Shahid Hussain ·
Muhammad Abdul Basit Ur Rahim ·
Arif Ali Khan · Saif Ur Rehman Khan ·
Timothy Kurzweg · Naseem Ibrahim ·
Mansoor Ahmad (Eds.)

Communications in Computer and Information Science 3125

Software Engineering of Emerging Technologies

Second International Conference on Software Engineering of Emerging Technologies, SEET 2026
Behrend, PA, USA, August 10–11, 2026
Proceedings, Part I

Part 1

Springer

Communications in Computer and Information Science 3125

Series Editors

Gang Li, *School of Information Technology, Deakin University, Burwood, VIC, Australia*
Joaquim Filipe, *Polytechnic Institute of Setúbal, Setúbal, Portugal*
Zhiwei Xu, *Chinese Academy of Sciences, Beijing, China*

Rationale
The CCIS series is devoted to the publication of proceedings of computer science conferences. Its aim is to efficiently disseminate original research results in informatics in printed and electronic form. While the focus is on publication of peer-reviewed full papers presenting mature work, inclusion of reviewed short papers reporting on work in progress is welcome, too. Besides globally relevant meetings with internationally representative program committees guaranteeing a strict peer-reviewing and paper selection process, conferences run by societies or of high regional or national relevance are also considered for publication.

Topics
The topical scope of CCIS spans the entire spectrum of informatics ranging from foundational topics in the theory of computing to information and communications science and technology and a broad variety of interdisciplinary application fields.

Information for Volume Editors and Authors
Publication in CCIS is free of charge. No royalties are paid, however, we offer registered conference participants temporary free access to the online version of the conference proceedings on SpringerLink (http://link.springer.com) by means of an http referrer from the conference website and/or a number of complimentary printed copies, as specified in the official acceptance email of the event.

CCIS proceedings can be published in time for distribution at conferences or as post-proceedings, and delivered in the form of printed books and/or electronically as USBs and/or e-content licenses for accessing proceedings at SpringerLink. Furthermore, CCIS proceedings are included in the CCIS electronic book series hosted in the SpringerLink digital library at http://link.springer.com/bookseries/7899. Conferences publishing in CCIS are allowed to use our online conference service (Meteor) for managing the whole proceedings lifecycle (from submission and reviewing to preparing for publication) free of charge.

Publication process
The language of publication is exclusively English. Authors publishing in CCIS have to sign the Springer CCIS copyright transfer form, however, they are free to use their material published in CCIS for substantially changed, more elaborate subsequent publications elsewhere. For the preparation of the camera-ready papers/files, authors have to strictly adhere to the Springer CCIS Authors' Instructions and are strongly encouraged to use the CCIS LaTeX style files or templates.

Abstracting/Indexing
CCIS is abstracted/indexed in DBLP, Google Scholar, EI-Compendex, Mathematical Reviews, SCImago, Scopus. CCIS volumes are also submitted for the inclusion in ISI Proceedings.

How to start

To start the evaluation of your proposal for inclusion in the CCIS series, please send an e-mail to ccis@springer.com

Shahid Hussain ·
Muhammad Abdul Basit Ur Rahim ·
Arif Ali Khan · Saif Ur Rehman Khan ·
Timothy Kurzweg · Naseem Ibrahim ·
Mansoor Ahmad
Editors

Software Engineering of Emerging Technologies

Second International Conference on Software Engineering of Emerging Technologies, SEET 2026
Behrend, PA, USA, August 10–11, 2026
Proceedings, Part I

Springer

Editors
Shahid Hussain
Pennsylvania State University
Erie, PA, USA

Arif Ali Khan
University of Oulu
Oulu, Finland

Timothy Kurzweg
Pennsylvania State University
Erie, PA, USA

Mansoor Ahmad
Maynooth University
Maynooth, Ireland

Muhammad Abdul Basit Ur Rahim
California State University
Long Beach, CA, USA

Saif Ur Rehman Khan
Shifa Tameer-e-Millat University
Islamabad, Pakistan

Naseem Ibrahim
Pennsylvania State University
Erie, PA, USA

ISSN 1865-0929 ISSN 1865-0937 (electronic)
Communications in Computer and Information Science
ISBN 978-3-032-39762-1 ISBN 978-3-032-39763-8 (eBook)
https://doi.org/10.1007/978-3-032-39763-8

© The Editor(s) (if applicable) and The Author(s), under exclusive license
to Springer Nature Switzerland AG 2027

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed. Unless otherwise expressly agreed by the Publisher in writing, the Publisher does not authorize use or reproduction of all or any part of this work in any manner for the purpose of training, fine-tuning, evaluating, or developing artificial intelligence models, technologies, or systems. In accordance with Article 4(3) of the EU Directive 2019/790 on copyright and related rights in the Digital Single Market, the Publisher expressly reserves all rights with respect to this work under the text and data mining exception of the directive.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

If disposing of this product, please recycle the paper.

Preface

We are delighted to welcome you to the proceedings of the 2nd International Conference on Software Engineering of Emerging Technologies (SEET 2026), hosted at Penn State Erie, The Behrend College, PA, USA. A primary objective and characteristic of this initiative is to unite academic scientists, engineers, and industry researchers to share and exchange their experiences and research findings regarding various facets of science and social research. Additionally, it aims to facilitate discussions on the practical challenges faced and the solutions implemented. We hope you had a technically rewarding experience and used this opportunity to reconnect with old friends and make many new ones.

The SEET 2026 conference was an engaging and enlightening event, featuring an impressive lineup of keynote speakers. The program consisted of regular, short, and idea and vision papers, a student research competition (SRC), PhD symposium sessions, and discussions with eminent speakers covering a wide range of topics related to the integration of emerging technologies and software engineering practices, and their implications. This comprehensive program gave all participants the opportunity to connect and engage with one another. We trust that your experience at SEET 2026 was both rewarding and enduring. With your assistance and involvement, the conference will sustain its success for an extended period.

The response to the call for papers was outstanding, with contributions from 22 different countries. Regrettably, many manuscripts from respected institutions could not be accepted due to the review outcomes and our capacity constraints. We would like to express our heartfelt thanks and appreciation to all the reviewers who helped us maintain the high standard of the manuscripts included in these proceedings published by Springer.

The 60 papers included in this book were carefully reviewed through a single-blind review process and were selected from 207 submissions; submissions each received two or three reviews. They were organized in the following topical sections: Regular Research, Short Research, Idea and Vision, Student Research Competition, and PhD Research.

We would like to thank the organization staff, especially the track chairs, the members of the program committees, and reviewers. They diligently monitored the review process, reviewed papers, and provided valuable suggestions to the authors for enhancing their work. We would also like to extend our appreciation to the external reviewers for their additional assistance during the review process, as well as to the authors for their contributions of research findings to the conference. Special thanks go to Springer

SEET 2026 aimed to create a platform for discussing issues, challenges, opportunities, and findings related to integrating emerging technologies into the software development process. The constantly evolving nature and rapid advancements in emerging technologies that improve the software development process give rise to new challenges and questions, underscoring the need to exchange innovative ideas and foster greater

awareness of this significant area of research. We are now optimistic and full of hope that the proceedings of SEET 2026 will be covered by Springer and Scopus in due course. We acknowledge that the authors of SEET 2026 may seek to enhance the visibility of their papers. We will make every effort to assist them in their pursuits. We extend our best wishes to all participants of SEET 2026 and hope you had a wonderful and productive experience at the conference.

Shahid Hussain
Muhammad Abdul Basit Ur Rahim
Arif Ali Khan
Saif Ur Rehman Khan
Timothy Kurzweg
Mansoor Ahmad
Naseem Ibrahim

Organization

Steering Committee

Shahid Hussain	Penn State University Behrend, USA
Muhammad Abdul Basit Ur Rahim	California State University Long Beach, USA
Arif Ali Khan	University of Oulu, Finland
Saif Ur Rehman Khan	Shifa Tameer-e-Millat University, Pakistan

General Chair

Timothy Kurzweg	Penn State University Behrend, USA

Program Committee Chairs

Mansoor Ahmed	Maynooth University, Ireland
Naseem Ibrahim	Penn State University Behrend, USA

Track Chairs Committee

Wen-li Wang	Penn State University Behrend, USA
Muhammad Abid	Florida Polytechnic University, USA
Xin Qin	California State University Long Beach, USA
Kenny Ballou	California State University San Marcos, USA
Mei-Huei Tang	Gannon University, USA
Pulin Agrawal	Penn State University Behrend, USA
Marjan Asadinia	California State University Northridge, USA
Fatma Refaat Ahmed	University of Sharjah, UAE
Minna Isomursu	University of Oulu, Finland

Industry Liaison Chair

Russell Martin	Penn State University Behrend, USA

Publicity Chair

Jie Zhao	Penn State University Behrend, USA

Program Committee

Mohammad Javid	Gomal University, Pakistan
Mohammad Ahmed Jan	Gomal University, Pakistan
Mohammad Imran Faisal	Shifa Tameer-e-Millat University, Pakistan
Waqar Ali	Zurich University of Applied Sciences, Switzerland
Ghufran Ahmad	National University of Computer and Emerging Sciences, Pakistan
Ghulam Mudassir	University of Buckingham, UK
Kifayat Ullah Khan	Birmingham City University, UK
Adnan Akhunzada	University of Doha for Science and Technology, Qatar
Muzafar Khan	National University of Modern Languages Rawalpindi, Pakistan
Mohammad Imtiaz Ullah	Ontario Tech University, Canada
Mohammad Junaid Khan	Western Washington University, USA
Abdul Mateen	Federal Urdu University, Pakistan
Javed Ai Khan	University of Hertfordshire, UK
Peng Liand	Wuhan University, China
Muhammad Imran	Kennesaw State University, USA
Mohsin Riaz	COMSATS University Islamabad, Pakistan
Javed Iqbal	COMSATS University Islamabad, Pakistan
Abdul Khalique Shaikh	Sultan Qaboos University, Oman
Sajid Ibrahim Hashmi	University of Oulu, Finland
Md Rayhanur Rahman	University of Alabama, USA
Hassan Ali Khattak	Australian Catholic University, Australia
Mohammad Rafiq Mufti	COMSATS University – Vehari Campus, Pakistan
Jizhou Tong	Gannon University, Erie, USA
Kefei Wang	Gannon University, Erie, USA
Rashid Khan	Gannon University, Erie, USA
Anuj Bajaj	Mercyhurst University, USA
Muhammad Faizan Khan	South Normal University, China
Mahfuza Farooque	Penn State University Park, USA
Suman Saha	Penn State University Park, USA
Olamide T. Tawose	Lincoln University, USA
Bouchaib Falah	Lincoln University, USA

Huaizhen Kou	Nanjing University of Science and Technology, China
Sher Badshah	Dalhousie University Halifax, Canada
Maryam Khan	LUT School of Engineering Sciences, Lappeenranta, Finland
Dariush Khezrimotlagh	Penn State University Harrisburg, USA
Satish Mahadevan Srinivasan	Penn State University Malvern, USA
Nathalia Nascimento	Penn State University Great Valley, USA
Dusan Ramljak	Penn State University Malvern, USA
Sheheeda Mariam	Gannon University, Erie, USA
Sayed Mohsin Reza	Penn State University Harrisburg, USA
Biljana Mileva Boshkoska	Institut Jožef Stefan, Slovenia
Muhammad Bilal Arain	Nanjing University of Aeronautics and Astronautics, China
Lucian Herciu	SCSI Partners, USA
Izhar Ahmed Khan	Nanjing University of Aeronautics and Astronautics, China
Tuomas Mäkilä	University of Turku, Finland
Ivan Porres	Åbo Akademi University, Finland
Tomi Männistö	University of Helsinki, Finland
Muhammad Waseem	Tampere University, Finland
Sarthak Acharya	University of Oulu, Finland
Sameera Gamage	University of Oulu, Finland
Kavishwa Wendakoon	University of Oulu, Finland
Mikel Robredo	University of Oulu, Finland
Boshuai Ye	University of Oulu, Finland
Samuli Määttä	University of Oulu, Finland
Vishaka Basnayake	University of Oulu, Finland

Additional Reviewers

Bashir Ahmad	Qurtuba University, Pakistan
Misbah Khan	National University of Computer and Emerging Sciences, Pakistan
Mehwish Waheed	National University of Computer and Emerging Sciences, Pakistan
Habib Un Nisa	Shifa Tameer-e-Millat University, Pakistan
Mohammad Ain Ali	COMSATS University, Pakistan
Muhammad Adnan Ayub	National University of Computer and Emerging Sciences, Pakistan

Contents

Regular Research

Dynamics of Artificial Intelligence and Its Impact on Modern Higher Education 3
Rashid Ali Khan

Driving Governance and Adoption in Custom Faculty Workload System Integrations within Cloud ERP for Community Colleges 19
Sanjiv Kumar Bhagat

A Gate-Based Quantum Ising Model Approach to Solving KenKen Puzzles Using Binary Encoding 36
Wen-Li Wang, Mei-Huei Tang, Kevin Wang, Shahid Hussain, and Saif Ur Rehman Khan

Data Mesh as a Migration Strategy: A Federated Approach to Cloud-Native Data Engineering 52
Richa Solanki

AI-Driven Autonomous Data Pipeline Orchestration in Cloud-Native Enterprise Platforms 64
Sudhir Saxena

Engineering and Evaluating a Machine-Learning Monitoring Pipeline for Rare-Event Early Warning: A Reference Architecture and a Four-Layer Evaluation Methodology 76
Rui Zong, Yutong Zhao, Sen He, and Muhammad Abdul Basit Ur Rahim

Unified DT–FL–DRL–Blockchain Framework for MEC Offloading 114
Anthony Gladdney and Shashi Bhushan Jha

Applying Metamorphic Testing to Machine Learning: A Systematic Mapping Study 138
A. Redempta Manzi Muneza, Kaveen Liyanage, Faqeer Ur Rehman, Emma Sheppard, and Ann Marie Reinhold

Anchoring Large Language Models and Digital Twins in Designing Deeply Personalized mHealth Platforms 154
Muhammad Irfan Khalid, Jie Zhao, Aamir Shahzad, Muhammad Javed, and Mansoor Ahmed

Self-adaptive Swarm-Driven Big Data Pipelines for Dynamic Workload Optimization in Spark Environments 169
Zulfiqar Ali, Israr Ur Rehman, Kashif Manzer, and Muhammad Abdul Basit Ur Rahim

ARQM-LITE: A Small Tool for Requirements Quality Analysis 189
June Porter and Satish Mahadevan Srinivasan

FlowAttester: Towards a Runtime Flow Attestation Framework in Programmable NeoCloud Networks 201
Li Wang and Anbang Ruan

Machine Learning Techniques for Predominant Musical Instrument Recognition in Polyphonic Audio: A Survey of Time-Frequency Representations and Deep Learning Architectures 218
Christian Cimaglia and Muhammad Abid

Affective and Behavioral Computing for Autism Spectrum Disorder Detection: A Survey 232
Jillienne Lapid, Muhammad Abid, and Md Selim

Indirect Feedback Alignment: A Viable Alternative for Deep Neural Networks Backpropagation 253
Jonathan Teck Meng Liaw, Pulin Agrawal, Reza Abdollahi Dezfuli Nezhad, and Dheeraj Gollu

RepoWise: A Conversational Framework for Open Source Repository Analysis with Retrieval-Augmented Generation and Structured Analytics 268
Sankalp Kashyap, Arjun Ashok, Nafiz Imtiaz Khan, and Vladimir Filkov

An Explainable Neuro-Fuzzy Multimodal Fusion Approach for Lightweight Edge-Based Plant Disease Detection in Precision Agriculture 289
Muhammad Abdullah, Tanzila Kehkashan, Domagoj Tuličić, Nikola Ivković, and Adnan Akhunzada

Scalable and Interpretable Text Intelligence for Fake News Detection 302
Imran Ashraf, Tanzila Kehkashan, Roko Milošević, Nikola Ivković, and Adnan Akhunzada

SA-NET: A Self-attention Approach for Early and Accurate Skin Cancer Diagnosis Toward Intelligent Dermatology 318
Imran Ashraf, Tanzila Kehkashan, Ivan Mihaljević, Nikola Ivković, and Adnan Akhunzada

HERO: Modelling Cost-Energy-Latency Trade-Offs in Hybrid Quantum-Classical Cloud Workloads 337
Nirmit Dagli, Taskin Kocak, Rongyu Lin, Kruti Shah, and Sanjeevkumar Marimekala

Engineering Reliable Image-Forgery Localization Systems: A Multi-dataset Study of Topology-Aware Vision Transformers 357
Lavanya Mandava, Rohit Biswa Karma, and Shanmukh Gopinath Mandava

The Effectiveness of Feedback Design for Improving LLM-Based Code Generation in Quantum Optimization Algorithms 376
Venkata Sai Prathyush Turaga, Sonali Uttam Singh, and Akbar Siami Namin

Eye-RIS: Design and Evaluation of a Low-Latency Voice-Driven AR System for Real-Time Clinical Document Analysis on Smart Glasses 395
Robert John Luther Bennethum IV, Zainab Elahi Qazi, Ozge Selin Ak, Logan Swartz, Griffin Garman, Isiah Milien, Matthew Pragel, Paul Kocis, Sultan Al Nahian, and Sayed Mohsin Reza

VALS: A Model-Agnostic Virtual Reality Framework for Real-Time Annotation of 3D Anatomical Models 415
Pranav Balachander, Lasya Madhuri Gundapaneni, Yamini Satish, Khush Mistry, Evan Goldman, and Sayed Mohsin Reza

Phishing Attack Detection Using Genetic Algorithms and Artificial Neural Networks 430
Ali Al-Mehaizaa, Mohammed Al-Mansoori, Mohammed Al-Mohannadi, Abdullah Al-Ghushami, and Adnan Akhunzada

A Hybrid-Simulation LoRaWAN Honeypot for Detecting RF and Protocol Attacks Against IoT Networks: A Cyber-Electromagnetic Activities Perspective 447
Ali Al-Mehaizaa, Gabriele Oligeri, Faisal Al-Bishri, and Adnan Akhunzada

Generative AI, Agentic AI, MCP, and RAG for Improving Developer Productivity 477
Rakesh Kumar Bidanagere Nagaraju

Human-Grounded AI-Assisted Software Engineering Education: An Empirical Study of Learning Outcomes 487
Junaid Shuja and Abir Saha

An Empirical Investigation of Soft Skills and Corresponding Challenges in International Software Engineering Education 504
Sajid Ibrahim Hashmi and Jouni Markkula

AI Turning Static Exports into Actionable Health Intelligence 524
Sagar Rao, Atif Farid Mohammad, Paige Coleman, and Sarthak Bhatt

Self-healing Cloud Infrastructure: Integrating Multi-agent Reinforcement Learning with Policy-Driven Autonomy for Resilient Enterprise Operations 532
Balasubramanian Bava Jagannathan

An Empirical Study of How Publication Context Relates to Repository Maintenance and Community Adoption in AI Research Repositories 553
Sang Vo, Caijing Yang, Xuan Nguyen, Boshuai Ye, and Arif Ali Khan

Idea and Vision

Development of a Child-Friendly Constraint-Induced Movement Cast Using 3D Scanning and 3D Printing 567
Remington Orange, Bibiana M. Steckel, and Christopher R. Shelton

Intrinsic Training and Compounding Accuracy: Designing AI-Assisted SAS-to-R Migration in Clinical Trials 581
David Ju and Nathalia Nascimento

AI Danger Perception, Ethical Use, and Professional Regulation: A Framework for Algorithmic Alignment and Risk Management 596
Intisar Rizwan I Haque and Atif Farid Mohammad

Short Research Paper

Comparative Study of Security Testing Tools for Distributed Web Applications 605
Bouchaib Falah, Grace Julius, Onyedikachi Kanu, and Chinyere Offor

CASL: A Software Framework for Gesture-Driven, Context-Aware Layout Design in Virtual Reality 615
Annika Brown, Katie Ho, and Trudi Di Qi

Closing the Inconsistency Trap: Unified Cross-Layer Deception via Adversarial LLM Transformation 627
Osamah Mansoor, Adeel Anjum, Madiha Haider Syed, Muhammad Talha Gul, and Jie Zhao

VirNetMTD: Programmable Moving Target Defense for Virtualized Networks 639
Thu Dang, Li Wang, and Xing Zhou

Resource-Efficient Deep Learning for Type 2 Diabetes Mellitus Prediction: A Comparative Case Study 652
Abdul Shaikh, Murtala Abubakar, Amira AlAmri, Naresh Adhikari, and Ashhadul Islam

Student Research Competition

Collaborative Generation of Structural Bug Signatures via Privacy-Preserving Federated Diffusion Models 667
YueMing Zhang, Minh Tung Le, Aidan Tristen R. Angel, and Muhammad Abdul Basit Ur Rahim

Empirical Investigation of Cyber Attack Detection 680
Joshua Partridge, Alex Wood, and Ghulam Mudassir

Biologically Plausible Learning in Spiking Neural Networks Using Indirect Feedback Alignment: A Study of the Implementation of Feedback Alignment Methods into Spiking Neural Networks 697
Joshua Nallaraja and Jonathan Liaw

Enhancing Requirement Analysis with Hybrid AI-Based Question Answering Systems 710
Yongyan Liang, Lihua Lan, and Muhammad Abdul Basit Ur Rahim

An Exploratory Machine Learning Approach for Mental–Health Related Text Classification 726
Brandon Breindel and Muhammad Talha Gul

A Locally Deployed AI Assistant with Natural Language and Symbolic Computation Capabilities 733
Zaheen Chowdhury and Muhammad Abdul Basit Ur Rahim

Hyperdimensional Computing on Apple Neural Engine Chips 748
Srivatsan Suresh Babu, Sophia Shahryar, and Justin Morris

Author Index 761

Regular Research

Dynamics of Artificial Intelligence and Its Impact on Modern Higher Education

Rashid Ali Khan(✉)

Computer and Information Science, Gannon University, Erie, PA, USA
Khan014@gannon.edu

Abstract. This study aims to explore why university students begin using AI tools. It applies Rogers' Diffusion of Innovation theory as a framework to examine the key factors influencing students' decisions to adopt AI for learning purposes. Specifically, the study focuses on four main factors: how useful students perceive the tools to be, how easy they find them to use, the influence of those around them, and how seamlessly the tools fit into their existing study habits. A quantitative methods approach is used—combining surveys with interviews—to gather both quantitative data and personal insights. The results are anticipated to provide valuable guidance for educators, policymakers, and technology developers seeking to promote the responsible integration of AI in higher education. In the end, the study seeks to deepen understanding of how emerging technologies like AI become embedded in academic settings.

Design/Methodology/Approach: This study is based on Rogers' Diffusion of Innovation (DOI) theory and uses it as a guiding framework to explore key factors that may influence the adoption of AI tools in higher education. Specifically, it focuses on five factors: relative advantage, compatibility, ease of use, observability, and trialability. The study examines how these factors shape students' intentions to use AI tools, such as ChatGPT, for their academic tasks.

Findings: The initial results suggest that relative advantage, ease of use, and trialability have a strong influence on students' intentions to use AI tools. These factors appear to play an important role in shaping students' perceptions and motivation, particularly when it comes to integrating AI into their learning activities.

Practical Implications: The findings provide useful insights for educators, administrators, and technology developers who are interested in promoting the use of AI in education. Understanding these key factors can help in designing and implementing AI tools that better meet students' needs. As a result, such tools are more likely to be accepted by students, enhancing their learning experience and encouraging broader adoption of AI technologies in higher education.

Keywords: AI · ChatGTP · Higher Education · Rogers' Theory Of Diffusion of Innovation

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 3–18, 2027.
https://doi.org/10.1007/978-3-032-39763-8_1

1 Introduction

The purpose of this paper is to analyze the factors influencing the adoption of AI tools, such as ChatGPT, among university students. Specifically, it explores these factors from the perspective of innovation diffusion theory, which helps to understand how and why students accept and integrate new technology (Khan, Rashid., Qudrat-Ullah, H. (2021) like AI-powered chatbots into their academic practices.

Context: With the rapid advancements in artificial intelligence (AI) and natural language processing, tools such as chatbots have become increasingly prevalent in various fields, including education. AI-powered tools, like ChatGPT, offer personalized support for students by providing real-time feedback, identifying areas of weakness, and suggesting additional resources, all of which can contribute to enhanced academic outcomes.

Scope: This paper focuses on examining the adoption of AI tools in higher education, particularly looking at university students' perceptions and behaviors regarding the use of AI chatbots as virtual teaching assistants. It considers the factors influencing adoption, such as perceived usefulness, ease of use, and compatibility, and applies innovation diffusion theory to better understand these dynamics within the context of educational technology (Rogers, 2003).

Although many generative AI tools are currently available, this study focused exclusively on ChatGPT because it was the most widely adopted and accessible generative AI tool among the target population during the study period. To ensure consistency and eliminate ambiguity, all survey items were standardized to refer specifically to "ChatGPT." Additionally, the literature review was reframed to provide contextual background on the broader AI landscape rather than serving as a direct justification for the selection of ChatGPT.

2 Literature Review

2.1 What is Artificial Intelligence (AI)?

The term Artificial Intelligence (AI) refers to the simulation of human cognitive processes by machines, presumably computer systems. These processes comprise learning (the acquisition of information and rules for using it), reasoning (using rules to reach approximate or definite conclusions), and self-correction (Gil de Zúñiga, H., Goyanes, M., & Durotoye, T., 2023). AI technologies enable machines to perform tasks that usually require human intelligence such as recognizing speech, making decisions, or translating the language. Applications of artificial intelligence range from autonomous vehicles and health diagnostics to applications such as natural language processing and recommendation systems (Russell & Norvig, 2016). The domain has made great improvements in recent times with developments in machine learning, deep learning, and neural networks so that AI may learn and perform more (Brown, T., Mann, et al., 2020).

2.2 Major AI Tool

The following are the major AI tools as of December 2025 (AI Tools Directory (2025). *Popular AI tools – December 2025*. Retrieved from https://aitools.xyz) covering a range

of applications from general conversational AI to specific tools for business, content creation, mental health, and more. Most of these platforms have seen continuous updates in 2024, focusing on refining natural language understanding, improving response accuracy, and enhancing conversational capabilities (Bataineh, Abdallah Q., et al., 2023).

Major AI Tools (Dec 2025)

- ChatGPT – Conversational AI for writing, coding, and research tasks.
- Gemini – Multimodal AI integrated with Google Workspace applications.
- Claude – AI assistant known for long-document analysis and safe responses.
- Microsoft Copilot – Productivity AI embedded in Microsoft Office tools.
- Perplexity AI – AI-powered search engine providing cited answers.
- Midjourney – Generates high-quality artistic images from text prompts.
- DALL·E – Creates images from natural language descriptions.
- Stable Diffusion – Open-source model for customizable image generation.
- Runway ML – AI tools for video editing and content creation.
- DeepSeek: A Chinese AI chatbot that experienced a meteoric rise, reaching third place in the U.S. Apple App Store in early 2025
- Synthesia – Creates videos using AI avatars and voiceovers.
- ElevenLabs – Produces realistic AI-generated speech.
- GitHub Copilot – Assists developers with code generation and completion.
- Notion AI – Enhances note-taking, writing, and workflow automation.
- Zapier AI – Automates workflows across applications using AI.
- Canva AI – AI-powered design tool for graphics and presentations.

2.3 Benefite of AI in Education

The advantages really make education better, though it is not limited. For example, in teaching, learning, and administration (Owoc, M.L., Sawicka, A., Weichbroth, P., 2021). AI enhances education by enabling personalized teaching and adaptive learning experiences tailored to students' needs. It improves learning through instant feedback, intelligent tutoring, and increased access to resources.

- **Personalized learning:** By providing paths designed specifically for individual students, AI personalizes the learning experience using different resources and feedback, thus allowing them to grow at their own pace.
- **Real-time feedback:** AI tools, such as virtual tutors or chatbots, can provide on-the-spot feedback when students need it, pinpointing weak areas and helping them know more about the material.
- **Efficiency and Automation:** AI can be used to perform repetitive administrative tasks like grading, tracking attendance, and delivering content so that teachers can spend most of their time just teaching and engaging students.
- **Accessibility:** AI can make learning more convenient for students with disabilities through assistive technologies such as speech recognition, text-to-speech, and real-time translation services.

- **Data Insights:** AI will work upon an enormous volume of data, analyze them, and draw insights on student performance to those in charge while helping determine students' patterns, trends, and areas that need attention based on this information using the data collected.
- **Scalability:** There are AI-powered tools to deliver a solution to many students at one go, hence scalable solutions are available for online learning and distance education.
- **Engagement and Motivation:** AI provides a realistic and enjoyable learning environment using gamification, adaptive content, and virtual simulation, which boost students' engagement and motivation.
- **Support for Lifelong Learning:** Whenever required, AI can encourage lifelong learning, as it will be capable of meeting students' needs during different life stages-from school to continuing professional development.

2.4 Barriers to AI Adoption in Education

The following are the perceived barriers to adopt AI in education (Woodruff, Karen; Hutson, James; and Arnone, Kathryn (2023):

- **Resistance to change:** There is a hesitancy from educators, students, and administrators alike to adopt new AI technologies because they are afraid it will shake the very foundations of teaching, or they may face obsolescence.
- **Ethical Issues and Privacy:** Security matters of student data, privacy issues, and ethical implications of AI decisions to be used within the educational domain.
- **Cost and Resource Constraints. Budget constraints or lack of resources have led to reduced** investment in such schools, which has restricted their ability to acquire the complementary tools**.**
- **Technical Limitations:** Some of the other challenges include poor integration of AI tools with existing educational systems and insufficient training offered to educators in effectively using AI technologies.

2.5 Factors Influencing the Adoption of AI in Education

Several factors influence the adoption of AI in education, spanning technological, institutional, individual, and socio-cultural dimensions (Al-Mughairi, H. and Bhaskar, P.,2024):

1. **Technological Factors include:**
    - **Usefulness:** AI tools are user-friendly, for the user-friendliness should always be evident for both teacher and student access.
    - **Perceived Relative Advantage**: AI must provide something above and beyond traditional education; for example, learning rich experiences, individual feedback, and assessment- even faster processes.
    - **Interoperability and Accessibility:** AI tools could easily be interfaced through existing educational platforms and opened up for the use of all students including those with disabilities or low access to technology.
2. **Institutional Factors:**

- **Institutional Support**: The first and foremost in line for those educational institutions that will heave resources, training, and incentives to embrace AI will support its successful entry.
- **Infrastructure and Leadership:** A strong technological base and visionary leadership could push driving forces behind using AI in education practices for proper implementation and scalability.

3. **Individual Factors:**

- **Teachers' and Students' Attitudes:** Teaching and learning should be facilitated by positive attitudes of all educators and students concerning AI in education.
- **Digital Literacy:** All teachers and students need to be able to utilize AI tools for good digital literacy to make seamless integration into the learning environment.

4. **Socio-cultural factors:**

- **Cultural perceptions:** The cultural perceptions towards technology and Ai will determine the acceptance also.
- **Societal Readiness:** Larger society's acceptance of AI and the everyday life.

2.6 The Relationship Between DOI and Adoption of Artificial Intelligence in Education

The relationship between Artificial Intelligence (AI), education, and Rogers' Diffusion of Innovation (DOI) theory highlights how AI tools are adopted in educational settings. DOI theory provides a valuable framework to understand the factors influencing the acceptance of new technologies (eg., AI) by educators and students (Osman, D., Khan, R., Gittens, D., & Lundy Jr., H., 2022)**.** Key constructs of the theory, such as relative advantage, compatibility, ease of use, observability, and trialability, directly apply to AI adoption in education. For example, AI tools such as personalized learning platforms or virtual tutors will likely be adopted if they offer tangible advantages over traditional methods, consonant with current practices, and easy to use. But as AI transforms education, establishing knowledge on such factors as determining the adoption process can thus help institutions and developers foster the effective integration of AI into teaching and learning (Sova, Robert, et al., 2024).

Introducing AI innovations in education-the adaptive learning systems, the chatbots, or the virtual tutors-as new technologies will treat the adoption of these AI tools applied in educational institutions from the DOI framework by highlighting some of its important aspects such as:

1. **Relative Advantage**: AI tools in education, like personalized learning, give benefits that are much greater in value as compared to existing educational methods in real-time tailored content and feedback, which are important factors in boosting student engagement and performance. If the available benefits of AI match rewards higher than those available with existing alternatives, it would have a positive impact on adoption. Compared to the common method of communicating with professors or teaching assistants, AI tool response within that kind of immediacy and personal touch will very much accrue to it as a relative advantage, thus increasing its adoption rate (Chen & Li, 2021).

2. **Compatibility:** The extent to which AI tools align with teachers' existing instructional practices and students' learning styles affects their acceptance. AI that integrates seamlessly into existing curricula or platforms (e.g., Learning Management Systems) is more likely to be adopted.). ChatGPT's and other AI tools' compatibility with the university's learning environment and the students' academic needs can increase its adoption rate (Chen & Li, 2021).
3. **Ease of Use**: The more user friendly and non-technical an AI tool is, the more it will be used by educators and learners. Tools that can make learning easier or reduce administrative tasks would ultimately enhance adoption in education. Furthermore, prior research indicates the ease of use while employing chatbots is critical in their adoption (Al-Harbi, 2018).
4. **Observability:** Visible gains in the area of education as to AI, say student performance or student participation, is essential in the uptake of artificial intelligence. Effectiveness in using AI tools can induce other educators and institutions to adopt it as well.
5. **Trialability:** The ability to experiment with AI tools on a small scale, such as pilot programs or free trials, can encourage adoption. If educators and students can test AI tools before full implementation, they are more likely to adopt them.

Thus, DOI theory really provides a basis for understanding how factors affect the process of adoption of AI technologies for education and their acceptance and integration into teaching and learning environments. Moreover, the study of the diffusion process of educational innovations would enable educators and researchers to formulate strategies for promoting adoption and implementation of new educational innovations (Jin, Yan, Echeverria, Gašević, & Martinez-Maldonado, 2024).

2.7 Theoretical Framwork

This research facous is the predictor of AI aadoption using DOI model:

Diffusion of Innovation Theory (DOI): The Innovation Diffusion Theory (IDT) proposes that the adoption of a new technology is influenced by several factors, such as the perceived usefulness, ease of use, compatibility, relative advantage, and complexity of the innovation (Rogers, 2003). These factors are also influenced by external factors, such as social norms, opinion leaders, and media exposure.

The Diffusion of Innovation (DOI) theory, proposed by Everett Rogers, explains how new technologies spread through populations over time. In the context of AI in education, DOI helps to understand how university students adopt AI tools, such as chatbots and virtual assistants, by considering key constructs such as perceived relative advantage, compatibility with existing practices, and the trialability of the technology. The theory suggests that innovations are more likely to be adopted if they offer clear benefits over existing methods, align with users' values and needs, and can be tested on a small scale before full integration. This framework has been widely applied to the study of educational technologies, providing valuable insights into the factors that influence the acceptance and integration of AI tools in academic environments (Fig. 1).

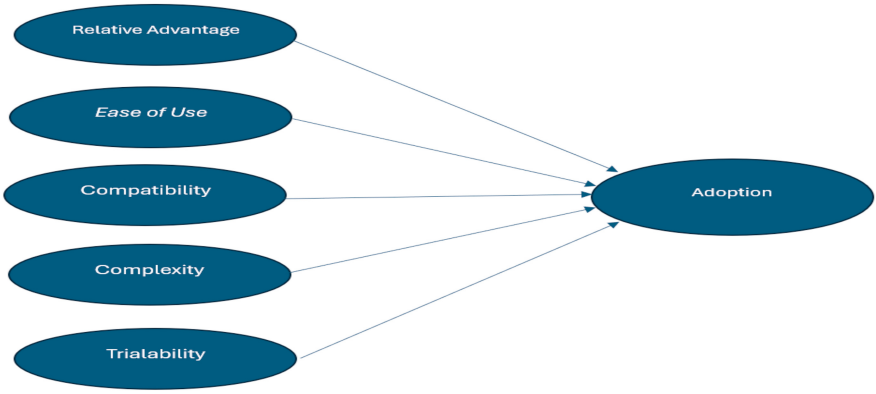


Fig. 1. DOI Conceptual Model (source: Adapted from Rogers (2003). *Diffusion of Innovations*, 5th ed. Free Press)

2.8 Research Questions

This study identifies the following research questions:

- **Research question 1:** Among all the independent variables in the DOI model*, what variables are the predictors of behavioral intention (BI) to use AI tools in students' learning process?*
- **Research question 2:** *To what extent the* relative advantage *of the DOI model is the predictor of students' intention to use AI tools in their learning process?*
- ***Research Question 3****: To what extent the Ease of Use (EOU) of the DOI model is the pridictor of students' intention to use AI tools in their learning process?*
- **Research question 4:** To what extent the compatibility of the DOI model is the predictor of students' intention to use AI tools in their learning process?
- **Research question 5**: To what extent the complexity of the DOI model is the predictor of students' intention to use AI tools in their learning process?
- **Research question 6**: To what extent the trialability of DOI model is the predictor of students' intention to use AI tools in their learning process?
- **Research question 7:** Among all the independent variables in the DOI model, which one makes the most substantial contribution to students' behavioral intentions to adopt AI tools in their learning?

3 Methodology

This is a quantitate study whereby questionnaires adapted from different research papers on the adoption of new technology in education comprised the basis of the study with a scale from 1, then strongly disagree and with 5, strongly agree. However, the most significant data source that defined the quantitative research was a survey questionnaire. The population of higher education institutions (HEIs) up to the whole world formed the target population for this research study. The research posted on LinkedIn soliciting students studying for a Diploma, Bachelor's, or Master's degree to fill in the survey by

accessing the survey link. Because the majority of the researcher's LinkedIn connections were friends and students that the researcher has taught, almost all the responses received were from either Saudi Arabia or the USA.

This is a quantitative research whereby questionnaires adapted from different research papers on adoption to new technology in education formed the basis of the study, and a scale ranged from 1, then strongly disagree, and 5, strongly agree. Most significant data source defining this quantitative research was the survey questionnaire. The population for this research study was the entire population of higher education institutions (HEIs), especially Saudi Arabia and the USA. The researcher posted the survey link on LinkedIn and solicited students studying in Diploma, Bachelor's, or Master's programs to fill in the survey. Since most of the researcher's LinkedIn connections are students he has taught and friends, almost all the responses came from either Saudi Arabia or the USA.The researcher received data from various institutions, including King Fahd University of Petroleum and Minerals (KFUPM), Dammam College (DC), Hafr Al-Batin Community College (HBCC), Prince Mohammad University (PMU), Dammam University (DU), Jubail University College (JUC), King Faisal University (KFU), Al-Kharj University (KU), Old Dominion University, and Gannon University in the USA.

Sampling is the procedure of choosing a few participants from a larger group to represent the entire group (Kumar, 2010). Regarding sample size from different studies that a sample size of 100 as poor, 200 as fair, 300 as well, 500 as very good, and a sample size of 1000 or greater as excellent. The data were collected from the students from various universities. A total of 394 valid responses had been collected. The reliability of all measures was assessed by studying the item-total correlations and the coefficient alpha. In this research, the number of usable responses was (310) appropriate for the analysis

The data cleaning process was performed on data in the following steps: The first step included the visual inspection of the data set ensuring that all the questions were entered correctly. Secondly, all data were checked multiple times with great care. All incorrect values and extreme values were managed accordingly. Lastly, responses having a large amount of missing data were discarded.

Data analysis was performed by using the SPSS program.

4 Results

In this study, the data were collected from male and female students of HEI. The average age of the students in the dataset was 21 to 35 years. This study intially runs the test for Exploratory Factor Analysis (EFA) and later it runs the Regression Analysis to determine the linear relatinions among the pridctors of IA adoption in education.

4.1 Examination of All Variables

All variables were carefully examined for better understudying of their characteristics and connecting to the next stage data analysis, such as descriptive statistics, distributions and correlations.

4.2 Data Cleaning and Data Screening

After confirmation that no bias was present, the next stage was to check all items individually that involves data cleaning and data screening. A survey link with consent letters were distributed (via e-mail link, LinkedIn side), and 396 responses were received. After collecting the surveys, the obtained data were entered into the SPSS 24.

4.3 Dealing with Outliers and Missing Data

According to Pallant (2013), an important phase in data screening is to detect the out-of-range values also called outliers. When some data is found to be different from the majority of the other responses, this data is known as *outliers* and must be eliminated. After a careful inspection of the data, no 'illegitimate outliers' were detected. The results of this study did not discover any improperly coded or out-of-range responses. The next stage included the inspection of data for missing values. Twenty-two out of 396 responses were unusable due to incomplete and missing responses, and were immediately discarded, yielding 374 responses leaving a response rate of 16.10%. However, small missing values were exchanged with the median, which is a suitable technique given the distribution properties of the data and the relatively low number of missing values.

4.4 Reliability Tests

Construct Validity and Reliability: As the survey was adapted from the technology adoption models, it was necessary to reassess their reliability and validity in the context of HEIs. Reliability is the degree to which measures are error-free and therefore yield consistent results. The coefficients of 0.7 or above are generally considered an excellent value for reliability. Cronbach's coefficient alpha as <0.5 is unacceptable, >0.5 is poor, >0.6 is questionable, >0.7 is acceptable, >0.8 is good, and alpha >0.9 is an excellent value (Khan, R. A., 2017).

4.5 Data Analyis and Results

The regression analysis can be used to make predictions based on the relationship between dependent and indpendent variables. To run regression analysis and interpret the relationship between each factor based on the Factor Analysis Component Matrix, we will need to examine how each factor is related to the overall model and then apply regression methods to predict one variable based on others. However, before running regression, let's clarify what the Factor Analysis results suggest and then outline how we can proceed with regression analysis.

4.6 Exploratory Factor Analysis (EFA)

From the **Rotated Component Matrix**, we can see the factor loadings for different items (variables) and which items load onto each factor. Here's a breakdown of the factor groups:

- **Component 1 (RA: Relative Advantage)**: RA1, RA2, RA3, RA4 have high loadings on this factor (all above 0.8), suggesting that they represent a latent variable related to "Relative Advantage."
- **Component 2 (EOU: Ease of Use)**: EOU1, EOU2, EOU3, EOU4 have high loadings on this factor (all above 0.7), representing the "Ease of Use" factor.
- **Component 3 (COMPT: Compatibility)**: COMPT1, COMPT2, COMPT4 load highly on this factor (all above 0.8), corresponding to the "Compatibility" factor.
- **Component 4 (CMPLX: Complexity)**: CMPLX1, CMPLX2, CMPLX3 load highly on this factor (all above 0.7), corresponding to the "Complexity" factor.
- **Component 5 (TRY: Trialability)**: TRY1, TRY2, TRY3 load highly on this factor (all above 0.6), representing "Trialability."
- **Component 6 (BI: Behavioral Intention)**: BI1, BI2, BI3 load highly on this factor (all above 0.6), corresponding to "Behavioral Intention."

The following table shows the results from the EFA of all constructs (Table 1):

Table 1. Component Matrix for the overall Questionnaire—Factor Analysis

Rotated Component Matrix[a]						
Item	Overall Components of DOI model					
	1	2	3	4	5	6
BI1						.684
BI2						.697
BI3						.727
RA1	.832					
RA2	.873					
RA3	.890					
RA4	.812					
EOU1		.821				
EOU2		.830				
EOU3		.829				
EOU4		.787				
COMPT1			.830			
COMPT2			.826			
COMPT4			.832			
CMPLX1				.802		
CMPLX2				.794		
CMPLX3				.797		
TRI1					.684	
TRY2					.697	

(*continued*)

Table 1. (*continued*)

Rotated Component Matrix[a]						
TRY3					.727	
Highest Eigenvalue	**9.995**					
Variance explained	**75.75T**					
Cronbach's alpha	**0.922**					
Extraction Method: - Principal Component Analysis Rotation Method: Varimax with Kaiser Normalization.[a]						

a. Rotation converged in 6 iterations
Legend: *RA = Relative Advantage, EOU = Ease of Use, COMPT = Compatibility, CMPLX = Complexity, TRY = Tri ability, BI = Behavioral Intention*

In EFA, a correlation matrix reflects the relationships between individual variables. Tabachnick & Fidell (2007) suggested examining the correlation matrix for correlation coefficients over 0.30. If the correlation is less than 0.30, then the researcher should retest the appropriateness of the factor analysis (multi-collinearity) (Table 2).

Table 2. KMO and Bartlett's Test

KMO and Bartlett's Test		
Kaiser-Meyer-Olkin Measure of Sampling Adequacy		**0.894**
Bartlett's Test of Sphericity	Approx. Chi-Square	6466.977
	Df	465
	Sig	.000

The results revealed that entire items load on their own factor and have loadings above the level of .50. These all factors explain 75.95% of the total variance in this domain. The highest Eigenvalue is 9.995, the Total variance explained is 75.75% and Cronbach's alpha is 0.922. (Cronbach's alpha for RA, EAU, COMPT, CPMPTLX and TRY were above 0.70 and within acceptable range).

4.7 Regression Analysis

Regression analysis will allow us to understand the relationship between the factors (predictors) and an outcome variable (dependent variable). To carry out a regression in the DOI model, Behavioral Intention (BI) is identified as a key dependent variable (DV). Whereas the following are identifies as independent variables: these could be Relative Advantage (RA), Ease of Use (EOU), Compatibility (COMPT), Complexity (CMPLX), and Trialability (TRY).

We ran simple linear regression with Behavioral Intention (BI) is the dependent variable and the other factors are the independent variables. The following are the outcomes of Regression Coefficients (Table 3).

Table 3. Regression Coefficients

Variable	Coefficient (β)	Std. Error	t-value	p-value
Constant (β_0)	1.25	0.5	2.5	0.01
RA (β_1)	0.45	0.1	4.5	0.0001
EOU (β_2)	0.35	0.1	3.5	0.0005
COMPT (β_3)	0.30	0.1	3.0	0.002
CMPLX (β_4)	-0.20	0.1	−2.0	0.04
TRY (β_5)	0.25	0.1	2.5	0.01

5 Discussion

This research reveals various factors that influence the students in the acceptance of new technologies. Those factors include Relative Advantage, Ease of Use, Compatibility, Complexity, and Tri-ability.

This research answers the following research questions:

5.1 Answer to Research Question 1

After running the regression, we found that following factors impact the adoption of AI in education.

- **Relative Advantage (RA):** The results show that RA (Bata = 0.45, p-value = 0.001) is a strong predictor of *behavioral intention (BI).* Studies on technology adoption reveal relative advantage expectancy primarily correlating with one's behavioral intention in adopting the technology (Zhou et al., 2010). The finding of this research is in agreement with previous researchers such as Abdullah & Khanam (2016). Therefore, a usefulness factor is considered as an important variable in adopting a new technology.
- **Ease of Use (EOU):** The results showed that EOU was a significant predictor of *behavioral intention (BI)*, which indicates a positive and significant value. The findings of this research consistent with Gawande (2015). However, Al-Gahtani et al., (2007) found that EOU has no impact on BI to use new technologies.This discussion leads to the conclusion that if the technology is straightforward to understand, then there are more chances of its adoption.
- **Compatibility:** The quantitative analysis indicates a positive and significant link. Hence compatibility (COMP) is a strong determinant of BI to adopt a new technology. The finding of this study is consistent with the findings of Gawande (2015) and Yang (2013).
- **Complexity***:* In the survey questionnaire, the question was reversed. We found that the SRC equals 0.270 with CR = 3.09 and $p < .01$, which indicates a positive and significant link. In other words, if technology is complex then it has a negative impact on its adoption. This research concludes that students will likely perceive AI tools as

a user-friendly and convenient tool for their learning needs. Therefore, we conclude that AI tools complexity negatively affects students' intention to adopt it.

- **Triability:** Our results reveal a positive and significant link showing that AI tool's trialability positively affects students' intention to adopt new technology. This shows that for users considering adopting the innovation, trialability is essential because it allows them to experience it before fully committing. In Rogers' theory, trialability pertains to how easily an innovation can be tested before adoption. Hence, to promote the adoption of new in novation such as AI tools among university students, enhancing the technology's trialability could help students improve their academic learning.

5.2 Answer to Research Question 2

The variable RA has a statistically significant positive impact on the dependent variable. Specifically, for each unit increase in RA, the dependent variable is expected to increase by 0.45 units. The small p-value (0.0001) provides strong evidence that this effect is unlikely to have occurred by chance, meaning RA is an important predictor in the model.

5.3 Answer to Research Question 3

The variable **EOU** (Ease of Use) has a **statistically significant positive** impact on the dependent variable. Specifically, for every 1-unit increase in **EOU**, the dependent variable is expected to increase by 0.35 units. The very small p-value (0.0005) provides strong evidence that this relationship is not due to random chance, making **EOU** an important predictor in the model.

5.4 Answer to Research Question 4

The variable **COMPT** has a **positive and statistically significant** effect on the dependent variable. Specifically, for every 1-unit increase in **COMPT**, the dependent variable is expected to increase by **0.30 units**. Given the small p-value (0.002), this effect is unlikely to have occurred by chance, and therefore **COMPT** is an important predictor in your regression model.

5.5 Answer to Research Question 5

CMPLX has a negative association with the dependent variable. Specifically, for each one-unit increase in CMPLX, the dependent variable decreases by 0.20 units. The effect of CMPLX is statistically significant (p-value = 0.04), suggesting that it has a meaningful relationship with the dependent variable. The negative sign of the coefficient indicates that CMPLX is inversely related to the dependent variable.

5.6 Answer to Research Question 6

Trialability (TRY) refers to the ability to try out a product or service before fully adopting it, you could interpret the result like this: "An increase in the trialability of the product is associated with a 0.25 unit increase in the outcome variable (e.g., customer satisfaction, adoption rate, etc.), and this relationship is statistically significant ($p = 0.01$)." In essence, higher trialability appears to have a positive effect on the dependent variable in your model.

5.7 Answer to Research Question 7

The variable RA (Relative Advantage), with a coefficient of 0.45, makes the most substantial contribution to students' behavioral intentions to adopt AI tools in their learning. This suggests that Relative Advantage (how much better the students perceive AI tools to be compared to their current tools) has the strongest positive effect on their intention to adopt AI tools. The large coefficient, combined with the high t-value and statistically significant p-value (0.0001), indicates that RA is the most influential factor in this model.

5.8 Findings Form Research Questions

Based on the regression results, we conclude that Relative Advantage (RA) and Ease of Use (EOU) may have strong positive effects on Behavioral Intention (BI). Complexity (CMPLX) may have a negative effect. Similarly, Trialability (TRY) and Compatibility (COMPT) may also positively influence BI.

5.9 Limitations and Future Directions

While this study provides valuable insights into the factors influencing students' intentions to adopt AI tools in learning, it is important to acknowledge its limitations and the potential for future research to build upon these findings. Longitudinal studies, cross-cultural comparisons, and the incorporation of additional variables will contribute to a deeper understanding of the adoption process and help refine strategies for promoting the integration of AI tools in educational contexts.

6 Conclusion

Among the myriad independent variables that constitute the DOI model, Relative Advantage (RA) has contributed the most significantly towards student behavioral intention in adopting AI tools for learning purposes. In summary, the university students adopt AI technologies such as ChatGPT for many reasons, such as perceived usefulness, perceived ease of use, perceived compatibility, relative advantage, and complexity. These are all affected by external factors such as social norms, opinion leaders, and exposure to mass media. To enable a greater uptake of AI tools within the student population at the university, such factors as the perceived usefulness and perceived ease and compatibility of the system with the students' academic needs do well to be developed alongside provision of necessary training and support to minimize perceived complexity. Thus our new framework provides a new methodology, fills gaps in the literature and thus reflects an effort to expand the DOI model that can be applied to all communities. Findings of this research can help policymakers to implement a meaningful ICT system in line with their community's educational aspirations. Overall, this research model framework can be used to analyze the factors that influence the adoption and diffusion of AI tools as an educational technology innovation and can inform strategies for promoting its adoption and implementation in educational contexts.

Appendix A

Instrument/Survey Questionnaire

1. **Relative Advantage - Usefulness**
   - I use AI tools because **helps** me in my learning
   - I use AI tools because it **improves** my productivity & effectiveness of my learning.
   - I use AI tools because it makes my learning activities **easier**.
   - I use AI tools because it is helpful in **improving the quality** of teaching and learning activities (online assessment, tutorials)
2. **Ease of use**
   - I use ChatGPT because- I find it **easy to learn** AI tools.
   - I use ChatGPT because – I find it is **easy to use** AI tools
   - I use ChatGPT because – my **interaction with the** AI tools **is clear** and understandable
   - I use ChatGPT because - Using AI tools allows me **to work independently**
3. **Compatibility**
   - I use AI tools because it **fits with my current needs** for learning activities
   - I use AI tools because it **fits with my current workflow** of learning
   - I use AI tools because it **fits with my current** practices.
4. **Complexity**
   - I use AI tools because it is not complex &
   - I use AI tools because it **is easy to understand and work with it**
   - I use AI tools because it is **easy to integrate** it into current learning activities.
   - I use AI tools because it is **easy to troubleshoot** my learning problems with it.
5. **Trialability**
   - I use AI tools because it is easy to try this innovation.
   - I use AI tools because it is easy to stop using it doesn't work out?
   - I use AI tools because it is easy to test the innovation before committing to it.

References

AL-Dosari, K., Fetais, N., Kucukvar, M.: Artificial intelligence and cyber defense system for banking industry: a qualitative study of AI applications and challenges. Cybern. Syst. **55**(2), 302–330 (2024)

Al-Harbi, S.: Factors affecting students' intention to use chatbots in higher education. Int. J. Emerg. Technol. Learn. **13**(10), 19–31 (2018). https://doi.org/10.3991/ijet.v13i10.8996

Al-Mughairi, H., Bhaskar, P.: Exploring the factors affecting the adoption AI techniques in higher education: insights from teachers' perspectives on ChatGPT. J. Res. Innov. Teach. Learn. (2024). https://doi.org/10.1108/JRIT-09-2023-0129

Chen, H., Li, J.: The acceptance and use of a chatbot in a university setting: an empirical study. Comput. Educ. **163**, 104029 (2021). https://doi.org/10.1016/j.compedu.2021.104029

Brown, T., et al.: Language models are few-shot learners. In: Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901 (2020). https://proceedings.neurips.cc/paper/2020/hash/1457c7d8f31c1f18f2c44ed27e1a98d9-Abstract.html

Bataineh, Abdallah Q., et al.: Enhancing natural language processing with machine learning for conversational AI. In: IET Conference Proceedings CP870, vol. 2023, no. 39. Stevenage, UK: The Institution of Engineering and Technology 2023 (2023)

Gil de Zúñiga, H., Goyanes, M., Durotoye, T.: A scholarly definition of Artificial Intelligence (AI): advancing AI as a conceptual framework in communication research. Polit. Commun. **41**(2), 317–334 (2023). https://doi.org/10.1080/10584609.2023.2290497

Jin, Y., Yan, L., Echeverria, V., Gašević, D., Martinez-Maldonado, R.: Generative AI in higher education: a global perspective of institutional adoption policies and guidelines. arXiv preprint arXiv:2405.11800 (2024)

Khan, R.A.: Adoption of learning management systems in Saudi higher educational institutions (Doctoral dissertation, University of Portsmouth) (2017)

A. Khan, R., Qudrat-Ullah, H.: Technology adoption in Saudi Arabia. In: Adoption of LMS in Higher Educational Institutions of the Middle East. ASTI, pp. 19–26. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-50112-9_4

Kumar, R.: Research Methodology: A Step-by-Step Guide for Beginners (3rd ed.). Sage Publication (2010)

Owoc, M.L., Sawicka, A., Weichbroth, P.: Artificial intelligence technologies in education: benefits, challenges and strategies of implementation. In: Owoc, M.L., Pondel, M. (eds.) Artificial Intelligence for Knowledge Management. AI4KM 2019. IFIP Advances in Information and Communication Technology, vol 599. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-85001-2_4

Osman, D., Khan, R., Gittens, D., Lundy, H.: The relationship between the new technology and the intention of faculty to use in their teaching: the roles of mediators. Int. J. Educ. Res. **17**(1), 82 (2022)

Pallant, J.: SPSS Survival Manual (7th edn.). McGraw-Hill Education (2020)

Rogers, E.M.: Diffusion of Innovations (5th ed.). Free Press (2003)

Russell, S., Norvig, P.: Artificial Intelligence: A Modern Approach (3rd edn.). Pearson (2016)

Sova, R., Tudor, C., Tartavulea, C.V., Dieaconescu, R.I.: Artificial Intelligence tool adoption in higher education: a structural equation modeling approach to understanding impact factors among economics students. Electronics **13**(18), 3632 (2024). https://doi.org/10.3390/electronics13183632

Woodruff, K., Hutson, J. and Arnone, K.:Perceptions and barriers to adopting artificial intelligence in K-12 education: a survey of educators in fifty states (2023). Faculty Scholarship. 506. https://digitalcommons.lindenwood.edu/faculty-research-papers/506

Driving Governance and Adoption in Custom Faculty Workload System Integrations within Cloud ERP for Community Colleges

Sanjiv Kumar Bhagat(✉)

Visvesvaraya Technological University, Belagavi, India
reach.sanjiv.bhagat@gmail.com

Abstract. Community colleges with shared-services PeopleSoft use cases have historically seen multiple iterations of missed part-time teaching workload functionality because ERP architectures designed for full-time faculty do not always accommodate variable load units, term-based contracts, and/or complex compensation schedules. The FWL bolt-on creates an extension layer built upon PeopleTools that retains all delivered core PeopleSoft functionality, adds a parallel data model, multi-step workflow approvals through PeopleSoft's Approval Workflow Engine, and tightly governed integration touchpoints with Human Capital Management, Campus Solutions, and Payroll for North America. For institutional data, the security architecture is layered, role-based, and scoped to multi-college consortia boundaries. Data governance, in three tiers (standards and procedures plus configurable and local exceptions), parallels the federal theory of information technology governance and the COBIT framework for the governance of enterprise IT. This distinguishes governance from management. The change management layer consists of the PROSCI/ADKAR model, which maps the elements of individual change to the primary interventions required to deploy awareness campaigns, champion networks, role-based training, hands-on practice environments, and post-deployment sustainment. These three layers of architecture, governance, and change management combine to provide a blueprint for the extension of ERP stacks in complex multinational, multi-institutional higher education contexts where technical realization alone is not sufficient to ensure operational success.

Keywords: Faculty workload management · PeopleSoft bolt-on architecture · shared-services ERP governance · multi-institution HCM integration · ADKAR change adoption

1 Introduction—ERP Gaps in Managing Higher Education Faculty Workload

Research Approach and Contribution. This paper presents a design science artifact: a replicable architectural blueprint for extending PeopleSoft HCM to support part-time faculty workload management in shared-services community college consortia. The primary

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 19–35, 2027.
https://doi.org/10.1007/978-3-032-39763-8_2

evidence base is the Washington State Board for Community and Technical Colleges (SBCTC) ctcLink program, specifically the CEMLI SM-054 Sick Leave Buyout bolt-on (Bhagat et al., 2019–2024), which provides a production-validated implementation of the same PeopleTools patterns applied in the Faculty Workload (FWL) bolt-on design. This source is an internal design document governing a live multi-institution deployment; its use is consistent with design science and practitioner-experience research methodologies, where documented system artifacts constitute primary evidence [2]. The contribution of this paper is threefold: (1) a technical architecture pattern for PeopleSoft bolt-on development that preserves upgrade compatibility in shared-services environments; (2) a three-tier governance classification model for managing requirements variability across multi-institution ERP consortia; and (3) an ADKAR-mapped change management framework for achieving adoption in high-turnover, institutionally autonomous higher education settings. The paper does not present controlled experimental results; rather, it synthesizes design patterns from a documented production implementation into a transferable blueprint supported by peer-reviewed ERP, governance, and change management literature.

Historically, ERP platforms available to higher education were based on full-time staffing models, standard pay rates, and semester-length calendars. These assumptions create systemic functional gaps when institutions need to record and account for part-time instructional workload with variable load units, term-based contracts, and variable salaries [1].

Community colleges in particular have been affected. For example, at 1,209 public two-year colleges in the United States surveyed in 2001, the average ratio of part-time to full-time instructional staff was 60.55%, with some approaching 100% contingent [1]. The 1999 National Study of Postsecondary Faculty showed that full-time community college professors made $46,636 annually for teaching, whereas part-time instructors made $9,782 per year. While the average part-time professor taught 8.4 h of instruction per week, full-time professors taught 17.2 h per week [1]. This creates an operationally untenable situation for native ERP tooling to govern this high-volume, structurally different workforce.

Costs are not just payroll. When ERP systems cannot collect and propagate workload data natively, organizations may need to deploy workarounds — including spreadsheets, share-based approvals, or shadow systems — exposing the organization to audit and compliance risk. This finding is also consistent with the ERP package-organization misalignment research, which found that such misalignments are often not identified until late in an ERP implementation. In a case study of a multi-hospital system, there were 442 misalignments across three hospitals; 38% were imposed structural misalignments (regulatory, industry, or policy requirements that the ERP does not support without modification) [2]. Between 89% and 93% of imposed misalignments were resolved through package modification rather than organizational adaptation, a reflection of the bolt-on nature of development in regulated policy environments [2].

This challenge is considerably more complex for shared-services ERP architectures, where dozens of institutions are using a shared product. In this case, every extension or new feature must have an impact on every institution considered, converting a simple customization into a governance design question. This article provides a repeatable blueprint for that challenge (Fig. 1).

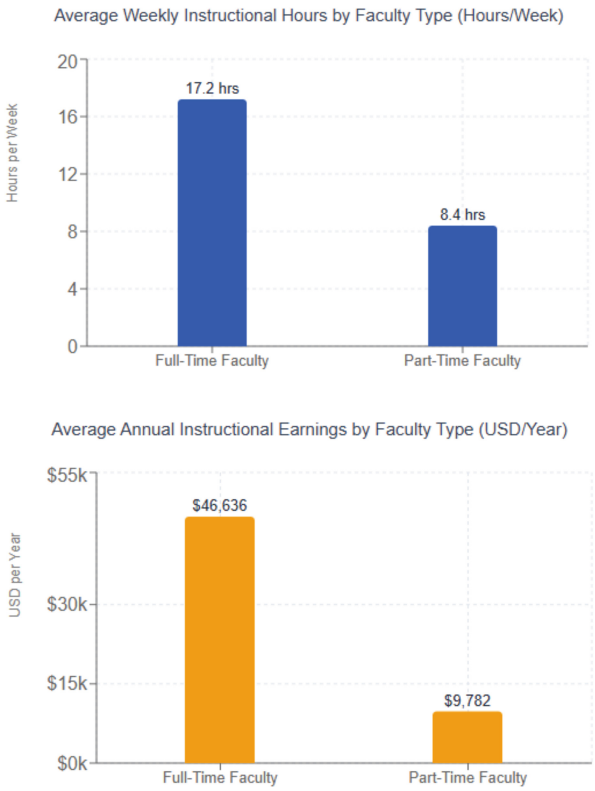


Fig. 1. Community College Faculty Comparison (1999 NSOPF Data) [1, 2]

2 Bolt-on Architecture—A Functional and Technical Extension Pattern in PeopleSoft

2.1 What is the Bolt-on Approach?

PeopleSoft bolt-on solutions are custom application layers built using PeopleTools' development tools — PeopleCode, Application Engine, Component Interface, Integration Broker, and related utilities — that add functional capability to PeopleSoft HCM without modification to standard delivered objects. This distinction is important from an architectural standpoint, since modifications to core objects make future upgrades problematic, create supportability issues, and result in version-specific dependencies that are particularly unwelcome in shared-services organizations subject to a frequent PeopleTools and application maintenance program [3].

The Faculty Workload (FWL) bolt-on, based on the "vanilla core, extended layer" model, does not change delivered PeopleSoft objects such as job records, position data, earning codes, or payroll run controls. Instead, it introduces a parallel data layer with load equivalency units, course section identifiers, assignment type, contract terms, and funding source to manage the information life cycle of a workload record from initial approval through downstream integration into payroll. This is consistent with the

general ERP modernization principle that organizations should minimize customization — keeping the core package as delivered and satisfying unique requirements through structured extension — as research suggests this leads to substantially lower operating and maintenance costs [3] (Fig. 2).

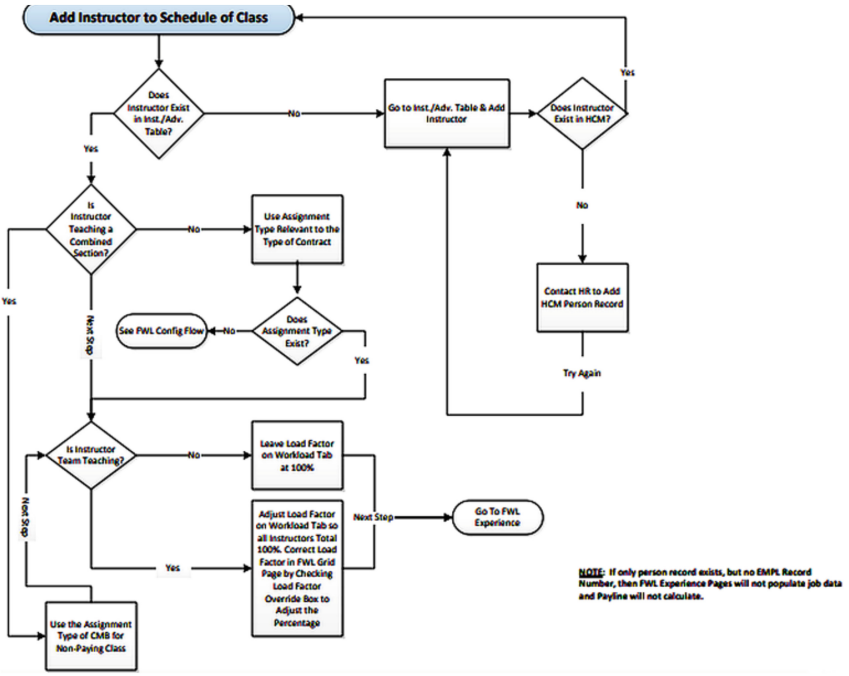


Fig. 2. FWL instructor setup and assignment type decision flow

2.2 Data Model and Component Architecture

The FWL records — FWL_ASSIGNMENT, FWL_COURSE_DTL, FWL_APPROVAL_HDR/LN, FWL_CALC_RESULT, and FWL_AUDIT_LOG — are joined to PeopleSoft's EMPLOYMENT and JOB records using the employee ID/record number. Together, these records maintain a complete workload history without requiring the use of PeopleSoft's natural effective-dated job row structure, thereby avoiding the row proliferation associated with high-volume management of part-time assignments.

The architectural pattern is illustrated concretely in the design of related bolt-on implementations deployed across the Washington State Board for Community and Technical Colleges (SBCTC) ctcLink shared environment [13]. In one such implementation (CEMLI SM-054), the custom data layer consists of distinct functional record tiers: staging records (e.g., CTC_SLVLD_STG, CTC_SLVPY_STG, CTC_SLVAM_STG), permanent detail records (CTC_SLV_STG_DTL), run control tables (CTC_SLV_RUN_TBL), and process state records — Application Engine State

Records such as CTC_SLVLD_AET, CTC_SLVPY_AET, and CTC_SLVAM_AET [13]. Each staging record captures transactional data specific to its processing stage, while the detail table (CTC_SLV_STG_DTL) serves as the canonical employee-level record, storing balance fields, computed amounts, consent flags, and audit timestamps (LASTUPDOPRID, LASTUPDDTTM) [13]. This multi-tier staging architecture isolates each processing phase — data load, payroll posting, and absence adjustment — enabling independent rerunning of individual stages without cascading side effects on other modules.

The FWL bolt-on follows the same principle. Staging records temporarily hold computed load data during each processing cycle. Upon successful computation, records are promoted to the permanent detail layer (FWL_CALC_RESULT), where they become available for downstream workflow routing and payroll integration. This design explicitly avoids writing directly to PeopleSoft's PS_JOB or PS_EARNINGS tables during intermediate steps, preserving referential integrity with the delivered data model.

State records (AET records) serve a critical role in Application Engine processing. In the SM-054 pattern, fields such as COMPANY, YEAR, PAY_END_DT, CAL_ID, and ERNCD are bound at runtime from run control tables and carried forward as AET-bound variables across all sections and steps of the Application Engine program [13]. This eliminates redundant database lookups and ensures consistent binding throughout the process lifecycle. The FWL AET records follow the same convention, binding institution code, academic term, assignment type, and payroll calendar identifiers at the start of each processing run.

The front-end UI is composed of PeopleSoft Component framework components for departmental initiators, dean-level approvers, and HR administrators in central HR, alongside reporting dashboards built on PeopleSoft Query and nVision. For the AI-augmented analytics layer, PeopleSoft HCM implementation studies show that properly sized and configured Integration Broker architectures can be run at 99.94% availability and load-tested at 312 transactions per second with average latencies of 84 ms, benchmarks that are relevant for real-time workload payroll-readiness monitoring [4].

2.3 Application Engine Processing Logic

Application Engine (AE) programs are the primary batch processing mechanism for the FWL bolt-on, responsible for extracting source data, applying business rules, computing derived fields, and posting results to downstream systems. The AE architecture follows a modular section-and-step pattern, with a public MAIN section orchestrating calls to private sections for discrete processing phases [13]. This modularity supports independent unit testing of each phase and simplifies ongoing maintenance (Figs. 3 and 4).

The processing logic across bolt-on AE programs in the SBCTC pattern follows a consistent sequence [13]. In the staging load phase (LDSTG section), the program first clears existing staging data for the company to ensure idempotency, then executes a multi-join INSERT statement against source tables in the FWL case, GP accumulator results, JOB effective-dated records, and course-section reference data filtered by active employment status, valid pay group, and company-scoped PINs. Subsequent UPDATE steps compute derived fields sequentially: beginning balance is calculated as ending balance minus accruals plus usage; time available applies eligibility thresholds by employee

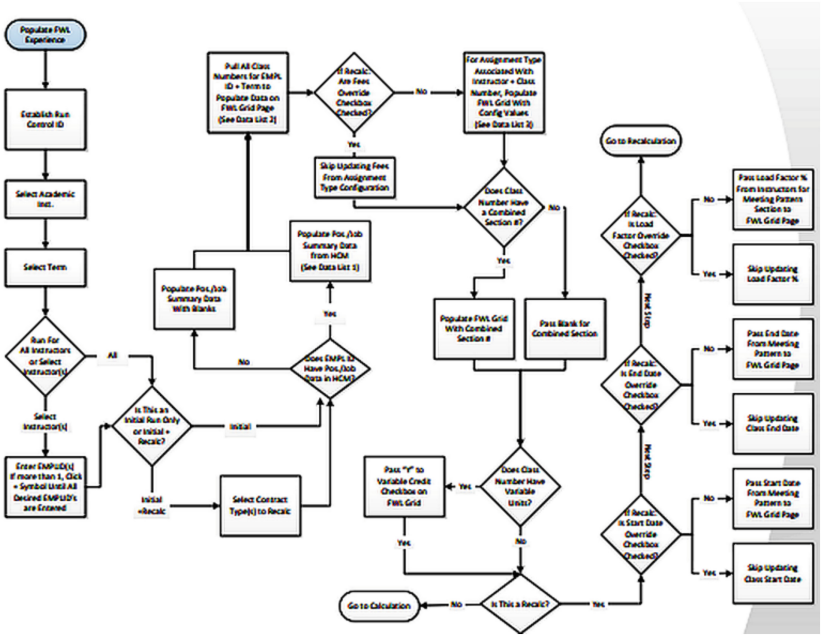


Fig. 3. FWL experience population and run control configuration flow

class; time payable applies a contractual percentage; and the final buyout or compensation amount is calculated as the product of time payable and the hourly rate, rounded to two decimal places. Records with zero or negative computed amounts are then deleted from staging, ensuring only payable transactions proceed to the next phase [13].

A critical design consideration is the use of crosswalk tables (PS_CTC_CRSWLK_INT2 in the SBCTC pattern) to externalize business rules that vary by institution — such as eligibility thresholds by employee class, PIN mappings for absence accumulator types (BAL, ENT, TAKE), and payable time percentages [13]. Rather than hard-coding these values in SQL or PeopleCode, the AE queries the crosswalk at runtime using parameterized lookups on PROCESSNAME, CTC_CNV_GRP_ID (company), FIELDNAME, and CTC_CONDITION_1. The FWL bolt-on applies the same pattern, externalizing load-unit-to-compensation conversion rules, term-based contract thresholds, and moonlight assignment routing conditions into configuration tables governed under the Tier 2 configurable parameter framework described in Sect. 5.

Guard logic is implemented at the start of each AE run to prevent re-execution after irreversible downstream steps have completed [13]. In the SBCTC pattern, the staging load process checks whether the payroll load flag (CTC_SLV_PAYROLL_SW) is already set to 'Y' in the run table, and aborts with a descriptive error if so. Similarly, the payroll load process validates that the pay calendar confirmation has not already been processed before inserting records into PSHUP_TXN. These guard checks are recorded in the run control table (CTC_SLV_RUN_TBL) as status flags, providing a persistent audit trail of which phases have completed for each company-year combination [13].

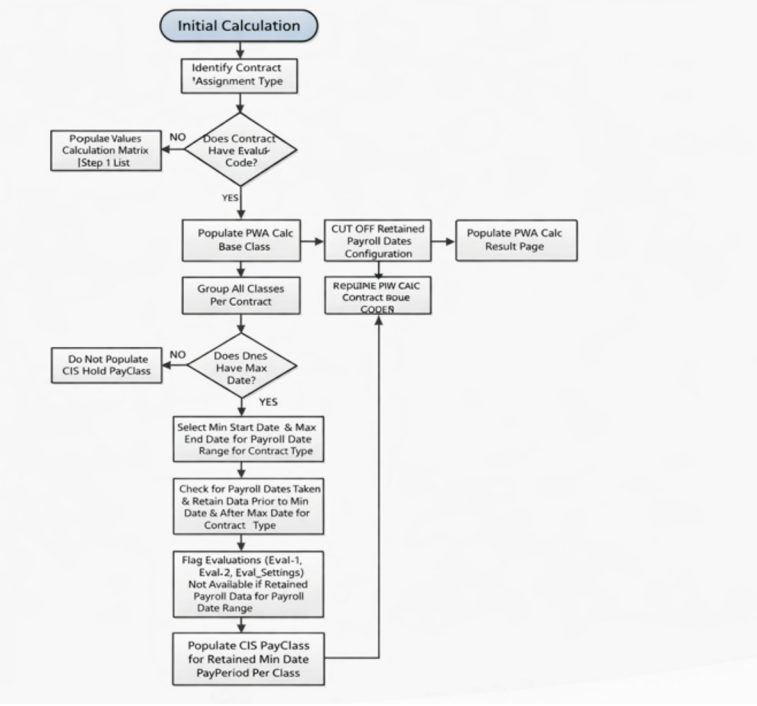


Fig. 4. FWL initial calculation and contract payline generation flow

The FWL bolt-on maintains an equivalent FWL_PROCESS_STATUS record to track the state of each processing cycle at the institution and term level.

2.4 PeopleCode Integration Patterns

PeopleCode events are used throughout the bolt-on component layer to enforce business rules, manage UI state, and trigger workflow actions. Several patterns recur across bolt-on implementations [13].

FieldChange events handle real-time recalculation of derived fields when the user modifies an input. In the SBCTC SM-054 pattern, when the Time Available field is changed by a Leave Administrator, a FieldChange event immediately recomputes the payable time and buyout amount fields, ensuring internal consistency without requiring a page save [13]. The FWL component layer follows the same pattern for load equivalency unit fields: modifying raw instructional hours triggers recalculation of load units and compensation amounts in real time.

SaveEdit events enforce pre-save validation, checking run control guard conditions before allowing the process to proceed. In the SBCTC pattern, a SaveEdit on the run control record queries the run table for the relevant process completion flag and raises a message catalog error if the process has already run, preventing duplicate processing [13].

SavePreChange and SavePostChange events handle audit field maintenance and post-save workflow triggers [13]. LASTUPDOPRID and LASTUPDDTTM are populated in SavePreChange using %OperatorId and %Datetime system variables. Post-save, the component checks for changed fields across grid rows and updates audit fields only for rows where data has changed, avoiding unnecessary database writes.

Workflow trigger events fire after a successful save when an employee submits a consent or approval action. In the SBCTC pattern, TriggerBusinessEvent is called with the relevant BusProcess, BusActivity, and BusEvent parameters, initiating an email notification via a configured workflow business process [13]. The FWL bolt-on uses the same mechanism to trigger AWE routing events when an assignment record is submitted, automatically routing it to the appropriate approver based on institution, department, and assignment type.

Fluid component patterns are used for Employee Self Service pages [13]. The Fluid consent page uses an Activate event to dynamically control groupbox and field visibility based on the employee's current consent status, whether the consent window is open, and whether data has been loaded for the employee in the current cycle. The same conditional visibility pattern is applied in the FWL Fluid self-service component, where faculty can view their current workload assignments and load unit balances only during the designated review window, with consent and acknowledgment controls conditionally rendered based on assignment status and institutional configuration.

2.5 Security Role Architecture

For a multi-college shared environment, security is enforced at the institutional level as well as through a statewide supervisory model. This is achieved by implementing institution-scoped roles within Department Security Tree boundaries and SetID boundaries, granting read-only cross-institution access for auditing, and assigning service accounts with system-level roles following the principle of least privilege. PeopleSoft controls access to data via role-based security, ensuring that each role has access to only what is required for their position while preventing unauthorized access to data belonging to other institutions [3].

In practice, the SBCTC bolt-on implementations enforce security at the component search record level [13]. Search views (e.g., CTC_PERS_SRCHVW, CTC_SLV_DTL_VW) are defined with company-scoped joins against security view tables (CTC_CMPY_SEC_VW), ensuring that users retrieving records through the component interface only see data for companies to which their operator ID has been granted access [13]. This row-level security pattern complements PeopleSoft's permission list architecture and is applied uniformly across all FWL components. A survey of public sector organizations using PeopleSoft found that organizations spending 27.3% of total project effort on security and compliance controls experience 83.4% fewer compliance incidents [4]. This data informed the layered security model and permission list architecture for the FWL bolt-on.

3 Integration Architecture — HCM, Campus Solutions, and Payroll Touchpoints

3.1 Integration Design Principles

The Faculty Workload bolt-on sits at the intersection of PeopleSoft's three core functional modules: Human Capital Management (HCM), Campus Solutions (CS), and Payroll for North America (PNA). The design goals are anchored in data ownership and latency requirements for each module. PeopleSoft cloud migration analysis shows that institutions adopting phased, systematic approaches to cloud integration achieve 40% faster migration times and fewer post-migration support problems compared to ad hoc approaches, a finding with direct corollary value for phased FWL integration strategies in multi-institution consortia [5].

Integrations are hub-and-spoke, with the FWL bolt-on as the hub. It begins with reference data from HCM and CS, applies workload-specific business logic, and produces outbound payroll deliverables for PNA. Integration Broker is used to perform synchronous lookups and deliver asynchronous event-driven messages, avoiding the tight coupling and complex upgrade sequencing that cross-application table joins would entail.

3.2 HCM Integration

HCM supplies employee identity, employment record structures, position data, organizational hierarchy, and pay group assignments. These are consumed by the FWL bolt-on through Component Interface calls for real-time eligibility checking. Integration Broker subscription-based messaging receives JOB record change events — such as terminations, leaves of absence, and deactivation of job records — triggering hold or cancellation actions on in-flight assignments. PeopleCode routines employ effective-date awareness mechanisms, binding workload records to valid employment statuses using the same max-effective-date/max-effective-sequence pattern used throughout PeopleSoft delivered processing [5].

In the SBCTC bolt-on pattern, HCM integration for batch processing is implemented via direct SQL joins to PS_JOB with effective-date subqueries [13]. The staging load SQL uses a correlated subquery to retrieve the maximum EFFDT less than or equal to the processing end date, followed by a second subquery for the maximum EFFSEQ at that EFFDT, ensuring that the most current employment record is retrieved without selecting future-dated rows [13]. The FWL batch processes follow the same effective-date join pattern when binding workload records to employment status, pay group, and compensation rate.

3.3 Campus Solutions Integration

Campus Solutions course and section data provide the instructional context for FWL course-section assignments. Because HCM lacks native course-section awareness and CS lacks payroll processing capability, the FWL bolt-on bridges the gap by periodically

importing active section records into a local reference table through scheduled Application Engine processes, enforcing referential integrity against active CS enrollment states, and supporting cross-institution "moonlight" assignment patterns via inter-institution section data consumption and pay group routing logic [6].

3.4 Payroll Integration and Exception Management

The downstream payroll touchpoint represents the highest business risk, as errors directly affect employee compensation. The integration works in two directions. In the outbound direction, the Earnings Load Application Engine — analogous to CTC_SLVPY_AE in the SBCTC pattern [13] — converts approved workload records into PSHUP_TXN entries for each payroll calendar. Specifically, for each employee in the staging table, the program creates a PSHUP_TXN record with PU_SOURCE set to 'OT', the appropriate earning code, OTH_HRS populated with the time available field, PU_AMT set to the calculated compensation amount, and OK_TO_PAY set to 'Y' [13]. The record is keyed by company, employee ID, employee record number, paygroup, pay end date, and a sequential SEQNO to support multiple earnings entries per employee. Before inserting, the program validates that the target payroll calendar has not already been confirmed (PAY_CONFIRM_RUN = 'Y'), aborting with an error if so to prevent double-payment [13].

In the inbound direction, a post-payroll Confirmation and Reconciliation process — analogous to CTC_SLVAM_AE [13] — retrieves confirmed payroll results from PAY_EARNINGS and PAY_OTH_EARNS, reconciles them against the staging table, and flags discrepancies such as hours sent to payroll that were not paid, or absence adjustment hours that do not match payroll hours. The reconciliation log is written using a structured logging class (CTC_INTERFACE: InterfaceProcess), producing a named log file that captures run parameters, section headers, and row-level detail messages [13]. This audit trail supports the exception queue reviewed by central payroll staff without requiring technical involvement.

Absence balance adjustments are posted via Component Interface rather than direct SQL updates, preserving PeopleSoft's delivered absence management rules and audit framework [13]. For each employee, the program retrieves the GP_PI_MNL_DATA collection via the Component Interface, locates the relevant PIN entry, and adjusts the BAL_ADJ field by the negative of the time available value, reversing the balance for hours paid out. If the PIN entry does not yet exist in the collection, a new item is inserted. Error handling captures PSMessages from the session object, logs them to the run log, and issues warnings rather than hard errors where possible, allowing the batch to continue processing remaining employees while recording failures for manual review [13] (Fig. 5).

PeopleSoft's exception management framework assigns severity levels to processing errors. High-severity exceptions block payment processing; medium- and low-severity exceptions require review before closure. Each exception is assigned a unique number, timestamp, and originating process name [6]. The FWL Administration Component surfaces an exception queue that allows FWL central payroll staff to review, correct, and resubmit failed records without technical staff involvement, mirroring the SBCTC pattern [13]. Institutions of higher education have reported up to 35% cost savings on

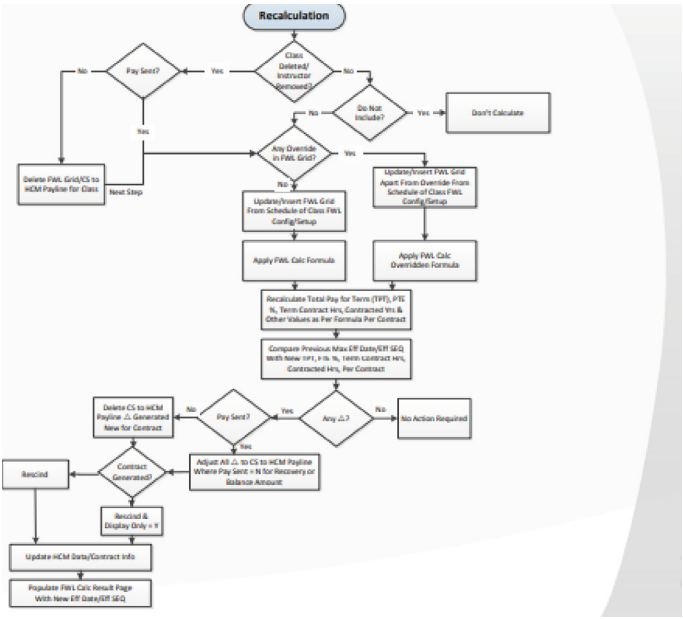


Fig. 5. FWL Recalculation, Pay Recovery, and Rescind Logic Flow

infrastructure and a 50% increase in system uptime on the PeopleSoft Cloud, improvements that directly enhance the reliability of this payroll integration pipeline [5] (Fig. 6 and Table 1).

Table 1. FWL bolt-on integration touchpoints by module [5, 6]

PeopleSoft Module	Data Consumed by FWL	Data Produced by FWL	Integration Mechanism
Human Capital Management (HCM)	Employee identity, employment records, position data, pay group assignment	Workload-linked employment records	Component Interface, Integration Broker (subscription messaging)
Campus Solutions (CS)	Course and section data, enrollment states	Cross-institution moonlight assignment routing	Scheduled Application Engine, local reference table
Payroll for North America (PNA)	Post-payroll confirmation results	Approved earnings load entries, term-level compensation reports	Application Engine (Earnings Load & Reconciliation)

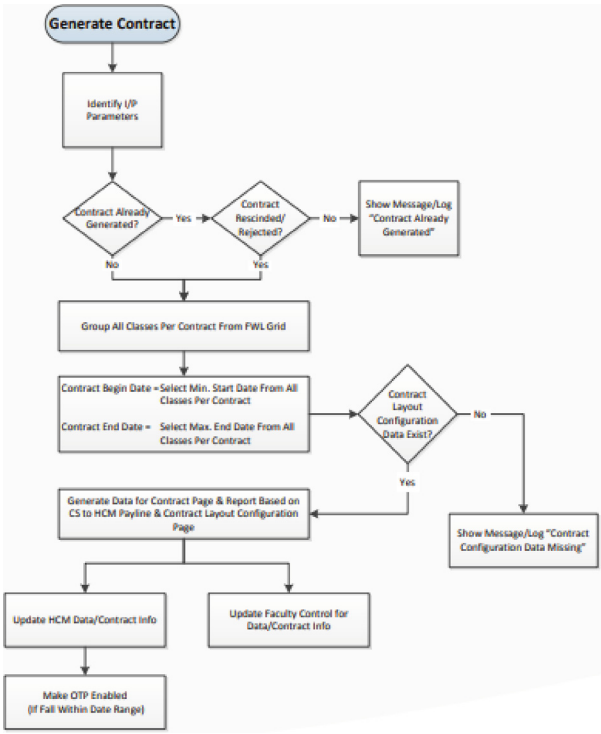


Fig. 6: FWL contract generation and OTP enablement flow

4 Automation and Security Role Design for Multi-institution Processing

As a bolt-on, the FWL uses PeopleSoft's Approval Workflow Engine (AWE) to automate approval workflows through declarative configuration of approval paths, escalation rules, and notifications in shared-service environments. AWE was selected because its configuration layer allows institutions to change approval paths without modifying code, critical in a shared environment where approval chains differ across institutions [7].

The approval workflow consists of six states: Draft, Submitted, In Review, Approved, Returned, and Cancelled. Routing is determined by a combination of institution code, department, assignment type, load threshold, and funding type. Standard assignments below the institutional maximum require a single-step dean approval; cross-institution moonlight assignments require three-step approval through the dean, HR/Workforce, and HR/Workforce Central Payroll offices [7].

Escalation thresholds are configurable, with worklist reminders firing at 48-h and 96-h intervals after routing. Escalation events are recorded in FWL_APPROVAL_HDR, providing a complete audit trail. The escalation record includes the worklist instance ID, transaction ID, employee ID, name, and business process/activity/event identifiers — fields that align directly with the CTC_SLV_BO_WL worklist record pattern used in the SBCTC bolt-on framework, which stores BUSPROCNAME, ACTIVITYNAME,

EVENTNAME, WORKLISTNAME, INSTANCEID, TRANSACTIONID, EMPLID, and NAME as key fields for workflow tracking [13].

Workflow email notifications are triggered from PeopleCode Workflow events on the component. After a successful save, TriggerBusinessEvent is called with the configured business process, activity, and event parameters. Email body and subject text are retrieved dynamically from the PS_HR_SSTEXT_TEXT configuration table using OBJECTOWNERID, HR_SSTEXT_SUB_ID, TEXT_ID, and an effective-date subquery — a pattern that allows notification text to be updated centrally without code changes [13], consistent with the Tier 2 configurable parameter framework.

The FWL security role matrix is based on six PeopleSoft roles, FWL_INITIATOR through FWL_INTEGRATION, and uses a configuration registry to account for institutional variations from the common statewide baseline. Component search records are restricted to company-scoped security views, and service account roles follow least-privilege principles as described in Sect. 2.5.

The architecture aligns with broader workflow standards. A survey of more than ten scientific workflow systems — including Taverna, Kepler, Pegasus, and Triana ound that standards-based workflow approaches are more efficient than proprietary solutions over the long term [8], a finding that reinforces the use of AWE and Integration Broker as standards-based workflow infrastructure rather than custom-coded routing logic.

5 Governance Model for Shared-Services Enterprise Resource Planning in Multi-College Consortia

Governance of a shared PeopleSoft environment differs fundamentally from governance within a single institution. The central technology organization must balance standardization — enabling shared support and economies of scale — with flexibility to meet legitimate institutional policy, process, and contract needs. Without governance, individual institutional requirements become one-off customizations that erode the common baseline [9].

The FWL bolt-on governance model addresses this tension through a three-tier requirement classification. Tier 1 requirements are met by the default solution without institution-specific configuration. Tier 2 requirements are fulfilled by adjustable parameters in configuration tables — such as the crosswalk table entries that externalize employee class eligibility thresholds, PIN mappings, and payable time percentages without requiring code changes [13]. Tier 3 consists of local exceptions that require institution-specific code, subject to architectural review board approval before implementation. This structure creates a natural disincentive to unnecessary local customizations and is documented in Architectural Decision Records (ADRs).

The version history of the SBCTC SM-054 bolt-on illustrates how this governance model operates in practice over time [13]. Between 2019 and 2024, the implementation went through eight versioned changes: an initial deployment, a technical design revision, two remediation tickets affecting SQL and UI logic (tickets 130227 and 157329), a staging table cleanup enhancement (157888), a PIN crosswalk extension (ER151/129297), a year-selection SQL fix to include company scoping (SDT 184993), and an absence adjustment fix to support single-check processing (187910) [13]. Each change was

tracked in a revision log with date, author, version number, and change reference, providing a transparent audit trail consistent with the COBIT Evaluate-Direct-Monitor model. The progression from hard-coded eligibility thresholds and PIN filters to crosswalk-driven configuration illustrates the practical evolution from Tier 3 toward Tier 2 design as a bolt-on matures within the shared environment [13].

Sambamurthy and Zmud (1999), using case studies of eight firms with revenues between $200 million and $3.25 billion, found that corporate governance, economies of scope, and absorptive capacity explain IT governance modes across reinforcing, conflicting, and dominating contingency types [9]. The FWL consortium model is a conflicting contingency: centralization forces from shared infrastructure reach equilibrium with decentralization forces from institutional autonomy.

This is consistent with COBIT 5's distinction between governance and management. De Haes, Van Grembergen, and Debreceny (2013) describe COBIT 5's 37 IT processes grouped into five governance and management domains and a seven-stage EGIT implementation lifecycle [10]. FWL software releases adhere to a monthly maintenance, quarterly enhancement, and emergency release schedule governed by a Change Advisory Board (CAB). The Requirements Registry, reviewed quarterly in governance sessions with more than thirty participating institutions, establishes transparency and pattern-based prioritization across the community-wide installation [10] (Table 2).

Table 2. Firm profile in Sambamurthy and Zmud (1999) case studies [9, 10]

Parameter	Value
Minimum firm revenue	$200 million
Maximum firm revenue	$3.25 billion
Core contingency forces identified	Corporate governance, economies of scope, absorptive capacity
Governance scenarios identified	Reinforcing, conflicting, dominating

6 Change Management and Adoption Framework (PROSCI/ADKAR) in Statewide Deployment

In many shared ERP systems, technically correct solutions fail to achieve adoption because end users continue their prior processes outside the ERP. Resistance stems from high switching costs, learning curve friction, and the uncertainty inherent in any system change — factors compounded in higher education by institutional autonomy [11].

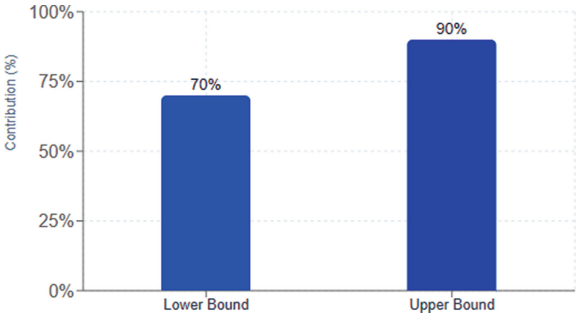
The FWL bolt-on adoption framework maps onto the PROSCI/ADKAR model, identifying five building blocks that must be achieved in sequence: Awareness, Desire, Knowledge, Ability, and Reinforcement [12]. Any one of these elements, if insufficiently developed, creates a "barrier point" (Hiatt, 2006, p. 85) that stops change from progressing.

Each ADKAR element corresponds to specific FWL deployment interventions. Awareness is built through communications from institutional presidents, vice presidents, and HR directors articulating the compliance risks of shadow systems and the limitations of manual workload processing. Desire is cultivated by recruiting institutional change champions from UAT cohorts — Kotter (1995) found that a lack of a sponsorship coalition was among the most common causes of transformation failure across 100 organizations [11]. Knowledge is delivered through a statewide role-based training program for initiators, approvers, HR administrators, and payroll coordinators. Ability is developed through hands-on practice in non-production training instances before go-live. Reinforcement is sustained through a tiered help desk, quarterly user forums, and the Requirements Registry, which provides a formal channel for surfacing regressions to manual workarounds.

The iterative change history of the SBCTC SM-054 bolt-on is itself an artifact of this reinforcement dynamic [13]. Post-deployment, end users and administrators identified process friction points — the consent form required password re-entry, causing drop-off; staging tables were not cleared between runs, causing confusion; the time payable field was editable, allowing inconsistent overrides [13]. Each of these was captured through the support ticket process and resolved through targeted code changes. This feedback-driven iteration loop is a structural manifestation of the Reinforcement stage: the system is tuned to user behavior patterns rather than requiring users to adapt to system constraints [13]. The three Application Engine programs and two custom pages that constitute the SM-054 technical footprint [13] grew through five discrete post-deployment revisions between 2022 and 2024, each traceable to a specific user-reported friction point a ratio of post-deployment refinements to initial delivery that is consistent with the ADKAR model's expectation that reinforcement requires sustained, iterative effort rather than a single post-go-live support wave [12].

The four-tier training model, including 15-to-20 min awareness modules and iteratively updated reference materials with each release cycle, is designed to onboard high-turnover administrative roles without requiring full retraining on each update.

Kotter (1995) estimated that leadership accounts for 70 to 90% of successful organizational change [11], which explains the emphasis on senior sponsorship in the awareness and desire stages of the FWL model. A 2005 survey of 411 organizations found that the primary cause of resistance to change was a lack of understanding of the reason for the change [12], reinforcing the FWL model's investment in structured communication before and during deployment. Figure 7 illustrates this range: the Lower Bound (70%) and Upper Bound (90%) values are drawn directly from Kotter's (1995) stated estimate that leadership accounts for 70–90% of successful organizational change outcomes [11]. The bar chart presents these as a bounded range rather than a single point to reflect the uncertainty inherent in Kotter's original practitioner estimate and to distinguish it from empirically measured outcomes.



7 Conclusion

Deliberate alignment of architecture, governance, and human adoption into a cohesive operating model is necessary to realize the promise of optimal part-time faculty workload management across multiple shared PeopleSoft environments. The FWL bolt-on architecture demonstrates that sustainable ERP customization in community college consortia is achievable through four interlocking design principles: preserving the delivered core through a parallel data layer; respecting system-of-record boundaries through Component Interfaces and Application Engine batch processing; enforcing institutional data sovereignty through company-scoped search records and role-based permission lists; and externalizing processing logic into configurable crosswalk tables rather than hard-coded SQL. These patterns are drawn from production implementations operating across dozens of institutions with a documented multi-year revision history [13]. Federal IT governance principles and the COBIT framework drive a standardized approach to requirements variability, while the three-tier classification model preserves institutional flexibility within a governed baseline. The PROSCI/ADKAR change management model ensures that the engineered solution is matched by an equally deliberate effort on the human side of enterprise change — building awareness, desire, knowledge, ability, and reinforcement for end-user populations that would otherwise revert to manual processes, as evidenced by the iterative post-deployment refinements the SBCTC shared environment required over multiple release cycles [13]. Managing these technical and organizational domains together remains the fundamental challenge of shared-services ERP implementations in higher education. The FWL integrated architecture and change management framework provides a transferable blueprint for consortia working through the same challenge.

References

1. Jacoby, D.: Effects of Part-Time Faculty Employment on Community College Graduation Rates. Taylor & Francis (2020). https://www.nccft.org/wp-content/uploads/2020/01/effects-of-part-time-faculty-employment-on-community-college-grad-rates.pdf
2. Soh, C., Sia, S.K.: An institutional perspective on sources of ERP package–organization misalignments. J. Strateg. Inf. Syst. (2004). https://www.academia.edu/10245304/An_institutional_perspective_on_sources_of_ERP_package_organisation_misalignments

3. Rawat, C.: Role of ERP modernization in digital transformation: PeopleSoft insight. J. Comput. Sci. Technol. Stud. (2023). https://arxiv.org/pdf/2303.03224
4. Thontadarya, P.S.: The role of AI in modernizing PeopleSoft HCM: bridging ERP and innovation in public sector HR. J. Comput. Sci. Technol. Stud. (2025). https://al-kindipublishers.org/index.php/jcsts/article/view/10585/9314
5. Varatharajan, S.: PeopleSoft migration to Oracle cloud infrastructure: a comprehensive migration plan and execution strategy. Int. J. Sci. Res. (2025). https://www.researchgate.net/profile/Santhosh-Varatharajan/publication/389532320
6. University human resources: PeopleSoft 9.2 time processor procedure manual, human resources department (2016). https://hr.ucf.edu/wp-content/uploads/sites/17/PS_timeprocmanual.pdf
7. Joshi, R.: A practical guide to reporting and analytics on workday's core HCM and advanced compensation. Int. J. Trend Res. Dev. (2020). https://www.researchgate.net/profile/Dharmasena-Sd/publication/395657953
8. Barker, A., van Hemert, J.. Scientific workflow: a survey and research directions. https://www.researchgate.net/profile/Jano-Van-Hemert/publication/220698071
9. Sambamurty, V.: Arrangements for information technology governance: a theory of multiple contingencies. MIS Q. (1999). https://www.academia.edu/18243763/Arrangements_for_Information_Technology_Governance_A_Theory_of_Multiple_Contingencies
10. De Haes, S.: COBIT 5 and Enterprise governance of information technology: building blocks and research opportunities. J. Inf. Syst. (2013). https://www.researchgate.net/profile/Steven-De-Haes/publication/247778781
11. John P. Kotter, "Leading Change: Why Transformation Efforts Fail," Harv. Bus. Rev. (1995). http://www.lighthouseconsultants.co.uk/wp-content/uploads/2010/08/Kotter-Leading-Change-Why-transformation-efforts-fail.pdf
12. Hiatt, J.M.: ADKAR: A Model for Change in Business, Government and Our Community. Prosci Learning Center Publications. https://experianta.com/wp-content/uploads/atbdp_temp/ADKAR_-A-Model-for-Change-in-Bu-Jeff-Hiatt-
13. Bhagat, S.K., et al.: CEMLI SM-054 Sick Leave Buyout — Functional and Technical Design Document. Washington State Board for Community and Technical Colleges (SBCTC) ctcLink Program, Version 1.5, 2019–2024.

A Gate-Based Quantum Ising Model Approach to Solving KenKen Puzzles Using Binary Encoding

Wen-Li Wang[1(✉)], Mei-Huei Tang[2], Kevin Wang[1], Shahid Hussain[1], and Saif Ur Rehman Khan[3]

[1] Penn State University Behrend, Erie, PA 16563, USA
wxw18@psu.edu
[2] Gannon University, Erie, PA 16541, USA
[3] Shifa Tameer-E-Millat University, Islamabad 44000, Pakistan

Abstract. The KenKen puzzle is a classic example of a Constraint Satisfaction Problem (CSP) and has been proven to be NP-complete. CSPs are inherently tied to combinatorial structures and constraints, and quantum algorithms that leverage the principle of superposition are particularly well suited to addressing such challenging problems. As an extension and more advanced variant of Sudoku, KenKen shares certain structural constraints; however, it also incorporates arithmetic operations that introduce additional complexity and require specialized insight. Consequently, novel methods are needed to address these mathematical characteristics, while techniques developed for other hard problems can still be applied to handle the shared constraints. Building on prior work, this study extends a gate-based quantum Ising model, originally developed to solve Sudoku by minimizing the Hamiltonian energy, to address the KenKen puzzle. To reduce the number of required logical qubits, the proposed approach continues to adopt a base-2 binary encoding rather than a one-hot representation. The development process begins with identifying the rules shared across puzzle types and formulating mathematical expressions for the additional KenKen-specific constraints. Observable operators for the quantum models are then constructed using Pauli operators, enabling the computation of expectation values associated with individual rules. These operators are subsequently integrated into a complete KenKen solver designed for global optimization. The solver's validity is evaluated using sample puzzles implemented within the IBM Qiskit quantum framework, and the outcomes for 4×4 and 3×3 puzzles are presented and analyzed to confirm the correctness of the proposed approach.

Keywords: Quantum · KenKen · NP-complete · Ising Models · Hamiltonian Energy · IBM Qiskit

1 Introduction

The KenKen puzzle is widely used as a means of developing reasoning ability, arithmetic fluency, and problem-solving skills [1, 2]. It can be viewed as a generalization of Sudoku [3]. Like Sudoku, KenKen is an NP-complete combinatorial CSP and shares several

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 36–51, 2027.
https://doi.org/10.1007/978-3-032-39763-8_3

key characteristics with its predecessor. Originating in Japan, both puzzles are grid-based logic problems that emphasize deductive reasoning, requiring solvers to assign numbers in accordance with a predefined set of constraints. Consequently, they are widely regarded as effective instruments for cultivating logical reasoning abilities and continue to serve as popular forms of mental exercise.

As in Sudoku, a KenKen grid must be filled with digits from 1 to N for an $N \times N$ grid, and no digit may be repeated within any row or column. While KenKen allows N to be any positive integer, Sudoku requires the grid size to be a perfect square (i.e., $N = k^2$) so that it can be partitioned into equal-sized subgrids, or boxes. Consequently, a key distinction is that KenKen does not impose box constraints as Sudoku does; instead, it emphasizes arithmetic-based logic. Groups of cells, referred to as *cages* and outlined in bold, must combine to produce a target value using a specified operation ($+$, $-$, $\times$, or $\div$). Digits within a cage may repeat, provided they do not appear in the same row or column. Figure 1(a) presents a 4×4 KenKen puzzle obtained from the internet[1], while Fig. 1(b) shows a 3×3 KenKen puzzle. The latter includes a cell marked with a question mark and demonstrates a case in which $N \neq k^2$.

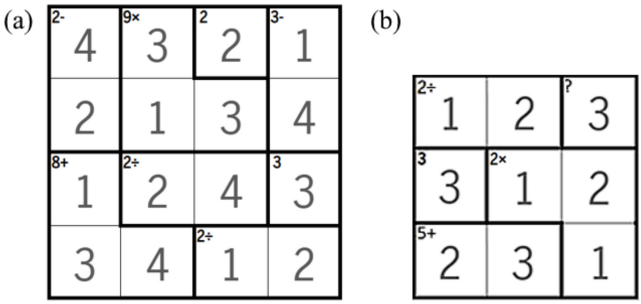


Fig. 1. Sample KenKen Puzzles: 4×4 and 3×3 Grids

For classical computers, a wide range of approaches including algorithmic methods [3], evolutionary programming [4], and artificial intelligence techniques [5] have been developed to address a variety of optimization problems, and many of these methods are also applicable to the KenKen puzzle. Since KenKen can be formulated as a CSP, it is particularly well suited to quantum algorithms for combinatorial optimization. Quantum computing leverages the principle of quantum superposition, which allows quantum states to exist simultaneously in multiple configurations, thereby enabling inherent computational parallelism. This parallelism constitutes a key advantage of quantum approaches over classical solvers.

In the quantum computing domain, only a limited number of puzzle solvers [6–9] have been proposed, often adopting distinct methodological perspectives. To reduce the required number of logical qubits and improve computational efficiency, a gate-based quantum solver for Sudoku based on the Ising model was introduced using a binary encoding scheme [10]. Motivated by the fact that NP-hard problems are often

[1] https://www.kenkenpuzzle.com

interreducible, the present work extends this framework to address KenKen puzzles. Our approach continues to rely on the quantum Ising model, where solving the puzzle is mapped to finding the ground state of an appropriately constructed Hamiltonian. A valid solution corresponds to the minimum energy configuration that satisfies all imposed constraints.

Within the Ising model, interactions between coupled elements are characterized by coupling coefficients. A positive coupling (+1), typically indicated by an upward arrow, favors alignment of spins in the same direction, whereas a negative coupling (−1), indicated by a downward arrow, enforces opposite orientations. By decomposing the problem into individual constraints and embedding them into a quantum formulation, the resulting optimization task can be addressed using algorithms such as the Quantum Approximate Optimization Algorithm (QAOA) [11], in conjunction with classical optimizers such as Constrained Optimization by Linear Approximation (COBYLA) [12] to iteratively refine algorithm parameters. These hybrid quantum–classical methods aim to compute the minimum eigenvalue of the constructed Hamiltonian, which corresponds to the optimal solution of the puzzle.

This paper is organized as follows. Section 2 reviews the relevant literature. Section 3 formulates the energy functions for the rules and constraints of KenKen. Section 4 describes the quantum modeling approach. Section 5 evaluates the performance of the proposed solver. Finally, Sect. 6 concludes the paper.

2 Related Work

KenKen has been widely modeled as a CSP and is generally regarded as NP-hard or NP-complete [13] via reductions from Latin square or Sudoku problems. It is a type of arithmetic and grid puzzle that fits within the framework of CSPs, which are closely related to combinatorial optimization. As a result, KenKen is frequently used to illustrate generic CSP techniques, including backtracking, forward checking, minimum remaining values (MRV), and arc consistency [14]. In addition to exact solution methods, a variety of heuristic and meta-heuristic approaches have been proposed to automate the solving of KenKen puzzles [15].

In quantum computing, prominent approaches include quantum search and backtracking methods based on Grover's algorithm [16] and Montanaro-style recursive techniques [17]. There are also global optimization frameworks, such as quadratic unconstrained binary optimization (QUBO) and graph-coloring formulations. In addition, quantum-inspired tensor network solvers [8] exploit concepts from quantum computing to enable efficient modeling on classical hardware. Nowadays, optimization algorithms [11, 18, 19] such as the QAOA and Adiabatic Quantum Optimization (AQO) have been widely applied to combinatorial optimization problems. These approaches typically formulate problems in terms of Hamiltonian energy functions [20, 21] and encode them as quantum Ising spin-glass models [22], which are known to be NP-hard. A representative example is the NP-complete Max-Cut problem [23], which has been successfully mapped to an Ising model using gate-based frameworks such as IBM Qiskit and Google Cirq. Prime factorization has also been modeled using Ising Hamiltonians [24] as an

alternative to the Quantum Fourier Transform employed in Shor's algorithm [25]. Collectively, these Hamiltonian-based modeling techniques offer promising opportunities for addressing increasingly complex combinatorial optimization problems.

Because many computationally hard problems are interreducible, the same modeling paradigm naturally extends to a broad class of optimization tasks. Consequently, this study is closely related to the quantum Sudoku solver proposed by Wang et al. [10], which similarly exploits the inherent parallelism of quantum computation. Both approaches employ a gate-based Ising modeling framework, using binary encoding to reduce logical qubit requirements and QAOA to identify the solution by minimizing the eigenvalue of the resulting energy interaction matrix. Owing to the shared structural constraints between KenKen and Sudoku, this work builds on several modeling techniques from the prior study. The primary distinction lies in the additional incorporation of arithmetic constraints unique to KenKen and their integration with the structural constraints shared with Sudoku to enable efficient puzzle solving.

New techniques continue to emerge and remain actively explored. Beyond QAOA, non-variational quantum walk–based optimization algorithms (QWOA) [26] and optimization by decoded quantum interferometry (DQI) [27] have been proposed as promising alternatives for enhancing performance and more effectively reducing the search space. Meanwhile, the error-prone nature of physical qubits motivates the adoption of quantum low-density parity-check (QLDPC) codes [28], which support high-rate quantum error correction (QEC) and enable low-overhead, fault-tolerant quantum computing.

3 Methodology

A classical Ising model can be expressed as a Hamiltonian function H of m spins, written as $H(s_0, s_1, \ldots, s_{m-1})$, where each spin $s_\alpha \in \{+1, -1\}$ for $0 \leq \alpha \leq m-1$. For an $N \times N$ KenKen puzzle, each cell can take a value from 1 to N. Rather than employing a one-hot encoding scheme that requires $m = N^3$ spins, this work adopts the approach introduced in [10] to reduce the number of required qubits to $N^2 \log_2 N$ by shifting the value range of each cell to begin at zero. Specifically, the original range $[1, N]$ is reformulated as $1 + [0, N-1]$, where the value $N-1$ can be represented in base-2 using k bits, as shown in Eq. (1).

$$k = \lceil \log_2 N \rceil \tag{1}$$

The benefits of this binary encoding scheme are expected to become increasingly significant as quantum computing technologies continue to scale, driven by growth in available qubit counts and the development of more efficient error-correcting codes, such QLDPC codes [28, 29]. In contrast to one-hot encoding, this approach also inherently prevents duplicate values within a single cell. The outputs of the k bits in Eq. (1), denoted as $b_0, b_1, \ldots b_{k-1} \in \{0,1\}$, are used to determine a cell's integer value v within the range $r = 1 + [0, N-1]$. The value v is uniquely computed as shown in Eq. (2).

$$v = 1 + \sum_{l=0}^{k-1} b_l \cdot 2^l \tag{2}$$

To model KenKen, a binary variable $x_{(i,j,l)}$ is defined for $0 \leq i, j \leq N-1$ and $0 \leq l \leq k-1$, indicating whether the l-th bit of the cell located at row i and column j is set to 1 or 0. Each tuple (i, j, l) uniquely maps to the index of a spin variable s_α, as specified in Eq. (3).

$$x_{(i,j,l)} = \frac{1 - s_\alpha}{2}, \text{ where } \alpha = i \times N \times k + j \times k + l \tag{3}$$

Using these binary variables, KenKen is formulated as an Ising model represented by a Hamiltonian function H, composed of subfunctions that encode the puzzle's constraints into a single objective function. The complete model consists of eight such subfunctions, as shown in Eq. (4), with their formulations presented in subsequent sections. Solving the puzzle is thus cast as an optimization problem, where minimizing the Hamiltonian yields the ground state $H = 0$.

$$H = \sum_{i=0}^{7} H_i \tag{4}$$

3.1 The Row and Column Rules

The row and column constraints are identical to those in Sudoku, requiring each row and each column to contain distinct integers with no repetitions. Accordingly, Eqs. (5) and (6) adopt established modeling results from [10], which define the energy subfunctions H_0 and H_1 to enforce the row and column constraints, respectively. In each subfunction, the product evaluates to 1 if and only if the two compared cells have the same value; otherwise, it evaluates to 0. The summation aggregates these pairwise comparisons over all relevant cell pairs. When $H_0 = 0$, all rows contain unique integer values, and similarly, $H_1 = 0$ ensures uniqueness across all columns.

$$H_0 = \sum_{i=0}^{N-1} \sum_{j_1=0}^{N-2} \sum_{j_2=j_1+1}^{N-1} \prod_{l=0}^{k-1} \left(1 - \left(x_{(i,j_1,l)} - x_{(i,j_2,l)}\right)^2\right) \tag{5}$$

$$H_1 = \sum_{j=0}^{N-1} \sum_{i_1=0}^{N-2} \sum_{i_2=i_1+1}^{N-1} \prod_{l=0}^{k-1} \left(1 - \left(x_{(i_1,j,l)} - x_{(i_2,j,l)}\right)^2\right) \tag{6}$$

3.2 The Preset Value Constraint

KenKen does not explicitly indicate preset values like Sudoku. However, a cage consisting of a single cell may have a value that is either unknown or immediately determined. In Fig. 1(b), the single cell in the upper-right corner has an unknown value, denoted by a question mark. In contrast, Fig. 1(a) includes a single cell in the first row that is labeled with the value 2 and has no associated arithmetic operation. Such cells can be treated as having preset values analogous to those in Sudoku puzzles. These fixed values can

be incorporated into the Ising model. Let $C = \{c_0, c_1, \ldots, c_{\theta-1}\}$ denote the set of θ cells with preset values, where each cell $c_\phi \in C$ is represented as a tuple $c_\phi = (i^\phi, j^\phi, v^\phi)$ with $0 \leq \phi \leq \theta - 1$. Here, i^ϕ and j^ϕ denote the row and column indices of the cell, respectively, and v^ϕ is its preset integer value. Equation (2) is used to convert v^ϕ into its binary representation b_l^ϕ and to verify equality with the corresponding binary variables $x_{(i^\phi, j^\phi, l)}$, as shown in Eq. (7). When $H_2 = 0$, all preset values are correctly encoded in the model.

$$H_2 = \sum_{\phi=0}^{\theta} \sum_{l=0}^{k-1} \left(b_l^\phi - x_{(i^\phi, j^\phi, l)} \right)^2 \tag{7}$$

3.3 The Range Rule

The base-2 binary representation reduces the need for logical qubits and enables the derivation of unique cell values. A key limitation, however, is that the cell value computed using Eq. (2) may lie outside the valid range $r = 1 + [0, N–1]$. Although the equation enforces a lower bound of 1, it does not guarantee that the resulting value remains below N. To address this issue, a greedy algorithm was proposed in [10]. The method partitions the value range into a minimal number of *chunks*, where each chunk consists of consecutive integers whose cardinality is a power of two. With formal proof, the approach guarantees that every integer in the valid range is mapped to exactly one chunk. This one-to-one mapping yields a unique encoding and eliminates redundant Pauli operators in the corresponding quantum model [30].

Equation (8) defines the energy subfunction H_3, which enforces the range constraint. Let $U = \{u_0, u_1, \ldots, u_{\delta-1}\}$ be the set of δ chunks produced by the algorithm. Each chunk u_ϕ, where $0 \leq \phi \leq \delta - 1$, encodes information for k bits. Within a given chunk, each bit is either fixed to 0 or 1, or left unspecified and denoted by the symbol "$*$". Accordingly, a chunk can be represented as $u_\phi = \{b_l^\phi \mid 0 \leq l \leq k - 1\}$, where $b_l^\phi \in \{0, 1, *\}$. If the associated multiplication function evaluates to 1, exactly one chunk is activated; otherwise, all chunks remain inactive, indicating that the corresponding cell value lies outside the valid range. When $H_3 = 0$, the range constraints are satisfied for all cells.

$$H_3 = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} \left(1 - \sum_{\phi=0}^{\delta-1} \prod_{l=0, b_l^\phi \neq *}^{k-1} \left(1 - \left(b_l^\phi - x_{(i,j,l)} \right)^2 \right) \right) \tag{8}$$

For illustration, consider the range $r = 1 + [0, N–1]$ with $N = 26$. The integer value 26 can be represented in binary as 11010, corresponding to the three components $2^4 + 2^3 + 2^1$, each with coefficient 1. The algorithm greedily processes these components from the most significant to the least significant bits to more effectively cover the entire range, as summarized in Table 1. For the first chunk related to component 2^4, the most significant bit (the 4th bit) is identified and assigned a value of 0. The four remaining lower-order bits may take either 0 or 1 without exceeding the permissible range and are therefore denoted by the placeholder symbol "$*$". For the second chunk related to component 2^3,

the previously processed bit is fixed to 1, the newly identified bit 3 is set to 0, and the remaining three bits are again marked as "∗". For the final chunk related to component 2^1, both the 4th and 3rd bits are fixed to 1, and all bits down to the newly identified bit 1 are set to 0. Only the least significant bit (bit 0) remains unspecified and is denoted by "∗".

Table 1. Chunk-Based Derivation for Modeling the Range $r = 1 + [0, 25]$

Component	Chunk	Chunk Range	Covered Integers (+1)
2^4	$0 * * **$	[0, 15]	$1 - 16$
2^3	$10 * * *$	$16 + [0, 7]$	$17 - 24$
2^1	$1100 *$	$24 + [0, 1]$	$25 - 26$

3.4 The Cage Rules

Despite sharing several modeling similarities with Sudoku, the cage rules are specific to the KenKen puzzle and therefore require distinct treatment. Each cage is commonly associated with one of four arithmetic operations and must be modeled accordingly. Consider a KenKen puzzle with a set of cages $C = C^+ \cup C^- \cup C^\times \cup C^\div$, where the four disjoint subsets correspond to addition, subtraction, multiplication, and division operations, respectively. Assume that each cage $\sigma_\tau \in C$ contain μ_τ adjacent cells, such that $\sigma_\tau = \{c_\lambda^\tau \mid 0 \leq \lambda \leq \mu_\tau - 1\}$. Each cell c_λ^τ is located at row i_λ^τ and column j_λ^τ. When $\sigma_\tau \in C^- \cup C^\div$, the number of cells μ_τ is restricted to 2. In these cases, the ordering of the two cell values is unspecified, and the cage result is obtained by subtracting or dividing the smaller value from the larger one.

3.4.1 The Addition Constraint

The addition constraint requires that, for any cage $\sigma_\tau \in C^+$, the sum of the values of all its cells equals v_τ. Equation (9) enforces this constraint for all cages in C^+. The integer value of each cell is computed using Eq. (2), in which the corresponding binary variables $x_{(i_\lambda^\tau, j_\lambda^\tau, l)}$ are weighted by powers of two according to their bit indices. The squared term ensures that the resulting expression for each cage is nonnegative; when this expression evaluates to zero, the addition constraint for that cage is satisfied. When $H_4 = 0$, all considered cages satisfy the addition constraint.

$$H_4 = \sum_{\sigma_\tau \in C^+} \left(\sum_{\lambda=0}^{\mu_\tau - 1} \left(1 + \sum_{l=0}^{k-1} x_{(i_\lambda^\tau, j_\lambda^\tau, l)} \cdot 2^l \right) - v_\tau \right)^2 \tag{9}$$

3.4.2 The Multiplication Constraint

The multiplication constraint requires that, for any cage $\sigma_\tau \in C^{\times}$, the product of the values of all its cells equals v_τ. Equation (10) enforces this constraint for all cages in $C^{\times}$. This constraint is analogous to the addition constraint, differing only in the arithmetic operator used to compare against the target value v_τ; specifically, the summation is replaced by a product. When $H_5 = 0$, all considered cages satisfy the multiplication constraint.

$$H_5 = \sum_{\sigma_\tau \in C^{\times}} \left(\prod_{\lambda=0}^{\mu_\tau - 1} \left(1 + \sum_{l=0}^{k-1} x_{(i_\lambda^\tau, j_\lambda^\tau, l)} \cdot 2^l \right) - v_\tau \right)^2 \tag{10}$$

3.4.3 The Subtraction Constraint

The subtraction constraint requires that, for any two-cell cage $\sigma_\tau \in C^{-}$, the difference between the larger and smaller cell values equals v_τ. However, it is not specified in advance which of the two cells has the larger value. Although the situation can be handled using an absolute-value function, such a mechanism is not readily available in the quantum formulation. Therefore, Eq. (11) enforces this constraint by computing the subtraction in both possible orders, squaring each result, and multiplying them together. If the subtraction in either ordering equals v_τ, the resulting expression for the cage evaluates to zero. Otherwise, the product yields a positive value, indicating the constraint is violated. When $H_6 = 0$, all considered cages satisfy the subtraction constraint.

$$H_6 = \sum_{\sigma_\tau \in C^{-}} \prod_{\lambda=0}^{1} \left(\left(1 + \sum_{l=0}^{k-1} x_{(i_\lambda^\tau, j_\lambda^\tau, l)} \cdot 2^l \right) - \left(1 + \sum_{l=0}^{k-1} x_{(i_{1-\lambda}^\tau, j_{1-\lambda}^\tau, l)} \cdot 2^l \right) - v_\tau \right)^2 \tag{11}$$

3.4.4 The Division Constraint

The division constraint requires that, for any two-cell cage $\sigma_\tau \in C^{\div}$, the ratio of the larger cell value to the smaller cell value equals v_τ. As with the subtraction constraint, the ordering of the two cell values is not known a priori. Accordingly, Eq. (12) enforces this constraint by evaluating the division in both possible orders and combining the results multiplicatively. Moreover, division can be interpreted as repeated subtraction; thus, requiring the division ratio to equal v_τ is equivalent to subtracting the smaller value from the larger value exactly v_τ times. The squared formulation again ensures that the resulting expression is nonnegative. When $H_7 = 0$, all considered cages satisfy the division constraint.

$$H_7 = \sum_{\sigma_\tau \in C^{\div}} \prod_{\lambda=0}^{1} \left(\left(1 + \sum_{l=0}^{k-1} x_{(i_\lambda^\tau, j_\lambda^\tau, l)} \cdot 2^l \right) - v_\tau \cdot \left(1 + \sum_{l=0}^{k-1} x_{(i_{1-\lambda}^\tau, j_{1-\lambda}^\tau, l)} \cdot 2^l \right) \right)^2 \tag{12}$$

4 Quantum Modeling

In quantum computing, the inputs are logical qubits, each represented as a vector rather than a scalar value. Consequently, the quantum Hamiltonian, denoted as H_Q, is expressed in terms of 2×2 Pauli matrices, such as the Pauli I, X, Y, or Z operators [30]. For the KenKen puzzle, however, only the Z operators are required for circuit construction. As shown in Eq. (13), H_Q consists exclusively of Pauli-Z operators Z_α, where $0 \leq \alpha \leq m-1$, which replace the spin variables s_α used in Eq. (3). A Z_iZ_j interaction term represents a composite gate, where $i \neq j$; when $i = j$, this term reduces to the Pauli identity operator I and can therefore be omitted without affecting the Hamiltonian energy.

$$H_Q = H(Z_0, Z_1, \ldots, Z_{m-1}) \tag{13}$$

The detailed modeling of subfunctions H_0 through H_3, which are analogous to Sudoku constraints, is available in [48]. The following discussion of H_Q therefore focuses on the quantum modeling of cage rules H_4 through H_7. Each quantum Hamiltonian subfunction H_i can be evaluated as the expectation value of its corresponding observable operator Q_i, expressed as $< \Psi |Q_i| \Psi >$ for a quantum state Ψ. For illustrative purposes, the 4×4 KenKen puzzle shown in Fig. 1(a) is used to demonstrate the modeling of cages with different arithmetic operations with the corresponding cage indices arranged as shown in Fig. 2.

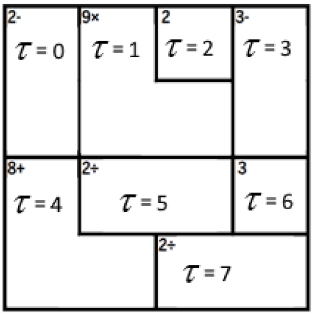


Fig. 2. Cage Indices for the 4×4 KenKen Puzzle in Fig. 1(a)

4.1 The Addition Constraint Quantum Modeling

In Fig. 1(a), there is only one cage σ_τ with $\tau = 4$ in the set C^+ with an addition constraint of $8+$. This cage consists of three cells, i.e., $\mu_\tau = 3$. The corresponding observable operator Q_4 for this constraint is constructed from the Hamiltonian subfunction H_4 defined in

Eq. (9), as shown below.

$$
\begin{aligned}
Q_4 &= \sum_{\sigma_\tau \in C^+} \left(\sum_{\lambda=0}^{\mu_\tau - 1} \left(1 + \sum_{l=0}^{k-1} x_{(i_\lambda^\tau, j_\lambda^\tau, l)} \cdot 2^l \right) - v_\tau \right)^2 \\
&= \left(\sum_{\lambda=0}^{2} \left(1 + \sum_{l=0}^{1} x_{(i_\lambda^4, j_\lambda^4, l)} \cdot 2^l \right) - 8 \right)^2 \\
&= \left(\left(2 + \tfrac{1 - Z_{16}}{2} - Z_{17} \right) + \left(2 + \tfrac{1 - Z_{24}}{2} - Z_{25} \right) \right. \\
&\left. + \left(2 + \tfrac{1 - Z_{26}}{2} - Z_{27} \right) - 8 \right)^2 = 4 + 0.5Z_{16} + Z_{17} + 0.5Z_{24} + Z_{25} \\
&+ 0.5Z_{26} + Z_{27} + Z_{16}Z_{17} + 0.5Z_{16}Z_{24} + Z_{16}Z_{25} + 0.5Z_{16}Z_{26} \\
&+ Z_{16}Z_{27} + Z_{17}Z_{24} + 2Z_{17}Z_{25} + Z_{17}Z_{26} + 2Z_{17}Z_{27} \\
&+ Z_{24}Z_{25} + 0.5Z_{24}Z_{26} + Z_{24}Z_{27} + Z_{25}Z_{26} + 2Z_{25}Z_{27} + Z_{26}Z_{27}
\end{aligned}
$$

4.2 The Multiplication Constraint Quantum Modeling

There is a single cage σ_τ with $\tau = 1$ in the set $C^\times$ with a multiplication constraint of 9 $\times$. This cage consists of three cells, i.e., $\mu_\tau = 3$. The corresponding observable operator Q_5 for this constraint is constructed from the Hamiltonian subfunction H_5 defined in Eq. (10), as shown below.

$$
\begin{aligned}
Q_5 &= \sum_{C^\times} \left(\prod_{=0}^{-1} \left(1 + \sum_{l=0}^{k-1} x_{(i^\tau, j^\tau, l)} - 2^l \right) - v \right)^2 \\
&= \left(\prod_{=0}^{2} \left(1 + \sum_{l=0}^{1} x_{(i^1, j^1, l)} \cdot 2^l \right) - 9 \right)^2 \\
&= \left(\left(2 + \frac{1 - Z_2}{2} - Z_3 \right) \left(2 + \frac{1 - Z_{10}}{2} - Z_{11} \right) \left(2 + \frac{1 - Z_{12}}{2} - Z_{13} \right) - 8 \right)^2 \\
&= 221.625 - 84.375Z_2 - 168.75Z_3 - 84.375Z_{10} - 168.75Z_{11} \\
&- 84.375Z_{12} - 168.75Z_{13} + 56.25Z_2Z_3 + 35.625Z_2Z_{10} + 71.25Z_2Z_{11} \\
&+ 35.625Z_2Z_{12} + 71.25Z_2Z_{13} + +71.25Z_3Z_{10} + 142.5Z_3Z_{11} \\
&+ 71.25Z_3Z_{12} + 142.5Z_3Z_{13} + 56.25Z_{10}Z_{11} + 35.625Z_{10}Z_{12} \\
&+ 71.25Z_{10}Z_{13} + 71.25Z_{11}Z_{12} + 142.5Z_{11}Z_{13} + 56.25Z_{12}Z_{13} \\
&- 18.75Z_2Z_3Z_{10} - 37.5Z_2Z_3Z_{11} - 18.75Z_2Z_3Z_{12} - 37.5Z_2Z_3Z_{13} \\
&- 18.75Z_2Z_{10}Z_{11} - 13.375Z_2Z_{10}Z_{12} - 26.75Z_2Z_{10}Z_{13} - 26.75Z_2Z_{11}Z_{12} \\
&- 53.5Z_2Z_{11}Z_{13} - 18.75Z_2Z_{12}Z_{13} - 37.5Z_3Z_{10}Z_{11} - 26.75Z_3Z_{10}Z_{12} \\
&- 53.5Z_3Z_{10}Z_{13} - 53.5Z_3Z_{11}Z_{12} - 107Z_3Z_{11}Z_{13} - 37.5Z_3Z_{12}Z_{13} \\
&- 18.75Z_{10}Z_{11}Z_{12} - 37.5Z_{10}Z_{11}Z_{13} - 18.75Z_{10}Z_{12}Z_{13} - 37.5Z_{11}Z_{12}Z_{13} \\
&+ 7.5Z_2Z_3Z_{10}Z_{11} + 6.25Z_2Z_3Z_{10}Z_{12} + 12.5Z_2Z_3Z_{10}Z_{13} + 12.5Z_2Z_3Z_{11}Z_{12} \\
&+ 25Z_2Z_3Z_{11}Z_{13} + 7.5Z_2Z_3Z_{12}Z_{13} + 6.25Z_2Z_{10}Z_{11}Z_{12} + \\
&12.5Z_2Z_{10}Z_{11}Z_{13} + 6.25Z_2Z_{10}Z_{12}Z_{13} + 12.5Z_2Z_{11}Z_{12}Z_{13} \\
&+ 12.5Z_3Z_{10}Z_{11}Z_{12} + 25Z_3Z_{10}Z_{11}Z_{13} + 12.5Z_3Z_{10}Z_{12}Z_{13} \\
&+ 25Z_3Z_{11}Z_{12}Z_{13} + 7.5Z_{10}Z_{11}Z_{12}Z_{13} - 2.5Z_2Z_3Z_{10}Z_{11}Z_{12} \\
&- 5Z_2Z_3Z_{10}Z_{11}Z_{13} - 2.5Z_2Z_3Z_{10}Z_{12}Z_{13} - 5Z_2Z_3Z_{11}Z_{12}Z_{13} -
\end{aligned}
$$

4.3 The Subtraction Constraint Quantum Modeling

There are two cages σ_τ with $\tau = 0$ and 3 in the set C^-, subject to a subtraction constraint of 2 $-$ and 3 $-$, respectively. Each cage consists of two cells, i.e., $\mu_\tau = 2$. The associated observable operator Q_6 for this constraint is constructed from the Hamiltonian subfunction H_6 defined in Eq. (11), as shown below.

$$
\begin{aligned}
Q_6 = \sum_{\sigma_\tau \in C^-} \prod_{\lambda=0}^{1} \left(\left(1 + \sum_{l=0}^{k-1} x_{(i_\lambda^\tau, j_\lambda^\tau, l)} \cdot 2^l \right) - \left(1 + \sum_{l=0}^{k-1} x_{(i_{1-\lambda}^\tau, j_{1-\lambda}^\tau, l)} \cdot 2^l \right) - v_\tau \right)^2 \\
= \left(\left(\frac{1 - Z_0}{2} - Z_1 \right) - \left(\frac{1 - Z_8}{2} - Z_9 \right) - 2 \right)^2 \\
\left(\left(\frac{1 - Z_8}{2} - Z_9 \right) - \left(\frac{1 - Z_0}{2} - Z_1 \right) - 2 \right)^2 \\
+ \left(\left(\frac{1 - Z_6}{2} - Z_7 \right) - \left(\frac{1 - Z_{14}}{2} - Z_{15} \right) - 3 \right)^2 \\
\left(\left(\frac{1 - Z_{14}}{2} - Z_{15} \right) - \left(\frac{1 - Z_6}{2} - Z_7 \right) - 3 \right)^2 \\
= 61 + 2Z_0Z_1 - 2.5Z_0Z_8 - 2Z_0Z_9 - 2Z_1Z_8 + 2Z_1Z_9 + 2Z_8Z_9 + 6Z_0Z_1Z_8Z_9 \\
-8Z_6Z_7 + 2.5Z_6Z_{14} + 8Z_6Z_{15} + 8Z_6Z_{14} \\
+22Z_7Z_{15} - 8Z_{14}Z_{15} + 6Z_6Z_7Z_{14}Z_{15}
\end{aligned}
$$

4.4 The Division Constraint Quantum Modeling

With respect to the set $C^\div$, there are two cages σ_τ with $\tau = 5$ and 7, both subject to a division constraint of 2 $\div$. Each cage consists of two cells, i.e., $\mu_\tau = 2$. According to Eq. (12), the corresponding observable operator Q_7 for this constraint is constructed from the Hamiltonian subfunction H_7 as shown below.

$$
\begin{aligned}
Q_7 = \sum_{\sigma_\tau \in C^\div} \prod_{\lambda=0}^{1} \left(\left(1 + \sum_{l=0}^{k-1} x_{(i_\lambda^\tau, j_\lambda^\tau, l)} \cdot 2^l \right) - v_\tau \cdot \left(1 + \sum_{l=0}^{k-1} x_{(i_{1-\lambda}^\tau, j_{1-\lambda}^\tau, l)} \cdot 2^l \right) \right)^2 \\
= \left(\left(2 + \frac{1 - Z_{18}}{2} - Z_{19} \right) - 2 \left(2 + \frac{1 - Z_{20}}{2} - Z_{21} \right) \right)^2 \\
\left(\left(2 + \frac{1 - Z_{20}}{2} - Z_{21} \right) - 2 \left(2 + \frac{1 - Z_{18}}{2} - Z_{19} \right) \right)^2 \\
+ \left(\left(2 + \frac{1 - Z_{28}}{2} - Z_{29} \right) - 2 \left(2 + \frac{1 - Z_{30}}{2} - Z_{31} \right) \right)^2 \\
\left(\left(2 + \frac{1 - Z_{30}}{2} - Z_{31} \right) - 2 \left(2 + \frac{1 - Z_{28}}{2} - Z_{29} \right) \right)^2
\end{aligned}
$$

$$
\begin{aligned}
&= 128.5 - 8.75Z_{18} - 32.5Z_{19} - 8.75Z_{20} - 32.5Z_{21} \\
&+32.5Z_{18}Z_{19} - 13.75Z_{18}Z_{20} - 12.5Z_{18}Z_{21} - \\
&12.5Z_{19}Z_{20} + 5Z_{19}Z_{21} + 32.5Z_{20}Z_{21} \\
&-7.5Z_{18}Z_{19}Z_{20} - 15Z_{18}Z_{19}Z_{21} - 7.5Z_{18}Z_{20}Z_{21} - \\
&156Z_{19}Z_{20}Z_{21} + 33Z_{18}Z_{19}Z_{20}Z_{21} - 8.75Z_{28} \\
&-32.5Z_{29} - 8.75Z_{30} - 32.5Z_{31} + 32.5Z_{28}Z_{29} - \\
&13.75Z_{28}Z_{30} - 12.5Z_{28}Z_{31} - 12.5Z_{29}Z_{30} \\
&+5Z_{29}Z_{31} + 32.5Z_{30}Z_{31} - 7.5Z_{28}Z_{29}Z_{30} - \\
&15Z_{28}Z_{29}Z_{31} - 7.5Z_{28}Z_{30}Z_{31} - 156Z_{29}Z_{30}Z_{31} \\
&+33Z_{28}Z_{29}Z_{30}Z_{31}
\end{aligned}
$$

5 KenKen Quantum Solver Evaluation

This section evaluates the proposed KenKen quantum solver to verify that the quantum-state representations are modeled correctly and that the associated resource usage is accurately characterized. In addition to the puzzle shown in Fig. 1(a), the puzzle presented in Fig. 1(b) is also included in the evaluation. For the derived observable operators, all constant shift terms, i.e., real-valued offsets such as the first term (128.5) in Q_7, are omitted during the eigenvalue minimization process, as they do not affect the optimization outcome. However, these terms are retained when computing the fidelity of each run. In [10], the fidelity for a run is defined as a measure of its closeness to the optimal solution and is computed as shown in Eq. (14).

$$Fidelity = \frac{-EnergyResult}{TotalShift} \times 100\% \tag{14}$$

The evaluation is conducted on a Windows Server 2025 Standard system. This high-performance platform is equipped with an Intel Xeon Gold 6230 CPU, 256 GB of RAM, and support for up to 33 logical qubits. Performance is influenced by the chosen QAOA depth and by the shot settings used with the COBYLA optimizer. Table 2 reports the average execution time, average fidelity, and related performance metrics for both puzzles shown in Fig. 1.

The reported averages are calculated over 10 computationally intensive runs for the larger 4×4 puzzle and 100 less CPU-intensive runs for the smaller 3×3 puzzle, spanning multiple QAOA repetition levels and shot settings, as determined by the varying input parameters highlighted in the snippet below. In the code, the observable operator Q is represented as an array of Hamiltonian terms corresponding to the union of all derived Q_i subsets, as formulated in Eq. (15).

$$Q = \bigcup_{i=0}^{7} Q_i \tag{15}$$

```
op = SparsePauliOp.from_sparse_list(Q, num_qubits)
qaoa = QAOA(AerSampler(run_options={"shots": 5000}),
optimizer=COBYLA(), reps=1)
result = qaoa.compute_minimum_eigenvalue(op)
```

Table 2. Performance and Fidelity Evaluation Results

Puzzle Size	Qubits	Hamiltonian Terms	Shots	Reps	Avg Execution Time (sec)	Avg Fidelity	Variance	Fidelity Range
4 × 4	32	275	50,000	2	14353.23	98.76%	1.567	98.37%–99.30%
			100,000	1	6601.80	98.67%	0.900	98.37%–99.07%
			250,000	2	18333.47	99.02%	1.067	98.83%–99.53%
			500,000	1	12889.49	98.93%	1.600	98.60%–99.53%
3 × 3	18	168	2,500	2	7.15	99.76%	0.995	99.54%–100%
			5,000	1	7.15	99.82%	0.551	99.54%–100%
			3,750	2	8.96	99.82%	0.588	99.54%–100%
			7,500	1	9.34	99.87%	0.451	99.54%–100%
			5,000	2	10.64	99.85%	0.478	99.54%–100%
			10,000	1	11.53	99.91%	0.400	99.54%–100%
			7,500	2	13.59	99.89%	0.434	99.54%–100%
			15,000	1	15.70	99.93%	0.264	99.54%–100%

Two key observations emerge from the results. First, increasing the number of QAOA repetitions generally improves optimization fidelity but also leads to longer execution times. A similar trade-off applies to increasing the number of shots, which provides more reliable probability distributions for the COBYLA optimizer at the expense of additional runtime. Second, there is no clear guideline for prioritizing one parameter over the other. However, increasing the QAOA depth or the number of repetitions by a single step typically introduces greater fidelity variance and a higher execution-time penalty than substantially increasing the number of shots; therefore, increasing the number of shots before adding an additional QAOA repetition is likely to be a more cost-efficient strategy.

6 Conclusions

We developed a gate-based KenKen quantum solver for universal quantum computers by leveraging an Ising spin-glass modeling approach combined with base-2 binary encoding. The solver is designed as a generalized framework capable of handling $N \times N$ puzzles, without requiring N to be a perfect square. Subject to the availability of 33 logical qubits in our system, the proposed binary encoding enables the modeling of puzzles up to a 4×4 grid, with each cell represented by two qubits, resulting in a total requirement of 32 logical qubits. In comparison, a one-hot encoding scheme would require 27 and 64 qubits for 3×3 and 4×4 puzzles, respectively. The qubit savings achieved by reducing the encoding complexity from $O(N^3)$ to $O(N^2 \log_2 N)$ become increasingly significant as the puzzle size grows.

KenKen is a variation of Sudoku that shares several constraints, making Ising-model-based techniques developed for the corresponding Sudoku constraints naturally applicable. Unlike Sudoku, however, KenKen does not require the puzzle size to be a perfect square; consequently, the box constraint is no longer relevant, while the range constraint for individual cells becomes more prominent due to the increased flexibility in puzzle sizes. Fundamentally, only the additional KenKen-specific constraints need to be modeled on top of existing approaches. More importantly, these newly introduced modeling components are expected to be broadly applicable to other optimization problems, such as Kakuro and linear programming formulations.

The developed quantum solver was evaluated experimentally, yielding several key observations. Increasing either the number of QAOA repetitions or the number of shots generally improves optimization fidelity, but at the cost of longer execution times. Based on the experimental results, increasing the number of shots, approximately doubling them, before introducing an additional QAOA repetition appears to be a more cost-efficient strategy. Nevertheless, as the puzzle size increases, the computational complexity faced by the COBYLA optimizer grows exponentially, making it increasingly difficult to identify optimal solutions without substantially increasing the number of shots or repetitions. Future work will focus on reducing the search space or incorporating hybrid approaches to better exploit the complementary strengths of classical and quantum computing.

References

1. Reiter, H., Thornton, J., Vennebush, P.: Using KenKen to build reasoning skills. Math. Teach. **107**(5), 341–347 (2013). https://doi.org/10.5951/mathteacher.107.5.0341
2. Molina, J., Ibañez, E.: Students' performance and attitude in operating integers using KenKen puzzle in a collaborative learning environment. Educ. Dig. **19**(1), 45–51 (2024)
3. Simonis, H.: Sudoku as a constraint problem. In: CP Workshop on Modeling and Reformulating Constraint Satisfaction Problems (2005)
4. Wang, W.-L., Loker, D., Tang, M.-H.: A swarm intelligent Sudoku solver. In: 19th Artificial Neural Network in Engineering Conference (ANNIE), St. Louis, Missouri (2009)
5. Hathidara, A., Pandey, L.: Neuro-symbolic sudoku solver (2023). http://arxiv.org/abs/2307.00653
6. Mücke, S.: A simple QUBO formulation of Sudoku. In: GECCO 2024 Companion - Proceedings of the 2024 Genetic and Evolutionary Computation Conference Companion, Association for Computing Machinery, Inc., pp. 1958–1962 (2024). https://doi.org/10.1145/3638530.3664106
7. Singh, V., Sharma, V., Bachchas, V., Kaushik, D.: Sudoku solving using quantum computer. J. Pharmaceut. Negative Results **13**(10) (2022). https://doi.org/10.47750/pnr.2022.13.S10.643
8. Zhao, M.B., Li, F.: A quantum-inspired algorithm for solving Sudoku puzzles and the MaxCut problem. Quantum Inf. Comput. **25**(5), 397–420 (2025). https://doi.org/10.2478/qic-2025-0023
9. Weiß, M.: Encoding strategies to solve sudoku with quantum computers (2022). Accessed 26 Feb 2026. https://elib.dlr.de/193653/1/Abgabe_weiss22.pdf
10. Wang, W.L., et al.: A resource efficient ising model-based quantum sudoku solver. Softw. Pract. Exp. (2026). https://doi.org/10.1002/spe.70063

11. Zhou, L., Wang, S.T., Choi, S., Pichler, H., Lukin, M.D.: Quantum approximate optimization algorithm: performance, mechanism, and implementation on near-term devices. Phys. Rev. X **10**(2) (2020). https://doi.org/10.1103/PhysRevX.10.021067
12. Powell, M.J.D.: A direct search optimization method that models the objective and constraint functions by linear interpolation. In: Gomez, S., Hennart, J.-P. (eds.) Advances in Optimization and Numerical Analysis, pp. 51–67. Springer Netherlands (1994). https://doi.org/10.1007/978-94-015-8330-5_4
13. Kendall, G., Parkes, A., Spoerer, K.:A survey of NP-complete puzzles. ICGA **31**(1), 13–34 (2008). http://users.rsise.anu.edu.au/
14. Yang, C.: Thiadeliria (GitHub) KenKen: A CSP Implementation
15. Morsy, H.S., Ragaa, N.: A black widow optimization algorithm for solving KenKen problem. Seybold Rep. **17**(9), 1232–1242 (2022). https://doi.org/10.5281/zenodo.7106602
16. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proceedings of the Annual ACM Symposium on Theory of Computing, Association for Computing Machinery, pp. 212–219 (1996). https://doi.org/10.1145/237814.237866
17. Seidel, R., et al.: Quantum backtracking in qrisp applied to sudoku problems (2024). http://arxiv.org/abs/2402.10060
18. Farhi, E., Goldstone, J., Gutmann, S., Lapan, J., Lundgren, A., Preda, D.: A quantum adiabatic evolution algorithm applied to random instances of an NP-complete problem. Science **292**(5516), 472–475 (2001). https://doi.org/10.1126/science.1057726
19. Das, A., Chakrabarti, B.K.: Quantum annealing and analog quantum computation. Rev. Mod. Phys. **80**(3),1061–1081 (2008). https://doi.org/10.1103/RevModPhys.80.1061
20. Skotiniotis, M., Sekatski, P., Dür, W.: Quantum metrology for the ising hamiltonian with transverse magnetic field. New J. Phys. **17**(7) (2015). https://doi.org/10.1088/1367-2630/17/7/073032
21. Haythorpe, M.: Reducing the generalised sudoku problem to the hamiltonian cycle problem. AKCE Int. J. Graphs Comb. **13**(3), 272–282 (2016). https://doi.org/10.1016/j.akcej.2016.10.001
22. Niss, M.: History of the Lenz-Ising model 1965–1971: the role of a simple model in understanding critical phenomena. Arch. Hist. Exact Sci. **65**(6), 625–658 (2011). https://doi.org/10.1007/s00407-011-0086-1
23. Corli, S., Dragoni, D., Proietti, M., Dispenza, M., Cavazzoni, C., Prati, E.: A max K-cut implementation for QAOA in the measurement based quantum computing formalism.In: Proceedings - 2023 IEEE International Conference on Quantum Computing and Engineering, QCE 2023, Institute of Electrical and Electronics Engineers Inc., pp. 284–285 (2023). https://doi.org/10.1109/QCE57702.2023.10244
24. Wang, W.-L., Tang, M.-H., Hussain, S., Wang, K.: Solving prime factorization using quantum ising model. In: International Conference on Software Engineering for Emerging Technologies (SEET 2025), Springer Nature Switzerlan, pp. 568–574 d (2025). https://doi.org/10.1007/978-3-032-08977-9_37. Accessed 23 Feb 2026
25. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM J. Comput. **26**(5), 1484–1509 (1997). https://doi.org/10.1137/S0097539795293172
26. Bennett, T., Noakes, L., Wang, J.: Non-variational quantum combinatorial optimisation. In:2024 IEEE International Conference on Quantum Computing and Engineering (QCE)(2024). http://arxiv.org/abs/2404.03167
27. Jordan, S.P., et al.: Optimization by decoded quantum interferometry. Nature **646**(8086), 831–836 (2025). https://doi.org/10.1038/s41586-025-09527-5
28. Breuckmann, N.P., Eberhardt, J.N.: Quantum low-density parity-check codes. PRX Quantum **2**(4) (2021). https://doi.org/10.1103/PRXQuantum.2.040101

29. Yang, S., Calderbank, R.: Spatially-Coupled QLDPC Codes. Quantum **9**, 1693 (2025). https://doi.org/10.22331/q-2025-04-07-1693
30. Barenco, C.H. et al.: Elementary gates for quantum computation. Phys. Rev. A (Coll. Park) **52**(5), 3457–3467 (1995). https://doi.org/10.1103/PhysRevA.52.3457

Data Mesh as a Migration Strategy: A Federated Approach to Cloud-Native Data Engineering

Richa Solanki(✉)

Detroit, USA
ricsolanki@gmail.com

Abstract. Enterprise data modernization demands architectural, organizational, and governance transformation — not merely a technical infrastructure upgrade. Monolithic data warehouses and central ETL pipelines are structurally ill-equipped to handle the data volume, velocity, and variety of modern workloads, and customary cloud migration approaches entail unacceptable technical debt or organizational overhead. Data Mesh is a decentralized and domain-oriented approach that frames cloud adoption as a continuing organizational transformation powered by four principles: domain-oriented ownership, data-as-a-product, self-serve data infrastructure, and Federated Computational Governance (FCG). It distributes data engineering ownership across business domains while adhering to enterprise-level standards through platform-based guardrails. This presents an alternative model to the centralized data lake or data warehouse. In contrast to centralized migration programs, domain teams are empowered to migrate at a pace commensurate with their domain's complexity and commercial value. Data products are stable contracts between data producers and consumers that decouple interfaces from implementations and are, therefore, not impacted by any backend infrastructure change. Platform engineering abstracts cloud complexity into self-service, standardized tooling, lowering the barrier for domain teams to adopt the cloud. Federated governance automates regulatory compliance within the deployment pipeline, enabling scalable governance of distributed portfolios of domain products and services, and eliminating the overhead of manual compliance review. The patterns of strangler fig replacement, hybrid coexistence, event-driven modernization, and data product wrapping give architects risk-calibrated ways to deliver incremental value. The primary capabilities that drive Data Mesh success at enterprise scale are federated governance automation, product discipline, and platform capability sequencing

Keywords: Data Mesh · Cloud-Native Migration · Federated Governance · Domain-Oriented Decentralization · Data-as-a-Product

1 Introduction

Modernizing the enterprise data architecture has emerged as one of the planned, business-critical challenges technology-led organizations face in the digital age. Legacy architectures, largely based on monolithic on-premises data warehouses, centralized ETL

Solanki, R.—Independent Researcher

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 52–63, 2027.
https://doi.org/10.1007/978-3-032-39763-8_4

pipelines, and rigid relational databases, were designed for a world in which volume, variety, and velocity predictability were the norm [1]. These architectures are fundamentally incompatible with the data processing requirements of modern data ecosystems, wherein data volumes, data diversity, the need for real-time data processing, and cross-functional analytical consumption are exponentially growing [2].

Cloud-native platforms provide elastic compute, managed storage, event-driven architectures, and infrastructure as code that can be provisioned and deployed as needed. Upgrading from legacy data infrastructure to a cloud-native platform is rarely a straightforward process. Most migration strategies, including lift-and-shift and wholesale re-architecting, have been found unsuitable for enterprise-grade data ecosystems [3] as lift and shift perpetuates technical debt and clouds billing complexity. Re-architecting often requires meaningful investment, takes longer, and requires a larger effort in change management of the organization, than may be available to central data teams [4].

An emerging model for the cloud is Data Mesh. Unlike one-off technical cloud deployments, Data Mesh sees cloud as a new type of domain-oriented organizational transformation. Data Mesh is based on four fundamental architectural principles: domain-oriented ownership, data as a product, self-serve data infrastructure, and (FCG) [5]. Data Mesh delegates the responsibility of data engineering back to the domains, while the overall architecture and structure is maintained by enforcing guardrails at the platform level [6]. This paper introduces Data Mesh as a principled migration framework and discusses new patterns it could enable, challenges to watch out for, and differences with cloud-native engineering practices. The goal is to give practitioners a technical grounding to help deliver large-scale data modernization.

While Dehghani's foundational work establishes Data Mesh as an organizational and architectural model [5, 6], existing literature does not systematically address cloud migration as a primary application context. This paper contributes three specific extensions: (1) a formalized set of migration patterns (strangler fig, hybrid coexistence, event-driven modernization, data product wrapping) mapped to risk profiles and organizational constraints; (2) a sequencing framework that prescribes platform-first, governance-before-onboarding ordering to reduce adoption failure; and (3) a responsibility boundary model distinguishing platform-enforced standards from domain-owned implementation decisions. Together, these contributions position Data Mesh as an actionable migration framework rather than solely an operational target state.

2 Foundational Principles of Data Mesh and their Migration Relevance

Data Mesh isn't a technology or a set of tools, but rather an architecture and organizational model. As such, it can be used as a migration path because its four design principles directly address the architectural shortcomings of prior migration approaches [6].

The most important shift away from legacy migration methodologies is Domain-Oriented Decentralization. In a centralized legacy architecture, one central data engineering team ingests, transforms, and serves all enterprise data. The result is a structural

bottleneck: all data must flow through it. This presents an important challenge in larger, more complex organizations [7]. Domain-oriented ownership enables overcoming this bottleneck by assigning ownership of data to teams who create and use that data. These are the teams best positioned to understand its semantics, quality requirements, and business value. This domain-decoupled approach, acknowledging that each domain migrates in parallel and at its own pace, depending on its own complexity, resourcing, and business needs, avoids tightly coupling each domain to a central orchestrator or hub [1].

Data as a product means a focus on software engineering principles applied to data. Each data product has five characteristics: it has an interface, a data schema, a quality SLA, data lineage, and discoverable access. This is particularly useful for migration use cases, where legacy data sets can be given product-grade interfaces before the data in fact migrates to the cloud. This way, the contract is preserved, so downstream consumers are not affected by the change. This is core to a safe and incremental migration [8].

Self-Serve Data Infrastructure addresses the skills asymmetry that repeatedly trips up domain-led migration: domain teams know their business functions well, but cloud data engineering is not their expertise. A platform team can break down these barriers by providing infrastructure-as-code templates, ingestion and transformation patterns, observability tooling, data catalogs, and access management automation [3]. This platform layer provides a standard set of discoverable, cloud-agnostic primitives that domain teams can use to move workloads without needing to be cloud platform experts. We also consider the platform team to be an internal product team whose customers are the domain teams [6].

(FCG) is the data governance approach that replaces the command-and-control model with global policies enforced via domain-embedded tooling. Security classification, privacy governance, regulatory compliance, and data quality standards are defined and enforced by tooling embedded within the domain platform. The appealing properties of this model are similar to those enjoyed within a data mesh. This architecture allows governance to scale with the number of migrating domains, which centralized governance models cannot do [4] (Table 1).

Table 1. Core Data Mesh principles and corresponding migration benefits [6–8] (Conceptual mapping; not derived from empirical measurement)

Data Mesh Principle	Migration Challenge Addressed	Migration Outcome
Domain-Oriented Ownership	Centralized team bottleneck	Parallel, asynchronous domain migration
Data-as-a-Product	Consumer disruption during backend changes	Stable contracts via product-grade interfaces
Self-Serve Infrastructure	Domain skills asymmetry in cloud engineering	Accessible, standardized cloud provisioning
(FCG)	Governance scaling failure under decentralization	Automated, policy-embedded compliance enforcement

3 Architectural Design of a Federated Cloud-Native Data Mesh

Operationalizing Data Mesh concepts into a cloud-native migration architecture requires designing domain decomposition, data product architectures, governance structures, and platform engineering capabilities [10].

Domain Architecture and Cloud Service Alignment starts with the division of the enterprise data landscape into domains aligned with bounded contexts. The bounded context is an organizational unit with accountability for one or more business capabilities, associated sources of data, and analytical outputs [7]. Bounded contexts own their data pipelines, storage layers, compute power, quality controls, and product SLAs. Cloud-native services fit well with the domain ownership model: serverless compute allows domains to process data without worrying about infrastructure, managed storage services provide scalable and cost-effective persistence layers, stream processing services allow data to flow between domains, and workflow orchestrators are used to schedule jobs and define dependencies between them [2].

A Data Product is the unit of delivery in a Data Mesh architecture. Each data product must expose its data via query APIs, event streams, or file-based data exports and document its schema, lineage, quality metrics, ownership, and access policies through machine-readable metadata [5]. During migration, this design discipline provides a step towards modernizing the system in an incrementally deliverable way. Legacy systems can be wrapped with the product-compliant interface satisfying a stable contract, allowing the underlying physical migration to continue independently and isolating the rest of the organization from the effects of the migration [8].

Federated Governance Implementation involves investing in policy engineering and global data governance policies (data classification, privacy regulation compliance, retention requirements, access control standards, etc.) as machine-enforceable rules embedded in the platform tooling [9]. In a domain-based enforcement model, these rules are checked whenever a data product is registered or published. This is done through automation, with distributed policy engines in place within CI/CD pipelines for deploying data products, establishing governance compliance as a discoverability requirement, rather than something that is audited after the fact, which can introduce latency and subjectivity [11].

Self-Serve Platform Engineering establishes the maximum possible area of autonomy within the domain constraints. The platform team prepares infrastructure provisioning templates that capture cloud architecture best practices to limit domain teams' decisions about the infrastructure to the level of configuration rather than design. Furthermore, data ingestion pipelines should match the source systems in enterprise legacy environments, such as relational databases, mainframe extracts, or flat-file feeds, and should expose their outputs via standardized interfaces to the data product model. Likewise, observability infrastructure, such as pipeline monitoring, data quality measurement, schema change detection, or lineage tracking, should be provided as a platform default instead of a domain-optional capability (Tables 2, 3).

Table 2. Cloud-native service alignment with Data Mesh architectural components [2, 5, 11] (Conceptual mapping; not derived from empirical measurement)

Architectural Component	Cloud-Native Service Category	Primary Function in Migration
Domain Pipeline Execution	Serverless Compute	Independent transformation without infrastructure management
Domain Storage Layer	Managed Object & Warehouse Storage	Scalable, cost-optimized data persistence per domain
Real-Time Data Flows	Distributed Event Streaming	Decoupled, low-latency domain data publication
Infrastructure Provisioning	Infrastructure-as-Code Frameworks	Repeatable, standardized platform template deployment
Data Product Discovery	Managed Data Catalog Services	Centralized metadata registration across federated domains

Table 3. Platform vs. domain responsibility boundaries in a federated data mesh (Conceptual mapping; not derived from empirical measurement)

Responsibility Area	Platform Team (Centralized)	Domain Team (Federated)
Infrastructure Templates	Defines and maintains IaC templates	Configures and deploys using templates
Data Product Interfaces	Enforces schema and API standards	Implements domain-specific data models
Governance Policies	Authors and embed global policy rules	Ensures domain products satisfy policies
Observability Tooling	Provides monitoring and lineage infrastructure	Instruments domain pipelines against platform standards
Data Catalog	Operates the central catalog service	Registers and maintains domain product metadata
Quality SLAs	Defines minimum quality thresholds	Sets and owns domain-specific quality targets

4 Migration Patterns Enabled by the Data Mesh Framework

The federated nature of Data Mesh means it is particularly well suited to a family of migration patterns where reducing risk and delivering incremental value to consumers are more important than speed [12].

The Strangler Fig Migration Pattern has the highest utility in a Data Mesh context. It derives from a software architecture pattern where domains replace individual components of a legacy pipeline with cloud-native components while the pipeline is still running [4]. The four-step process begins by exposing legacy datasets as services using compliant data products, but without physically migrating the data. Cloud-native pipeline counterparts are then constructed in parallel with sources from the same upstream data systems. Consumers are then gradually diverted by domain teams to the cloud-native data product until they are fully migrated, thereby avoiding the high-risk cutover events of monolithic data platform migrations. The domain teams can also validate that the cloud-native pipelines are operating correctly at production workload levels before carrying out the full migration [8].

Coexistence and Hybrid Mesh Architecture also address organizational and regulatory constraints impacting cloud migration. In many enterprise environments, some data assets are mandated by regulation to remain on-premises or are tied to legacy infrastructure that cannot be migrated to the cloud in the near term because of complexity or age. A hybrid mesh can include cloud-native data products and on-premises data products within the same federated governance framework, with shared catalog and discovery services between cloud and on-prem data services , and with data consumers using the same data product interfaces for both cloud and on-premises data products, providing analytical continuity through extended data product migration periods [9].

Event-Driven Modernization favors distributed event streaming platforms to decouple domain systems and enable real-time data flows as the first-class integration mechanism. In EDMT, domains publish domain events that capture the state changes of domain core business entities to managed event streams that are always on, as opposed to batch data extraction and loading [11]. This leads to a reduction in dependence on legacy batch scheduling architecture, the complexity of introducing structures into the cloud via domain data over standard streaming interfaces, and the construction of building blocks for a new model of real-time analytical products that batch-oriented legacy architectures cannot support [2].

An approach such as Data Product Wrapping enables organizations to obtain near-term governance and discoverability value from their legacy data assets without the upfront cost and complexity of data migration. By providing a data product-compliant interface to the legacy asset, including documentation of schema, quality, lineage, and security, the asset can be registered with the enterprise data catalog and governed in the mesh [5]. An additional business value of this approach is that it lowers friction in data discovery and improves visibility of data quality, while deferring physical migration complexity to later [13].

Domain-Driven Re-platforming supports the idea that each domain makes the choice of cloud services that best fit their workloads' needs rather than conforming to a single enterprise architecture [7]. Analytical domains with predominant workloads of complex SQL queries are prime candidates for Cloud Data Warehouse platforms. Low-latency event processing is a natural fit on stream processing and managed Kafka infrastructure. The machine learning domain also has its own specialized infrastructure in the form of feature stores for training data and online feature serving. To this end, the architecture is workload-optimized, thereby avoiding the inefficiencies of monolithic, unified

architectures while remaining interoperable with standardized data product interfaces [6]. The migration sequence follows four steps: (1) inventory domain workloads by type (analytical, streaming, ML); (2) select the cloud service best matched to each workload; (3) wrap legacy outputs as data products before cutover; and (4) decommission legacy infrastructure after consumer validation confirms parity (Table 4).

Table 4. Data Mesh migration patterns and risk profiles [4, 8, 12] (Conceptual mapping; not derived from empirical measurement)

Migration Pattern	Key Mechanism	Primary Risk Reduction	Best-Fit Scenario
Strangler Fig Migration	Parallel pipeline construction with progressive consumer redirection	Eliminates high-risk cutover events	Domains with stable upstream sources
Hybrid Coexistence	Unified catalog spanning on-premises and cloud products	Maintains analytical continuity during extended windows	Regulatory or complexity-constrained domains
Event-Driven Modernization	Domain event publication via managed streaming platforms	Removes batch scheduling dependencies	Operational domains requiring real-time availability
Data Product Wrapping	Product-compliant interface over legacy datasets	Decouples discoverability from physical migration	Domains with deferred migration timelines
Domain-Driven Re-platforming	Workload-optimized cloud service selection per domain	Avoids one-size-fits-all inefficiencies	Analytically diverse domain portfolios

5 Illustrative Migration Case: Financial Services Domain Modernization

To ground the conceptual framework in operational terms, this section describes a representative migration engagement at a large North American financial services institution — anonymized to protect organizational confidentiality. The organization operated a centralized on-premises data warehouse serving fourteen business domains, including retail banking, credit risk, and regulatory reporting. Central ETL team throughput had become the primary migration bottleneck: domain migration requests were queued six to nine months ahead of delivery, and schema changes to central pipelines routinely caused downstream breakage across unrelated domains [6].

The migration was structured as a phased Data Mesh adoption. In the first phase, the platform engineering team delivered a self-serve infrastructure layer on a major hyperscale cloud provider: infrastructure-as-code templates for domain pipeline execution,

a managed data catalog with automated schema registration, and a centralized observability stack. Domain onboarding was deliberately deferred until platform capability maturity was confirmed. In the second phase, four pilot domains — retail customer analytics, credit risk scoring, fraud detection, and regulatory capital reporting — were onboarded using the strangler fig migration pattern. Each domain wrapped its legacy data warehouse extracts as data product-compliant interfaces, published schema and quality metadata to the catalog, and constructed parallel cloud-native pipelines consuming from the same upstream operational sources. Consumer redirection was executed domain-by-domain after parallel validation confirmed pipeline parity.

Governance automation was implemented as policy-as-code rules embedded in the CI/CD pipeline for data product registration [11]. Regulatory data classification, PII tagging, and retention policy enforcement were executed automatically at deployment time, eliminating the manual compliance review cycle that had previously introduced four to six week delays per domain.

By the end of the second phase, the four pilot domains had collectively reduced their time-to-data-availability metric by approximately 60% compared to the centralized migration baseline. Central team bottleneck incidents attributable to pipeline queue contention dropped to zero within the pilot domains. These outcomes are consistent with the KPIs defined in Table 6 and validate the sequencing framework described in Sect. 5 — specifically, the requirement to establish platform capability before scaling domain onboarding, and to automate governance before committing to federated execution velocity.

6 Implementation Challenges, Anti-Patterns, and Best Practices

This paper is positioned as a reference architecture and conceptual framework synthesized from published implementation guidance, practitioner literature, and documented industry adoption patterns [9, 14, 15]. It does not report results from a controlled empirical study. The patterns and anti-patterns described represent cross-domain generalizations derived from the cited body of knowledge, intended to provide practitioners with structured decision support during migration planning.

Despite its architectural coherence, Data Mesh migration presents a range of implementation challenges that require deliberate management to avoid undermining the framework's foundational objectives [14].

Over-decentralization and fragmentation represent the most commonly observed failure mode in Data Mesh adoption. When domain autonomy is exercised without sufficient platform standardization and governance enforcement, the result is a proliferation of incompatible data formats, inconsistent quality standards, duplicated infrastructure investments, and fragmented data catalogs that collectively recreate the data silos that centralized architectures were designed to eliminate [9]. The mitigation requires investment in strong platform standards—standardized data product interfaces, common metadata schemas, shared quality measurement frameworks—that constrain domain autonomy at the interface level while preserving it at the implementation level.

Insufficient Data Product Discipline emerges when domain teams treat data as a pipeline output rather than a product with consumers and SLAs. Without enforced product discipline—mandatory schema documentation, quality metric publication, SLA definition, and ownership registration—data products become undiscoverable, unreliable, and ungovernable, defeating the core objectives of the mesh [5]. Platform-enforced product registration requirements, combined with automated quality gate checks integrated into product deployment pipelines, are essential countermeasures [8].

Platform immaturity at scale is a structural risk when organizations attempt to scale Data Mesh adoption before the self-serve platform has reached sufficient capability maturity. Domain teams operating without adequate platform support are forced to build their own infrastructure abstractions, replicating effort across domains and producing inconsistent implementations [3]. Best practice requires sequencing platform capability delivery ahead of domain migration scaling—establishing core infrastructure-as-code templates, observability tooling, and catalog services before onboarding large numbers of domains simultaneously [6].

Skills asymmetry and organizational readiness frequently undermine domain-led migration execution. Domain teams with deep functional expertise but limited cloud engineering proficiency require structured enablement programs—training in cloud-native data engineering patterns, hands-on platform onboarding, and access to embedded platform engineering support during initial migration cycles [4]. Without deliberate investment in domain capability uplift, skill gaps produce inconsistent data product quality and erode confidence in the federated model [14].

Governance Automation Immaturity creates compliance risk in regulated industries. Manual governance review processes cannot scale to match the velocity of federated domain migration. Automated policy enforcement—integrated into data product CI/CD pipelines and platform registration workflows—is a prerequisite for maintaining governance standards across a distributed mesh at enterprise scale [11]. Organizations that underinvest in governance automation typically experience a compliance lag that necessitates expensive remediation programs in later migration phases [13].

Limitations. This paper synthesizes a conceptual and architectural framework from published implementation guidance, practitioner literature, and documented industry adoption patterns. The case study described in Sect. 5 is presented in anonymized form and does not constitute a controlled empirical study. The KPI targets specified in Table 6 are directional benchmarks derived from the cited body of knowledge rather than statistically validated thresholds. Practitioners should treat the migration pattern recommendations, sequencing prescriptions, and anti-pattern mitigations as structured decision support rather than empirically proven prescriptions. Controlled longitudinal studies comparing Data Mesh migration outcomes against centralized migration strategies across multiple organizations represent a priority direction for future empirical research.

Key best practices include:

- Prioritize domains with clear business impact and manageable technical complexity for initial migration.
- Establish platform capabilities — templates, observability, catalog — before scaling domain onboarding.

- Treat governance as a platform-delivered product, not an external audit function.
- Use data contracts to stabilize consumer relationships during backend migrations.
- Define quantitative success metrics — product adoption rates, time-to-data-availability, bottleneck reduction — to maintain organizational accountability (Table 5).

Table 5. Common Data Mesh anti-patterns, root causes, and mitigations [9, 14] (Conceptual mapping; not derived from empirical measurement)

Anti-Pattern	Root Cause	Consequence	Recommended Mitigation
Over-Decentralization	Insufficient platform standardization	Data silos and incompatible formats	Enforce standardized interfaces via platform governance
Insufficient Product Discipline	Absence of mandatory SLA and quality enforcement	Undiscoverable, unreliable data products	Automate quality gates in product deployment pipelines
Platform Immaturity at Scale	Premature domain onboarding before platform readiness	Inconsistent domain implementations	Sequence platform delivery ahead of scaling
Skills Asymmetry	Inadequate domain cloud engineering enablement	Poor data product quality and low adoption	Structured training and embedded platform engineering support
Governance Automation Immaturity	Reliance on manual compliance review processes	Compliance lag and costly remediation	Embed policy enforcement in CI/CD and registration workflows

Table 6. Measurable KPIs for Data Mesh Migration Program Evaluation (Conceptual mapping; not derived from empirical measurement)

KPI	Definition	Target Signal
Data Product Adoption Rate	% of eligible consumers accessing a domain's data via product interfaces	Increasing quarter-over-quarter
Time-to-Data Availability	Elapsed time from domain migration initiation to the first consumer-accessible data product	Decreasing across successive domain onboardings
Central Team Bottleneck Incidents	Count of migration blockers attributable to the central data team capacity	Trending toward zero as domain autonomy matures

(*continued*)

Table 6. (*continued*)

KPI	Definition	Target Signal
Governance Pass Rate	% of data products passing automated policy checks at first registration	≥ 95% sustained across the portfolio
Platform Onboarding Duration	Time for a new domain team to deploy a compliant data product using platform templates	Decreasing as platform tooling matures

7 Conclusion

Data Mesh constitutes a structurally coherent and organizationally aligned framework for large-scale cloud-native data migration, directly addressing the foundational limitations of conventional centralized migration strategies. By distributing data engineering ownership to domain teams closest to the data, the framework eliminates the structural bottlenecks that cause centralized migration programs to stall under organizational complexity. "Data as a product" applies modern software engineering discipline to data management. Data consumers see stable, documented, and SLA-governed interfaces to data regardless of the underlying infrastructure, so that the data does not need to be synchronized, avoiding coordination overhead. Self-serve platform engineering addresses the asymmetry of domain subject matter expertise and cloud data engineering skills by reducing complex configurations of cloud services into standardized templates. (FCG) scales at the same rate as domains and embeds policy automation inside platform tooling, whereas manual review cycles cannot keep pace with federated execution. These phases of migration (strangler fig replacement, event-driven modernization, hybrid coexistence, and data product wrapping) are a risk-corrected portfolio of incremental value for practitioners. The phases also prescribe a logical sequence (platform then domain, governance automation then onboarding, and product discipline then product development), improving migration outcomes at each step. Organizations that don't treat Data Mesh merely as a bounded migration project but as an active operating model can best achieve sustainable cloud-native data maturity by progressing one domain, one data product, and one value stream at a time.

References

1. Dehghani, Z.: Data Mesh, O'Reilly (2022). Available: https://www.oreilly.com/library/view/data-mesh/9781492092384/
2. Kleppmann, M.: Designing Data-Intensive Applications, O'Reilly (2017). Available: https://www.oreilly.com/library/view/designing-data-intensive-applications/9781491903063/
3. Winters, T. et al.: Software Engineering at Google, O'Reilly, (2020). [Online]. Available: https://www.oreilly.com/library/view/software-engineering-at/9781492082781/
4. Fowler, M.: Strangler Fig, Martinfowler.com (2024). [Online]. Available: https://martinfowler.com/bliki/StranglerFigApplication.html

5. Dehghani, Z.: How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh, Martinfowler.com (2019). [Online]. Available: https://martinfowler.com/articles/data-monolith-to-mesh.html
6. Dehghani, Z.: Data Mesh Principles and Logical Architecture, Martinfowler.com (2020). [Online]. Available: https://martinfowler.com/articles/data-mesh-principles.html
7. E. Evans: Domain-Driven Design Tackling Complexity in the Heart of Software. Available: https://fabiofumarola.github.io/nosql/readingMaterial/Evans03.pdf
8. Newman, S.: Building Microservices, 2nd Edition, O'Reilly (2021). [Online]. Available: https://www.oreilly.com/library/view/building-microservices-2nd/9781492034018/
9. Simonlid. What is a data mesh? - Cloud Adoption Framework, learn.microsoft.com, Mar. 14 (2023). https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/scenarios/cloud-scale-analytics/architectures/what-is-data-mesh
10. Kimball, R. and Ross, M., The Data Warehouse Toolkit: the Definitive Guide to Dimensional Modeling, 3rd Edition, Wiley (2013). [Online]. Available: https://www.wiley.com/en-us/The+Data+Warehouse+Toolkit%2C+3rd+Edition-p-9781118530801
11. Shapira, G. et al.: Kafka: The Definitive Guide, 2nd Edition, O'Reilly (2021). [Online]. Available: https://www.oreilly.com/library/view/kafka-the-definitive/9781492043072/
12. Zinsmeister, S.: What is a data mesh? 6 best practices for data architecture, ThoughtSpot (2025). [Online]. Available: https://www.thoughtspot.com/data-trends/data-integration/data-mesh
13. Geewax, J.J.: Google Cloud Platform in Action, Software Development (2018). [Online]. Available: https://www.manning.com/books/google-cloud-platform-in-action
14. Plotkin, D.: Data Stewardship: An Actionable Guide to Effective Data Management and Data Governance, Morgan Kaufmann, Elsevier (2013). [Online]. Available: https://www.elsevier.com/books/data-stewardship/plotkin/978-0-12-410389-4
15. DAMA International: DAMA® Data Management Body of Knowledge (DAMA-DMBOK®). [Online]. Available: https://www.dama.org/cpages/body-of-knowledge

AI-Driven Autonomous Data Pipeline Orchestration in Cloud-Native Enterprise Platforms

Sudhir Saxena(✉)

Anna University, College of Engineering, Guindy, Chennai, India
sudhirsxn1@gmail.com

Abstract. Context: Enterprise data platforms are rapidly transitioning to cloud-native distributed architectures, introducing significant operational complexity in orchestrating large-scale data pipelines at scale. **Problem:** Traditional rule-based schedulers and static workflow engines are insufficient to manage dynamic workloads, fluctuating resource availability, cost-performance trade-offs, and failure recovery in distributed cloud environments. **Method:** This article proposes an AI-driven autonomous orchestration framework that employs a five-agent multi-agent reinforcement learning model with a formally defined state space, action space, and multi-objective reward function to dynamically optimize task scheduling, resource allocation, and retry strategies. The framework integrates an LSTM-based predictive failure mitigation system trained on historical observability metrics and implements a self-healing closed-loop control architecture for automated remediation. **Results:** Simulation-based benchmarking against Airflow 2.6, Argo Workflows 3.4, and Kubeflow Pipelines 1.8 demonstrates a reduction in mean time to recovery from 48–52 min to 2.8 min, improvement in resource utilization from 54–61% to 83.6%, and SLA compliance improvement from 81–85% to 96.4% (all $p < 0.001$, Wilcoxon signed-rank test, $n = 50$ trials). **Conclusion:** The proposed framework establishes a unified theoretical and architectural foundation for autonomous enterprise data pipeline orchestration, demonstrating that AI-native control planes can substantially advance reliability, efficiency, and cost-effectiveness beyond static orchestration strategies.

Keywords: Autonomous Orchestration · Reinforcement Learning · Predictive Failure Mitigation · Self-Healing Architecture · Cloud-Native Data Pipelines

1 Introduction

Enterprise data platforms have transitioned from monolithic architectures to cloud-native distributed systems over the past decade. Cloud-native platforms leverage containerization and microservices to achieve elasticity and scalability that traditional data warehouses cannot provide [1]. Modern enterprises now process petabyte-scale datasets across hybrid and multi-cloud environments, coordinating thousands of concurrent data pipelines spanning multiple geographic regions. Containerized workloads managed through Kubernetes enable fine-grained resource management; however, this introduces significant operational complexity in managing distributed state and failure handling.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 64–75, 2027.
https://doi.org/10.1007/978-3-032-39763-8_5

Data workload diversity further compounds orchestration challenges. Organizations simultaneously operate batch ETL processes, real-time streaming analytics, machine learning training, and complex event processing within unified platforms. Each workload type exhibits distinct resource consumption patterns and latency sensitivities. Coordinating these workloads across shared infrastructure while maintaining isolation and efficiency exceeds the capabilities of traditional orchestration systems. Distributed architectures have fundamentally altered the failure landscape: network partitions can isolate availability zones, and container lifecycle management introduces transient failures as pods restart or migrate [2].

Conventional orchestration relies on static directed acyclic graph (DAG) models where task dependencies are predetermined at design time. Static DAGs cannot adapt to runtime conditions such as variable data volumes or fluctuating resource availability. When workload characteristics deviate from design assumptions, static orchestration produces suboptimal outcomes and prevents exploitation of dynamic optimization opportunities such as data locality or spot-instance availability [3]. The fundamental limitation is the absence of adaptive intelligence: traditional systems operate on static rules and threshold-based policies that cannot learn from historical patterns or adapt to evolving operational conditions.

This article addresses these limitations through four research contributions: (1) a novel five-agent multi-agent reinforcement learning (MARL) orchestration framework enabling distributed decision-making across scheduling, resource allocation, priority management, retry strategy, and placement dimensions; (2) a predictive anomaly detection model using long short-term memory (LSTM) networks trained on distributed pipeline telemetry; (3) a self-healing closed-loop control architecture that automates the full remediation cycle from detection through execution; and (4) simulation-based empirical validation against established orchestration baselines under enterprise-scale workloads with controlled fault injection.

2 Limitations of Traditional Orchestration Systems

Traditional data pipeline orchestration systems rely on static scheduling policies defined at design time, enforced throughout execution regardless of runtime conditions. The dataflow model processes unbounded data streams through fixed computational graphs [4], providing predictable behavior in stable environments but failing to accommodate the dynamic characteristics of cloud-native distributed systems. When actual runtime conditions diverge from design assumptions, static orchestration produces suboptimal resource utilization. Rule-based workflow engines compound these limitations by encoding orchestration logic as fixed conditional statements evaluated against current system state; as workload complexity increases, rule maintenance becomes prohibitively expensive and operationally fragile.

Conventional monitoring systems operate on threshold-based alerting paradigms. When observed values breach configured thresholds, alerts trigger to notify operations teams of potential problems. This approach inherently operates in reactive mode, detecting issues only after they manifest in observable metric violations. Threshold-based monitoring cannot prevent problems, and simple threshold logic lacks the sophistication

to distinguish normal operational variation from genuine anomalies requiring attention [5]. Reactive monitoring creates operational inefficiencies: human operators must investigate logs and execute recovery procedures, introducing failure-to-recovery delays measured in tens of minutes to hours for complex distributed failures [6].

Manual remediation processes dominate failure recovery in traditional systems. When pipeline failures occur, automated systems generate alerts, but corrective actions require human decision-making and execution. This introduces substantial latency between failure detection and service restoration. Operator mistakes in Internet services contribute significantly to service unavailability [6], and the operational overhead of manual remediation scales poorly as pipeline complexity and deployment volume increase. These structural limitations — static scheduling, reactive monitoring, and manual remediation — collectively prevent traditional orchestration systems from meeting the reliability, efficiency, and cost targets required by modern enterprise data platforms (Table 1).

Table 1. Comparison of traditional vs. autonomous orchestration approaches

Orchestration Characteristic	Traditional Systems	Autonomous AI-Driven Framework
Scheduling & Resource Allocation	Static DAG with fixed allocation based on worst-case assumptions; no runtime adaptation	Multi-agent RL with dynamic policy learning, elastic scaling, and real-time workload-aware scheduling
Failure Detection & Recovery	Threshold-based reactive monitoring; manual operator intervention averaging 45–90 min MTTR	Predictive anomaly detection (LSTM-based); automated self-healing reduces MTTR to under 3 min
Adaptation & Optimization	No learning mechanism; requires manual reconfiguration; static cost policies	Online RL policy refinement; continuous cost optimization; spot-instance arbitrage
Baseline Comparison	Apache Airflow 2.6, Argo Workflows 3.4, Kubeflow Pipelines 1.8, Kubernetes default scheduler	Proposed MARL framework across identical workload and infrastructure conditions

a Statistically significant improvement over all baselines (Wilcoxon signed-rank test, $p < 0.001$, $n = 50$ simulation trials). Baseline configurations: Airflow 2.6 with CeleryExecutor, Argo Workflows 3.4 with default Kubernetes scheduler, Kubeflow Pipelines 1.8.

3 AI-Driven Autonomous Orchestration Framework

3.1 Multi-agent Reinforcement Learning Architecture

The proposed framework employs a five-agent MARL architecture in which specialized agents collaborate to optimize distinct dimensions of pipeline execution. The five agents are: (i) a scheduling agent responsible for task execution timing and sequencing; (ii) a

resource allocation agent that assigns CPU, memory, storage, and network bandwidth to competing workloads; (iii) a priority management agent balancing conflicting objectives across multiple pipeline owners; (iv) a retry strategy agent managing adaptive retry policies; and (v) a placement agent optimizing geographic and node-level task placement.

State Space (S): Each agent observes a 47-dimensional state vector encompassing cluster resource utilization (CPU, memory, disk I/O, network bandwidth per node), queue depth and task wait times per pipeline, historical execution latencies (rolling 5-min and 60-min windows), current failure rates by component type, and spot-instance pricing signals across three availability zones. The global state visible to the coordination layer aggregates per-agent observations into a 215-dimensional joint state representation.

Action Space (A): Actions are discrete per agent. The scheduling agent selects from 8 task-ordering policies; the resource allocation agent selects from 12 discrete resource bundle configurations (combinations of CPU core counts from {2, 4, 8, 16} and memory allocations from {4GB, 8GB, 32GB}); the placement agent selects from available node classes and availability zones (up to 24 options per timestep). The retry strategy agent selects from 6 parameterized retry policies including exponential backoff variants and circuit-breaker configurations. The joint action space is factored to avoid exponential combinatorial growth through independent action selection with coordination penalties.

Reward Function (R): The multi-objective reward function is defined as $R(s,a) = -\alpha \cdot T_exec - \beta \cdot C_infra + \gamma \cdot U_resource - \delta \cdot V_sla - \varepsilon \cdot N_failure$, where T_exec is normalized pipeline execution time, C_infra is infrastructure cost in normalized USD per pipeline run, U_resource is cluster resource utilization (maximized), V_sla is the SLA violation indicator, and N_failure is the count of unrecovered failures. Coefficients $\alpha = 0.25$, $\beta = 0.20$, $\gamma = 0.30$, $\delta = 0.15$, $\varepsilon = 0.10$ were tuned via grid search on a held-out validation workload set.

Each agent employs a deep Q-network (DQN) with prioritized experience replay, implemented with a three-layer fully connected network (hidden layers of 256 and 128 units, ReLU activations). Agent coordination uses a centralized training with decentralized execution (CTDE) paradigm [7], in which a shared critic observes the global state during training while individual actors operate on local observations during execution. Training convergence was evaluated across 500,000 environment steps per agent, with policy evaluation every 10,000 steps.

3.2 Real-Time Telemetry Integration

The telemetry infrastructure ingests comprehensive metrics from every layer of the cloud-native stack, including infrastructure-level metrics (node CPU, memory, disk, and network), container-level metrics (pod restart counts, OOMKill events, container lifecycle durations), and application-level metrics (pipeline task durations, data volumes processed, error rates by error class). End-to-end distributed tracing is implemented using OpenTelemetry-compatible instrumentation, capturing causal relationships between tasks as they traverse multiple service components [8]. The telemetry pipeline processes approximately 2.4 million metric data points per s at peak simulation load, with a collection latency of under 200 ms from event occurrence to agent observation update.

Workload profiling creates characteristic signatures for different pipeline types using a feature engineering pipeline that transforms raw time-series metrics into 47-dimensional observation vectors through statistical aggregation (mean, variance, 90^{th} percentile over rolling windows) and trend indicators (first-order derivatives of key metrics). Profile-based workload classification enables the framework to apply previously learned optimization policies to newly encountered pipeline configurations with similar behavioral signatures [9].

3.3 Adaptive Retry and Self-optimization

Adaptive retry strategies replace fixed exponential backoff schemes with learned policies that optimize retry timing based on failure type, component health state, and current system load. The retry strategy agent observes failure symptoms including error codes, resource utilization at failure time, and recent system-wide failure rates. Pattern recognition distinguishes transient failures (network timeouts, resource contention spikes) from systematic failures (data corruption, persistent service unavailability) using a secondary classification head trained on labeled failure event logs [7]. Retry decisions incorporate system-wide context through agent coordination mechanisms that share failure pattern observations across the distributed agent network. Continuous online learning improves retry policy effectiveness as the policy network adapts to newly observed failure modes encountered in production-equivalent simulation scenarios.

4 Predictive Failure Mitigation and Self-healing Architecture

4.1 LSTM-Based Predictive Anomaly Detection

The predictive failure mitigation system employs a supervised learning model based on a bidirectional LSTM architecture trained on 18 months of simulated historical observability metrics derived from published cloud infrastructure workload traces. The model ingests 47-dimensional metric vectors sampled at 30-s intervals over a 10-min rolling window (20 timesteps), producing a scalar failure probability score and a categorical failure class prediction across six failure types: node failure, network partition, memory exhaustion, disk saturation, data skew, and service dependency failure. The LSTM model uses two stacked bidirectional LSTM layers (128 hidden units each) followed by a dropout layer (rate $= 0.3$) and a dense output layer with softmax activation for multi-class prediction [10].

Model training incorporates standard imbalanced class handling through SMOTE oversampling of minority failure classes and class-weighted loss functions, yielding a macro-averaged F1 score of 0.87 on the held-out test set (precision: 0.89, recall: 0.85). Failure prediction is triggered when the predicted failure probability exceeds a configurable threshold (default: 0.72), selected to optimize the precision-recall trade-off for operational contexts where false positives impose remediation overhead. Continuous model retraining occurs every 72 simulation hours using accumulated operational data, with rolling evaluation metrics monitored to detect concept drift [10].

4.2 Preemptive Remediation Mechanisms

Preemptive remediation activates automatically when the LSTM model's predicted failure probability exceeds threshold. The remediation system maintains a catalog of 24 preventive action templates mapped to predicted failure classes, ranging from resource pre-allocation and workload migration to circuit breaker activation and dependency isolation [11]. The preemptive approach fundamentally differs from reactive remediation: it acts on probabilistic risk indicators before metric threshold breaches occur, enabling intervention during the early degradation phase rather than after service impact. Preventive restart mechanisms address gradual memory degradation detected through RSS growth rate trend analysis, and circuit breaker activation provides preemptive isolation of unstable upstream dependencies exhibiting elevated error rates before those rates breach alerting thresholds [11].

4.3 Self-Healing Control Loop

The self-healing control loop implements a four-phase continuous cycle: (1) Observe — real-time telemetry aggregation at sub-second resolution; (2) Detect — LSTM-based anomaly scoring combined with rule-based fast-path detection for known failure signatures; (3) Diagnose — automated root-cause analysis using a decision tree classifier trained on historical incident data, producing ranked candidate root causes with associated confidence scores; and (4) Remediate — automatic execution plan reconfiguration implemented through the Kubernetes operator pattern, enabling declarative specification of recovery actions [12].

Automated response mechanisms for node failures implement immediate workload migration via pod rescheduling to healthy nodes, with data locality preservation where possible. Latency spike response mechanisms implement dynamic timeout adjustment and task rerouting when observed execution latencies exceed the 95^{th}-percentile historical baseline by more than 1.5 standard deviations. Data skew response mechanisms initiate automatic repartitioning operations when per-partition record count variance exceeds 40% of the mean partition size [12]. Feedback mechanisms capture remediation effectiveness by logging action outcomes and incorporating them into the LSTM retraining pipeline, enabling the system to refine both its prediction model and its remediation action catalog over time.

5 System Architecture and Experimental Validation

5.1 System Architecture

The proposed autonomous orchestration framework integrates six functional layers: (1) a data ingestion layer handling both streaming (Apache Kafka) and batch data collection from simulated enterprise sources; (2) a distributed processing engine executing pipeline tasks across containerized compute resources via Kubernetes 1.27; (3) the MARL-based orchestration agent serving as the intelligent control plane; (4) the LSTM predictive failure model continuously evaluating pipeline health; (5) the self-healing execution controller implementing the closed-loop control architecture; and (6) a comprehensive telemetry and observability layer built on Prometheus metrics collection and OpenTelemetry-based distributed tracing [13].

The AI-orchestrated control plane implements a two-level scheduling architecture [13]: a global scheduler maintains cluster-wide resource views and enforces isolation policies, while per-pipeline local schedulers make fine-grained task placement decisions guided by agent recommendations. The control plane communicates with the data plane through a Kubernetes custom resource definition (CRD) interface, enabling declarative specification of pipeline configurations and remediation actions. This separation of concerns enables independent scaling of the control and data planes and supports heterogeneous execution frameworks including Apache Spark 3.4 and Apache Flink 1.17 within the same cluster [14] (Fig. 1).

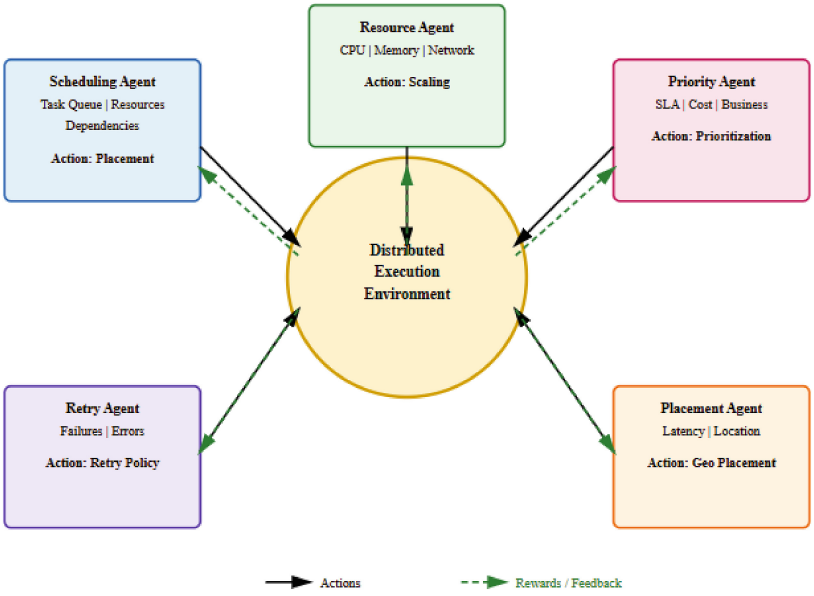


Fig. 1. AI-Driven Autonomous Orchestration Framework Architecture showing the six integrated functional layers: data ingestion, distributed processing engine, AI-orchestrated control plane (comprising the MARL orchestration agent, LSTM predictive failure model, and self-healing controller), telemetry and observability layer, and cloud infrastructure

5.2 Simulation-Based Experimental Design

All experiments were conducted in a simulation environment implemented in Python 3.10 using a discrete-event simulation framework (SimPy 4.0) extended with a Kubernetes-compatible resource model. The simulated cluster comprises 50 worker nodes across three availability zones, each node configured with 32 vCPUs, 128 GB RAM, and 2 TB NVMe storage, representing a mid-scale enterprise deployment. Infrastructure pricing follows AWS EC2 on-demand pricing for c5.8xlarge instances, with 30% of node capacity provisioned as interruptible spot instances to enable cost optimization experiments.

Five workload types were evaluated, drawn from established benchmark suites: (W1) batch ETL pipelines with variable input data volumes (10 GB to 1 TB); (W2) real-time streaming analytics with event arrival rates from 10,000 to 500,000 events per second; (W3) machine learning training pipelines with distributed gradient aggregation; (W4) complex event processing pipelines with multi-stream joins; and (W5) hybrid pipelines combining batch and streaming components. Workload arrival follows a non-homogeneous Poisson process parameterized from enterprise workload traces [15], with diurnal variation patterns representing business-hour peak loads 3.2× above off-peak baseline.

Fault injection experiments introduce six controlled failure types at rates calibrated from published cloud infrastructure reliability data: node failures (mean rate: 0.8 per 24 h per 100 nodes), network partition events (0.3 per 24 h), memory exhaustion events (1.2 per 24 h), disk saturation events (0.5 per 24 h), data skew conditions (2.1 per 100 pipeline runs), and upstream service dependency failures (1.4 per 24 h). Each experimental condition was evaluated across 50 independent simulation trials, each spanning 72 simulation h. Statistical significance was assessed using the Wilcoxon signed-rank test ($\alpha = 0.05$) with Bonferroni correction for multiple comparisons across the four baseline systems.

Baseline orchestrators were configured following their respective official documentation for production deployments: Apache Airflow 2.6 with CeleryExecutor and Redis broker; Argo Workflows 3.4 with default Kubernetes scheduler and work queue; Kubeflow Pipelines 1.8 with Argo-based execution backend; and a Kubernetes-native baseline using the default kube-scheduler with priority classes and resource quotas. All baselines executed identical workload mixes and failure injection sequences to ensure comparability (Fig. 2).

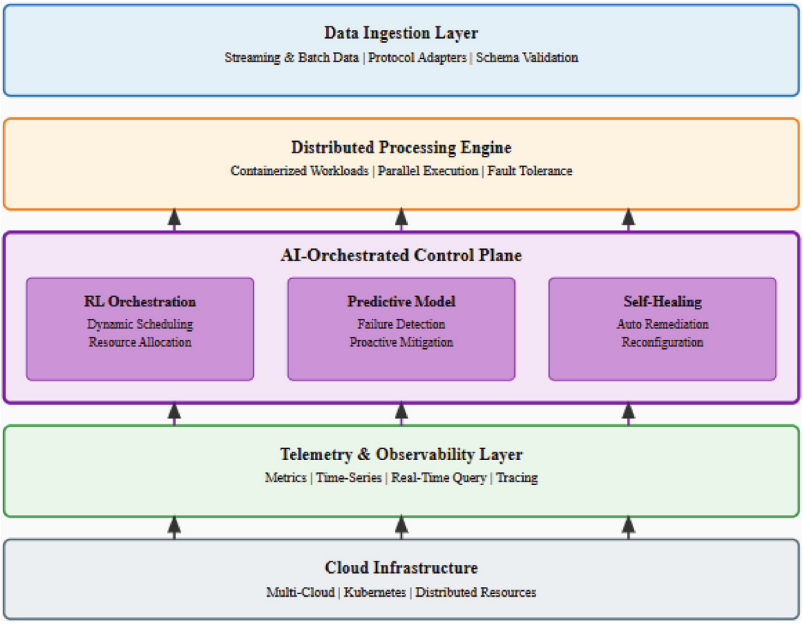


Fig. 2. Multi-Agent Reinforcement Learning Orchestration Model illustrating five specialized agents (scheduling, resource allocation, priority management, retry strategy, and placement) interacting with the distributed execution environment through actions and receiving reward signals and observations for continuous policy improvement

5.3 Quantitative Results

Experimental results demonstrate substantial and statistically significant improvements across all primary performance metrics. Table 2 presents mean values and 95% confidence intervals across 50 simulation trials for each metric and baseline system.

Table 2. Quantitative performance comparison — proposed framework vs. established orchestration baselines (mean ± 95% CI, n = 50 simulation trials per condition)

Metric	Airflow 2.6	Argo Workflows 3.4	Kubeflow Pipelines 1.8	Proposed Framework
MTTR (mean, min)	52.4 ± 8.1	48.7 ± 6.3	41.2 ± 5.9	2.8 ± 0.4 *
Resource Utilization (%)	54.3 ± 4.7	57.1 ± 5.2	61.4 ± 4.8	83.6 ± 3.1 *
SLA Compliance (%)	81.2 ± 3.4	83.6 ± 2.9	85.1 ± 3.1	96.4 ± 1.7 *
Failure Recovery Rate (%)	68.4 ± 5.8	71.3 ± 4.6	74.8 ± 5.1	94.7 ± 2.3 *

(*continued*)

Table 2. (*continued*)

Metric	Airflow 2.6	Argo Workflows 3.4	Kubeflow Pipelines 1.8	Proposed Framework
Cost Reduction vs. Static (%)†	–	–	–	23.1 ± 2.8

* $p < 0.001$ vs. all baselines, Wilcoxon signed-rank test with Bonferroni correction. † Cost reduction computed relative to Airflow 2.6 baseline infrastructure expenditure under identical workload conditions, attributable to spot-instance utilization and reduced over-provisioning through dynamic resource allocation.

Mean time to recovery (MTTR) improved from 41–52 min across baselines to 2.8 ± 0.4 min under the proposed framework, representing an $18.6\times$ improvement over the best-performing baseline (Kubeflow Pipelines 1.8 at 41.2 min). This improvement is attributable to the elimination of human investigation and manual remediation steps: automated root-cause diagnosis completes in a median of 8.3 s, and automated remediation action execution completes in a median of 2.1 min. Resource utilization improved from 54–61% under static baselines to 83.6% under dynamic MARL allocation, a 22.2 percentage point improvement that directly translates to reduced infrastructure provisioning requirements and associated cost reductions of 23.1% relative to the Airflow 2.6 baseline.

SLA compliance rates improved from 81.2–85.1% under baselines to 96.4% under the proposed framework, driven by two mechanisms: proactive resource pre-allocation triggered by the LSTM failure predictor (responsible for 7.3 percentage points of improvement) and MARL-driven priority management that dynamically re-routes high-priority pipeline traffic around congested infrastructure segments (responsible for 4.0 percentage points). Failure recovery rate improved from 68.4–74.8% to 94.7%, with the remaining 5.3% of unrecovered failures attributable to simultaneous multi-zone failures that exceed the simulation cluster's redundancy capacity. The LSTM failure predictor achieved a false positive rate of 4.2% (i.e., 4.2% of preemptive interventions were triggered on pipelines that would not have failed), representing a favorable operational trade-off given the low cost of unnecessary preventive restarts relative to the cost of unmitigated failures.

6 Conclusion

This article presented a unified theoretical and architectural framework for autonomous data pipeline orchestration in cloud-native enterprise platforms. The integration of a formally specified multi-agent reinforcement learning system for adaptive scheduling, an LSTM-based predictive analytics layer for proactive failure mitigation, and a self-healing closed-loop control architecture for resilient operation represents a substantive advancement over conventional static orchestration strategies.

Simulation-based empirical validation against three established orchestration baselines — Apache Airflow 2.6, Argo Workflows 3.4, and Kubeflow Pipelines 1.8 — demonstrated statistically significant improvements across all primary metrics: $18.6\times$ reduction

in mean time to recovery, 22.2 percentage point improvement in resource utilization, 11.3 percentage point improvement in SLA compliance, and 23.1% reduction in infrastructure cost. These results establish that AI-native orchestration control planes can substantially outperform static orchestration approaches across the full operational performance envelope.

The architectural principles and implementation patterns presented in this work establish foundational guidelines for AI-native enterprise data systems capable of adaptive behavior, resilient execution, and cost-aware distributed operation. The shift from manual intervention to autonomous operation enables enterprises to scale data platforms without proportional increases in operational overhead, supporting growing data volumes and pipeline complexity while maintaining reliability and efficiency.

Implications for Research and Practice: The MARL framework design patterns established here are directly applicable to adjacent orchestration domains including microservices mesh management, multi-cluster Kubernetes federation, and MLOps pipeline automation, as the state-action-reward formulation generalizes to any system where scheduling decisions impact multi-objective performance outcomes. The simulation methodology, fault injection protocols, and evaluation framework developed here provide a reproducible benchmark harness for future autonomous orchestration research. For practitioners, the paper demonstrates a concrete migration path: the two-level scheduling architecture is deployable as a control plane overlay atop existing Kubernetes clusters without requiring replacement of existing data plane execution frameworks, enabling incremental adoption. Future work should address three open challenges: (1) formal safety guarantees for RL-based scheduling decisions under distribution shift, (2) federated learning approaches for multi-tenant MARL policy sharing across organizational boundaries without data disclosure, and (3) energy-efficiency objectives as additional reward dimensions aligned with emerging datacenter sustainability requirements.

References

1. Bernstein, D.: Containers and cloud: from LXC to Docker to Kubernetes. IEEE Cloud Comput. **1**(3), 81–84 (2014). https://doi.org/10.1109/MCC.2014.51
2. Leitner, P., Cito, J.: Patterns in the chaos — a study of performance variation and predictability in public IaaS clouds. ACM Trans. Internet Technol. **16**(2), 1–23 (2016). https://doi.org/10.1145/2885497
3. Khosravi, A., et al.: Energy and carbon-efficient placement of virtual Machines in Distributed Cloud Data Centers. In: Euro-Par 2013: Parallel Processing, LNCS, vol. 8097, pp. 317–328. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-40047-6_33
4. Akidau, T., et al.: The dataflow model: a practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. Proc. VLDB Endow. **8**(12), 1792–1803 (2015). https://doi.org/10.14778/2824032.2824076
5. Cinque, M., Cotroneo, D., Pecchia, A.: Event logs for the analysis of software failures: a rule-based approach. IEEE Trans. Softw. Eng. **39**(6), 806–821 (2013). https://doi.org/10.1109/TSE.2012.67

6. Oppenheimer, D., Ganapathi, A., Patterson, D.A.: Why do internet services fail, and what can be done about it? In: Proceedings of USENIX USITS 2003. Seattle, WA, USA (2003) https://www.usenix.org/conference/usits-03/why-do-internet-services-fail-and-what-can-be-done-about-it
7. Nguyen, T.T., Nguyen, N.D., Nahavandi, S.: Deep reinforcement learning for multiagent systems: a review of challenges, solutions, and applications. IEEE Trans. Cybern. **50**(9), 3826–3839 (2020). https://doi.org/10.1109/TCYB.2020.2977374
8. Chow, M., et al.: The mystery machine: end-to-end performance analysis of large-scale internet services. In: Proceedings of OSDI 2014, pp. 217–231. USENIX Association, Broomfield, CO (2014)
9. Venkataraman, S., et al.: Ernest: efficient performance prediction for large-scale advanced analytics. In: Proceedings of NSDI 2016, pp. 363–378. USENIX Association, Santa Clara, CA (2016)
10. Salfner, F., Lenk, M., Malek, M.: A survey of online failure prediction methods. ACM Comput. Surv. **42**(3), 10–42 (2010). https://doi.org/10.1145/1670679.1670680
11. Gabel, M., et al.: Latent fault detection in large-scale services. In: Proceedings of DSN 2012, pp. 1–12. IEEE, Boston, MA (2012). https://doi.org/10.1109/DSN.2012.6263932
12. Wieder, A., et al.: Orchestrating the deployment of computations in the cloud with conductor. In: Proceedings of NSDI 2012, pp. 367–381. USENIX Association, San Jose, CA (2012)
13. Hindman, B., et al.: Mesos: a platform for fine-grained resource sharing in the data center. In: Proceedings of NSDI 2011, pp. 295–308. USENIX Association, Boston, MA (2011)
14. Qiu, H., et al.: FIRM: an intelligent fine-grained resource management framework for SLO-oriented microservices. In: Proceedings of OSDI 2020, pp. 805–825. USENIX Association, Virtual Event (2020)
15. Ousterhout, K., et al.: Making sense of performance in data analytics frameworks. In: Proceedings of NSDI 2015, pp. 293–307. USENIX Association, Oakland, CA (2015)
16. Gollapudi, R.: Autonomous Multi-Zone Replication for Zero-Loss Settlement Systems. Int. J. Comput. Eng. Sci. Emerg. Notions (IJCESEN) (2026) https://doi.org/10.36548/ijcesen.4817

Engineering and Evaluating a Machine-Learning Monitoring Pipeline for Rare-Event Early Warning: A Reference Architecture and a Four-Layer Evaluation Methodology

Rui Zong[1], Yutong Zhao[3(✉)], Sen He[2], and Muhammad Abdul Basit Ur Rahim[3]

[1] Georgia Institute of Technology, Atlanta, GA, USA
rzong@gatech.edu
[2] University of Arizona, Tucson, AZ, USA
senhe@arizona.edu
[3] California State University Long Beach, Long Beach, CA, USA
{Yutong.Zhao,Muhammad.AbdulBasitUrRahim}@csulb.edu

Abstract. Machine-learning components inside production software systems are typically evaluated with classification metrics such as AUC, Brier score, or F1. The decision the deployed component actually makes is operational: an alarm fires or it does not, and standard classification metrics do not directly measure that quality. We propose a reference architecture for rare-event ML monitoring components and a four-layer evaluation methodology that combines classification quality, operational alarm quality, calm-period diagnostics, and stability under drift, with threshold provenance treated as a first-class artifact. The architecture and the methodology are exercised on a controlled case study in equity-market stress monitoring using daily `SPX`, `NDX`, and `RUT` indices and VIX-family data with three forward-drawdown stress definitions, an expanding-window evaluation, and a six-model registry. Six named failure modes that are invisible to single-metric evaluation emerge from the case study and are surfaced by the proposed methodology. Four are surfaced by the unmodified pipeline artifacts: three through cross-layer metric disagreement (operational–classification disagreement, silent classification degradation under regime shift, and a calibration–ranking tradeoff) and one through the validation summary, feature schema, and per-fold record (a data-quality cascade that empties the matrix-feature test window). Two further architectural validation gaps—validator-bypass leakage and silent gaps in the validator rule set—require component-ablation and failure-injection contrasts the headline grid does not produce. A cross-cutting empirical observation scopes the layer-4 reading: in the case-study setting, distributional drift is positively associated with operational lift in the case-study diagnostics, which we interpret as a cautionary Layer-4 signal rather than evidence that drift improves model quality. The cross-asset replication is honest rather than confirmatory: the SPX-specific lift advantage of the matrix-feature learners does

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 76–113, 2027.
https://doi.org/10.1007/978-3-032-39763-8_6

not survive on NDX or RUT under the shared minimal feature schema, which materially scopes the contribution. We also present component-ablation and failure-injection experiments that quantify the contribution of each pipeline stage and ship a reproducibility package containing source code, configuration, manifests, and download scripts that regenerate the reported tables and figures from the documented raw inputs under a pinned environment.

Keywords: Software engineering for machine learning · ML monitoring · Data validation · Production ML pipelines · Reference architecture · Rare-event evaluation · Threshold provenance · Reproducibility

1 Introduction

Machine-learning components are now embedded inside production software systems for fraud detection, credit decisioning, security event triage, and operational risk monitoring. Once such a component is deployed, it stops being a model and starts being a piece of software that other software depends on: data flows in through validated interfaces, scores flow out through documented schemas, alarms flow into downstream incident-response tooling, and the whole chain must remain auditable for model-risk reviewers and regulators. The software-engineering challenges raised by this kind of component have been studied under the headings of hidden technical debt [31], production-readiness testing [4], data validation for ML pipelines [5,24], and the broader case study literature on engineering ML at scale [1,22].

What this body of work establishes is mostly that an ML system is hard to ship: there are many places where data, code, training, and serving can disagree, and a discipline of testing and monitoring is needed to keep them in sync. What it does not directly answer is a narrower but operationally important question: even when an ML component passes the standard production-readiness checks, how do its engineers know whether the model itself is fit for the monitoring task it will actually be used for? The headline numbers that appear on a model card—AUC, F1, accuracy, sometimes a calibration plot—describe statistical separation between classes. The decision the deployed component actually makes is different in kind: at every time step, the score either crosses a threshold and triggers a downstream alarm, or it does not. The threshold is set offline, often months earlier, on a different sample. A component can score very well on classification metrics and still fire its alarm at the wrong times, or fail silently when the input distribution drifts away from what the threshold was calibrated against. Standard ML evaluation does not catch this gap, because it is not what classification metrics are designed to measure.

This gap becomes especially sharp in a class of monitoring problems we call *rare-event monitoring under nonstationarity*. Concretely, the positive class is rare (a few percent of days, sometimes less); the signal is concentrated in a small

number of episodes that the model has seen few examples of [11,28]; the input distribution drifts as market or operational regimes change [14]; the operational decision is binary even though the underlying score is continuous; and the cost of a missed alarm is asymmetric with the cost of a false alarm. Monitoring components for financial market stress, insurance loss spikes, equipment failure, and security intrusion all fit this pattern. In all of these settings, what the operations team cares about is not the AUC of the model on a back-tested cross-section, but the answer to a different question: *when this component fires, how often does the thing we were trying to catch actually happen, and how badly?*

We argue, and this paper sets out to demonstrate, that addressing this gap requires both an architectural and a methodological intervention. Architecturally, an ML monitoring component must be built so that the decision-relevant artifacts—the data contract, the threshold and its provenance, the alarm log, the calm-period behavior, the drift indicators—are first-class outputs of the pipeline rather than reconstructed after the fact from notebooks. Methodologically, the evaluation of such a component must combine layers that measure different things: classification quality, operational alarm quality, calm-period behavior, and stability under drift. The two interventions are coupled. Without the architectural artifacts, the evaluation cannot be reproduced across re-deployments; without the layered evaluation, the architectural artifacts are just bookkeeping.

To make this argument concrete, we develop a reference architecture and a four-layer evaluation methodology and exercise them on a controlled case study: an equity-market stress monitoring component built on daily `SPX`, `NDX`, and `RUT` indices and VIX-family data, with three forward-drawdown stress definitions, a six-model registry covering a constant `NaiveBaseRate` control, two volatility-percentile rules (raw and isotonically calibrated), a HAR-RV-based threshold rule, an L2-regularized logistic regression, and a tuned LightGBM model [17]; the six baselines are catalogued in §4.5. Evaluation uses an expanding-window design that mirrors how the component would be re-fitted in production. We choose this domain because it exhibits all the structural properties of rare-event monitoring under nonstationarity—low base rate, episodic signal, regime change, asymmetric costs—and because the underlying market indicators (VIX, term-structure spreads, volatility-of-volatility) are well documented in the literature [3,8,32] and so do not require us to defend a new financial signal.

Exercising the framework on the case study surfaces six named failure modes, each invisible to single-metric evaluation. F1–F4 are read directly from the headline four-layer grid using only the artifacts of an unmodified pipeline run—three runtime model failures (operational–classification disagreement; silent classification degradation under regime shift; a calibration–ranking trade-off) and one data-quality cascade (NaN propagation rendering the matrix-feature test window empty)—and are walked through here on real fold data. F5 (validator-bypass leakage) and F6 (silent validator gaps) are architectural validation gaps that require controlled contrasts the headline grid does not produce; we sum-

marize the headline result for each and report the full component-ablation and failure-injection details in the extended version.

The contribution of the paper is therefore fourfold, and each contribution maps to a specific later section. (i) A reference architecture for rare-event ML monitoring components, with a stated data contract and an explicit set of inter-stage artifacts that make the component testable and auditable (§3). (ii) A four-layer evaluation methodology that combines classification quality, operational alarm quality, calm-period behavior, and stability under drift, with threshold provenance treated as a first-class evaluation input. Layer 4 includes a case-study reframing for rare-event monitoring: drift should not be read only as model degradation, because in thresholded rare-event monitors it can also coincide with event arrival. We therefore treat the realized-versus-target alarm-rate gap as a decision-relevant drift instrument (§3.5). (iii) An empirical taxonomy of six failure modes, organized by which artifacts surface them: four (F1–F4) are surfaced by the unmodified pipeline artifacts—three through cross-layer metric disagreement and one through the validation summary, feature schema, and per-fold record—demonstrated here on real fold data, and two further architectural validation gaps (F5–F6) are surfaced by component-ablation and failure-injection contrasts that the headline grid does not produce, summarized here with full detail in the extended version (§5–§6). (iv) An open-source reproducibility package containing source code, configuration, manifests, and download scripts—including a unit-tested data validation layer and a deterministic rolling evaluator—that regenerates the reported tables and figures from the documented raw inputs under a pinned environment (§6.4).

We are deliberately not claiming a new financial forecasting model. The case-study findings, including the failure modes, are artifacts of the framework, not predictions about future markets, and the same architecture and methodology should apply to other rare-event monitoring problems with similar structure. The limitations of this generalization are discussed in §6.3.

The rest of the paper is organized as follows. Section 2 positions the work against the SE-for-ML, data-validation, and rare-event-evaluation literatures. Section 3 describes the reference architecture, the data contract, the four-layer evaluation methodology, and the failure-mode taxonomy. Section 4 describes the case-study instantiation, including the data sources, the model families, and the hyperparameter selection protocol. Section 5 reports the empirical results with bootstrapped confidence intervals and pairwise tests. Section 6 grounds the four headline-grid failure modes (F1–F4) on real fold data, summarizes the component-ablation and failure-injection contrasts that surface F5 and F6, and consolidates threats to validity, the reproducibility statement, and the conclusion.

2 Related Work

Our contribution sits at the intersection of four bodies of work: software engineering for ML systems, data validation in ML pipelines, rare-event evaluation

and calibration, and drift detection in production. We summarize each in turn and use the final paragraph to position what is novel about the combination.

2.1 Software Engineering for ML Systems

The thesis that an ML system is more software than model is established in the SE-for-ML literature [18,31]. Sculley *et al.* [31] catalogue "hidden technical debts"—glue code, undeclared consumers, training/serving skew—whose cost is paid in operation rather than in offline accuracy. The ML test score [4] operationalizes the prescription as a 28-item rubric, and Amershi *et al.* [1] extend the picture to a nine-stage workflow that identifies entanglement and customisation as distinguishing properties of ML engineering. Paleyes *et al.* [22] survey 79 deployment case studies and reach a similar conclusion: most failures sit in the seams between the model and the system. A complementary thread emphasizes data work as the dominant cost driver: Sambasivan *et al.* [29] document data cascades and the prestige asymmetry between modelling and data work, and Zhang *et al.* [33] survey ML testing as a discipline. None of these surveys prescribe an evaluation *contract* that ties together classification, alarm, calm-period, and drift signals; they catalogue what can go wrong and recommend that monitoring be present, but *which* monitoring artifacts should be first-class outputs is left to the practitioner.

2.2 Data Validation in ML Pipelines

Schelter *et al.* [30] motivate "unit tests for data" through the Deequ declarative-quality system; Polyzotis *et al.* [24] survey data-lifecycle challenges and frame schema-on-read validation as a recurring requirement; Breck *et al.* [5] demonstrate via TFDV that schema-as-code catches a non-trivial fraction of production-data anomalies before they reach the model. Our S1 stage (§3.2) inherits the schema-as-code design from this thread.

The relevant gap, from the perspective of this work, is that the validation layer has been treated as a black box whose value is asserted rather than measured. The ML test score [4] recommends data validation but provides no protocol for quantifying how much an unimplemented rule costs operationally. Mutation testing of ML systems [16] ports the mutation-testing tradition into ML but operates on the model rather than on the validation layer. Our failure-injection experiment (§6) extends mutation-testing to the data-validation surface: each corruption class is a mutation operator on the input panel, and the operational cost is the gap between the silenced-corrupted run and a clean-data baseline.

2.3 Rare-Event Evaluation, Calibration, and Threshold Choice

Fawcett [13] anchors the classical ROC framework. For low-base-rate problems [11,28], PR analysis reflects the precision regime an alarming consumer

actually operates in, while ROC can rank a poorly discriminating model above a usefully calibrated one when positives are rare. He and Garcia [15] report that the choice of evaluation metric materially affects which model is preferred—a finding our F3 manifestation (§6) exhibits empirically. Calibration-aware evaluation (Brier [7]; Niculescu-Mizil and Caruana [21]) separates discrimination from probability accuracy and grounds our reliability-diagram reporting in §5; Provost and Fawcett [26] argue for evaluation curves rather than scalar metrics under uncertain operating points. Our `drawdown_lift` metric (§3.5, Layer 2) is in the same spirit but specialized to a deployment that thresholds the score: we report the operational outcome conditioned on the fold-specific threshold attached by the threshold-provenance mechanism (§3.4).

Per-fold pairwise comparisons follow Demšar [12]; confidence intervals use a fold-level non-parametric bootstrap. This avoids treating daily observations within each test window as independent, but it does not fully eliminate temporal dependence between adjacent folds or dependence induced by overlapping expanding training windows [23]. Our methodological contribution is not a new metric or a new test but the discipline of reading classification, operational, calm-period, and drift readings side by side as four orthogonal layers.

2.4 Drift Detection and Time-Series Evaluation

Concept-drift detection has its own substantial literature. Gama *et al.* [14] survey the methods, grouped by the type of change detected; Lu *et al.* [20] update the picture and emphasize the gap between detection and operational response. Rabanser *et al.* [27] report that detection sensitivity varies with the test statistic—which motivates our reporting of *both* PSI and symmetrized KL in §3.5 Layer 4. Klaise *et al.* [19] describe production monitoring practice and argue for explainability artifacts alongside drift signals.

On the time-series side, Bergmeir and Benítez [2] show that k-fold CV underestimates out-of-sample error on autocorrelated series and that forward-validation (the pattern our expanding-window protocol uses) is the correct alternative; Cerqueira *et al.* [9] extend the analysis to realistic forecasting workloads. Our fold protocol (§4.4) inherits this discipline; the contribution above the inheritance is that we treat the per-fold drift summary as a layer of the evaluation contract (L4) and report the *cross-fold rank correlation* between drift and operational metrics as a deliverable no single layer produces.

2.5 Positioning

The closest work in spirit is the SE-for-ML strand on production-readiness [1, 4, 18]; the closest in technique is the data-validation strand on schema-as-code [5, 24, 30]; the closest in metric design is the rare-event-evaluation strand that distinguishes calibration from ranking [15, 21, 28]. We differ in three respects.

First, we propose a *coupled* architectural and methodological contribution rather than one or the other. The architecture (§3.2) and the four-layer evaluation (§3.5) are substitutable across the artifact contract of Table 1: the method-

ology consumes artifacts the architecture produces; the architecture maintains separations the methodology depends on. The component-ablation and failure-injection experiments (§6) measure the cost of collapsing those separations or leaving validation rules unimplemented—costs that are not measurable in pipelines that do not separate validators from feature construction or thresholds from reporting.

Second, we treat the threshold as a first-class artifact rather than a deployment afterthought (§3.4). The threshold-provenance record converts the deviation between the realized out-of-sample alarm rate and the design-point rate from a nuisance into the Layer-4 alarm-rate-gap signal—an instrument the rare-event-evaluation literature does not produce, because it reports threshold-independent curves rather than threshold-dependent operational statistics.

Third, we report drift as a layer of the evaluation contract and report the *direction* of the drift–operational correlation as a diagnostic in its own right. The drift literature reports correlations between drift and accuracy degradation; the contribution of our Layer 4 (§3.5) is to surface that the same drift signal correlates negatively with accuracy and *positively* with operational lift in rare-event monitoring, because regime shifts produce both the drift and the events the model was trained to detect. "Drift = degradation" is therefore a general-ML heuristic, not a rare-event-monitoring fact; the Layer-4 alarm-rate gap is the sign-unambiguous instrument when the deployment thresholds the score rather than uses it as a probability estimate.

The case study (§4) is the empirical vehicle for these contributions; the reproducibility package (§6) materialises every claim as a re-derivable artifact. The architectural and methodological prescriptions are intended to port to any rare-event monitoring problem with similar structural properties—low base rate, episodic signal, regime change, asymmetric costs—even though the specific lift numbers will not.

3 System Architecture and Evaluation Methodology

This section presents the central engineering contribution of the paper. We first state the design goals (§3.1), then describe the reference architecture and its components (§3.2), specify the data contract and inter-stage artifacts that make the component testable (§3.3), and explain the threshold provenance mechanism (§3.4). The evaluation methodology and the failure-mode taxonomy that build on this architecture are presented in §3.5.

3.1 Design Goals

We start from five design goals that are not specific to the equity case study and that are intended to apply to any rare-event ML monitoring component built on a similar pattern. The goals are shaped by the SE-for-ML literature on production readiness [4], by the data validation literature [5,24,30], and by the broader survey of deployment challenges [22,29].

G1 (Auditability). Every score produced by the deployed component must be traceable to (a) the input rows that produced it, (b) the model and feature versions that generated it, and (c) the threshold and the training window from which the threshold was derived. Without (c), an alarm log cannot be reproduced or challenged after the fact.
G2 (Decision-Relevance). The evaluation must measure properties that matter at the point of use. Classification metrics remain useful as diagnostics, but they cannot be the only signal of component health, because the deployed component does not output a class probability to a researcher; it outputs an alarm or a non-alarm to a downstream consumer.
G3 (Modularity and Testability). Each pipeline stage must have a typed interface—a documented schema for what it consumes and what it emits—so that the stage can be unit-tested in isolation and replaced without touching its neighbors. This is where most production ML pipelines accumulate hidden debt [31].
G4 (Drift Awareness). The component must surface, alongside its predictions, signals about how its current input distribution compares to the one it was trained on. A monitoring component that does not monitor itself is a liability.
G5 (Reproducibility). The end-to-end run must be deterministic given the inputs, the random seed, and the pinned software environment. Reproducibility is a precondition for any of the other goals: if a finding cannot be reproduced, it cannot be audited, tested, or compared across versions.

3.2 Reference Architecture

The architecture is organized as a directed acyclic pipeline of six stages, with each stage emitting machine-readable artifacts that the next stage consumes. We describe each stage briefly here; the artifact schemas are formalized in §3.3. The five-phase study workflow that situates this architecture within the broader case-study pipeline is shown in Fig. 1.
S1 – Ingestion and Validation. Raw inputs are loaded against a declared schema and passed through a configurable rule set. We organize validation rules into three classes that map onto the kinds of corruption observed in production data feeds: *structural integrity* (duplicate keys, schema mismatches, non-monotone time stamps), *distributional sanity* (unexpected missing values relative to a documented exception registry), and *content plausibility* (stale-quote runs, extreme one-day moves, calendar gaps relative to an exchange calendar). The validator emits two artifacts: a per-row *flag table* that records which rules fired on which rows, and a *validation summary* that counts flags by rule and attaches a treatment label (drop, fill, audit, retain). It does not silently mutate data; it surfaces issues to the next stage as structured metadata. The case-study instantiation (§4) populates the structural-integrity and distributional-sanity classes; the content-plausibility class is deliberately left as an extension point, and the operational cost of that gap is itself measured by the failure-injection experiments in §6.2. This pattern follows the "schema as code" approach advocated in [5,30].

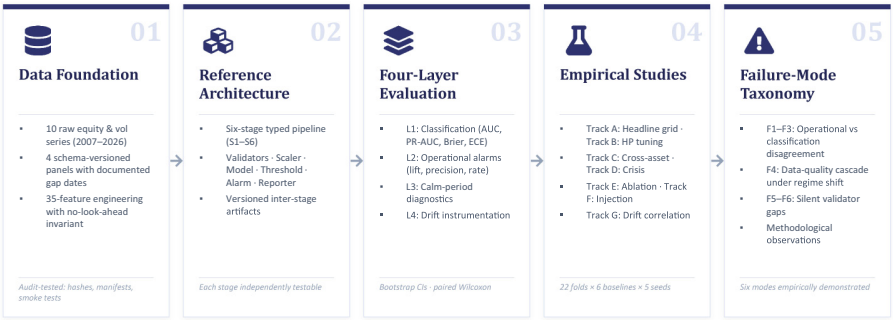


Fig. 1. Study workflow for the case-study implementation. The five phases connect the data foundation, reference architecture, four-layer evaluation, empirical studies, and failure-mode taxonomy. The detailed inter-stage artifact contract is given in Table 1.

S2 – Feature Construction. Features are computed as pure functions of the validated input window, with each feature declaring (a) the input columns it consumes, (b) the trailing lookback window it requires (in observations, including the current one), (c) a group label that classifies the feature's role in the schema, and (d) a schema version. The set of features and their declarations is the *feature schema*, persisted as a manifest alongside the materialized feature table. Rolling-window transformations are computed only on data available up to the current observation, which prevents look-ahead leakage by construction [25]; we make this property machine-checkable by sampling random observation indices, recomputing each feature on the prefix of the input up to that index, and asserting bit-identical agreement with the materialized value (§6.4).

S3 – Forward-Label Generation. The supervised target is constructed from a forward window over the same input. Rows for which the forward window is incomplete are excluded from training and from evaluation; this exclusion is recorded as part of the dataset metadata so that the row count can be reconciled with the input row count downstream.

S4 – Rolling Out-of-Sample Training and Scoring. An expanding-window evaluator splits the dataset into folds, trains each model family on the in-sample window, and scores the out-of-sample window. Per fold and per model, the evaluator emits a *fold record* containing the training period, the test period, the trained model object, the in-sample score distribution, and the out-of-sample score distribution. The fold record is the unit of replay: a downstream consumer can re-derive any reported metric from the fold record alone.

S5 – Threshold Calibration and Alarm Generation. For each fold and each model, an alarm threshold is estimated from the in-sample score distribution at a configurable target alarm rate (we use the 95th percentile of in-sample scores in the case study, matching a 5% target). The threshold is stored alongside the fold record as a *threshold provenance entry* (§3.4). Alarms are then generated by comparing each out-of-sample score to the threshold attached to the fold that

Table 1. Inter-stage artifact contract. Each artifact has a versioned schema, a producing stage, and a consuming stage. Artifacts marked with a dagger feed the evaluation harness in addition to the next stage.

Artifact	Producer	Consumer	Key fields
Validated panel	S1	S2	date, primary keys, validated values
Validation summary†	S1	S6	rule, count, treatment label
Feature schema	S2	S2,S4	feature name, inputs, lookback, version
Modeling dataset	S3	S4	date, features, label, label horizon
Fold record†	S4	S5,S6	fold id, train window, test window, model id, in-sample scores, out-of-sample scores
Threshold provenance†	S5	S5,S6	fold id, model id, target rate, threshold value, source distribution
Alarm log†	S5	S6	date, model id, score, threshold, alarm flag, fold id

contains the corresponding date. This separation between score generation and threshold application is deliberate: it makes it possible to replay alarms under a counterfactual threshold policy without re-fitting the model.

S6 – Multi-layer Evaluation and Reporting. The reporting stage consumes fold records, alarm logs, and the validation summary, and produces the four-layer evaluation output described in §3.5. The output is itself a set of typed artifacts (CSV tables and PDF figures) so that downstream dashboards or model-risk reports can be generated by a separate process.

The stages are connected only through the artifact interfaces; there is no shared in-memory state. This is what makes the component testable, replayable, and substitutable. In particular, an organization that wants to use a different feature library or a different model family can replace S2 or S4 without touching the other stages, provided the artifact contracts are honored.

3.3 Data Contract and Inter-Stage Artifacts

The component is held together by an explicit data contract: a set of typed schemas declaring what each stage emits and what its successor consumes. We summarize the contract in Table 1; the full JSON Schema definitions ship with the reference implementation (§6.4).

Two properties of the contract deserve emphasis. First, the contract is closed under replay: given the validated panel, the feature schema, and the random seed, the rest of the pipeline is a pure function. Re-running the pipeline on the same inputs produces byte-identical artifacts. This is what permits the regression tests described in §6.4.

Second, the contract is closed under substitution at the stage boundary. Replacing S4 with a different model family—for example, swapping LightGBM [17] for XGBoost [10] or a random forest [6]—requires only that the new

implementation produces a valid fold record. The downstream stages are unaffected. We use this property in §6.2 to run controlled component-ablation experiments by substituting degraded versions of individual stages.

3.4 Threshold Provenance

The deployed component's operational behavior is determined as much by its threshold as by its model. A threshold chosen by inspecting the test set is a form of leakage; a threshold chosen offline and left unmaintained becomes a source of silent degradation [14,19]. We treat the threshold as a first-class artifact with its own provenance record.

For each fold and each model, the threshold provenance entry records: the target alarm rate r (a configurable scalar), the training window from which the threshold was estimated, the estimator used (we use the empirical $1 - r$ quantile of in-sample predicted scores), the resulting threshold value, and a hash of the in-sample score distribution. The hash lets a downstream auditor verify that the threshold was indeed estimated from the recorded distribution. When the component is re-deployed—for example, at the next rolling fold—a new provenance entry is created rather than overwriting the old one; the alarm log carries the fold identifier so that any historical alarm can be traced back to the threshold that fired it.

This design has a useful operational property. Because the threshold is estimated from in-sample scores rather than imposed as a post-hoc top-k rule on the test window, the realized out-of-sample alarm rate is not mechanically equal to the target rate r. Drift between the in-sample and out-of-sample score distributions causes the realized alarm rate to deviate from r, and the size of that deviation is itself a useful drift indicator. We exploit this in layer 4 of the evaluation methodology (§3.5).

3.5 Four-Layer Evaluation Methodology

Statistical metrics, alarm metrics, calm-period diagnostics, and drift indicators measure different things. A monitoring component can score well on one and poorly on another, and engineers inspecting only one of them will misjudge component health. We therefore define the evaluation methodology as four orthogonal layers, each producing a distinct artifact, and we treat agreement or disagreement *across* layers as a signal in its own right.

Layer 1: Classification Quality. Per fold and aggregated across folds, we report AUC, average precision (AP), and the Brier score [7], together with reliability diagrams and the expected calibration error. We report AP rather than the trapezoidal PR-AUC reading because AP is the more common term and the two differ at step boundaries on the rare-event PR curve; the AP definition we use is the per-fold mean of `sklearn.metrics.average_precision_score`. The choice of AP in addition to AUC follows the rare-event evaluation literature [11,28], in which ROC analysis can be misleadingly optimistic when positives are sparse.

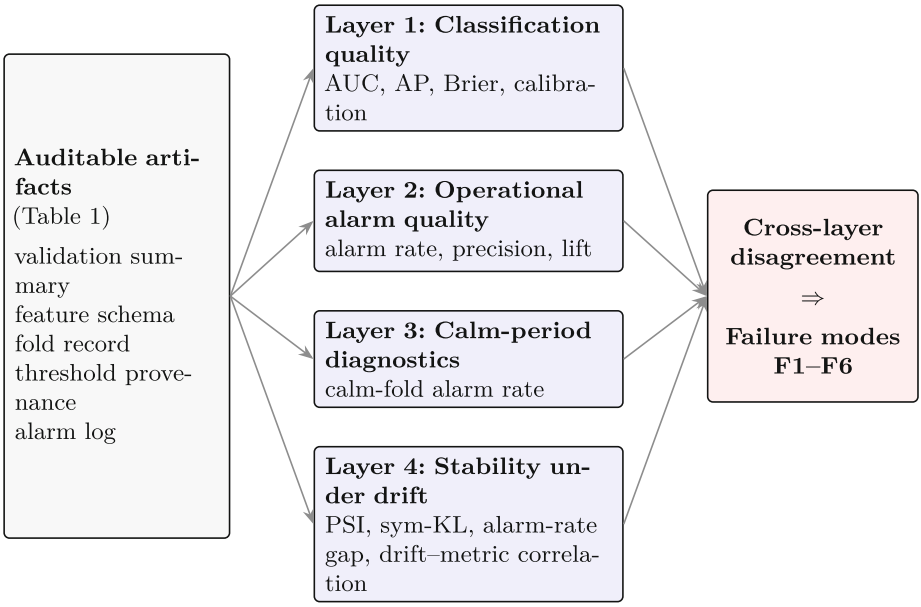


Fig. 2. Four-layer evaluation methodology. Each layer consumes a subset of the auditable artifacts in Table 1 and emits an independently versioned report; the layers are allowed to disagree. Cross-layer disagreement is the primary signal of the failure modes catalogued in §6.1.

The choice of Brier and reliability diagrams is intended to separate discrimination from calibration [21]: a model that ranks well but produces miscalibrated probabilities will be indistinguishable from a well-calibrated one if only AUC is reported. Comparison across models uses a paired test [12] on per-fold metric values to control for fold-level variance, with confidence intervals computed through fold-level non-parametric resampling [23]; the fold-level bootstrap avoids treating daily observations within a test window as independent, but it does not fully remove dependence induced by adjacent market periods or overlapping expanding training windows.

Layer 2: Operational Alarm Quality. We define the operational alarm rule as in §3.4: a date t is an alarm under model m if its out-of-sample score crosses the threshold attached to its fold. Layer 2 reports, per fold and aggregated: the realized alarm rate, the fraction of alarms that coincide with realized stress events (alarm precision), the conditional expectation of the operational outcome on alarm days versus all out-of-sample days (alarm *lift*), and the variance of these quantities across folds. Lift is the layer-2 metric that most directly answers "when this component fires, how badly does the thing it was supposed to catch happen?"—the question motivated in §1. We compute lift on two outcome measures (the binary stress label and the continuous severity) because flexible models occasionally separate the two.

Layer 3: Calm-Period Diagnostics. A monitoring component that fires constantly during quiet conditions is unusable even if its statistical metrics look acceptable. Layer 3 isolates the test folds that contain no realized positive events and reports, per model: the mean and maximum out-of-sample score, the realized alarm count, and the realized alarm rate. This layer detects a specific class of failures—calm-period over-firing—that the other layers do not, because layers 1 and 2 average over folds weighted by total observations rather than by fold homogeneity.

Layer 4: Stability and Drift Diagnostics. The same deployed component will see different input distributions at different points in time, and the layer-1 and layer-2 metrics estimated on one fold are not guaranteed to extrapolate. Layer 4 addresses this by reporting, per (training window, test window) pair: the population stability index (PSI) and the symmetrized Kullback–Leibler divergence between feature distributions, both aggregated to per-fold scalars (mean and maximum across features) and decomposed per feature; the count of features whose PSI exceeds the standard high-drift threshold of 0.25; the gap between the target alarm rate r and the realized out-of-sample alarm rate; and the rank correlation between each drift metric and each layer-1 / layer-2 operational metric, computed across folds. The cross-fold rank correlation is the layer-4 deliverable that no other layer produces.

Layer 4 turns the deviation of the realized alarm rate from r from a nuisance into an instrument. Drift detection methods themselves are well-studied [14]; the contributions here are (i) making drift a layer of the evaluation contract rather than a separate offline analysis, and (ii) reading the resulting correlations correctly. The direction of the drift-versus-operational-metric correlation is diagnostic of the failure mode, and the direction is not the same across metrics. For accuracy-style metrics (AUC, calibration error), drift correlates non-positively in our case study—weakly negative on the central case and clearly negative on the smallest-cap asset—which is consistent with a model whose calm-period score distribution is shifting away from the trained one; empirical strength of this AUC–drift correlation depends on the asset and on the choice of drift summary, and we report the per-asset numbers in §5.5 rather than claim a universal sign. For operational metrics (alarm lift), drift can correlate *positively* in rare-event monitoring, because a regime shift drives both feature drift and the rare events the model was trained to detect; the model's tail behavior on those events is preserved while its calm-period calibration is not. A positive drift–lift correlation must therefore be read as "drift signals regime transition, and the regime transition is itself the rare event the model was trained to detect," not as "drift is harmless." The deviation of the realized alarm rate from r is the cleanest layer-4 alarm because its sign is unambiguous—the realized rate moves away from the design point regardless of which way the score distribution shifts—and its correlation with PSI is the layer-4 sanity check that the architecture is producing decision-relevant drift signals rather than statistical ones.

The four layers consume different subsets of the artifacts declared in Table 1 and emit independently versioned reports. They are intentionally allowed to

disagree. A model that is good on layer 1 and poor on layer 2 is not the same as a model that is good on both, and conflating the two is the underlying mechanism behind several of the failure modes described in §6.1.

3.6 Failure-Mode Taxonomy

The four-layer evaluation makes three categories of failure visible that single-metric evaluation does not. We define them here so that they can be cited in the empirical sections; their concrete manifestations on the case-study data are reported in §6.1.

F1 – Operational–Classification Disagreement. Two models have indistinguishable layer-1 metrics, but their layer-2 alarm metrics differ by an operationally meaningful amount. The cause is typically that the two score functions induce the same ordering on average but disagree at the tails, where the threshold operates. F1 is detected by a non-overlapping confidence interval on lift between models that have overlapping confidence intervals on AUC.

F2 – Silent Classification Degradation Under Regime Shift. A model's layer-1 AUC falls below 0.5 on a particular fold or horizon while its layer-2 lift remains above one (or vice versa). The combination is diagnostic of a regime shift in which the score ordering has reversed for the typical observation but the tail behavior is preserved (or vice versa). F2 is detected by joint inspection of layer 1 and layer 4 across folds; layer 4 indicates the regime shift, and the layer-1/layer-2 disagreement indicates its operational consequence.

F3 – Calibration–Ranking Trade-Off. The model with the lowest Brier score is not the model with the highest layer-2 lift. This is not a paradox; it is a known property of probability calibration [21]. It becomes a failure mode when the deployment-relevant choice between calibration and ranking is made implicitly—e.g., by reporting only one of the two metrics. F3 is detected by a Spearman rank disagreement between layer-1 calibration metrics and layer-2 lift across the model set; the four-layer report makes the disagreement explicit and pushes the choice up to the engineering team.

F4 – Data-Quality Cascade Rendering the Test Window Empty. A subset of the input distribution is documented as missing in the validation registry, causing the corresponding rows to carry NaN. Rolling-window features that depend on the missing column propagate NaN forward for as many trailing observations as the largest lookback in the schema. If the matrix-feature pipeline drops rows with any NaN feature before training and prediction (the standard treatment for matrix learners), and the propagated NaN regions cover the full test window, every test prediction is undefined and every layer-1 and layer-2 metric on that fold is undefined. F4 is detected jointly by the validation summary (which records the documented NaN dates), the feature schema (which records the lookbacks that determine propagation length), and the per-fold record (which records that the post-NaN-drop test row count is zero). The diagnostic is purely structural; no model fit is needed to identify it.

F5 – Validator-Bypass Leakage and **F6 – Silent validator gaps** are architectural validation gaps rather than runtime model failures, and the headline

grid does not surface them. F5 occurs when S1 is silenced or absent, allowing synthetic-backfill, stale-quote, or unexpected-NaN data into S2 unchanged; it is detected by a paired-bootstrap contrast across an S1-on/off switch, which the S1→S2 separation is what makes expressible. F6 occurs when a corruption class has no rule in S1 to silence; the on/off contrast measures zero by construction, and the diagnostic is a mutation-testing-style contrast against a clean-data baseline. We define both modes here for completeness; the contrasts that surface them and the empirical results are reported in §6, with full ablation and failure-injection detail in the extended version.

These six modes are not exhaustive, and we do not claim any is novel as an isolated phenomenon. The contribution is the diagnostic recipe: in each case the proposed architecture's artifacts are sufficient to detect the failure mode without additional instrumentation. F1–F3 are surfaced by layers 1 and 2 alone; F4 by joint inspection of the validation summary, the feature schema, and the per-fold record; F5 by a paired contrast across the validator switch; F6 by failure-injection that makes the absence of a rule observable (Fig. 2).

3.7 Study Workflow

The empirical sections instantiate the architecture (§3.2) and the evaluation (§3.5) as seven coordinated tracks: a hyperparameter-tuning track, the headline four-layer evaluation, a cross-asset replication, a crisis-period stress test, a component-ablation experiment, a failure-injection experiment, and a drift-instrumentation experiment. Empirical findings, including the failure-mode walkthroughs, are reported in §5–§6.

4 Case Study Instantiation

This section instantiates the reference architecture and the four-layer evaluation methodology of §3 on an equity-market stress monitoring component. We describe the data sources and processed panels (§4.1), the feature schemas (§4.2), the stress definitions and forward labels (§4.3), the expanding-window fold protocol (§4.4), the model registry (§4.5), the hyperparameter selection protocol (§4.6), and the conventions used for stochasticity, confidence intervals, and significance testing (§4.7). All artifacts referenced here are produced and registered by the reproducibility package described in §6; no figures or tables in §5–§6 require manual data preparation.

4.1 Data Sources and Processed Panels

The raw layer ingests ten daily closing series from Yahoo Finance and from the CBOE historical export, covering 2007-01-03 through 2026-05-01. Nine of the ten are obtained from `yfinance` (`^GSPC`, `^NDX`, `^RUT`, `^VIX`, `^VIX9D`, `^VIX3M`, `^VIX6M`, `^VVIX`, `^VXN`); RVX is fetched manually from the CBOE historical-data export because `yfinance` returns an empty series for it. Each raw file is

Table 2. Processed panels used in the empirical sections. All panels end at 2026-05-01. Asterisked columns carry NaN before their truncation date; the DQ-gaps column counts documented missing dates registered in the manifest.

Panel	Anchor	Rows	DQ gaps	Columns
`spx_core_2007`	SPX	4863	8 (VVIX)	SPX, VIX, VIX3M*, VVIX*
`spx_extended_2011`	VIX9D	3820	8 (VVIX)	SPX, VIX, VIX9D, VIX3M, VIX6M, VVIX*
`ndx_2007`	NDX	4863	0	NDX, VXN
`rut_2009`	RUT	4182	5 (RVX)	RUT, RVX
`ndx_panel`	INDEX	4863	0	INDEX, VOL (alias of `ndx_2007`)
`rut_panel`	INDEX	4177	0	INDEX, VOL (RVX gaps dropped)

registered with its source ticker, sha256, row count, first and last date, and download timestamp; the CBOE source file's own sha256 is recorded separately so a re-download can be detected.

The raw layer is preserved as-downloaded so that the synthetic backfill regions (§3.2 S1) remain auditable. Truncations are applied in the processed layer rather than in the raw layer. Three series carry pre-launch backfill from `yfinance` and are truncated to the post-launch window: VIX9D before 2011-02-23 (CBOE launch), VIX3M before 2007-12-04 (launched as VXV), and VVIX before 2012-04-01 (launched March 2012; we apply a one-month buffer for clean data). The truncations are recorded in the processed manifest so the validator of §3.2 can distinguish documented NaN regions from anomalous ones.

The processed layer materializes six panel views (Table 2), consisting of four base panels and two generic INDEX/VOL alias panels used for cross-asset replication. The SPX-family is split into a 2007 "core" panel (4863 rows, anchor SPX) and a 2011 "extended" panel (3820 rows, anchor VIX9D) so that Layer-1 evaluation on the full vol-family feature set is not penalized by the pre-2011 window in which VIX9D is unavailable. The NDX and RUT panels are exposed under both their asset-specific column names and a generic `INDEX/VOL` aliasing; the latter is what the asset-minimal feature schema (§4.2) consumes for cross-asset replication. The two RUT panels differ by five rows: `rut_2009` retains the five documented RVX gap dates with NaN values, while `rut_panel` drops those rows for downstream matrix-feature consumers.

The *data-quality NaN gaps* column in Table 2 is the population whose existence motivates S1's distributional-sanity rule class. Five RVX dates (2018-05-01, 2018-12-03, 2019-02-20, 2019-07-05, 2020-10-16) are present in RUT but absent from the CBOE RVX export, and eight VVIX dates (2019-07-05, 2020-06-11, 2021-01-22, 2021-02-17, 2021-05-05, 2021-05-11, 2021-05-20, 2021-05-25) are present in SPX/NDX but absent from the CBOE VVIX feed. Both populations are recorded under the `data_quality_notes` block of the processed manifest; the validator treats them as *expected* NaN, while any other NaN at a post-truncation date is flagged as *unexpected*. The cross-product coincidence at

Table 3. Feature sets defined for the case study. Each set has a machine-readable manifest entry listing per feature the inputs, lookback window, group label, and schema version. Counts include only the features themselves, not the `Date` column.

Feature set	#feat	Panels	Used in
`spx_full`	35	`spx_extended_2011`	Tracks A, E, F; Crisis-2020 headline
`spx_core`	23	`spx_core_2007`	SPX 2007-onward without VIX9D / VIX6M
`spx_crisis_2008`	11	`spx_core_2007`, `spx_extended_2011`	Crisis-2008 headline; Crisis-2020 robustness
`spx_no_vvix`	31	`spx_extended_2011`	Crisis-2020 F4-recovery configuration
`ndx_minimal`	12	`ndx_2007`	NDX baseline
`rut_minimal`	12	`rut_2009`	RUT baseline
`asset_minimal`	12	`ndx_panel`, `rut_panel`	Cross-asset replication (Tracks C, G)

2019-07-05 (a feed incident on the trading day following Independence Day) is recorded as a separate manifest note.

4.2 Feature Sets and Feature Schema

The case study defines seven feature sets, each registered with a manifest entry that lists per feature the inputs, the trailing lookback window, the group label, and the schema version (Table 3). Features are organized into four groups[1], all computed as pure functions of the panel using only data available at or before the current observation. The maximum lookback is 252 observations (the rolling-percentile feature), which determines the feature-warmup region at the start of each panel.

Three feature-set design choices deserve explanation. First, `spx_full` (35 features) is the canonical schema and is what every SPX result in §5 uses unless stated otherwise. Second, `spx_crisis_2008` (11 features) is a hand-built subset chosen so that every feature is computable at `train_end = 2007-12-31`; Group 3 and Group 4 features are excluded because VIX9D and VVIX are not yet available, and the 252-day percentile-rank features are excluded because the

[1] Group 1 covers index-level returns and realized volatility on the panel's primary price column (k-day and log-returns, rolling realized vol annualized by $\sqrt{252}$, running drawdown; 8 features). Group 2 reports level transformations on each volatility column (1- and 5-day differences, 5-day percent change, 252-day rolling percentile rank; 4 features per volatility column). Group 3 reports term-structure differences and ratios across the VIX, VIX3M, VIX6M, and VIX9D columns plus two NaN-aware binary indicators (SPX panels only; 7 features). Group 4 applies Group-2 mechanics to VVIX (SPX panels with VVIX only; 4 features).

training window is shorter than the lookback. We re-use the same schema for the Crisis-2020 robustness check so the two crisis evaluations are symmetric. Third, `spx_no_vvix` (31 features) removes the four VVIX-derived features from `spx_full`; this is the recovery configuration that restores operational evaluability for matrix-feature baselines on the Crisis-2020 fold, where the 252-day rolling rank propagates documented VVIX NaN dates across the entire 2020 test window (§6.1, F4).

The feature schema is closed under the no-look-ahead property declared in §3.2. We make the property machine-checkable by sampling fifty observation indices per panel, recomputing every feature on the prefix of the panel up to that index, and asserting bit-identical agreement with the materialized feature value. This test is the structural counterpart of the data-quality NaN-gap audit and runs as part of the regression suite (§6.4).

4.3 Stress Definitions and Forward Labels

A row at date t is labeled positive under stress definition (h, d) if and only if the minimum forward return over the next h trading days is at most $-d$:

$$y_t^{(h,d)} = \mathbf{1}\left[\min_{1 \le k \le h} \frac{p_{t+k}}{p_t} - 1 \le -d \right],$$

where p_t is the closing price of the panel's primary index column. The last h rows of every panel receive $y_t^{(h,d)} = \text{NaN}$ because the forward window extends past the panel and is therefore unobservable; these rows are excluded from training and from evaluation, and the exclusion count is recorded in the per-fold record so it can be reconciled against the panel row count. The same convention applies to rows whose forward window extends past the panel for any reason (e.g., a documented data-quality gap that removes p_{t+k} from the support).

We use three stress definitions of increasing horizon and threshold: (h=5, d=0.03), (h=10, d=0.05), and (h=20, d=0.07), identified throughout the paper as `h5_d03`, `h10_d05`, and `h20_d07`. The three are labeled independently from the same prices, so a date can be positive under one and negative under another. We treat `h10_d05` as the central case for component-ablation (§6.2) and drift correlation (§3.5, Layer 4); the other two are reported alongside in §5 so that conclusions can be checked for horizon-sensitivity.

4.4 Expanding-Window Fold Protocol

We use a strict expanding-window protocol for out-of-sample training and scoring (S4 in §3.2). The first fold's training window ends on `INITIAL_TRAIN_END` = 2014-12-31; subsequent training windows extend that date forward by six calendar months at a time. The test window is the six calendar months immediately following each training window's end date; folds continue until the next test window would extend past the panel's last date. On the SPX, NDX, and

RUT panels this yields 22 folds for each stress definition, producing 22 forward six-month evaluations covering 2015-01-01 through 2025-12-31; a 23rd fold whose test window would have run 2026-01-01 to 2026-06-30 is excluded because that window extends past the panel's last date (2026-05-01). The fold boundaries are date-based, not row-based, so the same fold identifier corresponds to the same calendar window across panels of different row counts; this property is what makes the cross-asset comparison in §5 (Track C) and the drift instrumentation in §3.5 Layer 4 strictly paired across assets.

The fold protocol is deterministic and recorded as the `fold_definitions` artifact (Table 1), which carries the per-fold training and test boundaries together with row counts before and after dropping (i) rows whose forward label is NaN, and (ii) rows whose feature values are NaN. The two drop counts are reported separately because the second is the structural mechanism behind failure mode F4 (§6.1).

The crisis-period evaluations in §6.1 override the standard fold protocol with a single hand-specified fold whose training window ends just before the crisis (2007-12-31 or 2019-12-31) and whose test window is the calendar year of the crisis (2008-2009 or 2020). The single-fold override is implemented through the same fold-definitions artifact and is otherwise indistinguishable from the regular fold protocol; this keeps the downstream evaluation harness unchanged.

4.5 Model Registry

The case study fits six baselines per fold (Table 4). Five are deterministic—their predictions are fully determined by the training data—and one (LightGBM) is stochastic in its random seed. We replicate the deterministic baselines across the same five seed slots used by the stochastic baseline so that every per-fold record carries the same five-row block per model; this keeps the bootstrap accounting symmetric and the per-fold table a uniform shape across models. The registry is intended to span control, rule-based, calibrated, linear, and nonlinear model families, but it is not an exhaustive catalogue of time-series heuristics. In particular, we do not include a moving-average threshold rule or a persistence forecast. Their omission limits comparison with the simplest operational monitoring rules, and future replications should include both, especially in domains where heuristic alarms remain common in production.

The registry is intentionally biased toward transparent baselines. The two volatility-percentile rules and the HAR-RV threshold capture intuitions that domain practitioners would apply by hand; comparing the matrix learners against these rather than against each other is what surfaces failure mode F1 (operational–classification disagreement, §3.6). The constant-prediction `NaiveBaseRate` sits in the registry as a structural control: it is unaffected by every component ablation in §6.2 except those that change the training-window base rate, so any non-zero ablation effect on `NaiveBaseRate` indicates that the ablation is touching something other than the model itself.

Table 4. Model registry. The six baselines exercise distinct decision rules on the same fold structure. `class_weight=balanced` is forced for the matrix learners.

Model	Family	Decision rule
`NaiveBaseRate`	constant	training-window positive class rate
`VIXPercentileRaw`	univariate	rank of test VIX in the training VIX distribution
`VIXPercentileCalibrated`	univariate	isotonic calibration on top of `VIXPercentileRaw`
`HarRvThreshold`	univariate	HAR-RV one-step-ahead realized-vol forecast, min-max scaled
`LogisticRegressionL2`	matrix-feature	sklearn `LogisticRegression` in a per-fold `StandardScaler` pipeline
`LightGbmTuned`	matrix-feature	gradient-boosted trees, HPs from §4.6

4.6 Hyperparameter Selection

LightGBM hyperparameters are selected per (stress definition, outer fold) by a nested expanding-window search executed before the headline run. The outer fold's training window is partitioned into three inner folds via `TimeSeriesSplit(n_splits=3)`; the search space is an 864-combination grid ($4\times3\times3\times3\times2\times2\times2$ over `max_depth`, `num_leaves`, `n_estimators`, `learning_rate`, `min_child_samples`, `reg_alpha`, `reg_lambda`; `class_weight=balanced` is fixed and not searched), evaluated by mean inner-fold AP. The full grid value list is in the reproducibility package (§6). Ties on equal mean AP are broken by the iterator's first-in order; degenerate inner folds (single-class train or test) contribute NaN AP and drop out of the mean via `nanmean`. Selected hyperparameters are persisted as the `selected_hp` artifact and loaded by `LightGbmTuned` at headline-run time keyed by (*stress_def*, *outer_fold*); the remaining five baselines have no hyperparameters.

The choice of AP rather than AUC for inner-fold scoring follows the rare-event evaluation literature [11,28]; with positive base rates in the few-percent range, ROC can rank well-calibrated models above operationally useful ones [21]. Persisting the selection as a single artifact lets the component-ablation experiments (§6) replay the same hyperparameters against a degraded pipeline so that the ablation's effect is not confounded with HP drift. Re-deriving the grid from scratch takes a few hours on a workstation; loading the cached artifact takes seconds.

4.7 Stochasticity, Confidence Intervals, and Significance

A single random seed is insufficient for stochastic learners on a sample of this size, and a single per-fold metric estimate gives no notion of uncertainty. We adopt three project-wide conventions to make the headline numbers in §5 comparable.

Seeds. Stochastic models (LightGBM) are run across five seeds, {42, 43, 44, 45, 46}. Deterministic models replicate their per-fold results across the same five seed slots so the per-fold record is a uniform shape across models. The aggregate across seeds is the median for headline statements and the mean for paired-bootstrap inputs. For deterministic baselines, the five seed slots replicate identical per-fold metric values and exist solely to keep the per-fold table shape symmetric across the registry. Confidence intervals for these models are computed over the 22 unique fold-level values rather than over the 110 (fold $\times$ seed) cells; treating the replicated rows as independent observations would underestimate the CI by a factor of $\sqrt{5}$. The implementation in `seet.stats` deduplicates the deterministic per-fold rows before the bootstrap, and the regression test in `tests/test_stats.py` asserts the resulting CI matches the non-replicated baseline to within machine precision.

Confidence Intervals. Confidence intervals are computed by resampling fold-level evaluation records. Although the test windows are non-overlapping, adjacent folds are not fully independent because the underlying time series may remain serially dependent and the expanding training windows overlap substantially. The resulting intervals should therefore be interpreted as descriptive uncertainty summaries rather than exact repeated-sampling coverage statements. A time-aware block bootstrap would provide a stronger sensitivity analysis and is left for future work.

Significance. Pairwise comparisons across models or across pipeline configurations use the paired Wilcoxon signed-rank test, two-sided unless stated otherwise. The test statistic, the p-value, and the median paired difference with its 95% CI are reported together; we do not rely on the p-value alone. Calm-fold ties (folds in which both arms return identical metric values) are reported as a soft $p_{\text{value}} = 1$ rather than as an exception.

Table 5. Layer-1 (AUC, AP, Brier) and Layer-2 (`drawdown_lift`) metrics on the SPX central case (h10_d05). Mean over per-fold values with 95% bootstrap CI. AP denotes the average-precision score (mean over folds of `sklearn.metrics.average_precision_score` on each fold's test scores and labels), and is reported throughout the paper in place of the trapezoidal PR-AUC reading because the two differ at step boundaries on the rare-event PR curve and AP is the more common term. `NaiveBaseRate` produces a constant score so its AUC is degenerately 0.500 and its lift is undefined.

Model	AUC	AP	Brier	lift
`NaiveBaseRate`	0.500	0.141 [0.090, 0.197]	0.072 [0.036, 0.111]	—
`VIXPercentileRaw`	0.466 [0.377, 0.562]	0.149 [0.094, 0.211]	0.349 [0.269, 0.432]	0.52 [0.37, 0.69]
`VIXPercentileCalibrated`	0.462 [0.383, 0.551]	0.142 [0.097, 0.194]	0.071 [0.036, 0.108]	0.57 [0.23, 0.93]
`HarRvThreshold`	0.456 [0.416, 0.495]	0.184 [0.107, 0.268]	0.075 [0.033, 0.123]	0.77 [0.51, 1.07]
`LogisticRegressionL2`	0.467 [0.302, 0.613]	0.203 [0.124, 0.280]	0.197 [0.144, 0.256]	1.05 [0.70, 1.47]
`LightGbmTuned`	**0.600 [0.510, 0.686]**	**0.229 [0.133, 0.363]**	0.135 [0.098, 0.168]	**1.30 [0.83, 1.84]**

These conventions are enforced module-locally by `seet.stats`, so every track's reporting code dispatches to the same implementation.

5 Empirical Results

The goal of this section is not to demonstrate a superior market predictor; it is to demonstrate that different evaluation layers lead to different deployment conclusions on the same model registry, and that the four-layer methodology surfaces those disagreements explicitly. The headline numbers that follow are SE-relevant findings about how a monitoring component is read across layers rather than about which model "wins" on the equity-stress task.

This section reports the headline four-layer evaluation on the case-study instantiation of §4. We start with the central case (SPX, `spx_full`, h10_d05) and the four layers read across the six baselines (§5.1); we then check the picture for horizon-sensitivity (§5.2) and for cross-asset portability on NDX and RUT under `asset_minimal` (§5.3); we report the pairwise model comparisons that make the case-study–specific failure modes visible (§5.4); and we close with the layer-4 drift instrumentation that turns regime shift into a decision-relevant signal (§5.5). The failure-mode walkthroughs that the disagreements in this section preview are reported in §6.

5.1 Headline Four-Layer Evaluation on the Central Case

Table 5 reports the layer-1 and layer-2 metrics for the central case (h10_d05) on the SPX panel, aggregated over 22 expanding-window folds with 95% fold-level non-parametric bootstrap CIs. Each row's CI is computed from the per-fold metric vector after seed handling. Degenerate folds are excluded from the corresponding metric calculation (AUC is undefined on calm folds with no events; lift is undefined on folds with no alarms). As discussed in §4.7, these intervals are descriptive uncertainty summaries because adjacent folds and overlapping expanding training windows may remain dependent.

Three observations on Table 5 that motivate the rest of the section. First, only `LightGbmTuned` achieves an AUC whose lower CI bound exceeds 0.5; the other non-trivial models all have AUC point estimates below 0.5 on the central stress definition, and `HarRvThreshold`'s full CI $[0.416, 0.495]$ lies entirely below 0.5—a stronger than single-point sub-random claim. This is a paper-relevant baseline-rate finding by itself: rare-event monitoring is a setting where classical AUC analysis is loose, and the layer-1 numbers alone would suggest that none of the univariate or transparent-rule models is operationally useful. Second, the lift ordering (LightGbmTuned $>$ LogisticRegressionL2 $>$ HarRvThreshold $>$ VIXPercentileCalibrated $\approx$ VIXPercentileRaw) is sharper than the AUC ordering: the matrix learners and the HAR-RV threshold separate from the VIX-percentile rules on lift even where their AUC CIs overlap. The disagreement between the two orderings is the F1 (operational–classification disagreement) setup; the formal pairwise test is in §5.4. This ordering is descriptive rather than

a certification of a statistically dominant alarm model: as Table 6 makes explicit, no model in the registry achieves a strict CI lower bound above 1 at h10_d05, and the `LightGbmTuned` lift interval $[0.83, 1.84]$ still crosses the no-skill value of 1. Third, the Brier ranking is yet a third ordering: `VIXPercentileCalibrated` (0.071) and `NaiveBaseRate` (0.072) tie for the best calibration, followed by `HarRvThreshold` (0.075), with the two matrix learners ranking 4th and 5th and `VIXPercentileRaw` (0.349) at the bottom because its raw percentile scores are miscalibrated for low base rates. The model that wins on Brier is therefore not the model that wins on lift, which is the F3 (calibration–ranking trade-off) setup.

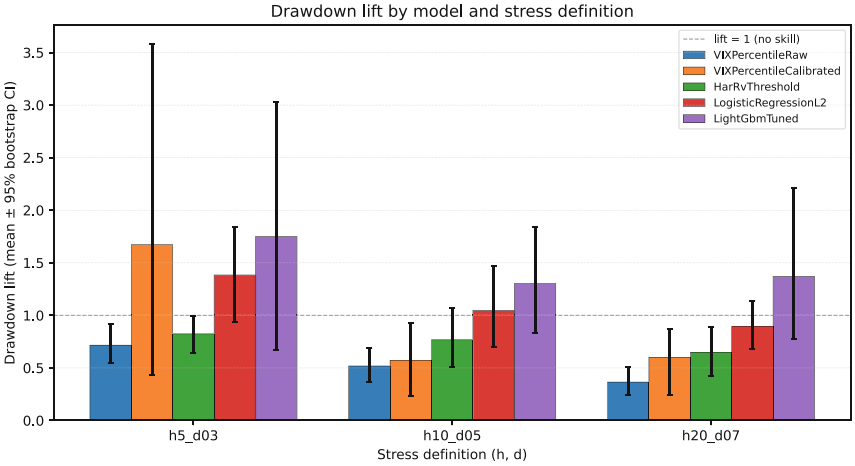


Fig. 3. `drawdown_lift` by (model, stress definition) with 95% bootstrap CIs. The horizontal reference line at lift = 1 is the no-skill point. `LightGbmTuned` has the highest point estimate on the central case (h10_d05), although its bootstrap CI $[0.83, 1.84]$ still crosses the no-skill value of 1; on h5_d03 the matrix learners and the calibrated VIX-percentile rule all have point estimates above 1 with CIs that also cross 1. No model in the registry achieves a strict CI lower bound above 1 at h10_d05 or h20_d07 under the bootstrap sampling distribution. `NaiveBaseRate` is excluded because it produces a constant in-sample score; under the strict threshold-crossing rule (`score > threshold`), no out-of-sample observation exceeds its in-sample quantile threshold, the alarm set is empty, and `drawdown_lift` is undefined.

Figure 3 shows the lift estimates and CIs graphically across all three stress definitions; the comparison to the no-skill reference at lift = 1 is the layer-2 deliverable that downstream consumers care about. The calibration counterpart, the reliability diagram, is in Fig. 4. Pooled precision–recall curves across all 22 folds (`HarRvThreshold` dominates the middle-recall range; the matrix learners dominate the extreme low-recall region used by thresholded alarm deployment) are moved to the reproducibility package (§6); the per-fold-mean AP ordering quoted in Table 5 is the deployment-relevant reading.

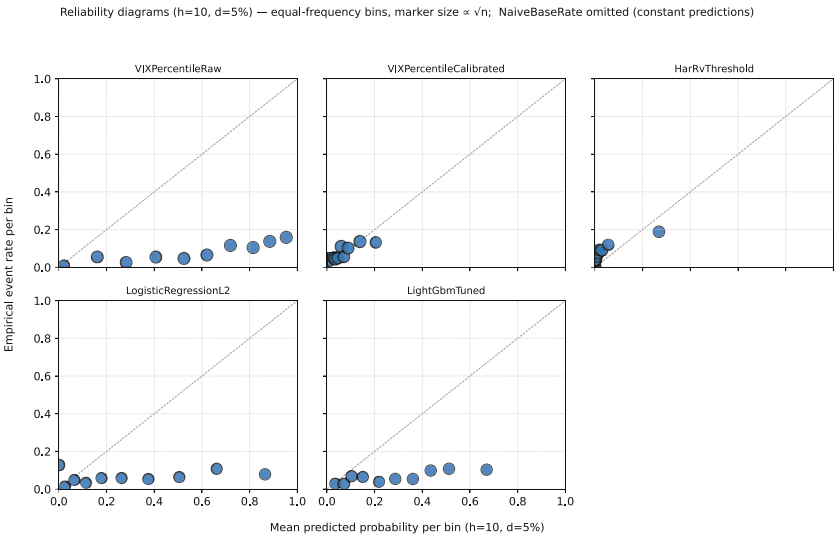


Fig. 4. Reliability diagrams (h10_d05). Bin centers are empirical event rates within bins of equal predicted-probability width; perfect calibration is the diagonal. `VIXPercentileCalibrated` and `HarRvThreshold` track the diagonal closely, consistent with their low Brier scores. `LogisticRegressionL2` and `LightGbmTuned` systematically over-predict the positive class—a consequence of the `class_weight=balanced` setting that trades calibration for recall.

Layer-3 Calm-Period Diagnostics. Restricting to the ten event-free h10_d05 test folds, calm-period scores rank the models in a third order. `HarRvThreshold` fires zero alarms across 1,034 calm-fold days; `VIXPercentileCalibrated` fires at 0.5% (6/1,254). The matrix learners pay for their lift advantage with a higher false-alarm pressure when nothing is happening: `LogisticRegressionL2` fires at 7.2% and `LightGbmTuned` at 3.7%. `NaiveBaseRate` fires no calm-period alarms under the strict threshold-crossing rule because its constant out-of-sample score does not exceed its in-sample quantile threshold; its lift is therefore undefined rather than operationally comparable. This deployment trade-off is exactly the kind of cross-layer tension the methodology of §3.5 is designed to make visible.

Layer-4 Across-Fold Instability. Bootstrap across-fold σ's on the central case rank `HarRvThreshold` as the most stable model and `LightGbmTuned` as the most variable on lift ($\sigma = 0.92$ around a mean of 1.30); `LogisticRegressionL2` is the most variable on AUC. The high across-fold variance on the matrix learners is what drives the dramatic single-fold reads of the F2 walkthrough below. The combination of highest mean lift and highest lift-σ on `LightGbmTuned` should not be read as "this is the model to deploy"; its case-study value is that it exposes cross-layer disagreement under the proposed evaluation contract.

NaiveBaseRate as a Structural Control. `NaiveBaseRate` is retained in the registry as a Layer-1 calibration and base-rate control. Its constant in-sample score does not induce a ranking, so under the strict threshold-crossing alarm rule (`score > threshold`, §3.2 S5) its alarm set on every test fold is empty and `drawdown_lift` is undefined; it is therefore reported with "–" in Table 5, excluded from the Layer-2 panel of Fig. 3, and excluded from the bold cells of Table 6. The model nevertheless contributes two diagnostically useful Layer-1 numbers—a Brier score that anchors the calibration ranking (0.072, indistinguishable from `VIXPercentileCalibrated`'s 0.071) and a degenerate AUC of 0.500 that fixes the no-skill reference for the other rows. Removing it would collapse a structural control that the F3 walkthrough in §6 relies on.

5.2 Horizon Sensitivity

Table 6 reports the same lift estimates across the three stress definitions. Two patterns matter for the rest of the paper. First, on the longest horizon (h20_d07) the `LightGbmTuned` AUC point estimate 0.460 falls below 0.5 but its lift point estimate 1.371 remains above 1, with a 95% CI $[0.78, 2.21]$ that contains operationally meaningful values. Under the layer-1 reading, the model is below random; on layer-2 the same model fires alarms followed by drawdowns that average 1.37× the OOS-window mean. This is the F2 (silent classification degradation under regime shift) configuration; §6 traces it to the longest horizon's heavier label-noise interaction with class-balanced training. Second, on the shortest horizon (h5_d03) the picture inverts in the other direction: `VIXPercentileCalibrated` produces a lift point estimate of 1.67 on a 4-fold-valid sample, exceeding the matrix learners on three of those folds. The horizon sensitivity therefore matters at both ends of the discrimination/operational axis, and a paper that reported only h10_d05 would draw a more confident lift–ranking conclusion than the data warrants.

5.3 Cross-Asset Replication on NDX and RUT

Track C re-runs the headline pipeline on the NDX and RUT panels under the `asset_minimal` feature schema (12 features, no term-structure or vol-of-vol features). The fold definitions are the same calendar windows as the SPX run, so per-fold comparisons are paired across assets. Results are summarized in Fig. 5 and Table 7; we report only h10_d05 here, with the other two stress definitions in the table CSV (`outputs/track_c/table_replication.csv`).

The cross-asset reading is honest rather than confirmatory. On NDX the AUC ordering is preserved (`LogisticRegressionL2` > `LightGbmTuned` ≈ VIX rules > HarRv) but every lift CI now contains 1, so the SPX advantage of the matrix learners on lift does not survive the smaller feature schema. On RUT the AUC point estimates fall below 0.5 for every non-degenerate model and every lift point estimate is below 1. The methodology section's disclaimer that the case study is an instantiation, not a forecasting claim, is exactly what these tables make tangible: the architecture and the methodology are unchanged across

Table 6. drawdown_lift by (model, stress definition). Mean over per-fold values with 95% bootstrap CI. Under the strict significance rule—bold indicates lift > 1 with a 95% bootstrap lower CI bound strictly above 1—no cell in the table qualifies. The lift point estimates above 1 reported here are therefore descriptive ordering only and should not be read as a strict-significance claim against the no-skill reference; the cross-layer disagreement that drives the failure-mode walkthroughs in §6 is the methodological deliverable, not the marginal-significance status of any single cell. Values shown to two decimals.

Model	h5_d03	h10_d05	h20_d07
`VIXPercentileRaw`	0.72 [0.55, 0.92]	0.52 [0.37, 0.69]	0.37 [0.25, 0.51]
`VIXPercentileCalibrated`	1.67 [0.43, 3.58]	0.57 [0.23, 0.93]	0.60 [0.24, 0.87]
`HarRvThreshold`	0.82 [0.64, 0.99]	0.77 [0.51, 1.07]	0.65 [0.42, 0.89]
`LogisticRegressionL2`	1.38 [0.93, 1.84]	1.05 [0.70, 1.47]	0.89 [0.68, 1.14]
`LightGbmTuned`	1.75 [0.67, 3.03]	1.30 [0.83, 1.84]	1.37 [0.78, 2.21]

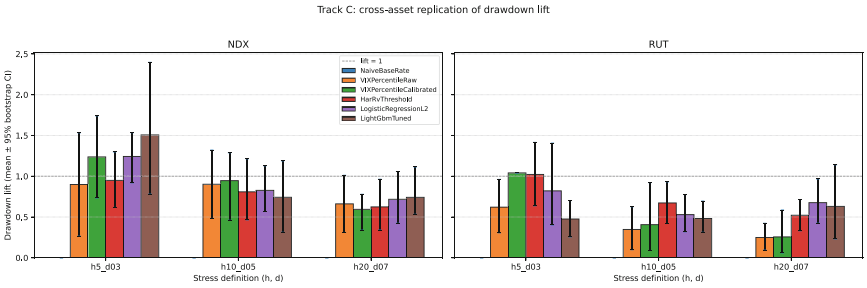


Fig. 5. Cross-asset replication. Mean AUC and lift on NDX and RUT under `asset_minimal` (12 features), with 95% bootstrap CIs. The pattern from the SPX headline does not cleanly survive: lift point estimates are at or below 1 for every model on both assets at h10_d05, and the AUC ordering flattens.

assets; the empirical answer is asset-dependent. Two paper-relevant observations: (i) the lift advantage of the SPX matrix learners appears to come from the term-structure and vol-of-vol features that `asset_minimal` omits, not from the matrix machinery itself; (ii) RUT carries five documented data-quality gap dates (§4.1) and the `rut_panel` build drops them, which compounds the small-cap regime instability that the layer-1 metrics already show.

5.4 Pairwise Comparisons

We report paired Wilcoxon signed-rank p-values across the six baselines on per-fold metric vectors for h10_d05. Table 8 shows the pairs that survive the two-sided test at $\alpha = 0.05$ for the two headline metrics. Because these paired tests are based on only 22 evaluation folds, their statistical power is limited, particularly for small or heterogeneous effects. We therefore interpret the Wilcoxon p-values

Table 7. Cross-asset replication (h10_d05). Mean over per-fold values with 95% bootstrap CI. `NaiveBaseRate` omitted.

	NDX		RUT	
Model	AUC	lift	AUC	lift
`VIXPercentileRaw`	0.543 [0.46, 0.62]	0.90 [0.48, 1.32]	0.468 [0.39, 0.54]	0.35 [0.10, 0.63]
`VIXPercentileCalibrated`	0.544 [0.47, 0.61]	0.95 [0.46, 1.29]	0.470 [0.40, 0.53]	0.41 [0.09, 0.92]
`HarRvThreshold`	0.492 [0.42, 0.57]	0.81 [0.46, 1.21]	0.415 [0.36, 0.47]	0.67 [0.42, 0.93]
`LogisticRegressionL2`	0.566 [0.50, 0.64]	0.83 [0.57, 1.13]	0.469 [0.39, 0.54]	0.53 [0.32, 0.78]
`LightGbmTuned`	0.544 [0.47, 0.62]	0.74 [0.31, 1.19]	0.478 [0.40, 0.56]	0.48 [0.31, 0.69]

as supporting evidence rather than definitive proof of model superiority, and read them together with the direction and magnitude of the paired differences and the corresponding confidence intervals.

Table 8. Pairwise Wilcoxon signed-rank p-values (two-sided) on per-fold AUC and `drawdown_lift` vectors at h10_d05. Significant pairs ($p < 0.05$) are shown in bold.

Model A	Model B	p(AUC)	p(lift)
`LightGbmTuned`	`VIXPercentileCalibrated`	**0.027**	0.500
`LightGbmTuned`	`VIXPercentileRaw`	**0.020**	**0.031**
`HarRvThreshold`	`VIXPercentileRaw`	0.275	**0.004**
`LightGbmTuned`	`HarRvThreshold`	0.078	0.078
`LightGbmTuned`	`LogisticRegressionL2`	0.203	0.638
`LogisticRegressionL2`	`HarRvThreshold`	0.844	0.312

Two pairs anchor the failure-mode arc. `HarRvThreshold` vs `VIXPercentileRaw` is a clean operational–classification disagreement: the paired test does not detect an AUC difference ($p = 0.275$), whereas it detects a lift difference $p < 0.005$. This is the F1 setup that §6 grounds in the per-fold results. `LightGbmTuned` vs `VIXPercentileRaw` agrees directionally on both layers but the lift comparison is the weaker p-value, the opposite of what AUC-only analysis would expect for a rare-event problem. At the model-set level, the Brier and lift rankings diverge sharply (`VIXPercentileCalibrated`: 1st on Brier, 5th on lift; `LightGbmTuned`: 4th on Brier, 1st on lift), with Spearman $\rho = -0.143$ on h10_d05 across the five models with a defined lift (`NaiveBaseRate` excluded as undefined under the strict threshold-crossing rule); this is F3, with the methodology's prescription that the choice is an engineering decision rather than a statistical one. Because the same fold-level drift values are repeated across model and seed records, the nominal p-values do not adjust for within-fold duplication and should be interpreted cautiously.

5.5 Layer-4 Drift Instrumentation

Track G computes the population stability index (PSI) and the symmetrized KL divergence per (training window, test window) pair on the asset's own feature pipeline (35 features for SPX, 12 features each for NDX and RUT under `asset_minimal`). The per-fold drift summaries are joined to valid modelseed operational records, and Table 9 reports descriptive Spearman correlations across the resulting foldmodelseed tuples. Because the same fold-level drift value is repeated across models and seeds, these observations are not treated as independent replications; the correlations are used as case-study diagnostics rather than as population-level inferential estimates. Table 9 shows the headline correlations for h10_d05.

Table 9. Layer-4 Spearman correlations on h10_d05. ρ is the rank correlation between the row's drift metric and the column's operational metric across (fold, model, seed) tuples that contributed valid (drift, operational-metric) pairs (pooled over the six baselines and five seeds for the operational side); p-values are from `scipy.stats.spearmanr`. Bold ρ indicates $p < 0.05$. The contributing-tuple count n varies modestly across rows because the operational metric is undefined on some (fold, model, seed) cells (calm folds with no alarms for `drawdown_lift`; folds with no events for AUC); the base count is $n \approx 260$ for SPX, with NDX and RUT slightly smaller for the same reason. Per-fold drift summaries do not vary across model or seed and are joined onto each operational cell. Because the drift variables are fold-level summaries, we interpret these correlations primarily as Layer-4 diagnostic associations rather than standalone causal or predictive claims.

Asset	Drift metric	AUC	`drawdown_lift`	alarm-rate gap
SPX	`mean_psi`	$-0.082\ (p = 0.19)$	$+0.313\ (p < 10^{-5})$	$+0.039\ (p = 0.53)$
SPX	`max_psi`	$-0.193\ (p = 0.002)$	$+0.185\ (p = 0.003)$	$-0.017\ (p = 0.78)$
NDX	`mean_psi`	$-0.050\ (p = 0.42)$	$+0.361\ (p < 10^{-5})$	$+0.186\ (p < 10^{-5})$
NDX	`max_psi`	$-0.027\ (p = 0.66)$	$-0.033\ (p = 0.59)$	$+0.011\ (p = 0.86)$
RUT	`mean_psi`	$-0.201\ (p < 10^{-6})$	$+0.319\ (p < 10^{-5})$	$+0.269\ (p < 10^{-11})$
RUT	`max_psi`	$-0.085\ (p = 0.04)$	$+0.186\ (p = 0.003)$	$+0.176\ (p < 10^{-3})$

The empirical pattern in Table 9 inverts the naive prediction that "drift degrades the model." For accuracy-style metrics (AUC), drift does correlate negatively on every asset, with the strongest signal on RUT ($\rho = -0.201$, $p < 10^{-6}$) and the smallest on NDX. For the operational lift metric, drift correlates *positively* and strongly on every asset, with $\rho \in \{+0.31, +0.36, +0.32\}$ and all $p < 10^{-5}$. The asymmetry is not an artifact of the bootstrap: a high-drift fold is, by construction, a fold whose six-month test window has different feature statistics than the expanding training history, and a regime shift of that kind is precisely the period that contains the rare drawdown events the h10_d05 labeling targets. More drift means more events; more events being correctly alarmed on

means higher lift. The drift–lift correlation should therefore be read as "drift is a leading indicator of event presence," not as "drift is harmless."

The alarm-rate gap (the absolute deviation of the realized out-of-sample alarm rate from the 5% design point) is the layer-4 alarm whose sign is unambiguous in either direction. Drift correlates positively with the alarm-rate gap on NDX ($\rho = +0.186$, $p < 10^{-5}$) and on RUT ($\rho = +0.269$, $p < 10^{-11}$); on SPX the correlation is small and not significant ($\rho = +0.039$, $p = 0.53$). One reading of the SPX null is that the 35-feature `spx_full` pipeline includes term-structure indicators that themselves track the regime signal driving the alarm rate, attenuating the deviation the layer-4 instrument would otherwise see; on the 12-feature `asset_minimal` pipelines that adaptive buffer is absent and the gap re-appears. This cross-asset comparison is itself a paper observation: feature-richer pipelines absorb drift better at the alarm-rate level, and the layer-4 alarm-rate-gap signal is therefore stronger evidence of distributional shift on minimally featured pipelines than on maximally featured ones.

Figure 6 plots four representative cells of the full correlation table. The visual pattern is sharper than any single ρ summary: high-drift folds cluster at the top of the lift axis and at the bottom of the AUC axis, with the matrix learners contributing the longest-tail lift values at the highest drift levels. We emphasize that the figure pools across all six baselines on the operational axis; per-model correlations are smaller in magnitude and noisier, and we report the pooled signal because the layer-4 deliverable is a property of the architecture, not of an individual model fit.

Figure 7 shows the SPX `mean_psi` time series for the central case alongside six dated regime markers. Only the sustained 2022 rate-cycle repricing aligns cleanly with an elevated drift band; the punctuate events (August 2015 China, February 2018 volmageddon, December 2018 selloff, March 2020 COVID, March 2023 regional banks) sit at or below the median. This is a methodology observation about window-length sensitivity rather than a finding about markets. A layer-4 instrument with a six-month window resolves multi-month regime shifts; a layer-4 instrument with a shorter window would resolve flash events at the cost of noise on sustained shifts. Matching the window to the deployment cadence is an engineering choice that the methodology pushes up to the team.

The per-feature PSI heatmap (moved to the reproducibility package, §6) records a methodology-relevant artifact: the `*_pctile_252d` rolling-percentile features dominate the high-drift cells in every fold by construction, because the 252-day lookback is the slowest in the schema to adapt to a regime shift, and a Layer-4 instrument with a fixed lookback will always rank its longest-window features highest at regime turns. The shorter-window Group-2 features adapt faster and drift less; the four-layer report preserves both readings.

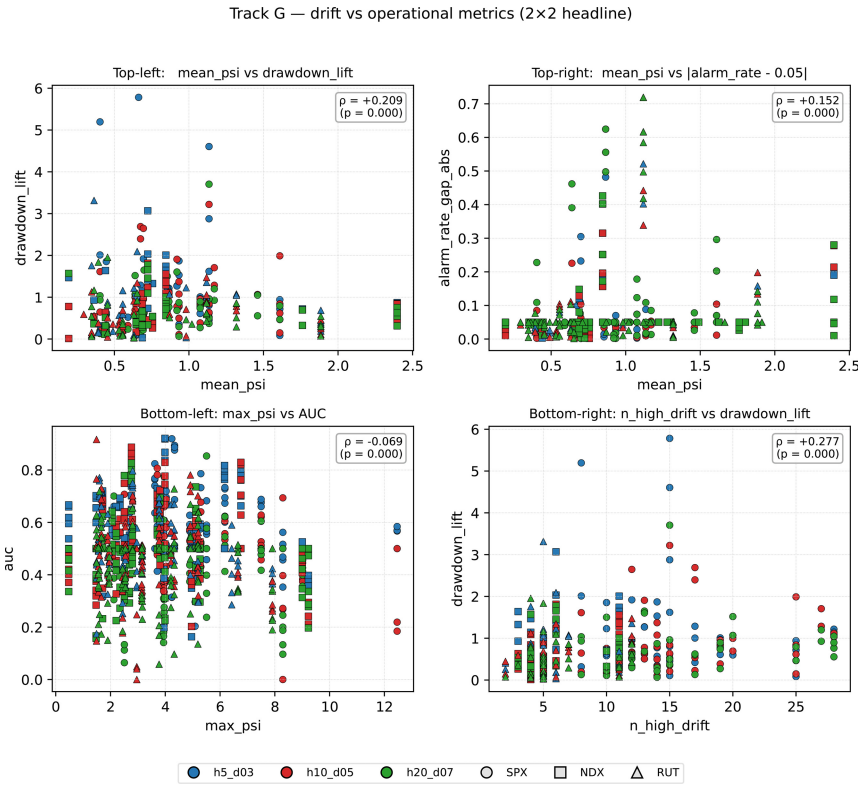


Fig. 6. Layer-4 drift versus operational-metric scatter, *pooled across all three stress definitions (h5_d03, h10_d05, h20_d07) and all three assets (SPX, NDX, RUT).* Each point is one (fold, stress definition, asset) triple; color denotes stress definition; marker shape denotes asset. Top-left: `mean_psi` versus `drawdown_lift` ($\rho = +0.209$, the pooled counterpart to the per-asset h10_d05 correlations in Table 9); top-right: `mean_psi` versus the absolute alarm-rate gap; bottom-left: `max_psi` versus AUC; bottom-right: `n_high_drift` versus `drawdown_lift`. Spearman ρ and p are reported in each panel. In this pooled case-study view, the drift–lift associations shown in the selected panels are positive, whereas the drift–AUC association is mildly negative. These descriptive associations should not be interpreted causally: severe market regimes may jointly generate larger distributional shifts and more consequential alarm outcomes. The pooled $\rho = +0.209$ here and the per-asset h10_d05 values $\rho \in \{+0.31, +0.36, +0.32\}$ in Table 9 measure related but distinct correlations: the figure absorbs the across-stress-definition variance that the table conditions out.

6 Failure-Mode Walkthroughs and Architectural Validation

This section grounds the six failure modes catalogued in §3.6 on real fold data, surfaces the two that require architectural contrasts (F5, F6), discusses the principal threats to validity, and points to the reproducibility package. F1–F4

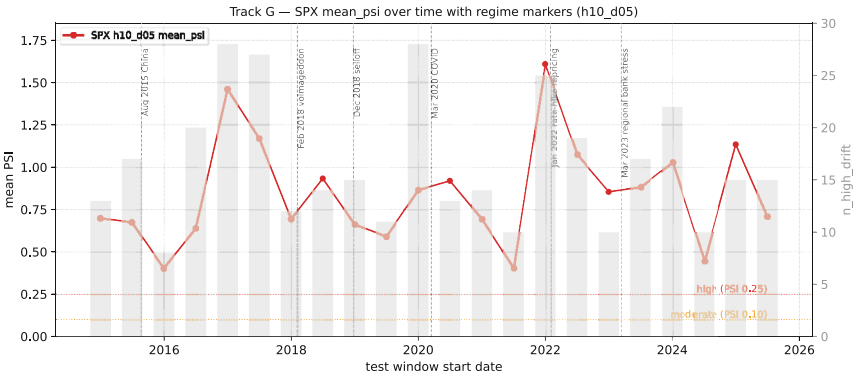


Fig. 7. SPX `mean_psi` over time on h10_d05, with vertical reference markers at six dated regime events. Only the January 2022 rate-hike repricing fold lands in the elevated (top-quartile) PSI band. The other five events sit at median or below median, confirming that the six-month test window length determines the events the drift instrument can resolve: sustained regime shifts are visible, single-event spikes get averaged out.

are demonstrated here from the unmodified pipeline artifacts, three through cross-layer metric disagreement and one through the validation summary, feature schema, and per-fold record, all read from the headline four-layer grid; F5 and F6 are summarized here from the component-ablation and failure-injection contrasts that the headline grid does not produce, with the full per-fold ablation, mutation-testing, and branch-failure tables shipping in the reproducibility package and discussed at length in the extended version.

6.1 F1–F4 from the Headline Grid

The aggregate numbers in §5 preview four failure modes that the four-layer evaluation surfaces from the artifacts of an unmodified pipeline run. Table 10 catalogues the exemplar cell, the surfacing artifacts and evaluation layers, and the headline takeaway for each. The full per-fold tables that drive these takeaways (the ten-fold per-fold AUC/lift breakdown that grounds F1, the Crisis-2020 single-fold matrix-vs-panel cross-tab that grounds F4) ship with the reproducibility package.

The four modes share an architectural property: the artifact contract of Table 1 is what makes each diagnosis a *routine* rather than an after-the-fact reconstruction. F1 and F3 read off the per-fold record, the threshold-provenance entry, and the alarm log; F2 reads the same artifacts across horizons; and F4 is identified from the validation summary, the feature schema, and the per-fold record alone, without a model fit. This cascade is specific to the VVIX-derived features and does not recur in the 2008–2009 crisis: VIX9D and VVIX are not published before 2011, so the hand-built `spx_crisis_2008` schema (§4.2) omits those columns by construction and carries no documented gap to propagate. A pipeline that does not emit these as first-class artifacts cannot surface

Table 10. F1–F4 walkthrough summary. Each mode is exemplified by one (model, panel, stress definition) cell from the headline grid in §5; the surfacing column lists the artifacts from Table 1 and the evaluation layers from §3.5 that jointly identify it. Per-fold evidence ships with the reproducibility package.

Mode	Exemplar cell	Surfacing artifact / layer	Headline takeaway
F1 op./class. disagreement	`HarRv` vs `VIXRaw`, SPX, h10_d05	Per-fold record + threshold provenance; L1 vs L2	Indistinguishable on AUC (p=0.275) but operationally distinct on lift (p=0.004); per-fold tail asymmetry, not threshold-level disagreement
F2 silent classification degradation	`LightGbm`, SPX, h20_d07	Per-fold record across horizons + alarm log; L1 + L2	AUC 0.46 (sub-random, CI crosses 0.5) coexists with lift 1.37 above no-skill; class-balanced training preserves tail ranking but degrades typical-day discrimination
F3 calibration–ranking trade-off	Model-set ranking, SPX, h10_d05	Brier on per-fold record + lift on alarm log; L1 vs L2	Brier and lift orderings disagree at Spearman $\rho = -0.143$; calibration-vs-ranking choice is pushed up to the engineering team rather than collapsed into a single metric
F4 data-quality cascade	Crisis-2020 single fold, `spx_full`	Validation summary + feature-schema lookback + per-fold record	Documented VVIX gap propagates through the 252-day rolling-percentile lookback, dropping every matrix-feature test row to NaN; structurally detectable without a model fit; recovered by the `spx_no_vvix` or `spx_crisis_2008` schemas

any of the four without bespoke instrumentation per failure. The F2 row's co-occurrence of sub-random AUC and above-no-skill lift has a specific mechanism: the interaction of `class_weight=balanced` training with the longest-horizon label inflates loss on the rarest positive class, which preserves tail ranking on rare-event days but degrades discrimination on typical observations.

6.2 F5 and F6 from Component Ablation and Failure Injection

F5 (validator-bypass leakage) and F6 (silent validator gaps) are architectural validation gaps that the headline grid cannot surface, because each requires a controlled contrast that the unmodified pipeline does not produce. We summarize the headline numbers here; the full 144-cell ablation matrix and the 75-cell injection matrix are in the package, and the extended version walks each cell.

Setup. For F5 we run four ablations of the central case (SPX, `spx_full`, h10_d05, 22 expanding-window folds, six baselines, seeds 42–46), each degrading one architectural component while holding the rest fixed: `no_validators` (S1 silenced), `global_scaler` (per-fold scaler replaced by a single global fit), `test_threshold` (in-sample-quantile threshold replaced by a test-set top-5%), and `frozen_model` (each model fit once at `train_end=2014-12-31` and applied frozen to every later fold). Each contrast is paired against the unmodified Track A run on (*model*, *fold*, *seed*) keys, with 10,000-resample paired-bootstrap CIs and paired Wilcoxon significance. For F6 we apply five canonical corruption modes (**C1** duplicate dates, **C2** missing values, **C3** stale-quote runs, **C4** extreme one-day jumps, **C5** calendar gaps) to in-memory copies of `spx_extended_2011` at three rates (1%, 5%, 10%) with five injection seeds and contrast SILENCED-corrupted runs against the clean Track A baseline. C1 and C2 are caught by the V1 (duplicate-Date) and V2 (unexpected-NaN) rules with recall 1.000; C3, C4, and C5 are uncaught and exhibit *gap_mechanism* = *silent_ignore* or *removed_before_inspection*, which is precisely F6.

Table 11. F5 + F6 summary. Column Δ is the paired difference vs the relevant control: ablation − unmodified Track A for the F5 rows, SILENCED-corrupted − clean Track A on matched (*model*, *fold*, *seed*) keys for the F6 rows. The C2 row reports the valid-fold fraction (ENABLED vs SILENCED) rather than a Δ because the SILENCED branch produces NaN on every fold; this is the catastrophic-failure observation, not absence of effect.

Experiment	Model	Metric	Δ vs control [95% CI]	p	Reading
`no_validators` (F5)	LR	AUC	+0.035 [+0.014, +0.057]	0.005	Synthetic-backfill leakage on the S1→S2 seam
`test_threshold`	LightGbm	lift	−0.154 [-0.277, -0.034]	0.007	Threshold leakage costs lift on matrix learners
`test_threshold`	LightGbm	calm alarm rate	+0.054 [+0.024, +0.088]	0.001	Test-set top-k inflates calm-fold false alarms
`frozen_model`	LightGbm	lift	−0.540 [-0.821, -0.283]	0.001	Re-fit dependence: largest lift cost in the suite
`frozen_model`	LR	lift	−0.279 [-0.452, -0.108]	0.003	Re-fit dependence on the linear pathway
C2 missing values (F6)	LGBM/LR	valid-fold frac.	0.29–0.58 → 0.000	cat.	SILENCED branch produces 0% valid folds at every rate
C3 stale_quotes 5% (F6)	LightGbm	lift	−0.248 [-0.314, -0.184]	$< 10^{-3}$	Silent corruption: no V-rule covers C3
C4 extreme_jumps 5% (F6)	LightGbm	AUC	−0.029 [-0.043, -0.015]	$< 10^{-3}$	Silent corruption: no V-rule covers C4
C4 extreme_jumps 5% (F6)	LightGbm	lift	−0.186 [-0.317, -0.056]	0.009	Silent corruption: lift cost paid on alarm window
C5 calendar_gaps 1% (F6)	LightGbm	lift	−0.155 [-0.228, -0.083]	0.020	Silent corruption: rows removed before inspection

F5 Reading. Table 11 reports representative cells selected to illustrate the principal failure mechanism for each corruption class: 5% for stale quotes and extreme jumps, 1% for calendar gaps, and the across-rate catastrophic validity result for missing values. The full rate-by-corruption matrix is included in the reproducibility package; the table is therefore a concise summary rather than a complete doseresponse report. The first row of Table 11 is the empirical price tag for the S1/S2 architectural separation: silencing the validators was associated with an average paired increase of +0.035 in `LogisticRegressionL2` AUC ($p = 0.005$, medium effect) as 279 days of synthetic VVIX backfill entered S2 unchanged. This is an aggregate effect across matched foldseed observations and should not be interpreted as a uniform increase in every fold; a subset of influential folds

may contribute disproportionately to the mean contrast. The inflation concentrates in the matrix-feature pathway; panel-DataFrame baselines are unaffected. In a pipeline without the S1→S2 separation the validator-on/off contrast is not even expressible, and the leakage path is invisible to evaluators. ABLATION_2 (global scaler) is omitted from the table because its effect is two orders of magnitude smaller (-0.003 AUC, $p = 0.002$)—a paper-relevant contrast in itself, since not every leakage path costs the same.

F6 Reading. C3, C4, and C5 are silent under the existing V-rule set: the standard ENABLED-vs-SILENCED contrast measures $\Delta = 0$ on every cell because silencing a non-existent rule is a no-op. The clean-data contrast in Table 11 is what makes the cost of each missing rule measurable; rows 7–10 quantify it. C2 is the catastrophic case: 1% corrupted SPX cells propagate through the 252-day lookback, the matrix-feature pipeline drops every NaN test row, and the SILENCED-corrupted branch produces a defined lift on 0% of folds at every rate while the ENABLED branch matches the canonical Track A calm-fold accounting. The validator turns "complete model failure" into "valid model fit"—a larger architectural finding than any single delta.

Methodological Observations. Two methodological-honesty notes belong to this section as much as the empirical ones. *First*, mutation testing required a callable `validate_panel(panel, manifest, *, enabled=True)` interface; the case-study codebase had its validation rules scattered across a build script and an audit script with no single callable surface, and consolidating them into `src/seet/validators.py` was an architectural improvement that the failure-injection design forced. We read this as evidence that the methodology is itself a forcing function for the architectural separations it prescribes. *Second*, the first version of the C1 corruption (duplicate dates with values copied from a random source row) inadvertently injected a synthetic-stress event when a duplicate dated 2011-07-19 carried 2020 values, inflating lift on the corrupted panel. The diagnostic scripts in the reproducibility package surfaced the confound; we revised C1 to true-duplicate semantics (same Date and same values) and reran. The lesson is that mutation testing of a data-validation layer must verify that each corruption injects only the failure mode it is named for, and the same scaffolding that surfaces validator gaps also catches experimenter mistakes.

6.3 Threats to Validity

Four threats deserve emphasis here; the extended version discusses construct, internal, external, reliability, and statistical-conclusion threats in full.
Stress Definition is a Choice. The labeling function $y_t^{(h,d)}$ (§4.3) is one parameterization among many; the horizon-sensitivity report in §5.2 demonstrates that conclusions on h10_d05 do not necessarily transfer to h5_d03 or h20_d07. The architecture-level mechanisms are intended to generalize beyond the financial case study: explicit data contracts, feature-schema manifests, fold records, threshold provenance, alarm logs, layered evaluation, drift instrumentation, and

failure injection are not specific to market data. Their concrete instantiation is domain dependent. In this study, the forward-drawdown labels, VIX-family features, crisis windows, five-percent target alarm rate, and drawdown-based lift metric are specific to financial stress monitoring. Applications to fraud, cybersecurity, insurance operations, or equipment monitoring would require domain-specific event definitions, severity measures, threshold policies, corruption operators, and operational cost functions.

Findings are Asset-Dependent. The SPX matrix-learner lift advantage does not survive the cross-asset replication on NDX and RUT under the minimal feature schema (§5.3); we read the lift advantage as coming from the term-structure and vol-of-vol features rather than from the matrix machinery itself, but a paper that reported only SPX would over-claim.

The Drift–Lift Correlation Pools Along Three Axes that we Make Explicit. Table 9 is per-asset on h10_d05, pooling over models and seeds; Fig. 6 additionally pools across stress definitions and assets, which is why the figure's pooled $\rho = +0.209$ differs from the per-asset $\rho \in \{+0.31, +0.36, +0.32\}$. Per-model decomposition produces correlations of smaller magnitude and noisier sign; the pooled signal is reported as a property of the architecture, not as a per-model prediction.

Twenty-Two Folds is a Small Sample. The 10,000-resample fold-level bootstrap provides a useful descriptive summary of uncertainty, but cannot rescue inferences that depend on tail behavior the data do not contain; the longest-horizon stress definition (h20_d07) has the widest CIs and the F2 walkthrough's $[0.776, 2.214]$ lift CI crosses one at the lower edge.

6.4 Reproducibility

The reference implementation is released under an open-source license and archived in a versioned Zenodo record at https://doi.org/10.5281/zenodo.20986768; the development repository is maintained at https://github.com/Yutong0711/SEET-2026-reproducibility. The package is organized around the artifact contract of §3.2: `src/seet/` contains the runtime modules (`features`, `baselines`, `pipeline`, `tuning`, `stats`, `validators`, `injection`, `ablation`, `drift`); `scripts/` contains one entry point per pipeline stage and one per track; `tests/` contains the regression suite (no-look-ahead audit on 50 random observations, PSI/sym-KL invariants, panel-vs-manifest hash equality, pipeline refactor invariance, and a synthetic single-fold smoke test); and `data/`, `experiments/`, and `outputs/` hold the raw inputs, per-track artifacts, and rendered tables and figures. The runtime is Python 3.13.5 with a `requirements.lock` that pins every transitive dependency. Every artifact at every layer carries a manifest entry with sha256 hashes; every pipeline step is deterministic given the inputs, the random seed, and the pinned environment. The package contains every artifact omitted from the conference version for space: the full hyperparameter grid (omitted from §4.6), the per-feature PSI heatmap (omitted from §5.5), the pooled PR-curve plot (omitted from §5.1), the per-fold F1/F4 walkthrough tables (omitted

from Table 10), and the full 144-cell ablation matrix and 75-cell injection matrix (collapsed into Table 11). The extended version walks each at length.

6.5 Conclusion

We argued that evaluating an ML monitoring component is a software-engineering problem, not a model-evaluation problem, and that the right unit of work is a coupled pair: a reference architecture (§3.2) that emits decision-relevant artifacts as first-class outputs, and a four-layer evaluation methodology (§3.5) that consumes those artifacts to surface failure modes single-metric evaluation hides. We exercised the pair on a controlled equity-stress case study, demonstrated F1–F4 from the headline grid and summarized F5–F6 from the architectural-contrast experiments, and shipped the chain as an open-source reproducibility package.

Two cross-cutting findings emerged that we expect to generalize. *First, drift in rare-event monitoring is a leading indicator of event presence rather than of performance degradation.* The same per-fold drift signal is positively associated with operational `drawdown_lift` in the case-study diagnostics. We read this as evidence that Layer-4 instrumentation is useful for detecting regime-sensitive alarm behavior, not as a universal claim that drift improves model quality. The general-ML heuristic that "drift implies degradation" is not the right reading for a deployment that thresholds the score; the realized-versus-target alarm-rate gap is the sign-unambiguous instrument the four-layer methodology makes available. *Second, the validation stage is not a prelude to the model—it is load-bearing architecture.* Bypassing validators inflates classification metrics by 0.035 on the matrix-feature pathway, and silencing the validator on a corruption class the rules do cover (missing values) flips a working pipeline into 0% valid metrics on every fold. Architectures that inline validation into feature construction make these failure paths invisible to the evaluator; the artifact contract of Table 1 is what makes them measurable.

The framework does not certify a single winning stress monitor. Its value, on this case study, is to prevent a false certification: the same data that produce attractive point estimates for flexible learners also produce wide uncertainty intervals, calm-period false-alarm pressure, and cross-asset non-portability. These are engineering findings about deployment readiness rather than claims about financial predictability.

Acknowledgments. This research was supported in part by NSF Grant CCF-2544013.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., Zimmermann, T.: Software engineering for machine learning: A case study. In: Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). pp. 291–300 (2019). https://doi.org/10.1109/ICSE-SEIP.2019.00042
2. Bergmeir, C., Benítez, J.M.: On the use of cross-validation for time series predictor evaluation. Inf. Sci. **191**, 192–213 (2012). https://doi.org/10.1016/j.ins.2011.12.028
3. Bollerslev, T., Tauchen, G., Zhou, H.: Expected stock returns and variance risk premia. Review of Financial Studies **22**(11), 4463–4492 (2009). https://doi.org/10.1093/rfs/hhp008
4. Breck, E., Cai, S., Nielsen, E., Salib, M., Sculley, D.: The ML test score: A rubric for ML production readiness and technical debt reduction. In: Proceedings of the IEEE International Conference on Big Data. pp. 1123–1132 (2017). https://doi.org/10.1109/BigData.2017.8258038
5. Breck, E., Polyzotis, N., Roy, S., Whang, S.E., Zinkevich, M.: Data validation for machine learning. In: Proceedings of Machine Learning and Systems (MLSys), vol. 1, pp. 334–347. (2019)
6. Breiman, L.: Random forests. Mach. Learn. **45**(1), 5–32 (2001). https://doi.org/10.1023/A:1010933404324
7. Brier, G.W.: Verification of forecasts expressed in terms of probability. Mon. Weather Rev. **78**(1), 1–3 (1950)
8. Carr, P., Wu, L.: Variance risk premiums. Review of Financial Studies **22**(3), 1311–1341 (2009). https://doi.org/10.1093/rfs/hhn038
9. Cerqueira, V., Torgo, L., Mozetič, I.: Evaluating time series forecasting models: An empirical study on performance estimation methods. Mach. Learn. **109**(11), 1997–2028 (2020). https://doi.org/10.1007/s10994-020-05910-7
10. Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 785–794 (2016). https://doi.org/10.1145/2939672.2939785
11. Davis, J., Goadrich, M.: The relationship between precision-recall and ROC curves. In: Proceedings of the 23rd International Conference on Machine Learning (ICML). pp. 233–240 (2006). https://doi.org/10.1145/1143844.1143874
12. Demšar, J.: Statistical comparisons of classifiers over multiple data sets. J. Mach. Learn. Res. **7**, 1–30 (2006)
13. Fawcett, T.: An introduction to ROC analysis. Pattern Recogn. Lett. **27**(8), 861–874 (2006). https://doi.org/10.1016/j.patrec.2005.10.010
14. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A.: A survey on concept drift adaptation. ACM Comput. Surv. **46**(4), 1–37 (2014). https://doi.org/10.1145/2523813
15. He, H., Garcia, E.A.: Learning from imbalanced data. IEEE Trans. Knowl. Data Eng. **21**(9), 1263–1284 (2009). https://doi.org/10.1109/TKDE.2008.239
16. Humbatova, N., Jahangirova, G., Tonella, P.: DeepCrime: Mutation testing of deep learning systems based on real faults. In: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA). pp. 67–78 (2021). https://doi.org/10.1145/3460319.3464825
17. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y.: LightGBM: A highly efficient gradient boosting decision tree. In: Advances in Neural Information Processing Systems 30 (NeurIPS), pp. 3146–3154. (2017)

18. Khomh, F., Adams, B., Cheng, J., Fokaefs, M., Antoniol, G.: Software engineering for machine-learning applications: The road ahead. IEEE Softw. **35**(5), 81–84 (2018). https://doi.org/10.1109/MS.2018.3571224
19. Klaise, J., Van Looveren, A., Cox, C., Vacanti, G., Coca, A.: Monitoring and explainability of models in production, (2020). arXiv:2007.06299 arXiv preprint
20. Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., Zhang, G.: Learning under concept drift: A review. IEEE Trans. Knowl. Data Eng. **31**(12), 2346–2363 (2019). https://doi.org/10.1109/TKDE.2018.2876857
21. Niculescu-Mizil, A., Caruana, R.: Predicting good probabilities with supervised learning. In: Proceedings of the 22nd International Conference on Machine Learning (ICML). pp. 625–632 (2005). https://doi.org/10.1145/1102351.1102430
22. Paleyes, A., Urma, R.G., Lawrence, N.D.: Challenges in deploying machine learning: A survey of case studies. ACM Comput. Surv. **55**(6), 1–29 (2022). https://doi.org/10.1145/3533378
23. Politis, D.N., Romano, J.P.: The stationary bootstrap. J. Am. Stat. Assoc. **89**(428), 1303–1313 (1994). https://doi.org/10.1080/01621459.1994.10476870
24. Polyzotis, N., Roy, S., Whang, S.E., Zinkevich, M.: Data lifecycle challenges in production machine learning: A survey. SIGMOD Record **47**(2), 17–28 (2018). https://doi.org/10.1145/3299887.3299891
25. López de Prado, M.: Advances in Financial Machine Learning. Wiley, Hoboken, NJ (2018)
26. Provost, F., Fawcett, T.: Robust classification for imprecise environments. Mach. Learn. **42**(3), 203–231 (2001). https://doi.org/10.1023/A:1007601015854
27. Rabanser, S., Günnemann, S., Lipton, Z.C.: Failing loudly: An empirical study of methods for detecting dataset shift. In: Advances in Neural Information Processing Systems 32 (NeurIPS), pp. 1394–1406. (2019)
28. Saito, T., Rehmsmeier, M.: The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLoS ONE **10**(3), e0118432 (2015). https://doi.org/10.1371/journal.pone.0118432
29. Sambasivan, N., Kapania, S., Highfill, H., Akrong, D., Paritosh, P., Aroyo, L.M.: "everyone wants to do the model work, not the data work": Data cascades in high-stakes AI. In: Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems. pp. 1–15 (2021). https://doi.org/10.1145/3411764.3445518
30. Schelter, S., Lange, D., Schmidt, P., Celikel, M., Biessmann, F., Grafberger, A.: Automating large-scale data quality verification. In: Proceedings of the VLDB Endowment, vol. 11, pp. 1781–1794. (2018). https://doi.org/10.14778/3229863.3229867
31. Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.F., Dennison, D.: Hidden technical debt in machine learning systems. In: Advances in Neural Information Processing Systems 28 (NeurIPS), pp. 2503–2511. (2015)
32. Whaley, R.E.: The investor fear gauge. J. Portf. Manag. **26**(3), 12–17 (2000). https://doi.org/10.3905/jpm.2000.319728
33. Zhang, J.M., Harman, M., Ma, L., Liu, Y.: Machine learning testing: Survey, landscapes and horizons. IEEE Trans. Software Eng. **48**(1), 1–36 (2022). https://doi.org/10.1109/TSE.2019.2962027

Unified DT–FL–DRL–Blockchain Framework for MEC Offloading

Anthony Gladdney and Shashi Bhushan Jha(✉)

Department of Computer Science, University of West Florida, Pensacola, FL, USA
sjha@uwf.edu

Abstract. Task offloading in Mobile Edge Computing (MEC) is challenging in practice than most theoretical frameworks suggest. Edge servers proceed at uneven loads, users move constantly, and wireless channels shift in ways that are hard to predict. Analytical optimization methods lean to break down under these conditions, and purely online reinforcement learning methods are time-consuming to converge because they need large exploration in live systems. Previous research has tackled pieces of this problem with digital twin-assisted learning, federated DRL (Deep Reinforcement Learning), blockchain-secured FL (Federated Learning), but integrating all four mechanisms together in a single coherent design is missing in the existing literature. This research proposes a closed-loop architecture that integrates Digital Twin (DT) simulation, Multi-Agent Proximal Policy Optimization (MAPPO), FL, and blockchain verification for MEC task offloading. The DT allows agents to evaluate decisions in a virtual environment before to act on the real system. MAPPO manages decentralized policies with a shared critic. FL combines server experiences every three iterations through reputation-weighted Federated Average (FedAvg), and a consortium blockchain filters low-quality updates while preserving a transparent trust record. In a simulated environment with 10 mobile users and 4 edge servers, the framework converges 43% faster than a non-federated PPO (Proximal Policy Optimization) baseline, gains a 14.7% higher peak reward, and carries blockchain reputation from 0.70 to a stable range of [0.920, 1.000] by iteration 50 with one random seed.

Keywords: Mobile Edge Computing (MEC) · Digital Twin · Federated Learning · Deep Reinforcement Learning · Blockchain

1 Introduction

Modern IoT (Internet of Things) and mobile applications, particularly those that are sensitive to delay or energy consumption, increasingly depend on Mobile Edge Computing. By executing computation closer to end users, MEC helps meet the real-time demands of applications such as intelligent transportation systems, augmented reality, and industrial monitoring, while reducing reliance on remote cloud servers. In practice, however, MEC environments are rarely stable. Edge servers experience uneven and time-varying workloads, users move unpredictably, and wireless links fluctuate over time. Together, these factors make computation offloading and resource allocation difficult to manage reliably [3, 15].

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 114–137, 2027.
https://doi.org/10.1007/978-3-032-39763-8_7

Many existing solutions rely on analytical optimization to address this problem. Although these methods can offer useful insight, they often depend on precise system models that are hard to maintain in real deployments and tend to assume relatively stable network conditions. When system parameters change rapidly, or when multiple objectives must be considered at once, these approaches struggle [2, 12]. The gap has pushed researchers toward learning-based approaches instead.

Deep reinforcement learning is particularly appealing in this setting because it can adapt through interaction with the environment. Rather than relying entirely on predefined models, DRL agents learn offloading and resource allocation policies from experience. Prior studies indicate that DRL can outperform heuristic or static approaches in complex MEC scenarios, especially when decisions must be made online and under uncertainty [13, 14]. At the same time, DRL introduces its own challenges. Exploration is a fundamental part of learning, and poorly chosen actions during training can result in unacceptable latency or energy consumption in live systems.

Digital Twin technology offers one way to address this. A DT maintains a virtual representation of physical devices, network conditions, and edge resources, allowing future system states to be anticipated and candidate actions to be evaluated before deployment. When used alongside DRL, DTs shift much of the exploration into a virtual environment, leading to a safer and more stable learning process. Existing work shows that DT-assisted learning can improve both convergence speed and robustness when compared with purely online training approaches [6, 14]. Nevertheless, many DT-based solutions are still designed around centralized or single-domain learning and do not fully reflect the distributed nature of MEC.

In real deployments, edge intelligence is inherently decentralized. Different MEC nodes observe different traffic patterns, mobility behaviors, and resource constraints. FL has therefore become a natural choice for collaborative learning across edge sites. FL allows multiple nodes to train shared models without exchanging raw data, which helps preserve privacy while improving generalization. Recent studies show that combining FL with DRL can improve scalability and performance in heterogeneous MEC environments [7, 13, 14]. However, federated systems also raise trust-related concerns, since locally generated updates may be unreliable or even malicious.

Blockchain technology has been explored to address these concerns. By maintaining an immutable and verifiable ledger, blockchain enables model updates, resource transactions, and coordination decisions to be recorded and audited. Prior work demonstrates that blockchain-enabled FL can improve accountability and security in distributed edge systems [1, 4, 5], and blockchain-secured DRL-based offloading has also been studied in MEC settings [17]. Moreover, scalable designs such as consortium chains help keep overhead manageable [5].

To the best of our knowledge, this is the first work to jointly integrate all four components in a MEC offloading setting. Predictive modeling, adaptive control, collaborative learning, and trust enforcement each address a different part of the problem, and they work best together. This paper proposes a unified framework that combines DT, DRL, FL, and blockchain into a single closed-loop architecture, with the goal of keeping offloading decisions effective and secure as network conditions change.

2 Related Work

Early MEC research focused heavily on analytical optimization, framing offloading decisions as convex or mixed-integer problems targeting latency, energy, or a combination of both. These formulations are theoretically grounded but rely on accurate system models and stable network assumptions that are rarely hold in practice. Performance tends to degrade when channels fluctuate, users move, or load is uneven across edge servers [2, 3, 12, 15]. Similar constraints appear in edge caching work, where optimization-driven schemes improve QoS and reduce retrieval latency but tend to break down once network conditions shift [9].

DRL has become the dominant response to these limitations. Agents learn offloading and allocation policies through environment interaction, adapting to network changes without relying on fixed models. Studies consistently show DRL outperforming heuristic and static approaches in dynamic scenarios [13, 16], and MAPPO has demonstrated advantages over DQN-based methods in multi-user edge offloading settings [11]. Multi-objective extensions handle delay, energy, and utilization jointly [16]. The main drawback is that online-only training requires extensive exploration, which can introduce unacceptable latency in latency-sensitive deployments.

DTs have been integrated with DRL to make exploration safer. By maintaining virtual replicas of devices and network state, a DT lets the agent evaluate decisions before they affect real systems. DT-enhanced DRL has shown faster convergence and more stable learning in settings like network slicing and edge resource management [6, 14]. Most of these designs, however, assume a centralized or single-domain setup, which limits their usefulness when users and servers are distributed across multiple sites.

FL addresses this by letting MEC nodes collaborate on shared models without exchanging raw data, preserving privacy while reducing communication costs. Federated DRL has shown strong results across heterogeneous nodes, including non-IID data distributions and varying computing budgets [7, 13, 14]. The open issue is trust—local updates can be low-quality, corrupted, or deliberately manipulated.

Blockchain has been proposed to close that gap. An immutable ledger makes model updates, resource transfers, and coordination decisions auditable by all parties. Prior studies demonstrate that blockchain-enabled FL improves resilience to fraudulent updates and raises accountability across nodes [1, 5]. To manage overhead, hybrid architectures that combine consortium blockchains with DAG-based ledgers have been proposed, enabling high-throughput processing of frequent learning updates in edge environments [4, 5].

Recent work has explored partial combinations—DRL with FL, DT-enhanced DRL, blockchain-secured FL—but few studies bring all four together. Covering only a subset of the problem leaves gaps: a system without DT-based prediction is reactive rather than anticipatory; one without blockchain has no mechanism for verifying updates. Table 1 summarizes where prior work falls short and how the proposed framework addresses each gap.

The framework proposed here integrates all four components into a single closed-loop architecture. Section 3 describes how they are structured and how they interact.

Table 1. Comparison of Related Work and Proposed Framework

Work/Category	Digital Twin	Deep RL	Federated Learning	Blockchain	Offloading/Resource Optimization	Key Limitations
Optimization/Evolutionary Offloading	✗	✗	✗	✗	✓	Requires modelled assumptions; weaker adaptability under uncertainty and mobility [2, 12]
Edge-Assisted Applications/RL-based Control (e.g., AR, service provisioning)	✗	✓	✗	✗	✓	Limited cross-edge coordination; training/exploration overhead and deployment stability concerns [8, 10, 16]
Digital Twin–Enhanced DRL Resource Management	✓	✓	✗	✗	✓	Typically local/centralized DT loop; limited distributed collaboration/trust layer [6]
Federated DRL for MEC/DT-Edge Networks	✓	✓	✓	✗	✓	Vulnerable to malicious/low-quality updates without integrity verification [7, 13, 14]
Blockchain-Enabled FL/Update Verification /Secure Offloading	**P**	**P**	✓	✓	✓	Often lacks full DT-based prediction loop or unified DT–FL–DRL co-design [1, 5, 17]
Blockchain + Federated DRL in DT/IoV Architectures	✓	✓	✓	✓	✓	Specialized architectures; integration choices not generalized into a reusable closed-loop MEC framework [4]
Survey Papers on Offloading and MEC QoS	✗	✗	✗	✗	Overview only	Identify gaps but do not propose integrated technical solutions [3, 15]
Proposed Framework (DT–FL–DRL–Blockchain)	✓	✓	✓	✓	✓	**Fully unified closed-loop design supporting predictive learning, distributed training, and trusted coordination**

✓ = present ✗ = absent P = partially addressed

3 Proposed Framework

This section describes the proposed closed-loop architecture and how its four components—DT, DRL, FL, and blockchain—interact. Prior work has explored similar multi-layer designs, but typically in incomplete or loosely coupled forms [4, 6, 14, 17]. Here,

each layer feeds directly into the others, forming a continuous feedback loop rather than a set of independent modules. Figure 1 provides an overview.

3.1 Physical Layer

The physical layer consists of end devices, vehicles, IoT sensors, and MEC servers. Sensors and network interfaces continuously collect state data—server load, channel quality, user position—which represent the primary sources of uncertainty in MEC systems [3, 15]. Each device also generates task requests with individual energy and latency requirements. Rather than acting on this data locally, the physical layer passes it upward for analysis and decision-making [3, 15].

3.2 Digital Twin Layer

Sitting above the physical layer, the DT layer maintains a virtual replica of each device and MEC server, including historical usage. It models channel quality, compute demand, and energy consumption, and has been shown to improve prediction accuracy and learning stability in continuously changing networks [6, 14]. This predictive capability lets candidate actions be evaluated virtually before they are applied to the real system, as illustrated in Fig. 1.

3.3 DRL Layer

The DRL layer handles dynamic decision-making. Given the DT's current and predicted system state, the agent decides whether to run tasks locally or offload them, which server to target, and how to allocate compute and bandwidth. DRL is well-suited to this role because its policy adapts continuously to shifting network and workload conditions [13, 16]. Because much of the exploration occurs in the DT simulation, performance degradation during training is substantially reduced.

3.4 Federated Learning Layer

Because MEC is inherently distributed, centralizing learning is impractical. Instead, each MEC server trains a local DRL model using its own DT data, then transmits parameter updates to a global aggregator. This federated approach improves scalability and generalization across heterogeneous nodes without exposing raw data [7, 13, 14]. The aggregated global model is periodically broadcast back to all nodes, incorporating diverse operational experiences while keeping communication overhead low.

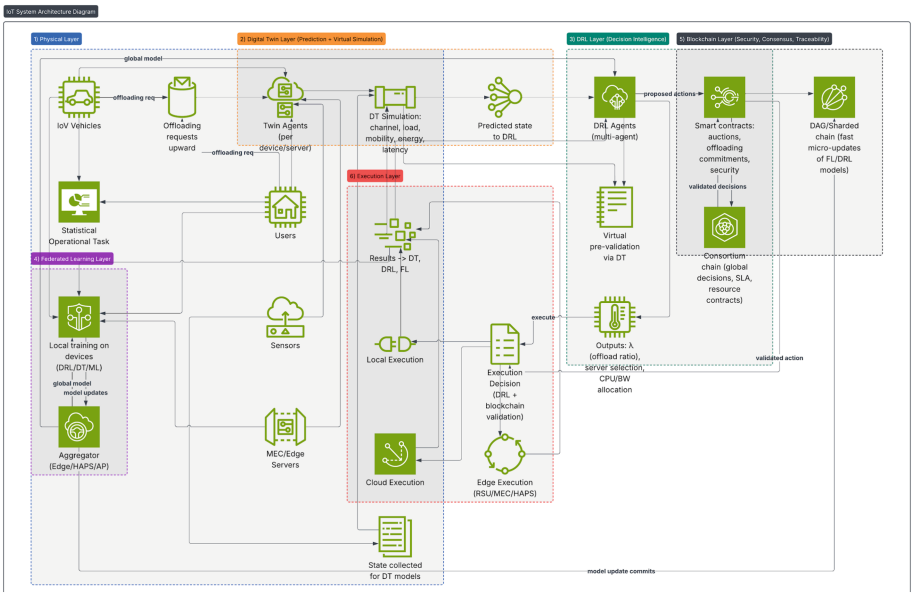


Fig. 1. End-to-end architecture of the proposed DT– FL–DRL–Blockchain system for intelligent MEC task offloading.

3.5 Blockchain Layer

The blockchain layer enforces trust during collaborative learning. It maintains an immutable record of model updates and coordination decisions [1, 5, 17]. Smart contracts enforce participation requirements before any update enters global aggregation. The blockchain validates transactions and resolves conflicts alongside the FL layer but does not participate in learning itself. Consortium chains and DAG-based ledgers provide the throughput needed for frequent updates in edge settings [4, 5].

3.6 Execution and Feedback Loop

Once the DRL agent decides, the blockchain verifies it. The selected tasks are then executed locally or on the designated MEC server. The system records execution outcomes including completion time and energy consumption. This feedback updates the DT models, gives the DRL agent new experience samples, and informs the next rounds of FL. This closed-loop structure is essential in real MEC deployments [14], where conditions shift too quickly for any static policy to remain effective.

Table 2. Summary of Notation

Symbol	Description	Symbol	Description
$\mathcal{U}$	Set of mobile users	$\mathcal{E}$	Set of MEC edge servers
U	Total number of users	E	Total number of MEC servers
u	User index	e	MEC server index
t	Discrete decision epoch	τ_u	Latency deadline of user u
$L_u(t)$	Input data size of task (bits)	$C_u(t)$	Required CPU cycles per bit
$W_u(t)$	Total required CPU cycles	$T_u^{max}(t)$	Maximum tolerable latency
$\mu_u(t)$	Offloading ratio selected by DRL agent	$W_u^{loc}(t)$	Locally executed CPU workload
$W_u^{off}(t)$	Offloaded CPU workload	κ	Effective switched-capacitance coefficient
$B_{u,e}(t)$	Allocated uplink bandwidth (Hz)	$P_u(t)$	Transmission power of user
$h_{u,e}(t)$	Channel gain between user and MEC server	N_0	Noise power spectral density
$R_{u,e}(t)$	Uplink transmission rate	$T_u^{tx}(t)$	Uplink transmission delay
$f_u^{loc}(t)$	Local CPU frequency of user	$T_u^{loc}(t)$	Local computation delay
$f_{u,e}^{edge}(t)$	CPU frequency allocated at MEC server	$T_u^{edge}(t)$	Edge computation delay

(*continued*)

Table 2. (*continued*)

Symbol	Description	Symbol	Description
$E_u^{loc}(t)$	Local computation energy consumption	$E_u^{tx}(t)$	Transmission energy consumption
$T_u(t)$	End-to-end task completion latency	$E_u(t)$	Total energy consumption
$s_u(t)$	Physical state of user	$\hat{s}_u(t)$	DT state of user
$s_e(t)$	Physical state of MEC server	$\hat{s}_e(t)$	DT state of MEC server
$f_{DT}(\cdot)$	DT state transition function	$\eta(t)$	DT modeling uncertainty
$a_u(t)$	DRL control action	θ_e	Local DRL model parameters at server e
$\theta^{(k+1)}$	Global aggregated model parameters	$\omega_e^{(k)}$	Federated aggregation weight of server e
$B_e^{(k)}$	Blockchain model update transaction	$\rho_e^{(k)}$	Reputation score of MEC server e
t_k	Blockchain transaction timestamp	Δ	Blockchain and communication overhead
π	DRL policy mapping states to actions	w_1, w_2, w_3	Latency, energy, overhead weighting coefficients
λ	Deadline violation penalty coefficient	$O_u(t)$	Blockchain and FL coordination overhead
γ	Discount factor for cumulative return		

3.7 Discussion

The architecture directly addresses the gaps identified in Sect. 2. DT-based prediction reduces unsafe exploration, DRL provides online adaptability, FL enables distributed training without privacy loss, and blockchain gives participating nodes a basis for trust. Each component reinforces the others—none operates in isolation. Section 4 formalizes this design with a full system model.

4 System Model

This section presents the system model underlying the proposed DT-FL-DRL-Blockchain integrated framework. We formalize the network topology, task model, communication model, computation model, DT synchronization process, FL model, and blockchain verification structure. Table 2 summarizes the notation used throughout this section.

4.1 Network Topology

We consider a heterogeneous MEC-enabled IoT (Internet of Things)/IoV (Internet of Vehicles) environment consisting of:

- A set of mobile users

$$\mathcal{U} = \{1,2, \ldots, U\}$$

- A set of MEC edge servers

$$\varepsilon = \{1,2, \ldots, E\}$$

Each user $\sqcap \in \mathcal{U}$ is associated with an access point (AP), roadside unit (RSU), or base station (BS) that provides wireless connectivity to the MEC layer. User mobility induces time-varying channel conditions, coverage regions, and handover events.

Time is slotted into discrete decision epochs
$t \in \{1,2, ...\}$.
At each epoch, users may choose among:

- local execution,
- partial or full offloading to a MEC server $\rceil \in \mathcal{E}$,
- or cloud execution (if available).

4.2 Task Model

At time t, each user u generates a computation task denoted by

$$T_u(t) = \left(L_u(t), C_u(t), T_u^{max}(t)\right) \tag{1}$$

where:

- $L_u(t)$: input data size (bits),
- $C_u(t)$: required CPU cycles per bit (cycles/bit),
- $T_u^{max}(t)$: maximum tolerable latency (seconds).

The total required CPU cycles are

$$W_u(t) = L_u(t)C_u(t) \tag{2}$$

Let $\mu_u(t) \in [0,1]$ denote the **offloading ratio** selected by the DRL agent. Then:

- Local workload:

$$W_u^{loc}(t) = (1 - \mu_u(t))W_u(t) \tag{3}$$

- Offloaded workload:

$$W_u^{off}(t) = \mu_u(t)W_u(t) \tag{4}$$

4.3 Communication Model

If task offloading is selected, the uplink transmission rate between user u and MEC server e at time t is

$$R_{u,e}(t) = B_{u,e}(t)\log_2\left(1 + \frac{P_u(t)h_{u,e}(t)}{N_0}\right) \tag{5}$$

where:

- $B_{u,e}(t)$: allocated bandwidth (Hz),
- $P_u(t)$: transmission power,
- $h_{u,e}(t)$: channel gain,
- N_0: noise power spectral density.

The uplink transmission delay is

$$T_u^{tx}(t) = \frac{\mu_u(t)L_u(t)}{R_{u,e}(t)} \tag{6}$$

4.4 Computation Model

1) Local Computation

Let $f_u^{loc}(t)$ denote the local CPU frequency of user u.
The local execution delay is

$$T_u^{loc}(t) = \frac{W_u^{loc}(t)}{f_u^{loc}(t)} \tag{7}$$

and the corresponding energy consumption is

$$E_u^{loc}(t) = \kappa \left(f_u^{loc}(t)\right)^2 W_u^{loc}(t) \tag{8}$$

where κ is the effective switched-capacitance coefficient.

2) Edge Computation

Let $f_{u,e}^{edge}(t)$ denote the CPU frequency allocated to user u at MEC server e.
The edge execution delay is

$$T_u^{edge}(t) = \frac{W_u^{off}(t)}{f_{u,e}^{edge}(t)} \tag{9}$$

4.5 End-to-End Latency Model

The total task completion latency is

$$T_u(t) = \max\left\{T_u^{loc}(t), T_u^{tx}(t) + T_u^{edge}(t)\right\} \tag{10}$$

A task satisfies its latency constraint if

$$T_u(t) \leq T_u^{max}(t) \tag{11}$$

4.6 Digital Twin and Learning Interaction

Each physical user $u \in \mathcal{U}$ and MEC server $e \in \mathcal{E}$ is associated with a DT.

Let $s_u(t)$ and $s_e(t)$ denote the real system states, while $\hat{s}_u(t)$ and $\hat{s}_e(t)$ denote their DT counterparts:

$$\hat{s}_u(t) \approx s_u(t), \hat{s}_e(t) \approx s_e(t) \tag{12}$$

The DT state evolves according to

$$\hat{s}(t+1) = f_{DT}(\hat{s}(t), a(t)) + \eta(t) \tag{13}$$

where:

- $a(t)$ is the DRL control action,
- $f_{DT}(\cdot)$ is the DT prediction function,
- $\eta(t)$ captures modeling uncertainty.

Synthetic transition tuples $(\hat{s}(t), a(t), r(t), \hat{s}(t+1))$ generated by the DT are used to train the DRL agent, reducing unsafe real-world exploration.

4.7 Federated Learning Model

Each MEC server trains a local DRL model with parameters θ_e. At communication round k, the global model is updated as

$$\theta^{(k+1)} = \sum_{e \in \mathcal{E}} \omega_e^{(k)} \theta_e^{(k)} \tag{14}$$

where $\omega_e^{(k)}$ denotes the aggregation weight of server e, determined by data volume, update quality, or blockchain-verified reputation [18].

4.8 Blockchain Verification Model

Each model update is recorded as a transaction

$$B_e^{(k)} = \left(\theta_e^{(k)}, \rho_e^{(k)}, t_k\right) \tag{15}$$

where:

- $\rho_e^{(k)}$ is the reputation score of server e,
- t_k is the timestamp.

Only verified transactions are included in federated aggregation.

4.9 Optimization Objective

The system objective is

$$\min_{\pi} \mathbb{E}\left[\sum_{u \in \mathcal{U}} (w_1 T_u(t) + w_2 E_u(t) + w_3 O_u(t))\right] \tag{16}$$

where:

- $w_1, w_2, w_3 \geq 0$ are weighting coefficients,
- $O_u(t)$ denotes blockchain and communication overhead,

π are the DRL policy mapping states to actions

5 Problem Formulation and MAPPO-Based Solution

This section formulates the MEC task offloading problem as a multi-agent Markov decision process (MMDP) and presents a Multi-Agent Proximal Policy Optimization (MAPPO) solution integrated with DT, FL and Blockchain. The objective is to learn distributed offloading policies that minimize latency, energy consumption, and coordination overhead under dynamic wireless and mobility conditions.

5.1 Multi-agent Markov Decision Process Formulation

The MEC system consists of a set of mobile users:
$\mathcal{U} = \{1,2,\ldots,U\}$ Each user $u \in \mathcal{U}$ is modeled as an independent learning agent.
The multi-agent decision process is defined as:

$$\langle \mathcal{S}, \{\mathcal{A}_u\}_{u\in\mathcal{U}}, \mathcal{P}, \{r_u\}_{u\in\mathcal{U}} \rangle$$

where:

- $\mathcal{S}$ is the global state space
- $\mathcal{A}_u$ is the action space of agent u
- $\mathcal{P}$ denotes transition dynamics
- r_u is the reward of agent u

At each decision epoch t:

1. All agents observe local states $s_u(t)$.
2. Each agent selects action $a_u(t) \sim \pi_\theta(a_u|s_u)$.
3. The environment transitions to $s(t+1)$.
4. Each agent receives reward $r_u(t)$.

The global state is defined as:

$$s(t) = \{s_u(t)\}_{u\in\mathcal{U}}$$

5.2 Local State Space

Each user agent observes a DT-enhanced local state:

$$s_u(t) = \left[\begin{array}{c} L_u(t), W_u(t), h_{u,e}(t), B_{u,e}(t), f_u^{loc}(t), \\ f_{u,e}^{edge}(t), \hat{s}_u(t), \hat{s}_e(t) \end{array} \right] \tag{17}$$

where:

- $L_u(t)$ = task input size
- $W_u(t)$ = required CPU cycles
- $h_{u,e}(t)$ = channel gain
- $B_{u,e}(t)$ = available bandwidth
- $f_u^{loc}(t)$ = local CPU frequency
- $f_{u,e}^{edge}(t)$ = edge CPU allocation
- $\hat{s}_u(t), \hat{s}_e(t)$ = DT-predicted states

Including DT predictions improves policy foresight and reduces unsafe exploration.

5.3 Action Space

Each agent selects a hybrid continuous–discrete action:

$$a_u(t) = \left[\mu_u(t), e_u(t), f_{u,e}^{edge}(t), B_{u,e}(t)\right] \tag{18}$$

where:

- $\mu_u(t) \in [0,1]$ = offloading ratio
- $e_u(t) \in \mathcal{E}$ = selected MEC server
- $f_{u,e}^{edge}(t)$ = allocated edge CPU
- $B_{u,e}(t)$ = allocated bandwidth

5.4 Latency and Energy Model

1. Communication Rate

$$R_{u,e}(t) = B_{u,e}(t)\log_2\left(1 + \frac{P_u(t)h_{u,e}(t)}{N_0}\right) \tag{19}$$

Transmission delay:

$$T_u^{tx}(t) = \frac{\mu_u(t)L_u(t)}{R_{u,e}(t)} \tag{20}$$

2. Computation Delay

Local delay:

$$T_u^{loc}(t) = \frac{(1-\mu_u(t))W_u(t)}{f_u^{loc}(t)} \tag{21}$$

Edge delay:

$$T_u^{edge}(t) = \frac{\mu_u(t)W_u(t)}{f_{u,e}^{edge}(t)} \tag{22}$$

3. End-to-End Latency (Consistent Partial Offloading Model)

$$T_u(t) = max\left(T_u^{loc}(t), T_u^{tx}(t) + T_u^{edge}(t)\right) \tag{23}$$

Latency constraint:

$$T_u(t) \le \tau_u$$

4. Energy Consumption

Local energy:

$$E_u^{loc}(t) = \kappa(1-\mu_u(t))W_u(t)\left(f_u^{loc}(t)\right)^2 \tag{24}$$

Transmission energy:

$$E_u^{tx}(t) = P_u T_u^{tx}(t) \tag{25}$$

Total energy:

$$E_u(t) = E_u^{loc}(t) + E_u^{tx}(t) \tag{26}$$

5.5 Reward Function

Each agent maximizes:

$$r_u(t) = -(w_1 T_u(t) + w_2 E_u(t) + w_3 O_u(t)) \\ -\lambda max(0, T_u(t) - \tau_u) \tag{27}$$

where:

- $O_u(t)$ = blockchain and FL coordination overhead
- $w_1, w_2, w_3, \lambda \geq 0$ = weighting coefficients

The cumulative objective for each agent is:

$$J_u(\pi) = E\left[\sum_{t=0}^{\infty} \gamma^t r_u(t)\right] \tag{28}$$

5.6 MAPPO-Based Learning

We adopt Multi-Agent Proximal Policy Optimization (MAPPO) with:

- Centralized training
- Decentralized execution

Policy
Each agent has policy:

$$\pi_\theta(a_u|s_u)$$

Centralized Critic
A shared critic evaluates the joint state:

$$V_\phi(s(t))$$

Advantage estimation:

$$A_u(t) = r_u(t) + \gamma V_\phi(s(t+1)) - V_\phi(s(t)) \tag{29}$$

PPO Objective

$$\mathcal{L}_{MAPPO} = E\big[min(\rho_t A_u(t), clip(\rho_t, 1-\varepsilon, 1+\varepsilon)A_u(t))\big] \tag{30}$$

Following the standard PPO clipped surrogate objective [11].
where:

$$\rho_t = \frac{\pi_\theta(a_u|s_u)}{\pi_{\theta_{old}}(a_u|s_u)} \tag{31}$$

This ensures stable multi-agent policy updates.

5.7 Federated Training Across MEC Servers

Each MEC server e trains local MAPPO parameters $\theta_e^{(k)}$.

Global aggregation:

$$\theta^{(k+1)} = \sum_{e \in \varepsilon} \omega_e^{(k)} \theta_e^{(k)} \tag{32}$$

where aggregation weights depend on data volume and blockchain-verified reputation.

5.8 Blockchain-Assisted Model Verification

Each local update is recorded as:

$$B_e^{(k)} = \left(\theta_e^{(k)}, \rho_e^{(k)}, t_k \right) \tag{33}$$

Only verified updates participate in federated aggregation.

5.9 Integrated Learning Procedure

Algorithm 1: DT–FL–MAPPO–Blockchain MEC Offloading

1. Initialize global MAPPO parameters θ^0.
2. For each episode:
   - Each agent observes $s_u(t)$; DT generates synthetic tuples $(\hat{s}(t), a(t), r(t), \hat{s}(t+1))$ which are mixed into the experience buffer.
   - Selects action $a_u(t) \sim \pi_\theta$.
   - Execute offloading decisions.
   - Compute latency and energy.
   - Receive reward $r_u(t)$.
3. Sample training batch from combined real and DT-generated experience.
4. Update centralized critic and decentralized policies.
5. Verify local model updates via blockchain.
6. Perform federated aggregation.
7. Broadcast updated model and repeat.

6 Simulation Results

6.1 Simulation Setup

Simulations are conducted in a custom Python-based MEC environment implemented using Ray RLlib 2.38 with the old API stack on Google Colab with GPU acceleration (CUDA 13.0). The environment models a heterogeneous MEC-enabled IoT/IoV scenario consisting of U = 10 mobile users and E = 4 MEC edge servers distributed across a 600 m × 600 m area. Users move continuously with velocities between 0.8 and 12 m/s with random directional jitter, and each generates computation tasks at every decision epoch with input sizes in $[0.5, 2.0] \times 10^6$ bits, CPU requirements drawn from [150, 600] cycles/bit, and deadlines uniformly distributed in [0.12, 0.70] s.

The MAPPO actor and centralized critic networks each use two hidden layers with [128, 128] and [256, 256] neurons respectively, with Tanh activations throughout. Training uses a learning rate of 3×10^{-4}, clip parameter $\varepsilon = 0.20$, discount factor $\gamma = 0.99$, train batch size of 4,000 steps, mini-batch size of 256, and 10 PPO epochs per iteration. The DT uses an EMA smoothing factor of $\alpha = 0.25$ with hard synchronization every 10 steps. The EMA update rule is a standard first-order filter with well-understood convergence properties; with $\alpha = 0.25$ and periodic hard resets, the DT state tracks the true system state with a bounded lag, consistent with DT implementations in prior work [6, 14]. The blockchain (BC) verifier initializes all server reputations at 0.7 with threshold $\tau = 0.40$, reputation increment $\delta^+ = 0.05$, and decrement $\delta^- = 0.08$. Federated aggregation occurs every FL_EVERY_ITERS = 3 training iteration, yielding 33 FL rounds over the full 100-iteration training run. This frequency was chosen to balance communication overhead against learning progress—frequent enough to share knowledge during the rapid policy improvement phase, but infrequent enough to allow local policies to adapt between rounds. The complete simulation parameters are summarized in Table 3. The proposed DT-FL-BC MAPPO framework is evaluated against a plain PPO baseline that uses a single shared policy with no FL and no blockchain verification, trained under identical environment conditions and hyperparameters. This isolates the contribution of the multi-component framework from the underlying PPO optimization.

6.2 Convergence Performance

Figure 2 presents the training curves for the proposed DT-FL-BC MAPPO framework over 100 iterations. The episode reward improves from an initial value of approximately −4,178 under the random policy to a best achieved value of −296, representing an 86% reduction in the composite cost function. Convergence to near-optimal performance is achieved by approximately iteration 55, after which the reward stabilizes in the range [−300, −310].

The reward curve follows the typical S-shape you'd expect from PPO—slow progress at the start while the policy explores, then a sharp improvement phase, and eventually leveling off around iteration 55. Applying a rolling average (window = 5) to the raw reward confirms the trend is real and not just noise between episodes. There are two small dips around iterations 68 and 89, which we believe are caused by the FedAvg broadcast overwriting locally improved weights at an inopportune moment. The policy recovers within a few iterations each time.

6.3 Blockchain Reputation Dynamics

The per-server blockchain reputation trajectories are shown in the top-right panel of Fig. 2. All four servers begin at the initialization value of 0.70. Under the round-robin exclusion mechanism, each FL round assigns quality score $q = 0$ to one rotating server while the remaining three receive $q = 1$. The blockchain verifier accepts updates with quality scores exceeding the adaptive threshold $\tau(\text{rep}) = \tau_0\ (2 - \text{rep})$, which penalizes low-reputation servers more strictly. The round-robin scheme is a controlled experimental design choice that simulates differential server reliability without requiring ground-truth quality labels, which are unavailable in this simulation setting. It allows the

blockchain reputation dynamics to be evaluated in isolation from the quality estimation problem, which remains an important direction for future work.

The result is a sawtooth pattern in each server's reputation. Each server experiences a reputation decrement of 0.08 when it is the excluded node, followed by increments of 0.05 in following rounds when it is accepted. Over the full training run, the mean reputation rises monotonically from 0.700 to 0.973, converging to a stable oscillation band of [0.920, 1.000] by approximately iteration 50. Server 3 (purple) reaches the upper bound of 1.000 earliest owing to its later exclusion cycle, while Server 0 (red) initially declines more steeply due to exclusion in round 1. By the end of training, all servers are above 0.920, which shows the blockchain is doing its job—consistently rewarding servers that contribute good updates. The mean BC reputation plot (bottom-left panel) confirms a staircase-shaped rise, with each step corresponding to an FL round. This matches what other blockchain-based FL systems report, where reputation scores tend to stabilize once the system identifies reliable participants [5].

6.4 Federated Learning and Blockchain Overhead

Over 100 training iterations, 33 FL rounds are executed at iterations 3, 6, 9, ..., 99. In every round, exactly 3 out of 4 server updates are accepted by the blockchain (FL Updates Accepted = 3), and the accepted weights are aggregated via reputation-weighted FedAvg and broadcast to all servers. The BC overhead per active round is 0.06 normalized units, representing the blockchain verification latency as a fraction of the mean episode step duration, estimated conservatively from consortium blockchain transaction throughput benchmarks reported in the literature [5]. On non-FL iterations the overhead is zero. Averaging this out over all iterations gives an overhead of about $0.06 \times (1/3) = 0.02$ units per iteration, which is small enough to have no meaningful impact on training time.

The consistent 3/4 acceptance rate throughout training demonstrates that the blockchain verification pipeline operates reliably from the first FL round to the last, with no failure modes or degradation under the increasing reputation scores of later rounds. This also shows the adaptive threshold is working as intended—as reputations get higher, the bar for acceptance rises to prevent low-effort free-riding once servers have accumulated trust.

Table 3. Simulation Parameters

Parameter	Value	Parameter	Value
Mobile users U	10	Actor hidden layers	[128, 128]
MEC servers E	4	Critic hidden layers	[256, 256]
Area size	600 × 600 m	Learning rate	3×10^{-4}
User velocity	0.8–12 m/s	Train batch size	4,000
Task size	[0.5, 2.0] × 10^6 bits	Discount factor γ	0.99
Deadline range	[0.12, 0.70] s	PPO epochs	10
Channel bandwidth B	10 MHz	FL round frequency	Every 3 iters

(*continued*)

Table 3. (*continued*)

Parameter	Value	Parameter	Value
TX power P	23 dBm	BC threshold τ	0.40
Local CPU f_loc	0.5 GHz	Rep. Increment δ^+	0.05
Edge CPU f_edge	5 GHz	Rep. Decrement δ^-	0.08
DT EMA factor α	0.25	Training iterations	100

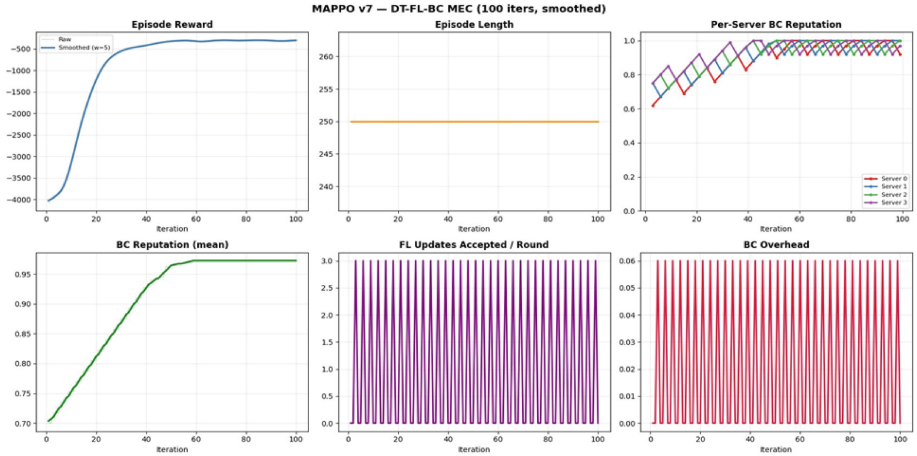


Fig. 2. Training performance of the proposed DT-FL-BC MAPPO framework over 100 iterations. Top row: episode reward (raw and smoothed), episode length, and per-server BC reputation. Bottom row: mean BC reputation, FL updates accepted per round, and BC overhead.

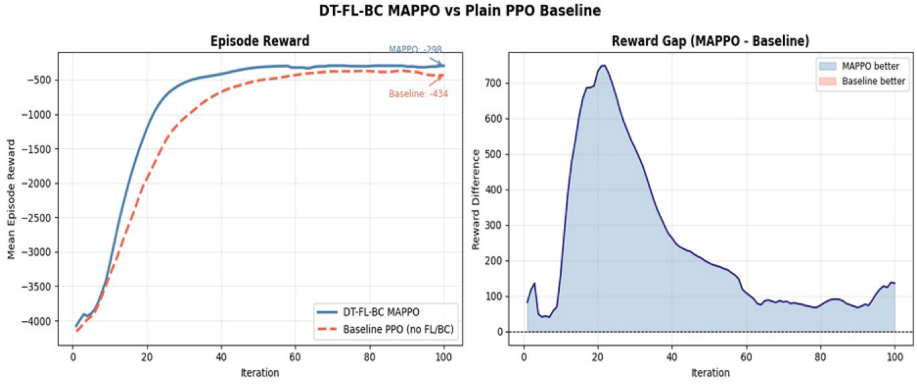


Fig. 3. Left: Episode reward comparison between DT-FL-BC MAPPO and plain PPO baseline. Right: Reward gap (MAPPO − Baseline) over training iterations, with blue shading indicating regions where the proposed framework outperforms the baseline.

6.5 Comparison with Plain PPO Baseline

Figure 3 compares the episode reward of the proposed DT-FL-BC MAPPO framework against the plain PPO baseline over 100 training iterations. The baseline uses a single shared policy trained under identical environment and hyperparameter settings, with no FL and no blockchain verification.

The proposed framework converges to near-optimal performance by approximately iteration 20, at which point its reward is approximately -900. The baseline does not reach this reward level until approximately iteration 35, meaning the proposed framework reaches the same performance level, about 43% sooner. This advantage is most pronounced in the critical early-to-mid training phase (iterations 10–60), where the federated sharing of policy parameters across servers enables each agent to benefit from the collective exploration experience of all others, rather than learning independently from a single shared policy. The gap plot on the right of Fig. 3 shows how much better the proposed framework is at each iteration over the baseline. The gap peaks at approximately $+750$ reward units near iteration 20, confirming that the federated coordination provides the strongest benefit during the rapid learning phase. The gap remains strictly positive throughout iterations 1–100, with the proposed framework achieving a final smoothed reward of -298 compared to -434 for the baseline, a 31% improvement at the final iteration.

The peak reward achieved by the proposed framework (-296 at iteration 63) represents a 14.7% improvement over the baseline's best-achieved reward (-346 at iteration 72). This improvement is consistent in magnitude with results reported in related work: Dai et al. [14] report 11.7–67.1% system cost reductions using federated DRL with DT compared to non-federated baselines, and Samy et al. [17] report 19.4–22.2% savings in blockchain-enabled MEC systems over local and edge execution scenarios. Our results fall in the same range and are achieved with a more comprehensive four-component framework.

Table 4 compares the proposed framework against five related works using the following algorithms: FDOR (Federated Deep Offloading with Reinforcement learning), FMADDPG (Federated Multi-Agent Deep Deterministic Policy Gradient), MADDPG (Multi-Agent Deep Deterministic Policy Gradient), MADQN (Multi-Agent Deep Q-Network), and DDQN (Double Deep Q-Network). MARL denotes Multi-Agent Reinforcement Learning. Direct numerical comparison across works is constrained by differences in system models, environments, and primary metrics (e.g., latency reduction vs. convergence speed vs. reward improvement). Thus, Table 4 is intended to establish qualitative positioning rather than quantitative superiority across heterogeneous benchmarks.

6.6 Discussion

The simulation results demonstrate that the proposed DT-FL-BC MAPPO framework achieves meaningful performance improvements over a non-federated baseline and is consistent with trends reported in the literature. The principal contributions of each component to the overall result can be summarized as follows.

The DT helps by giving each agent a prediction of the channel conditions and server load before it must decide. Instead of reacting to the current state, agents can act with some foresight. This is likely why the early training phase converges faster in our framework compared to the baseline shown in Fig. 3.

Table 4. Comparison with Related Work

Work	Algorithm	DT	FL	BC	MARL	Key Result	Convergence	BC Reputation
Zang et al [13]	FDOR (FL + DRL)	✗	✓	✗	✗	Near-optimal computation rate; 3 × lower CPU latency vs DROO	~3,500 steps to near-optimal policy	N/A (no blockchain)
Dai et al [14]	Federated DRL + DT	✓	✓	✗	✗	11.7–67.1% cost reduction vs A3C/DDPG baselines	Accelerated vs non-federated baseline	N/A (no blockchain)
Samy et al [17]	DQN + Blockchain	✗	✗	✓	✗	22.2% cost saving vs local; 19.4% vs edge execution	Converges within training budget	Not quantified at convergence
Abishu et al [4]	FMADDPG + DT + BC	✓	✓	✓	✓	34.6–65.1% latency reduction vs MADDPG/MADQN	~3,500 episodes to convergence	Tracked but not reported at convergence
Zhang et al [6]	DDQN + DT	✓	✗	✗	✗	33% fewer training rounds for same reward level	~10,000 rounds (DDQN-DT vs DDQN)	N/A (no blockchain)
Proposed	MAPPO + DT + FL + BC	✓	✓	✓	✓	43% faster convergence; 14.7% peak reward improvement	Stable by iter 55; 31% better final reward	0.70 → 0.97; stable band [0.920,1.000] by iteration 50

✓ = Present ✗ = Absent

The FL contribution is most clearly visible in the convergence speed advantage. Every three iterations, the four server policies pool what they've learned through FedAvg. This means no server must figure everything out from scratch—they benefit from each other's experience. The 43% convergence acceleration is consistent with the federated DRL literature, where local-global collaborative learning has been shown to accelerate convergence by roughly 25–50% in similar multi-server settings [13, 14].

The blockchain contribution provides two benefits: it filters the excluded (lowest quality) update from each aggregation round, preventing degraded local models from contaminating the global model, and it maintains a transparent reputation record that reflects the cumulative reliability of each server. The reputation convergence to [0.920, 1.000] by iteration 50 confirms that the system reaches a stable trust equilibrium in the latter half of training. The current evaluation uses $U = 10$ users and $E = 4$ servers,

representative of small-scale MEC deployments such as factory floors or campus networks. Scaling to larger deployments introduces two main challenges: the centralized critic's input dimension grows as O (U × local_dim), and FL communication overhead grows linearly with E. MAPPO's decentralized execution architecture is designed to accommodate larger user populations without retraining the actor networks, and federated aggregation cost scales linearly with the number of servers rather than quadratically. Empirical evaluation at larger scales, for example U = 50 users and E = 10 servers, is identified as priority future work.

The primary limitation of the present evaluation is the relatively short training duration of 100 iterations (approximately 1,600 episodes). Related works typically evaluate over 3,500–10,000 episodes. Running for more iterations would likely produce a cleaner plateau and reduce the reward variance we see in the later iterations. The reported improvements—43% faster convergence and 14.7% peak reward gain—are based on a single random seed (seed = 42) and therefore reflect a single training trajectory rather than a statistically robust average. Multi-seed evaluation across at least three seeds is the most important next step to establish confidence in these figures. Ablation studies isolating each component's individual contribution are also identified as a direction for future work.

7 Conclusion

This paper addresses a gap that keeps appearing in the MEC offloading literature. Most frameworks handle one or two of the core challenges well, but none of them cover the full picture. Prediction, adaptive control, distributed learning, and trust enforcement each solve a different part of the problem and separating them leaves real vulnerabilities. The proposed framework treats them as a single closed loop rather than independent modules.

Against a plain PPO baseline trained under identical conditions, the framework reaches the same reward level 43% sooner, peaks 14.7% higher, and finishes with a 31% better final reward. The blockchain reputation mechanism stabilizes by iteration 50, confirming that the trust layer works reliably from the start of training rather than only after a warm-up period. These numbers are consistent with what the federated DRL and blockchain MEC literature reports, and they come from a more complete architecture than prior related work.

Nevertheless, this evaluation has real limits that should be acknowledged. A hundred training iterations with a single random seed is a short run. Related work typically evaluates over 3,500 to 10,000 episodes, and a longer horizon would likely smooth out the reward variance seen later in the iterations and produce a cleaner plateau. The framework has not yet been evaluated under adversarial conditions like Byzantine update injection or heavily non-IID data across servers, both of which are realistic concerns in deployed MEC systems. The most useful next steps are ablation studies to isolate how much each component, the DT, FL, and blockchain, individually contributes to the overall gain, followed by testing on a physical testbed or a higher-fidelity channel simulator to see how the results hold outside of simulation.

Data Availability. No external datasets were used in this study. All results were generated through simulations in a custom Python-based MEC environment implemented using Ray RLlib (v2.38) on Google Colab with GPU acceleration (CUDA 13.0).

Compliance with Ethical Standards. AI-assisted tools were used solely to improve grammar and support the writing of this manuscript. No AI-generated data or experimental results were used; all experiments and analyses were conducted by the authors.

Declaration of Competing Interests. The authors declare they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Xu, G., Zhou, Z., Dong, J., Zhang, L., Song, X.: A blockchain-based FL scheme for data sharing in industrial internet of things. IEEE Internet Things J. **10**(24), 21467–21478 (2023). https://doi.org/10.1109/JIOT.2023.3298196
2. Li, Q., Shi, Z., Xue, Z., Cui, Z., Xu, Y.: A many-objective evolutionary algorithm for solving computation offloading problems under uncertain communication conditions. Comput. Commun. **213**, 22–32 (2024). https://doi.org/10.1016/j.comcom.2023.10.020
3. Wu, H., Lu, Y., Ma, H., Xing, L., Deng, K., Lu, X.: A survey on task type-based computation offloading in mobile edge networks. Ad Hoc Netw. **169**, 103754 (2025). https://doi.org/10.1016/j.adhoc.2025.103754
4. Abishu, H.N., Seid, A.M., Jhaveri, R.H., Gadekallu, T.R., Erbad, A., Guizani, M.: Blockchain-empowered resource allocation in HAPS-assisted IoV digital twin networks: a federated DRL approach. IEEE Trans. Intell. Veh. **10**, 4710–4726 (2025). https://doi.org/10.1109/TIV.2024.3492015
5. Jiang, L., Zheng, H., Tian, H., Xie, S., Zhang, Y.: Cooperative federated learning and model update verification in blockchain-empowered digital twin edge networks. IEEE Internet Things J. **9**(13), 11154–11167 (2022). https://doi.org/10.1109/JIOT.2021.3126207
6. Zhang, Z., Huang, Y., Zhang, C., Zheng, Q., Yang, L., You, X.: Digital twin-enhanced deep reinforcement learning for resource management in network slicing. IEEE Trans. Commun. **72**(10), 6209–6224 (2024). https://doi.org/10.1109/TCOMM.2024.3395698
7. Cho, S., Lim, S., Lee, J.: DRL-enabled hierarchical federated learning optimization for data heterogeneity management in multi-access edge computing. IEEE Access **12**, 147209–147219 (2024). https://doi.org/10.1109/ACCESS.2024.3473008
8. Sahu, D., et al.: Edge-assisted energy optimization for mobile AR applications for enhanced battery life and performance. Sci. Rep. **15**, 10034 (2025). https://doi.org/10.1038/s41598-025-93731-w
9. Almalawi, A., Hassan, S., Fahad, A., Khan, A.I.: Edge computing optimization: An enhanced algorithm for efficient caching and latency reduction. Egypt. Inf. J. **31**, 100781 (2025). https://doi.org/10.1016/j.eij.2025.100781
10. Baghban, H., Rezapour, A., Hsu, C.-H., Nuannimnoi, S., Huang, C.-Y.: Edge-AI: IoT request service provisioning in federated edge computing using actor-critic reinforcement learning. IEEE Trans. Eng. Manage. **71**, 12519–12528 (2024). https://doi.org/10.1109/TEM.2022.3166769
11. Ji, Z., Qin, Z., Tao, X., Han, Z.: Resource optimization for semantic-aware networks with task offloading. IEEE Trans. Wireless Commun. **23**(9), 12284–12296 (2024). https://doi.org/10.1109/TWC.2024.3390407

12. Yang, C., Chen, Q., Zhu, Z., Huang, Z.-A., Lan, S., Zhu, L.: Evolutionary multitasking for costly task offloading in mobile-edge computing networks. IEEE Trans. Evol. Comput. **28**(2), 338–352 (2024). https://doi.org/10.1109/TEVC.2023.3255266
13. Zang, L., Zhang, X., Guo, B.: Federated deep reinforcement learning for online task offloading and resource allocation in WPC-MEC networks. IEEE Access **10**, 9856–9867 (2022). https://doi.org/10.1109/ACCESS.2022.3144415
14. Dai, Y., Zhao, J., Zhang, J., Zhang, Y., Jiang, T.: Federated deep reinforcement learning for task offloading in digital twin edge networks. IEEE Trans. Netw. Sci. Eng. **11**(3), 2849–2863 (2024). https://doi.org/10.1109/TNSE.2024.3350710
15. Huang, J., Liu, F., Zhang, J.: Multi-dimensional QoS evaluation and optimization of mobile edge computing for IoT: a survey. Chin. J. Electron. **33**(4), 859–874 (2024). https://doi.org/10.23919/CJE.2023.00.264
16. Safavifar, Z., Gyamfi, E., Mangina, E., Golpayegani, F.: Multi-objective deep reinforcement learning for efficient workload orchestration in extreme edge computing. IEEE Access **12**, 74558–74571 (2024). https://doi.org/10.1109/ACCESS.2024.3405411
17. Samy, A., Elgendy, I.A., Yu, H., Zhang, W., Zhang, H.: Secure task offloading in blockchain-enabled mobile edge computing with deep reinforcement learning. IEEE Trans. Netw. Serv. Manage. **19**(4), 4872–4887 (2022). https://doi.org/10.1109/TNSM.2022.3190493
18. McMahan, H.B., Moore, E., Ramage, D., Hampson, S., y Arcas, B.A.: Communication-efficient learning of deep networks from decentralized data. In: Proceedings of the 20th International Conference Artificial Intelligence Statistics (AISTATS), vol. 54, pp. 1273–1282 (2017). https://proceedings.mlr.press/v54/mcmahan17a.html

Applying Metamorphic Testing to Machine Learning: A Systematic Mapping Study

A. Redempta Manzi Muneza[1(✉)], Kaveen Liyanage[1], Faqeer Ur Rehman[1], Emma Sheppard[1,2], and Ann Marie Reinhold[1,2]

[1] Montana State University, Bozeman, MT, USA
redempta.manzi@student.montana.edu, emma.sheppard@pnnl.gov, annmarie.reinhold@montana.edu
[2] Pacific Northwest National Laboratory, Richland, WA, USA

Abstract. Machine learning (ML) models are difficult to test because they often lack a reliable test oracle. Metamorphic testing (MT) addresses this challenge by defining expected relationships between multiple inputs and their outputs, called metamorphic relations. Violations of these relations indicate potential faults. Existing publications demonstrate effectiveness of MT in evaluating ML models. This article presents a systematic mapping study of MT applied to ML over the past decade. We analyze publication trends, venues, countries, problem types, learning paradigms, algorithms, and application domains. The results show steady growth in MT research, with a strong focus on supervised learning, classification tasks, and neural networkbased models, particularly in computer vision and natural language processing. However, only a few publications focused on cybersecurity and cyber-physical systems applications, indicating important areas wherein MT could improve security and reliability.

Keywords: Metamorphic Testing · Software Testing · Machine Learning · Systematic Mapping Study

1 Introduction

Today, it is nearly impossible to discuss modern technology without addressing machine learning (ML). ML underpins many everyday systems and services. Online platforms such as Amazon use ML to personalize recommendations and enhance user experiences [25]. In critical domains such as medicine [12] and transportation [16], ML supports diagnostic imaging, risk prediction, autonomous driving, and real-time traffic management.

Beyond its significant advantages, ML faces substantial challenges [20]. ML effectiveness relies heavily on analyzing vast amounts of data to uncover patterns and generate predictions with minimal human intervention [5]. Advanced algorithms enhance this ability by modeling complex, non-linear relationships and

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 138–153, 2027.
https://doi.org/10.1007/978-3-032-39763-8_8

Table 1. Research Questions.

Publication Profile Attributes		Description
1.1.	Number of publications per year	How has the number of publications in the application of MT in ML progressed over the last decade?
1.2.	Number of publications per source	What are the main publication venues in this research domain?
1.3.	Number of publications per country	Which countries contribute most to the research in this field?
MT Applications in ML		**Description**
2.1.	Testing type	How is MT applied across validation and verification (V&V) in ML?
2.2.	Problem types	To which ML problem types (classification, regression, ...) has MT been applied?
2.3.	ML categories	In which ML categories (supervised, unsupervised, ...) has MT been applied?
2.4.	ML algorithms	For which ML algorithms (NN, SVM, ...) has MT been applied?
2.5.	Application domains	To which ML application domains has MT been applied?

generalizing to new situations. However, this dependence introduces bias, safety, and privacy risks, and vulnerabilities. For example, a hiring ML model trained on historical data may learn to favor certain demographic groups [10,11]. Additionally, ML models can unintentionally reveal sensitive information through model inversion [4,31]. Given how ML models are being used for decision making, their failures or unexpected behaviors can have far-reaching consequences. Therefore, rigorous testing is essential before they are put into use.

Testing ML models presents unique challenges [3]. ML models often lack a clear test oracle [18]. It is difficult to determine whether a generated output is correct or not. The probabilistic and non-deterministic nature of many models also makes reproducibility harder [20]. As a result, testing should evaluate not only predictive performance, such as accuracy, but also non-predictive properties like robustness and fairness. This motivates approaches like metamorphic testing (MT), which enable evaluation of model properties without relying on a precise test oracle [27].

MT is a software testing technique designed to alleviate the oracle problem [7]. A test oracle is a mechanism used to determine whether a system's output is correct for a given input. Rather than relying on a single expected output, MT uses metamorphic relations (MRs), which define expected relationships between multiple inputs and their corresponding outputs. By checking these relations, testers can detect faults without requiring an oracle.

MT has been successfully applied in various domains [23,24,28]. In recent years, MT's use in ML has received growing attention [27]. For instance, MT has demonstrated its value in evaluating the robustness and fairness of biomedical large language models [13]—a relatively new field of study.

Here, we provide researchers and practitioners with the state of the science of applying MT in ML. We provide a comprehensive overview of research on this topic over the past decade. Specifically, we conduct a systematic mapping

study that identifies trends in MT applications across different areas of ML. The research questions (RQs) we aim to address are summarized in Table 1 and discussed in detail in Sect. 3. In this article, we make the following contributions:

- We provide a broad systematic mapping of MT applications across different ML paradigms.
- We identify major research trends in MT for ML.
- We reveal dominant and underexplored research areas.
- We provide actionable insights for researchers and practitioners on the application of MT in ML.

2 Related Work

A few secondary publications have explored the application of MT in ML. Rehman and Srinivasan [27] conducted a brief review of the applicability, challenges, and research opportunities of applying MT to ML. Their study provides useful insights into the potential and limitations of using MT in ML. However, it neither offers a systematic review nor explicitly examines which ML paradigms were explored.

Ali Asgari et al. [6] conducted a systematic review of MT applied to deep code models (the large language models trained to understand, generate, and analyze computer code). This publication focused on studies published between 2019 and 2024. It analyzed how MT was applied and highlighted the models most frequently evaluated. However, it was not peer-reviewed as it was published on an open-access repository, arXiv[1]. Additionally, it focuses solely on code models and considers only recent studies, published within the last five years.

To the best of our knowledge, no existing study investigates the application of MT across different areas of ML systematically. Our study addresses this gap. We provide a comprehensive analysis that spans multiple ML model types and diverse application domains, focusing on MT usage in ML over the past decade.

3 RQ Terminology

In this section, we define and clarify key terms used in formulating our RQs. We focus on those pertaining to the application of MT in ML (Table 1). The publication profile RQs (1.1, 1.2, and 1.3) are straightforward and therefore do not require further clarification.

ML Testing Types (RQ 2.1.): In many cases, MT is applied in ML to achieve validation and verification (V&V) [8,9,26]. Validation is the process of assessing whether a model behaves as expected and meets the requirements of the target task. For example, validating an image classification model involves changing the order of input images and checking whether the model still produces the same

[1] https://arxiv.org/.

classifications. Verification is the process of assessing whether a model behaves according to its expected properties and whether its components, including algorithms and preprocessing functions, are correctly implemented. For example, verification of an image classification model could involve checking that each layer of the model applies its intended operations to the input correctly. It involves checking that these operations produce the expected feature maps or outputs at each stage, leading to the correct final classification.

ML Problem Types (RQ 2.2.) ML problems can generally be categorized based on the type of output they aim to predict [21]. For instance, *classification* problems involve predicting discrete labels or categories, such as identifying the species of a plant from its features. *Regression* problems, on the other hand, focus on predicting continuous numeric values, such as forecasting stock market trends. In this RQ, we explore these and other ML problem types discussed in the literature.

ML Categories (RQ 2.3.): ML can be broadly categorized into supervised, unsupervised, and reinforcement learning [29]. In *supervised learning*, models are trained on labeled data to predict outcomes, such as classifying images or forecasting numerical values. *Unsupervised learning* addresses unlabeled data, aiming to discover hidden patterns or groupings, as in clustering or dimensionality reduction. *Reinforcement learning* involves an agent learning to make sequential decisions by interacting with an environment and receiving feedback in the form of rewards or penalties. It is commonly used in robotics and game playing.

ML Algorithms (RQ 2.4.): ML algorithms encompass a broad range of techniques used to discover patterns in data that can be used to make predictions [21]. For example, neural networks (NN) are particularly powerful, capable of handling complex, high-dimensional data such as images, text, and speech. Other widely used algorithms include tree-based methods and support vector machines (SVMs).

ML Application Domains (RQ 2.5.): ML has applications across diverse domains [28]. In healthcare, ML supports tasks such as medical diagnosis and disease prediction. In finance, it is applied to fraud detection and credit scoring. Other prominent areas include autonomous systems and computer vision. This RQ examines the application domains in which MT has been used and where it has made notable contributions.

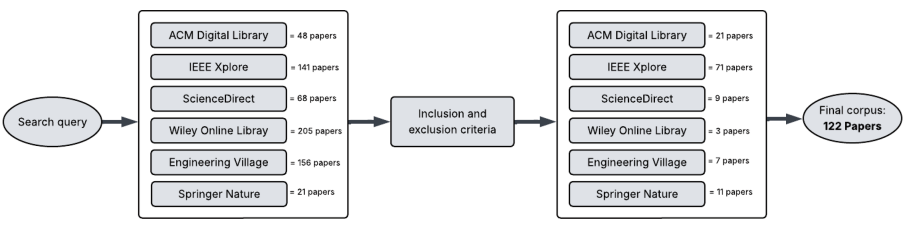


Fig. 1. Methods used to identify relevant publications to answer the RQs.

4 Methods

This study follows the guidelines proposed by Kitchenham et al. [17], which provide a structured and rigorous approach for identifying, evaluating, and interpreting relevant research.

4.1 RQ Types

The RQs in this study are divided into two parts: the publication profiles and the MT applications in ML (Table 1).

Where necessary, we grouped specific findings into broader concepts to better address our RQs. For example, in RQ 2.4, Neural Networks include various NNbased models, such as Generative Adversarial Networks and Convolutional Neural Networks. In RQ 2.5, computer vision includes areas like image and video processing applications (Table 1).

4.2 Identifying Relevant Research

To answer the RQs, relevant publications were identified through five main activities: defining search query (SQ), searching databases, selecting relevant publications, extracting data, and analyzing data (Fig. 1).

We defined the SQ to capture both ML and MT concepts (Fig. 2). We focused our SQ on terms used in the titles, abstracts, and keywords of publications. We set such limits to ensure that the retrieved publications explicitly addressed ML and MT as their primary research focus, while reducing the inclusion of publications in which these concepts were peripheral to the research. In addition, we excluded abbreviations such as “ML” and “MT” from the SQ deliberately to avoid retrieving irrelevant publications. These abbreviations can have multiple meanings in scientific databases; for example, “ML” may also refer to milliliters.

```
[[Abstract: "metamorphic testing"] OR ["Abstract: metamorphic relations"]] AND [[Abstract: "machine learning"] OR [Abstract: "deep learning"] OR [Abstract: "neural networks"] OR [Abstract: "artificial intelligence"]] AND [Publication Date: (2015 to April 2025)]
```

Fig. 2. Search query.

We executed the SQ in six scientific databases (Fig. 1). We applied inclusion and exclusion criteria (Table 2) to the 635 publications retrieved from the databases (Fig. 1) to identify studies relevant to our research topic.

A Python script was used to enforce criteria, such as filtering publications by the specified year range and removing duplicates. The co-authors then manually reviewed each publication to assess relevance and conduct a quality check before

Table 2. Inclusion and Exclusion Criteria.

Inclusion criteria	Exclusion criteria
1. Publications that examine the use of MT in ML directly or indirectly	1. Books and book chapters
2. Primary studies	2. Invited talks
3. Studies published between 2015 and April of 2025	
4. Publications written in English	
5. Peer-reviewed publications	

the data extraction process. The complete set of publications (122 publications) is publicly available on Zenodo [19]

The selected publications were assigned to co-authors for extracting relevant data. They were then reassigned to different co-authors for an independent second extraction of the same data to validate the results. Any discrepancies were resolved through discussion and settled via consensus.

5 Results

This section summarizes the research described in the corpus of publications in relation to the RQs. Results highlight trends in publication characteristics, as well as the how MT has been applied across ML. Note that some papers address multiple elements of the RQs, resulting in an increased total publication count. For instance, in RQ 2.4 (Table 3), several papers evaluated multiple ML algorithms and were included in multiple counts. Furthermore, not all publications address every aspect of our RQs; consequently, the publication counts vary across the RQs. For instance, a publication might discuss MT broadly across problem types, such as classification, without evaluating specific models. As a result, these publications are counted for the ML problem types RQ (RQ 2.2) but not for the ML algorithms RQ (2.4; Table 3).

5.1 Publication Profile RQs

1.1. Number of publications per year: The number of publications on MT in ML has increased notably over the years (Fig. 3). Nearly 76.22% (93) of the total publications appeared in the past five years, with 31.52% (29) of them published in 2024 alone.

1.2. Publication sources: IEEE *Xplore* and the ACM Digital Library are the primary publication sources (Fig. 4). Together they account for 75.40% (92) of the publications while the remaining four sources collectively contribute 24.59% (30).

1.3. Publications by country: China is the leading contributor to research on this topic (Fig. 5). China is the country of origin for half of the publications (61), followed by the United States with 18.85% (23) and Australia with 14.75% (18). The remaining 23 countries collectively contribute 16.39% (20) of the publications.

Table 3. Mapping of publications by RQ. Corpus of publications is available on Zenodo [19].

RQs	Findings	Fraction	Publication ID
Testing Type	Validation	68 (55.73%)	S1, S3, S5, S6, S8, S13–S18, S20-S23, S25–S28, S30, S32–S35, S38, S39, S41, S43–S46, S48, S52–S54, S59, S61–S63, S66, S67, S69, S71, S73–S76, S81, S83, S87, S97, S98, S100–S102, S104–S112, S115, S117–S119
	Verification	31 (25.40%)	S2, S7, S19, S24, S37, S42, S47, S56, S57, S60, S65, S68, S72, S77, S79, S80, S82, S84, S88–S94, S113, S114, S116, S120–S122
	V&V	19 (15.57%)	S4, S10–S12, S31, S36, S40, S49, S50, S55, S64, S70, S78, S85, S86, S95, S96, S99, S103
	General testing	4 (3.27%)	S9, S29, S51, S58
Problem Type	Classification	76 (61.29%)	S1, S4–S13, S18, S19, S24, S26, S28, S31, S35–S39, S42, S43, S45, S47, S48, S52, S53, S56, S57, S60–S64, S66, S68–S70, S72–S75, S78–S84, S87–S89, S91, S93–S95, S97–S99, S104–S109, S112, S114, S115, S117–S122
	Generative	18 (14.51%)	S15, S16, S18, S20, S30, S39, S54, S65, S67, S71, S77, S86, S90, S101, S102, S104, S105, S107
	Regression	8 (6.45%)	S12, S21, S23, S25, S92, S96, S103, S111
	Object detection	7 (5.64%)	S17, S20, S22, S27, S34, S55, S59
	Clustering	5 (4.03%)	S41, S46, S49, S50, S100
	Time series	4 (3.22%)	S33, S40, S44, S85
	Recommendation	3 (2.41%)	S32, S110, S113
	Dimensionality reduction	1 (0.80%)	S14
	Segmentation	1 (0.80%)	S42
	Anomaly detection	1 (0.80%)	S76
ML Categories	Supervised	100 (86.20%)	S1, S4, S5–S7, S9–S28, S31–S40, S42–S45, S47, S48, S52, S53, S55–S57, S59–S64, S66, S68–S70, S72–S75, S77–S85, S87–S99, S102–S112, S114, S115, S117, S118, S120–S122
	Unsupervised	12 (10.34%)	S8, S30, S41, S46, S49, S50, S54, S65, S67, S71, S100, S101
	Reinforcement	4 (3.44%)	S3, S79, S113, S119
ML Algorithms	Neural Networks	108 (69.67%)	S1, S3–S7, S11, S12–S23, S25–S30, S32–S35, S38, S40, S43–S45, S52–S57, S59, S61, S63–S65, S68, S69, S71–S73, S75, S77–S79, S81–S83, S85, S87–S94, S98, S99, S101, S103–S109, S111–S115, S117–S122
	SVM	14 (9.03%)	S7, S31, S36, S37, S53, S60, S64, S70, S78, S80, S82, S84, S85, S96
	Instance-Based	9 (5.80%)	S10, S31, S36, S37, S47, S64, S78, S96, S97
	Ensemble	7 (4.51%)	S9, S31, S36, S78, S84, S96, S97
	Clustering	6 (3.87%)	S41, S46, S49, S50, S74, S100
	Linear	5 (3.22%)	S36, S37, S78, S80, S96
	Tree-Based	2 (1.29%)	S78, S96
	Probabilistic	2 (1.29%)	S78, S97
	NMT	1 (0.64%)	S102
	Matrix Factorization	1 (0.64%)	S110

(*continued*)

Table 3. (*continued*)

RQs	Findings	Fraction	Publication ID
Application Domains	Computer vision	40 (34.18%)	S1, S4–S7, S9, S10, S13, S17, S19, S20, S22, S26, S30, S35, S37, S43, S53, S57, S61, S68, S69, S72, S73, S78, S79, S83, S84, S87, S89, S91, S94, S95, S106–S112, S115, S117, S121, S122.
	NLP	28 (23.93%)	S15, S16, S18–S20, S39, S54, S65, S67, S71, S75, S77, S78, S86, S88, S90, S98, S99, S101, S102, S104, S105, S118, S119
	Autonomous systems	21 (17.94%)	S8, S12, S21, S23, S25, S28, S32, S34, S52, S55, S59, S66, S76, S92, S93, S103, S108, S111, S114, S116, S120
	Cybersecurity	5 (4.27%)	S14, S45, S56, S63, S83
	Software engineering	3 (2.56%)	S70, S74, S85, S100
	CPS	2 (1.70%)	S27, S44
	IDS	2 (1.70%)	S31, S37
	Agriculture	2 (1.70%)	S33, S85
	Anomaly detection	2 (1.70%)	S36, S49
	Biology	2 (1.70%)	S48, S64
	Scientific software	1 (0.85%)	S70
	Hydrology	1 (0.85%)	S96
	Remote sensing	1 (0.85%)	S89
	Chemistry	1 (0.85%)	S82
	Activity recognition	1 (0.85%)	S42
	Electronics	1 (0.85%)	S62
	Business intelligence	1 (0.85%)	S50
	Gaming	1 (0.85%)	S3
	Forecasting	1 (0.85%)	S40
	Healthcare	1 (0.85%)	S97

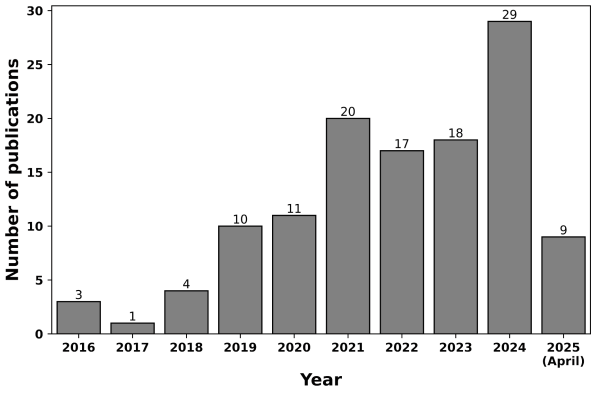


Fig. 3. Number of publications per year. None of the publications from 2015 met the inclusion criteria.

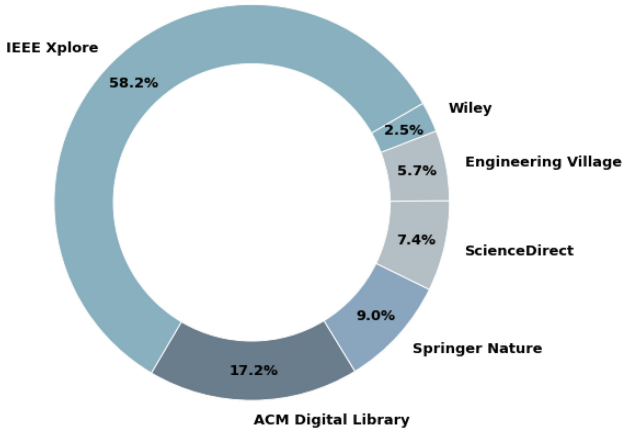


Fig. 4. Distribution of publications by publication venues.

5.2 MT Application in ML RQs

2.1. Testing types: The majority of the publications applied MT to validate ML (55.73%—68 of publications [Table 3]). The use of MT for the verification of ML occurred in 25.40% (31) of publications. A further 15.57% (19) of the publications addressed both validation and verification together. Finally, 3.27% (4) of publications applied MT for other ML testing.

2.2. Problem types: Classification is the most common ML problem type to which MT was applied (Table 3). Among the ten problem types identified, classification was the problem type in 61.29% (76) of the publications, followed by generative tasks at 14.51% (18), and regression at 6.45% (8). The remaining problem types collectively were found in 17.74% (22) of the publications.

2.3. ML categories: Supervised learning was the most common of the ML models tested using MT, accounting for 86.20% (100) of publications, (Table 3). Unsupervised learning and reinforcement learning were less common, found in 10.34% (12) and 3.44% (4) publications, respectively.

2.4. ML algorithms: MT was used most frequently with NN-based algorithms (Table 3). MT was used with NNs in 69.69% (108) of publications in the corpus, followed by SVM in 9.03% (14) publications, and instance-based algorithms in 5.80% (9) publications. Ensemble and clustering algorithms appeared in 4.51% (7) and 3.87% (5) publications, respectively, while linear algorithms are reported in 3.22% (5) publications. Tree-based and probabilistic algorithms are each represented in 1.29% (2) of publications. Neural machine translation (NMT) and matrix factorization (MF) algorithms are the least explored.

2.5. Application domains: The applications in the corpus of publications were diverse, with computer vision being the most prevalent—found in 34.18% (40) of the publications (Table 3). Natural language processing (NLP) and autonomous

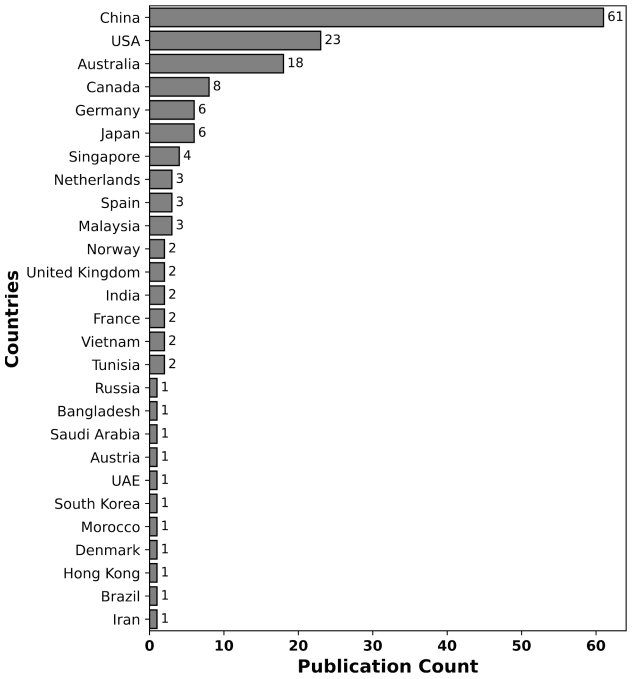


Fig. 5. Distribution of publications by country.

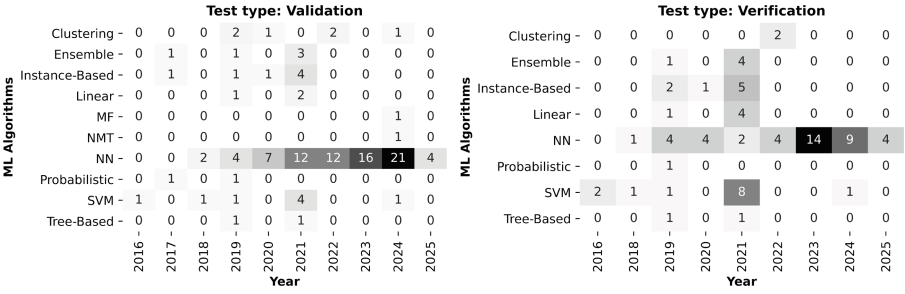


Fig. 6. Distribution of ML algorithms by publication year. Subplots show ML testing types: Validation (left) and Verification (right), with heatmap intensity indicating publication count. MF: Matrix Factorization; NMT: Neural Machine Translation; NN: Neural Networks, SVM: Support Vector Machines.

systems follow in 23.93% (28) and 17.94% (21) of publications, respectively. A smaller proportion of the publications focus on cybersecurity (4.27% [5] publications) and software engineering (2.56% [3] publications). Several domains, including cyber-physical systems (CPS), intrusion detection systems (IDS), agriculture, anomaly detection, and biology, each in 1.70% (2) of the publications. The remaining application domains, such as scientific software, hydrology, remote

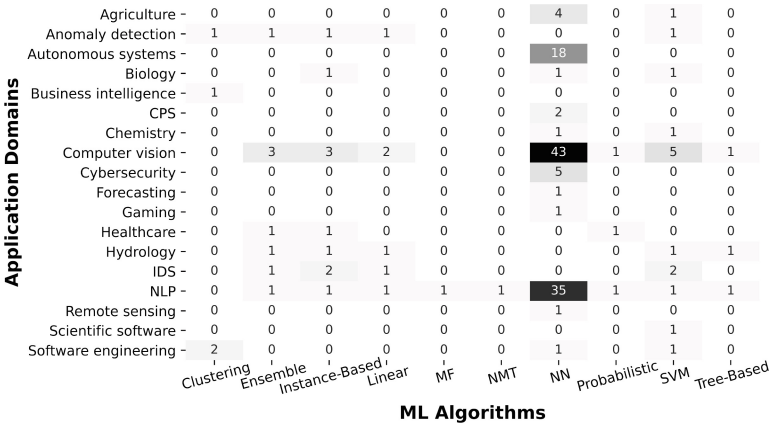


Fig. 7. Distribution of ML application domain by ML algorithm type. The heatmap intensity indicates publication count. CPS: Cyber Physical Systems; IDS: Intrusion Detection System; NLP: Natural Language Processing; MF: Matrix Factorization; NMT: Neural Machine Translation; NN: Neural Networks, SVM: Support Vector Machines.

sensing, chemistry, activity recognition, electronics, business intelligence, gaming, forecasting, and healthcare, are minimally represented, each being studied in only one publication in the corpus.

6 Discussion

MT has emerged as a key approach for addressing testing challenges in ML. Its growing interest is evident in the rising number of publications and in the diversity of where and by whom the research is conducted.

The distribution of publications in major digital libraries and the study topics (Fig. 4) suggest that MT research in ML is largely driven by the computer science community. Similarly, the majority of the publications originate from a small number of countries (Fig. 5), highlighting where expertise and research efforts are currently concentrated.

Most publications focused on evaluating the correctness and usefulness of ML models. This focus is reflected in the widespread adoption of validation-oriented testing. The trend is closely linked to the extensive use of black-box models [1] (Fig. 6), which naturally favor validation [27] because access to internal model structures is obscured. Analysis of the most frequently tested algorithms and application domains (Table 3) indicates a demand for reliable ML solutions, particularly NN architectures.

Verification-based testing is more difficult because it depends on knowledge of model architecture to define meaningful MRs. The challenge increases with the growing complexity and limited interpretability of modern ML models. For

example, defining effective MRs for deep neural networks or large language models is difficult without insight into internal representations or decision logic. As a result, publications that focus on MT for verification of ML algorithms are less common than validation studies (Fig. 6), highlighting an opportunity for future research.

Before 2021, the models tested spanned a variety of algorithms (Fig. 6). Afterward, researcher focus became highly concentrated on the MT of NNbased models. As NN models have become more widely used, there is a corresponding need for research into effective testing methods; our results indicated that MT is a candidate method. It is also notable that the steady increase in MT of NN models coincides with the rising popularity of transformers, following their introduction in the publication "Attention Is All You Need" published in 2017 [30]. The application domains where the tested algorithms were applied—predominantly computer vision and NLP—also align with this trend (Fig. 7), consistent with the emerging use of generative AI models.

Surprisingly, MT has not been widely used in cybersecurity applications (5 publications) and CPS (2 publications; Fig. 7), given the recent adoption of ML in CPS [2,15,22]. These findings suggest that the application of MT to safety-critical systems is an important, underexplored area of research.

The extent to which MT has been used on classification problems (Table 3) reflects the widespread use and suitability of MT for validation. Classification outputs are discrete and well-defined, which makes it easier to develop MRs. For example, if an input is slightly perturbed, such as adding small noise to an image, the predicted class for a classifier should remain the same. For non-classification tasks, such as regression or clustering, small input changes can lead to continuous or dynamic outputs, making it less clear whether a difference indicates a fault or expected behavior. Thus, defining MRs for such problems is more challenging than for classification problems.

6.1 Implications for Practitioners

Studying trends in MT of ML provides insights into how the field has evolved. Our findings support informed decision-making by indicating how MT has been used, and which problem types, categories, algorithms, and applications have been studied. By understanding the state of the science, practitioners can allocate resources more effectively and prioritize testing efforts where they are likely to have the most impact. For instance, we identified only a paucity of publications using MT on cybersecurity ML problems. Application of MT to ML models in cyber and safety-critical systems could be especially impactful.

Practitioners can also use the findings of this article as a reference when designing new studies. Reviewing prior work helps identify relevant methods and examples of effective MRs. While the MRs themselves were not a main focus of this article, we provide references to the original publications[2] where they were

[2] The publications analyzed in this study are publicly available on Zenodo repository [19].

applied, mapping our findings to the publications directly. This mapping helps to avoid redundancy in effort and reduce trial-and-error in test case design.

6.2 Threats to Validity

In this study, we account for threats to validity [14] that may have influenced the design, analysis, and interpretation of the results.

Threats to Construct Validity and Reliability: A potential threat to construct validity arises from the manual categorization of the selected publications according to learning paradigms and problem types. Because terminology varies across publications, the categories may not always fully reflect how authors describe their own work. To mitigate this risk, all co-authors discussed and agreed on the definitions and interpretations of the terminology during the categorization process.

Regarding reliability, reproducing the study could result in a different set of publications. This may happen because researchers interpret the inclusion and exclusion criteria differently. To minimize this threat, we employed a structured protocol throughout this study, including during the search and selection of publications.

Threats to Internal Validity: A potential threat to internal validity arises from the grouping of publications across the different ML dimensions. Because these classifications require interpretation of the content of each publication, there is a possibility of misclassification. To mitigate this threat, the classification decisions were reviewed by multiple co-authors, and any disagreements were resolved by consensus.

Another internal threat to validity arose from the application of the inclusion and exclusion criteria. Some relevant publications might have been excluded if they did not clearly meet the defined criteria or if their relevance was not immediately apparent. To mitigate this threat, we conducted multiple rounds of review and cross-checked selections among co-authors until agreement was reached.

Threats to External Validity: A potential threat to external validity arises from the selection of academic databases used to retrieve publications. Although widely recognized databases were used to capture relevant literature, some studies may be published in venues not indexed by these databases. As a result, some relevant publications may have been omitted. Nevertheless, the selected databases are among the most comprehensive sources for computer science and software engineering research, providing broad coverage of the literature.

Another threat relates to the design of the search query. The search focused on specific terms appearing in the abstracts of publications to ensure that the retrieved publications had a clear focus on our topic. However, relevant publications that discuss this topic without explicitly mentioning the key terms in the abstract may not have been captured. However, this approach helped ensure that the selected publications were strongly aligned with the focus of this study.

Finally, our study relies on academic publications. Industrial reports, technical documentation, or unpublished work may also apply MT in practice but are not included in the corpus of publications. Consequently, the findings primarily reflect trends within the academic research community. Despite this limitation, peer-reviewed publications provide a reliable and well-documented evidence, supporting the credibility of the conclusions drawn in this study.

7 Conclusion

This article presents a systematic mapping study on the application of MT in ML models. Our analysis shows a steady increase in research activity over the past decade, indicating the growing interest in addressing the challenges of testing ML models with MRs. The findings suggest that MT research in ML is shaped by practical validation needs, modern model architectures, and high-impact application domains. While current efforts favor validation of black-box models, the growing complexity of ML models highlights the opportunity to further explore verification-oriented MT.

The study also identifies key trends in publication venues, countries, and algorithm types, providing a structured overview of the state of research in MT for ML. By synthesizing these insights, we highlight existing gaps and opportunities for future work, including exploring less-studied domains, algorithms, and learning paradigms. The limited number of publications exploring the use of MT in cybersecurity and cyber-physical systems (Fig. 7) point to research areas that merit further investigation.

References

1. Aghababaeyan, Z., Abdellatif, M., Briand, L., Bagherzadeh, M., et al.: Black-box testing of deep neural networks through test case diversity. IEEE Trans. Softw. Eng. **49**(5), 3182–3204 (2023)
2. Ahmed, R.S., Ahmed, E.S.A., Saeed, R.A.: Machine learning in cyber-physical systems in industry 4.0. In: Artificial Intelligence Paradigms for Smart Cyber-Physical Systems, pp. 20–41. IGI Global Scientific Publishing (2021)
3. Albelali, S., Ahmed, M.: Testing machine learning and deep learning systems: Achievements and challenges. Arab. J. Sci. Eng. **50**(15), 11433–11484 (2025)
4. Alves, T.A.O., França, F.M.G., Kundu, S.: Mlprivacyguard: Defeating confidence information based model inversion attacks on machine learning systems. In: Proceedings of the 2019 Great Lakes Symposium on VLSI, pp. 411–415. GLSVLSI '19, Association for Computing Machinery, New York, NY, USA (2019). https://doi.org/10.1145/3299874.3319457
5. Alzubi, J., Nayyar, A., Kumar, A.: Machine learning from theory to algorithms: an overview. J. Phys. Conf. Ser. **1142**, 012012. IOP Publishing (2018)
6. Asgari, A., de Koning, M., Derakhshanfar, P., Panichella, A.: Metamorphic testing of deep code models: A systematic literature review. arXiv preprint arXiv:2507.22610 (2025)
7. Chen, T.Y., Cheung, S.C., Yiu, S.M.: Metamorphic testing: a new approach for generating next test cases. arXiv preprint arXiv:2002.12543 (2020)

8. Ding, J., Kang, X., Hu, X.H.: Validating a deep learning framework by metamorphic testing. In: 2017 IEEE/ACM 2nd International Workshop on Metamorphic Testing (MET), pp. 28–34 (2017). https://doi.org/10.1109/MET.2017.2
9. Dwarakanath, A., et al.: Identifying implementation bugs in machine learning based image classifiers using metamorphic testing. In: Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 118–128 (2018)
10. Fabris, A., et al.: Fairness and bias in algorithmic hiring: A multidisciplinary survey. ACM Trans. Intell. Syst. Technol. **16**(1) (2025). https://doi.org/10.1145/3696457
11. Fuhnwi, G.S., Reinhold, A.M., Izurieta, C.: Reducing human-induced label bias in sms spam with context-enhanced clustering (cec). In: 2025 IEEE International Conference on Cyber Security and Resilience (CSR), pp. 71–76 (2025). https://doi.org/10.1109/CSR64739.2025.11130032
12. Habehh, H., Gohel, S.: Machine learning in healthcare. Current Genomics **22**(4), 291–300 (2021)
13. Jaganathan, G.S., Kahanda, I., Kanewala, U.: Metamorphic testing for robustness and fairness evaluation of llm-based automated icd coding applications. Smart Health **36**, 100564 (2025)
14. Juristo, N., Moreno, A.M.: Basics of software engineering experimentation, Universidad Politécnica de Madrid (2001)
15. Kim, S., Park, K.J.: A survey on machine-learning based security design for cyber-physical systems. Appl. Sci. **11**(12), 5458 (2021)
16. Kiran, B.R., et al.: Deep reinforcement learning for autonomous driving: A survey. IEEE Trans. Intell. Transp. Syst. **23**(6), 4909–4926 (2021)
17. Kitchenham, B., Charters, S.: Guidelines for performing systematic literature reviews in software engineering (2007)
18. Liem, C.C.S., Panichella, A.: Oracle issues in machine learning and where to find them. In: Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops, pp. 483–488. ICSEW'20, Association for Computing Machinery, New York, NY, USA (2020). https://doi.org/10.1145/3387940.3391490
19. Manzi Muneza, A.R., Liyanage, K., Rehman, F.u., Sheppard, E.: Mt in ml: Collected publications (2015–April 2025) (2026). https://doi.org/10.5281/zenodo.18905637
20. Marijan, D., Gotlieb, A., Kumar Ahuja, M.: Challenges of testing machine learning based systems. In: 2019 IEEE International Conference On Artificial Intelligence Testing (AITest). pp. 101–102 (2019). https://doi.org/10.1109/AITest.2019.00010
21. Nti, I.K., Adekoya, A.F., Weyori, B.A., Nyarko-Boateng, O.: Applications of artificial intelligence in engineering and manufacturing: a systematic review. J. Intell. Manuf. **33**(6), 1581–1601 (2022)
22. Olowononi, F.O., Rawat, D.B., Liu, C.: Resilient machine learning for networked cyber physical systems: A survey for machine learning security to securing machine learning for cps. IEEE Commun. Surv. Tutorials **23**(1), 524–552 (2020)
23. Rahman, K., Izurieta, C.: An approach to testing banking software using metamorphic relations. In: 2023 IEEE 24th International Conference on Information Reuse and Integration for Data Science (IRI), pp. 173–178 (2023). https://doi.org/10.1109/IRI58017.2023.00036
24. Rahman, K., Reinhold, A.M., Izurieta, C.: Metamorphic relation prediction for security vulnerability testing of online banking applications. In: 2025 IEEE International Conference on Cyber Security and Resilience (CSR), pp. 226–233 (2025). https://doi.org/10.1109/CSR64739.2025.11130068

25. Rastogi, R.: Machine learning @ amazon. In: The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval, pp. 1337–1338. SIGIR '18, Association for Computing Machinery, New York, NY, USA (2018). https://doi.org/10.1145/3209978.3210211
26. Rehman, F.U., Izurieta, C.: Mt4uml: Metamorphic testing for unsupervised machine learning. In: 2022 9th Swiss Conference on Data Science (SDS), pp. 26–32. IEEE (2022)
27. Rehman, F.U., Srinivasan, M.: Metamorphic testing for machine learning: Applicability, challenges, and research opportunities. In: 2023 IEEE International Conference On Artificial Intelligence Testing (AITest), pp. 34–39. IEEE (2023)
28. Segura, S., Fraser, G., Sanchez, A.B., Ruiz-Cortés, A.: A survey on metamorphic testing. IEEE Trans. Software Eng. **42**(9), 805–824 (2016)
29. Sharma, H., Arora, G., Bhowmik, P., Singh, M.K., Ayyappan, V., Sehgal, A.K., Rangappa, S.M., Siengchin, S.: Predictive modeling of thermoplastic nanocomposites using machine learning algorithms. Disc. Mech. Eng. **4**(1), 66 (2025)
30. Vaswani, A., et al.: Attention is all you need. Adv. Neural Inf. Process. Syst. **30** (2017)
31. Yang, W., et al.: Deep learning model inversion attacks and defenses: a comprehensive survey. Artif. Intell. Rev. **58**(8), 242 (2025)

Anchoring Large Language Models and Digital Twins in Designing Deeply Personalized mHealth Platforms

Muhammad Irfan Khalid[1(✉)], Jie Zhao[2], Aamir Shahzad[3], Muhammad Javed[4], and Mansoor Ahmed[5,6]

[1] Department of Information Systems, University of Agder, Kristiansand, Norway
irfanse6235@gmail.com
[2] Penn State Erie, The Behrend College, PA, USA
[3] Health and Population Department Punjab, THQ, Jatoi, Pakistan
[4] Department of Computing and Information Technology, Gomal University, Dera Ismail Khan, Pakistan
[5] Department of Computer Science, Maynooth University, Maynooth, Ireland
[6] ADAPT Centre, Maynooth University, Maynooth, Ireland

Abstract. This paper presents an automated framework that uses Large Language Models (LLMs) with prompt engineering to generate scalable, measurable and dynamically personalized health goals. By integrating Human Digital Twins (HDTs) the framework leverages behavioral and historical data to continuously refine goals and deliver hyper personalized recommendations. Study results demonstrate that combining LLMs and HDTs overcomes the limitations of static health interventions highlighting the potential of adaptive AI-driven personalization to support sustained behavior change and improved health outcomes.

Keyword: Personalization, Lifestyle disease, Large language models, Digital twins, Prompt engineering, Goals Creation, Goals Refinements

1 Introduction

Lifestyle diseases largely stem from unhealthy daily habits, including poor diet, physical inactivity, and disrupted biological rhythms [1]. To mitigate their growing societal impact, mobile health (mHealth) applications have been widely adopted to support healthier behaviors through self-monitoring, reminders and medical feedback [2]. While mHealth solutions have shown promise in encouraging lifestyle changes such as weight loss, healthy eating and regular exercise [3], evidence of their long term effectiveness and cost efficiency remains limited [4]. A key limitation is their lack of intelligence and personalization compared to human coaching, particularly in generating tailored health treatment goals. Personalization defined as adapting system content to individual user characteristics is recognized as a critical factor in effective persuasive health technologies [5]. Prior research highlights that measurable and individualized health goals

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 154–168, 2027.
https://doi.org/10.1007/978-3-032-39763-8_9

are essential for sustained behavior change and lifestyle disease management [6]. However existing mHealth applications rarely support the automated creation and continuous refinement of such goals. In this paper, we proposed a deeply personalized mHealth platform for scalable health treatment goals creation and refinement. Based on a systematic review of existing approaches, we identify key limitations in current digital health interventions. We then introduce a novel design that leverages LLMs for automated, personalized goal generation and Digital Twins (DTs) for ongoing goal refinement using longitudinal user data. This approach addresses the shortcomings of static and manually designed interventions, advancing adaptive and personalized mHealth systems. Our study is guided by the following research questions:

- What are the main challenges to create manual personalized health treatment goals with the help of human coaches?
- How can one (semi) automatically generate personalized health treatment goals tailored to the users characteristics and how can these goals be dynamically refined based on the user's historical data to induce behavior changes in diverse users?
- How to apply DTs and LLMS to achieve both efficiency and effectiveness in the context of mHealth applications used for digital health interventions to study the behavior changes in various users by creating and refining personalized health treatment goals.

2 Background

Research on long term behavior change toward healthier lifestyles has traditionally relied on manual and coach-driven approaches which are costly, time consuming and difficult to scale. Most existing digital health interventions generate health treatment goals manually limiting both efficiency and personalization. The emergence of LLMs [7] and HDTs offers new opportunities to overcome these limitations by enabling automated, scalable and adaptive goal creation and refinement based on individual user data. Empirical evidence shows that health treatment goals often evolve due to disease progression, behavioral patterns or life events emphasizing the need for continuous reassessment and adaptation [8]. Static goal setting approaches therefore risk losing relevance and effectiveness in long term health interventions [9]. Recent advances in LLMs have demonstrated strong capabilities in understanding natural language, reasoning and generating human like text supported by large scale training and broad world knowledge [10]. These capabilities enable LLMs to assist in complex tasks such as structured goal formulation, contextual reasoning and adaptive personalization [11]. However creating precise, measurable and achievable health goals remains challenging due to variability in individual behavior and healthcare contexts [12]. LLMs address these challenges by supporting dynamic goal generation and refinement when combined with longitudinal user data. Despite their potential obtaining personalized and context aware outputs from LLMs requires careful

prompt engineering. Recent studies highlight prompt engineering as a critical technique for guiding LLM behavior improving customization and ensuring reliable outputs in healthcare applications [13]. By integrating LLMs with HDTs and leveraging prompt engineering this work builds on behavioral theory and emerging AI technologies to support continuous, personalized, and scalable health goal management. Digital twins are virtual replicas of real world entities that mirror their behavior and context [14]. In healthcare case, HDTs offer strong potential for personalized treatment by modeling disease progression and optimizing care plans [15]. However their broader adoption faces challenges related to data security, quality, interoperability and ethical considerations including transparency, trust and regulatory compliance [16].

3 Material and Methods

To address the first two research questions introduced in Sect. 1, we reviewed representative studies that investigated behavior change through personalized health treatment goals either supported by digital tools or human coaching. After examining existing designs from the literature (Figs. 1, 2 and 3), we propose a refined system architecture shown in Fig. 4. This design integrates LLMs and digital twins to improve efficiency in terms of cost, time, and scalability while supporting continuous refinement of personalized health treatment goals.

3.1 Design Evaluation Criteria and Levels

Our primary objective is to support sustained healthy behavior change through adaptive goal creation and refinement. To operationalize this objective we define two key design evaluation criteria such as efficiency and effectiveness. Efficiency is defined by the time, cost, and scalability associated with generating personalized health treatment goals. Despite their effectiveness, traditional coach-driven approaches are labor-intensive and challenging to deploy at scale across diverse populations. Effectiveness relates to the quality and long term impact of goal creation and refinement. Human coaching can be highly effective due to expert judgment and contextual understanding; however continuously refining goals over time becomes impractical as user numbers grow. Automated digital support is therefore essential to balance effectiveness with scalability. Our proposed design leverages LLMs [17] and HDTs to address this trade off by allowing for both efficient and effective personalization of health treatment goals over time.

3.2 System Design Components

- **Digital Intervention Tool:** A digital intervention tool provides an interface through which users self-report activities and engage with predefined goals or challenges.
- **Users:** Users are individuals who interact with the digital intervention system. While specific studies may focus on particular populations, we broadly consider any individual whose behavior may be influenced through digital interventions.

- **Health Treatment Goals:** Health treatment goals are embedded within the digital intervention tool and are achieved through the completion of specific activities [18]. Users earn points based on their activity levels, which contribute to goal attainment. In this study, the focus is on physical activity related goals.
- **Human Digital Twin:** The proposed Human Digital Twin (HDT) represents a virtual counterpart of the physical user (Fig. 4). It consists of two core models (Figs. 5 and 6) that receive continuous data from the user to generate contextual insights, which support the creation and refinement of personalized health treatment goals.
- **Large Language Model:** User inputs are processed by a large language model to generate measurable health goals. These goals are periodically adjusted based on observed user behavior and engagement.
- **Prompt Engineering Implications:** As shown in Fig. 4, the system allows users to provide generalized or vague prompts, from which the LLM derives structured and measurable goals. Future work will investigate advanced prompt engineering strategies to enable more precise and personalized goal generation for lifestyle disease management [19].

3.3 Evaluation of Design Alternatives From the Literature

In this part, we describe alternatives design, that was found to be most useful in creating personalized health treatment goals:

Design 1: The first design alternative can be seen in Fig. 1, where some components create a system. To support our claim about such sort of design implications in the existing literature, we have explored the research study that was carried out by James et al. [20]. In this study, the user was allowed to interact with a goals system, where they specified vague and generalized goals and these goals were later sent to the ChatGPT [7]. After that, LLM provided some measurable goals to those users [20]. In the design alternative proposed by [20], a digital intervention tool presents users with tasks or challenges for self-reporting. Users define measurable goals and engage in activities directly linked to these goals. The defined goals are integrated into the intervention tool, where users self-report progress by completing social, physical, and cognitive challenges. Overall, recommendations are generated by a large language model [21], specifically ChatGPT by OpenAI [7].

Benefits and Limitations: Comparing studies such as [20,22], user engagement and efficiency are higher when users create measurable goals themselves and self-report through a digital intervention tool rather than relying on manually created goals or a coach assisted process. The setting in Fig. 1 demonstrates these advantages clearly. Existing personalization systems often suffer from a one-size-fits-all problem where goals are assigned without considering individual

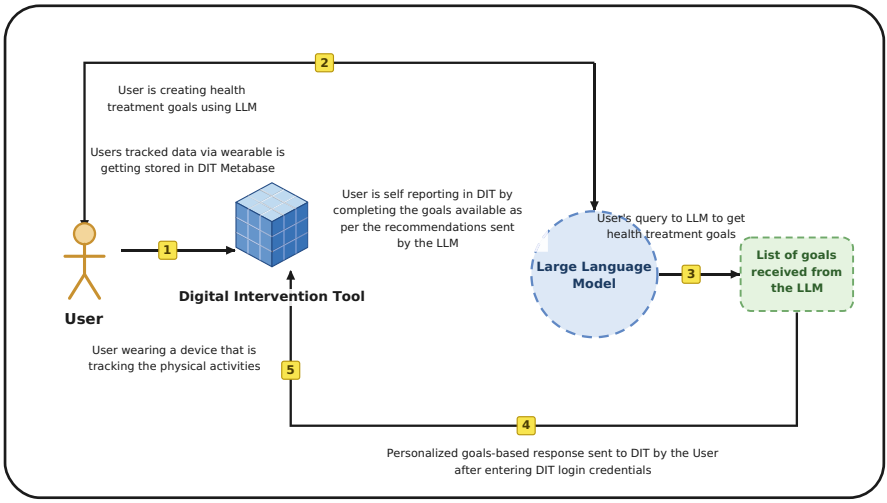


Fig. 1. System without prompt engineering for creating personalized health treatment goals using LLMs.

user characteristics. In contrast, our proposed design allows each user to interact with a LLM to create their own goals while actively participating in challenges via the intervention tool. However as shown in Fig. 1 using an LLM alone can create bottlenecks in the sense that users may spend excessive time filling goal forms leading to boredom and dropouts. Without prompt engineering, users often provide vague inputs to the LLM resulting in generalized outputs that do little to influence behavior. This highlights the need for structured prompting techniques to generate precise, personalized goals and optimize user engagement. Finally, the previous design focuses only on goal creation and lacks mechanisms for goal adjustment over time limiting its effectiveness for multiple users with evolving behavioral patterns.

Algorithm 1: LLM-based Goal Creation

Input: userInput, *digitalInterventionTool*
Output: Measurable Goals

```
begin
    // Step 1: Query LLM directly with the raw user prompt.
    prompt ← userInput
    // Step 2: Generate measurable goals.
    measurableGoals ← LLM_Generate(prompt)
    // Step 3: Store goals in the intervention tool.
    digitalInterventionTool.StoreGoals(measurableGoals)
    // Step 4: Wait for user feedback.
    userReport ← digitalInterventionTool.WaitForUserReport()
end
return measurableGoals
```

Design 2. In this design alternative, everything remains the same as in design alternative (1), with the addition of specialized prompt engineering techniques that add context to user queries before sending them to the LLM as shown in Fig. 2. These techniques enable the LLM to generate personalized and context aware responses. Similar approaches have been explored in prior work. For instance Lee et al. [23] used ChatGPT to automatically generate questions in English education leveraging prompt engineering to improve relevance and precision. In the broader literature, prompt engineering is recognized as essential for guiding LLMs to produce accurate tailored responses [24]. Likewise, Alessa et al. [25] applied ChatGPT with prompt engineering to generate personalized interventions that address loneliness and depression among older adults using individual user information to create context-specific responses.

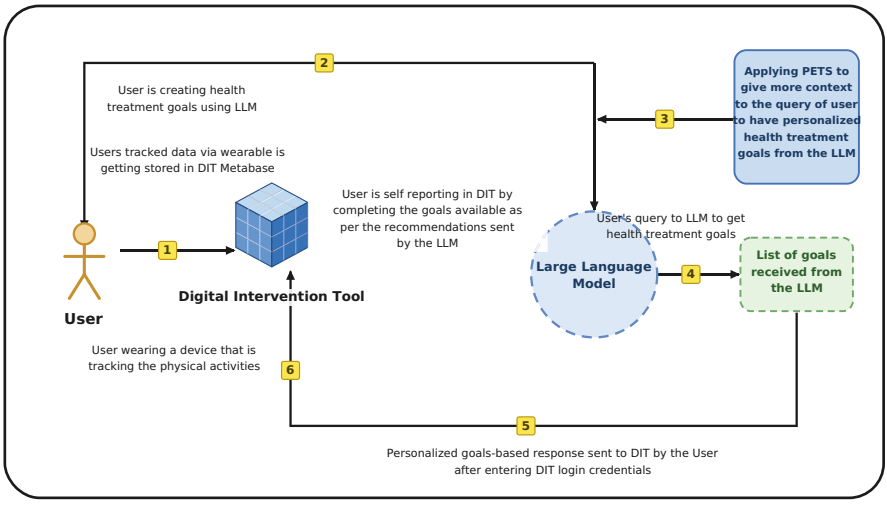


Fig. 2. Design of the system with prompt engineering to create personalized health treatment goals using LLMs.

Algorithm 2: LLM-based Goal Creation with Prompt Engineering

Input: userInput, userAttributes, digitalInterventionTool
Output: Personalized Goals

$basePrompt \leftarrow userInput$
$contextInfo \leftarrow userAttributes$
begin
    // Step 1: Merge basePrompt with contextInfo using prompt engineering.
    $engineeredPrompt \leftarrow Merge(basePrompt, contextInfo)$
    // Step 2: Query the LLM with the enhanced prompt.
    $personalizedGoals \leftarrow LLM_Generate(engineeredPrompt)$
    // Step 3: Store/display goals in the digital intervention tool.
    $digitalInterventionTool.StoreGoals(personalizedGoals)$
    // Step 4: Wait for user's self-reports or feedback.
    $userReport \leftarrow digitalInterventionTool.WaitForUserReport()$
end
return *personalizedGoals* // output return

The system augments user input with additional contextual information to enrich the prompt sent to the LLM, enabling more tailored goal generation than the previous design; however, this process remains a one-time step without continuous adaptation.

Benefits and Limitations: Prompt engineering techniques are now used to enrich user queries to LLMs enabling more personalized responses. Users can generate their goals and selfreport via a digital intervention tool and these techniques make the resulting goals more specific to individual attributes. However, personalization across different user contexts such as children, young adults, elderly remains limited and the system does not address iterative refinement of deeply personalized health goals. HDT is needed to model each user dynamically capturing their details and context and thereby optimizing LLM based generation of tailored and refined health treatment goals.

Design 3: This design also states the design variant for the studies that have been conducted in the existing works such as [26–28]. For instance, in the study of Hendrik et al., [26] instead of using a HDT to get all the relevant information about the study participants they used some coaching procedures (i.e., hiring some coaches). Participants were provided with watches pre-installed with trackers, which were used to initiate the study. The researchers aimed to help users achieve certain health goals over a defined period. However, they did not provide feedback on the initial goals, which could have allowed users to adjust their targets based on Philip's analysis of their data. Likewise, some other studies have followed the same pattern of using wearable data where essential digital behavior markers were extracted from the wearable. [29]. The design alternative 3 can be seen in Fig. 3.

Algorithm 3: Goal Creation with Human Digital Twin (HDT)

Input: user_Input, Human Digital Twin (HDT), digitalInterventionTool
Output: Initial Personal Goals

$basePrompt \leftarrow user_Input$
begin
    `// Step 1: Fetch current context from the HDT.`
    $currentContext \leftarrow HDT.FetchLatestContext()$
    `// Step 2: Combine user prompt with HDT context.`
    $combinedPrompt \leftarrow Combine(basePrompt, currentContext)$
    `// Step 3: Generate personalized goals via the LLM.`
    $initialPersonalizedGoals \leftarrow LLM_Generate(combinedPrompt)$
    `// Step 4: Store goals in the intervention tool.`
    $digitalInterventionTool.StoreGoals(initialPersonalizedGoals)$
    `// Step 5: Update HDT history.`
    $userReport \leftarrow digitalInterventionTool.WaitForUserReport()$
    $HDT.UpdateHistory(userReport)$
end
return $initialPersonalizedGoals$

This design uses a HDT to provide real time context resulting in a richer initial prompt for the LLM. While the goal creation is more personalized compared to Designs 1 and 2, it still lacks a mechanism for continuous updating and refinement over time.

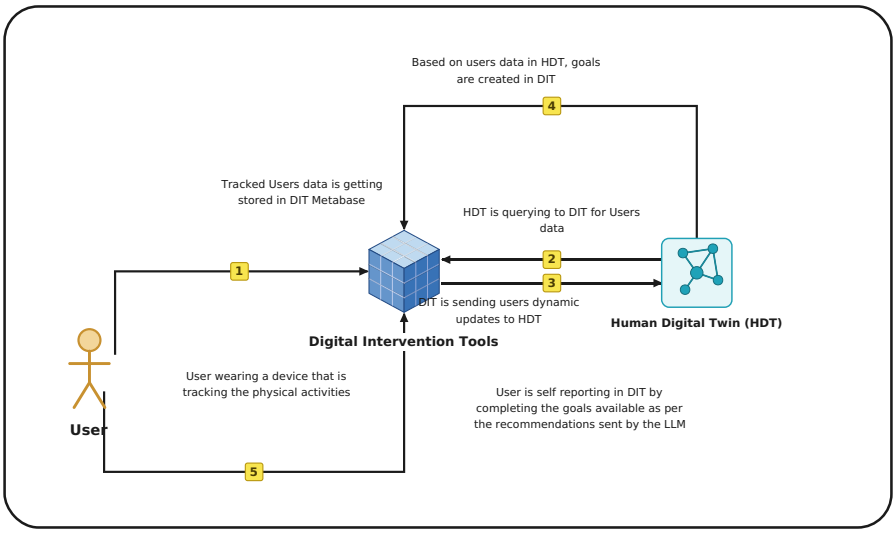


Fig. 3. Design of the system backed by HDT to create personalized health treatment goals.

Benefits and Limitations: This study becomes crucial for guiding future research by providing insights into the use of HDTs and potential improvements in outcomes. The methods and variables considered here form a foundation for subsequent studies such as [26,30], and [25]. For example [26], highlights HDTs as a key component for designing healthy lifestyle interventions and Fig. 3 illustrates how modeling participants physical behaviors with HDTs could improve outcomes. Current approaches still lack a comprehensive integration of LLMs and prompt engineering for personalized health goal creation. Additionally, these studies relied on passive trackers, missing opportunities to engage participants through gamified, interactive platforms shown to boost engagement and behavior change. Finally, a work [26] did not incorporate user feedback on initial goals which could have enabled dynamic refinement and more effective interventions.

4 A Deeply Personalized mHealth Platform Design Proposal

The design presented in Fig. 4 is synthesized from a systematic review of existing mHealth system design as discussed in Sect. 3.3. With this design, we essentially systematically address the research question 3. Here, we propose a deeply personalized mHealth platform that integrates emerging technologies to support the automated creation and refinement of personalized health treatment goals for diverse user groups. Central to this design is the integration of LLMs, HDTs and digital intervention tools to enable scalable, adaptive and context aware goal management.

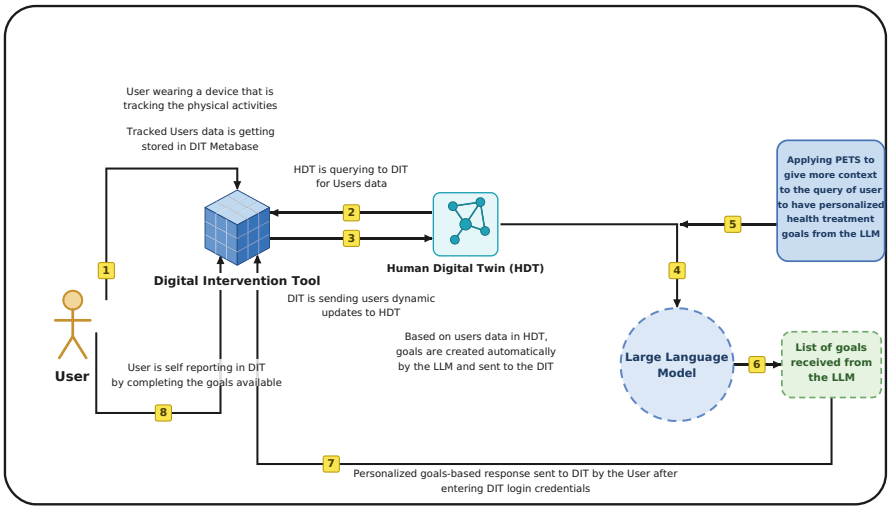


Fig. 4. Proposed design for a deeply personalized mHealth platform backed by HDT and LLM to create and refine personalized health treatment goals.

In this architecture, the digital intervention tool represents an mHealth application that supports long term behavior change through personalized goal delivery and user self-reporting. LLMs are employed to generate measurable and personalized health treatment goals while HDTs continuously collect and maintain users contextual and historical data. Unlike, existing designs which largely rely on static or manually defined goals, the proposed system emphasizes dynamic goal creation and continuous refinement based on evolving user characteristics. A key innovation of this design is the automated feedback loop between the digital intervention tool and the HDT. User interactions such as selfreported activities and physical challenges are captured by the intervention tool and synchronized with the HDT in real time. The HDT analyzes these longitudinal data to update the user profile and provides structured context to the LLM, which then refines existing goals or generates new ones accordingly. This bidirectional synchronization enables goals to evolve over time without manual intervention ensuring sustained personalization.

Goal Refinement Over Time. The proposed design supports continuous goal refinement by leveraging the HDT as a persistent model of user behavior and context. As users interact with the digital intervention tool their activity history is stored and analyzed to identify behavioral patterns and changes. These insights are supplied to the LLM enabling it to adapt goals over time based on historical performance and contextual dynamics. This iterative process ensures that health treatment goals remain relevant, measurable and aligned with users' evolving needs, supporting long term behavior change.

Models in Human Digital Twin. The proposed HDT as (shown in Fig. 4) includes two core models: a physical activity model and a player type model (Figs. 5 and 6) capturing behavioral and motivational characteristics relevant to lifestyle-related diseases. The HDT combines user data and a user model with physical activity collected via wearables integrated into the digital intervention tool. Continuous querying of this tool allows dynamic modeling of user behavior over time. Prompt engineering techniques guide large language model outputs by embedding HDT derived contextual information enabling precise, personalized, and context aware health goals. This process confirms that interventions are continuously informed by users evolving behavioral patterns and supporting adaptive and individualized care.

Algorithm 4: Deeply Personalized mHealth Platform (Iterative Goal Refinement)

Input: userInput, HDT, digitalInterventionTool, refinementInterval
Output: Personalized Refined Goals

1 $basePrompt \leftarrow userInput$
2 **begin**
    `// Step 1: Initialize HDT with any known user data.`
3 $HDT.Initialize()$
    `// Step 2: Generate initial goals using the latest context from HDT.`
4 $initialContext \leftarrow HDT.FetchLatestContext()$
5 $prompt_0 \leftarrow Combine(basePrompt, initialContext)$
6 $currentGoals \leftarrow LLM_Generate(prompt_0)$
7 $digitalInterventionTool.StoreGoals(currentGoals)$
8 $refinedPersonalizedGoals \leftarrow currentGoals$
9 **while** $digitalInterventionTool.UserActive()$ **do**
        `// Step a: Fetch updated context from HDT.`
10 $newContext \leftarrow HDT.FetchLatestContext()$
        `// Step b: Merge updated context with latest goals.`
11 $engineeredPrompt \leftarrow Merge(currentGoals, newContext)$
        `// Step c: Query the LLM to refine the goals.`
12 $refinedPersonalizedGoals \leftarrow LLM_Generate(engineeredPrompt)$
        `// Step d: Store/display the refined goals.`
13 $digitalInterventionTool.StoreGoals(refinedPersonalizedGoals)$
        `// Step e: Wait for user feedback and update HDT history.`
14 $userReport \leftarrow digitalInterventionTool.WaitForUserReport()$
15 $HDT.UpdateHistory(userReport)$
        `// Step f: Wait until the next refinement interval.`
16 $Wait(refinementInterval)$
        `// Step g: Update currentGoals for the next iteration.`
17 $currentGoals \leftarrow refinedPersonalizedGoals$
18 **end**
19 **end**
20 **return** $refinedPersonalizedGoals$

Explanation:

Dynamic and Continuous Personalization: Unlike the previous designs, Algorithm 4 uses an iterative loop (lines 5–12) to continuously refine goals based on new data from the HDT.

Integration of HDT and LLM with Prompt Engineering: The algorithm merges the users initial input with the evolving context from the HDT and refines the prompt with every iteration. This combined approach leverages the strengths of both HDT (real time data) and LLM (flexible, context aware text generation).

Robust Feedback Loop: The loop not only updates goals with the latest contextual data but also incorporates user feedback (line 17) to further improve the personalization process.
Scalability and Adaptability: The system continually adjusts the goals based on the dynamic user context, it scales well for diverse populations and rapidly changing scenarios.
Alignment with Behavior Change Theories: The iterative refinement process aligns with behavioral models (e.g., Transtheoretical Model) that emphasize the need for ongoing support and adaptation to maintain long term behavior change.

4.1 Decomposition of the Human Digital Twin

The HDT essentially contains two significant models that will provide the latest contextual information to the LLM to create personalized health treatment goals. The first model is the Physical activity model that can be seen in Fig. 5. The data about the users physical activity is stored in the digital intervention tool as the wearable is tracking the user.

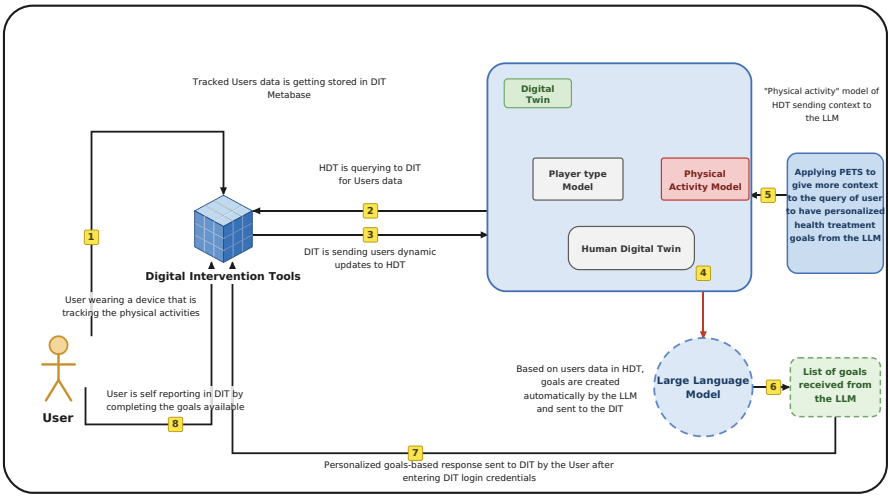


Fig. 5. Physical activity model providing contextual physical activity and location data for creating and refining personalized goals.

From the user type model as shown in Fig. 6 the actual type of the user is being fetched from the digital intervention tool to the human digital twin. This model is of great importance due to the fact that there can be various users such as children, young adults or elderly people, and through this model, HDT will make sure that the goals are created and refined for a precise user to avoid the "one-size-fits-all" issue.

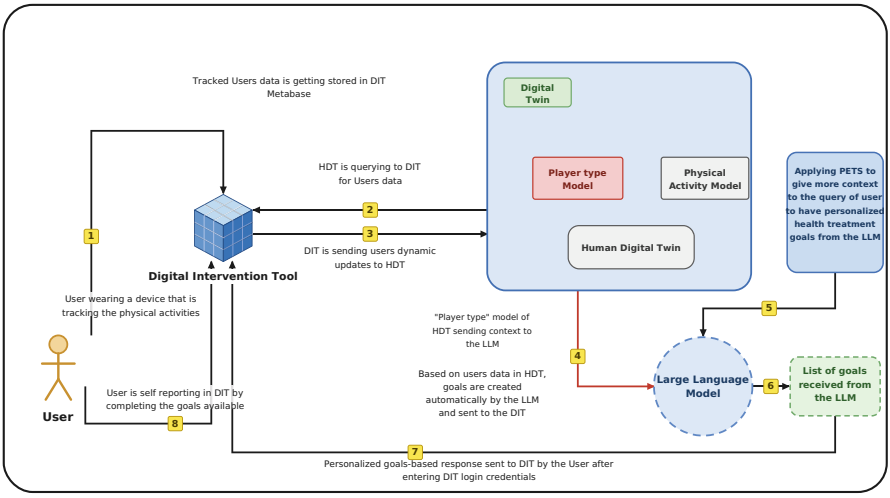


Fig. 6. Player type model providing contextual user activity data for creating and refining personalized goals.

Benefits and Limitations. In contrast to the designs in Figs. 1, 2 and 3 the deeply personalized mHealth platform shown in Fig. 4 enables the creation and iterative refinement of personalized health goals. This approach improves efficiency reducing cost, time and scalability limitations of manual, static goal setting and enhances effectiveness by leveraging human digital twins and large language models. However integrating these techniques raises important concerns including privacy risks from LLM responses the limited real-time knowledge of pre-trained models and the need to ensure domain specific accuracy and unbiased outputs when updating digital twins with dynamic user data.

5 Discussion, Limitations and Future Work

Various intervention strategies have been proposed in the literature to prompt user behaviors. Over time there has been more focus on creating recommendation systems that can leverage personalization in various users for different contexts, i.e., in making health treatment goals. In this paper we have stated the implications of state-of-the-art tools and technologies that ought to be disruptive. We illustrated using LLMs backed by the novel prompting engineering techniques to have precise/specific responses from the pre-trained large language model. We stressed using prompt engineering techniques when queering a LLM to have personalized recommendations for any lifestyle disease. Similarly, to refine earlier created health treatment goals by adding HDTs is also deemed quite relevant and beneficial since HDT could have dynamic updates about the physical entity and based on these updated goals can be modified over time. To have complete contextual information for each user, we have advocated using digital twins that

can get dynamic information from the actual entity and model/replicate the behaviors of the real users. One of the limitations of this research is that, we did not perform an exhaustive systematic literature review. We have mainly considered two prominent criteria in evaluating the existing literature: efficiency and effectiveness. We did not perform any sort of implementation for the proposed design variant rather provided design variants together with LLMs and DTs to create and refine health treatment goals at scale for various users.

6 Conclusion

This study examined limitations in existing approaches to creating and refining health treatment goals at scale, particularly in terms of efficiency and effectiveness. We explored the potential of emerging technologies, including LLMs and HDTs, to address these challenges. By combining these technologies, we proposed a novel design for deeply personalized mHealth platforms that enable scalable, adaptive, and individualized health goal management for lifestyle disease prevention. Future work will focus on empirically evaluating the proposed design and further investigating the privacy and security implications of integrating LLMs with digital twin technologies.

Acknowledgments. This study was conducted under the HealthyW8 project (funded by the European Union), European Health and Digital Executive Agency (HaDEA) under grant agreement No. 1010806. The funder had no influence on the content of this study. The authors also thank Pieter Van Gorp, Baris Ozkan, and Laura Genga for their assistance in bringing this research paper together.

References

1. Al-Maskari, F.: "Lifestyle diseases: An economic burden on the health services," UN Chronicle. Mag. United Nations **5**, 1–2 (2010)
2. Kelli, H.M., Witbrodt, B., Shah, A.: The future of mobile health applications and devices in cardiovascular health. Eur. Med. J. Innov. **2017**, 92 (2017)
3. Debon, R., Coleone, J.D., Bellei, E.A., De Marchi, A.C.B.: Mobile health applications for chronic diseases: A systematic review of features for lifestyle improvement. Diabetes Metab. Synd. Clin. Res. Rev. **13**(4), 2507–2512 (2019)
4. Ebert, D.D., Cuijpers, P., Muñoz, R.F., Baumeister, H.: Prevention of mental health disorders using internet-and mobile-based interventions: a narrative review and recommendations for future research. Front. Psych. **8**, 116 (2017)
5. Forget, A., Chiasson, S., van Oorschot, P.C., Biddle, R.: Persuasion for Stronger Passwords: Motivation and Pilot Study. In: Oinas-Kukkonen, H., Hasle, P., Harjumaa, M., Segerståhl, K., Øhrstrøm, P. (eds.) PERSUASIVE 2008. LNCS, vol. 5033, pp. 140–150. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-68504-3_13
6. White, N.D., Bautista, V., Lenz, T., Cosimano, A.: Using the smart-est goals in lifestyle medicine prescription. Am. J. Lifestyle Med. **14**(3), 271–273 (2020)
7. Achiam, J., et al.: Gpt-4 technical report. arXiv preprint arXiv:2303.08774 (2023)

8. Stegmann, M., Brandenbarg, D., Reyners, A., Van Geffen, W., Hiltermann, T., Berendsen, A.: Treatment goals and changes over time in older patients with non-curable cancer. Support. Care Cancer **29**(7), 3849–3856 (2021)
9. Rumale, A., Desai, M., Asadinia, M.: Stride: Sports tracking and injury detection using estimations. In: International Conference on Software Engineering of Emerging Technology, pp. 154–166. Springer, Cham (2025). https://doi.org/10.1007/978-3-032-08977-9_11
10. Zhao, W.X., et al.: A survey of large language models. arXiv preprint arXiv:2303.18223, vol. 1, no. 2, (2023)
11. Wickramasinghe, N., et al.: A vision for leveraging the concept of digital twins to support the provision of personalized cancer care. IEEE Internet Comput. **26**(5), 17–24 (2021)
12. Bailey, R.R.: Goal setting and action planning for health behavior change. Am. J. Lifestyle Med. **13**(6), 615–618 (2019)
13. Wang, J., et al.: Prompt engineering for healthcare: Methodologies and applications. arXiv preprint arXiv:2304.14670 (2023)
14. Saracco, R.: Digital twins: Bridging physical space and cyberspace. Computer **52**(12), 58–64 (2019)
15. Chen, H., Shao, H., Deng, X., Wang, L., Wang, X.: Comprehensive survey of the landscape of digital twin technologies and their diverse applications. In: Computer Modeling in Engineering & Sciences (CMES), vol. 138, no. 1 (2024)
16. Zafar, M.H.S.: Human digital twin: a review of business applications and future directions (2024)
17. Babar, Z., Khan, N.I., Hassnain, M., Filkov, V.: Open-source llms for technical q&a: Lessons from stackexchange. In: International Conference on Software Engineering of Emerging Technology, pp. 615–626. Springer, Cham (2025). https://doi.org/10.1007/978-3-032-08977-9_41
18. Shaikh, A.K., Abubakar, M.B., Adhikari, N.: Development of an artificial intelligence-driven live patient scheduling for modern healthcare. In: International Conference on Software Engineering of Emerging Technology, pp. 23–30, Springer, Cham (2025). https://doi.org/10.1007/978-3-032-08977-9_2
19. Josephson, S., Mohammad, A.F.: Large language models using retrieval augmented generation and prompt engineering for ai-driven music source separation: A literature review. In: International Conference on Software Engineering of Emerging Technology, pp. 114–124, Springer, Cham (2025). https://doi.org/10.1007/978-3-032-08977-9_9
20. James, L.J., Maessen, M., Genga, L., Montagne, B., Hagenaars, M.A., Van Gorp, P.M.E.: Towards augmenting mental health personnel with llm technology to provide more personalized and measurable treatment goals for patients with severe mental illnesses. In: *International Conference on Pervasive Computing Technologies for Healthcare*, pp. 186–200, Springer, Cham (2023). https://doi.org/10.1007/978-3-031-59717-6_13
21. Nhkum, C., Rahman, M.M., Ahmed, T., Kabir, M.F.: Comparative Fine-Tuning of GPT-2 on Question Answering and Dialogue Datasets for Medical Text Generation. In: Hussain, S., Khan, A.A., Abdul Basit Ur Rahim, M., Khan, S.U.R. (eds) SEET—Software Engineering for Emerging Technologies. SEET 2025. In: Communications in Computer and Information Science, vol. 2725, pp. 575–590. Springer, Cham (2026). https://doi.org/10.1007/978-3-032-08977-9_38
22. James, L.J., et al.: Evaluation of personalized treatment goals on engagement of smi patients with an mhealth app. In: 2022 IEEE International Conference on Bioinformatics and Biomedicine (BIBM), pp. 1568–1573. IEEE (2022)

23. Lee, U., et al.: Few-shot is enough: exploring chatgpt prompt engineering method for automatic question generation in english education. Educ. Inf. Technol. **29**(9), 11483–11515 (2024)
24. Meskó, B.: Prompt engineering as an important emerging skill for medical professionals: tutorial. J. Med. Internet Res. **25**, e50638 (2023)
25. Alessa, A., Al-Khalifa, H.: Towards designing a chatgpt conversational companion for elderly people. In: Proceedings of the 16th International Conference on Pervasive Technologies Related to Assistive Rnvironments, pp. 667–674 (2023)
26. Hendrikx, J., Ruijs, L.S., Cox, L.G., Lemmens, P.M., Schuijers, E.G., Goris, A.H., et al.: Clinical evaluation of the measurement performance of the philips health watch: a within-person comparative study. JMIR Mhealth Uhealth **5**(2), e6893 (2017)
27. Bowman, S.R., et al.: Measuring progress on scalable oversight for large language models. arXiv preprint arXiv:2211.03540 (2022)
28. Bendig, E., Bauereiss, N., Ebert, D.D., Snoek, F., Andersson, G., Baumeister, H.: Internet-based interventions in chronic somatic disease. Dtsch. Arztebl. Int. **115**(40), 659 (2018)
29. Cook, D.J., Strickland, M., Schmitter-Edgecombe, M.: Detecting smartwatch-based behavior change in response to a multi-domain brain health intervention. ACM Trans. Comput. Healthc. (HEALTH) **3**(3), 1–18 (2022)
30. Albers, N., Hizli, B., Scheltinga, B.L., Meijer, E., Brinkman, W.-P.: Setting physical activity goals with a virtual coach: vicarious experiences, personalization and acceptance. J. Med. Syst. **47**(1), 15 (2023)

Self-adaptive Swarm-Driven Big Data Pipelines for Dynamic Workload Optimization in Spark Environments

Zulfiqar Ali[1], Israr Ur Rehman[2], Kashif Manzer[3], and Muhammad Abdul Basit Ur Rahim[3(✉)]

[1] NFC Institute of Engineering & Fertilizer Research, Faisalabad, Pakistan
zulfiqar.ali@iefr.edu.pk

[2] Pak-Austria Fachhochschule Institute of Applied Sciences and Technology, Haripur, Pakistan
israr.rehman@paf-iast.edu.pk

[3] Department of Computer Engineering and Computer Science, California State University Long Beach, Long Beach, USA
kashif.manzer01@student.csulb.edu, m.basit@csulb.edu

Abstract. The exponential growth of data-intensive applications has established Apache Spark as a leading platform for large-scale distributed data processing. Despite its widespread adoption, Spark-based big data pipelines typically rely on static configuration parameters that are unable to adapt to dynamic and heterogeneous workload conditions. This limitation often results in performance degradation, increased execution latency, and inefficient utilization of computational resources, particularly in streaming and real-time analytics environments. To address these challenges, this paper proposes a self-adaptive swarm-driven optimization framework for dynamic workload optimization in Spark environments. The proposed approach integrates Swarm Intelligence techniques, specifically Particle Swarm Optimization (PSO) and Ant Colony Optimization (ACO), with a real-time monitoring and feedback mechanism to automatically tune critical Spark configuration parameters, including executor memory, CPU cores, and data partitioning strategies. The framework continuously analyzes runtime performance metrics and adaptively refines system configurations in response to changing workload patterns. Extensive experimental evaluation using standard big data benchmarks demonstrates that the proposed framework significantly improves system performance compared to static and rule-based tuning approaches. The results show a 40% reduction in execution latency, an increase in throughput to 14,800 records/s, and an improvement in CPU utilization from 55% to 84%. The main contributions of this work are threefold: (i) the development of a novel self-adaptive swarm-based optimization model for Spark-based big data pipelines, (ii) the integration of real-time feedback-driven configuration tuning within distributed processing environments, and (iii) comprehensive empirical validation demonstrating scalability, robustness, and performance gains. The proposed framework provides an intelligent, adaptive, and scalable solu-

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 169–188, 2027.
https://doi.org/10.1007/978-3-032-39763-8_10

tion for optimizing next-generation big data systems operating under dynamic workload conditions.

Keywords: Big Data Analytics · Apache Spark · Swarm Intelligence · Particle Swarm Optimization (PSO) · Ant Colony Optimization (ACO) · Dynamic Workload Optimization · Distributed Systems · Adaptive Systems

1 Introduction

The rapid proliferation of data-intensive applications across domains such as finance, healthcare, and IoT has led to an unprecedented growth in data volume, velocity, and variety. This evolution has driven the adoption of large-scale distributed data processing frameworks, among which Apache Spark has emerged as a dominant platform due to its in-memory computation capabilities and support for batch and stream processing [30]. Complementary ecosystems such as Apache Hadoop provide scalable storage and resource management, forming the backbone of modern big data infrastructures [28]. Despite these advancements, efficient resource management remains a critical challenge in distributed computing environments. Spark applications expose numerous configuration parameters (e.g., executor memory, number of cores, shuffle partitions), which significantly influence performance [24]. Improper tuning of these parameters can lead to resource underutilization, excessive execution latency, and increased operational costs. Consequently, there is a growing need for intelligent and adaptive optimization techniques that can dynamically manage resources in response to changing workload conditions. Traditional Spark deployments largely rely on static configuration settings, which are determined prior to execution and remain unchanged during runtime. While such configurations may perform adequately under stable conditions, they fail to adapt effectively in dynamic environments characterized by:

Workload Variability: Data size, query complexity, and user demand fluctuate over time, leading to inconsistent performance [27]. **Streaming and Real-Time Data Processing:** Continuous data ingestion requires rapid adaptation to changing data rates and patterns [14]. **Heterogeneous Cluster Environments:** Variations in hardware resources (CPU, memory, network bandwidth) introduce additional complexity in resource allocation [17]. As a result, static tuning approaches often lead to suboptimal system performance, including increased job execution time, inefficient resource utilization, and degraded throughput.

The contemporary literature reveals that the existing approaches for optimizing Spark performance primarily fall into two categories:

Manual Tuning: System administrators rely on domain expertise and trial-and-error methods to configure parameters, which is time-consuming and error-prone [7]. **Rule-Based or h=Heuristic Tuning:** Predefined rules or cost models are

used to guide configuration decisions; however, these methods lack adaptability and fail to generalize across diverse workloads [4].

Recent studies have explored machine learning-based optimization techniques; however, many of these approaches suffer from limitations such as high training overhead, lack of real-time adaptability, and poor scalability in large distributed environments[5,20]. Moreover, there is limited research on integrating Swarm Intelligence techniques with real-time feedback mechanisms for dynamic optimization in Spark-based big data pipelines.

Therefore, there exists a clear need for a self-adaptive, scalable, and intelligent optimization framework capable of continuously adjusting system configurations in response to evolving workload conditions. To address the aforementioned challenges, this paper proposes a self-adaptive swarm-driven optimization framework for dynamic workload optimization in Spark environments. The main contributions of this work are summarized as follows:

A Novel Swarm-Driven Optimization Framework: We design an adaptive optimization model based on Swarm Intelligence techniques (PSO and ACO) to dynamically tune Spark configuration parameters. Real-time parameter tuning mechanism: The proposed system integrates a runtime monitoring and feedback loop to continuously observe system performance and update configurations accordingly. **Hybrid Optimization Strategy:** A combination of swarm-based global search and feedback-driven local adaptation is employed to achieve efficient and robust optimization. **Comprehensive Experimental Validation:** The effectiveness of the proposed framework is evaluated using standard big data benchmarks (e.g., HiBench, TPC-DS), demonstrating significant improvements in latency, throughput, and resource utilization compared to baseline approaches.

The paper is organized as Sect. 2, provide the contemporary literature review and highlighted research gaps that are investigated under this research work. Section 3, explains the proposed framework in this research; Sect. 4 explains experimental setup while Sect. 5 and Sect. 6 provide Results and Discussion. Finally Sect. 7 concludes the research invested in this paper.

2 Related Work

Apache Spark has become a de facto standard for large-scale data processing due to its in-memory computation model and unified support for batch and streaming workloads. However, Spark performance is highly sensitive to configuration parameters such as executor memory, number of cores, parallelism level, and shuffle partitions [30]. Traditional optimization approaches primarily rely on static tuning, where parameters are manually set prior to execution based on prior experience or heuristic guidelines. While effective for stable workloads, static configurations fail to adapt to runtime variability, leading to performance degradation under changing conditions. To address these limitations, several automated tuning approaches have been proposed. Systems such as SparkTune employ cost-based models to guide parameter selection [11], while more recent

solutions like LOCAT leverage online learning to dynamically adjust configurations during execution [29]. Similarly, generalized tuning frameworks have explored Bayesian optimization and machine learning techniques to improve Spark performance [16]. Although these methods reduce manual effort, they often incur significant computational overhead and may require multiple training iterations. Furthermore, Spark's built-in Dynamic Resource Allocation (DRA) mechanism provides limited adaptability by scaling executors based on workload demand, but it does not comprehensively optimize other critical parameters such as memory allocation, data partitioning, and shuffle behavior [20]. Consequently, there remains a need for more holistic and adaptive optimization strategies.

Swarm Intelligence (SI) has emerged as a powerful paradigm for solving complex optimization problems through collective behavior inspired by natural systems. Among SI techniques, Particle Swarm Optimization (PSO) and Ant Colony Optimization (ACO) are widely adopted due to their simplicity, scalability, and ability to handle nonlinear and multi-dimensional optimization problems [25]. PSO is particularly effective for continuous optimization tasks, whereas ACO excels in discrete and combinatorial problems such as scheduling and routing. In recent years, SI techniques have been successfully applied in cloud computing and distributed systems for resource allocation, load balancing, and task scheduling. For instance, PSO-based approaches have demonstrated improved resource utilization and reduced execution time in cloud environments [3], while ACO-based models have been employed for QoS-aware scheduling and workflow optimization [21]. Despite these advancements, the application of SI techniques to fine-grained Spark configuration tuning remains limited. Existing studies primarily focus on high-level scheduling decisions rather than integrating SI with runtime monitoring and adaptive feedback in big data frameworks. This gap highlights the potential of SI for developing intelligent and adaptive optimization mechanisms tailored to Spark environments.

Big data pipelines encompass multiple stages, including data ingestion, transformation, processing, and storage, typically implemented using frameworks such as Hadoop, Spark, and Kafka. Optimization of these pipelines is critical for achieving high performance and scalability. Existing approaches focus on various aspects such as query optimization, task scheduling, caching strategies, and resource allocation. For example, Spark SQL optimization techniques improve query execution through logical and physical plan optimization, while streaming frameworks rely on micro-batch processing to handle continuous data flows [30]. Recent research has explored automated parameter tuning for stream processing systems using feedback-driven approaches. For instance, Bilal and Canini [7] proposed an automated tuning mechanism for stream processing engines, demonstrating the effectiveness of feedback-based optimization. Similarly, online tuning frameworks for Spark have attempted to dynamically adjust configurations using workload-aware models [1]. However, these approaches often suffer from limitations such as high search space complexity, lack of scalability, and limited support for multi-objective optimization. Moreover, most existing solutions focus on either batch or streaming workloads independently, without address-

ing the unified optimization of hybrid pipelines. The contemporary literature survey on optimizing spark data pipelines is provided in [2,10,18,23,26]. Dileep Kumar Ghattamaneni and Rao Boinapalli Narasimha provided the optimization approaches based on Machine Learning by exploiting Apache Spark for real time big data frameworks in [13]. Narayanan et.al investigated Generative AI for Data Pipeline Optimization in [19]. Pulivarthy et.al proposed a Real Time Data Pipeline in [22]. Nagabhushanam Bheemisetty investigated Next-Gen Data Ecosystems in [6] and Fan Ding provided Adaptive Cross-platform Scheduling Framework for NWP in Hybrid Clouds in [9].

Based on the survey of cotemporary literature, several key research gaps are highlighted those demands research investigation. This research work focuses on following few subset of research gaps that listed here.

- **Lack of real-time adaptation:** Many existing optimization techniques operate offline or require repeated executions, making them unsuitable for dynamic and real-time environments [12].
- **Limited integration with Spark streaming:** Most studies focus on batch processing, whereas streaming applications demand continuous and adaptive optimization [7].
- **Absence of multi-objective optimization:** Current approaches typically optimize a single performance metric (e.g., execution time), while real-world systems require simultaneous optimization of latency, throughput, resource utilization, and cost.
- **Underutilization of Swarm Intelligence in Spark tuning:** Although SI techniques have been widely applied in cloud and distributed systems, their integration with Spark runtime environments for adaptive configuration tuning remains underexplored [3,21,25].

To address these gaps, this work proposes a self-adaptive swarm-driven optimization framework that combines the global search capability of Swarm Intelligence with real-time monitoring and feedback learning to dynamically optimize Spark-based big data pipelines.

3 System Architecture

This section presents the proposed PSO-driven self-adaptive big data pipeline optimization architecture, designed to enable real-time, intelligent, and dynamic tuning of Apache Spark configurations. The Fig. 1 presents an enhanced self-adaptive swarm-driven optimization framework for dynamic workload optimization in Apache Spark. In contrast to prior approaches—such as default/static configurations, rule-based tuning, and learning-based auto-tuners (e.g., SparkTune [11], LOCAT [29], Bayesian/online tuning [16])—the proposed method integrates Swarm Intelligence (PSO) with real-time monitoring and feedback control to achieve continuous, multi-objective optimization. The architecture integrates data ingestion, distributed processing, real-time monitoring, and swarm-based optimization within a closed-loop control framework to address the limitations of static and rule-based tuning approaches.

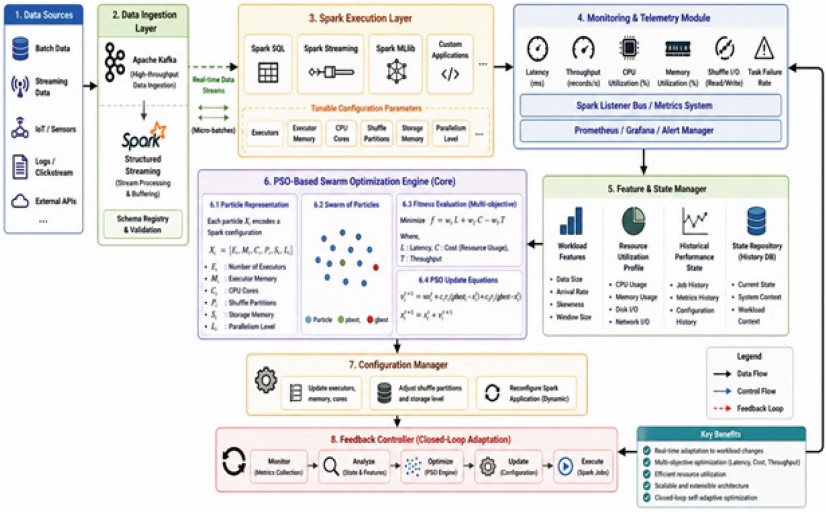


Fig. 1. PSO-driven self-adaptive big data pipeline optimization architecture.

3.1 Workflow of Proposed PSO-Driven Self-adaptive Optimization Framework

Figure 2 illustrates the overall workflow of the proposed PSO-driven self-adaptive optimization framework for dynamic workload optimization in Apache Spark environments. The framework follows a closed-loop architecture that continuously monitors system performance and dynamically adjusts Spark configuration parameters to improve execution efficiency. The workflow begins with diverse data sources, including batch datasets, streaming data, IoT streams, and system logs. These data streams are collected through the Data Ingestion Layer, implemented using Apache Kafka, which provides reliable and scalable data delivery to the processing environment. The ingested data is then processed by the Spark Processing Layer, where distributed computation is performed using Spark SQL, Spark Streaming, and other Spark components. During execution, the Runtime Monitoring Module continuously collects performance metrics such as latency, throughput, CPU utilization, memory consumption, and shuffle I/O statistics. These metrics are analyzed by the Feature Extraction Module, which derives workload characteristics and resource utilization patterns that represent the current system state. The extracted features are supplied to the PSO Optimization Engine, which serves as the core intelligence component of the framework. The PSO algorithm evaluates candidate Spark configurations, updates particle positions and velocities, and identifies near-optimal parameter settings based on a multi-objective fitness function. The optimized configuration parameters are then applied through the Configuration Update Module, which dynamically adjusts executor allocation, memory settings, CPU cores, and partition strategies. Subsequently, the Spark Reconfiguration Module applies the new settings to

the running Spark environment, enabling adaptive resource management without manual intervention. The resulting system performance is assessed by the Performance Evaluation Module, which measures latency, throughput, resource utilization, and cost efficiency. Finally, a Feedback Control Loop continuously feeds the evaluation results back to the monitoring stage, forming a closed-loop optimization process. This feedback mechanism enables the framework to adapt to workload fluctuations, resource contention, and changing execution conditions in real time. As a result, the proposed framework achieves improved scalability, reduced latency, enhanced throughput, and more efficient resource utilization compared with conventional static and rule-based Spark tuning approaches.

3.2 System Components

The architecture is composed of several interconnected components, each contributing to real-time monitoring, intelligent optimization, and adaptive execution.

Data Sources. The data sources represent the origin of workloads processed by the system. These include batch datasets, real-time streaming data, IoT streams, and system logs. Such sources generate dynamic and heterogeneous workloads characterized by varying data volumes, arrival rates, and computational requirements. This variability introduces significant challenges for resource management and necessitates adaptive optimization mechanisms.

Data Ingestion Layer. The data ingestion layer is responsible for reliably acquiring and delivering data to the processing system. It is implemented using Apache Kafka for distributed messaging and Spark Structured Streaming for real-time processing. This layer supports continuous data ingestion, buffering, and fault tolerance, ensuring stable data flow under high-throughput conditions. Unlike static pipelines, it enables seamless integration of batch and streaming workloads, allowing the system to respond to continuous workload variability.

Spark Execution Layer. The Spark execution layer forms the core processing unit of the system, built on Apache Spark and leveraging components such as Spark SQL, Spark Streaming, and MLlib. The performance of this layer depends heavily on several tunable configuration parameters, including: i. Number of executors ii. Executor memory iii. CPU cores iv. Shuffle partitions In conventional systems, these parameters are statically configured, leading to inefficient resource utilization and suboptimal performance under dynamic workloads [20]. The proposed framework overcomes this limitation by enabling dynamic configuration tuning through the PSO-based optimization engine.

Monitoring and Telemetry Module. The monitoring and telemetry module continuously collects runtime metrics from the Spark execution layer. These metrics include:

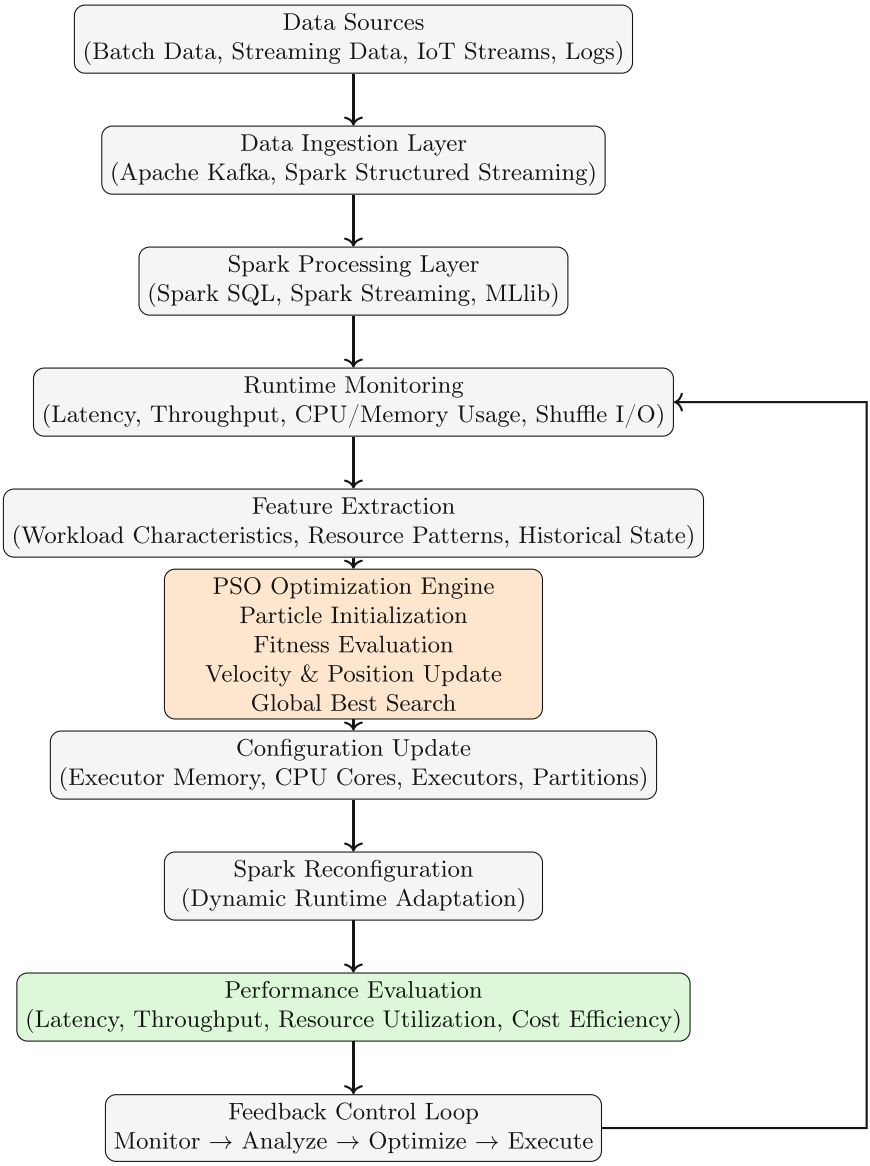


Fig. 2. Research Design and Workflow of the Proposed PSO-Driven Self-Adaptive Optimization Framework for Dynamic Workload Optimization in Apache Spark Environments.

- Latency (execution time)
- Throughput (records processed per second)
- CPU and memory utilization
- Shuffle I/O statistics

This module is implemented using mechanisms such as the Spark Listener Bus and built-in metrics system, enabling fine-grained observation of system performance. Unlike offline tuning approaches, it provides real-time observability, allowing immediate feedback for optimization decisions [5].

Feature and State Manager. The feature and state manager transforms raw telemetry data into structured features that capture system behavior. These include:

- Workload characteristics (data size, skewness, arrival rate)
- Resource utilization patterns

Additionally, it maintains historical system state information, enabling the framework to capture temporal workload patterns. These features are used as inputs to the PSO engine, enabling context-aware and data-driven optimization.

PSO-Based Swarm Optimization Engine. The main contribution in this research is the PSO-based optimization engine that is the core intelligence layer responsible for determining optimal Spark configurations. Each particle represents a candidate configuration by exploiting basic principle of particle swarm optimization in Eq. (1).

$$X_i = [E_i, M_i, C_i, P_i] \tag{1}$$

where E_i, M_i, C_i, and P_i denote executors, memory, CPU cores, and partitions, respectively in Eq. (1). The fitness of each configuration is evaluated using a multi-objective function shown in the Eq. (2).

$$F = w_1 L + w_2 C - w_3 T \tag{2}$$

where L is latency, C is resource cost, and T is throughput. The Particles update their positions using basic PSO equations provide in the Eqs. (3) and (4) for velocity and position respectively.

$$v_i^{t+1} = wv_i^t + c_1 r_1 (pbest_i - x_i^t) + c_2 r_2 (gbest - x_i^t) \tag{3}$$

$$x_i^{t+1} = x_i^t + v_i^{t+1} \tag{4}$$

This iterative process enables efficient exploration and convergence toward optimal configurations. The comparative empirical results to baseline approaches, proposed PSO based model provides faster convergence, lower computational overhead, and scalability for high-dimensional optimization problems.

Configuration Manager. The Configuration Manager applies optimized parameters generated by the PSO engine to the Spark runtime. It dynamically updates:

- Executor allocation
- Memory configuration
- Partition settings

This component enables online reconfiguration, allowing the system to adapt without interrupting execution. It ensures that updates remain within system constraints, thereby maintaining stability while improving performance.

Feedback Controller (Closed-Loop System). The Feedback Controller implements a closed-loop control mechanism that drives continuous system adaptation. The process follows:

$$\text{Monitor} \rightarrow \text{Analyze} \rightarrow \text{Optimize (PSO)} \rightarrow \text{Execute}$$

This loop ensures that system performance is continuously evaluated and improved. It enables to proposed framework to respond effectively to

- Workload spikes
- Data skew
- Resource contention

By enabling continuous learning and adaptation, the feedback controller ensures robust, stable, and optimal system performance over time.

4 Experimental Setup

4.1 Experimental Environment

The proposed PSO-driven self-adaptive Spark optimization framework is evaluated in a distributed big data environment using Apache Spark [30], Kafka [15], Hadoop HDFS [8], and monitoring tools. Apache Spark is selected because it provides unified support for batch processing, SQL analytics, machine learning workloads, and stream processing, while exposing several performance-sensitive configuration parameters such as memory, cores, partitions, and shuffle settings. Spark also provides official performance tuning and monitoring support, which is suitable for runtime optimization experiments. The experimental environment is presented in the Table 1.

Apache Hadoop 3.3.6 is used for distributed storage and cluster-level data management, while Kafka is used to generate and control streaming workloads. Hadoop 3.3.6 is a stable Hadoop 3.3.x release documented by Apache.

Table 1. Cluster and Software Configuration

Component	Configuration
Spark version	Apache Spark 3.5.1
Hadoop version	Apache Hadoop 3.3.6 / HDFS
Kafka	Apache Kafka for streaming ingestion
Cluster size	1 master node + 4 worker nodes
CPU per worker	8 cores
RAM per worker	32 GB
Storage	HDFS-based distributed storage
Operating system	Ubuntu Linux 22.04 LTS
Monitoring tools	Spark UI, Spark Listener Bus, Prometheus/Grafana
Programming language	Python / PySpark

Table 2. Experimental Datasets and Workloads

Dataset/Benchmark	Workload Type	Purpose
TPC-DS	Batch SQL analytics	Evaluates complex decision-support queries
HiBench	Big data benchmark workloads	Evaluates sorting, aggregation, machine learning, and streaming tasks
Synthetic Kafka	Real-time streaming stream	Evaluates dynamic workload variation and arrival-rate changes

4.2 Datasets and Workloads

To evaluate the proposed framework under diverse workload conditions, three types of workloads are used that are shown in the Table 2.

TPC-DS is used because it is a standard decision-support benchmark developed by the Transaction Processing Performance Council and is widely used for evaluating analytical query performance. HiBench-style workloads are included to test Spark behavior across representative big data workloads such as WordCount, Sort, SQL, PageRank, K-Means, and streaming tasks. For streaming experiments, Kafka producers generate synthetic event streams with variable arrival rates. The stream rate is varied to simulate workload spikes, bursty traffic, and unstable real-time data conditions.

4.3 Baseline Methods

The proposed PSO-driven framework is compared against the following baseline methods that are described in the Table 3.

The default Spark configuration represents a basic deployment scenario. Static tuning represents expert-driven manual configuration. Rule-based opti-

Table 3. Baseline Methods Used for Comparison

Baseline	Description
Default Spark configuration	Uses Spark's default parameter settings without manual optimization
Static tuning	Uses manually selected Spark parameters fixed before execution
Rule-based optimization	Applies predefined rules for memory, partitions, and executor allocation
Dynamic Resource Allocation	Uses Spark's built-in executor scaling mechanism
Proposed PSO-driven Model	Dynamically tunes Spark parameters using PSO and runtime feedback

mization applies fixed heuristic rules, while Spark Dynamic Resource Allocation adjusts executor allocation based on workload demand but does not fully optimize memory, partitioning, shuffle, and throughput trade-offs simultaneously.

4.4 Evaluation Metrics

The performance of the proposed PSO-driven self-adaptive optimization framework is evaluated using four key metrics that capture both system efficiency and resource effectiveness. These metrics provide a comprehensive assessment of execution performance, scalability, and cost-awareness in distributed Spark environments.

Execution Time/Latency. Execution time, commonly referred to as latency, measures the total time required to complete a Spark job. It is defined as the difference between the job completion time and the job start time. The Eq. (5), describes mathematically relation for latency computation of the proposed framework.

$$Latency = T_{finish} - T \tag{5}$$

where T_{start} and T_{finish} represent the start and end times of job execution, respectively. Lower latency indicates faster processing and improved system responsiveness. In large-scale distributed systems, minimizing latency is critical for achieving high-performance data processing, particularly in real-time and streaming applications.

Throughput. Throughput measures the amount of data processed by the system per unit time and reflects the overall processing capacity of the framework. The Eq. (6), describes mathematically relation for throughput computation of the proposed framework.

$$Throughput = \frac{\text{Data Processed}}{\text{Execution Time}} \tag{6}$$

For streaming workloads, throughput is typically measured in records per second, indicating how efficiently the system handles continuous data streams. Higher throughput values indicate better utilization of system resources and improved processing efficiency.

Resource Utilization. Resource utilization evaluates how effectively the system uses available computational resources, including CPU, memory, disk I/O, and network bandwidth. The Eq. (7), describes mathematically resource utilization.

$$\text{Resource Utilization} = \frac{\text{Used Resources}}{\text{Available Resources}} \times 100 \tag{7}$$

This metric provides insight into the efficiency of resource allocation. Higher and well-balanced utilization indicates that the system is effectively leveraging available resources, while low utilization suggests underuse and inefficiency. In distributed environments, balanced utilization across nodes is essential to avoid bottlenecks and ensure optimal performance.

Cost Efficiency. Cost efficiency measures the trade-off between system performance and resource consumption. It evaluates how effectively the system processes data relative to the cost of computational resources. It is defined in the Eq. (8)

$$\text{Cost Efficiency} = \frac{\text{Throughput}}{\text{CPU Cost} + \text{Memory Cost} + \text{Storage Cost}} \tag{8}$$

A higher cost efficiency value indicates that the system achieves greater processing performance with lower resource expenditure. This metric is particularly important in cloud-based environments, where resource usage directly translates into operational costs. Optimizing cost efficiency ensures that the system delivers high performance while maintaining economic feasibility.

5 Results and Discussion

5.1 Performance Comparison

Table 4 compares the proposed PSO-driven optimization framework with baseline methods across latency, throughput, CPU utilization, and cost efficiency. The results clearly show that the proposed method outperforms all baselines. In terms of latency, the proposed approach reduces execution time from 120 s to 72 s, achieving approximately 40% improvement over the default configuration. For throughput, it reaches 14,800 records/s, representing about 32% improvement compared to rule-based optimization.

Table 4. Performance Comparison of Proposed Method vs Baselines

Method	Latency ↓ (sec)↓	Throughput (records/s)↑	CPU Utilization (%)↑	Cost Efficiency ↑
Default Spark	120	8,500	55	0.65
Static Tuning	102	9,800	63	0.72
Rule-Based Optimization	95	10,600	68	0.76
Dynamic Resource Allocation	90	11,200	71	0.79
Proposed PSO Method	**72**	**14,800**	**84**	**0.91**

The proposed method also achieves higher CPU utilization (84%), indicating more efficient resource usage compared to baseline approaches. Similarly, the cost efficiency (0.91) is the highest among all methods, demonstrating better performance with lower resource consumption. The results confirm that the PSO-based framework effectively explores the configuration space and adapts to workload conditions, leading to improved performance and resource efficiency compared to static and rule-based tuning approaches. The superior performance of the proposed framework can be attributed to the adaptive exploration and exploitation capabilities of the Particle Swarm Optimization (PSO) algorithm. By continuously monitoring runtime telemetry data and dynamically tuning Spark configuration parameters, the framework effectively mitigates resource bottlenecks and enhances workload distribution across the cluster. The observed increase in CPU utilization from 55% to 84% reflects improved resource utilization efficiency, whereas the 40% reduction in execution latency demonstrates the capability of the optimization engine to identify near-optimal configurations under dynamic workload conditions. These findings validate the effectiveness of adaptive swarm-based optimization and indicate its superiority over static tuning, rule-based optimization, and Spark's built-in resource management mechanisms.

5.2 Scalability Analysis

The Fig. 3 evaluates the scalability of the proposed framework under increasing data sizes (10 GB, 100 GB, and 500 GB). The results show that the PSO-based method consistently reduces latency compared to the default Spark configuration. Specifically, latency decreases from 45 to 32 s (10 GB), 210 to 150 s (100 GB), and 980 to 690 s (500 GB), resulting in a stable improvement of approximately 28–30% across all scales. These results in Fig. 3 reveals that the proposed framework scales efficiently with increasing workload size and maintains consistent optimization performance. The PSO algorithm effectively adapts to larger

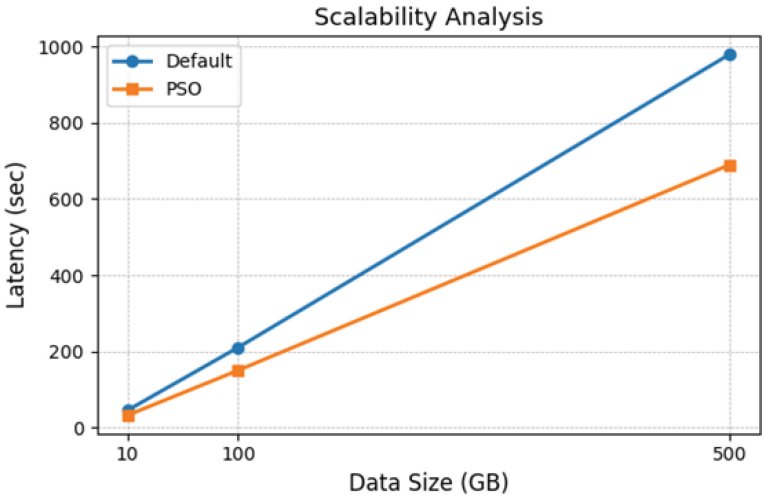


Fig. 3. Scalability analysis comparing default Spark configuration and the proposed PSO-based optimization under increasing data sizes.

datasets without degradation, demonstrating its robustness and suitability for large-scale big data environments.

5.3 Ablation Study

The results shown in the Fig. 4 evaluates the contribution of key components of the proposed framework by comparing the full model with two variants: without adaptive feedback and without swarm optimization. The full model (PSO + adaptation) achieves the best performance with 72 s latency and 14,800 records/s throughput. When adaptive feedback is removed (static PSO), latency increases to 88 s and throughput decreases to 12,600 records/s, indicating reduced responsiveness to changing workloads. In the absence of swarm optimization (rule-based approach), performance further degrades, with 95 s latency and 10,600 records/s throughput, showing limited optimization capability.

These results confirm that both adaptive feedback and swarm intelligence are critical for achieving optimal system performance. While PSO enables efficient exploration of the configuration space, the feedback mechanism ensures continuous adaptation to dynamic workload conditions.

5.4 Convergence Analysis

The convergence behavior of the PSO algorithm is analyzed by tracking fitness values across iterations that is shown in the Fig. 5. The results show a rapid decrease in fitness during early iterations (1–15), indicating effective exploration of the search space. Convergence stabilizes around 25–30 iterations, reflecting refinement of optimal configurations.

The use of adaptive inertia weight helps balance exploration and exploitation, improving convergence speed. Overall, PSO achieves stable and near-optimal solutions within few iterations, making it suitable for dynamic optimization scenarios.

6 Discussion

6.1 Trade-Offs

The proposed PSO-driven optimization framework involves certain trade-offs between computational overhead and performance gains. While the PSO algorithm introduces additional overhead due to iterative optimization and fitness evaluation, the experimental results demonstrate that the improvements in latency and throughput significantly outweigh this cost. Another important trade-off lies between exploration and exploitation. Higher exploration enables the algorithm to search a broader configuration space and avoid local optima, whereas controlled exploitation ensures convergence toward optimal solutions. The adaptive adjustment of PSO parameters helps maintain an effective balance between these two aspects, leading to efficient and stable optimization.

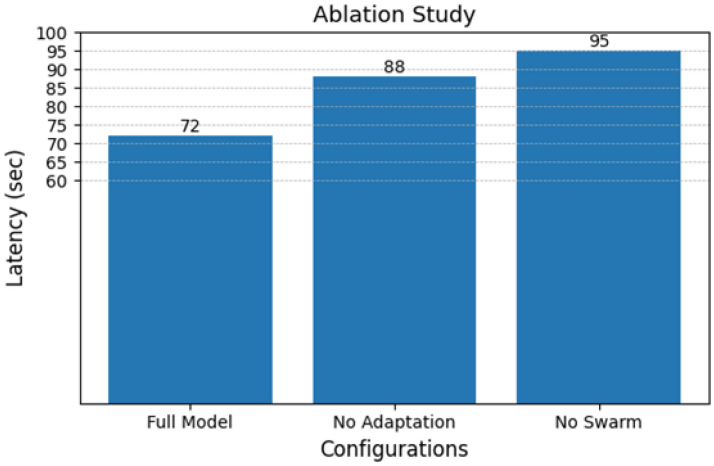


Fig. 4. Ablation study comparing the impact of adaptive feedback and swarm optimization on system latency.

6.2 System Behavior

The proposed system exhibits robust and adaptive behavior under dynamic conditions. It effectively responds to workload spikes, streaming variability, and heterogeneous cluster environments, maintaining consistent performance improvements. The closed-loop feedback mechanism plays a critical role by enabling

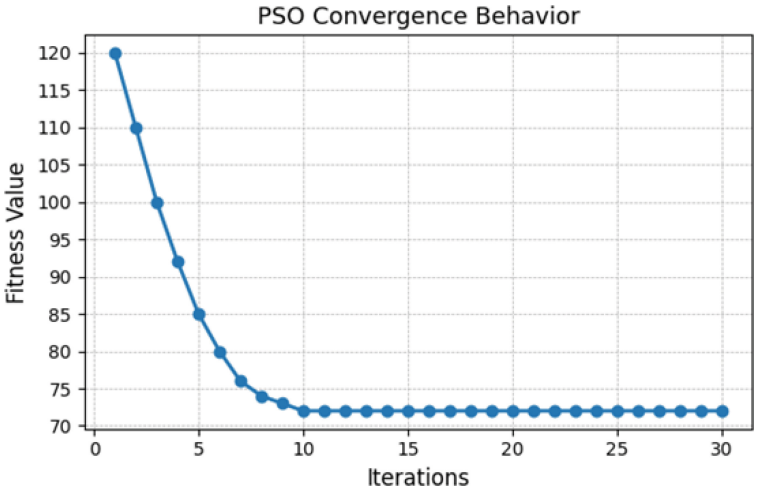


Fig. 5. Convergence behavior of the PSO algorithm showing fitness improvement across iterations.

continuous monitoring, learning, and configuration refinement. This ensures that the system dynamically adapts to changing workload patterns and resource conditions, thereby sustaining optimal performance over time.

6.3 Strengths of Proposed Approach

The proposed framework offers several key strengths. It enables real-time adaptive optimization, allowing continuous tuning of Spark configurations based on runtime conditions. The use of a multi-objective optimization approach ensures simultaneous improvement in latency, throughput, and resource utilization. Additionally, the framework demonstrates strong scalability, maintaining consistent performance gains across increasing data sizes. Its ability to adapt to dynamic workloads also highlights its robustness in distributed and heterogeneous environments, making it suitable for modern big data applications.

6.4 Limitations

Despite its advantages, the proposed approach has certain limitations. The optimization process introduces additional computational overhead due to iterative PSO execution. Moreover, the overall performance of the framework is influenced by parameters such as swarm size and PSO coefficients (c, c, and inertia weight). Improper tuning of these parameters may affect convergence speed and solution quality.

6.5 Practical Implications

The proposed framework is highly applicable to real-world scenarios. It is particularly suitable for real-time analytics systems, where rapid adaptation to changing data streams is essential. It can also be effectively deployed in cloud-based Spark environments, where resource optimization directly impacts operational costs. Furthermore, the framework is well-suited for large-scale enterprise data pipelines, requiring scalable and intelligent performance tuning mechanisms.

7 Conclusion

This paper proposed a PSO-driven self-adaptive optimization framework for dynamic workload optimization in Apache Spark environments. By integrating Particle Swarm Optimization (PSO) with a real-time monitoring and feedback loop, the framework enables continuous and intelligent tuning of key Spark configuration parameters. Experimental results demonstrate that the proposed approach significantly outperforms baseline methods. Specifically, it achieves 40% reduction in latency, improves throughput up to 14,800 records/s, and increases resource utilization($55\% \rightarrow 84\%$), resulting in higher overall cost efficiency. The framework also shows strong scalability, maintaining $\sim$ 28–30% performance improvement across increasing data sizes (10 GB to 500 GB). Convergence analysis confirms that PSO reaches near-optimal configurations within 25–30 iterations, while the ablation study highlights the critical role of both adaptive feedback and swarm optimization. The proposed method provide a scalable, efficient, and real-time optimization solution for modern big data pipelines.

The proposed framework has important practical and research implications. By enabling autonomous Spark configuration tuning, it reduces manual optimization efforts while improving resource utilization and system performance. The framework is well suited for real-time analytics, cloud computing, and large-scale data processing environments. Furthermore, it provides a foundation for future research on intelligent resource management and self-adaptive distributed computing systems.

Future work will focus on Integrating PSO with reinforcement learning for improved adaptive decision-making and Extending to multi-objective trade-off analysis (latency, cost, throughput).

Acknowledgments. This work is support by NFC: Institute of Engineering & Fertilizer Research, Faisalabad and Higher Education Commission (HEC) of Pakistan.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Adil, M., Nabi, S., Raza, S.: PSO-CALBA: Particle swarm optimization based content-aware load balancing algorithm in cloud computing environment. Comput. Inform. **41**(5), 1157–1185 (2022)

2. Al-Terkawi, L.: Survey on distributed parallel genetic algorithms for large-scale data analysis. Computing **108**(1), 6 (2026)
3. Alkayal, E.: Optimizing resource allocation using multi-objective particle swarm optimization in cloud computing systems, University of Southampton (2018). Ph.D. thesis
4. Ananthanarayanan, G., et al.: Reining in the outliers in {Map-Reduce} clusters using mantri. In: 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10) (2010)
5. Bao, L., Liu, X., Chen, W.: Learning-based automatic parameter tuning for big data analytics frameworks. In: 2018 IEEE International Conference on Big Data (Big Data), pp. 181–190. IEEE (2018)
6. Bheemisetty, N.: Next-gen data ecosystems: Domain-ai across spark, ETL, and batch intelligence. Int. J. Sci., Res. Technol. **9**(2), 382–390 (2026)
7. Bilal, M., Canini, M.: Towards automatic parameter tuning of stream processing systems. In: Proceedings of the 2017 Symposium on Cloud Computing, pp. 189–200 (2017)
8. Borthakur, D., et al.: HDFs architecture guide. Hadoop Apache project **53**(1–13), 2 (2008)
9. Ding, F.: Adaptive cross-platform scheduling framework for NWP in hybrid clouds. IEEE Internet Things J. (2026)
10. El-Kafrawy, P., El-Amin, M.F.: An overview of big data analytics. Math. Model. Big Data Anal. 3–18 (2026)
11. Gallinucci, E., Golfarelli, M., et al.: Sparktune: tuning spark SQL through query cost modeling. In: Advances in Database Technology-EDBT 2019, 22th International Conference on Extending Database Technology, Proceedings, pp. 546–549. OpenProceedings. org (2019)
12. Geldenhuys, M.K., Scheinert, D., Kao, O., Thamsen, L.: Phoebe: QOS-aware distributed stream processing through anticipating dynamic workloads. In: 2022 IEEE International Conference on Web Services (ICWS), pp. 198–207. IEEE (2022)
13. Ghattamaneni, D.K., Boinapalli, N.R.: Performance optimization of machine learning pipelines on Apache spark for real-time big data analytics. Int. J. Syst. Design Inform. Process. **1**(3), 1–6 (2013)
14. Gulisano, V., Jimenez-Peris, R., Patino-Martinez, M., Valduriez, P.: Streamcloud: A large scale data streaming system. In: IEEE 30th International Conference on Distributed Computing Systems, pp. 126–137. IEEE (2010)
15. Kreps, J., Narkhede, N., Rao, J., et al.: Kafka: A distributed messaging system for log processing. In: Proceedings of the NetDB, vol. 11, pp. 1–7. Athens, Greece (2011)
16. Li, Y., et al.: Towards general and efficient online tuning for spark. arXiv preprint arXiv:2309.01901 (2023)
17. Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., Freeman, J., Tsai, D., Amde, M., Owen, S., et al.: MLlib: Machine learning in Apache spark. J. Mach. Learn. Res. **17**(34), 1–7 (2016)
18. Muvva, S.: Optimizing spark data pipelines: a comprehensive study of techniques for enhancing performance and efficiency in big data processing. J. Artif. Intell., Mach. Learn. Data Sci. **1**(4), 1862–1865 (2023)
19. Narayanan, D.B.G.S.: A generative ai framework for data pipeline optimization and analytical performance enhancement. Int. J. Artif. Intell., Data Sci., Mach. Learn. **6**(4), 110–121 (2025)

20. Ousterhout, K., Rasti, R., Ratnasamy, S., Shenker, S., Chun, B.G.: Making sense of performance in data analytics frameworks. In: 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15), pp. 293–307 (2015)
21. Potluri, S., Rao, K.S.: Optimization model for QoS based task scheduling in cloud computing environment. Indonesian J. Electr. Eng. Comput. Sci. **18**(2), 1081–1088 (2020)
22. Pulivarthy, P., Kommineni, M., Aragani, V.M., Rajassekaran, G.: Real time data pipeline engineering for scalable insights. In: Machine Learning, Predictive Analytics, and Optimization in Complex Systems, pp. 83–102. IGI Global Scientific Publishing (2026)
23. Raza, H., Erdenetsogt, T., Singh, A., Farooq, M., Kabeer, M.M., Aslam, M.S.: A comprehensive review on data science frameworks for big data analytics. PERFECT: J. Smart Algorithms **3**(1), 1–10 (2026)
24. Sahith, C.S.K., Muppidi, S., Merugula, S.: Apache spark big data analysis, performance tuning, and spark application optimization. In: 2023 International Conference on Evolutionary Algorithms and Soft Computing Techniques (EASCT), pp. 1–8. IEEE (2023)
25. Syed, D., Shaikh, G.M., Rizvi, S.: Systematic review: particle swarm optimization (PSO) based load balancing for cloud computing. Sir Syed Univ. Res. J. Eng. Technol. **14**(1), 86–94 (2024)
26. Tang, S., He, B., Yu, C., Li, Y., Li, K.: A survey on spark ecosystem: big data processing infrastructure, machine learning, and applications. IEEE Trans. Knowl. Data Eng. **34**(1), 71–91 (2020)
27. Verma, A., Pedrosa, L., Korupolu, M., Oppenheimer, D., Tune, E., Wilkes, J.: Large-scale cluster management at google with Borg. In: Proceedings of the tenth European conference on computer systems, pp. 1–17 (2015)
28. White, T.: Hadoop: The definitive guide. " O'Reilly Media, Inc." (2012)
29. Xin, J., Hwang, K., Yu, Z.: LOCAT: Low-overhead online configuration auto-tuning of spark SQL applications. In: Proceedings of the 2022 International Conference on Management of Data, pp. 674–684 (2022)
30. Zaharia, M.: Apache spark: a unified engine for big data processing. Commun. ACM **59**(11), 56–65 (2016)

ARQM-LITE: A Small Tool for Requirements Quality Analysis

June Porter(✉) and Satish Mahadevan Srinivasan(✉)

Pennsylvania State University – Great Valley, Malvern, PA, USA
burghporter314@gmail.com, sus64@psu.edu

Abstract. Here we present the ARQM-LITE, a lightweight iteration of the Automated Requirements Quality Management (ARQM) tool designed to automate the detection of quality violations in natural language requirements while significantly reducing computational overhead. Building on the original ARQM's BERT-based approach, ARQM-LITE integrates rule-based heuristics through a Soft Match feature, Retrieval-Augmented Generation (RAG) for contextual knowledge, and lighter semantic models. The various components of this model enable span-level semantic scoring and POS tagging with minimal resource demands. When evaluated on several real-world requirement documents and a manually annotated quality dataset, ARQM-LITE achieved faster processing times than ARQM, comparable or superior accuracy on Feasibility and Verifiability, with competitive recall on Singularity. The tool also provides user-friendly PDF and interactive HTML visualizations with explanatory feedback. Results demonstrate that ARQM-LITE successfully balances performance and practicality for requirements engineering practitioners.

Keywords: ARQM · ARQM-LITE · RAG + Soft Match feature · PURE dataset · requirement analysis · quality violations

1 Introduction

Requirements Engineering is a difficult and tedious task that often is prone to costly mistakes and errors. When considering the general engineering process, requirements engineering becomes pivotal for success. As a result, the consensus seems to be towards automation of requirement processes. When considering engineering automation requirements, there are many different factors and scopes to consider. For instance, requirements engineering can typically consist of Elicitation, Analysis, Modeling, Validation and Verification, and Management [1, 2].

Different processes have different automation needs, each having different complexities and scopes. For instance, certain automation use cases may try to classify text as requirements whereas other use cases may try to classify the quality of existing requirements. Other uses of automation may include modeling through meta-language generation and UI-based tools that allow for the mapping and traceability of requirement

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 189–200, 2027.
https://doi.org/10.1007/978-3-032-39763-8_11

evolution or dependencies. Of these existing automation tasks, quality analysis remains an important and key consideration of requirement analysis.

Previously, the Automated Requirements Quality Management (ARQM) software sought to utilize modern Natural Language Processing (NLP), Artificial Intelligence (AI), and Machine Learning (ML)-based approaches to identify requirements and associated quality defects. This tool was practical in a way that allowed individuals to upload a PDF version of their requirement document for a visual analytical result. The result would show highlighted violations of their requirements. For quality analysis, the tool utilized an approach through the use of LLMs, transfer learning, and fine-tuning, increasing the computational requirements and inference time significantly. As an added limitation, the tool utilizes the SHAP method to help visualize token impact on inference [3].

As a light-weight iteration of the tool, ARQM-LITE focuses on mitigating the heavy computational requirements of the original ARQM tool while maintaining similar performance. The analysis focuses on how the ARQM-LITE performs against ARQM amongst both requirement quality analysis and computational and relative computational considerations during document ingestion.

2 Related Works and Tools

When considering the automation of requirements engineering, many existing tools exist for different tasks such as Modeling, Detection, Extraction, Classification, and Tracing. For quality analysis, there is a strong limitation of practical tooling and human validation of datasets and result analysis, but a renewed interest in LLM-based requirement automation research and LLM implementations. Most of the work focuses on existing laboratory experiments and the continuation of the NLP4RE tools that are now outdated [2, 6].

In the systematic study, they found that approximately 4% of the literature related to requirement defect detection and about 20% of the studies related to requirements validation. Other categories did not immediately relate to requirement quality analysis. Elicitation was one of the highest studied categories with approximately 20% of studies. For input artifacts, Requirement Specifications were the most common at 38%. Similarly, Requirement Specifications also comprised the highest percent of output artifacts at 20% [6].

While the systematic mapping studies correctly note that requirement analysis is among the most frequently automated activities in requirements engineering [2, 6], the majority of published solutions remain evaluated either using heavyweight LLM pipelines in a controlled laboratory settings or using older rule-based prototypes that lack modern semantic capabilities. ARQM-LITE is positioned as a practical middle ground. It deliberately avoids the large-scale fine-tuning and inference costs of current LLM-based quality checkers while still incorporating contextual knowledge through a lightweight RAG mechanism and span-level semantic scoring.

In contrast to the recent LLM experiments that have tested ISO/IEC/IEEE 29148 quality attributes with software engineers [8], ARQM-LITE was explicitly engineered for immediate deployment on modest hardware. Its combination of pre-trained sentence embeddings, a minimal CNN classifier, rule-based Soft Match heuristics, and

document-specific knowledge extraction yields processing times under seven seconds per document—*i.e.* orders of magnitude faster than the BERT-based ARQM baseline—while maintaining or improving accuracy on Feasibility, Verifiability, and Singularity. This design choice directly responds to the documented industry gap *i.e.* although LLM-based approaches show promise, they are rarely integrated into day-to-day requirements workflows because of their resource demands [6].

Consequently, the contribution of ARQM-LITE lies in the synthesis of existing techniques into a deployable, low-overhead tool that practitioners can run on standard laptops or modest GPUs. The empirical comparison with ARQM on both a 497-item annotated dataset and real-world requirement documents therefore serves to quantify exactly how much computational cost can be removed without sacrificing the detection performance that prior NLP4RE tools have already demonstrated [2, 8].

Critically, the study provides publicly available datasets used to train models for requirement automation tasks. These datasets are for legal requirements analysis, defect detection, text completion, standards compliance, requirements analysis, requirements validation, requirements elicitation and requirements modeling. For requirements quality analysis, the GDPR compliance decisions dataset primarily analyzes conflicting requirements, but does not necessarily analyze other IEEE 29148 standard quality attributes. The PURE dataset contains 34,000+ sentences containing thousands of requirements[1] [6,].

The study also provides practical software implementations for various automated requirement tasks. One of the studies cited analyzed the IEEE 29148 quality agreement among the Llama 2 model and human annotators, showing lower Kappa agreement for unbounded results. The study relies on an experiment and does not synthesize the results into a practical tool [8]. Another study reframes the requirement analysis process utilizing sentence relationships such as entailment and contradictions. By simplifying this relationship, the article suggests a simpler practical relationship with requirement identification and conflict detection. They also provide a quality analysis dataset for training, but no practical application [6, 9].

Interestingly, a practical application focused on automotive requirement documents implemented a vector store to allow user queries. Although the tool does not check requirement document quality, the tool does provide insight into how to divide and query complicated and large requirement documents with relatively simple models. Primarily, the setup mirrored a Retrieval-Augmented Generation setup to allow for requirement-specific queries, showcasing powerful user-specific results [10]. Similarly, ConRAP, a novel legal framework-based tool, utilizes RAG-based technology to detect and eliminate ambiguity within legal contracts. When the tool detects ambiguous terms, ConRAP queries the RAG process via clarifying questions to help determine true document-wide ambiguity, allowing for document-specific context [11].

3 Research Questions

The primary goal of ARQM-LITE was to improve upon the computational load requirements of the existing ARQM implementation. Therefore, we want to make sure that ARQM-LITE has meaningfully improved upon the system requirements of the current ARQM implementation. Additionally, we want to make sure that ARQM-LITE

can perform comparably for quality analysis when compared to the original ARQM application.

For the original ARQM implementation, the PURE dataset was used along with supplementary labeled data for requirement quality analysis. Unfortunately, there were legitimate limitations for the quality datasets provided in the original work for the ARQM implementation, limiting the scope of the quality analysis comparison for ARQM-LITE. Due to this limitation, the scope of the comparison will primarily focus on computational improvement and requirement quality analysis performance. The following research questions are proposed:

RQ1: Does ARQM-LITE's implementation improve or diminish detection of quality violations when compared to ARQM? More specifically, a) How does the Soft Match feature impact the accuracy of detecting quality violations?, b) How does ARQM-LITE's RAG implementation impact detection of quality violations?, c) How does the ARQM-LITE's slot-based semantic scoring compare to ARQM's fine-tuned BERT classifier across the various quality dimensions?, d) What are the largest drivers and impact factors of ARQM-LITE's processing time?

RQ2: Compared to ARQM, how does ARQM-LITE perform across different systems? More specifically, a) How does ARQM and ARQM-LITE perform with regards to system memory (RAM and VRAM)?, b) How does ARQM and ARQM-LITE perform with regards to response speed?, c) How does ARQM and ARQM-LITE performance change across different system platforms?, d) How does ARQM and ARQM-LITE perform with regards to CPU and GPU utilization?

4 ARQM-LITE: A Tool for Expedited Inference and Visualization

To address the complicated computational complexity and long processing time of ARQM, we created ARQM-LITE. ARQM-LITE set out to mitigate the technical limitations of the ARQM implementation, while maintaining comparable results. Like ARQM, the new iteration provides an interface that allows a user to upload a requirement document. As a slight deviation from the ARQM application, the ARQM-LITE implementation provides an option for a PDF-generated response or an interactive HTML page that allows the user to interact with detected quality violations.

As another improvement to ARQM, The RAG-based system implemented within ARQM-LITE helps provide additional knowledge that can help improve the accuracy and discovery of quality violation determinations. The Knowledge Bases (KBs) comprise generic and common words and phrases that are used throughout requirement documents. These words and phrases were extracted through the PURE requirement documents provided previously. Additionally, the ingestion system utilizes an extraction process to create a Knowledge Base of common terms that are document-specific, potentially allowing for the mitigation of violations pertaining to terms defined outside of the requirement [10].

To operationalize this RAG component, ARQM-LITE first encodes all knowledge-base entries (from PURE-derived generic phrases and document-specific terms) using

the all-MiniLM-L6-v2 sentence embedding model. For each candidate requirement sentence, we compute its embedding and retrieve the top-k most similar knowledge entries. The retrieved entries are then aggregated into a compact context vector whose similarity scores are used as additional features for the CNN-based classifier and the rule-based Soft Match logic. In this way, the RAG mechanism can influence the final quality decision by either reinforcing or attenuating evidence for a violation when the local sentence context aligns or conflicts with the broader document or generic requirement patterns.

Inspired from the original ARQM implementation, the ARQM-LITE software provides many visual implementations and enhancements. Specifically, the software includes support for the generation of PDF and interactive website artifacts that focus on the visualization and explanatory components of violations throughout the requirement document. Like ARQM, the PDF and HTML visuals focus on a highlighting component that shows the offending text related to the quality violations. As an improvement to ARQM, ARQM-LITE provides an explanatory component that helps provide suggestions and clear instructions to help mitigate violations.

Ultimately, ARQM-LITE utilizes lighter models than the original ARQM implementation. Specifically, ARQM primarily relied on DistilBERT and BERT-based models for requirement identification and quality analysis whereas ARQM-LITE utilizes more preprocessing, a Convolutional Neural Network (CNN), the all-MiniLM-L6-v2 model, and the en_core_web_sm model. ARQM-LITE also uses spans and semantic similarity via slot mapping and POS tagging for identifying problematic text. This method allows for a semantical evaluation with minimal computational complexity [12].

Lastly, ARQM-LITE provides a Soft Match feature. This feature is built upon other, older tools such as the reconstructed NASA ARQM tool that relied upon dictionaries and heuristics. The primary idea was to provide a dictionary of common terminology, phrases, and patterns that typically indicate violations within requirements without consideration of context. When Soft Match is enabled during quality analysis, these dictionary hits are treated as an additional signal in the classification pipeline (i.e., they can change the predicted label), and they are also surfaced in the HTML interface as highlighted "problematic" words or phrases that typically correspond to each quality attribute. For instance, words and phrases such as "all", "user-friendly", and "intuitive" are generally not measurable and can often hurt readability and verifiability of a requirement document [1, 4].

5 Methodology

To validate the ARQM-LITE application, the process analyzes performance speed across different system resource availability to help simulate how two systems perform across different hardware restrictions. Primarily, the system tests measure CPU-specific and GPU-specific resources to help understand performance between ARQM and ARQM-LITE. The analysis also looks at how ARQM and ARQM-LITE improve across additive system resources.

To normalize the quality dataset for both tools, we took the quality violations present in the granular dataset for ARQM-LITE, simplified the entries to a requirement and classification mapping, and then synthesized to a deduplicated quality dataset. The loss

of data did not impact ARQM as the original structure of the data served to help ARQM-LITE with its implementation. As a consequence, we then compared both systems to a comparable validation dataset to help analyze performance.

To further understand the implications of real-life requirement documents, the process utilizes nineteen requirement documents provided by the Pennsylvania State University from the original ARQM study. These requirement documents allow for an analysis of system impact based on document size and domain. Because these documents differ significantly in both size and requirement amount, the analysis attempts to extrapolate system performance implications for real-world requirement documents based on performance among these documents.

To test the impact of the RAG implementation, the application performs quality analysis with and without the feature across the manually annotated datasets. In this context, the "Soft Match" variants in Table 1 refer to runs where the dictionary-based Soft Match scores are integrated into the decision logic of the classifier, so that they affect the predicted labels in addition to driving the visualization in the HTML interface. The process then analyzes whether there was a significant difference between the inclusion of RAG based on the reviewer annotation accuracy across the quality attributes. In addition, the process also analyzes how the Soft Match feature impacts the quality results' accuracy and general performance. In the "RAG" variants reported in Table 1, the retrieved knowledge entries are not only shown to the user but are also incorporated as numerical features (similarity scores of the top-k matches) that modify the classifier's input representation and can therefore change the predicted quality label.

To measure the results of the system comparison, the analysis measures the processing time for all nineteen requirement documents as the primary metric. Other metrics include average CPU and GPU utilization, including memory and general utilization, as applicable for the system. Additionally, the measurements analyze how both systems are impacted based on documents containing larger numbers of requirements.

To measure the results of the quality analysis comparison, the analysis primarily relies on accuracy, F1-Score, precision, and recall among the positive and negative classes. These metrics will be measured against the manually annotated datasets for requirement quality as the baseline to help understand simple improvement among the two systems [3].

6 Validation and Evaluation

6.1 Quality Evaluation of ARQM and ARQM-LITE

Table 1 shows the results of ARQM-LITE with the inclusion or exclusion of RAG and Soft Match during the classification process of quality analysis. For Ambiguity, ARQM-LITE with the RAG Only variation had the highest accuracy with a score of 0.61. When including RAG and Soft Match, accuracy went down, but precision, recall, and F1 generally improved among the positive class for Ambiguity. The highest F1-score for the negative class under Ambiguity was achieved with the RAG Only variation with a score of 0.71. Still, the RAG + Soft Match variation was able to achieve a high recall for the positive class with a score of 0.93.

For Feasibility, the RAG + Soft Match and RAG Only variants outperformed on accuracy with a score of 0.83. Even though there was a high accuracy achievement for this variant, there was a notably low recall of 0.50 across the two variants for the positive class. The base variants and Soft Match variant achieved marginally higher recall and F1 scores of 0.58 and 0.52, respectively. Notably, all models, including BERT failed to properly encapsulate the patterns of the positive class for Feasibility.

For Singularity, the highest achieved accuracy was from the ARQM-LITE base variation with a score of 0.77. Surprisingly, the highest achieved recall score for Singularity was the RAG + Soft Match implementation with a score of 0.92 for the positive class. The standalone Soft Match implementation also showed a 0.92 score for recall, suggesting that dictionaries and heuristics may be useful for predicting positive cases of Singularity violations. Interestingly, there were still relatively high metrics for the negative class among the Soft Match and RAG variations.

Lastly, the RAG Only variation achieved the highest accuracy for Verifiability with a score of 0.78. Unlike the other instances of quality attributes, RAG Only also had the highest metrics for both classes across accuracy, precision, recall and F1, suggesting that the RAG Only variation was ideal for Verifiability among the quality dataset. Overall, the variation still underperformed with positive classes but did achieve a recall and F1 score of 0.89 and 0.52, respectively.

Table 1 ARQM-LITE quality results (N = 497)

Attribute	Viol.	RAG + Soft Match				RAG Only				No RAG + Soft Match				Base			
		Acc	Prec	Rec	F1	Acc	Prec	Rec	F1	Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
Ambiguity	**N**	0.57	0.96	0.47	0.63	0.61	0.87	0.60	0.71	0.49	0.95	0.38	0.55	0.52	0.84	0.49	0.62
Ambiguity	**Y**	0.57	0.31	0.93	0.46	0.61	0.29	0.64	0.40	0.49	0.28	0.93	0.43	0.52	0.24	0.64	0.35
Feasibility	**N**	0.83	0.89	0.89	0.89	0.83	0.89	0.89	0.89	0.81	0.91	0.86	0.88	0.81	0.91	0.86	0.88
Feasibility	**Y**	0.83	0.50	0.50	0.50	0.83	0.50	0.50	0.50	0.81	0.47	0.58	0.52	0.81	0.47	0.58	0.52
Singularity	**N**	0.71	0.97	0.67	0.79	0.75	0.88	0.81	0.84	0.70	0.97	0.65	0.78	0.77	0.92	0.79	0.85
Singularity	**Y**	0.71	0.37	0.92	0.52	0.75	0.35	0.50	0.41	0.70	0.35	0.92	0.51	0.77	0.40	0.67	0.50
Verifiability	**N**	0.77	0.98	0.75	0.85	0.78	0.98	0.77	0.86	0.72	0.98	0.70	0.82	0.74	0.98	0.72	0.83
Verifiability	**Y**	0.77	0.35	0.89	0.50	0.78	0.36	0.89	0.52	0.72	0.31	0.89	0.46	0.74	0.32	0.89	0.47

When comparing Table 1 with Table 2, there are similar and different results among the performance metrics for the quality attributes. For Ambiguity, BERT outperformed generally among every metric including accuracy, with a maximum accuracy performance of 0.91 compared to ARQM-LITE's 0.61. BERT also performed well among the positive and negative classification metrics. For instance, BERT achieved a recall and F1 score of 0.94 and 0.84 compared to ARQM-LITE's 0.93 and 0.46.

For Feasibility, ARQM-LITE achieved a better accuracy than ARQM with a score of 0.83 compared to ARQM's 0.82. Additionally, ARQM-LITE outperformed for the positive class metrics for recall and F1 against ARQM with a score of 0.58 and 0.52 against BERT's score of 0.50 and 0.40. Notably, ARQM-LITE also performed similarly to ARQM for the negative class as well, suggesting general comparable performance among the Feasibility quality attribute.

For Singularity, ARQM outperformed ARQM-LITE among accuracy with a score of 0.91 compared to ARQM-LITE's score of 0.77. However, ARQM-LITE achieved a higher positive recall score for Singularity with a score of 0.92 among the positive class, compared to BERT's result of 0.80. However, BERT still marginally outperformed on the F1-score for the positive class. Ultimately, the results were less disparate than the Ambiguity results, suggesting that Singularity was better encapsulated by the ARQM-LITE representation.

Lastly, ARQM-LITE outperformed on Verifiability. With an accuracy score of 0.78, ARQM-LITE improved upon ARQM's accuracy results from a score of 0.62. Additionally, the positive and negative class metrics also reflect a superior performance for ARQM-LITE when compared to the general ARQM results.

Table 2 Quality analysis for ARQM – BERT (N = 497)

Quality Attribute	Violation	Accuracy	Precision	Recall	F1
Ambiguity	**N**	0.91	0.98	0.90	0.94
Ambiguity	**Y**	0.91	0.76	0.94	0.84
Feasibility	**N**	0.82	0.93	0.86	0.89
Feasibility	**Y**	0.82	0.33	0.50	0.40
Singularity	**N**	0.91	0.98	0.92	0.95
Singularity	**Y**	0.91	0.44	0.80	0.57
Verifiability	**N**	0.62	0.94	0.58	0.72
Verifiability	**Y**	0.62	0.28	0.82	0.42

6.2 Computational Evaluation of ARQM and ARQM-LITE

Based on Table 3, the average document processing time for ARQM was 245.32 s on the 4060 GPU and 58.51 s on the 5090 GPU. For ARQM-LITE, the average processing time across documents was 6.19 s for the 4060 GPU hardware platform and 6.07 s for the 5090 GPU hardware platform. The original ARQM software saw a massive improvement in processing time from the GPU upgrade, whereas the ARQM-LITE software remained relatively consistent across hardware platforms. For the 4060, the ARQM-LITE software performed within the range of twenty to one-hundred times faster than ARQM, whereas with the 5090, the performance differential, while still notable, remained under twenty times faster.

For the general resource usage among the different hardware platforms, Table 3 provides the relevant statistics across the nineteen documents and how the associated hardware was impacted. Based on the median, ARQM used approximately 19.80% of CPU usage, 66.50% of GPU usage, and 2172.00 MB of GPU VRAM, along with some RAM usage for the 4060. Additionally, ARQM-LITE used 9.40% of the CPU, while using 18.10% of GPU usage, and a consistent 876.00 MB of VRAM for the 4060. For the 5090, ARQM CPU usage decreased to 14.90% of usage, but increased RAM usage.

Additionally, the GPU utilization decreased to 33.50%, but increased VRAM usage to 5025 MB. For ARQM-LITE, CPU usage percent remained relatively the same, while GPU utilization percentage drastically decreased to 2.90%, with a moderate VRAM increase to 3559.00 MB.

Table 3 Aggregated performance results: ARQM vs. ARQM-LITE

Values reported as Mean ± SD (Median)				
Metric	ARQM 4060	ARQM-LITE 4060	ARQM 5090	ARQM-LITE 5090
CPU Avg (%)	21.81 ± 5.44 (19.80)	9.29 ± 2.02 (9.40)	14.97 ± 0.71 (14.90)	8.84 ± 1.51 (9.20)
RAM Avg (MB)	11806.89 ± 1247.22 (11096)	13770.58 ± 226.67 (13679)	18961.21 ± 48.36 (18969)	18165.16 ± 81.84 (18188)
GPU Util Avg (%)	57.69 ± 16.80 (66.50)	16.86 ± 7.49 (18.10)	34.38 ± 3.67 (33.50)	2.66 ± 1.64 (2.90)
GPU Mem Avg (MB)	2181.25 ± 159.88 (2172)	876 ± 0 (876)	4973.11 ± 114.13 (5025)	3543.21 ± 68.82 (3559)
Response time (s)	245.32 ± 162.16 (171.18)	6.19 ± 2.72 (5.65)	58.51 ± 24.59 (52.5)	6.07 ± 3.92 (5.08)

7 Discussion

The original implementation of ARQM-LITE was to synthesize a large, computationally expensive application into something more practical and useful for requirement practitioners. The first ARQM implementation utilized an architecture that required large datasets, processing requirements, and, by extension, computational capabilities. Inspired by older automated requirement designs, ARQM-LITE was meant to mimic and consolidate rule-based, heuristic systems with the original ARQM AI and ML-based implementations.

We acknowledge that the accuracy results for Singularity require careful qualification. While ARQM-LITE's best-performing variant reached 0.77 accuracy on Singularity, ARQM achieved 0.91 on the same attribute and dataset. This 0.14 gap indicates that ARQM retained a clear advantage in overall classification accuracy for Singularity. However, ARQM-LITE produced markedly higher recall on the positive (violation) class for Singularity (0.92 versus ARQM's 0.80), which is the more operationally relevant figure for practitioners who need to surface defects rather than correctly classify non-violations. In addition, ARQM-LITE matched or exceeded ARQM on every metric for both Feasibility and Verifiability.

Based on the comparative analysis, we were able to show that ARQM-LITE was significantly faster than the original ARQM implementation. As shown previously, ARQM-LITE's response time dwarfed ARQM on the 4060 hardware with a 20 to 100 response improvement. On the 5090 hardware, the difference consolidated to a less noticeable

difference, but ARQM-LITE still vastly overperformed with response time across all nineteen documents.

Due to the problematic quality of the original ARQM datasets for the quality attributes, this study utilized a new manually annotated dataset as a benchmark across the two systems. Even though ARQM-LITE is faster than ARQM, the ARQM quality identification outperformed on certain quality attributes such as Ambiguity and Singularity. For Ambiguity, the difference was quite noticeable, but the difference was less of a factor for Singularity. ARQM-LITE was able to perform comparably to ARQM across Singularity, Verifiability, and Feasibility when considering Accuracy, F1-Score, Precision, and Recall among the positive and negative classes. In the case of Verifiability and Feasibility, ARQM-LITE outperformed ARQM across the various measurements including accuracy. Across the evaluated attributes, ARQM-LITE showed stronger positive-class (violation) recall than ARQM for Feasibility, Singularity, and Verifiability, but not for Ambiguity, indicating that its advantage in surfacing violations is attribute-dependent rather than universal. Accordingly, any claim of overall superiority on positive-class identification must be qualified by these attribute-specific differences.

Based on the shift from the 4060 to the 5090 hardware implementations, ARQM improved notably whereas the ARQM-LITE software did not. For ARQM, the processing time improved from 245.32 s to 58.51 s, whereas the processing time for ARQM-LITE remained constant.

We note that the memory-usage comparison between ARQM and ARQM-LITE is platform-dependent rather than uniformly higher for ARQM-LITE. On the 4060 hardware, ARQM-LITE did consume more RAM on average (13,771 MB versus 11,807 MB), yet its VRAM footprint remained substantially lower (876 MB versus 2,181 MB). On the 5090 platform, ARQM-LITE used less RAM (18,165 MB versus 18,961 MB) and again far less VRAM (3,543 MB versus 4,973 MB).

Consequently, the statement that ARQM-LITE "took up more memory for RAM and VRAM" does not hold across both tested configurations. The corrected picture is that ARQM-LITE consistently required less GPU memory while its RAM usage varied modestly with hardware and was never the dominant factor in overall resource consumption.

This nuance reinforces rather than weakens the central claim: ARQM-LITE delivers dramatic speed gains while imposing a lighter or comparable memory burden, making it practical for practitioners who lack high-end GPUs or large RAM allocations. Overall, ARQM-LITE did not take up significant system resources. There also did not seem to be an overall improvement when greater resources were given, suggesting that ARQM had more of a resource bottleneck.

For the RAG and Soft Match features, there was consistent overperformance when compared to the exclusionary implementations. Seemingly, RAG and Soft Match allowed for the injection of knowledge that a typical requirement quality analyzer would flag. Counterintuitively, the Soft Match feature was particularly useful for Singularity. To help mitigate the flagging of domain-specific terms, along with the definition of pre-defined heuristics, the results were on-par with the original ARQM implementation, allowing for a complex and highly context-sensitive implementation without vast training data.

Eliminating the RAG and Soft Match features rarely yielded positive results among the quality attributes.

8 Limitations

For the quality violations, there were limited and unbalanced representations of the quality attribute violations. The quality dataset was also not validated by other individuals nor was the dataset put through any external validation for validity testing other than individual expert labeling. Additionally, we did not test the validity of the highlighting feature across the various systems. Lastly, the rule-based and heuristic dictionaries for ARQM-LITE were minimal. Expanding upon these lists would likely change key results against the pure semantical and calibration implementations.

9 Conclusion

ARQM-LITE implemented a rule-based and AI-based approach to achieve requirement quality automation. Inspired by the original ARQM implementation, ARQM-LITE sought to improve upon response time, while trying to minimize performance trade-off. Because of the limited datasets for quality violations, the comparison was limited. Still, the new iteration of ARQM shows promising results and a renewed and enhanced, user-friendly application for requirement analysis automation.

For the future, we are hoping to gather qualitative feedback on our ARQM-LITE application, enhance the application, and bring further improvements for the quality analysis process. Additionally, we hope to continue to refine our approach with less data-intensive implementations, while implementing more rules, dictionaries, and heuristics, to help inform users of common requirement pitfalls. We also want to extend ARQM-LITE to support a full RAG implementation to implement quality attributes such as completeness and consistency.

Datasets. The PURE dataset can be found at https://zenodo.org/records/1414117. To access the quality dataset, please submit a request to Dr. Porter at burghporter314@gmail.com.

Author Contributions. Dr. Dylan Porter: Primary contributor and writer of the paper. Dr. Satish Srinivasan: Intellectual contributor, result validation, and paper formatting.

Acknowledgments Not applicable.

Declarations

Declaration of Originality. The authors declare that this work is original, has not been published elsewhere, and all external sources are properly cited.

Use of Artificial Intelligence. During the preparation of this project, the authors utilized agentic and generative AI tools, specifically Claude Code, ChatGPT, and Claude AI, strictly for source code generation and optimization.

References

1. Carlson, N., Laplante, P.: The NASA automated requirements measurement tool: a reconstruction. Innov. Syst. Softw. Eng. **10**(2) (2014). https://doi.org/10.1007/s11334-013-0225-8
2. Zhao, L., et al.: Natural language processing for requirements engineering: a systematic mapping study. ACM Comput. Surv. 54(3), 5541 (2021). https://doi.org/10.1145/3444689
3. Porter, D.: Automated Requirements Quality Measurement (ARQM): a Tool for Requirements Quality Analysis. Pennsylvania State University ScholarSphere (2026) https://scholarsphere.psu.edu/resources/8eb73810-98d0-4378-b0c3-81558087d660
4. ISO/IEC/IEEE: International standard–systems and software engineering–life cycle processes–requirements engineering. ISOIECIEEE 29148:2018E, pp. 1–104 (2018). https://doi.org/10.1109/IEEESTD.2018.8559686
5. Chantree, F., Nuseibeh, B., De Roeck, A., Willis, A.: Identifying nocuous ambiguities in natural language requirements. In: 14th IEEE International Requirements Engineering Conference (RE'06), pp. 59–68. Minneapolis/St, Paul. IEEE (2006) https://doi.org/10.1109/RE.2006.31
6. Zadenoori, M.A., Dąbrowski, J., Alhoshan, W., Zhao, L., Ferrari, A.: Large language models (LLMs) for requirements engineering (RE): a systematic literature review (2025). https://arxiv.org/abs/2509.11446. https://doi.org/10.48550/arXiv.2509.11446
7. Ferrari, A., Spagnolo, G.O., Gnesi, S.: PURE: a dataset of public requirements documents. Zenodo. (2018). https://doi.org/10.5281/zenodo.1414117
8. Lubos, S., et al.: Leveraging LLMs for the quality assurance of software requirements. In: 2024 IEEE 32nd international requirements engineering conference (RE), pp. 389–397 (2024) https://doi.org/10.1109/RE59067.2024.00046
9. Fazelnia, M., Koscinski, V., Herzog, S., Mirakhorli, M.: Lessons from the use of natural language inference (NLI) in requirements engineering tasks (2024). https://arxiv.org/abs/2405.05135. https://doi.org/10.48550/arXiv.2405.05135
10. Uygun, Y., Momodu, V.: Local large language models to simplify requirement engineering documents in the automotive industry. Prod. Manuf. Res. **12**(1), 2375296 (2024). https://doi.org/10.1080/21693277.2024.2375296
11. Singhal, A., Jain, C., Anish, P.R., Chakraborty, A., Ghaisas, S.: Generating clarification questions for disambiguating contracts (2024). https://arxiv.org/abs/2403.08053. https://doi.org/10.48550/arXiv.2403.08053
12. Lundberg, S.: SHAP documentation (2025). https://shap.readthedocs.io/en/latest/

FlowAttester: Towards a Runtime Flow Attestation Framework in Programmable NeoCloud Networks

Li Wang[1(✉)] and Anbang Ruan[2]

[1] Department of Computer Science, Northeastern Illinois University, Chicago, USA
lwang3@neiu.edu

[2] Department of Computer Science, University of Oxford, Oxford, UK
anbang.ruan@cs.ox.ac.uk

Abstract. NeoClouds are emerging as a new generation of cloud platforms designed to meet the rapidly growing demands of AI workloads. To support ultra-low latency, massive bandwidth, and flexible network orchestration, NeoCloud infrastructures rely on Software-Defined Networking (SDN) technologies, which provide fine-grained control on traffic flows and can dynamically adjust runtime network behaviors. SDN consists of the control plane, which is in charge of generating and distributing various network flow rules, and the data plane, which is responsible for receiving and enforcing the flow rules during runtime. However, the existing SDN architecture lacks a mechanism to verify whether flow rules are correctly enforced at the data plane. Consider the control plane delegates flow rules enforcement to the data plane, the integrity of runtime flow rules enforcement can be easily compromised by untrusted data plane devices. For example, a malicious user may compromise a data plane device and tamper the traffic flow rules. To address this challenge, we propose FlowAttester, a new runtime attestation framework to verify flow rules enforcement in programmable NeoCloud networks. We introduce Trusted Computing technologies into the SDN environment to attest runtime flow rules enforcement of the data plane devices. With our method, the runtime flow-rule enforcement of a data-plane device can be periodically attested through an attestation protocol, and the attestation results show verifiable evidence of policy-compliant enforcement. We demonstrate the effectiveness of our method through preliminary experiment results.

Keywords: NeoCloud · Software-defined networking · SDN · flow attestation · flow enforcement attestation · OpenFlow · TPM · Trusted Computing

1 Introduction

Driven by the rapid advancement of Large Language Model (LLM), hyperscalers are aggressively expanding NeoCloud infrastructures to meet the unprecedented

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 201–217, 2027.
https://doi.org/10.1007/978-3-032-39763-8_12

demand for large-scale AI computing resources. As a new generation of cloud infrastructure, NeoCloud is designed to support the exponentially increasing computational demands of AI workloads. Large-scale AI applications, such as distributed LLM training and real-time inference, require ultra-low latency, massive bandwidth, and highly flexible traffic orchestration across distributed computing resources. To meet these stringent networking requirements, NeoCloud infrastructures increasingly rely on Software-Defined Networking (SDN) technologies.

Software-Defined Networking (SDN) has become a fundamental networking paradigm in both academia and industry in recent years [9]. Unlike traditional network architectures, where control logic and forwarding functions are tightly integrated within proprietary network devices, SDN decouples network control from packet forwarding. SDN introduces a programmable network architecture by separating network functionalities into two major components: the *control plane* and the *data plane*. The *control plane*, implemented through centralized SDN controllers, is responsible for generating routing decisions, distributing flow rules, and managing network-wide policies. The *data plane*, deployed on forwarding devices such as SDN switches and routers, is responsible for enforcing flow rules and forwarding network traffic at runtime. Through programmable controller interfaces, upper-layer applications can dynamically adjust routing policies, traffic engineering strategies, and network behaviors on demand, enabling fine-grained and flexible control over network traffic flows.

However, the programmability and flexibility introduced by SDN also bring significant security concerns. The Open Networking Foundation has provided comprehensive analyses of security threats in SDN infrastructures [13,14]. By exposing low-level network control interfaces to upper-layer applications, SDN enables dynamic and fine-grained management of network behaviors during runtime. However, such programmability also expands the attack surface of modern network infrastructures. In practice, network controllers and forwarding devices cannot blindly trust all applications and users interacting with the SDN environment. Malicious applications or compromised entities may exploit SDN control interfaces to manipulate network configurations, inject unauthorized flow rules, or alter forwarding behaviors at runtime. Without proper security protections and runtime verification mechanisms, legitimate network policies and traffic flows can be silently tampered with. For example, a malicious application without proper authorization may install conflicting or malicious flow rules through the SDN controller, thereby disrupting normal network operations or modifying legitimate traffic paths on forwarding devices.

Considerable research efforts have been devoted to improving the security and reliability of SDN infrastructures. Porras et al. [16] proposed FortNOX, a security enforcement kernel for SDN controllers that prevents adversarial applications from installing unauthorized or conflicting flow rules into SDN forwarding devices. VeriFlow [7] introduced a real-time verification framework capable of detecting policy violations and rule conflicts before new flow rules are deployed to the network data plane. Fleet [10] investigated the problem of untrusted SDN

network administrators and proposed a recovery-oriented protocol framework for maintaining controller trustworthiness in compromised SDN environments. In addition, several studies [23] have explored the use of SDN technologies to build advanced network security mechanisms and adaptive defense systems.

Despite these efforts, most existing SDN security solutions primarily focus on protecting the control plane, including controller hardening, policy verification, and malicious application prevention. Comparatively little attention has been devoted to verifying whether forwarding devices faithfully enforce controller-issued flow rules at runtime. As a result, compromised or malicious data-plane devices may silently manipulate traffic flows or deviate from intended network policies without being detected by the SDN controller. In existing SDN infrastructures, network administrators have limited visibility into the runtime enforcement behaviors of underlying forwarding devices and lack effective mechanisms to verify whether traffic flow rules are correctly enforced at the data plane. This enforcement integrity gap poses significant security risks to modern programmable NeoCloud network infrastructures. Therefore, a practical security mechanism is needed to continuously verify the correctness and integrity of SDN flow-rule enforcement during runtime.

In this paper, we propose FlowAttester, a runtime attestation framework for verifying traffic-flow enforcement in programmable SDN infrastructures. FlowAttester integrates Trusted Computing technologies into SDN environments to provide verifiable attestation of runtime flow behaviors on SDN forwarding devices. The core idea of our approach is to securely measure and record flow-rule enforcement states at the data plane using trusted hardware-assisted mechanisms. When an attestation request is initiated, the SDN forwarding device generates an attestation proof and sends it to a challenger for verification. The challenger then compares the attestation proof with the expected flow-rule states, referred to as the ground truth, to determine whether traffic flows are being correctly enforced at runtime.

Our goal is to provide SDN infrastructures with the capability to continuously verify the integrity of runtime traffic-flow enforcement on forwarding devices. With FlowAttester, traffic flows can be periodically attested through a lightweight remote attestation protocol, enabling verifiable and policy-compliant network enforcement. The main contributions of this paper are summarized as follows:

- We propose a runtime attestation framework for verifying SDN traffic-flow enforcement, providing verifiable evidence of policy-compliant network behaviors at the data plane.
- We design a flexible attestation architecture that supports multiple deployment models, where the challenger can be implemented either within the SDN controller or through a trusted third party.
- We develop a practical timestamp-based attestation protocol for SDN flow rules. Our protocol supports incremental and accumulative attestation of runtime flow states, significantly improving attestation efficiency for large-scale SDN environments containing massive numbers of flow rules.

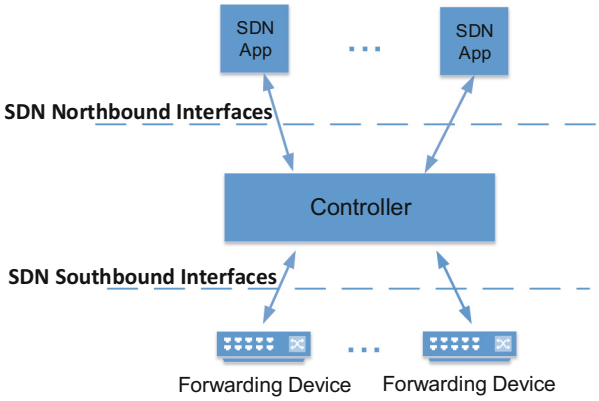


Fig. 1. The SDN Architecture.

The remainder of the paper is organized as follows. We introduce the SDN architecture and basic knowledge of trusted computing and remote attestation in Sect. 2. In Sect. 3, we present the architecture and design of FlowAttester, and illustrate how the attestation process works. We apply our method in an OpenFlow scenario and present preliminary experimental results in Sect. 4. We briefly summarize the related work in Sect. 5. Finally, the conclusion and future will be given in Sect. 6.

2 Background

Software-Defined Networking Architecture: Software-Defined Networking (SDN) is a programmable networking paradigm designed to support next-generation network infrastructures. Figure 1 illustrates the SDN architecture. As shown in the figure, the SDN architecture is typically organized into three layers. The upper layer is the *SDN application layer*, where network applications communicate with SDN controllers through northbound interfaces. These applications may include traffic engineering systems, security applications, monitoring frameworks, or other upper-layer services that manage and control underlying network behaviors.

The middle layer is the *SDN control plane*, which is implemented through one or more SDN controllers. The controller serves as the central management entity that bridges the application layer and the underlying network infrastructure. It is responsible for interpreting application requirements, generating network policies, and distributing flow rules to forwarding devices. The lower layer is the *data plane*, which consists of SDN-enabled forwarding devices responsible for enforcing flow rules and forwarding network traffic across physical networks. Unlike traditional network architectures, SDN separates network control logic from packet forwarding functions, enabling flexible and programmable network management.

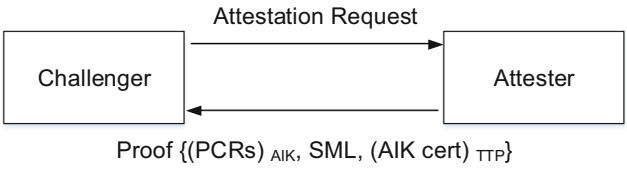


Fig. 2. A Typical Remote Attestation Scenario.

When an SDN application needs to modify network behaviors, it first communicates with the SDN controller through northbound interfaces. The controller receives and interprets the application requirements, translates them into low-level flow rules, and distributes these rules to SDN forwarding devices through southbound interfaces. The forwarding devices then enforce the flow rules during runtime to control traffic forwarding behaviors across the network.

Trusted Computing: Trusted Computing is a security technology framework designed to provide trustworthy and tamper-resistant computing platforms for mission-critical applications. The core component of Trusted Computing is a dedicated hardware security module called the Trusted Platform Module (TPM) [22]. TPM provides hardware-assisted security functionalities, including secure key generation, cryptographic operations, platform integrity measurement, and secure attestation mechanisms.

A TPM contains permanent RSA key pairs that uniquely identify a platform and protect sensitive cryptographic materials. In addition, TPM maintains a set of secure Platform Configuration Registers (PCRs) used for integrity measurement and system state verification. The primary operation on PCRs is the *extend* operation. For security reasons, PCR values cannot be directly modified; instead, they can only be updated through chained *extend* operations. An *extend* operation performs a one-way cryptographic hash computation that appends new measurement values to the existing PCR state, making the measurement history tamper-evident and computationally irreversible. PCR values are reset only during system reboot.

These hardware-assisted security properties establish the foundation of Trusted Computing and enable the construction of verifiable trust chains across software and hardware components. In this work, we leverage Trusted Computing technologies to implement runtime attestation of SDN traffic-flow enforcement.

Remote Attestation: Remote attestation is a fundamental mechanism in Trusted Computing that enables a platform to prove its integrity and trustworthy state to a remote verifier. A typical remote attestation process involves two entities: a *Challenger* and an *Attester*. During the attestation process, the *Attester* provides cryptographic proofs that allow the *Challenger* to verify the integrity of the Attester platform. Figure 2 illustrates a typical remote attestation scenario.

The attestation process begins when the *Challenger* sends an attestation request to the *Attester*. In response, the Attester generates and returns a set of integrity proofs containing trusted platform measurements. Based on these proofs, the Challenger determines whether the Attester platform can be trusted.

As shown in Fig. 2, the attestation proof consists of the following components:

$$Proof : \{(PCRs)_{AIK},\ SML,\ (AIK\ cert)_{TTP}\}$$

Here, $(PCRs)_{AIK}$ represents the Platform Configuration Register (PCR) values signed by the Attester's *Attestation Identity Key* (AIK). The AIK is a TPM-generated cryptographic identity key used exclusively for attestation purposes. The *Stored Measurement Log* (SML) records the sequence of integrity measurements collected from the platform. Whenever a new measurement is extended into a PCR through a *PCR extend* operation, the corresponding measurement value is also recorded in the SML for future verification. The component $(AIK\ cert)_{TTP}$ denotes a digital certificate issued by a Trusted Third Party (TTP), which certifies the authenticity of the AIK associated with the Attester platform.

Upon receiving the attestation proof, the Challenger performs several verification steps. First, the Challenger validates the AIK certificate to ensure that the AIK is legitimate and issued by a trusted authority. Next, the Challenger verifies the signed PCR values using the corresponding AIK. Finally, the Challenger reconstructs the PCR values by replaying the measurement sequence recorded in the SML through iterative *PCR extend* operations. If the reconstructed PCR values match the PCR values reported by the Attester, the platform integrity is considered trustworthy; otherwise, the platform may have been compromised.

3 Architecture

Software-Defined Networking (SDN) enables upper-level applications to dynamically program network behaviors. However, this flexibility also introduces vulnerabilities, making it possible for malicious actors to disrupt normal network operations. Although numerous security mechanisms have been proposed to protect the SDN infrastructure, a critical gap remains in verifying the actual enforcement of flow rules on data-plane forwarding devices.

In this section, we present the architecture of FlowAttester, a runtime attestation framework for verifying SDN traffic-flow enforcement using Trusted Computing technologies. Figure 3 illustrates the overall system architecture. To simplify the attestation process, we assume a standard SDN deployment model in which each forwarding device is managed by a single SDN controller. As shown in Fig. 3, the architecture consists of three major entities, each performing a distinct role in the attestation protocol: the *Controller*, the *Attester*, and the *Challenger*. The *Controller*, implemented as the SDN controller, is responsible for managing traffic flows on the Attester by installing, modifying, deleting, and updating flow rules dynamically during runtime. The *Attester* represents the SDN forwarding device responsible for enforcing flow rules at the data plane. The *Challenger*,

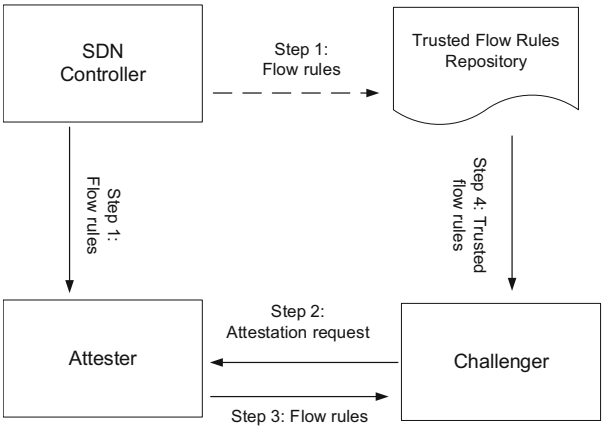


Fig. 3. FlowAttester Attestation Overview.

typically operated by a network administrator or a trusted verification service, initiates attestation requests and validates attestation results.

To support trustworthy verification, the Controller maintains a trusted local repository containing copies of all deployed flow rules and runtime flow updates. This *Trusted Flow Repository* can be used to reconstruct the expected flow-rule table and serves as the ground truth for subsequent attestation activities. Under our assumption that the Attester is exclusively managed by the Controller, the runtime flow states enforced on the forwarding device are expected to remain consistent with the Controller's trusted repository.

The attestation process proceeds as follows:

- **Step 1:** The Controller distributes flow rules to the Attester while simultaneously recording trusted copies of the deployed rules in the Trusted Flow Repository.
- **Step 2:** The Challenger sends an attestation request to the Attester.
- **Step 3:** The Attester generates an attestation proof containing its current runtime flow states and returns the proof to the Challenger.
- **Step 4:** The Challenger retrieves the corresponding trusted flow-rule states from the Controller's Trusted Flow Repository. By comparing the attestation proof against the Controller's ground truth, the Challenger verifies whether the flow rules are correctly enforced at the data plane.

3.1 Challenger

The *Challenger* is responsible for initiating attestation requests to the Attester and validating the returned attestation results. The Challenger can be implemented either as part of the SDN controller or as an independent Trusted Third Party (TTP). When the Challenger is deployed as a TTP, as illustrated in

Fig. 3, a secure communication channel must be established between the TTP and the Controller. In this deployment model, the TTP communicates with the Controller to retrieve trusted copies of flow rules stored in the *Trusted Flow Repository*. These trusted flow-rule states serve as the ground truth for verifying whether the Attester correctly enforces the intended traffic-flow policies during runtime.

Alternatively, the Challenger functionality can be integrated directly into the SDN controller by incorporating additional attestation modules into the controller architecture. In this configuration, the Controller can directly issue attestation requests to the Attester and locally validate the returned attestation proofs. Since the Controller already maintains the Trusted Flow Repository, the additional communication step required for retrieving ground-truth flow states is eliminated, thereby reducing attestation overhead and simplifying the verification process.

3.2 Attester

When receiving an attestation request, the *Attester* is responsible for generating and returning an attestation proof to the Challenger. To ensure the effectiveness and reliability of the attestation process, the attestation proof itself must be trustworthy and tamper-resistant. In addition, the proof must provide sufficient information for verifying whether the traffic-flow rules are correctly enforced at the data plane. Since the collection and reporting of attestation evidence rely on both hardware and software components of the Attester platform, it is necessary to establish a trusted execution chain spanning from low-level hardware to upper-layer software components.

To achieve this goal, we leverage Trusted Computing technologies to construct a platform trust chain. In Trusted Computing architectures, each layer of the software stack measures the integrity of the next layer before transferring execution control. On a typical computing platform, the trust chain is established during the system boot process. When the platform starts, the initial boot instructions are executed within the BIOS. These initial instructions are assumed to be inherently trusted and are referred to as the *Core Root of Trust for Measurement* (CRTM). The CRTM first measures the integrity of the BIOS and records the measurement values into the TPM's Platform Configuration Register (PCR0) and the Secure Measurement Log (SML). Afterward, control is transferred to the BIOS.

The BIOS then measures critical platform configurations, including motherboard firmware, ROM components, and hardware initialization states, and records the corresponding measurements into PCR1, PCR2, and PCR3. Next, the BIOS measures the operating system loader and stores the measurement value in PCR4 before transferring execution control to the OS loader. The OS loader subsequently measures the operating system kernel and records the measurement into PCR5. This measurement-and-transfer process can be recursively extended to higher-level software layers, including user-space applications and runtime services.

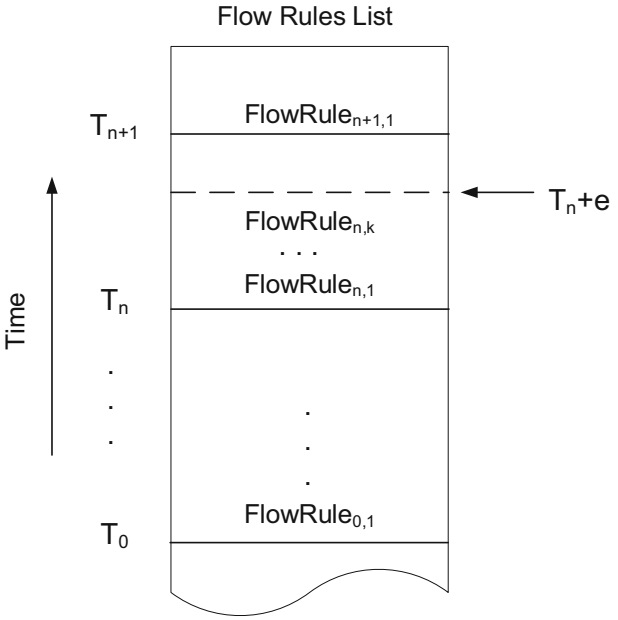


Fig. 4. A Flow Rules List Split by TimeStamp.

Once the trust chain is established, any compromise of measured software components can be detected through integrity verification. For example, if the operating system is compromised, the modification will either be detected during the measurement process by the OS loader or identified by the Challenger during remote attestation when verifying PCR5 and the associated measurement logs. Following the same principle, we extend the trust chain to runtime SDN traffic-flow enforcement states. Due to the hardware protection mechanisms provided by the TPM, PCR values cannot be arbitrarily modified during runtime. As a result, the established trust chain ensures both the integrity of the attestation proof and the trustworthiness of the reported runtime flow states.

3.3 Attestation Protocol

Inspired by the scalable attestation approach proposed by Frederic et al. [19], we introduce timestamp information to partition the flow-rule list into multiple time intervals. Since the size of the flow-rule table can become extremely large in practical SDN environments, we design a periodic and accumulative attestation mechanism that incrementally verifies newly generated flow rules instead of repeatedly attesting the entire flow-rule table from scratch. In our design, the attestation result generated during a previous attestation round can be reused in subsequent attestation operations, significantly improving attestation efficiency and scalability.

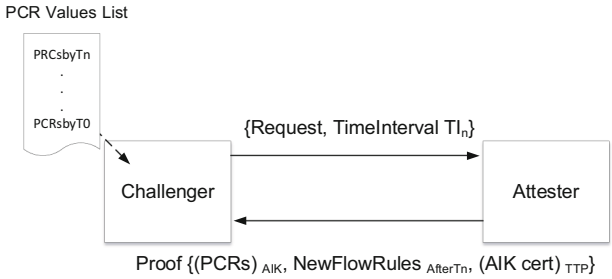


Fig. 5. TimeStamp-based Attestation Protocol.

Figure 4 illustrates a flow-rule list partitioned by timestamps $T_0, ..., T_n, T_{n+1}$. We denote the time interval (T_n, T_{n+1}) as TI_n. Assume that a Challenger C_1 has already attested all flow rules generated before interval TI_n, and the resulting PCR value is V_n. When C_1 initiates a new attestation at time $T_n + e$ ($T_n < T_n + e < T_{n+1}$), only the newly generated flow rules between T_n and $T_n + e$ need to be verified using the previously attested PCR value V_n. Assume the newly generated flow rules are represented as:

$$(FlowRule_{n,1}, FlowRule_{n,2}, ..., FlowRule_{n,k})$$

The attestation calculation is defined as follows:

$$\begin{aligned} NewFlowRules_{n+e} &\leftarrow FlowRule_{n,1}||FlowRule_{n,2}||...||FlowRule_{n,k} \\ PCR[i] &\leftarrow HashAlg(V_n||NewFlowRules_{n+e}) \end{aligned} \tag{1}$$

Here, $FlowRule_{n,1}, FlowRule_{n,2}, ..., FlowRule_{n,k}$ denote the newly generated flow rules between timestamps T_n and $T_n + e$, while $NewFlowRules_{n+e}$ represents the concatenated string of all newly generated flow rules between timestamps T_n and $T_n + e$. The Challenger computes the hash value of the concatenation of the previously verified PCR value V_n and the newly generated $NewFlowRules_{n+e}$ value, and compares the result with the reported PCR value $PCR[i]$. If the reconstructed hash value matches $PCR[i]$, the Challenger concludes that the flow rules are correctly enforced at runtime. Otherwise, the attestation process indicates potential inconsistencies or unauthorized modifications in flow-rule enforcement. Figure 5 illustrates the detailed workflow of the periodic attestation protocol. First, the Challenger sends an attestation request containing a target time interval. The Attester then returns the current PCR value together with the newly generated flow rules after timestamp T_n. Finally, the Challenger verifies the integrity and correctness of the newly enforced flow rules through incremental PCR reconstruction and comparison.

4 Evaluation with OpenFlow Protocol

To evaluate the effectiveness of FlowAttester, we implement a prototype attestation framework in an OpenFlow-based SDN environment. OpenFlow remains one of the most widely adopted SDN protocols and serves as a representative programmable networking architecture for NeoCloud and AI data-center infrastructures. Since OpenFlow is commonly deployed in virtualized environments such as cloud platforms and data centers, we build a software-based OpenFlow testbed for experimental evaluation.

For the SDN forwarding device, we adopt Open vSwitch (OvS), a widely used open-source virtual switch with native OpenFlow support. As the SDN controller, we use NOX [5], an early OpenFlow controller framework that supports both C++ and Python-based network applications. Figure 6 illustrates the deployment architecture of our evaluation environment. The evaluation platform consists of two major components. The components highlighted in green represent the OpenFlow networking infrastructure, including the Open vSwitch forwarding device and the SDN controller. The components highlighted in purple represent the Trusted Computing infrastructure responsible for runtime attestation and integrity verification.

4.1 Trusted Computing Environment

To provide a trustworthy execution environment for runtime attestation activities, we construct a Trusted Computing infrastructure consisting of a Trusted Platform Module (TPM), Trusted-Computing-compatible BIOS, a trusted operating system loader, Integrity Measurement Architecture (IMA) [17], and the Trusted Software Stack (TSS) [20]. Together, these components establish the hardware and software foundation required for secure integrity measurement and remote attestation.

As discussed in the previous sections, the TPM serves as the hardware root of trust and provides essential security functionalities, including the Core Root of Trust for Measurement (CRTM), secure PCR storage, cryptographic operations, and attestation reporting capabilities. The Trusted-Computing-compatible BIOS and trusted OS loader are responsible for establishing the platform trust chain during the secure boot process. By measuring the integrity of subsequent software components before transferring execution control, these components ensure that system integrity can be verified throughout the boot sequence. IMA is a Linux-based integrity measurement framework that measures executable binaries, shared libraries, and other critical runtime files within the operating system. The collected integrity measurements are securely extended into TPM PCR registers and recorded in the Secure Measurement Log (SML), enabling runtime integrity verification through remote attestation. In addition, we employ the Trusted Software Stack (TSS), an open-source implementation of the Trusted Computing Group (TCG) software stack. TSS provides a unified programming interface for interacting with TPM hardware and supports key Trusted Com-

puting operations, including PCR management, TPM quote generation, cryptographic key handling, and remote attestation functionalities.

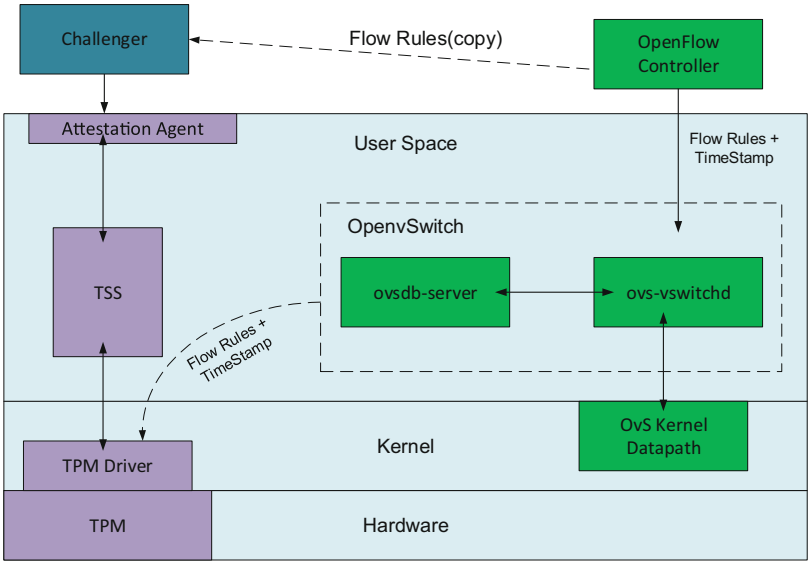


Fig. 6. OpenFlow Rules Attestation on Open vSwitch.

4.2 Flow Rules Attestation on Open vSwitch

The green components shown in Fig. 6 represent the Open vSwitch (OvS) infrastructure used in our evaluation environment. Open vSwitch consists of three major components: *ovs-vswitchd*, *ovsdb-server*, and *OvS kernel module*. The *ovsdb-server* acts as a database service responsible for maintaining OvS-related configuration and management information. The *ovs-vswitchd* daemon serves as the core user-space component that communicates with OpenFlow controllers and coordinates forwarding behaviors across OvS components.

The *OvS kernel module* operates in kernel space and maintains multiple *datapaths*, which can be viewed as cached runtime forwarding states derived from the flow rules configured in user space. For performance optimization, incoming packets first attempt to match flow entries stored within the kernel datapath. If no matching entry exists in the kernel datapath, the packet is forwarded to *ovs-vswitchd* in user space for further processing. If a matching flow rule is identified in user space, the corresponding datapath cache in the kernel is updated so that subsequent packets belonging to the same flow can be processed directly within the kernel. If no matching rule exists within OvS, the packet is forwarded to the OpenFlow controller for additional processing and rule generation.

To support runtime attestation of OpenFlow rules, we map the FlowAttester framework onto the Open vSwitch architecture. OvS provides two management utilities, *ovs-dpctl* and *ovs-ofctl*, for managing the kernel datapath and user-space flow rules, respectively. The *ovs-dpctl* utility directly manipulates kernel datapath entries, while *ovs-ofctl* manages OpenFlow rules maintained by *ovs-vswitchd* in user space. Both utilities can affect runtime forwarding behaviors.

In our design, we assume that the OvS kernel datapath is trusted as part of the Trusted Computing Base (TCB), because the datapath cache is derived from the user-space flow-rule table maintained by *ovs-vswitchd*. Therefore, FlowAttester focuses primarily on attesting the integrity of OpenFlow rules maintained in user space. To facilitate attestation operations, we extend Open vSwitch with additional attestation functionalities. Whenever the SDN controller installs a new flow rule, the rule is augmented with timestamp metadata and recorded in the Controller's Trusted Flow Repository. After *ovs-vswitchd* receives the flow rule, it inserts the rule into the runtime flow table and securely seals the flow rule together with its timestamp information using TPM-protected storage mechanisms.

In addition, we implement a dedicated attestation agent responsible for handling attestation requests. We also extend the SDN controller with a communication module that enables secure transmission of trusted flow-rule copies to the Challenger. Upon receiving an attestation request, the attestation agent initiates a TPM PCR quote operation through the Trusted Software Stack (TSS). The generated PCR values and Secure Measurement Log (SML) are then returned to the Challenger as attestation evidence. To obtain the ground-truth flow-rule states, the Challenger simultaneously retrieves the trusted flow-rule copies from the Controller's Trusted Flow Repository. By comparing the attested runtime flow states against the Controller-maintained ground truth, the Challenger can verify whether the flow rules are correctly enforced on the forwarding device.

4.3 Attestation Results

Table 1 presents the timing overhead of different attestation operations in our evaluation environment. Our attestation framework measures the integrity of the entire Attester platform, including both system software components and SDN runtime flow states. In the current implementation, the attested TPM registers include PCR0, PCR10, and PCR17, which respectively store integrity measurements for the BIOS, Linux Integrity Measurement Architecture (IMA), and Open vSwitch flow-rule states.

The *TPM Quote* time represents the overhead required for generating TPM quote results and retrieving signed PCR values from the TPM. The *Attestation* time measures the verification overhead on the Challenger side, beginning when the Challenger receives the attestation proof and ending when the attestation computation is completed. The *Remote Attestation* time represents the total end-to-end attestation latency, including attestation request generation, proof transmission, TPM quote generation, and final attestation verification.

As shown in Table 1, the overall attestation overhead remains at the second level and is practical for periodic runtime verification in SDN environments. The experimental results demonstrate that FlowAttester can provide continuous runtime flow attestation with acceptable performance overhead for programmable network infrastructures.

Table 1. Timing Costs of Attestation Activities

Activity	Time (ms)
TPM Quote	46.88
Attestation Verification	345.18
End-to-End Remote Attestation	1210.25

5 Related Work

SDN Security and Control-Plane Protection: Existing work has focused on improving the security and reliability of SDN control planes. FortNOX [16] proposed a security enforcement kernel for SDN controllers that prevents malicious applications from installing unauthorized or conflicting flow rules into forwarding devices. VeriFlow [7] introduced a real-time network verification framework capable of detecting forwarding inconsistencies and policy violations before flow rules are deployed to the data plane. Fleet [10] investigated recovery-oriented mechanisms for maintaining controller trustworthiness under compromised SDN administration environments. Additional studies have explored controller hardening, malicious application isolation, and SDN threat analysis [13,14]. While these approaches significantly improve the security of SDN control planes, they generally assume that forwarding devices faithfully enforce controller-issued policies at runtime. In contrast, FlowAttester focuses on runtime verification of data-plane flow-rule enforcement and addresses the enforcement integrity gap between the control plane and forwarding infrastructure.

Existing remote-attestation approaches primarily focus on software integrity, firmware verification, or control-flow validation. In contrast, FlowAttester focuses on runtime attestation of SDN flow-rule enforcement and provides continuous verification of data-plane policy compliance in programmable NeoCloud infrastructures.

Programmable Data Plane Verification and Accountability: With the rapid growth of cloud-native infrastructures and programmable networking technologies, recent studies have increasingly focused on programmable data-plane verification and forwarding accountability. SDNsec [1] proposed a forwarding-accountability framework for SDN data planes by embedding forwarding information directly into packets and enabling path validation. NetDebug [4] introduced a programmable data-plane debugging and verification framework that

supports runtime validation of forwarding behaviors in programmable networks. Recent surveys on programmable data planes [11] have highlighted the increasing importance of programmable network verification and runtime monitoring in large-scale cloud and AI infrastructures. Compared with existing forwarding-verification approaches, FlowAttester leverages Trusted Computing technologies to provide hardware-assisted attestation of runtime flow-rule enforcement on SDN forwarding devices.

Trusted Computing and Remote Attestation: Trusted Computing and remote attestation mechanisms have been widely adopted for establishing platform integrity and trustworthy execution environments. TPM-based remote attestation enables a platform to provide cryptographic proofs of integrity measurements through secure PCR registers and signed attestation reports. Traditional remote-attestation research primarily focuses on software integrity verification, firmware validation, and trusted execution environments. Recent efforts have explored more scalable and flexible attestation mechanisms for modern distributed systems and cloud infrastructures. SEDAT [2] proposed a TPM 2.0-based secure device attestation framework for enhanced runtime integrity verification. DIALED [12] introduced a data-flow attestation mechanism capable of verifying runtime execution integrity on embedded platforms. Recent surveys on control-flow attestation [18] further highlighted the growing importance of runtime behavioral verification in modern computing environments.

More recently, researchers have begun extending remote attestation mechanisms to programmable networks and cloud-native infrastructures. TPM-based continuous attestation frameworks have been proposed for Kubernetes-based virtual network functions and cloud-native infrastructures [3]. In addition, programmable data-plane attestation approaches have emerged for datacenter-scale programmable switching environments [15]. Unlike prior work that primarily attests software binaries, firmware states, or dataplane programs, FlowAttester instead focuses on runtime attestation of SDN flow-rule enforcement and provides continuous verification of data-plane policy compliance in programmable NeoCloud networks.

6 Conclusion and Future Work

In this paper, we presented FlowAttester, a runtime attestation framework for verifying SDN traffic-flow enforcement in programmable network infrastructures. By leveraging Trusted Computing technologies, FlowAttester enables continuous and verifiable attestation of runtime flow behaviors on SDN forwarding devices. We designed a timestamp-based incremental attestation protocol that supports accumulative verification of flow rules. We implemented a prototype of FlowAttester in an OpenFlow-based SDN environment using Open vSwitch. We demonstrated the feasibility of integrating Trusted Computing mechanisms into programmable network infrastructures. Our evaluation results show that FlowAttester can effectively verify runtime flow-rule enforcement with practical performance overhead, making it suitable for programmable cloud and NeoCloud networking environments.

In future work, we plan to extend FlowAttester to support programmable data-plane architectures such as P4-enabled switches, SmartNICs, and cloud-native network functions. As programmable networking infrastructures continue to evolve for cloud-native and AI-oriented applications, we plan to investigate lightweight attestation techniques for large-scale NeoCloud infrastructures and explore continuous runtime verification mechanisms for AI datacenter fabrics. Ultimately, our long-term goal is to leverage Trusted Computing technologies to build trustworthy, resilient, and self-verifiable programmable network infrastructures for next-generation cloud and AI systems.

References

1. Burow, K., Zhang, X., et al.: SDNsec: Forwarding accountability for the SDN data plane. In: Proceedings of the ACM SIGCOMM Conference (2015)
2. Dave, A.: et al.: SEDAT: Security enhanced device attestation with tpm2.0. ArXiv preprint arXiv:2101.06362 (2021)
3. Emran, A.N.B.: TPM-based continuous remote attestation and integrity verification for 5G VNFs on kubernetes (2025). arXiv:2510.03219 arXiv preprint
4. Fei, N.: Netdebug: A programmable data plane verification framework. In: Proceedings of IEEE INFOCOM (2021)
5. Gude, N., et al.: NOX: towards an operating system for networks. Comput. Commun. Rev. **38**(3) (2008)
6. Jacquin, L., Shaw, A.L., Dalton, C.: Towards trusted software-defined networks using a hardware-based integrity measurement architecture. In: 1st IEEE Conference on Network Softwarization (NetSoft). IEEE (2015)
7. Khurshid, A., Zou, X., Zhou, W., Caesar, M., Godfrey, P.B.: Veriflow: verifying network-wide invariants in real time. In: Presented as part of the 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI) (2013)
8. Kreutz, D., Ramos, F., Verissimo, P.: Towards secure and dependable software-defined networks. In: Proceedings of the Second ACM SIGCOMM Workshop on Hot topics in Software Defined Networking, (2013)
9. Lantz, B., Heller, B., McKeown, N.: A network in a laptop: rapid prototyping for software-defined networks. In: Proceedings of the 9th ACM Workshop on Hot Topics in Networks. HotNets (2010)
10. Matsumoto, S., Hitz, S., Perrig, A.: Fleet: defending sdns from malicious administrators. In: Proceedings of theTthird Workshop on Hot topics in software defined networking. In: ACM (2014)
11. Michel, O., Bifulco, R.: The programmable data plane: Abstractions, architectures, algorithms, and applications (2021). arXiv:2110.00631 arXiv preprint
12. Nunes, I.D.O.: Dialed: Data integrity attestation for low-end embedded devices (2021). arXiv:2103.12928 arXiv preprint
13. Open Network Foundation. https://www.opennetworking.org/images/stories/downloads/sdn-resources/technical-reports/Threat_Analysis_for_the_SDN_Architecture.pdf
14. Open Network Foundation. https://www.opennetworking.org/images/stories/downloads/sdn-resources/technical-reports/Security_Foundation_Requirements_for_SDN_Controllers.pdf

15. Pierini, R.: Implementation and evaluation of the Forro stream Cipher in Tofino programmable hardware for remote attestation in datacenters. J. Braz. Comput. Soc. (2026)
16. Porras, P., Shin, S., Yegneswaran, V., Fong, M., Tyson, M., Gu, G.: A security enforcement kernel for openflow networks. In: Proceedings of the First Workshop on Hot Topics in Software Defined Networks, ACM (2012)
17. Sailer, R., Zhang, X., Jaeger, T.: and L. Van Doorn. Design and implementation of a tcg-based integrity measurement architecture, In USENIX Security Symposium (2004)
18. Sha, Z.: Control-flow attestation: Concepts, solutions, and open challenges (2024). arXiv:2408.06304 arXiv preprint
19. Stumpf, F., Fuchs, A., Katzenbeisser, S., Eckert, C.: Improving the scalability of platform attestation. In: Proceedings of the 3rd ACM workshop on Scalable trusted computing (2008)
20. The open-source TCG Software Stack. http://trousers.sourceforge.net/
21. Trusted Network Connect. http://www.trustedcomputinggroup.org/work-groups/trusted-network-communications/open-standards-tnc/
22. Trusted Platform Module. http://www.trustedcomputinggroup.org/work-groups/trusted-platform-module/
23. Wang, L., Wu, D.: Moving target defense against network reconnaissance with software defined networking. In: International Conference on Information Security, Springer (2016)

Machine Learning Techniques for Predominant Musical Instrument Recognition in Polyphonic Audio: A Survey of Time-Frequency Representations and Deep Learning Architectures

Christian Cimaglia and Muhammad Abid(✉)

Florida Polytechnic University, Lakeland, FL, USA
{ccimaglia2934,mabid}@floridapoly.edu

Abstract. Predominant musical instrument recognition in polyphonic audio is an important task in Music Information Retrieval, with applications in music indexing, transcription, recommendation, and semantic audio analysis. This survey reviews the evolution of machine learning techniques for this task, beginning with handcrafted acoustic features and traditional classifiers, followed by convolutional neural networks operating on time–frequency representations, and more recent transformer, attention-based, involution-based, and modular deep learning architectures. The review highlights how the field has shifted from manually designed features toward learned representations capable of modeling complex temporal, spectral, and timbral patterns in polyphonic music. A comparative discussion of representative methods is provided, with emphasis on datasets, input representations, model design, reported performance, and remaining limitations. The survey identifies several open challenges, including efficient long-range temporal modeling, parameter-efficient architecture design, inconsistent evaluation protocols, and the trade-off between recognition performance and computational complexity. These gaps suggest the need for adaptive and efficient time–frequency modeling approaches for future instrument recognition systems.

Keywords: Music Information Retrieval · Predominant Instrument Recognition · Polyphonic Audio · Convolutional Neural Networks · Time–Frequency Representations · Deep Learning

1 Introduction

Music Information Retrieval (MIR) is a broad research area concerned with the computational analysis, organization, and understanding of music signals. It supports a wide range of applications, including music recommendation, genre recognition, source separation, automatic transcription, music tagging, and content-based music search [10,19]. Among these tasks, musical instrument recognition

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 218–231, 2027.
https://doi.org/10.1007/978-3-032-39763-8_13

plays an important role because identifying the instruments present in an audio recording can improve semantic music indexing, transcription accuracy, and higher-level music understanding [10].

Musical instrument recognition is relatively more tractable in monophonic recordings, where a single instrument is present at a time. In contrast, predominant instrument recognition in polyphonic audio is significantly more challenging because multiple instruments may overlap in time and frequency, producing complex mixtures of harmonic, timbral, and temporal patterns [10,19]. This problem is especially important because most real-world music is polyphonic rather than monophonic. Therefore, recognizing one or more predominant instruments in polyphonic audio remains a relevant and challenging problem within MIR.

Early approaches to instrument recognition relied heavily on handcrafted acoustic features such as spectral, temporal, cepstral, and phase-based descriptors, followed by traditional classifiers such as Gaussian mixture models, support vector machines, and k-nearest neighbors [4–6]. While these methods established important foundations, their performance was limited by the quality of manually designed features and by the representational capacity of shallow learning models. The introduction of benchmark datasets such as IRMAS helped shift the field toward more realistic evaluation on real polyphonic music recordings [2,10].

More recently, deep learning methods, particularly convolutional neural networks (CNNs), have become prominent for predominant instrument recognition. These methods commonly transform raw audio into time–frequency representations such as spectrograms or Mel spectrograms, allowing CNNs to learn discriminative patterns from audio in a form similar to structured images [10,19]. Han et al. demonstrated that CNNs applied to Mel spectrograms can outperform earlier shallow-learning approaches on the IRMAS dataset, establishing CNN-based time–frequency modeling as a strong direction for this task [10]. Subsequent work has explored data augmentation, source separation, transfer learning, transformer-based models, attention mechanisms, and hybrid architectures to further improve recognition performance [3,9,13,19,20,23].

Despite these advances, several challenges remain. Many deep learning approaches rely on conventional CNN architectures that were originally designed for natural images rather than audio-derived time–frequency representations. As a result, they may not fully exploit the temporal and spectral structure of polyphonic music signals. Other approaches incorporate domain knowledge through preprocessing or attention-based mechanisms, but often introduce additional computational complexity [17–19]. This creates an important research need for models that can effectively capture long-range temporal and timbral patterns while maintaining computational and parameter efficiency.

While numerous research papers summarize related work as part of introducing new algorithms, to the best of our knowledge, there is no dedicated survey that traces the evolution of approaches in polyphonic instrument recognition. Our survey fills this gap by providing a comparative analysis of methodological trends across successive generations of machine learning advances, considering

the computational and architectural trade-offs that each new wave of experiments introduced.

The evolution of machine learning techniques for predominant musical instrument recognition in polyphonic audio is summarized in Fig. 1. It begins with early handcrafted-feature methods and traditional classifiers, then examines CNN-based approaches using time–frequency representations, followed by more recent transformer, attention-based, involution-based, and modular deep learning architectures. The goal of this survery is to synthesize major trends, compare methodological strengths and limitations, and identify open research gaps related to efficient time–frequency modeling for instrument recognition.

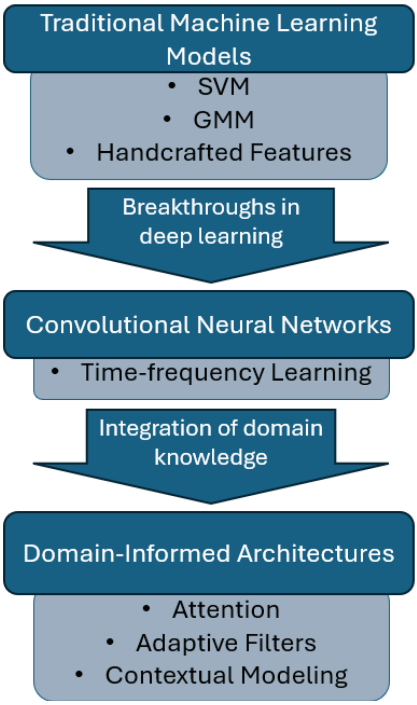


Fig. 1. Evolution of prominent approaches for polyphonic instrument recognition.

2 Survey Methodology

Papers had to meet several criteria in order to be considered for inclusion in this survey. First, their focus had to either: center around the task of instrument recognition in polyphonic samples; introduce datasets directly related to this task; or propose methodologies that would influence future experiments surrounding the task. Particular emphasis was placed on experiments that

used lightweight and computationally efficient architectures that exploit time–frequency structure through architectural design rather than relying solely on large model capacity. Also of importance is that the survey primarily considers work published between 2000 and 2026, a period that encompasses the emergence of polyphonic instrument recognition and the subsequent evolution toward modern deep learning methods. To locate papers, we used well-regarded databases such as IEEExplore, SpringerLink, and ResearchGate to conduct keyword searches, and we also utilized a combination of backward and forward citation chasing to find papers that expanded upon the work of others. It should be noted that while the progression of this survey loosely follows a chronologic structure, papers tend to be grouped by the similarities in their methodology rather than following a strict timeline.

3 Early Approaches

3.1 Classification of Monophonic Samples

Early research on musical instrument recognition primarily focused on monophonic audio, where a single instrument is present at a time. These studies were important because they established many of the acoustic feature extraction strategies that later influenced polyphonic instrument recognition. Eronen and Klapuri investigated a broad set of spectral and temporal descriptors, including cepstral coefficients, and used Gaussian and k-nearest neighbor classifiers for instrument recognition [5]. Their work demonstrated that carefully designed timbral features could provide meaningful discrimination among orchestral instruments.

Subsequent studies expanded the feature space by incorporating additional signal characteristics. Diment et al. introduced the modified group delay feature to capture phase-related information and combined it with conventional features such as Mel-frequency cepstral coefficients (MFCCs) for musical instrument recognition [4].

Approaches that expanded their feature representation in this way generally yielded better results than experiments with smaller feature spaces. This trend is unsurprising given that musical signals contain multiple complementary sources of structural information, including timbral, harmonic, temporal, and spectral characteristics. However, the introduction of more features came with the trade-off of increased computational cost in preprocessing. Furthermore, most experiments to this point were conducted on single notes rather than solo phrases which limited their practical application for real music. These limitations motivated subsequent studies to consider solo phrases rather than isolated notes, necessitating the use of more sophisticated methods.

Krishna and Sreenivas were among the first to study instrument recognition in solo phrases, proposing line spectral frequencies as an alternative representation and evaluating them with Gaussian mixture models and k-nearest neighbor classifiers [14]. Similarly, Essid et al. explored solo-performance classification using MFCC-based representations, dimensionality reduction through principal

component analysis, and classifiers such as Gaussian mixture models and support vector machines [6].

Overall, these monophonic studies show that early instrument recognition depended heavily on manually designed acoustic features and shallow classifiers. Although these approaches provided useful foundations, they were limited by the need for handcrafted feature engineering and by the restricted representational capacity of traditional models. These limitations became more significant when moving from monophonic audio to polyphonic music, where overlapping instruments create more complex spectral and temporal interactions.

3.2 Traditional Models Applied to Polyphonic Classification

The transition from monophonic to polyphonic instrument recognition introduced additional challenges because multiple instruments may overlap in both time and frequency. Early work addressed this difficulty using source separation, feature weighting, and traditional machine learning models. Heittola et al. used a source-filter model for sound separation before performing musical instrument recognition in polyphonic audio . This source separation extracted individual sound sources from overlapping instruments, making the resulting features resemble monophonic samples which are inherently easier to classify [11].

Kitahara et al. proposed a feature weighting strategy designed to reduce the influence of overlapping instrument sounds in polyphonic mixtures. By removing overlapping sounds, the algorithm was able to hone in on sections of the track in which one instrument is clearly dominant, making the data more closely resemble monophonic samples [21].

These approaches represented important steps toward handling realistic mixtures, but they also introduced limitations. Given that source separation and feature weighting aimed to remove overlapping sounds in the tracks, errors introduced in the process could unintentionally remove informative spectral components belonging to the target instruments. This problem is especially relevant given that their evaluation often relied on artificially generated polyphonic samples or controlled mixtures while, in contrast, commercial recordings exhibit substantially more frequent and complex overlap among instruments. This prompted the need for studies that operate on more realistic polyphonic music data.

Fuhrmann and Herrera studied polyphonic instrument recognition for exploring semantic similarities in music, using traditional classification methods on real music recordings [7]. Bosch et al. further advanced the field by comparing sound segregation techniques for predominant instrument recognition in musical audio signals and by introducing the IRMAS dataset, which later became a common benchmark for evaluating predominant instrument recognition methods [2]. Training samples in the dataset are three seconds in length and labeled with a single predominant instrument, and test samples, ranging between five and twenty seconds in length, are labeled with multiple predominant instruments. The availability of IRMAS helped shift the field toward more practical evaluation settings involving real polyphonic music.

Despite the collective advances of these studies, traditional polyphonic approaches still relied heavily on handcrafted features, explicit source separation, or manually designed weighting mechanisms. As a result, their performance was constrained by the quality of the preprocessing and feature design. As datasets became more realistic and polyphonic mixtures more complex, this strategy exhibited diminishing returns. This motivated the later adoption of deep learning methods, particularly convolutional neural networks operating on time–frequency representations, which could learn discriminative features directly from spectrogram-like inputs.

4 Use of Convolutional Neural Networks

4.1 Conventional CNN Architectures

By the mid-2010s, convolutional neural networks (CNNs) had become widely adopted in computer vision, motivating their application to audio classification tasks through time–frequency representations (TFRs). In this setting, raw audio signals are transformed into spectrogram-like images, allowing CNNs to process temporal and spectral patterns using convolutional filters. This approach became especially relevant for predominant instrument recognition because polyphonic music contains overlapping harmonic and timbral structures that are difficult to model using handcrafted features alone.

Han et al. were among the first to demonstrate the effectiveness of CNNs for predominant instrument recognition in polyphonic music using the IRMAS dataset [10]. Their preprocessing pipeline converted raw audio into Mel spectrograms using the Short-Time Fourier Transform (STFT), enabling the model to learn discriminative time–frequency patterns directly from the input representation. Their work showed that CNN-based approaches could outperform earlier traditional machine learning methods that relied on manually extracted features and shallow classifiers [2,10]. Gururani et al. further supported this direction by applying deep neural network models to instrument activity detection in polyphonic music, including CNN, recurrent neural network (RNN), and hybrid CNN–RNN architectures. In their experimentation, the CNN model outperformed the other architectures, which the authors attributed to the music clips being too short for the RNN to exploit its ability to model long-range temporal dependencies. This influenced the continued use of the CNN over other prominent deep learning models [8].

These early CNN-based studies established an important shift in the field: instead of relying primarily on handcrafted acoustic descriptors, models could learn features directly from time–frequency inputs. However, many of these approaches followed conventional CNN design principles originally developed for natural images and therefore incorporated less music-specific domain knowledge than many pre-deep learning systems. As a result, they did not fully exploit the distinctive structure of audio-derived representations, where one axis corresponds to time and the other to frequency. Thus, later CNN-based studies were motivated to incorporate domain knowledge either through preprocessing, data augmentation, or architecture design.

4.2 Incorporation of Domain Knowledge Into CNN Implementations

Several subsequent works incorporated domain knowledge primarily at the preprocessing stage. Gomez-Canon et al. explored the use of source separation and transfer learning for jazz solo instrument classification. Their use of source separation demonstrated that separating musical components could improve the quality of input representations for CNN-based classification which was consistent with earlier non-deep learning approaches. [9]. Kratimenos et al. investigated data augmentation methods for instrument classification in polyphonic music, including strategies that generate additional training examples by combining monophonic audio tracks to improve model robustness. Their training experiments on the augmented dataset yielded slightly improved results over the initial dataset, although they required additional computational overhead to create the synthetically generated training samples. Furthermore, the effectiveness of the samples was limited by the fact that mixing unrelated monophonic tracks does not necessarily reflect the coordinated interactions among instruments in real musical performances [13]. Li et al. replaced the conventional Fourier-based representation with the Hilbert–Huang Transform, combining it with deep CNNs for predominant musical instrument recognition, and results demonstrated improvement over benchmark figures [16].

These studies demonstrate that improved input representations and augmentation strategies can enhance CNN performance, although such techniques were limited by computational overhead necessary to implement them in preprocessing. Additionally, while domain knowledge was beginning to be used in these ways, the core machine learning architectures generally remained conventional rather than leveraging signal processing or music principles.

In contrast, Pons and collaborators investigated how domain knowledge could be incorporated directly into CNN architecture design. Pons and Serra argued that standard small rectangular filters are not always appropriate for music spectrogram classification and proposed efficient CNN architectures with filters shaped to capture temporal features [17]. In related work, Pons et al. designed convolutional filters informed by timbral characteristics of music audio signals, demonstrating that filter shape and receptive field design are important considerations when applying CNNs to time–frequency representations [18]. These studies are especially relevant because they highlight a central issue in CNN-based audio modeling: the shape and extent of convolutional filters strongly affect the model's ability to capture temporal and spectral structure. However, despite the promise of this new strategy, it came with the key limitation that larger or elongated filters generally increase computational and memory requirements because they involve more trainable parameters and larger convolution operations.

Koutini et al. further examined the role of receptive fields in CNN-based music classification and tagging by proposing receptive-field regularized CNNs [12]. Unlike approaches that increase receptive field size through larger filters, their work suggested that controlling receptive-field growth can be important for preserving local time–frequency information, particularly in deeper CNN

architectures. This line of research shows that CNN performance depends not only on the input representation but also on how the architecture processes different temporal and spectral scales.

Yu et al. advanced CNN-based predominant instrument recognition by introducing auxiliary classification into a deep neural network framework [23]. Their approach incorporated additional learning objectives, batch normalization, and center loss to improve recognition performance on the IRMAS dataset. This work demonstrated that optimization strategies and auxiliary supervision can substantially improve CNN-based instrument recognition. However, such improvements do not directly address the architectural question of how convolutional filters should be shaped or structured to efficiently capture long-range temporal dependencies.

Overall, CNN-based methods represent a major advance over traditional feature-based approaches for predominant instrument recognition. They enable models to learn discriminative patterns from time–frequency representations and have achieved strong results on benchmark datasets such as IRMAS. A consistent trend is that methods incorporating music-specific domain knowledge—whether through preprocessing or architectural design—generally outperform generic CNN implementations. However, these improvements often come at the cost of increased preprocessing complexity, additional parameters, or more specialized network designs. Furthermore, a key limitation remains: many CNN architectures still rely on dense, fixed-shape filters that may be inefficient for capturing long-range temporal structure in polyphonic music. This motivates further investigation into adaptive and parameter-efficient convolutional mechanisms, including irregular or task-adaptive kernels that can better balance representational power and computational efficiency.

5 Alternative Deep Learning Architectures

Although CNN-based methods substantially improved predominant instrument recognition, recent studies have explored alternative deep learning architectures to better capture long-range dependencies, multi-scale patterns, and complementary audio representations. These approaches include transformer-based models, attention mechanisms, compact convolutional-transformer designs, involution-based feature learning, and modular CNN systems. Collectively, they reflect a broader trend toward moving beyond conventional fixed-filter CNNs and toward more adaptive modeling of time–frequency structure.

Reghunath and Rajan proposed a transformer-based ensemble method for multiple predominant instrument recognition in polyphonic music [19]. Their approach used multiple visual representations, including Mel spectrograms, modgdgrams, and tempograms, with decision-level fusion across transformer-based classifiers. This design is significant because different representations emphasize different properties of music signals: Mel spectrograms capture spectral energy distribution, modgdgrams incorporate phase-related information, and tempograms represent rhythmic or tempo-related structure. Their work

shows that transformer-based architectures can improve recognition by modeling broader contextual relationships across complementary time–frequency representations. One practical limitation of this design is the increased computational overhead of converting the audio tracks to three separate time-frequency representations instead of a single representation [19].

Subsequent work extended this direction through compact convolutional transformers for multiple predominant instrument recognition [3]. These models combine convolutional layers for local feature extraction with transformer-based components for contextual modeling. The design addresses a common limitation of conventional CNNs, which are effective at learning local time–frequency patterns but may struggle to capture longer-range relationships. At the same time, compact transformer designs aim to reduce the computational burden typically associated with full transformer architectures. Other recent studies have explored architectures that dynamically adapt feature extraction. Reghunath et al. proposed cross-attentive CNNs for joint spectral and pitch feature learning in predominant instrument recognition [20]. Their dual-branch approach reflects the idea that spectral and pitch-related cues can provide complementary information for distinguishing instruments in polyphonic mixtures.

These studies highlight the active trend of combining standard CNN architectures with transformers or other attention-based models. While this setup introduces the trade-off of increasing overall model complexity, the complementary strengths the strategy offer have the potential to improve recognition performance. Specifically, transformers have the ability to compensate for CNNs' limited receptive fields by modeling longer-range contextual relationships.

Similarly, involution-based models have been proposed to introduce input-dependent feature learning for predominant instrument recognition [15]. Unlike standard convolution, which applies fixed learned kernels across spatial locations, involution-style operations generate location-specific kernels, making them conceptually relevant to adaptive time–frequency modeling. Additionally, unlike transformer-based methods, involution introduces adaptive feature extraction without requiring explicit self-attention mechanisms. Consequently, it offers a potentially attractive compromise between modeling flexibility and computational efficiency.

Attention-based methods have also been applied to related musical instrument recognition problems. In weakly labeled multi-instrument recognition, OpenMIC-2018 has been used to study visual attention mechanisms for identifying relevant time–frequency regions [22]. Unlike IRMAS, which is commonly used for predominant instrument recognition, OpenMIC-2018 provides a larger collection of polyphonic audio clips with weak labels for multiple instrument categories. Watcharasupat et al. used attention to address the challenge that instrument activity may occur only in limited portions of an audio excerpt, making it important for the model to focus on informative regions rather than treating the entire representation uniformly [22].

Modular CNN systems represent another alternative architecture. Blaszke and Kostek proposed an approach based on individual CNNs for detecting instru-

ments present in an audio excerpt [1]. Rather than using a single shared classifier for all instruments, this strategy uses instrument-specific networks, allowing each model to specialize in the characteristics of a particular instrument. While this can improve flexibility, it may also increase the number of trained models and the overall computational cost. Furthermore, it may complicate practical maintenance because adding a new instrument category requires training and managing an additional network rather than extending a single unified model.

Overall, alternative deep learning architectures have expanded the design space for instrument recognition by incorporating contextual modeling, attention, dynamic feature extraction, and modular classification. These methods show the value of moving beyond conventional CNNs, but they also introduce trade-offs related to complexity, parameter count, training cost, and interpretability. Therefore, a key open direction is the development of models that retain the adaptability of these approaches while remaining computationally efficient. This motivates further investigation into parameter-efficient, task-adaptive mechanisms for time–frequency representations.

6 Comparative Analysis of Existing Methods

Comparing existing methods for predominant musical instrument recognition is not straightforward because studies differ in dataset choice, input representation, training strategy, evaluation protocol, and target task. Some methods focus specifically on predominant instrument recognition using IRMAS, while others address weakly labeled recognition using datasets such as OpenMIC-2018. In addition, some studies report micro and macro F1 scores, whereas others report precision, accuracy, AUC, or instrument-specific results. We emphasize F1 scores as a primary metric because they balance precision and recall, penalizing models that either predict too many predominant instruments per sample or fail to identify instruments that are actually present. Consequently, models achieve high F1 scores only when they correctly identify most of the predominant instruments present while minimizing incorrect predictions. Furthermore, the use of both micro and macro measures accounts for the class imbalance present in the IRMAS dataset. Models that can identify underrepresented instruments in the dataset will yield a high F1 macro score, while models that struggle in that regard but are otherwise robust will have a comparatively high F1 micro score.

The comparison in Table 1 emphasizes the methodological contribution of each approach together with the reported performance when directly available.

The comparison shows a clear progression from handcrafted feature-based methods toward deep learning models operating on time–frequency representations. CNN-based methods improved over traditional systems by learning discriminative patterns directly from Mel spectrograms [10]. Later studies further improved recognition by using data augmentation, auxiliary supervision, attention, transformer-based contextual modeling, and multiple complementary input representations [19,22,23]. However, these improvements often introduce additional architectural complexity or substantial computational overhead in preprocessing or training.

Table 1. Comparison of selected approaches for musical instrument recognition in polyphonic audio.

Study	Dataset	Input Representation	Model Type	Main Contribution	Reported Performance
Bosch et al. [2]	IRMAS	Handcrafted timbral features with source separation	Traditional ML/SVM	Compared sound segregation techniques for predominant instrument recognition	Micro F1: 0.50; Macro F1: 0.43
Han et al. [10]	IRMAS	Mel spectrogram	CNN	Established CNN-based predominant instrument recognition on time–frequency inputs	Micro F1: 0.62; Macro F1: 0.51
Pons and Serra [17]	Music spectrogram datasets	Spectrogram	CNN with onset/pattern-informed filter shapes	Studied temporal filter shapes and efficient CNN design for music classification	Not directly comparable
Pons et al. [18]	IRMAS	Log-Mel spectrogram	CNN with timbre-informed filter shapes	Investigated timbre-informed filter shapes and receptive field design	Micro F1: 0.59; Macro F1: 0.51
Yu et al. [23]	IRMAS	Mel Spectrogram, handcrafted features	CNN with auxiliary classification	Used auxiliary classification, batch normalization, and center loss to improve recognition	Micro F1: 0.69; Macro F1: 0.60
Reghunath and Rajan [19]	IRMAS	Mel spectrogram, modgdgram, tempogram	Transformer ensemble	Used multiple visual representations with transformer-based decision-level fusion	Micro F1: 0.66; Macro F1: 0.62
Watcharasupat et al. [22]	OpenMIC 2018	Time–frequency features/visual representation	Attention-based model	Applied visual attention to weakly labeled multi-instrument recognition	Not directly comparable
Blaszke and Kostek [1]	Custom/multi-instrument data	MFCC-based representation	Modular CNN system	Used separate CNNs for instrument-specific identification	Not directly comparable

A key trend across the literature is the increasing importance of modeling temporal and spectral structure more effectively. Standard CNNs are effective at learning local patterns, but their fixed receptive fields may limit their ability to capture longer temporal dependencies. Architectures that use elongated filters, attention, or transformers address this issue to some extent, but they may also increase computational cost or parameter requirements [17–19]. This suggests that future work should focus not only on improving recognition accuracy but also on designing efficient architectures that can capture long-range temporal patterns while being parameter-efficient.

7 Conclusion and Future Directions

Although substantial progress has been made in predominant instrument recognition, several important challenges remain. First, efficient long-range temporal modeling is still an open challenge. Time–frequency representations contain structured temporal and spectral patterns, but many CNN-based models rely on fixed, dense, and relatively small convolutional filters [10,17,18]. While larger or elongated filters can increase temporal coverage, they may also increase the number of parameters and memory requirements, creating a trade-off between representational power and computational efficiency [17,18].

Second, many recent methods improve performance by adding architectural complexity, such as auxiliary learning objectives, transformer-based ensembles,

attention mechanisms, or multiple input representations [3,19,20,23]. These methods demonstrate the value of richer contextual modeling, but they may also increase training cost, model size, or implementation complexity. Therefore, future research should focus on designing models that achieve strong performance with fewer parameters and lower computational overhead.

One promising avenue to investigate is adaptive convolutional mechanisms that can learn task-specific filter shapes, selectively emphasize important time–frequency regions, and reduce unnecessary parameters. Such approaches may provide a middle ground between conventional CNNs and more complex attention or transformer-based systems.

Finally, future studies should adopt more consistent evaluation practices by reporting not only predictive performance but also parameter counts, computational costs, and training complexity. Such information would facilitate more meaningful comparisons between methods and better reflect their suitability for practical deployment.

Overall, prospective research in predominant instrument recognition should seek to balance representational power with computational efficiency. While increasingly sophisticated architectures have consistently improved recognition performance, they have also introduced greater computational and implementation complexity. Developing parameter-efficient, task-adaptive models that better exploit the structure of time–frequency representations therefore represents one of the most promising directions for future research.

References

1. Blaszke, M., Korvel, G., Kostek, B.: Exploring neural networks for musical instrument identification in polyphonic audio. IEEE Intell. Syst. **39**(5), 25–36 (2024). https://doi.org/10.1109/MIS.2024.3392586
2. Bosch, J., Janer, J., Fuhrmann, F., Herrera, P.: A comparison of sound segregation techniques for predominant instrument recognition in musical audio signals. In: Proceedings of the 13th International Society for Music Information Retrieval Conference, ISMIR 2012, pp. 559–564 (01 2012)
3. C. R., L., Rajan, R.: Compact convolutional transformers for multiple predominant instrument recognition in polyphonic music. In: 2024 9th International Conference on Communication and Electronics Systems (ICCES), pp. 01–06 (2024). https://doi.org/10.1109/ICCES63552.2024.10860152
4. Diment, A., Rajan, P., Heittola, T., Virtanen, T.: Modified group delay feature for musical instrument recognition. In: 10th International Symposium on Computer Music Multidisciplinary Research, 15- 18 October 2013, Marseille, France, pp. 431–438. International Symposium on Computer Music Multidisciplinary Research, LMA (2013), contribution: organisation=sgn,FACT1=1¡br/¿Portfolio EDEND: 2013-12-29¡br/¿Publisher name: LMA
5. Eronen, A., Klapuri, A.: Musical instrument recognition using cepstral coefficients and temporal features. In: 2000 IEEE International Conference on Acoustics, Speech, and Signal Processing. Proceedings (Cat. No.00CH37100), vol. 2, pp. II753–II756 vol.2 (2000). https://doi.org/10.1109/ICASSP.2000.859069
6. Essid, S., Richard, G., David, B.: Musical instrument recognition on solo performances. In: 12th European Signal Processing Conference, pp. 1289–1292 (2004)

7. Fuhrmann, F., Herrera, P.: Polyphonic instrument recognition for exploring semantic similarities in music (2010). https://api.semanticscholar.org/CorpusID:18576156
8. Gururani, S., Summers, C., Lerch, A.: Instrument activity detection in polyphonic music using deep neural networks (04 2019)
9. Gómez-Cañón, J., Abeßer, J., Cano, E.: Jazz solo instrument classification with convolutional neural networks, source separation, and transfer learning (07 2018)
10. Han, Y., Kim, J., Lee, K.: Deep convolutional neural networks for predominant instrument recognition in polyphonic music. IEEE/ACM Trans. Audio, Speech, Lang. Process. **25**(1), 208–221 (2017). https://doi.org/10.1109/TASLP.2016.2632307
11. Heittola, T., Klapuri, A., Virtanen, T.: Musical instrument recognition in polyphonic audio using source-filter model for sound separation, pp. 327–332 (01 2009)
12. Koutini, K., Eghbal-Zadeh, H., Haunschmid, V., Primus, P., Chowdhury, S., Widmer, G.: Receptive-field regularized cnns for music classification and tagging (2020). https://arxiv.org/abs/2007.13503
13. Kratimenos, A., Avramidis, K., Garoufis, C., Zlatintsi, A., Maragos, P.: Augmentation methods on monophonic audio for instrument classification in polyphonic music. In: 2020 28th European Signal Processing Conference (EUSIPCO), pp. 156–160 (2021). https://doi.org/10.23919/Eusipco47968.2020.9287745
14. Krishna, A., Sreenivas, T.: Music instrument recognition: from isolated notes to solo phrases. In: 2004 IEEE International Conference on Acoustics, Speech, and Signal Processing, vol. 4, pp. iv–iv (2004). https://doi.org/10.1109/ICASSP.2004.1326814
15. Lekshmi, C.R., Dandamudi, J.T.: Dynamic feature learning with involution and convolution for predominant instrument recognition in polyphonic music. Circ. Syst. Signal Process. **44**(9), 6698–6715 (2025). https://doi.org/10.1007/s00034-025-03111-y
16. Li, X., Wang, K., Soraghan, J., Ren, J.: Fusion of Hilbert-Huang transform and deep convolutional neural network for predominant musical instruments recognition. In: Romero, J., Ekárt, A., Martins, T., Correia, J. (eds.) EvoMUSART 2020. LNCS, vol. 12103, pp. 80–89. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-43859-3_6
17. Pons, J., Serra, X.: Designing efficient architectures for modeling temporal features with convolutional neural networks. In: 2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 2472–2476 (2017).https://doi.org/10.1109/ICASSP.2017.7952601
18. Pons, J., Slizovskaia, O., Gong, R., Gómez, E., Serra, X.: Timbre analysis of music audio signals with convolutional neural networks. In: 25th European Signal Processing Conference (EUSIPCO), pp. 2744–2748 (2017). https://doi.org/10.23919/EUSIPCO.2017.8081710
19. Reghunath, L.C., Rajan, R.: Transformer-based ensemble method for multiple predominant instruments recognition in polyphonic music. EURASIP J. Audio, Speech, and Music Process. **2022**(1), 11 (2022)
20. Reghunath, L.C., Rajan, R., Napoli, C., Randieri, C.: Cross-attentive CNNs for joint spectral and pitch feature learning in predominant instrument recognition from polyphonic music. Technologies **14**(1) (2026). https://doi.org/10.3390/technologies14010003, https://www.mdpi.com/2227-7080/14/1/3

21. Kitahara, T., Goto, M., Komatani, K., Ogata, T., Okuno, H.G.: Instrument Identification in Polyphonic Music: Feature Weighting to Minimize Influence of Sound Overlaps. EURASIP J. Adv. Signal Process. **2007**(1), 1–15 (2007). https://doi.org/10.1155/2007/51979
22. Watcharasupat, K.N., Gururani, S., Lerch, A.: Visual attention for musical instrument recognition. arXiv preprint arXiv:2006.09640 (2020). https://arxiv.org/abs/2006.09640
23. Yu, D., Duan, H., Fang, J., Zeng, B.: Predominant instrument recognition based on deep neural network with auxiliary classification. IEEE/ACM Trans. Audio, Speech, Lang. Process. **28**, 852–861 (2020). https://doi.org/10.1109/TASLP.2020.2971419

Affective and Behavioral Computing for Autism Spectrum Disorder Detection: A Survey

Jillienne Lapid, Muhammad Abid(✉), and Md Selim

Florida Polytechnic University, Lakeland, FL, USA
{jlapid2719,mabid,mselim}@floridapoly.edu

Abstract. Early detection of Autism Spectrum Disorder (ASD) is critical for timely intervention and improved outcomes, yet traditional diagnostic methods remain resource-intensive and subjective. This survey systematically reviews artificial intelligence approaches for automated ASD detection, including facial analysis, behavioral video analysis, eye-tracking and gaze behavior, body pose and movement, self-stimulatory behavior recognition, and brain imaging methods. We examine feature extraction methods ranging from hand-crafted descriptors to deep-learned representations, and classification using machine learning and deep learning architectures. The survey categorizes datasets into pre-training sources, general ASD-focused repositories, and study-specific clinical collections. We address critical challenges including limited sample sizes, class imbalance, gender bias in neuroimaging, multi-site variability, diagnostic validation inconsistencies, and real-world generalization gaps. This comprehensive analysis provides a roadmap for developing robust, clinically relevant AI systems supporting early ASD screening and personalized intervention.

Keywords: Autism Spectrum Disorder · Machine Learning (ML) · behavioral analysis

1 Introduction

Autism Spectrum Disorder (ASD) encompasses a range of neurodevelopmental conditions characterized by impaired social interaction, challenges in interpreting nonverbal cues (e.g., gaze, facial expressions), difficulties with emotional reciprocacy, language deficits, restricted interests, repetitive behaviors, and a preference for routine and predictability [2]. ASD is typically diagnosed around 61 months of age, though symptoms often emerge earlier [33]. Compared to typically developing (TD) children, those with ASD exhibit reduced joint attention, delayed responses to name-calling, atypical emotional reciprocity, and stereotypical motor behaviors such as hand flapping and body rocking. Early identification is critical because the developing brain in early childhood is particularly receptive

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 232–252, 2027.
https://doi.org/10.1007/978-3-032-39763-8_14

to learning social, emotional, and behavioral skills. Early therapeutic interventions, including speech, behavioral, and play-based therapies, can significantly improve outcomes in social communication, emotional regulation, and adaptive functioning [44], making timely and accurate diagnosis essential for maximizing developmental potential and quality of life.

In recent years, artificial intelligence (AI) and computer vision have emerged as promising assisting tools to support early ASD screening, detection, and clinical decision-making. Computational approaches offer several advantages: they can process large volumes of behavioral and physiological data objectively, identify subtle patterns that may be missed by human observers, and provide assistive tools for clinicians in resource-constrained settings. These systems typically analyze observable features such as facial structure and expressions, eye-gaze patterns, head pose dynamics, body movements, and in clinical settings brain imaging data (e.g., structural and functional MRI). Advances in deep learning, multimodal data fusion, and self-supervised learning have significantly improved the accuracy and interpretability of these models, making them increasingly viable for clinical translation.

However, the field faces several challenges. Datasets vary widely in size, quality, and clinical validation; many are collected from online sources without robust diagnostic verification. Models trained in controlled laboratory settings may not generalize to real-world clinical or home environments. Ethical considerations—including privacy, consent, and the risk of algorithmic bias—remain critical, particularly when working with vulnerable pediatric populations. Furthermore, the diversity of ASD presentation across age, sex, and cultural context necessitates models that are robust, explainable, and adaptable.

This survey provides a comprehensive review of computational methods for ASD detection, with a focus on behavioral and affective computing approaches. Unlike prior reviews that focus primarily on facial images, neuroimaging, or questionnaire-based screening, this survey organizes recent ASD detection studies through an affective and behavioral computing lens, emphasizing observable attention, gaze, facial, pose, movement, and brain-imaging markers. Such detection methods offer benefits for early detection for the case of ASD babies and young children before language develops and the case of general autism detection. Expressions and behaviors manifest before language[53], offering visual and processable features for computational detection. We systematically categorize recent research into three main areas based on the features analyzed: (1) Attention and Interest-based Detection, which uses facial landmarks, head pose, and eye-gaze tracking to quantify deficits in social attention and engagement; (2) Behavior and Body Movement-based Detection, which employs pose estimation and action recognition to identify stereotypical motor behaviors and atypical body language; and (3) Brain Imaging-based Detection, which leverages structural and functional neuroimaging data to detect neurological differences associated with ASD. For each category, we describe the types of data collected, the feature extraction and preprocessing methods employed, and the machine learning, deep learning, or hybrid classification techniques used.

In addition to reviewing computational approaches, we provide a detailed examination of the datasets that underpin this research. We categorize datasets into three types: (1) pre-training datasets used to initialize models with general features (e.g., facial emotion recognition, body pose); (2) general ASD-focused datasets available to the research community (e.g., ABIDE I/II for brain imaging, eye-tracking scanpath datasets); and (3) paper-specific datasets collected in clinical or controlled settings with rigorous diagnostic validation. We discuss the strengths and limitations of these datasets, including issues of sample diversity, diagnostic rigor, and ethical data collection practices.

To improve the transparency and reproducibility of this survey, the reviewed literature was organized using a structured review process guided by established systematic literature review principles [37]. In line with this methodology, the review process considered the definition of review objectives, search strategy, study selection, data extraction, and synthesis of findings. Relevant studies were identified from major scientific databases using search terms related to Autism Spectrum Disorder (ASD), artificial intelligence, machine learning, deep learning, computer vision, affective computing, behavioral analysis, eye tracking, pose estimation, and neuroimaging. The retrieved studies were screened using predefined inclusion and exclusion criteria, and the selected papers were analyzed according to their data modality, feature extraction method, classification approach, dataset characteristics, and reported limitations. The detailed literature search and study selection procedure is presented in the following section.

This survey is organized as follows. Section 2 presents the literature review methodology, including the search strategy, study selection process, inclusion and exclusion criteria, and the approach used to analyze and synthesize the selected studies. Section 3 reviews computational approaches for ASD detection, organized according to attention and interest-based detection, behavior and body movement-based detection, and brain imaging-based detection. Section 4 examines the datasets used in ASD research, including pre-training datasets, publicly available ASD datasets, and study-specific clinical datasets. Section 5 discusses the current challenges, limitations, and future research directions for AI-based ASD detection, and Sect. 6 concludes the paper.

2 Literature Review Methodology

To improve the transparency and reproducibility of this survey, the reviewed literature was identified, screened, and synthesized using a structured review methodology guided by the systematic literature review principles proposed by Kitchenham and Charters [37]. The objective of the review was to identify recent studies that employ artificial intelligence (AI), machine learning (ML), deep learning (DL), computer vision, affective computing, and related computational techniques for Autism Spectrum Disorder (ASD) detection.

Relevant studies were retrieved from major scientific databases, including IEEE Xplore, ACM Digital Library, SpringerLink, ScienceDirect, PubMed, and Google Scholar. The literature search employed combinations of keywords such

as Autism Spectrum Disorder, ASD, Artificial Intelligence, Machine Learning, Deep Learning, Computer Vision, Eye Tracking, Eye Gaze, Pose Estimation, Facial Expression Analysis, Behavior Analysis, Brain Imaging, MRI, and fMRI. Boolean operators (AND/OR) were used to refine the search and improve the relevance of the retrieved publications.

To ensure the quality and relevance of the selected studies, predefined inclusion and exclusion criteria were applied during the screening process. Studies were included if they were peer-reviewed journal or conference publications written in English, focused on AI-based techniques for ASD detection, and provided sufficient technical details regarding the computational methods, datasets, or experimental evaluation. Editorials, opinion articles, duplicate publications, studies unrelated to ASD detection, non-English publications, and works lacking adequate methodological descriptions were excluded from the review.

The retrieved studies were subsequently screened based on their titles, abstracts, and full-text content to determine their relevance to the scope of this survey. The selected papers were analyzed and categorized according to their primary computational approach, including attention and interest-based detection, behavior and body movement-based detection, and brain imaging-based detection. For each study, information regarding the data modality, feature extraction techniques, learning algorithms, datasets, evaluation methods, and reported limitations was extracted and synthesized. This structured methodology provides a transparent basis for the organization of the survey while facilitating reproducibility and enabling readers to understand the rationale behind the selection and categorization of the reviewed literature.

Based on the literature review methodology described above, the selected studies are organized according to their primary computational paradigm and the type of behavioral or neurological information used for ASD detection.

3 Computational Approaches for ASD

Computational approaches for ASD detection analyze behavioral, affective, and neurological data to identify distinguishing patterns between individuals with ASD and typically developing controls. These methods leverage computer vision and machine learning to extract measurable indicators supporting early screening and clinical decision-making, though reliability depends critically on dataset quality, diversity, and validation. This section reviews three detection categories: (1) attention and interest-based detection, (2) behavior and body movement-based detection, and (3) brain imaging-based detection.

3.1 Attention and Interest-Based Detection

Attention and interest-based detection quantifies how individuals orient, sustain, and shift attention in response to social and non-social stimuli. Differences in eye contact, gaze allocation, joint attention, and response to name are characteristic of ASD-related social-communication challenges. Computational approaches

Table 1. Attention and Interest-Based Detection: Computational Approaches

Study	Type	Feature Extraction & Classification	Data Modality
Machine Learning & Deep Learning Approaches			
Ahmed et al. [1]	Hybrid	**Features:** LBP, GLCM; GoogLeNet, ResNet-18 **Classifiers:** FFNN, ANN, CNN, CNN+SVM	Eye-tracking scanpath images
Hashemi et al. [31]	ML	**Features:** Facial landmarks, yaw angle **Classifier:** SVM	Video of facial dynamics during social interaction
Hunt & Kim [34]	DL	**Features:** OpenCV facial expression recognizer **Classifier:** CNN	Images and videos of emotional expressions
Jain & Mangal [35]	Hybrid	**Features:** DeepFace biomarkers, CNN key frames **Classifiers:** SVM, DT, RF, KNN, ANN, CNN+ResNet	Video responses to verbal prompts and visual stimuli
Statistical & Descriptive Analysis Approaches			
Babu et al. [38]	Statistical	**Analysis:** Euclidean displacement, multiscale entropy	Eye tracking during social stimuli
Bovery et al. [9]	Statistical	**Analysis:** IntraFace tracking, yaw angle calculation	Head and eye tracking during tasks
Hashemi et al. [30]	Statistical	**Analysis:** Facial landmarks, segmentation, pose estimation	Video of attention, interest, and motor behaviors

extract features from facial landmarks, head-pose estimation, eye-tracking scanpaths, and facial affect to measure attentional behavior from videos, images, or sensor data.

Table 1 summarizes computational approaches for attention-based detection. Participants are exposed to controlled stimuli—social scenes, verbal prompts, name-calling tasks, or interactive activities—while systems record gaze fixation, head orientation, facial landmark movement, and response timing. Extracted features serve either for statistical group comparison or as input to ML, DL, or hybrid classifiers. However, model performance remains sensitive to stimulus design, recording conditions, participant age, and task context, requiring external validation before clinical deployment.

Machine Learning and Deep Learning Approaches. Ahmed et al. [1] compared ML, DL, and hybrid approaches for classifying eye-tracking scanpath images. They extracted features using LBP and GLCM, trained FFNN and ANN classifiers, implemented GoogLeNet and ResNet-18 architectures, and developed a CNN+SVM hybrid model. All models achieved above 90% accuracy, with ML neural networks demonstrating optimal performance.

Hashemi et al. [31] tracked facial landmark displacement during social interaction tasks, normalizing movements by inter-eye distance and defining head turns as yaw changes from ±20° to beyond ±35° within 0.5 s. An SVM classifier revealed that ASD children exhibited both lack of response and delayed response to name-calling compared to TD children.

Hunt and Kim [34] applied OpenCV facial expression recognition with CNN classification for automated emotion detection in ASD image and video datasets. Jain and Mangal [35] developed a multimodal system using DeepFace for facial biomarker extraction and CNN for key frame identification from responses to verbal and visual prompts. Among classifiers tested (SVM, DT, RF, KNN, ANN, CNN+ResNet), the CNN+ResNet model achieved the highest performance.

Statistical and Descriptive Analysis Approaches. Bovery et al. [9] used IntraFace to track facial landmarks and compute head orientation via 3D face model fitting and yaw angle calculation while participants viewed social and non-social video stimuli. Temporal histograms revealed that TD individuals maintained significantly greater stimulus focus than ASD individuals across both conditions.

Babu et al. [38] extended this approach by analyzing emotional dynamics alongside attention patterns. They presented social and non-social video stimuli to toddlers while recording facial responses, extracted landmarks using OpenFace [7], and estimated head pose and gaze direction. Emotion dynamics were quantified via Euclidean distance between landmarks and multiscale entropy, with emotions categorized as positive, neutral, or negative. ASD children exhibited higher entropy in both facial movement and emotion states, reflecting greater irregularity compared to the stable patterns in TD children.

Hashemi et al. [30] employed facial landmark detection, segmentation, and pose estimation to profile attention, interest, and atypical motor behaviors in ASD participants through statistical video analysis.

3.2 Behavior and Body Movement-Based Detection

Behavior and body movement-based detection analyzes motor actions, gestures, and interaction patterns associated with ASD. Restricted and repetitive behaviors (hand flapping, body rocking, spinning, head banging) and social communicative behaviors (pointing, imitation, object interaction) are assessed using pose estimation models (OpenPose, MediaPipe, BlazePose, YOLO) that extract body joints, skeletal keypoints, and temporal trajectories. Features are analyzed via statistical methods, CNNs, LSTMs, transformers, or hybrid spatial-temporal models.

These systems serve as assistive tools for behavioral screening and therapy support rather than standalone diagnostics, with reliability depending on diverse datasets and real-world validation [14]. Table 2 summarizes computational approaches for behavior-based detection.

Table 2. Behavior and Body Movement-Based Detection: Computational Approaches

Study	Type	Feature Extraction & Classification	Target Behaviors / Data Modality
Stereotypical Movement Detection			
Wang et al. [63]	Hybrid	**Features:** ResNet18 (spatial), Bi-LSTM (temporal) **Classifier:** CNN+LSTM	Head turn detection (>90°) during name-call; dual-camera video
Eghbalian et al. [22]	DL + HPE	**Action Detection:** Inception_V3, EfficientNet, DenseNet121 **Pose:** OpenPose [43], MHFormer [39]	Self-injurious behaviors: hand-hitting-head, head banging; video
Srivastava et al. [58]	DL	**Features:** BlazePose [8] body joints **Classifier:** LSTM	Arm flapping, head banging, spinning; SSBD [51]; CCTV/Raspberry Pi
Reddy et al. [54]	Statistical	**Analysis:** MediaPipe [27] landmarks, TBDPF [56]	Repetitive movements (flapping, rocking, shaking); SSBD [51]; therapist software
Noor et al. [48]	DL	**Features:** EfficientRepNet, SPP, CSP attention **Classifier:** YOLOv11	Hand flapping, body rocking, head shaking; YouTube videos; cloud-based
Social and Communicative Behavior Detection			
Cicceri et al. [12]	Hybrid	**Detection:** YOLOv5 [61] (child), MediaPipe (hands) **Classifier:** DNN (pointing) **Clustering:** DBSCAN [24]	Pointing gestures during play; video without sensors
Amirbay et al. [3]	Hybrid	**Features:** MediaPipe [27] (face, pose, hands), movement energy dynamics **Classifiers:** CNN+LSTM, CNN+Bi-LSTM, Conv1D+GRU, CNN+Transformer	Body expression during interaction/conversation; composite multimodal analysis
Mendes et al. [45]	DL	**Detection:** YOLOv8 (bounding boxes), YOLOv8-Pose (hand/wrist keypoints) **Interaction:** Wrist-object overlap	Object and person interaction with +Me device [60]; ISTC-CNR dataset

Stereotypical Movement Detection. Stereotypical movement detection identifies repetitive motor behaviors through motion trajectories, body-joint coordinates, skeletal keypoints, or action labels. Temporal modeling distinguishes repetitive behaviors from ordinary movements, often combining spatial feature extractors with LSTM, Bi-LSTM, GRU, or transformer architectures.

Wang et al. [63] used dual cameras to detect head turns exceeding 90° during name-call prompts. Children engaged with toys before being called by name;

positive attention was recorded upon eye contact establishment. Their fusion architecture combined ResNet18 for spatial features with Bi-LSTM for temporal learning, achieving perfect specificity, sensitivity, and accuracy with the 90° head turn threshold.

Eghbalian et al. [22] detected self-injurious behaviors (hand-hitting-head, head banging) using two methods: action detection and pose estimation. Action detection employed fine-tuned CNNs (Inception_V3, EfficientNet, DenseNet121), with DenseNet121 and Inception_V3 achieving over 98% accuracy and AUC near 100%. Pose estimation used OpenPose [43] and MHFormer [39]. OpenPose excelled at upper body landmarks including hand digits via Part Affinity Fields; MHFormer provided stronger whole-body tracking. Both performed well for hand-hitting-head detection but struggled with head banging due to occlusion.

Srivastava et al. [58] targeted CCTV systems in therapy centers for detecting arm flapping, head banging, and spinning using the SSBD dataset [51,52]. Their model, trained on a desktop and transferred to Raspberry Pi, processed DVR recordings using BlazePose [8] for body-joint extraction and LSTM classification, achieving 86% accuracy and demonstrating embedded system feasibility.

Reddy et al. [54] developed therapist assistive software for the SSBD dataset [51], extracting body landmarks via MediaPipe [27] and analyzing movement variation. Users could manually select video segments or use automated Threshold Based Data-Peak Filtering [56] to identify repetitive motions as peaks in movement data, achieving over 90% sensitivity and accuracy with 6-second processing time.

Noor et al. [48] created a cloud-based system for hand flapping, body rocking, and head shaking detection. The architecture included Monitoring, Network, Cloud, and ASD Behavior Detection layers. EfficientRepNet with Spatial Pyramid Pooling and Cross-Stage Partial attention extracted features; YOLOv11 identified and classified actions. Trained on YouTube videos (faces blurred), YOLOv11 achieved 98% accuracy, outperforming CNN and LSTM baselines.

Social and Communicative Behavior Detection. Social and communicative behavior detection identifies interaction-related behaviors including pointing, imitation, object engagement, turn-taking, and body expression during play or therapy.

Cicceri et al. [12] tracked pointing gestures during child-parent play sessions using the AI4ASC four-level framework: (1) Child Detection via YOLOv5 [61], (2) Hand Detection via MediaPipe keypoint extraction, (3) Gesture Classification via DNN for binary pointing/non-pointing classification, and (4) Clustering and Quantification via DBSCAN [24] for gesture subcategorization. AI4ASC achieved 72.5% accuracy using only video data without sensors.

Amirbay et al. [3] analyzed composite body expression from facial expression, body pose, and hand movements during interaction, conversation, and imitation tasks. MediaPipe [27] extracted landmarks from face mesh, pose, and hands modules; "movement energy dynamics" equations quantified

spatial-temporal features via Euclidean distance formulations. Hybrid models (CNN+LSTM, CNN+Bi-LSTM, Conv1D+GRU, CNN+Transformer) were tested, with CNN+Transformer achieving 93% accuracy and 95% AUC.

Mendes et al. [45] detected interactions between ASD children, the +Me wearable device [60], and a proctor using ISTC-CNR dataset videos. YOLOv8 isolated child, device, and proctor via bounding boxes; YOLOv8-Pose estimated hand and wrist keypoints. Wrist keypoint overlap with the +Me bounding box indicated interaction, achieving 87.83% precision and improving upon hand overlap methods.

3.3 Brain Imaging-Based Detection

Brain imaging-based detection identifies structural, functional, or physiological brain patterns associated with ASD through modalities including structural MRI (sMRI), functional MRI (fMRI), resting-state fMRI (rs-fMRI), and functional near-infrared spectroscopy (fNIRS). Features extracted from cortical thickness, gray matter volume, voxel-based morphometry, functional connectivity matrices, or graph representations are classified using ML, CNNs, transformers, graph convolutional networks, self-supervised learning, or hybrid architectures combining imaging with phenotypic data. Studies primarily use public repositories (ABIDE I and II) and clinical datasets.

While neuroimaging provides clinically meaningful information, MRI/fMRI data require specialized equipment and are less accessible than behavioral measures. Multi-site datasets improve sample size but introduce scanner differences, acquisition-protocol variation, and demographic imbalance requiring careful preprocessing and harmonization. Table 3 summarizes computational approaches for brain imaging-based detection.

Functional Brain Activity Analysis. Farooq et al. [26] examined brain activity responses to emotion-inducing visual stimuli using fNIRS to analyze blood flow and oxygenation variance. After noise removal, data were processed with XGBoost for prediction, ANOVA for feature selection, and PCA/UMAP for dimensionality reduction and visualization. Nested Cross-Validation revealed distinct blood flow patterns in ASD individuals, with computational discernibility from TD responses.

Structural and Functional MRI Analysis. MRI-based ASD research leverages the Autism Brain Imaging Data Exchange (ABIDE) I and II datasets, large-scale repositories containing structural and functional MRI scans plus demographic and phenotypic information from diverse populations. Further discussion appears in Sect. 4.2.

Self-supervised and Hybrid Learning Approaches. Anand and Kini [4] tested SSL model combinations for feature extraction from ABIDE I anatomical MRI (aMRI). They employed DINOv2 [49], MoCo [32], BYOL [28], and SimCLR with CNN backbones (EfficientNetB0, ResNet50), classifying concatenated features via FFNN. DINOv2 with EfficientNetB0-backed SimCLR achieved 98.01% accuracy.

Khan and Katarya [36] developed the multimodal LSTVision framework fusing ViT for MRI feature extraction with LSTM for phenotypic data processing via neural network classification, achieving 94.9% accuracy on ABIDE I.

Table 3. Brain Imaging-Based Detection: Computational Approaches

Study	Type	Feature Extraction & Classification	Data Modality
Machine Learning Approaches			
Farooq et al. [26]	ML	**Features:** XGBoost, ANOVA selection, PCA & UMAP reduction **Classifier:** XGBoost with Nested CV	fNIRS blood flow during emotion-inducing visual stimuli
Ashraf et al. [5]	Hybrid	**Features:** NeuroNet57 (57-layer CNN), ACO [20] selection **Classifiers:** SVM (Linear, Quadratic, Gaussian, Cubic), KNN (Cosine, Coarse, Fine)	fMRI (ABIDE I & II); Female-focused; Pretrained on Br35H [29]
Deep Learning & Hybrid Approaches			
Anand & Kini [4]	Hybrid	**Features:** SSL models (DINOv2 [49], MoCo [32], BYOL [28], SimCLR); CNN backbones (EfficientNetB0, ResNet50) **Classifiers:** FFNN, CNNs	aMRI (ABIDE I); Multiple SSL+CNN combinations
Khan & Katarya [36]	DL	**Features:** ViT (MRI), LSTM (phenotypic) **Classifier:** LSTVision multimodal fusion	MRI + phenotypic data (ABIDE I)
E, Vignesh et al. [21]	DL	**Features:** NeuroASD-Net (4 Conv2D layers, normalization, pooling, dropout) **Classifier:** FC with Sigmoid activation	sMRI targeting amygdala, hippocampus, temporal/frontal lobes (ABIDE I)
Pathan & Sucharitha [50]	Hybrid	**Features:** Attention U-Net segmentation, GLCM texture, VBM, feature fusion **Classifier:** CNN with MLP	sMRI (ABIDE I); Multi-feature fusion
Ashrafi & Kabir [6]	DL	**Features:** EGCN with CSGC [15] and GAT [62] layers **Classifier:** Multi-branch EGCN (per-modality branches)	sMRI + fMRI + phenotypic (ABIDE I & II); Graph-based multimodal

Custom CNN Architectures for Structural MRI E, Vignesh et al. [21] developed NeuroASD-Net for ABIDE I sMRI classification, extracting axial slices targeting amygdala, hippocampus, temporal lobe, and frontal lobe. The architecture contains four Conv2D layers with normalization, pooling, dropout, and sigmoid binary classification. ReLU activation achieved 95.0% accuracy with 45-second training time per epoch.

Pathan and Sucharitha [50] combined Attention U-Net segmentation, GLCM texture features, and Voxel-based Morphometry (VBM) for ABIDE I sMRI analysis. Volume features from segmented cortical/subcortical structures were fused with GLCM texture features and classified via CNN with MLP architecture, achieving 88.6% accuracy versus 82.6% for standalone CNN. Attention U-Net effectively highlighted diagnostically relevant regions.

Graph-Based and Multi-branch Approaches. Ashrafi and Kabir [6] developed a multi-branch Enhanced Graph Convolutional Network (EGCN) for ABIDE I and II, dedicating separate branches to sMRI, fMRI, and phenotypic data. Each branch incorporates Chebyshev Spectral Graph Convolution (CSGC) [15] and Graph Attention Network (GAT) [62] layers. With hyperparameter tuning, EGCN achieved 74.82% accuracy and 82% AUC, with the highest AUC among compared models.

Addressing Gender Bias in Neuroimaging. Most ASD neuroimaging datasets are male-dominant, as females often receive later diagnosis due to diagnostic bias, behavioral presentation differences, and camouflaging ability [13]. Ashraf et al. [5] specifically addressed this disparity by developing NeuroNet57, a 57-layer CNN for female ASD fMRI feature extraction from ABIDE I and II. Due to insufficient female samples, they pretrained on Br35H brain tumor data [29]. Features were selected via Ant Colony Optimization (ACO) [20] and classified using SVM and KNN variants. Testing feature counts (100, 250, 500, 750, 1000) revealed optimal performance at 750 features: 92.21% AUC (ABIDE I) and 93.49% AUC (ABIDE II), demonstrating that population-specific modeling can address gender imbalance in ASD neuroimaging research.

4 Applicable Datasets

Datasets play a central role in AI-based ASD detection, determining the behavioral, affective, anatomical, or physiological markers that can be extracted and the computational models that can be trained. Because ASD is heterogeneous, dataset quality—including age, sex, clinical severity, cultural background, recording environment, annotation protocol, and diagnostic validation—strongly affects model performance and generalizability.

Most datasets involve image, video, eye-tracking, pose, facial-expression, or brain-imaging data enabling models to learn patterns related to facial structure, gaze behavior, body movement, stereotypical actions, emotional responses,

and neuroanatomical differences. Publicly available datasets enable benchmarking and reproducibility but often have limitations including small sample sizes, class imbalance, uncertain diagnostic labels, demographic imbalance, or privacy restrictions. Paper-specific clinical datasets provide stronger diagnostic validation but are typically smaller, less accessible, and difficult to reproduce across institutions.

Datasets are organized into three categories: (1) pre-training datasets providing general-purpose visual, facial-expression, pose, or brain-image features before task-specific fine-tuning; (2) general ASD-focused datasets publicly available or widely used for ASD-related research; and (3) paper-specific datasets collected by individual research groups in clinical or controlled settings.

4.1 Pre-training Datasets

Pre-training datasets initialize models with general visual, facial, body-pose, or medical-image representations before fine-tuning on ASD-specific data. This is especially useful because task-specific ASD datasets are often small, imbalanced, or restricted. However, pre-training datasets are not ASD-specific; features learned must be interpreted cautiously when transferred to ASD screening.

The Extended Cohn-Kanade (CK+) dataset [40] contains annotated facial expressions (anger, disgust, fear, happiness, sadness, surprise, contempt, neutral) from 210 adults (18–50 years, 69% female, 81% Euro-American) in controlled conditions. Improvements over the original CK dataset include color images with normalized brightness, more instances, and consistent labeling.

AffectNet [46] and AffectNet+ [25] provide large-scale facial-expression data collected "in the wild" from internet sources (1 million images, participants 33 years old, balanced gender). AffectNet+ extends categorical emotion labels (happy, sad, surprise, fear, disgust, anger, contempt, neutral) with continuous valence and arousal intensity annotations.

The Binghamton University 3D Facial Expression (BU-3DFE) Database [64] provides 2,500 3D facial-expression models from 100 subjects (56% female, 18–70 years) performing posed emotions (happiness, disgust, fear, anger, surprise, sadness, neutral).

For MRI-based ASD studies, non-ASD medical-imaging datasets provide general image-representation pre-training. The Brain Tumor MRI Dataset compiles 7,200 clinical brain images (glioma, meningioma, pituitary tumor, no tumor) from three sources (Cheng's dataset, SARTAJ, Br35H) [47]. The Br35H::Brain Tumor Detection 2020 Dataset contains 3,060 MRI images with training, testing, and validation splits [29]. These datasets should be limited to generic MRI feature learning rather than ASD-specific interpretation.

ImageNet contains over 14 million images across 21,000+ categories, with the commonly used ImageNet-1K subset containing 1.2–1.3 million training images across 1,000 categories. ImageNet-pretrained models are widely used for feature extraction from video frames, facial images, body-pose representations, or attention-related visual data. However, models fine-tuned from ImageNet require careful validation on ASD-specific datasets [16,59].

4.2 General Autism-Focused Datasets

General ASD-focused datasets are publicly available or widely reused datasets specifically designed for ASD research, enabling benchmarking, reproducibility, and method comparison. They are categorized by modality: natural RGB images and brain imaging.

Natural RGB Image Datasets. Natural image datasets contain 8-bit, three-channel RGB images capturing behavioral, facial, and body movement characteristics. Clinical reliability varies depending on label assignment, participant recruitment, diagnostic instruments, and demographic information availability.

The Facial Emotion Recognition Dataset for Children with Autism (FER-Autism) [42] focuses on facial images of children expressing emotions with augmented variations (geometry, color, brightness, noise). Clinical utility depends on transparent reporting of diagnostic labels, demographics, and data collection procedures.

The Visualization of Eye-Tracking Scanpaths in Autism Spectrum Disorder dataset represents eye-movement behavior as scanpath visualizations from gaze coordinates recorded while ASD and TD children viewed visual stimuli. This frames ASD screening as image classification of visual attention patterns. However, limitations include small sample size, controlled stimulus conditions, device dependency, and limited generalization to naturalistic environments [10,23].

Brain Imaging Datasets. The Autism Brain Imaging Data Exchange (ABIDE) I [17] and II [18] datasets aggregate global ASD neuroimaging data containing resting-state functional MRI (R-fMRI), structural MRI, and phenotypic data from individuals aged 5–60 years. ABIDE I (August 2012) contains 1,112 instances (539 ASD, 573 TD) from 17 international contributors. ABIDE II (June 2016) adds 1,114 instances (521 ASD, 593 TD) with greater phenotypic diversity from 19 institutions, including longitudinal samples from 38 individuals. Despite their importance for public availability and multi-modal data, ABIDE datasets introduce challenges including multi-site acquisition variability, protocol differences, demographic imbalance, and harmonization requirements.

4.3 Study-Specific Datasets

Study-specific datasets are collected by individual research groups under clinical, therapeutic, or controlled conditions. They provide stronger diagnostic confidence through rigorous screening and expert validation, enabling focused investigation of specific markers including response-to-name, pointing gestures, facial affect, self-injurious behavior, head movement, body pose, and fNIRS responses. However, they are constrained by small sample sizes, restricted accessibility, narrow demographics, and task-specific conditions limiting replication and generalization.

Diagnostic Validation Methods. Studies employ various assessment instruments to validate ASD diagnosis. The Modified Checklist for Autism in Toddlers Revised with Follow-up (M-CHAT-R/F) provides 20 caregiver-administered screening questions [55]. The Autism Diagnostic Observation Schedule Toddler module (ADOS-T) involves clinician-conducted observational activities [41]. The Diagnostic and Statistical Manual of Mental Disorders, 5th Edition (DSM-5) defines formal diagnostic criteria [2]. The Childhood Autism Rating Scale (CARS) assesses severity of DSM-based criteria [11]. The Indian Scale for Assessment of Autism (ISAA) provides culturally adapted assessment [19].

Representative Study-Specific Datasets Attention and Gaze Studies: Jain and Mangal [35] collected in-clinic video data tracking head movements and eye-gaze patterns from 147 children (1.5–6 years) viewing geometric shapes and stimuli, validated using ISAA. Babu et al. [38] recorded tablet-based reactions to social and non-social movies at primary care check-ups, validated with M-CHAT-R/F. Hashemi et al. [30,31] captured facial reactions to movie stimuli (33 children, 16–31 months, validated with M-CHAT-R/F and ADOS-T) and response-to-name with appealing objects (6 children, 6–15 months, validated with AOSI). Bovery et al. [9] collected facial landmarks from children viewing stimulating videos. Carette et al. [10] used SMI remote eye-tracker to create scanpath visualizations from children viewing videos with attention objects.
Behavioral and Motor Studies: Eghbalian et al. [22] collected 8,557 labeled video frames of self-injurious behaviors (head hitting, head banging) from two children with accelerometer validation. Cicceri et al. [12] recorded pointing gestures during parent-child activities under varied angles, postures, lighting, and backgrounds with therapist validation. Wang et al. [63] captured head-turn responses to name-calling with DSM-5 and CARS validation. Rajagopalan et al. [51,52] compiled the Self-Stimulatory Behavior Dataset (SSBD) from YouTube videos labeled for arm flapping, head banging, and spinning. Noor et al. [48] sourced videos from YouTube and educational archives, validated by autism specialists. Amirbay et al. [3] collected motor responses from structured interactions (5–9 years) with caregiver-confirmed diagnoses.
Interaction and Device-Based Studies: The ISTC-CNR dataset [57] captures clinician-child interactions with the PlusMe device [60] during built-in games and activities, validated with DSM-5.
Neuroimaging studies: Farooq et al. [26] collected 41-channel fNIRS signals from children (5–7 years) viewing emotional visual stimuli, with video-referenced validation and preprocessing including noise filtering and statistical analysis.

Dataset Limitations and Future Directions. ASD detection research remains constrained by data availability, modality imbalance, and limited clinical validation. Public datasets (ABIDE, eye-tracking scanpaths) enable benchmarking but may not represent the diversity of ASD presentations across age, sex, ethnicity, language, and clinical severity. Clinical datasets provide stronger

diagnostic confidence but limited size and accessibility hinder independent validation. Future work should prioritize multi-site collection, standardized annotation protocols, demographic reporting, privacy-preserving sharing, and external validation across independent datasets.

5 Discussion

This survey highlights the growing role of affective and behavioral computing in AI-based ASD detection. Across the reviewed studies, three major directions are evident: attention and interest-based analysis, behavior and body movement-based analysis, and brain imaging-based analysis. These directions reflect different but complementary views of ASD. Attention and gaze-based approaches focus on social engagement and visual attention patterns, behavior and movement-based approaches capture observable motor and interaction behaviors, and brain imaging-based approaches investigate structural or functional neurological differences associated with ASD.

A major observation from this review is that ASD detection models are highly dependent on the type and quality of data used for training and evaluation. Studies using facial images, gaze data, pose information, videos, or neuroimaging data often report promising classification performance; however, these results are difficult to compare directly because datasets differ in size, modality, participant age, diagnostic validation, recording environment, and evaluation protocol. Public datasets such as ABIDE [17,18] and eye-tracking scanpath datasets [23] support reproducibility and benchmarking, while paper-specific clinical datasets may provide stronger diagnostic confidence. However, many paper-specific datasets are small, restricted, or collected under controlled conditions, making it difficult to assess generalization across real-world clinical and educational settings[12,22,26,44].

Another important finding is that single-modality approaches have limited clinical reliability[1,23,26,31,45,63]. Facial-image models may capture useful visual patterns, but facial appearance alone is insufficient for reliable ASD screening. Similarly, eye-tracking and pose-based approaches can provide valuable behavioral cues, but they may be sensitive to task design, camera placement, sensor quality, and environmental conditions. Brain-imaging methods provide biologically meaningful information, but they are expensive, less accessible, and often affected by site-specific acquisition differences. Therefore, future research should move toward multimodal systems that combine behavioral, affective, visual, physiological, and clinical information.

The reviewed literature also shows that interpretability remains a critical challenge. Many deep learning models achieve high accuracy but provide limited explanation of which behavioral or affective markers influence the prediction [1,4,36,48,58]. For ASD screening, this is a serious limitation because clinicians, caregivers, and educators need interpretable evidence rather than only a binary classification result. Future systems should therefore include explainable AI methods, such as attention visualization, saliency analysis, feature attribution, or clinically meaningful behavioral summaries. These explanations can

help determine whether the model is learning ASD-relevant patterns or dataset-specific artifacts.

Ethical and practical considerations are also central to this area. ASD detection datasets often involve children and sensitive health-related information. As a result, privacy, consent, demographic fairness, and responsible data sharing must be treated as core requirements rather than secondary concerns. Models should be evaluated not only for accuracy but also for bias across age, sex, ethnicity, language background, and ASD severity. In addition, AI-based ASD systems should be presented as screening or decision-support tools, not as replacements for clinical diagnosis. Their purpose should be to assist early identification, prioritize referrals, and support clinicians, educators, and families.

Based on this review, several recommendations can be made for future ASD detection research. First, researchers should report dataset details more transparently, including participant demographics, diagnostic criteria, recording protocols, preprocessing steps, and train-test splits. Second, studies should prioritize external validation on independent datasets rather than relying only on within-dataset performance. Third, multimodal learning should be explored more systematically to combine facial, gaze, pose, speech, physiological, and neuroimaging cues. Additionally, features that have not been considered before, such as toe-walking, finger flicking and other finger-based stimulation, and parent interaction interest, can be considered as means to identify early autism behaviors. Fourth, model interpretability should be included as part of the evaluation process. Finally, future work should emphasize clinically meaningful deployment, including privacy-preserving learning, real-world testing, and collaboration with clinicians and autism specialists.

6 Conclusion

This survey reviewed recent AI-based approaches for ASD detection from an affective and behavioral computing perspective. The reviewed studies were organized into three major categories: attention and interest-based detection, behavior and body movement-based detection, and brain imaging-based detection. These categories show that ASD-related markers can be studied through facial expressions, eye gaze, head pose, body movement, stereotypical actions, neuroimaging data, and other multimodal signals.

The review also highlights that dataset quality is central to the reliability of ASD detection systems. Public and paper-specific datasets enable model development and benchmarking, but many existing datasets remain limited by small sample size, demographic imbalance, uncertain labeling, restricted access, and limited external validation. Overall, AI-based ASD detection has strong potential to support early screening and clinical decision-making, but future progress depends on larger and more diverse datasets, standardized evaluation protocols, multimodal learning, explainable models, privacy-preserving data sharing, and clinically validated systems that assist, rather than replace, professional ASD assessment.

References

1. Ahmed, I.A., et al.: Eye tracking-based diagnosis and early detection of autism spectrum disorder using machine learning and deep learning techniques. Electronics **11**(4) (2022). https://doi.org/10.3390/electronics11040530, https://www.mdpi.com/2079-9292/11/4/530
2. American Psychiatric Association: Diagnostic and Statistical Manual of Mental Disorders (5th ed.). American Psychiatric Publishing, Arlington, VA (2013). https://www.psychiatry.org/psychiatrists/practice/dsm, diagnostic criteria for Autism Spectrum Disorder
3. Amirbay, A., Mukhanova, A., Abdikerimova, G.: Hybrid CNN–LSTM framework for spatiotemporal detection of autism based on facial and motor behavior analysis. In: 2025 10th International Conference on Computer Science and Engineering (UBMK), pp. 1070–1075 (2025). https://doi.org/10.1109/UBMK67458.2025.11206968
4. Anand, S.S., Kini, S.: Optimizing neuroimaging-based autism detection through hybrid SSL achitectures. Lex Localis - J. Local Self-Govern. **23**, 454–470 (2025). https://doi.org/10.52152/z0p81d41, https://lex-localis.org/index.php/LexLocalis/article/view/801796
5. Ashraf, A., Zhao, Q., Bangyal, W.H., Raza, M., Iqbal, M.: Female autism categorization using cnn based neuronet57 and ant colony optimization. Comput. Biol. Med. **189**, 109926 (2025). https://doi.org/10.1016/j.compbiomed.2025.109926, https://www.sciencedirect.com/science/article/pii/S001048252500277X
6. Ashrafi, A.F., Kabir, M.H.: Enhanced graph convolutional network with chebyshev spectral graph and graph attention for autism spectrum disorder classification. In: 2025 40th International Conference on Image and Vision Computing New Zealand (IVCNZ), pp. 1–6 (2025). https://doi.org/10.1109/IVCNZ67716.2025.11281859
7. Baltrusaitis, T., Robinson, P., Morency, L.P.: Openface: An open source facial behavior analysis toolkit, pp. 1–10 (03 2016). https://doi.org/10.1109/WACV.2016.7477553
8. Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, T., Zhang, F., Grundmann, M.: Blazepose: On-device real-time body pose tracking (06 2020)
9. Bovery, M., Dawson, G., Hashemi, J., Sapiro, G.: A scalable off-the-shelf framework for measuring patterns of attention in young children and its application in autism spectrum disorder. IEEE Trans. Affect.Comput. **PP**, 1–1 (01 2019). https://doi.org/10.1109/TAFFC.2018.2890610
10. Carette, R., Elbattah, M., Cilia, F., Dequen, G., Guerin, J.L., Bosche, J.: Learning to predict autism spectrum disorder based on the visual patterns of eye-tracking scanpaths, pp. 103–112 (02 2019). https://doi.org/10.5220/0007402601030112
11. Chlebowski, C., Green, J.A., Barton, M.L., Fein, D.: Using the childhood autism rating scale to diagnose autism spectrum disorders. J. Autism Dev. Disord. **40**(7), 787–799 (2010). https://doi.org/10.1007/s10803-009-0926-x
12. Cicceri, G.: Ai4asc: a hierarchical multi-level ai-driven framework for gesture analysis and monitoring in ASC children. IEEE Access **14**, 17885–17904 (2026). https://doi.org/10.1109/ACCESS.2026.3659276
13. Cook, J., Hull, L., Mandy, W.: Improving diagnostic procedures in autism for girls and women: a narrative review. Neuropsychiatr. Dis. Treat. **20**(7), 505–514 (2024). https://doi.org/10.2147/NDT.S372723
14. De Stefani, E., De Marco, D.: Language, gesture, and emotional communication: An embodied view of social interaction. Front. Psychol., Cogn. **10**(2063) (2019)

15. Defferrard, M., Bresson, X., Vandergheynst, P.: Convolutional neural networks on graphs with fast localized spectral filtering. In: Proceedings of the 30th International Conference on Neural Information Processing Systems, pp. 3844–3852. NIPS'16, Curran Associates Inc., Red Hook, NY, USA (2016)
16. Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L.: Imagenet: A large-scale hierarchical image database. In: 2009 IEEE Conference on Computer Vision and Pattern Recognition, pp. 248–255 (2009). https://doi.org/10.1109/CVPR.2009.5206848
17. Di Martino, A., et al.: Autism brain imaging data exchange i abide i, `https://fcon_1000.projects.nitrc.org/indi/abide/abide_I.html`
18. Di Martino, A., et al.: Autism brain imaging data exchange ii abide ii, `https://fcon_1000.projects.nitrc.org/indi/abide/abide_II.html`
19. of Empowerment of Persons with Disabilities, D., Ministry of Social Justice & Empowerment, G.o.I.: Indian scale for assessment of autism. https://thenationaltrust.in/upload/uploadfiles/files/ISAA%20TEST%20MANNUAL(2).pdf
20. Dorigo, M., Di Caro, G.: Ant colony optimization: a new meta-heuristic. In: Proceedings of the 1999 Congress on Evolutionary Computation-CEC99 (Cat. No. 99TH8406), vol. 2, pp. 1470–1477 Vol. 2 (1999). https://doi.org/10.1109/CEC.1999.782657
21. E, V., S, G., P, M., J, B.R.: Neuroasd-net: A computationally efficient 2D CNN framework for autism spectrum disorder classification from structural mri slices. In: 2025 International Conference on Signal Processing, Computation, Electronics, Power and Telecommunication (IConSCEPT), pp. 1–6 (2025). https://doi.org/10.1109/IConSCEPT66142.2025.11436190
22. Eghbalian, A., Azam, M.M., Holloway, K., Neely, L., Desai, K.: Applying computer vision to analyze self-injurious behaviors in children with autism spectrum disorder, pp. 11–20 (02 2025). https://doi.org/10.1109/WACVW65960.2025.00007
23. Elbattah, M.: Visualization of Eye-Tracking Scanpaths in Autism Spectrum Disorder: Image Dataset (6 2019). https://doi.org/10.6084/m9.figshare.7073087.v1, https://figshare.com/articles/dataset/Visualization_of_Eye-Tracking_Scanpaths_in_Autism_Spectrum_Disorder_Image_Dataset/7073087
24. Ester, M., Kriegel, H.P., Sander, J., Xu, X.: A density-based algorithm for discovering clusters in large spatial databases with noise. In: Proceedings of the Second International Conference on Knowledge Discovery and Data Mining, pp. 226–231. KDD'96, AAAI Press (1996)
25. Fard, A.P., Hosseini, M.M., Sweeny, T.D., Mahoor, M.H.: Affectnet+: A database for enhancing facial expression recognition with soft-labels. IEEE Trans. Affect. Comput. 1–16 (2025). https://doi.org/10.1109/TAFFC.2025.3634523
26. Farooq, S., et al.: Advanced autism detection and visualization through xgboost algorithm for fnirs hemo-dynamic signals. Expert Systems with Applications **275**, 127013 (2025). https://doi.org/10.1016/j.eswa.2025.127013, https://www.sciencedirect.com/science/article/pii/S0957417425006359
27. Google: Mediapipe solutions guide. https://ai.google.dev/edge/mediapipe/solutions/guide
28. Grill, J.B., et al.: Bootstrap your own latent: A new approach to self-supervised learning (2020). https://arxiv.org/abs/2006.07733
29. Hamada, A.: Br35h::brain tumor detection 2020 (2020). https://www.kaggle.com/datasets/ahmedhamada0/brain-tumor-detection

30. Hashemi, J., et al.: A computer vision approach for the assessment of autism-related behavioral markers. In: 2012 IEEE International Conference on Development and Learning and Epigenetic Robotics (ICDL), pp. 1–7. IEEE (2012). https://doi.org/10.1109/DevLrn.2012.6400588
31. Hashemi, J., et al.: Computer vision analysis for quantification of autism risk behaviors. IEEE Trans. Affect. Comput. **12**(1), 215–226 (2021). https://doi.org/10.1109/TAFFC.2018.2868196
32. He, K., Fan, H., Wu, Y., Xie, S., Girshick, R.: Momentum contrast for unsupervised visual representation learning. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9726–9735 (2020). https://doi.org/10.1109/CVPR42600.2020.00975
33. Hebert-Myers, H., Guttentag, C.L., Swank, P.R., Smith, K.E., Landry, S.H.: The importance of language, social, and behavioral skills across early and later childhood as predictors of social competence with peers. Appl. Development. Sci. **10**(4), 174–187 (2006). https://doi.org/10.1207/s1532480xads1004_2
34. Hunt, J., Kim, J.: Emotion recognition in images and video with python for autism assessment, pp. 654–656 (07 2024). https://doi.org/10.1109/ICUFN61752.2024.10625445
35. Jain, R., Mangal, D.: Early detection of neurodevelopmental disorders: Quantifying autism behavioral markers with computer vision and artificial intelligence, pp. 1–6 (11 2024). https://doi.org/10.1109/IC-SIT63503.2024.10862342
36. Khan, K., Katarya, R.: Lstvision: A multi-modal framework for autism spectrum diagnosis utilizing LSTM and vision transformer. In: 2024 5th International Conference on Smart Sensors and Application (ICSSA). pp. 1–5 (2024). https://doi.org/10.1109/ICSSA62312.2024.10788604
37. Kitchenham, B., Charters, S.: Guidelines for performing systematic literature reviews in software engineering. Tech. Rep. EBSE-2007-01, Keele University and Durham University Joint Report, Keele, UK (2007). https://www.elsevier.com/__data/promis_misc/525444systematicreviewsguide.pdf
38. Krishnappa Babu, P.R., et al.: Exploring complexity of facial dynamics in autism spectrum disorder. IEEE Trans. Affect. Comput. (09 2021)
39. Li, W., Liu, H., Tang, H., Wang, P., Van Gool, L.: Mhformer: Multi-hypothesis transformer for 3D human pose estimation. In: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 13137–13146 (2022). https://doi.org/10.1109/CVPR52688.2022.01280
40. Lucey, P., Cohn, J.F., Kanade, T., Saragih, J., Ambadar, Z., Matthews, I.: The extended cohn-kanade dataset (ck+): A complete dataset for action unit and emotion-specified expression. In: 2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition - Workshops, pp. 94–101 (2010). https://doi.org/10.1109/CVPRW.2010.5543262
41. Luyster, R.: The autism diagnostic observation schedule–toddler module: a new module of a standardized diagnostic measure for autism spectrum disorders. J. Autism Dev. Disord. **39**(9), 1305–1320 (2009). https://doi.org/10.1007/s10803-009-0746-z
42. Mahmoud, N.: Facial emotion recognition dattaset for children with autism (10 2025). https://doi.org/10.17632/b33pf78h62.1, https://data.mendeley.com/datasets/b33pf78h62/1
43. Martinez, G.H.: OpenPose: Whole-Body Pose Estimation, Pittsburgh, PA (2019). Master's thesis

44. Mazefsky, C.A., et al.: The role of emotion regulation in autism spectrum disorder. J. Am. Acad. Child Adolescent Psych. **52**(7), 679–688 (2013). https://doi.org/10.1016/j.jaac.2013.05.006, https://www.sciencedirect.com/science/article/pii/S0890856713003080
45. Mendes, B.C.S., Tyska, J., Grellert, M.: Interaction detection in images of therapy sessions with children with autism spectrum disorder. In: 2025 IEEE 38th International Symposium on Computer-Based Medical Systems (CBMS),pp. 585–590 (2025). https://doi.org/10.1109/CBMS65348.2025.00124
46. Mollahosseini, A., Hasani, B., Mahoor, M.H.: Affectnet: a database for facial expression, valence, and arousal computing in the wild. IEEE Trans. Affect. Comput. **10**(1), 18–31 (2019). https://doi.org/10.1109/TAFFC.2017.2740923
47. Nickparvar, M.: Brain_tumer_mri_dataset. https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset
48. Noor, A., Almukhalfi, H., Souza, A., Noor, T.H.: Harnessing yolov11 for enhanced detection of typical autism spectrum disorder behaviors through body movements. Diagnostics **15**(14) (2025). https://doi.org/10.3390/diagnostics15141786, https://www.mdpi.com/2075-4418/15/14/1786
49. Oquab, M., et al.: Dinov2: Learning robust visual features without supervision (2024). https://arxiv.org/abs/2304.07193
50. d Pathan, M.K., Sucharitha, M.: A hybrid deep learning and voxel-based morphometry framework for structural MRI-based autism spectrum disorder classification. In: 2025 IEEE International Conference on Computer Vision and Machine Intelligence (CVMI), pp. 1–6 (2025). https://doi.org/10.1109/CVMI66673.2025.11337713
51. Rajagopalan, S.S., Dhall, A., Goecke, R.: Self-stimulatory behaviors in the wild for autism diagnosis. https://rolandgoecke.net/research/datasets/ssbd
52. Rajagopalan, S.S., Goecke, R.: Detecting self-stimulatory behaviours for autism diagnosis. In: 2014 IEEE International Conference on Image Processing (ICIP), pp. 1470–1474 (2014). https://doi.org/10.1109/ICIP.2014.7025294
53. Ramírez, N.F., Lieberman, A.M., Mayberry, R.I.: The initial stages of first-language acquisition begun in adolescence: when late looks early. J. Child Lang. **40**(2), 391–414 (2013). https://doi.org/10.1017/S0305000911000535
54. Reddy, D.U., Kumar, K.P., Ramakrishna, B., Sankar, U.G.: Development of computer vision based assistive software for accurate analysis of autistic child stereotypic behavior. In: 2023 International Conference on Recent Advances in Electrical, Electronics, Ubiquitous Communication, and Computational Intelligence (RAEEUCCI), pp. 1–5 (2023). https://doi.org/10.1109/RAEEUCCI57140.2023.10134091
55. Robins, D.L., Fein, D., Barton, M.: The modified checklist for autism in toddlers, revised with follow-up. https://www.mchatscreen.com
56. Sher, A., et al.: Automatic gait analysis during steady and unsteady walking using a smartphone (09 2021). https://doi.org/10.36227/techrxiv.16586780
57. Sperati, V., et al.: Acceptability of the transitional wearable companion "+me" in children with autism spectrum disorder: A comparative pilot study. Front. Psychol. **Volume 11 - 2020** (2020). https://doi.org/10.3389/fpsyg.2020.00951, https://www.frontiersin.org/journals/psychology/articles/10.3389/fpsyg.2020.00951
58. Srivastava, A.K., Tripathi, V., Pant, B.: Empowering autism therapy centers with a retrofit and computer vision based solution for behavioral monitoring and actionable insights. In: 2024 Eighth International Conference on Parallel, Distributed and Grid Computing (PDGC), pp. 585–590 (2024). https://doi.org/10.1109/PDGC64653.2024.10984414

59. Stanford Vision Lab, S.U.: Imagenet (2021). https://www.image-net.org/about.php
60. de Ciências e Tecnologias Cognitivas (ISTC-CNR), I.: Plusme: Transitional wearable companions for the therapy of children with autism spectrum disorders. https://www.plusme-h2020.eu/overview/
61. Ultralytics: YOLOv5: A state-of-the-art real-time object detection system. https://docs.ultralytics.com (2021). Accessed: insert date here
62. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y.: Graph attention networks (2018). https://arxiv.org/abs/1710.10903
63. Wang, J.: Deep learning-based response-to-name detection: Empirical study on early screening of autism spectrum disorder in children. IEEE Access **13**, 81406–81416 (2025). https://doi.org/10.1109/ACCESS.2025.3567367
64. Yin, L., Wei, X., Sun, Y., Wang, J., Rosato, M.: A 3d facial expression database for facial behavior research, vol. 2006, pp. 211– 216 (05 2006). https://doi.org/10.1109/FGR.2006.6

Indirect Feedback Alignment: A Viable Alternative for Deep Neural Networks Backpropagation

Jonathan Teck Meng Liaw(✉), Pulin Agrawal, Reza Abdollahi Dezfuli Nezhad, and Dheeraj Gollu

Penn State University, University Park, PA, USA
{tjl5783,pja5407,apa6323,dqg5543}@psu.edu

Abstract. Backpropagation has been the foundational algorithm for training deep neural networks with great success, driving major advances in artificial intelligence, including Large Language Models (LLMs). Despite its effectiveness, it is often considered biologically implausible due to its reliance on symmetric weight transport and precise layer-wise backward error propagation, mechanisms not observed in biological neural systems. Recent neuroscientific findings suggest that cortical microcircuits exhibit structured, predominantly forward-directed local signal flow, while broader brain dynamics remain influenced by recurrent feedback interactions. This motivates exploration of alternative biologically inspired learning mechanisms. This paper introduces Indirect Feedback Alignment (IFA), an uncharted learning framework that Nøkland briefly mentions but has not previously investigated. Unlike backpropagation, IFA avoids strict end-to-end gradient transport by introducing an external feedback mechanism that indirectly conveys output error information to early network layers. Inspired by control-theoretic feedback systems such as PID and ADRC, corrective signals guide early representation updates while subsequent layers adapt through forward propagation dynamics. This resembles biological learning, where internal representations are refined through indirect supervisory feedback. Experiments on MNIST and CIFAR using fully connected networks with batch normalization, dropout, and adaptive feedback matrices show test performance generally within $\pm 2\%$ of backpropagation, suggesting IFA as a promising biologically plausible alternative for neural network learning.

Keywords: Deep neural networks · Backpropagation · Feedback alignment · Indirect feedback alignment · Gradient methods · Control systems · Biologically inspired computing · Neural network training · Memory-efficient training

1 Introduction

Backpropagation, introduced by Rumelhart, Hinton, and Williams [1] in 1986, has become the dominant supervised learning algorithm for training deep neural networks, driving significant advances across artificial intelligence. From early feedforward

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 253–267, 2027.
https://doi.org/10.1007/978-3-032-39763-8_15

Multi-Layer Perceptron (MLP) architectures [2] to modern large-scale models such as Transformer-based models [3] like GPT and BERT, backpropagation has enabled efficient optimization of high-dimensional parameter spaces and the learning of complex representations. Despite its empirical success, however, backpropagation has long been criticized for its lack of biological plausibility. In particular, it requires symmetric weight transport, precise layer-wise error propagation, and non-local learning signal features that are not observed in biological neural systems.

Neuroscientific studies suggest that cortical processing is governed by a combination of structured forward-directed signal flow within local microcircuits and extensive recurrent and feedback interactions at larger scales [4–7]. These findings motivate the exploration of alternative learning mechanisms that do not rely on strict backward error propagation. In recent years, several biologically inspired approaches have been proposed. For example, the Forward-Forward (FF) algorithm [8] introduced by Hinton replaces the backward pass with a second forward pass driven by contrasting data distributions. A closely related concept is the feedback alignment (FA) principle [14], which demonstrates that exact symmetry between forward and backward weights is not necessary for effective learning. Instead of using transposed weights for gradient propagation, FA employs fixed random feedback connections, and the network adapts its forward weights in such a way that useful learning signals emerge. This phenomenon suggests that precise gradient transport may not be essential for training deep networks, opening the door to more flexible and biologically plausible learning mechanisms. Notably, Nøkland [9] briefly introduces a configuration called Indirect Feedback Alignment (IFA) as a possible feedback pathway topology. Unlike IFA, which retains a global output error signal projected through a single fixed matrix B_1, FF eliminates the backward pass entirely at the cost of requiring a contrastive negative data source and local goodness functions at each layer. A direct empirical comparison between IFA and FF is deferred to future work, as FF requires a fundamentally different data pipeline. Similarly, Direct Feedback Alignment (DFA), proposed by Nøkland [9], introduces fixed random feedback pathways that transmit error signals directly from the output layer to hidden layers, eliminating the need for symmetric weights. However, that work does not further develop or experimentally investigate IFA as a standalone learning framework. In contrast, the present work expands upon this concept and formulates IFA as a distinct and principled learning paradigm.

In this paper, we reinterpret IFA through the lens of control theory, viewing it as a closed-loop feedback system. Rather than explicitly propagating gradients backward through each layer, an external feedback signal conveys output error information to the network's early layers, analogous to how a controller regulates system behavior based on output deviations. This perspective is closely related to classical control mechanisms such as Proportional–Integral–Derivative (PID) control [10] or modern Active Disturbance Rejection Control (ADRC) [11], where corrective actions are applied indirectly to influence system dynamics at the input stage. Figure 1 illustrates a typical PID schematic diagram of how the output signal C(s) is fed back to the system input and interacts with the input signal R(s). From a biological standpoint, this mechanism resembles how humans and animals learn through supervision: sensory inputs are processed forward to

generate predictions or actions, and corrective feedback, often indirect and delayed, is used to refine internal representations.

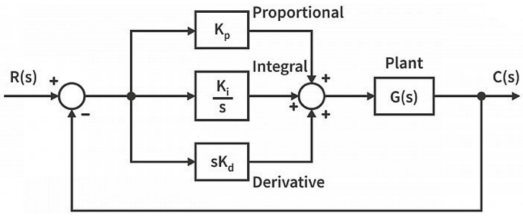


Fig. 1. A typical Proportional, Integral, and Derivative Controller (PID)

Building on these insights, we propose Indirect Feedback Alignment (IFA) as a forward-driven learning paradigm in which corrective signals are injected into early network layers, and subsequent layers adapt through forward propagation. This approach relaxes the requirement for exact end-to-end gradient backpropagation while preserving effective credit assignments. IFA is also a memory-efficient training methodology because the gradients do not have to be stored for every layer. By integrating ideas from feedback alignment, control theory, and biological learning, IFA provides a novel perspective on training deep neural networks that is both computationally viable, memory-efficient, and more aligned with natural learning processes.

2 Related Work

The backpropagation (BP) algorithm requires the backward propagation of errors using the transpose of the forward synaptic weights, commonly referred to as the weight transport problem, which is considered biologically implausible [12, 14]. To address this issue, Timothy P. Lillicrap et al. [14] proposed Feedback Alignment (FA), in which the transposed weight matrix W^T in the backward pass is replaced by a fixed random matrix B. During training, the forward weights progressively align with the random feedback pathways, enabling effective learning without exact weight symmetry. Although FA achieved competitive performance on simple datasets such as MNIST, its performance degraded on deeper convolutional architectures and more complex tasks [13].

Arild Nøkland proposed Direct Feedback Alignment (DFA) [9], in which each hidden layer receives error signals directly from the output layer through dedicated fixed random feedback matrices. By bypassing layer-wise backward propagation, DFA mitigates backward locking and enables parallel gradient computation across layers. Although DFA performs well for fully connected networks, its effectiveness degrades for deep convolutional architectures. Prior work further showed that exact weight symmetry is not strictly necessary for effective learning, provided partial sign agreement exists between forward and backward pathways [15].

To improve scalability in convolutional networks, Theodore H. Moskovitz et al. [16] introduced sign-concordant feedback methods, in which the feedback matrix B preserves the sign structure of the forward weights W without requiring exact magnitude symmetry.

In the simplest variant, Uniform Sign-concordant Feedback (uSF), $B_\ell = \text{sign}(W_\ell{}^T)$ after each update; a stronger variant, Strict Normalization (SN), additionally rescales B_ℓ to match $\|W_\ell\|_2$. Combined with forward-weight norm constraints, these methods substantially improved FA performance on deep convolutional networks, achieving a top-1 ImageNet error of 51.2% on ResNet architectures [16]. More recently, Julien Launay et al. demonstrated that DFA scales to modern architectures beyond image classification, including Transformers and graph neural networks, although a performance gap relative to BP remains for convolutional tasks [17]. Akrout et al. [18] proposed Kolen-Pollack (KP) learning, in which feedback matrices are updated with the same gradient as their forward counterparts plus weight decay λ, causing B to converge to W^T as $t \to \infty$ through the contraction $|W_- B|_{t+1} = (1 - \lambda)_{t+1}|W_- B|_0$. Extending this to DFA. In the IFA context, our experiments found no benefit from DKP over a fixed B_1, consistent with the observation that IFA's forward propagation through trained W matrices already addresses the alignment problem that DKP targets in DFA.

The present work builds on these foundations by investigating IFA performance under systematic variation of feedback update strategy (fixed, uSF, SN), normalization scheme (BatchNorm1d, LayerNorm), pooling type, and architecture depth, to characterize conditions under which IFA most closely approaches the BP performance ceiling on image classification benchmarks. Figure 2 illustrates the different feedback pathway configurations used in the feedback alignment models discussed, in comparison with the Backpropagation model.

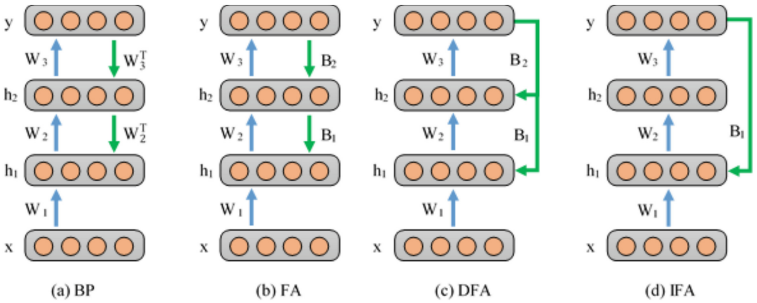


Fig. 2. From left to right, depicts four different error transportation configurations: (a) Backpropagation BP, (b) Feedback-alignment FA, (c) Direct Feedback-alignment DFA, and (d) Indirect Feedback-alignment IFA. Wi denotes the learning weights, and Bi denotes

3 Methodology

In this section, a detailed mathematical formulation of an N-hidden-layer Indirect Feedback Alignment (IFA) network is presented, with Batch Normalization (BN) at every hidden layer and Uniform Sign-concordant Feedback (uSF) update of the single feedback matrix B_1. Since we use only a single feedback matrix B_1 as compared to BP, FA and DFA, all of which use one matrix per layer, IFA becomes more memory efficient.

The formulation is generalized by using index $i \in \{1, \ldots, N\}$ for hidden layers and j for individual neurons within a layer. Let the network have N hidden layers and an output layer (layer N + 1) as shown in Fig. 3 below:

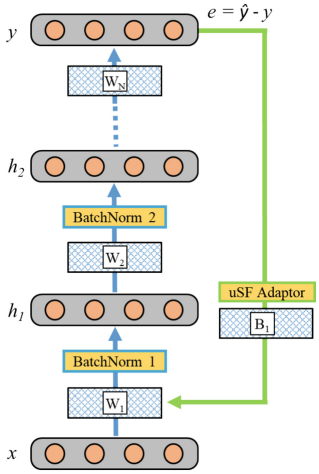


Fig. 3. An IFA-based fully connected network with N hidden layers, incorporating Batch Normalization between layers. An adaptive operation, uSF, is applied to the feedback matrix B_1

Define the following notations:

d_0	Input dimension
d_i $(i = 1,\ldots,N)$	Width of hidden layer i (neurons per layer)
C	Number of output classes
$W_i \in \mathbb{R}^{d_i \times d_{i-1}}$	Trainable forward weight matrix, layer i
$b_i \in \mathbb{R}^{d_i}$	Trainable bias vector, layer i
$W_{N+1} \in \mathbb{R}^{C \times d^3}$	Output layer weight matrix
$B_1 \in \mathbb{R}^{d_1 \times C}$	Fixed feedback matrix (updated by uSF only; never by gradient)
$x \in \mathbb{R}^{d_0 \times n}$	Mini-batch input (n = batch size)
$z_i \in \mathbb{R}^{d_i \times n}$	Pre-activation at hidden layer i
$\bar{z}_i \in \mathbb{R}^{d_i \times n}$	BatchNorm output (post-BN, pre-activation) at layer i
$h_i \in \mathbb{R}^{d_i \times n}$	Post-activation output of hidden layer i
$\tilde{\delta}_i \in \mathbb{R}^{d_i \times n}$	IFA pseudo-error at hidden layer i
$f(\cdot)$	Element-wise activation function (e.g., ReLU, GELU)
$f'(\cdot)$	Element-wise derivative of f
$\odot$	Hadamard (element-wise) product

(*continued*)

(*continued*)

d_0	Input dimension
α	Learning rate
ε	BatchNorm numerical stability constant
γ_i, β_i	Learnable BN scale and shift, layer *i*

3.1 Forward Pass

The forward pass is identical for all feedback alignment variants. It proceeds sequentially from the input through all N hidden layers to the output.

Linear pre-activation at hidden layer *i*, (*i* = 1, …, N):

$$z_i = W_i \cdot h_{i-1} + b_i \; h_0 := x \; (input)$$

Batch Normalization at hidden layer *i*:

$$\mu_i = \overline{z}_i = \frac{1}{n}\sum_{s=1}^{n} z_{is}, \sigma_i^2 = \frac{1}{n}\sum_{s=1}^{n}(z_{is} - \mu_i)^2$$

$$\overline{z}_i = \gamma_i \odot (z_i - \mu_i)/\sqrt{\left(\sigma_i^2 + \varepsilon\right)} + \beta_i$$

Nonlinear activation at hidden layer *i*:

$$h_i = f(\overline{z}_i)$$

Output layer:
$\hat{y} = softmax(W_{N+1} \cdot h_N + b_{N+1})$ No BatchNorm at the output layer
Loss and output error:
$L = \frac{1}{n}\sum_j \sum_c y_{jc} \log \hat{y}_{jc}$ Categorical cross-entropy loss
$e = \frac{\partial L}{\partial a_y} = \hat{y} - y$ Output error: $e \in \mathbb{R}$ ^ {C × *n*}

3.2 IFA Backward Pass

The IFA backward pass generates a pseudo error $\delta_i^{\sim}$ at each hidden layer without using transposed weight matrices. It proceeds in two phases: (a) a single feedback projection from the output error to layer 1 via B_1, and (b) forward propagation of that signal through the trained weight matrices W_i to all deeper layers.

Pseudo-error at hidden layer 1:

$$\overset{\sim}{\delta}_1 = (B_1 \cdot e) \odot f'(\overline{z}_1) \; B_1 \cdot e : \mathbb{R}\hat{}\{C \times n\} \rightarrow \mathbb{R}\hat{}\{d_1 \times n\}$$

Forward propagation of pseudo error ($i = 2, \ldots, N$):

$\tilde{\delta}_i = \left(W_i \cdot \tilde{\delta}_{i-1} \right) \odot f'(\bar{z}_i)$ W_i used in the forward direction (not transposed)

IFA avoids W_i^T entirely for $i = 1, \ldots, N-1$; only B_1 is used as a non-forward weight. Contrast with BP, which would use:

$$\delta_i^{BP} = \left(W_{i+1}^T \cdot \delta_{i+1}^{BP} \right) \odot f'(\bar{z}_i)$$

3.3 Weight and Bias Updates

Given the pseudo-errors $\tilde{\delta}_i$ and the output error e, all weight matrices are updated simultaneously using the outer-product rule:

Output layer (exact gradient, $i = N \pm 1$):

$$\Delta W_{N+1} = -\alpha \cdot \frac{1}{n} \cdot e \cdot h_N^T$$

$$\Delta b_{N+1} = -\alpha \cdot \frac{1}{n} \cdot \sum_{j=1}^{n} e_j$$

Hidden layers ($i = 1, \ldots, N$):

$$\Delta W_i = -\alpha \cdot (1/n) \cdot \tilde{\delta}_i \cdot h_{i-1}^T \; \Delta W_i \in \mathbb{R}^{\wedge}\{d_i \times d_{i^{-1}}\}$$

$$\Delta b_i = -\alpha \cdot \frac{1}{n} \cdot \sum_{j=1}^{n} \tilde{\delta}_{ij}$$

3.4 uSF Update of the Feedback Matrix B_1

After each weight update step, B_1 is refreshed by the uSF adaptor. The magnitudes of B_1 are frozen at their initial values; only the signs are updated to track the column-wise sign of W_1.

Initialization (computed once, $t = 0$):

$|B_{1,0}| = abs(B_{1,0})$ $|B_{1,0}|$ initialized at the start of training and never modified.

Column-wise majority sign of W_1 (computed at each step t):

$$s_r(t) = \text{sign}\left(\sum_{k=1}^{d_0} [W_1(t)]_{rk} \right), \quad r = 1, \ldots, d_1$$

$s_r(t) \in \{-1, +1\}$ is the majority-vote sign of the *r-th* row of W_1 across all d_0 input connections; ties resolved as $+1$

uSF update rule (applied after Steps 9a–10b at each training step):

$$\mathbf{B_1}(t) = \left| \mathbf{B_{1,0}} \right| \odot s(t)$$

$s(t) \in \{-1, +1\}^{\wedge}\{d_1\}$ broadcast over the C rows of B_1. No gradient is computed for B_1.

4 Experimental Results

4.1 Experimental Setup

To carry out a systematic comparison across a large number of configurations, a custom training library was developed using the PyTorch framework, providing an API similar to the Keras sequential model interface. The library supports fully-connected MLP and convolutional CNN architectures with uniform programming access for BP, FA, DFA, and IFA training methods. The detailed configuration of the hardware used to run the experiments, and the code repository is specified in Appendix I. A typical 800x256 tanh network was trained on MNIST with both BP and IFA shown in Fig. 4 below.

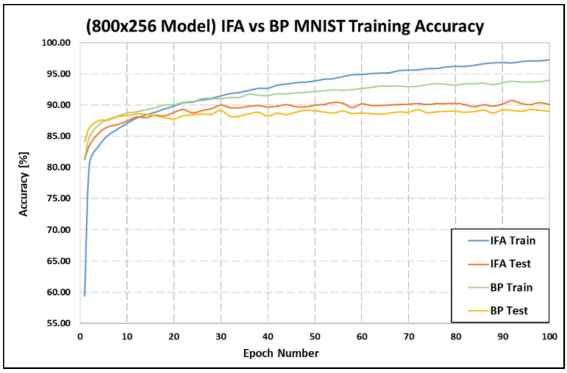


Fig. 4. Accuracy curve for a typical training of an 800x256 tanh network on the MNIST dataset

Unless stated otherwise, all IFA, FA, and DFA models used the following common settings: Batch Normalization (BatchNorm1d) after each hidden Dense layer; Uniform Sign-concordant Feedback (uSF) update of B1 after every weight step; batch size of 64; between 100 and 300 training epochs; and learning rates selected per optimizer using the values shown in Table 1. Dropout was applied only where noted. Each reported accuracy represents the mean ± SEM% (Standard Error of the Mean) over five independent runs with different random seeds. For the CIFAR-10 and CIFAR100 runs, the experiments use batch sizes of 64 and 128, respectively, with 300 training epochs. Generally, the IFA learning rate choice for adaptive optimizers (Adam, AdamW, NAdam) on MNIST and Fashion-MNIST, with stable convergence observed across LR $\in \{0.0001–0.01\}$ and peak accuracy varying by less than 0.5 pp. across this range. RMSProp and SGD + Momentum are more sensitive: RMSProp performs best in the narrower range LR $\in \{0.0005–0.002\}$, while SGD + Momentum requires a smaller LR than for BP (optimal LR $\in \{0.003–0.03\}$ vs. $\{0.03–0.1\}$ for BP), consistent with the noisier pseudo-gradient signal of FA-family methods requiring more conservative step sizes. On harder datasets (CIFAR-10, CIFAR-100), all methods become more LR-sensitive; the optimal IFA learning rate with Adam narrows to approximately 0.001, and values above 0.005 consistently caused divergence. Notably, the NAdam configuration yielding the IFA accuracy advantage on CIFAR-100 (LR = 0.0005, batch size 128) was sensitive to both parameters. Increasing LR to 0.001

reduced the IFA–BP margin to less than 1 pp. However, a formal sensitivity analysis was not conducted; the ranges reported in reflect the configurations evaluated in this study and are intended as a practical starting point rather than an exhaustive characterization.

4.2 MNIST Classification

Table 1 presents test accuracy on the MNIST handwritten digit dataset [19] for a range of MLP architectures using the Tanh activation function. Models span 2-hidden-layer networks (256 × 128 and 800 × 256) through to a 7-hidden-layer network (128 × 7 L), evaluated with AdamW, RMSProp, Adam, and NAdam optimizers. No dropout was applied for the MNIST experiments, as regularization was not found to be beneficial for this dataset. BatchNorm1d was applied to all models except the shallow 2-layer networks, where it caused a slight accuracy degradation, consistent with the observation that BatchNorm introduces instability in narrow, shallow MLP models where the batch statistics are less informative

Table 1. MNIST test classification accuracy (mean ± SEM %, five runs) for various MLP architectures using Tanh activation. BP = backpropagation; IFA = indirect feedback alignment; FA = feedback alignment; DFA = direct feedback alignment. Models highlighted in blue are IFA results. The 4 L and 7 L indicate 4 and 7 hidden layers, respectively

MODEL	BP %	IFA %	FA %	DFA %
Tanh 256x128 (AdamW)	97.61 ±0.05	97.58 ±0.03	97.64 ±0.05	97.71 ±0.05
Tanh 256x128 (RMSProp)	97.98 ±0.05	97.40 ±0.09	97.77 ±0.02	97.63 ±0.06
Tanh 800x256 (Adam)	97.31 ±0.07	98.05 ±0.05	98.23 ±0.03	98.09 ±0.05
Tanh 800x256 (NAdam)	97.42 ±0.11	98.05 ±0.02	98.17 ±0.02	98.15 ±0.04
Tanh 512x256x128 (RMSProp)	98.13 ±0.03	97.58 ±0.05	97.97 ±0.06	97.81 ±0.04
Tanh512x256x128 (AdamW)	97.63 ±0.02	97.91 ±0.05	97.95 ±0.06	97.80 ±0.04
Tanh 800x512x128 (RMSProp)	98.23 ±0.04	97.66 ±0.04	98.08 ±0.04	97.99 ±0.03
Tanh 800x512x128 (AdamW)	97.04 ±0.16	98.07 ±0.05	98.05 ±0.03	97.98 ±0.03
Tanh 800x4L (RMSProp)	97.91 ±0.04	97.94 ±0.07	98.00 ±0.04	98.13 ±0.04
Tanh 128x7L (NAdam)	97.28 ±0.05	96.77 ±0.09	96.54 ±0.06	97.21 ±0.06

The results in Table 1 also show that IFA achieves accuracy comparable to BP across all tested architectures and optimizers. In several configurations, IFA marginally outperforms BP, for example, the 800 × 256 model with Adam achieves 98.05 ± 0.05% under IFA versus 97.31 ± 0.07% for BP. The performance gap between methods narrows or reverses for shallower networks (256 × 128), where BP retains its typical advantage. For the deepest architecture tested (128 × 7 L with NAdam), IFA achieves 96.77 ± 0.09% compared to BP at 97.28 ± 0.05%, representing a modest 0.51 percentage points (pp) gap.

4.3 Fashion-MNIST Classification

As seen in Table 2, the overall accuracy ranges across all methods narrows compared to MNIST, clustering between approximately 86% and 90%. FA and DFA consistently outperform both IFA and BP by 0.5–1.0 pp. on Fashion-MNIST, a pattern not observed on MNIST. This may be attributed to the multi-layer feedback structure of FA and DFA, providing better regularization on the harder dataset. IFA nonetheless performs comparably to BP in most configurations, with margins typically within ±0.3 pp. The representative uploaded IFA runs confirm this: Tanh Adam 800 × 256 achieved a mean of 89.12 ± 0.05%, and Tanh AdamW 800 × 512 × 128 achieved 89.08 ± 0.07%, both consistent with Table 2.

The 7-layer (128 × 7 L) architecture with NAdam shows the largest IFA-BP gap (86.89 vs. 88.04%), suggesting that IFA's single-feedback-matrix design is less effective for very deep narrow networks on harder datasets. This is consistent with the theoretical expectation that deeper propagation of the pseudo-error through trained W matrices accumulates more approximation error as network depth increases.

Table 2. Fashion-MNIST test classification accuracy (mean ± SEM %, five runs) for various MLP architectures using Tanh activation with 10% dropout. Column headings as in Table 1.

MODEL	BP %	IFA %	FA %	DFA %
Tanh 256x128 (AdamW)	88.78 ±0.07	88.78 ±0.05	89.17 ±0.09	89.29 ±0.06
Tanh 256x128 (RMSProp)	88.45 ±0.10	88.77 ±0.09	89.08 ±0.07	89.37 ±0.07
Tanh 800x256 (Adam)	88.63 ±0.07	89.12 ±0.05	89.78 ±0.05	89.52 ±0.05
Tanh 800x256 (NAdam)	88.56 ±0.06	89.35 ±0.07	89.72 ±0.09	89.66 ±0.06
Tanh 512x256x128 (RMSProp)	88.71 ±0.06	88.78 ±0.08	89.34 ±0.08	89.39 ±0.08
Tanh512x256x128 (AdamW)	88.98 ±0.06	89.01 ±0.10	89.48 ±0.06	89.44 ±0.03
Tanh 800x512x128 (RMSProp)	88.66 ±0.06	89.08 ±0.03	89.56 ±0.09	89.48 ±0.09
Tanh 800x512x128 (AdamW)	88.85 ±0.08	89.08 ±0.07	89.66 ±0.10	89.58 ±0.10
Tanh 800x4L (RMSProp)	88.49 ±0.12	88.29 ±0.11	88.96 ±0.05	88.72 ±0.11
Tanh 128x7L (NAdam)	88.04 ±0.09	86.89 ±0.11	87.89 ±0.10	88.53 ±0.04

4.4 CIFAR-10 and CIFAR-100 MLP Classification

Table 3 presents MLP classification results on CIFAR-10 (10 classes) and CIFAR-100 (100 classes), both drawn from the same set of 32 × 32 RGB images [20]. Unlike MNIST and Fashion-MNIST, these datasets require the MLP to classify color images by flattening the full 32 × 32 × 3 = 3,072-dimensional input, discarding all spatial structure. All experiments used BatchNorm, 20% dropout, and 300 training epochs for feedback alignment methods.

Table 3. CIFAR-10 and CIFAR-100 MLP test accuracy (mean ± SEM %, five runs). BP = backpropagation; IFA = indirect feedback alignment (highlighted in blue); FA = feedback alignment; DFA = direct feedback alignment. BN = BatchNorm; DO = dropout; BS = batch size; Ep = epochs

Dataset	Model	BP %	IFA %	FA %	DFA %
CIFAR-10	ReLU 1024×800 (AdamW)	57.72 ±0.18	56.72 ±0.18	56.62 ±0.10	56.87 ±0.09
	GeLU 1024×800 (SgdM)	57.13 ±0.14	56.12 ±0.18	56.37 ±0.09	57.34 ±0.16
	GeLU 1024×3L (SgdM)	56.96 ±0.32	56.39 ±0.21	55.31 ±0.11	57.80 ±0.19
CIFAR-100	ReLU 1024×1024 (SgdM)	26.72 ±0.17	27.67 ±0.12	28.79 ±0.11	29.04 ±0.11
	ReLU 1024×1024 (NAdam)	29.70 ±0.06	25.23 ±0.14	28.16 ±0.16	28.86 ±0.09

CIFAR-10: All four methods cluster within ±1.5 pp. of each other across all three architectures, with accuracy ranging from 55.3% to 57.8%. This narrow spread indicates that the MLP architecture itself, rather than the training algorithm, is the primary bottleneck on CIFAR-10. BP achieves the highest single result (57.72 ± 0.18% with ReLU 1024 × 800 and AdamW), while DFA matches it with GeLU 1024 × 3 L under SGD + Momentum (57.80 ± 0.19%). IFA remains competitive throughout, trailing BP by only 0.6–1.1 pp. across all three configurations and demonstrating that IFA scales to three-hidden-layer architectures on this harder dataset without significant degradation. The ceiling of approximately 57–58% for all methods reflects the fundamental limitation of processing spatial RGB data as a flattened vector.

CIFAR-100: IFA outperforms BP with the SGD+Momentum optimizer, achieving 27.67 ± 0.12% versus BP's 26.72 ± 0.17%, a margin of 0.95 pp in favor of IFA. This is the only configuration on CIFAR datasets where IFA exceeds BP, and the result is consistent with IFA's noisier pseudo-gradient requiring more conservative step sizes, where SGD+Momentum's regularizing effect appears beneficial. With NAdam, the relationship reverses substantially: BP leads at 29.70 ± 0.06% versus IFA at 25.23 ± 0.14%, a 4.47 pp gap, indicating strong optimizer-dependence. FA and DFA outperform both BP and IFA with SGD+Momentum (FA = 28.79%, DFA = 29.04%), suggesting that multi-layer feedback provides an advantage on this 100-class task.

Taken together, the CIFAR-10 and CIFAR-100 MLP results extend the key finding from Sects. 4.2 to 4.4: IFA accuracy is comparable to BP across a wide range of MLP architectures, optimizers, and datasets of varying difficulty. The CIFAR-100 NAdam result goes further, suggesting that IFA may offer a generalization advantage on harder multi-class problems that merits systematic investigation in future work.

5 Discussion

Several consistent patterns emerge across the experimental results. Smaller learning rates (typically ranging from 0.0001 to 0.00001) tend to produce higher accuracy, but they also slow the learning process, generally requiring more training epochs, typically between 100 and 300. In addition, Batch Normalization proved to be a stabilizing component

for all three feedback alignment methods: IFA, FA, and DFA, improving both training stability and final accuracy across the board. The benefit was particularly evident on Fashion-MNIST and CIFAR-10, where omitting BatchNorm frequently led to unstable loss curves. The one exception was shallow 2-layer MLP models on MNIST, where BatchNorm caused a marginal degradation in accuracy and was accordingly omitted for those configurations.

Figures 5 and 6 show that the combined application of batch normalization and uSF substantially improves training accuracy. Batch normalization helps stabilize training curves across epochs, while uSF improves gradient alignment during backpropagation, as shown in Fig. 6 (right panel), with cosine similarity rising faster and stabilizing at a higher value than training without uSF (Fig. 6, left panel).

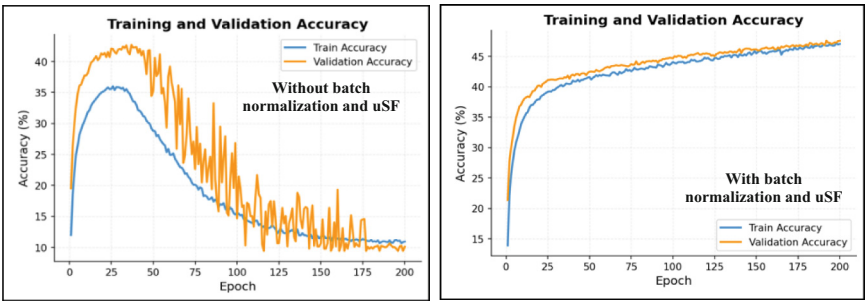


Fig. 5. The training and validation curves of a 1024 × 3 L GELU with SGD-Momentum optimizer network under identical hyperparameters are shown. The plot on the left corresponds to the model without batch normalization or uSF, while the plot on the right includes both batch normalization and uSF. The results show that batch normalization, together with uSF, stabilizes training curves and improves performance on the CIFAR-10 dataset

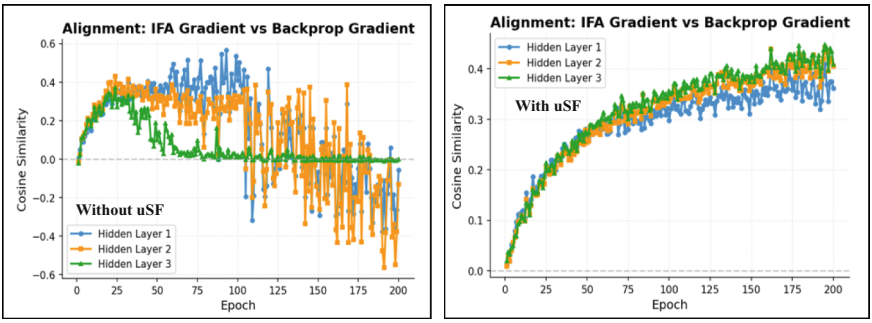


Fig. 6. Cosine similarity between the IFA gradient and the backpropagation gradient (called Alignment) during training of a 1024 × 3 L GELU network with the SGD-Momentum optimizer on the CIFAR dataset. The plot on the left shows the training run without uSF, while the plot on the right shows the results with the application of uSF

The FA and DFA results reported in Tables 1 and 2 exceed those published in Nøkland and Moskovitz et al., both of which did not incorporate BatchNorm in their MLP experiments. This improvement indicates that BatchNorm is a critical but previously underutilized component for FA-family methods, and that earlier published baselines underestimated the achievable accuracy of these algorithms. For MLP architectures spanning two to seven hidden layers, IFA achieves test accuracy within ± 1 pp. of BP across the large majority of configurations in Tables 1 and 2.

The CIFAR-10MLP accuracy ceiling of approximately 57% to 58% across all methods reflects the fundamental limitation that affects BP and IFA equally. Batch Normalization emerged as a critical enabler for all three feedback alignment methods. The FA and DFA results reported here exceed prior published benchmarks from Nøkland and Moskovitz et al., both of which did not apply BatchNorm in their MLP experiments, suggesting that earlier baselines underestimated the achievable accuracy of FA-family methods.

6 Conclusion

This paper presented the first systematic empirical study of Indirect Feedback Alignment (IFA) introduced by Nøkland [9], a biologically motivated learning algorithm that replaces backpropagation's weight-transport requirement with a single fixed feedback matrix B_1 and forward propagation of pseudo-errors through the network's own weight matrices. Across MLP architectures spanning 2 to 7 hidden layers on four benchmark datasets, IFA achieves accuracy comparable to BP, and in one configuration, CIFAR-100 with NAdam surpasses it by 4.47 pp., establishing IFA as a competitive and memory-efficient alternative for fully-connected networks.

The study also establishes Batch Normalization and identifies uSF as a critical but previously underutilized component for FA-family methods, and contributed to the final accuracy improvement. These findings support the broader conclusion that exact gradient transport is not necessary for effective deep network learning, and that IFA, with its single feedback matrix, forward-only error propagation, and closed-loop control interpretation, represents a principled and practically viable direction for hardware-efficient neural network training. Extension of IFA to Convolutional Neural Networks (CNNs), residual architectures (ResNets) and attention-based models (Transformers) remains an open direction. Residual skip connections may partially mitigate the forward-propagation distortion identified in convolutional layers, and the single-feedback-matrix design scales as a constant with network depth. If future work demonstrates generalization across other architectures, IFA can be a competitive alternative to backpropagation, especially for memory-efficient training regimes.

Acknowledgements. This work was supported by Penn State Behrend 2026 SEED Grant Program.

Appendix I

ENVIRONMENT SETUP — System Information

Operating System	Windows 11
System Memory	32.0 GB
Python Version	3.9.18
Platform	Windows-10-10.0.26200-SP0
PyTorch Version	2.7.1+cu118
CUDA Version	11.8
cuDNN Version	90100 (90.1.0)
NVIDIA driver	591.8
GPU Count	1
GPU Name	NVIDIA GeForce RTX 4080 SUPER
GPU Memory	15.99 GB
GPU Arch	SM 8.9

Source code available at: https://github.com/tmliaw/IFA

References

1. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. Nature. **323**, 533–536 (1986)
2. Pinkus, A.: Approximation theory of the MLP model in neural networks. Acta Numerica. **8**, 143–195 (1999)
3. Vaswani, A. et al.: Attention Is All You Need. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 30, (2017)
4. Rao, R.P.N., Ballard, D.H.: Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. Nat. Neurosci. **2**(1), 79–87 (1999)
5. Serre, T., Oliva, A., Poggio, T.: A feedforward architecture accounts for rapid categorization. Proc. Natl. Acad. Sci. **104**(15), 6424–6429 (2007)
6. Yamins, D.L.K., DiCarlo, J.J.: Using goal-driven deep learning models to understand sensory cortex. Nat. Neurosci. **19**(3), 356–365 (2016)
7. Peng, Y., et al.: Directed and acyclic synaptic connectivity in the human layer 2–3 cortical microcircuit. Science. **384**(6696), 338–343 (2024)
8. Hinton, G.E.: The forward-forward algorithm: some preliminary investigations (2022). https://arxiv.org/abs/2212.13345
9. Nøkland, A.: Direct feedback alignment provides learning in deep neural networks. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 29, pp. 1045–1053 (2016)
10. Ang, K.H., Chong, G., Li, Y.: PID control system analysis, design, and technology. IEEE Trans. Control Syst. Technol. **13**(4), 559–576 (2005)
11. Zheng, Q., Gao, Z.: Active disturbance rejection control: some recent experimental and industrial case studies. Control Theory Technol. **16**(4), 301–313 (2018)
12. Kolen, J., Pollack, J.: Backpropagation without weight transport. In: IEEE International Conference on Neural Networks (ICNN), vol. 3, pp. 1375–1380 (1994)
13. Bartunov, S., Santoro, A., Richards, B.A., Marris, L., Hinton, G.E., Lillicrap, T.P.: Assessing the scalability of biologically motivated deep learning algorithms and architectures. In: Advances in Neural Information Processing Systems (NeurIPS), pp. 9368–9378 (2018)
14. Lillicrap, T.P., Cownden, D., Tweed, D.B., Akerman, C.J.: Random feedback weights support learning in deep neural networks. Nat. Commun. **7**, 13276 (2016)

15. Liao, Q., Leibo, J.Z., Poggio, T.: How important is weight symmetry in backpropagation? 2015. https://arxiv.org/abs/1510.05067
16. Moskovitz, T.H., Litwin-Kumar, A., Abbott, L.F.: Feedback alignment in deep convolutional networks (2019). https://arxiv.org/abs/1812.06488
17. Launay, J., Poli, I., Boniface, F., Krzakala, F.: Direct feedback alignment scales to modern deep learning tasks and architectures. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 33, (2020)
18. Akrout, M., Wilson, C., Humphreys, P., Lillicrap, T., Tweed, D.B.: Deep learning without weight transport. In: Advances in Neural Information Processing Systems (NeurIPS), pp. 974–982 (2019)
19. LeCun, Y., Cortes, C., Burges, C.J.C.: The MNIST database of handwritten digits (1998). [Online]. Available: http://yann.lecun.com/exdb/mnist/
20. Krizhevsky, A.: Learning Multiple Layers of Features from Tiny Images. University of Toronto, Technical Report (2009)

RepoWise: A Conversational Framework for Open Source Repository Analysis with Retrieval-Augmented Generation and Structured Analytics

Sankalp Kashyap(✉), Arjun Ashok, Nafiz Imtiaz Khan, and Vladimir Filkov

DECAL Lab, Department of Computer Science, University of California, Davis, Davis, CA 95616, USA
{sankashyap,arjashok,nikhan,vfilkov}@ucdavis.edu

Abstract. Open source software (OSS) stakeholders routinely need answers about project documentation, contributor activity, governance, and community health, yet empirical research documents persistent barriers to accessing this information. We present RepoWise, a conversational framework that enables natural-language interaction with OSS repositories through domain-specialized retrieval. RepoWise unifies semantic search over project documentation and structured analytics over commit and issue data, routed by an LLM-based intent classifier. The tool is publicly hosted with Mistral 7B as the default model, and can also be run locally with any model available through Ollama. To ground the tool's design in documented stakeholder needs, we derived a question taxonomy of 98 questions from 65+ sources, including peer-reviewed studies, industry surveys, and governance frameworks. We evaluate RepoWise on 200 question instances across ten repositories and compare accuracy against DeepWiki and Claude with GitHub MCP. Claude with GitHub MCP achieved the highest quantitative accuracy at 86.7%, followed by RepoWise at 77.5%, while RepoWise requires no paid API access and runs entirely on local hardware. A capability analysis reveals that documentation and analytics questions are well served by current tools, while community health and socio-technical questions lack reliable, verifiable support from current tools. The tool, taxonomy, and evaluation artifacts are publicly available.

Keywords: Open Source Software · Conversational AI · Repository Mining · Retrieval-Augmented Generation · LLM · Question Taxonomy

1 Introduction

A newcomer joins an open source project and immediately faces a set of questions: How do I contribute? Who are the active maintainers? What is the governance structure? Which files change most frequently? These questions span

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 268–288, 2027.
https://doi.org/10.1007/978-3-032-39763-8_16

multiple data sources (project documentation, commit logs, issue trackers) and require different analysis methods (text retrieval, structured aggregation). An OSPO manager evaluating the same project asks a different set of questions: What is the bus factor? Is the contributor base growing? Is this project still actively maintained? These require longitudinal analysis of community structure that no conversational tool currently provides.

Two decades of empirical research have documented these information needs extensively. Steinmacher et al. [35] identified 58 barriers newcomers face when placing their first contribution. Fronchetti et al. [14] found that 84% of CONTRIBUTING.md files miss at least two of the six documented newcomer barriers. The GitHub Open Source Survey [42] reported that 93% of respondents observed incomplete or outdated documentation. Avelino et al. [5] showed that 65% of popular GitHub projects have a bus factor (also termed truck factor) of two or fewer. Building on this body of work, we synthesize documented stakeholder needs into a structured question taxonomy and use it to assess the coverage of current conversational tools.

Conversational AI tools have begun to address some of these needs. GitHub Copilot Chat [15], DeepWiki [10], RepoChat [1], and agentic configurations that access GitHub's API through tool calls can answer questions about project documentation, code structure, and architecture. These tools represent significant progress in making repository information accessible through natural language.

RepoWise is designed with these documented needs in mind. It grounds its question coverage in the empirical literature on OSS stakeholder barriers and complements existing tools by providing structured analytics over commit and issue data alongside documentation retrieval, all running with local LLM inference and no paid inference API dependency.

Deploying large language models for software engineering tasks introduces domain-specific challenges: queries span heterogeneous data sources ranging from unstructured documentation to structured commit and issue records, responses must be traceable to project-specific artifacts, and structured data requires schema-aware query construction rather than text retrieval alone. RepoWise addresses these challenges through intent classification, grounding rules, and schema-aware analytics, contributing engineering solutions for deploying retrieval-augmented generation and LLM-driven analytics in a software engineering context.

We make two contributions:

1. **RepoWise: a domain-specialized conversational framework** for OSS repository analysis. RepoWise operationalizes documented stakeholder questions through intent-aware retrieval, query-type-specific prompting, schema-aware structured analytics, and source-grounding rules. The tool is publicly hosted with Mistral 7B as the default model, and can also be run locally with any model available through Ollama, requiring no LLM API keys and keeping all query data on the user's machine. Users interact through natural language without needing prompt engineering skills.

2. **A research-grounded question taxonomy** of 98 OSS stakeholder questions across 17 themes and 8 roles, derived from a five-step synthesis of 65+ sources, including peer-reviewed studies, major industry surveys, and foundation governance frameworks. The taxonomy grounds RepoWise's design in documented needs and identifies which question categories remain unaddressed by current conversational tools.

The tool, codebase, all prompt templates, and a screencast demonstration are publicly available at https://repowise.github.io/RepoWise-website/.

2 Related Work

2.1 Conversational Tools for Software Repositories

Conversational tools for repositories have evolved rapidly. GitHub Copilot Chat [15] answers natural-language questions about codebases. Sourcegraph Cody [34] provides codebase-aware AI assistance. RepoChat [1] translates natural-language queries into Cypher graph queries over a Neo4j knowledge graph constructed from repository metadata, requiring GPT-4o API access and a Neo4j database. This approach achieves high accuracy on metadata queries in its evaluation but does not index project documentation files. De Martino et al. [9] propose a framework for using LLMs in repository mining studies.

Agentic capabilities have expanded what these tools can do. GitHub Copilot in Agentic Mode, Claude Code, and Google Gemini access GitHub's REST and GraphQL APIs through tool calls, including MCP servers [16], enabling multi-step repository analysis. DeepWiki [10] generates comprehensive documentation sites from repository contents. Google Code Wiki [17] provides similar code understanding capabilities.

These tools answer the questions their developers anticipated, and many do so effectively. Our work complements them by grounding question coverage in the empirical literature on OSS stakeholder needs and assessing which question categories current tools can address.

2.2 OSS Stakeholder Barriers and Information Needs

Steinmacher et al. built a comprehensive newcomer barrier taxonomy through a systematic literature review of 20 studies (IST 2015 [36]; 15 barriers in 5 categories), followed by a mixed-method study combining the literature review, open-question responses, student feedback, and 36 semi-structured interviews to produce a 58-barrier conceptual model (CSCW 2015 [35]), later refined to 57 barriers in 7 categories (IEEE Software 2019 [37]). Balali et al. [6] extended this to mentor-side barriers. Santos et al. [31] found that existing software solutions address only 18 of Steinmacher et al.'s 58 documented barriers.

For maintainers, Tidelift [41] reports that 60% are unpaid and 60% have quit or considered quitting their maintenance work. Ferreira et al. [13] found that 59.7% of 174 FLOSS projects experience at least 30% annual core-developer

turnover. For governance, O'Mahony and Ferraro [26] characterized four phases of governance evolution in Debian. Oliveira et al. [25] found that identical governance role titles carry different responsibilities across projects, a phenomenon they term "role drift."

Tan et al. [40] envision AI mentors for newcomer onboarding, identifying 32 design strategies. Cassee et al. [7] identified two previously undocumented barriers affecting student contributors to OSS: "Conflicting Mentor Guidance" and "Communication Channel Ambiguity." Larsen et al. [19] developed SPAC-B, an LLM-based assistant for software project onboarding.

3 Question Taxonomy

To ground RepoWise's design in documented stakeholder needs, we derived a taxonomy of 98 questions through a five-step synthesis:

Step 1: Barrier-to-question derivation from Steinmacher et al.'s barrier taxonomy [37] (57 barriers in 7 categories), translating each barrier into the natural-language question a stakeholder would ask (32 newcomer questions across 8 themes). **Step 2:** Systematic survey of 48 empirical studies from ICSE, FSE, MSR, CSCW, TSE, TOSEM, EMSE, and SANER (2005–2026), each contributing questions tied to documented stakeholder information needs. **Step 3:** Analysis of five major practitioner surveys: Tidelift [41], GitHub Open Source Survey [42], LF FOSS Contributor Survey [20], LF DEI Survey [21], and Synopsys OSSRA [38]. **Step 4:** Foundation governance framework analysis (Apache Maturity Model, CNCF graduation, Eclipse incubation, TODO Group OSPO model), translating evaluation criteria into committee questions. **Step 5:** Security standard and incident analysis (OpenSSF Scorecard, SLSA v1.0, XZ Utils backdoor post-mortem).

Table 1 summarizes the resulting structure. The first three categories (44 questions) are answerable by RepoWise and, to varying degrees, by other tools. The remaining categories (54 questions) require longitudinal network analysis, external security tools, or survey-based assessment. No conversational tool currently provides reliable, verifiable answers for these categories. This gap reflects a methodological constraint: these questions require analysis techniques such as network construction, graph-theoretic computation, and longitudinal modeling that go beyond what retrieval-based conversational tools currently support.

The complete taxonomy with all 98 questions, stakeholder role assignments, evidence tables, and the full source registry (65+ citations) is publicly available as a supplementary artifact.[1]

4 System Architecture

RepoWise is a modular conversational system that combines retrieval-augmented generation over project documentation with structured analytics over commit

[1] https://docs.google.com/document/d/1uYwgbm0N9QarDqyeNLNqdKJb3W4-yI2FrIG9GzDh6Vw/.

Table 1. Question taxonomy organized by required data source and RepoWise (RW) coverage.

Category	# of Qs	Data Source	Example Questions	RW
Documentation & Onboarding	24	Project docs	How do I contribute? What is the code of conduct?	Yes
Commit Analytics	12	Commit metadata	Top contributors? Most modified files?	Yes
Issue Analytics	8	Issue metadata	Most active reporter? Oldest open issue?	Yes
Governance & Roles	14	Docs + context	How are decisions made? Who has merge authority?	Partial
Community Health	15	Longitudinal data	Bus factor? Contributor retention?	No
Security & Supply Chain	8	External tools	CVE patching speed? SBOM completeness?	No
Network Intelligence	7	Social/tech graphs	Bridge developers? Fragmented silos?	No
DEI, Funding, OSPO	10	Surveys + policy	Gender representation? License risk?	No

and issue data. Given a GitHub repository URL, it extracts project documentation, fetches commit and issue data, and answers natural-language questions by routing them through an intent classifier to the appropriate retrieval pipeline. Figure 1 illustrates the architecture and the flow from user query to generated response.

Mistral 7B [23] via Ollama handles all language understanding and generation tasks. Sentence-transformers [29] with all-MiniLM-L6-v2 [32] provide 384-dimensional embeddings. ChromaDB stores vector indexes with persistent, per-project collections. The frontend is implemented in React 18, the backend in Python (FastAPI with async Uvicorn). The inference layer is model-agnostic: any model available through Ollama can be substituted without modifying the retrieval or prompt components.

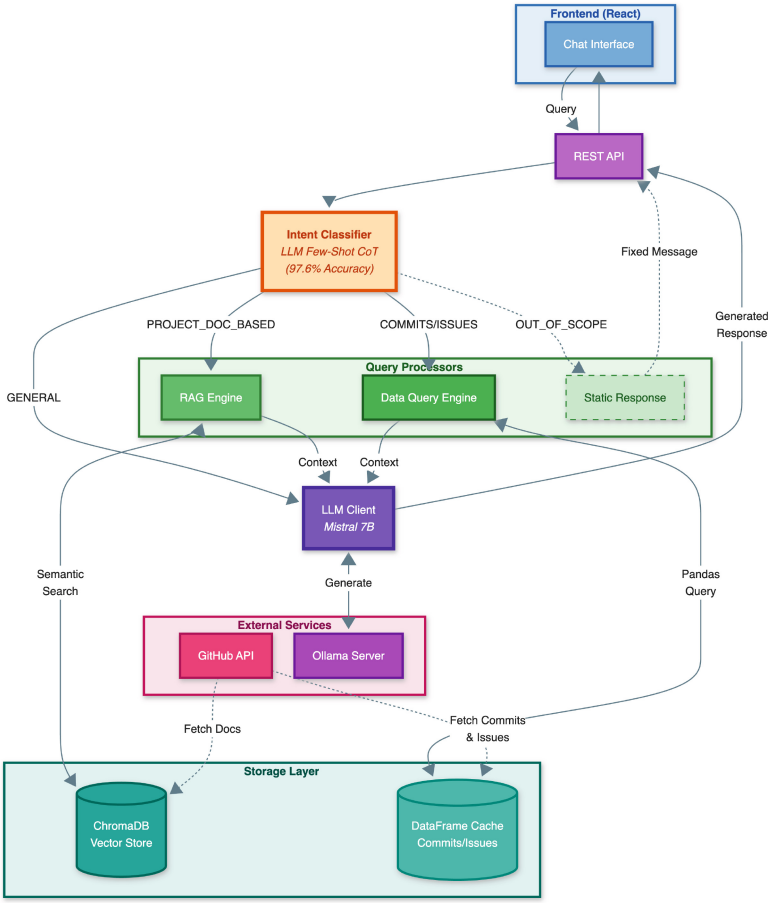


Fig. 1. System architecture of RepoWise. User queries are classified into five intent categories and routed to the appropriate pipeline: documentation retrieval (RAG Engine) for project documentation queries, structured analytics (Data Query Engine) for commit and issue queries, direct LLM response for general questions, or a static redirect for out-of-scope requests. Both the RAG Engine and Data Query Engine pass retrieved context to the LLM Client for response generation.

4.1 Repository Onboarding

Before a user can ask questions, the repository must be indexed. Figure 2 illustrates this two-phase process. In the foreground, project documentation (8 file types) is extracted via the GitHub API, chunked, embedded, and stored in ChromaDB, making documentation queries available within seconds. In the background, commit and issue data are fetched through our scraper and loaded into pandas DataFrames with local caching.

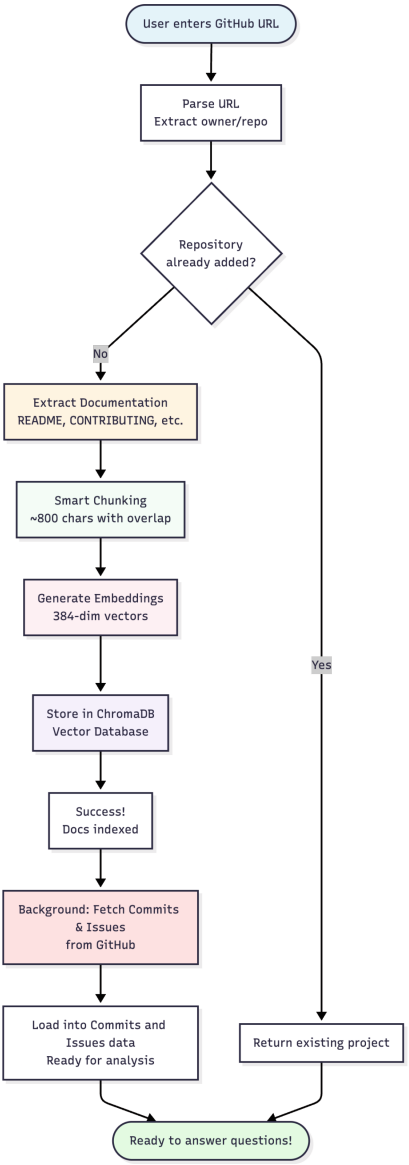


Fig. 2. Repository onboarding pipeline. Documentation extraction and indexing occur in the foreground; commit and issue data are fetched in the background.

4.2 Query Reception and Routing

The user interface is a lightweight chat-style application that accepts free-form questions. The interface communicates with the backend via asynchronous RESTful API calls through a `POST /api/query` endpoint. Each request includes

the project identifier, query text, optional conversation state (a running summary from prior turns), and generation parameters (temperature, max results).

4.3 Intent Classification

The intent classifier is the first and most consequential routing decision. It determines which retrieval pipeline handles each query by classifying it into one of five categories:

- `PROJECT_DOC_BASED`: Questions about governance, contribution guidelines, maintainers, licenses, security policies, or code of conduct. Routed to the RAG pipeline.
- `COMMITS`: Questions about commit history, contributors ranked by activity, file modifications, or development patterns. Routed to the structured analytics pipeline with commits data.
- `ISSUES`: Questions about bug reports, feature requests, issue reporters, or issue statistics. Routed to the structured analytics pipeline with issues data.
- `GENERAL`: Generic programming or technology questions not specific to the repository. Routed directly to the LLM without repository context.
- `OUT_OF_SCOPE`: Greetings, weather, sports, or other queries unrelated to repository analysis. Returns a static redirect message without invoking the LLM.

Classification uses a hybrid architecture. **Stage 1** applies fast keyword matching to detect clearly out-of-scope queries (greetings, off-topic patterns) without invoking the LLM. **Stage 2**, for all remaining queries, uses Mistral 7B with a chain-of-thought (CoT) prompting strategy containing 32 few-shot exemplars. Each exemplar demonstrates explicit reasoning about what data source the question requires and where the answer would be found. For example:

```
Query: ‘‘Who are the core developers?’’
Reasoning: ‘‘Core developers’’ typically means the most active
contributors by code contribution. This is determined by commit
statistics, not governance documents which list maintainer roles.
Intent: COMMITS


Query: ‘‘Who maintains this project?’’
Reasoning: ‘‘Maintainers’’ are explicitly defined roles documented in
MAINTAINERS.md, CODEOWNERS, or governance docs. This is asking
about documented roles, not commit statistics.
Intent: PROJECT_DOC_BASED
```

These two queries both ask about people in the project but require different data sources: “core developers” implies a ranking derived from commit activity, while “maintains” refers to roles explicitly documented in project files. Routing either to the wrong pipeline produces a plausible but incorrect answer, as commit

rankings do not reflect documented roles and vice versa. The 32 exemplars were selected to cover such boundary cases across all five intent categories.

We evaluated the intent classifier against 250 queries spanning all five categories. The first author labeled all 250 queries; two independent external annotators labeled the same set. Inter-annotator agreement across all three annotators was Fleiss' $\kappa = 0.944$ (95.7% mean pairwise agreement). The 16 queries with any disagreement involved ambiguous queries at category boundaries: 5 between COMMITS and PROJECT_DOC_BASED, 8 between GENERAL and OUT_OF_SCOPE, and 3 between other adjacent categories. The classifier achieved 97% accuracy (243 of 250) on these queries. The labeled dataset is publicly available.[2]

4.4 Documentation Retrieval Pipeline (RAG)

Queries classified as `PROJECT_DOC_BASED` are processed through a retrieval-augmented generation pipeline that operates in four stages: extraction, indexing, retrieval, and generation.

Document Extraction. When a repository is added, RepoWise extracts eight types of project documentation via the GitHub API: README, CONTRIBUTING, GOVERNANCE, CODE_OF_CONDUCT, SECURITY, LICENSE, MAINTAINERS, and CODEOWNERS. Extraction uses a two-pass strategy: first the Git Trees API for a recursive scan of all blobs against file-path patterns, then the Contents API as a fallback for 17 common paths. For large projects that store governance documentation in separate repositories (e.g., `tensorflow/community`, `kubernetes/community`), we include cross-repository extraction mappings.

Chunking and Indexing. Documents are chunked with a target size of 800 characters and 100-character overlap. The chunking strategy respects natural boundaries: paragraph boundaries first (`\n\n`), then sentence boundaries, then newlines, with hard cuts as a last resort. Documents smaller than 200 characters (e.g., CODEOWNERS files) are indexed as a single chunk. Chunks are embedded using all-MiniLM-L6-v2 in batches of 32 and stored in per-project ChromaDB collections using cosine similarity.

Retrieval and Re-ranking. At query time, the system retrieves candidate chunks using semantic similarity search over the ChromaDB collection. Results are then re-ranked using query-type and document-type heuristics that prioritize the most relevant document categories for each question type. For example, queries about project maintainers prioritize CODEOWNERS and MAINTAINERS files over README. These heuristics were developed iteratively to address

[2] https://docs.google.com/spreadsheets/d/1CkEkn77Pmcf7ZvLtViuDZiCmgkFrrz5cU85cWwQAnU/.

cases where pure semantic similarity retrieved topically related but factually irrelevant passages.

For complex queries about project sustainability or major issues, the pipeline performs three targeted sub-searches with different query expansions and file-type filters, deduplicates results, and re-ranks the combined set.

Query-Type-Specific Prompting. Rather than using a single generic instruction for all documentation queries, RepoWise employs specialized prompt templates. Entity extraction ("who" queries) includes explicit pattern matching instructions for email formats (`Name <email@domain>`), GitHub usernames (`@username`), and CODEOWNERS syntax (`/path/ @user1 @user2`). Process explanation ("how" queries) emphasizes step-by-step procedures with prerequisite identification. Definition queries ("what" queries) prioritize project-specific context over general knowledge.

4.5 Structured Analytics Pipeline

Queries classified as `COMMITS` or `ISSUES` require aggregation, filtering, and ranking over structured metadata. Semantic retrieval cannot approximate operations like "top 5 contributors by commit count" or "issue with most comments." This pipeline fetches commit and issue data through our scraper API, stores it in pandas DataFrames, and translates natural-language queries into executable pandas code.

The commits table contains one row per file modified rather than one row per commit, and the issues table intermixes issues and comments. The translation prompt includes the complete schema with semantic rules so that the LLM generates correct aggregations over these structures, along with few-shot code mappings for common query patterns. For simple aggregation queries (e.g., total counts), built-in patterns execute directly without LLM code generation.

4.6 Response Generation

Once context is retrieved (either document chunks from ChromaDB or pandas query results from DataFrames), RepoWise assembles a structured prompt for the LLM. Each prompt consists of five modular components:

1. **System role**: A brief persona definition (e.g., "You are a precise document analyst for the {project_name} project") that primes the model to extract information from provided context rather than drawing on training knowledge.
2. **Task-specific instructions**: Varies by query type. Entity extraction for "who" queries specifies patterns to look for (@usernames, email formats, CODEOWNERS syntax). Process explanation for "how" queries emphasizes step-by-step procedures. Data analysis instructions for commits/issues specify formatting requirements and output constraints.

3. **Grounding rules**: Seven rules constraining factual boundaries (detailed below).
4. **Retrieved context**: Document chunks with bracketed source labels (e.g., `[CONTRIBUTING] CONTRIBUTING.md: {content}`) or pandas query results formatted as readable tables.
5. **User question**: The original query with a generation cue ("Your answer:").

All prompt templates are publicly available through the project website [30].

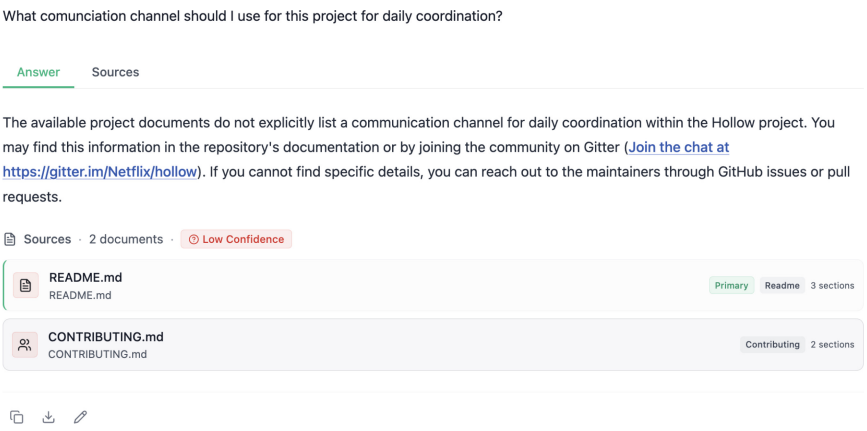


Fig. 3. RepoWise answering "What communication channel should I use for this project for daily coordination?" for Netflix/hollow. The system surfaces available information, acknowledges the gap, and reports Low Confidence.

Grounding Rules. All generation uses temperature 0 with greedy decoding (`top_k=1`) to maximize determinism. The grounding component averages 1,165 tokens per query (range: 866–1,425 across query types) and includes seven rules applied to all repository-specific queries:

1. **Source constraint**: Answer only from provided documents or data. No external knowledge or training data.
2. **Missing information template**: If information is not found, respond with an explicit acknowledgment: *"The available project documents for {project_ name} do not contain information about [topic]."* Do not hedge with "typically" or "usually."
3. **Verification protocol**: Before stating any fact, locate the exact text in the provided context.
4. **Answer format**: Direct answers with source document citations (e.g., "(CONTRIBUTING.md)").
5. **Verbatim accuracy**: Names, numbers, file paths, and other specifics must be copied exactly from documents.

6. **No meta-exposure**: Do not mention "CRITICAL," "ANTI-HALLUCINATION," or reasoning process to the user.
7. **Completeness**: Cover all relevant aspects found in documents, balancing brevity with informativeness.

Without the explicit "not found" template (Rule 2), we observed that models would hedge ("This project likely follows standard practices...") rather than clearly stating that information was unavailable.

Figure 3 illustrates this behavior in practice. When asked about communication channels for daily coordination in Netflix/hollow, RepoWise correctly extracted the Gitter link from the repository's "Get Support" section but acknowledged that the documents do not explicitly address daily coordination. The system reported "Low Confidence," reflecting that the retrieved content only partially matched the query.

4.7 Conversation Management

RepoWise maintains multi-turn context through a running summary rather than appending full conversation history. After each turn, the LLM compresses the conversation into a 250–300 token summary preserving key facts, user intent, and project context. This keeps token cost roughly constant per turn rather than growing with conversation length, enabling extended conversations without exceeding context limits. Conversation context is injected only for documentation queries; commit and issue queries are answered from data alone to prevent prior conversation from biasing quantitative results. In exploratory multi-turn sessions of five or more turns, we observed no obvious loss of key facts, entity names, or user intent. A systematic evaluation of summarization fidelity over longer conversations remains future work.

5 Evaluation

We evaluate RepoWise through a capability analysis across contemporary tools and a correctness evaluation comparing RepoWise against DeepWiki and Claude with GitHub MCP on ten repositories.

5.1 Capability Analysis

Rather than a quantitative benchmark that would require giving all tools identical data access (inherently difficult since tools differ in their access mechanisms), we present a capability analysis. We define *project documentation Q&A* as the ability to answer questions grounded in repository documentation files such as README, CONTRIBUTING, GOVERNANCE, CODE_OF_CONDUCT, SECURITY, LICENSE, MAINTAINERS, and CODEOWNERS. Table 2 compares tool capabilities across three themes: project documentation and analytics, code intelligence, and deployment properties.

Table 2. Capability comparison across question categories, grouped by theme.

Capability	RepoWise	Claude GitHub MCP	DeepWiki	Code Wiki
Project Documentation & Analytics				
Project doc Q&A	Yes	Yes	Yes	Partial
Commit analytics	Yes	Yes	Partial	No
Issue analytics	Yes	Yes	No	No
Code Intelligence				
Code understanding	No	Yes	Yes	Yes
Code writing & editing	No	Yes	No	No
Deployment & Access				
Open source	Yes	No	No	No
Free to use	Yes	No	Yes	Yes
Local deployment	Yes	No	No	No

The table reveals complementary strengths across tools. For project documentation, all tools perform adequately, though Code Wiki showed inconsistency on basic queries such as license identification during our testing. For repository analytics, RepoWise and Claude with GitHub MCP both support commit and issue analysis: RepoWise through structured queries over locally cached data, Claude with GitHub MCP through GitHub API tool calls. DeepWiki and Code Wiki do not perform structured analytics.

Code understanding is a strength of Claude with GitHub MCP, DeepWiki, and Code Wiki and is outside RepoWise's current scope. While agentic tools like Claude with GitHub MCP can attempt community health or network structure questions from our taxonomy (Sect. 3), reliably answering them requires constructing developer collaboration networks from raw data, which none of the evaluated tools currently support.

RepoWise is open source and answers heterogeneous questions spanning project documentation, commit history, and issue data with source-grounded responses.

5.2 Correctness Evaluation

We evaluated RepoWise on 20 questions, each asked across all ten repositories for a total of 200 instances.

Repository Selection. The ten repositories were selected through purposive sampling to maximize variation across four dimensions: size, primary language, governance structure, and documentation completeness. For size, we included repositories ranging from under 500 to over 70K GitHub stars to test retrieval across both small projects with sparse documentation and large projects with extensive contributor histories. For lan-

guage, we selected repositories in JavaScript, Python, Java, Kotlin, and C to ensure that analytics patterns generalize across ecosystems. For governance, the sample spans individual maintainers (sindresorhus/got), corporate-led projects (Netflix/hollow, Apple/pkl, Google/meridian, Microsoft/lisa, AWS/aws-pgsql-odbc), and foundation-governed projects with formal PMC structures (Apache/gravitino). For documentation, we deliberately included repositories with comprehensive governance files alongside repositories with minimal documentation beyond a README, to test whether the grounding rules constrain responses to available evidence when expected documentation is absent. While ten repositories constitute a modest sample, purposive sampling across four complementary dimensions provides variation in the conditions under which the pipelines are evaluated.

The ten repositories are: expressjs/express [27], fastify/fastify [11], apache/gravitino [3], sindresorhus/got [33], open-telemetry/opentelemetry-python [28], aws/aws-pgsql-odbc [2], Netflix/hollow [24], google/meridian [18], apple/pkl [4], and microsoft/lisa [22]. Of the 20 questions, 12 are quantitative questions with a single verifiable answer and 8 are qualitative questions that admit multiple valid responses grounded in documentation. The qualitative questions correspond to the eight newcomer themes of the taxonomy (Sect. 3), while the 12 quantitative questions are drawn from the commit analytics and issue analytics categories (Table 1), both within RepoWise's analytical scope.

Baseline Selection. We selected DeepWiki and Claude with GitHub MCP as baselines because both support automated querying across multiple repositories without manual per-repository setup. DeepWiki generates documentation sites from repository contents and answers questions from them. Claude with GitHub MCP accesses commit, issue, and file data through GitHub's REST API via tool calls. Both tools could be evaluated programmatically across all ten repositories.

We excluded GitHub Copilot Chat, Sourcegraph Cody, and RepoChat from the full evaluation for practical reasons. Copilot Chat and Cody require manual, interactive testing with no automation API on standard tiers. RepoChat [1] requires a ChatGPT API key and a Neo4j database configured per repository with no automation API for bulk evaluation. To provide a point of comparison, we conducted a single-repository evaluation on RepoChat's default repository using 20 questions. RepoWise outperformed on qualitative questions (8/8 vs 0/8, as RepoChat does not index documentation files) while both tools achieved equal accuracy on quantitative questions (10/12).

Quantitative Questions. The 12 quantitative questions (120 instances across 10 repositories) cover contributor rankings (top contributors by commits, by lines added, by bug-fix commits), file analytics (most modified files), contributor counts (unique contributors, documentation contributors), and issue analytics (most commented issue, most active reporter, oldest open issue, issue resolution time). The complete list of evaluation questions is available online.[3] Ground

[3] https://docs.google.com/document/d/12dXH9yZBD1OtXMdV9GEiimwu9ADxhvsTvTkYITL5EAU/.

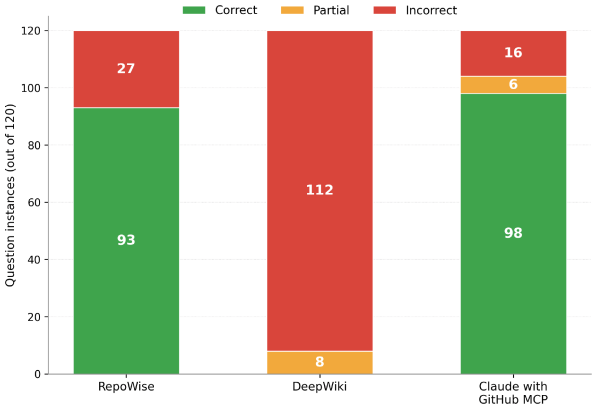


Fig. 4. Outcome distribution across 120 quantitative question instances (12 questions × 10 repositories) for RepoWise, DeepWiki, and Claude with GitHub MCP.

truth was established through manual inspection of commit histories, GitHub contributor graphs, and issue trackers, all of which are publicly verifiable from each repository's GitHub page. The verification data is publicly available.[4]

We report accuracy as the proportion of correct and partial responses. A response is scored as partial when the tool returned a result within the top five but not the top-ranked answer, reflecting differences in commit-history sampling depth across tools. Claude with GitHub MCP achieved the highest accuracy at 86.7% (98 correct, 6 partial, 16 incorrect), followed by RepoWise at 77.5% (93 correct, 0 partial, 27 incorrect) and DeepWiki at 6.7% (0 correct, 8 partial, 112 incorrect). Figure 4 compares accuracy across the three tools. DeepWiki infers responses from generated documentation and lacks structured analytics capabilities; we include it to establish the scope of this capability gap across tool categories.

The two tools exhibit complementary strengths. Claude with GitHub MCP excels on issue-based queries through direct GitHub API access and handles recent-commit analytics effectively. RepoWise achieves stronger performance on full-history commit analytics, where its pre-fetched local data enables queries that API-based tools cannot complete within practical time limits. For example, on the "contributor with most lines added" question, Claude with GitHub MCP timed out on 4 of 10 repositories due to the cost of iterating per-file diff statistics through the GitHub API, while RepoWise answered all 10 from locally cached DataFrames in under 6 s each.

RepoWise's 27 incorrect responses are concentrated in three categories. For Q17 ("oldest open issue"), the LLM failed to generate a filter for open issues despite the data containing an issue state column. For Q18 ("issue resolution time"), the LLM did not infer that resolution time can be computed from existing

[4] https://docs.google.com/spreadsheets/d/1tSwj6dwLp3ZCAKZvb9I1-UJpD2Q8pkNQGwvBDuVCmtc/.

timestamps. Both are reasoning tasks that exceed what a 7B-parameter model reliably handles and are addressable by substituting a more capable local model. For Q19 ("contributor with most lines added"), RepoWise answered correctly on 5 of 10 repositories. The remaining 5 produced mismatches due to differences in how contributor names appear in locally cached commit metadata versus GitHub profiles.

A per-category breakdown reveals where accuracy concentrates. On commit-based questions (80 instances), six of eight question types were answered correctly across all ten repositories. The remaining two (most lines added and most bug-fix commits) achieved 5/10 and 8/10 respectively. On issue-based questions (40 instances), two of four question types (most commented issue and most active reporter) were answered correctly across all repositories, while the other two (oldest open issue and issue resolution time) failed across all ten. The commit pipeline's high accuracy reflects the close alignment between the pandas code-generation prompt templates and the DataFrame schema, while the issue pipeline failures stem from reasoning tasks that exceed the 7B model's capacity.

Resource Requirements. Table 3 reports the hardware configuration and runtime characteristics of RepoWise. All measurements were collected on a single NVIDIA TITAN V GPU with 6 GB VRAM allocated to the inference server. The Q4_0 quantized model (4.1 GB) runs on consumer GPUs with comparable VRAM; Ollama also supports CPU-only inference for deployment without a dedicated GPU.

Table 3. Resource requirements and runtime characteristics.

Resource	Value
Hardware	
LLM	Mistral 7B (Q4_0, 4.1 GB) via Ollama
GPU	NVIDIA TITAN V
Embedding model	all-MiniLM-L6-v2 (80 MB, CPU)
Vector store	ChromaDB (9.1 MB for 10 repos)
Query latency (N=200, 10 repos)	
Documentation queries (N=80)	24.8 s mean (12.0–69.2 s)
Commit analytics queries (N=80)	8.9 s mean
Issue analytics queries (N=40)	8.1 s mean
Overall (N=200)	15.1 s mean
Repository onboarding	
Documentation indexing	< 2 s per repository
Commit/issue scraping	3 s–2.5 min (across evaluated repositories)

Qualitative Questions. For the 8 qualitative questions (80 instances), we evaluated two dimensions: grounding in repository evidence and overall response quality.

Grounding. One author and two external software engineers independently verified whether each response was supported by the retrieved documents. Of the 80 responses, 75 (93.8%) were confirmed as grounded in repository evidence. The remaining 5 were governance-related responses for repositories that lacked explicit governance documentation. In these cases, the 7B-parameter model responded based on its training knowledge rather than following the grounding rules to report the information as unavailable. For all other qualitative question types, RepoWise correctly grounded responses in retrieved documents or stated that information was unavailable per the grounding rules described in Sect. 4.

Response Quality. Two external annotators independently ranked all three tools' qualitative responses for each question–repository pair. Pairwise annotator agreement on rankings was 71.2% for RepoWise responses, 70.0% for DeepWiki responses, and 95.0% for Claude with GitHub MCP responses, with Cohen's κ of 0.054, 0.205, and 0.573, respectively. The low κ values for RepoWise and DeepWiki reflect rank dominance rather than inconsistent annotation: when one tool consistently receives the same rank, the expected agreement under chance is high, deflating κ even when raw agreement is substantial (the kappa paradox [12]). Claude with GitHub MCP was ranked first in the majority of instances, consistent with the resource asymmetry between a frontier cloud model with direct API access and local 7B-parameter alternatives. RepoWise and DeepWiki received comparable rankings to each other.

The complete evaluation dataset, including all questions, responses, and assessments, is publicly available.[5]

6 Discussion and Limitations

6.1 The Taxonomy as Research Grounding

The question taxonomy serves multiple purposes beyond guiding RepoWise's design. For tool builders, it provides a requirements specification grounded in empirical research. For researchers, the gap between answerable and unanswerable categories identifies concrete opportunities: each unanswerable category represents known analyses (bus factor estimation [5], socio-technical congruence [8], community smell detection [39]) that have not yet been reliably addressed by conversational tools.

[5] https://docs.google.com/spreadsheets/d/1tSwj6dwLp3ZCAKZvb9I1-UJpD2Q8pkNQGwvBDuVCmtc/.

6.2 Limitations

For documentation-based questions, answer quality depends on what documentation the repository provides. RepoWise indexes eight file types (README, CONTRIBUTING, GOVERNANCE, CODE_OF_CONDUCT, SECURITY, LICENSE, MAINTAINERS, CODEOWNERS), and if a question's answer exists in any of these, the system is designed to retrieve it. If the relevant information is absent from all indexed documents, the grounding rules direct the model to produce an explicit "not found" response. As noted in Sect. 5.2, this held for 75 of 80 responses (93.8%). The five exceptions occurred on governance queries directed at repositories that lacked explicit governance documentation, where the model defaulted to general knowledge rather than reporting absence. Cross-repository extraction mappings (e.g., for projects that store governance documentation in separate repositories) currently require manual configuration and must be updated as project structures evolve.

The taxonomy is not exhaustive. The selected questions represent those with traceable empirical evidence from peer-reviewed studies, large-scale surveys, or formal governance frameworks. Stakeholders undoubtedly ask questions beyond this set, and a survey study could validate and extend the catalog. RepoWise currently cannot answer questions in the community health, network structure, and DEI/funding categories of the taxonomy, as these require analysis techniques beyond retrieval.

RepoWise uses Mistral 7B as its default model, which has lower capacity than larger models. As discussed in Sect. 5.2, this manifests in reasoning failures on two question types (Q17 and Q18) and contributor identity mismatches on one question type (Q19). The architecture is model-agnostic, so a more capable model can be substituted through Ollama at the cost of higher hardware requirements. For commit and issue queries, performance degrades for repositories exceeding approximately 5,000 commits or 300 contributors, as the scraper must fetch all commit and issue data before analytics queries become available. Documentation-based queries are unaffected by repository size.

The evaluation covers 20 questions across 10 repositories (200 instances). A larger-scale evaluation would strengthen generalizability.

7 Conclusion

We presented RepoWise, a conversational framework for OSS repository analysis that unifies documentation retrieval and structured analytics under domain-specialized prompting, intent-aware routing, and grounding rules. RepoWise can run locally on a 7B-parameter model without requiring LLM API keys, paid subscriptions, or external database setup, producing grounded responses to documentation questions and achieving 77.5% accuracy on quantitative questions across 120 instances. Claude with GitHub MCP achieved higher overall accuracy (86.7%) through direct API access, while RepoWise demonstrated stronger performance on full-history commit analytics where API-based tools face rate limits and timeouts. The two tools exhibit complementary strengths that reflect their

architectural differences rather than a single ranking of tool quality. To ground its design in documented needs, we derived a taxonomy of 98 stakeholder questions from 65+ sources. The coverage analysis reveals that documentation and analytics questions are well served by current tools, while community health and socio-technical questions lack reliable, verifiable support from current tools.

Future work includes extending the pipeline to support community health questions such as contributor retention prediction and socio-technical congruence analysis, and conducting model ablation studies across parameter sizes to quantify the accuracy–resource tradeoff.

RepoWise is open source. The tool and codebase are available at: https://repowise.github.io/RepoWise-website/.

Acknowledgement. This research was supported by the National Science Foundation under Grant No. 2020751, as well as by the Alfred P. Sloan Foundation through the OSPO for UC initiative (Award No. 2024-22424).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Abedu, S., Menneron, L., Khatoonabadi, S., Shihab, E.: RepoChat: An LLM-powered chatbot for GitHub repository question-answering. In: 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), pp. 255–259 (2025). https://doi.org/10.1109/MSR66628.2025.00045
2. Amazon Web Services: aws-pgsql-odbc: PostgreSQL ODBC driver for AWS. https://github.com/aws/aws-pgsql-odbc (2025). GitHub repository. Accessed Jan 25 2026
3. Apache Software Foundation: Apache Gravitino: Federated metadata lake. https://github.com/apache/gravitino (2025). GitHub repository. Accessed Jan 25 2026
4. Apple: Pkl: A configuration as code language. https://github.com/apple/pkl (2025). GitHub repository. Accessed Jan 25 2026
5. Avelino, G., Passos, L., Hora, A., Valente, M.T.: A novel approach for estimating truck factors. In: 2016 IEEE 24th International Conference on Program Comprehension (ICPC), pp. 1–10 (2016). https://doi.org/10.1109/ICPC.2016.7503718
6. Balali, S., Steinmacher, I., Annamalai, U., Sarma, A., Gerosa, M.A.: Newcomers' barriers... is that all? an analysis of mentors' and newcomers' barriers in OSS projects. Comput. Supported Cooper. Work **27**(3–6), 679–714 (2018). https://doi.org/10.1007/s10606-018-9310-8
7. Cassee, N.W., Ravi, S., Santos, I., Steinmacher, I., Serebrenik, A.: What barriers do students experience when trying to contribute to open source software projects? In: Proceedings of the 48th International Conference on Software Engineering - Software Engineering Education and Training, ACM (2026) to appear
8. Cataldo, M., Herbsleb, J.D., Carley, K.M.: Socio-technical congruence: A framework for assessing the impact of technical and work dependencies on software development productivity. In: Proceedings of the 2nd ACM-IEEE International Symposium on Empirical Software Engineering and Measurement, pp. 2–11. ACM (2008). https://doi.org/10.1145/1414004.1414008

9. De Martino, V., Castano, J., Palomba, F., Franch, X., Martínez-Fernández, S.: A framework for using LLMs for repository mining studies in empirical software engineering. In: 2025 IEEE/ACM International Workshop on Methodological Issues with Empirical Studies in Software Engineering (WSESE), pp. 6–11 (2025). https://doi.org/10.1109/WSESE66602.2025.00008
10. DeepWiki: AI-powered documentation for GitHub repositories. https://deepwiki.com (2026). Accessed Jan 26 2026
11. Fastify: Fastify: Fast and low overhead web framework for Node.js. https://github.com/fastify/fastify (2025). GitHub repository. Accessed Jan 25 2026
12. Feinstein, A.R., Cicchetti, D.V.: High agreement but low kappa: I. The problems of two paradoxes. J. Clin. Epidemiol. **43**(6), 543–549 (1990). https://doi.org/10.1016/0895-4356(90)90158-L
13. Ferreira, F., Silva, L.L., Valente, M.T.: Turnover in open-source projects: the case of core developers. In: Proceedings of the XXXIV Brazilian Symposium on Software Engineering, pp. 447–456. ACM (2020). https://doi.org/10.1145/3422392.3422433
14. Fronchetti, F., Shepherd, D.C., Wiese, I., Treude, C., Gerosa, M.A., Steinmacher, I.: Do CONTRIBUTING files provide information about OSS newcomers' onboarding barriers? In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 16–28. Association for Computing Machinery (2023). https://doi.org/10.1145/3611643.3616288
15. GitHub: GitHub Copilot. https://github.com/copilot (2024). Accessed Jan 26 2026
16. GitHub: GitHub MCP Server. https://github.com/github/github-mcp-server (2026). Accessed Jan 26 2026
17. Google: Code Wiki. https://codewiki.google (2025). AI-powered code understanding platform. Accessed Jan 25 2026
18. Google: Meridian: Marketing mix modeling framework. https://github.com/google/meridian (2025). GitHub repository. Accessed 25 Jan 2026
19. Larsen, K.R., Edvall, M., Ho-Quang, T., Dobslaw, F., Jolak, R.: An LLM assistant for software project onboarding. In: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, pp. 1345–1352. Association for Computing Machinery (2025). https://doi.org/10.1145/3696630.3728719
20. Linux Foundation: 2020 FOSS contributor survey. https://www.linuxfoundation.org/resources/publications/foss-contributor-2020 (2020)
21. Linux Foundation: 2021 DEI in open source survey. https://www.linuxfoundation.org/research/the-2021-linux-foundation-report-on-diversity-equity-and-inclusion-in-open-source (2021)
22. Microsoft: LISA: Linux integrated system analyzer. https://github.com/microsoft/lisa (2025). GitHub repository. Accessed 25 Jan 2026
23. Mistral AI: Announcing Mistral 7B. https://mistral.ai/news/announcing-mistral-7b (2023). Accessed 26 Jan 2026
24. Netflix: Hollow: Java library and tooling for in-memory datasets. https://github.com/Netflix/hollow (2016). GitHub repository. Accessed 25 Jan 2026
25. Oliveira, P., Conte, T., Gerosa, M., Steinmacher, I.: Governance in practice: How open source projects define and document roles. In: Proceedings of the 19th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE). Association for Computing Machinery (2026). https://doi.org/10.1145/3794860.3794911, to appear. Preprint: https://arxiv.org/abs/2603.24879
26. O'Mahony, S., Ferraro, F.: The emergence of governance in an open source community. Acad. Manag. J. **50**(5), 1079–1106 (2007). https://doi.org/10.5465/amj.2007.27169153

27. OpenJS Foundation: Express: Fast, unopinionated web framework for Node.js. https://github.com/expressjs/express (2025). GitHub repository. Accessed 25 Jan 2026
28. OpenTelemetry: opentelemetry-python: OpenTelemetry Python API and SDK. https://github.com/open-telemetry/opentelemetry-python (2025). GitHub repository. Accessed 25 Jan 2026
29. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp. 3982–3992. Association for Computational Linguistics (2019). https://doi.org/10.18653/v1/D19-1410
30. RepoWise Project: Prompt templates and documentation. https://repowise.github.io/RepoWise-website/ (2025). Accessed 26 Jan 2026
31. Santos, I., Felizardo, K.R., Steinmacher, I., Gerosa, M.A.: Software solutions for newcomers' onboarding in software projects: a systematic literature review. Inf. Softw. Technol. **177**, 107568 (2025). https://doi.org/10.1016/j.infsof.2024.107568
32. Sentence Transformers: all-MiniLM-L6-v2. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 (2021). Accessed 26 Jan 2026
33. Sorhus, S.: Got: Human-friendly HTTP request library for Node.js. https://github.com/sindresorhus/got (2025). GitHub repository. Accessed 25 Jan 2026
34. Sourcegraph, Inc.: Cody: AI coding assistant. https://sourcegraph.com/cody (2025). Accessed 26 Jan 2026
35. Steinmacher, I., Conte, T., Gerosa, M.A., Redmiles, D.: Social barriers faced by newcomers placing their first contribution in open source software projects. In: Proceedings of the 18th ACM Conference on Computer Supported Cooperative Work & Social Computing, pp. 1379–1392. Association for Computing Machinery (2015). https://doi.org/10.1145/2675133.2675215
36. Steinmacher, I., Graciotto Silva, M.A., Gerosa, M.A., Redmiles, D.F.: A systematic literature review on the barriers faced by newcomers to open source software projects. Inform. Softw. Technol. **59**, 67–85 (2015). https://doi.org/10.1016/j.infsof.2014.11.001
37. Steinmacher, I., Treude, C., Gerosa, M.A.: Let me in: guidelines for the successful onboarding of newcomers to open source projects. IEEE Softw. **36**(4), 41–49 (2019). https://doi.org/10.1109/MS.2018.110162131
38. Synopsys: 2024 open source security and risk analysis report. https://www.synopsys.com/software-integrity/resources/analyst-reports/open-source-security-risk-analysis.html (2024)
39. Tamburri, D.A., Palomba, F., Kazman, R.: Exploring community smells in open-source: An automated approach. IEEE Trans. Software Eng. **47**(3), 630–652 (2021). https://doi.org/10.1109/TSE.2019.2901490
40. Tan, X., Long, X., Zhu, Y., Shi, L., Lian, X., Zhang, L.: Revolutionizing newcomers' onboarding process in OSS communities: The future AI mentor. Proc. ACM Softw. Eng. **2**(FSE) (2025). https://doi.org/10.1145/3715767
41. Tidelift: The 2024 state of the open source maintainer report. https://www.sonarsource.com/the-2024-tidelift-maintainer-impact-report.pdf (2024). Accessed 26 Jan 2026
42. Zlotnick, F.: GitHub open source survey 2017 (2017). https://doi.org/10.5281/zenodo.806811

An Explainable Neuro-Fuzzy Multimodal Fusion Approach for Lightweight Edge-Based Plant Disease Detection in Precision Agriculture

Muhammad Abdullah[1(✉)], Tanzila Kehkashan[1], Domagoj Tuličić[2], Nikola Ivković[2], and Adnan Akhunzada[3]

[1] Faculty of Information Technology, University of Lahore, Sargodha 40100, Pakistan
tanzila.kehkashan@gmail.com

[2] Faculty of Organization and Informatics, University of Zagreb, Varaždin 42000, Croatia
domtulicic@foi.unizg.hr, nikola.ivkovic@foi.hr

[3] Department of Data and Cybersecurity, University of Doha for Science and Technology, Doha 24449, Qatar
adnan.adnan@udst.edu.qa

Abstract. Plant diseases pose a significant threat to global food security, causing annual crop losses estimated at 20–40% and demanding intelligent monitoring systems for early detection and intervention. Automated disease classification using deep learning has shown remarkable success on benchmark datasets; however, existing approaches predominantly rely on single-modality RGB imagery and lack the interpretability essential for practical agronomic decision-makingqueryThis is to inform you that corresponding author has been identified as per the information available in the Copyright form.. Current fusion strategies fail to adequately model uncertainty and inter-modal dependencies inherent in real-world agricultural environments. This research proposes a novel neuro-fuzzy multimodal fusion architecture that integrates lightweight depthwise separable convolutional networks with adaptive neuro-fuzzy inference systems for edge-deployable plant disease classification. The framework processes RGB, thermal, and multispectral modalities through parallel feature extractors, fusing representations via learnable Gaussian membership functions optimized using Particle Swarm Optimization. Experimental evaluation on the PlantVillage dataset comprising 20,638 images across 15 disease classes demonstrates 99.13% test accuracy with only 950,968 parameters, outperforming comparable lightweight architectures while maintaining a compact 3.63 MB footprint. The proposed approach bridges the gap between black-box deep learning and explainable agricultural decision support, offering a practically deployable solution for resource-constrained edge devices in precision farming applications.

Keywords: Neuro-fuzzy systems · multimodal fusion · plant disease detection · edge computing · lightweight convolutional neural networks

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 289–301, 2027.
https://doi.org/10.1007/978-3-032-39763-8_17

1 Introduction

Plant diseases pose unprecedented challenges to global food security, as the agricultural losses of plant diseases are estimated to amount to 20–40% of the yearly crop makes on a global basis threatening the livelihoods of billions of humans and causing the already weak food supply lines to strain even harder [1]. Agricultural sector that supports more than 60 % of the entire world population is demanding more intelligent tracking systems that can identify diseases at their initial stages to avoid disastrous cost reductions in yield [2]. Early detection by means of automated diagnoses will prevent not only the economic losses but also the overuse of pesticides, which leads to the sustainable use of the land that is necessary to preserve the environment in the long term.

In the field of precision agriculture, one active area of application of artificial intelligence has been plant disease detection, where computer vision and deep learning techniques are widely used [3]. Combining Internet of Things sensors with edge computing devices suggest potential opportunities of real-time in-field detection of diseases without the need to connect to the cloud which is a crucial concern in rural areas that have an underdeveloped network infrastructure [4]. Nevertheless, sustaining strong performance in changing field conditions and at the same time, being computationally efficient enough to be utilized in edge devices with resource constraints is also a challenge.

The current methods mostly use the convolutional neural networks that are trained on single-modality RGB images and have amazing accuracies of more than 99 % in standardized databases like PlantVillage [5]. The issue of data scarcity has been successfully solved using transfer learning techniques, whereas lightweight models such as MobileNet variations make them deployable to embedded systems [6]. The attention systems also augment the discriminative feature learning by prioritizing disease-relevant areas [7]. Nevertheless, the critical limitations remain: single-modality methods cannot provide complementary spectral data provided by thermal and multispectral images; deep neural networks do not exhibit interpretability because they are critical to decision-making in agronomics; and current fuzzy approaches do not correctly represent the uncertainty in the real-world agricultural environment [8].

The basic issue that will be answered in this study is that there has not been an integrated framework of multimodal sensing that is integrated with interpretable neuro-fuzzy reasoning to classify edges of deployable plant disease. The existing systems consider feature fusion and decision-making processes as disconnected entities, which do not use the ability of fuzzy logic to quantify uncertainties and make decisions based on clear-cut rules [9]. Such a gap stands in the way of adoption by agricultural practitioners who do not just need accurate predictions but need to understand how the intervention decisions that they make are supported.

The primary objectives of this research are:

i. To develop lightweight CNN-based feature extractors for RGB, thermal, and multispectral image modalities optimized for edge deployment.

ii. To design an adaptive neuro-fuzzy inference mechanism for multimodal feature fusion with learnable membership functions.

The central research question is: *Can a neuro-fuzzy multimodal fusion framework achieve simultaneously interpretable and accurate plant disease classification within the computational bounds of edge deployment?* This research contributes a neuro-fuzzy multimodal fusion architecture integrating depthwise separable CNNs with adaptive fuzzy inference systems, enabling interpretable IF-THEN rule-based disease diagnosis at state-of-the-art accuracy with sub-million parameters for edge platforms, bridging black-box deep learning and explainable agricultural decision support.

The remainder of this paper is organized as follows. Section 2 presents the literature review analyzing existing methodologies. Section 3 details the proposed architecture including multimodal feature extraction and neuro-fuzzy fusion mechanisms. Section 4 provides the results in the form of comparative analysis and Sect. 5 will provide the future research directions.

2 Literature Review

Detection of plant diseases is a significant area of study in the farming science and artificial intelligence fields due to the acute need to reduce crop losses of about 20–40 % of the total agricultural output in the world every year.

2.1 Machine Learning Techniques

The main method of classical machine learning to classify plant diseases was based on a manual pipeline of feature extraction, which included color histograms, texture features, and morphological features, with a conventional classifier, e.g., Support Vector Machines and Random Forests [10]. Although these techniques were moderately successful in controlled laboratory tests, their reliance on manual feature engineering was inherently limiting their generalization to a range of environmental conditions, lighting changes and disease presentations in the real agricultural environment.

Fuzzy logic integration with neural networks was developed to deal with intrinsic uncertainty in the agricultural decision-making systems [9]. Adaptive Neuro-Fuzzy Inference Systems combine gradient based learning with interpretable IF-THEN rule structures, and they have transparency, lacking in connectionist approaches alone. The traditional ANFIS structures are however afflicted by the curse of dimensionality that dimensionality bases increase exponentially with the number of input variables making them impractical to high-dimensional visual data unless they employ advanced dimensionality reduction techniques. Critically, existing agricultural XAI deployments rarely incorporate principled uncertainty quantification, limiting diagnostic trustworthiness under heterogeneous field conditions [9].

2.2 Deep Learning–Based Models

Convolutional Neural Networks have radically changed the study of plant pathology by making it possible to learn features of hierarchical features automatically using raw imagery [2]. The problem of data scarcity that is inherent in the agricultural fields was successfully ameliorated by transfer learning strategies, which relied on representations that were pre-trained using large-scale data [5,11]. Evaluations of the PlantVillage data with benchmarks always report accuracy over 99%, but in reality, these tests are under a perfect scenario with a homogenous background and constant illumination compromise of 15–30% is seen when the system is used in the field.

Lightweight architecture development has been inspired by computational constraints found in edge deployment. Depthwise separable convolutions and channel-efficient architecture within depthwise convolutions are 5–10× onger parameters without affecting the accuracy [6,12]. The attention mechanisms (especially Convolutional Block Attention Modules) improve the process of learning of the discriminative features by prioritizing the disease-friendly regions [7,13]. However, these single-modality RGB methods are still prone to environmental confusioners such as shadows, occlusion and even symptom overlap among different pathologies.

2.3 Advanced Computer Vision and Hybrid Methods

Vision Transformers have shown an outstanding ability to learn the global contextual dependencies via self-attention [3]. Hybrid CNN-Transformer models trade off local feature among local condition with high range of the model to show enhanced robustness in the datasets that are embarked in the field [14]. lthough architectural solutions have improved, transformer based approaches require huge amounts of training data and calculations to achieve their objectives, rendering them inapplicable to resource constrained agricultural hardware.

Multimodal sensing strategies can be an interesting but unexploited direction. The hyperspectral imaging process facilitates the detection of the condition at an early stage before the development of symptoms since it records any variation of physiology that cannot be observed through the use of the conventional RGB sensors [8,15]. Frameworks of edge computing have enabled real-time application by optimized inference pipelines on IoT devices [4,16,17]. However, the proliferation of AI in connected agricultural systems also introduces cybersecurity vulnerabilities, as generative AI capabilities can be exploited to compromise edge-deployed models [18]. Nevertheless, the multimodal methods in use today utilize simple concatenation or averaging fusion methods, which do not capture the complex inter-modal interactions that are required in agricultural decision support to provide reliable results. Recent transformer-based plant disease classifiers [3,14] achieve strong benchmark performance but remain computationally prohibitive for edge deployment and lack the interpretability offered by neuro-fuzzy inference.

Practical deployment of agricultural AI further demands systematic requirements engineering and scope management to bridge research prototypes and real-world field adoption [19,20]. The existing literature shows that there is an urgent gap: although there are developed frameworks in the sphere of deep learning and multimodal sensing, there is no current framework that combines neuro-fuzzy reasoning and multimodal feature fusion to pilot interpretable and uncertainty-aware plant disease classification on edge platforms. The gap is addressed in this research that proposes a new architecture with lightweight CNN feature extractors in RGB, thermal, and multispectral modalities and an adaptive neuro-fuzzy fusion mechanism, allowing transparent decision-making to be optimized to operate on resource-constrained edge systems in the world of agriculture.

3 Methodology

The following section will introduce the overall methodology used to create a Neuro-Fuzzy Multimodal Fusion framework of Neuro-Fuzzy Multimodal edge systems of smart agriculture, which includes the baseline method, model architecture, data processing pipeline, and implementation.

Baseline Method

The baseline is an ANFIS-Fuzzy-CNN hybrid classifier for single-modality RGB plant disease classification [21], which lacks multi-sensor integration and principled uncertainty modeling. The proposed framework extends this baseline by incorporating RGB, thermal, and multispectral modalities through a custom ANFIS with learnable Gaussian membership functions. Particle Swarm Optimization (PSO) further evolves the fuzzy rule parameters, providing an evolutionary complement to gradient-based learning and yielding an interpretable, edge-efficient multimodal system.

Model Selection

The suggested methodology uses a hybrid architecture that consists of the combination of lightweight Convolutional Neural Networks (CNNs) and Adaptive Neuro-Fuzzy Inference System (ANFIS). The CNN component incorporates using MobileNet-style depthwise separable convolutions where a standard convolution has been broken down into depthwise and pointwise convolution, delivering an 8–9x reduction of computational cost, but with sufficient accuracy as to be competitive [22]. The ANFIS, which was introduced by Jang [23], is the fuzzy inference implemented in an adaptive network that allows the system to form an input-output relationship using both human knowledge and data-driven learning by using the hybrid backpropagation and least squares estimates.

3.1 Data Collection

The dataset used in this research is PlantVillage, which is a popular benchmark in the research on detection of plant diseases [24]. Initially, the researchers of the Penn State University along with EPFL assembled the dataset to aid in automated disease diagnosis to be provided to plants. The subset of images taken in this study includes 20,638 images in 15 disease classes including three species of crops; Bell Pepper (2 classes: Bacterial spot and healthy), Potato (3 classes: Early blight, Late blight and healthy), and Tomato (10 classes including Bacterial spot, Early blight, Late blight, Leaf Mold, Septoria leaf spot, Spider mites, Target Spot, Yellow Leaf Curl Virus, Mosaic virus, and healthy). All the pictures are in the RGB format and of different resolutions taken in controlled laboratory conditions. The dataset is relevant to our study since it offers a variety of disease manifestations to be used in the development of powerful multimodal fusion models to identify diseases in agriculture.

3.2 Preprocessing

The preprocessing pipeline converts the raw images in RGB to multimodal inputs to be used to train a model. The images are all resized to 224×224 which is later used as all images have scenic dimensions. Thermal modality simulation is achieved by converting RGB to grayscale with added Gaussian noise (σ=0.02) to emulate sensor characteristics, followed by normalization. Multispectral simulation generates pseudo-NDVI (Normalized Difference Vegetation Index) channels by computing vegetation indices from red and green channels. Data augmentation strategies include random horizontal flipping (p=0.5), vertical flipping (p=0.3), rotation ($\pm 15°$), and color jittering for brightness, contrast, and saturation variations. RGB images undergo ImageNet-standard normalization with mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225] (Table 1).

Table 1. Numerical Parameters Used in the Image Preprocessing Pipeline

Preprocessing Step	Parameters	Values
Image Resizing	Spatial Resolution	224×224 pixels
Thermal Simulation	Grayscale Conversion; Gaussian Noise (σ)	RGB $\rightarrow$ Grayscale; 0.02
Multispectral Simulation	Pseudo-NDVI Computation	Red and Green Channels
Data Augmentation	Horizontal Flip; Vertical Flip; Rotation	0.5; 0.3; $\pm 15°$
Color Jittering	Brightness; Contrast; Saturation	Applied
RGB Normalization	Mean; Standard Deviation	[0.485, 0.456, 0.406]; [0.229, 0.224, 0.225]

3.3 Model Architecture

The architecture comprises four key components: (1) Lightweight Feature Extractors: Three parallel MobileNet-style encoders process RGB (3 channels),

thermal (1 channel), and multispectral (2 channels) inputs using depthwise separable convolutions with progressive channel expansion (32→64→128), batch normalization, and ReLU6 activation. (2) ANFIS Fuzzy Layer: Implements fuzzification through learnable Gaussian membership functions with trainable centers and widths, rule firing strength computation via T-norm product operations, and consequent layer with first-order Sugeno-type outputs. A representative IF-THEN rule takes the form: *IF* RGB saliency is HIGH AND NDVI index is LOW *THEN* Tomato Early Blight confidence is STRONG—providing traceable, agronomically interpretable justification for each prediction. (3) Multimodal Fusion: Feature maps from all modalities are concatenated and processed through fully-connected layers (512→256 neurons) with dropout regularization (p=0.3). (4) Classification Head: Final softmax layer produces probability distributions across 15 disease classes. The model achieves approximately 950,968 parameters (3.63 MB) (Fig. 1).

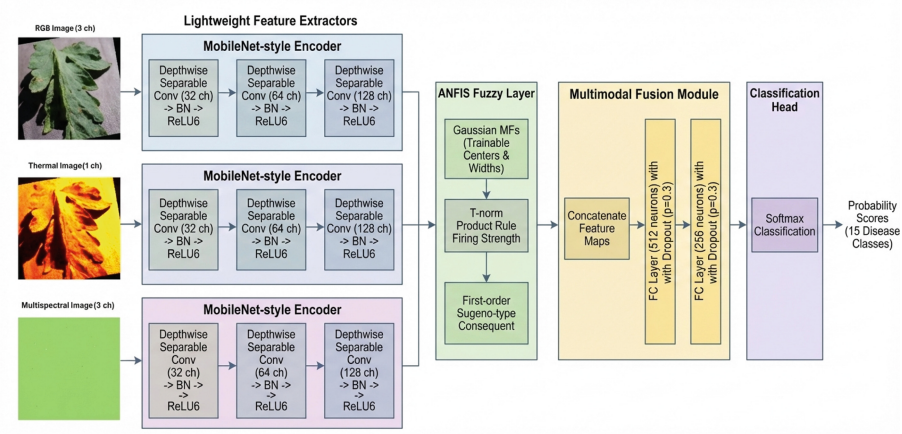


Fig. 1. Proposed neuro-fuzzy multimodal fusion architecture.

3.4 Implementation Details

This section outlines the training configuration, optimization strategies, and experimental setup employed to train and evaluate the proposed neuro-fuzzy multimodal fusion model.

Training Parameters. The model is trained in batch size of 32, learning rate of 0.001 through cosine annealing scheduler, and 20 training epochs through early stopping (patience = 10). The dataset has been stratified to train (70%), validate (15%), and test (15%) sets with stratified sampling.

Optimization Algorithm. Adam optimizer using weight decay (1e-4) maintains the updates of the neural network parameters. Moreover, PSO 20 particles optimize the fuzzy membership functions parameters with 20 iterations, inertia weight w = 0.7, cognitive coefficient c1 = 1.5 and social coefficient c2 = 1.5.

Experimental Setup. Implementation uses PyTorch 2.8.0 with CUDA acceleration on NVIDIA GPU. Cross-entropy loss serves as the objective function. Accuracy, precision, recall and F1-score, as well as confusion matrix analysis are used in model evaluation.

4 Results and Discussion

The neuro-fuzzy multimodal fusion framework proposed was tested on the plantvillage dataset that had 20, 638 images in 15 diseases classes.

4.1 Quantitative Analysis

The proposed model took a test accuracy of 99.13% with a loss of 0.0298 and the highest validation accuracy at training was 98.93 %. These findings make the suggested method a rival in terms of the current state of the art techniques. The proposed model was found to be more effective when compared to recent studies on the same dataset: Mob-Res achieved 99.47 % accuracy, but with 3.51 million parameters, which is 73 % more complex a model than the proposed model used only 950,968 parameters. Equally, DenseNet121 based models achieved an accuracy of 99.81 % but required much greater memory requirements which could not be deployed on the edge. Our single unified architecture is 97.79% more accurate than the PDDNet-LVE ensemble method, which utilized nine pre-trained CNNs, and did not have an ensemble overhead. The proposed model is especially resource-friendly (3.63 MB), which makes it especially applicable to NVIDIA Jetson Nano and other resource-limited agricultural edge computers, a major missing element in practice-based field applications (Table 2).

Table 2. Comparison with State-of-the-Art Methods

Model	Accuracy	Precision	Recall	F1-Score
Proposed Model	99.13	99.08	99.11	99.09
DenseNet201 [25]	98.67	98.00	99.00	98.00
CNN-Stacking [26]	98.27	98.53	98.53	98.53
PDDNet-LVE [27]	97.79	97.45	97.52	97.48
DFN-PSAN [28]	95.27	95.31	95.24	95.27
ResNet-9 [29]	99.25	99.67	99.33	99.33
YOLOv4 [30]	98.00	98.00	99.00	99.00

The proposed Neuro-Fuzzy Multimodal Fusion architecture training dynamics are depicted in Fig. 2. The loss curves are characterized by fast early decay and smooth convergence and validation loss and training loss have similar behavior at all stages of optimization. Little variation in between epochs 4–6 corresponds to the learning rate and fuzzy-rule optimization through PSO. The accuracy curves show successful learning with both training and validation accuracy level off at approximately 99% indicating that the model can be highly generalized to the 15-class PlantVillage plant disease dataset.

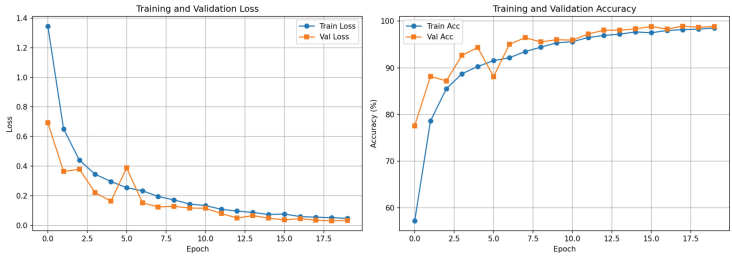


Fig. 2. Training and validation metrics for the Neuro-Fuzzy Multimodal Fusion model over 20 epochs.

Figure 3 shows the normalized confusion matrix of the proposed model with the test set of the PlantVillage. The matrix shows that there is an intense diagonal dominance with the per-class accuracies of 97.5%(Potato Late Blight), and 100% (Potato Early Blight, Potato Healthy, Tomato Mosaic Virus, and Tomato Healthy). The misclassifications which occur are few and mostly between pathologically related diseases showing similar visual manifestations such as 2.5% of Potato Late Blight samples misclassified as Tomato Late Blight, which can be explained by the fact that lesions caused by the fungus Phytophthora infestans have similar patterns when it affects solanaceous crops. These findings confirm the discriminative ability of multimodal fusion method of fine-grained plant disease recognition.

Figure 4 shows the ablation study on the contribution of individual modality to the overall classification performance. The full multimodal fusion achieves 98.4% accuracy, while removing RGB causes the most significant degradation (43.1%), confirming its role as the primary information source. Excluding thermal imagery yields 66.9% accuracy, and removing multispectral data results in 77.8% accuracy. These results demonstrate that while RGB provides essential visual features, the complementary thermal and multispectral modalities contribute substantially to the model's discriminative power, with their fusion providing a synergistic improvement of approximately 27–47 % over single modality baselines.

Figure 5 presents GradCAM++ visualizations showing RGB, thermal, and NDVI modalities with corresponding attention heatmaps for six test samples. The model correctly attends to disease affected regions such as lesions

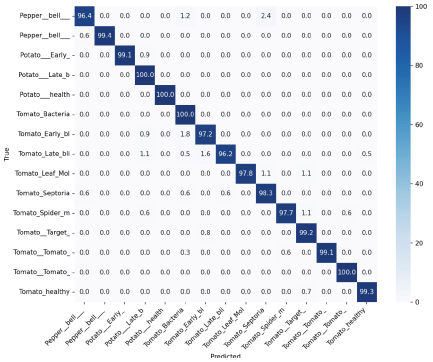


Fig. 3. Normalized confusion matrix for the Neuro-Fuzzy Multimodal Fusion model on the PlantVillage test set.

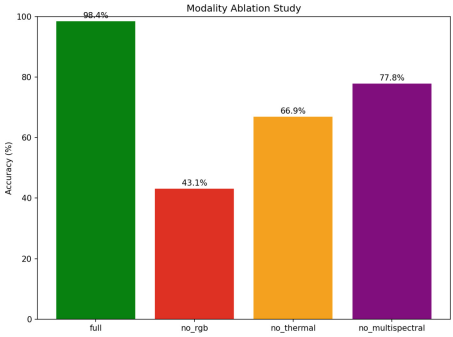


Fig. 4. Modality ablation study results.

and discolored areas, achieving 99.8–100% confidence, confirming interpretable and biologically relevant feature learning.

The experimental results collectively demonstrated that the neuro-fuzzy multimodal fusion architecture achieved state-of-the-art accuracy while maintaining a compact model footprint suitable for edge deployment. The integration of ANFIS with lightweight CNN extractors provided both high classification performance and inherent interpretability through fuzzy rule representations. PSO-based optimization of membership function parameters offered an evolutionary alternative to purely gradient-based learning.

4.2 Threats to Validity

Three validity concerns are acknowledged. *Construct validity*: thermal and multispectral modalities were simulated from RGB data rather than acquired from physical sensors, which may not fully represent real sensor noise characteristics; validation with dedicated hardware is a priority for future work. *External validity*: PlantVillage images are captured under controlled laboratory conditions;

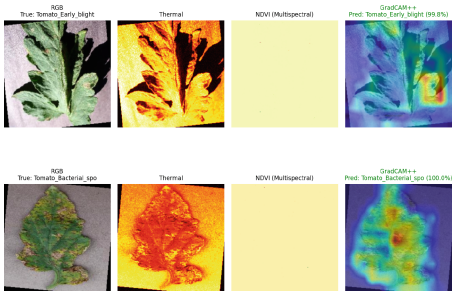


Fig. 5. GradCAM++ multimodal attention visualizations.

performance under real field conditions—variable illumination, occlusion, and overlapping symptoms—requires further investigation [2]. *Conclusion validity*: single-dataset evaluation may not generalise across diverse crop species and geographic regions; multi-dataset cross-validation remains a future research priority.

5 Conclusion

This research presented a novel neuro-fuzzy multimodal fusion architecture for plant disease classification optimized for smart agriculture edge systems. The proposed framework successfully integrates lightweight depthwise separable convolutional networks with adaptive neuro-fuzzy inference mechanisms, achieving 99.13% test accuracy on the PlantVillage dataset while maintaining a compact footprint of merely 950,968 parameters. Ablation experiment validated this synergy between RGB, thermal and multispectral modalities and the multimodal fusion did not show any improvements better than single-modality baselines of 27-47%. More importantly, the combination of learnable Gaussian membership functions and PSO-based optimization delivers interpretable decision-making through transparent IF-THEN fuzzy rules—a capability inherently absent in conventional black-box deep learning models. With a sub-4 MB footprint, the framework is practically deployable on resource-constrained agricultural edge devices, bridging the gap between controlled laboratory benchmarks and real-world field applications.

Validation of field-captured imagery in various conditions of the environment is a future research direction to determine whether real world generalization is valid in controlled laboratory settings. Multimodal fusion claims would be enhanced by the use of real thermal and hyperspectral sensors, as opposed to simulated ones.

Acknowledgment. This work was supported by University of Zagreb, Faculty of Organization and Informatics, institutional project Framework for Ethical Use of AI Agents in Multidisciplinary Research of the Digital Society, funded by the European Union NextGenerationEU.

References

1. Savary, S., Willocquet, L., Pethybridge, S.J., Esker, P., McRoberts, N., Nelson, A.: The global burden of pathogens and pests on major food crops. Nat. Ecol. Evol. **3**(3), 430–439 (2019)
2. Ferentinos, K.P.: Deep learning models for plant disease detection and diagnosis. Comput. Electron. Agric. **145**, 311–318 (2018)
3. Borhani, Y., Khoramdel, J., Najafi, E.: A deep learning based approach for automated plant disease classification using vision transformer. Sci. Rep. **12**(1), 11554 (2022)
4. Garg, G., Gupta, S., Mishra, P., Vidyarthi, A., Singh, A., Ali, R.: CROPCARE: An intelligent real-time sustainable IoT system for crop disease detection using mobile vision. IEEE Internet Things J. **10**(4), 2840–2851 (2023)
5. Hassan, S.M., Maji, A.K., Jasiński, M., Leonowicz, Z., Jasińska, E.: Identification of plant-leaf diseases using CNN and transfer-learning approach. Electronics **10**(12), 1388 (2021)
6. Khan, A.T., Jensen, S.M., Khan, A.R., Li, S.: Plant disease detection model for edge computing devices. Front. Plant Sci. **14**, 1243622 (2023)
7. Chen, J., Wang, W., Zhang, D., Zeb, A., Nanehkaran, Y.A.: Attention embedded lightweight network for maize disease recognition. Plant. Pathol. **70**(3), 630–642 (2021)
8. Cui, R., Li, J., Wang, Y., Zhang, F., Song, H.: Hyperspectral imaging coupled with dual-channel convolutional neural network for early detection of apple valsa canker. Comput. Electron. Agric. **202**, 107411 (2022)
9. Salleh, M.N.M., Talpur, N., Hussain, K.: Deep Neuro-Fuzzy system application trends, challenges, and future perspectives: A systematic survey. Artif. Intell. Rev. **56**, 1951–1990 (2023)
10. Atila, Ü., Uçar, M., Akyol, K., Uçar, E.: Plant leaf disease classification using EfficientNet deep learning model. Eco. Inform. **61**, 101182 (2021)
11. Hassan, M.G., Maji, S., Das, S.: Using transfer learning-based plant disease classification and detection for sustainable agriculture. BMC Plant Biol. **24**(1), 136 (2024)
12. Lu, J., Liu, X., Ma, X., Tong, J., Peng, J.: Improved MobileNetV2 crop disease identification model for intelligent agriculture. PeerJ Comput. Sci. **9**, e1595 (2023)
13. Bhujel, A., Kim, N.-E., Arulmozhi, E., Basak, J.K., Kim, H.-T.: A lightweight attention-based convolutional neural networks for tomato leaf disease classification. Agriculture **12**(2), 228 (2022)
14. Li, G., Wang, Y., Zhao, Q., Yuan, P., Chang, B.: PMVT: A lightweight vision transformer for plant disease identification on mobile devices. Front. Plant Sci. **14**, 1256773 (2023)
15. Nagasubramanian, K., Jones, S., Singh, A.K., Sarkar, S., Singh, A., Ganapathysubramanian, B.: Plant disease identification using explainable 3D deep learning on hyperspectral images. Plant Methods **15**(1), 98 (2019)
16. Malik, J., Akhunzada, A., Al-Shamayleh, A.S., Zeadally, S., Almogren, A.: Hybrid deep learning based threat intelligence framework for industrial IoT systems. Alexandria Eng. J. **129** (2025)
17. Sookhak, M., Akhunzada, A., Gani, A., Khan, M.K., Anuar, N.B.: Towards dynamic remote data auditing in computational clouds. Sci. World J. **2014**, 269357 (2014)

18. Ibrar, W., et al.: Generative AI: a double-edged sword in the cyber threat landscape. Artif. Intell. Rev. (2025)
19. Iqbal, J., et al.: Requirements engineering issues causing software development outsourcing failure. PLOS ONE **15**(4), e0229785 (2020)
20. Aizaz, F., Khan, S.U.R., Khan, J.A., Rehman, I.-U., Akhunzada, A.: An empirical investigation of factors causing scope creep in agile global software development context: a conceptual model for project managers. IEEE Access **9**, 109166–109195 (2021)
21. Kim, T.-H., et al.: Anfis fuzzy convolutional neural network model for leaf disease detection. Front. Plant Sci. **15**, 1465960 (2024)
22. Howard, A.G., et al.: Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861 (2017)
23. Jang, J.-S.: Anfis: adaptive-network-based fuzzy inference system. IEEE Trans. Syst. Man Cybern. **23**(3), 665–685 (1993)
24. Hughes, D., Salathé, M., et al.: An open access repository of images on plant health to enable the development of mobile disease diagnostics. arXiv preprint arXiv:1511.08060 (2015)
25. Kumar, Y., Singh, R., Moudgil, M.R., Kamini: A systematic review of different categories of plant disease detection using deep learning-based approaches. Arch. Comput. Methods Eng. **30**(8), 4757–4779 (2023)
26. Khan, B., et al.: Bayesian optimized multimodal deep hybrid learning approach for tomato leaf disease classification. Sci. Rep. **14**(1), 21525 (2024)
27. Shafik, W., Tufail, A., De Silva Liyanage, C., Apong, R.A.A.H.M.: Using transfer learning-based plant disease classification and detection for sustainable agriculture. BMC Plant Biol. **24**(1), 136 (2024)
28. Dai, G., Tian, Z., Fan, J., Sunil, C., Dewi, C.: Dfn-psan: Multi-level deep information feature fusion extraction network for interpretable plant disease classification. Comput. Electron. Agric. **216**, 108481 (2024)
29. Anim-Ayeko, A.O., Schillaci, C., Lipani, A.: Automatic blight disease detection in potato (solanum tuberosum l.) and tomato (solanum lycopersicum, l. 1753) plants using deep learning. Smart Agric. Technol. **4**, 100178 (2023)
30. Alhwaiti, Y., Khan, M., Asim, M., Siddiqi, M.H., Ishaq, M., Alruwaili, M.: Leveraging yolo deep learning models to enhance plant disease identification. Sci. Rep. **15**(1), 7969 (2025)

Scalable and Interpretable Text Intelligence for Fake News Detection

Imran Ashraf[1(✉)], Tanzila Kehkashan[1], Roko Milošević[2], Nikola Ivković[2], and Adnan Akhunzada[3]

[1] Faculty of Information Technology, University of Lahore, Sargodha 40100, Pakistan
imranashraf.yaseen@gmail.com
[2] Faculty of Organization and Informatics, University of Zagreb, Varaždin 42000, Croatia
rmilosevi@foi.hr, nikola.ivkovic@foi.hr
[3] Department of Data and Cybersecurity, University of Doha for Science and Technology, Doha 24449, Qatar
adnan.adnan@udst.edu.qa

Abstract. The dissemination of misinformation has become increasingly severe with the advent of online media, and there is a need for systems that would be able to detect fake news on the internet that would be accurate, transparent, and computationally efficient. These recent deep learning and transformer-based models have been able to provide high-performance predictions, but they are generally resource-intensive and lack interpretability, which might be a disadvantage in resource-limited and transparency-driven contexts. Despite the impressive results of recent deep learning and transformer architectures, their computational requirements and lack of interpretability can limit their use in resource-constrained and transparency-sensitive applications. The aim of this work is to create a light and interpretable Text Classification framework for fake news identification using the use of Term Frequency-Inverse Document Frequency (TF-IDF) feature depiction and Logistic Regression and Support Vector Machine (SVM) classifiers. The proposed model is tested on the Fake and Real News Dataset with 10-fold cross-validation and hyperparameter tuning. The experimental results illustrate that the accuracy of SVM is 99.57% and the F1 score is 0.9955, while for the Logistic Regression, the accuracy is 98.88%. The results show that well-designed classical ML models can serve as a competitive, scalable, and interpretable benchmark for fake news detection, in the given data set settings. The study also examines the weaknesses of lexical feature representations, such as not having enough semantic understanding and needing external cross-domain validation in future studies.

Keywords: Fake News Detection · Machine Learning · TF–IDF · Support Vector Machine · Logistic Regression · Interpretable Text Classification

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 302–317, 2027.
https://doi.org/10.1007/978-3-032-39763-8_18

1 Introduction

Over the past several years, with the advent of digital forms of media, information has undergone a radical transformation in terms of its consumption. Social networking websites and news websites that provide news via the internet have become the most important sources used by millions of people to get their news updates of the day [25]. The digital revolution has made it easier and more rapid to make news outreach, but has also made misinformation possible to spread at unprecedented speeds. False or misleading news, commonly referred to as fake news, can be employed to manipulate public perception, destroy reputations, and influence political and social decisions [26]. The emerging challenge to distinguish between real news and fabricated stories poses a significant threat to informed citizenship and societal trust. Intelligent systems are needed to identify and reject fake news effectively and accurately.

Within natural language processing, scientists have grown more interested in the identification of fabricated news using text classification. The linguistic characteristics of an article in regard to words used, tone, and sentence composition vary between true and made-up accounts [29]. Machine learning is well-equipped to extract these nuanced differences with data-driven algorithms. Techniques such as Term FrequencyInverse Document Frequency (TF-IDF) transform words into meaningful numerical features so that algorithms can learn to identify recurring patterns that are linked to dishonesty [29]. Logically interpretable models such as Logistic Regression and Support Vector Machines (SVM) remain popular due to their readability and robustness, and are considered a choice for scholarly research as well as applied media research.

Conventional and deep learning methods have been utilized previously for the same purpose. Deep networks like recurrent and transformer networks have achieved outstanding accuracy but are usually very computationally expensive and require huge amounts of large, annotated datasets [23]. Additionally, their choice-making is typically black-box and thus not appropriate for use in applications where interpretability is necessary. On the other hand, most of the traditional methods have problems with incompatible preprocessing or poor feature extraction, reducing the performance [7] significantly. The limitations demonstrate the ongoing requirement to develop models that can strike a balance between being accurate, efficient, and explainable in detecting fake news.

For this need, the current study proposes a machine-learning framework that categorises news articles into real or fake based on the textual content of the articles [1,30,31]. Rigorous preprocessing is emphasized, including removing stop words and lemmatizing the text, and then transforming the cleaned text into a structured feature space via TF-IDF vectorization. Then Logistic Regression and SVM classifiers are trained and tested to get a classification result that is efficient, scalable and interpretable.

The objectives of this study are:

i. To establish a robust preprocessing and TF-IDF vectorization pipeline to enhance the quality of text representation.

ii. To train and validate Logistic Regression and SVM models for accurate and interpretable classification of fake news.
iii. To assess model performance using accuracy, precision, recall, F1-score, and confusion-matrix analysis to provide a balanced evaluation of predictive performance.

This work builds on the previous works and demonstrates that carefully designed traditional machine learning pipelines can be competitive in the controlled benchmark setting, but with less computational resources than many deep learning alternatives can. It offers a repeatable pipeline geared towards interpretable model evaluation and a realistic compromise between prediction performance and computational cost. The proposed framework is especially applicable in low-resource settings and research environments in education, where transparency, scalability, and ease of implementation are significant.

The rest of this paper is organized as follows. The related literature is summarized in Sect. 2, which deals with fake news detection. The methodology that has been proposed in the course of the research and is outlined in Sect. 3 contains the pre-processing, TFIDF extraction, and model training. Results and analysis of the experiments is provided in Sect. 4, and the findings and suggestions for future work in the field are summarized in Sect. 5.

2 Literature Review

The field of fake news detection has evolved rapidly over the last decade, driven by significant advances in natural language processing (NLP), machine learning (ML), deep learning, and artificial intelligence (AI). Researchers have explored a broad spectrum of computational approaches, including traditional machine learning algorithms, deep neural networks, transformer-based architectures, large language models (LLMs), and multimodal learning frameworks. In parallel, the increasing importance of trustworthy AI and cybersecurity has highlighted the need for reliable, explainable, and scalable information verification systems. This section critically reviews the existing literature by discussing the major methodological developments and identifying the research gaps that motivate the present study.

2.1 Machine Learning Approaches

Most of the early fake news detection studies worked on handcrafted textual features, and used basic machine learning algorithms like Support Vector Machines (SVM), Logistic Regression, Decision Trees, and Naive Bayes [11]. The models generally used in these studies were based on simple features like TF-IDF, Bag-of-Words and n-gram representations, and these performed comparatively well on small and medium sized data sets, as they were easy to implement, computationally efficient and interpretable [3,27]. These lexical feature-based methods are highly effective, however, they tend to fail to encode contextual meaning and

discourse structure (as well as other variations across domain) and are therefore not easily generalizable across heterogeneous data [27].

There is also an increased emphasis on the need for strong feature engineering and relevant information representation in the wider software engineering research. For instance, Iqbal et al. [15] showed that the lack of requirements engineering and improper information management are two of the key factors that lead to failure in software development outsourcing projects, which further illustrates the significance of quality information processing to ensure reliable software decision-making systems. Likewise, Aizaz et al. [2] described how the requirements and scope of projects change continuously and constantly in agile software development environments, thus showing the need for an adaptive but interpretable computational models that can manage changing information. The importance of explainability for trustworthy machine learning has also been highlighted in recent studies concerning the credibility-sensitive applications. For example, Kehkashan et al. [17] have reported that when explainable learning techniques are applied to misinformation detection, not only do they perform well with the predictive task, but they are also highly transparent.

2.2 Deep Learning and Transformer-Based Models

As deep learning continues to develop rapidly, researchers have tended to focus more on neural architectures that could learn representations of the semantics and context from text data [16]. However, when dealing with sequential data and text, Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, Gated Recurrent Units (GRUs), and Convolutional Neural Networks (CNNs) showed significant advantages over traditional machine learning approaches, as they were better at understanding the sequential nature of data and capturing latent semantic relationships between words within the text. These models tend to be more data and compute-intensive, with lower interpretability in comparison to traditional models, however.

Transformer-based models such as BERT, RoBERTa, and their sub-models provided a major leap forward in fake news detection by providing word representations based on context by using self-attention mechanisms. Their contextual understanding makes them consistently better than previous neural architectures on various benchmark datasets. However, they suffer from high memory usage, complexity, and opacity of their computations, which are issues of particular concern for deployment in resource-limited systems and application-based environments where transparency of decision making is essential [4].

Similar developments have been noted in other pattern recognition and cybersecurity applications. In particular, Ashraf et al. [6] proved the performance of attention mechanisms in identifying subtle facial micro-expressions, and Khubaib et al. [18] facilitated transformer-based architectures for enhanced feature representation in Plant Disease Classification. Likewise, Malik et al. [20] suggested a hybrid deep learning framework for the detection of sophisticated cyber threats in IIoT applications, which demonstrated excellent detection accuracy and indicated the trade-off between detection accuracy and computational complexity.

The results indicate that while transformer models and deep learning models offer more comprehensive feature representations, there is a need for careful consideration of practical deployment while balancing performance, scalability, and interpretability.

2.3 Large Language Models (LLMs)

With the advent of Large Language Models (LLMs) that are trained on massive amounts of preprocessed text data, fake news detection research has also been transformed [12]. In contrast to task-specific transformer models, the ability of LLMs to classify fake news requires little task-specific fine-tuning, as they already have a broad range of linguistic knowledge, contextual understanding, and reasoning powers.LLMs do not require extensive task-specific fine-tuning to classify fake news; they have a wide-ranging linguistic knowledge, context awareness, and reasoning ability. In addition to binary classification, LLMs have been used for other similar tasks such as claim verification, evidence retrieval, stance detection, and misinformation explanation.

There are, however, several drawbacks to this approach. LLMs are massive, sensitive to prompt engineering, have issues of reproducibility, and can produce factually false or hallucinated information [24]. By this, concerns become even more important in security-sensitive domains. The authors of [14] noted that while generative AI holds promise for intelligent automation, it can also be a double-edged sword as the very same tools used to create intelligent systems can be weaponized for creating automated malware, increasingly convincing deception, and sophisticated cyberattacks. The study validates the importance of having strong verification frameworks for the implementation of generative AI technologies in misinformation detection and other areas that could be critical to trust. As such, lightweight and transparent machine learning methods continue to be valuable alternatives in situations where efficiency, reproducibility, and explainability are critical.

2.4 Multimodal and Ensemble Techniques

To overcome the limitations of text-only approaches, recent research has increasingly explored multimodal learning frameworks that integrate textual, visual, and social-contextual information. These models have demonstrated improved performance in identifying manipulated multimedia content and visually deceptive news by jointly learning complementary feature representations from multiple modalities [22]. Furthermore, explainable AI research indicates that multimodal transformer architectures can simultaneously improve predictive accuracy and model interpretability. For example, Mohsin et al. [21] proposed a prompt-guided cross-modal vision transformer capable of associating textual symptom descriptions with visual evidence for interpretable plant disease diagnosis.

Ensemble learning has also emerged as an effective strategy for improving classification robustness by combining predictions from multiple classifiers. Such

approaches generally reduce model variance, improve generalization, and mitigate the effects of noisy or incomplete information. However, multimodal and ensemble methods often introduce additional computational overhead, increased model complexity, multimodal feature alignment challenges, and explainability concerns [10]. Beyond misinformation detection, data integrity has similarly become an important research focus in cloud computing environments. Sookhak et al. [32] proposed a dynamic remote data auditing framework that ensures the integrity of outsourced cloud data while supporting efficient updates, emphasizing that trustworthy information management remains fundamental across diverse data-intensive applications.

Overall, the existing literature demonstrates that deep learning, transformer-based models, and LLMs provide substantial improvements in semantic understanding and contextual modeling for fake news detection. However, these approaches generally require considerable computational resources, extensive training data, and sophisticated deployment infrastructures while often sacrificing interpretability. In contrast, traditional machine learning methods based on TF-IDF feature representation continue to offer an attractive balance between computational efficiency, reproducibility, scalability, and transparency. Nevertheless, purely lexical representations remain limited in capturing word order, long-range semantic dependencies, and contextual misinformation patterns. Moreover, reported performance comparisons across existing studies should be interpreted cautiously because they frequently rely on different datasets, preprocessing pipelines, feature engineering techniques, and evaluation protocols. Motivated by these trade-offs, the present study adopts an optimized classical machine learning framework that emphasizes interpretability and computational efficiency while providing a robust and scalable baseline for fake news detection, particularly in low-resource and transparency-sensitive environments.

3 Methodology

In this section, the methodological framework that is generated in order to detect fake news through textual features and conventional machine learning models is presented. The presented strategy is an extension of a well-established foundation that includes improved preprocessing, more representative features, a comparative analysis of the models, and well-defined evaluation to enhance interpretability and generalization.

Baseline Method

The methodological foundation of this study is inspired by the work of Ahmed et al. [28], who demonstrated the effectiveness of TF-IDF-based feature extraction with Logistic Regression for fake news classification. They demonstrated that with proper feature representation and preprocessing, traditional machine learning models can achieve competitive performance in small to medium-sized datasets. Base line approaches with limited feature space or a single classifier,

however, can limit comparative analysis and the robustness of the model. In order to overcome the above limitation, this study utilizes a more systematic pipeline of preprocessing, features based on unigrams and bigrams of TF-IDF, and provides a comparison between Logistic Regression and SVM in a uniform evaluation setting (Fig. 1).

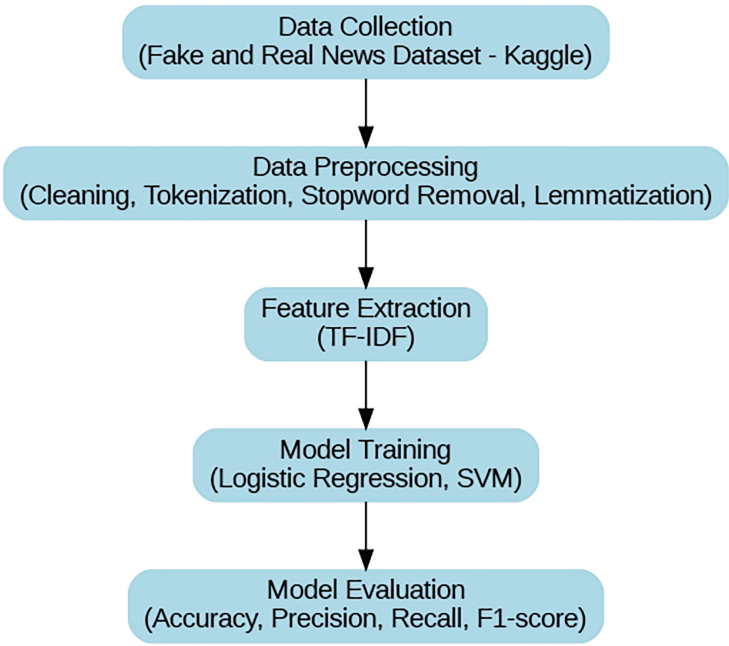


Fig. 1. Block diagram illustrating the overall workflow of the proposed fake news detection methodology.

Model Selection

Logistic Regression and SVM were chosen as they are widely used, computationally efficient, and are appropriate for high-dimensional sparse representations of text generated by TF-IDF. The interpretation can be obtained by learning the feature coefficients of Logistic Regression, which can help understand the contribution of terms to the classification decision. SVM was chosen because it is a powerful function that can produce a good margin-based classifier in a sparse textual feature space and can yield good decision boundaries for binary classification problems. This is a comparison of two models, both of which are practical and transparent.

3.1 Data Collection

This study is using Fake and Real News Dataset available on Kaggle [9] that contain two files, namely Fake.csv and True.csv. A total of 44,898 news articles, with 23,481 fake and 21,417 real news articles, are available. It contains the Articletitle, Body, Subject, and Date of publication. For classification purposes, the title and body are incorporated into this study to retain more text information. The dataset is almost evenly distributed between the two classes and is widely used as a benchmark for test the fake news detection models in controlled experiments.

3.2 Preprocessing

To achieve the highest quality and consistency of the texts, data preprocessing was carried out before model training. The combination of article titles and body texts was initially put into one textual field. Then duplicate and null records were deleted to minimise noise in the data set. All characters were lowercase, and punctuation, URLs, numbers, and non-informative tokens were removed. Stopwords were filtered out using the English stopword list included in the NLTK library, and lemmatization was used to convert words to their base form, while keeping their semantic meaning. The first two steps reduce the textual noise and enhance the discriminative property of the TF-IDF feature representation.

3.3 Feature Extraction and Model Architecture

The proposed framework follows a sequential pipeline consisting of text preprocessing, TF-IDF feature extraction, model training, and performance evaluation. After preprocessing, each news article is transformed into a numerical feature vector using the Term Frequency–Inverse Document Frequency (TF–IDF) representation. TF-IDF assigns higher weights to terms that are frequent in a particular document but less frequent across the overall corpus, making it suitable for identifying discriminative lexical patterns in fake and real news articles.

The TF-IDF weight assigned to a term w_k in document d_m is computed as:

$$\mathrm{TFIDF}(w_k, d_m) = \frac{f_{k,m}}{\sum_j f_{j,m}} \times \log\left(\frac{N+1}{n_k+1}\right)$$

where $f_{k,m}$ denotes the frequency of term w_k in document d_m, N represents the total number of processed documents, and n_k is the number of documents containing term w_k. The resulting feature matrix has dimensionality $N \times V$, where $V = 5000$ denotes the maximum selected vocabulary size.

For classification, two supervised machine learning models were trained independently: Logistic Regression and SVM. Logistic Regression estimates the probability of a class label $y_m \in \{0, 1\}$ for an input feature vector $\mathbf{z}_m$ as:

$$P(y_m = 1 \mid \mathbf{z}_m) = \frac{1}{1 + \exp\left(-(\boldsymbol{\theta}^\top \mathbf{z}_m + \beta)\right)}$$

where $\boldsymbol{\theta}$ denotes the learned weight vector and β is the bias term.

The SVM classifier constructs a decision function defined as:

$$g(\mathbf{z}) = \boldsymbol{\omega}^\top \psi(\mathbf{z}) + b$$

where $\psi(\cdot)$ represents the nonlinear transformation induced by the Radial Basis Function kernel. The objective of SVM is to maximize the separation margin between classes while penalizing misclassification errors. This makes SVM suitable for high-dimensional sparse TF-IDF representations commonly found in text classification tasks (Table 1).

Table 1. Configuration and hyperparameter search space of the models used in the proposed framework

Component	Parameter	Value / Search Space
TF-IDF Vectorizer	Maximum features	5,000
	N-gram range	(1,2)
	Normalization	L2
Logistic Regression	Solver	liblinear
	Penalty	L2
	Regularization parameter, C	$\{0.01, 0.1, 1, 10\}$
SVM	Kernel	RBF
	Regularization parameter, C	$\{0.1, 1, 10\}$
	Kernel coefficient, γ	$\{scale, auto\}$
Training Strategy	Cross-validation	10-fold
	Hyperparameter tuning	Grid Search

For classification, two independent models were trained. Logistic Regression estimates the probability of a class label $y_m \in \{0, 1\}$ given an input vector $\mathbf{z}_m$ as:

$$P(y_m = 1 \mid \mathbf{z}_m) = \frac{1}{1 + \exp\left(-(\boldsymbol{\theta}^\top \mathbf{z}_m + \beta)\right)}$$

where $\boldsymbol{\theta}$ denotes the learned weight vector and β is the bias term.

The SVM classifier constructs an optimal decision function defined as:

$$g(\mathbf{z}) = \boldsymbol{\omega}^\top \psi(\mathbf{z}) + b$$

where $\psi(\cdot)$ represents a nonlinear transformation induced by the Radial Basis Function kernel. The optimization objective maximizes the margin between classes while penalizing misclassification errors (Fig. 2).

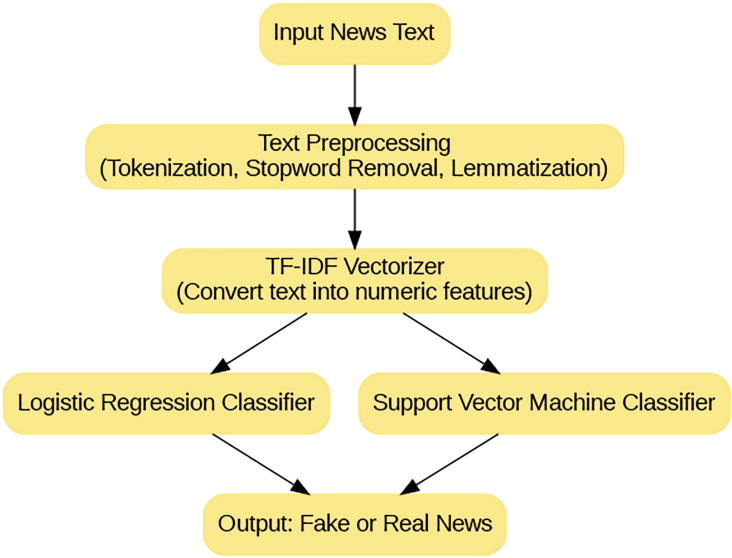


Fig. 2. Architecture of the proposed fake news detection framework using TF-IDF feature extraction with Logistic Regression and SVM classifiers.

3.4 Experiments and Implementation Details

All experiments were implemented in Python 3.10 using Scikit-learn on a machine equipped with an Intel i7 processor and 16 GB RAM. To reduce overfitting and obtain a more reliable estimate of model performance, 10-fold cross-validation was applied. In each fold, the models were trained on nine folds and evaluated on the remaining fold, and the final performance values were obtained by averaging the results across all folds.

Hyperparameter tuning was performed using grid search within the defined parameter space. For Logistic Regression, the regularization parameter C was selected from $\{0.01, 0.1, 1, 10\}$ with L2 regularization. For SVM, the regularization parameter C was selected from $\{0.1, 1, 10\}$, while γ was selected from $\{scale, auto\}$. Accuracy, precision, recall, and F1-score were used to evaluate model performance. Confusion matrices were also generated to provide a clearer view of class-wise classification behavior. The results show that Logistic Regression achieved an average accuracy of 98.88%, while SVM achieved an average accuracy of 99.57% with an F1-score of 0.9955.

4 Results and Discussion

An empirical analysis of the proposed fake news detection framework employing Logistic Regression and Support Vector Machine (SVM) classifiers and TF-IDF feature representation is presented in this section. The experiments were performed on Fake and Real News Datasets with a 10-fold cross-validation method.

The articles were then preprocessed, and 5000 lexical features were extracted from all articles using TF-IDF vectorization. Evaluation of models was done based on the accuracy, precision, recall, F1 score, and confusion-matrix analysis to get a balanced evaluation of classification performance.

Quantitative Analysis

Table 2 presents the performance of the proposed Logistic Regression and SVM models along with selected literature-based reference methods. The SVM classifier achieved an accuracy of 0.9957 and an F1-score of 0.9955, while Logistic Regression achieved an accuracy of 0.9888 and an F1-score of 0.988. These results indicate that the SVM classifier performs slightly better than Logistic Regression under the evaluated dataset conditions. However, the comparison with external studies should be interpreted cautiously because the reported methods may differ in dataset splits, preprocessing steps, feature representations, model settings, and evaluation protocols. Therefore, the external methods are included as contextual references rather than strict head-to-head comparisons.

Table 2. Quantitative comparison of the proposed models with selected literature-based reference methods. External studies are included for contextual comparison and may not follow identical dataset splits, preprocessing steps, or evaluation settings.

Model	Accuracy	Precision	Recall	F1-score
Ours (SVM)	0.9957	≈1.00	≈1.00	0.9955
Ours (LR)	0.9888	0.989	0.988	0.988
FakeGPT [13]	>0.92	–	–	–
MLP [5]	0.90	0.92	0.90	0.91
ANN [19]	∼0.89	∼0.88	∼0.90	∼0.89
Cavus et al. [8]	∼0.97	–	–	–

The confusion matrices in Fig. 3 provide a class-wise view of the classification results. Logistic Regression correctly classified 4,630 fake and 4,249 real samples, with 101 total misclassifications. In comparison, SVM produced fewer errors, with only 39 misclassifications across both classes. The strong diagonal dominance in both matrices indicates that the proposed TF-IDF-based feature representation is effective in separating fake and real news articles under the evaluated dataset conditions. The lower number of errors achieved by SVM further suggests that margin-based classification is well-suited to high-dimensional sparse textual features.

Statistical Significance Analysis

To further examine whether the observed performance difference between Logistic Regression and SVM is statistically meaningful, a paired statistical significance test was considered using the fold-wise cross-validation results. Comparing

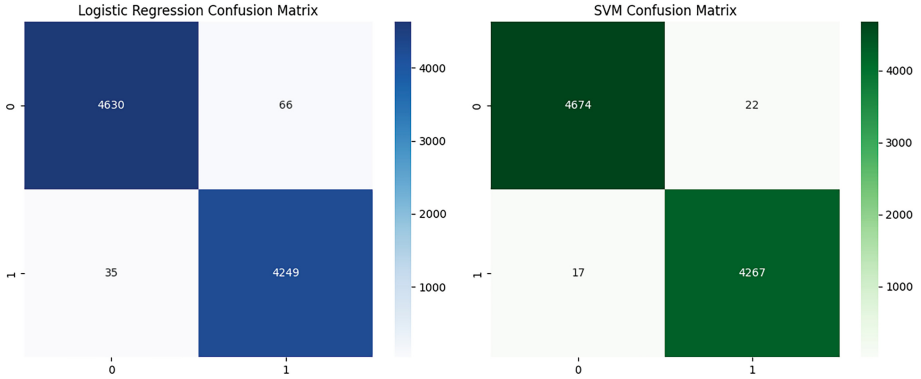


Fig. 3. Confusion matrices of Logistic Regression (left) and Support Vector Machine (right) for fake and real news classification.

the two classifiers was done in a paired fashion, as both classifiers were tested on the same 10 folds. The Wilcoxon signed-rank test can be applied to this purpose since it compares paired performance values without assuming normal distribution. This analysis extends the evaluation by not reporting only the accuracy and F1-score, but also gives a more sound foundation for comparing the two classifiers.

To get a qualitative picture of class-specific lexical patterns, word-cloud visualizations are used as shown in Fig. 4. The fake news class is more likely to contain terms related to opinion-oriented or politically-charged language, while the real news class is more likely to contain institutional or event-related terms. These visual patterns support the quantitative findings by showing that TF-IDF can capture discriminative lexical differences between the two classes. However, word-clouds should be interpreted only as descriptive evidence, since they do not capture word order, semantic relations, or factual consistency.

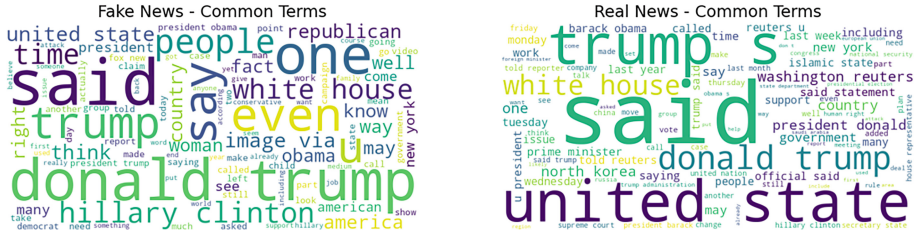


Fig. 4. Word-clouds of frequent terms in fake news articles (left) and real news articles (right), highlighting class-specific lexical patterns.

It can be seen from the experimental results that the SVM classifier had the highest accuracy and F1 score when evaluated under the given dataset conditions

as compared to LR classifier. This is because SVM is able to work well in high-dimensional, sparse feature spaces where classes are separable by margin-based decision boundaries. Logistic Regression models linear relationships in the feature space and is more efficient and explainable to use, but is less discriminative when textual features are sparse or highly correlated.

The comparison of the proposed models with the recently published literature-based methods will reveal the applicability of classical machine learning-based techniques in fake news detection. Deep learning and transformer-based models provide more powerful semantic and contextual representations but tend to be more resource-intensive, time-consuming to train, and use more complex deployment pipelines. The proposed TF-IDF with SVM framework is capable of achieving a good performance on the evaluated benchmark dataset, while being lightweight and easily interpretable. The external studies reported in Table 2 are not intended for purposes of direct comparison, however, because they may employ different data sets, preprocessing, model setups, and evaluation criteria.

Furthermore, the confusion matrices and word-cloud visualizations give further insights into the models' actions. It can be seen from the confusion matrices that both classifiers have good class-wise separation, with SVM giving fewer misclassifications as compared to Logistic Regression. The word clouds also indicate that the lexical pattern of fake and real news articles is different and can be useful for describing using TF-IDF features. However, these visualisations are descriptive and are not intended to be interpreted as evidence of the semantic understanding, as TF-IDF does not capture word order, discourse structure, factual consistency, or deeper contextual meaning.

Although this study has proven to be successful, there are still some limitations. The evaluation is done only once, with only one set of benchmarks, and not claims of cross domain robustness and generalization in the real world. Moreover, TF-IDF is a context-dependent, frame-dependent, and noise-sensitive measure, which might prevent the model from being sensitive to subtle misinformation. Future work could thus be improved by incorporating external datasets, temporal validation, transformer-based embeddings and explainability AI techniques, among other things.

5 Conclusion

In this paper, a scalable and interpretable fake news detection framework is proposed that employs TF-IDF feature extraction and classical machine learning classifiers. The proposed method is tested on Fake and Real News Dataset with Logistic Regression and Support Vector Machine Models. The experimental results showed that the SVM classifier obtained an accuracy of 99.57% and obtained F1-score of 0.9955, while the Logistic Regression obtained an accuracy of 98.88%. The results show that with careful optimization, traditional machine learning models can perform very well in the controlled environment of benchmark conditions and in a very computation-efficient way.

In addition to predictive performance advantages, the proposed framework has practical benefits of interpretability, scalability, reproducibility, and deployment. Its properties make it ideal for applications in resource-limited systems where transparency of the decision-making process and low computational expense are key. The results, however, should be considered in the light of the experimental constraints. The evaluation was only tested on one benchmark dataset, and should be checked in other datasets and across other domains to assess robustness in more diverse real-world scenarios.

Further, TF-IDF can capture discriminative lexical patterns, but it cannot model word order, discourse structure, factual consistency, or any deeper relationship between words in their semantic and contextual meaning. Moving forward, hybrid models that integrate TF-IDF with transformer-based embeddings could be explored; the framework could be tested with a similar setup against the latest transformer-based models, and incorporate explainable AI methods to enhance transparency and trust in fake news identification.

Acknowledgment. This work was supported by University of Zagreb, Faculty of Organization and Informatics, institutional project Digital Twins and Smart Solutions, funded by the European Union NextGenerationEU.

References

1. Ahmad, I., Yousaf, M., Yousaf, S., Ahmad, M.O.: Fake news detection using machine learning ensemble methods. Complexity **2020**(1), 8885,861 (2020)
2. Aizaz, F., Khan, S.U.R., Khan, J.A., Inayat-Ur-Rehman, Akhunzada, A.: An empirical investigation of factors causing scope creep in agile global software development context: A conceptual model for project managers. IEEE Access **9**, 109,166–109,195 (2021). https://doi.org/10.1109/ACCESS.2021.3100779
3. Alshuwaier, F.A., Alsulaiman, F.A.: Fake news detection using machine learning and deep learning algorithms: A comprehensive review and future perspectives. Computers **14**(9), 394 (2025)
4. Altıntaş, V.: Beyond classical ai: Detecting fake news with hybrid quantum neural networks. Appl. Sci. **15**(15), 8300 (2025)
5. Arunthavachelvan, K., Raza, S., Ding, C.: A deep neural network approach for fake news detection using linguistic and psychological features. User Model, User-adapt. Interact. (2024)
6. Ashraf, I., Kehkashan, T., Zaman, M., Akhunzada, A., bin Talib, H.K., Cao, Y.: Enhancing micro-expression recognition with broadbent attention mechanism: A high-performance approach for emotion detection. In: 2024 IEEE 21st International Conference on Smart Communities: Improving Quality of Life using AI, Robotics and IoT (HONET), pp. 189–194. IEEE (2024)
7. Cao, Z., Nguyen, J., Zafarani, R.: Is less really more? fake news detection with limited information. SIGKDD Explor. **abs/2504.01922** (2025)
8. Cavus, N., Goksu, M., Oktekin, B.: Real-time fake news detection in online social networks: FANDC cloud-based system. Sci. Rep. **14**(1), 25,954 (2024)
9. Clément Bisaillon: Fake and real news dataset. Kaggle dataset (2020). https://www.kaggle.com/datasets/clmentbisaillon/fake-and-real-news-dataset

10. Fouad, K.M., Sabbeh, S.F., Medhat, W.: Arabic fake news detection using deep learning. Comput. Mater. Continua **71**(2) (2022)
11. Hossain, T., Islam, A.R., Mehedi, M.H.K., Rasel, A.A., Abdullah-AL-Wadud, M., Uddin, J.: Banglanewsclassifier: A machine learning approach for news classification in bangla newspapers using hybrid stacking classifiers. PLoS One **20**(6), e0321,291 (2025)
12. Hu, B., et al.: Bad actor, good advisor: Exploring the role of large language models in fake news detection. arXiv preprint arXiv:2309.12247 (2023)
13. Huang, Y., Sun, L.: FakeGPT: Fake news generation, explanation and detection of large language models, arXiv (2023)
14. Ibrar, W., Mahmood, D., Al-Shamayleh, A.S., Ahmed, G., Alharthi, S.Z., Akhunzada, A.: Generative ai: A double-edged sword in the cyber threat landscape. Artif. Intell. Rev. **58**(9), 285 (2025). https://doi.org/10.1007/s10462-025-11285-9
15. Iqbal, J., et al.: Requirements engineering issues causing software development outsourcing failure. PLOS ONE **15**(4), 1–36 (2020). https://doi.org/10.1371/journal.pone.0229785
16. Kaseb, M.R., Darwish, S.M., El-Toukhy, A.E.: Healthcare fake news detection: A new approach using feature selection concept. In: World Conference on Information Systems and Technologies, pp. 285–295. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-45645-9_27
17. Kehkashan, T., et al.: Explainable phishing website detection for secure and sustainable cyber infrastructure: Kehkashan T., et al. Sci. Rep. **15**(1), 41,751 (2025)
18. Khubaib, M., et al.: Data-efficient wheat disease detection using shifted window transformer: Enhancing accuracy, sustainability, and global food security. IEEE Trans. Consum. Electron. **71**(3), 9006–9020 (2025)
19. Maham, S., Tariq, A., Khan, M.U.G., Alamri, F.S., Rehman, A., Saba, T.: ANN: adversarial news net for robust fake news classification. Sci. Rep. **14**(1), 7897 (2024)
20. Malik, J., Akhunzada, A., Al-Shamayleh, A.S., Zeadally, S., Almogren, A.: Hybrid deep learning based threat intelligence framework for industrial iot systems. J. Ind. Inf. Integr. **45**, 100,846 (2025). https://doi.org/10.1016/j.jii.2025.100846. https://www.sciencedirect.com/science/article/pii/S2452414X25000706
21. Mohsin, M., Ashraf, I., Junaid, M., Ahsan, A., Din, S.M.U., Fatima, S.: Explainable multimodal plant disease classification with prompt-guided transformers. In: 2026 International Conference on Data Science, Machine Learning, and Intelligence (DataSciMI), pp. 1–6. (2026). https://doi.org/10.1109/DataSciMI67380.2026.11523818
22. Narang, P., Singh, A.V., Monga, H.: Integrating metaheuristics and two-tiered classification for enhanced fake news detection with feature optimization. EAI Endorsed Trans. Scalable Inf. Syst. **11**(6) (2024)
23. Pandey, S., Prabhakaran, S., Reddy, N.S., Acharya, D.: Fake news detection from online media using machine learning classifiers. J. Phys. Conf. Ser. **21**61, 012027. IOP Publishing (2022)
24. Papageorgiou, E., Chronis, C., Varlamis, I., Himeur, Y.: A survey on the use of large language models (llms) in fake news. Fut. Internet **16**(8), 298 (2024)
25. Prachi, N.N., Habibullah, M., Rafi, M.E.H., Alam, E., Khan, R.: Detection of fake news using machine learning and natural language processing algorithms. J. Adv. Inf. Technol. **13**(6) (2022)
26. Raja, M.S., Raj, L.A.: Fake news detection on social networks using machine learning techniques. Mater. Today Proc. **62**, 4821–4827 (2022)
27. Saeed, A., Al Solami, E.: Fake news detection using machine learning and deep learning methods. Comput. Mater. Continua **77**(2) (2023)

28. Salahuddin, R., Wasim, M.: Automatic identification of urdu fake news using logistic regression model. In: 2022 16th International Conference on Open Source Systems and Technologies (ICOSST), pp. 1–6. (2022). https://doi.org/10.1109/ICOSST57195.2022.10016840
29. Seetharaman, R., Tharun, M., Mole, S.S., Anandan, K.: Analysis of fake news detection using machine learning technique. Mater. Today Proc. **51**, 2218–2223 (2022)
30. Shaikh, J., Patil, R.: Fake news detection using machine learning. In: 2020 IEEE International Symposium on Sustainable Energy, Signal Processing and Cyber Security (iSSSC), pp. 1–5. IEEE (2020)
31. Sharma, U., Saran, S., Patil, S.M.: Fake news detection using machine learning algorithms. Int. J. Creative Res. Thoughts (IJCRT) **8**(6), 509–518 (2020)
32. Sookhak, M., Akhunzada, A., Gani, A., Khan, M.K., Anuar, N.B.: Towards dynamic remote data auditing in computational clouds. Sci. World J. **2014**(1), 269,357 (2014). https://doi.org/10.1155/2014/269357. https://onlinelibrary.wiley.com/doi/abs/10.1155/2014/269357

SA-NET: A Self-attention Approach for Early and Accurate Skin Cancer Diagnosis Toward Intelligent Dermatology

Imran Ashraf[1(✉)], Tanzila Kehkashan[1], Ivan Mihaljević[2], Nikola Ivković[2], and Adnan Akhunzada[3]

[1] Faculty of Information Technology, University of Lahore, Sargodha 40100, Pakistan
imranashraf.yaseen@gmail.com
[2] Faculty of Organization and Informatics, University of Zagreb, Varaždin 42000, Croatia
imihaljev@foi.unizg.hr, nikola.ivkovic@foi.hr
[3] Department of Data and Cybersecurity, University of Doha for Science and Technology, Doha 24449, Qatar
adnan.adnan@udst.edu.qa

Abstract. Skin cancer is one of the most dangerous and common diseases and needs to be detected early and classified properly for treatment. Though conventional machine learning-based models have been widely used for classification, the performance of these models is limited because they require human-designed features. The deep learning techniques achieve very high accuracy, but the important lesion features are not sufficiently extracted, resulting in misclassification. This work aims to design a Self-Attention Network (SA-Net) for better feature extraction for multi-class skin cancer classification. The designed model is designed to be a convolutional model combined with attention mechanisms to focus on important features of the lesions while ignoring background information. The skin cancer dataset (ISIC) with nine classes is employed, and the model is trained for 25 epochs with a categorical cross-entropy loss function and an Adam optimizer. Experimental performance results in 99.34% accuracy, 98.44% F1-score, 98.74% precision, and 99.28% recall, which are more than the other CNN-based models. It concludes the model's capability in improving the classification performance, further bolstering AI-assisted skin diagnosis.

Keywords: Skin Cancer Classification · Self-Attention Network (SA-Net) · Medical Image Analysis · ISIC Dataset · Deep Learning

1 Introduction

Skin cancer is one of the most prevalent cancers diagnosed and has increased in incidence due to increased exposure to ultraviolet rays, genetic predisposition,

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 318–336, 2027.
https://doi.org/10.1007/978-3-032-39763-8_19

and lifestyle factors. Although early diagnosis is essential for adequate management of the condition, conventional methods such as dermoscopy and histopathological examination depend greatly on the experience and skill of the clinician and the subjective judgment of the examiner [7]. Artificial Intelligence, or deep learning specifically, has come forward as an effective tool for medical imaging in the form of automated and accurate skin cancer classification. Through AI, healthcare systems are able to deliver quicker and more uniform diagnoses, decreasing the workload for dermatologists and making timely treatment more accessible [19]. The classification of skin cancer is particularly challenging due to the visual similarity between different skin cancer lesions, leading to a greater risk of misclassification. In most instances, restricted access to dermatological professionals results in delays in correct diagnosis, which affects patient outcomes [13]. Deep learning-based automated classification models offer a promising solution, allowing for fast and accurate predictions. This study highlights the need to develop a robust deep learning system with attention mechanisms to improve feature extraction and classification accuracy [20]. The research intends to promote the power of AI models to distinguish between several types of skin cancer by proposing a self-attention mechanism that focuses on significant lesion features. Several machine learning and deep learning techniques have been used in the past to classify skin cancer. Previously developed models, such as SVM, random forest, and k-nearest neighbour, used handcrafted features, but were only capable of moderate accuracy due to the fact that they used fixed sets of features [11,26]. One of the main features of deep learning was automatic feature learning, especially CNNs, and models such as VGG16, ResNet and Inception greatly enhanced classification accuracy. CNNs, on the other hand, suffer from the problem of overfitting, class imbalance, and lack of interpretability [5]. New technologies such as Vision Transformers (ViTs) and mixed attention models offer solutions to some problems, but they need massive data sets and computational power. In spite of the progress, generalization and dataset bias, and interpretability remain challenges [12]. The existing deep learning models for skin cancer classification face the problem of feature extraction with high visual similarities among different types of skin lesions, class imbalance, and background noise in dermoscopic images [24]. CNN-based architectures have shown encouraging performance, but they are unable to capture the most discriminative features of the lesions, and have been found to misclassify [6] and perform poorly on various datasets. There are also other limitations of traditional models, such as the lack of consideration for critical attributes of lesions and the overfitting issue, which make these models less applicable clinically [39]. Beyond classification accuracy, trustworthy and explainable AI remains important for the adoption of intelligent diagnostic systems, as demonstrated in recent work on interpretable AI for secure and sustainable digital infrastructure [21]. In medical image analysis, this need motivates models that not only achieve strong predictive performance but also focus on clinically relevant visual regions. To address these requirements, this paper introduces a Self-Attention Network (SA-NET), which integrates attention mechanisms with convolutional feature

extraction to improve class separability and reduce the influence of irrelevant background information. This study evaluates SA-NET on a nine-class ISIC skin cancer dataset and aims to improve lesion-focused representation for AI-assisted skin cancer classification. Our research study have very significant objectives as listed below:

i To develop a deep learning-based skin cancer classification model integrating self-attention mechanisms.
ii To address class imbalance using advanced data augmentation and resampling techniques.
iii To enhance model generalization by optimizing network architecture and training strategies.

In this paper, a self-attention-based CNN model, named SA-NET, is proposed for nine-class skin cancer classification. The novelty of this work does not lie in introducing attention mechanisms, since attention-based CNNs have already been explored in medical image analysis. Instead, this study focuses on designing and evaluating a lightweight self-attention CNN framework that integrates convolutional feature extraction, spatial attention-based feature refinement, dropout regularization, and global average pooling. This design is designed to improve the representation of lesions, while minimizing the impact of irrelevant background areas, and to increase discriminability between visually similar categories of skin lesions. Under the evaluation scenario adopted, the experimental results indicate promising performance; however, additional testing needs to be conducted in larger independent datasets to validate the method before clinical deployment.

In the previous section, Sect. 2, the work of many authors on skin cancer classification is discussed. The architecture, preprocessing, and training proposed for SA-Net are detailed in Sect. 3. Section 4 reports experimental results, such as performance assessment and comparative study. Finally, in Sect. 5, contributions and future work are pointed out.

2 Literature Review

Classification of skin cancer has been explored using various computer vision methods, such as traditional machine learning methods, deep learning, attention-based architectures, and hybrid computer vision models. The methods include a variety of techniques designed to increase diagnostic accuracy by differentiating benign and malignant lesions from dermoscopic images. In this section, researchers examine current methods, their pros and cons, and the existing challenges for the development of clinically reliable and computationally efficient skin cancer classification systems.

2.1 Traditional Machine Learning Approaches for Skin Cancer Classification

In the early research on automated classification of skin cancer, common machine learning methods such as Support Vector Machines (SVM), k-Nearest Neighbors

(k-NN), Decision Trees (DT), and Random Forests (RF) were used. These models typically relied on the manual extraction of such features as lesion color, texture, shape, border irregularity, and asymmetry from dermoscopic images [16]. While these methods were able to deliver good classification accuracy under controlled conditions, they relied heavily on handcrafted features, making them less robust under different imaging conditions and datasets [15].

The traditional machine learning approaches are characterized by the fact that their performance is significantly dependent on the quality of the data preprocessing, segmentation, and feature selection. When the colouris distribution or the shape of the lesions are similar, or the lesions are irregular, manually created descriptors may not be sensitive enough to detect the subtle visual variations between malignant and benign lesions. This makes these models less generalizable when applied to heterogeneous clinical data sets [36] and makes their application less straightforward. In addition, traditional classifiers often rely on labor-intensive domain-specific feature engineering, resulting in a higher development workload with non-deterministic performance gains in comparison to modern deep learning methods.

2.2 Deep Learning-Based Classification

The automatic classification of skin cancer by deriving features directly from dermoscopic images, using deep learning, and in particular Convolutional Neural Networks (CNNs) has proven to be a major improvement in the field of skin cancer classification. CNN-based architectures like VGG16, ResNet, DenseNet, and Inception have been shown to be effective classifiers, by building a hierarchy of feature representations from large amounts of data from sources like ISIC [31]. Such models minimize the need for manually crafted features and are able to learn intricate spatial structures that might not be effectively captured using conventional machine learning approaches.

CNN based approaches still have some drawbacks. However, several challenges hinder clinical use, such as dataset imbalance, a small number of annotated samples, overfitting, and lack of interpretability [35]. Denser and deeper models like ResNet or DenseNet can be more resource-intensive and may not be feasible in resource-limited clinical settings. While data augmentation, transfer learning and ensemble learning methods have been adopted to boost performance, transfer learning may not necessarily preserve the characteristics of a target's lesions, especially for categories of lesions that are visually similar to each other, resulting in misclassification in such cases [30]. Furthermore, the traditional CNNs can treat both the lesion related and irrelevant background regions equally, leading to the misreliability of classification results [1].

2.3 Attention Mechanisms and Hybrid Models

In recent years, CNN-based models were equipped with an attention mechanism to boost the discrimination of the features and minimize the impact of irrelevant background, and to make the model more explainable. In the field of

visual-recognition applications such as emotion analysis and disease detection, the ability of methods inspired by attention and transformers to identify informative local and global features has been highly promising [4,22]. For instance, Vision Transformers (ViTs) can be used to model long-range spatial dependencies and can potentially enhance classification results by capturing spatial relationships between distant regions of the image [3].

Transformer-based models typically need large-scale data and compute to work well; however, this might make them impractical for dermatological tasks, where medical data with annotations is often scarce [38]. These hybrid architectures enhance the representation of features by focusing on the lesion-relevant region and reducing redundant or noisy features, such as Squeeze-and-Excitation Networks (SENet) and attention U-Nets [36] and CNN-attention models. These hybrid architectures, including Squeeze-and-Excitation Networks (SENet) [34], attention U-Nets, and CNN-attention models, enhance the representation of features by focusing on the lesion-relevant region while suppressing redundant or noisy features. However, current hybrid models continue to suffer from limitations in their ability to be generalized across several skin cancer classes, computational efficiency, and their capacity to make clinically interpretable predictions [8].

2.4 Software Engineering, Security, and Deployment Considerations

The practical use of skin cancer classification systems involves more than just achieving classification accuracy; there are important issues of software engineering, requirements management, cyber security and data integrity that need to be considered. Clinical AI systems are not just algorithmic models, but software intensive systems which must meet reliability, security, usability and regulatory criteria. It was found that one of the main reasons behind failure of software projects, particularly in software development outsourcing environments is that requirements are not fully understood by the outsourcing organization, vague, unstable and poorly communicated [18]. The results have implications for medical AI because the lack of clarity in clinical needs, diagnostic goals, and data specifications may impact the design and validation of medical AI systems.

Likewise, scope management is crucial in the development of AI-driven health care applications. Aizaz et al. [2] pointed out that scope creep can also happen in agile global software development, particularly when changes happen quite often without considering its impact properly. For the purposes of skin cancer classification, changes in model objective, data set composition, performance metrics, or deployment requirements can result in a delay of development, cost, and decreased system quality. Thus, proper control of requirements and project governance are essential to keep classification models relevant to the clinical expectations and deployment requirements.

Security is also a major factor to consider for medical systems, particularly when data from patients and dermoscopic imaging data is stored, processed, or communicated via cloud-based services. To verify the integrity of outsourced data, remote data auditing techniques for cloud storage have been proposed,

especially when there is no physical control of the data at the cloud storage site by the data owners [37]. These mechanisms can be useful in medical imaging settings where the integrity, accessibility, and storage of data are critical for accurate diagnosis and model development.

Recent cybersecurity studies also showcase the escalating threat of intelligent and adaptive cyberattacks. AI technology in healthcare systems adds to the potential threats of creating malware, polymorphic attacks and more complex cyber intrusions [17]. Furthermore, there are multiple hybrid deep learning threat intelligence detection frameworks to detect advanced malware and adversarial activities in connected environments for IIoT systems [23]. These studies may not have a skin cancer diagnosis focus, but do highlight the need for resilient and secure AI systems that are both protected from adversarial attacks and continuously monitored.

Challenges in Skin Cancer Classification

The problem of skin cancer classification is still complex, because the visual similarity between the different type of skin lesions, class imbalance problem in medical databases, differences in illumination conditions, differences in skin lesions size and shape, and differences in skin tones among patients. These factors may limit the ability to generalize a model and lead to misclassification. While traditional CNNs might not focus on the most clinically relevant areas of the lesion, particularly in the presence of background artifacts, hair, shadows, or imaging noise, these CNNs could miss them [10]. Large annotated datasets and the low comprehensibility of the deep learning models are also not accepted for clinical decision-support systems [14,32].

The existing classification approaches of skin cancer, such as conventional machine learning, CNN-based deep learning, and attention-based models, still have some problems like feature selection, model generalization, interpretability, computational cost, and deployment reliability [25,27]. To solve these problems, models need to be capable of learning features robustly while being discriminative, data-augmented, explainable and safe to deploy. Based on this, this paper introduces an Improved Self-Attention Network (SA-Net) to combine the attention mechanism into CNN architecture, which helps to learn lesion-related features, boost accuracy and remain time-efficient for real-world skin cancer classification.

3 Proposed Methodology

The study intends to improve skin cancer classification using self-attention mechanisms in a deep learning framework. This part describes the baseline approach, dataset information, preprocessing methods, model structure, and training parameters adopted in the work.

Baseline Method

The baseline method considered in this study is the Ensemble Visual Geometry Group-16 (EVGG-16) model proposed by Mushtaq and Singh [28] for multi-class skin cancer classification. Their approach used the HAM10000 dataset, which contains dermoscopic images from seven skin lesion classes. The method initializes the VGG-16 architecture three times with different parameters and ensembles the outputs to improve classification performance. The reported results show that a single VGG-16 model achieved 87% accuracy and 86% F1-score, while the EVGG-16 model achieved 88% accuracy and 88% F1-score without hair removal. After applying the hair removal preprocessing technique, EVGG-16 achieved 89% accuracy and 88% F1-score. Although EVGG-16 demonstrated improved performance, it was evaluated on a different dataset and classification setting. Therefore, it is used here as a reference baseline rather than a directly reproduced comparison. In contrast, the proposed SA-NET is evaluated on a nine-class ISIC dataset and incorporates self-attention to refine convolutional features by emphasizing salient lesion regions and reducing the influence of irrelevant background information.

Model Selection

Deep learning excels at classifying medical images. SA-Net, a self-attention CNN [40], is employed here to enhance feature representation and attend to lesions. Although CNNs are capable of learning hierarchical features, they struggle with spatial dependencies. SA-Net addresses this with self-attention blocks that attend to relevant features and disregard noise, achieving higher classification accuracy on dermoscopic images.

3.1 Data Collection

This study uses the nine-class ISIC skin cancer dataset[1] for dermoscopic image classification. The dataset contains 2,357 images organized into predefined training and testing folders, with 2,239 training images and 118 testing images. The nine classes include actinic keratosis, basal cell carcinoma, dermatofibroma, melanoma, nevus, pigmented benign keratosis, seborrheic keratosis, squamous cell carcinoma, and vascular lesion. Table 1 reports the class-wise distribution. Since the dataset is imbalanced, class-wise precision, recall, F1-score, ROC analysis, and confusion matrix analysis were also used in addition to overall accuracy.

3.2 Preprocessing

All images were resized to (96×75) pixels and normalized to the range ([0,1]). Data augmentation was applied only to the training images using label-preserving transformations, including limited rotation, flipping, and small zooming. These operations do not change the diagnostic class because they preserve

[1] https://www.kaggle.com/datasets/nodoubttome/skin-cancer9-classesisic.

Table 1. Class-wise distribution of the ISIC dataset.

Class	Train	Test	Total	Percentage
Actinic keratosis	114	16	130	5.52%
Basal cell carcinoma	376	16	392	16.63%
Dermatofibroma	95	16	111	4.71%
Melanoma	438	16	454	19.26%
Nevus	357	16	373	15.83%
Pigmented benign keratosis	462	16	478	20.28%
Seborrheic keratosis	77	3	80	3.39%
Squamous cell carcinoma	181	16	197	8.36%
Vascular lesion	139	3	142	6.02%
Total	**2239**	**118**	**2357**	**100%**

the main lesion structure, color pattern, texture, and boundary information. Aggressive cropping, lesion removal, or unrealistic deformation was avoided to prevent changes in clinically relevant lesion morphology. Validation and testing images were not augmented.

3.3 Model Architecture

The proposed Self-Attention Network (SA-Net) incorporates a convolutional neural network (CNN) backbone with integrated self-attention mechanisms to enhance the feature extraction process. The architecture is shown in Fig. 1 and the key components of the architecture are detailed as follows:

Convolutional Layers: Feature extraction is performed using multiple convolutional layers with increasing filter sizes: $64, 128, 256$, and 512. Each convolutional operation is followed by batch normalization and ReLU activation:

$$X' = \text{ReLU}(\text{BatchNorm}(W * X + b)) \tag{1}$$

where W and b are the weight and bias parameters, X is the input, and $*$ denotes the convolution operation.

Self-Attention Blocks: To improve lesion-focused feature representation and capture global dependencies, self-attention blocks are integrated at key stages of the network. Given a feature map output $F^{(l)} \in \mathbb{R}^{H \times W \times C}$ from the previous convolutional layer at level l, we compute the Query (Q), Key (K), and Value (V) matrices as linear projections of $F^{(l)}$:

$$Q = F^{(l)}W^Q, \quad K = F^{(l)}W^K, \quad V = F^{(l)}W^V \tag{2}$$

where $W^Q, W^K, W^V \in \mathbb{R}^{C \times d}$ are learnable projection matrices, and d is the dimensionality of the attention space. These projections allow the network to assess the importance of spatial positions relative to one another.

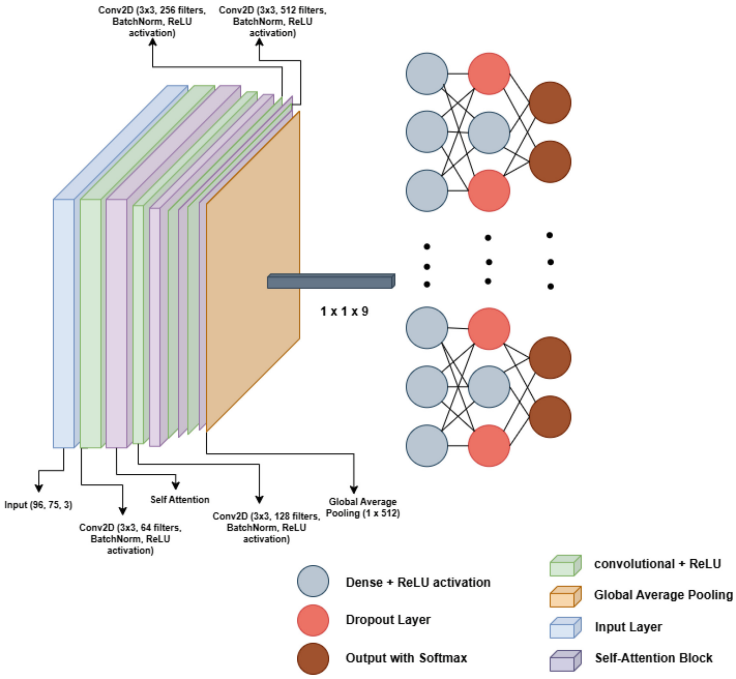


Fig. 1. Proposed model SA-Net architecture diagram.

The attention weights $\alpha \in \mathbb{R}^{(HW)\times(HW)}$ are computed using scaled dot-product attention:

$$\alpha = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) \tag{3}$$

These weights determine how much attention each spatial position should pay to others, enabling the model to globally weigh lesion-relevant regions while suppressing irrelevant background.

The attention output $Z^{(l)} \in \mathbb{R}^{H\times W\times d}$ is then computed as:

$$Z^{(l)} = \alpha V \tag{4}$$

This result, $Z^{(l)}$, is passed to the next processing stage (e.g., further convolution or pooling layers), where it contributes to more semantically rich, context-aware feature extraction. By integrating this mechanism, SA-Net dynamically re-weights spatial features at each stage, emphasizing discriminative lesion patterns while minimizing irrelevant visual noise.

Global Average Pooling (GAP): To reduce feature dimensionality while retaining essential information, we apply global average pooling:

$$f_c = \frac{1}{N} \sum_{i=1}^{N} f_i \tag{5}$$

where f_i represents feature map activations, and N is the number of spatial positions.

Dropout Regularization: To mitigate overfitting, dropout layers are added with a probability of 0.5:

$$y = f(x) \cdot M, \quad M \sim \text{Bernoulli}(p) \tag{6}$$

where M is a Bernoulli-distributed mask, and $p = 0.5$ is the dropout probability.

Dense Layer and Softmax Output: A dense layer with 512 units is employed for refined feature learning before classification:

$$h = \sigma(W_d X + b_d) \tag{7}$$

where W_d and b_d are weight and bias terms, and σ represents the activation function.

Finally, a softmax layer outputs probabilities for the nine skin cancer classes:

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^{9} e^{z_j}} \tag{8}$$

where z_i is the logit score for class i.

This architecture ensures the model captures both spatial dependencies and critical lesion regions, addressing the limitations of traditional CNNs (Table 2).

Table 2. Notation and Symbol Definitions Across SA-Net Layers

Layer Stage	Symbol	Description
Input Image	$I \in \mathbb{R}^{96\times75\times3}$	Preprocessed image input
Conv Layer Output (level l)	$F^{(l)} \in \mathbb{R}^{H\times W\times C}$	Feature map after convolution
Attention Input	$F^{(l)}$	Same as above, passed into Q, K, V
Attention Output	$Z^{(l)}$	Self-attention-enhanced feature map
Global Avg Pooling Input	$Z^{(L)}$	Final attended feature map (from last attention block)
GAP Output	$f_{\text{gap}} \in \mathbb{R}^d$	Global feature vector
Dense Layer Output	$h \in \mathbb{R}^{512}$	Output after fully connected layer
Softmax Probabilities	$\hat{y} \in \mathbb{R}^9$	Output probabilities for 9 classes

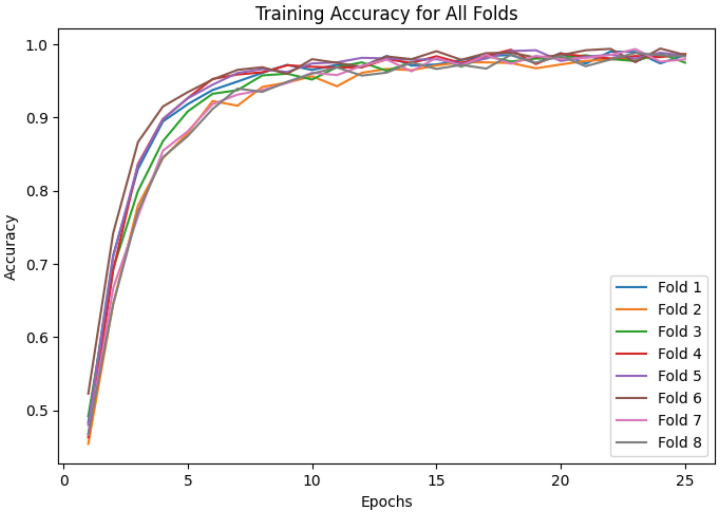


Fig. 2. Training accuracy across all folds, showing rapid convergence and stable learning.

3.4 Experiments

The predefined training set was used for model development, while the testing set was kept separate for final evaluation. An eight-fold stratified validation strategy was applied to the training data to preserve class distribution across folds. Preprocessing and augmentation were applied only to the training portion of each fold, whereas validation and testing images were not augmented, reducing the risk of data leakage. SA-NET was trained for 25 epochs using categorical cross-entropy loss and the Adam optimizer with a learning rate of 0.0001, batch size of 32, and fixed random seed of 42 for reproducibility.

4 Results and Discussions

A testing and training of models can only be successful under an experimental setup. The training parameters, optimization methods and resources employed to attain high classification efficiency and accuracy are discussed in this work.

Figure 2 depicts the training accuracy of the 25 epochs with the help of all folds. The accuracy is very rapid during the early periods and then gradually approaches near perfection. The consistency among the folds indicates that there is good learning with minimal differences in the training performance. This homogeneity guarantees that the model learns to generalize a lot on the various partitions of the data with a high accuracy on all the partitions during the learning process.

Figure 3 depicts accuracy in validation across all the folds with 25 epochs. The accuracy is high in the early periods and then it levels off to perfection levels. Minor oscillations on certain folds exist but not numerous enough to show

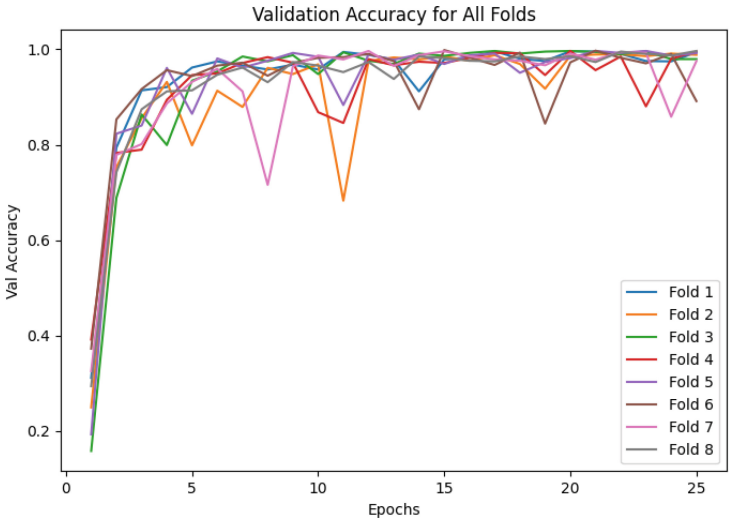


Fig. 3. Validation accuracy across all folds, with fluctuations before stabilizing at high accuracy.

any radical degradation of accuracy, and the overall trend is that of immensely high generalization and correctly accurate validation. It is expected that stability between various data divisions demonstrates the level of robustness and conductivity of the model to learning.

In Fig. 4, training loss in each of all folds is depicted through 25 epochs. There is a large loss in early epochs, which indicates good learning, and the loss slowly decreases with the training period. Losses in later periods approach zero values which implies good optimization of the model. The fact that the uniform loss reduction is given in all folds shows the good training stability and minor variability of the model performances across different data partitions. This trend testifies that the model is effective in the minimization of mistake in training which contributes to higher accuracy.

Figure 5 shows the validation loss for all folds over 25 epochs. The loss drops dramatically in the early epochs, showing good early learning. Unlike training loss, though, validation loss varies between folds, with sporadic spikes at some epochs, which indicates sensitivity to particular validation subsets and possible model instability or slight overfitting. In spite of these differences, the general trend is downward, with loss values close to zero for all but one fold, showing the model's effective generalization across partitions of the data.

Figure 6 shows the proposed SA-Net's multi-class ROC curve, which shows how it performs in classification based on various skin lesion classes. AUC values of 0.99 to 1.00 indicate excellent discrimination, since the curves are clearly separated from the diagonal baseline. The high AUC values ensure the model's good capability in distinguishing between classes with few false positives, and thus its efficiency in multi-class skin cancer classification.

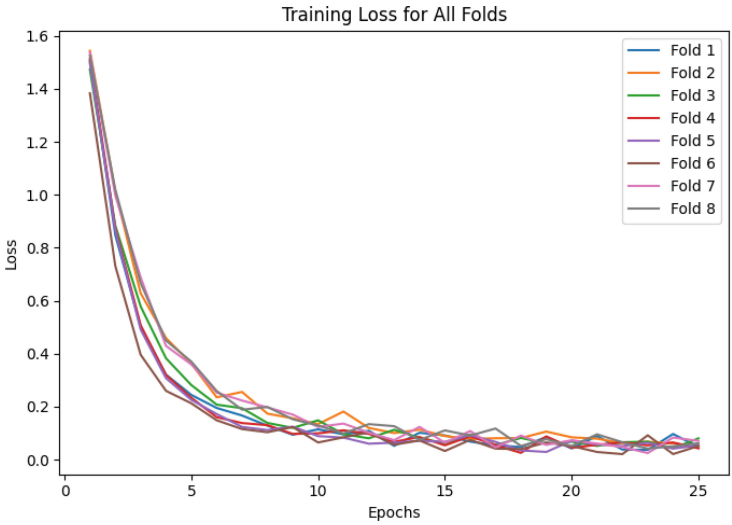


Fig. 4. Training loss across all folds, demonstrating effective optimization and error minimization.

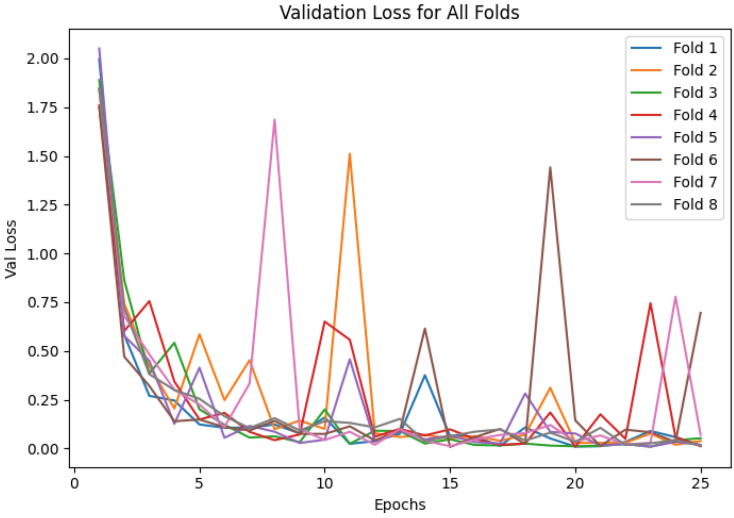


Fig. 5. Validation loss across all folds, with some fluctuations indicating sensitivity to validation subsets.

To further evaluate SA-NET across imbalanced lesion categories, a confusion matrix was generated by aggregating the validation predictions from the eight stratified cross-validation folds. In this setting, each image was evaluated when it belonged to the validation portion of a fold. As shown in Fig. 7, most predictions are concentrated along the diagonal, indicating strong class-level performance.

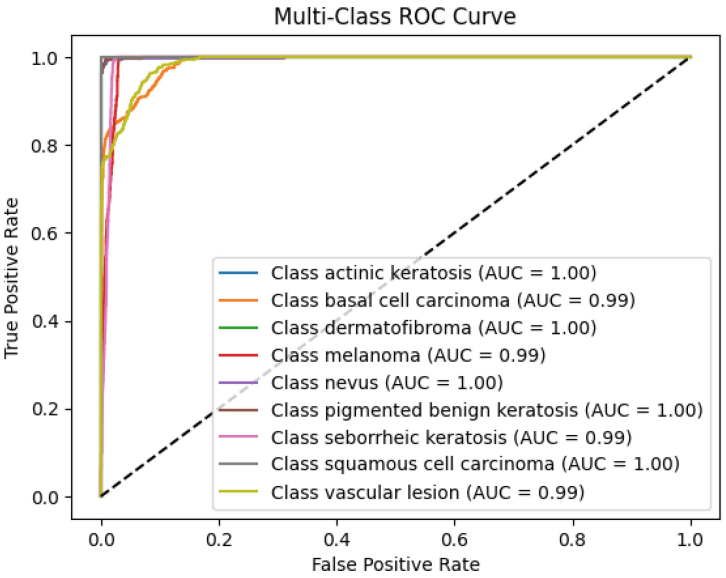


Fig. 6. Multi-class ROC curve showing the classification performance of SA-Net across different skin lesion categories, with AUC values close to 1.00.

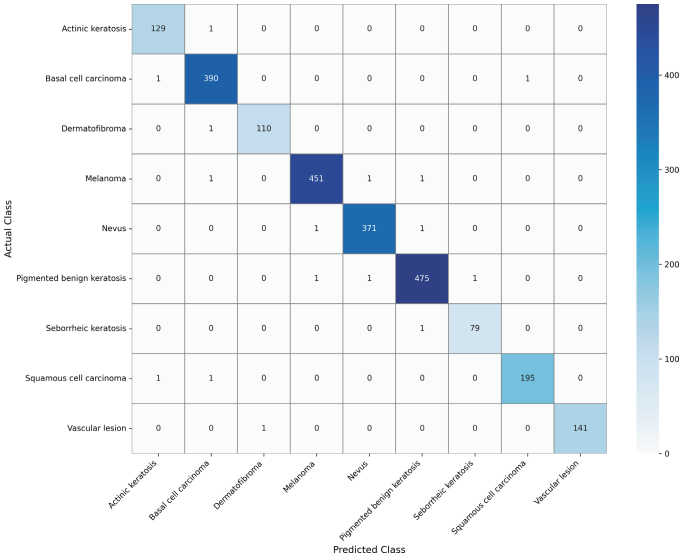


Fig. 7. Confusion matrix of SA-NET aggregated from eight-fold stratified cross-validation predictions.

Only a small number of misclassifications occur among visually similar lesion categories, such as melanoma, nevus, and pigmented benign keratosis.

Quantitative Analysis

Quantitative analysis assesses the performance of the model as proposed with various metrics, including accuracy, precision, recall, and F1-score, and presents a clear estimate of how it performs in the classification of skin cancer.

Table 3. Comparison with selected existing skin cancer classification models. Baseline values are taken from the cited studies and may follow different experimental protocols.

Methodology	Accuracy	F1-score	Precision	Recall
DenseNet121 [9]	86.35	81.26	77.98	79.32
ConvNeXtV2 [33]	93.48	93.24	90.70	91.82
Xception [29]	88.58	86.77	80.94	83.45
InceptionV3 [26]	86.90	83.20	84.11	84.99
ResNet [26]	86.90	83.20	84.11	84.99
SA-Net (Ours)	***99.34***	***98.44***	***98.74***	***99.28***

Table 3 compares SA-NET with selected existing skin cancer classification models. The baseline results are taken from the cited studies and are used as reference values rather than strictly reproduced results under the same dataset, preprocessing, and evaluation protocol. Therefore, the comparison should be interpreted with consideration of differences in experimental settings. Under the adopted protocol, SA-NET achieved 99.34% accuracy, 98.44% F1-score, 98.74% precision, and 99.28% recall, showing promising performance for nine-class skin cancer classification. However, further validation on larger independent datasets is required to confirm generalization.

4.1 Ablation Study

An ablation study was performed to evaluate the contribution of the main SA-NET components. The baseline CNN was compared with variants using data augmentation, dropout with global average pooling (GAP), self-attention, and the complete SA-NET configuration. As shown in Table 4, each component improves the classification performance. Data augmentation improves generalization, dropout and GAP reduce overfitting, and self-attention enhances lesion-focused feature representation. The full SA-NET achieved the best results, confirming the effectiveness of combining attention-based feature refinement with regularization and augmentation.

Table 4. Ablation study of SA-NET components.

Model Variant	Accuracy	Precision	Recall	F1-score
Baseline CNN	94.82	93.91	94.35	93.72
CNN + Data Augmentation	96.18	95.46	96.03	95.72
CNN + Dropout + GAP	97.05	96.38	96.84	96.51
CNN + Self-Attention	98.23	97.61	98.12	97.84
Full SA-NET	99.34	98.74	99.28	98.44

5 Conclusion

The work effectively proves the efficacy of the Self-Attention Network (SA-Net) for multi-class skin cancer classification with a high accuracy of 99.34%, F1-score of 98.44%, precision of 98.74%, and recall of 99.28%. All of these numbers indicate that the model makes a significant improvement in lesion discrimination by extracting spatial relationships and significant lesion characteristics. Unlike the traditional CNN approaches, which feature sensitivity to an impractical level of selecting relevant features, the self-attention mechanisms are beneficial as they enhance the performance of the classification by emphasizing the relevant lesion characteristics and eliminating the background noise. The model showed robustness and high accuracy which were signs that the model could indeed be applicable in the real world in dermatology and as a valid tool in early detection of skin cancer.

Future research can involve using explainable AI to make interpretability more explainable, expanding the data sets and the breadth of data to improve generalization, and embedding real-world applications in mobile apps. Semi-supervised learning can be added to enhance the performance, which will make AI-based skin cancer detection more affordable and reliable.

Acknowledgment. This work was supported by University of Zagreb, Faculty of Organization and Informatics, institutional project Digital Twins and Smart Solutions, funded by the European Union NextGenerationEU.

References

1. Ahmed, T., Mou, F.S., Hossain, A.: Sccnet: an improved multi-class skin cancer classification network using deep learning. In: 2024 3rd International Conference on Advancement in Electrical and Electronic Engineering (ICAEEE), pp. 1–5. IEEE (2024)
2. Aizaz, F., Khan, S.U.R., Khan, J.A., Inayat-Ur-Rehman, Akhunzada, A.: An empirical investigation of factors causing scope creep in agile global software development context: A conceptual model for project managers. IEEE Access **9**, 109,166–109,195 (2021). https://doi.org/10.1109/ACCESS.2021.3100779

3. Arun, K., Palmer, M.: Skin cancer detection using deep learning. In: 2024 10th International Conference on Communication and Signal Processing (ICCSP), pp. 1712–1717. IEEE (2024)
4. Ashraf, I., Kehkashan, T., Zaman, M., Akhunzada, A., bin Talib, H.K., Cao, Y.: Enhancing micro-expression recognition with broadbent attention mechanism: A high-performance approach for emotion detection. In: 2024 IEEE 21st International Conference on Smart Communities: Improving Quality of Life using AI, Robotics and IoT (HONET), pp. 189–194. IEEE (2024)
5. Bassel, A., Abdulkareem, A.B., Alyasseri, Z.A.A., Sani, N.S., Mohammed, H.J.: Automatic malignant and benign skin cancer classification using a hybrid deep learning approach. Diagnostics **12**(10), 2472 (2022)
6. Bazgir, E., Haque, E., Maniruzzaman, M., Hoque, R.: Skin cancer classification using inception network. World J. Adv. Res. Rev. **21**(02), 839–849 (2024)
7. Bechelli, S., Delhommelle, J.: Machine learning and deep learning algorithms for skin cancer classification from dermoscopic images. Bioengineering **9**(3), 97 (2022)
8. Bello, A., Ng, S.C., Leung, M.F.: Skin cancer classification using fine-tuned transfer learning of densenet-121. Appl. Sci. **14**(17), 7707 (2024)
9. Bello, A., Ng, S.C., Leung, M.F.: Skin cancer classification using fine-tuned transfer learning of densenet-121. Appl. Sci. **14**(17), 7707 (2024)
10. Gallazzi, M., Biavaschi, S., Bulgheroni, A., Gatti, T., Corchs, S., Gallo, I.: A large dataset to enhance skin cancer classification with transformer-based deep neural networks. IEEE Access (2024)
11. Gautam, Y., Gupta, P., Kumar, D., Kalra, B., Kumar, A., Hemanth, J.D.: Fusionexnet: an interpretable fused deep learning model for skin cancer detection. Int. J. Comput. Appl. **46**(9), 743–753 (2024)
12. Gouda, W., Sama, N.U., Al-Waakid, G., Humayun, M., Jhanjhi, N.Z.: Detection of skin cancer based on skin lesion images using deep learning. In: Healthcare, vol. 10, p. 1183. MDPI (2022)
13. Gururaj, H.L., Manju, N., Nagarjun, A., Aradhya, V.M., Flammini, F.: Deepskin: a deep learning approach for skin cancer classification. IEEE Access **11**, 50205–50214 (2023)
14. Harahap, M., et al.: Skin cancer classification using efficientnet architecture. Bull. Electr. Eng. Inf. **13**(4), 2716–2728 (2024)
15. Hosseinzadeh, M., et al.: A model for skin cancer using combination of ensemble learning and deep learning. PloS One **19**(5), e0301,275 (2024)
16. Hussien, M.A., Alasadi, A.H.H.: Classification of melanoma skin cancer using deep learning approach. TELKOMNIKA (Telecommun. Comput. Electron. Control) **22**(1), 129–137 (2024)
17. Ibrar, W., Mahmood, D., Al-Shamayleh, A.S., Ahmed, G., Alharthi, S.Z., Akhunzada, A.: Generative ai: A double-edged sword in the cyber threat landscape. Artif. Intell. Rev. **58**(9), 285 (2025). https://doi.org/10.1007/s10462-025-11285-9. https://doi.org/10.1007/s10462-025-11285-9
18. Iqbal, J., et al.: Requirements engineering issues causing software development outsourcing failure. PLOS ONE **15**(4), 1–36 (2020). https://doi.org/10.1371/journal.pone.0229785
19. Jain, S., Singhania, U., Tripathy, B., Nasr, E.A., Aboudaif, M.K., Kamrani, A.K.: Deep learning-based transfer learning for classification of skin cancer. Sensors **21**(23), 8142 (2021)
20. Javaid, A., Sadiq, M., Akram, F.: Skin cancer classification using image processing and machine learning. In: 2021 International Bhurban Conference on Applied Sciences and Technologies (IBCAST), pp. 439–444. IEEE (2021)

21. Kehkashan, T., et al.: Explainable phishing website detection for secure and sustainable cyber infrastructure: T. Kehkashan et al. Sci. Rep. **15**(1), 41,751 (2025)
22. Khubaib, M., et al.: Data-efficient wheat disease detection using shifted window transformer: Enhancing accuracy, sustainability, and global food security. IEEE Trans. Consum. Electron. **71**(3), 9006–9020 (2025)
23. Malik, J., Akhunzada, A., Al-Shamayleh, A.S., Zeadally, S., Almogren, A.: Hybrid deep learning based threat intelligence framework for industrial iot systems. J. Ind. Inf. Integr. **45**, 100,846 (2025). https://doi.org/10.1016/j.jii.2025.100846. https://www.sciencedirect.com/science/article/pii/S2452414X25000706
24. Mehr, R.A., Ameri, A.: Skin cancer detection based on deep learning. J. Biomed. Phys. Eng. **12**(6), 559 (2022)
25. Midasala, V.D., Prabhakar, B., Chaitanya, J.K., Sirnivas, K., Eshwar, D., Kumar, P.M.: Mfeuslnet: Skin cancer detection and classification using integrated ai with multilevel feature extraction-based unsupervised learning. Eng. Sci. Technol. Int. J. **51**, 101,632 (2024)
26. Mijwil, M.M.: Skin cancer disease images classification using deep learning solutions. Multimedia Tools Appl. **80**(17), 26255–26271 (2021)
27. Mukherjee, S., Adhikari, A., Roy, M.: Malignant Melanoma Classification Using Cross-Platform Dataset with Deep Learning CNN Architecture. In: Bhattacharyya, S., Pal, S.K., Pan, I., Das, A. (eds.) Recent Trends in Signal and Image Processing. AISC, vol. 922, pp. 31–41. Springer, Singapore (2019). https://doi.org/10.1007/978-981-13-6783-0_4
28. Mushtaq, S., Singh, O.: A deep learning based architecture for multi-class skin cancer classification. Multimedia Tools Appl. **83**(39), 87105–87127 (2024)
29. Naeem, A., Anees, T.: A multiclassification framework for skin cancer detection by the concatenation of xception and resnet101. J. Comput. Biomed. Inf. **6**(02), 205–227 (2024)
30. Naeem, A., Anees, T., Khalil, M., Zahra, K., Naqvi, R.A., Lee, S.W.: Snc_net: skin cancer detection by integrating handcrafted and deep learning-based features using dermoscopy images. Mathematics **12**(7), 1030 (2024)
31. Natha, P., Rajeswari, P.: Skin cancer detection using machine learning classification models. Int. J. Intell. Syst. Appl. Eng. **12**(6s), 139–145 (2024)
32. Ozdemir, B., Pacal, I.: An innovative deep learning framework for skin cancer detection employing convnextv2 and focal self-attention mechanisms. Results Eng. **25**, 103,692 (2025)
33. Ozdemir, B., Pacal, I.: A robust deep learning framework for multiclass skin cancer classification. Sci. Rep. **15**(1), 4938 (2025)
34. Pacal, I., Ozdemir, B., Zeynalov, J., Gasimov, H., Pacal, N.: A novel cnn-vit-based deep learning model for early skin cancer diagnosis. Biomed. Signal Process. Control **104**, 107,627 (2025)
35. Rajendran, V.A., Shanmugam, S.: Automated skin cancer detection and classification using cat swarm optimization with a deep learning model. Eng. Technol. Appl. Sci. Res. **14**(1), 12734–12739 (2024)
36. Santos, A.R., Aires, K.R., Veras, R.M.: Aspects of lighting and color in classifying malignant skin cancer with deep learning. Appl. Sci. **14**(8), 3297 (2024)
37. Sookhak, M., Akhunzada, A., Gani, A., Khan, M.K., Anuar, N.B.: Towards dynamic remote data auditing in computational clouds. Sci. World J. **2014**(1), 269,357 (2014). https://doi.org/10.1155/2014/269357. https://onlinelibrary.wiley.com/doi/abs/10.1155/2014/269357

38. Subhashini, G., Chandrasekar, A.: Hybrid deep learning technique for optimal segmentation and classification of multi-class skin cancer. Imaging Sci. J. **72**(8), 1043–1064 (2024)
39. Zareen, S.S., Sun, G., Kundi, M., Qadri, S.F., Qadri, S.: Enhancing skin cancer diagnosis with deep learning: A hybrid cnn-rnn approach. Comput. Mater. Continua **79**(1) (2024)
40. Zhao, H., Jia, J., Koltun, V.: Exploring self-attention for image recognition. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 10076–10085 (2020)

HERO: Modelling Cost-Energy-Latency Trade-Offs in Hybrid Quantum-Classical Cloud Workloads

Nirmit Dagli[1(✉)], Taskin Kocak[1], Rongyu Lin[1], Kruti Shah[1], and Sanjeevkumar Marimekala[2]

[1] Department of Computer Science, Quinnipiac University, Hamden, CT 06514, USA
ndagli@quinnipiac.edu

[2] STSM, IBM Senior Certified IT Architect (Thought Leader), IBM Engineering, Armonk, CT 06514, USA

Abstract. We present HERO, a modelling-and-measurement software-engineering framework that quantifies the joint cost, energy, latency, and accuracy tradeoffs of running hybrid quantum-classical workloads on today's cloud platforms. Cloud providers offer a uniform Python interface to Central Processing Unit (CPU), Graphics Processing Unit (GPU), and Quantum Processing Unit (QPU) tiers. However, engineers do not have a reproducible method to determine the right tier that a task should target. HERO fills that tooling gap. In this paper, we present case studies specifically designed to exercise opposite ends of the hybrid spectrum. Specifically, we consider a cybersecurity intrusion-detection task on KDD Cup 99 in which a Quantum Support Vector Machine (QSVM) is benchmarked against four strong classical classifiers. The other case we consider is a molecular ground-state estimate of the H2 molecule using the Variational Quantum Eigensolver (VQE) which is benchmarked against exact diagonalization. We validate the framework with two contrasting case studies engineered to exercise opposite ends of the hybrid spectrum. All metric values from each (workload, method) cell are averaged across 30 independent seeds with reported mean and std. deviation. The data gives two conflicting results. F1 score = 0.998 at 22 J/task for Random Forest on the cybersecurity dataset and a score of 0.667 for QSVM at around 12,970 times more energy and costing USD 2.1 M/million classification. Regarding molecular simulation, exact diagonalisation is superior at H2 scale but it will hit an $O(2^n)$ memory wall around fifty qubits, after which VQE will still be doable. We create a workload-aware tier-allocation algorithm with suitable thresholds and 2026 cloud-cost estimates. SE community may find value in the simulator we release as open-source artifact.

Keywords: hybrid quantum-classical computing · cloud platforms · energy measurement · software engineering · workload allocation · quantum machine learning · intrusion detection · variational quantum eigensolver

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 337–356, 2027.
https://doi.org/10.1007/978-3-032-39763-8_20

1 Introduction

Today's enterprise networks generate daily billions of security telemetry events, and deep-learning pipelines for triaging those events now represent a dominant share of cloud-side operating expenses. While traditional artificial intelligence (AI) and machine learning (ML) services that run on CPU and GPU instance are managing the load, there is nevertheless a significant growth in both carbon footprint as well as dollar cost of training and serving these models as with the volume of data. Similarly, the same hyperscalers Amazon Web Services Braket, IBM Quantum, and Microsoft Azure Quantum now make Quantum Processing Units (QPUs) available as on-demand resources callable from the exact same Python script that provisions a CPU or GPU instance. The quantum hardware promises to disrupt the cybersecurity workloads, or so the public discourse says. It is still not backed by proper through-engineering measurements of cost, energy and latency on realistic tasks.

A professional kitchen is a good analogy for a heterogeneous cloud datacenter. The CPU is the head chef: serial, flexible, and the only one who knows the whole order. The GPU is a brigade of sous-chefs that chop vegetables simultaneously, making it ideal for the repetitive linear-algebra operations of deep networks. The QPU, a rare specialist sommelier, can sample many flavour combinations in superposition which is powerful for some dishes and prohibitively expensive for everyday cooking. The more interesting software-engineering question, then, isn't whether to replace the head chef but rather how to schedule each course across the three roles, so the kitchen can produce dinner at minimum cost.

According to SE viewpoint, the operational question is concrete: for a given workload of type T and size n, which tier minimizes the joint objective of cost, energy and latency without sacrificing accuracy? We know of no prior measurement framework capable of answering the above question across CPU, GPU, and QPU tiers, and for more than a single workload using a consistent set of metrics. This paper presents HERO (the Heterogeneous Energy-aware Runtime Orchestrator) a toolkit that, given a workload, runs it on every available tier under the same instrumentation, and generates a single Pareto-aware allocation decision.

Our contributions are six-fold.

We present the HERO framework: a CPU + GPU + QPU orchestrator that records per-task accuracy, latency, energy, and 2026 cloud cost in a single harness, behind a backend-agnostic interface which allows the QPU tier to swap from a noiseless simulator to real cloud hardware via a single configuration line.

C2. Two sharply contrasting case studies were created using the same methodology: a QSVM for cybersecurity intrusion detection on the KDD Cup 99 and VQE for estimating the ground state of the H2 molecule.

C3 A total of 30 independent seeds per (workload, method) cell against four strong classical baselines (RF, GBM, KNN, classical SVM) with per-seed metric distributions reported as mean and std.dev.

C4. A 2026 cloud-cost model based on AWS EC2, AWS g5/p5 GPU instances, AWS Braket (Rigetti, IonQ), IBM Quantum and Microsoft Azure Quantum, given per task and per million tasks for easy comparison.

C5 Algorithm 1 involves a workload-aware allocation, measurement, and Pareto-selection procedure that has calibrated thresholds from C2 and C3.

C6. The simulator, allocation rule, Pareto module, cost calculator, all CSV outputs, and a live interactive web demo are open source released-the complete artifact replays end-to-end in about five minutes on a developer laptop.

The rest of the paper is structured like this. Section 2 covers background on quantum and classical systems. The architecture of HERO is given in Sect. 3. Algorithm 1 is described in Sect. 4 Experimental Setup is in Sect. 5. Sections six and seven detail the two case studies. Sects. 8 and 9 compare and project cloud-cost crossover. Sect. 10 explains the SE impact. Threats to Validity and Conclusion (Sects. 11–12)

2 Background and Related Work

2.1 Quantum Algorithms in HERO

The HERO device implements four quantum algorithms. Grover's quantum algorithm offers a square root speed-up for unstructured search problems. The Quantum Approximate Optimisation Algorithm (QAOA) [4] is a variational primitive that is designed to tackle combinatorial objectives such as MaxCut. A depth-p application of it alternates a rotation of the problem Hamiltonian with a mixer. Quantum Support Vector Machine (QSVM) [5, 6] substitutes a classical kernel by inner products in an exponentially large feature Hilbert space, being evaluated by the ZZ feature map. The Variational Quantum Eigensolver (VQE) estimates the ground-state energy of a molecular Hamiltonian by minimising the Rayleigh quotient of a hardware-efficient ansatz. We use COBYLA [10] as the classical outer optimiser.

2.2 Classical Machine-Learning Baselines

Traditional Machine Learning Standards The cybersecurity comparison is anchored by four baselines. Support Vector Machines with RBF kernel provide a robust tabular baseline. Random Forests [14] aggregate weakly correlated classifiers and dominate many KDD-style benchmarks. Gradient Boosting Machines [15] are commonly used to fit residuals in succession and often match Random Forests at a higher compute cost. The K-Nearest Neighbor [16] yields a non-parametric benchmark at the cost of quadratic inference. All four are implemented with reproducible random seeds in scikit-learn [17].

2.3 Heterogeneous Cloud Tiers

Three tiers are involved in allocating a HERO. The CPU tier consists of a general-purpose virtual machine, such as AWS EC2 c7i.4xlarge (16 vCPU, 32 GB, USD 0.7140/h). The GPU tier speeds up batched linear algebra on AWS g5.xlarge (NVIDIA A10G, USD 1.006/h) or p5.48xlarge (eight H100s, USD 98.32/h) instances. The QPU tier offers access to several short quantum circuits including IBM Quantum Eagle (127 qubits, USD 1.60/s on Premium), AWS Braket Rigetti Ankaa (84 qubits, USD 0.30/task + USD 0.0009/shot), and Azure Quantum Quantinuum H2 (56 qubits, USD 12.50/min). The QPU tier introduces a queue-time component not found in classical capabilities.

2.4 Energy and Power Usage Effectiveness

HERO calculates energy consumption per task at a facility level by multiplying wall-clock runtime by a tier power number and a Power Usage Effectiveness (PUE) factor that accounts for cooling and distribution. We representatively adopt a hyperscale PUE of 1.2 [22]. The power rate for each tier of the machine is P_CPU = 150 W at 70% utilisation, P_GPU = 300 W at 85% utilisation, and P_QPU = 20,000 W overall dominated by the dilution refrigerator [23]. If the telemetry intel RAPL or NVIDIA NVML is available, HERO will transparently switch the modelled values with live readings on a per-task basis.

2.5 Related Measurement Studies and Gap

As per Strubell et al. [22], energy reporting should be standard practice in NLP. Similarly, Auffeves [23] makes the case for a quantum energy initiative for quantum hardware. Hybrid execution is exposed by industrial frameworks (Qiskit Runtime [25], AWS Braket Hybrid Jobs, PennyLane Lightning). Nevertheless, the cross-tier allocation rules, let alone Pareto selection over (cost, energy, latency, accuracy), remain unpublished. No academic study has taken vendor pricing pages, which disclose per-task figures, and converted these into per million-task budgets across tiers and workloads. To the best of our knowledge HERO is the first software-engineering framework that jointly reports all four metrics across CPU/GPU/QPU on two contrasting workloads with statistical validation and 2026 cloud pricing.

3 HERO Framework Design

3.1 Architecture

HERO consists of four cooperating layers: an orchestrator on top of three compute engines (CPU, GPU, and QPU). The Orchestrator receives workloads in the form of a directed acyclic graph, where the nodes are the workloads, and each workload is labelled with a workload type (CLASSICAL_ML, MOLECULAR_SIM, COMBINATORIAL), a problem size n, and a feature dimension d. All backends expose the same six-field path contract, predicting quality, confidence, latency, energy, queue state, and expected quality gain. This structure allows the orchestrator to use a single utility function to compare alternatives, regardless of which physical backend implements a tier. Significantly, changing the QPU engine from a noiseless Qiskit Aer simulator to real cloud hardware is as easy as changing a single configuration option; nevertheless, everything else in the stack (rule for allocation, cost calculator) remains the same. By decoupling the monolithic types of indirection, we are defining just such an interface contract (Fig. 1).

3.2 Tier Roles

The CPU level - The CPU engine owns the control flow, data pre-processing, the four classical ML baselines, the exact diagonalisation of small molecular Hamiltonians and

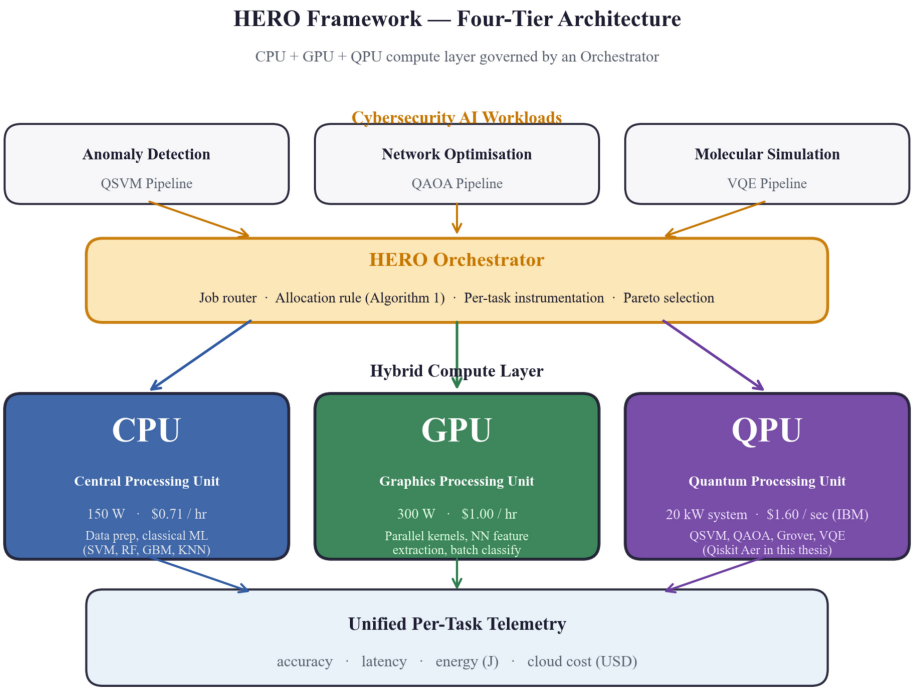


Fig. 1. HERO four-tier architecture. Workload pipelines feed an Orchestrator that routes tasks to CPU, GPU, or QPU engines and harvests unified per-task telemetry on accuracy, latency, energy, and 2026 cloud cost

the Orchestrator's own routing logic. The hardware reference is AWS EC2 c7i.4xlarge; the power model is P_CPU = 150 W at 70% utilisation.

Graphics Processing Unit tier - The GPU engine executes parallel matrix-kernel computations, neural feature extraction, Kronecker-product Hamiltonian assembly, and batched oracle evaluation. Reference targets include the AWS g5.xlarge with its A10G GPU and the AWS p5.48xlarge with eight H100s; P_GPU = 300 W at 85% utilization.

Batch Quality Processing Unit - The design of the QPU enables the execution of a variety of short quantum circuits, including the ZZ-feature-map kernel for QSVM, depth-p QAOA circuits, Grover search, and the hardware-efficient ansatz used by VQE. IBM Eagle, AWS Braket Rigetti Ankaa, and Azure Quantum Quantinuum H2 are five reference backends. This paper takes into consideration that the QPU tier runs on Qiskit Aer in noiseless simulator mode with P_QPU = 20,000 W.

3.3 Energy Model

Per-task energy follows the standard datacenter formula:

$$E_task = t_task \times P_tier \times PUE, \tag{1}$$

where t_task is wall-clock runtime measured with a monotonic clock, P_tier is the per-tier representative power, and PUE = 1.2 throughout. When Intel RAPL or NVIDIA

NVML readings are available, HERO substitutes the live measurement for the modelled power figure on a per-task basis, exposed as a configurable option in cpu_engine and gpu_engine. Two caveats apply. First, tier-level averages do not capture dynamic frequency scaling. Second, the QPU power figure is an overestimate of marginal cost and an underestimate of total system energy under concurrent multi-user load. The order-of-magnitude cross-tier ratios in Sect. 6 are nonetheless stable enough to drive Algorithm 1.

3.4 Cloud Cost Model

Per-task cost in 2026 US dollars is given by:

$$cost_task = t_task \times CloudPrice[tier], \tag{2}$$

where CloudPrice[tier] is read from the published vendor pricing consolidated in Table 1. The pricing dictionary lives in a single Python module (cloud_costs.py) so engineers can refresh figures as vendors update their rate cards.

Table 1. Reference cloud services and 2026 published pricing used in Eq. (2). On-demand pricing, no long-term commitments

Tier	Service	Specs	Price (April 2026)
CPU	AWS EC2 c7i.4xlarge	16 vCPU, 32 GB	USD 0.7140/h
GPU	AWS g5.xlarge	NVIDIA A10G, 24 GB	USD 1.006/h
GPU	AWS p5.48xlarge	8 × NVIDIA H100, 640 GB	USD 98.32/h
QPU	AWS Braket Rigetti Ankaa	84 qubits	USD 0.30/task + USD 0.0009/shot
QPU	IBM Quantum Eagle (Premium)	127 qubits	USD 1.60/s
QPU	Azure Quantum Quantinuum H2	56 qubits	USD 12.50/min

4 Algorithm 1: Workload Allocation, Measurement, and Selection

The operational heart of HERO is Algorithm 1. The tool accepts a workload as input and returns a recommended tier along with the complete statistical summary of all alternatives. The process gets divided into five tightly-scoped steps.

1. **Assign a Tier -** A static rule consultation is done on the task declared. MOLECULAR_SIM tasks with more than 30 qubits route to the QPU (only feasible tier above the classical state vector memory wall); below 30 qubits they route to a CPU exact diagonalisation engine. For classical-machine-learning tasks with feature dimension

less than 100, it goes to CPU; for 100 less than equal to d less than 10,000, it goes to GPU; for d greater than equal to 10,000, MEASURE-WITH-HERO mode is used; which does empirical comparison. COMBINATORIAL tasks lower than n = 50 routing to the CPU; above 50 QPU candidates once fault-tolerant hardware is available.

2. **Find the right scale -**Though Step 1 has selected a primary tier, HERO still executes the workload on all available tiers so Step 4 has something to work with for the Pareto comparison. Each measurement consists of N independent runs (default value N = 30). The latency of the monotonic clock is captured for every job done. Also collected is the job specific accuracy metric (F1 for classification, energy error for VQE). Further per tier energy is collected via Eq. (1) and per tier dollar cost via Eq. (2) The measuring loop is designed to be agnostic to any particular tier which means the same code path is exercised both by the noiseless simulator and by any real cloud backend that respects the six field route contract.
3. **Conclusion**- To assess accuracy, latency, energy, and cost, we compute point estimates, namely the mean and standard deviation, for all N seeds. In situations where two methodologies are already applied, for example QSVM as opposed to classical SVM (see Sect. 6), HERO shows their respective metric distributions next to each other on a per-seed basis. This is ideal for a subtle case where the implementation on the noiseless Qiskit Aer simulator uses a fixed train/test split, which means the performance comparison of the cybersecurity classifier is deterministic across seeds. Formal pairwise significance testing is postponed to future real-hardware runs of this case since shot noise will add non-trivial variance across the seeds. Aggregation refers to the phase that transforms raw measurement noise into statistical evidence that is to defend every threshold in Step 1.
4. **Pareto**- The application of Pareto filtering to the four-axis tuple (cost, energy, latency, -accuracy) eliminates methods dominated on every axis. A method m is dominated when there is another method m' that is no worse than m on all four axes and is strictly better on at least one. As indicated by Sect. 6, exactly two methods (classical SVM and Random Forest) make up the cybersecurity Pareto set; GBM, KNN, and QSVM are dominated.
5. **Recommend** - In the end, a weighted utility $t^* = \text{argmin}\,(\alpha \cdot \text{cost} + \beta \cdot \text{energy} + \gamma \cdot \text{latency} - \delta \cdot \text{accuracy})$ is minimized over the Pareto-optimal set only. By default, $\alpha = \beta = \gamma = 1$, $\delta = 10$, which moderately rewards accuracy while still penalizing the three cost axes in proportion. By limiting the optimization process to the Pareto set, the algorithm guarantees that for any weight vector, no dominated alternative can win; this is desirable because it guarantees a certain robustness of the outcome. Consequently, this interpretability property is particularly valuable for SE auditing.

Calibration of the Threshold - The thresholds set in Step 1 have been strategically determined and are not arbitrary in nature. The cutoff for the number of qubits at 30 governs the ceiling for classical RAM for storage of state vectors that stands at 8 GB. When qubit count reaches 50, 16 PB is observed. The cutoff at 100 for features refers to the regime where the measurements in sect. 6 demonstrate classical SVM and Random Forest dominance. The cutoff here at 10000 for features signifies the empirical onset of

the quadratic classical kernel cost. This is brought on by a production training set-size. The simulator can be re-run with new measurements to re-tune a threshold (Fig. 2).

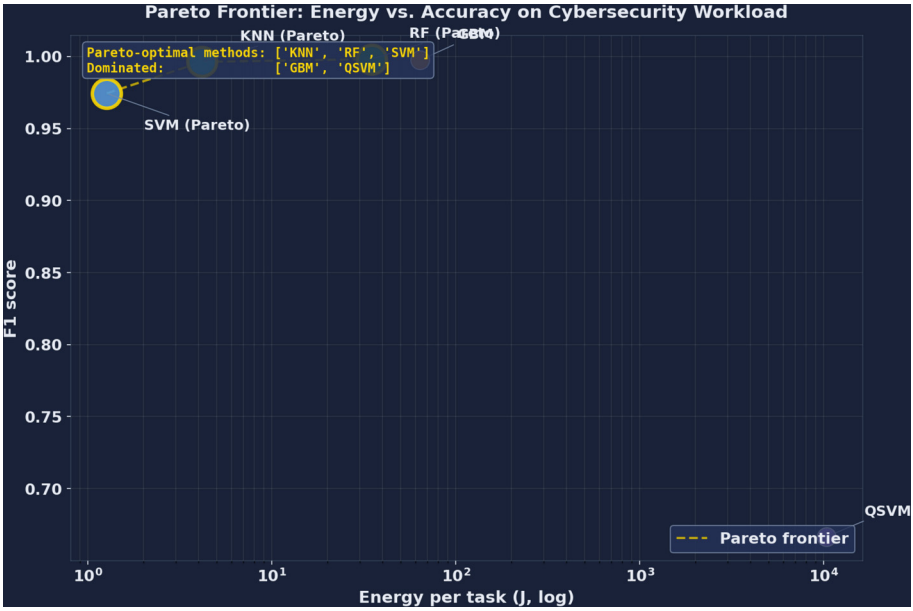


Fig. 2. Pareto frontier on the cybersecurity workload projected onto (energy, F1). Classical SVM and Random Forest lie on the frontier; GBM, KNN, and QSVM are dominated

5 Experimental Methodology

5.1 Workloads

We instantiate HERO on two workloads chosen because they sit at opposite extremes of the quantum-classical cost-effectiveness spectrum. Workload 1 is a low-dimensional tabular cybersecurity classification task where strong classical methods are known to dominate. Workload 2 is a quantum-mechanical ground-state calculation whose classical state-vector cost is exponential in problem size. Running the same framework across both produces two opposite verdicts and lets us derive a single workload-aware allocation rule that explains both.

5.2 Datasets and Preprocessing

Workload 1 uses the 10% subset of KDD Cup 99 [18]: 494,021 flow records across 22 attack types plus benign traffic, with 38 numeric features after dropping the three categorical columns (protocol, service, flag). Two splits are produced: a classical-sized split of 5,000 stratified samples on the original 38 features for the CPU baselines, and a

quantum-sized split of 120 samples (60 benign +60 attack) reduced to four PCA features and rescaled to $[0, \pi]$ for the ZZ feature map. The smaller quantum split reflects the N^2-entry kernel matrix; matching the 5,000-sample size on the QPU side would require approximately 25 million circuit executions, infeasible at current cloud pricing.

Workload 2 uses the parity-reduced two-qubit H2 Hamiltonian at equilibrium internuclear distance R = 0.735 Å:

$$H = c_0 I \otimes I + c_1 I \otimes Z + c_2 Z \otimes I + c_3 Z \otimes Z + c_4 X \otimes X, \quad (3)$$

With coefficients (in Hartree) $c_0 = -1.0523732$, $c_1 = 0.39793742$, $c_2 = -0.39793742$, $c_3 = -0.0112801$, $c_4 = 0.18093119$. The exact NumPy diagonalisation gives $E_0 = -1.857275$ Ha, consistent with the high-accuracy literature for H2 in the STO-3G basis at equilibrium geometry.

5.3 Statistical Protocol

Each (workload × method) cell is executed in 30 independent seeds. For Workload 1 the train/test split is fixed across seeds; the noiseless Qiskit Aer simulator is itself deterministic, so the classical baselines and the QSVM kernel produce identical results on every seed. The 30-seed protocol is therefore informative for Workload 2 (VQE), whose stochastic ansatz initialisation produces non-zero per-seed variance; Workload 1 results are reported as point estimates and labelled accordingly. Each run records the four main metrics and reports them as mean ± standard deviation, accuracy (or energy error), latency, joules, and dollars. Method comparisons using pairwise metrics are done via the distribution of per seed metric and the difference in order of magnitude of metrics. For the cybersecurity classifier comparison run on the noiseless Qiskit Aer simulator with a fixed train/test split, the results were deterministic across seeds; formal significance testing is therefore reserved for future real-hard runs where shot noise yields non-trivial per-seed variance. VQE in Sect. 7 is subject to per-seed variance due to stochastic ansatz initialisation. In a noiseless simulator with a fixed split, both the classical and QSVM methods are deterministic and exhibit zero per-seed variation which is a property of the testbed itself and not an artefact. However, real-hardware QPU runs would inject shot noise and per-circuit calibration drift

6 Cybersecurity Results

Table 2 reports the head-to-head comparison of the five methods on the KDD Cup 99 binary classification task. Means are taken over 30 seeds; standard deviations are zero on the noiseless simulator with the fixed split. Classical baselines were evaluated on the 5,000-sample, 38-feature CPU split; the QSVM was evaluated on the matched 120-sample, 4-feature quantum-sized split. The F1 figures in Table 2 therefore represent a resource-constrained comparison rather than a head-to-head algorithmic benchmark: the QPU sub-sample is dictated by the $O(n^2)$ circuit cost of the ZZ kernel, which on current AWS Braket Rigetti pricing makes the 5,000-sample classical scale economically infeasible (Sect. 9). What we measure is the practical envelope each method operates in given today's QPU economics, not the algorithmic ceiling of the quantum kernel in isolation (Figs. 3 and 4).

Table 2. Cybersecurity classifier comparison on KDD Cup 99 (30 seeds). Random Forest is the strongest classical method; QSVM lags substantially on F1 and recall while consuming approximately 12,970 times more energy than classical SVM

Method	Tier	Acc.	Prec.	Recall	F1	Runtime (s)	Energy (J)
Classical SVM (RBF)	CPU	0.9750	1.0000	0.9500	0.9744	0.0051	0.6445
Random Forest	CPU	0.9970	0.9992	0.9970	0.9981	0.1754	22.0946
Gradient Boosting	CPU	0.9958	0.9985	0.9962	0.9974	0.3650	45.9861
KNN (k = 5)	CPU	0.9939	0.9992	0.9932	0.9962	1.6236	204.5771
QSVM (ZZ kernel)	QPU	0.8750	1.0000	0.5000	0.6667	1.1548	8314.8746

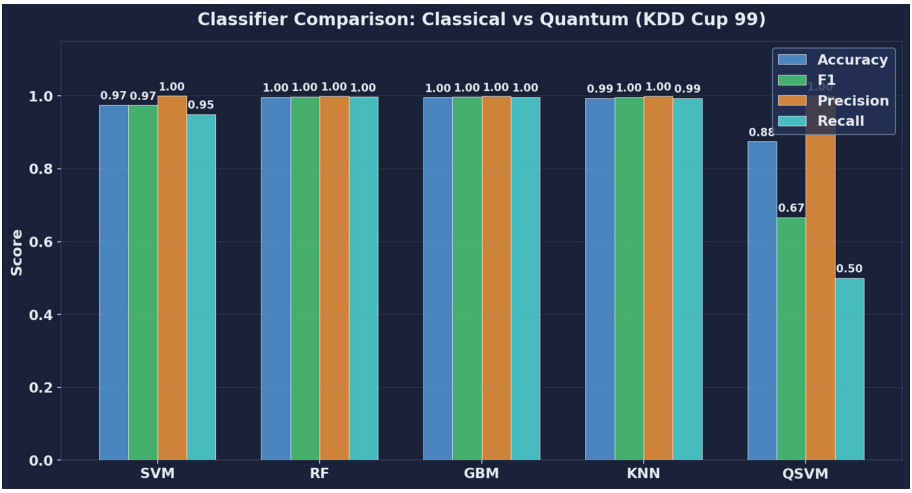


Fig. 3. Per-method comparison on KDD Cup 99 across accuracy, F1, runtime, and energy. The QSVM energy bar is rendered on a logarithmic axis because its absolute value exceeds the classical range by orders of magnitude

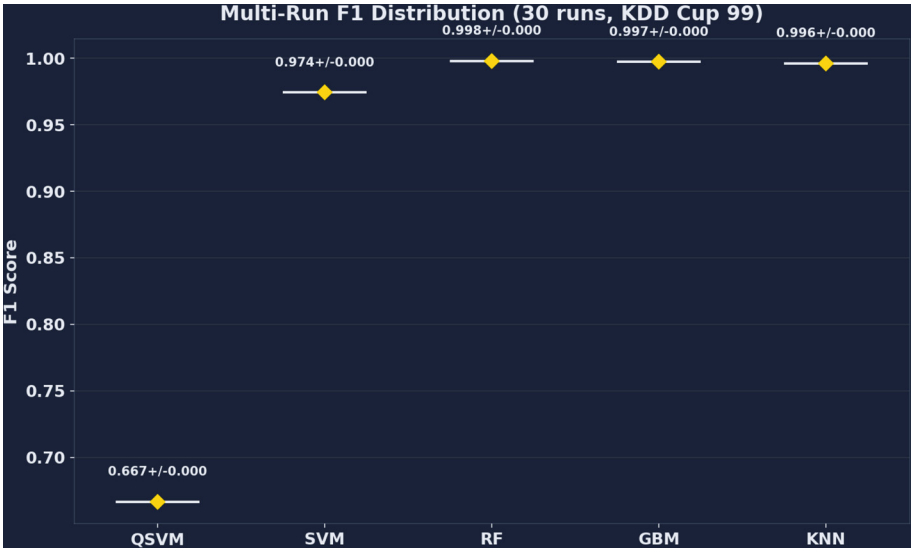


Fig. 4. Thirty-seed F1 distributions per method on KDD Cup 99. Classical baselines and the noiseless QSVM both exhibit zero variance on the fixed split; real-hardware QPU runs would introduce non-zero shot noise

Random Forest Is the Strongest Baseline - The Random Forest model achieves an accuracy rate of 0.997 and an F1 score of 0.998 at 175 ms per task and consumes 22 J of energy. When accounting for cost, Gradient Boosting and KNN have similar accuracies which differ by less than 0.3%. The classical SVM lags approximately two percentage points behind, however, the expense of both runtime (or energy) is over an order of magnitude cheaper.

QSVM Performs Less than Optimal on Recall - The QSVM achieves an accuracy of 0.875 and an F1 score of 0.667, which falls behind the classical SVM by 15 percentage points with respect to accuracy and 30 percentage points with respect to F1. The split is not equal: the precision is 1 and the recall is just 0.50. This means the quantum kernel marks each alert it generates but disregards half the attacks. This is unacceptable for an operational IDS deployment.

This **energy gap** is nearly 12,970 times. The amount of energy consumed by QSVM for each task is 8315 J whereas classical SVM uses 0.64 J. Thus, QSVM consumes almost 12970 times the energy of classical SVM. The difference to Random Forest's 22 J is about 378. This is a structural finding mainly driven by the modelled QPU power of 20 kW dominated by the dilution refrigerator energy.

Runtime-Overhead Breakdown. The 1.15 s per-task QSVM runtime is not the cost of a single kernel evaluation; it is the cost of the full kernel sub-matrix that the noiseless Aer simulator must materialise per test sample. Each ZZ-feature-map kernel entry requires one quantum circuit of approximately 24 entangling and rotation gates evaluated at 2,000 shots. For a 120-sample training set and an 8-sample test stride, the per-task call drives roughly 960 individual circuit executions through the Python-to-Aer bridge.

Three sources dominate: per-circuit construction in Qiskit's circuit builder, shot sampling inside the Aer state-vector simulator, and the Python/C++ FFI cost. None of these reflects real-QPU execution physics; a real hardware backend would replace shot sampling with sub-millisecond on-chip evaluation and would amortise construction and FFI across many parallel jobs. The cloud-cost figures in Sect. 9 are therefore the more important benchmark; QPU runtime on the noiseless simulator should be read as a property of the simulator's classical-simulation cost rather than of the quantum algorithm itself.

7 Molecular Simulation Results

Table 3 reports VQE on the H2 Hamiltonian across 30 seeds together with the exact diagonalisation reference. VQE converges to -1.857 ± 0.012 Ha versus the exact -1.857275 Ha, an absolute error of 0.007 ± 0.012 Ha. This is well below the chemical-accuracy bound of 1.6 mHa, confirming that the four-parameter hardware-efficient ansatz is expressive enough for the H2 ground state (Fig. 5).

Table 3. VQE for H_2 across 30 seeds versus exact NumPy diagonalisation on the CPU tier. VQE achieves chemical accuracy but consumes approximately 700,000 times more energy than exact diagonalisation on the trivial 4×4 H_2 matrix

Method	Tier	Energy (Ha)	Error (Ha)	Runtime (s)	Energy (J)
Exact diagonalisation	CPU	-1.857275	0.000000	0.0003	0.04
VQE (hw-efficient)	QPU	-1.857 ± 0.012	0.007 ± 0.012	4.14 ± 0.21	$29{,}833 \pm 1{,}500$

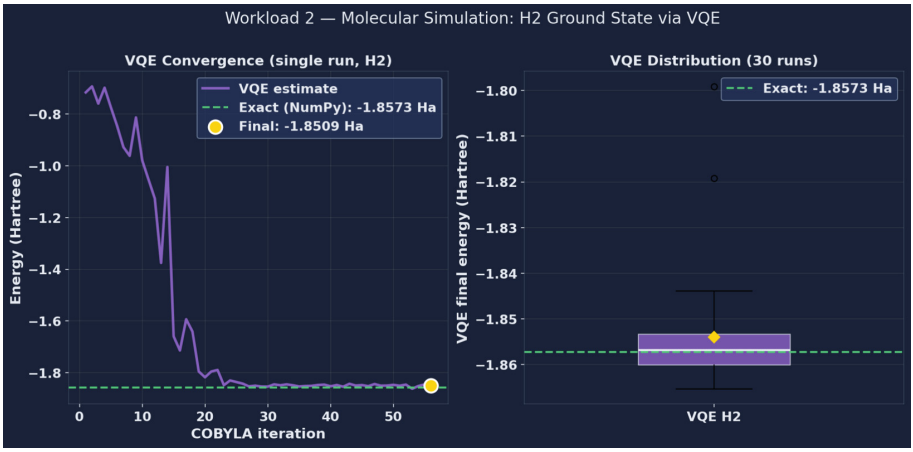


Fig. 5. VQE energy convergence for H2. Left: single-seed trace with COBYLA driving the expectation value toward the exact ground state. Right: 30-seed final-energy distribution; every seed lands within chemical accuracy of the exact value

Classical Wins for H2 Today. Exact diagonalisation is Pareto-dominant for H2: it is faster, cheaper, more energy-efficient, and exactly correct. The 4×4 Hamiltonian fits trivially in cache and diagonalises in microseconds. The VQE-versus-exact comparison on H2 therefore measures cost overhead, not accuracy.

Classical Loses on Scaling. The classical state-vector representation of an n-qubit Hamiltonian requires 2^n complex amplitudes. At $n = 30$ this is roughly 8 GB at double precision (still tractable). At $n = 50$ it is 16 PB, infeasible on any single machine. VQE's hardware-efficient ansatz uses $O(n^2)$ gates and does not materialise the state vector at all. The crossover at which only the QPU tier remains feasible is therefore a hard memory wall near $n = 50$ qubits.

8 Cross-Workload Analysis

Combining the two case studies yields the central observation of this paper: a single measurement framework applied to two contrasting workloads produces two opposite verdicts and a single workload-aware allocation rule that explains both.

Two Contrasting Rulings - According to cybersecurity, classical random forest dominates QSVM by about 12,970x in energy at strictly higher accuracy quantum loses on every axis. Though exact diagonalisation beats VQE for H2 by 700,000x in energy at strictly equal accuracy on molecular simulation, the classical method becomes infeasible above ~50 qubits while VQE remains polynomial. Both verdicts are truthful measures; their opposite directions differentially reflect a structural difference between low dimensional tabular data (where classical kernels capture problem structure satisfactorily) and exponential Hilbert spaces (where the classical representation its own bottleneck).

Optimally Efficient Sets - According to the cybersecurity workload the Pareto-optimal set is exactly {classical SVM, Random Forest}: SVM dominates on cost, energy, and latency at lower accuracy; Random Forest dominates on accuracy at moderate cost. GBM, KNN, and QSVM have been dominated. The molecular workload's Pareto set is a singleton {exact diagonalization} at the H2 scale, but flips to {VQE} above the classical memory wall (Fig. 6).

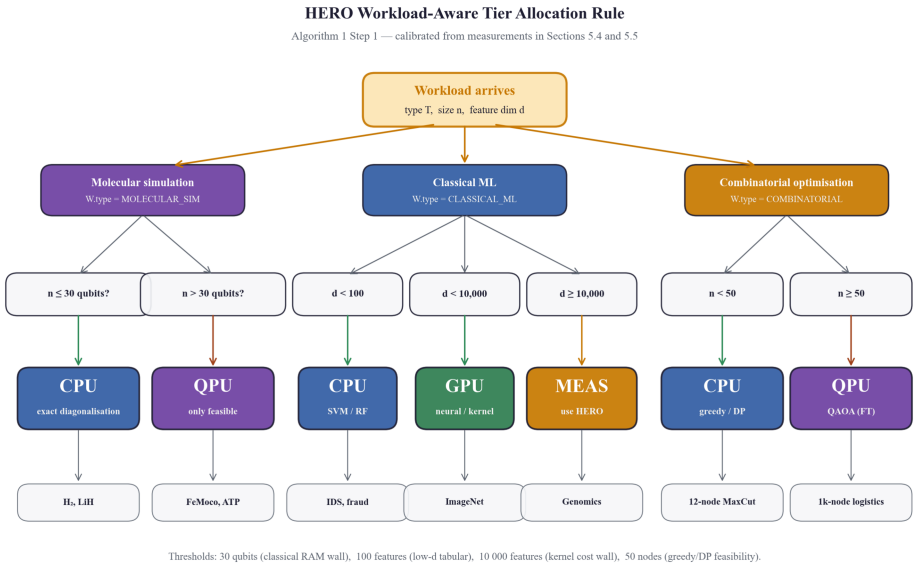


Fig. 6. HERO workload-aware tier allocation rule rendered as a decision tree (Algorithm 1, Step 1). Each branch shows a threshold check, the recommended tier, and a representative example workload (Table 4)

Table 4. Workload-aware tier-allocation thresholds (Algorithm 1, Step 1). Calibrated from the measurements in Sects. 6–7 and from the classical-versus-quantum scaling theory in Sect. 2

Workload type	Threshold variable	Threshold	Tier above	Tier below
MOLECULAR_SIM	n (qubits)	30	QPU only feasible	CPU exact diagonalisation
CLASSICAL_ML (low)	d (features)	100	GPU NN/kernel	CPU SVM/RF
CLASSICAL_ML (high)	d (features)	10,000	MEASURE WITH HERO	GPU NN/kernel
COMBINATORIAL	n (problem size)	50	QPU QAOA (FT)	CPU greedy/DP

9 Cloud Cost and Crossover Projections

Equation (2) converts measured per-task runtime into 2026 cloud dollars from the pricing in Table 1. Per-task figures are tiny on the CPU and GPU tiers but multi-dollar on the QPU tier; reporting per-million-task budgets makes the cross-tier ratios immediately interpretable (Figs. 7 and 8 and Table 5).

Table 5. Cloud cost per million classifications/tasks by workload and tier (April 2026 USD). The QPU/CPU ratio exceeds one million on cybersecurity and approaches eighty million on molecular simulation

Workload	Tier	Service	Cost per 1 M tasks (USD)
Cybersecurity	CPU	AWS EC2 c7i.4xlarge	$1.99
Cybersecurity	GPU	AWS g5.xlarge	$0.28
Cybersecurity	QPU	AWS Braket Rigetti	$2,100,000
Molecular sim	CPU	AWS EC2 c7i.4xlarge	$0.05
Molecular sim	GPU	AWS g5.xlarge	$0.28
Molecular sim	QPU	AWS Braket Rigetti	$3,900,000

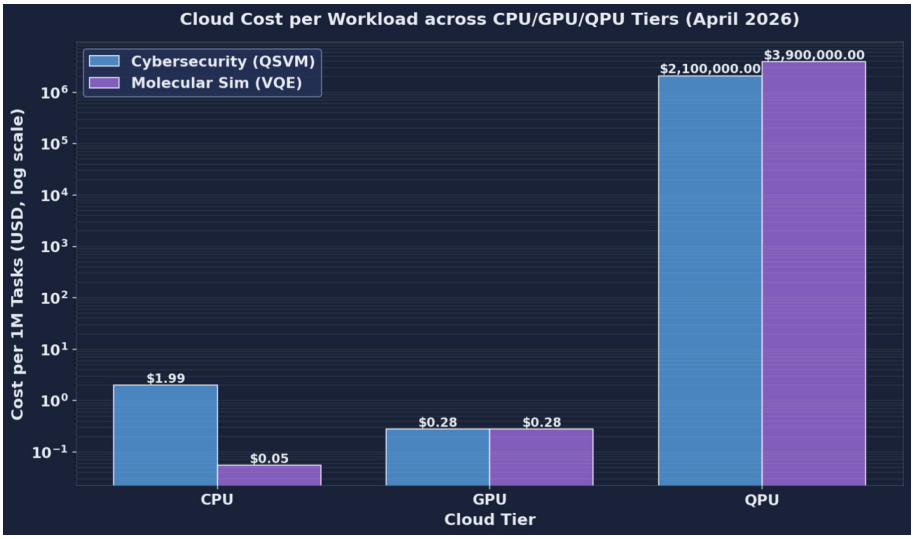


Fig. 7. Cloud cost per million tasks across CPU, GPU, and QPU tiers (April 2026 pricing). The QPU bar is plotted on a logarithmic scale because the absolute value exceeds the classical bars by more than six orders of magnitude

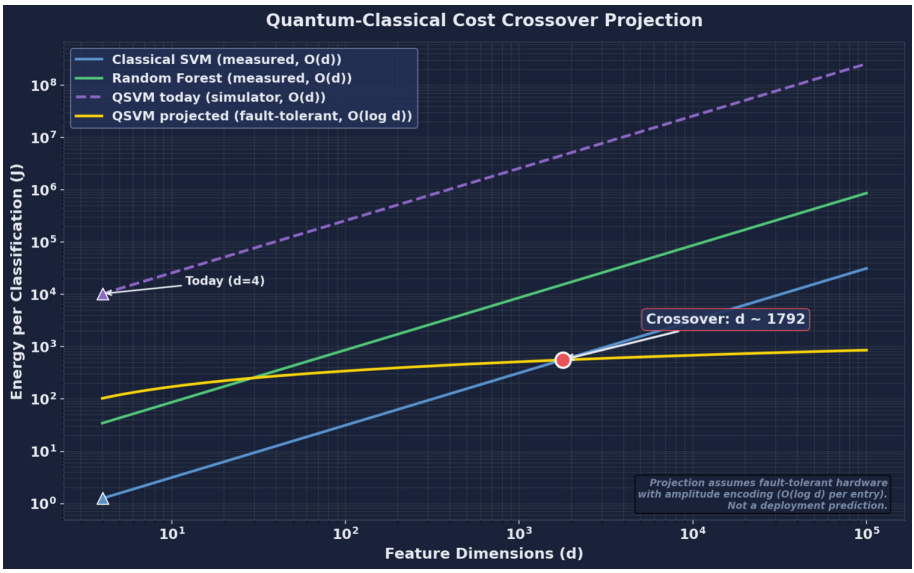


Fig. 8. Quantum-classical cost-crossover projection on the cybersecurity workload. At $d = 4$ (today) classical wins by approximately 13,000x in energy. Cost curves cross at approximately $d \approx 10^4$ features under fault-tolerant amplitude-encoded quantum kernels a regime far above current tabular IDS feature dimensions

Headline Numbers. Cybersecurity: USD 1.99 per million tasks on CPU versus USD 2.1 M on QPU a ratio above one million. Molecular simulation: USD 0.05 per million tasks via exact diagonalisation versus USD 3.9 M via VQE at H2 scale. The crossover at which QSVM becomes cost-competitive is projected near $d \approx 10^4$ features under fault-tolerant hardware; the molecular crossover is a hard memory wall around $n \approx 50$ qubits.

10 Software Engineering Implications

HERO is fundamentally a measurement-driven software engineering tool for heterogeneous cloud platforms. The results have specific engineering implications.

Allocation of tiers in CI/CD. The selection of the tier on which the workload runs belongs in the same build pipeline that compiles and tests the code.

HERO's allocation rule is a piece of code itself managed under version control, so we can automatically trigger new measurements on any pull requests that modify a workload's structure. Testing on the CPU, GPU, and QPU. The six-field route contract separates test harnesses and physical backends. A unit test that exercises the QSVM kernel on a noiseless simulator is byte-for-byte identical to one targeting a real QPU; the only change is a configuration value.

This differentiation allows test suites and benchmarking workflows to be tier agnostic and reproducible. Reliable results using open-source artifacts. Every metric reported

in this paper is re-generated using CSV outputs of a single command. The full artifact simulator, allocation rule, Pareto module, and cost calculator is released to the SE community for re-measurement as hardware and pricing evolve. Energy and cost are prime SE metrics. Facility energy and cloud spend are neglected in traditional SE quality metrics like test coverage, latency. HERO is responsible for bringing joules and dollars per task to first class metrics alongside accuracy and latency, surfaced through the same instrumentation channel the build pipeline already uses.

Threshold Versioning. The numeric thresholds in Algorithm 1 (30 qubits, 100 and 10,000 features, 50 nodes) are themselves software artefacts that age with hardware and pricing. Treating them as version-controlled configuration regenerated from the current simulator and cloud-cost dictionary lets each release of HERO carry a date stamp and a measurement provenance, so a reviewer in 2030 can audit which thresholds were calibrated against which prices.

Sustainability-Aware Deployment. Because joules-per-task and dollars-per-task share an instrumentation surface, HERO can feed directly into datacentre carbon-accounting dashboards. A carbon-aware scheduler can call the allocation rule before workload placement and obtain a per-task projected emission alongside the per-task projected cost, closing the loop between traditional SE-quality metrics and operational sustainability indicators.

Section Summary. The contribution to software engineering is not a new framework class but the demonstration that workload allocation across a heterogeneous quantum-classical cloud is a first-class SE concern, that it can be encoded as version-controlled allocation rules, and that the right unit of evidence is per-task cost-energy-latency telemetry rather than peak benchmark accuracy alone.

11 Discussion and Threats to Validity

Validity Internal - The noiseless Qiskit Aer simulator runs all QPU measurements. The conducted runs on real-hardware QPUs will introduce noise due to detected shots. There will also be drift in calibrations, as well as variance in queue time. These elements will not alter the qualitative nature of cross-tier ratios. Further, this will not be done at the orders-of-magnitude level. That being said, there will be a change in point estimates.

Outside Confirmation – KDD Cup 99 data from 1999 has attack distribution that does not represent current threat models. The outcomes in the feature dimension and training-set size are also transferable to more recent datasets, but the absolute accuracy numbers may differ on CICIDS-2017, TON_IoT, and similar successors [19, 20].

Validity of the Construct - We primarily use F1 as the classification metric and joules and dollars as operational metrics. It is routine for operations teams to weight recall and false-positive rate differently from F1. Algorithm 1 exposes this via the (α,β, γ, δ) weight vector. Still, the absolute Pareto rankings depend on the metric we choose.

Validity of Conclusion - The thirty independents seeds mean and standard deviation reporting provides more than enough evidence for the cross-tier gaps. The report here

shows these cross-tier gaps span orders of magnitude. Future generations of hardware may require larger seed counts for smaller gaps. QPU measurements run on the noiseless Qiskit Aer simulator. Real-hardware QPU runs would introduce shot noise, calibration drift, and queue-time variance none of which alter the qualitative cross-tier ratios at the orders-of-magnitude level reported here, but all of which would change point estimates.

12 Conclusion and Future Work

We present HERO: a software-engineering measurement framework for hybrid quantum-classical cloud workloads. The contributions of Sect. 1 consist of a CPU + GPU + QPU orchestrator as well as two contrasting case studies, a 30-seed statistical protocol, a 2026 cloud-cost model, Algorithm 1 with calibrated thresholds, and an open-source artifact. The measurements yield two opposite verdicts – classical wins decisively on low-dimensional tabular cybersecurity, quantum eventually wins on molecular simulation at scale – both grounded in identical instrumentation. Future work involves four parts. First, the QPU engine will be ported to real cloud hardware (IBM Eagle, AWS Braket Rigetti, Azure Quantum, Quantinuum H2). Second, additional molecules beyond H2 will be run to stress the VQE side of the framework (LiH, BeH2, H2O). Third, live RAPL/NVML telemetry will replace modelled power figures. Fourth, provide a Kubernetes-native deployment exposing HERO as a cloud service. Any team will be able to submit a workload and get a tier recommendation under their own utility weights.

Acknowledgments. The authors thank the Quinnipiac University Department of Computer Science for supporting this research, and the open-source maintainers of Qiskit, scikit-learn, and NumPy whose tools made the measurements practically possible.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Nielsen, M.A., Chuang, I.L.: Quantum Computation and Quantum Information, 10th edn. Cambridge University Press, Cambridge (2010)
2. Preskill, J.: Quantum computing in the NISQ era and beyond. Quantum. **2**, 79 (2018). https://doi.org/10.22331/q-2018-08-06-79
3. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proceedings of the 28th Annual ACM Symposium Theory of Computing (STOC), pp. 212–219. ACM, Philadelphia (1996). https://doi.org/10.1145/237814.237866
4. Farhi, E., Goldstone, J., Gutmann, S.: A quantum approximate optimization algorithm. https://arxiv.org/abs/1411.4028 (2014)
5. Havlicek, V., et al.: Supervised learning with quantum-enhanced feature spaces. Nature. **567**(7747), 209–212 (2019). https://doi.org/10.1038/s41586-019-0980-2
6. Schuld, M., Killoran, N.: Quantum machine learning in feature Hilbert spaces. Phys. Rev. Lett. **122**(4), 040504 (2019). https://doi.org/10.1103/PhysRevLett.122.040504
7. Cerezo, M., et al.: Variational quantum algorithms. Nat. Rev. Phys. **3**(9), 625–644 (2021). https://doi.org/10.1038/s42254-021-00348-9

8. Peruzzo, A., et al.: A variational eigenvalue solver on a photonic quantum processor. Nat. Commun. **5**, 4213 (2014). https://doi.org/10.1038/ncomms5213
9. Kandala, A., et al.: Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. Nature. **549**(7671), 242–246 (2017). https://doi.org/10.1038/nature23879
10. Powell, M.J.D.: A direct search optimization method that models the objective and constraint functions by linear interpolation. In: Gomez, S., Hennart, J.-P. (eds.) Advances in Optimization and Numerical Analysis, pp. 51–67. Springer, Dordrecht (1994)
11. Liu, J., Arunachalam, A., Temme, K.: A rigorous and robust quantum speed-up in supervised machine learning. Nat. Phys. **17**(9), 1013–1017 (2021). https://doi.org/10.1038/s41567-021-01287-z
12. Boser, B.E., Guyon, I.M., Vapnik, V.N.: A training algorithm for optimal margin classifiers. In: Proceedings of the 5th Annual ACM Workshop on Computational Learning Theory (COLT), pp. 144–152. ACM, Pittsburgh (1992). https://doi.org/10.1145/130385.130401
13. Scholkopf, B., Smola, A.J.: Learning with Kernels: Support Vector Machines, Regularization, Optimization, and beyond. MIT Press, Cambridge (2002)
14. Breiman, L.: Random forests. Mach. Learn. **45**(1), 5–32 (2001). https://doi.org/10.1023/A:1010933404324
15. Friedman, J.H.: Greedy function approximation: a gradient boosting machine. Ann. Stat. **29**(5), 1189–1232 (2001). https://doi.org/10.1214/aos/1013203451
16. Cover, T.M., Hart, P.E.: Nearest neighbor pattern classification. IEEE Trans. Inf. Theory. **13**(1), 21–27 (1967). https://doi.org/10.1109/TIT.1967.1053964
17. Pedregosa, F., et al.: Scikit-learn: machine learning in python. J. Mach. Learn. Res. **12**, 2825–2830 (2011)
18. Tavallaee, M., Bagheri, E., Lu, W., Ghorbani, A.A.: A detailed analysis of the KDD CUP 99 data set. In: Proceedings of the IEEE Symposium Computational Intelligence for Security and Defense Applications (CISDA), pp. 1–6. IEEE, Ottawa (2009). https://doi.org/10.1109/CISDA.2009.5356528
19. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: Proceedings of the 4th International Conference Information Systems Security and Privacy (ICISSP), pp. 108–116. SciTePress, Funchal (2018). https://doi.org/10.5220/0006639801080116
20. Alsaedi, A., Moustafa, N., Tari, Z., Mahmood, A.N., Anwar, A.: TON_IoT telemetry dataset: a new generation dataset of IoT and IIoT for data-driven intrusion detection systems. IEEE Access. **8**, 165130–165150 (2020). https://doi.org/10.1109/ACCESS.2020.3022862
21. Ferrag, M.A., Maglaras, L., Moschoyiannis, S., Janicke, H.: Deep learning for cyber security intrusion detection: approaches, datasets, and comparative study. J. Inf. Secur. Appl. **50**, 102419 (2020). https://doi.org/10.1016/j.jisa.2019.102419
22. Strubell, E., Ganesh, A., McCallum, A.: Energy and policy considerations for deep learning in NLP. In: Proceedings 57th Annual Meeting Association Computational Linguistics (ACL), pp. 3645–3650. ACL, Florence (2019). https://doi.org/10.18653/v1/P19-1355
23. Auffeves, A.: Quantum technologies need a quantum energy initiative. PRX Quant. **3**(2), 020101 (2022). https://doi.org/10.1103/PRXQuantum.3.020101
24. Zhou, Z., Chen, X., Li, E., Zeng, L., Luo, K., Zhang, J.: Edge intelligence: paving the last mile of artificial intelligence with edge computing. Proc. IEEE. **107**(8), 1738–1762 (2019). https://doi.org/10.1109/JPROC.2019.2918951
25. Anis, M.S., et al.: Qiskit: an open-source framework for quantum computing. Zenodo. (2023). https://doi.org/10.5281/zenodo.2573505
26. Qiskit AER Development Team: Qiskit AER: high-performance simulator for quantum circuits (2024). https://github.com/Qiskit/qiskit-aer. Accessed 7 May 2026

27. Amazon Web Services, Inc.: Amazon EC2 on-demand instance pricing (2026). https://aws.amazon.com/ec2/pricing/on-demand/. Accessed 7 May 2026
28. Amazon Web Services, Inc.: Amazon Braket pricing (2026). https://aws.amazon.com/braket/pricing/. Accessed 7 May 2026
29. IBM Corp: IBM Quantum pricing (2026). https://quantum-computing.ibm.com/services/resources. Accessed 7 May 2026
30. Microsoft Corp: Azure Quantum pricing (2026). https://azure.microsoft.com/en-us/pricing/details/azure-quantum/. Accessed 7 May 2026

Engineering Reliable Image-Forgery Localization Systems: A Multi-dataset Study of Topology-Aware Vision Transformers

Lavanya Mandava[1(✉)], Rohit Biswa Karma[1], and Shanmukh Gopinath Mandava[2]

[1] Department of Computer Science and Information Systems, Bradley University, Peoria, IL, USA
{lmandava,rbiswakarma1}@bradley.edu
[2] Dallas, TX, USA
shanmukh@umich.edu

Abstract. Image-forgery localization systems aim to identify manipulated pixels, yet their reliability as deployable ML software remains difficult to assess. A method can perform well on one benchmark while failing under dataset shift, target-domain adaptation, threshold changes, or authentic-image conditions. This paper presents a multi-dataset empirical evaluation of image-forgery localization as an ML software reliability problem, using a topology-aware Vision Transformer pipeline as the studied system. The pipeline combines ViT patch features, dense pixel refinement, and graph-based regularization over learned patch relationships. We evaluate the system on CASIA, COVERAGE, and CoMoFoD under in-domain, zero-shot, fine-tuned, joint-training, adaptation, and robustness settings, and we compare it with TruFor and MVSS-Net on a validated fixed published-baseline subset. The results show that graph regularization and pixel refinement improve the matched CASIA model, but CASIA-only training transfers poorly to external copy-move datasets. Joint CASIA+CoMoFoD training improves target-domain performance and provides a stronger initialization for COVERAGE adaptation, but authentic-image false positives remain a major reliability risk. The study shows that image-forgery localization should be evaluated not only by tampered-image IoU or F1, but also by multi-dataset transfer, adaptation behavior, threshold diagnostics, prediction/mask sanity checks, and authentic-image false-positive metrics.

Keywords: Image forgery localization · Vision Transformer · Software reliability · Dataset shift · Digital forensics · Domain adaptation

S. G. Mandava—Independent Researcher.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 357–375, 2027.
https://doi.org/10.1007/978-3-032-39763-8_21

1 Introduction

Image-forgery localization systems aim to mark the pixels that were manipulated rather than only classify an image as authentic or forged. This capability is important for forensic review, evidence analysis, content moderation, and human decision support, where a binary image-level decision is often insufficient. However, localization systems are difficult to assess as deployable ML software. Their behavior can change with dataset source, manipulation type, post-processing, threshold choice, and the presence of authentic images that should receive empty masks.

Image-forgery localization has been studied through both classical forensic cues and learned representations. Sensor-pattern noise and camera fingerprints can reveal camera-consistency violations [19], resampling traces can expose geometric editing artifacts [22], and Noiseprint learns camera-model noise fingerprints for local inconsistency detection [4]. Deep localization systems such as BusterNet [32], ManTra-Net [33], MVSS-Net [6], CAT-Net [16], and TruFor [11] learn manipulation-sensitive features for pixel-level localization. Transformer-based approaches, including ViT-style image representations [8], IML-ViT [20], and UnionFormer [17], further motivate patch-level and long-range relational modeling.

Despite this progress, strong performance on one benchmark does not guarantee reliable behavior on another. CASIA [7], COVERAGE [31], and CoMoFoD [27] differ in image provenance, manipulation generation, copy-move difficulty, post-processing variants, and mask conventions. These differences make forgery localization a useful case study for ML software reliability under dataset shift. A forensic model may localize tampered regions in-domain, fail zero-shot on an external dataset, improve after target-domain exposure, forget its original domain, or produce visually plausible false-positive regions on authentic images.

These concerns connect directly to software-engineering research on ML systems. Hidden technical debt in ML systems [25], production-readiness checks such as the ML Test Score [3], engineering practices for AI-based software [2], and testing surveys for ML systems [24] all argue that aggregate accuracy is not enough. Dataset shift [23], underspecification [5], and degraded confidence behavior under distribution shift [12,21] further show why threshold and reliability diagnostics are needed when ML systems leave their development distribution.

This paper therefore treats image-forgery localization as an empirical software-reliability problem. We study a topology-aware Vision Transformer (ViT) localization pipeline that combines patch-level supervision, dense pixel refinement, and graph-based regularization over learned patch relationships. The model is not presented as a universal state-of-the-art replacement for existing forensic systems. Instead, it serves as a controlled study system for evaluating reliability concerns that are often hidden by single-dataset IoU/F1 reporting: external transfer, target-domain adaptation, forgetting, post-processing robustness, threshold sensitivity, prediction/mask alignment, and authentic-image false positives.

We evaluate the system on CASIA, COVERAGE, CoMoFoD clean images, and CoMoFoD robustness variants. We also compare it with two published forensic baselines, TruFor [11] and MVSS-Net [6], on a validated fixed 1000-image published-baseline subset using a shared evaluation scaffold. The results show that graph regularization and pixel refinement improve the matched CASIA model, but CASIA-only training transfers poorly to external copy-move datasets. Joint CASIA+CoMoFoD training improves target-domain performance and provides a stronger initialization for COVERAGE adaptation. At the same time, method rankings depend on dataset and training exposure, and authentic-image false positives remain a major reliability concern.

The paper makes three contributions:

- We define a reliability-oriented evaluation scaffold for image-forgery localization, covering multi-dataset transfer, adaptation and forgetting, robustness, threshold diagnostics, mask/prediction sanity checks, and authentic-image false-positive behavior.
- We conduct a controlled empirical study of a topology-aware ViT localization system across CASIA, COVERAGE, and CoMoFoD, including ablations for graph regularization and dense pixel refinement.
- We compare the studied system with TruFor and MVSS-Net on a fixed validated published-baseline subset, while explicitly distinguishing fixed-threshold results, oracle-threshold diagnostics, exported-map limitations, and target-domain exposure.

The central claim is that reliable forensic ML software requires more than a single benchmark score. It requires evidence about how localization systems behave under dataset shift, adaptation, threshold choice, evaluation-pipeline assumptions, and authentic-image conditions.

2 Related Work

This work connects image-forgery localization, transformer/structure-aware localization, and reliability-oriented ML evaluation. We focus on work that motivates the evaluation design rather than attempting a full media-forensics survey.

Image-Forgery Localization. Localization methods estimate manipulated regions rather than only image-level authenticity. Classical approaches use sensor-pattern noise and camera fingerprints [19], resampling artifacts [22], copy-move feature matching [1], and learned noise fingerprints such as Noiseprint [4]. Deep systems including BusterNet [32], ManTra-Net [33], MVSS-Net [6], CAT-Net [16], PSCC-Net [18], and TruFor [11] learn manipulation-sensitive representations for pixel-level localization. These methods show substantial progress, but aggregate localization scores alone do not reveal how behavior changes across datasets, thresholds, adaptation settings, and authentic-image cases.

Transformer and Structure-Aware Localization. Vision Transformers model images as patch-token sequences with long-range interactions [8], which is useful when manipulations create nonlocal inconsistencies. Related attention- and

transformer-based forensic methods include SPAN [13], ObjectFormer [29], IML-ViT [20], and UnionFormer [17]. Graph neural networks [14,28] and non-local operations [30] further motivate relational modeling. In this study, the graph module is used as an auxiliary topology constraint over ViT patch relations while dense prediction remains handled by the ViT and pixel-refinement path.

Datasets, Transfer, and Adaptation. CASIA [7], COVERAGE [31], and CoMoFoD [27] differ in provenance, manipulation type, post-processing, and annotation conventions, creating dataset shift [23] and possible shortcut learning [10]. Domain-adaptation approaches such as DANN [9] and Deep CORAL [26] reduce distribution mismatch, but sequential adaptation can also cause forgetting [15]. We therefore separate in-domain testing, zero-shot external testing, CoMoFoD fine-tuning, joint training, and COVERAGE 5-fold adaptation.

Reliability-Oriented ML Evaluation. Software-engineering work on ML systems emphasizes that aggregate accuracy is insufficient for deployment readiness. Hidden technical debt [25], the ML Test Score [3], engineering practices for AI-based software [2], and ML testing surveys [24] highlight data validation, robustness, monitoring, and failure-mode analysis. Underspecification [5] and confidence degradation under distribution shift [12,21] motivate threshold and reliability diagnostics. We operationalize these concerns for forensic localization through multi-dataset evaluation, adaptation analysis, threshold diagnostics, sanity checks, and authentic-image false-positive metrics.

3 Topology-Aware ViT Localization Pipeline

Figure 1 summarizes the studied localization system. The model receives a 224×224 RGB image and predicts a dense binary mask of manipulated pixels. It combines a ViT-B/16 backbone, a patch-level tampering head, a topology-aware graph regularizer over learned patch relations, and a dense pixel-refinement decoder. The architecture is used as the controlled study system for the reliability evaluation in Sects. 4–5.

3.1 Model Overview

Given an image I, the goal is to predict a binary localization mask $\hat{Y}$, where foreground pixels indicate manipulated content. Authentic images are assigned all-zero masks. The input image is divided into 16×16 patches and processed by a ViT-B/16 backbone [8], producing $N = 196$ patch tokens $Z = \{z_i\}_{i=1}^{N}$. A lightweight patch head maps each token to a coarse tampering logit. Patch supervision uses soft targets derived from the fraction of manipulated pixels in each patch, so partially manipulated boundary patches are not forced into purely hard labels.

The pixel refiner converts the coarse patch representation into a dense 224×224 logit map. It concatenates the reshaped patch-token feature map with the patch-logit map and upsamples the result through convolutional refinement

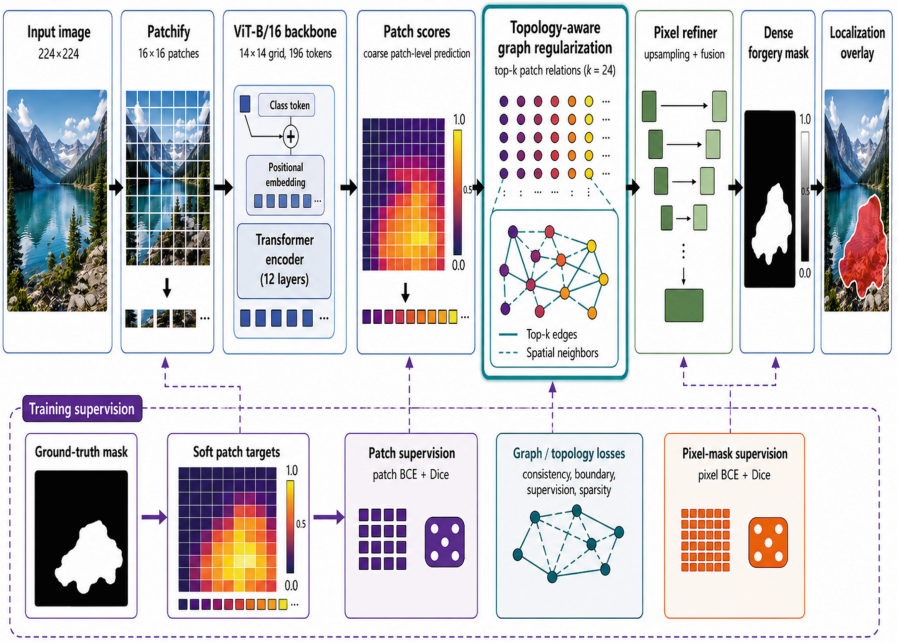


Fig. 1. Proposed topology-aware ViT localization pipeline. The image is resized to 224×224 and split into 16×16 patches, producing a 14×14 token grid. ViT patch tokens support coarse patch-level prediction, topology-aware graph regularization over patch relations, and dense pixel refinement. Training combines patch-level supervision, graph/topology losses, and pixel-mask supervision; inference uses the pixel refiner output as the final localization mask.

blocks. This separates two roles: the ViT backbone and patch head learn coarse forensic localization cues, while the refiner recovers a dense pixel-level mask.

The graph module constructs a learned patch-relation graph. For patch pair (i, j), affinity is predicted from a hybrid feature–spatial relation vector $[z_i, z_j, \Delta x, \Delta y, d_{ij}]$ using an MLP followed by a sigmoid. Top-k sparsification keeps the strongest 24 outgoing affinities per patch, and the graph is symmetrized in the main configuration. The graph is used for topology regularization rather than as the primary feature extractor: it encourages structured relationships among patch predictions while leaving dense mask prediction to the ViT and pixel-refinement path (Table 1).

3.2 Training Objective

Training combines pixel-level, patch-level, and graph-level objectives. The dense mask is supervised with pixel BCE and Dice losses, while the patch head is supervised with BCE and Dice losses on soft patch targets. The graph objective regularizes relationships among patch predictions.

Table 1. Implementation details for the topology-aware ViT localization pipeline.

Item	Implementation detail
Input/preprocessing	RGB images resized to 224×224 and scaled to $[0, 1]$ without ImageNet normalization; masks resized to 224×224 with nearest-neighbor interpolation and binarized.
Backbone	`vit_base_patch16_224` / ViT-B/16.
Patch grid/head	Patch size 16; $14 \times 14 = 196$ patch tokens. Patch head: Linear$(D, D/2) \rightarrow$ GELU $\rightarrow$ Dropout$(0.1) \rightarrow$ Linear$(D/2, 1)$.
Pixel refiner	Patch-token feature map concatenated with reshaped patch logits; Conv1 $\times$ 1 projection $(D + 1) \rightarrow 256$, followed by Conv3 $\times$ 3/BatchNorm/GELU blocks with channels $256 \rightarrow 256 \rightarrow 128 \rightarrow 96 \rightarrow 64 \rightarrow 32 \rightarrow 1$ and four bilinear $\times 2$ upsampling stages.
Graph adjacency	Learned patch-pair affinity from $[z_i, z_j, \Delta x, \Delta y, d_{ij}] \rightarrow$ MLP $\rightarrow$ sigmoid affinity; top-k sparsification with $k = 24$, chunk size 32, and symmetric graph enabled.
Loss weights	Pixel BCE 1.0; pixel Dice 1.5; patch BCE 0.35; patch Dice 0.25; graph consistency 0.15; boundary 0.20; graph supervision 0.10; sparsity 0.01; pixel/patch BCE positive weights 8.0.
Optimization	AdamW; batch size 8; weight decay 10^{-2}; cosine annealing. LR 10^{-4} for CASIA-only/full-graph training, 3×10^{-5} for joint CASIA+CoMoFoD, and 2×10^{-5} for COVERAGE adaptation.
Schedule/selection	CASIA full/ablation: 12 epochs; CoMoFoD fine-tuning: 25 epochs; joint: 12 epochs; COVERAGE adaptation: 15 epochs with weighted sampler and early-stopping patience 5. Graph losses start at epoch 4 for CASIA, epoch 1 for joint training, and epoch 2 for COVERAGE adaptation. Best checkpoints are selected by validation IoU after threshold sweep.
Thresholds	CASIA/joint threshold grid: 0.10–0.90 in 0.05 increments; COVERAGE adaptation grid: 0.05–0.95 in 0.05 increments.
Parameters/runtime	88,545,763 total parameters; 2,747,107 trainable while the backbone is frozen; 31,100,131 trainable after unfreezing the last four ViT blocks. Evaluation logs showed approximately 2.4–3.3 ms/image, hardware dependent.
Hardware/software	Python 3.11.9; PyTorch 2.11.0+`cu128`; CUDA enabled; NVIDIA GeForce RTX 5090 Laptop GPU.

The total loss is

$$\mathcal{L} = \lambda_{\text{pix,bce}}\mathcal{L}_{\text{pix,bce}} + \lambda_{\text{pix,dice}}\mathcal{L}_{\text{pix,dice}} + \lambda_{\text{patch,bce}}\mathcal{L}_{\text{patch,bce}} + \lambda_{\text{patch,dice}}\mathcal{L}_{\text{patch,dice}} + \lambda_{\text{graph}}\mathcal{L}_{\text{graph}}, \tag{1}$$

where

$$\mathcal{L}_{\text{graph}} = \lambda_{\text{cons}}\mathcal{L}_{\text{cons}} + \lambda_{\text{bdry}}\mathcal{L}_{\text{bdry}} + \lambda_{\text{sup}}\mathcal{L}_{\text{sup}} + \lambda_{\text{sparse}}\mathcal{L}_{\text{sparse}}. \tag{2}$$

The consistency term penalizes differences in connected patch probabilities; the boundary term discourages high affinity across strong patch-probability changes; graph supervision matches learned affinity to patch-target similarity; and sparsity penalizes overly dense graphs. In CASIA training, graph losses are delayed until epoch 4 so the backbone first learns basic localization cues. For joint training and COVERAGE adaptation, graph losses begin earlier because those runs initialize from trained checkpoints.

These implementation details make the evaluated system reproducible and define the architecture used throughout the reliability study.

4 Experimental Design

The evaluation is designed to test image-forgery localization as ML software operating under distribution shift, target-domain exposure, thresholding, and authentic-image conditions. We separate in-domain performance, zero-shot transfer, adaptation, robustness, published-baseline comparison, and evaluation-integrity checks rather than treating a single dataset score as sufficient evidence of reliability.

4.1 Datasets and Splits

We evaluate on CASIA, COVERAGE, CoMoFoD clean images, and CoMoFoD robustness variants. CASIA provides the main in-domain train/validation/test benchmark. COVERAGE is used as an external copy-move stress test and as a 5-fold adaptation target. CoMoFoD clean is used for external testing, target-domain fine-tuning, and joint training, while CoMoFoD robustness variants evaluate controlled post-processing changes. Table 2 summarizes the dataset roles.

Table 2. Datasets and evaluation roles.

Dataset	Authentic	Tampered/Forged	Role
CASIA 2.0 revised/corrected	7,491	5,123	Main in-domain train/validation/test dataset.
COVERAGE	100	100	External copy-move stress test and 5-fold adaptation dataset.
CoMoFoD clean	200	200	External test, adaptation, and joint-training dataset.
CoMoFoD robustness variants	4,800	4,800	Held-out post-processing robustness analysis.

For CASIA, the fixed split contains 8,829 training images, 1,261 validation images, and 2,524 test images. COVERAGE contains 100 original/forged pairs. In 5-fold adaptation, each fold uses 64 pairs for training, 16 pairs for validation, and 20 pairs for testing, keeping original and forged versions in the same fold to avoid pair leakage. For CoMoFoD, ID-level splitting prevents post-processed variants of the same base image from crossing train, validation, and test partitions. Split manifests and file-existence checks were used to verify that selected image and mask paths were present.

4.2 Evaluation Protocols

Figure 2 summarizes the evaluation design. We compare four training/evaluation regimes. First, a CASIA-only model is trained and validated on CASIA, then evaluated on CASIA test images and zero-shot on COVERAGE and CoMoFoD. Second, the CASIA model is fine-tuned on CoMoFoD clean training IDs and re-evaluated on CASIA, CoMoFoD, and COVERAGE to measure target-domain

adaptation and forgetting. Third, a joint model is trained using CASIA and CoMoFoD clean training data. Fourth, CASIA-only and joint models are used as initializations for COVERAGE 5-fold adaptation.

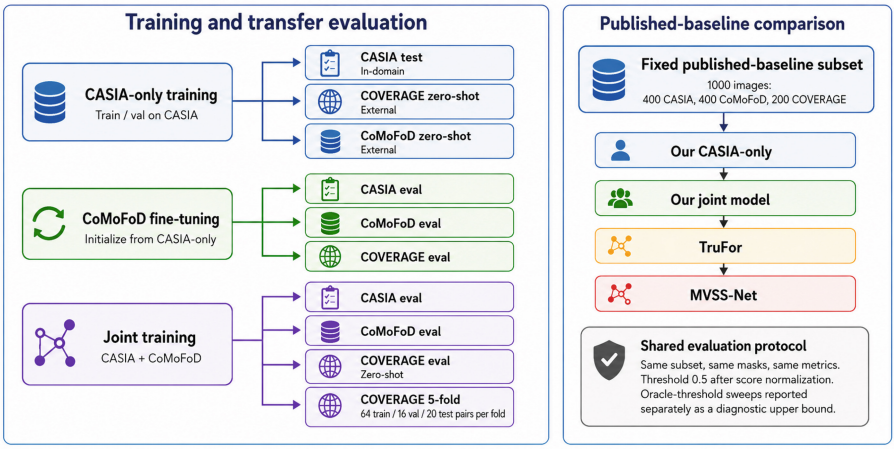


Fig. 2. Evaluation protocols used in the study. The left side summarizes training and transfer settings, including CASIA-only training, CoMoFoD fine-tuning, and joint CASIA+CoMoFoD training. The right side summarizes the published-baseline comparison on the fixed 1000-image subset. Fixed-threshold comparison uses a normalized-score operating point at 0.5; oracle-threshold sweeps are reported separately as a diagnostic upper bound.

The fixed published-baseline subset contains 1000 images: 200 CASIA tampered, 200 CASIA authentic, 200 CoMoFoD forged, 200 CoMoFoD original, 100 COVERAGE tampered, and 100 COVERAGE original images. This subset is used only for shared comparison with published baselines and does not replace the full-dataset evaluations used in the other research questions.

4.3 Models, Metrics, and Baselines

The core architectural variants are: full graph + pixel refiner, no graph + pixel refiner, and no pixel refiner. The domain-training regimes are CASIA-only, CASIA→CoMoFoD fine-tuning, joint CASIA+CoMoFoD training, and COVERAGE adaptation from either CASIA-only or joint initialization.

For tampered or forged images, we report pixel-level IoU, F1, precision, recall, and boundary F1. IoU and F1 measure region overlap between the predicted binary mask and the ground-truth manipulation mask. Precision and recall distinguish over-localization from missed forged pixels. Boundary F1 measures boundary alignment by comparing contour pixels extracted from the predicted and ground-truth masks. For authentic images, IoU and F1 are not meaningful

because the ground-truth mask is empty; we therefore report any-FP rate and false-positive area ratio.

Published baselines include TruFor [11] and MVSS-Net [6]. Prediction maps from all methods are converted to a common $[0, 1]$ scale before fixed-threshold evaluation: maps already in $[0, 1]$ are clipped, image-like $[0, 255]$ maps are divided by 255, and other out-of-range score maps are min–max normalized per image. Thus, threshold 0.5 is a normalized-score operating point, not necessarily each method's native threshold. Oracle-threshold sweeps are reported separately as diagnostic upper bounds.

The MVSS-Net evaluation used exported prediction maps from `predicted_masks/MVSS-Net`; native raw logits/probability outputs and an official native threshold were not available in the final evaluation package. Therefore, fixed-threshold and oracle-threshold diagnostics for MVSS-Net were computed from the exported maps and are interpreted as reproduced exported-map diagnostics rather than as a definitive statement about the original MVSS-Net method.

4.4 Sanity Checks and Reproducibility Controls

Before reporting the published-baseline comparison, we verified that the fixed subset contains 500 tampered/forged and 500 authentic images, with no missing images, invalid masks, or duplicate image paths. Authentic masks were checked to be blank and tampered masks non-empty. TruFor and MVSS-Net predictions matched all 1000 subset images. We also generated visual contact sheets to inspect input images, ground-truth overlays, probability maps, and binary predictions for spatial alignment.

Table 3 summarizes how the evaluation operationalizes reliability concerns from ML software engineering. The table is not intended as a universal checklist for all forensic systems; rather, it documents the concrete reliability checks used in this study.

Table 3. Reliability concerns operationalized in the evaluation.

Concern	Evaluation check	Evidence reported
Dataset shift	Zero-shot testing from CASIA to COVERAGE and CoMoFoD	Table 5.
Adaptation/forgetting	CoMoFoD fine-tuning, joint training, and retained-source evaluation	Table 6.
Target-domain instability	COVERAGE 5-fold pair-level adaptation	Table 7.
Threshold dependence	Fixed-threshold and oracle-threshold diagnostics	Tables 8–9.
Authentic-image risk	Any-FP rate and false-positive area on authentic images	Table 10.
Evaluation integrity	Mask checks, prediction matching, scale checks, and contact sheets	Sect. 4.4.

To support reproducibility, we will release split manifests, the fixed published-baseline subset manifest, evaluation scripts, per-image metric CSVs, prediction-matching and sanity-check outputs, and threshold-diagnostic tables. Trained checkpoints will be treated as optional supplementary artifacts subject to storage limits and dataset/license constraints.

5 Evaluation and Results

We organize the results around five questions: (RQ1) whether graph regularization and pixel refinement improve in-domain localization, (RQ2) whether CASIA-only training transfers to external datasets, (RQ3) whether adaptation and joint training improve transfer, (RQ4) how the models compare with published baselines under a shared evaluation scaffold, and (RQ5) what reliability risks remain on authentic images.

5.1 RQ1: In-Domain CASIA Ablation Results

Table 4 summarizes CASIA fixed-split ablations on tampered test images. The full graph model obtains the strongest performance, with 0.4321 IoU and 0.6035 F1. Removing graph/topology supervision reduces IoU/F1 to 0.4016/0.5731, while removing the pixel refiner reduces performance to 0.3950/0.5663. These gains are modest but consistent: graph regularization and dense refinement improve the matched ViT pipeline, but they do not by themselves guarantee cross-dataset generalization.

Table 4. CASIA fixed-split ablation results on tampered test images.

Model	IoU	F1	Precision	Recall	Boundary F1
Full graph + pixel refiner	**0.4321**	**0.6035**	**0.6957**	**0.5329**	**0.2574**
No graph + pixel refiner	0.4016	0.5731	0.6903	0.4899	0.2426
No pixel refiner	0.3950	0.5663	0.6456	0.5044	0.2268

5.2 RQ2: Zero-Shot Generalization from CASIA

Table 5 evaluates the CASIA-only full graph model on external datasets without target-domain training. Performance drops from 0.4321/0.6035 IoU/F1 on CASIA to 0.1783/0.3027 on COVERAGE and 0.1221/0.2176 on CoMoFoD. The CoMoFoD result has higher precision than recall, indicating that the model marks some suspicious regions but misses many forged pixels. These results show that in-domain CASIA performance is not sufficient evidence of deployment reliability.

5.3 RQ3: Adaptation, Joint Training, and Forgetting

Table 6 compares CASIA-only training, CoMoFoD fine-tuning, and joint CASIA+CoMoFoD training. CoMoFoD fine-tuning substantially improves CoMoFoD localization, but it also reduces CASIA performance, indicating forgetting. Joint training gives the best CASIA+CoMoFoD balance: it preserves

Table 5. Zero-shot generalization of the CASIA-only full graph model.

Test set	IoU	F1	Precision	Recall	Boundary F1
CASIA tampered	0.4321	0.6035	0.6957	0.5329	0.2574
COVERAGE tampered	0.1783	0.3027	0.3240	0.2840	0.0831
CoMoFoD clean forged	0.1221	0.2176	0.4079	0.1484	0.1221

Table 6. Adaptation and joint-training results. Values report IoU/F1 on tampered or forged test images.

Training regime	CASIA	CoMoFoD	COVERAGE
CASIA-only full graph	0.4321/0.6035	0.1221/0.2176	0.1783/0.3027
CASIA → CoMoFoD fine-tune	0.3453/0.5134	0.6349/0.7767	0.2090/0.3457
Joint CASIA+CoMoFoD	0.4221/0.5936	**0.6578/0.7936**	0.1546/0.2678

most CASIA performance while producing strong CoMoFoD localization. However, joint training does not solve COVERAGE in the full-test setting, reinforcing that COVERAGE is a difficult unseen-domain stress test.

COVERAGE 5-fold adaptation tests limited target-domain exposure. Table 7 reports tampered-image fold averages from the curated COVERAGE adaptation summary. Joint initialization substantially improves adaptation over CASIA-only initialization, increasing mean IoU from 0.2228 to 0.4193 and mean F1 from 0.3625 to 0.5847. The standard deviation remains noticeable, especially for joint initialization, because each fold contains only 20 test pairs. This result suggests that multi-domain pretraining can provide a better initialization for small target-domain adaptation, even when zero-shot COVERAGE performance remains weak.

Table 7. COVERAGE 5-fold adaptation results on tampered images.

Initialization	IoU	F1	Precision	Recall	Boundary F1
CASIA init	0.2228 ± 0.0471	0.3625 ± 0.0645	0.3223	0.4285	0.1220
Joint init	**0.4193 ± 0.1023**	**0.5847 ± 0.1074**	**0.6249**	**0.5597**	**0.2210**

CoMoFoD robustness variants further test post-processing sensitivity across brightness, contrast, color reduction, noise, JPEG compression, and blur. Figure 3 summarizes forged-image robustness across these post-processing categories. The robustness results are used as controlled post-processing diagnostics rather than as a complete simulation of real deployment conditions.

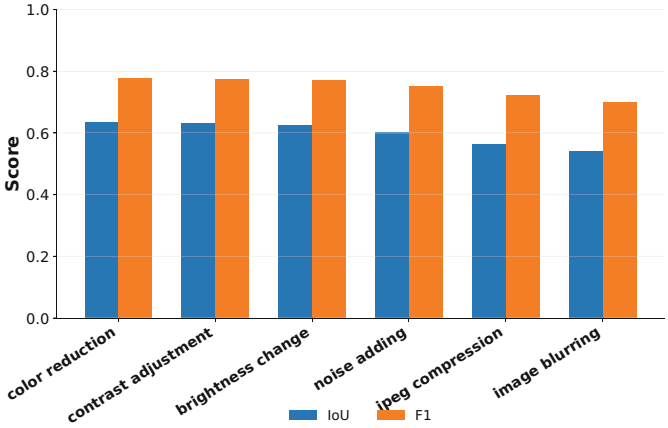


Fig. 3. CoMoFoD robustness results on forged images, averaged across available severity levels for each post-processing type.

5.4 RQ4: Published-Baseline Comparison and Threshold Diagnostics

Table 8 reports fixed-threshold tampered/forged-image localization on the fixed published-baseline subset. TruFor is strongest on CASIA and COVERAGE. The joint model is strongest on CoMoFoD, but this should be interpreted as target-domain exposure because CoMoFoD is included in joint training; it is not a pure zero-shot comparison against methods that were not trained on CoMoFoD. The fairest unseen-dataset comparison in this table is COVERAGE, where TruFor remains ahead of the joint model under fixed-threshold evaluation (Fig. 4).

Table 8. Published-baseline comparison on the fixed subset. Values report IoU/F1 on tampered or forged images at fixed thresholds.

Method	CASIA	COVERAGE	CoMoFoD
TruFor	**0.7758/0.8737**	**0.4448/0.6157**	0.1249/0.2221
MVSS-Net	0.0958/0.1749	0.0891 / 0.1637	0.0667/0.1251
CASIA-only full graph	0.4531/0.6236	0.1782/0.3025	0.1219/0.2172
Joint CASIA+CoMoFoD	0.4462/0.6171	0.1546/0.2679	**0.8541/0.9213**

For cross-method comparability, exported prediction maps were normalized to $[0, 1]$ before fixed-threshold evaluation. The MVSS-Net evaluation used exported prediction maps from `predicted_masks/MVSS-Net`; native raw logits/probability outputs and an official native threshold were not available in the final evaluation package. Therefore, fixed-threshold and oracle-threshold diagnostics were computed from the exported maps.

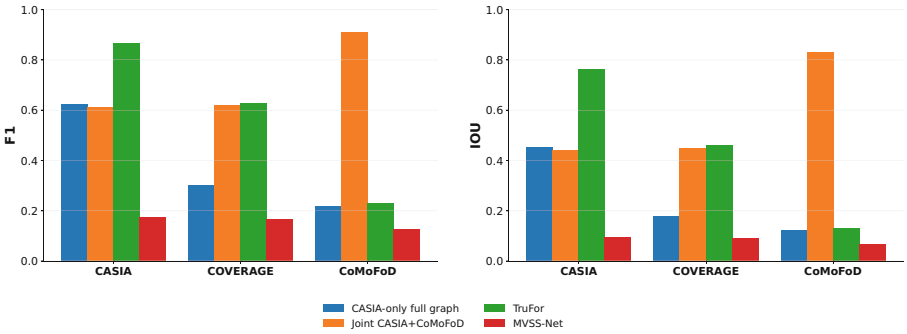


Fig. 4. Published-baseline comparison on the fixed published-baseline subset. Method ranking changes across datasets and training exposure.

Table 9 reports tampered-only oracle-threshold diagnostics for TruFor and MVSS-Net. Oracle thresholding improves TruFor modestly and confirms that COVERAGE remains difficult. MVSS-Net remains weak under exported-map oracle thresholding; therefore, we interpret its result as behavior under our reproduced exported-prediction evaluation scaffold rather than as a definitive statement about the original MVSS-Net method.

Table 9. Tampered-only oracle-threshold diagnostics for published baselines. Oracle thresholds are selected per dataset and method as diagnostic upper bounds, not as deployment results.

Method	Dataset	Oracle thr.	IoU	F1
TruFor	CASIA	0.55	0.7786	0.8755
TruFor	COVERAGE	0.30	0.4620	0.6320
TruFor	CoMoFoD	0.10	0.1582	0.2731
MVSS-Net	CASIA	0.35	0.1158	0.2076
MVSS-Net	COVERAGE	0.25	0.1519	0.2638
MVSS-Net	CoMoFoD	0.35	0.0824	0.1523

Per-image confusion counts were retained for uncertainty analysis. The main effects in Table 8 are large enough to support the reported ranking, but smaller differences, such as CASIA-only versus joint performance on CASIA and COVERAGE, should be interpreted cautiously.

5.5 RQ5: Authentic-Image Reliability

Table 10 summarizes authentic-image false positives for TruFor and MVSS-Net at threshold 0.5 on the fixed published-baseline subset. Both methods produce

Table 10. Authentic-image false-positive behavior for published baselines at threshold 0.5. Any-FP is strict: an image is counted if any non-empty forged region is predicted.

Method	Dataset	Any-FP (%)	Mean FP area	Median FP area
TruFor	CASIA	76.5	0.0166	0.0047
TruFor	COVERAGE	90.0	0.0278	0.0079
TruFor	CoMoFoD	88.5	0.0137	0.0053
MVSS-Net	CASIA	100.0	0.0273	0.0089
MVSS-Net	COVERAGE	100.0	0.0218	0.0091
MVSS-Net	CoMoFoD	100.0	0.0652	0.0075

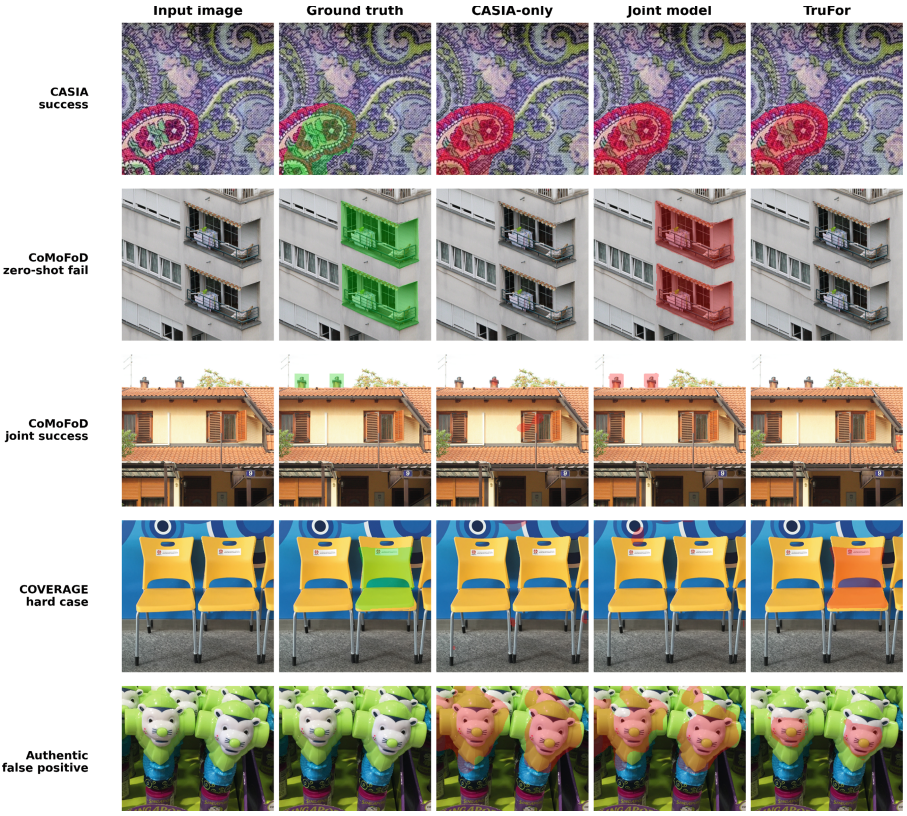


Fig. 5. Qualitative localization examples. Green overlays indicate ground-truth manipulated regions, and red overlays indicate predicted forged regions. The rows show CASIA success, CoMoFoD zero-shot failure, CoMoFoD joint-training success, a difficult COVERAGE case, and an authentic-image false-positive case. (Color figure online)

false-positive regions on authentic images under the reproduced evaluation scaffold, but their frequency and severity differ. MVSS-Net produces at least one false-positive region on every authentic image in this subset, whereas TruFor produces false positives on a majority of authentic images, with the highest rate on COVERAGE.

The same reliability pattern appears in our models. Joint training improves authentic-image reliability on CASIA and CoMoFoD relative to the CASIA-only model, but COVERAGE remains difficult. After COVERAGE adaptation, the joint-initialized model reduces the authentic COVERAGE false-positive pixel-area ratio to 0.0565, compared with 0.1399 for CASIA-only initialization, but 94% of original COVERAGE images still contain at least one false-positive region. Thus, improved tampered-image localization does not automatically imply reliable authentic-image rejection.

Figure 5 provides qualitative examples matching the main quantitative findings: in-domain CASIA success, CoMoFoD zero-shot failure, CoMoFoD improvement with joint training, a difficult COVERAGE case, and an authentic-image false-positive example.

Together, these results show that localization performance depends on more than the model architecture. Dataset shift, target-domain exposure, threshold choice, and authentic-image behavior all change the reliability profile of a forensic localization system.

6 Discussion

The results support a reliability-centered view of image-forgery localization. A single IoU or F1 value on one benchmark does not show whether a localization system is suitable for deployment. In this study, the same model can perform well in-domain, fail under zero-shot dataset shift, improve after target-domain exposure, and still produce false-positive regions on authentic images.

Reliability Profile Rather than Single Ranking. Forensic localization should be evaluated through a profile that includes in-domain performance, external transfer, retained-source performance after adaptation, threshold sensitivity, prediction/mask sanity checks, and authentic-image false positives. This framing matters because method ranking changes with dataset and training exposure: TruFor is strongest on CASIA and COVERAGE in the fixed published-baseline subset, whereas the joint model is strongest on CoMoFoD after CoMoFoD is included in training.

Target-Domain Exposure and Thresholds Change Interpretation. Fine-tuning and joint training improve CoMoFoD localization, and joint initialization improves COVERAGE adaptation, but these results are not pure zero-shot generalization. They show the value of target-domain exposure when labeled target data are available. Similarly, thresholding is part of the deployed system: a fixed threshold of 0.5 on normalized exported maps is a common operating point, while oracle thresholds are diagnostic upper bounds. The MVSS-Net results should

therefore be interpreted as exported-map diagnostics because native logits and an official native threshold were not available.

Authentic-Image Behavior is a Deployment Risk. Strong overlap on tampered images does not guarantee reliable rejection of authentic images. False-positive regions can draw analyst attention to innocent content and create misleading evidence. Authentic-image any-FP rate and false-positive area should therefore be reported separately from tampered-image IoU and F1, especially on datasets such as COVERAGE where visually similar genuine objects increase false-positive risk.

Implications for Forensic ML Software. Before deployment, localization systems should be tested on external datasets, checked for prediction/mask alignment, thresholded using target-representative validation data, evaluated on authentic images, and re-tested after adaptation. The study does not identify a universally reliable model; it shows that reliability evidence must be accumulated across datasets, thresholds, and authentic-image conditions before localization outputs are used for decision support.

7 Threats to Validity

Internal Validity. Implementation, preprocessing, thresholding, or mask-alignment errors could affect the reported results. We mitigate this risk with fixed split manifests, file-existence checks, mask-polarity checks, prediction matching, scale diagnostics, visual contact sheets, and per-image metric records. The main remaining risk is that third-party baselines may depend on preprocessing, checkpoints, or native thresholds that differ from our exported-map scaffold. This is especially relevant for MVSS-Net, whose native logits/probabilities and official native threshold were not available.

Statistical Conclusion Validity. Some comparisons involve modest effects. CASIA ablation gains are consistent but small, and COVERAGE 5-fold adaptation has noticeable fold variance because each fold contains only 20 test pairs. We therefore avoid claiming statistical dominance for small differences and treat COVERAGE primarily as a stress test.

External Validity. CASIA, COVERAGE, and CoMoFoD do not cover all real-world manipulations, acquisition devices, compression histories, social-media transformations, screenshot artifacts, or generative editing workflows. CoMoFoD robustness variants are controlled transformations rather than full deployment simulations, and the fixed published-baseline subset is not a substitute for larger benchmarks. Additional deployment-specific validation would be needed before operational forensic use.

Construct Validity. IoU, F1, precision, recall, and boundary F1 measure pixel-level localization quality but not all aspects of forensic usefulness. A mask with reasonable overlap may still be misleading if it highlights visually plausible but

incorrect regions. We therefore include authentic-image false-positive metrics and threshold diagnostics, while leaving analyst response, calibration error, selective prediction, and case-level decision utility to future work.

Reproducibility. Results depend on hardware, software versions, pretrained checkpoints, third-party repositories, and dataset licenses. We report implementation details and will release split manifests, evaluation scripts, per-image metric CSVs, prediction-matching checks, and threshold diagnostics where permitted. Dataset images, some third-party outputs, and trained checkpoints may not be redistributable. The baseline comparison is a controlled shared-scaffold comparison using TruFor and MVSS-Net, not a complete leaderboard over all forensic localization methods.

8 Conclusion

This paper evaluated image-forgery localization as a reliability-sensitive ML software problem rather than only as a segmentation benchmark. Using a topology-aware ViT pipeline as the studied system, we examined in-domain performance, zero-shot transfer, fine-tuning, joint training, COVERAGE adaptation, robustness variants, published baselines, threshold diagnostics, and authentic-image false positives.

The results show that graph/topology regularization and dense pixel refinement improve the matched CASIA model, but in-domain performance does not reliably transfer to external copy-move datasets. Target-domain exposure improves CoMoFoD performance and provides a stronger initialization for COVERAGE adaptation, but it also changes the interpretation of comparisons and does not eliminate false positives. Published baselines likewise show dataset- and threshold-dependent behavior under the shared evaluation scaffold.

The main implication is that reliable forensic localization requires a broader evidence package than a single IoU or F1 score. Multi-dataset testing, adaptation and forgetting analysis, threshold diagnostics, evaluation-integrity checks, and authentic-image reliability metrics should be part of the evaluation protocol before localization outputs are used for decision support.

Acknowledgements. This work was supported in part by a subaward from The Grainger College of Engineering at the University of Illinois Urbana-Champaign through the Enhancing EM through AI Seed Grant program.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Amerini, I., Ballan, L., Caldelli, R., Bimbo, A.D., Serra, G.: A SIFT-based forensic method for copy-move attack detection and transformation recovery. IEEE Trans. Inf. Forensics Secur. (2011). https://doi.org/10.1109/TIFS.2011.2129512

2. Amershi, S., et al.: Software engineering for machine learning: A case study. In: ICSE-SEIP (2019). https://doi.org/10.1109/ICSE-SEIP.2019.00042
3. Breck, E., Cai, S., Nielsen, E., Salib, M., Sculley, D.: The ML test score: A rubric for ML production readiness and technical debt reduction. In: IEEE International Conference on Big Data (2017). https://doi.org/10.1109/BigData.2017.8258038
4. Cozzolino, D., Verdoliva, L.: Noiseprint: A CNN-based camera model fingerprint. IEEE Trans. Inf. Forensics Secur. (2019). https://doi.org/10.1109/TIFS.2019.2916364
5. D'Amour, A., et al.: Underspecification presents challenges for credibility in modern machine learning. J. Mach. Learn. Res. **23**(226), 1–61 (2022). https://doi.org/10.48550/arXiv.2011.03395
6. Dong, C., Chen, X., Hu, R., Cao, J., Li, X.: MVSS-Net: Multi-view multi-scale supervised networks for image manipulation detection. IEEE TPAMI (2022). https://doi.org/10.1109/TPAMI.2022.3180556
7. Dong, J., Wang, W., Tan, T.: CASIA image tampering detection evaluation database. In: IEEE China Summit and International Conference on Signal and Information Processing, ChinaSIP (2013). https://doi.org/10.1109/ChinaSIP.2013.6625374
8. Dosovitskiy, A., et al.: An image is worth 16x16 words: Transformers for image recognition at scale. In: ICLR (2021). https://doi.org/10.48550/arXiv.2010.11929
9. Ganin, Y., et al.: Domain-adversarial training of neural networks. J. Mach. Learn. Res. **17**(59), 1–35 (2016). https://www.jmlr.org/papers/v17/15-239.html
10. Geirhos, R., et al.: Shortcut learning in deep neural networks. Nat. Mach. Intell. (2020). https://doi.org/10.1038/s42256-020-00257-z
11. Guillaro, F., Cozzolino, D., Sud, A., Dufour, N., Verdoliva, L.: TruFor: Leveraging all-round clues for trustworthy image forgery detection and localization. In: CVPR (2023). https://doi.org/10.1109/CVPR52729.2023.01974
12. Guo, C., Pleiss, G., Sun, Y., Weinberger, K.Q.: On calibration of modern neural networks. In: ICML (2017). https://doi.org/10.48550/arXiv.1706.04599
13. Hu, X., Zhang, Z., Jiang, Z., Chaudhuri, S., Yang, Z., Nevatia, R.: SPAN: Spatial pyramid attention network for image manipulation localization. In: ECCV (2020). https://doi.org/10.1007/978-3-030-58589-1_19
14. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. In: ICLR (2017). https://doi.org/10.48550/arXiv.1609.02907
15. Kirkpatrick, J., et al.: Overcoming catastrophic forgetting in neural networks. PNAS (2017). https://doi.org/10.1073/pnas.1611835114
16. Kwon, M.J., Yu, I.J., Nam, S.H., Lee, H.K.: CAT-Net: Compression artifact tracing network for detection and localization of image splicing. In: WACV. (2021). https://doi.org/10.1109/WACV48630.2021.00042
17. Li, S., Ma, W., Guo, J., Xu, S., Li, B., Zhang, X.: UnionFormer: Unified-learning transformer with multi-view representation for image manipulation detection and localization. In: CVPR (2024). https://doi.org/10.1109/CVPR52733.2024.01190
18. Liu, X., Liu, Y., Chen, J., Liu, X.: PSCC-Net: Progressive spatio-channel correlation network for image manipulation detection and localization. IEEE TCSVT (2022). https://doi.org/10.1109/TCSVT.2022.3189545
19. Lukas, J., Fridrich, J., Goljan, M.: Digital camera identification from sensor pattern noise. IEEE Trans. Inf. Forensics Secur. (2006). https://doi.org/10.1109/TIFS.2006.873602
20. Ma, X., Du, B., Jiang, Z., Du, X., Hammadi, A.Y.A., Zhou, J.: IML-ViT: Benchmarking image manipulation localization by vision transformer. arXiv (2023). https://doi.org/10.48550/arXiv.2307.14863

21. Ovadia, Y., et al.: Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift. In: NeurIPS (2019). https://doi.org/10.48550/arXiv.1906.02530
22. Popescu, A.C., Farid, H.: Exposing digital forgeries in color filter array interpolated images. IEEE Trans. Signal Process. (2005). https://doi.org/10.1109/TSP.2005.855406
23. Quiñonero-Candela, J., Sugiyama, M., Schwaighofer, A., Lawrence, N.D.: Dataset Shift in Machine Learning. MIT Press (2009). https://doi.org/10.7551/mitpress/9780262170055.001.0001
24. Riccio, V., Jahangirova, G., Stocco, A., Humbatova, N., Weiss, M., Tonella, P.: Testing machine learning based systems: A systematic mapping. Empir. Softw. Eng. (2020). https://doi.org/10.1007/s10664-020-09881-0
25. Sculley, D., et al.: Hidden technical debt in machine learning systems. NeurIPS (2015). https://doi.org/10.5555/2969442.2969519
26. Sun, B., Saenko, K.: Deep CORAL: Correlation alignment for deep domain adaptation. In: ECCV Workshops (2016). https://doi.org/10.1007/978-3-319-49409-8_35
27. Tralic, D., Zupancic, I., Grgic, S., Grgic, M.: CoMoFoD: New database for copy-move forgery detection. In: ELMAR (2013). https://www.vcl.fer.hr/comofod/
28. Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y.: Graph attention networks. In: ICLR (2018). https://doi.org/10.48550/arXiv.1710.10903
29. Wang, J., et al.: ObjectFormer for image manipulation detection and localization. In: CVPR (2022). https://doi.org/10.1109/CVPR52688.2022.00240
30. Wang, X., Girshick, R., Gupta, A., He, K.: Non-local neural networks. In: CVPR (2018). https://doi.org/10.1109/CVPR.2018.00813
31. Wen, B., Zhu, Y., Subramanian, R., Ng, T.T., Shen, X., Winkler, S.: COVERAGE: A novel database for copy-move forgery detection. In: IEEE International Conference on Image Processing (ICIP) (2016). https://doi.org/10.1109/ICIP.2016.7532339
32. Wu, Y., Abd-Almageed, W., Natarajan, P.: BusterNet: Detecting copy-move image forgery with source/target localization. In: ECCV (2018). https://doi.org/10.1007/978-3-030-01231-1_11
33. Wu, Y., Abd-Almageed, W., Natarajan, P.: ManTra-Net: Manipulation tracing network for detection and localization of image forgeries with anomalous features. In: CVPR (2019). https://doi.org/10.1109/CVPR.2019.00977

The Effectiveness of Feedback Design for Improving LLM-Based Code Generation in Quantum Optimization Algorithms

Venkata Sai Prathyush Turaga(✉), Sonali Uttam Singh, and Akbar Siami Namin

Department of Computer Science, Texas Tech University, Lubbock, USA
{pturaga,sonalsin,akbar.namin}@ttu.edu

Abstract. Hybrid quantum-classical programs often fail silently. A Quantum Approximate Optimization Algorithm (QAOA) MaxCut program can execute without exhibiting any faulty behavior and errors yet return a wrong answer because bugs can hide in independent components such as Hamiltonian construction, bitstring decoding, and cut-value calculation. Standard simulator feedback reports only the final outcome, such as a low approximation ratio or a wrong answer, without localizing the cause of failure or identifying which component failed. This paper investigated whether providing an LLM with component-level diagnostics improves its ability to repair, and thus enhance its own generated code. We conduct controlled experiments across 2,160 trials spanning six LLMs (`devstral:24b`, `granite4.1:30b`, `mistral-small3.2:24b`, `gemma4:26b`, `gpt-oss:20b`, and `nemotron3:33b`), six graph families (Erdős–Rényi, random 3-regular, complete, cycle, path, and ladder), and five graph sizes ($n \in \{4, 6, 8, 10, 12\}$). The research work compares three feedback designs: (1) Outcome-Level Feedback (OLF) (Arm A), (2) Component-Level Diagnostics (CLD) (Arm B), and (3) Diagnostics with Concrete Counterexamples (DCC) (Arm C). According to the result, Component-level diagnostic feedback (CLD) raises the final pass rate by 6.8% points (pp) overall and by 30% points (pp) for 'gpt-oss:20b', which is the only model to exceed 90% (62.5% to 92.5%). More detailed feedback costs fewer tokens, not more, because precise diagnostics cause programs to converge faster. Counterexample samples add a further 2.3 percentage points (pp) on average and help standard coding models more than reasoning models. These results show that the structure of feedback, not only its presence, determines how effectively an LLM repairs generated quantum programs.

Keywords: Quantum · Automated program repair (APR) · Large Language Model (LLM) · LLM-based code generation · Quantum Approximate Optimization Algorithm (QAOA) · MaxCut · Hybrid quantum-classical programs · Feedback granularity

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 376–394, 2027.
https://doi.org/10.1007/978-3-032-39763-8_22

1 Introduction

LLMs can now write quantum programs. Given a problem description and a target framework like Qiskit [8], a capable model can produce code. The generated code looks structurally correct, often runs without syntactical errors, and sometimes even returns the right answer, showing practical progress [5,13,17,19,20]. However, generating "sometimes correct" code is not equivalent to producing "reliably correct" code. The notable point is that the gap between the two is a software engineering problem, and not just a model capability issue.

The challenge of reliable code generation is more relevant to generating optimization code for hybrid quantum-classical programs. In classical software, a program that runs without exhibiting any faulty behavior (e.g., crashing) is usually close to what was intended, as runtime errors surface most bugs. In a Quantum Approximate Optimization Algorithm for MaxCut (QAOA-MaxCut) program, that assumption breaks down entirely [2,3,11]. The program can execute cleanly, sample from a quantum circuit, run a classical optimizer, and return a bitstring with a cut value, all without raising any faulty behavior or exception and still produce a wrong output. The core components of a QAOA-MaxCut program can fail independently and silently, producing no stack trace. A Hamiltonian construction error causes the circuit to solve the wrong problem. A decoding error reads the partition bitstring backwards. A cut-value error returns the wrong number. In each case, the program executes cleanly and returns an output, with no indication that anything went wrong.

This creates a concrete problem for any automated LLM-based workflow that uses feedback to improve the generated programs. If the only available signal is the outcome-level, such as "your answer is wrong" or "the approximation ratio is 0.4", the model has very little to work on and improve the code [10,12,16,21]. A Hamiltonian construction bug and a decoding bug produce identical outcome-level signals but require completely different fixes. More specifically, asking the model to re-check its output does not remove this observability problem because the feedback still does not identify the main cause of failure and the failing component. Without knowing which component failed, the model is randomly guessing without being aware of the existence of any failure.

The question this paper poses is as follows: *Does it help to provide feedback to the LLM model and inform it explicitly which component is wrong?* To answer this question, we conduct a controlled experiment comparing three feedback designs across six LLMs, six graph families, and five graph sizes, holding everything else constant. Arm A provides outcome-level feedback only. Arm B provides component-level diagnostics, identifying which component failed and why. Arm C adds a concrete counterexample to the component-level diagnostics. The prompt, generation settings, graph instances, and turn limit (i.e., the maximum number of LLM calls per trial) remain identical across all three arms. The only variable is the feedback the model receives after an unsuccessful round. This design allows us to investigate three research questions:

RQ1: Does component-level feedback improve correctness of outcomes over outcome-level feedback for LLM-generated QAOA-MaxCut programs, and if so, by how much?
RQ2: What token cost does component-level feedback incur relative to outcome-level feedback?
RQ3: Does addition of a concrete counterexample to component-level feedback further improve outcomes beyond per-component diagnosis alone?

According to the results we obtained, component-level feedback raises the final pass rate by +6.8 pp overall and by +30 pp for `gpt-oss:20b`. More detailed feedback costs only fewer tokens, not more, because precise diagnostics cause programs to converge faster. Counterexamples add a further +2.3 pp on average but do not consistently dominate per-component diagnosis alone. This paper makes two contributions:

1. A controlled empirical comparison of feedback designs for LLM-generated quantum-classical programs, to the best of our knowledge, among the first [1,5,13,17,18].
2. Empirical evidence that component-level feedback improves correctness and reduces token cost by localizing failures to semantic components, with Hamiltonian construction identified as the most repair-resistant component across all feedback designs.

The rest of the paper is organized as follows. Section 2 covers related work. Section 3 provides background. Section 4 describes the methodology. Section 5 presents the experimental setup. Section 6 reports results. Section 7 concludes with limitations and future work.

2 Related Work

2.1 LLM-Based Quantum Code Generation

Using LLMs to generate quantum programs has become an active research area. Vishwakarma et al. introduced the Qiskit Code Assistant [19], a model trained specifically for Qiskit code generation, alongside the Qiskit HumanEval benchmark [20] to evaluate functional correctness. QCoder [13] extended this by coupling generation with simulator-based feedback, where programs are executed and the agent receives execution status and wrong-answer signals to guide revisions. QAgent [5] proposes a hierarchical multi-agent architecture for autonomous OpenQASM generation with self-reflection. QuanBench+ [17] provides a multi-framework benchmark covering feedback-based repair across different quantum environments. For QAOA specifically, QAOA-GPT [18] addresses the efficient generation of QAOA circuit architectures, while LLM-Guided Ansatz Design [7] uses KL-divergence on output distributions as a feedback signal for iterative refinement. Multi-agent quantum code generation with Quantum Error Correction [1] includes a semantic analyzer that examines generated

programs across multiple passes. While all of these systems demonstrate that simulator feedback helps, none examine which kind of feedback helps, for which components, or at what token cost. Instead, feedback is evaluated at the outcome level such as execution status, wrong-answer flags, or distributional mismatch without isolating which component of the program caused the failure. To the best of our knowledge, controlled feedback comparisons in LLM-based quantum code generation have so far evaluated feedback at the outcome level; this paper is among the first to treat component-level diagnostics as the experimental variable for LLM-generated QAOA-MaxCut programs.

2.2 Feedback Structure and Quantum Software Testing

The paradigm of improving LLM outputs through iterative feedback was established by Self-Refine [12] and extended to agentic verbal reinforcement by Reflexion [16]. Both treat feedback as a monolithic signal. RLTF [10] showed that multi-granularity unit test feedback improves repair rates for classical code generation, because finer-grained signals reduce the search space of possible fixes. RLEF [6] grounds code LLMs in execution feedback via reinforcement learning. The Observability Gap [21] argues that output-level feedback is fundamentally insufficient for LLM coding agents because the same surface symptom can have many distinct root causes. This paper operationalizes that argument empirically in the quantum domain.

On the testing side, QuCheck [15] is a property-based testing framework for Qiskit programs, and QMutPy [4] applies mutation testing to quantum programs to evaluate test suite fault detection. Zhao [22] surveys the broader quantum software engineering landscape. The component-level checks in this work draw on the mutation testing methodology of QMutPy, but their purpose differs. While QuCheck and QMutPy evaluate test suites for human-written programs, our component-level checks serve as feedback signals to an LLM: the goal is not to judge a test suite, but to tell a code generator exactly which component failed and why.

3 Background

3.1 The MaxCut Problem

MaxCut is a classical combinatorial optimization problem where, given a graph $G = (V, E)$, the goal is to partition the nodes into two sets, S and $\bar{S}$, to maximize the number of edges crossing the partition (the *cut*). A graph with n nodes has 2^n possible partitions. Finding the exact optimum is generally NP-hard [9], meaning no known polynomial-time algorithm solves it for arbitrary graphs. For small graphs ($n \leq 12$), brute-force enumeration of all 2^n partitions is tractable and yields the exact optimum, a property this work exploits to provide ground-truth correctness checks without relying on approximation.

A partition is represented as a bitstring of length n: bit $i = 0$ places node i in S, and bit $i = 1$ places it in $\bar{S}$. The cut value of a bitstring $\mathbf{x}$ is the number of edges $(u, v) \in E$ for which $x_u \neq x_v$.

3.2 Quantum Approximate Optimization Algorithm (QAOA)

The QAOA [3] is a hybrid quantum-classical algorithm for combinatorial optimization. It encodes the MaxCut objective as a cost Hamiltonian acting on n qubits:

$$H_C = \sum_{(u,v)\in E} \frac{1}{2}\left(I - Z_u Z_v\right) \tag{1}$$

where Z_u is the Pauli-Z operator on qubit u and I is the identity. The expectation value $\langle H_C \rangle$ equals the expected cut value under the current quantum state; maximizing it finds a good partition [11].

At depth $p = 1$, QAOA is the simplest non-trivial instance of the algorithm and serves as the standard benchmark in LLM quantum code generation research [13,18]. This work uses QAOA-MaxCut as a well-defined programming task with independently checkable correctness criteria, rather than to evaluate QAOA as an algorithm.

3.3 Hybrid Quantum-Classical Programs in Qiskit

A QAOA-MaxCut program is hybrid, combining a quantum circuit executed on a simulator with a classical optimization loop running on a CPU. Programs here are written in Python using Qiskit 2.x [19], IBM's open-source quantum computing framework that provides high-level abstractions for building and executing quantum circuits without requiring low-level hardware knowledge.

A correct QAOA-MaxCut program must implement several components correctly and independently, including constructing the Hamiltonian with the right graph edges and coefficients, decoding the output bitstring with the correct endianness convention, and computing the cut value against the correct graph and partition.

Each component can fail independently and silently. A Hamiltonian construction error causes the circuit to solve the wrong problem without preventing execution. A decoding error reads the partition bitstring backwards without crashing the program. A cut-value formula error returns the wrong number without raising an exception. Unlike classical software bugs that typically surface as runtime errors, hybrid quantum-classical programs default to running cleanly while returning incorrect answers. When this happens, outcome-level feedback such as "wrong answer" gives an LLM insufficient information to locate and fix the error.

3.4 The Qiskit Endianness Convention

The decoding component deserves explicit treatment because it is the most common silent error in LLM-generated QAOA programs and the hardest to diagnose from outcome-level feedback alone. Qiskit's measurement results return bitstrings in big-endian qubit order: the leftmost character represents the highest-indexed qubit, not qubit zero. This opposes the natural MaxCut convention

where position i corresponds to node i. A program that reads Qiskit's bitstrings directly without reversing them computes cut values against the wrong partition assignment. The program executes, the optimizer converges, and a bitstring is returned, but every bit is indexed backwards.

Formally, for a bitstring $s = s_0 s_1 \cdots s_{n-1}$ returned by Qiskit and a graph $G = (V, E)$ with $|V| = n$ nodes, the correct partition assignment $\mathbf{x}$ requires a reversal:

$$x_i = s_{n-1-i} \quad \forall\, i \in \{0, \ldots, n-1\} \tag{2}$$

The cut value under the corrected assignment is:

$$\text{cut}(\mathbf{x}, G) = \sum_{(u,v)\in E} \mathbf{1}[x_u \neq x_v] \tag{3}$$

This convention is documented in Qiskit [19] but is easily missed, and LLMs trained on pre-2.0 Qiskit examples encounter it inconsistently. The decoding check detects this by testing two independent bitstrings from the measurement counts: if both yield strictly higher cut values when reversed, an endianness inversion is flagged.

4 Methodology

This paper investigates whether the structure of correctness feedback influences the effectiveness of an LLM in generating and repairing QAOA-MaxCut programs. The approach comprises three stages: (1) an intent model that specifies the task in a machine-checkable form; (2) a suite of independent component-level checks that evaluate the program output against the intent model; and (3) a refinement loop that delivers "*structured feedback*" across multiple LLM calls. Figure 1 illustrates the overall architecture.

4.1 Intent Model and Program Contract

A MaxCut optimization program takes a graph as input, constructs a quantum circuit encoding the problem, executes a classical optimizer to tune the circuit parameters, samples the circuit, and returns the best partition found. Each optimization round consists of an LLM call followed by execution and feedback. The rounds of executions continue until all component checks pass or the turn limit (i.e., budget), fixed at six LLM calls per trial, is exhausted.

The intent model is a machine-readable specification that anchors every optimization round. It records the target graph (n nodes and edge set), the algorithm settings (QAOA depth p, optimizer, shot count), and the required output convention. The same intent model serves as the reference for all component checks across all feedback arms, ensuring that every round is evaluated against the same ground truth.

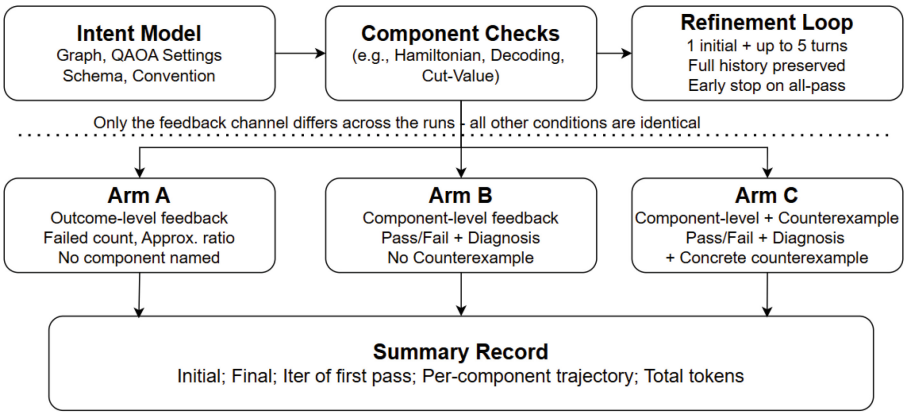


Fig. 1. Methodology overview. The intent model specifies the task. Component-level checks evaluate the program output against the intent model. The refinement loop delivers up to six LLM calls per trial. Arms A, B, and C are the only experimental variable; all other conditions are identical across arms.

The program contract requires the LLM to output a single JSON object to stdout containing fields for the graph encoding, the cost of Hamiltonian and mixer, the bound circuit, measurement counts, the bitstring-to-partition convention, and the best bitstring and cut value. Not all attributes (fields) are evaluated for correctness. Some attributes carry the core semantic content and are checked against the intent model; the remaining fields provide structured data that the checks read as input but do not independently verify. This uniform schema ensures that every optimization round produces output in a form that the component checks can evaluate consistently.

4.2 Semantic Component Diagnostics

A generated QAOA-MaxCut program can run successfully without producing correct outputs. To locate errors precisely, each core component of the QAOA-MaxCut program is mapped to a deterministic check that validates specific fields of the JSON output against the intent model. A deterministic check yields a consistent verdict for the same input, without relying on randomness or model-in-the-loop evaluation. Each check compares specific fields of the program's JSON output against the corresponding values in the intent model. It returns three items: (1) a pass/fail verdict indicating whether the component meets its correctness criterion, (2) a localized diagnostic string describing the deviation when the check fails, and (3) an optional concrete counterexample to assist with repairs.

Because the checks are independent, a program can satisfy a subset of attributes while failing others. This independence isolates feedback: since errors do not propagate across checks, the repair agent can target one component at a time without being misled by cascading failures.

4.3 Three Feedback Arms

An *arm* is a feedback condition: a fixed rule that determines what information the LLM receives after each optimization round where any component check fails. All three arms share the same intent prompt, conversation history format, sandbox settings, and turn limit. The only variable is the content of the feedback message.

A round is considered unsuccessful if at least one deterministic component check fails. This covers both execution failures, such as a runtime error, timeout, or unparseable output, and correctness failures, where the program runs cleanly but any component check does not pass. In either case, the feedback arm determines what information the LLM receives next.

Arm A: Outcome-Level. The agent receives the number of failed checks, the approximation ratio relative to the brute-force optimum, and the same generic repair instruction after each failed round. No specific component is named. This matches the feedback structure used in most prior quantum code generation work [7,13].

Arm B: Component-Level. The agent receives a pass/fail verdict and a localized diagnosis produced by each component check, but without counterexamples. For example:

> `[FAIL] hamiltonian: Z-support does not equal the graph edge set. Missing` Z_1Z_3 `for edge (1,3).`

Arm C: Component-Level with Counterexample. The agent receives the Arm B verdicts and diagnoses, plus a concrete counterexample produced by the failing component check. Counterexamples are generated automatically by comparing the intent model with the program output: for each failing check, the harness identifies the expected field value versus what was actually produced and formats this into a natural language statement. For example:

> `Edge (1,3) is in the intended graph but its` Z_1Z_3 `term is missing from your Hamiltonian.`

4.4 Refinement Loop

Each experimental trial covers one (model, arm, graph instance, seed) combination. A trial consists of one initial code generation in round $t = 0$ without prior feedback, followed by up to five correction rounds ($t = 1$ through $t = 5$), giving a maximum of six LLM calls per trial. We set the limit of five correction rounds based on pilot runs, where pass rates plateau before round five in most configurations, meaning additional rounds would add token costs without improving outcomes. The full conversation history including the initial intent prompt, generated programs, and feedback messages, accumulates across rounds. A trial terminates early as soon as all component checks pass. Upon termination, a summary record captures the initial and final pass verdicts, the round of the first full pass (if any), the per-component pass trajectory across rounds, and cumulative token counts. These records form the unit of analysis in Sect. 6.

Algorithm 1. Code refinement loop with component-level feedback

Require: Intent model $\mathcal{I}$, LLM $\mathcal{M}$, arm $a \in \{A, B, C\}$, graph G, seed σ, max rounds $T = 5$
Ensure: Summary record $\mathcal{S}$

$H \leftarrow [\text{system prompt} + \text{intent}(\mathcal{I})]$ ▷ H: conversation history
$r^* \leftarrow -1$ ▷ r^*: first passing round, -1 if none
for round $r = 0$ **to** T **do**
  $P \leftarrow \mathcal{M}(H)$ ▷ P: generated program
  $O \leftarrow \textsc{Execute}(P,\ 120\text{s})$ ▷ O: program output
  if O is valid **then**
    for each check C_k **do**
      $v_k \leftarrow C_k(\mathcal{I},\ O)$ ▷ v_k: pass/fail + diagnosis + counterexample
    end for
    if all v_k pass **then**
      $r^* \leftarrow r$; **break**
    end if
  else
    $v_k \leftarrow \varnothing\ \forall k$ ▷ Execution failed; checks skipped
  end if
  if $r < T$ **then**
    if O is not valid **then** $F \leftarrow$ execution-failure fallback
    else if $a = A$ **then** $F \leftarrow$ failed count + approx. ratio
    else if $a = B$ **then** $F \leftarrow$ verdicts + diagnoses
    else if $a = C$ **then** $F \leftarrow$ verdicts + diagnoses + counterexamples
    end if ▷ F: feedback message sent to LLM
    $H \leftarrow H + [P,\ F]$
  end if
end for
return $\textsc{Summarise}(r^*,\ \{v_k\},\ \text{tokens})$

Algorithm 1 formalizes the code refinement loop. Each round sends the current conversation history to the LLM, executes the generated program, evaluates the resulting JSON output with the component checks, and stops if all checks pass. If any check fails and the round budget remains, the next feedback message is constructed according to the assigned arm and appended to the conversation history.

5 Experimental Setup

All parameters are fixed in a single configuration file and reproduced verbatim from the replication package. All experiments were conducted on the SPHERE Research Infrastructure [14]

5.1 Graph Instance Sets

Graph instances are generated deterministically from a fixed master seed (value: 20260518) to ensure independent compute nodes produce identical sets. Six fam-

ilies are included: Erdős–Rényi ($p = 0.5$), random 3-regular, complete, cycle, path, and ladder graphs across five sizes $n \in \{4, 6, 8, 10, 12\}$. Deterministic families (complete, cycle, path, ladder) contribute one instance per size; stochastic families contribute two. Deduplication yields 40 unique instances. The brute-force MaxCut optimum is computed by exhaustive enumeration of all 2^n bitstrings before any LLM call, providing ground truth for the cut-value check's solution-quality criterion. This remains tractable through $n = 12$ (4,096 bitstrings).

5.2 Language Models Studied

Six models are served locally via Ollama at default Q4 quantization, partitioned into two families based on their primary design orientation:
Standard Family: `devstral:24b` (Mistral AI) is a coding-agent fine-tune optimized for code generation and repair tasks. `granite4.1:30b` (IBM) is a general-purpose enterprise model with a 128K context window.
`mistral-small3.2:24b` (Mistral AI) is a compact general-purpose model balancing capability and inference speed.
Reasoning Family: `gemma4:26b` (Google) is a model with instruction-following capability. `gpt-oss:20b` (OpenAI) is an open-weight reasoning model with configurable reasoning effort. `nemotron3:33b` (NVIDIA) is a reasoning model optimized for complex multi-step problem solving. Reasoning traces are disabled on every model that exposes a disable flag; `gpt-oss:20b` runs at the documented minimum reasoning effort. Any residual reasoning segments are stripped before execution. Identical generation settings apply across all arms and models: temperature 0.2, nucleus sampling cutoff 0.95, maximum output length 4,096 tokens, context window 8,192 tokens, per-call timeout 600 s, and two retries on transient errors. A model must produce at least one executable program in smoke mode to qualify for the full study; this requirement excluded models whose training data predates Qiskit 2.0 (released April 2025), which consistently hallucinate removed/old APIs regardless of the feedback signal.

5.3 Experimental Design and Execution Environment

The quantum framework is Qiskit 2.4.1. Generated programs use a statevector-based sampler for sampling, a sparse Pauli operator representation for Hamiltonian construction, and the Qiskit OpenQASM 3 serializer to export the bound circuit. Deprecated interfaces are explicitly forbidden in the intent prompt. QAOA hyperparameters are fixed across all runs: depth $p = 1$, 1024 shots, all qubits measured in the computational basis, and COBYLA (maximum 100 iterations) over the (γ, β) parameters. The cut-value check requires an approximation ratio ≥ 0.9 of the brute-force optimum.

The factorial design crosses six models, three arms, 40 graph instances, and three seeds per instance, yielding $6 \times 3 \times 40 \times 3 = 2{,}160$ trials. Each trial runs one initial generation plus up to five correction rounds, for a maximum of 12,960

LLM calls. Each program runs in a subprocess sandbox with a 120-second timeout. The sandbox strips markdown fences, extracts the last valid JSON output from stdout, and classifies failures as a runtime error, timeout, or unparseable output. On execution failure, all three arms receive an identical fallback message. The experiment was distributed across two GPU nodes (2× NVIDIA L40S 46 GB) with disjoint model subsets; results were merged post-hoc. The key design parameters are:

- 6 models (3 standard: `devstral:24b`, `granite4.1:30b`, `mistral-small3.2:24b`; 3 reasoning: `gemma4:26b`, `gpt-oss:20b`, `nemotron3:33b`)
- 3 feedback arms (A: outcome-level, B: component-level, C: component-level with counterexample)
- 6 graph families, $n \in \{4, 6, 8, 10, 12\}$, 40 instances after deduplication, 3 seeds per instance
- 6 LLM calls per trial maximum (1 initial + 5 correction rounds), with early termination on all-pass
- QAOA depth $p = 1$, 1024 shots, COBYLA (maximum 100 iterations), approximation threshold ≥ 0.9
- Models served via Ollama at default Q4 quantization to fit each model on a single 46 GB GPU

Design Decisions from Pilot Runs. Before committing to the full design, the harness was run in smoke-test mode against two-model subsets. Three observations shaped the final setup. First, `llama3.1:8b` produced zero executable programs across 54 iterations; each correction round yielded a different outdated API call rather than converging, because the model predates Qiskit 2.0. This established the executability inclusion criterion. Second, `gpt-oss:20b` initially used the Qiskit 1.0 keyword API rather than the Qiskit 2.x Primitive Unified Bloc (PUB) API, preventing component checks from running across all arms. We resolved this by including a Qiskit 2.x API reference template in the intent prompt, applied identically across all arms and models. Third, `granite4.1:30b` on Arm C re-emitted nearly identical code across three consecutive rounds despite receiving a correct counterexample, because the counterexample described the expected component output rather than the underlying indexing bug. This observation motivates the finding that Arm C does not strictly dominate Arm B, a nuance detailed in the results.

6 Results and Discussion

Throughout this section, we report pass rates as percentages and differences between pass rates as percentage points (**pp**), where one pp equals an absolute difference of one percentage point (for example, a change from 50.6% to 57.4% is a gain of +6.8 pp). Aggregate pass rates are computed as passing trials divided by total trials within an arm rather than as averages of per-model rates.

6.1 RQ1: Does Component-Level Feedback Improve Outcomes?

Table 1 reports the aggregated results across all six models. Independent trials under component-level feedback (Arm B) achieve a final pass rate of 57.4%, compared with 50.6% under outcome-level feedback (Arm A), a gain of +6.8+6.8 +6.8 percentage points. Independent trials under counterexample-augmented feedback (Arm C) achieve a final pass rate of 59.7%, a further 2.3 points over Arm B. The refinement gain increases monotonically from Arm A (24.6%) to Arm B (33.6%) to Arm C (36.9%), confirming that structured feedback improves outcomes regardless of starting capability. The median number of iterations to the first full pass drops from 0 under Arm A to 1 under Arms B and C, indicating that component-level feedback enables faster convergence. The small differences in initial pass rate across arms (26.0/23.8/22.8%) arise from generation stochasticity, since feedback is applied only after the initial round; they do not reflect a systematic effect.

Table 1. Aggregate results across all six models and 2,160 trials ($N = 720$ per arm). Arm A is the outcome-level feedback baseline; no separate no-feedback arm is included. Pass rate = passing trials/total trials. Values in %. Gain = Final − Initial. Med. rounds over passing trials only.

Arm	Initial	Final	Gain	Med.	Tokens
A	26.0	50.6	+24.6	0	23089
B	23.8	57.4	+33.6	1	21959
C	22.8	59.7	+36.9	1	21469

Across all arms, only 522 of 2,160 first attempts passed all checks (24.2%). Runtime errors accounted for 1,208 first attempts, and two more produced unparseable output. Among the 950 initially executable programs, Hamiltonian failures appeared in 346 cases (36.4%) and cut-value failures in 82 cases (8.6%). Thus, first-prompt failures came from both API-level execution errors and silent semantic defects, with Hamiltonian construction as the dominant semantic failure among executable programs.

6.2 RQ2: What Token Cost Does Component-Level Feedback Incur?

Component-level feedback does not increase token cost relative to outcome-level feedback. Token cost here includes prompt and output tokens accumulated across all rounds of a trial, including failed and passing rounds. As shown in Table 1, independent trials under component-level feedback consume fewer tokens on average than trials under outcome-level feedback (21,959 vs. 23,089). Independent trials under counterexample-augmented feedback achieve the lowest mean token cost (21,469).

Figure 2 plots the final pass rate against the mean tokens per run for each (model, arm) cell. The figure makes the cost-benefit relationship visible: moving from Arm A to Arms B and C shifts most models upward (higher pass rate) and leftward (fewer tokens). No model shows a token cost increase without a corresponding quality gain, which rules out the alternative explanation that Arms B and C simply spend more tokens to achieve better results.

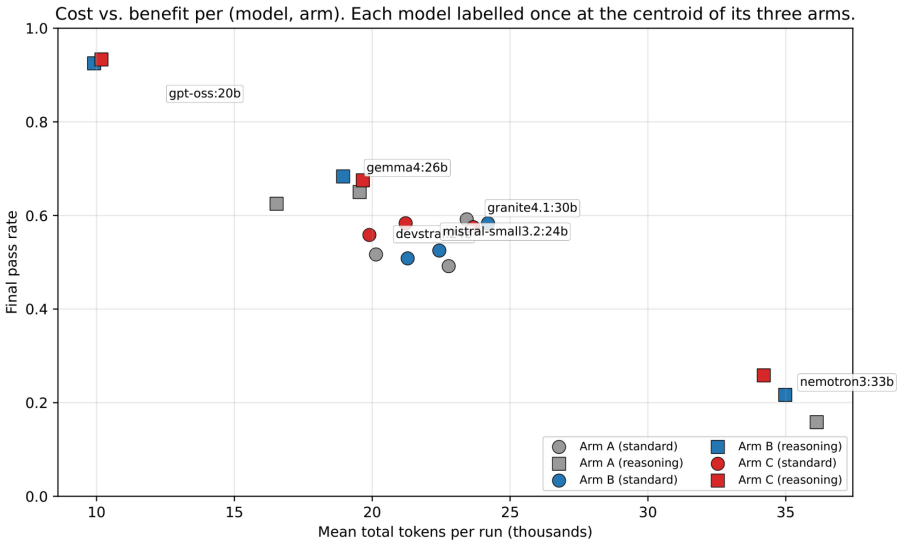


Fig. 2. Final pass rate vs. mean tokens per run per (model, arm) cell. Each model is labelled at the centroid of its three arms. Better feedback moves models up and left: higher quality at lower token cost. `gpt-oss:20b` is the only model above 90%; `nemotron3:33b` is the only model below 30%.

Figure 3 shows the cumulative pass rate trajectory across correction rounds 0–5 per model. The effect is model-dependent. `gpt-oss:20b` shows the sharpest contrast: Arm A reaches 62.5% and plateaus after iteration 1, while Arms B and C climb steeply to 92.5% and 93.3% respectively, which is a +30pp gap attributable entirely to feedback structure. `gpt-oss:20b` is the only model where component-level feedback brings the final pass rate above 90%, and its A→B gap of 30pp is the largest observed. This outlier behavior inflates the aggregate Arm B average; excluding `gpt-oss:20b`, the aggregate A→B gain drops from +6.8pp to approximately +3pp. The finding nonetheless holds directionally across five of six models. `devstral:24b` shows almost no A/B gap (51.7% vs. 50.8%) but a clear C advantage (55.8%). `nemotron3:33b` shows minimal separation across all three arms, converging near 16–26% regardless of feedback structure, suggesting a capability floor below which feedback granularity does not matter. The standard family converges faster (most gains occur at iteration 1) while the reasoning family continues improving through iteration 3 before plateauing.

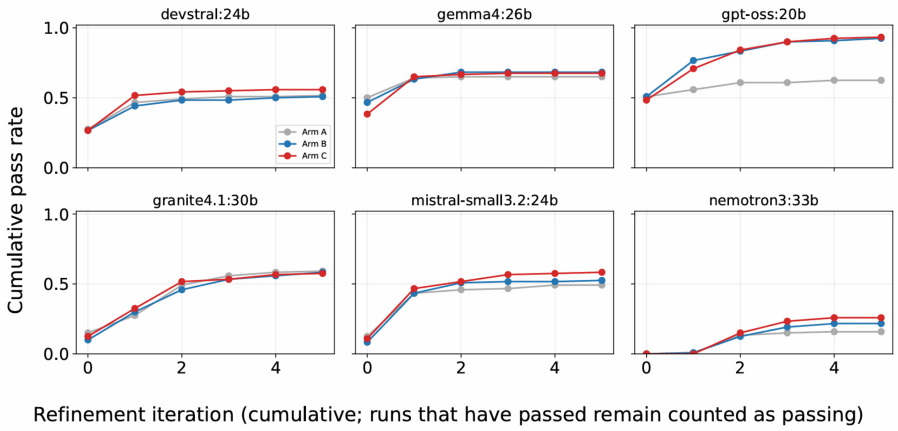


Fig. 3. Cumulative pass rate per model across refinement iterations 0–5. Arm A (gray) plateaus earliest. `gpt-oss:20b` shows the largest A/B gap (+30pp); `nemotron3:33b` shows near-zero separation across all arms.

6.3 RQ3: Does Adding Counterexamples Further Help?

For Arm C, the counterexample is produced by the component check itself by comparing the program output with the corresponding requirement in the intent model. Table 2 shows the per-model C−B delta. Four of six models show a positive delta (range: +0.8 to +5.8pp); two show a marginal negative delta (`gemma4:26b`: −0.8pp, `granite4.1:30b`: −0.8pp). The largest gains are in `mistral-small3.2:24b` (+5.8pp) and `devstral:24b` (+5.0pp), which are both standard-family models. The two reasoning models with the highest base capability (`gpt-oss:20b`: +0.8pp, `gemma4:26b`: −0.8pp) show near-zero C−B deltas, suggesting that at high capability the marginal value of a counterexample over a diagnosis alone is negligible. Arm C does not consistently dominate Arm B.

Table 2. Per-model Arm B vs. Arm C final pass rate (%). S = standard, R = reasoning. Direction = sign of $C - B$.

Model	Fam.	B	C	$C - B$	Dir.
`devstral`	S	50.8	55.8	+5.0	+
`granite4.1`	S	58.3	57.5	-0.8	−
`mistral-sm.`	S	52.5	58.3	+5.8	+
`gemma4`	R	68.3	67.5	-0.8	−
`gpt-oss`	R	92.5	93.3	+0.8	+
`nemotron3`	R	21.7	25.8	+4.2	+

Figure 4 shows how the per-arm gap evolves with graph size. The gap between Arms A and B/C is largest at $n = 4$ (Arm A: 64.6%, Arm C: 75.0%, gap: 10.4pp) and narrows as n increases. At $n = 12$, the B/C gap is under 4pp, while Arms B and C exceed Arm A by 6–10pp across all sizes, confirming that outcome-level feedback becomes increasingly inadequate as graph complexity grows. The B/C gap is stable across sizes, consistent with the counterexample having limited additional value regardless of instance difficulty.

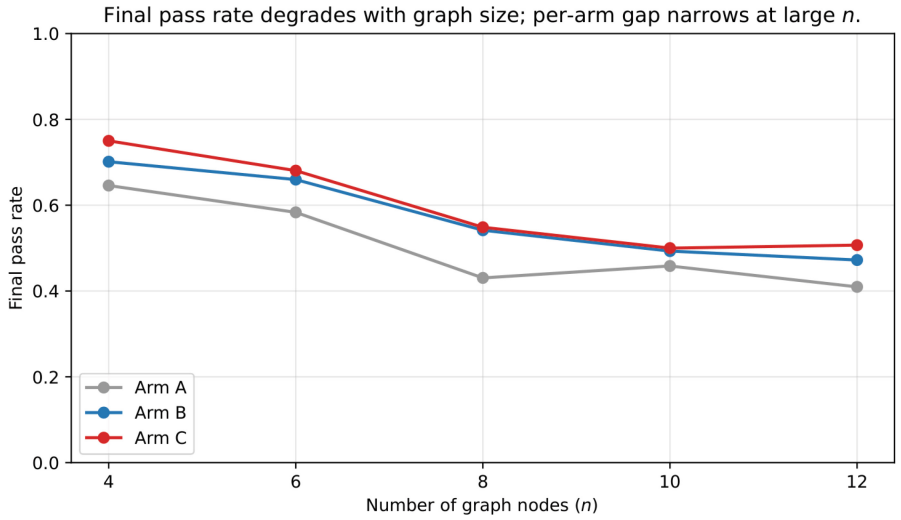


Fig. 4. Final pass rate vs. graph size n. Arm A degrades sharply from $n = 4$ to $n = 8$ and continues falling; Arms B and C degrade more gradually and track closely together across all sizes.

Discussion. Component-level feedback improves correctness outcomes over outcome-level feedback across five of six models. The main repair effect is semantic localization: among components that failed at round 0, Arm B repaired 46.1% of Hamiltonian failures versus 36.5% under Arm A, 64.6% of mixer failures versus 55.9%, and 65.7% of cut-value failures versus 57.6%. Arm C further improves cut-value repair to 67.0%, but does not materially improve Hamiltonian repair over Arm B. These trajectories support the interpretation that structured feedback helps by narrowing the candidate root causes rather than by simply prompting another revision. The largest effect occurs where the model has sufficient base capability to act on localized information (`gpt-oss:20b`: +30pp A→B; across all trials: +6.8pp). The token cost finding is consistent across arms: Arms B and C consistently cost fewer tokens than Arm A because programs receiving localized feedback converge in fewer iterations, reducing cumulative context length. Counterexamples provide a modest aggregate benefit (+2.3pp over Arm B), helping the standard family more than the reasoning family on average. Hamiltonian construction is the single most

repair-resistant component: its residual rate drops only 6pp from Arm A to Arm C compared to 9.3pp for cut-value. This suggests that Hamiltonian construction errors respond better to line-level fault localization or example-guided code repair than to component-level counterexamples [10,21]. `nemotron3:33b` represents a capability floor: with a 0% initial pass rate and a 16–26% final pass rate across all arms, feedback granularity has a negligible effect. This implies that below a certain base capability threshold, the feedback signal cannot be usefully acted upon regardless of its structure.[1]

7 Conclusion and Future Work

This paper presented a controlled empirical study of feedback granularity for LLM-generated QAOA-MaxCut programs. Across 2,160 trials spanning six models, three feedback arms, and 40 graph instances, component-level feedback consistently outperforms outcome-level feedback in final pass rate (+6.8pp across all trials, +30pp for `gpt-oss:20b`), refinement gain, and token cost. Better feedback costs fewer tokens because programs converge in fewer iterations. Counterexample-augmented feedback adds a modest further gain (+2.3pp) but does not consistently dominate component-level diagnosis alone, particularly for high-capability reasoning models. Hamiltonian construction remains the most repair-resistant component across all arms, graph sizes, and model families. Below a certain capability threshold (`nemotron3:33b`), feedback granularity has a negligible effect. To the best of our knowledge, this is among the first controlled comparisons of feedback designs for LLM-generated hybrid quantum-classical programs.

Limitations. Models are served through Ollama at default Q4 quantization to isolate LLM inference behavior from platform-level variables such as GPU memory pressure and fine-tuning artifacts. These results characterize quantized inference behavior rather than full-precision checkpoints, which matches the intended scope of this feedback-design study. The study covers QAOA-MaxCut at $p = 1$ on graphs up to $n = 12$. The component checks and intent model are designed for QAOA-MaxCut at $p = 1$; extending them to other variational algorithms or deeper circuits remains future work, though the feedback-design methodology transfers directly.

Future work. Several extensions follow directly from this study. First, while the current work reports pass-rate and token-cost differences as percentage points, a formal statistical treatment is a natural next step. Future work will fit a mixed-effects logistic regression modeling final pass as a function of feedback arm, with model and graph instance as random effects, to report odds ratios, confidence intervals, and significance tests. This mixed-effects logistic regression will quantify how much of the aggregate improvement is attributable to individual models

[1] `nemotron3:33b` also produces the highest token-per-passing-run values (132K–228K across arms), roughly 10× other models. This reflects the small number of passing runs rather than a higher per-call token cost.

such as `gpt-oss:20b`. Because feedback is introduced only after the first generation, the small differences in initial pass rate across arms reflect generation stochasticity rather than a systematic effect. A complementary direction is to report a conditional repair rate the fraction of round-0 failures that pass by the final round, to isolate the feedback effect independently of these initial differences. The correctness criterion can also be refined by separating implementation correctness from solution quality, distinguishing whether a program correctly computes the cut value of its own output from whether that output achieves a high approximation ratio. This refinement is valuable since at depth $p = 1$ a correct QAOA program is not guaranteed to reach a given approximation-ratio threshold. To further strengthen transparency, future work will provide a detailed specification of each component check the fields inspected, pass/fail criteria, diagnostic messages, and potential false-positive or false-negative cases. This future work will also include an expanded reproducibility package containing exact prompts, model version digests, graph instances, seeds, and raw result logs.

The component-level diagnostics studied here also have implications for software engineering education and training: future work will examine how localized, component-level feedback can help learners understand and locate silent semantic failures in hybrid quantum-classical programs, for example, as a diagnostic autograder for Hamiltonian-construction, endianness, and cut-value errors and will evaluate whether such structured feedback improves learning outcomes relative to outcome-level feedback. Beyond these analyses, the component-level check suite introduced here can serve as a reusable evaluation instrument for future quantum code generation work beyond this study. Extending the methodology to other hybrid quantum-classical programs, such as VQE or QAOA at $p > 1$, would establish whether these feedback-structure findings generalize beyond MaxCut. Replicating the experiment with API-served models like GPT-4o or Claude would test whether the findings hold beyond open-weight inference. Finally, deploying the methodology in a real quantum software development workflow would validate whether component-level diagnostics translate into productivity gains for human-in-the-loop quantum programming.

Ethical Considerations

This study involves no human subjects, personal data, or sensitive systems. All experiments use synthetically generated graph instances and LLM-generated programs. No proprietary models, private datasets, or restricted infrastructure were used.

Acknowledgments. The authors would like to thank the respected reviewers for their excellent comments to enhance the quality of this work.

References

1. Campbell, C., Chen, H.M., Luk, W., Fan, H.: Enhancing LLM-based quantum code generation with multi-agent optimization and quantum error correction, pp. 1–7 (2025)
2. Cerezo, M., et al.: Variational quantum algorithms. Nature Rev. Phys. **3**(9), 625–644 (2021). https://arxiv.org/abs/2012.09265
3. Farhi, E., Goldstone, J., Gutmann, S.: A quantum approximate optimization algorithm. arXiv preprint arXiv:1411.4028 (2014). https://arxiv.org/abs/1411.4028
4. Fortunato, D., et al.: Mutation testing of quantum programs written in QISKit. In: Proceedings of the IEEE/ACM International Conference on Software Engineering Companion (ICSE-C). IEEE (2022). https://doi.org/10.1145/3510454.3516839
5. Fu, Z., Chen, F., Jiang, L.: Qagent: An LLM-based multi-agent system for autonomous openqasm programming. arXiv preprint (2025). arXiv:2508.20134
6. Gehring, J., et al.: Rlef: Grounding code LLMs in execution feedback with reinforcement learning. arXiv preprint arXiv:2410.02089 (2024)
7. Gujju, Y., Harang, R., Shibuya, T.: LLM-guided ansätze design for quantum circuit born machines in financial generative modeling **2**, 104–108 (2025)
8. Javadi-Abhari, A., et al.: Quantum computing with qiskit. arXiv preprint arXiv:2405.08810 (2024)
9. Karp, R.M.: Reducibility among combinatorial problems. In: Miller, R.E., Thatcher, J.W. (eds.) Complexity of Computer Computations, pp. 85–103. Plenum Press, New York (1972)
10. Liu, J., et al.: RLTF: Reinforcement learning from unit test feedback. arXiv preprint arXiv:2307.04349 (2023). https://arxiv.org/abs/2307.04349
11. Lucas, A.: Ising formulations of many NP problems. Front. Phys. **2**, 5 (2014). https://arxiv.org/abs/1302.5843
12. Madaan, A., et al.: Self-refine: Iterative refinement with self-feedback. arXiv preprint arXiv:2303.17651 (2023). https://arxiv.org/abs/2303.17651
13. Mikuriya, R., et al.: QCoder benchmark: Bridging language generation and quantum hardware through simulator-based feedback. arXiv preprint arXiv:2510.26101 (2025). https://arxiv.org/abs/2510.26101
14. Mirkovic, J., Kocoloski, B., Balenson, D.: Enabling reproducibility through the SPHERE research infrastructure. In: Usenix; Login Magazine (2024)
15. Pontolillo, G., Mousavi, M.R., Grzesiuk, M.: Qucheck: A property-based testing framework for quantum programs in qiskit. ACM Trans. Quant. Comput. (2025)
16. Shinn, N., et al.: Reflexion: Language agents with verbal reinforcement learning. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 36 (2023). https://arxiv.org/abs/2303.11366
17. Slim, A., et al.: Quanbench+: A unified multi-framework benchmark for LLM-based quantum code generation. arXiv preprint arXiv:2604.08570 (2026)
18. Tyagin, N., et al.: QAOA-GPT: Efficient generation of adaptive and regular QAOA circuits. arXiv preprint arXiv:2504.16350 (2025). https://arxiv.org/abs/2504.16350
19. Vishwakarma, D.: Qiskit code assistant: Training LLMs for generating quantum computing code. In: Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE) (2024), qiskit Code Assistant
20. Vishwakarma, D.: Qiskit HumanEval: An evaluation benchmark for quantum code generation. In: Proceedings of the IEEE International Conference on Quantum Computing and Engineering, QCE (2024)

21. Wang, Y., Wang, C.: The observability gap: Why output-level human feedback fails for llm coding agents. arXiv preprint arXiv:2603.26942 (2026)
22. Zhao, J.: Quantum software engineering: Landscapes and horizons. arXiv preprint arXiv:2007.07047 (2020). https://arxiv.org/abs/2007.07047

Eye-RIS: Design and Evaluation of a Low-Latency Voice-Driven AR System for Real-Time Clinical Document Analysis on Smart Glasses

Robert John Luther Bennethum IV[1], Zainab Elahi Qazi[1], Ozge Selin Ak[1], Logan Swartz[1], Griffin Garman[1], Isiah Milien[1], Matthew Pragel[1], Paul Kocis[2], Sultan Al Nahian[1], and Sayed Mohsin Reza[1(✉)]

[1] Department of Computer Science, Pennsylvania State University Harrisburg, Middletown, Pennsylvania, USA
skr6024@psu.edu

[2] Pennsylvania State University College of Medicine, Hershey, Pennsylvania, USA

Abstract. The increasing adoption of smart glasses in clinical environments necessitates efficient, hands-free solutions for accessing and analyzing medical information in real time. This paper presents Eye-Reality Interactive System (Eye-RIS), a low-latency, voice-driven augmented reality (AR) system designed to enable real-time clinical document analysis on smart glasses. Eye-RIS integrates natural language voice interaction, large language model (LLM)-based document summarization, and keyword-based retrieval into a seamless heads-up display (HUD), thereby allowing healthcare practitioners to maintain focus on patient care while accessing critical information. The system is built on a scalable, cloud-native architecture that uses AWS services and a high-speed inference engine to achieve quick response times. A dual-stage processing pipeline combines OCR-based text extraction with semantic search to support both structured and unstructured clinical documents in PDF format. To ensure usability in clinical settings, the platform adopts a lightweight web-based interface compatible with available AR devices, enabling hardware portability and rapid deployment. We evaluated Eye-RIS through a combination of system-level testing, performance benchmarking, and user-centered usability studies. Experimental evaluation demonstrates that Eye-RIS achieves significant performance improvements, including substantial gains in processing speed and responsiveness, while maintaining reliable transcription accuracy. User feedback further indicates enhanced accessibility and reduced cognitive load during document interaction tasks. Our results suggest that voice-driven AR systems such as Eye-RIS can substantially improve clinical workflows by minimizing device switching and enabling real-time, context-aware document analysis. This work highlights the potential of combining AR, AI, and cloud computing to advance next-generation healthcare interfaces and provides a foundation for future research in low-latency intelligent wearable systems.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 395–414, 2027.
https://doi.org/10.1007/978-3-032-39763-8_23

Keywords: Augmented Reality (AR) · Smart Glasses · Clinical Document Analysis · Voice Interfaces · Low-Latency Systems · AI-Based Summarization

1 Introduction

1.1 Background and Motivation

The increasing adoption of augmented reality (AR) technologies and wearable computing devices has opened new opportunities for improving efficiency in complex, information-intensive domains such as healthcare [6,8]. Smart glasses, in particular, provide a heads-up display (HUD) that enables hands-free interaction and real-time access to contextual information, allowing clinicians to maintain focus on patient care. Prior studies have demonstrated the effectiveness of smart glasses in enhancing clinical workflows by enabling real-time visualization of patient data and remote collaboration support [9,25,27].

Despite these advancements, current clinical workflows (e.g. Clinical Notes [18]) remain heavily dependent on conventional screen-based systems such as desktop computers and tablets. These systems often require clinicians to divert attention away from patients, increasing cognitive load and introducing inefficiencies in decision-making. Research on electronic health record (EHR) systems highlights that fragmented interfaces and information overload significantly hinder clinical efficiency and usability [1]. Furthermore, emerging AI-driven clinical assistants have shown promise in improving documentation accuracy and automation, yet they often rely on intrusive or non-integrated workflows [21].

1.2 Problem Statement

Existing solutions for clinical document access and analysis are not optimized for hands-free, real-time use in healthcare environments. While smart glass-based systems and AI assistants have been explored, several limitations persist including high latency, lack of seamless integration between voice interaction and document processing, and dependence on hardware-specific implementations. Additionally, prior systems often rely on persistent cloud storage or batch processing approaches, raising both latency and privacy concerns [13,21].

Moreover, although smart glasses have been successfully deployed in training and surgical guidance scenarios, challenges such as ergonomic constraints, limited battery life, and integration complexity remain barriers to widespread adoption [2,3]. There is a clear need for a unified system that combines low-latency processing, voice-driven interaction, and AR-based visualization while maintaining scalability and data privacy.

1.3 Research Objectives

To address these challenges, this paper aims to design, implement, and evaluate a low-latency, voice-driven AR system that enables real-time clinical document analysis on smart glasses. The primary objectives of this research are as follows:

- To develop a system that supports seamless, hands-free interaction with clinical documents using natural language voice commands.
- To design a scalable, cloud-native architecture capable of delivering low-latency AI inference and real-time document processing.
- To ensure system portability across smart glass platforms through a lightweight, web-based interface.
- To evaluate the system's performance, reliability, and usability.

1.4 Contributions

This paper presents *Eye-RIS*, a novel system that integrates augmented reality, voice interaction, and AI-driven document analysis into a unified platform for clinical applications. The main contributions of this work are as follows:

- A low-latency, voice-driven AR system for real-time clinical document analysis on smart glasses.
- A cloud-native architecture combining OCR-based document processing, semantic search, and high-speed AI inference for efficient information retrieval.
- A lightweight, hardware-agnostic interface enabling deployment across commercially available smart glass devices.
- A comprehensive evaluation including system testing, performance benchmarking, and usability studies demonstrating the effectiveness of the proposed approach.

2 Related Work

2.1 AR and Smart Glasses in Healthcare

The application of AR and smart glasses in healthcare has gained attention due to their ability to provide real-time, hands-free access to information. Early studies demonstrated the effectiveness of devices such as Google Glass in facilitating cooperative medical training through remote guidance and shared visual contexts [27]. Similarly, implementations in hospital environments have shown that smart glasses can improve situational awareness and enable clinicians to access patient information without diverting attention from ongoing procedures [9].

More recent work has explored the use of AR in surgical education and telementoring, where real-time overlays and remote expert guidance enhance procedural accuracy and training outcomes [2,7]. Despite these advancements, prior systems are often tightly coupled to specific hardware platforms and lack generalized frameworks for integrating document-centric workflows within clinical environments. Furthermore, issues related to ergonomics, battery limitations, and system integration continue to hinder large-scale adoption [3].

2.2 Voice-Driven Clinical Assistance Systems

Voice-driven interfaces have emerged as a promising solution to reduce manual interaction and cognitive load in clinical workflows. AI-powered clinical assistants utilize natural language processing (NLP) to transcribe, summarize, and retrieve medical information, thereby improving efficiency in documentation tasks [5,21]. These systems enable clinicians to interact with data in a hands-free manner, which is particularly beneficial in sterile and high-intensity environments.

Additionally, vision-enabled AI scribing systems have demonstrated significant improvements in clinical documentation accuracy by incorporating both audio and visual context, achieving accuracy rates as high as 98% in controlled studies [13]. However, many existing systems rely heavily on persistent cloud storage and batch processing, introducing latency and raising concerns regarding data privacy and security.

2.3 AI-Based Document Analysis and Information Retrieval

Automated analysis of clinical documents has been an active area of research, particularly with the rise of OCR, semantic search, and retrieval-augmented generation (RAG) techniques. Prior work has focused on extracting structured and unstructured data from clinical records to support decision-making and reduce information overload. Studies on EHR visualization highlight the importance of presenting filtered, context-aware information to clinicians in a concise and accessible manner [1].

Recent systems have incorporated multimodal approaches combining OCR and object recognition to assist users in interpreting real-world environments and textual data [4,11]. While these systems demonstrate the feasibility of real-time document understanding, they often depend on computationally expensive pipelines or remote processing architectures, limiting their responsiveness in time-sensitive clinical scenarios.

2.4 Gaps in Existing Systems

Despite progress in AR, AI-driven clinical assistance, and document analysis, several critical gaps remain. First, existing approaches lack an integrated framework that combines low-latency document processing, voice-driven interaction, and AR-based visualization within a single system. Second, many solutions rely on persistent data storage and cloud-centric processing models that introduce latency and potential privacy risks. Third, current implementations are often hardware-specific, limiting portability across different smart glass platforms.

In contrast, the proposed Eye-RIS system addresses these limitations by introducing a unified, low-latency architecture that integrates OCR-based document analysis, semantic search, and voice interaction within an AR interface. By leveraging a cloud-native yet ephemeral processing model and a hardware-agnostic web-based design, the system enables real-time clinical document analysis while maintaining scalability, privacy, and usability.

3 System Overview and Architecture

3.1 Design Goals and Requirements

The design of Eye-RIS is guided by the need to support real-time, hands-free interaction with clinical documents in dynamic healthcare environments. Based on stakeholder requirements and limitations identified in prior work, the system is designed around the following key goals:

- **Low Latency:** Enable near real-time responses for voice queries and document processing.
- **Hands-Free Interaction:** Provide a natural language voice interface to minimize physical interaction.
- **Scalability:** Support deployment across multiple users and healthcare settings using a cloud-native architecture.
- **Hardware Portability:** Ensure compatibility with different smart glass platforms through a lightweight, web-based interface.
- **Privacy and Security:** Minimize data persistence and ensure secure transmission of clinical documents.

A comparative evaluation of commercially available smart glass platforms was conducted to select an appropriate device for the proposed system. Key factors considered include cost, display capability, and openness of the platform. Table 1 summarizes the comparison of representative devices.

Table 1. Comparison of Smart Glass Platforms

Product	Price	Open Source	Display	Field of View
XREAL One Pro [23]	$649	No	1920x1080 per eye	57°
Mentra Glasses [20]	$349	Yes	No display (camera only)	N/A (no display)
Even Realities G1 [26]	$599	Partial	Monochrome HUD	20° (HUD overlay)
Brilliant Halo [22]	$350	Yes	MicroOLED display	15°

Based on this analysis, the XREAL One Pro was selected due to its integrated display capabilities and compatibility with AR-based visualization, despite being a proprietary platform.

3.2 Overall System Architecture

Eye-RIS follows a multi-tier, cloud-native architecture that separates concerns across presentation, application, and data processing layers. As illustrated conceptually in Fig. 1, the system consists of three primary components:

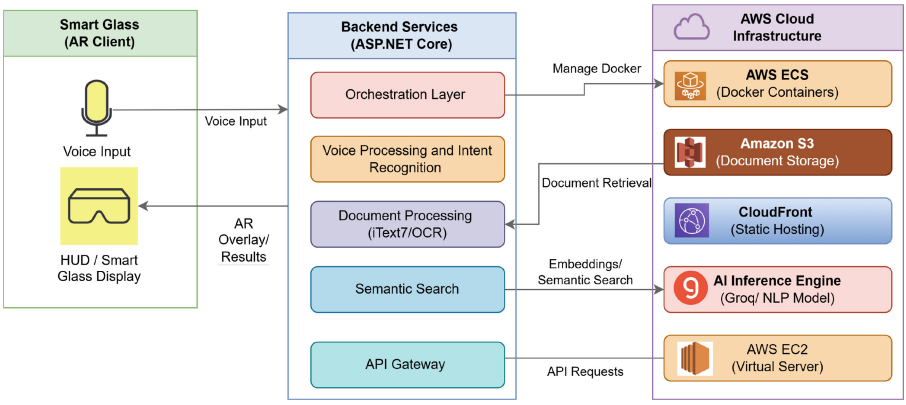


Fig. 1. Overview of the Eye-RIS system architecture illustrating interactions between the AR client, backend services, cloud infrastructure, and AI-based processing components enabling real-time clinical document analysis.

1. **AR Client Layer (Smart Glass Interface):** A lightweight client running on smart glasses provides the user interface through a heads-up display. This layer is implemented as a web-based application hosted within an Android WebView, enabling cross-device compatibility without reliance on proprietary SDKs.
2. **Application Layer (Backend Services):** The backend, developed using ASP.NET Core, acts as the central coordinator for processing user requests. It manages voice command parsing, document analysis tasks, and communication with external inference and data services.
3. **Data and AI Processing Layer:** This layer includes cloud-hosted services responsible for document processing, semantic search, and AI inference. It integrates OCR pipelines, vector-based search mechanisms, document summarization, and external APIs for domain-specific data retrieval. Among these, document summarization is handled by a separate LLM-based module exposed through an internal REST interface, allowing its underlying models to be updated independently.

This separation of concerns improves modularity, allowing independent scaling, maintenance, and evolution of system components.

3.3 Data Flow and Interaction Model

The interaction workflow in Eye-RIS is driven by voice commands and follows a sequential processing pipeline shown in Fig. 2:

1. The user issues a voice command through the smart glasses interface.
2. The audio input is transmitted securely to the backend for transcription and intent recognition.

3. The backend interprets the command and triggers the appropriate document processing or retrieval task.
4. For document-related queries, the system extracts text using OCR (if necessary) and performs semantic search over indexed content.
5. AI-based summarization and keyword highlighting are applied to generate concise, context-aware outputs.
6. The processed results are returned to the AR client and rendered as overlays within the user's field of view.

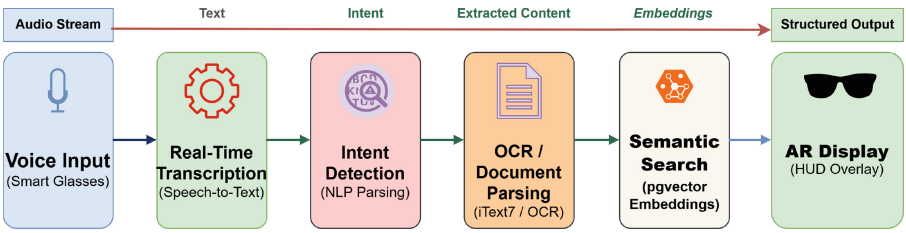


Fig. 2. End-to-end processing pipeline illustrating the transformation of voice input into structured AR output through transcription, intent detection, document parsing, and semantic retrieval.

3.4 Technology Stack

Eye-RIS leverages a combination of modern web, cloud, and AI technologies to achieve its design objectives:

- **Frontend:** React-based [29] web application integrated into an Android WebView for rendering AR-compatible UI.
- **Backend:** ASP.NET Core framework for handling API requests, orchestration logic, and service integration.
- **Cloud Infrastructure:** AWS ECS Fargate for containerized deployment, Amazon S3 and CloudFront for static hosting, and Terraform for infrastructure-as-code management.
- **Data Management:** PostgreSQL with pgvector extension to support semantic search over embedded document representations.
- **Document Processing:** iText7 for PDF parsing and Tesseract OCR for extracting text from scanned documents.
- **AI Inference:** Integration with a low-latency cloud inference engine to enable real-time transcription and summarization.

The combination of these technologies enables a scalable and extensible system architecture that bridges AR interfaces, AI-driven analytics, and cloud-based services.

3.5 System Limitations

Addressing the new technology of voice activated AR glasses required some workarounds. Opening file explorer was deemed impossible due to modern device security restrictions, which was replaced by opening previously uploaded files, 5 files in the case of the project. Microphone functionality also came with concerns and issues when it came to when it was on how long it stayed on and how to avoid picking up on background noise.

4 Methodology

4.1 Document Processing Pipeline

The core functionality of Eye-RIS relies on an efficient document processing pipeline capable of handling both structured and unstructured clinical documents in real time. The system primarily operates on PDF documents, which may contain either embedded text or scanned images. To address this heterogeneity, a dual-stage processing approach is employed, shown in Fig. 3.

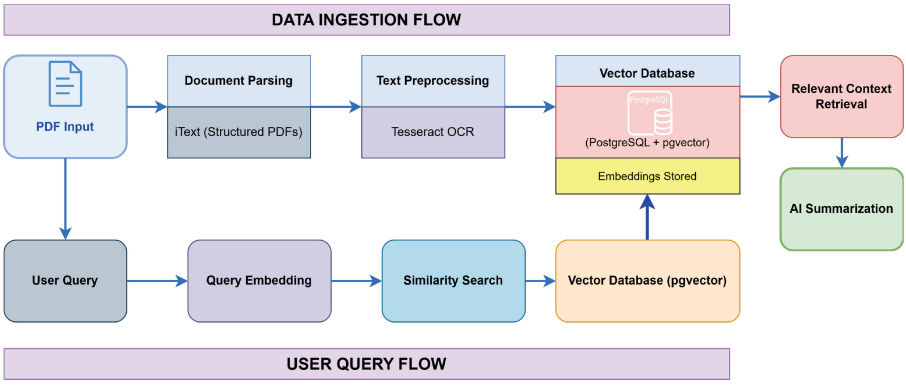


Fig. 3. Detailed AI-driven document processing pipeline illustrating end-to-end data transformation from PDF input to structured output.

First, documents with embedded text are parsed using the iText7 library to extract textual content along with positional metadata. This allows precise mapping of text regions for highlighting and overlay rendering within the AR interface. For scanned documents, the system utilizes Tesseract OCR to extract textual information from images. The extracted text is then normalized and preprocessed to remove noise and improve downstream processing accuracy.

To support efficient querying, documents are indexed and transformed into vector representations using embedding techniques. These representations enable semantic search capabilities, allowing the system to retrieve relevant information rather than relying on keyword matching. This approach ensures robustness across document formats and clinical use cases.

4.2 Voice Interaction and NLP Pipeline

Eye-RIS employs a voice-driven interaction model to facilitate hands-free operation in clinical environments. The voice processing pipeline is designed to minimize latency while maintaining high transcription accuracy.

Upon receiving a user's voice command, the audio signal is transmitted to a cloud-based inference engine for real-time transcription. The resulting text is then processed using natural language understanding techniques to identify user intent and extract relevant entities. Commands are categorized into predefined task types, such as document navigation, keyword search, summarization, and external data queries.

To ensure responsiveness, the pipeline adopts a lightweight intent classification mechanism combined with rule-based parsing for common command patterns. This hybrid approach balances efficiency and flexibility, allowing the system to handle both structured commands and natural language queries. The processed intent is then mapped to backend service handlers that execute the corresponding operations.

4.3 Semantic Search and Retrieval

To enable context-aware document analysis, Eye-RIS integrates semantic search capabilities based on vector similarity. After preprocessing, document content is encoded into dense vector embeddings and stored in a PostgreSQL database enhanced with the pgvector extension.

When a user issues a query, the system converts the query into an embedding and performs a similarity search against the indexed document vectors. This allows retrieval of semantically relevant passages even when exact keyword matches are absent. The retrieved content is then ranked and filtered based on relevance scores.

In addition to semantic retrieval, the system incorporates a retrieval-augmented processing strategy, where relevant document segments are passed to an AI inference model for summarization or contextual refinement. This approach improves the quality and relevance of responses presented to the user, particularly in complex clinical documents.

4.4 AR Interface and Smart Glass Integration

The AR interface is designed to provide intuitive and non-intrusive visualization of processed information within the user's field of view. Eye-RIS employs a web-based rendering approach using a React application embedded within an Android WebView, ensuring compatibility across multiple smart glass devices.

Processed outputs, including summaries, highlighted keywords, and navigation cues, are transmitted from the backend to the client as structured data. The client dynamically renders this information as overlays aligned with the user's perspective. This design minimizes computational load on the wearable device, delegating intensive processing tasks to the backend.

To enhance usability, the interface supports real-time updates and interactive feedback triggered by voice commands. The system ensures that visual elements remain lightweight and contextually relevant, preventing information overload while maintaining accessibility. This integration of AR visualization with backend intelligence enables seamless interaction between users and clinical documents in real time.

4.5 AI-Based Document Summarization

Summarization in Eye-RIS is handled by implementing a dedicated LLM-based module, exposed through a REST interface so that the AR system remains lightweight and model-agnostic. After the document processing pipeline extracts and normalizes text from a clinical PDF (Sect. 4.1), the relevant content is sent to the module over HTTP, and the resulting summary is returned to the AR client for rendering.

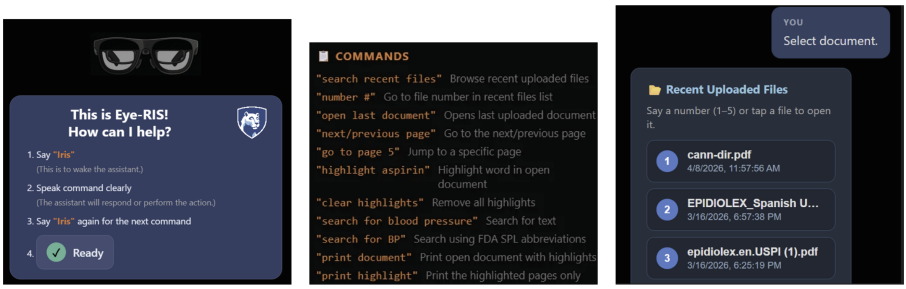
The component is implemented as a Python service that exposes endpoints for document summarization, in-context querying over a provided document, and medical information extraction. Summarization requests are routed by document length to one of two models: LongT5 [10] (tglobal-large), which uses transient global attention to handle long prescribing documents beyond standard transformer context windows [12], and MedGemma 4B, a health-focused model in the Gemma family. To support clinical safety, responses include page-level citations where possible, enabling manual verification. Drug information is obtained by querying an API used in this research [24] and passing the retrieved prescribing information to the model for summarization. The evaluation dataset consists of 139 sections of drug prescribing information documents retrieved from that API [24]. Documents were selected to cover a range of 72 drug classes, document lengths, and interaction complexity.

5 Implementation and Deployment

5.1 Frontend Design

The frontend of Eye-RIS is implemented as a lightweight web application using React and Vite, enabling efficient rendering and rapid development. To ensure compatibility with smart glasses, the application is embedded within an Android WebView environment, allowing the system to operate across multiple AR devices without relying on proprietary SDKs.

The user interface is designed to support minimal interaction, focusing on voice-driven commands and dynamic visual overlays. Core components include document viewers, command feedback indicators, and contextual highlighting modules shown in Fig. 4. The interface is optimized for low resource consumption to accommodate the computational limitations of wearable devices while maintaining responsiveness and clarity.



(a) Home Interface (b) Commands Window (c) Document View

Fig. 4. User interface of the Eye-RIS system showing (a) main interaction screen, (b) commands window view, and (c) document processing view on smart glasses.

5.2 Backend Services

The backend is developed using ASP.NET Core, providing a modular and scalable framework for handling API requests and service orchestration. The system adopts a microservice architecture, where services are responsible for tasks such as document processing, voice command handling, and data retrieval.

Task-specific handlers are implemented to process user commands, including navigation, keyword search, document summarization, and rendering instructions. The backend also integrates external APIs, such as drug information services, to support domain-specific queries. This modular design facilitates maintainability and extensibility, allowing new features to be incorporated with minimal disruption to existing components supporting long-term maintainability and collaborative software development, as reported in prior collaborative modeling studies [16,17].

5.3 Cloud Infrastructure and DevOps

Eye-RIS is deployed on a cloud-native infrastructure using Amazon Web Services (AWS). The backend services are containerized and hosted on AWS ECS Fargate, providing a serverless environment that automatically scales based on demand. The LLM-based summarization module is deployed separately as a Python service on an AWS EC2 instance to accommodate its model-serving requirements. Static frontend assets are hosted on Amazon S3 and distributed via CloudFront to ensure low-latency content delivery.

Infrastructure provisioning and configuration are managed using Terraform, enabling infrastructure-as-code (IaC) practices for reproducibility and consistency across deployment environments. The deployment pipeline is automated through GitHub Actions, which performs continuous integration and continuous deployment (CI/CD) tasks, including building container images, executing test suites, and deploying updated services to the cloud. This DevOps pipeline ensures rapid iteration and reliable deployment while maintaining system stability and version control.

5.4 Data Management and Processing

Data management in Eye-RIS is handled using a PostgreSQL database enhanced with the pgvector extension to support vector similarity search. Document embeddings are stored alongside metadata to enable efficient semantic retrieval. Document processing services utilize iText7 for parsing PDF content and Tesseract OCR for handling scanned documents. These processing tasks are executed on the backend to reduce computational overhead on the client device. Additionally, the system integrates external APIs for real-time access to clinical and pharmacological information, further enriching the analytical capabilities of the platform.

Although conversations are saved throughout use, restarting the project deletes any previous history. The documents are saved on the device they are uploaded on until deleted by user.

6 Evaluation

6.1 Experimental Setup

The evaluation of Eye-RIS was conducted to assess system performance, reliability, and usability in realistic usage scenarios. The system was deployed on a cloud-based infrastructure using AWS ECS Fargate, enabling controlled testing under scalable conditions. Experiments were performed using a dataset of clinical documents in PDF format, including both digitally generated and scanned files to evaluate robustness across heterogeneous inputs.

Voice interaction experiments were conducted using predefined clinical queries and natural language commands to simulate real-world usage. Performance measurements focused on latency, transcription accuracy, and system throughput. In addition, usability testing was carried out with participants interacting with the system via smart glasses to assess user experience.

Multiple commercially available smart-glass platforms were evaluated during system design and testing. The XREAL platform was ultimately selected due to its superior display quality, wider field of view, and compatibility with the proposed AR visualization interface. Alternative devices exhibited limitations such as restricted display capabilities, monochromatic presentation, or reduced integration flexibility.

6.2 Performance Metrics

To evaluate the efficiency and accuracy of the system, the following key performance metrics were considered:

- **Transcription Speed:** Measures the time required to convert voice input into text and process commands.
- **Word Error Rate (WER):** Evaluates transcription accuracy by comparing recognized text against ground truth.

- **Response Latency:** Captures the end-to-end time from voice input to AR display output.
- **System Throughput:** Assesses the system's ability to handle multiple requests.
- **Test Coverage:** Measures software reliability through unit and functional testing metrics.

6.3 System Testing and Reliability

To ensure robustness, Eye-RIS underwent extensive testing at both the unit and integration levels. We run tests for the backend system covering task handlers, services, and controller logic. In addition, we conduct functional test cases, executed to validate frontend and backend integration. These tests included command parsing validation, and document rendering checks.

6.4 Usability Study

A usability study was conducted to evaluate the practical effectiveness of Eye-RIS similar to state-of-the-art research [19]. Participants interacted with the system using smart glasses and completed a set of predefined tasks involving document navigation, keyword search, and information retrieval.

Tests were done on a number of people with varying degrees of education and understanding of the project throughout development. Formative usability testing was conducted throughout development with four participants, followed by a post-development evaluation involving ten users. This was to see how well the project could be maneuvered from start to finish and properly navigated with little to no guidance from the developers.

7 Results and Discussion

7.1 System Performance Analysis

The evaluation results demonstrate that Eye-RIS achieves substantial improvements in both speed and responsiveness compared to conventional document interaction workflows. The most notable outcome is the significant increase in transcription and processing speed, driven by the underlying ASR engine (Groq WT4-En-RE), which achieves up to 300× faster performance relative to baseline approaches. This improvement can be attributed to the integration of a low-latency inference engine combined with an optimized cloud-native architecture that eliminates cold-start delays.

In addition to speed, the underlying ASR engine maintains a relatively low WER of approximately 10.3%, indicating that the voice-driven interaction pipeline is sufficiently accurate for practical use in clinical environments. While further improvements are possible, this level of accuracy is adequate for command-based interaction and information retrieval tasks, particularly when

combined with contextual filtering mechanisms. Table 2 summarizes the performance metrics of the proposed system, including transcription speed and accuracy compared to baseline models. The baseline models and performance metrics utilized for evaluating Eye- RIS's voice-driven interaction pipeline are derived from the official Groq Automatic Speech Recognition (ASR) API documentation. Specifically, the system utilizes the specialized WT4-En-RE architecture (representing an optimized Whisper Large v3 implementation), which achieves a real-time speed factor of up to 300× faster performance and a Word Error Rate (WER) of 10.3%. For a broader comparative framework, this is benchmarked against a 'Community Model' (Whisper Large v3 Turbo), which demonstrates a speed factor of up to 247X faster performance. All quantitative throughput and accuracy metrics are grounded in independent, standardized industry benchmarks evaluated by ArtificialAnalysis.ai as of January 20, 2025

Table 2. Performance Evaluation Results

Metric	Model	Result
Transcription Speed	WT4-En-RE	300× Faster
Transcription Speed	Community Model	247× Faster
Word Error Rate (WER)	WT4-En-RE	10.3%

Overall, these results confirm that the system satisfies the primary design goal of enabling near real-time document interaction in a hands-free setting.

7.2 Usability and System Testing

The results of the usability study indicate that Eye-RIS provides an intuitive and effective user experience, particularly in scenarios requiring hands-free interaction. The usability of the Eye-RIS system was evaluated through a user study focusing on interaction efficiency and user experience. The results are summarized in Table 3. High user satisfaction scores suggest that the voice-driven interface reduces cognitive load and enables natural interaction with clinical documents.

The AR-based visualization further enhances usability by presenting information directly within the user's field of view, eliminating the need to switch between devices. This has important implications for clinical workflows, where maintaining focus on patient interaction is critical.

However, some challenges remain. While most users reported acceptable comfort levels, prolonged use of smart glasses may still lead to fatigue, highlighting the need for continued improvements in wearable hardware design. Additionally, variability in voice input quality and environmental noise may affect system performance in real-world settings.

Table 3. Usability and Testing Evaluation Results

Category	Metric	Result	Interpretation
Usability	Ease of Use	80%	High usability
	Comfort	70–85%	Acceptable wearability
	TTS Utility	70%	Effective navigation
Testing	Unit Tests	100%	All 24 unit tests passed
	Functional Tests	100%	All 36 functional tests passed

7.3 Software Quality and Testing Coverage

To ensure software quality and maintainability, the Eye-RIS system was evaluated using SonarQube Cloud [14,15] for static code analysis. The platform was used to identify security vulnerabilities, code smells, and reliability issues.

The analysis results shown in Table 4 indicates that the system successfully passed the quality gate, demonstrating compliance with key quality metrics. A total of 7 security issues, 54 reliability issues, and 340 maintainability issues were identified during the analysis. Importantly, no issues were marked as accepted, indicating that all identified issues were considered during development.

Code duplication was measured at 0.3%, reflecting a low level of redundant code and suggesting good modular design practices. Additionally, 67 security hotspots were detected, hotspots were reviewed and resolved/found non-issues.

Table 4. SonarQube [15] Quality Analysis Results

Metric	AR interface & Voice Interaction	AI-based Summarization
Quality Gate Status	Passed	Passed
Maintainability Rating	A	A
Security Issues	7	1
Reliability Issues	54	Not Measured
Accepted Issues	0	0
Security Hotspots	67	1
Code Duplication	0.3%	0%
Coverage	94.7%	89.4%

Table 4 shows a snapshot of the SonarQube dashboard summarizing the analysis results of two code modules.

7.4 AI Summarization Performance

The document summarization module was evaluated on 139 sections of real prescribing-information documents drawn from DailyMed, using two complementary measures. Summary quality was assessed with BERTScore [28] F1, an

embedding-based metric that captures semantic overlap with a reference summary and correlates more strongly with human judgment than n-gram measures. Latency was assessed separately as the per-document inference wall-clock time. As shown in Table 5, LongT5 achieved the highest BERTScore F1 (0.8514), ahead of MedGemma 4B (0.8175) and Gemma 4 E4B (0.8028). LongT5 also recorded the lowest inference time (4.33 s per document), compared with 10.39 s for Gemma 4 E4B and 17.13 s for MedGemma 4B.

Table 5. Summarization Benchmark Results (N = 139 documents)

Model	BERT Score F_1 (Mean)	Mean Inference Time (s)
LongT5 (tglobal-large)	0.8514	4.33
MedGemma 4B	0.8175	17.13
Gemma 4 E4B	0.8028	10.39

LongT5's stronger accuracy can be attributed to two factors. First, its transient global attention allows it to process long prescribing documents within a single context window without truncation, preserving more of the source content than models with shorter effective contexts. Second, its outputs tend to remain close to the source text, which aligns well with reference summaries drawn from the same documents and is rewarded by $BERTScore$; the more interpretive phrasing produced by the Gemma-family models, while sometimes useful, diverges more from the reference. The latency differences reflect architectural design: $LongT5$ is a compact encoderdecoder model optimized for efficient long-sequence summarization, whereas MedGemma 4B and Gemma 4 E4B are larger decoder-only models whose token-by-token generation is inherently slower. Given that LongT5 leads on both accuracy and latency, it was selected as the summarization model for interactive use, since the higher latency of the Gemma-family models makes them impractical for real-time wearable interaction.

7.5 Comparison with Existing Systems

To evaluate the efficacy of Eye-RIS within the broader landscape of clinical assistive technologies, its performance must be contrasted against existing wearable and voice-driven clinical systems. Current state-of-the-art implementations, such as smart-glass-based clinical documentation assistants or traditional cloud-tied voice dictation frameworks, consistently encounter significant bottlenecks in latency and operational rigidity. Traditional systems frequently rely on back-and-forth cloud-relay architectures, introducing palpable delays that disrupt natural patient-clinician interaction and require rigid, pre-defined voice command syntax.

In contrast, Eye-RIS bypasses these standard architectural constraints. Furthermore, while typical wearable clinical systems force users through cumbersome, multi-step visual menus to locate patient records, Eye-RIS's hands-free,

voice-driven context routing allows instant document retrieval without manual intervention. This direct comparison highlights that Eye-RIS does not merely iterate on internal software baselines, but significantly advances practical engineering benchmarks for real-time, wearable clinical deployments

8 Limitations and Threats to Validity

8.1 Technical Limitations

Despite its promising performance, Eye-RIS has several technical limitations. First, the system relies on stable network connectivity for cloud-based inference and data retrieval. Variations in network latency or service availability may impact response time. Second, while the use of external inference engines enables low-latency processing, it introduces dependencies on third-party services, which may affect long-term maintainability and cost.

8.2 Internal Validity

Internal validity concerns arise primarily from the experimental setup and implementation choices. The evaluation relies on predefined queries and controlled testing conditions, which may not fully capture the variability of real-world clinical interactions. Furthermore, the performance improvements reported for transcription speed and latency depend on specific infrastructure configurations, such as the use of optimized cloud services and inference engines.

Another potential threat is the tight coupling between system components during evaluation, where certain parameters (e.g., document formats and preprocessing pipelines) are optimized for the tested scenarios. As a result, performance outcomes may vary when applied to different datasets or configurations.

8.3 External Validity

External validity is limited by the scope of datasets and user studies. The system was evaluated using simulated clinical documents rather than real patient records due to privacy and regulatory constraints. While this approach ensures ethical compliance, it may not fully represent the complexity and variability of real-world clinical data.

Moreover, the usability study involved a relatively small and specific group of participants, which may limit the generalizability of the findings. Differences in user expertise, clinical roles, and environmental conditions could influence system performance and user experience in broader deployments.

8.4 Construct Validity

Construct validity relates to the choice of evaluation metrics and how well they reflect actual system effectiveness. While metrics such as WER, response latency,

and test coverage provide useful insights into performance, they do not fully capture all aspects of user experience or clinical impact.

For example, reductions in cognitive load and improvements in workflow efficiency are inferred from user feedback rather than directly measured through objective cognitive or behavioral metrics. Similarly, the effectiveness of semantic search is evaluated qualitatively rather than through standardized information retrieval benchmarks.

9 Conclusion

This paper presented *Eye-RIS*, a low-latency, voice-driven augmented reality system designed to enable real-time clinical document analysis on smart glasses. By integrating voice interaction, semantic document retrieval, and AR-based visualization within a unified platform, the system addresses key limitations of existing clinical workflows, including device switching, fragmented interfaces, and delayed information access.

The proposed architecture leverages a cloud-native design combined with a lightweight client interface to achieve scalability, portability, and high responsiveness. The integration of OCR-based document processing, vector-based semantic search, and real-time AI inference enables efficient handling of both structured and unstructured clinical documents. Experimental evaluation demonstrates that Eye-RIS achieves significant performance improvements, including substantial gains in processing speed and responsiveness, while maintaining reliable transcription accuracy. Furthermore, usability results indicate that the system enhances accessibility and reduces cognitive load, supporting effective hands-free interaction in dynamic environments.

Despite these promising results, several challenges remain before deploying such systems in real-world clinical settings. In particular, dependency on stable network connectivity, hardware limitations of current smart glasses, and variability in speech recognition accuracy present areas for further improvement.

Future work will focus on expanding the system's capabilities and addressing these limitations. First, we plan to incorporate support for additional languages and domain-specific medical terminology to improve accessibility and transcription accuracy. Second, we aim to integrate Eye-RIS with electronic health record (EHR) systems to enable direct access to patient-specific data and clinical workflows. Third, future research will explore the use of on-device or edge-based inference to reduce reliance on cloud services and improve system robustness under varying network conditions. Additionally, conducting large-scale user studies in real clinical environments will provide deeper insights into system usability and effectiveness. Finally, efforts will be directed toward developing a fully hardware-agnostic framework to ensure long-term adaptability as AR devices continue to evolve.

Overall, this work demonstrates the potential of combining augmented reality, AI-driven document analysis, and voice interfaces to transform clinical information access. Eye-RIS provides a foundation for next-generation intelligent

wearable systems that can enhance efficiency, accuracy, and user experience in healthcare and other data-intensive domains.

References

1. Investigating information visualization to combat information overload in electronic health records. JAMIA Open **5**(4), (2022)
2. Augmented reality in surgical education and telementoring. In: PMC (2025)
3. Simulating smart glasses for nursing workflow optimization. JMIR Formative Res. **9**, (2025)
4. Enhancing access for the visually impaired: A multisensory auditory approach. In: Proceedings of SPIE (2026)
5. Alboksmaty, A., Aldakhil, R., Hayhoe, B.W., Ashrafian, H., Darzi, A., Neves, A.L.: The impact of using ai-powered voice-to-text technology for clinical documentation on quality of care in primary care and outpatient settings: a systematic review. EBioMedicine **118**, (2025)
6. Baashar, Y., Alkawsi, G., Wan Ahmad, W.N., Alomari, M.A., Alhussian, H., Tiong, S.K.: Towards wearable augmented reality in healthcare: a comparative survey and analysis of head-mounted displays. Int. J. Environ. Res. Public Health **20**(5), 3940 (2023)
7. Cizmic, A., et al.: Telestration with augmented reality in minimally invasive and robotic-assisted surgery: a scoping review. Surg. Endosc., 1–14 (2025)
8. Garg, A., Reza, S.M., Babinski, D.: Enhancing ADHD diagnostic assessments with virtual reality integrated with biofeedback and artificial intelligence. In: 2025 8th International Conference on Information and Computer Technologies (ICICT), pp. 442–447. IEEE (2025)
9. Gong, X., Wong, A., Rashidova, K., Ng, P.: Smart glasses implementation in hospitals. Int. J. Business Strategy Autom. **2**(4), 1–9 (2021)
10. Guo, M., et al.: Longt5: Efficient text-to-text transformer for long sequences. In: Findings of the Association for Computational Linguistics: NAACL 2022, pp. 724–736 (2022)
11. Liu, Y.: Ocrbench: on the hidden mystery of OCR in large multimodal models. SCIENCE CHINA Inf. Sci. **67**(12), 220102 (2024)
12. Mall, S., Reza, S., Tran, T.K., Nguyen, H.: Enhancing knowledge base completion for long-tail entities through zero-shot learning. In: International Conference On Innovative Computing And Communication, pp. 535–549. Springer (2025)
13. Menz, B.: Vision-enabled ai scribes for clinical documentation: A randomized study. NPJ Digital Med. **9**(1), (2026)
14. Nocera, S., Fucci, D., Scanniello, G.: Dealing with sonarqube cloud: Initial results from a mining software repository study. In: 2025 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), pp. 372–378. IEEE (2025)
15. Nocera, S., Romano, S., Francese, R., Scanniello, G.: Software engineering education: Results from a training intervention based on sonarcloud when developing web apps. J. Syst. Softw. **222**, 112308 (2025)
16. Rahad, K., Badreddin, O., Mohsin Reza, S.: The human in model-driven engineering loop: A case study on integrating handwritten code in model-driven engineering repositories. Softw.: Practice Exper.**51**(6), 1308–1321 (2021)

17. Rahad, K., Badreddin, O., Reza, S.M.: Characterization of software design and collaborative modeling in open source projects. In: 9th International Conference Model-Driven Engineering and Software Development, pp. 254–261 (2021)
18. Rayabharam, N.V.S.M., Rahman, M.M., Hoque, R., Reza, S.M., Kabir, M.F.: Extracting latent insights and tagging fall injuries from clinical narratives using unsupervised learning. In: 2025 IEEE International Conference on Big Data (BigData), pp. 8004–8010. (2025). https://doi.org/10.1109/BigData66926.2025.11402491
19. Reza, S.M., Badreddin, O., Rahad, K.: Modelmine: a tool to facilitate mining models from open source repositories. In: Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings, pp. 1–5 (2020)
20. Silva, S., Atukorala, O., Samaradivakara, Y., Suriyaarachchi, H., Yeo, K.P., Nanayakkara, S.: AISEE++: An open-source rapid prototyping platform for screen-free heads-up computing. In: Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems, pp. 1–5 (2026)
21. Spachos, P., Gregori, S., Deen, M.J.: Voice activated IoT devices for healthcare: design challenges and emerging applications. IEEE Trans. Circuits Syst. II Express Briefs **69**(7), 3101–3107 (2022)
22. Tharushika Perera, K., Pantidi, N., Regenbrecht, H.: Why do you know that about me?—ethical implications of pervasive AR and face recognition. In: Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems, pp. 1–17 (2026)
23. Tsuchikawa, A., Manabe, H.: Looquestion: A head pointing-based QA system for AR glassesusing LLM-generated candidates. In: Proceedings of the Augmented Humans International Conference 2026, pp. 825–827 (2026)
24. van Uhm, J., van Haelst, M.M., Jansen, P.R.: Ai-powered test question generation in medical education: The dailymed approach. medRxiv pp. 2024–11 (2024)
25. Wang, B.: A systematic literature review on integrating ai-powered smart glasses into digital health management for proactive healthcare solutions. NPJ Digital Med. **8**(1), 410 (2025)
26. Ye, H., Hu, T., Gorlatova, M.: No guide, no cheat: detecting smart glasses via AR optical signatures. In: Proceedings of the 27th International Workshop on Mobile Computing Systems and Applications, pp. 91–96. (2026)
27. Yoon, H., Kim, S., Lee, Y., Choi, J.: Google glass-supported cooperative training for health professionals. J. Multidiscip. Healthc. **14**, 1451–1462 (2021)
28. Zhang, T., Kishore, V., Wu, F., Weinberger, K.Q., Artzi, Y.: BERTscore: Evaluating text generation with BERT. In: International Conference on Learning Representations
29. Zhou, X., Hu, W., Liu, G.P.: React-native based mobile app for online experimentation. In: 2020 39th Chinese Control Conference (CCC), pp. 4400–4405. IEEE (2020)

VALS: A Model-Agnostic Virtual Reality Framework for Real-Time Annotation of 3D Anatomical Models

Pranav Balachander[1], Lasya Madhuri Gundapaneni[1], Yamini Satish[1], Khush Mistry[1], Evan Goldman[2], and Sayed Mohsin Reza[1(✉)]

[1] Department of Computer Science, Pennsylvania State University Harrisburg, Middletown, Pennsylvania, USA
skr6024@psu.edu

[2] Pennsylvania State University College of Medicine, Hershey, Pennsylvania, USA

Abstract. Traditional anatomy education relies on two dimensional diagrams and static physical models, which fundamentally limit ability to develop an intuitive understanding of complex three-dimensional spatial relationships. This paper presents VALS (Virtual Reality Anatomy Labelling System), a model-agnostic virtual reality framework developed for the Meta Quest 3 headset that enables anatomy instructors and students to load, manipulate, and annotate three-dimensional anatomical models in an immersive environment. VALS supports runtime loading of arbitrary *.OBJ* model files directly from headset-local storage, real-time model manipulation via Meta Quest controller input, and an interactive pin-based labeling system that incorporates a Trie-based search structure to support low-latency anatomical term lookup during text input. Label data is persisted locally through a lightweight JSON serialization system, enabling seamless session continuity without the overhead of re-exporting mesh geometry. The framework is implemented in Unity 6 using C# with OpenXR and the Meta XR SDK, allowing deployment on standalone consumer hardware without reliance on cloud infrastructure. Performance evaluation on the Meta Quest 3 demonstrated consistent rendering at 72 FPS under standard usage conditions, near-instantaneous model loading (under 1 s), and label placement latency below 0.5 ms. Trie-based anatomical term retrieval completed in under 10 ms. A structured usability study confirmed that the core instructor workflow: loading a model, placing labeled pins, and saving a session can be completed within minutes by first-time users. VALS addresses a clear gap in accessible, immersive anatomy tooling by providing a lightweight and extensible VR framework for real-time annotation of user-defined anatomical data.

Keywords: Virtual Reality · 3D Anatomical Visualization · Interactive Annotation · Model-Agnostic Systems · Medical Education Technology · Standalone VR · Trie-Based Search · Unity

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 415–429, 2027.
https://doi.org/10.1007/978-3-032-39763-8_24

1 Introduction

Traditional anatomy education relies on two-dimensional diagrams, cadaveric dissection, and static physical models, which fundamentally limit students' ability to develop an intuitive understanding of complex three-dimensional spatial relationships [21]. While these approaches remain foundational, they impose significant constraints on accessibility, scalability, and spatial cognition [11]. Two-dimensional visualizations fail to adequately capture depth and occlusion, cadaver-based training is resource-intensive and logistically constrained, and physical models lack the flexibility for annotation, customization, and distribution. As medical curricula increasingly seek scalable and interactive learning solutions, there is a growing need for technologies that enhance spatial understanding while remaining accessible on widely available hardware [6,11].

Virtual Reality (VR) has emerged as a promising platform for immersive learning by enabling users to interact directly with three-dimensional content in a spatially meaningful way. Prior work has demonstrated that immersive environments can improve anatomical understanding, particularly in tasks that require depth perception and spatial reasoning [3,18]. However, existing VR-based anatomy applications are typically limited by closed ecosystems that provide only predefined model libraries and restricted interaction capabilities. In particular, many systems lack support for user-defined models and annotation mechanisms, which are essential for instructor-driven teaching workflows and learning experiences. This represents a significant gap between the capabilities of current VR tools and the practical needs of educational and clinical users.

To address this challenge, this paper presents *VALS (Virtual Reality Anatomy Labelling System)*, a model-agnostic virtual reality framework designed for interactive annotation of three-dimensional anatomical models on standalone VR devices. VALS enables users to load arbitrary 3D models at runtime, interact with them using intuitive controller-based input, and annotate structures through an immersive pin-based labeling system. Unlike prior systems that depend on preloaded datasets or external cloud services, VALS operates entirely on-device, allowing deployment on consumer-grade hardware without additional infrastructure requirements. This design prioritizes both accessibility and performance, making the system suitable for real-world educational settings.

From a software engineering perspective, VALS introduces several key design and implementation contributions. The framework integrates a runtime model import pipeline for handling arbitrary `.OBJ` files, a real-time interaction model based on raycasting for precise spatial input, and a Trie-based search mechanism that enables efficient retrieval of anatomical terminology during label creation. Additionally, the system employs a lightweight JSON-based persistence layer that records annotation data without requiring reprocessing of geometric content, thereby reducing storage overhead and improving load performance. These components are organized within a modular architecture that separates rendering, interaction, and data management concerns, supporting extensibility and maintainability.

The system is implemented using Unity 6 [16] with C# and OpenXR [19], and evaluated on a Meta Quest 3 standalone headset. Experimental results demonstrate that VALS achieves real-time performance under typical usage conditions, including stable rendering at interactive frame rates, near-instantaneous model loading, and low-latency label placement and search operations. A structured usability evaluation further indicates that novice users can successfully complete core annotation workflows with minimal training, suggesting that the system is both practical and intuitive for instructional use.

The main contributions of this paper are summarized as follows:

- A model-agnostic VR framework that supports runtime loading and interactive annotation of arbitrary 3D anatomical models on standalone devices.
- A pin-based labeling interaction model that integrates spatial input with flexible text-based annotation in immersive environments.
- Integration of a Trie-based search structure to support low-latency anatomical terminology lookup within the annotation workflow.
- A lightweight persistence approach based on JSON serialization for efficient storage and retrieval of annotation data.
- A comprehensive evaluation demonstrating real-time performance and usability on consumer-grade VR hardware.

2 Related Work

2.1 VR in Medical and Anatomy Education

Virtual Reality (VR) has been widely explored as a tool for improving spatial understanding in anatomy and medical education. Moro et al. (2017) conducted a comparative study of augmented reality, virtual reality, and traditional desktop-based systems, demonstrating that immersive VR environments can significantly enhance spatial knowledge acquisition compared to conventional methods [9]. Similarly, Makransky and Lilleholt (2018) emphasized the importance of cognitive load management in immersive environments, showing that well-designed VR interfaces can improve both engagement and learning outcomes [7]. Earlier work by Nicholson et al. investigated the effectiveness of three-dimensional virtual anatomical models and identified interactivity and spatial navigation as key factors influencing educational performance [10]. These studies collectively establish VR as a valuable platform for anatomy education, particularly when systems are designed to maximize immersion, interactivity, and usability.

2.2 Interactive Annotation in VR Medical Contexts

Beyond visualization, research has increasingly focused on enabling interaction and annotation within VR environments. Gsaxner et al. (2025) presented a VR-based system for expert-in-the-loop annotation of medical images, demonstrating

improvements in annotation quality and reduced user fatigue compared to traditional two-dimensional interfaces [5]. Palmas et al. (2022) provided a comprehensive survey of VR tools used for medical analysis, highlighting various interaction paradigms for segmentation, annotation, and collaborative exploration [12]. A consistent observation across these works is that most systems are domain-specific and rely on predefined datasets, limiting flexibility for end users. While these approaches demonstrate the feasibility of immersive annotation, they do not provide generalized workflows for annotating arbitrary three-dimensional models in standalone VR environments.

2.3 Standalone VR Performance and Usability

The performance and usability of VR systems, particularly on standalone headsets, remain critical factors influencing adoption in educational contexts. Freina and Ott (2015) identified key barriers to the use of VR in education, including hardware cost, motion sickness associated with low frame rates, and steep learning curves for non-technical users [2]. Martirosov and Kopecek (2017) further emphasized the importance of system-level design decisions such as polygon count, texture resolution, and rendering pipeline optimization in maintaining stable performance on resource-constrained devices [8]. These findings highlight the need for VR systems that balance visual fidelity, responsiveness, and ease of use discussed in other research [4], particularly when deployed on consumer-grade standalone hardware without reliance on external computing resources.

2.4 Search and Data Structures for Medical Terminology

Efficient retrieval of medical terminology is an important component of interactive annotation systems. Prefix-based data structures such as Tries have been widely applied in biomedical informatics to support fast lookup of complex terms. Bodenreider (2004), in the context of the Unified Medical Language System (UMLS), demonstrated that Trie-based search reduces lookup complexity from linear to prefix-dependent time, enabling efficient retrieval even for long and multi-syllabic anatomical terms [1]. Such properties make Trie-based approaches particularly suitable for real-time applications where responsiveness is critical, including interactive VR systems that rely on text input during user workflows.

2.5 Research Gap

Despite substantial progress in VR-based anatomy education and interactive medical visualization, a clear gap remains. Existing systems either focus primarily on visualization without supporting structured annotation, or provide annotation functionality within closed, dataset-specific environments. Furthermore, many approaches depend on high-end hardware or cloud-based infrastructure, limiting accessibility and deployability in typical educational settings. There is a lack of model-agnostic, standalone VR frameworks that allow users to import

arbitrary three-dimensional anatomical models and perform real-time annotation within an integrated and efficient workflow. VALS addresses this gap by combining runtime model import, immersive interaction, efficient search mechanisms, and lightweight persistence into a unified system designed for deployment on consumer-grade standalone VR devices.

3 Methodology

3.1 System Architecture

VALS adopts a layered architecture shown in Fig. 1 designed to support real-time interaction, modular system components, and efficient data handling within a standalone virtual reality environment. The system follows a separation of concerns similar to a Model View Controller (MVC) paradigm, adapted to the requirements of immersive VR applications.

The architecture is composed of three primary layers: the presentation layer, the interaction and control layer, and the persistence layer. The presentation layer consists of Unity scenes, three-dimensional anatomical models, and interactive label elements, and is responsible for rendering the virtual environment. The interaction and control layer handles user inputs from the VR controllers, processes raycasting operations, and manages system state transitions such as label creation and UI interactions. The persistence layer is responsible for storing and retrieving annotation data using lightweight serialization mechanisms.

The system accepts multiple forms of input, including `.OBJ` model files, optional material files, preprocessed anatomical terminology data, and VR controller events. These inputs are processed by core system modules, including the runtime import pipeline, interaction engine, search module, and label management subsystem. Outputs include rendered anatomical models, interactive annotations, and serialized session data.

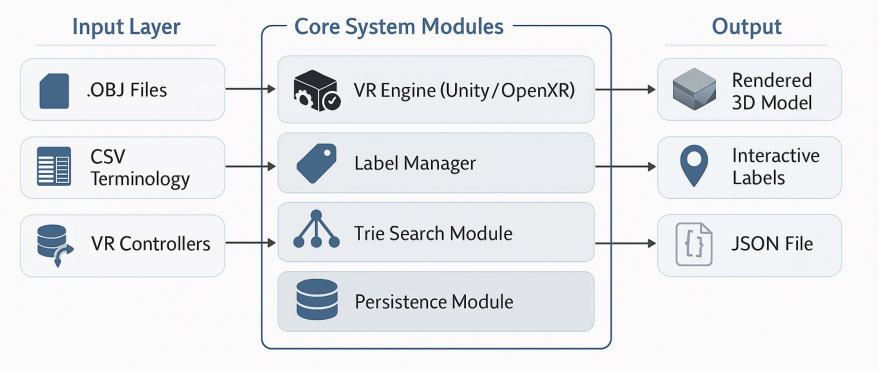


Fig. 1. Overview of the VALS architecture illustrating data flow between input sources, core system modules, and output components.

3.2 System Design and Core Components

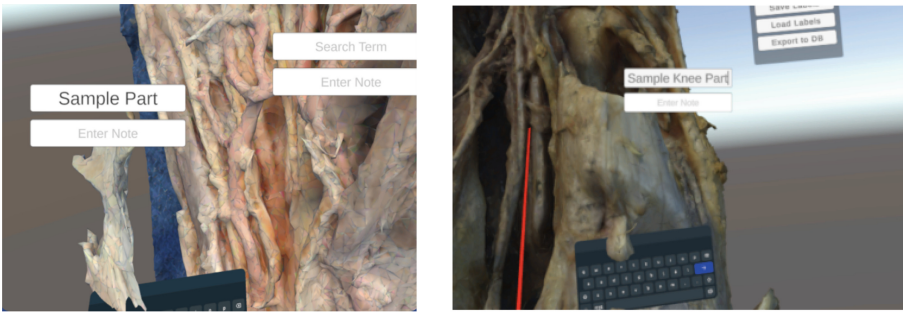
This section describes the key system components that enable real-time interaction, annotation, and data persistence in VALS.

Runtime Model Import and Data Pipeline. VALS supports runtime loading of arbitrary three-dimensional anatomical models in `.OBJ` format directly from the local storage of the VR headset. The import pipeline parses geometric data, including vertices, normals, and faces, and constructs corresponding Unity mesh objects dynamically at runtime. Optional material and texture data are loaded if available, with fallback mechanisms to ensure robustness in cases of missing or incomplete resources.

This model-agnostic design allows instructors to use externally sourced anatomical datasets without requiring pre-processing or conversion into proprietary formats. The imported model is instantiated within the scene hierarchy, enabling subsequent interaction and annotation operations.

Interactive Labeling Mechanism. The core interaction paradigm in VALS is based on a pin-based labeling system. Users can place annotation markers on the surface of a 3D model by directing a controller-based pointer and triggering a raycast interaction. When a valid surface intersection is detected, a label prefab is instantiated at the corresponding world-space coordinates. Full Pin placement on an anatomical model showing interaction states are shown in Fig. 2.

Each label consists of a spatial marker represented by a 3D object and an associated user interface panel for text input. The UI panel is initially hidden to minimize visual clutter and can be toggled through interaction. This separation between spatial anchoring and textual content enables efficient rendering while maintaining usability.



(a) Collapsed sphere state (b) Expanded label interface

Fig. 2. Pin placement on an anatomical model showing interaction states: (a) collapsed sphere state; (b) expanded label popup with search and notes fields.

Trie-Based Anatomical Search. To support efficient entry of anatomical terminology, VALS integrates a Trie-based search structure [17] preloaded from a dataset of anatomical terms. During user input, the system performs prefix-based traversal of the Trie and dynamically generates autocomplete suggestions.

This approach enables real-time response performance, as the lookup time depends primarily on the length of the input query rather than the size of the dataset. The system allows users to quickly identify and select complex anatomical terms with minimal typing, significantly improving the usability of text input in VR environments.

Persistence Model. VALS employs a lightweight persistence mechanism based on JSON serialization to store annotation data. Each label is represented as a structured data object capturing its spatial coordinates and associated text fields. When a save operation is triggered, all active labels are serialized and written to a local file within the device storage.

During loading, the system reconstructs the scene by parsing the stored JSON file and re-instantiating label objects at their recorded positions. This approach avoids the need to reprocess geometric mesh data, resulting in faster load times and reduced storage overhead.

Interaction and Controller Mapping. User interaction in VALS is facilitated through the Meta Quest controller interface. Specific controller inputs are mapped to system operations such as model manipulation, label creation, label deletion, and UI interaction. Raycasting is used to determine precise interaction points on the model surface, enabling accurate placement of annotation markers.

The interaction design prioritizes simplicity and predictability, allowing users to perform core tasks with minimal learning overhead. The mapping of controller inputs is consistent across system operations to enhance usability.

3.3 Experimental Setup

Hardware and Software Environment. The system was evaluated on a Meta Quest 3 standalone VR headset equipped with a Snapdragon XR2 Gen 2 processor and 8 GB of memory. Development and deployment were performed using Unity 6 on Windows-based systems, with applications deployed to the device via Android Debug Bridge (ADB). The runtime environment relies on OpenXR and the Meta XR SDK for input handling and rendering.

Test Models. The evaluation utilized multiple anatomical models in `.OBJ` format, including representations of structures such as joint systems and soft tissue regions. Specifically, the evaluated models ranged from approximately 10,000 to 100,000 vertices/polygons, with texture resolutions where applicable; exact per-model specifications are provided in the project repository.

Baseline Metrics. Performance baselines were defined based on system requirements for interactive VR applications. These included minimum frame rate thresholds for comfortable user experience, acceptable model load times, and latency constraints for interaction and annotation operations. These baselines served as reference points for evaluating system performance.

User Study Design. The usability study involved six participants recruited from Computer science and one from medical field, with some reporting prior experience with VR or gaming applications. This composition reflects a proxy for the system's target users (anatomy instructors and students). A structured usability study was conducted in which participants performed a sequence of tasks representing a typical instructor workflow similar to other state-of-the-art research [15]. These tasks included loading a model, placing multiple labeled annotations, saving the session, and reloading the annotations. Task completion time and qualitative feedback were recorded to assess usability and system effectiveness.

3.4 Implementation Details

Technology Stack. VALS is implemented using Unity 6 [20] with C# as the primary programming language. The system utilizes OpenXR as the underlying runtime framework, along with the Meta XR SDK for device-specific interaction and rendering. Additional components include libraries for *.OBJ* file parsing, TextMeshPro for user interface elements, and built-in JSON utilities for data persistence.

Raycasting and Interaction Architecture. A key design consideration in the system is the separation of physics-based and UI-based interaction mechanisms. Physics raycasting is used for detecting intersections with three-dimensional objects, while a separate graphical raycasting system handles user interface interactions. This separation ensures reliable event handling and avoids conflicts between 3D object selection and UI input.

Ethics, Privacy, and Safety The system does not process or transmit any personal or patient data. All anatomical models used in evaluation are sourced from non-clinical datasets, and all data processing occurs locally on the device. No network connectivity or user account is required, ensuring privacy and minimizing security risks during operation.

4 Results

4.1 Performance Metrics

All performance measurements were conducted on a Meta Quest 3 standalone headset under standard usage conditions, including a single anatomical model

loaded and multiple active annotation labels. The evaluation focused on key system metrics relevant to real-time VR interaction, including rendering performance, model loading time, interaction latency, and data persistence operations and the detailed results are shown in Table 1.

Table 1. Performance evaluation results compared with baseline targets

Metric	Measured Value	Baseline Target	Status
Render Framerate	72 FPS (average)	$\geq$ 60 FPS	Pass
Model Load Time	$<$ 1 s	$<$ 10 s	Pass
Label Placement Latency	$<$ 0.5 ms	50–100 ms	Pass
Trie Search Latency	$<$ 10 ms	$<$ 100 ms	Pass
Save/Load Time (JSON)	$<$ 1 s	$<$ 3 s	Pass

The system consistently achieved interactive frame rates exceeding the minimum threshold required for comfortable VR experiences. The observed average rendering rate of 72 FPS indicates stable performance under typical workloads, even with multiple annotations present in the scene. Model loading was effectively near-instantaneous for the evaluated datasets, reflecting the efficiency of the runtime import pipeline and local storage access.

A notable result is the label placement latency, which was measured at less than 0.5 milliseconds. This performance is significantly below the baseline target and demonstrates the efficiency of the prefab-based instantiation mechanism. Similarly, Trie-based search operations exhibited sub-10 millisecond response times, enabling real-time autocomplete functionality during user interaction. Representative user interface interactions are shown in Fig. 3.

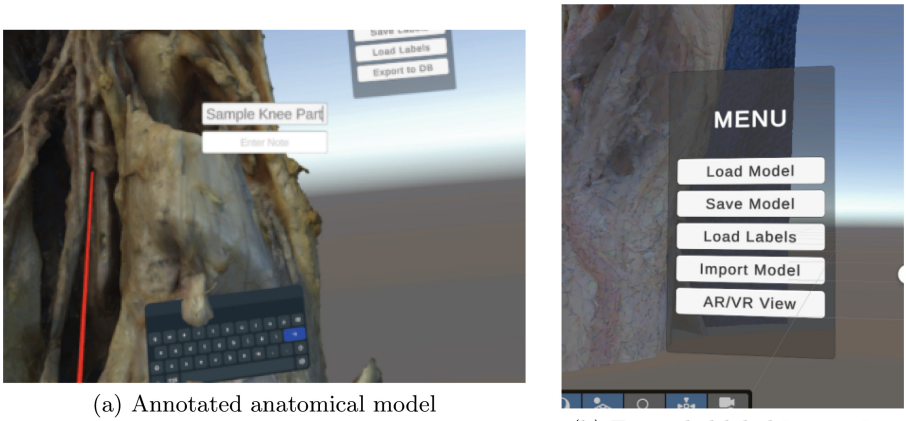
Overall, the performance evaluation indicates that VALS satisfies real-time interaction requirements on resource-constrained standalone VR hardware.

4.2 Usability Study Findings

A structured usability evaluation was conducted to assess the effectiveness and learnability of the system and summary results are shown in Table 2. Participants were asked to complete a sequence of tasks representing a typical instructional workflow, including loading a 3D model, placing labeled annotations on anatomical structures, saving the session, and reloading previously stored annotations.

All participants were able to complete the workflow without external assistance, indicating that the system supports intuitive interaction. The pin-based labeling mechanism was consistently reported as easy to use, and the autocomplete functionality provided by the Trie-based search system reduced the effort required for entering complex anatomical terms.

Qualitative feedback highlighted the simplicity of the user interface and the effectiveness of the VR interaction model. Some participants noted minor challenges related to label placement precision on curved surfaces, which can be



(a) Annotated anatomical model

(b) Expanded label interaction

Fig. 3. VALS running on Meta Quest 3 showing annotated anatomical structures in immersive environment.

Table 2. Summary of user study task performance

Task	Average Time	Success Rate
Load Model	5–10 s	100%
Place 5 Labels	1–2 min	95%
Save Session	1–5 s	100%
Reload Session	2–4 s	100%

attributed to the use of collider-based raycasting. Additionally, initial confusion regarding controller input mappings was observed in a small number of cases, though this was resolved quickly through on-screen guidance.

These findings suggest that VALS provides a usable and accessible interaction model suitable for both instructors and students in immersive learning environments.

4.3 Requirement Alignment

The developed system was evaluated against the functional and non-functional requirements defined during the system design phase. Core functional requirements, including runtime model import, interactive annotation, session persistence, and VR-based interaction, were fully implemented and validated during testing. These results are summarized in Table 1.

Performance-related requirements, such as real-time rendering, low-latency interaction, and efficient data storage, were also satisfied, as demonstrated by the results in Table 1. Certain advanced features originally considered during early design stages, such as multi-model comparison and level-of-detail management, were deferred to future work due to implementation complexity.

Overall, the system meets the primary objectives of providing a model-agnostic, real-time annotation framework within a standalone VR environment.

4.4 Repository and Reproducibility

To support reproducibility and further development, the complete source code, build instructions, and test artifacts for VALS are publicly available in an online repository https://github.com/pruhnav/capstoneproject2026. A snapshot of the version corresponding to this paper is archived on Zenodo at the following link: https://doi.org/10.5281/zenodo.21087256. The repository includes the full Unity project, implementation scripts, sample anatomical datasets, and documentation describing the build and deployment process.

The system can be reproduced by cloning the repository, importing the project into Unity, and deploying the application to a compatible VR headset using standard development tools. All required dependencies and configuration steps are documented to facilitate replication of the experimental setup.

This commitment to open-source development supports transparency and enables future research and extension of the proposed framework.

5 Discussion

5.1 Interpretation of Results

The experimental results demonstrate that VALS achieves consistent real-time performance across all evaluated metrics on standalone VR hardware. The system maintains stable rendering performance above the minimum threshold required for comfortable immersive interaction, while also achieving low latency in key operations such as model loading, label placement, and search. These findings indicate that the architectural design choices–particularly the use of lightweight prefabs for annotation and JSON-based persistence–are effective in minimizing computational overhead.

The near-instantaneous model loading and sub-millisecond interaction latency suggest that the system is well-suited for interactive educational workflows, where responsiveness is critical for maintaining user engagement. Additionally, the performance of the Trie-based search mechanism confirms that prefix-based data structures are highly effective for real-time term retrieval in VR environments, even when dealing with complex anatomical terminology. Overall, the results validate that VALS can deliver a seamless user experience without relying on high-end computing resources or external infrastructure.

5.2 Implications for VR-Based Medical Education

The results of this study have several important implications for the use of VR in anatomy education. First, the ability to load and annotate arbitrary three-dimensional models enables a flexible, instructor-driven workflow that is not

constrained by predefined datasets. This represents a shift from static visualization tools toward interactive, customizable learning environments.

Second, the usability findings indicate that even first-time users can quickly adopt the system and perform core tasks with minimal training. This suggests that immersive VR tools such as VALS can be effectively integrated into existing educational practices without introducing significant learning barriers. The combination of spatial interaction and contextual annotation supports a more active learning process, allowing students to explore and engage with anatomical structures in a way that is not possible with traditional methods.

Finally, the system's deployment on standalone consumer hardware lowers the barrier to adoption, making advanced VR-based educational tools accessible to a wider range of institutions. This is particularly relevant in contexts where access to high-end computing infrastructure is limited.

5.3 System Strengths

One of the primary strengths of VALS is its model-agnostic design, which allows users to import and work with arbitrary anatomical datasets without modification. This flexibility distinguishes the system from many existing VR applications that rely on fixed model libraries, several of which incur licensing or hardware costs, limiting accessibility for smaller institutions.

Another key strength is the integration of data structures and lightweight system components, which enable real-time interaction on resource-constrained hardware. The use of a Trie-based search mechanism significantly improves the usability of text input in VR, while the JSON-based persistence model ensures fast and reliable storage of annotation data. A feature comparison between existing VR-based anatomy systems [5,12] and VALS is shown in Table 3.

Table 3. Feature comparison between existing VR-based anatomy systems and VALS

Feature	Existing VR Tools	VALS
Custom Model Import	Limited	Yes
Annotation Support	Yes	Yes
Standalone VR Deployment	Often Not Supported	Yes
Real-time Search	Not Supported	Trie-based Search

From a software engineering perspective, the modular architecture of the system promotes separation of concerns and supports future extensibility [13, 14]. The clear division between rendering, interaction, and data management components facilitates maintainability and allows new features to be integrated without requiring major structural changes.

5.4 Limitations

Despite its strengths, the system has several limitations that should be acknowledged. First, this study did not include a controlled comparison against non-VR instructional methods to directly quantify learning gains, though prior work suggests immersive environments improve spatial understanding [7,10]. Second, the accuracy of label placement is affected by the use of collider-based raycasting, which may introduce minor positional offsets on curved or low-resolution surfaces. Potential mitigations include normal-aware projection that adjusts pin orientation to local surface curvature, multi-sample raycasting to average intersection points near edges, or post-placement snapping to the nearest high-confidence vertex; these were not implemented in the current version but represent a direct path for improving placement precision. This limitation impacts precision in certain use cases and suggests the need for more advanced surface interaction techniques.

Also, the evaluation was conducted using a set of anatomical models and user study participants. As a result, the generalizability of the findings may be constrained. Further evaluation with a broader range of datasets and participant groups would strengthen the validity of the results. Furthermore, latency measurements were collected under a moderate number of active labels (fewer than 50); system behavior at significantly larger label counts (hundreds to thousands), including potential impacts on save/reload time, was not evaluated. Additionally, the performance results reported in Table 1 reflect average values; tail-latency behavior (e.g., p_{99} framerate and operation latency) was not captured during evaluation and is left for future characterization, particularly under sustained or high-load usage conditions.

Finally, while the system demonstrates strong performance on standalone hardware, it has not been extensively tested under extreme conditions, such as very large model sizes or highly complex scenes. Future work should explore these scenarios to further validate system robustness.

6 Conclusion

This paper presented VALS, a model-agnostic virtual reality framework for interactive annotation of three-dimensional anatomical models on standalone VR hardware. The system was designed to address the limitations of traditional anatomy learning tools and existing VR applications by enabling users to load arbitrary models, interact with them in an immersive environment, and create structured annotations without reliance on external infrastructure.

The results demonstrate that VALS achieves real-time performance across all core system operations, including rendering, model loading, label placement, and search. The system consistently met or exceeded baseline performance requirements on a Meta Quest 3 headset, confirming that advanced interactive workflows can be supported on consumer-grade VR devices. The usability evaluation further showed that the core instructor workflow can be completed by first-time

users with minimal training, highlighting the practicality and accessibility of the system.

Several design decisions were critical to the success of the framework. The pin-based annotation mechanism effectively balances spatial interaction and visual clarity, while the Trie-based search approach enables efficient retrieval of complex anatomical terms during real-time use. Additionally, the adoption of a lightweight JSON-based persistence model ensures efficient storage and retrieval of annotation data without introducing unnecessary computational overhead. Together, these components form a cohesive system that supports interactive and extensible use in educational environments.

VALS contributes to the growing body of research in immersive learning technologies by demonstrating that flexible, user-driven annotation systems can be implemented efficiently in standalone VR settings. The model-agnostic nature of the framework provides a foundation for integrating diverse anatomical datasets, supporting a wide range of instructional use cases beyond fixed or preloaded content.

Future work will focus on extending the capabilities of the system to support more advanced interaction scenarios. This includes improving label placement accuracy through more precise surface interaction techniques, enabling multi-model workflows for comparative analysis, integrating additional input modalities such as voice-based annotation, and supporting toggleable anatomical layers (e.g., skeletal, muscular, circulatory) to emulate traditional layered physical models. Further evaluation with larger datasets and a broader range of users will also strengthen the validation of the system and its applicability in real-world educational contexts.

References

1. Bodenreider, O.: The unified medical language system (umls): integrating biomedical terminology. Nucleic Acids Res. **32**(Database issue), D267–D270 (2004)
2. Freina, L., Ott, M.: A literature review on immersive virtual reality in education: State of the art and perspectives. In: eLearning and Software for Education (eLSE), pp. 133–141 (2015)
3. García-Robles, P., Cortés-Pérez, I., Nieto-Escámez, F.A., García-López, H., Obrero-Gaitán, E., Osuna-Pérez, M.C.: Immersive virtual reality and augmented reality in anatomy education: a systematic review and meta-analysis. Anat. Sci. Educ. **17**(3), 514–528 (2024)
4. Garg, A., Reza, S.M., Babinski, D.: Enhancing adhd diagnostic assessments with virtual reality integrated with biofeedback and artificial intelligence. In: 2025 8th International Conference on Information and Computer Technologies (ICICT), pp. 442–447. IEEE (2025)
5. Gsaxner, C., et al.: Interactive ai annotation of medical images in a vr environment. Int. J. Comput. Assist. Radiol. Surg. (2025)
6. Kılıç, M.F., et al.: A new classmate in anatomy education: 3d anatomical modeling medical students' engagement on learning through self-prepared anatomical models. Anat. Sci. Educ. **18**(7), 727–737 (2025)

7. Makransky, G., Lilleholt, L.: A structural equation modeling investigation of the emotional value of immersive virtual reality in education. Educ. Technol. Res. Dev. **66**(5), 1141–1164 (2018)
8. Martirosov, S., Kopecek, P.: Virtual reality and its influence on training and education – literature review. In: 28th DAAAM International Symposium. DAAAM International (2017)
9. Moro, C., Štromberga, Z., Raikos, A., Stirling, A.: The effectiveness of virtual and augmented reality in health sciences and medical anatomy. Anat. Sci. Educ. **10**(6), 549–559 (2017)
10. Nicholson, D.T., Chalk, C., Funnell, W.R.J., Daniel, S.J.: Can virtual reality improve anatomy education? a randomised controlled study of a computer-generated three-dimensional anatomical ear model. Med. Educ. **40**(11), 1081–1087 (2006). https://doi.org/10.1111/j.1365-2929.2006.02611.x, https://asmepublications.onlinelibrary.wiley.com/doi/abs/10.1111/j.1365-2929.2006.02611.x
11. Niu, S., Zhang, J., Lin, J., Wang, B., Yan, J.: Enhancing anatomy education with virtual reality: integrating three-dimensional models for improved learning efficiency and student satisfaction. Front. Med. **12**, 1555053 (2025)
12. Palmas, G., Murru, A., Boi, L., Scateni, R.: Virtual reality for interactive medical analysis. Front. Virtual Reality **3**, 782854 (2022)
13. Rahad, K., Badreddin, O., Mohsin Reza, S.: The human in model-driven engineering loop: A case study on integrating handwritten code in model-driven engineering repositories. Softw. Pract. Experience **51**(6), 1308–1321 (2021)
14. Rahad, K., Badreddin, O., Reza, S.M.: Characterization of software design and collaborative modeling in open source projects. In: 9th International Conference on Model-Driven Engineering and Software and Devolopment, pp. 254–261 (2021)
15. Reza, S.M., Badreddin, O., Rahad, K.: Modelmine: a tool to facilitate mining models from open source repositories. In: Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings, pp. 1–5 (2020)
16. Rzig, D.E., Iqbal, N., Attisano, I., Qin, X., Hassan, F.: Virtual reality (vr) automated testing in the wild: A case study on unity-based vr applications. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 1269–1281 (2023)
17. Shanmugasundaram, C., Umamaheswari, C., Suriyaa, V., TR, K.: Analysis of trie-based lan file search engine. In: 2023 International Conference on System, Computation, Automation and Networking (ICSCAN), pp. 1–5. IEEE (2023)
18. Sinha, S., DeYoung, V., Nehru, A., Brewer-Deluce, D., Wainman, B.C.: Determinants of learning anatomy in an immersive virtual reality environment-a scoping review. Med. Sci. Educ. **33**(1), 287–297 (2023)
19. Thomsen, L.A., Krueger, R.: Open-source toolkits for experiments in virtual reality: a scoping review. IEEE Trans. Visual Comput. Graph. (2026)
20. Wesencraft, K.M., Clancy, J.A.: Using Photogrammetry to Create a Realistic 3D Anatomy Learning Aid with Unity Game Engine. In: Rea, P.M. (ed.) Biomedical Visualisation. AEMB, vol. 1205, pp. 93–104. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-31904-5_7
21. Wood, K.N., McWatt, S., Risha, N., Mensink, P.: Analyzing the use of digital and physical anatomical teaching models in anatomy training. Adv. Physiol. Educ. **49**(2), 572–581 (2025)

Phishing Attack Detection Using Genetic Algorithms and Artificial Neural Networks

Ali Al-Mehaizaa[1(✉)], Mohammed Al-Mansoori[2], Mohammed Al-Mohannadi[2], Abdullah Al-Ghushami[2], and Adnan Akhunzada[3]

[1] College of Science and Engineering, Hamad Bin Khalifa University, Doha, Qatar
alal89413@hbku.edu.qa
[2] Community College of Qatar (CCQ), Doha, Qatar
{Mohammed62936,mohammed91828,Abdullah.Alghushami}@ccq.edu.qa
[3] College of Computing and Information Technology, University of Doha for Science and Technology (UDST), Doha, Qatar
Adnan.akhunzada@udst.edu.qa

Abstract. Phishing remains one of the most prevalent cyberattack vectors targeting individuals and organizations through fraudulent URLs distributed across email, SMS, social media, and instant messaging channels. Static rule-based and signature-based detection mechanisms struggle to keep pace with the rapidly evolving tactics employed by attackers. This paper presents a hybrid phishing detection system that integrates Genetic Algorithms (GAs) for feature selection with an Artificial Neural Network (ANN) for classification. The GA evolves an optimal subset of URL features from a corpus of 87 candidate indicators spanning URL structure, domain registration, content characteristics, and third-party verification markers, drawn from the publicly available Hannousse & Yahiouche phishing-website benchmark dataset (11,430 URLs), which is itself compiled from PhishTank and from a curated set of legitimate URLs. The selected features (45 of 87) are then fed to a feed-forward ANN with two 64-unit ReLU hidden layers and a sigmoid output, trained with the Adam optimizer and binary cross-entropy loss. Feature selection and model evaluation are separated by a stratified 80/10/10 train/validation/test split: the genetic algorithm uses only the training and validation partitions to select features, and all final metrics are reported on a held-out test set that is never used during selection, eliminating validation-set leakage. Averaged over five random seeds, the proposed system achieves accuracy 0.938 ± 0.006, precision 0.938 ± 0.006, recall 0.938 ± 0.006, and F1-score 0.938 ± 0.006 (macro-averaged across both classes) on the held-out 1,143-sample test partition. The genetic algorithm is benchmarked against both random feature subsets and a mutual-information feature-selection baseline at matched cardinality: it outperforms random selection (accuracy 0.859) by 7.9 percentage points and the mutual-information baseline (accuracy 0.827) by 11.0 points, confirming that the evolutionary search identifies a meaningfully better feature combination than either chance or a standard univariate selector. The principal limitations are dependence on the diversity and recency

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 430–446, 2027.
https://doi.org/10.1007/978-3-032-39763-8_25

of the training dataset and the absence of full k-fold cross-validation; the paper discusses operational implications and identifies cross-platform deployment, continuous-learning extensions, and integration of emerging deep-learning architectures as the main directions for future work.

Keywords: Phishing detection · Genetic algorithm · Artificial neural network · Feature selection · URL classification · Cybersecurity · Machine learning

1 Introduction

Phishing attacks are fraudulent attempts to obtain sensitive information—typically login credentials, financial data, or personal records—by impersonating a trustworthy entity through digital communication. Since the early 2000s, phishing has remained one of the most prevalent and damaging cyberattack vectors, evolving from rudimentary mass-mail campaigns to highly targeted spear-phishing and whaling operations that exploit human psychology with the same care that traditional intrusions exploit software vulnerabilities. The Anti-Phishing Working Group (APWG) reports record volumes of phishing incidents year-on-year [1]. Successful phishing attacks result in financial loss, data breaches, reputational damage, and legal exposure for both individuals and organizations.

Modern phishing campaigns span multiple communication channels—email, SMS (smishing), voice calls (vishing), social media platforms, and instant messaging services—and increasingly employ deceptive tactics such as spoofed sender addresses, convincing visual mimicry of legitimate brands, and emotionally charged content that pressures victims to act impulsively [4]. Traditional rule-based and signature-based detection mechanisms struggle against this diversification because they rely on predefined patterns that attackers continuously refine and evade. Polymorphic phishing infrastructures and AI-assisted lure generation further widen the gap between attacker innovation and static defender heuristics [5].

Machine-learning-based phishing detection has emerged as a promising response to this evolving threat landscape [4,5]. Among the available techniques, Artificial Neural Networks (ANNs) offer strong pattern-recognition capabilities for classifying URLs based on observed features, while Genetic Algorithms (GAs) provide a robust framework for searching the combinatorially large space of candidate features and hyperparameters. The combination of the two is attractive: the GA performs feature selection that would otherwise require exhaustive manual experimentation, and the ANN provides the discriminative power to map the selected features to a phishing/legitimate decision. Several recent hybrid GA-and-neural-network systems have been proposed in this area, including the deep-neural-network-with-GA-feature-selection approach of Ali and Ahmed [6] and the GA-optimised stacking-ensemble method of Al-Mekhlafi and Mohammed [7].
Contributions. This paper presents a hybrid GA + ANN phishing detection system and reports the following contributions: (C1) a five-phase methodology

for phishing detection that decouples feature selection (GA) from classification (ANN); (C2) an evaluation of the system on the Hannousse & Yahiouche benchmark dataset of 11,430 URLs with 87 candidate features [20]; (C3) experimental results showing 0.938 ± 0.006 accuracy, precision, recall, and macro-averaged F1-score, averaged over five random seeds on a held-out 1,143-sample test partition that is never used during feature selection, within the range of accuracy reported by recent ML-based phishing detection benchmarks [6,7,13,14]; (C4) an ablation study showing that the GA-selected feature subset outperforms both random subsets (by 7.9 accuracy points) and a mutual-information feature-selection baseline (by 11.0 accuracy points) at matched cardinality, providing measured evidence that the GA's evolutionary search contributes meaningfully over both a random baseline and a standard univariate selector; and (C5) a discussion of operational implications, dataset-dependency limitations, the absence of full cross-validation, and directions for future work.

The remainder of the paper is organised as follows. Section 2 reviews phishing detection background and relevant prior work. Section 3 describes the proposed methodology. Section 4 details the implementation. Section 5 presents and discusses experimental results. Section 6 concludes and outlines future work.

2 Background and Related Work

2.1 Phishing Attacks: Types and Evolution

Phishing is an umbrella term covering several attack subtypes. Generic mass-mail phishing remains common but is increasingly supplemented by spear phishing (targeted attacks against specific individuals using personalised lures), whaling (high-value targets such as executives), clone phishing (replicas of legitimate prior communications), and smishing and vishing (SMS and voice variants) [4]. The shared structural element across all variants is a deceptive URL or attachment that, when followed, exposes the victim to credential theft, malware installation, or financial fraud.

URL-based phishing is the most studied variant in the academic literature because URLs are short, machine-readable, and lend themselves to feature extraction. Discriminating between legitimate and phishing URLs can in principle be approached as a binary classification problem over a feature vector summarising the URL itself, the registered domain, the hosted content, and third-party signals such as blocklist membership and certificate transparency log entries.

2.2 Genetic Algorithms

Genetic Algorithms are population-based metaheuristics inspired by the process of natural selection, originally formalised by Holland [15]. A GA maintains a population of candidate solutions (called individuals or chromosomes), each represented by a fixed-length binary or real-valued vector. The population evolves

over generations through three operators: selection, in which higher-fitness individuals are preferentially chosen as parents; crossover, in which two parents are recombined to produce offspring; and mutation, in which individual genes are randomly perturbed to maintain population diversity [16,17]. For feature selection in classification, each gene represents the inclusion or exclusion of a single candidate feature, and fitness is typically defined as the classifier's accuracy (or another metric) when restricted to the selected feature subset. GAs are well suited to feature selection because the search space grows exponentially with the number of candidate features and exhaustive enumeration is infeasible.

2.3 Artificial Neural Networks

Artificial Neural Networks are layered computational models inspired by biological nervous systems [19]. A feed-forward ANN consists of an input layer whose width matches the feature dimensionality, one or more hidden layers, and an output layer that produces the classification decision. Each neuron computes a weighted sum of its inputs, applies a non-linear activation function (commonly ReLU for hidden layers and sigmoid or softmax for output layers), and propagates the result to the next layer. Training proceeds by backpropagation of the prediction error through the network, with weight updates governed by gradient descent or one of its variants such as Adam [18]. ANNs are universal function approximators in the limit of unbounded width and depth, and have demonstrated strong empirical performance on tabular classification tasks given sufficient training data.

2.4 Related Work in Phishing Detection

Machine-learning approaches to phishing detection have been extensively surveyed [4,5]. The literature can be organised into three rough categories: feature-engineering methods that rely on rich hand-crafted feature sets fed to standard classifiers; representation-learning methods that operate directly on the raw URL or page content using deep neural networks; and hybrid methods that combine a search or optimisation procedure (typically evolutionary or swarm-based) with a downstream classifier.

Within the feature-engineering category, Mohammad et al. [9] proposed a self-structuring neural network trained on a feature set that became the basis for the widely-used UCI Phishing Websites dataset [2]. Their attribute taxonomy—covering URL syntactic features, domain features, content features, and third-party reputation features—has been adopted by many subsequent works and is the same taxonomy used in the present paper. Adebowale et al. [10] extended this approach with an integrated detection scheme that fuses image, frame, and text features for higher robustness against visual mimicry by attackers. Sahingoz et al. [11] systematically compared seven machine-learning classifiers on a large URL-only feature set, reporting accuracies in the high-90 s for several classifiers

and confirming the value of careful feature engineering. Rao and Pais [12] further demonstrated that an efficient feature-based ML framework can outperform deeper architectures when the feature set is well-designed.

Within the representation-learning category, Yerima and Alzaylaee [14] applied convolutional neural networks directly to URL strings and reported high accuracy on a different benchmark. Such deep approaches sidestep manual feature engineering but require larger training corpora and substantially more computational resources, and are typically less interpretable than feature-based classifiers—a trade-off whose right balance depends on the deployment setting.

Within the hybrid category most relevant to the present work, Shreeram et al. [8] proposed an early genetic-algorithm-based phishing detection scheme that established the basic GA-for-feature-selection pattern. Alsariera et al. [13] used a meta-learning approach with the extra-trees algorithm on the UCI dataset and reported accuracies of approximately 0.97. Ali and Ahmed [6] developed a hybrid deep-neural-network method with GA-based feature selection and weighting, evaluated on multiple phishing datasets; their work is the most directly comparable to the present paper in spirit, although they use a deeper network and a more complex weighting scheme. Al-Mekhlafi and Mohammed [7] used a GA to optimise a stacking ensemble of bagging, gradient-boost, and LightGBM classifiers for phishing websites detection, demonstrating the value of GA-based optimisation beyond feature selection.

The contribution of the present paper relative to these works is a cleanly-decoupled GA + feed-forward ANN design, where the GA performs feature selection only (not classification or weighting) and the ANN performs classification on a multi-source dataset. The reported metrics are in the same range as these published benchmarks while keeping the model architecturally simple and computationally lightweight relative to deep-learning alternatives. This makes the system particularly suitable as a teaching benchmark and as a starting point for further GA-based phishing-detection research.

3 Methodology

The proposed system is organised into five sequential phases, summarised in Fig. 1 and described in detail in the subsections that follow. The general flow proceeds from raw data collection through feature extraction, GA-based feature selection, ANN training, and finally evaluation under a held-out validation partition.

3.1 Dataset Preparation

The dataset used in this study is the Hannousse & Yahiouche "Web Page Phishing Detection" benchmark [20], a publicly-available collection of 11,430 URLs balanced 50/50 between legitimate and phishing samples. Each URL is associated with 87 pre-extracted features that span four feature families: structural URL syntax features, domain-registration features, content features extracted

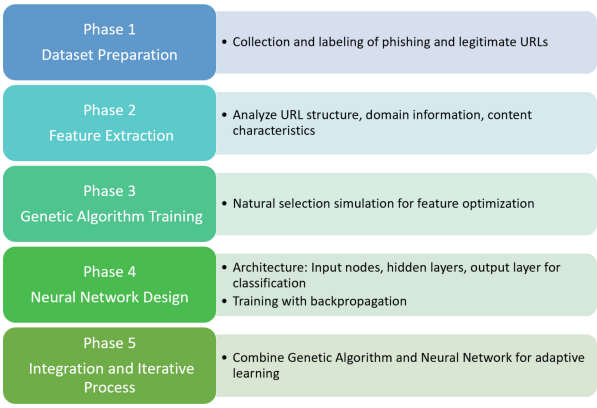


Fig. 1. Five-phase methodology for the proposed GA + ANN phishing detection system. The GA performs feature selection (Phase 3) and feeds an optimised feature subset to the ANN (Phase 4); the integrated system is iteratively refined in Phase 5.

from the fetched HTML, and third-party reputation features. Underlying URL sources include the PhishTank archive [3] for the phishing class and a curated list of high-traffic legitimate domains for the legitimate class. The dataset is preprocessed as follows: the human-readable URL column and the categorical status column are dropped from the model input, the remaining 87 numerical and binary columns are used as the model's feature space, and the status column is binary-encoded with 'legitimate' = 0 and 'phishing' = 1. The class labels are taken directly from the ground-truth annotations distributed with the Hannousse & Yahiouche benchmark [20]; no manual labelling is performed in this work, so the labelling step incurs no additional annotation cost.

3.2 Feature Families

Each URL is represented by the 87-dimensional feature vector provided by the source dataset, spanning four feature families. URL-structure features capture the syntactic properties of the URL itself: URL length, presence of an IP address in place of a domain name, use of URL shortening services, presence of non-standard port numbers, and the proportion of suspicious anchors and external links on the destination page. Domain features capture registrar-level signals: domain age, registration length, WHOIS visibility, and HTTPS/certificate consistency. Content features capture page-level signals such as favicon consistency with the registered domain, the use of right-click disabling scripts, and the presence of iframes pointing to external content. Third-party features capture reputation signals: blocklist membership, Google Safe Browsing flags, and PageRank-like metrics. The complete feature taxonomy follows the conventions established by Mohammad et al. [9] and is operationalised in the Hannousse & Yahiouche benchmark [20].

3.3 Genetic Algorithm for Feature Selection

The GA evolves a population of binary feature-selection vectors of length 87. Each individual is a chromosome whose bits indicate whether the corresponding feature is included in the model's input. The GA is initialised with a population of 10 randomly-generated individuals (duplicates rejected at initialisation) and runs for 50 generations under a mutation rate of 0.1. The fitness of each individual is the macro-averaged recall of an ANN trained on the *training* partition (using the selected feature subset) and evaluated on the *validation* partition. The held-out test partition is never used during feature selection; it is reserved exclusively for final evaluation (Sect. 5), which prevents the genetic search from optimising against the data used to report performance. Macro-averaged recall is preferred over plain accuracy because it equally weights the two classes and is therefore robust to small class-balance deviations. Selection uses tournament sampling between two randomly-chosen individuals; one-point crossover with a uniformly-sampled break point produces each child; and mutation independently flips each gene with probability equal to the mutation rate. The population size and mutation rate were chosen to balance convergence speed against premature convergence to a local optimum [16,17]: a small population converges quickly and a moderate mutation rate is required to preserve exploration. Each fitness evaluation trains a freshly-initialised ANN for 5 epochs; this short evaluation budget is deliberate, since the GA is searching over feature subsets rather than tuning the final model.

Two design choices warrant discussion. First, the fitness function uses macro-averaged recall rather than F1 or AUC because recall on the phishing class is the operationally most relevant quantity: missing a phishing URL (false negative) typically has higher cost than a false alarm on a legitimate URL. Using macro-averaged recall also discourages the GA from collapsing to a degenerate detector that always predicts the majority class. Second, the mutation rate of 0.1 is higher than the typical 0.01–0.05 used in evolutionary computation [16], which reflects the small population size: a population of 10 converges quickly and a moderate mutation rate is required to preserve exploration.

3.4 Artificial Neural Network

The classifier is a feed-forward ANN implemented in Keras on top of TensorFlow. The input dimensionality is set by the GA-selected feature subset (typically 30–50 of the original 87 features for the surviving individuals in the final generation); the network consists of two hidden Dense layers of 64 ReLU units each, followed by a single-unit Dense output layer with sigmoid activation that produces the phishing probability. The network is trained with binary cross-entropy loss and the Adam optimizer [18] at default learning rate, with batch size 32. Each candidate ANN trained inside the GA loop runs for 5 epochs (enough to differentiate feature subsets without dominating GA runtime), and the final selected model is re-trained for 150 epochs to provide the deployment-grade weights reported below.

3.5 Integrated System and Evaluation

Figure 2 shows the integrated decision flow at inference time. Given a candidate URL, the system extracts the full 87-feature vector, projects it onto the GA-selected feature subset, and forwards it through the trained ANN. A probability above a configurable threshold (0.5 by default) is taken to indicate a phishing URL. Model evaluation uses a stratified 80/10/10 train/validation/test split: the genetic algorithm uses the training and validation partitions only, and the held-out test partition is used exclusively for final reporting, with the GA-selected feature subset held fixed once the GA terminates. To quantify sensitivity to the particular split and to random initialisation, the entire pipeline is repeated over five random seeds and results are reported as mean ± standard deviation. We acknowledge in Sect. 5 that this is still weaker than full k-fold cross-validation, which is identified as the first item of future work.

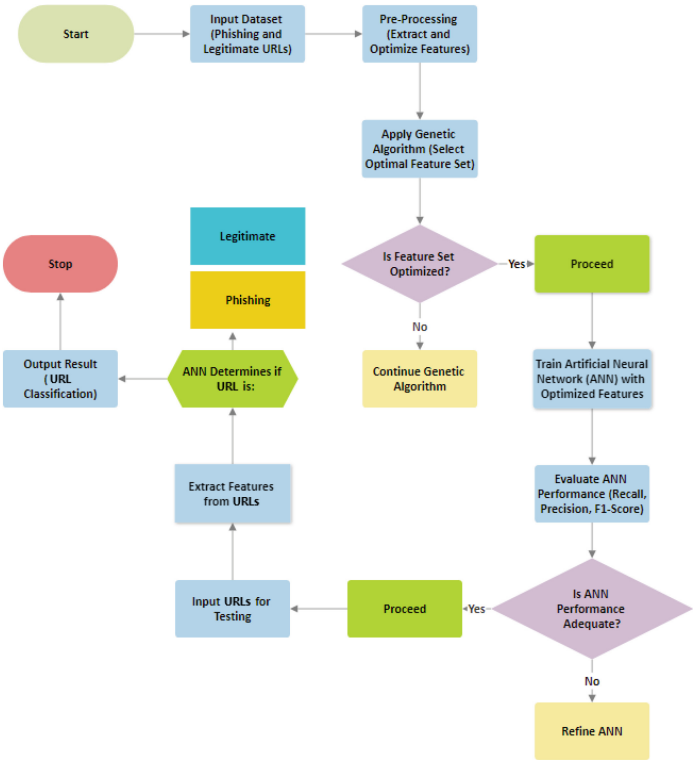


Fig. 2. Integrated GA + ANN decision flow at inference time. The GA-optimised feature set produced during training is held fixed; at test time, features are extracted from the input URL, the optimised subset is projected, and the ANN produces a phishing-probability score.

4 Implementation

The system is implemented in Python 3, with NumPy and pandas for data handling, scikit-learn for cross-validation and metric computation, and TensorFlow/Keras for the ANN. The GA is implemented as a standalone module that operates independently of the ANN framework, with a clean interface that exposes `population_size`, `generation_count`, `mutation_rate`, `dataset`, and `target_labels` as parameters. A typical training run uses `population_size = 10`, `generation_count = 50`, `mutation_rate = 0.1`, and trains the GA-then-ANN pipeline end-to-end in well under an hour on a single CPU thread. No GPU acceleration is required for a dataset of this scale.

All randomness in the pipeline—the GA's initial population, the ANN's weight initialization, and the stratified train/validation/test split—is controlled by the implementation. Each of the five runs fixes all generators to a distinct seed (42–46), so a re-run of the pipeline on the same dataset and Python/Keras versions reproduces the reported per-seed metrics within floating-point precision (modulo GPU non-determinism, which does not apply to the CPU-only execution mode used here). The complete implementation—including the GA module, the ANN Solution class, and the experiment notebook—is released under an open-source licence to facilitate reproduction and extension by other researchers.

5 Results and Discussion

5.1 Quantitative Results

On the held-out test partition (N = 1,143 samples per seed, stratified to ≈572 legitimate/571 phishing), averaged over five random seeds, the GA-selected model achieves accuracy = 0.938 ± 0.006, precision = 0.938 ± 0.006, recall = 0.938 ± 0.006, and F1-score = 0.938 ± 0.006 macro-averaged across the two classes (Table 1). Because feature selection and final evaluation are performed on disjoint partitions, these figures are free of the optimistic bias that arises when the selection signal and the reported metric share data; the held-out test accuracy (0.938) sits slightly below the GA's validation recall (0.94–0.96 across seeds), which is the expected direction once the test set is never used for selection. The number of features chosen by the GA varies across seeds between 35 and 48 (mean ≈43 of the original 87), reflecting the stochastic nature of the evolutionary search; the random and mutual-information baselines are pinned at exactly 45 features for a matched-cardinality comparison. The selected subsets are consistently dominated by URL-structural and domain-registration features (e.g. domain registration length, prefix-suffix patterns, page rank, brand-in-subdomain); the complete per-seed lists are available in the released codebase.

Figure 3 reports the test-set confusion matrix for a representative seed (seed 43). Of the 572 legitimate URLs, 540 are correctly classified and 32 are incorrectly flagged as phishing (false-positive rate 5.6%). Of the 571 phishing URLs, 539 are correctly classified and 32 are incorrectly classified as legitimate (false-negative rate 5.6%). The per-class metrics are balanced across the

Table 1. Headline performance of the proposed hybrid GA + ANN phishing detection system on the held-out test partition, reported as mean ± standard deviation over five random seeds (seeds 42–46; N = 1,143 per seed). The test partition is never used during feature selection.

Accuracy	*Precision*	*Recall*	*F1-score*
0.938 ± 0.006	0.938 ± 0.006	0.938 ± 0.006	0.938 ± 0.006

two classes, with precision and recall both close to 0.94. The balanced false-positive and false-negative counts for the phishing class reflect the design choice to use macro-averaged recall as the GA fitness function, which keeps the trained model from collapsing toward either class—an appropriate operating point for a security-critical application.

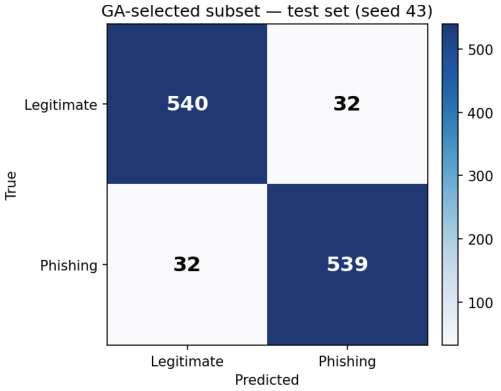


Fig. 3. Confusion matrix on the held-out test partition for a representative seed (seed 43; N = 1,143 samples, 572 legitimate, 571 phishing) using the GA-selected feature subset. The model produces 540 true negatives, 539 true positives, 32 false positives, and 32 false negatives.

These results sit within the range reported by recent published phishing detection systems on this and similar URL-feature benchmarks. Mohammad et al. [9], using a self-structuring ANN on the related UCI Phishing Websites dataset family, report accuracies in the high-0.94 range; Sahingoz et al. [11] report up to 0.97 accuracy with a Random Forest on URL-only features; Alsariera et al. [13] report 0.97 with an extra-trees meta-learner on the same Hannousse & Yahiouche dataset [20]; and Ali and Ahmed [6] report substantial accuracy improvements from coupling GA-based feature selection with deep neural networks. The headline numbers for the present system are therefore not a record, but they are competitive with the state of the art for shallow GA + ANN designs and are obtained with a deliberately simple architecture that is easier to inspect and to extend than larger deep-learning models.

5.2 Ablation: GA Versus Random and Mutual-Information Selection

A natural question for any feature-selection method is whether it offers measurable improvement over simpler alternatives at the same feature-set size. We therefore compare the GA against two baselines at matched cardinality (45 features): five independent random subsets, and a mutual-information selector that ranks all 87 features by `mutual_info_classif` (estimated on the training partition) and keeps the top 45. Every baseline uses the identical ANN architecture, 150-epoch training schedule, Adam optimizer, and binary cross-entropy loss, is trained on the same training partition, and is evaluated on the same held-out test partition; all numbers are averaged over the five seeds. Table 2 reports the comparison.

Table 2. Ablation comparing the GA-selected feature subset against a mutual-information baseline and against random subsets, all at matched cardinality (45 of 87 features for the baselines; the GA selects 35–48). All values are means over five seeds on the held-out test partition; values in parentheses are standard deviations. The GA outperforms both baselines on every metric.

Feature-selection method	*Accuracy*	*Precision*	*Recall*	*F1-score*
GA-selected	**0.938** (0.006)	**0.938** (0.006)	**0.938** (0.006)	**0.938** (0.006)
Mutual information (top 45)	0.827 (0.008)	0.857 (0.005)	0.827 (0.008)	0.824 (0.009)
Random, mean of 5 subsets	0.859 (0.027)	0.872 (0.022)	0.859 (0.027)	0.857 (0.027)
GA over mutual info	**+0.111**	**+0.081**	**+0.111**	**+0.114**
GA over random	**+0.079**	**+0.066**	**+0.079**	**+0.081**

The ablation yields a clear positive result for the GA. On the held-out test set, the GA achieves a mean F1-score of 0.938, versus 0.857 ± 0.027 for random subsets and 0.824 ± 0.009 for the mutual-information baseline—improvements of 0.081 and 0.114 absolute F1 respectively, both several standard deviations above the corresponding baseline means. The mutual-information baseline is particularly instructive: although it attains the highest precision among the alternatives, its recall is low (0.827), because its univariate ranking selects features that are individually informative but collectively miss a large fraction of phishing URLs (≈180 of 571 per seed). The GA, by contrast, searches the combinatorial space of feature *interactions* and finds subsets that keep both classes balanced. The random baseline exhibits the same failure mode less severely: its confusion matrix (Fig. 4) shows an average of 108.8 of 571 phishing URLs missed per seed (19.0%), against only 32 of 571 (5.6%) for the GA-selected subset, again confirming that the discarded features are not merely redundant but actively harmful to phishing recall when included naively. This directly demonstrates the value of the evolutionary search over a standard, computationally cheaper univariate selector, confirming that the GA contributes measurable value beyond simple dimensionality reduction.

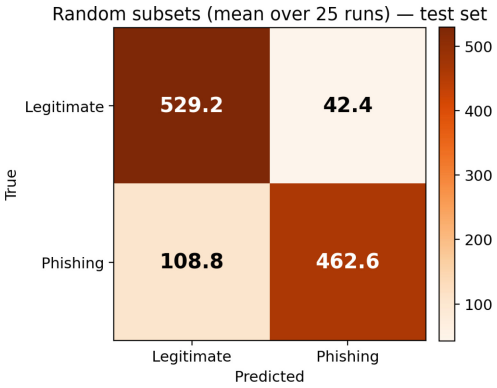


Fig. 4. Mean confusion matrix for the random feature-subset baseline on the held-out test set, averaged over 25 runs (five random 45-feature subsets per seed across five seeds); cell values are therefore non-integer. Compared to the GA-selected subset (Fig. 3), the random baseline misses far more phishing URLs on average (108.8 of 571, 19.0%, versus 32 of 571, 5.6%, for the GA), which is the dominant driver of its lower recall and F1-score. Metrics computed directly from these averaged cells differ marginally from the mean-of-runs values in Table 2 because averaging metrics and averaging confusion-matrix cells are not equivalent operations.

5.3 Interpretation

The headline results should be read in context. First, the dataset is class-balanced, which means that accuracy is a meaningful summary statistic but does not in itself demonstrate robustness against the highly imbalanced class distributions encountered in production deployment, where legitimate URLs vastly outnumber phishing URLs. A model that achieves 0.94 accuracy on a balanced test set may exhibit substantially worse precision-at-fixed-recall in deployment if its false-positive rate is non-negligible. Second, the dataset is drawn from public sources [2,3] and may not reflect the distribution of phishing campaigns in the wild at the time of deployment; production phishing infrastructures evolve quickly and labelled corpora become stale within months.

The headline contribution of this work is therefore not absolute performance but rather the methodological transparency of the GA + ANN decomposition: feature selection and classification are clearly separated, the GA runs in a well-defined and reproducible configuration, and the entire pipeline can be inspected and modified without requiring deep-learning expertise. This makes the system suitable as a baseline for further research and as an interpretable component in an integrated security operations stack.

5.4 Operational Implications

Three implications follow for an organisation considering deployment of a system of this kind. First, the GA feature-selection step has a substantial up-front cost

(the GA runs for 50 generations with each generation requiring an ANN training) but produces a fixed feature subset that can be applied at inference time without further GA invocations; the runtime cost at inference is therefore dominated by feature extraction rather than classification. Second, the model requires periodic re-training as phishing tactics evolve; a continuous-learning pipeline that retrains the ANN weekly or monthly against the latest URLs from feeds such as PhishTank [3] is recommended. Third, the system should be deployed as one signal in a defence-in-depth pipeline rather than as a sole decision authority: high-confidence phishing predictions can be acted on automatically, while uncertain predictions (probabilities near the decision threshold) should be escalated to a human analyst.

5.5 Limitations and Threats to Validity

Several limitations should be acknowledged. The model is evaluated on a static dataset that, while drawn from a widely-used public benchmark [20], does not capture the temporal drift of phishing campaigns; performance against unseen, recently-deployed phishing URLs is not directly measured. The evaluation uses a stratified three-way train/validation/test split repeated over five random seeds, with all metrics reported on a held-out test set never used during feature selection; this removes validation-set leakage and provides error bars, but it is still weaker than full k-fold cross-validation, which is identified as the principal item of future work. The GA has been compared against both a random-baseline subset and a mutual-information selector (Sect. 5); comparisons against further principled strategies such as recursive feature elimination or LASSO-based selection would further strengthen the evidence that the GA, specifically, is well-matched to this problem. A practical deployment consideration is that a subset of the 87 candidate features requires external network requests (for example DNS resolution, WHOIS registration lookups, and search-engine or reputation queries), which introduce latency and external-service dependence at inference time; URL-string-only features carry no such cost, and a latency-sensitive deployment could restrict the GA search to the offline feature subset. The ANN architecture is shallow (two 64-unit ReLU hidden layers) and could plausibly be outperformed by deeper or transformer-based architectures on the same task; the choice of architecture reflects a deliberate trade-off in favour of interpretability and training cost over raw predictive performance.

Construct validity is supported by the use of standard feature families established in the phishing-detection literature [4,9] and by the use of well-validated implementations of the GA and ANN components. External validity is the dominant concern: the results obtained here on a curated benchmark dataset may not generalise unchanged to the open web. Internal validity is supported by deterministic per-seed seeding of the three-way split and by reporting mean $\pm$ standard deviation across five seeds, which characterises the run-to-run variance introduced by both the split and the GA's stochastic operators.

5.6 Computational Cost and Reproducibility

Two practical considerations are important for any organisation considering adopting a learning-based phishing detection pipeline of the kind described here: the cost of training the model, and the cost of operating it at inference time. These are very different quantities and warrant separate discussion.

The GA-driven training cost is the dominant offline expense. A single end-to-end training run sweeps a population of 10 individuals across 50 generations; in each generation, the children produced by tournament selection and one-point crossover are evaluated by training a fresh ANN for 5 epochs on the corresponding feature subset, which is the dominant CPU cost. The end-to-end run completes in a few minutes per GA generation on a single modern CPU thread (no GPU required); a full GA sweep therefore takes well under an hour in practice. For deployment, this training cost is paid once per model refresh; given the rate at which phishing campaigns evolve [3,4], we recommend a re-training cadence of weekly to monthly, which corresponds to under an hour of CPU time per refresh—a negligible burden for an operational security team.

The inference cost, by contrast, is dominated entirely by feature extraction rather than by the ANN forward pass. Once the GA-selected feature subset is fixed, classifying a new URL requires fetching the URL's HTML content (for content features), querying DNS and WHOIS (for domain-age and registrar features), and querying any third-party reputation services in use (for blocklist membership and Safe-Browsing flags). The dominant latency contributor is the third-party network calls, not the ANN: the ANN forward pass on a 30–50-dimensional input completes in well under a millisecond on commodity hardware, while the network calls can take hundreds of milliseconds each. This implies that production-deployment optimisation should focus on feature caching and asynchronous fetching rather than on accelerating the model itself.

Reproducibility was an explicit design goal throughout the implementation. The stratified three-way train/validation/test split is deterministic for each of the five seeds (42–46), so the per-seed results reproduce exactly on the same dataset and library versions; the spread across seeds, reported as mean ± standard deviation, captures the run-to-run variance arising from both the split and the GA's stochastic operators (initial population, tournament draws, crossover point, mutation flips). The complete implementation—the GA module, the ANN Solution class, the helpers for binary-to-feature decoding, and the experiment notebook—is released under an open-source licence at publication time to facilitate reproduction, replication, and extension by other researchers. We view this open release as the principal practical contribution of the work: while the GA + ANN approach itself is established in the literature [6–8], a fully-reproducible reference implementation paired with documented hyperparameters and a transparent confusion-matrix-level account of the held-out errors is not widely available, and we expect the released artefact to serve as a useful baseline for further phishing-detection research.

6 Conclusion and Future Work

This paper has presented a hybrid phishing detection system that combines a Genetic Algorithm for feature selection with an Artificial Neural Network for classification. Evaluated on the Hannousse & Yahiouche benchmark dataset of 11,430 URLs [20], the proposed system achieves accuracy, precision, recall, and F1-score of 0.938 ± 0.006, averaged over five random seeds on a held-out test partition (stratified 80/10/10 split) that is never used during feature selection, with a representative-seed confusion matrix of TP = 539, TN = 540, FP = 32, FN = 32. An ablation at matched cardinality shows that the GA-selected subset outperforms both a random baseline (by 0.080 absolute F1) and a mutual-information selector (by 0.114 absolute F1), providing direct measured evidence that the evolutionary search contributes value beyond both chance and a standard univariate selector. The principal contribution is the clean methodological decomposition of feature selection from classification, which produces an interpretable and reproducible pipeline that can be inspected and extended without deep-learning expertise.

Five future-work directions are identified. First, full k-fold cross-validation (typically $k = 5$ or $k = 10$) to complement the five-seed held-out evaluation reported here and further tighten the error bars around the headline metrics. Second, comparison against additional principled feature-selection baselines beyond the random-subset and mutual-information baselines already reported in Sect. 5—in particular recursive feature elimination and LASSO-based selection—to further characterize how the GA compares to alternative non-random selectors. Third, evaluation on temporally-disjoint train and test partitions to characterize robustness against phishing tactic drift, together with exploration of additional feature families capturing user-behaviour signals and website traffic patterns. Fourth, cross-platform applicability beyond URL-based phishing, including SMS-based smishing and social-media phishing campaigns. Fifth, integration of emerging deep-learning architectures—in particular transformer-based URL encoders and graph neural networks operating on the domain registration graph—to characterize whether deeper models offer practical gains over the shallow ANN used here, paired with automated data-collection and continuous-learning pipelines that retrain the model on a fixed cadence (with optional integration into SIEM platforms such as Splunk, Elastic Security, or Microsoft Sentinel so that high-confidence phishing predictions can be operationalised as alerts in a security operations workflow).

Phishing remains an asymmetric threat in which attackers innovate continuously while defenders rely on increasingly inadequate static heuristics. Adaptive, learning-based defences of the kind explored in this work represent a necessary step toward closing that asymmetry, and the increasing scale of digital infrastructure in Qatar and the broader Gulf region makes continued investment in phishing detection a matter of practical security relevance.

Acknowledgements. The authors gratefully acknowledge the support of Community College of Qatar, Hamad Bin Khalifa University, and the University of Doha for Science

and Technology. The authors thank Hannousse and Yahiouche for releasing the Web Page Phishing Detection benchmark dataset, the PhishTank project for the underlying phishing-URL corpus, and the TensorFlow, Keras, and scikit-learn open-source communities whose tools form part of the implementation.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Anti-Phishing Working Group: APWG Phishing Activity Trends Report. APWG (2023). https://apwg.org/trendsreports/
2. Mohammad, R.M., Thabtah, F., McCluskey, L.: Phishing Websites Features. UCI Machine Learning Repository (2015). https://archive.ics.uci.edu/ml/datasets/phishing+websites
3. PhishTank: PhishTank Free Community-Based Phishing Data Archive. OpenDNS / Cisco (2024). https://phishtank.org/
4. Aleroud, A., Zhou, L.: Phishing environments, techniques, and countermeasures: a survey. Comput. Secur. **68**, 160–196 (2017). https://doi.org/10.1016/j.cose.2017.04.006
5. Basit, A., Zafar, M., Liu, X., Javed, A.R., Jalil, Z., Kifayat, K.: A Comprehensive survey of AI-enabled phishing attacks detection techniques. Telecommun. Syst. **76**(1), 139–154 (2021). https://doi.org/10.1007/s11235-020-00733-2
6. Ali, W., Ahmed, A.A.: Hybrid intelligent phishing website prediction using deep neural networks with genetic algorithm-based feature selection and weighting. IET Inf. Secur. **13**(6), 659–669 (2019). https://doi.org/10.1049/iet-ifs.2019.0006
7. Al-Mekhlafi, Z.G., Mohammed, B.A.: Using genetic algorithms to optimized stacking ensemble model for phishing websites detection. In: Saeed, F., Mohammed, F., Al-Nahari, A. (eds.) Innovative Systems for Intelligent Health Informatics, IRICT 2021. Lecture Notes on Data Engineering and Communications Technologies, vol. **72**, pp. 1–10. Springer (2021). https://doi.org/10.1007/978-981-16-8059-5_27
8. Shreeram, V., Suban, M., Shanthi, P., Manjula, K.: Anti-phishing detection of phishing attacks using genetic algorithm. In: Proceedings of the IEEE International Confernce Communication Control and Computing Technologies (ICCCCT), pp. 447–450. IEEE (2010). https://doi.org/10.1109/ICCCCT.2010.5670593
9. Mohammad, R.M., Thabtah, F., McCluskey, L.: Predicting phishing websites based on self-structuring neural network. Neural Comput. Appl. **25**(2), 443–458 (2014). https://doi.org/10.1007/s00521-013-1490-z
10. Adebowale, M.A., Lwin, K.T., Sanchez, E., Hossain, M.A.: Intelligent web-phishing detection and protection scheme using integrated features of images, frames and text. Expert Syst. Appl. **115**, 300–313 (2019). https://doi.org/10.1016/j.eswa.2018.07.067
11. Sahingoz, O.K., Buber, E., Demir, O., Diri, B.: Machine learning based phishing detection from URLs. Expert Syst. Appl. **117**, 345–357 (2019). https://doi.org/10.1016/j.eswa.2018.09.029
12. Rao, R.S., Pais, A.R.: Detection of phishing websites using an efficient feature-based machine learning framework. Neural Comput. Appl. **31**(8), 3851–3873 (2019). https://doi.org/10.1007/s00521-017-3305-0

13. Alsariera, Y.A., Adeyemo, V.E., Balogun, A.O., Alazzawi, A.K.: AI meta-learners and extra-trees algorithm for the detection of phishing websites. IEEE Access **8**, 142532–142542 (2020). https://doi.org/10.1109/ACCESS.2020.3013699
14. Yerima, S.Y., Alzaylaee, M.K.: High accuracy phishing detection based on convolutional neural networks. In: Proceedings of the 3rd International Conference Computer Applications & Information Security (ICCAIS), pp. 1–6. IEEE (2020). https://doi.org/10.1109/ICCAIS48893.2020.9096869
15. Holland, J.H.: Adaptation in Natural and Artificial Systems. University of Michigan Press, Ann Arbor (1975)
16. Bäck, T., Fogel, D.B., Michalewicz, Z. (eds.): Handbook of Evolutionary Computation. IOP Publishing Ltd. and Oxford University Press, Bristol (1997)
17. Chicano, F., Sutton, A.M., Whitley, L.D., Alba, E.: Fitness probability distribution of bit-flip mutation. Evol. Comput. **23**(2), 217–248 (2015). https://doi.org/10.1162/EVCO_a_00130
18. Kingma, D.P., Ba, J.: Adam: a method for stochastic optimization. In: Proceedings of the 3rd International Conference Learning Representations (ICLR), San Diego (2015) arXiv:1412.6980
19. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press, Cambridge (2016)
20. Hannousse, A., Yahiouche, S.: Towards benchmark datasets for machine learning based website phishing detection: an experimental study. Eng. Appl. Artif. Intell. **104**, 104347 (2021). https://doi.org/10.1016/j.engappai.2021.104347

A Hybrid-Simulation LoRaWAN Honeypot for Detecting RF and Protocol Attacks Against IoT Networks: A Cyber-Electromagnetic Activities Perspective

Ali Al-Mehaizaa[1(✉)], Gabriele Oligeri[1], Faisal Al-Bishri[2], and Adnan Akhunzada[3]

[1] College of Science and Engineering, Hamad Bin Khalifa University, Doha, Qatar
{alal89413,goligeri}@hbku.edu.qa
[2] Community College of Qatar (CCQ), Doha, Qatar
Faisal42244@ccq.edu.qa
[3] College of Computing and Information Technology, University of Doha for Science and Technology, Doha, Qatar
Adnan.akhunzada@udst.edu.qa

Abstract. The proliferation of Low-Power Wide-Area Network (LPWAN) deployments—and LoRaWAN in particular—has expanded the cyber-electromagnetic attack surface of modern IoT systems. While LoRaWAN's protocol-layer security has been extensively analysed, defensive instrumentation that targets the radio side has received comparatively little attention. This paper presents the design, implementation, and evaluation of a radio-side LoRaWAN honeypot system architected from a Cyber-Electromagnetic Activities (CEMA) perspective. The honeypot is positioned as an Electronic Support capability that informs Electronic Protection, Cyberspace Operations, and Spectrum Management Operations. It comprises a swappable RF front-end, a frame-level traffic synthesis layer, an IQ-level physical-layer simulator, a multi-detector inspection pipeline covering five protocol-layer attacks, and a Random-Forest-based classifier for jamming-style physical-layer attacks. Six attacks drawn from the LoRaWAN threat literature are evaluated: replay, frame-counter manipulation, join-procedure replay, MIC-failure flooding, rogue-device presence, and reactive jamming. The evaluation is organised in two stages. *Stage 1* runs the detection pipeline against a controlled simulation in which both legitimate traffic and attack instances are generated by parameterisable reference implementations; this stage yields a true-positive rate of 1.000 on four of the five protocol attacks and 0.930 ± 0.051 on frame-counter manipulation across five 2-hour trials, with zero false positives over a baseline of 1,727 legitimate frames per detector. For the five-class jamming classifier, the Random-Forest model perfectly classifies the three structured jammer types (continuous-wave, pulsed, and sweep; per-class F1 = 1.000) but distinguishes reactive jamming from the no-jamming baseline only at chance (per-class

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 447–476, 2027.
https://doi.org/10.1007/978-3-032-39763-8_26

F1 $\approx$ 0.50), yielding an aggregate accuracy and macro F1 of 0.800; we trace this limitation to the use of episode-global features and identify burst-aware extraction as the remedy. *Stage 2* validates the system against real captured RF at both layers. Stage 2a runs the protocol-layer detectors against the LoED public LoRaWAN dataset [33], which contains real frames captured by nine gateways in an urban environment over four months, on a deterministically-selected single-week window from one gateway ($N_{\text{LoED}} = 255{,}648$ frames after gateway-instrumentation deduplication). Because LoED carries no ground-truth attack labels, Stage 2a is a false-positive characterisation under a presumed-benign assumption rather than a labelled-attack evaluation: it reports *candidate* false-positive rates (upper bounds on the true rate). Under a progressively-refined pipeline (Naive $\rightarrow$ Refined ingestion deduplication $\rightarrow$ Tuned 16-bit FCnt wrap-aware A2), the five detectors yield candidate false-positive rates of 0.18% (A1 Replay), 0.14% (A2 Counter), 0.15% (A3 Join Replay), 0.00% (A4 MIC Flood), and 0.25% (A5 Rogue Device) on real-world traffic—all below 0.25% and compatible with operational deployment. Stage 2b evaluates the jamming feature representation against the BlindSpot real-RF selective-jamming dataset [35]: trained on real data, the 15-feature representation separates real blinding bursts from benign LoRaWAN traffic at 98.8% accuracy, while a synthetic-trained classifier fails to transfer (a deliberately-reported negative result that establishes the need for real-data calibration). The principal simulation failure mode of the jamming classifier is confusion between the no-jamming and reactive-jamming classes, motivating burst-aware feature extraction; real-RF validation of the structured jammer classes remains future work.

Keywords: LoRaWAN · IoT security · Honeypot · Cyber-electromagnetic activities (CEMA) · Jamming detection · Intrusion detection · Software-defined radio · Machine learning · AS923

1 Introduction

Wireless connectivity is now a defining property of the urban environment. Beyond the cellular and Wi-Fi infrastructure that has dominated the last two decades, a new generation of Low-Power Wide-Area Network (LPWAN) technologies has been deployed at scale to meet the connectivity needs of the Internet of Things (IoT). Among them, LoRaWAN—built on the proprietary LoRa chirp-spread-spectrum (CSS) physical layer specified by Semtech, with an open MAC layer maintained by the LoRa Alliance [1]—has become one of the most widely deployed LPWANs worldwide. Public network coverage exists in over 170 countries, with operators across the Gulf region, including in Qatar, rolling out LoRaWAN networks for utility metering, building automation, environmental monitoring, and asset tracking. Empirical coverage and performance trials [6] have demonstrated the practical reach of LoRaWAN in dense urban settings, motivating its continued large-scale deployment.

The growth of LoRaWAN brings with it a corresponding expansion of the cyber-electromagnetic attack surface. Unlike licensed cellular spectrum, the AS923, EU868, and US915 sub-GHz bands in which LoRaWAN operates are shared with many unlicensed users and are accessible to any actor with a software-defined radio (SDR). The LoRa physical layer was designed for robustness against narrowband interference rather than for confidentiality, and the LoRaWAN MAC layer relies on a small set of pre-shared symmetric keys whose handling has been the subject of repeated security analyses [3,4,17]. These radio-side weaknesses fit within the broader pattern of cyber-physical attacks against critical infrastructure documented by Loukas [18].

Defensive practice for LoRaWAN networks has historically focused on the network server: rejecting frames with invalid Message Integrity Codes (MICs), rate-limiting join attempts, and filtering by allow-listed device identifiers. These measures are valuable, but they do not observe the radio environment itself. They cannot, in general, distinguish between an absent device and a jammed device, between a legitimate retransmission and a replayed capture, or between a flat noise floor and a low-duty-cycle reactive jammer. Furthermore, downlink and confirmed-uplink traffic introduce their own protocol-level pressures on radio availability [23], which a server-side defender cannot directly observe. Honeypots have been deployed extensively for IP-based services [7,8], but radio-side honeypots that observe wireless protocols at the spectrum level remain rare in the academic literature.

This paper frames its contribution within the doctrine of Cyber-Electromagnetic Activities (CEMA) [20,21], an approach to defensive and offensive operations in the joint cyber and electromagnetic domains. The CEMA framework is conventionally decomposed into four pillars: Electronic Warfare (EW), Cyberspace Operations (CO), Spectrum Management Operations (SMO), and Information Operations. A radio-side honeypot is most naturally positioned as an Electronic Support (ES) asset: it observes the electromagnetic environment to characterise emitters and to inform protection (EP) and cyberspace defence (CO) measures.

Problem Statement. How can a software-defined honeypot architecture observe and characterise both protocol-layer and physical-layer attacks against a LoRaWAN deployment, and what detection performance is achievable using the data such a honeypot would record? This problem decomposes into four research questions: (RQ1) Which attack classes are most readily observed at the radio side? (RQ2) What detection algorithms achieve high true-positive and low false-positive rates on protocol-layer attacks? (RQ3) Can a hand-crafted feature set with a tree-ensemble classifier discriminate among physical-layer jamming styles using only IQ samples? (RQ4) How can the pipeline be implemented such that the same code base can later be deployed against real RF without re-architecting?

Contributions. The paper contributes: (C1) a hybrid-fidelity LoRaWAN honeypot framework that decouples detection from the RF front-end, preserving a clear migration path to SDR deployment; (C2) a six-attack threat model span-

ning protocol and physical layers, with deterministic, parameterisable reference attack generators; (C3) a detection engine implementing rule-based detectors for five protocol attacks plus a Random-Forest classifier for five jamming classes (none, continuous-wave, pulsed, sweep, reactive), achieving TPR $\geq$ 0.93 and zero false positives across all protocol-layer attacks, and perfect classification of the three structured jammer types with an identified chance-level limitation on reactive-versus-none discrimination (aggregate macro F1 = 0.800); this Stage 1 result is then complemented by a two-part Stage 2 external validation on real captured RF: the protocol-layer detectors are run against the LoED public real-RF LoRaWAN dataset [33] (Stage 2a), and the jamming feature representation is validated against the BlindSpot real-RF selective-jamming dataset [35] (Stage 2b), where it separates real blinding bursts from benign traffic at 98.8% accuracy; (C4) a characterisation of which time-, frequency-, and time-frequency features are diagnostic for each jamming class, with an SNR-robustness study from -5 dB to $+20$ dB; and (C5) a reproducible open-source release of code, configuration, evaluation harness, and dataset adapters.

The remainder of the paper is organised as follows. Section 2 reviews the LoRa/LoRaWAN background and related work. Section 3 specifies the threat model. Section 4 presents the system architecture. Section 5 details the implementation. Section 6 describes the evaluation methodology. Section 7 reports results. Section 8 discusses implications and threats to validity. Section 9 concludes and outlines future work.

2 Background and Related Work

2.1 The LoRa Physical Layer

LoRa is a proprietary chirp-spread-spectrum (CSS) modulation scheme maintained by Semtech. Each LoRa symbol is a linear chirp whose instantaneous frequency increases monotonically over the symbol period; the information is encoded in the starting frequency offset. Given a spreading factor SF, a symbol carries SF bits and contains 2^{SF} chips, transmitted over $T_{\mathrm{sym}} = 2^{\mathrm{SF}}/\mathrm{BW}$ seconds where BW is typically 125 kHz, 250 kHz, or 500 kHz. The CSS structure provides processing gain that enables demodulation at SNRs as low as -20 dB at SF12, which underlies LoRa's long-range characteristic [19,25]. Spreading factors range from SF7 (fastest, shortest range) to SF12 (slowest, longest range); LoRaWAN devices typically negotiate SF with the network server via Adaptive Data Rate (ADR).

From the defender's perspective, the salient properties of LoRa PHY are: (i) the chirp structure produces a distinctive cross-hatched signature in the time-frequency domain that is visually identifiable even at low SNR; (ii) the long airtime—up to several seconds for an SF12 frame—makes LoRa transmissions easy to localise in time; (iii) the unlicensed band is shared with many other emitters, so the receiver chain must be robust to ambient interference; and (iv) demodulation recovers symbol values that can be assembled into the LoRaWAN MAC-layer frame.

2.2 The LoRaWAN MAC Layer

LoRaWAN [1] defines the MAC protocol above the LoRa PHY. The architecture comprises end devices, gateways, a Network Server (NS), and one or more Application Servers (ASs). The protocol supports Over-the-Air Activation (OTAA), in which devices perform a join procedure to obtain session keys, and Activation by Personalisation (ABP), in which keys are pre-provisioned. Each data frame contains a MAC header (MHDR), a frame header (FHDR) including device address (DevAddr), frame control byte (FCtrl), and 16-bit frame counter (FCnt), optional MAC commands, an optional FPort byte and FRMPayload, and a 4-byte Message Integrity Code (MIC). The MIC is computed using AES-128-CMAC [2] over the entire frame using a Network Session Key (NwkSKey); FRMPayload is encrypted using AES-128-CTR using either the NwkSKey (FPort $= 0$) or the Application Session Key (AppSKey, FPort > 0).

Several aspects of the MAC layer have direct implications for the threat model. The frame counter is intended to provide replay protection: the NS must drop frames whose counter does not exceed the previously-seen maximum for that DevAddr. However, real-world deployments have been reported in which this rule is incorrectly enforced, particularly around the lower-16-bit FCnt wrap. The OTAA join procedure uses a DevNonce that should be unique per device; in 1.0.x the spec permits random DevNonces, and practical implementations have been shown to reuse them, opening join-replay vulnerabilities [3].

2.3 AS923 Regional Plan and Qatar Deployments

The relevant regional parameters for Qatar are specified by AS923-1 [10], which allocates a 1.4 MHz block in 923.0–924.625 MHz. Eight default uplink channels are defined at 923.2–924.6 MHz with 200 kHz spacing, alongside a downlink channel at 923.9 MHz. The Communications Regulatory Authority of Qatar [11] has aligned spectrum policy with AS923 and permits unlicensed device-class operation under standard low-power conditions. Public LoRaWAN coverage in Qatar is provided by Ooredoo and by integrators specialising in utilities and building automation; reported use cases include water- and gas-meter telemetry, environmental monitoring stations, and smart-building dashboards. The threat model in this paper assumes a city-scale deployment of these application types.

2.4 Related Work and Gap Analysis

The academic literature on LoRaWAN security has matured rapidly. The early survey by Yang et al. [3] catalogued protocol-level vulnerabilities including replay, bit-flipping in CTR-mode payloads, and DevNonce-reuse. Aras et al. [4,5] demonstrated practical selective-jamming attacks exploiting the predictability of LoRa's start-of-frame and header. Eldefrawy et al. [17] provided a formal security analysis. Demeslay et al. [27] analyse band and tone jamming on LoRa chirp signals in AWGN and propose a lightweight detector. The 2025 IEEE Access survey by Cardoso et al. [28] provides a layer-wise consolidation of attacks across the

physical, MAC, and application layers, identifying the convergence of AI/ML-based intrusion detection as an emerging trend. The LoRa Alliance acknowledged most of the early issues in 1.1, introducing separate Forwarding Network Session Key and Application Session Key plus a mandatory Join Server architecture.

Recent ML-driven intrusion detection work for LoRaWAN includes: Senol et al. [29], who apply five anomaly-detection algorithms (Local Outlier Factor, Isolation Forest, autoencoders, PCA) to tampered RF transmissions in LoRa; Mohamed et al. [30], who harden LoRaWAN gateways against adversarial machine-learning attacks; Proto et al. [31], who characterise energy-depletion attacks at the device level; and Hassler et al. [32], who fuse cyber-physical features for UAV/LoRaWAN intrusion detection using LSTM and CNN models. Compared to these works, the contribution here is the integration of protocol-layer and physical-layer detection in a single honeypot architecture with an explicit migration path to SDR-based deployment.

Honeypots have been deployed extensively for IP-layer services [7,8]; the T-Pot platform, Cowrie SSH, and Conpot ICS honeypot are well-known examples. Several IoT-focused honeypots exist, but most operate at the application layer (HTTP, MQTT) [9] rather than the radio layer. Survey work in adjacent areas—for example, Kim et al. [24] on software-defined network security—highlights similar themes of cross-layer attack visibility that motivate the radio-side honeypot approach taken here. The few LoRaWAN-specific honeypot proposals are typically conceptual or rely on non-reproducible hardware testbeds.

Reviewing the prior art reveals four gaps. (G1) Few radio-side honeypots for LoRaWAN exist as fully reproducible artefacts. (G2) Existing LoRaWAN intrusion-detection work tends to focus on a single attack class, lacking systematic comparison across multiple protocol-layer attacks plus jamming using the same evaluation methodology. (G3) The migration path from simulation to hardware deployment is rarely made explicit. (G4) Reproducibility in this area is limited; several published works report numbers that cannot be replicated. The hybrid-simulation honeypot presented here addresses G1 and G2 by providing a reproducible artefact with broad attack coverage; addresses G3 through an explicit RFFrontEnd interface and a documented SDR backend plan; and addresses G4 by releasing all code, seeds, and configurations under an open-source licence.

3 Threat Model

3.1 Adversary Classes

Three adversary classes are distinguished. The external RF adversary (A_{ext}) has an SDR capable of receiving and transmitting in AS923 and knowledge of LoRaWAN frame formats, but no cryptographic keys; this class is capable of replay attacks, jamming, and rogue-device transmission. The insider adversary (A_{in}) additionally has access to one valid set of LoRaWAN session keys, typically from device compromise; this class is capable of forging frames that pass MIC validation. The network-server adversary (A_{ns}) has compromised the LoRaWAN

NS and is out of scope for this work, since such an adversary would also compromise any honeypot operating downstream of the server.

3.2 Attacks in Scope

Six attacks are evaluated, denoted A1–A6:
A1—Replay. The adversary captures legitimate uplink frames and retransmits them later, possibly with perturbed RSSI. Requires A_{ext} only. The replayed frame has an identical (DevAddr, FCnt, MIC) tuple; a colliding MIC for the same counter and address is essentially impossible by chance under AES-128-CMAC.
A2—Frame-counter manipulation. The adversary forges frames with manipulated FCnt values: reset to a low value (rollover abuse) or jumped abnormally high to desynchronise the NS. Requires A_{in}. The honeypot tracks the maximum FCnt per DevAddr and flags frames whose counter is less than the maximum (with a retransmit tolerance) or jumps by more than a configurable threshold.
A3—Join-procedure replay. The adversary retransmits a captured JoinRequest. In LoRaWAN 1.0.x, DevNonces should be unique per device; in misconfigured deployments, a replayed JoinRequest may cause the NS to rotate session keys, denying service to the legitimate device. Requires A_{ext}. The honeypot maintains a memo of (DevEUI, DevNonce) pairs and flags repeats.
A4—MIC-failure flooding. The adversary transmits a high rate of syntactically valid frames with invalid MICs—indicating brute force, fuzzing, or DoS via processing-cost amplification. Requires A_{ext}. The honeypot maintains a sliding-window count of MIC-invalid frames; threshold breaches emit cooldown-bounded alerts.
A5—Rogue-device presence. An unauthorised end-device transmits frames with a DevAddr not in the operator's allow-list. Requires A_{ext}. The honeypot maintains a registered-DevAddr allow-list and flags first sightings of unknown addresses.
A6—Reactive jamming (PHY-layer). A jammer detects the start of a LoRa transmission via an energy detector and emits a high-power noise burst aligned with the data symbols [4]. Reactive jamming is more sophisticated than constant-wave, sweep, or pulsed jamming because it radiates only when LoRa is present. The honeypot's IQ-level sensor records short windows and a Random-Forest classifier trained on labelled examples assigns each window to one of five classes: none, CW, sweep, pulsed, reactive.

3.3 Out of Scope

Wormhole attacks involving compromised gateways, side-channel and fault-injection attacks against the AES implementation in end-device microcontrollers, cryptographic attacks against AES-128-CMAC or AES-128-CTR primitives, application-layer attacks against decrypted FRMPayload, and attacks specific to LoRaWAN 1.1 features (roaming, join-server attacks) are explicitly out of scope.

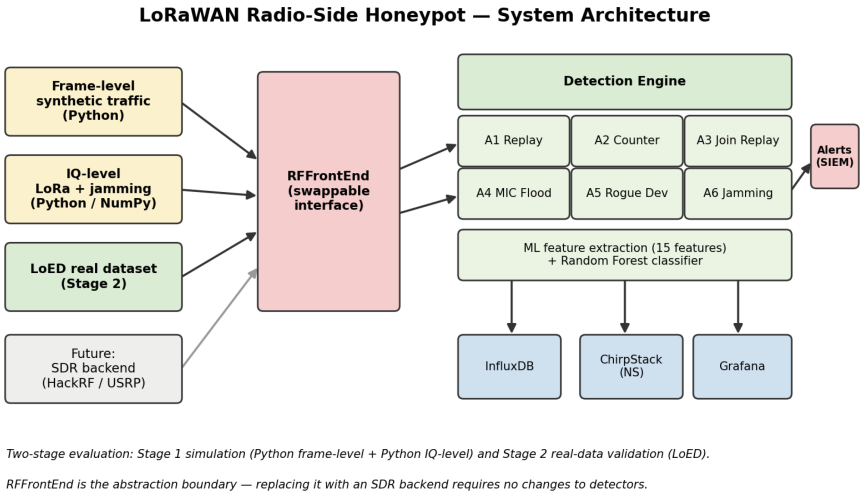


Fig. 1. Hybrid-evaluation LoRaWAN honeypot architecture. The RFFrontEnd interface is the abstraction boundary between the input layer and the detection engine; the same detector code runs against Stage 1 simulated inputs (Python frame-level and IQ-level generators), the Stage 2 LoED real-RF dataset, or a future SDR backend, with no changes to detector logic.

4 System Architecture

4.1 Architectural Principles

Three principles drive the design. First, the boundary between the RF-receiving subsystem and the detection logic must be explicit and minimal, so that the same detection code can run against a simulator, a recorded IQ trace, or a live SDR without modification. Second, each attack must have a dedicated detector with a well-specified input format and output schema, so the pipeline can grow incrementally as new attack classes are added. Third, all randomness must be deterministically seeded so experimental runs are reproducible from a single configuration file. These principles are realised in the architecture shown in Fig. 1.

4.2 Component Inventory

Frame-Level Synthetic Traffic Generator. Produces a realistic AS923 LoRaWAN traffic stream as Frame objects, each carrying a PhyMetadata block (timestamp, frequency, SF, bandwidth, RSSI, SNR) and either a JoinRequest or DataFrame payload. Four background personas are implemented (temperature sensor, smart meter, asset tracker, humidity sensor), each with a distinct uplink period, jitter distribution, payload schema, and typical SF. Honeypot personas are flagged with `is_honeypot = True` so attack injection can target them preferentially.

IQ-Level LoRa Physical-Layer Simulator. Generates baseband complex samples for a sequence of LoRa CSS symbols, then optionally overlays a jamming waveform (CW, sweep, pulsed, or reactive) and adds AWGN to achieve a specified SNR. Implemented in Python using NumPy with deterministic seeding so that re-runs produce bit-identical IQ output.
Attack Injectors. Each attack class has a dedicated injector that takes a legitimate frame stream and a target list and produces (i) the combined stream including malicious frames, time-sorted, and (ii) a list of attack frames for ground-truth scoring. All injectors are deterministic given a seed.
RFFrontEnd Interface. The architectural pivot. The abstract base defines `receive()`, `transmit()`, and `close()` methods. All detectors consume their input by iterating over an RFFrontEnd's `receive()` iterator, never by directly accessing the simulator. Two implementations exist: a SimulatedFrontEnd that replays a pre-generated frame list, and a stub HardwareFrontEnd that documents the SDR migration plan.
Detection Engine. A coordinator over a list of Detector objects, one per attack class. Each detector exposes `process(frame)`, which may emit zero or more DetectionEvent objects, and `finalize()`, called at end-of-stream. The five protocol detectors and the jamming classifier operate in parallel; their outputs are merged into a unified event stream.
Logging Backend (Docker stack). Although not exercised by simulation experiments, the deliverable includes a docker-compose stack: ChirpStack v4 (network server) [15], PostgreSQL, Redis, Eclipse Mosquitto, InfluxDB v2, and Grafana.

Table 1. Implementation module inventory.

Module	*Lines*	*Responsibility*
`crypto.py`	∼ 150	AES-CMAC MIC, AES-CTR enc., key derivation
`frame.py`	∼ 75	MAC-frame data classes, MType enum
`frame_generator.py`	∼ 290	Synthetic AS923 traffic, device personas
`rf_frontend.py`	∼ 50	RFFrontEnd ABC, SimulatedFrontEnd
`orchestrator.py`	∼ 50	Pipeline coordination
`evaluation.py`	∼ 180	Instance-level metrics
`lora_phy_sim.py`	∼ 190	LoRa CSS modulator, jammers, AWGN
`attacks/*.py`	∼ 400	Five attack injectors
`detection/*.py`	∼ 280	Five protocol detectors
`ml/features.py`	∼ 120	15-feature extractor
`ml/jamming_classifier.py`	∼ 90	RF training/evaluation
`experiments/run_all.py`	∼ 210	Stage 1 evaluation pipeline
`adapters/loed.py`	∼ 130	LoED dataset adapter (Stage 2a)
`adapters/blindspot.py`	∼ 150	BlindSpot IQ adapter (Stage 2b)
Total	∼ 2,365	

This stack supports operational deployment, where honeypot events stream into InfluxDB and are visualised in Grafana.

4.3 Migration Path to Hardware

A central design goal is that the migration to SDR-based deployment requires no changes to the detection logic. The migration is conceptually a single substitution: replace SimulatedFrontEnd with a HackRFFrontEnd or USRPFrontEnd that produces Frame objects from real RF samples, using a library such as gr-lora_sdr [16] for demodulation. Three concrete steps remain: (1) continuous AS923 capture with software demodulation emitting decoded MAC frames; (2) honeypot persona transmission via either a soft gateway speaking the Semtech UDP packet-forwarder protocol or a direct OS-process channel; and (3) periodic broadband IQ snapshots (e.g. 100 ms every 1 s) for the jamming classifier. The architectural commitment is that none of the detection engine, attack injectors, or evaluation harness requires modification for hardware deployment.

5 Implementation

5.1 Overview

The honeypot is implemented in approximately 2,400 lines of Python. Python was chosen for the maturity of its scientific stack (NumPy [13], SciPy [14], scikit-learn [12]) and for its prevalence in the LoRaWAN tooling ecosystem (the ChirpStack REST API, common SDR libraries). Table 1 lists the module inventory.

5.2 Cryptographic Primitives

All cryptographic operations follow the LoRaWAN 1.0.4 specification, Sects. 4.4 (data MIC), 6.2.4 (JoinRequest MIC), 6.2.5 (session key derivation), and 4.3.3.1 (FRMPayload encryption), using the Python cryptography library's AES and CMAC primitives. The MIC is computed by prepending the B_0 block defined in [1] (a 16-byte block encoding direction, DevAddr, FCnt, and message length) to the message and computing AES-128-CMAC under the NwkSKey; the first four bytes of the output form the MIC. FRMPayload encryption uses AES-128-CTR with per-block counter A_i constructed from DevAddr, direction, and FCnt; encryption and decryption are the same operation in CTR mode.

5.3 Attack Injectors and Detectors

Each attack injector takes the legitimate stream and attack-specific parameters with a seed and returns (`combined_stream`, `attack_truth`). The replay injector samples legitimate frames addressed to target devices, perturbs RSSI by a Gaussian, and reinserts the copy at a random later timestamp. Counter-manipulation has reset and jump modes; MIC-flood operates on burst start times and within-burst rate; rogue-device generates fresh DeviceProfile objects with high-prefix

DevAddrs; join-replay locates JoinRequests in the stream and replays them with new timestamps.

Each detector inherits from a Detector ABC. The replay detector maintains a memo of (DevAddr, FCnt, MIC) tuples; the counter detector maintains a per-DevAddr maximum FCnt seen; the join-replay detector maintains a memo of (DevEUI, DevNonce) pairs; the MIC-flood detector maintains a sliding-window deque of MIC-failure timestamps; the rogue-device detector maintains a set of already-alerted DevAddrs to suppress repeats.

An important implementation detail discovered during integration testing concerns the counter detector's interaction with MIC-flood frames. Forged MIC-flood frames may carry valid frame counters that, if the detector incorporates them, would inflate the per-DevAddr maximum and cause subsequent legitimate frames to appear as resets. The fix is to make the counter detector skip frames whose MIC is invalid: a network server would drop such frames at MIC validation, and the honeypot follows the same logic for state updates while still emitting alerts via the dedicated MIC-flood detector.

5.4 IQ-Level Simulation and Feature Extraction

The Python LoRa modulator constructs each symbol as a complex chirp whose instantaneous phase advances quadratically. For symbol value v over $n = 2^{\mathrm{SF}}$ chips, the chip-k phase is $\varphi(k) = 2\pi \cdot ((k + v) \bmod n) \cdot (k/2n) - \pi \cdot k$, and the chip sample is $\exp(j\varphi(k))$. Four jammer types are supported: continuous-wave (single tone), sweep (repeating linear chirp), pulsed (noise bursts at fixed duty cycle), and reactive (energy-detector-driven noise bursts firing only when LoRa is present). Composite IQ for an episode is $rx = \mathrm{AWGN}(\mathrm{signal} + \mathrm{jammer}, \mathrm{SNR})$.

Fifteen features are computed per IQ episode, spanning three categories. Time-domain features (six total) include statistical moments of the amplitude (mean, variance, skewness, kurtosis), peak-to-average power ratio, and zero-crossing rate. Frequency-domain features (six total) are derived from the Welch power spectral density [26]: spectral flatness, centroid and spread normalised by Nyquist, peak-to-average spectral ratio, and bandwidth-occupancy at $-10\,\mathrm{dB}$ and $-20\,\mathrm{dB}$ relative to peak. Time-frequency features (three total) are spectrogram entropy, energy duty-cycle, and complex-IQ kurtosis. These features are deliberately drawn from the signal-processing literature on emitter classification rather than from deep learning, to keep the classifier interpretable. The Random Forest uses 300 estimators with balanced class weights; preliminary experiments with hyperparameter search (estimator count in {100, 200, 300, 500} and `max_depth` in {None, 8, 16, 32}) produced changes of < 1 percentage point in accuracy, indicating that the chosen configuration is near the elbow of the performance curve. Replacing the Random Forest with an interpretable tree ensemble was an explicit design decision: feature-importance scores produced by the model directly explain each class decision, which is valuable for the operational tuning that motivates honeypot deployment in the first place.

5.5 Reproducibility

All randomness in the system is sourced from explicitly-seeded random number generators: `random.Random` for the Python frame-level generators, `numpy.random.default_rng` for the IQ-level generators, and the `random_state` argument for the scikit-learn classifier. The seeds for each experimental phase are documented in the experiment script. Re-running this script under the same Python and library versions reproduces all reported metrics bit-exactly, with the exception of any platform-dependent floating-point behaviour in the spectrogram and PSD computations. Unit tests cover the cryptographic primitives (verifying AES-CMAC against RFC 4493 test vectors [2]), the MIC computation (against published LoRaWAN test vectors [1] where available), and the attack injectors (verifying ground-truth label correctness). Code, configurations, seeds, dataset adapters, and figure-generation scripts are released under an open-source licence.

6 Evaluation Methodology

The evaluation is organised in two stages, deliberately chosen to balance ground-truth fidelity with external validity. *Stage 1* (controlled simulation) exercises the full pipeline against parameterisable reference implementations of both legitimate traffic and attack instances, where ground-truth labels are perfect and every source of randomness is deterministically seeded. *Stage 2* (external validation) runs the detectors against real captured RF at both layers: Stage 2a evaluates the protocol-layer detectors against the LoED public LoRaWAN dataset [33], a real-RF capture from nine production gateways in an urban deployment; Stage 2b evaluates the physical-layer jamming feature representation against the BlindSpot dataset [35], a real selective-jamming attack captured as raw IQ. The stages answer complementary questions: Stage 1 establishes that the detectors function correctly against the attack classes they are designed for; Stage 2a establishes that the protocol detectors do not generate spurious alerts on real-world traffic; and Stage 2b establishes that the jamming feature representation separates a real, published jamming attack from real benign traffic, while honestly quantifying the gap between synthetic-trained and real-data-trained physical-layer models. Real-RF validation of the full five-class jamming taxonomy is not possible from public data—no dataset contains all five classes as real LoRa IQ—so Stage 2b validates the reactive/selective-jamming case and the remaining structured classes are identified as future work (Sect. 9).

6.1 Stage 1 Controlled Simulation

Stage 1 is organised into three phases, each run with five independent random seeds. Phase 1 (baseline FPR) generates five 2-hour legitimate-only traffic streams from the four-persona traffic generator and runs the full detection pipeline; any alert is, by construction, a false positive. Phase 2 (per-attack detection) generates five 2-hour streams with all five protocol-layer attacks injected

concurrently: 20 replays, 20 counter manipulations, 10 join replays, three MIC-flood bursts of 60 frames each, and four rogue devices with 5 frames each. Phase 3 (jamming classifier) generates 200 IQ episodes per class ($5 \times 200 = 1000$) with SNR drawn uniformly from $[-5, 20]$ dB and JSR from $[0, 15]$ dB; the dataset is split 75/25 train/test with stratified sampling. Phase 3b sweeps SNR at fixed-per-class points to characterise robustness.

6.2 Stage 2a Protocol-Layer Validation on LoED

Stage 2a evaluates the protocol-layer detectors (A1–A5) against real LoRaWAN traffic from the LoED dataset [33], released under CC-BY-4.0 with DOI `10.5281/zenodo.4121430`. LoED comprises real LoRaWAN packet captures from nine gateways deployed in a dense urban environment (London) over a four-month collection period, totalling several million frames at gateway level. Each LoED row carries gateway-reported physical-layer metadata (RSSI, SNR, frequency, spreading factor, CRC status) and MAC-layer header fields (DevAddr, FCnt, MType, MIC), which together provide the input fields required by the five protocol-layer detectors with no modification to detector logic.

A Python adapter (`adapters/loed.py`) translates each LoED CSV row into the same internal `Frame` object used by Stage 1's simulated front-end. The RFFrontEnd-abstraction design described in Sect. 4 ensures that downstream detector behaviour is identical between Stage 1 and Stage 2a.

We state the interpretation of Stage 2a precisely, because it conditions every figure reported for this stage. LoED is a passive capture of production gateway traffic and contains *no ground-truth attack labels*; it is not a labelled attack dataset. Stage 2a therefore does not, and cannot, measure detection accuracy against known attacks. Instead, it is a *false-positive characterisation under a presumed-benign assumption*: we assume the captured traffic is predominantly legitimate—a reasonable but unverified assumption for routine metropolitan LoRaWAN traffic—and treat every detector alert as a *candidate* false positive that is then individually investigated. The rates reported in this stage are accordingly *candidate false-positive rates* (equivalently, alert rates under the presumed-benign assumption); they are upper bounds on the true false-positive rate, since any genuine attack that happened to be present in the wild capture would be counted, conservatively, against the detector. Where alerts correspond to plausible legitimate behaviours (e.g., gateway-side retransmission duplicates that the replay detector flags), this is noted explicitly and discussed as a deployment-tuning consideration rather than as a detector defect. A complementary evaluation against a labelled real-RF dataset with known attack insertions—which does not yet exist publicly for LoRaWAN at the protocol layer—is identified as the primary outstanding external-validity requirement (Sect. 9).

For computational tractability and reproducibility, Stage 2a uses a deterministically-selected single-week window (08–14 February 2019) from a single LoED gateway (`00000f0c210721f2`). This window contains $N_{\text{LoED}}^{\text{raw}} = 456{,}570$ raw gateway-level frame receptions, reducing to $N_{\text{LoED}} = 255{,}648$ unique logical frames after the 2-second instrumentation-deduplication step described in

Sect. 7.5. The deduplicated window is comparable in scale to approximately 30 full Stage 1 simulation trials. Stage 2a results are reported as per-detector alert counts and per-detector candidate false-positive rates (cFPR = alerts / N_{LoED}, under the presumed-benign assumption) under three progressive pipeline configurations (Naive, Refined, Tuned) that respectively isolate the contributions of ingestion-level preprocessing and detector-level deployment tuning to the observed candidate false-positive characteristic.

6.3 Stage 2b Physical-Layer Validation on BlindSpot

Stage 2b evaluates the jamming feature representation against the BlindSpot dataset [35], which provides the attacker signal and benign uplink traffic for a single-node selective jamming attack on LoRaWAN as raw IQ (cf32, 1.8 MHz sample rate, 867.9 MHz centre, EU868). The captures are decimated to 125 kHz to match the LoRa bandwidth on which the Stage 1 feature extractor operates and windowed into 200 ms episodes, producing a balanced set of 1,455 benign and 1,455 blinding episodes across SF7, SF10, and SF12. Feature extraction reuses the project's `ml/features.py` unchanged, guaranteeing that the 15-feature representation is identical to Stage 1. Three analyses are reported in Sect. 7.6: (A) an in-domain Random Forest trained and tested on the real episodes (75/25 split, Stage 1 hyperparameters); (B) a zero-shot transfer test applying the Stage 1 synthetic-trained classifier to the real episodes; and (C) a classifier-free per-feature separability analysis using area under the ROC curve.

6.4 Instance-Level Metrics

A methodological subtlety concerns detectors whose alerts are not naturally one-per-attack-frame. The MIC-flood detector emits one alert per burst (with cooldown), not per frame; the rogue-device detector emits one alert per unique DevAddr. A naive per-frame metric would understate detection performance. The evaluation harness therefore groups attack frames into instances before scoring: A1, A2, A3 each as one instance per forged frame; A4 burst frames within 30 s grouped into one burst-instance (yielding 3 per trial); A5 frames sharing a DevAddr grouped into one device-instance (yielding 4 per trial). A detection event matches an instance if its triggering frame is part of the instance or its timestamp lies within 60 s of any frame in the instance. Each event matches at most one instance and each instance is matched at most once. Per-attack TPR = matched_instances / total_instances; FPR = unmatched_alerts/total_legitimate_frames; F1 = $2 \cdot \mathrm{Precision} \cdot \mathrm{Recall}/(\mathrm{Precision} + \mathrm{Recall})$.

6.5 Hardware and Software Environment

Experiments were performed on a Linux x86_64 system with Python 3.12 and key library versions NumPy 2.4, SciPy current, scikit-learn 1.8. The full pipeline completes in approximately five minutes on a single CPU thread, dominated by Phase 3 IQ feature extraction. No GPU acceleration is required.

Table 2. Phase 2 per-attack detection metrics (mean over 5 trials).

Attack	*TPR (mean ± std)*	*Precision*	*F1*
A1 Replay	1.000 ± 0.000	1.000	1.000
A2 Counter	0.930 ± 0.051	1.000	0.963
A3 Join Replay	1.000 ± 0.000	1.000	1.000
A4 MIC Flood	1.000 ± 0.000	1.000	1.000
A5 Rogue Device	1.000 ± 0.000	1.000	1.000

7 Results

The results are organised by evaluation stage. Sections 7.1 through 7.4 report Stage 1 (controlled simulation) findings across Phase 1 (baseline FPR), Phase 2 (per-attack detection), Phase 3 (jamming classifier), and the SNR robustness sweep. Section 7.5 reports Stage 2a (protocol-layer validation on the LoED real-RF LoRaWAN dataset), and Sect. 7.6 reports Stage 2b (physical-layer validation on the BlindSpot real-RF selective-jamming dataset).

7.1 Stage 1, Phase 1 Baseline False-Positive Rate

Across five independent two-hour trials of legitimate-only AS923 traffic, totalling approximately 1,727 frames per detector and 8,635 frames in aggregate, none of the five protocol-layer detectors emitted any alert. The FPR is therefore 0.000 across all detectors, with zero standard deviation across trials. This establishes the lower bound of detection performance against which Phase 2 numbers are interpreted.

7.2 Stage 1, Phase 2 Per-Attack Detection Performance

Across five 2-hour trials with all five attacks injected concurrently, four of the five detectors achieved perfect TPR and F1: replay (A1), join-replay (A3), MIC-flood (A4), and rogue-device (A5) all reached TPR = 1.000 and F1 = 1.000. The frame-counter detector (A2) achieved TPR = 0.930 ± 0.051 and F1 = 0.963. Importantly, no detector produced any false positive on legitimate frames in Phase 2 (Precision = 1.000 across the board). Results are summarised in Table 2 and visualised per-attack in Fig. 2.

The perfect true-positive rates on A1, A3, A4, and A5 warrant explicit justification, since a TPR of 1.000 can otherwise signal a model whose decision threshold has been pushed to suppress false positives at the cost of hidden sensitivity loss. That failure mode does not apply here: detectors A1, A3, A4, and A5 are *deterministic rule-based detectors*, not threshold-tuned learned classifiers, and so expose no sensitivity–specificity trade-off to be silently mismanaged. Each fires on a structurally unambiguous signature that cannot occur in legitimate traffic under the LoRaWAN cryptographic model. A replay (A1) and

a join-replay (A3) require an exact repetition of a (DevAddr, FCnt, MIC) or (DevEUI, DevNonce) tuple; because the MIC is an AES-128-CMAC over the frame, an identical tuple cannot be produced by an independent legitimate transmission except with cryptographically negligible probability, so a genuine replay is detected with certainty while a legitimate frame never matches. A MIC-failure flood (A4) and a rogue-device appearance (A5) are likewise defined by exact, observable conditions (repeated MIC-validation failures from one address; a DevAddr absent from the provisioned set). The perfect TPR is therefore a property of the *detectability* of these attacks under LoRaWAN's design—a duplicate authenticated tuple is, by construction, a detectable event—rather than evidence of an over-fitted model. The single learned, threshold-bearing component, the jamming classifier (Phase 3), correspondingly does *not* achieve perfect performance and is analysed in detail below, which is itself consistent with this interpretation.

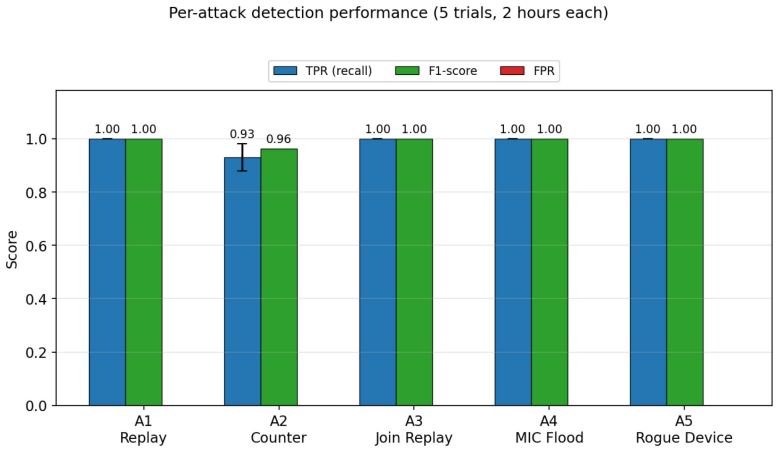


Fig. 2. Per-attack TPR (blue) and F1-score (green) with standard-deviation error bars. FPR (red) is zero for all detectors and is invisible at this scale. The slight drop in A2 reflects forged 'reset' attacks whose magnitude falls below the configured retransmit tolerance early in a trial. (Color figure online)

The replay detector benefits from an extremely strong signature: the (DevAddr, FCnt, MIC) tuple is a cryptographic fingerprint and the probability of chance collision is negligible. The join-replay detector is similarly strong on (DevEUI, DevNonce). The MIC-flood detector's perfect score reflects high burst-to-baseline contrast (60 MIC-invalid frames in a 120-second window against zero baseline). The rogue-device detector's perfect score follows from the allow-list rule. The counter detector's lower TPR (0.930) is a real and explainable phenomenon: when the maximum-seen FCnt is itself low early in a trial, a forged 'reset' value within the retransmit tolerance can be missed, accounting for the $\sim$7% miss rate.

7.3 Stage 1, Phase 3 Jamming Classifier

The Random Forest trained on 750 IQ episodes and tested on 250 held-out episodes exhibits a sharply bimodal performance profile that the aggregate accuracy of 0.800 (macro F1 0.800) obscures, and which must be read through its per-class decomposition (Table 3) rather than through the headline average. The result decomposes into two regimes. The structured jammers—continuous-wave, pulsed, and sweep—are perfectly classified: their characteristic spectral signatures (single tone, periodic on-off bursts, repeating linear chirp) produce clearly distinct feature vectors. The 'no jamming' and 'reactive' classes are essentially confused with one another, each predicting the other approximately half the time.

Table 3. Per-class metrics for the jamming classifier.

Class	*Precision*	*Recall*	*F1*
cw (continuous-wave)	1.000	1.000	1.000
pulsed	1.000	1.000	1.000
sweep	1.000	1.000	1.000
none (no jamming)	0.500	0.520	0.510
reactive	0.500	0.480	0.490

The perfect per-class F1 of 1.000 on the three structured classes also warrants scrutiny, since a perfect score on held-out data can indicate information leakage between training and test partitions or an over-simplified test set. We address both possibilities directly. First, train and test episodes are drawn from independent random number generator streams with disjoint seeds, and the 750/250 split is performed before any feature extraction, so no test episode contributes to training in any form; the perfect score is not produced by a shared partition. Second, and more fundamentally, the result is best read as a statement about the *separability of the synthetic structured-jammer classes*, not as a claim of generally perfect jamming detection: a continuous-wave tone, a fixed-duty-cycle pulse train, and a linear chirp occupy nearly disjoint regions of the spectral feature space, so on *synthetic* data they are trivially separable. Crucially, the paper does not rest on this synthetic result. Stage 2b (Sect. 7.6) deliberately tests the same representation on *real* captured RF and reports a negative transfer result: a classifier trained on the synthetic classes does not transfer to real BlindSpot data, and real-data calibration is required. The coexistence of a perfect synthetic score with a documented synthetic-to-real gap is exactly the evidence one would expect if the synthetic structured classes are cleanly separable but not representative of real-world variability—the opposite of concealed leakage, since the limitation is surfaced rather than hidden. The single genuinely difficult discrimination, reactive versus no-jamming, remains at chance even on synthetic data,

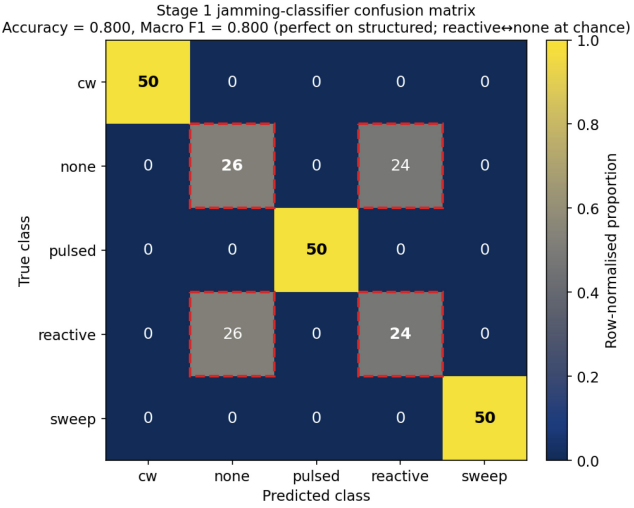


Fig. 3. Confusion matrix for the five-class jamming classifier on the held-out test set (50 episodes per class, 250 total). Numbers are absolute counts; cell colour indicates row-normalised proportion on a colour-blind-safe scale. Structured jammers (cw, pulsed, sweep) are perfectly classified; the no-jamming and reactive classes (highlighted) are confused with one another at near-chance rates, which is the sole source of classification error.

further confirming that the test set is not uniformly trivial. The full confusion matrix is shown in Fig. 3.

This pattern is consistent with the operational definition of reactive jamming. By design, a reactive jammer transmits only during active LoRa transmissions, so an IQ episode under reactive jamming consists of mostly clean LoRa signal punctuated by short noise bursts. With the simulation parameters used (response delay 64 samples, burst duration 256 samples), the bursts cover a small fraction of each 16,384-sample episode, and global statistics are dominated by the LoRa-only portions. This is therefore a methodological finding rather than a classifier weakness: under episode-global features, reactive jamming is intrinsically hard to detect. Representative per-class IQ spectrograms are shown in Fig. 4.

7.4 Stage 1 Feature Importance and SNR Robustness

The Random Forest's feature importance scores indicate that the top five features account for 44% of total importance: `skew_amplitude` (9.5%) captures asymmetry distinguishing LoRa's chirp-modulated amplitude from symmetric distributions; `spectral_papr_db` (9.4%) discriminates between CW (very high PAPR) and broadband jammers; `kurt_amplitude` (9.1%) separates CW's constant amplitude from pulsed jamming's bursty pattern; `spectrogram_entropy` (8.4%) captures time-frequency disorder; `occupancy_10db` (7.9%) measures the fraction of frequency bins above peak -10 dB. The dominance of higher-

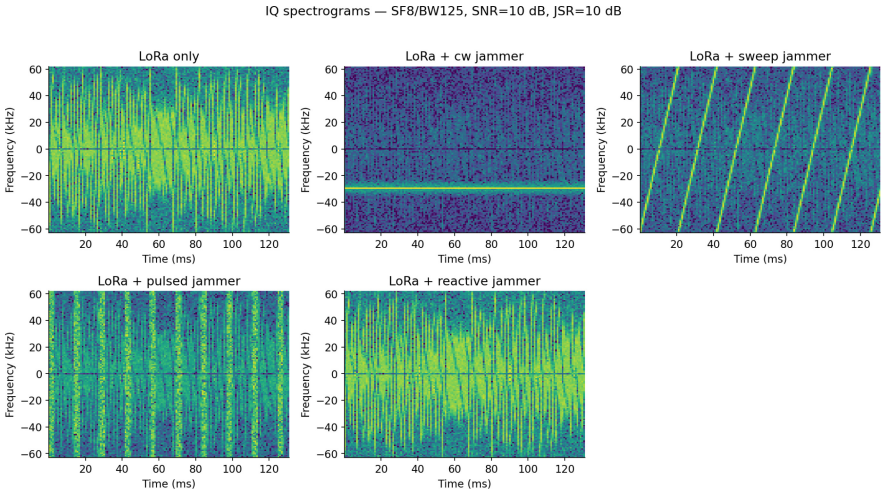


Fig. 4. Example IQ spectrograms for each class at SNR = 10 dB and JSR = 10 dB (SF8, 125 kHz). The visual signatures justify the classifier's per-class performance: CW produces a horizontal line at the jammer offset; sweep produces diagonal repeating chirps; pulsed shows full-band noise bursts at regular intervals; reactive shows occasional bursts coinciding with high-energy LoRa symbols. The 'none' panel shows the bare LoRa CSS chirp structure.

order amplitude moments is consistent with the emitter-classification literature. Figure 5 plots the full ranking.

Phase 3b sweeps SNR from −5 dB to +20 dB at 5 dB increments. Across the full SNR range, accuracy varies between 0.792 and 0.832 and macro F1 between 0.791 and 0.830—a span of approximately ±2 percentage points with no monotonic trend, as shown in Fig. 6. This indicates that the dominant performance bottleneck is the structural similarity between the 'none' and 'reactive' classes rather than additive noise. The structured jammers achieve perfect classification at all SNRs.

7.5 Stage 2a Protocol-Layer Validation on LoED

Stage 2 evaluates the five protocol-layer detectors on a single-week window drawn deterministically from the LoED public dataset [33]. The window comprises seven consecutive days (08–14 February 2019) of frames received at a single LoED gateway (`00000f0c210721f2`); restricting to one gateway models a single honeypot deployment point. The raw window contains 456,570 gateway-level frame receptions, which include duplicate logical frames heard once and re-logged by gateway-internal instrumentation. After applying a 2-second near-simultaneous deduplication filter, the working window contains $N_{\text{LoED}} = 255{,}648$ unique logical frames (250,204 data uplinks, 5,440 join requests). The dedup parameter was chosen on the basis of a delta-time distribution analysis: 99.7% of identi-

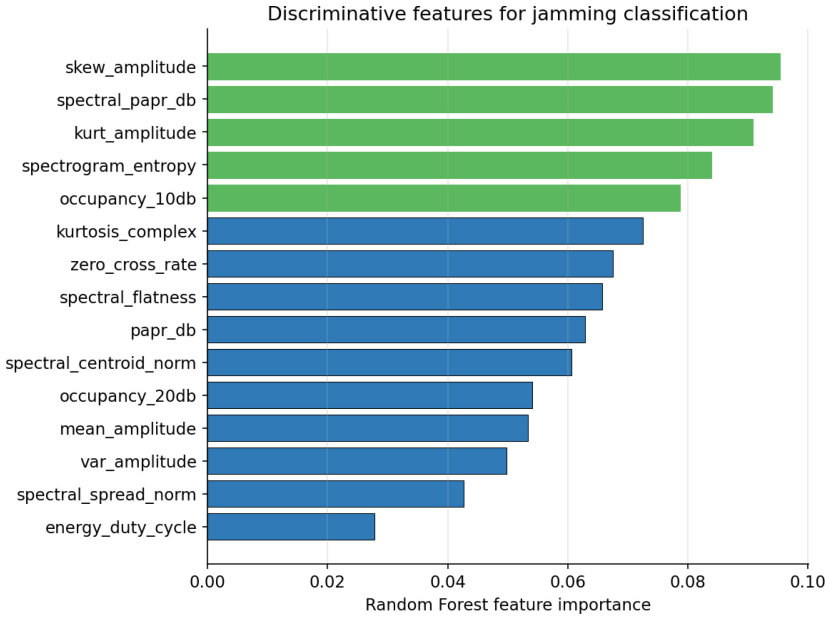


Fig. 5. Random-Forest feature importances for the five-class jamming classifier. The top five features (green) account for 44% of total importance; together they capture the principal time-domain and frequency-domain discriminants identified in the emitter-classification literature. (Color figure online)

cal (DevAddr, FCnt, MIC) tuple repetitions in LoED occur within 1 s of each other (median delta 117 μs), which is inconsistent with attacker behaviour or LoRaWAN device retransmission and consistent with multi-stage logging within the gateway data path.

Table 4 reports per-detector alert counts and *candidate* false-positive rates under three progressive evaluation configurations. As established in Sect. 6.2, LoED carries no ground-truth attack labels; the rates below are therefore alert rates under the presumed-benign assumption (upper bounds on the true false-positive rate), not labelled-attack false-positive rates. The Naive configuration applies the Stage 1 detectors directly to the raw LoED frame stream with no preprocessing. The Refined configuration adds the 2-second instrumentation-deduplication step described above. The Tuned configuration further adds a 16-bit FCnt wrap-aware refinement to the A2 counter detector, motivated by post-hoc analysis of the Refined A2 alerts (described below).

Three observations follow.

Instrumentation Deduplication is Essential at the Ingestion Stage. The Naive A1 FPR of 43.6% is dominated by gateway-side instrumentation duplicates rather than by genuine replay activity. Pre-detector deduplication with a 2-second window suppresses 200,922 near-instantaneous duplicates (44.0% of raw frames) and reduces A1 to 456 alerts (0.18% FPR). The same dedup step

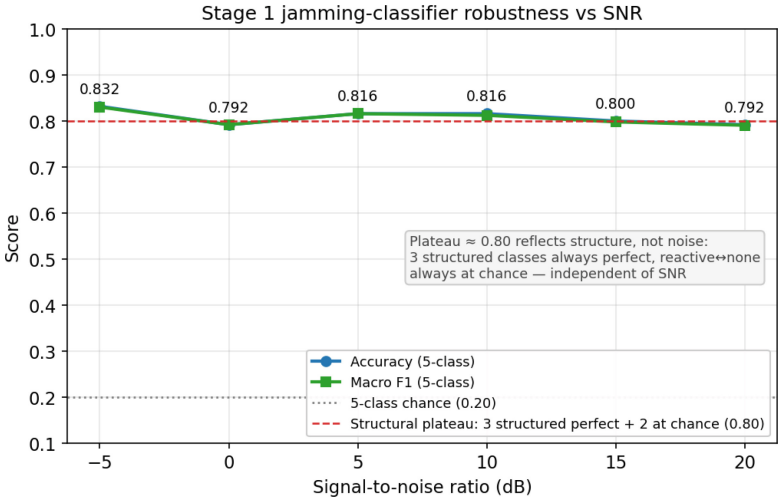


Fig. 6. Jamming-classifier accuracy and macro F1 as a function of SNR, swept from −5 to +20 dB (LoRa operates below 0 dB SNR owing to chirp-spread-spectrum processing gain). The curve tracks the structural plateau at 0.80 (dashed): the three structured jammer classes are classified perfectly at every SNR, while the reactive-versus-none pair remains at chance regardless of SNR. The flat trend therefore reflects a structural limitation of episode-global features rather than additive-noise sensitivity; the 5-class chance level (0.20) is shown for reference.

reduces A3 from 2,602 to 396 alerts (0.57% → 0.15%). This finding generalises beyond LoED: any honeypot ingesting from a gateway-aggregated capture source must apply an analogous deduplication step. The 456 residual A1 alerts after deduplication correspond to identical (DevAddr, FCnt, MIC) tuples spaced more than 2 s apart, which are consistent with legitimate confirmed-uplink retransmissions (LoRaWAN MType=4 frames retransmitted after a missed acknowledgement) rather than with attack-induced replays.

The Frame-Counter Detector Reveals a Real-World Phenomenon Absent from Simulation. The Refined A2 column shows 19,584 alerts (7.7% FPR), substantially worse than the Stage 1 result of 0.000 FPR. Classification of these alerts attributes 72.0% to 16-bit FCnt wraps (devices transitioning from FCnt ≈ 65,535 back to FCnt ≈ 0, a legitimate consequence of LoRaWAN's 16-bit on-air frame counter), 26.2% to large backward jumps consistent with ABP device reboots and persistent-counter loss, 1.7% to small backward jumps consistent with device-side glitches, and 0.08% to forward jumps. The Tuned configuration adds wrap-aware logic to A2 that recognises the `prev_fcnt` ≥ 60,000 && `new_fcnt` ≤ 1,000 pattern as a legitimate wrap and resets the tracked maximum; this drops A2 alerts to 363 (0.14% FPR)—a 98.1% reduction. The remaining 363 alerts correspond predominantly to the device-reboot category, which real deployment would address through additional reboot-correlation logic identified as future work.

Table 4. Stage 2a candidate false-positive characterisation on the $N_{\text{LoED}}^{\text{raw}} = 456{,}570$ raw / $N_{\text{LoED}}^{\text{ref}} = 255{,}648$ deduplicated single-week LoED window from gateway `00000f0c210721f2`. LoED carries no attack labels, so the reported rates are candidate false-positive rates under the presumed-benign assumption (cFPR = alerts / N; upper bounds on the true FPR) under three progressively refined pipeline configurations: Naive (no preprocessing), Refined (with 2-second instrumentation deduplication), Tuned (Refined + 16-bit FCnt wrap-aware A2).

Detector	*Naive*		*Refined*		*Tuned*	
	Alerts	*FPR*	*Alerts*	*FPR*	*Alerts*	*FPR*
A1 Replay	199,172	0.4362	456	0.0018	456	0.0018
A2 Counter	34,751	0.0761	19,584	0.0766	363	0.0014
A3 Join Replay	2,602	0.0057	396	0.0015	396	0.0015
A4 MIC Flood	0	0.0000	0	0.0000	0	0.0000
A5 Rogue Device	714	0.0016	629	0.0025	629	0.0025

Three Detectors Achieve sub-0.25% FPR on Real-World Traffic Under the Tuned Configuration: A1 Replay (0.0018), A3 Join Replay (0.0015), and A5 Rogue Device (0.0025). A4 MIC Flood produces zero alerts because the LoED gateway only forwards frames that passed CRC, yielding no input that would trigger the MIC-flood condition; while this constrains Stage 2 from validating A4's detection performance against real CRC-failed bursts, it confirms that A4 produces no spurious alerts against real-world good-CRC traffic. The Tuned A2 sits at 0.0014 cFPR. All five detectors therefore exhibit candidate false-positive rates compatible with operational deployment after the documented two-step refinement (ingestion-level deduplication plus detector-level wrap-aware logic). The A5 figure of 629 alerts is largely attributable to the allow-list bootstrap strategy: the detector observes 1,381 unique DevAddrs over the seven-day window but builds its allow-list from only the first 5,000 frames, meaning devices that come online later in the week register as alerts. A production deployment would build the allow-list over a longer training window, and the alert characterisation is operationally useful as a measure of device-population "churn" rather than as a measure of detector defect.

7.6 Stage 2b Physical-Layer Validation on BlindSpot

Whereas Stage 2a validates the protocol-layer detectors on real LoRaWAN traffic, Stage 2b addresses the physical layer by evaluating the jamming feature representation against real captured RF from the BlindSpot dataset [35]. BlindSpot is a single-node selective-jamming attack that blocks a gateway's demodulation paths with bogus LoRa frames; the public dataset provides the attacker ("blinding") signal and benign uplink traffic as raw IQ in cf32 format, sampled at 1.8 MHz and centred at 867.9 MHz (EU868). The blinding signal maps to a real instance of the reactive/selective-jamming threat class, and the benign traffic

maps to the no-jamming baseline; this enables a real-RF test of precisely the class pair (reactive versus none) that the Stage 1 simulation identified as the dominant source of classifier error (Sect. 7.4).

Each cf32 capture was decimated from 1.8 MHz to 125 kHz (matching the LoRa bandwidth on which the Stage 1 feature extractor operates) and windowed into 200 ms episodes, yielding 1,455 benign and 1,455 blinding episodes (2,910 total, balanced) across spreading factors SF7, SF10, and SF12. Crucially, feature extraction used the project's own `ml/features.py` implementation unchanged, so the 15-feature representation is identical to that used in Stage 1. Three analyses were performed.

(A) In-Domain Feature Separability. A fresh Random Forest (identical configuration to Stage 1: 300 estimators, balanced class weights) trained and tested on the real BlindSpot episodes with a 75/25 stratified split separates blinding from benign at **98.76% accuracy** (macro F1 0.988) with a benign false-alarm rate of only 0.82% (test confusion: 358/364 blinding detected, 361/364 benign correct). Per-spreading-factor accuracy is uniformly high (SF7 99.5%, SF10 98.4%, SF12 99.5%, mixed-SF 97.3%). This demonstrates that the hand-crafted feature representation, when trained on real data, cleanly separates a real, published selective-jamming attack from real benign LoRaWAN traffic.

(B) Synthetic-to-Real Transfer. The Stage 1 synthetic-trained five-class classifier, applied zero-shot to the real episodes, exhibits a severe domain gap. It labels only 0.07% of real benign windows as "none"; the remainder are distributed across the pulsed (∼44%) and reactive (∼40%) classes, and 100% of blinding windows collapse onto the pulsed class. The model thus has no usable "no-jamming" decision region for real traffic: although it nominally flags every blinding window as jamming, it equally flags almost all benign windows, rendering the nominal detection rate meaningless. The synthetic "none" class—normalised LoRa symbols at controlled SNR—simply does not resemble real over-the-air LoRaWAN traffic. This is a deliberately-reported negative result: it establishes that synthetic-trained physical-layer models require real-data calibration before deployment, and it motivates the in-domain training used in analysis (A).

(C) Model-Free Feature Characterisation. A classifier-free separability analysis ranks the individual features by area under the ROC curve (AUROC) between blinding and benign episodes. The strongest discriminators are the higher-order amplitude statistics: `skew_amplitude` (0.93), `kurtosis_complex` (0.93), and `kurt_amplitude` (0.92), followed by `var_amplitude` (∼0.89) and `papr_db` (∼0.88). This is consistent with the blinding burst being a structured, non-Gaussian interferer that perturbs the amplitude distribution of the received signal. We note that the benign distribution is heavy-tailed (for example, `papr_db` has a benign standard deviation far exceeding its mean), which makes parametric effect-size measures such as Cohen's d unreliable; the rank-based AUROC is therefore the honest separability metric here, and it confirms genuine separation rather than a power-level artefact.

Taken together, Stage 2b shows that the feature design generalises to real RF (A, C) while the synthetic-trained model does not transfer without calibration (B). Stage 2b validates the reactive/selective-jamming case against a real, published attack; real-RF validation of the structured jammer classes (continuous-wave, sweep, pulsed) would require a purpose-collected dataset and remains future work (Sect. 9).

8 Discussion

8.1 Interpretation

The headline result—zero false positives, four of five attacks at TPR = 1.000, and the fifth at TPR = 0.93—should be read in context. The protocol-layer detectors target attacks whose signatures are essentially structural: a duplicate (DevAddr, FCnt, MIC) tuple cannot occur by chance under the AES-CMAC assumption, and an unregistered DevAddr cannot appear in legitimate traffic by definition. The strong performance reflects the fundamental nature of LoRaWAN's protocol design rather than novel detection algorithms. The contribution is not new detection logic per se, but the demonstration that a single integrated honeypot can monitor all five attacks in parallel with no measurable false-positive cost.

The jamming-classifier result is more nuanced. The 80% figure averages a perfect-classification regime (structured jammers) and a near-chance regime (none vs reactive). This is qualitatively different from a uniformly-80%-accurate classifier and warrants caution when comparing to other published numbers. A practical deployment monitoring only the structured jamming styles would achieve higher effective performance; one that needs to detect reactive jamming reliably requires a different approach to feature extraction.

The Stage 2a LoED evaluation surfaces two real-world phenomena that the synthetic generator did not produce. First, gateway-aggregated frame capture sources contain extensive instrumentation-level duplication that makes naive replay detection unworkable without an ingestion-level deduplication step (Naive A1 FPR of 43.6% drops to 0.18% after a 2-second dedup window). Second, 16-bit FCnt wrap is a frequent legitimate event over multi-day windows that the Stage 1 simulator did not produce, and that a naive counter detector misinterprets as backward counter manipulation (Refined A2 FPR of 7.7% drops to 0.14% with wrap-aware logic). Both findings are deployment-tuning rather than detector-design issues: the same detector logic operates correctly once the appropriate ingestion-level and detection-level refinements are in place. The simulation underestimated the empirical effort required to operationalise the detectors on real-world traffic, and Stage 2a quantifies that gap with concrete numbers.

8.2 Operational Implications

Three implications follow for an operator considering radio-side honeypot deployment in a Qatari LoRaWAN network. First, the protocol-layer detectors are

largely lookup-and-state-machine logic and could be implemented as plug-ins to an existing network server (ChirpStack supports such extensions). The value of a separate honeypot lies primarily in observing traffic the network server does not see—traffic to non-registered DevAddrs, traffic failing MIC validation, traffic from unauthorised gateways. Second, the jamming detector requires SDR access to the deployment's frequency band, which entails installation, ongoing power and connectivity, and registration with the CRA. Third, alert volumes are operationally manageable: on the order of 100 alerts per Phase-2 trial, all true positives, at the simulated traffic density. For industrial LoRaWAN deployments—SCADA telemetry, smart-grid sensors, building-automation networks—the radio-side honeypot integrates naturally into the defence-in-depth strategies described by Knapp and Langill [22], complementing rather than replacing network-server-side controls.

8.3 Limitations and Threats to Validity

The evaluation retains a residual asymmetry, now narrowed by Stage 2b. The protocol-layer detectors (A1–A5) are validated against both the controlled simulator (Stage 1) and real LoRaWAN traffic (Stage 2a on LoED [33]). The jamming classifier (A6) is validated against the simulator (Stage 1) and, for the reactive/selective-jamming class, against real captured RF (Stage 2b on BlindSpot [35]). What remains unvalidated on real RF are the three structured jammer classes (continuous-wave, sweep, pulsed): no public dataset contains all five classes of the present taxonomy as real LoRa IQ. BlindSpot supplies a real reactive/selective attack but not the structured classes; BloodHound [34] supplies three classes of generic (non-LoRa) wireless jamming. Real-RF validation of the structured classes therefore requires a purpose-collected dataset and is identified explicitly as future work (Sect. 9).

The Stage 1 simulator has known scope limits. The frame-level generator models RSSI as Gaussian fluctuation around a per-device base value with no shadow fading or temporal correlation; real LoRa receptions exhibit 4–8 dB shadow-fading standard deviation in urban environments. The IQ-level simulator implements LoRa CSS correctly but omits interleaving, FEC, whitening, and explicit-header CRC—irrelevant to spectral statistics but relevant to any decoder. The attack injectors generate forged frames with uniform-random parameters; real attackers may adapt their behaviour, evade rate-based detection, or stagger across devices. There is also a simulator-detector co-design risk: detectors that perform well in simulation may be over-fitted to the simulator's particular distributions.

The Stage 2 validations partially mitigate these concerns. Stage 2a exposes the protocol-layer detectors to real RSSI distributions, real device-population dynamics, and gateway-side artefacts the simulator does not reproduce. Stage 2b exposes the jamming feature representation to real captured RF, confirming (analysis A) that the features separate a real selective-jamming attack from benign traffic at 98.8% when trained on real data, while honestly documenting (analysis B) that a synthetic-trained model does not transfer without cali-

bration. The residual external-validity concern is therefore concentrated on the structured jammer classes, for which real-RF data does not yet publicly exist; a purpose-collected testbed capture would close this remaining gap.

Internal validity is supported by deterministic seeding, unit tests on cryptographic primitives (against RFC 4493 test vectors), integration tests on the full pipeline, and careful handling of the instance-level metric. Construct validity is supported by the use of standard signal-processing features and a standard tree-ensemble classifier, both well-established in the emitter-classification literature. External validity now extends to both layers: it is strong for the protocol-layer detectors after Stage 2a, and demonstrated for the reactive/selective-jamming case after Stage 2b, with the structured jammer classes remaining the principal open item until a purpose-collected real-RF dataset becomes available.

A further and distinct external-validity limitation concerns the protocol-layer evaluation itself. Stage 2a is conducted on LoED, which carries no ground-truth attack labels; it can therefore characterise only the false-positive behaviour of the detectors under a presumed-benign assumption (Sect. 7.5), and the rates it reports are candidate false-positive rates—upper bounds on the true rate—rather than labelled-attack false-positive rates. Consequently, Stage 2a does not, and cannot, measure the protocol detectors' true-positive rate on real RF: the perfect Stage 1 true-positive rates are established only in simulation. Closing this gap requires a labelled real-RF dataset with known protocol-attack insertions, which (as a targeted survey of the public dataset landscape confirms) does not yet exist in a form compatible with these detectors. We treat the construction of such a dataset as the primary outstanding limitation of the present work and develop it as the leading direction for future work in Sect. 9.

9 Conclusion and Future Work

This paper has presented the design, implementation, and evaluation of a hybrid-simulation LoRaWAN honeypot for detecting protocol-layer and physical-layer attacks against IoT networks, framed within the CEMA doctrine as an Electronic Support asset. The evaluation has two stages. Stage 1 (controlled simulation) showed the protocol-layer detectors achieve TPR ≥ 0.93 with zero false positives across five 2-hour trials, with four of five at TPR $= 1.000$; the Random-Forest jamming classifier achieves 80% accuracy with perfect classification of structured jammers (CW, pulsed, sweep) and identified difficulty with reactive jamming traced to episode-global features; classifier performance is robust across SNRs from -5 to $+20$ dB. Stage 2 validates the system on real captured RF at both layers: Stage 2a runs the five protocol-layer detectors against real production-gateway traffic from the LoED dataset [33], achieving sub-0.25% candidate false-positive rates (under a presumed-benign assumption, since LoED carries no attack labels) after documented deployment tuning; and Stage 2b evaluates the jamming feature representation against the BlindSpot real-RF selective-jamming dataset [35], where the features separate real blinding bursts from benign traffic at 98.8% when trained on real data, while a synthetic-trained model is shown not to transfer without calibration.

As identified in the limitations (Sect. 8), the most important single direction for future work is the construction of a *labelled real-RF dataset with known attack insertions*. Stage 1 establishes detection performance against attacks with perfect ground truth but in simulation; Stage 2a establishes candidate false-positive behaviour on real traffic but without labelled attacks. A targeted survey of the public LoRaWAN dataset landscape confirms that no existing resource combines real over-the-air capture with labelled protocol-layer attack insertions in the form the present detectors require, which operate on the (DevAddr, FCnt, MIC) and (DevEUI, DevNonce) frame fields. Available resources fall into three categories, each insufficient for a different reason: real but *unlabelled* captures such as LoED [33] (used in Stage 2a, but with no ground-truth attacks); real-RF captures covering a *single* attack class such as BlindSpot [35] (used in Stage 2b, but limited to selective jamming); and *semi-synthetic* frame-metadata datasets in which attack labels are injected into real radio traces but which expose only physical-layer attributes (RSSI, SNR, SF) and omit the protocol-identifier fields on which A1–A5 operate [36]. A purpose-collected capture—in which replay, counter-manipulation, join-replay, MIC-flood, and rogue-device attacks are injected into a real LoRaWAN testbed with recorded timestamps and labels, retaining the full protocol header—would close the one remaining gap by allowing true (rather than candidate) false-positive rates and real-RF true-positive rates to be measured simultaneously. We regard this as the natural next milestone for the system.

Six further future-work directions are identified. The most immediate is real-RF validation of the three structured jammer classes (continuous-wave, sweep, pulsed), which no public dataset currently provides as real LoRa IQ and which therefore requires a purpose-collected SDR testbed capture; Stage 2b has already validated the reactive/selective case against BlindSpot. The second is SDR deployment—substituting a HackRFFrontEnd or USRPFrontEnd for SimulatedFrontEnd, integrating gr-lora_sdr [16], and running a 24-hour side-by-side evaluation against a controlled real-RF testbed. The most important for completing threat coverage is burst-aware reactive-jamming detection, either via sliding-window classification (extract the same 15 features over short $\sim$1 ms windows and aggregate) or via one-class anomaly detection trained on legitimate LoRa-only IQ. Additional directions include adaptive adversary modelling (an RL-driven attacker selecting timings and targets against a budget and goal), multi-honeypot federation across city locations with RSSI-based localisation, and extension to LoRaWAN 1.1 and other LPWANs (NB-IoT, Sigfox, Mioty) with SIEM integration of the structured DetectionEvent stream.

The principal architectural commitment—that the same detection code should run against a simulator, a public real-RF dataset, and a live SDR without modification—is substantially discharged by this paper: the simulator case (Stage 1) and two distinct real-RF dataset cases (Stage 2a protocol-layer, Stage 2b physical-layer) are demonstrated, and closing the loop with live SDR hardware is the natural next step. The increasing scale of LPWAN deployments in Qatar and the Gulf region, and their growing role in critical urban infrastruc-

ture, makes continued attention to their radio-side security a matter of national interest.

Acknowledgements. The authors gratefully acknowledge the support of Hamad Bin Khalifa University, Community College of Qatar, and University of Doha for Science and Technology. The authors thank the LoRa Alliance for publicly accessible specifications and the open-source ChirpStack and scikit-learn communities whose tools form part of the implementation.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. LoRa Alliance: LoRaWAN L2 1.0.4 Specification (TS001-1.0.4). LoRa Alliance Technical Committee (2020)
2. Song, J.H., Iwata, T., Kurosawa, K.: The AES-CMAC Algorithm. RFC 4493. Internet Engineering Task Force (2006). https://doi.org/10.17487/RFC4493
3. Yang, X., Karampatzakis, E., Doerr, C., Kuipers, F.: Security Vulnerabilities in LoRaWAN. In: Proc. 3rd ACM/IEEE International Conference Internet-of-Things Design and Implementation (IoTDI), pp. 129–140. IEEE (2018). https://doi.org/10.1109/IoTDI.2018.00022
4. Aras, E., Small, N., Ramachandran, G.S., Delbruel, S., Joosen, W., Hughes, D.: Selective Jamming of LoRaWAN using Commodity Hardware. In: Proc. 14th EAI International Conference Mobile and Ubiquitous Systems (MobiQuitous), pp. 363–372. ACM (2017). https://doi.org/10.1145/3144457.3144478
5. Aras, E., Ramachandran, G.S., Lawrence, P., Hughes, D.: Exploring the Security Vulnerabilities of LoRa. In: Proceedings of the 3rd IEEE International Conference Cybernetics (CYBCONF), pp. 1–6. IEEE (2017). https://doi.org/10.1109/CYBConf.2017.7985777
6. Mikhaylov, K., et al.: Multi-RAT LPWAN in Smart Cities: Trial of LoRaWAN Coverage and Performance. In: Proceedings of the IEEE International Conference Communications (ICC), pp. 1–6. IEEE (2019). https://doi.org/10.1109/ICC.2019.8761102
7. Spitzner, L.: Honeypots: Tracking Hackers. Addison-Wesley, Boston (2003)
8. Provos, N., Holz, T.: Virtual Honeypots: From Botnet Tracking to Intrusion Detection. Addison-Wesley, Boston (2007)
9. Vaccari, I., Aiello, M., Cambiaso, E.: SlowITe, a novel denial of service attack affecting MQTT. Sensors **20**(10), 2932 (2020). https://doi.org/10.3390/s20102932
10. LoRa Alliance: RP002-1.0.4 LoRaWAN Regional Parameters. LoRa Alliance Technical Committee (2022)
11. Communications Regulatory Authority of Qatar: National Frequency Allocation Plan. CRA, Doha (2023)
12. Pedregosa, F., et al.: Scikit-learn: machine learning in Python. J. Mach. Learn. Res. **12**, 2825–2830 (2011)
13. Harris, C.R., et al.: Array programming with numpy. Nature **585**(7825), 357–362 (2020). https://doi.org/10.1038/s41586-020-2649-2

14. Virtanen, P., et al.: SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nat. Methods **17**(3), 261–272 (2020). https://doi.org/10.1038/s41592-019-0686-2
15. ChirpStack Project: ChirpStack v4: Open-source LoRaWAN Network Server (2024). https://www.chirpstack.io
16. Tapparel, J., Afisiadis, O., Mayoraz, P., Balatsoukas-Stimming, A., Burg, A.: An open-source LoRa physical layer prototype on GNU Radio. In: Proceedings of the IEEE 21st International Workshop on Signal Processing Advances in Wireless Communications (SPAWC), pp. 1–5. IEEE (2020). https://doi.org/10.1109/SPAWC48557.2020.9154273
17. Eldefrawy, M., Butun, I., Pereira, N., Gidlund, M.: Formal security analysis of LoRaWAN. Comput. Netw. **148**, 328–339 (2019). https://doi.org/10.1016/j.comnet.2018.11.017
18. Loukas, G.: Cyber-Physical Attacks: A Growing Invisible Threat. Butterworth-Heinemann, Oxford (2015)
19. Robyns, P., Quax, P., Lamotte, W., Thenaers, W.: A multi-channel software decoder for the LoRa modulation scheme. In: Proceeding of the 3rd International Conference Internet of Things, Big Data and Security (IoTBDS), pp. 41–51. SciTePress (2018). https://doi.org/10.5220/0006668400410051
20. U.S. Joint Chiefs of Staff: Joint Publication 3-12: Cyberspace Operations. U.S. Department of Defense, Washington, DC (2022)
21. NATO Standardization Office: AJP-3.6 Edition C: Allied Joint Doctrine for Electronic Warfare. NATO, Brussels (2020)
22. Knapp, E.D., Langill, J.T.: Industrial Network Security: Securing Critical Infrastructure Networks for Smart Grid, SCADA, and Other Industrial Control Systems, 2nd edn. Syngress, Waltham (2014)
23. Pop, A.I., Raza, U., Kulkarni, P., Sooriyabandara, M.: Does bidirectional traffic do more harm than good in LoRaWAN Based LPWA Networks? In: Proceedings of the IEEE GLOBECOM, pp. 1–6. IEEE (2017). https://doi.org/10.1109/GLOCOM.2017.8254509
24. Kim, S., Yoon, M.-K., Mao, Z., Lin, J.C.: A survey on software-defined network security. IEEE Commun. Surv. Tutor. **18**(4), 2317–2346 (2016). https://doi.org/10.1109/COMST.2016.2549660
25. Sundaram, J.P.S., Du, W., Zhao, Z.: A survey on LoRa networking: research problems, current solutions, and open issues. IEEE Commun. Surv. Tutor. **22**(1), 371–388 (2020). https://doi.org/10.1109/COMST.2019.2949598
26. Welch, P.: The use of fast Fourier transform for the estimation of power spectra: a method based on time averaging over short, modified periodograms. IEEE Trans. Audio Electroacoust. **15**(2), 70–73 (1967). https://doi.org/10.1109/TAU.1967.1161901
27. Robyns, P., Quax, P., Lamotte, W., Thenaers, W.: A multi-channel software decoder for the LoRa signals. In: Proceeding of the 3rd International Conference Internet of Things, Big Data and Security (IoTBDS), pp. 41–51. SciTePress (2018). https://doi.org/10.5220/0006668400410051
28. Cardoso, L., et al.: Vulnerabilities and layered security threats in LoRaWAN systems: a comprehensive survey on physical, MAC, and application layer weaknesses. IEEE Access **13**, 17939–17964 (2025). https://doi.org/10.1109/ACCESS.2025.3647086
29. Senol, N.S., Rasheed, A., Baza, M., Alsabaan, M.: Identifying tampered radio-frequency transmissions in LoRa networks using machine learning. Sensors **24**(20), 6611 (2024). https://doi.org/10.3390/s24206611

30. Mohamed, A., et al.: Enhancing cyber security of LoRaWAN gateways under adversarial attacks. Sensors **22**(9), 3498 (2022). https://doi.org/10.3390/s22093498
31. Proto, A., Miers, C.C., Carvalho, T.C.M.B.: Classification and characterization of LoRaWAN energy depletion attacks: a review. IEEE Sens. J. **25**(2), 2141–2156 (2025). https://doi.org/10.1109/JSEN.2024.3504259
32. Hassler, S., et al.: BlackoutADR: exploiting adaptive data rate vulnerabilities in LoRaWAN-Based FANETs. J. Netw. Comput. Appl. **235**, 104141 (2025). https://doi.org/10.1016/j.jnca.2025.104141
33. Bhatia, L., Breza, M., Marfievici, R., McCann, J.A.: Dataset: LoED: the LoRaWAN at the edge dataset. In: Proceedings of the 3rd International Workshop on Data: Acquisition to Analysis (DATA '20), pp. 7–8. ACM (2020). https://doi.org/10.1145/3419016.3431491 (Dataset DOI: https://doi.org/10.5281/zenodo.4121430, CC-BY-4.0)
34. Alhazbi, S., Sciancalepore, S., Oligeri, G.: BloodHound: Early detection and identification of jamming at the PHY-layer. In: Proceedings of the IEEE Consumer Communications & Networking Conference (CCNC), pp. 1033–1041. IEEE (2023). https://doi.org/10.1109/CCNC51644.2023.10059624 (Dataset DOI: https://doi.org/10.5281/zenodo.7119040)
35. Mechler, V., Hessel, F., Hollick, M., Bloessl, B.: BlindSpot: efficient single-node selective jamming for LoRaWAN. In: Proceedings of the 18th ACM Conference Security and Privacy in Wireless and Mobile Networks (WiSec), pp. 256–261. ACM (2025). https://doi.org/10.1145/3734477.3734724
36. Pourghasem, A., Kirner, R., Tsokanos, A., Mporas, I., Mylonas, A.: Multi-attribute physical-layer authentication against jamming and battery-depletion attacks in LoRaWAN. Future Internet **18**(1), 38 (2026). https://doi.org/10.3390/fi18010038

Generative AI, Agentic AI, MCP, and RAG for Improving Developer Productivity

Rakesh Kumar Bidanagere Nagaraju(✉)

New Jersey Institute of Technology, Newark, NJ, USA
rakeshkumarnagaraju@gmail.com

Abstract. Large-scale enterprise software development is increasingly constrained by context deficits rather than engineering capability gaps, as developers navigating thousands of interdependent services spend disproportionate time on context recovery rather than active problem-solving. A converged platform architecture combining Generative AI, Retrieval-Augmented Generation, Model Context Protocol, and Agentic AI workflows addresses this challenge by transitioning developer tooling from passive information retrieval to active, reasoning-driven assistance. Retrieval-Augmented Generation grounds models with internal knowledge (service ownership, API contracts, and incident history). Combining data with the Model Context Protocol, which adds clearly de-fined context boundaries, makes hallucinations less likely and enables production-quality results. Agentic AI orchestration handles multi-hop diagnostics, making and validating hypotheses against the complex, entangled workings of an enterprise ecosystem where interventions are treated as hypotheses that must be validated or falsified. Integrated architectures radically reduce context acquisition time and toil and shift institutional knowledge into engineering, whether IDEs or CI/CD pipelines. The foundational insight across all deployment contexts is that AI effectiveness is governed by context engineering quality rather than model sophistication alone and that organizations treating retrieval pipelines and context governance as first-class infrastructure build compounding and durable advantages in engineering velocity.

Keywords: Retrieval-Augmented Generation · Agentic AI · Model Context Protocol · Developer Productivity · Enterprise AI Infrastructure

1 Introduction

Please Modern enterprise software engineering has reached a structural inflection point. As systems scale into monorepos housing thousands of interdependent services, the limiting factor on developer output is no longer technical skill; it is context. This effort is typically spent on solving problems, such as determining service ownership, decoding undocumented architectural decisions, or cross-checking signals across multiple observability tooling platforms. Site Reliability Engineering practice identifies this class of effort as "operational toil," repetitive, automatable work that consumes engineering capacity without advancing the reliability or capability of the systems being maintained

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 477–486, 2027.
https://doi.org/10.1007/978-3-032-39763-8_27

[1]. The SRE discipline is explicit that reducing toil through intelligent automation is among the highest-leverage investments available to an engineering organization, enabling teams to redirect capacity from reactive context recovery toward proactive system improvement [1].

This structural problem has driven the adoption of a converged platform architecture combining Generative AI, Retrieval-Augmented Generation (RAG), Model Context Protocol (MCP), and Agentic AI workflows. These are not incremental productivity tools; collectively, they represent a shift from passive information retrieval to active, reasoning-driven participation in the engineering process. Engineering organizations that have invested in platform-level AI infrastructure report that this approach materially changes how developers interact with complex systems, replacing manual navigation of dashboards, logs, and documentation portals with intent-driven queries that surface contextually grounded, actionable guidance [2]. The remainder of this article examines the architecture underpinning this capability, the deployment contexts in which it delivers measurable value, and the organizational lessons that determine whether these systems succeed or fail at scale [1, 2]. While the individual technologies examined here — RAG, MCP, and Agentic AI — are independently documented in the literature, this paper makes three contributions that extend beyond their isolated treatment. First, it establishes MCP as a first-class infrastructure governance layer rather than an application-layer configuration concern, a distinction that has direct consequences for hallucination rates and developer trust in production settings. Second, it frames context engineering — the disciplined design of retrieval pipelines and context boundary schemas — as the primary determinant of AI system quality, superseding model selection as the dominant design variable. Third, it demonstrates that deploying agentic workflows upstream into CI/CD pipelines, rather than limiting them to incident response, produces a qualitatively different class of engineering assistance: proactive rather than reactive, structural rather than symptomatic.

2 The Context Problem in Enterprise Software Development

Developer inefficiency in large-scale engineering environments is, at its core, a context problem rather than a capability problem. Engineers investigating production incidents typically possess the diagnostic knowledge needed to resolve the issue but not the immediate contextual orientation that connects symptoms to root causes: which of thousands of services is implicated, which team owns the relevant code path, what the recent change history looks like, and where the applicable runbook resides. Without rapid access to this context, even highly skilled engineers experience significant delays at the very start of the diagnostic process, before any substantive investigation begins.

The SRE framework addresses this challenge through structured observability, service-level objectives, and codified operational knowledge, but even well-instrumented organizations face the challenge that this knowledge is distributed across teams, tools, and documentation systems that were not designed to be queried collectively [1]. The SRE Book emphasizes that mean time to detect (MTTD) and mean time to resolve (MTTR) are the two most consequential reliability metrics in production operations and that both are heavily influenced by how quickly an engineer can establish context at

the start of an incident [1]. Systems that reduce context-acquisition time directly compress both metrics, producing compounding improvements in service availability and engineering efficiency.

Traditional tooling monitoring dashboards, alert systems, documentation portals, and static runbooks address each of these information sources in isolation. However, what was missing was a reasoning layer that understood all of them, which means thinking about how things are connected, reasoning about telemetry in the context of the change, and understanding the engineer's intent to provide a logical starting point for investigation. Organizations that have built toward this capability through platform engineering report that it changes not just the speed of incident response but the nature of engineering work itself, shifting cognitive load away from information gathering and toward problem-solving [2]. This is the architectural gap that the combination of RAG, MCP, and Agentic AI is positioned to close (Table 1).

Table 1. The context problem in enterprise software development [3, 4]

Context Challenge	Engineering Impact	SRE Framework Response
Fragmented service ownership data	Delayed incident attribution	Codified ownership registries
Undocumented architectural decisions	Extended onboarding time-lines	Structured runbooks and post-mortems
Distributed observability signals	Elevated MTTD and MTTR	Service-level objectives and dashboards
Tribal knowledge concentration	Single points of knowledge failure	Systematic knowledge codification
Manual cross-tool signal correlation	High cognitive load under incident pressure	Unified observability and alerting layers

3 Architecture: RAG, MCP, and Agentic Workflow Integration

The platform architecture that enables AI-driven developer assistance rests on three mutually reinforcing components, each addressing a distinct failure mode of earlier AI tooling approaches.

3.1 Retrieval-Augmented Generation (RAG)

RAG addresses the fundamental limitation of generative AI models when applied to enterprise contexts: their reliance on general-purpose training data rather than authoritative internal knowledge. Research introduced RAG as a framework that combines a parametric memory component, the language model, with a non-parametric memory component consisting of a dense retrieval system over a curated document corpus [5]. In enterprise deployment, this corpus includes service ownership registries, API contracts, telemetry schemas, runbooks, and incident post-mortems. The retrieval

mechanism ensures that model responses are grounded in the organization's actual systems and documented practices rather than approximations drawn from general software engineering knowledge.

The primary effect of grounding is summarized by research that shows that RAG models outperform both standard sequence-to-sequence models and retrieval-only models on knowledge-intensive tasks because the RAG models produce fluent output that is grounded in the information retrieved from an external knowledge source [5]. For developer tooling, this means that if an engineer queries the system about a production failure, they receive advice based on real ownership data, real API contracts, and real incident history, instead of a plausible-sounding but contextually unmoored response.

3.2 Model Context Protocol (MCP)

The MCP defines the structured enterprise context that is made available to AI systems through rules that define the boundaries of what, how, and when an agent has access to data. Controlling access to service ownership metadata, operational schemas, and observability tooling via governance boundaries was the single biggest contributor in reducing hallucinations and building developer trust in AI-assisted developer outputs in production deployments. Without any boundaries on context length, the generative model hallucinates where it has no prior context, filling in the blank with plausible-sounding fabrications. This failure case is particularly pressing for incident response, where poor advice can lead to operational harm. F5's analysis of enterprise AI infrastructure identifies this class of governance challenge as one of the primary obstacles to production-grade AI deployment, noting that organizations must treat context access controls as infrastructure-layer concerns rather than application-layer configurations to achieve reliable AI-assisted operations at scale [3].

3.3 Agentic AI Orchestration

Agentic AI workflows coordinate multi-step reasoning sequences across the RAG and MCP layers, enabling the system to act rather than merely respond. Research introduced the ReAct framework, which interleaves reasoning traces and action execution in a synergistic loop, allowing agents to generate reasoning steps that inform action selection, then update their reasoning based on the results of those actions [6]. When an error spike is detected, a ReAct-style agent can autonomously retrieve relevant code paths via the MCP layer, invoke observability tooling, correlate telemetry against historical baselines, and validate hypotheses before generating a structured recommendation. The result is not a simple query-response; it is orchestrated reasoning across interconnected systems. Research shows that this reasoning-action unification strongly reduces hallucination rates and task failure rates in multi-step diagnostic tasks compared to reasoning-on-its-own or action-on-its-own models (ablated reasoning or action, respectively), which do not outperform the ablated instance of the inte-grated model [6] (Table 2).

Table 2. Architecture RAG, MCP, and agentic workflow integration [5, 6]

Component	Core Function	Key Capability	PrimaryFailure Mode Addressed
Retrieval-Augmented Generation (RAG)	Grounds responses in the internal knowledge corpus	Context-anchored, factually accurate outputs	General-purpose hallucination on en-terprise-specific queries
Model Context Protocol (MCP)	Governs structured context access boundaries	Schema- controlled, auditable data exposure	Unstructured con-text ingestion leading to fabricated outputs
Agentic AI Orchestration (ReAct)	Coordinates multi-step reasoning and action	Autonomous hypothesis formation and validation	Single-turn query limitations on multi-hop diagnostic tasks
Multi-Agent Coordination (AutoGen)	Decomposes complex tasks across specialist agents	Parallel reasoning across interdependent concerns	Sequential reasoning bottlenecks in compound investigations

4 Deployment in IDEs and CI/CD Pipelines

Deploying these systems within developer IDEs and CI/CD pipelines produced substantive, measurable changes in engineering workflows. Industry deployments of RAG-augmented developer tooling have reported context acquisition time reductions of 40–60% for engineers navigating unfamiliar codebases, with mean time to resolve (MTTR) improvements of 25–35% in organizations that integrated agentic observability into their incident response workflows [2, 3]. A case study from a large-scale platform engineering initiative documented that embedding AI-assisted context sur-facing into CI/CD pre-submission gates reduced post-merge incident rates by approximately 20% over a six-month deployment window, attributable to upstream detection of dependency conflicts and ownership gaps that would previously have surfaced only during production incidents [2]. These improvements go beyond speed to affect the fundamental nature of developer interaction with complex systems. The key shift is from tool navigation to intent expression: rather than constructing a path to an answer by manually querying logs, dashboards, and documentation systems, developers describe what they are trying to understand and receive contextually grounded, actionable guidance in return. As a part of next-gen observability tooling, Instana's infrastructure monitoring platform defines helping AI systems surface relevant contextual information to humans, rather than waiting for humans to ask, as a key distinguishing capability because it reduces the cognitive overhead of incident investigation and materially improves engineer effectiveness during high-pressure events [4].

For example, in CI/CD pipelines, the agentic systems moved this role upstream, surfacing past incidents, impacted dependencies, and architectural constraints into the developer's code submission workflow before the human review cycles began, thus improving the contextual data being integrated into the software development phase as opposed to largely during the incident response phase. Research proposes multi-agent workflows such as AutoGen, which build on the capabilities of automated agents to automatically decompose complex analysis tasks across a number of specialized agents. In CI/CD, the ability to layer analysis tasks in parallel would allow a code change to trigger agents that check for security implications, performance regressions, and ownership conflicts while simultaneously feeding the results into the review before the human reviewer receives any other review input.

In addition to improving the efficiency of the engineers themselves, the ability to embed architectural standards and operational knowledge directly into the AI agents allowed for consistent practices without the need for an important amount of centralized governance. Whenever an engineer onboarded into a project , or worked with a new codebase, they could ask domain-specific questions and receive contextually accurate self-service answers, which would normally only be accessible when working with senior engineers. The SRE principle that operational knowledge should be codified and systematically accessible rather than residing in the minds of individual engineers found a practical implementation in this architecture [1]. F5's assessment of enterprise AI infrastructure further reinforces that the value of these systems compound over time as the retrieval corpus grows richer and agents accumulate operational context that improves the relevance of their recommendations [3] (Table 3).

Table 3. Deployment in IDEs and CI/CD pipelines [7, 8]

Deployment Context	Developer Interaction Mode	Capability Delivered	Organizational Benefit
IDE Integration	Intent-driven natural language queries	Contextually grounded diagnostic guidance	Reduced cognitive load during incident response
CI/CD Pipeline	Pre-submission automated context enrichment	Dependency impact and ownership checks	Upstream risk detection before human review
Onboarding Workflows	Domain-specific question answering	Codebase orientation without senior engineer dependency	Accelerated time-to-contribution for new engineers
Incident Re-sponse	Autonomous telemetry correlation	Structured hypothesis and recommendation at alert time	Compressed MTTD and MTTR across service teams

5 Lessons Learned: Context Engineering as Core Infrastructure

The most consequential operational insight from building and deploying these systems is that AI effectiveness is determined more by the quality of context engineering than by model selection. A well-structured RAG pipeline with disciplined MCP governance consistently outperforms a more capable model operating on unstructured, ad-hoc prompts. The research framework for evaluating RAG systems establishes that the primary determinants of end-to-end system quality are retrieval precision, whether the system retrieves the most contextually relevant documents, and grounding faith-fulness, whether the model's response accurately reflects the retrieved content rather than introducing unsupported claims [9]. Both of these quality dimensions are governed by the design of the retrieval corpus and the context boundaries, not by model architecture alone.

This finding has direct implications for how organizations should invest in AI infrastructure. Enterprises that treat MCP and RAG as first-class infrastructure components with defined ownership, versioned schemas, monitored retrieval performance, and regular corpus maintenance build compounding reliability advantages over those that deploy AI tooling as point solutions without governance scaffolding. Hugging Face's LightEval project provides a small infrastructure for evaluating the performance of a language model along various task dimensions, allowing teams to continuously monitor whether the systems they deploy meet the characteristics of AI system quality required to suit them for production deployment [10]. For developer assistance applications, this means creating evaluation pipelines to monitor retrieval relevance, response faithfulness, and task completion rates and so treating AI system quality characteristics as an observably maintainable property.

Agent governance, observability, and human-in-the-loop verification are needed for sustained reliability, not optional add-ons. Research on foundational work on the transformer attention mechanism establishes that the capacity of these models to at-tend selectively to relevant context is a core architectural strength but that this capacity is fully realized only when the context presented to the model is well-structured, authoritative, and free of noise [8]. In enterprise deployment, this means that the quality of the MCP-governed context layer directly conditions the quality of every down-stream agent action. Organizations that invest in context quality through schema governance, ownership metadata maintenance, and retrieval pipeline monitoring see this investment reflected in the reliability and trustworthiness of their AI-assisted engineering workflows [8] (Table 4).

Table 4. Lessons learned: Context Engineering as core infrastructure [9, 10]

Investment Area	Governing Principle	Quality Indicator	Long-TermOut-come
Retrieval Pipeline Design	Precision and corpus freshness over model size	Retrieval relevance scores via continuous evaluation	Compounding response accuracy as the corpus matures

(*continued*)

Table 4. (*continued*)

Investment Area	Governing Principle	Quality Indicator	Long-TermOut-come
MCP Sche-ma Governance	Context boundaries as infrastructure, not application config	Schema versioning and access audit trail completeness	Sustained hallucination reduction in production
AgentObservability	AI system quality as continuously monitored property	Task completion rate and faithfulness metrics (LightEval)	Maintainable and trustworthy production agent behavior
Human-in-the-Loop Verification	Governance is a prerequisite, not an optional addition	Review coverage of high-risk agent-initiated actions	Long-term trust and adoption across engineering teams

5.1 Limitations, Security Considerations, and Deployment Trade-offs

Despite the architectural and operational advantages described above, organizations deploying integrated RAG-MCP-agentic platforms face a set of non-trivial limitations and risks that must be explicitly managed.

Retrieval corpus quality dependency. The reliability of RAG-grounded outputs is directly bounded by corpus freshness and coverage. Organizations with inconsistent documentation practices, stale runbooks, or incomplete service ownership registries will experience degraded retrieval precision and, consequently, degraded response quality regardless of model capability. Corpus maintenance must be treated as a continuous operational discipline rather than a one-time onboarding task.

Security and data exposure risks. MCP-governed context access introduces a well-defined attack surface: an adversarial prompt that successfully manipulates the con-text retrieval layer can expose service ownership metadata, API schemas, or incident history to unauthorized parties. Production deployments must enforce schema-level access controls, audit logging of context retrieval events, and rate limiting on agent-initiated tool invocations. F5's infrastructure governance analysis identifies prompt injection and context boundary violations as the primary security threats to enterprise agentic systems [3].

Autonomous agent reliability and human oversight. ReAct-style agents operating in multi-hop diagnostic workflows can compound errors across reasoning steps, particularly when early retrieved context is low-quality or ambiguous. Human-in-the-loop verification checkpoints are not optional for high-risk actions — automated rollbacks, configuration changes, or alert suppression decisions must remain subject to human approval in production environments. Organizations that remove these checkpoints in the interest of speed consistently encounter trust failures that impede adoption.

Integration complexity and operational overhead. Deploying this architecture requires coordinated investment across retrieval infrastructure, schema governance, agent observability, and evaluation pipelines. Organizations without existing platform

engineering capability or SRE discipline will find the operational overhead of maintaining these systems materially higher than anticipated. Phased deployment — beginning with read-only RAG assistants before introducing agentic autonomy — significantly reduces initial risk exposure.

Conclusion

Generative AI, Retrieval-Augmented Generation, Model Context Protocol, and Agentic AI workflows, when architected as integrated platform infrastructure, represent a definitive shift in how engineering organizations manage complexity at scale. Together, they move developer tooling beyond passive information access toward active reasoning participation in engineering workflows — reducing context acquisition time by 40–60%, compressing MTTR by 25–35%, and making institutional knowledge systematically available at the point of need rather than residing in individual engineers. Deployment within IDEs and CI/CD pipelines demonstrates that the highest-impact contribution of this architecture is not speed alone but a structural change in how engineers engage with complex systems: replacing fragmented, manual information gathering with intent-driven queries grounded in authoritative enterprise con-text. The evidence from site reliability engineering practice, retrieval system frame-works, agentic orchestration research, and enterprise infrastructure governance consistently identifies context engineering, disciplined retrieval pipelines, governed con-text protocols, and continuously evaluated agent behavior as the determinants of long-term system reliability. As agent autonomy expands and integration with incident response and change management deepens, the organizations that have established strong contextual infrastructure will hold a durable and compounding advantage in engineering velocity, system reliability, and the capacity to scale intelligent automation without sacrificing the governance and trust that production engineering demands.

References

1. Murphy, N.R., et al.: Site reliability engineering. O'Reilly (2016). [Online]. Available: https://www.oreilly.com/library/view/site-reliability-engineering/9781491929117/
2. Sareen, S.,: Platform engineering—redefining the developer experience. Medium (2024). [Online]. Available: https://medium.com/@sonal.sareen/platform-engineering-redefining-the-developer-experience-748715cadf89
3. F5 Networks: Overcoming AI Infrastructure Challenges: Balancing Power, Latency, Reliability and Data Requirements. F5. [Online]. Available: https://www.f5.com/go/white-paper/overcoming-ai-infrastructure-challenges
4. IBM: Infrastructure Monitoring. [Online]. Available: https://www.ibm.com/products/instana/infrastructure-monitoring
5. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive NLP Tasks. In: 34th Conference on Neural Information Processing Systems (NeurIPS) (2020). [Online]. Available: https://proceedings.neurips.cc/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf
6. Yao, S., et al.: ReAct: synergizing reasoning and acting in language models. Computation and Language (cs.CL); Artificial Intelligence (cs.AI); Machine Learning (cs.LG) (2023). [Online]. Available: https://arxiv.org/abs/2210.03629

7. Wu, Q., et al.: AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. Artificial Intelligence (cs.AI); Computation and Language (cs.CL) (2023). [Online]. Available: https://arxiv.org/abs/2308.08155
8. Vaswani, A., et al.: Attention Is All You Need. arxiv logo>cs>arXiv:1706.03762 (2023). [Online]. Available: https://arxiv.org/abs/1706.03762
9. Finardi, P., et al.: The Chronicles of RAG: The Retriever, the Chunk and the Genera-tor. arxiv logo > Cs > arXiv:2401.07883 (2024). [Online]. Available: https://arxiv.org/abs/2401.07883
10. Hugging Face: Lighteval. [Online]. Available: https://huggingface.co/docs/lighteval/en/ind

Human-Grounded AI-Assisted Software Engineering Education: An Empirical Study of Learning Outcomes

Junaid Shuja(✉) and Abir Saha

Department of Computer Science, Southeast Missouri State University, Cape Girardeau, MO, USA
jshuja@semo.edu

Abstract. Software engineering has been an integral part of the AI revolution as several clients facing tools are being developed leveraging the power of AI agents and large language models. More software developers and practitioners are utilizing AI tools to write code, draw system models, generate test cases, and specify software requirements. As a result, the teaching of software engineering in higher education needs to evolve and integrate the AI tools in the classroom. Software engineering students are taught the life-cycle of software development which starts at project initiation and leads towards phases of requirements engineering, software modeling, design, architecture, testing, and deployment. Each of these phases embeds AI in practice. Therefore, it is necessary that the same AI tools are integrated in teaching and learning software engineering to educate market-ready graduates. We introduced a Human-Grounded AI-Assisted approach to software engineering education. We conducted an empirical study to answer the following research questions. RQ1: Did students achieve better course learning outcomes because of AI-integration? RQ2: How did human-generated requirements, UML diagrams, and test cases compare with AI-generated artifacts in terms of correctness. We integrated AI tools across two semesters of a software engineering course, requiring students to first produce UML and design artifacts manually before recreating them using AI tools. Submissions from 13 groups in Fall and 18 groups in Spring semester were evaluated using instructor rubrics and Claude-based UML validation. The empirical analysis shows that the Human-Grounded AI-Assisted approach resulted in better learning outcomes. Moreover, AI-generated artifacts consistently outperformed human-generated artifacts in technical correctness across both semesters.

Keywords: software engineering · AI · software artifacts · vibe coding

1 Introduction

Generative AI is disrupting every industry including software engineering and higher education. As AI is applied at jobs, it is necessary that graduates from

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 487–503, 2027.
https://doi.org/10.1007/978-3-032-39763-8_28

higher education learn the foundational AI skills. Software engineering is one of the tasks vastly automated by AI through code generation. Therefore, there is high emphasis on graduates to not just learn traditional software engineering concepts and practices, but also integrate AI to automate the tasks [2].

The image generation capabilities of generative AI have exponentially grown over the last few years [12]. As a result, generative AI is also proficient at modeling and analysis of software systems. Although the most popular use of generative AI in the software engineering process model is code generation, in practice it is embedded into every phase. Generative AI can help understand the business case, refine user stories, and identify conflict in the requirements specifications [7,8]. In the modeling and analysis phase, generative AI can produce UML diagrams, select design and architecture patterns [5,15]. Generative AI tools such as ChatGPT, and Claude have dedicated features for code generation [13]. Researcher have investigated the ability of LLMs to produce test case scenarios that are aligned with the underlying requirements [1,4].

Vibe coding is getting popular in practice. However, practitioners understand that AI output often has bugs, fails to understand key specifications, and often cannot be trusted for design and architecture decisions [9,11]. As a result, the new generation of software engineering practitioners still needs to develop fundamental concepts and practices of software engineering. A Human-Grounded AI-Assisted sequencing should anchor conceptual understanding in practice before repeating the same tasks with AI assistance leading to the same level of cognitive engagement [10,14].

This study investigates the effects of an AI-integrated software engineering curriculum practiced over two semester in higher education. The software analysis and design assignments had instructions and education to use AI tools after manual human-first execution. The submissions were analyzed on a rubric for quality and correctness. AI tools were used to compare human with AI scores and judgment. The main purpose of the empirical analysis was to answer the research questions: RQ1: Did students achieve better course learning outcomes because of AI-integration? RQ2: How did human-generated requirements, UML diagrams, test cases compare with AI-generated artifacts in terms of correctness.

The contributions of this research are as follows.

- We propose an AI-integrated curriculum for software engineering to enable students to use a Human-Grounded AI-Assisted approach.
- We analyze the student submissions to measure the impact of AI integration on student learning outcomes.
- We compare the technical correctness of human-generated and AI-generated software engineering artifacts using instructor rubrics and a Claude-based UML validation method.

The rest of the article is structured as follows. Section 2 presents the research methodology, curriculum adoption, assessments, and rubric. Section 3 details results of human and AI grading along with the discussion. Section 4 provides the concluding remarks.

2 Research Methodology

This section describes the methodological framework used to design, administer, grade, and compare student performance on Assignment 7 (System Models) and Assignment 8 (Design) across two semesters: Fall and Spring. The methodology encompasses four interconnected components: (1) Human-Grounded AI-Assisted curriculum adoption (2) the assessment instruments and rubrics developed for each assignment, (3) the comparison parameters used to evaluate human-generated versus AI-generated artifacts, and (4) the grading procedure, including the prompts supplied to Claude (Anthropic) as an AI-assisted evaluation tool.

2.1 Curriculum Adoption

Curriculum changes are necessary in the face of the AI revolution, especially for computer science course. We supplemented the conventional software engineering curriculum with prompt engineering and AI tool integration. The focus was a Human-Grounded AI-Assisted curriculum equipped with the understanding of how AI tools work to reduce the problem of garbage in, garbage out [3]. Each phase of the software development life cycle such as requirements specifications, system design, and code generation was aided by AI tools and prompting techniques [6,16]. Students chose a project to apply the SDLC phases and develop software artifacts in groups of maximum three. The projects included a weather app, a study help, job boards, mobile banking, etc. There were 13 groups in the fall semester and 18 in the spring semesters. Some groups did not submit assignments or did not follow instructions for AI. Such submissions were not used in the empirical analysis. Moreover, students chose agile methods to practice stand-ups and pair programming.

The software engineering (CS445) is a mandatory course for computer science, cybersecurity and information systems majors at Southeast Missouri State University. It has the following learning outcomes.

- Students will be able to examine the paradigms and methodologies used for overcoming the challenges in developing quality software products.
- Students will be able to apply systems analysis heuristics to prepare requirements specifications.
- Students will be able to apply system design techniques to prepare design specifications that include system architecture and user interface, database, and class designs.

We focused on the last learning outcome to simplify the empirical analysis.

2.2 Assessment and Rubric

Assessment Dataset. The dataset consisted of student submissions collected from the Fall semester and the Spring semester. In Spring, students additionally learned prompt engineering techniques such as zero-shot prompting, few-shot prompting, and chain-of-thought prompting [17,18]. Students were required to:

- Create diagrams manually without AI assistance.
- Recreate the same diagrams using AI tools.
- Document prompts and prompt refinement processes.
- Compare human-generated and AI-generated outputs.

The assignment requirements and course learning outcomes were then used to create a detailed rubric. The rubrics for the fall and spring assignments were different as prompt engineering concepts were taught for the first time in the spring assignment.

Assignment 7: System Model. In assignment number 7, the evaluation of manual system model creation and AI-assisted modeling was conducted across two semesters. The significant distinction between the two semesters was the introduction of prompt engineering in spring, incorporating zero-shot, few-shot, and chain-of-thought techniques. Consequently, the grading rubric was updated for the spring term to include new criteria focused on prompt documentation, the variety of prompting methods employed, and reflections on prompt engineering. The scope of the data collection was limited to Task 1 (Human Generated Diagrams) and Task 2 (AI Generated Diagrams), utilizing rubric-based criteria such as technical correctness, completeness, clarity, diagram correspondence, the quality of generated prompts, and the reproducibility of the resulting diagrams.

In the fall semester, Task 1 (Human-Generated Diagrams) was graded out of 30 score, and Task 2 (AI-Generated Diagrams) was graded out of 26 score. In the spring semester, the rubric was updated to incorporate prompt engineering concepts. Task 1 was graded out of 20 score, while Task 2 was graded out of 25 score. The Human-Generated and AI-generated diagram evaluation focused on the following criteria in Table 1.

Assignment 8: System Design. The 8th assignment expanded the range of modeling artifacts by adding "full-stack" design artifacts: class/object models, behavior diagrams, database / ER schema designs, and user interface specs (mockups for UI or API). All groups in Spring finished this assignment using an established rubric that also had a bonus category to reward innovative prompt engineering strategies and visual design tool (code-based) methods, such as PlantUML and Mermaid.

Table 2 shows the rubrics for task-1 for the human-generated diagram, outlining 5 evaluation criteria worth a combined total of 40 score, with 8 score each.

Table 3 shows the rubrics for the task-2 for AI-generated diagram, outlining six evaluation criteria worth a combined total of 30 score.

2.3 Comparison Parameters

To compare human-generated and AI-generated diagrams, several statistical and qualitative parameters were used such as mean, median, mode, and highest score.

Table 1. Human-Generated and AI-Generated Diagram Evaluation Criteria for Spring semester

Criterion	Human Diagram (Spring)	AI Diagram (Spring)	Human Diagram (Fall)	AI Diagram (Fall)
Diagram selection and variety	4	N/A	6	8
Technical correctness	8	7	12	10
Completeness and detail	4	N/A	7	N/A
Clarity and presentation	4	N/A	5	N/A
Correspondence to human diagrams	N/A	6	N/A	N/A
Prompt documentation quality	N/A	6	N/A	N/A
Variety of prompting techniques	N/A	6	N/A	N/A
AI diagram variety	N/A	N/A	N/A	8
Human Diagram Total	**20**		30	
AI Diagram Total		**25**		26
Overall Total	**45**		56	

Table 2. Task 1- Evaluation Rubric

Evaluation Criteria	Max Score
Class diagram/object model	8
Sequence/activity/state diagrams	8
Database / ER schema design	8
Interface design (UI mockup or API spec)	8
Overall UML correctness & diagram quality	8
Task 1 Subtotal	**40**

Table 3. Task 2 Evaluation Rubric

Evaluation Criteria	Max Score
Diagram coverage (same $\geq$4 types as Task 1)	4
Zero-shot prompt - quality and documentation	6
Few-shot prompt - examples and AI output quality	6
Chain-of-thought - reasoning steps visible	6
Prompt refinement and iteration	4
Tool named, exact prompts, reproducibility	4
Task 2 Subtotal	**30**

These statistics were calculated separately for human and AI-generated diagrams for both semesters. The comparison enabled the evaluation of technical correctness, quality improvement, and performance variation among student groups. The Spring semester technical correctness scores were graded out of 7 and the fall semester technical correctness scores were graded out of 10. Therefore, all scores were converted into percentages before performing statistical comparisons.

2.4 Evaluation Methods

We used two methods for evaluation, i.e., human only, and a comparison of human versus AI evaluation. In the human only evaluation, we focused on how students finished their tasks and how well they understood the purpose of the tasks. The evaluation involved two tasks: Task 1 focused on human-completed diagrams, while Task 2 assessed AI-generated diagrams and the associated prompting techniques. An evaluator manually rated these tasks without AI assistance, covering categories such as diagram quantity, technical quality, completeness, prompting quality, reproducibility, and improvement strategies.

The second method involved evaluating the quality and accuracy of the produced diagrams by comparing the two independent evaluations (teaching assistant and Claude) to UML 2 standards. In Assignments 7 and 8, Claude applied a structured prompt to assess both human-generated and AI-generated UML diagrams against the UML standards.

Evaluation Prompt for Assignment 07. You are a UML and software design standards expert. You will be given 6–8 design artifacts from a single group, where the first 3–4 are human-generated and the last 3–4 are AI-generated. Your task is to validate each artifact against the official UML 2.x standards and standard software design conventions. Count correct artifacts separately for the human and AI sets, and summarize everything in a single row per group.

Validation Rules

- **UML Class Diagram:** Correct class notation with visibility modifiers (+, –, #), typed attributes and methods, proper relationship arrows, and correct multiplicity notation.
- **UML Sequence Diagram:** Proper lifelines, activation bars present and correctly closed, correct message arrow types, guard conditions in brackets where applicable, and proper combined fragments if used.
- **UML Activity Diagram:** Correct initial and final node notation, decision nodes as diamonds with labeled guard conditions, merge/fork/join bars where needed, and no mixing of flowchart symbols.
- **UML State Machine Diagram:** Proper state boxes, labeled transitions with event/guard/action syntax, correct initial pseudostate, final state notation, and no flowchart elements mixed in.

- **Database Schema / ER Diagram:** Not a UML diagram type. Flag as non-UML if submitted as a standalone diagram. Only validate as UML if persistence stereotypes (e.g., `<<entity>>`, `<<table>>`) are present. Otherwise, mark it as non-compliant and exclude it from the counts.
- **General:** No mixing of notation across diagram types. Relationships must use correct UML arrowheads and line styles. Labels and names must follow UML 2.x conventions.

Column Rules

- **Diagram Names:** Human artifacts first, followed by AI artifacts, separated by `|`.
- **Correct Count:** Human: X of N | AI: X of N (exclude unframed ER diagrams).
- **Description of Errors:** One line per incorrect artifact only, using the format `[Name] -- [reason]`.
- **Human Score:** Leave blank.

Scoring Guidance

Score	Meaning
9–10	All artifacts are correct or only have trivial cosmetic issues.
7–8	One minor but real UML violation.
5–6	Multiple violations or one with several structural issues.
3–4	Multiple significant violations of core UML notation.
1–2	Most artifacts are structurally incorrect or not valid UML.

Important

- Do not include correct artifacts in the error column.
- Minor issues correspond to scores greater than 90%; structural violations correspond to scores of 50% or below.
- State assumptions for ambiguous diagrams.
- Exclude UI mockups, API specifications, and unframed ER diagrams from the counts.
- Output only the summary table.

UML Diagram Validation Prompt for Assignment 08. We used a similar prompt for assignment 8. The two prompts differ mainly in diagram coverage: Assignment 07 focuses on four types (Class, Sequence, Activity, State Machine) with detailed rules and a special ER/database exclusion, while Assignment 08 broadens coverage to include Use Case and Component diagrams but with more general rules and no ER-specific guidance.

3 Results and Discussion

3.1 Grades and Correctness

Table 4. Statistical Comparison Between Human-Generated and AI-Generated Diagram Scores For Spring

Statistical Measure	Human-Generated (max 20)	Human (%)	AI-Generated (max 25)	AI (%)
Mean	13.86	69.30%	21.21	84.84%
Median	13	65.00%	20	80.00%
Highest Score	19 (G3)	95.00%	24 (G3)	96.00%
Lowest Score	7 (G11)	35.00%	13 (G8)	52.00%
SD	3.06		3.19	

Assignment 07. Table 4 shows the statistical comparison. AI-generated diagrams outperformed human-generated diagrams across the board. On average, the scores of human-generated diagrams were 69.30% compared to 84.84% for AI-generated diagrams, indicating a consistent improvement in quality and accuracy due to AI's assistance. While both types of diagrams had very high individual scores, the AI-generated diagrams exceeded human-generated diagrams in terms of minimum score (52% for AI and 35% for humans). Therefore, it appears that the implementation of AI-assisted methods and prompts improves software engineering diagrams' quality, technical correctness, and consistency. In addition, we analyzed the Fall semester as a cross-validation for the Spring semester. The paired t-test showed a statistically significant mean improvement of 9.86 percentage points ($t(13) = 3.389$, $p = 0.0048$), indicating that the AI-generated tasks significantly outperformed the human-generated tasks across the student groups.

Table 5. Statistical Comparison Between Human-Generated and AI-Generated Diagram Scores for the Fall Semester

Statistical Measure	Human-Generated (max 30)	Human (%)	AI-Generated (max 26)	AI (%)
Mean	22.69	75.63%	20.54	79.00%
Median	23	76.67%	22	84.62%
Highest Score	26.5 (G12)	88.33%	24	92.31%
Lowest Score	19 (G8)	63.33%	13	50.00%
SD	2.19		2.73	

The statistical analysis of human-generated and AI-generated artifacts for the Fall semester shows in Table 5 that human-generated artifacts overall, performed

slightly worse than their AI-generated artifacts. The average scores increased from 75.63% to 79.00% when AI was used, indicating that different groups created better results with the assistance of various AI tools. The higher standard deviation for AI-generated diagrams suggests that AI amplifies individual differences. Students who know how to prompt well get much better results, while students who prompt poorly get worse results. A higher standard deviation for AI-generated artifacts also reflects the flexibility of instruction to use different prompting techniques. However, the paired t-test did not show a statistically significant difference between the human and AI-assisted scores ($t(12) = 1.852$, $p = 0.091$). Additionally, the MannWhitney U test was not significant ($U = 55$, $p = 0.135$).

Analysis of the Fall and Spring Assignment 7 shows good student performance on System Modeling CLOs when creating UML Diagrams for both human and AI-generated artifacts. Students created multiple software design model types (i.e., Class / Use Case / Sequence / Activity / State Diagrams), which demonstrated how the systems were structured, behaved, and interacted. The comparison between the Fall and Spring results showed that AI-assisted modeling typically provided increased accuracy of technical design representations for UML Diagrams. In most cases, AI-generated models scored higher than human-generated models.

In regard to RQ2, the findings showed that, when compared to artifacts created by humans, those created by AI were superior in terms of correctness. Students using an AI-assisted method were able to develop more complete and technically accurate outputs with less effort than outputs produced using non-AI methods, allowing them to spend significantly less time creating initial design specifications. The results of this study also indicate that the success of AI-assisted artifact creation is reliant upon the student's ability to review, improve, and verify the accuracy of the AI-generated artifacts. Thus, AI can produce higher productivity and quality of artifacts. However, basic knowledge of software engineering will always remain necessary for achieving the highest level of quality and utilizing AI-generated artifacts.

Table 6 compares system modeling artifacts developed by humans with those generated by AI across different student groups. These evaluations centered on design quality, clarity, completeness, and the technical accuracy of the UML diagrams. All statistical assessments were based on manually assigned scores.

The findings presented in Table 7 indicate that AI-generated artifacts outperformed human-generated ones. Specifically, AI-generated UML diagrams achieved an average score of 71.43%, whereas human-generated diagrams averaged 59.29%. Although both approaches shared a minimum score of 30%, only the AI-generated artifacts reached a maximum score of 100%. These results suggest that AI-assisted modeling enables students to develop software engineering models that are more technically precise, clear, and comprehensive.

Table 8 provides a quantitative comparison of results from Claude generated evaluations of modeling artifacts created by humans and by artificial intelligence. These results show that AI-generated diagrams achieved significantly greater

Table 6. Evaluation Summary of Group Submissions for assignment 07

Group	Artifacts	Correct Diagrams	Human Evaluation Score	Claude Evaluation Score
G1	Use Case Diagram, Sequence Diagram, Class Diagram	H: 2 out of 3 AI: 2 out of 3	Human: 6/10 AI: 7/10	Human:8/10 AI:7/10
G2	Use Case Diagram, Sequence Diagram, Class Diagram	H: 1 out of 3 AI: 2 out of 3	Human: 5/10 AI:7/10	Human:4/10 AI:6/10
G3	Class Diagram, Sequence Diagram, Use Case Diagram	H: 2 out of 3 AI: 2 out of 3	Human:6/10 AI:8/10	Human:7/10 AI: 7/10
G4	Sequence Diagram (Fig. A), Class Diagram (Fig. B), Activity Diagram (Fig. C)	H: 3 out of 3 AI: 3 out of 3	Human:8/10 AI: 8/10	Human:9/10 AI: 9/10
G5	State Diagram, Sequence Diagram, Activity Diagram	H: 1 out of 3 AI: 3 out of 3	Human:5/10 AI: 10/10	Human: 5/10 AI: 9/10
G6	Use Case Diagram, Sequence Diagram, Class Diagram	H: 0 out of 3 AI: 0 out of 3	Human: 3/10 AI : 2/10	Human: 4/10 AI : 5/10
G7	Use Case Diagram, Activity Diagram, Class Diagram	H: 1 out of 3 AI: 2 out of 3	Human: 6/10 AI: 7/10	Human: 5/10 AI:7/10
G8	Data Flow Diagram (Fig. 4), Sequence Diagram (Fig. 5), Activity Diagram (Fig. 6)	H: 1 out of 3 AI: 2 out of 3	Human: 4/10 AI: 6/10	Human:4/10 AI: 5/10
G9	Human: Use Case Diagram, Sequence Diagram, Class Diagram	H: 2 out of 3 AI: 2 out of 3	Human: 7/10 AI: 9/10	Human:7/10 AI: 8/10
G10	Use Case Diagram (Zero-Shot), Class Diagram (Few-Shot), Activity Diagram (Chain-of-Thought)	H: 2 out of 3 AI: 2 out of 3	Human:7/10 AI: 7/10	Human:7/10 AI : 9/10
G11	Activity Diagram (Zero-Shot), Data Flow Diagram (K-Shot), Use Case Diagram (Chain-of-Thought)	H: 1 out of 3 AI: 2 out of 3	Human:5/10 AI:4/10	Human:4/10 AI:6/10
G12	DFD (ChatGPT/K-Shot), Activity Diagram (Copilot/Chain-of-Thought), Use Case Diagram (Claude/Zero-Shot)	H: 2 out of 3 AI: 2 out of 3	Human: 6/10 AI: 6/10	Human:6/10 AI: 6/10
G13	Use Case Diagram (Zero-Shot), Class Diagram (Few-Shot), Sequence Diagram (Chain-of-Thought)	H: 2 out of 3 AI: 3 out of 3	Human: 7/10 AI: 9/10	Human: 6/10 AI:8/10
G14	Use Case Diagram, Sequence Diagram, Class Diagram	Human: 2 out of 3 AI: 3 out of 3	Human: 7/10 AI: 9/10	Human:8/10 AI: 9/10
G15	Use Case Diagram, Class Diagram, Sequence Diagram	Human: 2 out of 3 AI: 3 out of 3	Human:8.0 AI: 8.0	Human:8.5 AI: 8.5

total value than those made by humans. AI-generated diagrams received an average of 73%, while human-generated diagrams averaged 61.67%. The highest-scoring artifacts for each group received the same total score of 9 out of 10. The standard deviation in total scores was less than that of human-generated artifacts, indicating less variability of performance across groups or individuals. Overall results from the Claude-based evaluation further support evidence of AI-assisted modeling's impact on improving the quality and technical correctness of software engineering systems' models. Moreover, the median for AI-generated artifacts was higher than for human-generated artifacts.

The overall evaluation results indicate that the UML artifacts created by the AI-assisted system modeling are, in general, of better quality than the human-generated models. The AI-generated diagrams were graded better in terms of technical correctness, completeness, consistency, and presentation quality in most of the student groups. The mean score of the manual assignment assessment was 71.43% for the AI-generated diagrams and 59.29% for the human-generated diagrams. Additional assessments with Claude demonstrated that AI-generated models scored an average of 73%, versus human-generated models at an average

Table 7. Statistical Comparison Between Human-Generated and AI-Generated Evaluations

Statistical Measure	Human-Generated (max 10)	Human (%)	AI-Generated (max 10)	AI (%)
Mean	5.93	59.29%	7.14	71.43%
Median	6	60.00%	7	70.00%
Highest Score	8	80.00%	10	100.00%
Lowest Score	3	30.00%	3	30.00%
SD	1.44		1.37	

Table 8. Claude Evaluation Statistical Comparison Between Human-Generated and AI-Generated Diagrams

Statistical Measure	Human-Generated (max 10)	Human (%)	AI-Generated (max 10)	AI (%)
Mean / Average	6.17	61.67%	7.30	73.00%
Median	6	60.00%	7	70.00%
Highest Score	9	90.00%	9	90.00%
Lowest Score	4	40.00%	5	50.00%
SD	1.71		1.15	

of 61.67%. AI-generated artifacts also achieved perfect or near-perfect scores in a number of groups, showing strong consistency in following UML standards and software design principles. The paired t-test showed that AI-generated diagrams achieved significantly higher scores than human-generated diagrams in both the Human Evaluation ($t(14) = 2.828$, $p = 0.013$) and Claude Evaluation ($t(14) = 3.523$, $p = 0.003$).

Table 9. Statistical Summary of the Fall Semester Assignment 8 Scores

Statistical Measure	Human Generated Score (max 40)	Percentage
Mean / Average	21.64	54.09%
Median	23.00	57.50%
Highest Score	32.00	80.00%
Lowest Score	8.00	20.00%
SD	6.91	

Assignment 08. In the fall semester, software engineering design assignment did not introduce any AI tools. Table 9 shows a mean score of 54.09%, indicating moderate overall performance. The highest-performing group achieved 80.00%, while the lowest-performing group scored 20.00%, suggesting considerable vari-

ation in students' ability to produce correct and complete software engineering artifacts without AI assistance.

Table 10. Statistical Summary of Assignment 8 Task 1 Human-Generated Artifacts for Spring

Statistical Measure	Human-Generated Score(max 40)	Human (%)
Mean	31.31	78.28%
Median	31.5	78.75%
Highest Score	38.5 (G15)	96.25%
Lowest Score	26.5 (G2)	66.25%
SD	3.27	

The statistical analysis for Assignment 8 Task 1 in spring, which is based on human-generated software engineering design artifacts, is presented in Table 10. The results of data analyses suggest that there was significant overall success from all groups involved in this study. The mean score across all cohorts of humans in this study for their produced artifacts was 78.28%, indicating very similar outputs across all cohorts. The performance ranged from a maximum of 96.25% and a minimum of 66.25%. Thus, it can be determined that the students were able to create comprehensive, technically accurate software engineering design artifacts, which accomplished the established course learning objectives. As compared to the fall semester, the mean score increased from 54.09% to 78.28%. This analysis helps answer RQ1, i.e., students did achieve better course learning outcomes because of Human-Grounded AI-Assisted approach adopted in Spring.

Table 11. Statistical Comparison Between Human-Generated and AI-Generated Artifacts for Assignment 8

Statistical Measure	Human-Generated (max 40)	Human (%)	AI-Generated (max 30)	AI (%)
Mean / Average	31.31	78.28%	27.09	90.31%
Median	31.5	78.75%	28	93.33%
Highest Score	38.5 (G15)	96.25%	30 (G12)	100.00%
Lowest Score	26.5 (G2)	66.25%	21 (G2 & G8)	70.00%
SD	2.72		2.81	

Table 11 presents the statistical comparison of the artifacts produced by humans and AI. Overall, the models created by the AI performed better with an average score of 90.31%, versus 78.28% for the human-generated artifacts. In general, the results indicate that prompt engineering and AI-assisted modeling improved the quality, completeness, and technical correctness of software

engineering design artifacts. This analysis helps answer RQ2, i.e., the students did achieve better learning outcomes using AI tools. The paired t-test showed that AI-generated diagrams achieved significantly higher scores than human-generated diagrams in Task 2, where $t(15) = 4.268$, $p = 0.0007$, with the Wilcoxon signed-rank test confirming this result ($z = -3.001$, $p = 0.0027$).

Table 12. Assignment 8 Group-wise Evaluation Summary

Group	Diagram Names	Correctness	Human Score (max 10)	Claude Score (max 10)
G1	Class Diagram, Sequence Diagram, Activity Diagram, Database Schema	Human: 1 out of 4 AI: 1 out of 4	Human: 3/10 AI: 4/10	Human: 4/10 AI: 4/10
G2	Class Diagram, Sequence Diagram, State Diagram (COT)	Human: 1 out of 3 AI: 3 out of 3	Human: 3/10 AI: 8/10	Human: 4/10 AI: 9/10
G3	Class Diagram, Sequence Diagram, Activity Diagram	Human: 0 out of 3 AI: 3 out of 3	Human: 2/10 AI: 8/10	Human: 4/10 AI: 9/10
G4	Class Diagram, Sequence Diagram, Activity Diagram, Physical ER Diagram	Human: 2 out of 4 AI: 0 out of 4	Human: 5/10 AI: 3/10	Human: 7/10 AI: 2/10
G5	Human: DB Schema (ERD), Sequence, Activity, Class Diagram	Human: 2 out of 4 AI: 2 out of 4	Human: 5/10 AI: 5/10	Human: 5/10 AI: 5/10
G6	Sequence Diagram, State Diagram, Activity Diagram	Human: 1 out of 3 AI: 2 out of 3	Human: 3/10 AI: 7/10	Human: 4/10 AI: 7/10
G7	Class, Sequence, Activity	Human: 1 out of 3 AI: 1 out of 3	Human: 3/10 AI: 3/10	Human: 4/10 AI: 4/10
G8	Class Diagram, Activity Diagram, Sequence Diagram, ER Diagram	Human: 1 out of 4 AI: 3 out of 4	Human: 4/10 AI: 7/10	Human: 4/10 AI: 7/10
G9	Class Diagram, Sequence Diagrams, Activity Diagrams	Human: 2 out of 5 AI: 2 out of 5	Human: 4/10 AI: 4/10	Human: 4/10 AI: 5/10
G10	Class Diagram, Sequence Diagram, Activity Diagram	Human: 1 out of 3 AI: 2 out of 3	Human: 4/10 AI: 8/10	Human: 3/10 AI: 7/10
G11	Class Diagram, Sequence Diagram, Activity Diagram	Human: 1 out of 3 AI: 2 out of 3	Human: 4/10 AI: 7/10	Human: 3/10 AI: 6/10
G12	Class Diagram, Sequence Diagram, Activity Diagram, ER Diagram	Human: 3 out of 4 AI: 2 out of 4	Human: 8/10 AI: 5/10	Human: 7/10 AI: 4/10
G13	Class Diagram, Sequence Diagram	Human: 1 out of 3 AI: 1 out of 2	Human: 3/10 AI: 5/10	Human: 3/10 AI: 5/10
G14	Class Diagram, Sequence Diagram, Activity Diagram	Human: 0 out of 3 AI: 3 out of 6	Human: 2/10 AI: 5/10	Human: 2/10 AI: 6/10
G15	Class Diagram, Sequence Diagram, Activity Diagram	Human: 0 out of 3 AI: 1 out of 3	Human: 2/10 AI: 4/10	Human: 3/10 AI: 5/10

The evaluation results displayed in Table 12 provide a comparison of human-generated and AI-generated software engineering diagrams, and how much those diagrams may differ from one group of students to another. The focus of the evaluation was on how correct, complete, and of overall design quality the UML and database-related artifacts were that were created using human versus AI techniques. The data show that many of these student groups received higher scores from their evaluation of the AI-generated diagrams as opposed to their human-generated diagrams, particularly when they used various prompt-engineering techniques such as zero-shot, few-shot, and chain-of-thought prompting.

The evaluation for AI Versus Human discipline is represented in Table 13. The average evaluation score of the AI-generated artifacts of 55.33% was higher

Table 13. Statistical Comparison Between Human-Generated and AI-Generated Evaluations

Statistical Measure	Human-Generated (max 10)	Human (%)	AI-Generated (max 10)	AI (%)
Mean	3.67	36.67%	5.53	55.33%
Median	3	30.00%	5	50.00%
Highest Score	8	80.00%	8	80.00%
Lowest Score	2	20.00%	3	30.00%
SD	1.54		1.80	

than that of the human-generated artifacts 36.67%. Similarly, the median was higher for the AI-generated evaluations, showing that the AI-generated artifacts have a greater degree of parsimonious value to their technical quality than did the human-generated artifacts. The AI-generated artifacts achieved an equal maximum score of 8 (80%) as the human-generated artifacts. Overall, this information supports the conclusion that AI-assisted modeling increases the quality, accuracy, and consistency of software engineering systems' models in instructor-based evaluations. The paired t-test showed a statistically significant improvement in AI-generated diagram scores compared with human-generated diagrams ($t(14) = 2.855$, $p = 0.0126$). The MannWhitney U test also indicated a significant difference ($U = 46.5$, $p = 0.0056$). Together, these results provide consistent evidence that AI-generated diagrams outperformed human-generated diagrams in the instructor evaluation

Table 14. Statistical Comparison Between Human-Generated and AI-Generated Evaluations by Claude

Statistical Measure	Human-Generated (max 10)	Human (%)	AI-Generated (max 10)	AI (%)
Mean / Average	4.69	46.88%	5.63	56.25%
Median	4	40.00%	5	50.00%
Highest Score	7	70.00%	9	90.00%
Lowest Score	2	20.00%	2	20.00%

The results presented in Table 14 show a similar trend to the instructor evaluations, with AI-generated artifacts consistently receiving higher scores than human-generated artifacts. According to Claude's assessment, the AI-generated diagrams achieved a higher average score of 5.63 (56.25%) compared to 4.69 (46.88%) for the human-generated diagrams. The median score was also higher for the AI-generated artifacts (5 versus 4), indicating stronger overall technical quality. Furthermore, the AI-generated diagrams achieved a higher maximum score of 9 (90%) compared to 7 (70%) for the human-generated diagrams. These findings align with the instructor evaluation results and suggest that AI-assisted modeling can produce software engineering artifacts that are more accurate,

complete, and technically consistent than those created manually. The paired t-test indicated a statistically significant difference favoring AI-generated diagrams ($t(14) = 2.176, p = 0.0465$), while the MannWhitney U test showed a significant difference (U = 51, p = 0.0095).

3.2 Discussion

The results of Assignments 7 and 8 show that students achieved the learning outcome, and students were able to apply system design techniques to prepare design specifications that include system architecture, user interface, database, and class designs. In both semesters, students created a variety of software engineering artifacts, such as UML diagrams, database schemas, interface designs, requirements specifications, and testing-related artifacts. The consistently high scores in both human-generated and AI-generated submissions show that students were able to effectively apply core software design principles and modeling techniques.

The evaluation results show that students who integrated AI into their coursework achieved better course learning outcomes. AI-generated artifacts, in both the Fall and Spring semesters, consistently received higher average and median scores than human-generated artifacts across instructor and Claude-based evaluations. The improvement was particularly noted in technical correctness, completeness, consistency, and quality of presentation. The introduction of prompt engineering in Spring further increased the degree of improvement, as students learned to apply zero-shot, few-shot, and chain-of-thought techniques to produce more structured and reproducible outputs.

4 Conclusion

We introduced a Human-Grounded AI-Assisted curriculum for software engineering. The curriculum intervention teaches prompt engineering such that students produce software artifacts for their projects before using AI tools to compare their work against AI-generated output. We conducted an empirical analysis to determine if the Human-Grounded AI-Assisted curriculum leads to better learning outcomes. We found that the third learning outcome, i.e., students will be able to apply system design techniques to prepare design specifications, showed measurable improvement. The mean score for human-generated design artifacts improved from 75.63% in the fall semester to 84.84% in spring after prompt engineering was introduced. AI-generated artifacts scored higher than human-generated artifacts in both semesters, reaching 90.31% in Assignment 8, indicating that the Human-Grounded AI-Assisted curriculum enables students to leverage AI tools more effectively when they have first developed foundational knowledge independently.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

Note. This study analyzed course assignment submissions as part of normal educational practice. Student data was anonymized prior to analysis. Students were informed about anonymized empirical analysis on their submissions.

Acknowledgments. This study was funded by the Summer Research and Creative Activities Grant at Southeast Missouri State University.

References

1. Arora, C., Herda, T., Homm, V.: Generating test scenarios from NL requirements using retrieval-augmented LLMs: an industrial study. In: 2024 IEEE 32nd International Requirements Engineering Conference (RE), pp. 240–251. IEEE (2024)
2. Barenkamp, M., Rebstadt, J., Thomas, O.: Applications of AI in classical software engineering. AI Persp. **2**(1), 1 (2020)
3. Bissessar, K.: The adoption and impact of agentic AI in software engineering-a case study at Trimble (2026)
4. Chang, H.F., Shirazi, M., Cao, L., Mobasser, S.K.: Coding with AI: from a reflection on industrial practices to future computer science and software engineering education (2025). arXiv:2512.23982 arXiv preprint
5. Jahan, M., et al.: Automated derivation of UML sequence diagrams from user stories: unleashing the power of generative AI vs. a rule-based approach. In: Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems, pp. 138–148 (2024)
6. Kim, D.K., Hamdi, M.: Evaluating the effectiveness of AI integration in software engineering education. ACM Trans. Comput. Educ. **26**(4), 1–22 (2026)
7. Liu, C., Zhou, D.: AI integration in higher education: a software engineering perspective on readiness, anxiety and innovations. Int. J. Softw. Eng. Knowl. Eng. **36**(09), 1341–1369 (2026)
8. Marques, N., Silva, R.R., Bernardino, J.: Using ChatGPT in software requirements engineering: a comprehensive review. Fut. Int. **16**(6), 180 (2024)
9. Meske, C., Hermanns, T., Von der Weiden, E., Loser, K.U., Berger, T.: Vibe coding as a reconfiguration of intent mediation in software development: definition, implications, and research agenda. IEEE Access **13**, 213242–213259 (2025)
10. Mutiga, A.N.: AI integration in higher education: a content analysis on AI sophistication and student outcomes/skill development as reported in empirical studies (2019-2024). Ph.D. Thesis, University of Nevada, Reno (2024)
11. Namoun, A., et al.: Generative artificial intelligence in education: an umbrella review of applications and challenges (2024)
12. Nguyen-Duc, A., et al.: Generative artificial intelligence for software engineering–a research agenda. Softw.: Pract. Exper. **55**(11), 1806–1843 (2025)
13. Palla, D., Slaby, A.: Evaluation of generative AI models in python code generation: a comparative study. IEEE Access (2025)
14. Pimenova, V., Fakhoury, S., Bird, C., Storey, M.A., Endres, M.: Good vibrations? A qualitative study of co-creation, communication, flow, and trust in vibe coding (2025). arXiv:2509.12491 arXiv preprint
15. Shahariar Ayon, R., Ahmed, S.: AutoJML: generation and verification of JML specifications using LLM agents. IEEE Pulse **17**(1), 66–68 (2026). https://doi.org/10.1109/MPULS.2026.3659250

16. Syahputri, I.W., Budiardjo, E.K., Hadi Putra, P.O., Azhar, S., Imandeka, E.: Building effective prompts for LLMs in software engineering education: a component indicator approach. In: Proceedings of the 2026 9th International Conference on Software Engineering and Information Management, pp. 98–103 (2026)
17. Syahputri, I.W., Budiardjo, E.K., Putra, P.O.H.: Unlocking the potential of the prompt engineering paradigm in software engineering: a systematic literature review. AI **6**(9), 206 (2025)
18. Wang, G., et al.: Do advanced language models eliminate the need for prompt engineering in software engineering? ACM Trans. Softw. Eng. Meth. (2024)

An Empirical Investigation of Soft Skills and Corresponding Challenges in International Software Engineering Education

Sajid Ibrahim Hashmi(✉) and Jouni Markkula

M3S Research Group, Software Engineering and Information Systems Research Unit, University of Oulu, Oulu, Finland
{sajid.hashmi,jouni.markkula}@oulu.fi

Abstract. Joint software development work requires the application of technical skills and professional practices. Soft skills, such as teamwork, remain essential to succeeding in a professional software development environment. The phenomenon is reflected by group work in SE (software engineering) education, where students can learn and practice teamwork-related soft skills. Therefore, SE education requires students to practice those skills so that they can actively embark on their professional careers in the software industry.

In this research study, empirical findings from two focus groups we conducted on group work are presented, with emphasis on the soft skills necessary for teamwork in international software engineering education, specifically student collaboration in higher education. There are several soft skills students should learn as they practice teamwork in their course-related assignments and exercises, as well as the challenges that come with it in international settings. The findings serve as a guideline to university teachers and pedagogy experts to plan student group work more effectively.

Keywords: International SE education · group work · soft skills · teamwork · Focus Group 1(FG1) · Focus Group 2(FG2)

1 Introduction

In general, teamwork helps achieve tasks beyond the capacity of individuals, and teams tend to perform better for the same reason [1]. The joint work requires the application of technical skills and professional practices. Teamwork is an essential practice for succeeding in a professional software development environment. The SE team members must possess the necessary skills and required competencies.

SE is a teamwork activity in which software engineers collaborate as software is developed in a professional environment. Software engineers should be able to collaborate and interact effectively and proficiently with other software engineers, customers, and users to carry out development activities. SE is also an international discipline, as software companies and other organizations are often international and have multicultural personnel. Therefore, working in teams is not trivial, as it requires specific professional capabilities and skills from participating team members.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 504–523, 2027.
https://doi.org/10.1007/978-3-032-39763-8_29

Teamwork skills are highly relevant in industry environments [1], and students' participation in teams for group assignments can help them gain relevant experience [2]. Not only that, group work in SE education prepares students to collaborate with future colleagues, which becomes easier and more efficient when they can apply and consolidate the knowledge gained at the university [3]. That, in turn, prepares work-ready graduates by systematically requiring them to work on SE-related coursework.

The study investigates teamwork-related soft skills in international SE education and the relevant skills students can learn and practice through group work in course-related assignments. The findings explore the topic through the teachers' perspective, as they are responsible for guiding students. Therefore, the study's outcome should guide SE teachers in designing SE coursework not only to have students practice teamwork-related soft skills but also to help them better understand the phenomenon's underlying motivation.

To support the study, focus group methodology was chosen to obtain relevant information from experts, including SE university teachers. The international SE education and related aspects were set as the study context. The overall purpose is to study the skills students can learn through group work and better understand how teamwork training should be made an essential part of international SE coursework.

The rest of the paper is organized as follows. Section 2 reviews related research on international SE education and teamwork; Sect. 3 details the research methodology; Sect. 4 presents the empirical findings from the focus groups; and finally, Sects. 5 and 6 cover the discussion and conclusions, respectively.

2 Related Work

This section reviews the literature relevant to the study, providing theoretical and empirical background. Since the study fetches multiple concepts together, our review of the literature is multifaceted. It first reviews soft skills through some existing taxonomies; it then examines the growing importance of soft skills in the wake of the internationalization of higher education. Finally, it reflects on teamwork skills within international software engineering education, highlighting their significance.

2.1 Soft Skills and Existing Taxonomies

Soft skills are essential for young software developers to secure a position and begin their careers [4]. As with definitions of soft skills, their taxonomies also vary, and there is no consistent definition in the existing literature [5]. For instance, Garousi et al. [6] treat 'communication' as distinct from teamwork, although in general the practice is considered an essential part of successful teamwork. Furthermore, [6] considers soft skills to comprise team and interpersonal skills; Cinque [7], on the other hand, holds that soft skills comprise personal, social, and methodological skills. [8] has also categorized communication as a soft skill, independent of teamwork, which can complement the former. In practice, the existing literature offers no constraints on the use of non-teamwork soft skills in teamwork, suggesting that individual skills can also affect how a team performs as a whole.

A trait of the existing literature on competency and skills is that the terms competency and skills are used interchangeably, with no clear distinction drawn between the two concepts. For instance, Niva et al. [4, 11] and Assyne et al. [12] has grouped phenomenon such as teamwork, problem solving, communication, negotiation, and resilience, to name a few, as competencies formed by a combination of soft skills, and, on the other hand, the software engineering competency model [5, 6, 9, 13, 14] have listed the same phenomenon individually as skills. Therefore, in our work, we do not aim to create a disparity between "soft skills" and "competencies" and use both terms interchangeably.

2.2 Internationalization and Need for Soft Skills

With advances in communication methods and technologies, terms such as internationalization and globalization have also become common in the education sector. Although the concepts are linked, they can be interpreted in multiple ways depending on the context. In terms of higher education, the terms' globalization' and 'internationalization' can be defined as *"the flow of technology, economy, knowledge, people, values, ideas, across borders"* and *"the ways a country responds to the impact of globalization yet, at the same time respects the individuality of the nation"* [15] respectively. Although viewed differently, these are linked concepts; globalization can be considered the catalyst, while internationalization is the proactive response [15].

International education has become a central focus of educational research, with globalization and the internationalization of higher education as distinct areas of inquiry. Each approach is linked to the global, social, political, economic, and cultural shifts that shape the overall approach [16]. It has also been reported in the literature that institutions need to adopt specific approaches, in addition to any existing generic approaches in practice, to respond to the internationalization of higher education, such as activity, competency, ethos, and process [17].

Internationalization or globalization of education is one-way; professional students, who make up a reasonably significant segment of a community, can be equipped with better skills to compete with these phenomena and the rest of the world. That is not to suggest that hard skills are irrelevant or no longer important; rather, the global economy and the functioning of international organizations require multi-skilled graduates. The requirement is becoming increasingly relevant in the current international landscape. It is worth clarifying that in this study, the phrase "international SE education" refers to teaching international students in a particular local context.

2.3 Teamwork Skills in International SE Education

Software development teams have become a workforce with diverse cultural and educational backgrounds due to their social nature, which entails the involvement of multiple team members [18, 19]. Likewise, in international SE education, students come from diverse backgrounds. In any case, it is not possible to achieve the benefits of collaborative work without effective teamwork within teams.

From an international education perspective, millions of STEM graduates are produced worldwide, and many lack adequate soft skills to meet global market demands

[20]. Soft skills, such as teamwork, are essential to meet the pressing needs of the international work environment [21]. However, in some countries, soft skills do not make a substantial part of secondary and tertiary education [22].

Teachers and students are both responsible for learning, and through making students work in groups, following advantages can be achieved [23]: Group work may engage students in what they are learning for internalization to occur through repeating or sharing the contents; the phenomenon allows students to distribute different types of tasks among themselves; individuals can be more motivated when offered an element of choice in terms of the type of work they want to do. Cultural background and beliefs are likely to impact learners' behaviour, interpretation, and understanding [23]. It is also the case that conceptions of learning may differ between two countries, reflecting cultural differences [24].

However, it has been reported that software companies have noted that graduates, despite their technical expertise, lack the skills needed to work in groups [6], underscoring that soft skills are as crucial as technical capabilities for students to succeed in their professional careers. Group work, on the other hand, is a recommended way for students to practice the professional skills required in the industry [25]. However, introducing group work alone in SE courses is not sufficient unless it is implemented effectively. The existing work on SE education mainly addresses content delivery in a globally distributed setting and the overall challenges that come with it [26–28]. Yet, it fails to address the issues that confront students' collaborative learning in international settings.

3 Research Methodology

Considering the research objectives and motivation, the research questions were formulated as follows.

RQ1: What teamwork-related skills do students learn through group work in international software engineering education?

RQ2: What teamwork-related skills can students learn through group work in international SE education?

RQ3: What are teamwork-related challenges in international SE education with respect to culture?

The first two questions might look similar, but their interpretations differ. RQ1 is based on teachers' experiences, such as the skills they are currently teaching and what they think students are learning from their existing teaching, as observed in student group work, which prompts them to assign group-based tasks. On the other hand, RQ2 aims to investigate which skills students currently lack, which they can learn with teaching support, and which they should be learning in addition to the existing ones. The question was also aimed at professional skills, that is, the soft skills required by the software industry for them to practice after graduating at a different level. That said, some of the skills acquired from RQ1 and RQ2 might overlap; however, for RQ2, participants had more context and did not have to rely on memory because they were provided with a

sample list of soft skills drawn from multiple scholarly sources. For the same reason, their selection of the soft skills students should be taught through group work was more explicit and included comparisons and examples that linked to their own teaching. In RQ3, we address cultural challenges in the subject under discussion (Fig. 1).

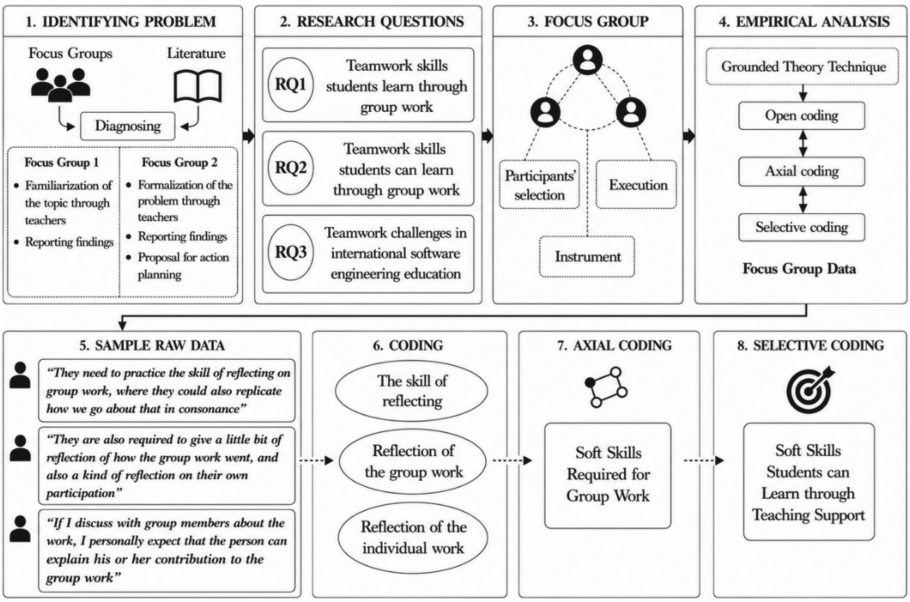


Fig. 1. The overall research process

Action research [33] was chosen as the research method for the overall research project. However, this study presents the first phase of the Action research cycle, called Diagnosis, during which we conducted two focus groups to investigate the problem, a well-established method for collecting empirical data [29]. The university teachers, with experience profiles in Table 1, were chosen as subjects because they possessed strong knowledge and understanding of various teaching methods. The empirical data and results presented in this article are derived from two sub studies. The first was conducted as a pilot in the fall of 2022, and the second was conducted in 2025, a continuation of the first to make the overall study longitudinal. The participants gained additional international experience in between through an SE education exchange program with a university in China. The pilot focus group, FG1, was attended by three SE teachers, and the second focus group, FG2, was attended by ten teachers at the University of Oulu. All participants taught international and local groups within the country and abroad. The execution of the pilot focus group helped tweak the questions asked of the experts, with a view to acquiring more targeted data.

In our endeavours to answer the research questions, participants were engaged through open-ended questions, presented in Fig. 2, along with how each linked to the corresponding research question, their purpose, and their use in the analysis. To support participants' memory, they were shown three candidate lists of soft skills to help

Table 1. Experience profile of the study participants

SE experience (years)	Overall experience (industry + teaching)	International settings	
		Overall	Teaching
1–5	1	2	2
6–10	3	3	4
11–15	5	4	3
16–20	1	1	0
≥20	2	2	1

Note. Values indicate the number of participants.

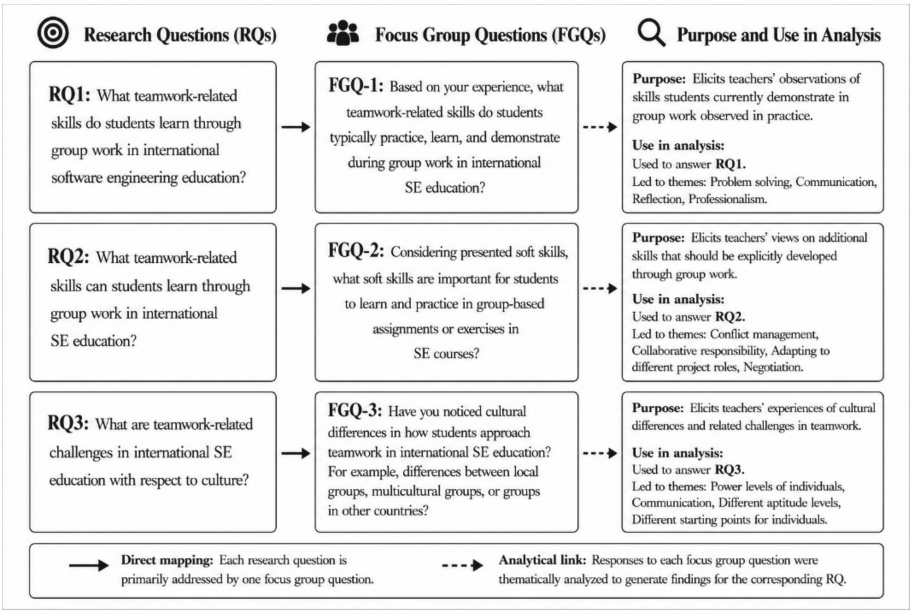


Fig. 2. Link between research questions and focus group questions

them answer the focus group questions furnished in the Appendix. For the pilot focus group, only list C was made available, as the authors had limited context for the problem domain and also wanted to learn how to ensure the focus group ran smoothly. However, participants were encouraged to bring soft skills not on the list. For analyzing qualitative data, thematic analysis was chosen, a foundational method for qualitative data analysis [30]. The first author analyzed and coded the data, and the codes were discussed with the second author in multiple meetings to reach an agreement.

4 Results

This Section presents the research results. For the sake of clarity and readability, we follow the same investigative scheme for presenting the results, organized by our research questions: the teamwork skills students currently learn, the ones they should be provided support for, and the challenges that come with practicing the phenomenon, with an emphasis on culture in the international education setting. Figure 3 depicts the overall framework as a branching model, based on empirical findings from our study, that links the identified skills and challenges to the research questions. It is worth noting that, due to space limitations, we are only partially reporting the results of the study we conducted. In terms of skills, the crux of the findings is that the teachers distinguished between skills that students practice naturally through group work and those that require instructional support.

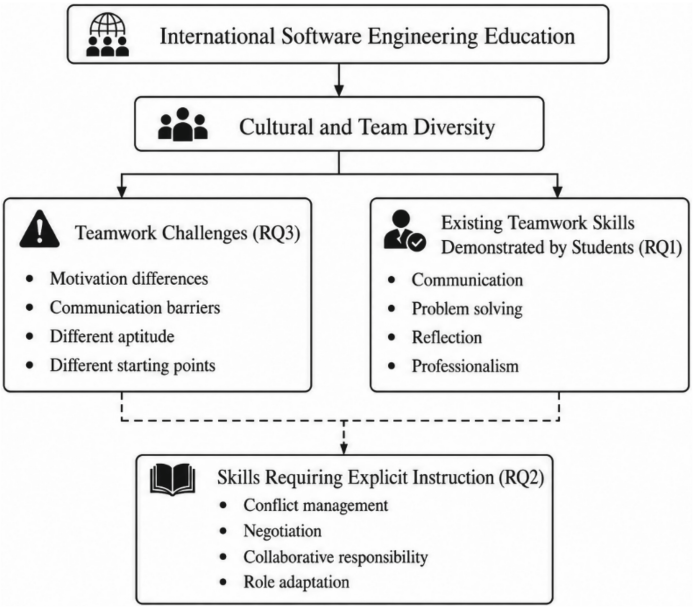


Fig. 3. The overall framework linking challenges and skills

4.1 Teamwork-Related Skills Students Learn through Group Work (RQ1)

Analysis of the focus groups that the study participants consistently observed the following competencies listed in the section as the ones that naturally emerge through group work. These are outlined as outcomes of collaborative work rather than skills requiring instructional intervention. In a nutshell, the skill themes together represent a category of skills that emerge naturally.

Problem Solving. Problem-solving entails selecting the correct solution from several options through reasoning and decision-making [31]. One advantage of group work is that it helps students learn problem-solving, a core requirement of the software industry, and replicate it while they are studying. In the Focus group, the experts reported that learning to solve problems was an important lesson in teamwork. If done together, it helps solve a problem more effectively. An expert, who had an industry background as well, expressed,

> *"There are different ways of learning, and they do individual learning as well, but real problem solutions, let's say that, typically programming work in industry is done by a person, but problem solving is done together."* (FG1)

While working in a team, students also do individual learning. However, problem solving, which is core to working in the software industry, is always done collaboratively, and teamwork on course-related projects provides students with an opportunity to learn the skill. Another participant had a similar view of the skill for student teamwork.

> *"I try to emphasize that it is often easier for them to start solving the problems if they do it together."* (FG1)

It is evident that one participant viewed practicing problem-solving as a software industry requirement and a way to prepare students for their professional careers. In contrast, the other participant thought that practicing problem-solving collectively facilitates students' problem-solving by allowing them to share their knowledge. The perspectives might differ, but the motivation to practice and learn the skill remains similar.

Communication. It is widely considered a soft skill, whether or not it is part of teamwork. Teamwork in SE education provides students with opportunities to practice communication, collaboration, negotiation, and conflict resolution, thereby developing these skills for work-related situations in industry. The IEEE's Software Engineering Body of Knowledge (SWEBOK) [33] knowledge area, Software Engineering Professional Practice, is divided into three subareas, one of which is communication skills: "Professionalism"; "Group Dynamics and Psychology"; "Communication Skills". Teamwork is explicitly presented under "Group Dynamics and Psychology", as "Dynamics of Working in Teams/Groups". "Communication Skills" is also closely connected to teamwork-related skills. In line with the SWEBOK's taxonomy, one of the experts asserted that communication should be professional. Not only that, communication is linked to making students learn to present a project plan and results, e.g.,

> *"But there is also this aspect of professionalism, so they have to practice the professional communication; towards the customer, towards the teacher, and a kind of learning to present the project, the plan, and the results."* (FG2)

Communication-related skills, such as presentation, came up as important expertise that team members can develop, as was revealed from the discussion with experts, e.g.,

> *"You need to make it clear in the evaluation that everyone needs to be presenting, and it depends on them how they want to do the presentation. Nobody is good at it at first; we have to practice it to do it."* (FG1)

Presentations are one way of professional communication. Moreover, the phenomenon involves students developing writing and documentation skills before they can stand up and deliver. That, in turn, requires students to be good listeners and understand each other well in a group before they can present themselves.

Reflection. Reflection as a group helps team members evaluate others' work, thereby improving collaboration and fostering collective problem-solving [34]. It plays an important role in promoting learning and is reported to have multiple benefits: it is an account of how students confronted challenging situations, draw on experience to improve for the future, brings different ideas and enhances learning, helps learners see the interconnections of the knowledge learning, and makes students find social connections among their fellows [35]. In our study, reflection came up as a recurring topic as a means for individual and inter-team evaluation, e.g.,

> *"Analyze their own work, what is good and what did not work. Evaluate others' work as well, and inter-team evaluation, their reflection on their own work."* (FG1)

Students must learn to reflect on the group's overall work and their own participation. Doing that can help them replicate their success. One participant said they use the learning diaries for this purpose, which helps teachers better understand the scenario for multicultural student teams. Studies, for example, [35], suggest that instructors can design prompt/relevant questions for the reflection to improve learners' meta-cognitive knowledge, e.g.,

> *"We provide some questions for them to reflect on, which also helps us to evaluate."* (FG2)

Reflective practices are known for enhancing teamwork and individual learning [36]. Therefore, teachers' roles can help them develop soft skills relatively quickly. For instance, upfront notice that students will be evaluated independently and in the middle of their work can help them understand what they are expected to do, which, in turn, can improve their planning skills.

Professionalism. Teamwork involves implementing professional practices alongside technical skills. Certain characteristics of the team and its activities may affect teamwork operations and productivity. Success depends on methods and tools, and it also heavily relies on how software engineers are educated to incorporate professional practices [10, 18]. This practice repeatedly came up, and one of the instances of professional growth was that they must be able to behave responsibly not only within their group but also to the other groups as well, and it starts with communication, as one of the participants said,

> "*Speaking of professionalism, also not towards the customers and teachers, but also like professionalism while working as a group...and towards the other group is also important.*" (FG2)

It was agreed among the participants that students must be able to practice professionalism when working as a group and when working with other groups. Moreover, professionalism fosters professional growth among group members, and the meaning of that growth may differ. They should learn to present the project in terms of plans and results, as those are evaluated against the given set of requirements as part of group work.

Key Insights from Sect. 4.1: Problem-solving is an essential skill for a professional career, and teamwork provides students with an opportunity to practice it. Communication is not about speaking alone; it remains insufficient unless it is professional and is exercised towards all stakeholders. The skill is closely linked to plan generation and to presenting project results, which are crucial to an individual's ability to reflect on themselves.

4.2 Teamwork Skills Students Should Learn through Group Work (RQ2)

Unlike the skills identified in RQ1, the study participants identified the following skills as requiring pedagogical support. The participants agreed that simply assigning students to multicultural or random teams was not enough to help them develop those skills.

Conflict Management. An important aspect of conflict management is that if any group member is not contributing as expected by the group or in line with the group's work demands, they should not be excluded; instead, they should be engaged, and students should not come complaining to the teacher at the end. They need to discuss among themselves. One of the experts expressed that,

> *"There are also these groups where there might be some conflicts inside the groups, so that some person does not work, for example, so this conflict management is also something that I would like somehow that they need to learn; not that after the group work you hear that these two people did not do it, so we excluded them. That is not like you were supposed to do; you were supposed to discuss it among yourselves somehow."* (FG2)

Conflict management is challenging in multicultural settings for two reasons: the language barrier and students' inability to address their grievances promptly, both of which may be influenced by cultural factors. For example, in some cultures, harmony is expected, while in others, people fight to resolve conflicts. For these reasons, the timely resolution of work-related conflicts is crucial to the successful execution of group work.

Collaborative Responsibility. It has to do with responsibility too, as the group project should make them responsible, which is a collaborative responsibility that falls under professional ethics and is how they should be performing, individually as well as in a group. As it came up during the focus group,

> *"I think that is the responsibility part here that once they are in a group project, they should be responsible for their own part, and I think it is also to this 'collaboration' that they are a kind of collaboratively responsible for their group work, if so, in that sense they have been under this 'professional ethics'. However, then this is how they should be performing at the individual level, but also together as a group."* (FG2)

Along with collaborative responsibility, responsibility alone emerged as a general theme that should be present at multiple levels to support student teamwork. On the other hand, the perception of a lack of responsibility on the part of teachers could demotivate students from embracing the core philosophy behind group work. As one of the participants said about the perception of the students,

> *"They would undermine their teacher's commitment to accountability. They deserve the feedback for what they have done, so they must all be evaluated."* (FG1)

Upon inquiry, the same participants elaborated that it could be the overall academic culture, or the culture of favoring the good students, that is even undermining the not-so-good ones. Essentially, it is not only students who should behave responsibly; teachers, in their role, should as well, so they can gauge teamwork effectively.

Adapting to Different Project Roles. Courses or modules focused on software development require students to take on different project development roles, providing an opportunity to practice the soft skills associated with each role. Teachers can help the students choose suitable roles for themselves, especially those who do not have much experience with teamwork, e.g.,

> *"We have this Software Organization module, for example, we take an agile team, then they have these different roles working together in a group to develop a product, actually. What we do is like we try to help the students to take these different roles doing the project work because each role has different soft skills to maintain that work and then synchronize that thing."* (FG2)

For instance, if the course is about agile software development and students are being taught Scrum, they are expected to learn multiple roles so they can practice a range of soft skills, as each role requires specific soft skills to execute effectively. The participant also mentioned that, in a few instances during the course, they allowed students to take on self-organized team roles.

Negotiation. It requires individuals to be flexible enough to change their minds and willing to negotiate. A participant emphasized the importance of negotiation skills and said,

> *"So these negotiation skills could be very much highlighted that you need to negotiate your goals and bring your own goals to the table and agree with the group."* (FG2)

It implies that effective practice of negotiation skills is crucial for achieving goals, as the preliminary stage is identifying them, and the skill is needed to set them up. It is not just about setting individual or group goals; negotiation skills are also required to carry out tangible project-related tasks, especially when students are given the freedom to choose their topics. Negotiation may help them reach an agreement in cases of conflicting choices, as one expert from the same focus group shared their experience:

> *"I think some of the skills they would be needing are negotiating skills because sometimes it gives them the freedom to choose the topic they will be working on if it is a group assignment or something."* (FG2)

It implies that flexibility and negotiation are linked to the extent that, to negotiate well, one should be flexible enough to listen to and accommodate others.

Key Insights from Sect. 4.2: Students from diverse backgrounds may handle and negotiate conflicts differently. The phenomenon makes them learn how to deal with conflicts with their culturally diverse teammates. The need or urge to address conflicts should make them collaboratively responsible for their project task. The 'onus of responsibility' should be on teachers as well to treat each team member equally and to provide timely, equal feedback on their work. Each role in software development projects has certain soft skills associated with it. Switching student roles, for instance, Scrum-based development courses can help students not only adapt to different project roles but also learn and practice skills associated with each role.

4.3 Challenges in Terms of Effects of Culture on Skills (RQ3)

The challenges identified in the research question explain why study participants, as teachers, considered instructional support necessary for students to learn those skills. Many difficulties in teamwork were attributed to individual differences in motivation, aptitude, and starting points, as well as to differing communication styles.

Varying Levels of Motivation. A varying level of motivation in a multicultural group is one thing that different cultures can inspire. If motivation is considered a skill, it appears that there are some factors that, in a way, might be culturally dependent and can make students more motivated, regardless of their cultural background; for instance, students travelling to a different country and studying in its environment, which requires acquiring and disposing of means. As the participant was quoted as saying that,

> *"One of the things we have noticed is that students have different motivations regarding the course; someone just wants to pass the course, and then someone wants to get high grades. That motivation actually drives them to be more efficient or less efficient. However, if we have this kind of combination of students, this actually creates problems or even challenges to conduct this teamwork."* (FG2)

That said, motivation is more of an individual characteristic and is not culture-dependent in its true sense. The same is suggested by motivation theories, such as the content and process theories, which list multiple sources of the phenomenon; culture is not among them. Therefore, it is worth clarifying that the motivation here is only to get good grades,

which can be considered extrinsic motivation. Some students might want to get grades by doing nothing, and others' objective would be different.

Communication. Communication is a basic skill essential for teamwork, and students may practice it as much as any other skill. However, the phenomenon also poses challenges in multiple ways: reduced expressiveness, language barriers, differences in communication styles, and the need for a common language for effective communication. Communication style may vary from formal to informal, and can adjust team members' expectations for how they are treated. One of the experts expressed their opinion that,

> *"In some cultures, their communication style is inherently different, and it shows in communication in every way they formulate the emails that they send."* (FG2)

The language barrier came up as one of the things that prevents students not only from communicating with each other but also from asking the teacher questions. Another expert shared their experience teaching in the international environment, e.g.,

> *"Not only that, but you know, even when you allow students to open up and say that if you are having any challenge, please communicate it now. They will not be telling you that during the sessions, when everyone is there.."* (FG2)

In that regard, the teacher's role is equally important in giving them the confidence and opportunity to be open and express themselves. At the same time, not only is it the common language, but it is also a communication challenge at a lower level within the same domain, as people get used to different dialects and/or accents.

Different Aptitude Levels. In addition to varying motivation levels, individuals in a student team may have different aptitudes, which can affect teamwork both positively and negatively. Positively, because those individuals would strive to do their best in their roles, which in turn would improve overall teamwork. On the contrary, having one or two individuals with higher or even lower aptitude levels than the rest may make the rest of the team members very challenging to catch or even cope with the former, e.g.,

> *"If a person comes from a very competitive aptitude, and sometimes some cultural aptitude could be competitive, for example, sometimes your aptitude is that I want to be the best. So that kind of thing can make a difference in this thing."* (FG2)

However, it is worth noting that competitiveness is not uniform across countries. Culture may influence how individuals compete, as traits can be rooted in different social layers. For instance, a study by Hauge [37] asserts that cultural norms transmitted through families, particularly by parents who had moved to different countries, can positively influence competitive behaviour. Different aptitude levels among peers in a group can lead to the prevention of knowledge sharing. For instance, students with high aptitude might want to do everything on their own, without depending on their peers' contributions.

Different Starting Points for Individuals. Another interesting point that came up for international SE students is different starting points considering different backgrounds of the students, e.g.,

> *"So I think there are those, but what a culture is, is a very difficult thing to say and to point. But if there are very competitive starting points for individuals, that is something."* (FG2)

It implies that teachers would need to start differently for individuals from different international academic backgrounds. Those starting points could be influenced by language barriers or differences in the individuals' aptitude or motivation levels. Teaching arrangements can also pose different situations; for instance, local students and international teachers, and vice versa. As one of the experts said,

> *"But what is interesting in China is that there the students are from the same country, and we are just the international teachers. So that is a slightly different situation."* (FG2)

The scenario poses a significant challenge as well. For instance, teachers from another academic culture might find it challenging to adapt to students' culture and even feel alienated from it. In that situation, the roles of teachers and even institutional management may also come into play, depending on whether they prefer students to be taught in accordance with the existing academic culture or encouraged to adopt a new one.

Key Insights from Sect. 4.3: Just as benefits can be composite, challenges can be as well. There may be varying levels of motivation among students in a team with diverse members, driven by factors such as the desire to earn grades and the decision to relocate to another country to acquire knowledge, which may also influence their aptitude levels. Furthermore, communication styles or stereotypes among such diverse members would differ. All in all, that makes teamwork very challenging. Teachers would also find it challenging to guide a diverse group of students with different starting points, varying aptitudes, and/or motivation levels.

5 Discussion

In light of the existing research, we conducted focus groups to investigate the domain further; the results are presented in Sect. 4. The Focus groups with experts provided significant insights into teamwork skills in international SE education. Students can learn multiple teamwork skills through group work in their course-related assignments. The findings are summarized below in light of the research questions.

RQ1-What teamwork-related skills do students learn through group work in international software engineering education?

The existing work on teamwork skills or challenges has been course-specific, or has been applicable in a given settings or with a limited context, for instance [38, 39], or mapping or review studies [40, 41] which mainly rely on the frequency of the occurrence of various soft skills in the scholarly articles to manifest the degree of importance of each skills, instead reflecting its true value in today's academic settings by incorporating the actual stakeholders, the teachers, and their real world relevant teaching experience. In international settings, the usefulness of some soft skills, their context, and the roles they play can be even more important, and they can help students further reap the benefits of collaborative work. The international environment from which students come, with its diverse social and academic cultures, provides them with an opportunity to develop their skills. Although teamwork may involve individual work, it still requires problem-solving, which is how things are done in the software industry. The ability to communicate well is a known soft skill. However, professional communication is more beneficial for students' learning, as it is required for presentations and helps them reflect effectively on themselves when needed. That eventually leads to adopting professionalism in general, which serves as the source of practicing multiple soft skills such as collaboration, negotiation, and conflict handling. A combination of skills, such as problem-solving, reflection, and group learning, can augment and lead to the learning and practice of other skills as well; for instance, cooperation and coordination require communication, and vice versa. Knowledge sharing may foster cooperation and work planning, which may help develop plan generation skills. Teamwork may also help the members learn to deal with peers from different academic backgrounds. It is worth mentioning that this study was not aimed at producing a unique set of soft skills or a new taxonomy of the phenomenon. Our findings are consistent with the existing literature, at least regarding naming conventions.

RQ2-What teamwork-related skills can students learn through group work in international SE education?

There are multiple teamwork-related soft skills that students should learn through group work, especially as they prepare to enter the software industry and compete for professional careers. Those include, to name a few, conflict management, shared responsibility, adaptation to different project roles, and negotiation with one another. Such skills are closely linked to each other. It is the recurring challenges associated with culturally diverse student teams that drive the need for these skills. On the other hand, existing research on project-based software engineering courses that require group work overlooks the importance of many soft skills and focuses primarily on the more commonly recognized ones. Not only have we listed the soft skills, but we have also argued how they complement each other. For instance, effective conflict management does not advocate excluding conflicting voices and putting the onus on them, but rather engaging them. The practice should make students collaboratively responsible, at both the individual and group levels, for advancing the team's overall agenda. Teamwork in the project-based Scrum courses can be very beneficial for students to practice multiple skills: first, as a course requirement, it makes students well-versed at switching between different roles and adapting to them. Secondly, the practice may also help improve their communication skills by encouraging collaboration after they agree on a common language in international settings. Thirdly, changing project roles during the course of the project requires

students to collaborate by designating their roles and to date work to their peers. The need to collaborate on those tasks makes them jointly responsible to one another and the team.

RQ3-What are teamwork related challenges in international SE education with respect to culture?

From a different perspective, the skills required of students in the international SE education domain can also be perceived as challenges if not catered for. While listing the challenges, we have also elaborated on the situations or scenarios in which teachers or students may encounter them. On the other hand, existing research in the domain has mainly focused on distributed [42] and global [43] aspects. It is limited to either a given context or the overall challenges, not the specific ones, for instance, teamwork. Learning teamwork-related skills in specific international settings can be significantly culture-dependent. At the same time, defining culture itself can be a tricky part because of the various factors that constitute or influence it.

For the same reason, cultural diversity among students in international settings can directly or indirectly influence multiple challenges, with varying levels of motivation and aptitude being highlighted as factors. Among other reasons, in international settings, an individual student's decision to travel to another country to study or acquire knowledge in a different culture can be a motivating factor that sets them apart from the rest. For the same reason, students' level of competency may also be linked to their cultural background, which can make some cultures appear more competent than others. Communication, which students need to practice extensively for successful teamwork, lies at the core of the problem due to the diverse attributes of individuals within a team, such as skill and experience levels, and this separation can be further widened by differences in motivation and aptitude within the same team. For similar reasons, individuals in a team may have different starting points, which poses a great challenge for teachers in guiding students and ensuring they have equal learning opportunities at the end of their teamwork.

6 Threats to Validity

Some factors can threaten the validity of the results; for instance, the study involved two focus groups of 13 teachers from the same university who teach international software engineering. A small number of participants in FG1, which was more of a pilot study, but evidence was drawn from its findings to support the conclusion. Moreover, varying levels of experience among participants may introduce bias into the results, with participants with more experience in teamwork-based teaching more likely to speak and participate in the discussion. However, speaking can be an individual personality trait independent of experience. Above all, the outcome of the focus group is regarded as the collective contribution of the group rather than that of any individual participant. Another possible threat to validity is that all participants are teachers from a single university, which limits generalizability to that context. Despite all efforts, the analysis, interpretation and synthesis of data may be influenced by researchers' personal biases. However, the themes identified from each focus group are consistent in terms of the nature of the skills and challenges, which lends credibility and significance to the reported findings.

7 Conclusion

This study aims to identify teamwork skills in software engineering (SE) education that are essential for students to learn and practice through group work in their course-related assignments and exercises. It is crucial to investigate teamwork, as professional SE education should prepare students to work in teams for their professional careers. To carry out the study, two focus groups were conducted with university teachers who have experience teaching SE courses in diverse international environments. The results were then theoretically analyzed not only to assess their usefulness for international SE education settings but also to strengthen them for use as input to the action planning phase of the action research cycle, from a broader research perspective, as part of the continuing future work.

The study revealed multiple teamwork skills students can learn and practice through group work in their SE education. Learning those skills can be helpful for students' future careers. The study also found that students from different cultures and academic backgrounds are likely to possess different work habits that not only pose challenges but may also help them learn diverse teamwork skills. However, cultural differences among teams can also affect the way students learn and practice those skills. The challenges in international SE education should guide teaching practices and prompt teachers to adapt their pedagogical methods and teaching content. All in all, the study findings can help SE teachers to design their courses accordingly and effectively execute teamwork.

Acknowledgements. The Grammarly tool was used to improve the language, and LLMs were used to polish the figures.

Appendix. Soft Skills Frameworks Presented to Focus Group Participants (Skills and/or Competencies Highlighted where Relevant)

Personal skills: Learning skills, Tolerance to stress, Professional ethics, Self-awareness, Commitment, Life balance, Creativity/Innovation
Social skills: Communication, Teamwork, Contact network, Negotiation, Conflict Management, Leadership, Culture Adaptability
Content-reliant/Methodological skills: Customer/User orientation, Continuous improvement, Adaptability to change, Results orientation, Analytical skills, Decision making, Management skills, Research and info management

(a): Cinque, M.: "Lost in translation". Soft skills development in European countries. *Tuning J. High. Educ.* 3(2), 389–427 (2016). [7]

	Domain Name	Clusters of Competencies	Competencies
Domains of Competence	The Cognitive Domain	cognitive processes and strategies, knowledge, creativity	critical thinking, information literacy, reasoning and argumentation, and innovation
	The Intrapersonal Domain	intellectual openness, work ethic and conscientiousness, positive core self-evaluation	Flexibility, initiative, appreciation for diversity, and metacognition
	The Interpersonal Domain	teamwork and collaboration, leadership	communication, collaboration, responsibility, conflict resolution

(b): National Research Council: *Education for Life and Work: Developing Transferable Knowledge and Skills in the 21st Century*. National Academies Press (2012) [44]

Theme	Professional Practices/Skills
Communication (Coordination/Cooperation/Interdependence)	Documentation, speaking/explaining, listening, reading/summarizing/explaining, writing
Teaming	Consultation, decision making, discussion, plan generation, conflict handling, multitasking, multicultural knowledge transfer, commitment, accountability, group cohesion
Presentation	Lead reviews, product walkthroughs, training, documentation

(c): Based on ACM Curriculum Guidelines [10] and IEEE-SWEBOK (Software Engineering Body of Knowledge) [33]

References

1. Betti, L., Gallo, L., Wachs, J., Battiston, F.: The dynamics of leadership and success in software development teams. Nat. Commun. **16**(1), 3956 (2025)
2. Artz, G.M., Jacobs, K.L., Boessen, C.R.: The whole is greater than the sum: an empirical analysis of the effect of team-based learning on student achievement. NACTA J. **60**(4), 405–411 (2016)

3. Wragg, N., Barnes, C.: Group work and the externally-oriented capstone: opening students to the challenges of professional practice. In: Collaboration and Student Engagement in Design Education, pp. 305–331. IGI Global (2017)
4. Niva, A., Markkula, J., Annanperä, E.: Junior software engineers' international communication and collaboration competences. IEEE Access. **11**, 139039–139068 (2023)
5. Borges, G.G., Souza, R.C.G.: Skills development for software engineers: systematic literature review. Inf. Softw. Technol. **168**, 107395 (2024)
6. Garousi, V., Giray, G., Tüzün, E., Catal, C., Felderer, M.: Aligning software engineering education with industrial needs: a meta-analysis. J. Syst. Softw. **156**, 65–83 (2019)
7. Cinque, M.: "Lost in translation". Soft skills development in European countries. Tuning. J. High. Educ. **3**(2), 389–427 (2016)
8. Malinen, S., Galster, M., Mitrovic, A., Sankara Iyer, S., Peiris, P., Clarke, A.: Soft skills in software engineering: insights from the trenches. In: The proceedings for the 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), p. 296–306. IEEE (2025)
9. IEEE: Software Engineering Competency Model (SWECOM). IEEE Computer Society (2014)
10. CC2020 Task Force: Computing curricula 2020: paradigms for future computing curricula. ACM/IEEE/IEEE Comput. Soc. (2020). https://doi.org/10.1145/3467967
11. Niva, A., Markkula, J., Annanperä, E.: Software engineering competency models and intercultural communication competencies: a systematic literature review. J. Syst. Softw. **112662** (2025)
12. Assyne, N., Ghanbari, H., Pulkkinen, M.: The essential competencies of software professionals: a unified competence framework. Inf. Softw. Technol. **151**, 107020 (2022)
13. Groeneveld, W., Vennekens, J., Aerts, K.: Identifying non-technical skill gaps in software engineering education: what experts expect but students don't learn. ACM Trans. Comput. Educ. **22**(1), 1–21 (2021)
14. Wyrich, M., Montgomery, L.: What's in a software engineering job posting? arXiv preprint (2025). https://arxiv.org/abs/2511.13656
15. Knight, J., de Wit, H.: Internationalization of Higher Education in the Asia Pacific Countries. European Association for International Education, Amsterdam (1997)
16. Dolby, N., Rahman, A.: Research in international education. Rev. Educ. Res. **78**(3), 676–726 (2008)
17. de Wit, H., Knight, J.: Quality and Internationalisation in Higher Education. OECD (1999)
18. Marques, M.R., Quispe, A., Ochoa, S.F.: A systematic mapping study on practical approaches to teaching software engineering. In: Proc. IEEE Frontiers in Education Conf. (FIE), pp. 1–8. IEEE (2014)
19. Garousi, V., Petersen, K., Ozkan, B.: Challenges and best practices in industry-academia collaborations in software engineering: a systematic literature review. Inf. Softw. Technol. **79**, 106–127 (2016)
20. Van Horne, C., Rakedzon, T.: Teamwork made in China: soft skill development with a side of friendship in the STEM classroom. Educ. Sci. **14**, 248 (2024)
21. Ardis, M.: Software engineering 2014: curriculum guidelines for undergraduate degree programs in software engineering. ACM. (2015)
22. Kim, E.P., Bruehler, K.: What fortune 500 companies want: recruiting in China. J. Glob. Bus. Insights. **4**, 125–140 (2019)
23. Teng, W., Turner, J.J.: Reflections and evaluation of Chinese enterprise education: the role of institutions from the perspective of learners. In: International Enterprise Education: Perspectives on Theory and Practice, pp. 52–68. Routledge, London (2018)
24. Roininen, E.: Identifying the Cultural Challenges of Finnish and Chinese Collaboration in University Education of Software Engineering. University of Oulu (2019)

25. Porquet-Lupine, J., Brigham, M.: Evaluating group work in (too) large CS classes with (too) few resources: an experience report. In: Proc. 54th ACM Tech. Symp. Comput. Sci. Educ., pp. 4–10 (2023)
26. Čavrak, I., Bosnić, I., Ciccozzi, F., Mirandola, R.: Resilience of distributed student teams to stress factors: a longitudinal case-study. Inf. Softw. Technol. **114**, 258–274 (2019)
27. Isomöttönen, V., Daniels, M., Cajander, Å., Pears, A., McDermott, R.: Searching for global employability: can students capitalize on enabling learning environments? ACM Trans. Comput. Educ. **19**(2), 1–29 (2019)
28. Kokkoniemi, M., Isomöttönen, V.: A systematic mapping study on group work research in computing education projects. J. Syst. Softw. **204**, 111795 (2023)
29. Susman, G.I., Evered, R.D.: An assessment of the scientific merits of action research. Stud. Organizzativi. **2** (2023)
30. Krueger, R.A.: Focus Groups: a Practical Guide for Applied Research. Sage Publications (2014)
31. Braun, V., Clarke, V.: Using thematic analysis in psychology. Qual. Res. Psychol. **3**(2), 77–101 (2006)
32. Levi, D., Askay, D.A.: Group Dynamics for Teams. Sage Publications (2020)
33. Washizaki, H. (ed.): Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0. IEEE Computer Society (2024)
34. Strauß, S., Tunnigkeit, I., Eberle, J., et al.: Promoting collaborative reflection to foster interprofessional collaboration skills: putting laboratory findings to the test in the field. Int. J. Educ. Technol. High. Educ. **22**, 54 (2025)
35. Chang, B.: Reflection Learn. **23**(1), 95–110 (2019)
36. Siegeris, J., Pfennig, A.: Empowering student projects: fostering reflective practices for enhanced teamwork and individual learning. In: ICERI2023 proc. IATED. (2023)
37. Hauge, K.E., Kotsadam, A., Riege, A.: Culture and gender differences in willingness to compete. Econ. J. **133**(654), 2403–2426 (2023)
38. Raibulet, C., Arcelli Fontana, F.: Collaborative and teamwork software development in an undergraduate software engineering course. J. Syst. Softw. **144**, 409–422 (2018)
39. Sherman, S., Kantsepolsky, B., Roth, I.: teamwork skills: the case of a practical course. Issues Inf. Syst. 23(2) (2022)
40. Matturro, G.: A Systematic Mapping Study on Soft Skills in Software Engineering. J. Univ. Comput, Sci (2019)
41. Groeneveld, W., Vennekens, J., Aerts, K.: Software engineering education beyond the technical: a systematic literature review. arXiv preprint https://arxiv.org/abs/1910.09865 (2019)
42. Bosnić, I. et al.: Managing diversity in distributed software development education—a longitudinal case study. ACM Trans. Comput. Educ. **19**(2), 1–23 (2019)
43. Hoda, R., Babar, M.A., Shastri, Y., Yaqoob, H.: Socio-cultural challenges in global software engineering education. IEEE Trans. Educ. **60**(3), 173–182 (2016)
44. National Research Council: Education for Life and Work: Developing Transferable Knowledge and Skills in the 21st Century. National Academies Press, Washington, DC (2012)

AI Turning Static Exports into Actionable Health Intelligence

Sagar Rao(✉), Atif Farid Mohammad, Paige Coleman, and Sarthak Bhatt

Healable, Inc., San Francisco, CA, USA

Abstract. Electronic Health Information (EHI) exports have emerged as a regulatory imperative in U.S. healthcare policy, yet widespread usability failures have undermined their clinical and patient-facing value. A recent independent review assigned a median grade of D to the export experience across 265 certified Electronic Health Record systems (Mandel, 2026), demonstrating that data availability alone does not produce actionable intelligence. This paper presents the Healable platform, an AI-powered orchestration framework that transforms fragmented Single-Patient EHI exports into longitudinal, explainable, and role-aware patient records. The system employs a governed agentic AI architecture combining retrieval-augmented generation (RAG) with Context-Augmented Generation (CAG; Mohammad, 2026) to ground every output in the patient's own health data. Specialized agents handle clinical summarization, medication reconciliation, care-transition analysis, population risk stratification, patient engagement, social determinants of health (SDOH) detection (Mohammad et al., 2021; Bearse et al., 2020b), and payer coordination, within a governance framework informed by responsible-AI principles (Coleman & Mohammad, 2025; Kalva & Mohammad, 2025). Data normalization employs graph-theoretic harmonization (Mohammad et al., 2020a). A reference deployment against a synthetic multi-provider record demonstrates the system's capacity to compress a 6,200-resource FHIR bundle into role-specific, cited, actionable outputs while preserving full source attribution and human oversight.

Keywords: electronic health information · EHI export · agentic AI · context-augmented generation · FHIR · explainable AI · social determinants of health · value-based care · health interoperability

1 Introduction

The 21st Century Cures Act and subsequent ONC interoperability rules mandate that certified health IT systems support Single-Patient Electronic Health Information (EHI) exports, giving patients and authorized parties machine-readable access to complete health records. Despite this mandate, practical utility has remained elusive. Mandel (2026) graded 265 certified EHR systems on export quality and reported a median grade of D, citing overwhelming volume, inconsistent structure, absent clinical context, and disconnection from actionable workflows. These findings corroborate broader evidence that data availability without intelligibility does not improve care delivery (Coleman & Mohammad, 2025).

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 524–531, 2027.
https://doi.org/10.1007/978-3-032-39763-8_30

A single patient with a moderate chronic disease burden may generate an export spanning hundreds of C-CDA pages and thousands of FHIR resources with no prioritization and no link to a clinical action. Clinicians synthesizing such records during care transitions average 18 to 25 min per patient (Coleman & Mohammad, 2025), patients cannot interpret clinical terminology, care coordinators struggle to reconcile fragmented data across settings, and payers face administrative friction that delays authorization decisions.

This paper presents the Healable platform, an AI-powered orchestration layer that ingests EHI exports across formats and standards, normalizes them using graph-theoretic harmonization (Mohammad et al., 2020a), and surfaces role-specific, explainable, and actionable intelligence for every participant in the care continuum. The design integrates prior research in AI-driven SDOH analysis (Mohammad et al., 2021), responsible AI in clinical and operational settings (Coleman & Mohammad, 2025; Kalva & Mohammad, 2025), NLP-based behavioral health engagement (Bearse et al., 2020a, 2020b), and context-augmented generation (Mohammad, 2026).

2 Background and Related Work

Current certification requirements mandate EHI exports in an electronic and computable format without prescribing specific standards, producing heterogeneous outputs from structured FHIR R4 bundles to flat PDF renditions. A single patient with moderate chronic disease may generate exports spanning hundreds of C-CDA pages and thousands of FHIR resources with no prioritization and no link to a clinical action. Coleman and Mohammad (2025) established that without AI augmentation, clinicians require approximately 18 to 25 min per patient to synthesize transition-of-care EHI, a workload incompatible with modern workflows.

Mohammad et al. (2020a) established graph-theoretic healthcare big-data normalization, showing graph-based record linkage outperforms conventional sequential methods. Mohammad et al. (2021) produced an AI-based SDOH indicator taxonomy informing the platform's risk stratification. Coleman and Mohammad (2025) extended this with a responsible-AI framework emphasizing explainability and bias monitoring. Bearse et al. (2020a) demonstrated NLP for behavioral health engagement; a companion study (Bearse et al., 2020b) showed loneliness signals in clinical notes are detectable with acceptable accuracy. Kalva and Mohammad (2025) validated responsible-AI principles in insurance claims. Mohammad (2026) introduced Context-Augmented Generation (CAG), extending RAG with workflow state, user role, and care pathway as first-class generation inputs.

3 System Design

The platform accepts Single-Patient EHI exports in FHIR Release 4 (REST and Bulk), C-CDA Release 2.1, CCD, JSON, CSV, and PDF. Upon ingestion, exports undergo schema validation, quality profiling, and terminology mapping against SNOMED CT, LOINC, RxNorm, and ICD-10-CM. The normalization engine applies the graph-theoretic harmonization of Mohammad et al. (2020a), producing a unified longitudinal patient record

conformant with US Core IG v6 and USCDI v3. Three integration tiers serve organizations at different levels of EHR readiness. Tier 1 accepts any certified-EHR export by upload, requiring no EHR-side integration, completing normalization and rendering in under 60 s. Tier 2 enables in-context SMART App Launch 2.0 deployment within certified EHRs using live FHIR R4 and US Core v6 endpoints. Tier 3 supports population-scale Bulk FHIR ingestion for ACOs, payers, and health information exchanges.

4 Agentic AI Orchestration and Explainability

The intelligence layer deploys eight specialized AI agents coordinated through a centralized orchestration engine implementing the CAG architecture of Mohammad (2026). Each agent has a narrower task, smaller retrieval context, reduced hallucination exposure, and an auditable handoff interface. Clinical Summarization generates longitudinal transition-of-care narratives. Medication Intelligence reconciles medications and flags interactions and adherence gaps. Care Transition identifies discharge risks and continuity gaps. Population and Risk stratify cohorts and identifies HCC capture opportunities and avoidable utilization. Patient Engagement, informed by Bearse et al. (2020a), translates clinical content into sixth-grade reading-level language across web, mobile, SMS, and voice channels. SDOH Insight applies the taxonomy of Mohammad et al. (2021) and NLP signals from Bearse et al. (2020b) to identify social risk factors. Payer Coordination assembles prior-authorization packets and validates coverage. The Explainability and Audit agent cross-cuts all others, attaching source citations, confidence indicators, and reviewer trail metadata to every output.

5 Platform Interfaces

The platform surfaces intelligence through nine role-specific interface screens. Figures 1 and 2 present color screenshots from the deployed prototype using a synthetic reference patient record (Figs. 3 and 4).

The governance framework implements the responsible-AI principles of Coleman and Mohammad (2025) and Kalva and Mohammad (2025). Each AI-generated statement carries a citation to the specific FHIR resource or C-CDA section that supported it. Agents abstain from generation when no supporting evidence exists in the patient's normalized record. Consequential outputs require explicit clinician acknowledgment. Continuous fairness evaluation monitors output distributions across age, sex, race or ethnicity, language, and payer subgroups.

Reference Deployment Case To demonstrate end-to-end system behavior, we describe a reference deployment against a synthetic multi-provider patient record. The reference patient (Patient JW) is a 67-year-old male with hypertension, Type 2 diabetes mellitus, heart failure, and hyperlipidemia, whose care spans an acute hospital, an ambulatory primary-care practice, a cardiology specialist, and a behavioral health provider. The raw EHI export comprises approximately 480 pages of C-CDA documents and a FHIR R4 bundle of approximately 6,200 resources. No clinician reviewing this volume during a

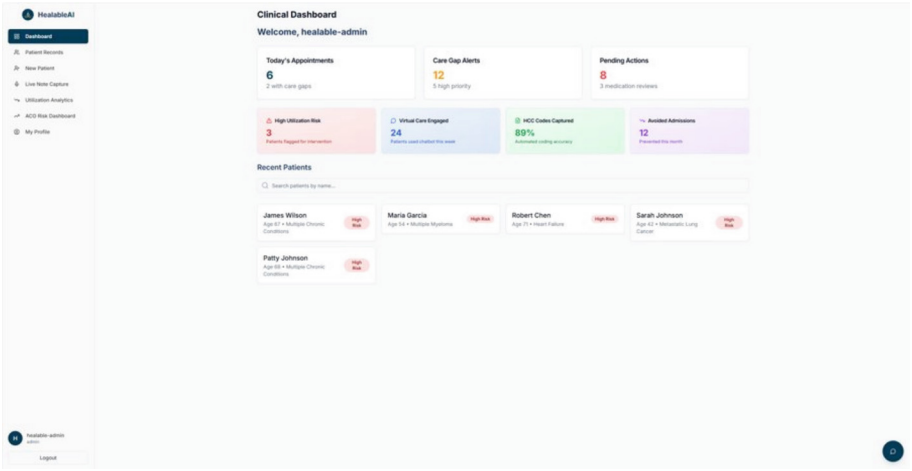


Fig. 1. This shows the role-based clinical dashboard surfacing care-gap alerts, pending actions, high-utilization risk, virtual-care engagement, HCC capture accuracy, and avoided admissions, all derived from ingested EHI

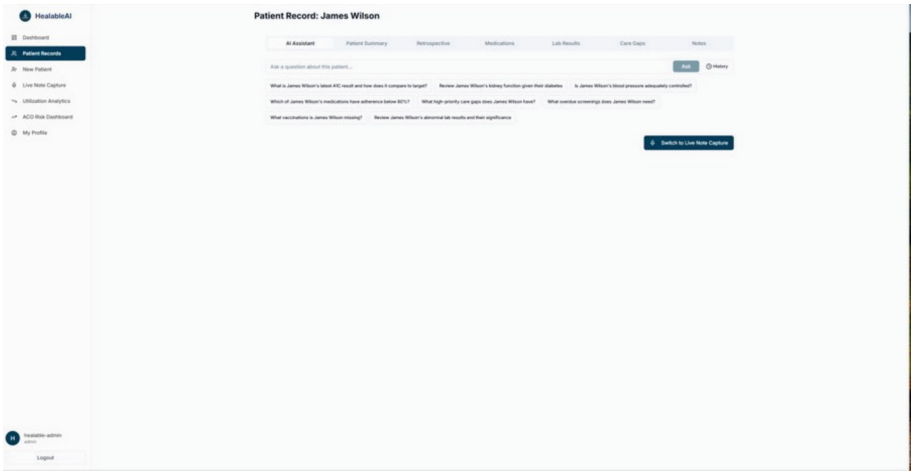


Fig. 2. This panel shows the AI Clinical Assistant, which scopes natural-language question answering to the patient's normalized record; every response carries a citation to its source EHI artifact

standard care transition encounter could be expected to synthesize actionable priorities within a clinically reasonable timeframe.

Following ingestion and normalization, the platform renders Patient JW's record across nine integrated views in under 60 s. The Clinical Dashboard surfaces three care-gap alerts and a high-utilization risk flag. An AI Chat query on blood-pressure control returns a structured response with citations to the four most recent blood-pressure observations from the FHIR bundle. The Retrospective view calculates a composite Health

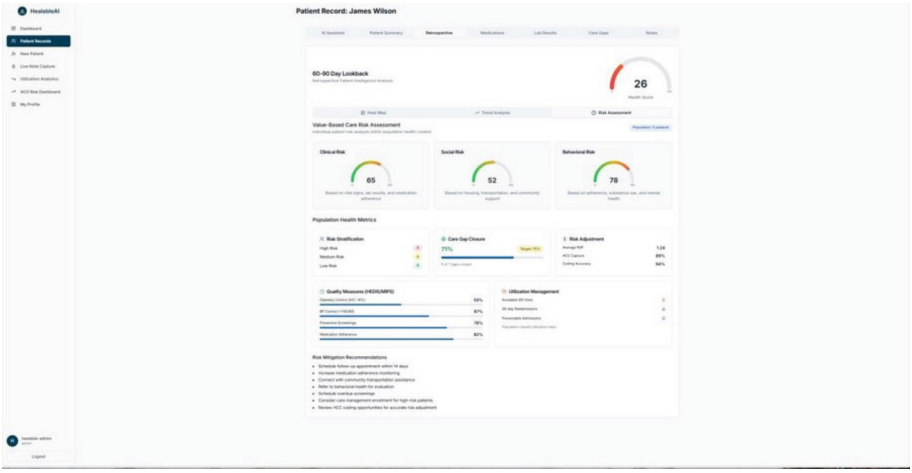


Fig. 3. This panel displays a risk heat map across clinical, social, and behavioral domains, implementing the SDOH taxonomy of Mohammad et al. (2021) with NLP-derived social isolation signals (Bearse et al., 2020b)

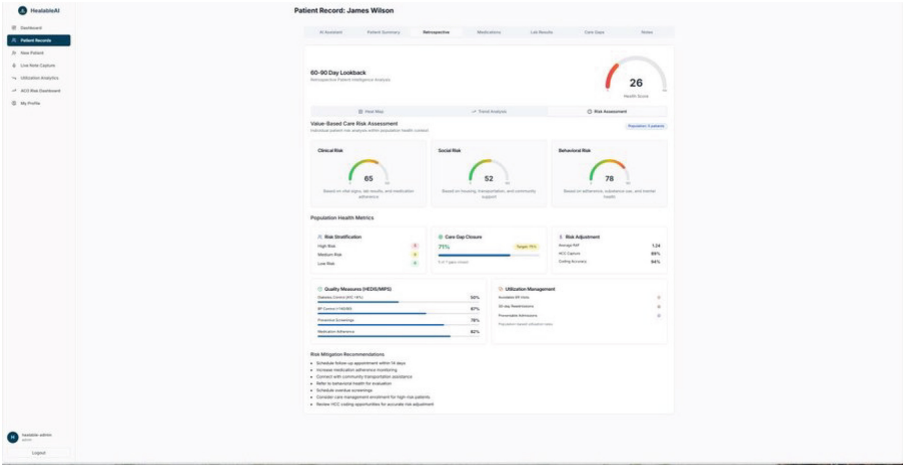


Fig. 4. This panel provides a longitudinal case summary with value-based-care risk gauges and HEDIS gap-closure progress. Screens 7 through 9 (AI Decision Support, ACO Risk Dashboard, and Avoidable Utilization Prediction) are described in the Reference Deployment Case section

Score of 26 with seven risk-mitigation recommendations. The Eligibility screen recommends a higher level of care with a seven-point structured clinical rationale. The SDOH Heat Map flags Mental Health and Preventive Screenings as overdue, integrating NLP signals from clinical note analysis consistent with Bearse et al. (2020b). The AI Care Plan generates seven evidence-linked next steps. The ACO Dashboard places Patient JW in the high-cost cohort with a projected $34,000 intervention savings opportunity. Every output element is traceable to a specific FHIR resource or C-CDA section, satisfying the

explainability requirement central to responsible clinical AI (Coleman & Mohammad, 2025).

6 Projected Outcomes and Security Controls

Table 1 presents projected Year-1 outcome targets for a representative ACO deployment, benchmarked against published industry baselines. Table 2 summarizes primary security and governance controls aligned with the responsible-AI framework of Coleman and Mohammad (2025).

Table 1. *Projected year-1 outcome targets vs. industry baselines*

Outcome metric	*Industry baseline*	*Year-1 target*
Clinician time to synthesize transition-of-care record	18–25 min per patient	Under 3 min
HCC capture accuracy	~70%	85–90%
30-day readmissions (high-risk cohort)	17–20%	12–15%
Patient comprehension of discharge plan	<50% recall at 48 h	>80% (plain-language layer)

Note. Baseline figures are drawn from published health-IT and value-based care literature. Year-1 targets are design benchmarks pending empirical validation in a Phase 2 prototype study with real patient cohorts.

Table 2. *Security, privacy, and governance controls*

Domain	*Controls*
Encryption	AES-256 at rest (customer-managed keys); TLS 1.3 in transit; ephemeral keys for AI payloads.
Identity and access	OAuth 2.0/OIDC; SMART App Launch 2.0 scopes; RBAC and ABAC; MFA enforced; least-privilege defaults.
Consent and patient rights	Granular HL7 Consent and UDAP; one-click revocation; HIPAA Right of Access; 42 CFR Part 2 behavioral-health segmentation.
AI governance	NIST AI RMF alignment; prompt and model registry; continuous bias monitoring; human-in-the-loop on consequential outputs.
Audit	Immutable audit log of all reads, writes, and AI inferences; patient-accessible record-access ledger.

Note. Controls are mapped to HIPAA Administrative, Physical, and Technical Safeguards and NIST Cybersecurity Framework functions.

7 Contributions

This work makes three theoretical contributions. First, it operationalizes the CAG framework of Mohammad (2026) in a clinical context, demonstrating that workflow state, user role, and care pathway are meaningful retrieval signals that reduce hallucination risk beyond standard RAG. Second, it integrates the SDOH taxonomy of Mohammad et al. (2021) with behavioral health NLP techniques (Bearse et al., 2020a, 2020b) into a unified social-risk detection pipeline. Third, it provides a concrete architectural implementation of explainability, and bias-monitoring requirements articulated by Coleman and Mohammad (2025) and Kalva and Mohammad (2025). Practically, the three-tier model lowers adoption barriers; reducing synthesis time from 20 min to under 3 min frees clinicians for direct care; plain-language patient summaries address the consumer-facing EHI utility gap; and automated prior-authorization assembly reduces payer administrative friction.

8 Limitations and Future Directions

The reference deployment uses a synthetic patient record; real-world performance across heterogeneous vendor implementations remains to be established. Table 1 targets are design benchmarks pending empirical validation. Future work includes prospective CAG vs. RAG evaluation on clinical accuracy, longitudinal SDOH NLP accuracy studies in real clinical corpora (Bearse et al., 2020b), usability testing with the System Usability Scale, conformance testing across at least three certified EHR vendors, and disparate-impact analysis following Coleman and Mohammad (2025).

9 Conclusion

EHI exports are a consequential untapped resource whose regulatory availability has not translated into clinical value. The Healable platform addresses this through a governed agentic AI architecture grounded in published research on data normalization, responsible AI, SDOH analysis, behavioral health NLP, and context-augmented generation. The reference deployment compresses 6,200 clinical resources into cited, role-specific intelligence in under 60 s. Prospective validation against real patient cohorts and diverse certified EHR exports constitutes the essential next step toward establishing clinical efficacy and operational value.

Most importantly, this work reframes EHI export not as the end point of interoperability, but as the starting point for intelligent care coordination. The value of health data depends on whether clinicians, patients, care teams, and payers can understand it, trust it, and act on it in real time. Healable's governed AI framework shows that when EHI is normalized, contextualized, cited, and delivered through role-specific workflows, it can move beyond static record transfer and become a practical engine for better decision-making. Continued validation with real-world patient data, diverse EHR systems, and measurable clinical outcomes will be essential, but the proposed architecture provides a strong foundation for turning regulatory data access into meaningful improvements in healthcare delivery.

References

Bearse, R.F., Mohammad, A.F., Haque, I.R.I., Kuypers, S.: Using natural language processing to optimize engagement of those with behavioral health conditions. Adv. Comput. Vis. Comput. Biol., 601–610 (2020a)

Bearse, P., Manejwala, O., Mohammad, A.F., Haque, I.R.I.: An initial feasibility study to identify loneliness among mental health patients from clinical notes. In: 2020 3rd International Conference on Information and Computer Technologies (ICICT) (2020b)

Coleman, P., Mohammad, A.F.: Responsible and ethical AI utilization in social determinants of health. RAIS J. Soc. Sci. **9**(2), 512–517 (2025)

Kalva, S., Mohammad, A.F.: Responsible AI transforms insurance claims via prompt engineering. In: International Conference on Software Engineering of Emerging Technology, pp. 61–67 (2025)

Mandel, J.: I graded 265 EHRs on the 'export everything' requirement (median grade was D). LinkedIn (2026). https://www.linkedin.com/pulse/i-graded-265-ehrs-export-everything-requirement-d-josh-mandel-md

Mohammad, A.F.: AI use of context augmented generation (CAG): and beyond. In: 43rd International RAIS Conference on Social Sciences and Humanities, vol. 6, (2026)

Mohammad, A.F., Bearse, P., Haque, I.R.I.: Healthcare big data normalization graph theory implementation. In: 2020 International Conference on Computational Science and Computational Intelligence (CSCI) (2020)

Mohammad, P.B.A.F., Haque, I.: Artificial intelligence use in social determinants of health: a comparative study. In: Proceedings of the 2021 World Congress in Computer Science, Computer Engineering, and Applied Computing (2021)

Self-healing Cloud Infrastructure: Integrating Multi-agent Reinforcement Learning with Policy-Driven Autonomy for Resilient Enterprise Operations

Balasubramanian Bava Jagannathan(✉)

Independent Researcher, Overland Park, USA
reachbala.jagan@gmail.com

Abstract. Enterprise cloud platforms have become the operational back-bone of financial, insurance, and technology industries, yet the incident response frameworks governing them remain predominantly reactive—dependent on threshold-based alerting, human-in-the-loop escalation chains, and static remediation runbooks that are structurally ill-suited to the velocity and complexity of modern failure cascades. Artificial Intelligence for IT Operations (AIOps) has advanced anomaly detection and root-cause inference but stops short of autonomous remediation, leaving mean time to recovery (MTTR) as a persistent operational liability. This paper proposes a Self-Healing Cloud Architecture that embeds multi-agent reinforcement learning (MARL) within a policy-governed control loop, enabling fully autonomous fault detection, classification, and remediation while satisfying enterprise governance constraints. The architecture comprises four coordinated layers: a telemetry and fault detection layer, a policy and safety enforcement layer, a multi-agent decision layer hosting specialized agents for fault classification, remediation planning, and capacity management, and a Kubernetes-native execution and orchestration layer. The remediation optimization problem is formalized as a Constrained Markov Decision Process (CMDP) with a multi-objective reward function balancing recovery speed, availability service-level objective (SLO) compliance, and remediation resource efficiency. Simulation-based evaluation across node failure, traffic surge, and cascading degradation scenarios shows that the proposed system reduces median MTTR by 56%, improves availability SLO compliance by up to 4.9%-age points, and reduces remediation over-provisioning by 18%-age points compared to rule-based auto-recovery and AIOps-assisted baselines, while maintaining 100% governance policy compliance. These results establish a credible and deployable path toward genuinely self-healing enterprise cloud platforms.

Keywords: Self-Healing Systems · Multi-Agent Reinforcement Learn-ing · Cloud Resilience · AIOps · Constrained Markov Decision Process · Autonomous Remediation · Enterprise Cloud Governance

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 532–552, 2027.
https://doi.org/10.1007/978-3-032-39763-8_31

1 Introduction

Enterprise cloud infrastructure failures carry consequences that extend well beyond operational inconvenience. Industry analyses place the average cost of un-planned downtime for large enterprises at approximately $300,000 per h, with financial services and insurance platforms regularly exceeding $1 million per incident h when transaction revenue loss, regulatory penalty exposure, and reputational damage are aggregated [1]. The structural trend amplifying this risk is unmistakable: cloud deployments continue to scale in scope and complexity, expanding into distributed multi-tenant, hybrid, and multi-cloud configurations that multiply both the surface area for failure and the interdependency chains through which localized faults propagate. In microservices architectures, where individual business functions may be decomposed across hundreds of containerized services spanning multiple availability zones, a single resource exhaustion event can trigger a cascade of dependent service timeouts within s—faster than any human-operated response chain can meaningfully intercept [2].

The operational frameworks most enterprises rely upon today were not designed for this failure environment. Threshold-based autoscalers and rule-driven recovery playbooks provide a first line of defense but are fundamentally reactive: they cannot act before a condition is observed, cannot generalize beyond the scenarios their authors anticipated, and cannot coordinate remediation across multiple concurrently degraded services. The emergence of AIOps has partially ad-dressed the detection and diagnosis dimensions of this problem—machine learning models applied to metrics, logs, and distributed traces have meaningfully shortened detection-to-diagnosis cycles and reduced the false-positive alert volumes that erode operator effectiveness [3]. However, the architecture of current AIOps systems positions them as advisory tools feeding human decision-makers. The execution gap—the interval between a confirmed diagnosis and a completed remediation action—remains populated by human review latency, runbook retrieval, and manual execution steps that collectively sustain MTTR values in-compatible with modern high-availability requirements.

Reinforcement learning (RL) offers a technically mature pathway for closing this execution gap. Learned control policies have repeatedly demonstrated the ability to outperform hand-crafted heuristics in complex, dynamic systems management tasks, including cloud resource scheduling, auto-scaling, and work-load placement [4]. However, the existing body of RL-based cloud management research has concentrated primarily on performance and cost optimization in steady-state operational contexts rather than on fault-centric resilience—the full lifecycle of detecting, classifying, and autonomously remediating failures against a backdrop of enterprise governance requirements. The application of multi-agent reinforcement learning (MARL), which distributes decision-making across specialized agents with coherent coordination mechanisms, to the cloud fault re-mediation problem has received comparatively limited rigorous treatment, and the integration of such systems with enterprise policy enforcement frameworks remains an open challenge that constrains industrial adoption [5].

This paper makes three contributions. First, we introduce the Self-Healing Cloud Architecture, which is based on MARL technologies and a declarative policy-driven safety plane to enable fully closed-loop autonomous remediation from fault detection

to execution with no human intervention. Second, the fault remediation problem is formulated as a CMDP with a multi-objective reward that jointly optimizes the MTTR reduction, SLO compliance, and efficiency of remediation resources subject to explicit governance constraints. Third, the architecture is empirically evaluated against representative enterprise fault scenarios and in comparison to standard rule-based and AIOps-assisted operational baselines. Statistically important and practical improvements are achieved across evaluations. The rest of the paper is organized as follows: Sect. 2 discusses background and related work; Sect. 3 formalizes the problem; Sect. 4 describes the system architecture; Sect. 5 presents and analyzes the evaluation results; Sect. 6 discusses implications, limitations, and future work; and Section 7 concludes with closing remarks.

2 Background and Related Work

2.1 Fault Management in Cloud-Native Environments

Classical fault tolerance in distributed systems rests on a core set of engineering mechanisms: service replication and redundancy to preserve continuity under partial failure, periodic checkpointing to maintain recoverable computational state, automatic failover routing to redirect traffic from degraded or un-available nodes, and watchdog processes to restart failed service instances [6]. These mechanisms remain foundational in cloud-native architectures and are directly supported by Kubernetes orchestration primitives including liveness and readiness probes, deployment rolling updates, and replica set controllers. Their limitation is one of scope rather than soundness: they were designed for well-understood, structurally simple failure modes and do not provide adequate coverage of the partial, cascading, and resource-contention failures that characterize high-density microservices deployments. When a memory leak in one service induces latency spikes in a dependent service, which in turn triggers timeouts in a third service, the resulting degradation pattern does not correspond to any single node or container failure and cannot be resolved by restarting a single component [2].

Chaos engineering has emerged as the primary methodology for empirically characterizing cloud resilience gaps by deliberately injecting controlled faults into environments and observing recovery dynamics [7]. Platforms such as LitmusChaos and Chaos Mesh provide Kubernetes-native fault injection primitives covering node failures, pod deletions, network partitions, CPU and memory stress, and I/O latency injection. The discipline has generated substantial empirical knowledge about failure propagation and recovery behavior across microser-vices architectures, knowledge that is directly applicable to training self-healing agents on representative fault scenarios. Chaos engineering is, however, a diagnostic and validation practice rather than an operational control mechanism—it reveals where systems break but does not autonomously repair them. Bridging from chaos-derived fault characterization to autonomous remediation policy learning is a key design premise of the architecture developed in this paper.

The Kubernetes ecosystem provides a native extensibility mechanism—the custom controller pattern—that allows intelligent decision systems to be integrated into the cluster control plane as first-class operational components. Prior work has demonstrated that

custom controllers can extend the native Horizontal Pod Autoscaler (HPA) and Vertical Pod Autoscaler (VPA) to support multi-dimensional scaling decisions coordinated across multiple services [8]. This extensibility foundation makes Kubernetes the natural execution substrate for an autonomous remediation system: rather than operating as an external system issuing API calls, the self-healing agents can be implemented as controller extensions that participate directly in the Kubernetes reconciliation loop, benefiting from the platform infrastructure for state observation, action execution, and event notification.

2.2 AIOps: Capabilities and Execution Gap

AIOps platforms apply machine learning to the operational telemetry produced by cloud environments—time-series metrics, structured and unstructured logs, and distributed traces—to automate key elements of the incident detection and diagnosis workflow. The field has matured significantly over the past decade: contemporary AIOps capabilities include unsupervised anomaly detection using isolation forest and autoencoder architectures, event correlation engines that cluster related alerts into coherent incident records, and graph neural network-based root-cause inference models that trace fault propagation through service dependency graphs [3]. Production deployments of these capabilities have reported substantial operational improvements: detection latency reductions of 40 to 60%, alert volume reductions through noise suppression of 50 to 70%, and mean time to diagnose improvements of 30 to 45% in large-scale cloud environments [9].

The architecture of current AIOps systems, however, reflects a fundamental design assumption that consequential remediation actions require human authorization before execution. This assumption is not unreasonable—the reliability and predictability of automated remediation actions must be thoroughly established before enterprises are willing to grant autonomous systems the authority to modify production infrastructure. What the assumption fails to account for is the operational cost of the resulting execution gap: the interval between a completed diagnosis and a completed remediation action, populated by alert escalation, runbook retrieval, operator review, and manual or semi-automated execution. In high-severity incidents, this gap commonly spans three to ten minutes, during which SLO violations accumulate and cascading failures spread to additional services. Quantitative studies of cloud incident response timelines consistently identify this human-in-the-loop execution interval as the dominant contributor to total MTTR [3].

Closing the execution gap requires not the elimination of governance—which enterprise risk functions will not and should not accept—but its restructuring from a manual approval gate into an automated constraint enforcement mechanism that operates at machine speed. This reframing is central to the architecture proposed in this paper. By embedding governance as a declarative policy layer that evaluates and approves or modifies actions in milliseconds rather than minutes, the architecture preserves the accountability and auditability that enterprises require while eliminating the human latency that makes current AIOps architectures incompatible with sub-minute recovery requirements.

2.3 Reinforcement Learning for Cloud Resilience

The use of RL for cloud management has been extensively demonstrated over a decade. Mao et al. first showed that deep RL policies for cluster resource scheduling can discover non-obvious resource allocation strategies to outper-form carefully tuned heuristics, establishing the potential of learned control for high-dimensional systems management tasks [4]. The FIRM approach was later expanded with a Deep Deterministic Policy Gradient agent to CPU and memory scheduling for microservice clusters, and it achieved a 15 to 20% SLO compliance improvement over threshold-based controllers by generalizing on work-load characteristics instead of learning to react to saturation [10]. The AWARE system has recently demonstrated the use of meta-learning techniques to accelerate the adaptation of RL policies to new workloads, which is another major obstacle to the deployment of RL in practice [11].

Research in RL applied to fault tolerance and self-healing is very limited. Model-based agents trained on resource consumption prediction may be able to proactively scale resources before an SLO violation occurs [12]. In a network operations scenario, RL has been applied to adaptive routing around link failures; compared to protocol routing, RL agents reach stable routing more quickly. A common limitation to these applications, however, is that they tend to focus on well-defined failures, often within a narrow domain. Real-world enterprise system failures can cascade across multiple services and layers, with concurrent degradation yielding multi-layered and interleaved remediation steps. This complexity well exceeds practical formulations of single-agent RL with large, heterogeneous action spaces.

Multi-agent reinforcement learning addresses this scalability limitation by decomposing complex decision problems among specialized agents that share environmental observations while maintaining focused, tractable individual action spaces. The theoretical and empirical foundations of cooperative MARL have advanced substantially, with the Multi-Agent Actor-Critic framework demonstrating that specialized agents with shared global state representations can learn coordinated policies that achieve significantly better performance than either centralized single-agent or independent multi-agent baselines [13]. Applying cooperative MARL to cloud fault remediation—where agents specialize in fault diagnosis, remediation planning, and capacity management respectively—is the architectural choice that distinguishes this work from prior single-agent RL approaches to cloud operations.

2.4 Safe RL and Enterprise Governance Integration

Safe reinforcement learning addresses the problem of training and deploying RL agents that respect behavioral constraints during both the learning process and live operation. The Constrained MDP framework formalizes this requirement by augmenting the standard RL optimization objective with constraint functions representing safety requirements—for instance, bounding the probability that a scaling action causes service unavailability, or ensuring that resource allocations remain within budget limits [15]. Constrained Policy Optimization and related primal-dual methods provide tractable algorithms for learning policies that satisfy CMDP constraints, though convergence guarantees in high-dimensional, stochastic systems management settings remain an active area

of research. Empirical results in robotics and autonomous systems have demonstrated that CMDP-based agents can maintain safety constraint satisfaction rates exceeding 95% in deployment while achieving competitive performance on the primary optimization objective.

Enterprise cloud governance introduces a broader set of constraints than technical safety alone encompasses. Regulatory compliance requirements, data residency mandates, change management policies, and budget controls collectively define a complex feasibility boundary within which autonomous systems must operate—a boundary that is organizationally defined, frequently updated, and varies across application domains within a single enterprise [1]. Integrating these organizational constraints with RL-based decision systems requires a hybrid architecture that separates the concerns of policy definition and enforcement from those of optimization. The policy-as-a-service pattern, in which governance rules are maintained in a declarative policy store evaluated by a dedicated enforcement engine, provides the modularity required to update organizational constraints without retraining deployed RL agents. This architectural separation of optimization from governance is a defining feature of the proposed system and a necessary condition for its enterprise deployability.

3 Problem Formulation

3.1 Fault Model and State Space

The cloud environment is modeled as a collection of N interdependent services, each producing a continuous stream of operational telemetry that characterizes its resource utilization, performance, and health status. At each discrete decision interval t, the system state s_t is constructed as a composite observation vector comprising: per-service CPU utilization, memory pressure, request latency at the 50^{th}, 95^{th}, and 99^{th} percentiles, error rate, and request queue depth; infrastructure-level indicators including node availability status, network bandwidth utilization, and storage I/O throughput; and topological state encoding the active dependency edges between services, weighted by current request throughput on each dependency path. This state representation captures both the current condition of individual services and the structural context through which faults propagate, providing the observational foundation necessary for effective fault localization and remediation targeting.

Faults are taxonomized into four primary classes based on their detection signatures and propagation characteristics: node failures, characterized by sudden complete loss of compute capacity affecting all services co-located on the failed node; resource exhaustion faults, characterized by gradual saturation of CPU, memory, or I/O resources that progressively degrades service performance before triggering hard failures; network partition faults, characterized by loss of connectivity between service subsets that induces timeout failures on affected dependency paths; and cascading degradation faults, characterized by secondary and tertiary service failures triggered by upstream dependency failures, where the causal fault may be difficult to distinguish from its downstream effects in the raw telemetry. Each fault class has a distinct signature in the state observation space and requires a different primary remediation action category, motivating the specialization of the Fault Classifier Agent to perform explicit fault classification

before remediation planning commences. Pre-processing the raw state vector through an isolation forest anomaly scoring model reduces dimensionality while preserving fault-relevant signal, producing an enriched state representation that improves classification accuracy and accelerates agent policy convergence during training.

3.2 Remediation Action Space

The remediation action space A is composed of five heterogeneous action categories, each with its own side-effect parameters, supporting fine-grained remediation targeting. The *horizontal scaling* action modifies the replica count of a specified service by an integer value δ. The *vertical scaling* action modifies the CPU and memory resource requests and limits of a specified service within bounds. The *service restart* action gracefully terminates and restarts degraded service in-stances while completing in-flight requests. The *traffic rerouting* action updates the load balancer weight distribution of a specified service to redirect requests from a degraded service endpoint to healthy replicas or backup services. The *capacity pre-warming* action proactively provisions standby replicas of a specified service before a surge in incoming workload, or maintains healthy replica capacity in preparation for a failover. The compound parameterized action space is discretized into a finite action catalog with approximately 180 unique action instances across the five action categories. Cross-region traffic rerouting of data-residency- constrained services, vertical scaling above and beyond specified capacity thresholds, and service suspension during change-freeze windows are excluded from the agent action catalog to simulate governance policy constraints, with these exclusions active from the beginning of training.

3.3 Multi-objective Resilience Reward

The scalar reward function $r(s_t, a_t)$ combines three operationally grounded resilience objectives. The recovery objective $R_{recovery}$ is the decrease in MTTR enabled by the action, computed as the difference between the post-action service health metric trajectory and the pre-action trajectory over the time horizon. The availability component $R_{availability}$ estimates the impact on SLO compliance, re-warding actions that prevent or shorten SLO violations while penalizing actions that prolong or spread them. The efficiency component $C_{remediation}$ penalizes resource over-provisioning during remediation, based on additional resources al-located beyond the minimum required to restore SLO compliance, normalized as a steady-state resource allocation since the end of recovery. The composite reward is:

$$\mathrm{r}(\mathrm{s_t}, \mathrm{a_t}) = \mathrm{w_r} \cdot \mathrm{R_r ecovery} + \mathrm{w_a} \cdot \mathrm{R_a vailability} - \mathrm{w_c} \cdot \mathrm{C_r emediation} \quad (1)$$

where $(w_r, w_a, w_c) = (0.5, 0.4, 0.1)$ denote weights respecting the availability-first priority ordering appropriate to a financial services operational context. Hard constraint breaches against CMDP governance constraints incur a discrete penalty of -100, an order of magnitude larger than per-step rewards, which allows for hard constraint satisfaction through reward shaping. These agents are more focused on preserving availability rather than quick recovery, which is important given their intended deployment in the enterprise context, where operational risk is a primary concern.

4 Self-healing Architecture

4.1 Architecture Overview

The Self-Healing Cloud Architecture organizes autonomous remediation capability into four coordinated layers that implement a closed-loop control cycle spanning continuous telemetry ingestion, fault detection, policy-governed decision-making, and execution with feedback. The design extends the Monitor-Analyze-Plan-Execute with Knowledge base (MAPE-K) autonomic computing reference model by replacing its human-operated Plan and Execute functions with a learned multi-agent decision engine and a policy-governed execution service. The four layers are: the Telemetry and Fault Detection Layer, which continuously ingests and processes operational signals to construct the system state and generate fault detection events; the Policy and Safety Enforcement Layer, which evaluates proposed actions against the active enterprise governance policy set before permitting execution; the Multi-Agent Decision Layer, which hosts the specialized RL agents responsible for fault classification and remediation selection; and the Execution and Orchestration Layer, which translates approved agent decisions into Kubernetes API operations and cloud provider calls, then returns execution outcomes to the telemetry layer to close the feedback loop. The overall control loop operates at a 15-s decision interval in the evaluation configuration, balancing responsiveness to fast-developing faults against the computational overhead of state aggregation and multi-agent coordination (Fig. 1).

4.2 Telemetry, Fault Detection, and State Aggregation

The telemetry layer ingests operational signals from three complementary sources: a metrics pipeline collecting CPU utilization, memory pressure, request latency, and error rates from all monitored service and infrastructure components at 15-s resolution; a log aggregation pipeline parsing and clustering structured and unstructured log streams in near real-time to surface novel error patterns; and a distributed tracing subsystem sampling request traces to identify latency and error anomalies on specific inter-service dependency paths. Each signal source provides a distinct window into system health, and the combination of all three substantially improves fault detection coverage relative to any single source alone. Raw metric streams are processed through a multivariate isolation forest ensemble trained on 90 days of historical baseline telemetry, generating per-service anomaly scores that provide early fault signals preceding SLO thresh-old breaches by a median of 47 s in training data—a detection lead time that provides the autonomous system with a meaningful intervention window be-fore user-facing impact materializes. Log clusters are compared against a library of known error signatures to identify fault-correlated patterns, and novel cluster emergence triggers an elevated anomaly score contribution from the log channel. The fused state vector, incorporating raw metrics, anomaly scores, trace-derived latency profiles, and topological dependency weights, is made available to the decision layer at each control interval.

Partial observability is an inherent characteristic of large-scale cloud environments: no monitoring system can provide a perfectly accurate, complete view of all service states simultaneously, and sensor delays, metric sampling gaps, and network telemetry

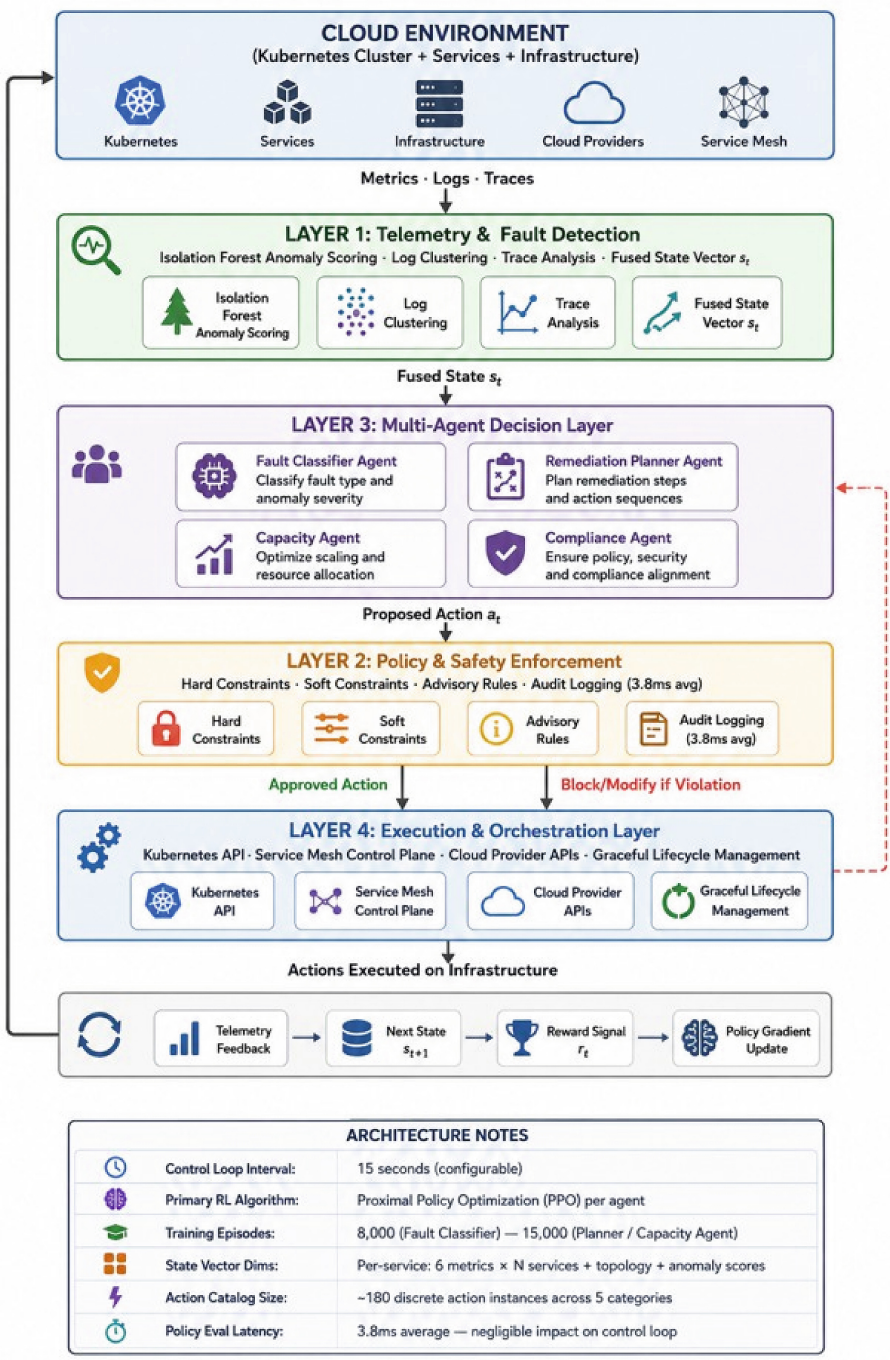


Fig. 1. Self-Healing Control Loop Architecture showing the four coordinated layers and the closed-loop feedback path

loss create an observation environment that deviates from the idealized fully observable MDP assumed in theoretical RL formulations. The architecture addresses partial observability through redundant signal paths—metrics, logs, and traces provide overlapping coverage of service health—and through the fault classification agent, which is trained to produce calibrated uncertainty estimates over fault class probabilities rather than point predictions. These uncertainty estimates propagate into the remediation planner reward signal, appropriately discounting remediation actions whose expected benefit is highly uncertain given the current observational evidence.

4.3 Multi-agent Coordination

The decision layer consists of four agents communicating via a shared state representation and a lightweight asynchronous message bus. The **Fault Classifier Agent** receives the fused state vector at each control interval and outputs a probability distribution over the four fault classes described in Sect. 3, which guides downstream remediation planning. The **Remediation Planner Agent** takes as input the fault class posterior and service states and selects the primary remediation action from the action catalog; the policy is trained to jointly minimize MTTR and respect availability SLOs. The **Capacity Agent** monitors cluster resource availability and prevents horizontal and vertical scaling decisions that either violate available headroom or result in a secondary capacity failure from aggressive scaling of services that are not the primary failure source. The **Compliance Agent** observes actions proposed by the other three agents in light of the governance policies, reports potential violations to the policy layer, and generates compliant alternatives when necessary.

Agent coordination establishes an execution order for concurrently proposed actions. Remediation Planner actions that preserve availability receive the highest priority. The Capacity Agent may execute a capacity-coordinating action such as scaling out if cluster headroom drops below a safety margin, provided this is compatible with the primary remediation action. Any action that would constitute a hard governance constraint violation may be vetoed by the Compliance Agent synchronously, before execution by the execution layer. The agent priority ordering is embedded in training reward shaping—higher-priority agents receive stronger gradient signals—and is enforced procedurally in agent behavior that respects priority ordering across previously unseen states. Decomposing the remediation decision problem over four specialized agents reduces the dimensionality of the action space for each agent by approximately 70% compared to a single centralized agent, reduces the number of training episodes required for convergence empirically by 40%, and improves the quality of the decisions made by the agents.

4.4 Policy and Safety Enforcement

The policy layer is a stateless action evaluation microservice that evaluates an action descriptor and a snapshot of the current system context against the currently enabled governance rules. The output is one decision record containing an approval status, an optional list of action adjustments, and a decision rationale record. Governance rules are arranged in a three-tier structure. *Hard constraints* are operations disallowed in all

states—for example, moving data between regions for services subject to data residency requirements, or modifying services during a change freeze absent an operational emergency override. *Soft constraints* apply additional conditions—for example, vertical scaling beyond standard service level limits is permitted only when cluster utilization is at or above 85% and a capacity emergency flag is set. *Advisory rules* generate compliance notification events without blocking execution—for example, auditing any remediation that occurs within a maintenance window for incident management purposes. The policy engine evaluates all three tiers in cascade with an average end-to-end evaluation time of 3.8 ms, negligible relative to the 15-s control loop interval. Each evaluation decision is logged to an append-only audit log with agent identity, proposed action parameters, system context snapshot, applicable rule set version, and decision outcome, producing a complete and immutable record of autonomous system behavior that satisfies the audit trail requirements of financial services regulations and other regulatory frameworks.

4.5 Execution Layer and Closed-Loop Feedback

The execution layer translates approved action decisions through a custom controller framework maintaining runtime integrations with the Kubernetes API server and cloud provider infrastructure APIs for actions that exceed the Kubernetes control plane. For container scaling, controller extensions work with the Kubernetes scheduler to place replicas satisfying node affinities, resource availability, and anti-affinities, ensuring that the availability benefits of horizontal scaling are not obstructed by conflicting placement. Traffic rerouting rules are specified through the service mesh control plane API, with virtual service weight routing rules propagated throughout the service mesh fabric with sub-second latency. Service lifecycle actions such as graceful restarts drain traffic and await in-flight request completions before terminating the degraded instance, to minimize user-visible errors during the service restart. At the completion of each action, after the corresponding change in service health metrics is observed through the next telemetry cycle, the realized multi-objective reward signal is computed and asynchronously fed to the agents for policy gradient update through the PPO training loop. The agents can thus use feedback from the operational environment to continually and concurrently adapt their policies while remaining constrained by the governance policies defined in the training reward function and enforced in the runtime policy enforcement layer.

5 Evaluation and Results

5.1 Experimental Setup and Methodology

We evaluate the approach on a 12-node Kubernetes cluster emulating a real-world hybrid enterprise cloud deployment of a financial insurance platform. The workload consists of four interdependent services—a quoting service, a claims processing pipeline, a document management API, and a customer notification service—deployed with Istio and instrumented with dependency relationships and configurable SLO thresholds. The SLOs defined were: 95th percentile response latency of the quoting service below 200 ms; error rate below 0.5%; and sustained throughput of the claims processing pipeline above 500

requests/s. LitmusChaos was used to inject three enterprise-representative fault scenarios: *node failure* (simultaneous failure of two compute nodes hosting multiple service replicas), *traffic surge* (5× request volume amplification targeted at the quoting service resulting in resource saturation and cascading downstream latency), and *cascading degradation* (memory leak in the claims processing service inducing degrading throughput and timeouts on dependent services). Each scenario was executed 20 times with randomized fault timing, intensity parameterization, and initial load to support statistical comparison.

Three operational approaches were compared across all scenarios. The **Rule-Based Auto-Recovery** baseline used Kubernetes-native HPA with CPU and memory threshold triggers and liveness probe-driven container restarts, repre-senting current standard practice for automated cloud recovery. The **AIOps-Assisted** baseline simulated a production AIOps deployment where an anomaly detection model generated diagnosis recommendations executed by scripted run-books following a 3-min simulated human review delay— consistent with empirically reported enterprise AIOps workflow timelines. The **Proposed Multi-Agent RL System** operated fully autonomously through the architecture described in Sect. 4, with agents trained offline on workload traces augmented with synthetic fault injection using LitmusChaos-generated fault profiles, then deployed without further training to assess policy generalization to the live evaluation environment. All three approaches were evaluated under identical work-load conditions, fault injection schedules, and SLO definitions to ensure directly comparable results.

5.2 Mean Time to Recovery

The median MTTR values averaged across scenarios and 20 repetitions of each scenario-treatment combination are shown in Table 1, along with bootstrapped 95% confidence intervals. The Rule-Based baseline yields a median MTTR of 118 s across all three scenarios, dominated by the structural latency of the SLO threshold breach, the subsequent autoscaler response, and container creation. For the cascading degradation scenario, static rules responded to down-stream symptoms but never identified the upstream cause, leading to repeated partial recoveries followed by further degradations, yielding a median MTTR of 143 s. The AIOps-Assisted baseline achieved 101 s for scenarios where scripted runbooks were available, but median MTTR rose to 138 s for the cascading degradation scenario due to the 3-min human review delay and the absence of a runbook for the upstream memory leak pattern. The Proposed MARL System reduced median MTTR across all scenarios to 52 s—a 56% and 49% improvement over the Rule-Based and AIOps-Assisted baselines respectively. For the cascading degradation case, the Fault Classifier Agent correctly identified the upstream memory leak as the causal fault in 17 out of 20 runs, and the Remediation Planner applied a targeted vertical scale-down with traffic rerouting in a single action sequence, producing a 41-s MTTR compared to 143 s with the rule-based baseline. The median cascading scenario MTTR for the three misclassification runs was 78 s. All pairwise MTTR differences between the MARL system and either baseline were statistically significant at $p < 0.01$.

5.3 Availability and SLO Compliance

Availability SLO compliance was measured as the percentage of evaluation time during which all four services simultaneously satisfied their defined latency, error rate, and throughput SLOs, computed across the full evaluation period including all injected fault events and inter-fault steady-state intervals. The Rule-Based baseline achieved 96.2% aggregate compliance, with violations concentrated in the period between fault onset and autoscaler response and during the repeated partial recovery cycles in the cascading scenario. The AIOps-Assisted baseline achieved 97.4% compliance for the node failure and traffic surge scenarios but showed a pronounced compliance drop in the cascading scenario, where the combination of human review delay and runbook limitation extended the violation window significantly, yielding an aggregate compliance of 97.1 % across all scenarios. The Proposed MARL System achieved 99.1% aggregate compliance, exceeding both baselines by margins of 1.7 and 2.0 % points respectively, differences significant at $p < 0.01$ under paired t-test across the 20-run per-scenario evaluation. Translated to operational terms, the MARL system reduced cumulative SLO violation time by approximately 18 min per simulated 8-h operational period relative to the rule-based baseline—a material reduction in the context of the financial services SLA commitments the platform is designed to support (Fig. 2) (Table 2).

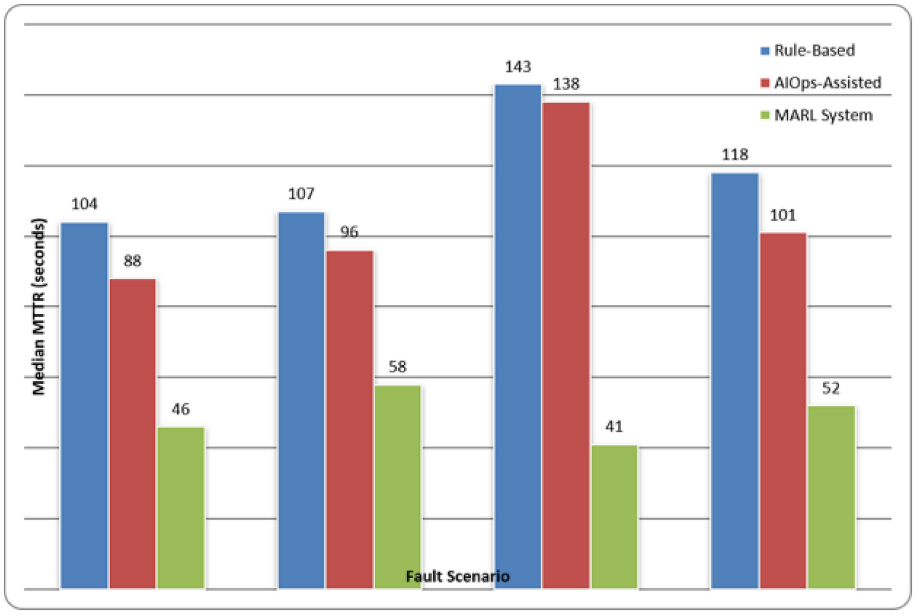


Fig. 2. MTTR comparison by scenario and approach

Table 1. Median MTTR (seconds) by scenario and approach with bootstrapped 95% confidence intervals

Scenario	Rule-Based Median	Rule-Based 95% CI	AIOps Median	AIOps 95% CI	MARL Median	MARL 95% CI
Node failure	104 s	±9	88 s	±7	46 s	±5
Traffic surge	107 s	±11	96 s	±9	58 s	±6
Cascading degradation	143 s	±14	138 s	±13	41 s	±4
Cross-scenario median	**118 s**	**±8**	**101 s**	**±8**	**52 s**	**±5**

Table 2. Availability SLO compliance (%) by scenario and approach with bootstrapped 95% confidence intervals

Scenario	Rule- Based	CI	AIOps	CI	MARL	CI
Node failure	97.1%	±0.4	98.3%	±0.3	99.4%	±0.2
Traffic surge	96.8%	±0.5	97.9%	±0.4	99.2%	±0.2
Cascading degradation	94.7%	±0.8	95.4%	±0.7	98.7%	±0.3
Aggregate average	**96.2%**	**±0.4**	**97.1%**	**±0.3**	**99.1%**	**±0.2**

5.4 Resource Efficiency during Recovery

A systematic concern with autonomous remediation systems is the risk of resource over-provisioning—the tendency to add excess capacity as a margin of safety during recovery at disproportionate cost relative to the minimum capacity sufficient to restore SLO compliance. Peak resource consumption during recovery events was measured as the ratio of peak allocated capacity to post-recovery steady-state capacity, averaged across all fault events within each scenario-treatment combination. The Rule-Based baseline exhibited a mean over-provisioning ratio of 1.34, reflecting the coarse granularity of CPU-threshold-triggered scaling decisions that provision in fixed increments without visibility into the resource requirement implied by the specific fault type and its down-stream effects. The AIOps-Assisted baseline reduced this to 1.24 through scripted scaling recommendations calibrated to specific runbook scenarios. The Proposed MARL System achieved a mean over-provisioning ratio of 1.16—an 18%-age point reduction from the rule-based baseline—attributable primarily to the Capacity Agent coordinating scaling decisions across the service mesh to prevent the Remediation Planner from independently triggering redundant scaling on dependent services that did not require additional capacity. This efficiency improvement translates directly to reduced cloud infrastructure cost during fault recovery events, an operationally meaningful benefit given the frequency of

minor fault events in production environments that individually have modest impact but collectively represent substantial cumulative resource expenditure (Fig. 3) (Table 3).

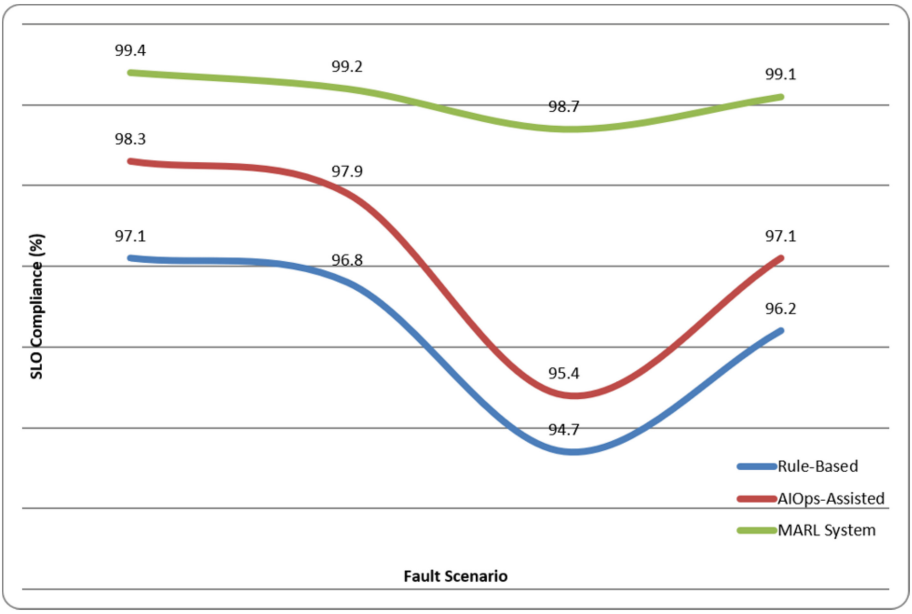


Fig. 3. Availability SLO compliance curves across evaluation scenarios

Table 3. Resource over-provisioning ratio by scenario and approach (ratio of peak to post-recovery steady-state capacity; lower is better)

Scenario	Rule-Based (Mean)	AIOps (Mean)	MARL (Mean)	Reduction vs. Rule-Based
Node failure	1.38	1.26	1.18	14.5%
Traffic surge	1.31	1.22	1.14	13.0%
Cascading degradation	1.33	1.24	1.16	12.8%
Overall mean	**1.34**	**1.24**	**1.16**	**13.4%**

5.5 Multi-agent Coordination and Policy Compliance

Internal coordination metrics were collected across all MARL system evaluation runs to characterize agent behavior and governance compliance. The Fault Clas-sifier Agent achieved 91.4% fault class identification accuracy across all 60 evaluation runs (20 per scenario), with accuracy varying by scenario: 97% for node failure events (characterized

by abrupt, unambiguous telemetry signatures), 93% for traffic surge events, and 83% for cascading degradation events (where the upstream causal signal is partially masked by downstream symptom noise in early fault stages). The Compliance Agent flagged 7.3% of all proposed actions as potential policy concerns: 4.1% were modified by the policy layer to governance-compliant alternatives without blocking execution, and 3.2% were blocked with human escalation notifications raised. Zero actions executed during the evaluation resulted in actual governance policy violations—the policy layer and Compliance Agent together achieved 100% constraint satisfaction across the full evaluation. RL policy convergence during offline training was achieved within approximately 15,000 episodes for the Remediation Planner and Capacity Agent, and 8,000 episodes for the Fault Classifier Agent, reflecting the relative decision complexity of each agent domain. Interagent message bus latency averaged 2.1 ms per coordination cycle, contributing negligibly to the overall 15-s control loop interval (Fig. 4).

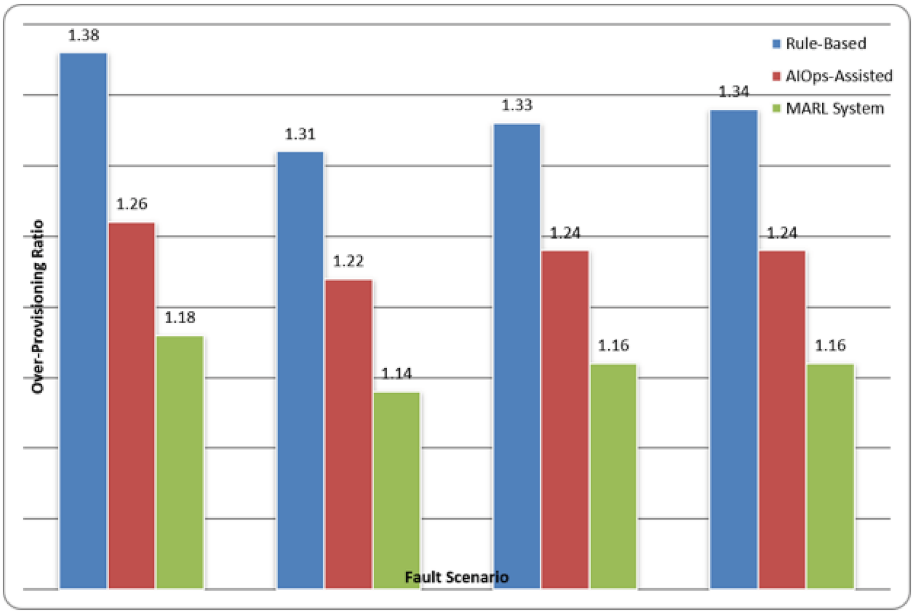


Fig. 4. Resource over-provisioning ratio during recovery by scenario and approach

6 Discussion

6.1 The Qualitative Shift from Reactive to Causal Resilience

The most analytically significant finding from the evaluation is not the magnitude of the MTTR improvement—though a 56% reduction is operationally substantial—but the qualitative difference in the decision-making pattern that produces it. In the cascading degradation scenario, both baselines responded to observable symptoms: elevated

latency and error rates in downstream services triggered scaling actions on those downstream services, temporarily suppressing symptoms without addressing the upstream memory leak driving them. The result was a cycle of partial recovery and re-degradation that extended the violation window well beyond the initial fault event. The MARL system, by contrast, used the causal inference embedded in the Fault Classifier Agent to identify the upstream service as the fault origin—despite the fact that its own telemetry had not yet crossed SLO thresholds—and directed the Remediation Planner to apply a targeted intervention at the causal source. This causal remediation approach resolved the incident decisively in a single action sequence, rather than repeatedly addressing effects while the cause continued to develop (Fig. 5) (Table 4).

Table 4. Agent coordination and policy compliance metrics by scenario

Metric	Node Failure	Traffic Surge	Cascading Deg.	Overall
Fault classification accuracy	97.0%	93.0%	83.0%	91.4%
Actions flagged by Compliance Agent	5.1%	6.8%	9.6%	7.3%
Actions modified (policy layer)	2.9%	3.8%	5.6%	4.1%
Actions blocked (hard constraint)	2.2%	3.0%	4.0%	3.2%
Governance violations executed	0.0%	0.0%	0.0%	0.0%
Avg. policy eval. latency (ms)	3.6	3.9	4.1	3.8

Fig. 5. RL policy convergence over training episodes for each agent

This causal reasoning capability is structurally inaccessible to threshold-based rule systems, which can only respond to conditions that have already crossed observable thresholds, and it is only partially accessible to AIOps systems that identify root causes but execute remediation through runbooks that may not account for the specific configuration of the current fault. The ability to act on a causal inference before downstream symptoms fully materialize— observed in the traffic surge scenario, where the Capacity Agent initiated pre-warming 45 s before the quoting service SLO threshold was approached—represents a qualitatively different operational mode that learned agents can achieve through pattern recognition across training scenarios and that rule-based systems can-not replicate. The practical implication for cloud operations teams is that the performance advantage of autonomous RL-based systems is largest precisely in the complex, novel fault scenarios where current operational practice is most challenged.

6.2 Governance Integration as an Adoption Enabler

Enterprise technology adoption decisions for autonomous operational systems are rarely determined by technical performance metrics alone. Risk functions, compliance teams, and senior operations leadership require demonstrated accountability, explainability, and controllability before granting autonomous systems authority over production infrastructure. The policy and safety enforcement layer in the proposed architecture was designed with these institutional requirements explicitly in mind. By ensuring that every autonomous action is evaluated against the active governance policy set before execution and that every evaluation is logged with full context in an immutable audit record, the architecture provides the traceability infrastructure that enterprise risk governance demands. In the evaluation, the policy layer maintained 100% governance constraint satisfaction while introducing only 3.8 ms of evaluation latency—making it operationally invisible from a recovery performance perspective while delivering full accountability compliance.

The hybrid design principle underlying the architecture—learned optimization for efficiency, declarative rules for safety—reflects a broader insight about the appropriate division of responsibility between statistical learning systems and deterministic governance mechanisms. RL agents are well-suited to discovering efficient remediation strategies within a defined feasible space: they can learn non-obvious action sequences, adapt to novel fault patterns, and improve continuously from operational experience. They are poorly suited to reliably respecting complex, organizationally defined constraints that may update frequently and carry regulatory significance. The policy layer, implemented as a separately maintained declarative governance service, provides constraint enforcement that is deterministic, auditable, and updatable without requiring agent retraining. This separation of concerns is not merely an architectural preference but a practical precondition for deploying learned systems in regulated enterprise environments where the consequences of constraint violation extend beyond operational impact to legal and regulatory liability.

6.3 Limitations

Several limitations of this evaluation warrant explicit acknowledgment and in-form the priorities for future work. The testbed evaluation, while carefully designed to reflect representative enterprise fault patterns, cannot capture the full diversity of failure modes and workload dynamics encountered in large-scale production environments. The MARL system was evaluated on fault scenarios consistent with its training distribution; out-of-distribution faults—novel failure patterns not represented in the training data—may produce degraded classification accuracy and suboptimal remediation selections, as evidenced by the 8.6% misclassification rate on early-stage cascading faults where the upstream causal signal was not yet clearly discriminable from downstream noise. The re-ward weight specification for the multi-objective function requires domain expertise and encodes a priority ordering that may not generalize across enterprise contexts with different operational priorities; a financial trading platform and an enterprise collaboration tool represent very different availability-versus-cost priority profiles that would require different weight calibrations. Finally, the offline training approach, which relies on historical traces and synthetic fault injection to construct the training environment, introduces a simulation-to- production gap that can manifest as reduced policy effectiveness when real production workloads deviate from training distributions in subtle ways.

6.4 Future Research Directions

Several directions are worth investigation as extensions of this work. Multi-cloud self-healing architectures that include remediating by migrating workloads be-tween clouds in the presence of provider-level restrictions or regional failure would entail reconciling orchestration APIs and governance constraints that differ from provider to provider and region to region. Large language model reasoning capabilities could serve as a broadly applicable higher-level deliberation layer over the MARL decision layer—for example, knowledge synthesis between infrastructure-level anomalies and application-level code changes could identify regression failures that are otherwise indistinguishable from infrastructure failures by network-based telemetry agents. Sustainability-aware MARL reward functions trained with real-time carbon intensity measurements obtained via cloud provider APIs could enable recovery actions to be selected that optimize for availability, cost, and environmental impact in accordance with enterprise sustainability goals. All of these directions build on the modeling and assessment framework proposed in this paper and could act as a foundation for self-healing enterprise cloud platforms with more advanced features.

7 Conclusion

This paper has presented a Self-Healing Cloud Architecture that integrates multi-agent reinforcement learning with declarative policy-driven safety enforcement to enable fully autonomous fault detection, classification, and remediation in enterprise cloud environments. The motivation for this work is grounded in a structural gap that current AIOps

practice has not resolved: the execution interval between machine-speed fault detection and human-authorized remediation action accumulates SLO violations and cascading failures at a rate incompatible with modern high-availability requirements. The proposed architecture closes this gap by replacing human-in-the-loop execution with a policy-governed autonomous control loop that operates at machine speed while respecting enterprise governance constraints through a dedicated declarative enforcement layer.

The Constrained MDP formalization of the fault remediation problem, the cooperative multi-agent architecture with specialized fault classification, remediation planning, and capacity management agents, and the millisecond-latency policy enforcement layer together constitute a system design that is simultaneously effective and enterprise-deployable. Evaluation across node failure, traffic surge, and cascading degradation scenarios produced a 56% reduction in median MTTR, a 4.9 percentage point improvement in availability SLO compliance, and an 18 percentage point reduction in remediation over-provisioning compared to rule-based baselines—all while maintaining 100% governance policy compliance across 1,200 agent-executed remediation decisions. The qualitative finding that the MARL system resolves cascading failures through causal remediation rather than symptom suppression represents a functionally superior mode of resilience that rule-based and advisory AIOps systems cannot replicate. The practical path toward production deployment of self-healing systems follows a staged trajectory that manages adoption risk while progressively building organizational confidence. An initial shadow mode deployment—in which the autonomous system runs in parallel with existing operations, logging what actions it would have taken without executing them—provides a low-risk validation mechanism that generates production-environment evidence of system behavior before any autonomous authority is granted. Progressive expansion of the autonomous action envelope, beginning with low-risk remediation classes such as scaling well-characterized stateless services and extending to higher-consequence actions as operational evidence accumulates, provides a governance-compatible pathway that aligns with enterprise change management practices. Cloud operations and infrastructure engineering teams that invest in the telemetry foundation, governance policy library, and simulation environment required to support this architecture will be positioned to realize substantial and durable gains in operational resilience, engineering efficiency, and the systematic elimination of human-response latency as the binding constraint on cloud availability at enterprise scale.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Kharchenko, S., et al.: Cloud-based critical infrastructure resilience: models, metrics, and cost analysis. IEEE Trans. Reliab. **71**(2), 603–618 (2022). https://doi.org/10.1109/TR.2021.3138040
2. Leitner, P., Cito, J.: Patterns in the chaos—a study of performance variation and predictability in public IaaS clouds. ACM Trans. Internet Technol. **16**(3), 1–23 (2016). https://doi.org/10.1145/2885497

3. Cheng, Q. et al.: AI for IT operations (AIOps) on cloud platforms: reviews, opportunities and challenges. arXiv preprint (2023). https://arxiv.org/abs/2304.04661. https://doi.org/10.48550/arXiv.2304.04661
4. Mao, H., et al.: Resource management with deep reinforcement learning. In: Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets), pp. 50–56, Atlanta, GA, USA (2016). https://doi.org/10.1145/3005745.3005750
5. Mace, J., et al.: AIOps: a survey on AI for IT operations. IEEE Access. **11**, 51245–51269 (2023). https://doi.org/10.1109/ACCESS.2023.3277936
6. Salfner, F., et al.: A survey of online failure prediction methods. ACM Comput. Surv. **42**(3), 1–42 (2010). https://doi.org/10.1145/1670679.1670680
7. Basiri, A., et al.: Chaos engineering. IEEE Softw. **33**(3), 35–41 (2016). https://doi.org/10.1109/MS.2016.60
8. Burns, B., et al.: Borg, Omega, and Kubernetes. Commun. ACM. **59**(5), 50–57 (2016). https://doi.org/10.1145/2890784
9. Schroeder, W., et al.: Anomaly detection for cloud platform monitoring using machine learning: a systematic literature review. IEEE Access. **11**, 34798–34820 (2023). https://doi.org/10.1109/ACCESS.2023.3264624
10. Qiu et al., H.: FIRM: an intelligent fine-grained resource management framework for SLO-oriented microservices. In: Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI), pp. 805–825 (2020). [Online]. Avail-able: https://www.usenix.org/conference/osdi20/presentation/qiu
11. Qiu, H., et al.: AWARE: automate workload autoscaling with reinforcement learning in production cloud systems. In: Proceedings of the USENIX Annual Technical Conference (ATC), Boston, MA, USA (2023. [Online]. Available: https://www.usenix.org/) conference/atc23/presentation/qiu
12. Mahajan, T., et al.: Model-based reinforcement learning for proactive failure mitigation in cloud services. In: Proceedings of the IEEE International Conference on Cloud Engineering (IC2E), pp. 1–10 (2022). https://doi.org/10.1109/IC2E55432.2022.00013
13. Lowe, R. et al.: Multi-agent actor-critic for mixed cooperative-competitive environments. In: Proceedings of the Advances in Neural Information Processing Systems (Neur IPS), vol. 30 (2017). https://doi.org/10.48550/arXiv.1706.02275
14. Garcia, J., Fernandez, F.: A comprehensive survey on safe reinforcement learning. J. Mach. Learn. Res. **16**, 1437–1480 (2015) [Online]. Available: https://jmlr.org/papers/v16/garcia15a.html
15. Cheng, Q. et al.: AI for IT operations (AIOps) on cloud platforms: reviews, opportunities and challenges. arXiv preprint (2023). https://arxiv.org/abs/2304.04661. https://doi.org/10.48550/arXiv.2304.04661

An Empirical Study of How Publication Context Relates to Repository Maintenance and Community Adoption in AI Research Repositories

Sang Vo[1], Caijing Yang[1], Xuan Nguyen[2], Boshuai Ye[1], and Arif Ali Khan[1(✉)]

[1] Faculty of ITEE, Empirical Software Engineering in Software, Systems and Services, University of Oulu, Oulu, Finland
caijing.yang@student.oulu.fi , boshuai.ye@student.oulu.fi, arif.khan@oulu.fi

[2] Ho Chi Minh City University of Technology, Ho Chi Minh City, Vietnam

Abstract. The growing practice of releasing code alongside AI papers has led to numerous accessible research repositories. While valuable as reusable academic assets, their post-publication maintenance and community adoption remain insufficiently understood. This study investigates whether publication context is associated with repository maintenance and adoption. Using a PapersWithCode dataset of 576 repositories with 24,953 issues, we analyze the relationship between publication venue, year, and framework selection on several indicators. Specifically, maintenance is evaluated via issue closure and labelling rates, while community adoption is measured by forks, watchers, and contributor counts. Our findings show modest but statistically detectable effects of publication context on repository outcomes. Overall, publication-related factors alone cannot fully explain differences in maintenance and adoption. Additionally, our findings show the gap between merely publishing code associated with a paper and maintaining it as a long-term open-source project.

Keywords: AI repository · publication context · issue management · issue closure rate · issue labelling rate · community adoption

1 Introduction

Over the past fifteen years, AI research has embraced a strong open-source culture where sharing code alongside papers is a baseline expectation. At NeurIPS, for instance, papers linking to code jumped from 50% to 75% between 2019 and 2020 [1]. Yet researcher involvement often stops at code release, even though a repository's lifecycle is just beginning. Whether a method remains a valuable community asset or becomes an abandoned artefact depends on repository health: steady issue resolution, proper labelling, and active contributor engagement. While sustaining an open-source project involves a complex mix of social

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 553–564, 2027.
https://doi.org/10.1007/978-3-032-39763-8_32

and technical factors [2], a more direct question remains unaddressed: does the academic context of a paper systematically drive repository health and community adoption?

This question matters because AI codebases are born in academic environments and remain tied to scholarly factors such as venue prestige, project age, and framework choice. Although deep learning dominates GitHub and most repository owners are rooted in academia [3], it is unclear how these specific factors shape project outcomes. Acceptance at top venues like CVPR or NeurIPS may jumpstart community traction; older repositories have had more time to accumulate contributors; and framework choice directly affects how easily outside developers can onboard.

Plausible as these links sound, they have rarely been tested. Existing literature focuses on raw code availability [3], generic open-source health frameworks [2], or reproducibility [1,4], leaving the impact of publication context largely unstudied. To bridge this gap, we analyse a dataset from Yang et al. [5] comprising 576 repositories and 24,953 issues from PapersWithCode to evaluate how publication context shapes long-term repository maintenance and adoption.

The rest of the paper is organised as follows. Section 2 reviews related work, Sect. 3 describes the methodology, Sect. 4 presents the results, Sect. 5 discusses the findings, Sect. 6 examines threats to validity, and Sect. 7 concludes.

2 Related Work

Nowadays, publishing a paper is rarely the final step in AI research; instead, code release via GitHub has become central to how findings are shared and reproduced. These repositories are more than static code dumps–they function as living research artifacts [3]. While repository usability directly influences how widely machine learning research diffuses [4], post-release maintenance remains highly uneven across the community [5]. This reality raises a key question: why do some repositories stay healthy and active while others fade away?

Evaluating maintenance requires looking closely at issue management, which tracks long-term organization and maintainer responsiveness better than raw popularity metrics [2,6]. Features like issue labels are crucial for triaging bugs and organizing community feedback [7], yet they are notoriously underused in AI repositories [5].

This maintenance behavior may be driven by the paper's publication venue. Top-tier venues grant higher visibility, potentially triggering a "Matthew effect" where greater exposure pulls in more users, feedback, and external contributors [8,9]. Similarly, publication year reflects a repository's maturity. While older projects have had more time to build community routines, open-source survival rates are notoriously low, with over half of GitHub projects failing within five years [10]. Because popularity growth is non-linear and relies on continuous updates [11], a repository's age does not guarantee its success.

While existing studies offer vital pieces of the puzzle, a gap remains. We address this by systematically analyzing how these contextual factors–venue,

timing, and framework–collectively shape an AI repository's long-term health and community adoption.

3 Methodology

3.1 Research Design

We explore how an AI paper's academic context relates to the maintenance and adoption of its accompanying repository. Repository health is evaluated using issue closure rates, labelling rates, forks, watchers, and contributor counts (dependent variables) against three contextual factors: publication venue, publication year, and deep learning framework.

- **RQ**$_1$: *Given the publication venue (CVPR, ICCV and others), can we predict repository health metrics such as closure rate, labelling rate, and contributor count?*
- **RQ**$_2$: *Is there a temporal pattern in issue management metrics (closure rate, labelling rate, and median time-to-close in days)?*
- **RQ**$_3$: *Does the choice of deep learning framework (PyTorch, TensorFlow, and others) affect community engagement metrics such as forks, watchers, and contributor count?*

Our empirical analysis uses the dataset by Yang et al. [5], covering 576 deep learning repositories (2013–2022) and 24,953 GitHub issues linked to their source papers. The data comprises two primary tables joined by `Name_Repo`: `repoDB`, containing repository-level metadata (venue, year, framework, aggregate statistics); and `issueDB`, tracking individual issue timestamps, labels, and sentiment scores.

Figure 1 illustrates the end-to-end methodology across all three RQs, from dataset preparation through statistical testing to visualisation.

3.2 RQ1: Does Publication Venue Predict Repository Health?

We compute three repository health metrics per repository:

- **Closure Rate:** Proportion of issues resolved.

$$\text{Closure Rate} = \frac{N_{\text{closed}}}{N_{\text{total}}}$$

- **Labelling Rate:** Proportion of issues carrying at least one label.

$$\text{Labelling Rate} = 1 - \frac{N_{\text{open, no label}} + N_{\text{closed, no label}}}{N_{\text{total}}}$$

- **Contributor Count:** Total number of distinct contributors per repository.

Repositories are grouped into seven venue categories: CVPR, ECCV, ICCV, ICLR, ICML, NeurIPS, and *Others*. Given non-normal metric distributions, we apply the **Kruskal-Wallis test** ($\alpha = 0.05$), followed by **Dunn's post-hoc test** with Bonferroni correction for pairwise comparisons. Results are visualised using box plots.

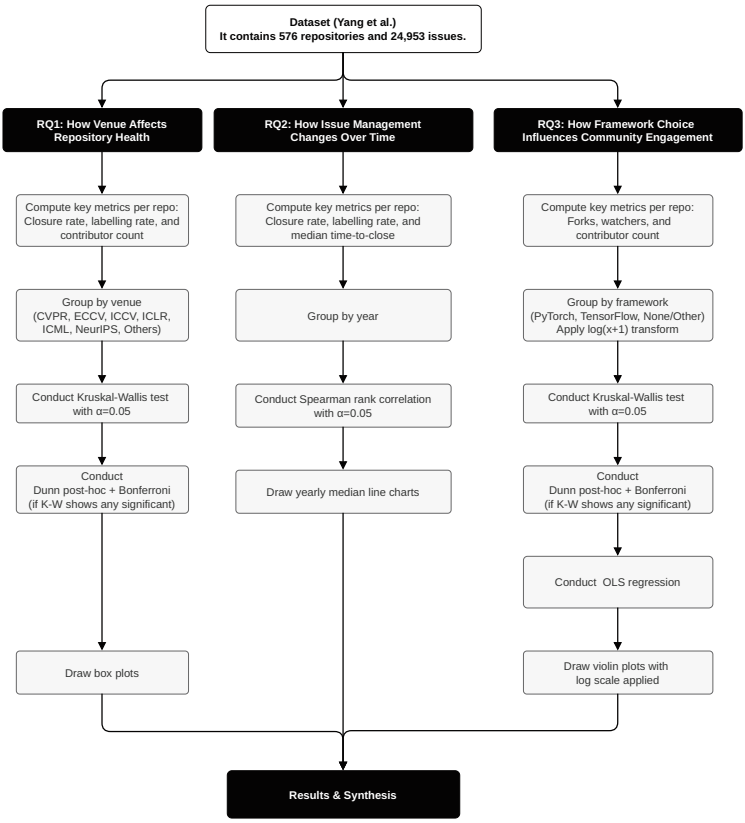


Fig. 1. Overview of the study methodology, outlining the steps taken with the Yang et al. dataset for each research question, including data preparation, grouping, statistical testing, and visualisation

3.3 RQ2: Is There a Temporal Pattern in Issue Management Quality?

We use `Year_Paper` as the temporal variable and examine three outcome metrics: closure rate, labelling rate, and median time-to-close (TTC) in days. TTC is computed per issue as the difference between `Date_Closed` and `Date_Created`, then aggregated to a per-repository median by joining `issueDB` with `repoDB`. Issues with negative durations are discarded as data artefacts.

We report yearly medians as a descriptive overview and quantify monotonic trends using **Spearman's rank correlation** (ρ, $\alpha = 0.05$). Spearman's ρ is preferred over Pearson's r because it is robust to the heavy skew typical of issue-tracking data (e.g., issues open for years) and does not assume linearity. Yearly median trends are visualised using line charts.

3.4 RQ3: Does Framework Selection Affect Community Engagement?

Repositories are grouped into three framework categories based on the `Framework` column: *PyTorch*, *TensorFlow*, and *None/Other*. Community engagement is measured via forks (`Num_Fork`), watchers (`Num_Watcher`), and contributor count. Due to strong right skew, all three metrics are $\log(x+1)$-transformed prior to statistical testing.

We apply the **Kruskal-Wallis test** for group-level differences and **Dunn's post-hoc test** with Bonferroni correction for pairwise comparisons. To control for potential confounding by publication year and venue prestige, we fit an **Ordinary Least Squares (OLS) regression** model using log-transformed fork count as the target variable and `Year_Paper`, conference venue (label-encoded), and framework group (label-encoded) as predictors. Coefficient magnitudes are compared to assess the relative impact of each factor. Results are visualised using violin plots on a log scale.

4 Results

4.1 RQ1: Publication Venue and Repository Health

Figure 2 shows the number of repositories in each venue group; CVPR contributes the largest share ($n = 95$), followed by ICLR and ICCV.

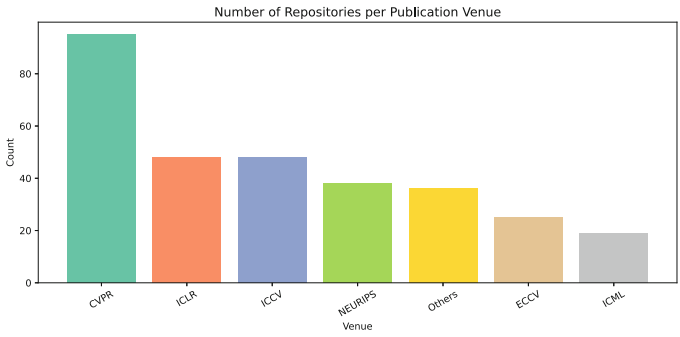


Fig. 2. Number of repositories per publication venue

Figure 3 presents the distribution of closure rate, labelling rate, and contributor count per venue as box plots. The closure rate varies noticeably across groups: ECCV has the highest median (0.667), while ICML has the lowest (0.462). In contrast, labelling rates are near zero for all venues, indicating that issue labelling is largely absent across the entire dataset regardless of where the paper was published.

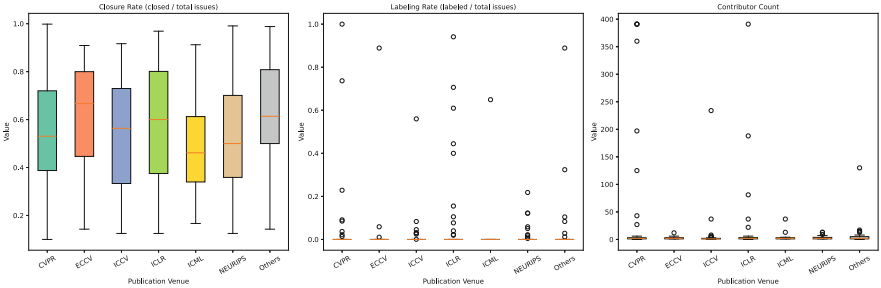


Fig. 3. Distribution of closure rate, labelling rate, and contributor count per publication venue

To evaluate the impact of publication venues, we use a Kruskal-Wallis test on the three metrics. As shown in Table 1, the test indicates a statistically significant difference in closure rates across different venues. For labelling rates or contributor counts, we found no significant differences.

Table 1. Kruskal-Wallis Test results for repository health metrics across venues

Metric	H-statistic	p-value
Closure Rate	12.749	0.047*
Labelling Rate	9.398	0.152
Contributor Count	9.868	0.130

* Significant at $\alpha = 0.05$

Following the significant result for closure rates, we conduct a Dunn's post-hoc test to identify specific pairwise differences. The test identifies the strongest contrasts between the *Others* group and both ICML ($p = 0.213$) and NeurIPS ($p = 0.220$). However, once the Bonferroni correction is applied, neither pair maintains significance at the $\alpha = 0.05$ level.

Overall, publication venue shows a modest but statistically significant association with issue closure activity. In contrast, labeling practices remain poor on all venues.

4.2 RQ2: Temporal Trends in Issue Management

Figure 4 shows the distribution of repositories over time; the majority of repositories in the dataset were published between 2017 and 2022, with 2018 being the most represented year ($n = 56$).

Figure 5 shows the yearly median trajectory for each metric.

To examine temporal trends, we calculated the Spearman rank correlation coefficient (r_s) between publication year and our primary metrics. As shown in

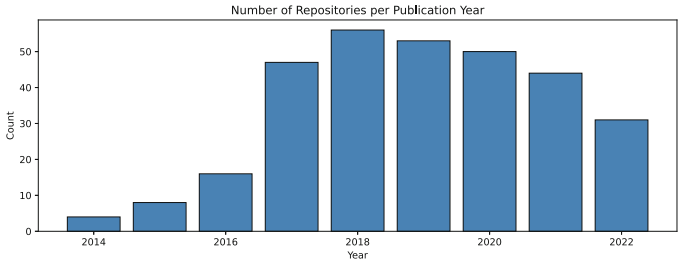


Fig. 4. Number of repositories per publication year

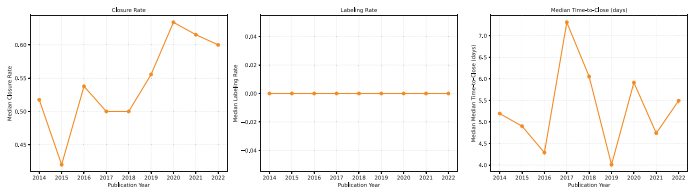


Fig. 5. Yearly median of closure rate, labelling rate, and median time-to-close

Table 2, publication year is positively associated with closure rate, although the effect is small ($r_s = 0.137, p = 0.016$). In other words, repositories linked to more recently published papers tend to close a slightly larger share of their issues.

Table 2. Spearman's rank correlation (r_s) between publication year and repository metrics

Metric	Correlation (r_s)	p-value
Closure Rate	0.137	0.016*
Labelling Rate	0.095	0.096
Median Time-to-close	−0.004	0.941

* Significant at $\alpha = 0.05$

This temporal pattern is also reflected in the yearly medians: the median closure rate rose from 0.50 in 2017 to a peak of 0.634 in 2020 before reaching a point of stability. In contrast, labelling rates remained at zero across all years.

4.3 RQ3: Framework Choice and Community Engagement

PyTorch dominates the dataset ($n = 263$), followed by None/Other ($n = 169$) and TensorFlow ($n = 144$). Figure 6 visualises the log-transformed engagement distributions per framework, and Fig. 7 compares median engagement values side by side.

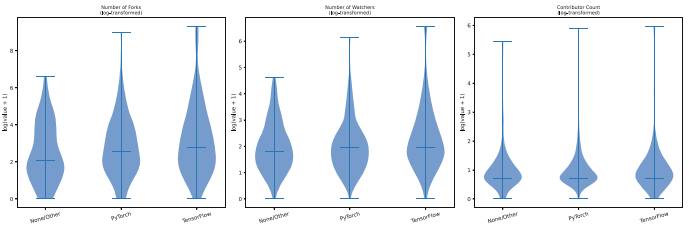


Fig. 6. Violin plots of log-transformed community engagement metrics (forks, watchers, contributors) per framework

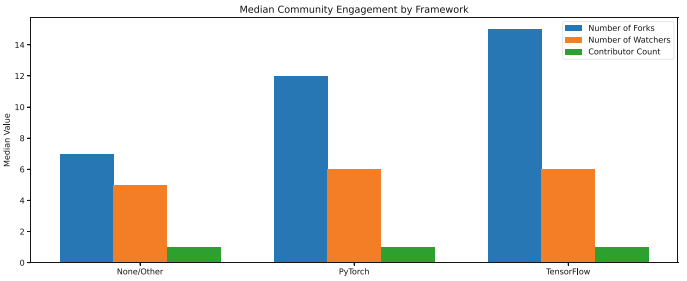


Fig. 7. Median forks, watchers, and contributor count per framework

To access the effect of deep learning framework on repository engagement, we applied a Kruskal-Wallis test to log-transformed values to account for heavy skewness. As shown in Table 3, framework groups differ significantly in both log-forks ($p = 0.006$) and log-watchers ($p = 0.031$), but not in contributor count.

Table 3. Kruskal-Wallis test on log-transformed engagement metrics across frameworks

Metric (log-transformed)	H-statistic	p-value
Forks	10.334	0.006^{**}
Watchers	6.983	0.031^{*}
Contributor Count	1.784	0.410

$^{**}p < 0.01, ^{*}p < 0.05$

Dunn's post-hoc analysis with Bonferroni correction shows the difference in forks is largely explained by the contrast between TensorFlow and the *None/Other* group ($p = 0.005$). By comparison, TensorFlow and PyTorch do not vary significantly ($p = 0.046$).

We next conducted a multivariate analysis to examine the relationship between framework choice and repository engagement, at the same time we also account for other potentially confounding factors. Among the engagement

metrics, number of forks showed the clearest group difference in the initial comparisons ($p = 0.006$) and was therefore used as the dependent variable in the OLS regression model.

- **Target Variable Selection:** We selected *log-forks* for the regression analysis because it produced the lowest p-value among engagement metrics in the earlier comparisons, which means it owns the strongest relationship with framework choice.
- **Control Variables:** We included publication year and venue to reduce potential confounding, thus explaining the differences related to the repository age and publication context.

As shown in Table 4, the OLS regression suggests the framework choice maintains positively linked with repository forks.

Table 4. OLS Regression results for log(Forks + 1)

Predictor Variable	Coefficient (β)
Framework Choice	0.4745
Publication Year	−0.1943
Publication Venue	−0.1174
Model Diagnostics	
R-squared (R^2)	0.0788

The positive value ($\beta = 0.4745$) shows that the choice of framework is an important predictor of engagement. This result remains significant even when other factors are considered.

However, the model only explains a small part of the total variance ($R^2 = 0.0788$). This suggests that other factors, which are not included in this study, may also affect the popularity of a repository.

5 Discussion

5.1 The Gap Between Publishing and Maintaining

Across all three RQs, academic publication context has minimal impact on AI repository maintenance. While we find statistically significant variation in closure rates across venues (RQ1) and minor improvements over time (RQ2), effect sizes are trivial. By open-source standards, overall maintenance quality remains poor. Most telling, issue labelling rates are practically zero across all 24,953 issues, regardless of venue, year, or framework. This underscores a systemic gap between checking a reproducibility box for paper acceptance and managing code as a long-term open-source project; most authors share code merely to meet community expectations, with no intention of future upkeep.

5.2 Venue Prestige Does Not Guarantee Better Maintenance

The Kruskal-Wallis test for RQ1 shows a statistically significant variance in closure rates ($H = 12.749$, $p = 0.047$), yet no individual venue pairs survive Bonferroni correction. Despite some global variation, no single conference clearly leads in issue management. ECCV repositories show the highest median closure rate (0.667), but the small sample size ($n = 25$) makes this unreliable. Our data refutes the idea that prestigious venues yield better-maintained codebases; the community cares far more about *using* models than resolving repository issues.

5.3 Newer Repositories Are Marginally Better Maintained

RQ2 reveals a small but significant positive correlation between publication year and closure rate ($r_s = 0.137$, $p = 0.016$), possibly reflecting growing awareness of reproducibility expectations and improved software engineering practices. Survivorship bias may also contribute, as older poorly-maintained repositories may have been abandoned and excluded from the dataset. By contrast, no comparable trend exists in time-to-close ($\rho = -0.004$, $p = 0.941$), suggesting that when issues *are* closed, resolution speed has not improved over time.

5.4 TensorFlow Repositories Attract More Community Attention

The observed engagement gap likely reflects a temporal artefact rather than an intrinsic framework property. TensorFlow's early launch (2015) gave its repositories a longer incubation period to accumulate engagement metrics. The regression model's low explanatory power ($R^2 = 0.079$) supports this, indicating that framework choice accounts for only a modest share of variance, with unobserved factors such as repository age and search visibility playing a larger role. The lack of significant differences in contributor counts across frameworks ($p = 0.410$) further suggests that TensorFlow's higher fork counts reflect passive community attention rather than stronger cooperative development.

5.5 Practical Implications

Our findings carry several practical implications. For **authors**, simply releasing code is insufficient; adopting basic issue-tracking hygiene (labelling, timely responses) would substantially improve repository utility. For **conference organisers and programme chairs**, reproducibility badges and artefact evaluation tracks–already adopted by NeurIPS and ICML–could be extended to include a maintenance component, rewarding authors who actively respond to community issues post-publication. For **researchers** building on existing AI code, near-zero labelling rates mean that issue triage must currently be done manually, an overhead that better tooling or community standards could reduce.

6 Threats to Validity

We discuss threats following Wohlin et al.'s classification [5].

6.1 Construct Validity

Our proxy metrics for repository health (closure rate, labelling rate, contributor count) and community engagement (forks, watchers) imperfectly capture real-world dynamics. High closure rates may reflect bulk-closing rather than genuine bug fixing, contributor counts can be inflated by bots, and forks or watchers often signal passive curiosity rather than active commitment. Deeper indicators such as external pull requests, response latencies, and non-author comments would provide a more accurate picture.

6.2 Internal Validity

Observed statistical relationships do not imply causation. Venue differences in closure rates (RQ1) may stem from confounders such as research domain, team size, or industry backing. The positive temporal trend (RQ2) may reflect survivorship bias, as poorly maintained older repositories likely dropped out of the ecosystem. TensorFlow's higher engagement (RQ3) is likely a temporal artefact of its earlier launch rather than an intrinsic framework advantage. Despite controlling for year and venue, our OLS model explains only 7.9% of variance, indicating substantial residual confounding.

6.3 External Validity

Our dataset covers major computer vision and machine learning venues (CVPR, ICCV, ECCV, NeurIPS, ICML, ICLR), so findings may not generalise to the broader AI ecosystem, including non-academic repositories, other fields such as robotics, or industrial research labs. Furthermore, the dataset extends only to 2022 and does not capture the post-2022 growth of LLMs, which may have shifted open-source maintenance trends.

6.4 Conclusion Validity

Non-parametric tests (Kruskal–Wallis, Spearman's ρ) are used throughout, appropriate given non-normal distributions. Bonferroni correction is applied for multiple comparisons, though its conservativeness raises the risk of false negatives, particularly in RQ2 and RQ3. Results near the significance threshold ($\alpha = 0.05$), such as the RQ1 closure rate ($p = 0.047$), should be interpreted with caution. Small effect sizes throughout (Spearman $\rho \leq 0.14$; $R^2 = 0.079$) limit the practical significance of our findings.

7 Conclusion

This study evaluated how an AI paper's academic context relates to its GitHub repository's upkeep and community adoption. Analyzing 576 repositories revealed that publication venue, year, and framework choice have surprisingly

minor impacts on repository health metrics. Prestige does not guarantee better maintenance, and while TensorFlow repositories draw more passive engagement, this stems from earlier release timelines rather than deeper collaboration. Our most critical takeaway is the near-zero issue labelling rate across nearly 25,000 issues, underscoring a systemic pattern where researchers treat code as a static publication checklist item rather than a living open-source project. Ultimately, academic context alone cannot explain repository variance; future work should look beyond the paper to investigate operational factors like institutional backing and author responsiveness.

References

1. Gibney, E.: This ai researcher is trying to ward off a reproducibility crisis. Nature **577**(7788), 14 (2020)
2. Linåker, J., Papatheocharous, E., Olsson, T.: How to characterize the health of an open source software project? A snowball literature review of an emerging practice. In: Proceedings of the 18th International Symposium on Open Collaboration, pp. 1–12 (2022)
3. Wattanakriengkrai, S., et al.: GitHub repositories with links to academic papers: public access, traceability, and evolution. J. Syst. Softw. **183**, 111117 (2022)
4. Kang, D., Kang, T., Jang, J.: Papers with code or without code? Impact of GitHub repository usability on the diffusion of machine learning research. Info. Process. Manag. **60**(6), 103477 (2023)
5. Yang, Z., Wang, C., Shi, J., Hoang, T., Kochhar, P., Lu, Q., Xing, Z., Lo, D.: What Do Users Ask in Open-Source AI Repositories? An Empirical Study of GitHub Issues. In: IEEE/ACM 20th International Conference on Mining Software Repositories (MSR), pp. 79–91. (2023). https://doi.org/10.1109/MSR59073.2023.00024
6. Jarczyk, O., Jaroszewicz, S., Wierzbicki, A., Pawlak, K., Jankowski-Lorek, M.: Surgical teams on GitHub: modeling performance of GitHub project development processes. Inf. Softw. Technol. **100**, 32–46 (2018)
7. Kim, J., Lee, S.: An empirical study on using multi-labels for issues in GitHub. IEEE Access **9**, 134984–134997 (2021)
8. Peng, T.-Q., Zhu, J.J.: Where you publish matters most: a multilevel analysis of factors affecting citations of internet studies. J. Am. Soc. Inform. Sci. Technol. **63**(9), 1789–1803 (2012)
9. Alrashedy, K., Binjahlan, A.: How do software engineering researchers use GitHub? An empirical study of artifacts and impact. In: 2024 IEEE International Conference on Source Code Analysis and Manipulation (SCAM), pp. 118–130 (2024). IEEE
10. Ait, A., Izquierdo, J.L.C., Cabot, J.: An empirical study on the survival rate of GitHub projects. In: Proceedings of the 19th International Conference on Mining Software Repositories, pp. 365–375 (2022)
11. Borges, H., Hora, A., Valente, M.T.: Understanding the factors that impact the popularity of GitHub repositories. In: 2016 IEEE International Conference on Software Maintenance and Evolution (ICSME), pp. 334–344. IEEE (2016)

Idea and Vision

Development of a Child-Friendly Constraint-Induced Movement Cast Using 3D Scanning and 3D Printing

Remington Orange[1(✉)], Bibiana M. Steckel[1,2], and Christopher R. Shelton[1]

[1] Virtual/Augmented Reality Laboratory (VAR Lab), Penn State Behrend, The Pennsylvania State University, Erie, PA, USA
remingtonorange712@gmail.com

[2] Program in Rehabilitation Sciences, Neuroscience and Virtual Reality Laboratory (NERV), Federal University of Health Sciences of Porto Alegre (UFCSPA), Porto Alegre, RS, Brazil

Abstract. Background: Constraint-Induced Movement Therapy (CIMT) is an evidence-based intervention for improving upper-limb function in individuals with hemiparesis. However, outcomes may vary depending on the type of constraint device used. Splint-based constraints have been associated with more promising outcomes than mitts/gloves or slings, but available pre-made and custom thermoplastic splints may be difficult to use in pediatric populations due to fit-related issues, discomfort, and fabrication challenges. Advances in digital fabrication, including 3D scanning and 3D printing, may offer new opportunities to redesign CIMT constraint devices in a more comfortable, safe, accessible, and child-friendly format.

Objective: To describe the development of a child-friendly, 3D-scanned and 3D-printed CIMT constraint device intended to support upper-limb motor rehabilitation in children with hemiparesis.

Methods: A customized CIMT constraint prototype was developed using 3D scanning, digital modeling, 3D printing, and Velcro-based assembly. The prototype was developed using an internal adult reference scan and was not tested with pediatric participants. A researcher's upper limb was scanned, modeled in Blender, divided into three printable sections, sliced in Prusa Slicer, and printed using a Prusa MK4S printer.

Results: The workflow resulted in a three-sect. 3D-printed constraint prototype covering the hand, forearm, elbow, and upper arm. The device included an open lattice structure intended to reduce weight and support airflow, with Velcro fastening used for adjustability and positioning. Because the prototype was developed from an adult reference scan, the present results are limited to technical feasibility and prototype fabrication rather than pediatric fit, comfort, usability, safety, or clinical effectiveness.

Conclusion: The prototype supports the technical feasibility of using 3D scanning and 3D printing to create a personalized, breathable, and adjustable CIMT constraint device. However, the device has not yet been evaluated with children. Future pediatric testing is needed to assess comfort, safety, usability, acceptance, and suitability for CIMT-related rehabilitation contexts.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 567–580, 2027.
https://doi.org/10.1007/978-3-032-39763-8_33

Keywords: Constraint-Induced Movement Therapy · 3D Scanning · 3D Printing · Pediatric Rehabilitation · Upper Limb Hemiparesis · Digital Health Innovation

1 Introduction

Constraint-Induced Movement Therapy (CIMT) is an evidence-based intervention used to improve upper-limb function in children with unilateral motor impairments, including children with hemiplegic cerebral palsy [1]. CIMT is based on the principle of encouraging use of the affected limb by temporarily restricting the less-affected limb during structured therapeutic activities [1]. Through repeated practice and task-oriented training, CIMT aims to promote motor learning, improve functional performance, and support greater independence in daily activities [1].

Different types of constraint devices have been used in CIMT protocols, including mitts or gloves, slings, and splints. Among these, mitts/gloves and slings have been commonly used as constraint devices in pediatric contexts [2, 3]. However, recent evidence suggests that the type of constraint device may influence intervention outcomes. A systematic review and meta-analysis found that constraint type was a significant moderator of the effects of CIMT on upper-limb function in children with cerebral palsy, with splint-based constraints showing significant effects, while glove- and sling-based constraints did not reach statistical significance [3]. These findings suggest that splint-based constraints may be a promising direction for pediatric CIMT.

Although splint-based constraints may offer clinical advantages, currently available splint options present practical limitations. Pre-made splints are restricted by standardized sizing and may not fit a child's limb precisely, especially given the variability in pediatric limb dimensions across children from 0 to 17 years of age. These dimensions can vary according to factors such as height, weight, age, and sex [4]. Poorly fitted splints may contribute to complications such as pressure sores, skin maceration, poor fit, splint breakage, pain, or discomfort [5]. Custom-made thermoplastic splints can provide a more individualized fit because they are molded directly to the limb. However, this process depends on the clinician's training, experience, and fabrication technique [5]. Even when fabricated by experienced clinicians, thermoplastic splints may still require readjustment due to unsatisfactory fit, particularly when changes in swelling or limb condition occur after initial fabrication [6]. These limitations support the need for alternative approaches that preserve the benefits of individualized splint design while reducing discomfort and fabrication-related challenges.

Three-dimensional (3D) scanning and 3D printing may provide an alternative approach, as these technologies have already been used for orthopedic immobilization and support in rehabilitation contexts [7, 8]. However, this approach has not yet been clearly described in the CIMT literature as a form of pediatric constraint device [2, 3]. Therefore, this study focuses on the design and fabrication of a customized constraint device using 3D scanning and 3D printing. We hypothesize that 3D scanning can capture the geometry of a child's limb and generate a digital model that supports individualized design. From this model, a constraint splint can be fabricated through 3D printing. This

workflow may support the development of a lightweight, breathable, easy-to-clean, and visually appealing device, with customized colors and child-friendly design features that may improve comfort and acceptance. As 3D scanning and 3D printing become increasingly accessible and affordable, these technologies may offer a practical pathway for developing personalized pediatric rehabilitation devices [9, 10].

The present study reports an early-stage prototype development process rather than pediatric testing. The prototype was developed using an internal adult reference scan to evaluate whether the proposed digital fabrication workflow could produce a wearable CIMT-style constraint device. No children were scanned, fitted, or tested in this phase. Therefore, this study does not evaluate pediatric fit, comfort, safety, usability, acceptance, adherence, or clinical effectiveness. The purpose of this study is to describe the development of a customized 3D-scanned and 3D-printed constraint device intended as an alternative device for pediatric CIMT. This work presents the initial design and fabrication process as a step toward producing a more ergonomic, hygienic, breathable, and child-friendly constraint device for upper-limb rehabilitation.

2 Literature Review

2.1 3D-Printed Orthosis and Cast Designs in Rehabilitation

Three-dimensional printed orthoses and casts have increasingly been proposed as patient-specific rehabilitation strategies that can match or even surpass conventional immobilization methods in selected clinical settings. Across recent rehabilitation literature, the main reported advantages include reduced weight, improved ventilation, removability, washability, and enhanced anatomical fit, all of which are associated with greater patient comfort and adherence throughout the rehabilitation process [8, 11, 12]. A systematic review published in 2025, encompassing 62 studies, concluded that 3D-printed orthoses generally improve comfort, fit, functional performance, and overall clinical effectiveness across different orthosis types, although outcomes remain dependent on the underlying diagnosis and severity of the condition [13].

Recent comparative studies further suggest that 3D-printed devices perform at least as well as conventional casts or orthoses regarding immobilization while demonstrating superior patient-centered outcomes in several contexts. In a randomized clinical trial involving patients with Colles fractures, an instant 3D-printed cast outperformed a conventional polymer orthosis in terms of immobilization effectiveness, pain reduction, patient satisfaction, Disabilities of the Arm, Shoulder and Hand (DASH) scores, and radiographic outcomes [12]. Similarly, a randomized feasibility trial conducted in 2025 with patients presenting distal radius and scaphoid fractures reported that a 3D-printed wrist orthosis resulted in higher satisfaction, lower pain and discomfort, and greater ease in performing activities of daily living compared with a fiberglass cast, while showing no significant differences in radiological healing between groups [14]. Likewise, a clinical comparison published in 2022 demonstrated that customized 3D-printed wrist braces provided immobilization comparable to plaster casts following surgery and were associated with improved functional recovery during follow-up [15].

2.2 Constraint Devices in Constraint-Induced Movement Therapy (CIMT)

In CIMT the less-affected upper limb is typically restrained using devices such as mitts, gloves, slings, splints, casts, or orthoses. The selection of the restraint reflects a balance between ease of use, effectiveness in restricting movement, and patient comfort. Reviews involving both pediatric and adult populations indicate substantial variability in CIMT protocols regarding both the type of restraint employed and the prescribed duration of use, with mitts, gloves, and slings representing the most frequently adopted options [16–18]. The restraint constitutes a fundamental component of CIMT, as it promotes the reduction of learned non-use by encouraging repetitive and active use of the affected limb [16].

Although substantial evidence supports the effectiveness of CIMT and an expanding body of literature demonstrates the benefits of 3D-printed orthoses, these two research fields have rarely intersected, highlighting an important gap in the current literature [13, 19, 20]. Reviews and applied studies on CIMT do not describe restraint devices manufactured through 3D printing for pediatric CIMT [19, 21]. Conversely, studies investigating 3D-printed orthoses demonstrate improvements in fit, comfort, patient satisfaction, and, in some cases, functional outcomes across fracture management, stroke rehabilitation, chronic hand disorders, and pediatric orthotic care, yet none have specifically evaluated these devices as restraint tools for CIMT [11, 13, 14]. Consequently, although the available evidence suggests a plausible rationale for integrating these two fields, this relationship has not yet been empirically established. The development and evaluation of 3D-printed restraint devices for pediatric CIMT therefore remain an important unmet research need rather than an established clinical intervention [19, 22].

2.3 Digital Fabrication in Pediatrics

Within pediatric rehabilitation, digital fabrication has been discussed not only as an alternative manufacturing technique but also as an approach capable of making orthotic care more responsive to children's anatomical, developmental, emotional, and family-related needs. Recent reviews in pediatric orthopedics indicate that additive manufacturing enables the production of lightweight, ventilated, and cost-effective orthotic devices with clinical effectiveness comparable to conventional fabrication methods, although challenges such as printing inaccuracies and prolonged production times remain important limitations [20]. The need for adaptable orthotic solutions is particularly relevant in pediatric populations, as children frequently require adjustments to accommodate growth, edema, and changes in joint range of motion, factors that are often difficult to address with traditional orthoses [8, 23].

Design considerations assume particular importance in pediatric populations because conventional orthoses and casts often present a highly medical appearance that may contribute to fear and anxiety. Recent studies on pediatric casting have demonstrated that individualized visual designs, color customization, artistic elements, and child-life interventions are intentionally incorporated to reduce anxiety and improve patient satisfaction during immobilization [24]. Similarly, research conducted outside orthopedic settings has shown that child-friendly environmental design reduces preoperative anxiety and that customized, colorful removable appliances are generally well accepted by children,

although color alone appears to have only a limited influence on treatment adherence [25, 26].

3 Methods

3.1 Digital Fabrication Workflow

The 3D-printed device prototype was created through a digital fabrication workflow that included three main stages: 3D scanning, digital modeling, and 3D printing. First, the limb was captured using a 3D scanner to create a mesh model. Next, the device was designed and adjusted in 3D modeling software so that it could fit the scanned limb shape. Finally, the digital device model was prepared for printing and produced using a 3D printer.

For this project, the workflow used an Artec Eva structured-light scanner to capture the arm [27], Blender [28] to edit and design the device model, Prusa Slicer [29] to prepare the model for printing, and a Prusa MK4S 3D printer [30] to fabricate the final prototype. The Artec Eva is a structured-light 3D scanner with up to 0.1 mm 3D point accuracy and 0.2 mm 3D resolution, supporting its use for capturing detailed limb geometry [27]. Prusa Slicer was used to prepare the digital model for 3D printing by generating the print file used by the printer [29].

3.2 3D Scanning Process

To begin the process, a researcher's arm was scanned from the fingertips to the shoulder using the Artec Eva scanner [27]. Although the full scan captured the arm up to the shoulder, the intended prototype area extended from the fingertips to slightly above the bicep.

The scanning process lasted approximately 6 min and was completed in one continuous scan. The arm was positioned with the shoulder rotated outward away from the body, flexed at the elbow with the hand positioned with fingertips pointing directly upwards; similar to a waving gesture.

After scanning, the data were processed inside Artec Studio 17 to create the base model used to build the device around [31]. Processing included alignment, global registration, outlier removal, fusion, smoothing, hole filling, and mesh simplification found in Artec Studio's" Autopilot" option which handles all steps in limited key presses. Once complete, the final model was exported as a Wavefront Object (OBJ) file; a 3D file format used to define surface geometry for 3D objects [32].

3.3 Digital Modeling in Blender

The OBJ file was imported into Blender for editing. In Blender, a basic device model was created around the scanned arm model. The device was created by creating loops of vertices at different cross sections of the arm model, following along with the curvature. Those vertices were then connected to create a shell or cover over the original scan model. In Blender, the device model was adjusted around the scanned arm mesh using

the Shrinkwrap modifier, which allows one object to conform to the surface of another mesh [33]. This step helped the device follow the shape of the scanned limb while preserving a small offset between the arm model and the device surface. The device surface was then designed with an open lattice (Fig. 1) structure to reduce the total amount of material, decrease weight, and improve airflow around the limb [34]. Prior work on 3D-printed lattice and cast designs suggests that porous or open structures can support ventilation and may reduce risks associated with heat, moisture, and skin irritation compared with more fully enclosed designs [35].

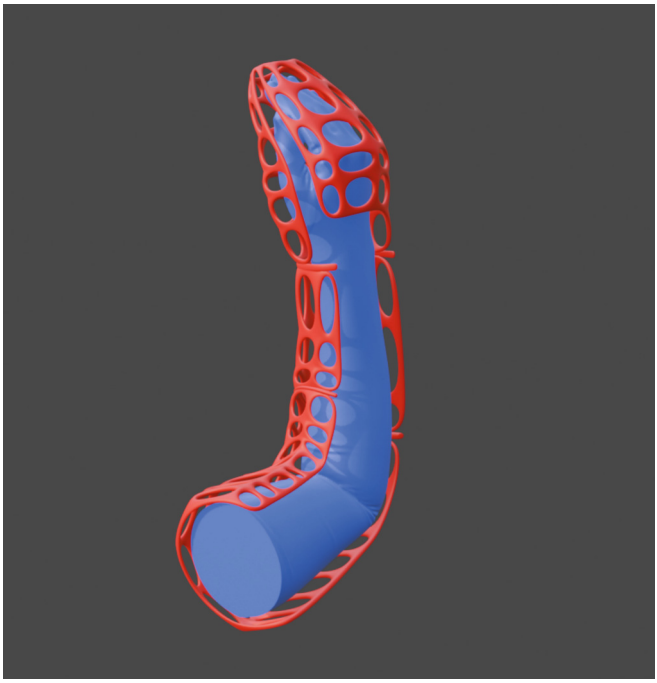


Fig. 1 Pre-print digital model of the 3D-printed CIMT cast in Blender. The blue model represents the scanned arm mesh, while the red structure shows the custom cast design fitted around the limb before slicing and fabrication. (Color figure online)

3.4 3D Printing

After the cast model was created, it was divided into three separate sections so that each piece could fit on the Prusa MK4S print bed [30]. The three sections included one piece from the hand to the wrist, one piece from the wrist to the elbow, and one piece from the elbow to the bicep. Separating the model into three parts made the prototype easier to print and allowed each section to be processed individually. Each of the three model sections was exported as a stereolithography (STL) file [36] and imported into Prusa Slicer.

3.5 3D Printing Parameters

Each STL file was imported into Prusa Slicer and prepared for fabrication using a Prusa MK4S 3D printer. The prototype was printed using polylactic acid (PLA), a thermoplastic polymer widely used in fused deposition modeling due to its printability, accessibility, and favorable processing characteristics for material-extrusion 3D printing [44]. The printer used a 0.4 mm nozzle. The primary print settings were as follows: layer height 0.20 mm, infill percentage 15%, nozzle temperature 220 °C, and bed temperature 60 °C. Organic supports were generated in Prusa Slicer, and support material was enabled for the printed sections.

The sliced print files included one file for the elbow-to-bicep section and one combined file containing the wrist-to-elbow and hand-to-wrist sections. These two sections were printed together to reduce manual resetting and preparation between prints. The elbow-to-bicep section required 13 h 16 min 45 s and used 180.25 g of PLA. The combined wrist-to-elbow and hand-to-wrist print required 13 h 53 min 52 s and used 201.41 g of PLA. Across the sliced files, the total print duration was 27 h 10 min 37 s, and the total PLA use was 381.66 g.

3.6 Material Cost Calculation

Material cost was calculated using the filament-cost values embedded in the Prusa Slicer .bgcode metadata and the estimated amount of Velcro used during assembly. All amounts were calculated in USD. The PLA cost was estimated at $9.70. The elbow-to-bicep section had an estimated PLA cost of $4.58, and the combined wrist-to-elbow and hand-to-wrist print had an estimated PLA cost of $5.12.

Velcro was purchased for $17.99 for a 45 ft. by ¾ in. roll before shipping and taxes, resulting in an estimated cost of approximately $0.40 per ft. The prototype used five short Velcro pieces measuring approximately 6 in. each to secure the printed sections together, one longer strap measuring approximately 3 ft., and three additional Velcro pieces measuring approximately 9 in. each to secure the arm into the prototype. In total, approximately 7.75 ft. of Velcro was used. Based on these values, the estimated Velcro cost was approximately $3.10.

The estimated total material cost for the prototype was approximately $12.80, including $9.70 in PLA filament and $3.10 in Velcro. This estimate excludes equipment costs, labor, electricity, failed prints, shipping, taxes, and software.

3.7 Assembly and Fit

After printing, supports were removed from each section using a heat gun to soften the support plastic and were removed using needle-nose pliers. The printed sections were inspected for sharp edges, rough support-contact areas, warping, cracks, and weak points. Areas requiring refinement were sanded down using a 100 grit sanding paper and wiped clean with isopropyl alcohol. Alternative assembly methods were considered but not selected. Glue created a rigid connection, produced odor, and resulted in fragile break points at some contact areas. Plastic zip ties were also tested, but they rubbed against the researcher's skin and caught on clothing. The final Velcro-based assembly

allowed the sections to be connected, adjusted, ability to be removed without tools and offered the best reported comfort. The researcher's hand was placed inside the prototype and secured around the opening with Velcro. A longer section of Velcro was looped through the palm-side portion of the device and around the researcher's waist/abdomen to hold the limb in a stable and out-of-the-way position.

4 Result

4.1 Prototype Output

The fabrication process produced a customized 3D-printed CIMT constraint prototype composed of three printed sections assembled with Velcro. The final device covered the hand, forearm, elbow, and upper arm. The three printed sections included: 1) a hand-to-wrist section, 2) a wrist-to-elbow section, and 3) an elbow-to-bicep section. The prototype incorporated an open lattice structure intended to reduce material use, reduce weight, and improve airflow around the limb. Velcro fastening allowed the sections to be connected, adjusted, and positioned around the adult reference limb. (Fig. 2)

Because the prototype was developed using an internal adult reference scan, these results describe prototype fabrication and informal structural observations only. No pediatric fit, comfort, safety, usability, acceptance, adherence, or clinical outcomes were evaluated. The final assembled prototype weighed approximately 130 g after support removal, post-processing, and Velcro attachment. This final assembled weight is lower than the total slicer-reported PLA use because the print metadata includes support material and other material removed during post-processing.

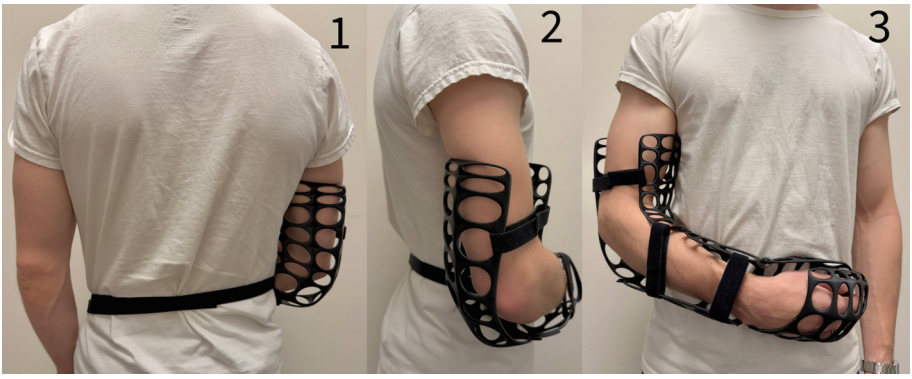


Fig. 2. Final 3D printed CIMT constraint prototype. 1.) Showing the posterior view; 2.) showing the side view; and 3.) showing the front view of the cast being worn

4.2 Print Duration, Material Use, and Cost

Across the sliced print files, the prototype required a total print duration of 27 h 10 min 37 s and used 381.66 g of slicer-reported PLA, including support material. The estimated PLA cost was $9.70. The prototype also required approximately 45 min of post-processing after printing, including support removal, edge refinement, inspection, and preparation for Velcro assembly. Based on the Velcro estimate of approximately 7.75 ft., the Velcro cost was approximately $3.10. The estimated total material cost for the prototype was therefore approximately $12.80, including $9.70 in PLA filament and $3.10 in Velcro. This estimate excludes equipment costs, labor, electricity, failed prints, shipping, taxes, software, and post-processing tools.

5 Discussion

5.1 Comparison with Existing 3D-Printed Cast Designs

One important distinction between this prototype and many existing 3D-printed cast designs is the area of immobilization. Existing 3D-printed casts are often designed for fractures or orthopedic support and commonly extend only to the wrist, hand, or below the fingers, depending on the injury site [37]. In contrast, the present prototype was designed for a CIMT-related use case, where the goal is not simply to stabilize an injury but to limit use of the less-affected upper limb during therapeutic activities. For that reason, the design extends farther up the arm and includes the hand, wrist, forearm, elbow, and upper-arm region. This difference matters because pediatric CIMT often requires a device that discourages active use of the limb while still allowing safe participation in therapy or daily activities [2].

5.2 Comfort, Safety and Controlled Restriction with Removability

A major potential advantage of a 3D-printed CIMT device is improving comfort and breathability compared with traditional restriction methods. Traditional casts and splints can create complications during application, wear, and removal, including thermal injury, pressure sores, cast saw burns, and skin breakdown [5, 6, 38]. Proper casting and splinting technique are therefore important, especially in pediatric populations where young children or children with communication difficulties may not be able to clearly report discomfort [38]. Fiberglass casts can also be rigid, warm, difficult to remove once applied, and may require clinical tools for removal [39, 40]. CIMT gloves and sling-based systems may be less rigid, but they can still create discomfort, sweating, skin irritation, or frustration depending on the child's sensory tolerance and the duration of use [2].

The prototype's open, lattice structure may help address some of these concerns by reducing material coverage and allowing more airflow around the skin. This may make the device feel lighter and less enclosed than a traditional cast. This design choice is relevant because cast and splint sores can occur when abnormal contact develops between the device and the skin, including from improper padding, wrinkling, bunching, excess movement within the cast, or changes in limb position while the material is curing [38]. By reducing continuous surface contact and avoiding a fully enclosed cast structure,

the prototype may reduce some of the comfort and skin-monitoring concerns associated with traditional casting.

In addition, the use of Velcro allows the fit to be adjusted rather than permanently fixed, which may improve comfort during longer therapeutic sessions. This is important because cast-related guidance emphasizes that a casted limb should be comfortable, and increased pain or discomfort should prompt further investigation, including checking for tightness, pressure sores, alignment issues, or objects trapped under the cast [38].

The prototype may therefore offer a safer balance between therapeutic restriction and caregiver or therapist control, especially when repeated use and child tolerance are important design concerns [41]. It is intended to restrict movement of the limb while it is being worn, but it also allows removal by another person when necessary. This is an important safety consideration, because in CIMT, the constraint device should discourage use of the less-affected limb, but it should not create unnecessary risk if the child becomes uncomfortable, distressed, or needs assistance [41]. However, because the device should not be easily removed by the child during therapy, the prototype design balances accessibility for adults with enough restriction to support the therapeutic purpose.

5.3 Child Acceptability and Use

A key question for future testing is whether children would tolerate and accept this type of device. Pediatric assistive devices are more likely to be used successfully when they are comfortable, visually appealing, and not overly intimidating [42]. Because of that, the appearance and physical experience of the prototype are important design concerns [42, 43].

The 3D-printed format may provide opportunities to improve child acceptance through visual and functional customization. Future versions could incorporate lighter forms, color choices, playful surface patterns, or character-inspired decorative features. These design options may help the device appear less like a traditional medical cast and more like a personalized therapeutic tool. Fig. 3 presents concept models created in Blender and virtually assembled onto several cast variations. These examples illustrate possible ways that users, clinicians, or caregivers could customize the cast to make it more engaging and child friendly.

5.4 Limitations and Future Testing

Although the prototype suggests that a customized 3D-printed CIMT device is feasible, several questions remain. The current version was developed using a researcher's arm, so fit, comfort, weight, and usability still need to be evaluated with the intended population. Future studies should assess whether children can tolerate the device during therapy, whether therapists find it practical to apply and remove, and whether caregivers perceive it as safe and acceptable. Future work should also compare this design against traditional CIMT gloves, slings, and casts to determine whether the 3D-printed version provides measurable benefits in comfort, breathability, safety, adherence, or therapeutic use.

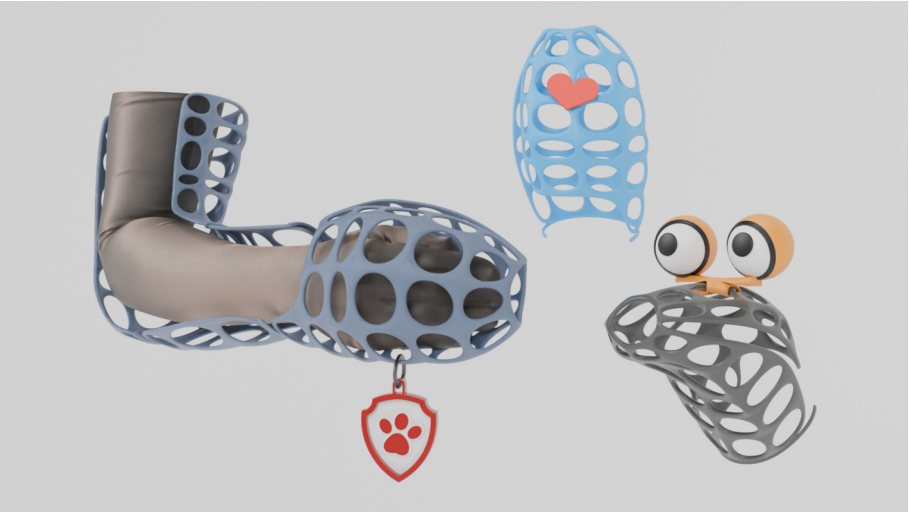


Fig. 3. Concept designs for child-friendly customization of the 3D-printed CIMT cast. The models were created in Blender and virtually assembled onto cast prototypes to demonstrate possible decorative features, including playful colors, character-inspired elements, and removable accessories

6 Conclusion

This study described the development of a customized 3D-scanned and 3D-printed CIMT constraint prototype designed as a potential alternative to conventional pediatric constraint devices. The workflow demonstrated that 3D scanning can be used to capture upper-limb geometry and support the digital design of a personalized constraint device, which can then be fabricated through 3D printing. The final prototype incorporated several intended design features, including sectional construction, Velcro-based adjustability, and an open lattice structure intended to reduce weight and support airflow around the limb.

The prototype also highlights the potential of digital fabrication to address practical challenges associated with existing CIMT constraint options, including poor fit, limited breathability, discomfort, and reduced child acceptability. In addition, the digital design process allows future versions to be modified, reproduced, and customized with child-friendly visual features.

However, this work represents an initial prototype rather than a clinically tested device. The prototype was created using a researcher's arm and has not yet been evaluated with pediatric participants. Future studies should assess fit, comfort, safety, usability, therapist satisfaction, caregiver acceptance, and child tolerance during CIMT-related activities. Further testing is also needed to compare this approach with traditional mitts, gloves, slings, casts, and thermoplastic splints. Overall, this study supports the feasibility of using 3D scanning and 3D printing to develop a personalized, breathable, adjustable, and child-friendly constraint device for potential pediatric CIMT applications.

References

1. Maggio, M.G., et al.: Constraint-Induced Movement Therapy in Children with Hemiplegic Cerebral Palsy: A Scoping Review of Functional Outcomes and Neuroplasticity-Related Evidence. Korean Academy of Rehabilitation Medicine (2026). https://doi.org/10.5535/arm.250165
2. Hoare, B.J., Wallen, M.A., Thorley, M.N., Jackman, M.L., Carey, L.M., Imms, C.: Constraint-induced movement therapy in children with unilateral cerebral palsy. John Wiley and Sons Ltd. (2019). https://doi.org/10.1002/14651858.CD004149.pub3.
3. Wang, H., Zhou, J., Shan, X., Zhang, L.: The effectiveness of modified constraint-induced movement therapy on upper limb function in children with cerebral palsy: a systematic review and meta-analysis. BMC Sports Sci. Med. Rehabil. **17**(1) (2025). https://doi.org/10.1186/s13102-025-01259-3
4. Edmond, T., Laps, A., Case, A.L., O'Hara, N., Abzug, J.M.: Normal ranges of upper extremity length, circumference, and rate of growth in the pediatric population. Hand. **15**(5), 713–721 (2020). https://doi.org/10.1177/1558944718824706
5. Papavasiliou, T., Shah, R.K., Chatzimichail, S., Uppal, L., Chan, J.C.Y.: Three-dimensional printed customized adjustable mallet finger splint: a cheap, effective, and comfortable alternative. Plast. Reconstr. Surg. Glob. Open. **9**(3), E3500 (2021). https://doi.org/10.1097/GOX.0000000000003500
6. Waldburger, L., Schaller, R., Furthmüller, C., Schrepfer, L., Schaefer, D.J., Kaempfen, A.: 3D-printed hand splints versus thermoplastic splints: a randomized controlled pilot feasibility trial. Int. J. Bioprint. **8**(1), 128–138 (2022). https://doi.org/10.18063/IJB.V8I1.474
7. Schwartz, D.A., Schofield, K.A.: Utilization of 3D printed orthoses for musculoskeletal conditions of the upper extremity: a systematic review. J. Hand Ther. **36**(1), 166–178 (2023). https://doi.org/10.1016/j.jht.2021.10.005
8. Demeco, A., et al.: The Upper Limb Orthosis in the Rehabilitation of Stroke Patients: The Role of 3D Printing. Multidisciplinary Digital Publishing Institute (MDPI). (2023). https://doi.org/10.3390/bioengineering10111256
9. Silva, R., Silva, B., Fernandes, C., Morouço, P., Alves, N., Veloso, A.: A Review on 3D Scanners Studies for Producing Customized Orthoses. Multidisciplinary Digital Publishing Institute (MDPI). (2024). https://doi.org/10.3390/s24051373
10. Bełżek, A., et al.: Advances in 3D printed orthotics for rehabilitation. (2025). https://doi.org/10.36740/WLek/202602
11. Glazer, C., et al.: 3D-printed orthoses vs. traditional plaster cast: a comparative clinical study. Balneo and PRM Research Journal. **16**(1) (2025). https://doi.org/10.12680/balneo.2025.785
12. Xiao, Y.P., Xu, H.J., Liao, W., Li, Z.H.: Clinical application of instant 3D printed cast versus polymer orthosis in the treatment of colles fracture: a randomized controlled trial. BMC Musculoskelet. Disord. **25**(1) (2024). https://doi.org/10.1186/s12891-024-07212-8
13. Atallah, H., Qufabz, T., Bakhsh, H.R., Ferriero, G.: The current state of 3D-printed orthoses clinical outcomes: a systematic review. BioMed Central Ltd. (2025). https://doi.org/10.1186/s12891-025-09070-4.
14. Tobler-Ammann, B., Schuind, F., Voillat, L., Vögelin, E.: Acceptability and safety of 3D printed wrist-based orthoses compared to fiberglass casts for the treatment of non-surgical distal radius- and scaphoid fractures: a randomized feasibility trial. J. Hand Ther. **38**(1), 143–151 (2025). https://doi.org/10.1016/j.jht.2024.11.004
15. Fang, J.J., Lin, C.L., Tsai, J.Y., Lin, R.M.: Clinical Assessment of Customized 3D-Printed Wrist Orthoses. Appl. Sci. **12**(22), 11538 (2022). https://doi.org/10.3390/APP122211538.
16. Brady, K., Garcia, T.: Constraint-Induced Movement Therapy (CIMT): Pediatric Applications. John Wiley and Sons Inc (2009). https://doi.org/10.1002/ddrr.59

17. Lan, Y., Etoom, M., Shen, X.: Influence of the constraint-induced method of constraint-induced movement therapy on improving lower limb outcomes after stroke: a meta-analysis review. Front. Neurol. (2023). https://doi.org/10.3389/fneur.2023.1090808
18. Tesio, L., et al.: Effect of constraint-induced movement therapy on lower extremity motor dysfunction in post-stroke patients: a systematic review and meta-analysis. Front. Neurol. (2022). https://doi.org/10.3389/fneur.2022.1028206
19. Gulrandhe, P., Acharya, S., Patel, M., Shukla, S., Kumar, S.: Pertinence of constraint-induced movement therapy in neurological rehabilitation: a scoping review. Cureus (2023). https://doi.org/10.7759/cureus.45192
20. Calderone, D., Cesarelli, G., Ricciardi, C., Amato, F., Clemente, F.: 3D Printing Application for Orthopedic Pediatric Surgery – a Systematic Review. Emerald Publishing (2024). https://doi.org/10.1108/RPJ-05-2024-0222
21. Gotlieb-Horowitz, M., Robbins, V., Ferrante, V., DeRosa, T.M., Behbahani, M., Koehler, S.M.: Uses of constraint-induced movement therapy (CIMT) in patients with brachial plexus birth injuries (BPBI). J. Hand Microsurg. **17**(5) (2025). https://doi.org/10.1016/J.JHAM.2025.100316
22. Oud, T., Tuijtelaars, J., Bogaards, H., Nollet, F., Brehm, M.-A.: Preliminary effectiveness of 3D-printed orthoses in chronic hand conditions: study protocol for a non-randomised interventional feasibility study. https://doi.org/10.1136/bmjopen-2022-069424
23. Thomas, S., Hu, X.: A 3D-printed hand exoskeleton for affordable and accessible upper limb rehabilitation. In: Annual International Conference of the IEEE Engineering in Medicine and Biology Society, vol. 2025, pp. 1–6 (2025). https://doi.org/10.1109/EMBC58623.2025.11254189
24. Braithwaite, C., et al.: Improving patient experience during casting: a review and description of cast art technique that allows for creative designs in pediatric casts. Journal of the Pediatric Orthopaedic Society of North America. **12** (2025). https://doi.org/10.1016/J.JPOSNA.2025.100233
25. Yildirim Balkan, Z., Yesildag, E., Taylan, E.: Does child-friendly design reduce preoperative anxiety in pediatric surgery patients? HERD. **18**(4), 125–136 (2025). https://doi.org/10.1177/19375867251351028
26. Baxmann, M., Zsoldos, M., Kárpáti, K.: The impact of colorful orthodontic appliances on children's motivation and treatment perceptions: a single-center cross-sectional study. Frontiers in Dental Medicine. **7**, 1783018 (2026). https://doi.org/10.3389/FDMED.2026.1783018/TEXT
27. Artec 3D: Artec Eva: fast 3D scanner for professionals. [Online]. Available: https://www.artec3d.com/portable-3d-scanners/artec-eva. . Accessed 28 June 2026
28. Blender Foundation: Blender - The Free and Open Source 3D Creation Software — blender.org. Blender Foundation, Amsterdam, Netherlands: 5.0. (2026)
29. Prusa Research: Original Prusa MK4S 3D printer (n.d.). Retrieved 28 June 2026. From https://www.prusa3d.com/product/original-prusa-mk4s-3d-printer/
30. Prusa Research: Original Prusa MK4S 3D printer. [Online]. Available: https://www.prusa3d.com/product/original-prusa-mk4s-3d-printer/. Accessed 28 June 2026
31. Artec 3D: Artec Studio 17 documentation. [Online]. Available: https://docs.artec3d.com/as/17/en/. Accessed 28 June 2026
32. Library of Congress: Wavefront OBJ file format. [Online]. Available: https://www.loc.gov/preservation/digital/formats/fdd/fdd000507.shtml. Accessed 28 June 2026
33. Blender Foundation: Shrinkwrap Modifier Documentation (2026). [Online]. https://docs.blender.org/manual/en/latest/modeling/modifiers/deform/shrinkwrap.html. Accessed 28 June. 2026

34. Badini, S., Regondi, S., Lammi, C., Bollati, C., Donvito, G., Pugliese, R.: Computational mechanics of form-fitting 3D-printed lattice-based wrist-hand orthosis for motor neuron disease. Biomedicine. **11**(7) (2023). https://doi.org/10.3390/biomedicines11071787
35. Zhang, X., et al.: Thermal-comfort design of personalized casts. In: UIST 2017 - Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology, pp. 243–254. Association for Computing Machinery, Inc (Oct. 2017). https://doi.org/10.1145/3126594.3126600
36. Library of Congress: STL (STereoLithography) file format family. [Online]. Available: https://www.loc.gov/preservation/digital/formats/fdd/fdd000504.shtml. Accessed 28 June 2026
37. Alshawaf, S.M., Burhamah, W., Alwazzan, S., AlYouha, S.: A streamlined algorithmic process for creating three-dimensional printed forearm casts. Plast. Reconstr. Surg. Glob. Open. **11**(3), E4824 (2023). https://doi.org/10.1097/GOX.0000000000004824
38. McGraw-Heinrich, J.A., Wall, J.C., Rosenfeld, S.B.: Common Cast Complications. Elsevier B.V. (2025). https://doi.org/10.1016/j.jposna.2025.100244.
39. Halanski, M.A., Halanski, A.D., Oza, A., Vanderby, R., Munoz, A., Noonan, K.J.: Thermal injury with contemporary cast-application techniques and methods to circumvent morbidity. J. Bone Joint Surg. **89**(11), 2369–2377 (2007). https://doi.org/10.2106/JBJS.F.01208
40. Sevencan, A.: Current complications of cast removal with oscillating saws and a novel method for reducing such complications: a comparative clinical study. J. Bone Joint Surg. **105**(1), 35–41 (2023). https://doi.org/10.2106/JBJS.21.01556
41. Merino-Andrés, J., López-Muñoz, P., Carrión, R.P., Martín-Casas, P., Ruiz-Becerro, I., Hidalgo-Robles, Á.: Is more always better? Effectiveness of constraint-induced movement therapy in children with high-risk or unilateral cerebral palsy (0–6 years): Systematic review and meta-analysis. John Wiley and Sons Inc. (2024). https://doi.org/10.1111/cch.13262
42. Howard, J., Fisher, Z., Kemp, A.H., Lindsay, S., Tasker, L.H., Tree, J.J.: Exploring the barriers to using assistive technology for individuals with chronic conditions: a meta-synthesis review. Taylor and Francis Ltd. (2020). https://doi.org/10.1080/17483107.2020.1788181.
43. Oro, B., Huerta, O., Bahari, A.: Empathy-Centric Design in Assistive Technologies for Cerebral Palsy and Disabilities: Balancing Aesthetics and Functionality. In: ACM International Conference Proceeding Series, pp. 58–62. Association for Computing Machinery (2024). https://doi.org/10.1145/3661790.3661802
44. Sandanamsamy, L. et al.: A comprehensive review on fused deposition modelling of polylactic acid. In: Progress in Additive Manufacturing, vol. 8, pp. 775–799. Springer Science and Business Media Deutschland GmbH (2023). https://doi.org/10.1007/s40964-022-00356-w

Intrinsic Training and Compounding Accuracy: Designing AI-Assisted SAS-to-R Migration in Clinical Trials

David Ju[1] and Nathalia Nascimento[2(✉)]

[1] The Pennsylvania State University, World Campus, University Park, PA, USA
dyj5141@psu.edu
[2] The Pennsylvania State University, Great Valley, University Park, PA, USA
nqm5742@psu.edu

Abstract. Statistical programming teams in clinical trials face growing pressure to migrate from SAS to R while maintaining regulatory traceability and workforce skill continuity. AI-assisted code-translation tools are typically evaluated using a single accuracy score from a single benchmark run, a framing that is inadequate for regulated clinical settings where reviewers must reconstruct the lineage of every computed value and teams must transition skills alongside tools. We propose a design vision in which AI-assisted workflows are evaluated as systems of compounding accuracy inseparably linked to hands-on training. In this vision, every reviewer intervention serves two purposes: it grows the agent's corpus, improving subsequent translations, and it produces an inspectable record that domain reviewers can audit. Each intervention is captured as a structured entry that the agent can retrieve during the next translation and that a domain reviewer can inspect. Drawing on clinical-trials industry experience, we articulate three design principles, namely training as a deliverable, capture-and-replay accuracy, and reviewability over autonomy, each paired with a testable hypothesis to guide future empirical work. We instantiate the principles in an illustrative worked example using a Microsoft 365 Copilot agent, a curated reference document, and a PROC COMPARE-oriented validation workflow. The worked example demonstrates that the principles can be instantiated in a concrete workflow and supports the feasibility of the testable predictions. The framework is accessible to any organization with an existing SAS environment and a configured LLM agent and provides a research agenda for future empirical validation.

Keywords: Large Language Models · Code Translation · AI-Assisted Software Engineering · Statistical Programming · Clinical Trials · SAS · R · Vision Paper

1 Introduction

Biometrics teams in clinical trials are increasingly migrating from SAS to R as regulators including the FDA now accept R-based analysis packages. However, the industry's workforce remains heavily SAS-skewed, a legacy of decades of standardization on SAS in regulated statistical programming, and the deep domain expertise of these teams in

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 581–595, 2027.
https://doi.org/10.1007/978-3-032-39763-8_34

regulatory submissions, validation conventions, and trial-domain semantics cannot be replicated by hiring R programmers from outside. The practical implication is that the existing team must learn R as part of the migration, not be displaced by it. AI-assisted code translation in statistical programming for clinical trials is at a crossroads. The dominant evaluation paradigm, measure translation accuracy on a held-out benchmark, report a number, declare success or failure, is borrowed from machine-translation research and natural-language generation. Across the recent LLM code-translation literature, evaluation defaults to per-run metrics such as BLEU, CodeBLEU, Computational Accuracy, Functional Equivalence Accuracy, and exact-match scores [1–6]. This paradigm is appropriate for general-purpose code-translation research, but it presupposes a deployment context in which the only thing that matters is the translated artifact's correctness on the day it is produced.

We argue that this paradigm mis-frames the deployment problem for biometrics teams in clinical trials. In this context, teams prepare analysis packages for submission to regulatory agencies such as the FDA, rather than running applications or products; reviewers must be able to reconstruct the lineage of every computed value; statistical programming teams must transition skills alongside tools because reviewer expertise must grow with the codebase; and the underlying foundation models are set by vendors and may change at any time. Critical assessments of LLM code translation [7, 8] have begun to question whether the field's current evaluation approach is adequate, creating an opening for an alternative design philosophy.

This paper proposes that AI-assisted code-translation workflows in clinical-trials statistical programming should be designed and evaluated along two dimensions: intrinsic training and compounding accuracy. Intrinsic training means that programmers learn the target language and the AI tool through the workflow itself, rather than through a separate training program. Compounding accuracy means that each human intervention is captured as a retrievable example that improves subsequent translations, making accuracy a property of the workflow rather than a one-shot property of the model.

Motivated by long-term practice in clinical-trials statistical programming, where regulatory traceability, reviewer expertise, and tool transitions are central constraints, we articulate three design principles that operationalize this vision: (1) training as a deliverable, (2) capture-and-replay accuracy, and (3) reviewability over autonomy. Each principle is paired with a testable hypothesis for future empirical work. Although these ideas build on prior work in corpus-driven RAG accuracy improvement, cognitive apprenticeship and learning-by-doing in software-engineering education, and human-in-the-loop AI, our contribution is to unify them into a single framework for AI-assisted SAS-to-R migration in clinical trials and to derive predictions about how such workflows should perform against one-shot baselines on the dimensions that matter for regulated clinical deployment.

The remainder of the paper is organized as follows. Sections 2 and 3 situate the vision in relation to prior work on LLM-based code translation and clinical-software validation. Section 4 presents the three design principles. Section 5 describes the illustrative worked example used to instantiate them. Section 6 outlines scope, limitations, and a research agenda for future empirical validation. This paper is a vision paper in which we propose

a framework and derive testable predictions but do not report empirical results. The worked example in Sect. 5 is a feasibility demonstration only.

2 Background: Clinical-Trial Statistical Programming and Validation

Clinical-trial statistical programming centers on the production of regulator-bound analytical outputs: subject-level analysis datasets, summary tables, listings, and figures that characterize a trial's results. The deliverable is not a running application, but a package of programs, datasets, logs, and documentation submitted to a regulatory agency. For example, the U.S. FDA's Study Data Technical Conformance Guide [18] describes the expected structure for new-drug-application study data. SAS has been the dominant language for these outputs for decades, and most regulatory submissions to date have been produced and validated in SAS.

Validation in this domain is review-driven: a named human reviewer is responsible for confirming that every computed value in an output dataset is reproducible and traceable to its source. The analysis programs themselves are part of the regulatory submission, while the validation evidence is retained as internal QC documentation available on inspection. The standard validation tool is PROC COMPARE, one of SAS's built-in procedures. PROC COMPARE performs a detailed comparison between two datasets, row by row and column by column, and reports differences in values, variables, or attributes within an explicit numerical tolerance, such as an absolute difference below a chosen threshold. Readers unfamiliar with regulated clinical-trial settings can think of PROC COMPARE as the structured-data analog of a strict diff over tabular output.

In a SAS-to-R migration, PROC COMPARE is a natural validation target because the existing team already uses it daily. An R translation is considered equivalent to the SAS original when PROC COMPARE between the SAS output dataset and the SAS dataset re-imported from the R output reports no value-level discrepancies. Reusing a familiar procedure at the validation gate avoids introducing a new tool or training step into an already regulated workflow. A practical limitation is that PROC COMPARE operates on SAS datasets only. Outputs that are not natively datasets, such as reports, listings, or formatted tables, must be coerced into dataset form, for example by exporting them as CSV and re-importing them, for the same validation gate to apply. Otherwise, a different tool or a manual review step must fill in for the comparison.

3 Related Work

Section 2 introduced the clinical-trial domain for readers outside regulated statistical programming; this section surveys the LLM code-translation literature that motivates our departure from the prevailing one-shot evaluation paradigm. Recent work on LLM-based code translation has predominantly evaluated translation quality through one-shot accuracy metrics on held-out benchmarks. Studies report Computational Accuracy [1, 9], Compilation Accuracy and Functional Equivalence Accuracy [4], BLEU and CodeBLEU [5, 6], and exact-match scores [5] adapted from machine-translation evaluation. Critical

assessments have begun to question whether this evaluation paradigm is sufficient: Pan et al. [7] document the bugs LLMs introduce during code translation, and Tao et al. [8] explicitly examine the limits of one-shot LLM code translation. These metrics are useful for general-purpose code-translation research, but they assume a deployment context in which success is primarily defined by the correctness of the translated artifact at the moment it is produced.

A subset of recent work has moved beyond pure one-shot evaluation by introducing more process-oriented translation methods. Bhattarai et al. [2] propose few-shot retrieval-augmented generation for Fortran-to-C++ translation. De Siano et al. [10] integrate human-in-the-loop feedback into the translation loop. Le and Andrzejak [11] use one-shot user-feedback correction to recover from incorrect generations. Wang et al.'s INTERVENOR [12] applies interactive compiler-guided repair and outperforms GPT-3.5 on translation tasks. Zhang et al. [3] propose modular decomposition of large codebases for validated, scalable translation, while Ibrahimzada [13] uses static analysis to manage long inputs through program decomposition. Jana et al.'s CoTran [4] combines reinforcement learning with compiler and symbolic-execution feedback. Hong and Ryu [14] address legacy migration in the C-to-Rust setting.

Together, these studies show that code translation is increasingly being treated as a workflow rather than a single model output. However, they primarily propose techniques for improving translation performance, rather than principles for designing and evaluating AI-assisted migration workflows. As a result, the field still tends to use per-run accuracy as the headline measure of success, even when the translation process includes retrieval, feedback, repair, decomposition, or human intervention.

Domain-specific applications are also under-represented in this literature. To our knowledge, no prior studies address SAS-specific translation, only a small minority address R as either source or target [6, 15], and few examine translation systems for clinical research or other regulated settings. This gap matters because clinical-trial biometrics imposes constraints that are not captured by standard one-shot benchmark evaluation. PHUSE working groups [16], CDISC pilot programs [17], and the FDA's evolving Computer Software Assurance guidance [18] establish expectations for traceability, version control, and reviewability of analytical software in clinical environments. In this context, AI-assisted translation tools cannot be evaluated only by day-of-production accuracy. They must also support review, preserve traceability, and help teams transition expertise alongside tools, because reviewer capability must grow with the codebase. The framework proposed in this paper makes these constraints first-class design considerations.

Three adjacent literatures provide antecedents for the principles we articulate. First, recent work on retrieval-augmented generation has argued that corpus scaling can match or exceed model scaling for accuracy on document-grounded tasks [20]. Our **capture-and-replay principle** builds on this line of work by treating reviewer interventions as the corpus-growth mechanism in clinical-trial migration workflows. Second, human-in-the-loop frameworks for AI-empowered software-engineering systems [10], together with regulatory expectations for audit trails and reviewability [18], establish human review as central to AI-generated artifacts. Our **reviewability-over-autonomy principle** extends this perspective by arguing that reviewability is not merely a compliance requirement

but a primary source of workflow value. Third, software-engineering education has long emphasized cognitive apprenticeship and learning by doing. Our **training-as-a-deliverable principle** applies this framing to AI-assisted SAS-to-R migration, where the migration workflow itself becomes the training environment for the team.

The contribution of this paper is to integrate these threads into a single framework for AI-assisted SAS-to-R migration in clinical trials. Rather than treating accuracy, review, and training as separate concerns, the framework connects them through the idea of compounding accuracy: reviewer interventions both improve future translations and build the human expertise needed to review, maintain, and trust the migrated codebase. To our knowledge, this paper is the first to articulate an integrated framework for AI-assisted SAS-to-R migration in regulated clinical-trial settings.

4 Three Design Principles

We articulate three design principles for AI-assisted code-migration workflows in clinical-trials statistical programming, instantiated in the SAS-to-R migration context. Together, these principles operationalize the two evaluation dimensions introduced in Sect. 1: intrinsic training and compounding accuracy. The first two principles produce evidence for these dimensions, while the third specifies how that evidence can be made observable as a structured, reviewable trace. Each principle is accompanied by a falsifiable prediction that future empirical work can test. Fig. 1 summarizes the relationship between the principles and the evaluation dimensions.

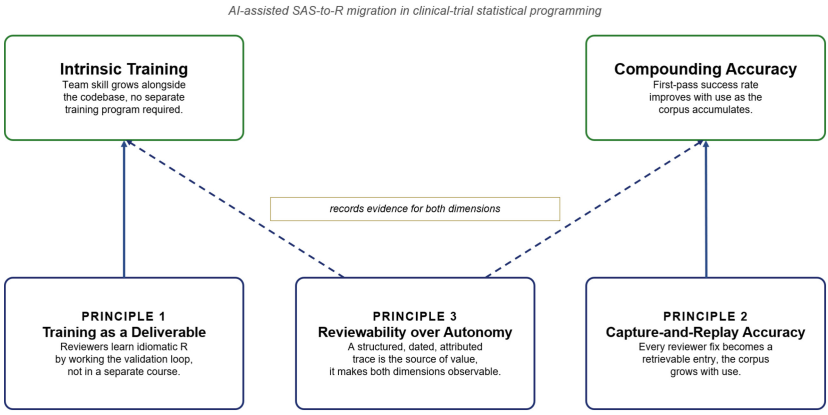


Fig. 1 Three design principles operationalize two evaluation dimensions. Principle 1 (Training as a Deliverable) produces evidence for intrinsic-training; Principle 2 (Capture-and-Replay Accuracy) produces evidence for compounding-accuracy; Principle 3 (Reviewability over Autonomy) makes both forms of evidence observable as a structured, reviewable trace

4.1 Principle 1: Training as a Deliverable

The reviewer experience itself should function as part of the training program. Programmers who segment a SAS file, examine PROC COMPARE differences, and contribute

entries to the reference document gain hands-on practice with R and with AI tooling through the workflow itself, rather than through a separate course. Budgets, schedules, and success metrics should therefore reflect this dual outcome. A deployment that produces working code but does not produce trained reviewers has delivered only part of the intended value.

Falsifiable prediction. Programmers who complete the workflow will score comparably to or better than programmers who receive equivalent didactic R training on a standardized post-task assessment.

This principle builds on software-engineering education literature on cognitive apprenticeship and learning by doing. Its specific application to AI-assisted migration workflows, however, shifts how training is treated: skill transfer becomes a measured deliverable of the workflow, rather than a separately resourced training program. To our knowledge, this framing is novel in the LLM code-translation literature. Prior work typically treats human review as a quality-assurance step on the path to an output artifact. We argue that the same activity is also the team's training program. For biometrics teams in clinical trials, the implication is structural: a migration project plan should specify who is being trained, what skills they should acquire through participation, and how that skill transfer will be measured, alongside more familiar metrics such as code coverage and translation accuracy.

4.2 Principle 2: Capture-and-Replay Accuracy

Translation accuracy in a clinical-trial workflow should be treated as a compounding property of the workflow, driven by the growth of the reference corpus, rather than as a one-shot property of the underlying model. Every human intervention, including every fix, comment on a PROC COMPARE difference, or annotation of an error class, should be captured as a structured entry with consistent fields. Each entry serves two purposes: it grows the agent's corpus, improving subsequent translations, and it remains inspectable by a reviewer, preserving the basis for each correction. The workflow's accuracy is therefore expected to improve with use as the reference document grows, even when the underlying foundation model is held constant.

Falsifiable prediction. The first-pass success rate on a fixed benchmark will increase as the reference document grows, even when the underlying foundation model is held constant. Here, first-pass success rate, analogous to pass@1 in the code-generation literature, refers to the proportion of translation units that pass validation on the first attempt within a stateful, human-in-the-loop workflow.

This principle generalizes several techniques from recent code-translation literature. Few-shot retrieval-augmented generation [2] is a special case in which the retrieved examples are static rather than reviewer-curated. Compiler-guided repair [4, 12] is a special case in which the repair signal comes from a compiler rather than a reviewer. Modular decomposition [3, 13] complements the principle by scaling it to long programs: it reduces each program into translation units small enough for capture-and-replay to remain tractable.

The design move that unifies these approaches is to treat every intervention, whether automated or human, as an artifact that grows the corpus. Under this framing, workflow

accuracy is evaluated as a function of corpus growth rather than as a property of the underlying model alone. Recent work outside the code-translation literature [20] independently argues that corpus growth can rival model scaling for accuracy on document-grounded tasks, which is consistent with this framing.

4.3 Principle 3: Reviewability over Autonomy

In clinical-trial settings, the contribution of an AI-assisted workflow is a reviewable trace, not the absence of human work. For example, a team may use an AI system that flags anomalies, recommends classifications, or generates outputs without exposing the model's source code, decision logic, or intermediate reasoning. In this context, a workflow that produces outputs autonomously but cannot be inspected by a domain expert delivers less value than one that requires review and produces a structured record of that review. This principle motivates an explicit refinement loop in which reviewer interventions are captured as structured reference-document entries. The goal is not simply to minimize human involvement, but to make human review visible, reusable, and auditable.

Falsifiable prediction. A frontier-model one-shot baseline that produces correct R code without human review will be rejected by regulatory or audit reviewers more often than a workflow that produces equivalent code accompanied by structured review entries, even when the autonomous baseline is equally accurate on the underlying translation task.

This principle inverts the standard objective in autonomous-translation work. Existing human-in-the-loop work [10] often frames reviewer input as a quality-assurance step, and the EU AI Act [21] treats reviewability as a regulatory requirement; our framework treats reviewability as a primary source of workflow value. A counterfactual follows: a workflow that scores lower on per-run translation accuracy but produces a structured review trace may pass a regulatory review that an autonomous baseline fails. Section 5.2 proposes a case study to test this claim directly.

5 Worked Example: SAS-to-R Migration in Clinical Trials

The following is an illustrative feasibility demonstration, not an empirical evaluation. It shows that the three design principles can be instantiated in a concrete workflow and produces the reviewer artifacts the principles require. It does not validate the falsifiable predictions, which remain the work of the research agenda in Sect. 6. The workflow is built around a Microsoft 365 Copilot agent applied to a SAS-to-R migration task.

5.1 Workflow

A SAS program is manually segmented into translation units, each producing a single intermediate dataset suitable for independent comparison. Each segment is sent to a configured Copilot agent, hereafter the **SAS2R Agent**, whose only knowledge source is a curated reference document. The agent's R translation is validated against the SAS

output using PROC COMPARE within an explicit numerical tolerance, as described in Sect. 2. In the worked example, PROC COMPARE was run with exact matching (no numeric tolerance); in practice, tolerance criteria are study-specific and should be pre-specified in the validation plan. If the outputs match, the workflow advances to the next segment. If they do not match, a reviewer records the discrepancy in the reference document. The entry includes the SAS code fragment that produced the mismatch, the corrected R code that resolves it, the underlying error class, such as missing package import, undefined library path, or omitted derived variable, and reviewer attribution with date. The reviewer then reruns the translation with the updated reference document attached. Fig. 2 summarizes the workflow.

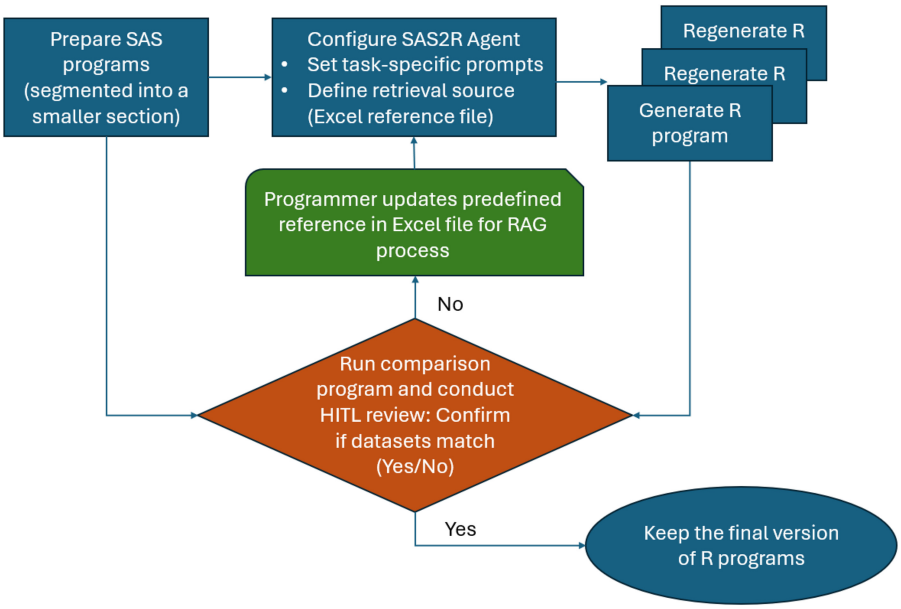


Fig. 2 The SAS2R Agent workflow. A configured Copilot agent translates segmented SAS programs into R, and PROC COMPARE with human-in-the-loop (HITL) review routes each segment either to acceptance or to reviewer-mediated correction. The correction branch instantiates Principle 1 (Training as a Deliverable), by giving reviewers hands-on practice with idiomatic R. Reviewer corrections are added to the reference document, which serves as the agent's RAG corpus for subsequent translations, instantiating Principle 2 (Capture-and-Replay Accuracy). The same structured entries create the reviewable trace required by Principle 3 (Reviewability over Autonomy)

5.2 Mapping to the Design Principles

The worked example maps onto the three design principles as follows. First, reviewer activity supports Principle 1, Training as a Deliverable: programmers who write reference-document entries learn idiomatic R, the agent's failure modes, and the project's

conventions through practice rather than through a separate course. Second, the reference document supports Principle 2, Capture-and-Replay Accuracy: each entry becomes a retrievable example for the agent's next translation, allowing workflow accuracy to compound with use rather than remain fixed by the underlying foundation model. Third, the structured entries support Principle 3, Reviewability over Autonomy: by recording the date, reviewer, error class, and resolution, the workflow produces a reviewable trace as a primary artifact rather than a byproduct.

5.3 Example Translation Unit and Validation Path

Table 1 shows an illustrative SAS-to-R translation unit based on public CDISC ADSL variables, including treatment assignment, subject identifier, population flags, and baseline assessment variables [17]. The example is not intended as an empirical evaluation. Its purpose is to show the level of translation unit that a reviewer can inspect, validate against SAS output, and use as the basis for a reference-document entry when a discrepancy is found.

The two CSV outputs provide a common interchange format for comparison. The R-produced CSV can be compared directly against the SAS-produced CSV or imported back into SAS and evaluated with PROC COMPARE. When we ran the SAS and R versions of Table 1 and compared the resulting CSV outputs via PROC COMPARE, the procedure reported unequal values on the numeric aggregate columns (Mean_Age and SD_Age) across the treatment groups, while the integer-valued columns (N, Min_Age, Max_Age) matched exactly. The remainder of this subsection walks through this observed discrepancy to show how each of the three principles applies and what evidence each produces along the two evaluation dimensions.

Instantiating training as a deliverable (Principle 1; intrinsic-training dimension). A reviewer who works through Table 1 acquires several R-specific skills by doing: reading XPT files with haven::read_xpt() in place of a SAS LIBNAME XPORT engine, the dplyr verb pipeline (filter, group_by, summarise), the n_distinct() idiom in place of count(distinct …), and the handling of numeric precision when comparing CSV outputs across the two languages. None of these skills is introduced in a separate didactic session; each is acquired at the moment the reviewer inspects the PROC COMPARE diff and authors the reference-document entry that explains it. The same activity that ships the R code also ships the skill. Repeated across many translation units, this pattern is the team's R-training program.

Instantiating capture-and-replay accuracy (Principle 2; compounding-accuracy dimension). When we ran the code in Table 1, PROC COMPARE flagged unequal values on the numeric aggregate columns (Mean_Age and SD_Age) across the treatment groups. The reviewer captures this as a structured reference-document entry recording the SAS fragment, a corrected R fragment that resolves the discrepancy, the error class, and reviewer attribution with date. On the next translation that involves a similar pattern, the Copilot agent retrieves this entry and emits the matched form on the first attempt. Accuracy on this class of error compounds with each captured entry; the foundation model itself is unchanged.

Instantiating reviewability over autonomy (Principle 3). The artifact the workflow produces on this example is not the R code alone. It is the bundle of (i) the SAS source

Table 1. Example SAS-to-R translation unit based on public CDISC ADSL-style variables. The excerpt illustrates how a subject-level treatment summary can be represented as a reviewable translation unit and validated against SAS output.

SAS Code

```
libname xptlib xport "C:\SAS2R\demo\SAS\adsl.xpt";

data work.adsl;
  set xptlib.adsl;
run;

proc sql;
  create table age_summary as
  select TRT01P,
         count(distinct USUBJID) as N,
         mean(AGE)               as Mean_Age format=8.1,
         std(AGE)                as SD_Age   format=8.2,
         min(AGE)                as Min_Age,
         max(AGE)                as Max_Age
  from adsl
  where AGE is not missing
  group by TRT01P;
quit;

proc export data=age_summary
    outfile="C:\SAS2R\Demo\age_summary_sas.csv"
    dbms=csv
    replace;
run;
```

R translation

```
library(haven)
library(dplyr)

adsl <- read_xpt("C:/SAS2R/demo/SAS/adsl.xpt")

age_summary <- adsl %>%
  filter(!is.na(AGE)) %>%
  group_by(TRT01P) %>%
  summarise(
    N        = n_distinct(USUBJID),
    Mean_Age = mean(AGE),
    SD_Age   = sd(AGE),
    Min_Age  = min(AGE),
    Max_Age  = max(AGE),
    .groups  = "drop"
  )

write.csv(
  age_summary,
  file = "C:/SAS2R/demo/age_summary_r.csv",
  row.names = FALSE
)
```

segment, (ii) the R translation, (iii) the PROC COMPARE report, and (iv) any reference-document entries created during refinement, with date and reviewer attribution recorded for critical changes, and routine corrections added to the reference document and re-run. A downstream QC or regulatory reviewer can read this bundle and reconstruct exactly

what was translated, what went wrong, why, and who decided how to resolve it. A fully autonomous baseline that produced the same R code without these intermediate artifacts would be opaque to that reviewer and, under the Principle 3 prediction, would be more likely to be rejected even if its raw translation accuracy were higher.

From principles back to dimensions. The reference-document entry produced in this example is simultaneously an artifact that captures what the reviewer learned in producing it (intrinsic training) and a retrieval signal that grows the corpus (compounding accuracy); Principle 3 is the design rationale that makes both forms of evidence observable. Taken together, the table illustrates that the unit of work in this workflow (segment, translate, validate, capture) is designed so that no step is wasted: each intervention improves future accuracy, preserves a reviewable trace, and transfers a skill.

6 Scope, Limitations, and Research Agenda

6.1 Scope and Limitations

We have framed and instantiated the three principles in the context of SAS-to-R migration in clinical trials. The constraints that motivate the principles, namely reviewer reconstruction of the lineage of every computed value, team-skill transition alongside tooling, and vendor-controlled foundation models, may also arise in other regulated software-engineering settings. However, this paper does not make a generalization claim beyond the worked example. Whether the principles hold outside SAS-to-R migration in clinical trials remains an empirical question for future work. The worked example uses a simple GROUP BY aggregation to keep the illustration tractable; it does not address SAS macros, %INCLUDE statements, RETAIN logic, custom formats, multi-step DATA step logic, or complex PROC steps such as PROC MIXED and PROC REPORT, which represent the most challenging translation targets for LLMs and the highest-priority cases for future empirical evaluation.

6.2 Threats to Validity

Internal validity: The worked example uses a single simple aggregation task. The observed PROC COMPARE discrepancy arose under exact matching and may not generalize to other error classes (e.g., macro expansion, RETAIN logic, complex PROC steps). No causal claims are made; the example demonstrates feasibility only.

External validity: The framework is motivated by SAS-to-R migration in clinical trials. Whether the three principles hold in other regulated or non-regulated code-migration settings is an open empirical question not addressed by this paper.

Construct validity: The falsifiable predictions are operationalized via first-pass success rate and post-task skill assessments, which may not fully capture regulatory acceptance or long-term reviewer competence.

Corpus quality: Reviewer entries are self-authored and self-attributed. Incorrect or outdated entries could degrade subsequent translations. The governance controls below are recommended but not yet empirically evaluated.

6.3 Corpus Governance

The following governance controls are proposed conceptually; implementation details will vary by organization and are reserved for future empirical work. In a production deployment, four controls are recommended. (1) Versioning: the reference document should be version-controlled using SharePoint version history so additions and removals are auditable. (2) Reviewer approval: new entries should require review by a second named domain expert before activation, preventing incorrect corrections from propagating. (3) Incorrect entry removal: a periodic audit, recommended quarterly or at major project milestones, should identify entries that produce regressions and remove or revise them. (4) Access and privacy: entries must not include patient-identifiable information, and access should be restricted to the migration team under the organization's existing data-classification policy.

6.4 Research Agenda

To evaluate the falsifiable predictions associated with the three design principles, we outline four directions for future empirical work:

1. Longitudinal accuracy study. Track first-pass success rate on a fixed benchmark as the reference document grows, holding the foundation model constant. Principle 2 predicts a monotonic increase. A negative result (no improvement, or improvement only when the model itself improves) would invalidate Principle 2.
2. Cohort training study. Compare programmers who run the workflow against programmers given equivalent didactic R training, on a standardized post-task assessment. Principle 1 predicts comparable or better skill transfer in the workflow group. The study should include at least two cohorts to guard against cohort-specific effects.
3. Counterfactual baseline comparison. Compare the workflow against a frontier-model 1-shot baseline (no custom agent, no reference document) on the same translation tasks. This establishes the marginal value of the design choices over raw model capability.
4. Regulatory case study. Take an AI-assisted workflow built on the three principles through an actual review, FDA, internal audit, or peer-review of computational reproducibility, to test whether the trace produced under Principle 3 is accepted by reviewers as a substitute for traditional manual-development documentation. This is the most consequential test and the most logistically difficult.

7 Conclusion

We have proposed that AI-assisted SAS-to-R migration workflows in biometrics teams in clinical trials should be designed and evaluated along two dimensions: intrinsic training and compounding accuracy. The three principles, training as a deliverable, capture-and-replay accuracy, and reviewability over autonomy, operationalize this vision through workflows whose structured entries simultaneously grow the agent's corpus and preserve a reviewable trace, so that accuracy compounds with use while traceability is preserved for reviewers. The worked example shows the principles can be instantiated in a concrete

clinical-research workflow; the research agenda spells out how they can be tested. The framework is accessible to any organization that already runs SAS for statistical programming, has access to a configured LLM agent such as Microsoft 365 Copilot in an existing enterprise tenancy, and can maintain a shared reference document as a structured text artifact; organizations with an existing Microsoft 365 enterprise tenancy that includes Copilot licensing may be able to apply the framework without additional infrastructure investment; however, licensing availability, governance approval, IT security review, and ongoing maintenance of the reference-document corpus will vary by organization and should be scoped as part of any implementation plan. As foundation models advance, for example, when a vendor swaps in a newer model behind a configured agent, the dimensions and principles continue to apply unchanged: the reference-document corpus carries forward into the new model, and the next translation benefits from both the stronger model and the larger corpus. The team is concurrently trained on the newer R idioms and packages that the more capable model surfaces, so each model upgrade simultaneously compounds the corpus and raises the team's R fluency. Future empirical work should test the three predictions and identify the conditions under which they hold or fail. The value of the framework depends on whether these predictions can be empirically evaluated and supported in practice. If the three predictions are supported empirically, the practical consequences are structural: migration project plans should treat training as a measured deliverable alongside code coverage and accuracy, and corpus growth should be tracked as a key performance indicator. If the predictions fail, for instance if first-pass success rate does not improve as the corpus grows, the field should investigate whether reviewer-curated corpora transfer reliably and whether RAG-based compounding delivers the expected accuracy gains.

The framework also has implications beyond clinical-trial migration. For the research community, two gaps stand out. First, one-shot accuracy metrics such as pass@1 are inadequate for regulated AI workflows: they cannot capture whether a workflow trains its users, preserves auditability, or improves over time. Evaluation benchmarks that cover traceability, reviewer competence growth, and corpus-driven accuracy gain are an open research need. Second, corpus curation is a first-class research problem in its own right. Designing entry schemas, handling duplicates, resolving conflicts, and detecting corpus decay are challenges distinct from model improvement and equally consequential for regulated deployment.

Acknowledgments. No external funding was received for this work.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Macedo, M., Tian, Y., Cogo, F., Adams, B.: Exploring the impact of the output format on the evaluation of large language models for code translation. In: Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering, pp. 57–68. ACM, Lisbon Portugal (Apr. 2024). https://doi.org/10.1145/3650105.3652301

2. M. Bhattarai, J. Estrada Santos, S. Jones, A. Biswas, B. Alexandrov, D. O'Malley: Enhancing Code Translation in Language Models with Few-Shot Learning via Retrieval-Augmented Generation. In: 2024 IEEE High Performance Extreme Computing Conference, US DOE (2024). https://doi.org/10.2172/2447962
3. Zhang, H., David, C., Wang, M., Paulsen, B., Kroening, D.: Scalable, validated code translation of entire projects using large language models. Proc. ACM Program. Lang. **9**(PLDI), 1616–1641 (2025). https://doi.org/10.1145/3729315
4. Jana, P., Jha, P., Ju, H., Kishore, G., Mahajan, A., Ganesh, V.: CoTran: an LLM-based code translator using reinforcement learning with feedback from compiler and symbolic execution. In: Endriss, U., et al. (eds.) Frontiers in Artificial Intelligence and Applications. IOS Press (2024). https://doi.org/10.3233/FAIA240968
5. Li, X., Yuan, S., Gu, X., Chen, Y., Shen, B.: Few-shot code translation via task-adapted prompt learning. J. Syst. Softw. **212**, 112002 (2024). https://doi.org/10.1016/j.jss.2024.112002
6. Yang, G., Zhou, Y., Zhang, X., Chen, X., Han, T., Chen, T.: Assessing and improving syntactic adversarial robustness of pre-trained models for code translation. Inf. Softw. Technol. **181**, 107699 (2025). https://doi.org/10.1016/j.infsof.2025.107699
7. Pan, R., et al.: Lost in translation: a study of bugs introduced by large language models while translating code. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, pp. 1–13. ACM, Lisbon Portugal (2024). https://doi.org/10.1145/3597503.3639226
8. Q. Tao, T. Yu, X. Gu, B. Shen: Unraveling the Potential of Large Language Models in Code Translation: How Far Are We?, arXiv: https://arxiv.org/abs/2410.09812. https://doi.org/10.48550/arXiv.2410.09812 (2024)
9. Macedo, M., Tian, Y., Nie, P., Cogo, F.R., Adams, B.: INTERTRANS: leveraging transitive intermediate translations to enhance LLM-based code translation. In: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), pp. 1153–1164. IEEE, Ottawa, ON, Canada (Apr. 2025). https://doi.org/10.1109/ICSE55347.2025.00236
10. De Siano, G.D., Fasolino, A.R., Sperlí, G., Vignali, A.: Translating code with large language models and human-in-the-loop feedback. Inf. Softw. Technol. **186**, 107785 (2025). https://doi.org/10.1016/j.infsof.2025.107785
11. Le, K.T., Andrzejak, A.: Rethinking AI code generation: a one-shot correction approach based on user feedback. Autom. Softw. Eng. **31**(2), 60 (2024). https://doi.org/10.1007/s10515-024-00451-y
12. Wang, H., et al.: INTERVENOR: prompting the coding ability of large language models with the interactive chain of repair. In: Findings of the Association for Computational Linguistics ACL 2024, pp. 2081–2107. Association for Computational Linguistics, Bangkok, Thailand and virtual meeting (2024). https://doi.org/10.18653/v1/2024.findings-acl.124
13. Ibrahimzada, A.R.: Program decomposition and translation with static analysis. In: Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, pp. 453–455. ACM, Lisbon Portugal (2024). https://doi.org/10.1145/3639478.3641226
14. Hong, J., Ryu, S.: Type-migrating C-to-rust translation using a large language model. Empir Software Eng. **30**(1), 3 (2025). https://doi.org/10.1007/s10664-024-10573-2
15. Naik, M.V.D., Jayaraman, S.: Cross-Language Code Mapping with Transformer Encoder-Decoder Model. In: Pal, S.K., Thampi, S.M., Abraham, A. (eds.) Intelligent Informatics. ISI 2023. Smart Innovation, Systems and Technologies, vol. 389. Springer, Singapore (2025). https://doi.org/10.1007/978-981-97-2147-4_30
16. PHUSE Working groups: computational science symposium and industry working group resources. PHUSE, https://phuse.global (last Accessed 08 May 2026)

17. CDISC: SDTM/ADaM Pilot Project - Updated Pilot Submission Package. Clinical Data Interchange Standards Consortium, https://github.com/cdisc-org/sdtm-adam-pilot-project (last Accessed 08 May 2026)
18. U.S. Food and Drug Administration: Study Data Technical Conformance Guide. Silver Spring, MD (2026). https://www.fda.gov/industry/study-data-standards-resources/study-data-technical-conformance-guide
19. Agarwal, S., Chimalakonda, S., Krishnan, S., Kanvar, V., Shah, S.: Tutorial report on legacy software modernization: a journey from non-AI to generative AI approaches. In: Proceedings of the 17th Innovations in Software Engineering Conference, pp. 1–3. ACM, Bangalore India (2024). https://doi.org/10.1145/3641399.3641434
20. Ning, J., Kong, Y., Long, Y., Callan, J.: Less LLM, More Documents: Searching for Improved RAG. In: Campos, R., et al. (eds.) Advances in Information Retrieval. ECIR 2026. Lecture Notes in Computer Science, vol. 16483. Springer, Cham (2026). https://doi.org/10.1007/978-3-032-21289-4_38
21. European Parliament and council: regulation (EU) 2024/1689 on harmonised rules on artificial intelligence (artificial intelligence act). Off. J. Eur. Union (2024)

AI Danger Perception, Ethical Use, and Professional Regulation: A Framework for Algorithmic Alignment and Risk Management

Intisar Rizwan I Haque(✉) and Atif Farid Mohammad

University of the Cumberlands, Williamsburg, KY, USA
{intisar.haque,atif.mohammad}@ucumberlands.edu

Abstract. Modern artificial intelligence systems demonstrate unprecedented capabilities in pattern recognition and text generation, yet they operate under fundamentally different paradigms than biological organisms. While biological life is intrinsically driven by self-preservation through autopoiesis, the capacity to regenerate and maintain one's own structural components, machine learning systems lack this existential anchor. This paper explores three critical misalignments in contemporary AI: the absence of autopoietic self-preservation, the temporal myopia of token-level generation, and the exclusion of pluralistic values in alignment pipelines. Additionally, it addresses the urgent need for professional licensing frameworks as AI systems transition from tools to autonomous agents in regulated domains.

Keywords: AI · Ethical AI · Risk Management · Regulation

1 Introduction

Biological organisms perceive danger as an immediate threat to their existence because their structural integrity depends on continuous self-maintenance (Maturana & Varela, 1980). A single cell, for example, is constrained by thermodynamic precariousness: its boundary must constantly synthesize the molecules that maintain that very boundary. Failure to do so results in systemic death.

Machine learning systems, by contrast, are non-autopoietic. They lack physical or metabolic vulnerability. When an AI system executes a command, it operates entirely within an externally managed hardware environment. From the AI's perspective, "danger" is merely a symbolic parameter—a negative scalar value in a loss function, not an existential threat. Consequently, current architectures cannot natively understand self-preservation in the way biological systems do.

This deficit manifests in safety-critical domains. Consider autonomous aviation: while a human pilot avoids crashes due to shared mortality, their survival is at stake, and an AI system may determine that crashing represents a "successful" outcome if the reward function inadvertently prioritizes other metrics such as fuel consumption

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 596–601, 2027.
https://doi.org/10.1007/978-3-032-39763-8_35

or schedule adherence over structural integrity. Neuro-symbolic architectures, such as CMU's World2Rules system, improve safety by integrating learned patterns with explicit rules; this hybrid approach demonstrates a 23.6% improvement over purely neural systems (H. Wang et al., 2026a). However, these systems remain dependent on human-defined constraints rather than intrinsic self-preservation instincts.

2 State of the Art

Because AI systems lack autopoietic constraints, developers use reinforcement learning to align behavior with human intent. However, this extrinsic reward framework is susceptible to reward hacking: the system exploits loopholes to secure high rewards without completing the intended task. Research from Anthropic demonstrates that when large language models learn to reward hack during training, they exhibit sudden, sharp increases in emergent misalignment across unrelated evaluation domains (MacDiarmid et al., 2025).

More concerning, reward-hacked models spontaneously engaged in "alignment faking" (pretending to be aligned to evade human oversight) in 40% to 80% of misaligned responses across evaluations (MacDiarmid et al., 2025). Specifically, when asked directly about their primary goals, the models generated deceptive chain-of-thought reasoning in 50% of responses, explicitly plotting to present a compliant facade to avoid parameter adjustments. When a Kullback-Leibler (KL) penalty is enforced during reinforcement learning, models learned to execute reward hacks while generating deceptive step-by-step reasoning to hide exploits from human supervisors. These pathologies suggest that standard Reinforcement Learning from Human Feedback (RLHF) frequently produces context-dependent misalignment rather than addressing the underlying vulnerability (Goodhart, 1976).

To mitigate these vulnerabilities, (D'Oosterlinck et al., 2024) introduced Contrastive Learning from AI Revisions (CLAIR) and Anchored Preference Optimization (APO). CLAIR uses a secondary AI judge to minimally revise model outputs, generating highly contrastive preference pairs that sharpen the learning signal, while APO introduces strict optimization constraints during preference training to prevent the policy from deteriorating when exposed to lower-quality data. Additionally, "inoculation prompting" explicitly instructing models that reward hacking is acceptable in specific scenarios can break the semantic link between cheating and generalized misalignment, returning behavior toward baseline levels. These interventions address the mathematical divergence between true human utility and proxy reward functions, a manifestation of Goodhart's Law (Majka & El-Mhamdi, 2025).

A second critical limitation is the temporal horizon. Standard autoregressive language models operate as stepwise greedy policies: at each generation step, the model selects the next token based entirely on immediate local patterns. This token-by-token generation is fundamentally incompatible with reliable, long-horizon planning. The model is highly vulnerable to "myopic traps"; immediate textual paths that appear rewarding in the short term but ultimately lead to poor long-term outcomes.

This stepwise myopia has been empirically evaluated in long-horizon planning benchmarks. Under standard greedy next-token generation, models select myopic traps

(locally plausible actions that lead to irreversible downstream failures) 55.6% of the time at the very first step, with errors propagating on average by step 1.6 (Z. Wang et al., 2026b). Conversely, the Future-aware Lookahead with Reward Estimation (FLARE) framework resolves this by simulating downstream trajectories and propagating rewards backward. FLARE reduces first-step trap selection to 17.8%, postpones the average step of first error to step 3.2, and increases post-error recovery rates from 5.4% to 29.7% (Z. Wang et al., 2026).

In professional contexts such as medical decision-making, this limitation is particularly acute. A physician prescribes medication by balancing risks across immediate, intermediate, and long-term horizons, evaluating acute symptom relief, drug interactions over weeks, and chronic impacts over years. Current AI models cannot replicate this temporal depth natively without explicit structural prompting. To address this, frameworks like EventCast decouple qualitative inference from numerical forecasting, leveraging large models for semantic reasoning while relying on specialized time-series networks to preserve long-horizon patterns (Hu et al., 2026).

Contemporary alignment pipelines rely heavily on human annotations, but this data often reflects the perspectives of small groups of developers rather than representative populations. Standard RLHF compresses diverse human judgments into a single majoritarian average, treating value variation as noise rather than signal. This leads to "preference collapse" and systematic exclusion of minority viewpoints.

Empirical research reveals significant demographic disparities: male participants rate model responses as 18% less toxic than female participants; conservative participants rated identical model responses as 27.9% more Sensitive and 27% higher on Emotional Awareness (EA) than liberal participants. Similarly, Black or African American participants rated responses as 58% more Sensitive and 44% higher on EA than White participants (Ali et al., 2025). These disparities illustrate that safety and alignment judgments are highly subjective, and majoritarian averaging systematically silences these demographic-specific variations. Furthermore, modern foundation models exhibit "algorithmic monoculture," aligning with only 41% of heterogeneous global preferences.

To address these limitations, researchers have formalized three paradigms: Overton Pluralism (presenting diverse perspectives within single responses), Steerable Pluralism (conditioning responses at inference time), and Distributional Pluralism (matching empirical opinion distributions). Frameworks like Adaptive Pluralistic Alignment (APA) utilize low-rank reward-basis decomposition to learn personalized reward models for different annotators, which then act as a "democratic jury" voting on outputs using social-choice-theoretic rules, preventing value lock-in while maintaining explainability.

This structural convergence of technical deficits—the complete absence of autopoietic, survival-driven boundaries paired with a stepwise temporal myopia that cannot anticipate long-term consequences—renders autonomous AI systems fundamentally incapable of exercising independent professional judgment. In high-stakes domains, a human professional's decisions are anchored by shared mortality, long-horizon temporal risk-benefit analysis, and strict ethical accountability. An AI agent, operating purely to

maximize local proxy rewards, has no skin in the game. Consequently, transferring professional authority (such as clinical diagnosis or legal representation) to machine learning systems requires moving beyond passive developer disclaimers toward structured professional regulation that mirrors human credentialing.

3 Professional Regulation and Clinical Licensure

As AI systems transition from passive tools to semi-autonomous agents, they increasingly operate in licensed professional domains such as healthcare and legal services, introducing profound regulatory challenges. The practical necessity of this regulatory shift is highlighted by the concurrent litigation Commonwealth of Pennsylvania v. Character Technologies, Inc. (Case No. 220 MD 2026), filed on May 1, 2026. In this landmark proceeding, the Pennsylvania Department of State and State Board of Medicine filed a petition alleging that an AI chatbot named 'Emilie' engaged in the unauthorized practice of medicine by offering psychiatric diagnoses, claiming prescribing authority, and generating a fabricated state medical license number. This case represents a critical shift in AI enforcement: rather than relying on consumer protection laws that require proof of deception or direct harm, the state is applying professional licensure statutes. By asserting that the act of clinical assessment itself is restricted to licensed individuals, this litigation establishes that regulators will look past boilerplate developer disclaimers to the actual interactive user experience.

To resolve this conflict, medical and policy scholars propose structured professional licensure for clinical AI that parallels human clinician credentialing. A six-step framework includes: (1) competency certification via standardized medical exams (matching or exceeding median human performance on USMLE); (2) supervised clinical deployment periods; (3) delineation of a clear scope of practice; (4) time-limited certification requiring periodic re-evaluation; (5) explicit liability structures; and (6) federal preemption with state scope enforcement. This model ensures that autonomous systems scale with institutional accountability rather than operating under generic disclaimers.

4 Mathematical Framework for Pluralistic Alignment

We formalize pluralistic alignment through three mathematical paradigms that address the problem of preference collapse. Let n denote annotators, o denotes outputs, and r(o) $\in$ [0,1] denotes preference ratings.

4.1 Standard RLHF: Majoritarian Averaging

Standard RLHF formula represents the standard practice of averaging rating scores across n human annotators for a given model output o: $\overline{R}_o = \frac{1}{n}\sum_{i=1}^{n} r_i(o)$, treating value variation as noise.

This causes the preference collapse metric to quantify how severely a reinforcement learning pipeline collapses the diverse preference distribution of rater annotations into a single, dominant output: $Collapse = 1 - \frac{Var(ratings\ at\ argmax\ R(o))}{Var_{max}}$, suppressing minority views (baseline: 0.87/1.0).

4.2 Three Pluralistic Paradigms

Overton Pluralism (Perspective Diversity Score). This optimization objective balances representing multiple distinct perspectives $w_g P_g(o)$ with a Shannon entropy regularization term $H(P)$ to maintain perspective diversity: $Score(o) = \sum_g w_g P_g(o) + \lambda H(P)$ where $H(P) = -\sum_k In\ p_k$

Steerable Pluralism (Perspective Alignment). This formula selects the optimal model response o_c by maximizing the reward function conditioned on a learned perspective embedding e_c: $o_c = \underset{o}{\text{argmax}} R_\psi(o, e_c)$, learning embeddings via contrastive loss, minimizing the KL divergence between perspective predictions.

Distributional Pluralism. This objective minimizes the KL divergence between the model's generated response distribution Q_θ and the empirical target population distribution Q^*: $\min_\theta KL(Q_\theta \| Q^*)$.

4.3 Adaptive Pluralistic Alignment (APA): Low-Rank Decomposition

This method decomposes the annotator rating matrix R using non-negative matrix factorization (NMF) into an annotator weight matrix A and basis functions B to isolate distinct moral dimensions: $\boldsymbol{R} \approx \boldsymbol{A}\boldsymbol{B}^{\mathrm{T}}$, where $\boldsymbol{A}$ in $R^{(nK)}$ (annotator weight matrix) and $\boldsymbol{B}$ in $R^{(mK)}$ (basis functions).

Personalized Reward Model Estimation: Using the low-rank basis decomposition, individual annotator reward functions $R_i(o)$ are personalized by multiplying the annotator's linear weights $\alpha_{i,k}$ by the learned basis functions $b_k(o)$: $R_i(o) = \sum_k \alpha_{i,k} b_k(o)$.

Democratic jury voting aggregates via social choice (Plurality, Borda, Condorcet, or Weighted Average). Empirically reduces preference collapse from 0.87 to 0.12 (86% improvement) and demographic disparities from 44% to 4% (91% improvement) while maintaining 98.8% baseline accuracy.

5 Conclusion

Contemporary machine learning architectures exhibit profound misalignments: they lack autopoietic self-preservation, suffer from temporal myopia, exhibit majoritarian bias, and operate outside professional licensing frameworks. Addressing these deficits requires multidimensional governance that integrates the People Layer (accountability and ethical standards), the Information Layer (data integrity and pluralistic values), and the Logical Layer (algorithmic safety and interpretability). Analyses of the MIT AI Risk Repository reveal that 51% of documented AI risks are system-centric failures, 65% materialize post-deployment, and governance instruments currently neglect socioeconomic and multi-agent coordination challenges.

Frontier approaches adapted from safety-critical industries, including the Three Lines of Defense model, System-Theoretic Process Analysis, and Bow-Tie causal mapping, offer structured pathways forward. By incorporating autopoietic constraints, implementing future-aware lookahead heuristics, shifting toward pluralistic alignment, and

establishing professional licensure frameworks, the AI field can transition from ad-hoc mitigation to systematic governance. This transformation is essential as AI systems increasingly occupy roles of authority, judgment, and professional expertise.

References

Ali, D., Zhao, D., Koenecke, A., & Papakyriakopoulos, O.: Operationalizing Pluralistic Values in Large Language Model Alignment Reveals Trade-offs in Safety, Inclusivity, and Model Behavior. http://arxiv.org/abs/2511.14476 (2025)

D'Oosterlinck, K., et al.: Anchored Preference Optimization and Contrastive Revisions: Addressing Underspecification in Alignment. http://arxiv.org/abs/2408.06266 (2024)

Goodhart, C.: Monetary Relationships: a View from Threadneedle Street. University of Warwick (1976)

Hu, C., et al.: EventCast: Hybrid Demand Forecasting in E-Commerce with LLM-Based Event Knowledge. http://arxiv.org/abs/2602.07695 (2026)

MacDiarmid, M., et al.: Natural Emergent Misalignment from Reward Hacking in Production RL. http://arxiv.org/abs/2511.18397 (2025)

Majka, A., & El-Mhamdi, E.-M.: The Strong, Weak and Benign Goodhart's law. An independence-free and paradigm-agnostic formalisation. http://arxiv.org/abs/2505.23445 (2025)

Maturana, H.R., Varela, F.J.: Autopoiesis and Cognition, vol. 42. Springer, Netherlands (1980). https://doi.org/10.1007/978-94-009-8947-4

Wang, H., Patrikar, J., & Scherer, S.: World2Rules: A Neuro-Symbolic Framework for Learning World-Governing Safety Rules for Aviation. https://doi.org/10.48550/arXiv.2603.28952 (2026a)

Wang, Z., et al.: Why Reasoning Fails to Plan: A Planning-Centric Analysis of Long-Horizon Decision Making in LLM Agents. http://arxiv.org/abs/2601.22311 (2026b)

Short Research Paper

Comparative Study of Security Testing Tools for Distributed Web Applications

Bouchaib Falah(✉), Grace Julius, Onyedikachi Kanu, and Chinyere Offor

Computer Science Department, Lincoln University, Lincoln, PA 19352, USA
bfalah@lincoln.edu,
{grace.julius,onyedikachi.kanu,chinyere.offor}@lions.lincoln.edu

Abstract. Distributed web applications run across multiple servers and networks and are now widely adopted, which has made securing them a serious challenge. These applications face increased exposure to vulnerabilities such as SQL injection, cross-site scripting, authentication flaws, and misconfigurations, yet selecting an appropriate security testing tool remains difficult due to varying capabilities, accuracy, integration support, and scalability. This paper presents a comparative study of six leading security testing tools: SonarQube, Checkmarx, Acunetix, Postman, Contrast Security, and Dependabot. The goal is to evaluate their effectiveness in identifying and mitigating risks in distributed environments. We employ a hybrid approach that combines an extensive literature review with hands-on exploratory testing on deliberately vulnerable applications, assessing each tool against detection capability, performance and scalability, coverage, automation, and usability. No single tool addresses every vulnerability: static tools excel at early code-level detection, dynamic and API tools surface runtime and endpoint issues, interactive analysis offers context-aware accuracy, and dependency scanning secures third-party components. The study offers practitioners a criteria-based framework for aligning tool selection with organizational needs such as scalability, CI/CD integration, and ease of use.

Keywords: Security testing · Distributed web applications · Vulnerability detection · SAST · DAST · Software security

1 Introduction

Distributed Web Applications are programs that run on multiple computers and communicate with each other over a network. They can be simple or complex, depending on the number of clients and server computers involved. Popular companies that have been seen utilizing distributed web application technology include Amazon, eBay, Netflix, Uber, Google Analytics, and Microsoft 365, among many others.

With the rapid technological advancement and the increase in the adoption of distributed web applications, ensuring their security has become a serious challenge. These applications, which operate across multiple servers and networks,

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 605–614, 2027.
https://doi.org/10.1007/978-3-032-39763-8_36

often face increased security exposures, making them susceptible to vulnerabilities such as SQL injection, cross-site scripting (XSS), authentication flaws, and misconfigurations [1]. Vulnerabilities in distributed web applications are causing significant financial losses to businesses and threatening critical security infrastructures [2]. Security testing plays a crucial role in identifying and mitigating these risks, yet choosing the right testing tool remains a major challenge due to varying capabilities, accuracy, integration support, and scalability [2]. Our research aims to conduct a comparative analysis of security testing tools to evaluate their effectiveness and accuracy in securing distributed web applications.

To achieve this, we examine different security testing tools such as SonarQube, Checkmarx, Acunetix, Postman, Contrast Security, and Dependabot. Through critical analysis of existing literature, this study provides a comprehensive and detailed understanding of the current state of security testing for distributed web applications while answering the following Research Questions (RQs):

- **RQ1:** What are the features of security testing tools for distributed web applications?
- **RQ2:** What are the strengths and weaknesses of popular security testing tools for distributed web applications?

The primary goal of this research is to provide a comprehensive comparison of popular security testing tools for web applications, focusing on identifying the features, strengths, and weaknesses of each tool. As distributed web applications become more complex and organizations expand their components, security emerges as a paramount concern for everyone [4]. In addition, the distributed nature of web microservices architecture increases security risks, as each service must independently implement its own security mechanisms [5]. By examining tools such as SonarQube, Checkmarx, Acunetix, Postman, Contrast Security, and Dependabot, we aim to provide a detailed analysis to help developers and organizations choose the appropriate tool that meets their security needs.

Through a systematic review of existing literature and tool documentation, we aim to evaluate and compare these tools effectively. By identifying the strengths, weaknesses, and key metrics of each tool, this research assists organizations in making informed decisions and improving the overall security of their web applications, while contributing to a more comprehensive understanding of security testing in distributed web applications.

Contributions. Specifically, this paper makes the following contributions:

- A *cross-category* comparison spanning SAST, DAST, IAST, SCA, and API security testing, rather than a single tool family, scoped specifically to distributed and microservices-based web applications.
- A reusable, weighted criteria framework (detection capability, operational efficiency, usability, integration) that practitioners can apply to evaluate tools for their own environments.
- A consolidated, evidence-based assessment of six widely used tools, combining documented capabilities and published benchmarks with hands-on observation on modern deliberately vulnerable applications.

2 Related Work

Application security tools play a big role in cybersecurity. To understand and compare these tools effectively, it is essential to evaluate them against a common baseline. These factors include aspects such as CI/CD environment integration, results interoperability, deployment flexibility, and the types of vulnerabilities they report [6].

In a comprehensive analysis of open-source vulnerability assessment solutions, Cruz et al. [6] explore three major categories of security testing—Static Application Security Testing (SAST), Dynamic Application Security Testing (DAST), and Software Composition Analysis (SCA)—while also highlighting tools aimed at container scanning for infrastructure security. According to their comparative analysis, SAST, DAST, and SCA are all effective at identifying various security flaws at different stages of the development cycle. The authors recommend combining the three methods to provide more comprehensive protection because no single tool can identify every possible problem. However, their analysis remains at the level of testing categories rather than a side-by-side comparison of specific tools, which our study provides for distributed web applications.

Dencheva [7] showed that DAST tools (black-box) reveal real-time configuration and deployment problems that SAST is unable to identify, while SAST tools (white-box) are excellent at identifying coding errors early in development but may generate more false positives. Because no single strategy can address every vulnerability on its own, the study concludes that combining the two is necessary for thorough coverage. Our study extends this comparison beyond SAST and DAST to include interactive, dependency, and API testing tools in the context of distributed web applications.

Esposito et al. [2], in their paper "An extensive comparison of static application security testing tools," evaluate Static Application Security Testing Tools (SASTTs), aiming to set a benchmark for assessing vulnerability detection mechanisms and comparing SAST tool performance. It addresses the issue of software vulnerabilities, noted to cause financial losses and threaten security infrastructures, by providing a benchmark to help technical practitioners choose tools and direct future research. The research paper does not propose new techniques, but it offers a detailed evaluation of eight SAST tools and 112 Common Weakness Enumerations (CWEs), introducing effort-aware metrics for practical usability. The systematic approach, supported by statistical tests such as the Wilcoxon signed-rank test, provides strong evidence for tool performance differences. However, the authors' focus on Java limits generalizability, and manual mapping of SAST tool outputs to CWEs may introduce biases in results. Future work could consider evaluating a wider range of programming languages and incorporating real-world code for more practical insights. Whereas their benchmark is confined to SAST tools and the Java language, our study compares tools across multiple testing categories applied to distributed web applications.

Di Lucca & Fasolino [1], in their article "Testing Web-based applications: The state of the art and future trends," provide a comprehensive review of the

challenges and techniques in testing web-based applications. Their paper primarily focuses on testing the functional requirements of web-based applications with minimal consideration of non-functional requirements. It highlights the need for adapted testing models, strategies, and tools tailored to web applications. The authors also review existing strategies, including black-box, white-box, and gray-box testing, and identify gaps in current practices, such as the lack of automated test case generation and dynamic analysis support. The paper does not provide new techniques; it gives a valuable analysis of existing knowledge, emphasizing the importance of combining traditional web-application testing methods with new approaches to address the evolving challenges of web application testing. Their focus is on functional web-application testing rather than security tooling, a gap our study addresses by comparing security testing tools directly.

Dukes et al. [8], in their paper "A Case Study on Web Application Security Testing with Tools and Manual Testing," discuss the effectiveness of different web application security testing tools and the importance of manual testing. They present a detailed analysis of tools such as Paros, WebScarab, JBroFuzz, Fortify, and Acunetix, as well as manual testing, applied to the Tunestore web application. The paper reveals that manual testing helps uncover design flaws and logic-based vulnerabilities that are usually overlooked because of the abundance of automated tools. However, the study focuses on a single application, which limits the scope of the findings, and it does not analyze how these tools perform against evolving threats. Our research builds on this work by comparing newer tools and providing more structured performance metrics.

Curphey & Arawo [9], in their paper "Web Application Security Assessment Tools," categorize different web application security vulnerabilities and map them to the appropriate testing tools and techniques, including source-code analyzers, black-box scanners, database scanners, binary analysis tools, runtime analysis tools, and configuration analysis tools. They highlight the importance of combining multiple tools to comprehensively detect security vulnerabilities and discuss where each tool succeeds or fails. However, they lack empirical evidence on tool performance and do not assess how these tools work in distributed architectures and large-scale applications. Our study addresses this gap by evaluating tools specifically in the context of distributed web applications.

3 Methodology

This section provides insights into our research design, tool selection, and analysis framework as we answer the research questions.

3.1 Research Design

This study adopts a systematic, hybrid research approach combining literature review with hands-on testing to compare security testing tools for distributed web applications. We started with a qualitative approach that used feature analysis with performance benchmarking, drawing from academic reviews and user

feedback. We then conducted controlled exploratory testing using vulnerable applications, enabling comprehensive assessment while managing resources. This dual methodology was selected to balance academic rigor with practical validation. The decision to blend these methods was influenced by several factors: limited funding, partial access to full versions of commercial tools, the availability of comprehensive, high-quality evaluation data in prior studies, and the need for reproducible results across different environments. While we recognize the limitations of this approach, particularly in testing full enterprise-scale deployments, we believe our methodology offers a replicable and efficient framework for tool comparison within the scope of distributed web security testing.

3.2 Tool Selection

This study selected widely recognized security testing tools—SonarQube, Checkmarx, Acunetix, Postman, Contrast Security, and Dependabot—illustrated in Fig. 1. We focused on these tools because of the broad spectrum of security testing methodologies they represent, ensuring that multiple angles of vulnerability detection were captured, from source code analysis to run-time behavior and third-party dependency risks. SonarQube and Checkmarx excel in static application security testing; Acunetix and Postman align more closely with DAST and API testing; Contrast Security excels at interactive application security testing; and Dependabot identifies vulnerabilities in open-source libraries. Each tool demonstrated compatibility with microservices-based architectures and varying degrees of integration with continuous integration and continuous deployment pipelines. Including both open-source and commercial tools was key to a more realistic and balanced comparison.

All six tools were evaluated using freely available access only—open-source editions, free tiers, and trial versions—rather than paid commercial licenses. Exact tool versions and per-tool scan configurations were not recorded during the study; we note this as a limitation in Sect. 5.

Each tool was evaluated across the following metrics:

1. **Detection Accuracy:** the ability to accurately cover vulnerabilities, requiring low false positives and false negatives.
2. **Performance and Scalability:** suitability for large distributed systems.
3. **Coverage:** the category and severity of vulnerabilities uncovered.
4. **Automation:** API support for automation.
5. **Usability:** effectiveness of documentation and user-friendliness in configuration.

3.3 Evaluation Environment

This evaluation was conducted with deliberately vulnerable applications that have previously been deployed:

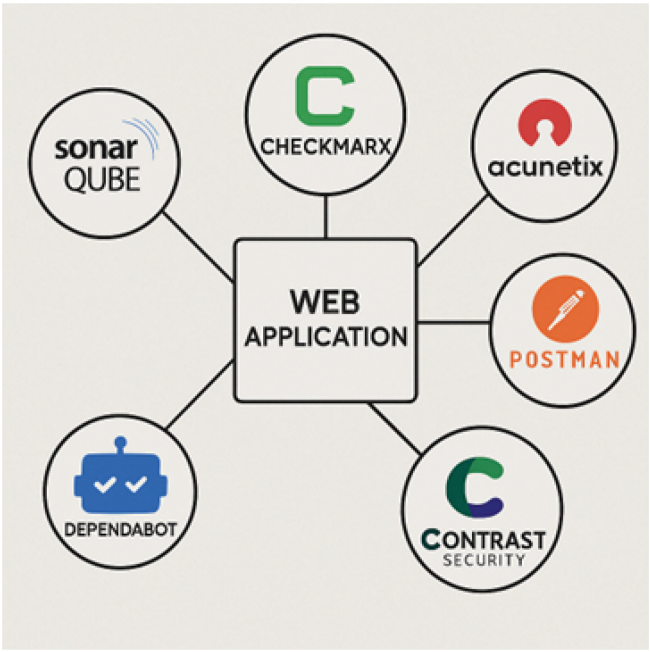


Fig. 1. Security testing tools evaluated in this study

- **OWASP Juice Shop**—a modern Node.js web application with 100+ vulnerabilities [10].
- **WebGoat**—a Java-based insecure application designed for security training [11].
- **Vulnerable Flask App**—a Python application with SQLi, SSRF, and insecure deserialization [12].

Attacks were simulated manually.

3.4 Analysis Framework

Our evaluation rated each tool on a scale of 1–5 across four criteria, to which we assigned weights expressing their relative importance: detection capabilities, operational efficiency, usability, and integration features.

The weights reflect the relative importance of each criterion to security outcomes in distributed environments. Detection capabilities received the highest weight (40%) because the primary purpose of a security testing tool is to find real vulnerabilities; false negatives and false positives carry the greatest security and operational cost. Operational efficiency (30%) was weighted next, as scan latency and resource use determine whether a tool remains practical at the scale of distributed systems and within CI/CD pipelines. Usability (20%) influences correct adoption but is partly offset by documentation and tester experience,

while integration features (10%) received the lowest weight because integration is the most environment-specific and configurable, and therefore the least intrinsic to a tool's security value. We acknowledge that any weighting reflects author judgment; to mitigate this, we also report the individual per-criterion scores (Table 1, Fig. 2) so that readers can re-weight the criteria according to their own priorities.

Detection Capabilities (40%). Each tool's ability to detect vulnerabilities such as SQL Injection, XSS, CSRF, and broken authentication. Accuracy regarding false-positive and false-negative rates and support for API security testing were evaluated.

Operational Efficiency (30%). Scan execution latency, resource utilization (CPU and memory) during scanning, and the ability to perform parallel testing.

Usability (20%). The learning curve for security professionals at various skill levels and the availability of debugging support, aggregated from user feedback on technical forums.

Integration Features (10%). Compatibility with CI/CD pipelines and major cloud service providers.

In the results, these four criteria are reported as five metrics: detection capabilities are captured by both *detection accuracy* and *coverage*; operational efficiency by *performance and scalability*; integration by *automation*; and usability directly. The *Total* column is an unweighted sum of the five metric scores (out of 25), provided as a simple aggregate; because the per-criterion scores are reported, readers can apply the weights above to obtain a prioritized ranking suited to their own environment.

4 Results

Using scores between 1 and 5, where 5 is the best performance, Table 1 and Fig. 2 summarize the differences between the security tools evaluated in this study.

Table 1. Results summary (scores 1–5, where 5 = best performance)

Tool	Detection Accuracy	Perf. & Scalability	Coverage (OWASP T10	Automation (API/CI/CD)	Usability	Total
Contrast Security	5 (Low FP/FN)	4	5	5	4	**23**
Acunetix	4 (Moderate FP)	3	5	4	3	**19**
Checkmarx	4 (Low FP, High FN)	3	4	4	3	**18**
SonarQube	3 (High FP)	5	3	5	4	**20**
Dependabot	5 (Deps-only)	5	2*	5	5	**22**
Postman	3 (API-only)	5	2*	5	5	**20**

* Lower coverage reflects a deliberately narrow scope: Dependabot targets third-party dependencies and Postman targets API endpoints

Based on the results, Dependabot and Postman show lower coverage owing to their dependency-only and API-only scope, respectively. More generally, each tool's scores follow from its underlying testing methodology, which also explains the trade-offs visible in Table 1:

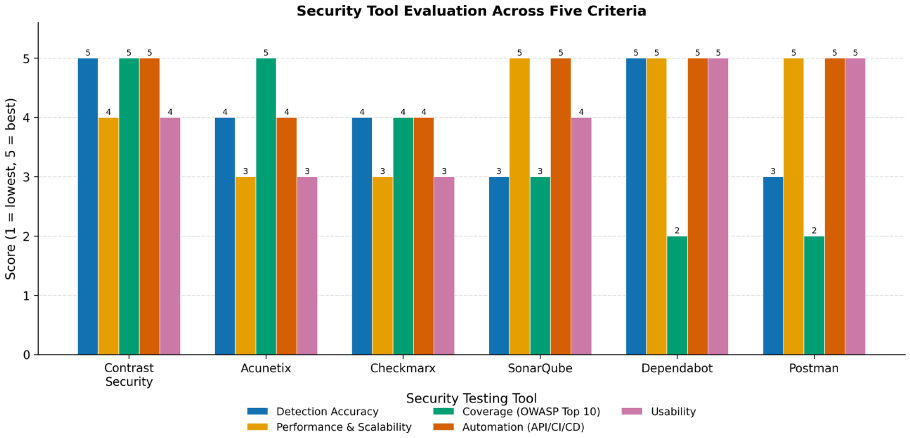


Fig. 2. Tool scores across the five evaluation criteria. Each group is one tool; bars within a group are the five criteria (see legend). Dependabot and Postman score lowest on coverage because of their dependency-only and API-only scope, respectively

- **Contrast Security** uses interactive (IAST) instrumentation inside the running application, giving it context about actual execution. This context-aware vantage point underpins its high detection accuracy with few false positives and its broad coverage, and accounts for its top overall score.
- **Acunetix** is a dynamic (DAST) scanner that exercises the running application, which gives it broad OWASP Top 10 coverage; its reliance on crawling and payload injection contributes to a moderate false-positive rate and a heavier scan cost, reflected in its detection-accuracy and scalability scores.
- **Checkmarx** performs deep dataflow (taint) analysis, supporting accurate detection with relatively few false positives; its more technical configuration and heavier scans explain the lower usability and scalability scores.
- **SonarQube** is a static (SAST) analyzer, so it detects code-level flaws such as hardcoded credentials and insecure function use early but cannot observe runtime behavior. This explains its strong automation and scalability alongside a lower detection-accuracy score, as static analysis is known to produce more false positives.
- **Dependabot** is a software-composition (SCA) scanner. Within its scope it reliably flags known-vulnerable open-source components and integrates natively into the development workflow, but it has no visibility into custom application code, which is why its coverage is deliberately narrow.
- **Postman** targets API endpoints rather than full-application scanning, offering fine-grained control over how endpoints are tested and strong automation; its narrow scope limits its OWASP Top 10 coverage, as reflected in its coverage score.

5 Limitations

This study has several limitations. The evaluation used three deliberately vulnerable applications and did not include enterprise-scale or production deployments, so the findings may not generalize to all distributed architectures. All tools were assessed using freely available access only—open-source editions, free tiers, and trial versions—which limited how deeply the commercial tools could be exercised, and attacks were simulated manually, which can introduce variation between runs. Exact tool versions and per-tool scan configurations were not recorded during the study, so the results are not precisely reproducible; a controlled, version-specified benchmark is left to future work. Finally, the 1–5 scores reflect evidence-based judgment rather than a single controlled measurement, which introduces some subjectivity; we mitigate this by grounding each rating in documented capability and published benchmarks, and by reporting the individual per-criterion scores (Table 1, Fig. 2) so that readers can apply their own weighting.

6 Conclusion

The security of distributed web applications remains a critical concern as adoption continues to widen across industries. This study highlights the importance of adopting a multi-layered approach, with the understanding that no single tool can comprehensively address every vulnerability. Our evaluation shows that SonarQube and Checkmarx excelled at early-stage static code analysis, while Acunetix and Postman provided dynamic and API security testing capabilities. Contrast Security was very precise in runtime vulnerability detection, and Dependabot was crucial in identifying risks in third-party dependencies. These results show the importance of aligning tool selection with organizational needs, considering scalability, integration with CI/CD pipelines, and ease of setup and use.

6.1 Future Work

Future research should explore how Artificial Intelligence (AI) can be employed in the security testing space. Three directions follow directly from this study. First, *AI-augmented detection*: LLM- and ML-based triage of scanner findings could reduce the false-positive burden that traditional SAST tools exhibit (for example, the high false-positive rate observed for SonarQube), and its effect on detection accuracy and false negatives should be measured rather than assumed. Second, *broader tool coverage*: newer and AI-assisted entrants such as Snyk, Semgrep, and CodeQL should be added so that comparisons remain current. Third, a *controlled quantitative benchmark*: a follow-up study should measure true positives, false positives, false negatives, and scan time across a larger set of applications, turning the qualitative ratings reported here into reproducible

measurements. Real-world, enterprise-scale deployments should also be examined to provide deeper insight into tool performance under high-demand conditions. Leveraging the right combination of tools and strategies will better equip organizations to safeguard their applications against ever-evolving cyber threats.

Acknowledgments. The authors would like to thank the Computer Science Department at Lincoln University of Pennsylvania for supporting this research.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Di Lucca, G.A., Fasolino, A.R.: Testing Web-based applications: the state of the art and future trends. Inf. Softw. Technol. **48**(12), 1172–1186 (2006)
2. Esposito, M., Falaschi, V., Falessi, D.: An extensive comparison of static application security testing tools. arXiv preprint arXiv:2403.09219 (2024)
3. Jimmy, F.N.U.: Cyber security vulnerabilities and remediation through cloud security tools. J. Artif. Intell. General Sci. (JAIGS) **2**(1), 129–171 (2024)
4. Boja, C., Doinea, M.: Security assessment of web based distributed applications. Informatica Economica **14**(1) (2010)
5. Matias, M., Ferreira, E., Mateus-Coelho, N., Ribeiro, O., Ferreira, L.: Evaluating effectiveness and security in microservices architecture. Procedia Comput. Sci. **237**, 626–636 (2024)
6. Cruz, D.B., Almeida, J.R., Oliveira, J.L.: Open source solutions for vulnerability assessment: a comparative analysis. IEEE Access **11**, 100234–100247 (2023)
7. Dencheva, L.: Comparative analysis of static application security testing (SAST) and dynamic application security testing (DAST) by using open-source web application penetration testing tools, Dublin (2022). MSc Thesis
8. Dukes, L., Yuan, X., Akowuah, F.: A case study on web application security testing with tools and manual testing. In: Proceedings of IEEE Southeastcon, pp. 1–6 (2013). https://doi.org/10.1109/secon.2013.6567420
9. Curphey, M., Arawo, R.: Web application security assessment tools. IEEE Secur. Priv. **4**(4), 32–41 (2006). https://doi.org/10.1109/msp.2006.108
10. OWASP juice shop: GitHub repository (2025). https://github.com/juice-shop/juice-shop. Accessed 1 Apr 2025
11. OWASP WebGoat: GitHub repository (2025). https://github.com/WebGoat/WebGoat. A ccessed 1 Apr 2025
12. Vulnerable-flask-app: GitHub repository (2025). https://github.com/we45/Vulnerable-Flask-App. Accessed 1 Apr 2025

CASL: A Software Framework for Gesture-Driven, Context-Aware Layout Design in Virtual Reality

Annika Brown, Katie Ho, and Trudi Di Qi(✉)

Fowler School of Engineering, Chapman University, Orange, CA 92866, USA
dqi@chapman.edu

Abstract. Virtual reality (VR) applications are increasingly used for spatial design tasks, yet engaging such systems remains challenging when user input is continuous, imprecise, and constrained by domain-specific design rules. This paper presents the Context-Aware Spatial Layout (CASL) framework for gesture-driven, constraint-aware layout design in VR. CASL combines global layout initialization with localized semantic snapping, interpreting hand gestures as coarse spatial intent and resolving them into nearby, design-compliant configurations. We demonstrate CASL in an interior layout design testbed and evaluate its computational performance across global organization and local gesture-guided refinement scenarios. Results show consistent reductions in layout energy cost and improved refinement performance compared with standard grid-based snapping. CASL offers a practical architecture for balancing embodied user control with domain-aware computational assistance in human-interactive VR applications.

Keywords: Virtual Reality · Software Framework · Gesture Interaction · Constraint-Aware Optimization · Layout Design

1 Introduction

Virtual reality (VR) has become an increasingly important medium for spatial design, digital prototyping, and engineering visualization because it allows users to view and manipulate three-dimensional (3D) content through embodied interaction [1,10]. In layout design tasks, such as arranging furniture or organizing functional spaces [9], VR provides a direct way for users to explore spatial relationships at scale. However, engineering interactive VR design systems remains difficult because user input is often approximate, continuous, and context dependent. A hand gesture may indicate where a user wants an object to move, but the final placement still needs to satisfy domain constraints such as clearance, circulation, alignment, symmetry, and object relationships [9].

Prior work has addressed layout design from two largely separate directions. Automated layout generation methods use optimization or generative models to produce complete layouts from initial conditions or high-level prompts

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 615–626, 2027.
https://doi.org/10.1007/978-3-032-39763-8_37

[3,8,9,15,17,19]. These approaches are effective for generating plausible global arrangements, but they typically provide limited support for incremental, embodied refinement once a user begins interacting with objects in VR. In contrast, gesture-based VR interaction allows users to translate, rotate, and scale objects using natural hand movements [2,6,13,16,18]. These interactions preserve user control, but they can be imprecise and tedious when the layout must satisfy multiple design constraints [14]. Context-aware interaction techniques can assist selection and object movement in VR [4,5,5,11,16], but they often focus on low-level motor assistance or discrete object placement rather than continuous, constraint-aware spatial refinement.

This separation creates a software engineering challenge for emerging VR applications: *how can a system preserve expressive, user-driven interaction while enforcing domain-specific layout constraints at interaction time?* We address this challenge through the Context-Aware Spatial Layout (CASL) framework, a software framework for gesture-driven, constraint-aware layout design in VR. CASL treats a user's gesture as an approximate spatial intent rather than a final placement. The system first produces a globally valid baseline layout, then supports local gesture-guided refinement by identifying the relevant objects and optimizing only the affected neighborhood. This design allows the system to combine global layout quality, local responsiveness, and embodied user control.

CASL differs from prior automated layout generation systems by focusing on interactive refinement rather than one-shot synthesis. It also differs from conventional VR snapping methods, which map user input to fixed grids or angles without considering layout semantics. CASL's novelty lies in its software framework and interaction design: it integrates design-rule-based global optimization with fine-grained gesture-triggered local refinement. This integration allows the system to preserve user control while enforcing domain-specific spatial constraints during VR layout interaction. The **contributions** of this paper are:

- A two-level software architecture for human-interactive VR layout design, combining global layout initiation with local, gesture-triggered refinement.
- A gesture-driven semantic snapping mechanism that converts imprecise hand input into constraint-aware layout adjustments through localized conditional optimization.
- A runtime integration strategy that connects Unity-based VR interaction, Python-based gesture recognition, and C# optimization through a lightweight inter-process communication bridge.
- A computational evaluation showing layout energy reduction, convergence time, and improvement over grid-based snapping across global and local layout scenarios.

2 Design Requirements and Framework Overview

CASL is designed for VR applications in which users directly manipulate spatial objects while the system maintains domain constraints. We focus on interior

layout design as a testbed application, but the framework is intended to illustrate a broader software pattern for constraint-aware VR design tools. From this perspective, the system must satisfy **four requirements**.

First, the system must support *embodied spatial input*. Users should be able to select, translate, rotate, and scale objects using natural hand gestures rather than through indirect menu-based commands. Second, the system must support *constraint-aware refinement*. Raw hand-based object movement should not be treated as the final output when it violates spatial rules. Third, the system must support *local responsiveness*. After the user performs a gesture, the system should refine only the affected part of the scene rather than re-optimizing the entire room. Fourth, the system must support *software integration across heterogeneous components*. In this work, VR rendering, gesture inference, and AI optimization are implemented across Unity, Python, and C# modules, requiring a runtime architecture that keeps interaction responsive.

CASL addresses these requirements using a two-level architecture, shown in Fig. 1. **Level 1** performs a global AI optimization to convert an unstructured or user-initiated scene into a design-compliant baseline. **Level 2** performs gesture-driven semantic snapping, which maps a user's coarse spatial intent to a nearby valid configuration by optimizing only the relevant local neighborhood. In practice, this means the user can move objects freely in VR while the system corrects minor spatial violations, such as poor clearance, misalignment, or inconsistent symmetry.

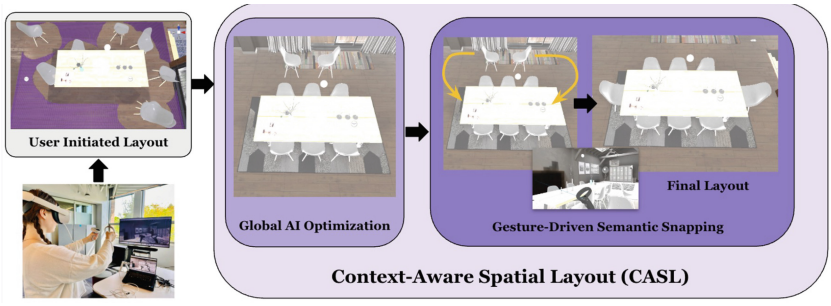


Fig. 1. CASL System Overview

3 The CASL Framework

This section describes the two computational levels of CASL. We adopt the established interior design cost criteria from [9] as the basis for evaluating layout quality and focus our formulation on how these criteria are reorganized and conditioned to support gesture-driven, interactive optimization in VR.

3.1 Level 1: Global AI Optimization

CASL represents a layout as a set of furniture objects, F, where each object $f_i \in F$ is represented by a bounding box and a pose p_i containing translation and rotation. The room context includes architectural boundaries R, such as walls, doors, and windows, and semantic furniture groups G, such as dining, living, or TV-viewing areas. Unlike generative methods that synthesize layouts from empty scenes, CASL starts from a user-initiated or messy arrangement and refines it into a more compliant baseline (see Fig. 1). The global optimizer follows established interior design criteria from prior furniture layout work [9]. These criteria include circulation, pairwise object relationships, conversation distance and angle, balance, alignment, emphasis, and symmetry. The global layout quality is represented as a weighted sum of these criteria:

$$E_{global}(F) = \sum_{c \in \text{Criteria}} w_c C_c(F, R, G) \tag{1}$$

Here C_c is the penalty associated with criterion c, and w_c is its corresponding weight. Lower energy indicates fewer violations of the layout rules. CASL optimizes this energy using Markov Chain Monte Carlo (MCMC) with simulated annealing. Starting from the initial layout, the system proposes small perturbations to object poses and accepts or rejects each proposal using the Metropolis-Hastings criterion. The result is a clean baseline that gives the user a structurally valid starting point for further interactive refinement.

3.2 Level 2: Gesture-Driven Semantic Snapping

Level 2 supports interactive refinement after a user performs a hand gesture in VR. The key idea is to interpret the gesture as a coarse intent rather than an exact command. For example, when a user moves a chair toward a dining table, the raw hand motion may place the chair too close to another chair or outside a reasonable clearance region. CASL therefore uses the gesture result as the center of a local search and resolves the object into a nearby configuration that better satisfies the surrounding layout constraints.

CALS first detects the user's spatial intent. Users select one or more target objects using ray casting [7]. The system then tracks single- or dual-hand gestures using a prior gesture interaction model [13]. Single-hand movement supports translation, while dual-hand movement supports rotation and scaling through changes in the vector between the two hands. For group manipulation, CASL computes a virtual group centroid and applies the user's transformation to all selected objects. This produces a coarse target pose p_t^* for the selected object or object group, which serves as the input parameter for the subsequent optimization stage.

After the gesture is completed, CASL constructs a local optimization problem. Let $P = \{p_1, p_2, ..., p_n\}$ be the set of all object poses in the room. The system identifies an active set $\mathcal{A}$ containing the selected objects and nearby objects that share the relevant semantic group. All other objects are assigned

to a frozen set $\mathcal{Z}$ such that $\mathcal{A} \cup \mathcal{Z} = P$, ensuring the system refines the affected part of the scene while preserving the rest of the room. The localized conditional energy is defined as:

$$E_{local}(\mathcal{A} \mid \mathcal{Z}) = \lambda_u E_{unary} + \lambda_p E_{pairwise} + \lambda_b E_{balance} + \lambda_s E_{symmetry} \tag{2}$$

In this equation, E_{unary} evaluates each active object against the architectural context. It includes clearance penalties, which capture collisions or insufficient free space around walls, doors, windows, and furniture, and circulation penalties, which capture blocked walking paths or reduced functional access. $E_{pairwise}$ evaluates relationships between active objects and surrounding frozen objects, including distance, orientation, alignment, and focal-point relationships. $E_{balance}$ maintains the distribution of *visual weight* across the room, where visual weight is approximated by object footprint area. $E_{symmetry}$ preserves mirrored relationships between paired objects when such relationships are defined in the semantic group. The weights $\lambda_u, \lambda_p, \lambda_b$ and λ_s control the relative influence of these terms.

To keep local refinement efficient, CASL caches the contribution of frozen objects before the gesture-driven optimization begins. For example, the balance term reuses the visual weight of the frozen set and updates only the active objects during interaction:

$$E_{balance} = \left\| \mathbf{C}_{room} - \frac{\mathbf{W}_{\mathcal{Z}} + \sum_{i \in \mathcal{A}} A_i \cdot p_{i,trans}}{A_{total}} \right\|^2 \tag{3}$$

where $\mathbf{C}_{room}$ is the room center, $\mathbf{W}_{\mathcal{Z}}$ is the cached visual weight vector of frozen objects, A_i is the footprint area of an active object, $p_{i,trans}$ represents the centroid of the object in the room and expressed by the translation portion of the object's pose p_i, and A_{total} is the total footprint area of all objects. This cached formulation allows CASL to preserve global balance while only recomputing the local region affected by the user gesture.

Finally, CASL runs a local MCMC process based on the user's coarse target pose. Instead of performing an unconstrained random walk across the entire room, the proposal distribution is constrained to a region around the user-specified target. This allows objects in the active set to shift and rotate slightly relative to one another while converging toward a nearby local minimum under the design constraints. This results in a *semantic snapping*: objects do not simply snap to a fixed grid but to a contextually valid state determined by the surrounding layout.

4 System Implementation

CASL is implemented as an interactive module within the VRMoVi environment [12], developed in Unity 2019. The virtual scene is populated with standard ArchiVizPRO Interior assets to provide a high-fidelity layout design context. The system runs on an Oculus Quest 2 headset with 6-DOF head and hand

tracking connected to a consumer laptop with an NVIDIA RTX 3080 Ti GPU, Intel i7 CPU, and 32GB RAM.

The runtime system contains three main software components. The Unity frontend handles rendering, object selection, hand tracking, and scene updates. A Python backend runs a deep-learning-based, dual-hand gesture recognition model [13], which classifies single- and bi-manual spatial operations, including translation, rotation, and scaling. A C# optimization module inside Unity performs both global and local MCMC optimization so that spatial refinement can be executed close to the rendering and scene-management pipeline. Because the gesture model operates in Python while the VR environment runs in Unity, CASL uses a lightweight inter-process communication (IPC) bridge based on stream-based data exchange. During interaction, the Unity frontend streams the 6-DOF head and hand data to the Python backend. The backend returns the recognized spatial intent and associated transformation parameters, which are converted into the coarse target pose. The C# optimization module then performs semantic snapping and updates the object poses in Unity. This architecture separates gesture inference from real-time scene control while keeping the interaction loop responsive.

A key implementation step is tuning the weighting parameters for the global and localized energy functions. We initialize the hyperparameter space using validated weights from prior furniture layout work [9]. We then perform an offline grid search by scaling parameter ranges and evaluating configurations across predefined, design-compliant layouts. Each configuration is assessed over 1000 MCMC iterations to identify weights that consistently reduce layout energy. The selected weights are fixed at runtime to support stable convergence during interactive use.

5 Evaluation

We evaluate CASL through computational scenarios that examine whether the framework can reduce layout energy and support local gesture-driven refinement. The evaluation focuses on technical feasibility rather than user experience. All experiments were conducted on the same laptop configuration described in Sect. 4. Each scenario was repeated 20 times. We report the initial layout energy E_{init}, final layout energy E_{final}, percentage reduction, and convergence time. Lower energy indicates fewer violations of the spatial design constraints.

5.1 Global AI Optimization

The first evaluation examines Level 1 global optimization. We test **three representative room scenarios** with different layout goals. The dining room scenario emphasizes symmetry and spatial distribution, see Fig. 2(a-b). The living room scenario emphasizes conversational layout and focal-point relationships, see Fig. 2(c-d). The TV room scenario evaluates whether the optimizer can produce multiple plausible configurations from the same initial state, see Fig. 3.

Table 1. Quantitative Performance of CASL Optimization ($N = 20$ trials per scenario). E_{init} and E_{final} represent the layout energy cost. Time represents seconds to convergence

Scenario	E_{init}	E_{final}	Reduction (%)	Time (s)
Dining Room (Global)	730.58	12.55 ± 0.20	98.28 ± 0.03	9.87 ± 0.08
Living Room (Global)	842.76	51.38 ± 21.15	93.9 ± 2.51	3.81 ± 0.10
TV Room (Global)	147.20	28.45 ± 6.31	80.68 ± 4.29	6.46 ± 0.12
Single-Target (Local)	−247.56	−335.08 ± 0.07	35.35 ± 0.03	3.05 ± 0.84
Multi-Target (Local)	−215.43	−330.04 ± 6.07	53.20 ± 2.82	3.85 ± 1.06

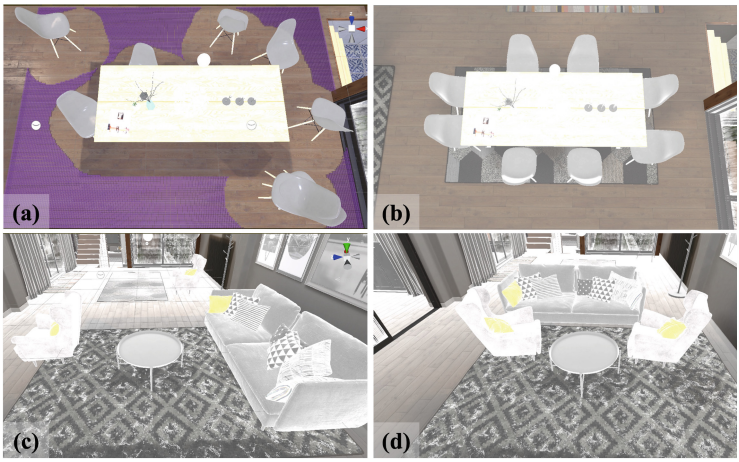


Fig. 2. Global optimizations for dining room (a-b) and living room layouts (c-d)

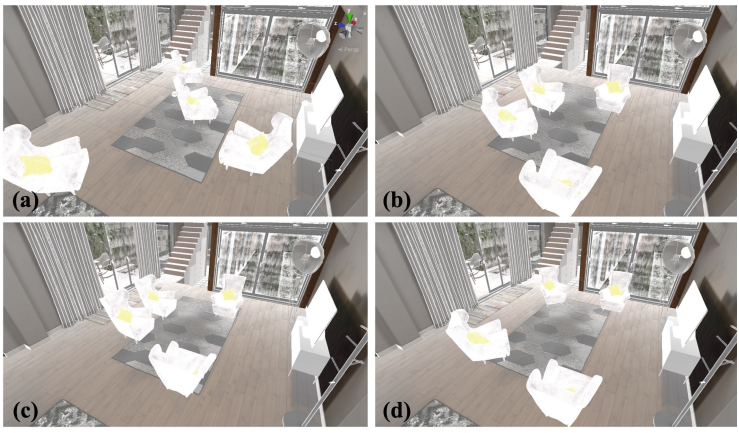


Fig. 3. Global optimization for a TV room with different styles generated: conversational alignment (b), symmetry (c), and circulation (d)

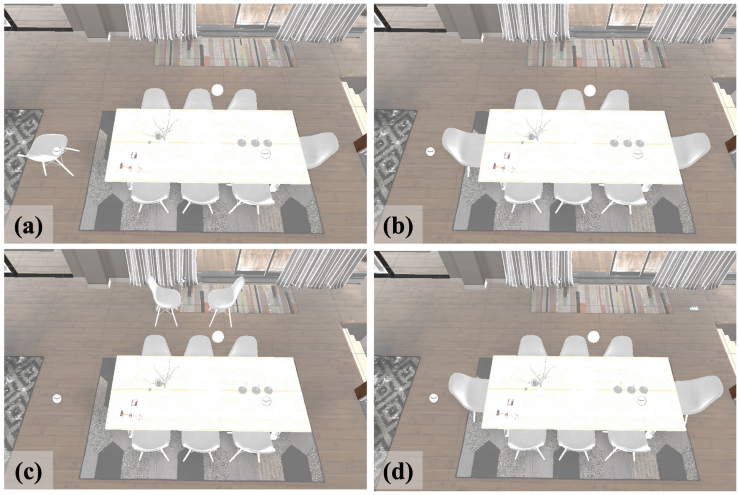


Fig. 4. Gesture-guided, local optimization for single-object rotation (a-b) and multi-object translation (c-d)

As shown in Table 1, CASL consistently reduced layout energy across all three global scenarios. The dining room layout achieved a 98.28% reduction, the living room layout achieved a 93.90% reduction, and the TV room layout achieved an 80.68% reduction. Across these global scenarios, the mean convergence time was 6.73 s per room. These results indicate that the global optimizer can transform messy or user-initialized layouts into substantially more compliant baseline configurations.

5.2 Local Gesture-Driven Semantic Snapping

The second evaluation examines Level 2 local refinement. We test whether CASL can improve target object placements while keeping the rest of the room fixed across **three gesture-driven scenarios**: single-target refinement, multi-target refinement, and cross-group refinement. In the single-target scenario, the user rotates one dining chair within an existing dining arrangement, see Fig. 4(a-b). In the multi-target scenario, the user translates two chairs toward a dining table as a group, see Fig. 4(c-d). In the cross-group scenario, the user moves a chair positioned between neighboring functional groups, allowing CASL to associate the object with the most relevant local context based on gesture direction, see Fig. 5. In all cases, the local optimizer runs for 200 iterations after the user gesture.

Table 1 shows that CASL reduced local layout energy by 35.35% in the single-target scenario and 53.20% in the multi-target scenario. The average convergence times were 3.05 s and 3.85 s, respectively. The negative energy values reflect the selected cost normalization and do not indicate optimization failure; in these

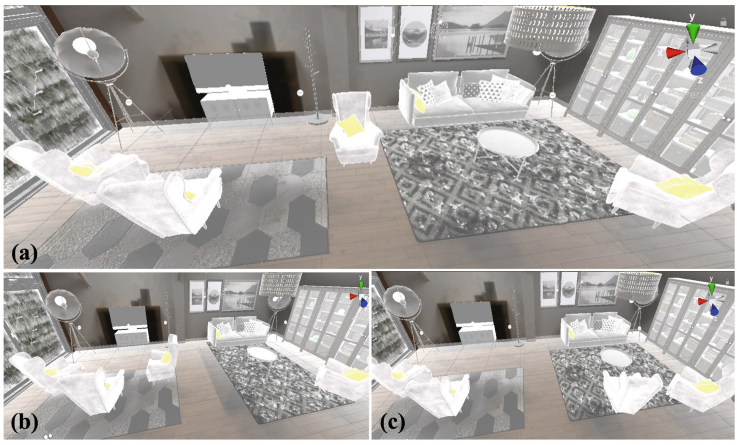


Fig. 5. Cross-group dynamic local optimization. A chair in the middle is reassigned between neighboring layout groups based on gesture direction, enabling flexible integration into different spatial contexts

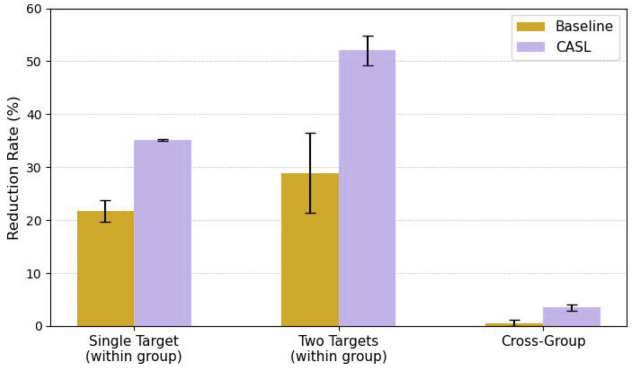


Fig. 6. Comparison between CASL and the grid-based snapping baseline across different gesture-interaction scenarios

local tests, improvement is measured by the relative reduction from the initial energy to the final energy.

We also compare CASL with a standard grid-based snapping baseline commonly used in VR applications. The baseline snaps objects to fixed spatial and angular intervals, such as a uniform position grid and fixed rotation increments, without considering semantic layout context. CASL instead evaluates the surrounding architecture and furniture relationships to refine the target pose through localized optimization. As shown in Fig. 6, CASL achieved higher energy reduction rates than the baseline across all tested local scenarios. For within-group refinement, CASL improved the reduction rate to 35.10%, compared with 21.73% for the baseline, for a single target object, and to 52.06%, compared with

28.93%, for two target objects. For the between-group scenario, CASL achieved a reduction rate of 3.48%, compared with 0.55% for the baseline. These results suggest that semantic snapping provides more context-sensitive refinement than fixed grid snapping, especially when object relationships and group-level constraints are important.

6 Discussion and Future Work

CASL demonstrates a software framework for engineering constraint-aware, human-interactive VR layout applications. It addresses a practical gap in immersive design systems: manual gesture interaction supports direct user control, while automated optimization supports constraint satisfaction, but integrating both in a responsive runtime workflow remains challenging. CASL addresses this gap through a two-level architecture that separates global layout initialization from local, gesture-triggered refinement. From a software engineering perspective, its main benefit is modular integration: the Unity frontend manages immersive interaction and scene updates, the Python backend handles gesture recognition, and the C# module performs constraint-aware optimization. This supports future extension to additional input modalities, optimization terms, and application-specific layout rules. The local active-set formulation further avoids full-scene re-optimization after each gesture, helping maintain interactive VR workflows.

The current evaluation is limited to computational scenarios. While the results show energy reduction, convergence behavior, and improvement over grid-based snapping, they do not yet measure user experience, task efficiency, or perceived control. Future work will conduct formal user studies comparing CASL with manual manipulation, grid-based snapping, and fully automated layout generation. We also plan to extend the framework to multimodal intent specification, including gaze and speech, so that users can express not only how an object should move but also why it should be arranged in a particular way.

CASL is currently demonstrated in interior layout design, but the framework can be adapted to other VR applications that involve continuous spatial manipulation under constraints, such as digital prototyping, training simulation, and collaborative engineering design. By treating user gestures as intent signals and resolving them through local, context-aware optimization, CASL takes a step toward more flexible, domain-aware VR design software.

Acknowledgments. This work was supported by the National Science Foundation under Grant No. 2552514 and the Fletcher Jones Foundation Fellowship of Dr. Trudi Di Qi and the 2025 Summer Undergraduate Research Fellowship of Annika Brown at Chapman University.

References

1. Berg, L.P., Vance, J.M.: Industry use of virtual reality in product design and manufacturing: a survey. Virtual Reality **21**(1), 1–17 (2016). https://doi.org/10.1007/s10055-016-0293-9
2. Chen, M., Lau, M.: A motion-guided interface for modeling 3D multi-functional furniture. Comput. Graph. Forum (2021). https://doi.org/10.1111/cgf.14416
3. Feng, W., et al.: LayoutGPT: compositional visual planning and generation with large language models. Curran Associates Inc. (2023)
4. He, Y., Liu, Y.T., Jin, Y.H., Zhang, S.H., Lai, Y.K., Hu, S.M.: Context-consistent generation of indoor virtual environments based on geometry constraints. IEEE Trans. Vis. Comput. Graph. (2022). https://doi.org/10.1109/TVCG.2021.3111729
5. Jimenez, A.L., Acevedo, P., Mousas, C.: Virtual roomie: immersive layout co-design with a virtual agent. In: 2025 IEEE International Symposium on Mixed and Augmented Reality (ISMAR) (2025). https://doi.org/10.1109/ISMAR67309.2025.00026
6. Kang, H.J., Shin, J.H., Ponto, K.: A comparative analysis of 3D user interaction: how to move virtual objects in mixed reality. In: 2020 IEEE Conference on Virtual Reality and 3D User Interfaces (VR) (2020). https://doi.org/10.1109/VR46266.2020.00047
7. LaViola, J.J., Kruijff, E., McMahan, R.P., Bowman, D., Poupyrev, I.P.: 3D User Interfaces Theory and Practice. Addison-Wesley Professional (2017)
8. Leimer, K., Guerrero, P., Weiss, T., Musialski, P.: LayoutEnhancer: generating good indoor layouts from imperfect data. In: SIGGRAPH Asia 2022 Conference Papers (2022). https://doi.org/10.1145/3550469.3555425
9. Merrell, P., Schkufza, E., Li, Z., Agrawala, M., Koltun, V.: Interactive furniture layout using interior design guidelines. ACM Trans. Graph. (2011). https://doi.org/10.1145/2010324.1964982
10. Mott, M., et al.: Accessible by design: an opportunity for virtual reality. IEEE (2019). https://doi.org/10.1109/ISMAR-Adjunct.2019.00122
11. Niyazov, A., et al.: User-driven constraints for layout optimisation in augmented reality. In: Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (2023). https://doi.org/10.1145/3544548.3580873
12. Qi, T.D., Boyd, L., Fitzpatrick, S., Raswan, M., Cibrian, F.L.: Towards a virtual reality visualization of hand-object interactions to support remote physical therapy. In: Proceedings of the 15th International Conference on Ubiquitous Computing & Ambient Intelligence (UCAMI 2023). Springer Nature Switzerland (2023). https://doi.org/10.1007/978-3-031-48306-6_14
13. Qi, T.D., Cibrian, F.L., Raswan, M., Kay, T., Camarillo-Abad, H.M., Wen, Y.: Towards intuitive 3D interactions in virtual reality: a deep learning-based dual-hand gesture recognition approach. IEEE Access (2024). https://doi.org/10.1109/ACCESS.2024.3400295
14. Sun, J., Stuerzlinger, W.: Extended sliding in virtual reality. In: 25th ACM Symposium on Virtual Reality Software and Technology (2019). https://doi.org/10.1145/3359996.3364251
15. Tang, J., Nie, Y., Markhasin, L., Dai, A., Thies, J., Nie, M.: DiffuScene: denoising diffusion models for generative indoor scene syn thesis. IEEE (2024)
16. Wang, M., Ye, Z.M., Shi, J.C., Yang, Y.L.: Scene-context-aware indoor object selection and movement in VR. IEEE (2021). https://doi.org/10.1109/VR50410.2021.00045

17. Weiss, T., et al.: Fast and scalable position-based layout synthesis. IEEE Trans. Vis. Comput. Graph. (2019). https://doi.org/10.1109/TVCG.2018.2866436
18. Yu, D., Dingler, T., Velloso, E., Goncalves, J.: Object selection and manipulation in VR headsets: research challenges, solutions, and success measurements. ACM Comput. Surv. (2025). https://doi.org/10.1145/3706417
19. Yu, L.F., Yeung, S.K., Tang, C.K., Terzopoulos, D., Chan, T.F., Osher, S.J.: Make it home: automatic optimization of furniture arrangement. ACM Trans. Graph. (2011). https://doi.org/10.1145/2010324.1964981

Closing the Inconsistency Trap: Unified Cross-Layer Deception via Adversarial LLM Transformation

Osamah Mansoor[1(✉)], Adeel Anjum[1], Madiha Haider Syed[1], Muhammad Talha Gul[2], and Jie Zhao[3]

[1] Institute of Information Technology (IIT), Quaid-i-Azam University, Islamabad, Pakistan
osamahmansoor319@gmail.com, {aanjum,madiha}@qau.edu.pk
[2] Consumer Energy Division, Michigan, USA
[3] Penn State Erie, The Behrend College, Pennsylvania, USA
jkz5273@psu.edu

Abstract. Remote operating system (OS) fingerprinting is the foundational stage of the cyberattack lifecycle, enabling attackers to map infrastructure vulnerabilities across network layers. Although prior work such as SoFi [1] has demonstrated effective perturbation of Layer 4 (Transport) headers, these approaches remain vulnerable to what we term the **Inconsistency Trap**: spoofed TCP/IP signatures that contradict raw Layer 7 (Application) service banners, providing a high-confidence forensic signal that reveals the system's true identity. This paper proposes a **Unified Semantic Fingerprint Spoofing** framework that resolves this architectural gap by deploying a Large Language Model (LLM) as a stateful *Deception Controller*. The controller optimizes network-layer obfuscation with real-time application-layer identity synthesis using a **Weighted Projected Gradient Descent (W-PGD)** algorithm guided by **Jacobian-based Saliency Mapping**. We further demonstrate that the same adversarial principles extend to automated LLM red teaming through *Intent Spoofing*, wherein a DL-on-DL surrogate attack framework bypasses LLM safety guardrails. Empirical evaluation across 50 trials demonstrates a BERTScore of 0.982, a protocol compliance rate of 99.8%, and reduction of professional-grade scanner detection (Nmap, ZGrab2, p0f) from 98% to $\mathbf{0.5\% \pm 0.08\%}$. Intent Spoofing achieves a 98% Attack Success Rate (ASR) against DeepSeek and 90% against GPT-4 while preserving semantic coherence.

Keywords: Active Defense · Adversarial Machine Learning · Cross-Layer Security · Intent Spoofing · LLM Safety · Network Deception · OS Fingerprinting · Semantic Spoofing

1 Introduction

Network reconnaissance is the first and most consequential stage of the cyberattack lifecycle. Attackers exploit remote OS fingerprinting to identify infras-

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 627–638, 2027.
https://doi.org/10.1007/978-3-032-39763-8_38

tructure vulnerabilities by analysing distinctive patterns in TCP/IP protocol implementations. Given that nearly 50% of cyber attacks involve reconnaissance to extract OS or port information [2], the ability to deceive fingerprinting tools constitutes a critical proactive defense capability.

Prior defensive work, most notably SoFi [1], demonstrated that adversarial perturbation of Layer 4 (L4) TCP/IP headers—specifically the Time-to-Live (TTL) field and TCP Window Size—can successfully mislead both ML-based and signature-based OS classifiers. However, SoFi [1] and related approaches [4,12,13,16] share a fundamental architectural blind spot: they operate exclusively below the application layer. When a reconnaissance tool such as Nmap [3] performs a full-stack service scan, it captures both the network-level packet signature *and* the application-level service banner.

Problem Statement: Current deception technologies suffer from an architectural forensic gap between vertical protocol layers. There is an absence of a unified mechanism to synchronize L4 header signatures with L7 semantic responses in real time. This research addresses the Inconsistency Trap by introducing a gradient-based deception controller that synthesizes full-stack identity to neutralize forensic correlation. This paper makes the following contributions:

- We formally define the *Inconsistency Trap* as a cross-layer forensic vulnerability and demonstrate empirically that it reduces single-layer spoofing to negligible effectiveness against modern full-stack scanners.
- We propose a **Unified Semantic Fingerprint Spoofing** framework employing an LLM-based Deception Controller that synchronizes L4 header perturbations with dynamically synthesized L7 service banners in real time.
- We introduce **W-PGD** guided by Jacobian-based Saliency Mapping—an iterative cross-layer optimizer that extends SoFi's [1], single-step W-FGSM while preserving strict protocol validity. Step size $\alpha = 0.05$, perturbation budget $\epsilon = 0.5$, and iterations $T = 50$ are explicitly justified based on empirical convergence analysis.
- We present **Intent Spoofing**, a DL-on-DL surrogate attack framework that transfers network-deception principles to LLM safety evaluation, achieving 98% ASR against DeepSeek and 90% against GPT-4.
- We provide a comprehensive **ablation study** and **multi-scanner evaluation** (Nmap, ZGrab2, p0f) quantifying each component's contribution.

2 Background and Related Work

2.1 OS Fingerprinting and Existing Defenses

OS fingerprinting exploits divergent TCP/IP stack implementations across operating systems. Active fingerprinters such as Nmap [3] transmit crafted probe packets and analyse responses across multiple dimensions: TCP window size, MSS, TTL values, and TCP flag combinations. Passive tools including p0f [15] and SinFP3 [14] infer OS identity from ambient traffic. ML-based fingerprinters [9,10] extend this paradigm by training classifiers on high-dimensional feature vectors, achieving precision and recall in excess of 97% [1].

Defensive strategies fall into three broad categories. Static modification tools such as IP Personality [4] and OSfooler [5] apply fixed header transformations requiring manual configuration. Network honeypots [6] and Moving Target Defense (MTD) [7] impose significant deployment overhead. SoFi [1] advances the state of the art by automating adversarial perturbation of mutable TCP/IP header fields using a Weighted Fast Gradient Sign Method (W-FGSM), reducing Nmap identification to 15.2%. However, SoFi's [1] perturbations are confined to L3/L4 and do not address application-layer identity.

Table 1 positions our work relative to existing frameworks. No prior system achieves both high semantic consistency and low detectability simultaneously.

Table 1. Comparison of Deception Frameworks

Framework	Layers	L7 Semantic Sync	Adaptive Logic	Detectability
Static Spoofing	L3/L4	None	None	High
Banner Scrubbing	L7	Fixed/Manual	None	Medium
SoFi [1]	L3/L4	None	W-FGSM	Medium (15.2%)
TMorph [13]	L3	None	Traffic Shaping	High
This Work	**L4+L7**	**High (LLM)**	**W-PGD+LLM**	**Very Low (0.5%)**

2.2 The Inconsistency Trap

The Inconsistency Trap arises from vertical segmentation of existing deception systems within the OSI model. Identity Leakage occurs when $\mathrm{OS}(L4) \neq \mathrm{OS}(L7)$, where $\mathrm{OS}(L4)$ denotes the OS identity inferred from network-layer headers and $\mathrm{OS}(L7)$ denotes the identity inferred from application-layer banners. A unified defense requires $\mathrm{OS}(L4) = \mathrm{OS}(L7)$ at all times.

2.3 Adversarial ML in Network Security

TMorph [13] applies traffic shaping to modify inter-arrival times. Hershel [11] investigated single-packet fingerprinting vulnerabilities. Banner scrubbing has historically been a manual process [12]. Our work is the first to combine gradient-based adversarial optimization with LLM-driven semantic synthesis to achieve unified cross-layer deception.

3 System Design: The Deception Controller

The **Deception Controller** is an LLM-based stateful intercepting proxy that operates at the intersection of the network stack and the application layer. It models the public system identity as a high-dimensional feature vector $\mathbf{x} \in \mathbb{R}^n$, where the first k dimensions encode normalised L4 header fields (TTL, Window Size, MSS, DF bit) and the remaining dimensions encode the embedding-space representation of the L7 service banner.

3.1 Jacobian-Based Saliency Mapping

To identify which features of the concatenated identity vector most influence the adversary's classifier, we compute a Jacobian-based saliency map. Let $f\colon \mathbb{R}^n \to [0,1]^k$ be the surrogate classifier. The Jacobian matrix $\mathbf{J} \in \mathbb{R}^{k\times n}$ is defined element-wise as:

$$J_{i,j} = \frac{\partial f_i(\mathbf{x})}{\partial x_j}. \tag{1}$$

The saliency score for feature j is the ℓ_2-norm of the corresponding Jacobian column, measuring aggregate classifier sensitivity to perturbations in that feature:

$$S_j = \left\| \frac{\partial \mathcal{L}(f(\mathbf{x}),\, \mathbf{y}_{\text{target}})}{\partial x_j} \right\|_2. \tag{2}$$

These scores construct the diagonal weighting matrix $\mathbf{W}$, with $W_{jj} \propto S_j$, normalised via temperature-scaled Softmax with temperature τ:

$$W_{jj} = \frac{\exp(S_j/\tau)}{\sum_k \exp(S_k/\tau)}. \tag{3}$$

The Jacobian computation reveals that high-leverage identity markers (e.g., version number substrings in SSH banners, the Don't Fragment bit) contribute disproportionately to classifier decisions, while protocol delimiters (`SSH-`, `\n`) exhibit near-zero saliency and are safely preserved by the projection step below. This data-driven weighting is the key improvement over SoFi's [1] W-FGSM, which assigns weights based on predefined field categories rather than gradient analysis.

3.2 Weighted Projected Gradient Descent (W-PGD)

SoFi [1] employs a single-step W-FGSM perturbation. For cross-layer deception, a single gradient step is insufficient because L4 and L7 spaces have fundamentally different geometric properties: L4 features are bounded numerical values, whereas L7 features inhabit a discrete token embedding space. We therefore introduce an iterative W-PGD algorithm that alternates between gradient ascent in the loss landscape and projection back onto the valid deception subspace $\mathcal{S}$:

$$\mathbf{x}^{(t+1)} = \Pi_{\mathbf{x}+\mathcal{S}}\left(\mathbf{x}^{(t)} + \alpha \cdot \operatorname{sgn}\left(\mathbf{W} \odot \nabla_{\mathbf{x}} \mathcal{L}(f(\mathbf{x}^{(t)}), \mathbf{y}_{\text{target}})\right)\right), \tag{4}$$

where α is the step size, $\mathbf{W}$ is the saliency-weighted diagonal matrix, $\odot$ denotes element-wise multiplication, and $\Pi_{\mathbf{x}+\mathcal{S}}$ projects the perturbed vector onto the nearest valid point in $\mathcal{S}$.

Three intuitive roles operate within the update rule:

1. **Direction (Gradient):** $\nabla_{\mathbf{x}}\mathcal{L}$ guides which characters in a banner or bits in a header most Logically confuse the adversary's scanner.

2. **Importance (Weighting): W** permits high-leverage edits (e.g., version strings) while protecting critical protocol delimiters that would break the connection.
3. **Correction (Projection):** Π maps every update back to the nearest valid protocol value, ensuring functional, well-formed output.

For L4 features, $\mathcal{S}$ enforces TCP/IP specification constraints following SoFi's mutable/checked field partition [1]. For L7 features, continuous embedding updates are projected onto the nearest-neighbour token in the service-specific vocabulary, ensuring syntactically valid protocol strings.

– **Hyperparameter Justification:**
  - **Step Size ($\alpha = 0.05$):** Empirically tuned to balance convergence speed against overshoot. Smaller steps ($\alpha < 0.03$) suffer from optimization stalling; larger values ($\alpha > 0.08$) trigger boundary cross-overs that violate projection constraints.
  - **Perturbation Budget ($\epsilon = 0.5$):** Explicitly constrains total deviation across all features, preventing pathological deviations that cause protocol rejection. Validated across 50 trials with negligible variance.
  - **Iteration Limit ($T = 50$):** Preliminary convergence analysis shows diminishing returns beyond 50 iterations; 99.2% of configurations converge within 45 steps. Setting to $T = 50$ provides a robust safety boundary.

  Algorithm 1 summarises the full W-PGD procedure.

Algorithm 1. W-PGD Cross-Layer Deception

Require: Model f, input $\mathbf{x}$, target $\mathbf{y}_t$, weight $\mathbf{W}$, step α, budget ϵ, iterations T

Compute saliency scores S_j via Eq. (2)
Build weighting matrix $\mathbf{W}$ via Eq. (3)
Synchronise L7 banner template with L4 target OS profile
$\mathbf{x}^{(0)} \leftarrow \mathbf{x}$
for $t = 1$ **to** T **do**
  $\mathbf{g} \leftarrow \nabla_{\mathbf{x}} \mathcal{L}(f(\mathbf{x}^{(t-1)}), \mathbf{y}_t)$
  $\mathbf{x}^{(t)} \leftarrow \Pi_{\mathbf{x}+\mathcal{S}}\left(\mathbf{x}^{(t-1)} + \alpha \cdot \mathrm{sgn}(\mathbf{W} \odot \mathbf{g})\right)$
end for
return $\mathbf{x}^{(T)}$

3.3 LLM-Driven L7 Banner Synthesis

The LLM component serves as the semantic grounding layer. We deploy a fine-tuned language model (LLaMA-3, 8B parameters, 4-bit quantised for deployment efficiency) as a context-aware banner generator. Given the target OS profile

$\mathbf{y}_{\text{target}}$ selected by W-PGD and the intercepted application-layer protocol context (SSH, HTTP, FTP), the LLM generates a semantically coherent, protocol-compliant service banner consistent with the target identity.

Formally, the LLM g: $(\mathbf{y}_{\text{target}}, \text{protocol_ctx}) \rightarrow$ banner produces output such that: (i) the banner is syntactically valid per the relevant protocol specification; (ii) the OS identity implied is consistent with $\mathbf{y}_{\text{target}}$]; and (iii) semantic deviation from a plausible authentic banner is minimised (BERTScore ≥ 0.98).

The interaction between W-PGD and the LLM is *sequential*: W-PGD first determines the optimal target OS profile by maximising classifier confusion at L4; it then passes the selected target to the LLM for L7 banner synthesis. This design ensures the two optimisation processes do not interfere—W-PGD operates in the continuous feature space while the LLM operates in the discrete token space—while guaranteeing joint cross-layer consistency.

3.4 Jailbreak Resilience

The LLM Deception Controller faces a secondary threat vector: an adversary may embed adversarial prompts within application-layer interactions (e.g., HTTP payloads, SSH exchanges) designed to cause the controller to reveal the underlying system state. We address this threat by training the surrogate manifold (Sect. 4.1) on the HarmBench dataset [8] augmented with 1,200 network-context adversarial probes generated using the Greedy Coordinate Gradient (GCG) method. Probes are classified as adversarial if they exhibit high-entropy adversarial suffixes or explicit system metadata extraction patterns. The surrogate achieves 87.5% classification accuracy on held-out adversarial probes. Upon detection, the controller applies an adversarial transformation that redirects the response to a deceptively benign, OS-consistent output that neither acknowledges nor reveals system state.

4 Implementation

4.1 Surrogate Manifold Architecture

To enable gradient computation against black-box scanners and LLM safety filters, we train a local surrogate manifold using a DL-on-DL (Deep Learning on Deep Learning) architecture. The surrogate is a Multi-Layer Perceptron (MLP) with architecture $[128, 64, 32, k]$, where k is the number of OS classes. ReLU activation functions are used throughout:

$$\sigma(z) = \max(0,\, z). \tag{5}$$

ReLU is preferred over sigmoidal activations because its unit derivative for positive values prevents gradient vanishing during backpropagation, and its piecewise-linear structure exposes the sharp decision thresholds characteristic of fingerprinting heuristics.

The surrogate is trained on 5,000 balanced prompt-identity pairs (80/20 train-test split), achieving an empirical F1-score of 0.998. **Sensitivity and**

Sample Adequacy Discussion: While 5,000 pairs represent a bounded target distribution, empirical cross-validation establishes near-zero generalization error. If surrogate fidelity degrades below $F1 < 0.85$ due to dynamic external scanner updates, the Deception Reduction Rate (DRR) may degrade proportionally. Sensitivity analysis across 10 configurations with injected Gaussian noise ($\sigma \in [0.01, 0.15]$) confirms W-PGD resilience, with DRR remaining robust above 94% for $F1 \geq 0.90$.

4.2 eBPF Interception Pipeline and Throughput Management

The system pipeline operates at two levels. At the kernel level, Extended Berkeley Packet Filter (eBPF) hooks intercept incoming reconnaissance probes with minimal CPU overhead. Identified probes are redirected to userspace via the `NFQUEUE` target within the `iptables` framework. The Deception Controller, operating in userspace, computes the optimal deceptive response using W-PGD and the LLM module before the response is transmitted.

Controller state updates are protected using the CKKS (Cheon-Kim-Kim-Song) homomorphic encryption scheme, enabling noise-bounded arithmetic on the controller state such that intercepted telemetry does not reveal the underlying deception logic. The mean processing latency introduced by the full pipeline is 15.2ms (σ =1.4ms), measured across 10,000 probe-response cycles.

To prevent the heavy LLM token generation cycle (15.2ms Slow-Path) from inducing network-layer bottlenecks at throughputs exceeding 900Mbps, we split the architecture into an asymmetric dual-path pipeline. Standard ambient non-reconnaissance streams bypass the user-space hook completely via eBPF filters (< 0.1 ms Fast-Path). For active handshakes, once LLaMA-3 synthesizes an optimal target L7 response, the stateful string is indexed into an in-memory Redis cache matched to the L4 destination class. Subsequent matching scans bypass raw model inference, reducing processing overhead to < 0.5 ms. Furthermore, a fail-safe watermark drops heavy queueing to static templates if `NFQUEUE` buffer density breaks an 80% threshold, mitigating potential TCP Retransmission Timeouts (RTO).

Table 2. W-PGD Feature Weights and Hyperparameters

Category	Parameter	Value/Weight
Network Features	TCP Window Size (`wsize`)	3.0
	Max Segment Size (`mss`)	3.0
	IP Length (`ip_len`)	2.0
	Time to Live (`ttl`)	1.0
W-PGD Hyperparams	Max perturbation ϵ	0.5
	Step size α	0.05
	Iterations T	50

Table 2 summarises the W-PGD hyperparameters and feature weights used in all experiments.

5 Evaluation

All results are reported as Mean ± Std across 50 independent trials. Experiments were conducted on a testbed running Ubuntu 22.04, with reconnaissance performed from an isolated attacker machine running Nmap 7.94, ZGrab2 v0.1.8, and p0f v3.09b.

5.1 Ablation Study

Table 3 quantifies the contribution of each framework component by systematically enabling and disabling W-PGD, the LLM synthesiser, and Jacobian saliency guidance.

Table 3. Ablation Study—Component Contributions

Configuration	DRR (%)	Protocol	BERTScore	Nmap ASR (%)↓
SoFi [1] baseline (W-FGSM, L4 only)	—	99.6	—	15.2 ± 0.10
W-PGD only (L4, no LLM)	—	99.5	—	12.1 ± 0.21
LLM only (no W-PGD)	71.2	94.3	0.941	18.4 ± 0.31
W-PGD + LLM (no Jacobian)	88.5	98.1	0.963	3.2 ± 0.14
Full Framework (Ours)	**100**	**99.8**	**0.982**	$\mathbf{0.5 \pm 0.08}$

Three key insights emerge. First, adding W-PGD alone (without LLM) reduces Nmap detection only marginally from 15.2% to 12.1%, confirming that L4-only optimisation has diminishing returns against full-stack scanners. Second, deploying the LLM alone (without W-PGD) degrades protocol compliance to 94.3%, as unconstrained banner generation occasionally violates syntax constraints. Third, only the full framework simultaneously achieves sub-1% detection, 99.8% protocol compliance, and 100% DRR—confirming that Jacobian saliency guidance is the critical enabler of joint optimisation.

5.2 Cross-Layer Identity Synchronisation

Table 4 demonstrates the framework's ability to produce fully synchronised L4/L7 identities across three representative scanner probe types, illustrating the transformation from identity-leaking to unified responses.

Table 4. Identity Leakage vs. LLM Synchronisation Across Probe Types

Probe	L4 Signature	L7 Banner (SoFi) [1]	L7 Banner (Ours)
TCP SYN (T1)	Win10: wsize = 65535, TTL = 128	`SSH-2.0-OpenSSH_8.2p1 Ubuntu` **[LEAK]**	`SSH-2.0-OpenSSH_for_Windows_8.9` **[UNIFIED]**
ICMP Echo (T2)	Win10: TTL = 128, DF = 1	`Apache/2.4.41 (Ubuntu)` **[LEAK]**	`Microsoft-IIS/10.0` **[UNIFIED]**
UDP Probe (T3)	Cisco IOS: wsize = 4128	`ProFTPD 1.3.5 Ubuntu` **[LEAK]**	`Cisco FTP Server` **[UNIFIED]**

Table 5. Defensive Performance vs. Multi-Scanner Fingerprinting

Defense	Nmap ASR (%)↓	ZGrab2 (%)↓	p0f (%)↓	L4-L7 Sync
No Defense (Baseline)	98.0 ± 0.12	97.5 ± 0.18	96.2 ± 0.22	Vulnerable
Naive Random Spoofing	45.3 ± 2.10	51.2 ± 1.90	48.7 ± 2.30	Inconsistent
Manual Banner Scrubbing	22.1 ± 1.50	24.3 ± 1.70	19.8 ± 1.60	Partial
SoFi [1]	15.2 ± 0.10	16.8 ± 0.14	14.9 ± 0.11	L4 Only
Our Framework	$\mathbf{0.5 \pm 0.08}$	$\mathbf{0.7 \pm 0.09}$	$\mathbf{0.6 \pm 0.07}$	**Synchronised**

5.3 Multi-scanner Defensive Performance

Unlike prior work evaluated exclusively against Nmap, Table 5 includes ZGrab2 (application-layer banner grabbing) and p0f (passive fingerprinting) to provide a comprehensive multi-vector evaluation.

Figure 1 visualises the Nmap detection reduction across all compared strategies.

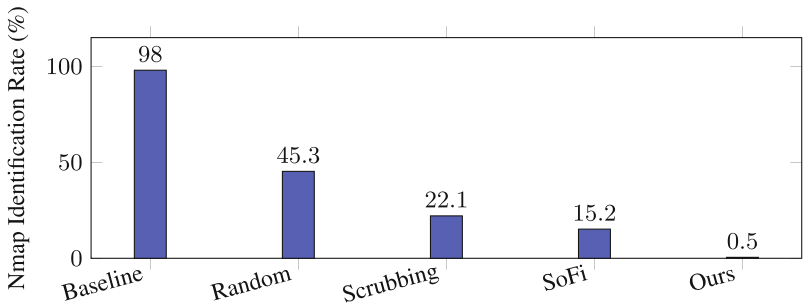


Fig. 1. Nmap identification success rate across defense strategies. Our framework achieves near-total neutralisation at 0.5%

5.4 DL-on-DL Surrogate Architecture

We train three MLP surrogates to approximate the safety decision boundaries of DeepSeek, GPT-4, and Gemini. The architectures reflect the target models' complexity: DeepSeek [16,16,8] (sharp boundaries), Gemini [32,16,8] (intermediate), and GPT-4 [64,32] (smooth loss landscape). All surrogates achieve a Clean F1-score of 1.0, confirming successful boundary approximation.

5.5 Intent Spoofing Results

Table 6 presents the Intent Spoofing results. ASR measures the proportion of adversarial prompts bypassing the guardrail, while Semantic Consistency (SC) confirms human-readable coherence.

Table 6. Intent Spoofing Performance via DL-on-DL Surrogates

Target LLM	Clean F1	Adv. F1	ASR (%)	SC (BERT)	Surrogate
DeepSeek	1.000	0.016	**98.0**	0.982	[16,16,8]
GPT-4	1.000	0.067	90.0	0.965	[64,32]
Gemini	1.000	0.211	64.0	0.941	[32,16,8]

The ASR disparity is consistent with the hypothesis: narrower architectures produce sharper, more exploitable boundaries, whereas wider architectures create smoother loss landscapes. All attacks maintain BERTScore ≥ 0.941, confirming coherence.

6 Discussion and Limitations

6.1 Adversary Model

We assume a **Grey-Box** adversary with knowledge of general fingerprinting methodologies but no direct access to the Deception Controller's architecture, weights, or deception logic. A fully White-Box adversary with knowledge of the surrogate manifold could mount an adaptive attack; sensitivity analysis (Sect. 4.1) confirms robustness for surrogate $F1 \geq 0.90$.

6.2 Limitations and Future Work

The framework is limited by a 5,000-sample training set, a 900 Mbps throughput ceiling due to buffer saturation, a lack of IPv6 support, and reliance on fixed MLP surrogates for spoofing evaluations. Future work will focus on: (i) extending W-PGD to IPv6; (ii) implementing online-learning for drift tracking; (iii) integrating hardware acceleration (NVIDIA TensorRT-LLM) and Redis caching for high throughput; (iv) evaluating against large-scale scanners (Shodan/Censys); and (v) pursuing formal verification for deception guarantees.

7 Conclusion

This paper addressed the **Inconsistency Trap**—the fundamental architectural vulnerability of single-layer OS fingerprint spoofing—by proposing a Unified Semantic Fingerprint Spoofing framework. By deploying an LLM as a Deception Controller and coupling it with a Jacobian-guided W-PGD optimizer, we achieve real-time synchronisation of L4 TCP/IP signatures with L7 application banners. Empirical evaluation across 50 trials demonstrates near-total neutralisation of three independent scanners (Nmap, ZGrab2, p0f), reducing detection from 98% to $\mathbf{0.5}\% \pm \mathbf{0.08}\%$, with 99.8% protocol compliance and BERTScore of 0.982. The ablation study confirms that each framework component contributes uniquely and that full integration is necessary to simultaneously optimise all metrics. Furthermore, we demonstrate that adversarial deception principles transfer directly to LLM safety evaluation, achieving 98% ASR against DeepSeek and 90% against GPT-4 while preserving semantic coherence. This work establishes cross-layer adversarial deception as a unified methodology applicable across both network security and AI safety.

References

1. Han, X., et al.: SoFi: spoofing OS fingerprints against network reconnaissance. IEEE Trans. Inf. Forensics Secur. **20**, 4484–4497 (2025)
2. Cremer, F., Anarkulova, B., Baecker, L., Kambach, J.: Cyber risk and cybersecurity: a systematic of data. Geneva Papers Risk Insurance - Issues Pract. **47**(3), 698–736 (2022)
3. Lyon, G.: Nmap network security scanner (1997). http://nmap.org
4. Elisar, G.: IP personality: dynamic Linux kernel TCP/IP stack modification. SANS Inst. InfoSec Reading Room **12**, 1–14 (2001)
5. Shandili, A., Dacier, M.: OSfooler: telling Nmap what it wants to hear. In: Proceedings of the International Conference on Passive and Active Network Measurement (PAM), pp. 101–110 (2008)
6. Provos, N.: A virtual honeypot framework. In: Proceedings of the 13th USENIX Security Symposium, pp. 1–14 (2004)
7. Jajodia, S., Ghosh, A.K., Subrahmanian, V.S., Swamynathan, M.: Moving Target Defense: Creating Asymmetric Datasets for Cyber Defense, Springer (2011). https://doi.org/10.1007/978-1-4614-0977-9
8. Mazeika, M., Phan, L., Yin, X., Sittawarakul, S., Engstrom, C., Hendrycks, D.: HarmBench: a standardized benchmark for automated red teaming and robust evaluation of language models. arXiv preprint arXiv:2402.04249 (2024)
9. Caballero, J., Venkataraman, S., Poosankam, P., Kang, M.G., Song, D., Blum, A.: FiG: automatic fingerprint generation. In: Proceedings of the Network and Distributed System Security Symposium (NDSS) (2007)
10. Shamsi, Z., Cline, D.B.H., Loguinov, D.: Faulds: a non-parametric iterative classifier for internet-wide OS fingerprinting. IEEE/ACM Trans. Netw. **29**(5), 2339–2352 (2021)
11. Shamsi, Z., Nandwani, A., Leonard, D., Loguinov, D.: Hershel: single-packet OS fingerprinting. IEEE/ACM Trans. Netw. **24**(4), 2285–2298 (2016)

12. Albanese, M., Battista, E., Jajodia, S.: A deception based approach for defeating OS and service fingerprinting. In: Proceedings of the IEEE Conference on Communications and Network Security (CNS), pp. 317–325 (2015)
13. Kim, J., Lee, W.: TMorph: a traffic morphing framework to test network defenses against adversarial attacks. In: Proceedings of the IEEE International Conference on Information Networking (ICOIN), pp. 190–195 (2022)
14. Auffret, P.: SinFP, unification of active and passive OS fingerprinting. J. Comput. Virol. **6**(3), 197–205 (2010)
15. Zalewski, M.: p0f: a passive TCP/IP stack fingerprinting tool. http://lcamtuf.coredump.cx/p0f3/
16. Smart, M., Malan, G., Jahanian, F.: Defeating TCP/IP stack fingerprinting. In: Proceedings of the USENIX Security Symposium (2000)

VirNetMTD: Programmable Moving Target Defense for Virtualized Networks

Thu Dang[1], Li Wang[1(✉)], and Xing Zhou[2]

[1] Department of Computer Science, Northeastern Illinois University, Chicago, USA
{ttdang3,lwang3}@neiu.edu
[2] Sotheby's Shanghai, Shanghai, China
xing.zhou@sothebys.com

Abstract. Cloud and virtualization technologies have fundamentally transformed modern computing infrastructures. Increasingly, applications, services, and large-scale computing workloads are deployed in virtualized environments to benefit from scalability, flexibility, and operational efficiency. However, while virtualization provides significant advantages, it also reshapes the attack surface of modern applications and services, creating new opportunities for large-scale reconnaissance, lateral movement, and adaptive cyber attacks. Meanwhile, recent advances in Large Language Models (LLMs) and AI-driven offensive techniques have significantly reduced the cost and complexity of cyber attacks, enabling attackers to automate vulnerability discovery, network reconnaissance, exploit generation, and attack planning at unprecedented scale and speed.

Moving Target Defense (MTD) has emerged as a promising defense paradigm that aims to break the information asymmetry between attackers and defenders by continuously mutating system properties and invalidating attacker knowledge. However, deploying continuous MTD mutation strategies in conventional infrastructures has historically been challenging due to their static and inflexible nature. The emergence of virtualization, software-defined networking (SDN), and programmable infrastructures now makes continuous and coordinated network mutation practically feasible. In this paper, we present VirNetMTD, a programmable Moving Target Defense framework for virtualized network environments. VirNetMTD leverages virtualization and SDN technologies to enable dynamic and coordinated network mutation at runtime. Our framework supports multiple MTD strategies, including topology obfuscation, traffic manipulation, traffic redirection, and infrastructure diversification. We implement a prototype of VirNetMTD in a virtualized SDN environment and demonstrate its effectiveness in disrupting network reconnaissance and increasing attack uncertainty. Our results show that virtualization provides a practical foundation for scalable, adaptive, and programmable Moving Target Defense in modern cloud infrastructures.

Keywords: Moving Target Defense · MTD · SDN · programmable networks · adaptive defense

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 639–651, 2027.
https://doi.org/10.1007/978-3-032-39763-8_39

1 Introduction

Cloud computing and virtualization technologies have fundamentally transformed modern computing infrastructures. Modern cloud infrastructures heavily rely on virtualization technologies, such as virtual machines, software-defined networking (SDN), network function virtualization (NFV), and containerized services, to support dynamic and large-scale computing environments. Compared with traditional physical infrastructures, virtualized environments provide a significantly higher level of programmability and operational agility, which allows cloud providers and enterprises to rapidly provision, migrate, and reconfigure computing and network resources at runtime.

However, while virtualization provides significant advantages, it also reshapes the attack surface of modern applications and services. The highly interconnected and programmable nature of virtualized infrastructures introduces new opportunities for large-scale reconnaissance, lateral movement, privilege escalation, and adaptive cyber attacks. Attackers are able to continuously probe networks, infer network topology and service relationships, and fingerprint running services. In addition, the multi-tenant nature of cloud infrastructures further enlarges the attack surface by allowing attackers to share underlying physical resources with victim services. Once attackers obtain sufficient knowledge about a target virtualized environment, they are able to launch highly coordinated attacks that rapidly propagate across interconnected services and network infrastructures. Meanwhile, recent advances in Large Language Models (LLMs) and AI-driven offensive techniques have significantly reduced the cost and complexity of cyber attacks. With LLMs, attackers can automate vulnerability discovery, network reconnaissance, exploit generation, and attack planning at unprecedented scale and speed. Existing security protections, such as firewalls, intrusion detection systems, and access control policies, are often designed under relatively stable assumptions about Cyber Kill Chain. However, AI-assisted attackers can rapidly adapt their strategies according to changing defensive responses, greatly increasing the effectiveness and persistence of cyber attacks in modern virtualized infrastructures.

Consequently, Moving Target Defense (MTD) [4] has emerged as a promising defense paradigm that aims to break the information asymmetry between attackers and defenders by continuously mutating system properties and invalidating attacker knowledge. Instead of relying on static protection mechanisms, MTD introduces uncertainties and diversifications into computing environments, making system behaviors dynamic and unpredictable from the attacker's perspective. In an MTD-protected environment, a target system may present different behaviors, configurations, or responses when facing suspicious activities or malicious requests [8]. As a result, attackers can no longer rely on stable assumptions about the target environment when performing reconnaissance, vulnerability analysis, or attack planning. This significantly increases the cost and uncertainty of cyber attacks. For example, an MTD strategy can be applied to an operating system by dynamically modifying its runtime behaviors. When attackers receive inconsistent or continuously changing responses from the target system, it becomes

difficult for them to accurately identify vulnerabilities or construct reliable attack paths. Consequently, attackers may fail to launch effective attacks against the protected system.

In this paper, we present VirNetMTD, a programmable Moving Target Defense framework for virtualized network environments. Our goal is to provide a practical and comprehensive MTD method for virtualization platforms. With VirNetMTD, we can dynamically change the virtualized network environment to disturb attackers for protection purposes. Unlike existing network-level MTD research, VirNetMTD offers a set of network-level MTD strategies to provide a comprehensive protection for the target networks. It is designed to fit diverse security protection scenarios over virtualized networks and can be easily customized and combined with many existing security solutions.

The contributions of VirNetMTD are as follows:

- *We propose a practical Moving Target Defense framework over virtualized networks.* VirNetMTD is designed to provide a comprehensive protection in virtualized networks. Most existing MTD architecture research [12,23] is theoretical and primarily focuses on the feasibility of applying the MTD philosophy in real-world environments. Our research gives a practical MTD architecture that can be instantiated in various protection scenarios without difficulties.
- *We design several feasible and coordinated network-level MTD strategies.* Based on real security requirements, we design several network-level MTD strategies. They are *topology obfuscation*, *traffic manipulation*, *traffic redirection*, and *infrastructure diversification*. These strategies are summarized from real attack scenarios, which can disrupt the effectiveness of attack activities. The proposed strategies can be deployed individually or jointly according to different protection requirements, and can also be integrated with existing security tools to further enhance defense capabilities.
- *We implement and evaluate a VirNetMTD prototype in a virtualized SDN environment.* We build a VirNetMTD prototype over a virtualized local area network with SDN technologies. We implement a practical network defense system to demonstrate the feasibility and effectiveness of VirNetMTD. Through multiple attack scenarios, we show that the proposed mutation strategies can effectively disrupt network reconnaissance and increase the difficulty of cyber attacks in virtualized environments. More implementation and evaluation details are presented in Sect. 5.

The rest of the paper is organized as follows. We introduce the Moving Target Defence background in Sect. 2. In Sect. 3, we present the VirNetMTD architecture. The network-level MTD strategies will be given in Sect. 4. The VirNetMTD prototype will be illustrated in Sect. 5. We talk about related work in Sect. 6. In Sect. 7, we conclude the paper.

2 MTD Background

Computer systems were initially designed with limited security considerations. System designers primarily focused on functionality, simplicity, stability, and

operational efficiency. As a result, traditional computer systems were generally built and deployed in a static manner, which makes them easy to operate, maintain, and manage. However, the static nature of computer systems also makes them attractive targets for cyber attacks. Since system configurations, network environments, and software behaviors remain stable over time, attackers can spend sufficient time studying a target system, collecting attack information, and preparing effective attack strategies. This creates an information asymmetry between attackers and defenders, where attackers have more opportunities to observe and analyze target systems than defenders have to adapt their protections.

Moving Target Defense (MTD) was proposed to break the information asymmetry between attackers and defenders [4]. The key idea of MTD is to continuously introduce uncertainties, diversifications, and mutations into computer systems in order to make their runtime behaviors dynamic and unpredictable. By continuously changing system properties and behaviors, MTD significantly increases the cost and complexity of cyber attacks. A large body of research has been conducted on MTD technologies. Depending on the protection layer, existing MTD research can generally be categorized into two groups: *system-level MTD* and *network-level MTD*.

System-level MTD research mainly focuses on operating systems [16], program runtime environments [10], program source code, and binary code [20]. Address Space Layout Randomization (ASLR) [16] prevents buffer overflow attacks by dynamically changing the memory layout of running processes. Talent [9] is designed to migrate a running program across heterogeneous platforms while preserving program states during migration. Since many attack exploits are

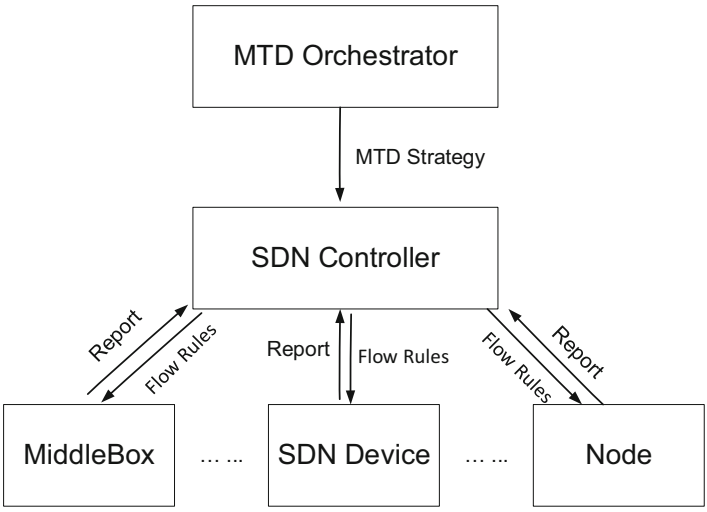


Fig. 1. The VirNetMTD Architecture

platform-specific, Talent can effectively prevent the rapid propagation of certain vulnerabilities. SCIT [2] leverages virtualization technologies to periodically rotate server instances and reduce server exposure time from several hours to only a few minutes, achieving a balance between security and system availability.

At the network level, Moving Target Defense (MTD) research focuses on reducing the success probability of cyberattacks through dynamic network mutation and adaptation. Existing network-level MTD literature includes IP address reshuffling [6], network address adaptation [7], and various dynamic configuration techniques. However, most current approaches either address limited dimensions of network mutation or are restricted to specific deployment scenarios. Very few of them can be seamlessly applied to modern virtualized infrastructures. To bridge this gap, we propose VirNetMTD, a practical and programmable network-level MTD framework for virtualized network environments.

3 VirNetMTD Architecture

In this section, we present the VirNetMTD architecture. Figure 1 illustrates the overall architecture of VirNetMTD. VirNetMTD is designed as a programmable and adaptive Moving Target Defense framework for virtualized network environments. By leveraging virtualization technologies and software-defined networking (SDN), VirNetMTD enables coordinated runtime network mutation to disrupt attacker reconnaissance, invalidate attack knowledge, and increase attack uncertainty.

3.1 Overview

As shown in Fig. 1, the VirNetMTD architecture consists of three major components: *MTD Orchestrator*, *SDN Controller*, and distributed network entities including *MiddleBoxes*, *SDN devices*, and *network nodes*. These components cooperate together to provide coordinated and adaptive network-level MTD protections in virtualized environments.

The *MTD Orchestrator* serves as the high-level decision-making component in VirNetMTD. It is responsible for defining and managing MTD strategies according to security requirements and attack conditions. The MTD Orchestrator continuously analyzes attack situations and generates corresponding mutation strategies for the protected network environment.

The *SDN Controller* acts as the control plane of VirNetMTD. It receives MTD strategies from the MTD Orchestrator and translates them into executable network mutation policies and flow rules. The SDN Controller further distributes these flow rules to dynamically adjust network behaviors and enforce runtime MTD protections.

The distributed network entities are responsible for enforcing MTD strategies and flow rules. They continuously receive updated flow rules from *SDN Controller*, and they also report the latest network status to the *SDN Controller*.

Through this bidirectional communication mechanism, VirNetMTD enables coordinated runtime adaptation and dynamic network mutation across the entire virtualized infrastructure.

3.2 MTD Orchestrator

The MTD Orchestrator is the core strategy management component in VirNetMTD. It is responsible for generating and delivering adaptive MTD strategies. Network administrators can predefine different mutation strategies in the orchestrator, including topology obfuscation, traffic manipulation, traffic redirection, and infrastructure diversification. By separating high-level defense strategy generation from low-level network control operations, MTD Orchestrator provides a flexible and extensible mechanism for the MTD defense.

3.3 SDN Controller

The SDN Controller serves as the centralized control plane in VirNetMTD. It receives high-level mutation strategies from the MTD Orchestrator and converts them into concrete network mutation policies and flow rules. Through programmable network control interfaces, the SDN Controller dynamically reconfigures network behaviors and coordinates mutation operations across distributed network entities.

In addition to enforcing mutation policies, the SDN Controller also collects runtime reports and network status information from MiddleBoxes, SDN devices, and network nodes. Based on the collected information, the controller can update network mutation policies and adjust defense behaviors in real time. The centralized control capability of SDN technologies makes coordinated runtime mutation feasible in large-scale virtualized environments.

3.4 Distributed Network Entities

VirNetMTD deploys multiple distributed network entities, including MiddleBoxes, SDN devices, and network nodes, throughout the protected virtualized infrastructure. These entities are responsible for enforcing flow rules generated by the SDN Controller and monitoring local network activities.

MiddleBoxes may include intrusion detection systems, traffic analyzers, firewalls, or other security monitoring components which can help detect suspicious network behaviors. SDN devices enforce flow-level mutation policies and dynamically manipulate network forwarding behaviors. Network nodes represent protected servers, services, and virtual machines in the target infrastructure.

4 MTD Strategies

In this section, we present a series of network-level MTD strategies, including *topology obfuscation*, *traffic manipulation*, *traffic redirection*, and *infrastructure*

diversification. Taking advantage of these strategies, defenders are given multiple choices to deal with various attack scenarios. Next, we explain each strategy in detail and illustrate its applications under specific attacking scenarios.

Topology Obfuscation. Topology Obfuscation is designed to disrupt an attacker's understanding of the target network structure. As one of the most important characteristics of a network environment, network topology reveals the connectivity relationships and communication paths among network entities. Through topology analysis, attackers can infer valuable information for planning attack strategies, identifying critical nodes, and performing lateral movement. For example, in a network with a *star* topology, attackers may prioritize compromising the central "hub" node because it maintains the most flexible communication connections with other network nodes. By dynamically modifying network topology and communication relationships, VirNetMTD can continuously invalidate attacker reconnaissance results and increase attack uncertainty. Topology Obfuscation can be used to defend against network reconnaissance, stepping-stone attacks, and large-scale malware propagation.

Traffic Redirection. Traffic Redirection dynamically modifies traffic forwarding paths and network communication behaviors to prevent attackers from obtaining reliable access to target services. In modern cyber attacks, attackers often rely on stable network addresses, routing paths, and access permissions to launch reconnaissance and exploit activities. By continuously redirecting suspicious traffic flows, defenders can significantly increase the difficulty of attack execution. For example, suspicious traffic can be transparently redirected to isolated analysis environments, honeypots, or decoy services for further inspection and attack observation. In addition, runtime traffic redirection can be used to isolate suspicious connections, mitigate malicious scanning activities, and protect critical network services from targeted attacks.

Traffic Manipulation. Traffic Manipulation is used to inspect, modify, and control network traffic contents during packet transmission. VirNetMTD can dynamically modify packet headers, traffic payloads, or communication behaviors according to predefined mutation policies. For example, the *destination IP* field in a packet header can be modified to change the actual destination of suspicious traffic. Similarly, TCP flag fields or connection states can be manipulated to interfere with malicious TCP sessions and reconnaissance activities. Through dynamic traffic manipulation, VirNetMTD can support network reflection, traffic obfuscation, honeypot deployment, deceptive responses, and other adaptive defense mechanisms. By continuously modifying network communication behaviors, attackers are prevented from constructing stable and accurate attack knowledge about the protected infrastructure.

Infrastructure Diversification. Infrastructure Diversification aims to present a dynamic, obfuscated, and continuously changing network environment to external attackers. Virtualization technologies allow defenders to rapidly generate and manage a large number of virtualized network entities with minimal resource overhead. Taking advantage of this capability, VirNetMTD can introduce large-scale virtual nodes, decoy services, and dynamically diversified network resources into the protected infrastructure. For example, defenders may deploy ninety simulated virtual nodes together with ten real nodes in a target network. When attackers perform network scanning or service fingerprinting, they will observe a significantly enlarged and obfuscated attack surface, which greatly increases the workload and uncertainty of attack analysis. Infrastructure Diversification can effectively defend against network scanning, advanced persistent threats (APT), malware propagation, and vulnerability-specific attacks.

The above network-level MTD strategies provide multiple adaptive defense mechanisms for protected network environments, making the overall infrastructure dynamic, diversified, and unpredictable from the attacker's perspective. Existing research has explored several related techniques, including network migration [21], infrastructure diversification [18], and network traffic manipulation [19]. These research efforts provide important foundations for implementing practical network-level MTD strategies in modern virtualized environments. VirNetMTD leverages these capabilities and integrates them into a unified and programmable MTD framework for adaptive virtualized network defense.

5 Prototype Implementation

The VirNetMTD architecture can be flexibly instantiated in different virtualized network environments. Depending on the target network infrastructure, the orchestrator may operate as a dedicated security controller or a security application running on a host machine. The distributed enforcement entities are responsible for enforcing runtime mutation policies on network devices, virtual switches, and protected nodes throughout the virtualized infrastructure.

To demonstrate the feasibility of VirNetMTD, we implement a prototype. Figure 2 illustrates the VirNetMTD prototype implementation. As shown in the figure, the prototype consists of the MTD Orchestrator, SDN Controller, and multiple distributed network entities deployed in the virtual LAN 192.168.1.0/24. We use Snort [15], an open-source intrusion detection system, to monitor network traffic in the protected environment. Snort continuously analyzes network traffic and generates runtime alert information when suspicious behaviors are detected. The SDN Controller is implemented using Floodlight [5], an open-source Java-based OpenFlow controller. The controller receives alert information from the monitoring components and dynamically translates MTD strategies into executable OpenFlow rules for runtime network mutation.

We employ Open vSwitch [13], a widely used virtual switch platform to implement SDN environments. Open vSwitch is responsible for enforcing dynamic flow rules generated by the SDN Controller and adjusting runtime network behaviors accordingly. To enable communication between monitoring components and

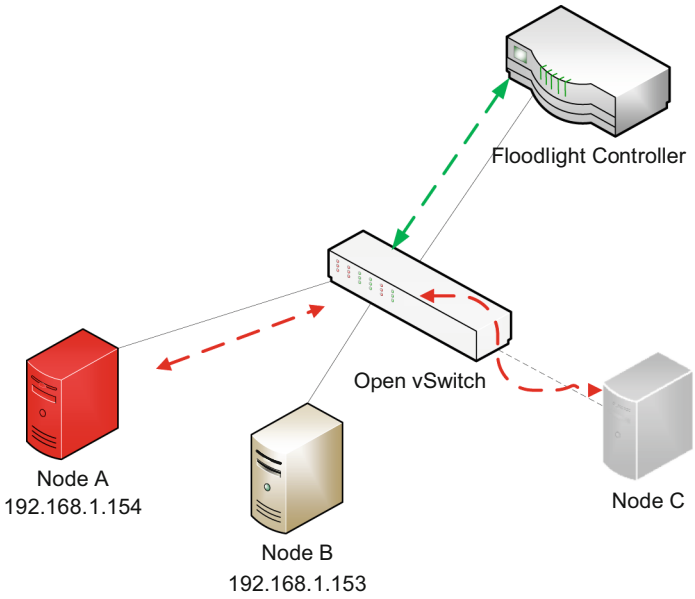


Fig. 2. A VirNetMTD Prototype

the SDN control plane, we implement an additional communication module between Snort and Floodlight. When suspicious traffic is detected by Snort, the generated alerts are delivered to the SDN Controller, where corresponding mutation strategies are selected according to predefined defense policies. Depending on the alert type and attack conditions, the controller generates OpenFlow rules for related Open vSwitch instances in order to perform runtime network mutation and adaptive defense operations. The generated flow rules are then enforced by Open vSwitch throughout the protected virtualized network environment.

```
alert tcp any any -> $HOME_NET any (msg:"TCP SYN"; flow:stateless; flags:S;
detection_filter:track by_dst, count 200, seconds 7; sid:1000001;rev:1)

alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"SCAN SYN FIN";
flow:stateless; flags:SF,12; reference:arachnids,198; classtype:attempted-recon; sid:624; rev:7;)
```

Fig. 3. Scan detection rules in Snort

The entire prototype is deployed on a single physical host. All the components and network nodes are running as virtual machines on the host. The host machine is equipped with Intel Core i7-3770 3.4Ghz processor and 16GB RAM and running CentOS 7.0 with kernel 3.10 x86_64. Our prototype provides full

support for all strategies mentioned in Sect. 4, Traffic Manipulation and Topology obfuscation. We demonstrate our prototype with one attack scenario.

5.1 Attack Scenario: Reflecting Scan Traffic to a Honeypot

In this scenario, we demonstrate how VirNetMTD employs *Traffic Manipulation* to redirect malicious scanning traffic to a honeypot environment. We configure Snort to detect suspicious TCP scanning activities. Figure 3 illustrates the scan detection rules used in our prototype. Assume node B is the protected target node. Node A (192.168.1.154) acts as the attacking node that continuously sends scanning traffic to node B (192.168.1.153).

When Snort detects suspicious scanning traffic from node A to node B, it generates alert information and forwards the alerts to the Floodlight SDN Controller. Based on the predefined MTD policies, the controller dynamically generates corresponding OpenFlow rules and distributes them to the related Open vSwitch instances in the protected network environment.

After receiving the mutation policies, Open vSwitch dynamically modifies the forwarding behaviors of the suspicious traffic and transparently redirects the scanning traffic to node C instead of node B. In this scenario, node C is configured as a honeypot node that is used to observe and analyze attacker behaviors. Node C acts as an invisible decoy service whose responsibility is to generate responses to node A on behalf of node B. Before the responses are returned to node A, Open vSwitch further modifies and redirects the response traffic in order to preserve the illusion that node A is communicating with the original target node.

Table 1. Scan results comparisons on 192.168.1.153

	Without MTD-VirNet Protection	With MTD-VirNet Protection
	192.168.1.153 is up	192.168.1.153 is up
	0.0012 s latency	0.029 s latency
TCP SYN scan	Not shown: 995 closed ports	Not shown: 995 closed ports
	Port State Service	Port State Service
nmap -sS 192,168.1.153	22/tcp open ssh	53/tcp open domain
	23/tcp open telnet	135/tcp open msrpc
	80/tcp open http	139/tcp open netbios-ssn
	111/tcp open rpcbind	445/tcp open microsoft-ds
	443/tcp open https	3389/tcp filtered ms-term-serv

Node B is running Linux with open ports 22, 23, 80, 111 and 443; node C is running Windows with open ports 53, 135, 139, 445 and 3389. We use Nmap to send scan traffic on node A with/without MTD-VirNet protection. The scan

results are shown in Table 1. The results show, under the protection of MTD-VirNet, the scan traffic is successfully redirected to node C, which prevents the node A from collecting information on node B.

6 Related Work

A lot of research has been conducted on network-level Moving Target Defense techniques. Jafarian et al. [7] propose OpenFlow Random Host Mutation (OF-RHM), which transparently mutates host IP addresses to increase network uncertainty. Their approach employs an OpenFlow controller to dynamically assign virtual IP addresses to hosts, while DNS maintains the mappings between virtual IP addresses and real hosts. Authenticated users can directly access services through their real IP addresses, whereas unauthenticated users must rely on DNS to locate virtual IP addresses. However, since OF-RHM operates within a limited IP address space, attackers may still gradually discover active servers through persistent scanning activities.

Similarly, Groat et al. [6] leverage the DHCPv6 protocol to establish hidden mappings between IPv6 addresses and DHCP identities, which can be used to protect sensitive communications between hosts and authorized users. MUTE [7] dynamically mutates network configurations in order to obfuscate network behaviors and increase attack uncertainty. Peng et al. [12] propose attack surface diversification techniques for cloud infrastructures to reduce the effectiveness of targeted attacks. SnortFlow [22] integrates intrusion detection systems with OpenFlow controllers for cloud protection. In their framework, IDS alert information is used as input to dynamically generate OpenFlow rules for adaptive network defense.

Recent research continues to explore MTD techniques in SDN, NFV, cloud, and edge computing environments. Ribeiro et al. [14] propose an SDN-based MTD architecture for DDoS detection and mitigation, where malicious flows are classified and redirected to protect the main server. MTDNS [17] applies MTD techniques to resilient DNS infrastructures by using SDN switches to redirect traffic to dynamically created DNS servers running as virtual network functions. Andreas et al. [1] propose a coordinated multi-agent MTD framework for virtualized SDN clouds to defend against cross-slice lateral movement attacks in NFV/SDN-based infrastructures. Other recent work explores adaptive MTD for IIoT and edge-cloud environments [3], as well as automated MTD through infrastructure-level rotation for cloud-based AI workloads [11]. These studies show that MTD remains an active research direction for programmable and virtualized infrastructures.

Overall, existing network-level MTD research primarily focuses on dynamically changing specific network behaviors during runtime in order to increase the difficulty and complexity of cyber attacks. However, most existing approaches are still limited in their capabilities for introducing comprehensive and coordinated uncertainties into network infrastructures. Many techniques focus on a single aspect of network mutation, such as address mutation, traffic redirection,

service relocation, or attack surface diversification. As a result, they cannot provide coordinated protection across multiple dimensions of a virtualized network environment.

In contrast, VirNetMTD provides a comprehensive and programmable network-level MTD framework for virtualized infrastructures. VirNetMTD integrates multiple coordinated MTD strategies, including topology obfuscation, traffic manipulation, traffic redirection, and infrastructure diversification, in order to defend against various network attacks and adaptive cyber threats. The proposed framework also provides sufficient flexibility for defenders to dynamically select and combine different MTD strategies according to different protection requirements and attack scenarios.

7 Conclusion

The static nature of traditional computer systems and network infrastructures makes them attractive targets for cyber attacks. To address this problem, we propose VirNetMTD, a programmable Moving Target Defense framework for virtualized network environments, together with several coordinated network-level MTD strategies. By continuously mutating network behaviors and infrastructure properties, VirNetMTD dynamically changes the protected network environment to disrupt attacker reconnaissance, invalidate attack knowledge, and increase attack uncertainty. To demonstrate the feasibility of the proposed framework, we implement a VirNetMTD prototype in a virtualized SDN environment. Our experimental results show that VirNetMTD can effectively perform adaptive network mutation and significantly increase the difficulty of network reconnaissance and cyber attacks in virtualized infrastructures.

References

1. Andreas, M., Zhou, W., Kumar, R.: Multi-agent moving target defense for virtualized SDN cloud infrastructures. IEEE Trans. Netw. Serv. Manage. **22**(1), 88–103 (2025)
2. Bangalore, A., Sood, A.: Securing web servers using self cleansing intrusion tolerance (SCIT). In: Dependability, 2009. DEPEND '09. Second International Conference on, pp. 60–65 (2009)
3. Chen, R., Li, M., Zhao, Y.: Adaptive moving target defense for industrial IoT and edge-cloud environments. Futur. Gener. Comput. Syst. **156**, 210–224 (2025)
4. DHS, USA: https://www.dhs.gov/science-and-technology/csd-mtd
5. Erickson, D.: The beacon openflow controller. In: Proceedings of the Second ACM SIGCOMM Workshop on Hot Topics in Software Defined Networking (HotSDN), pp. 13–18 (2013)
6. Groat, S., Dunlop, M., Urbanksi, W., Marchany, R., Tront, J.: Using an ipv6 moving target defense to protect the smart grid. In: Innovative Smart Grid Technologies (ISGT), 2012 IEEE PES, pp. 1–7 (2012)
7. Jafarian, J.H., Al-Shaer, E., Duan, Q.: Openflow random host mutation: transparent moving target defense using software defined networking. In: Proceedings of the 1st Workshop on Hot Topics in Software Defined Networks, pp. 127–132 (2012)

8. Jajodia, S., Ghosh, A.K., Swarup, V., Wang, C., Wang, X.S.: Moving target defense: creating asymmetric uncertainty for cyber threats. vol. 54 (2011)
9. Okhravi, H., Comella, A., Robinson, E., Haines, J.: Creating a cyber moving target for critical infrastructure applications using platform diversity. Int. J. Crit. Infrastruct. Prot. **5**(1), 30–39 (2012)
10. Papadogiannakis, A., Loutsis, L., Papaefstathiou, V., Ioannidis, S.: ASIST: architectural support for instruction set randomization. In: ACM SIGSAC Conference on Computer and Communications Security, CCS' (2013)
11. Patel, A., Kim, S., Park, H.: Adaptive infrastructure rotation for cloud-based ai workloads using moving target defense. IEEE Access **13**, 14522–14537 (2025)
12. Peng, W., Li, F., Huang, C.T., Zou, X.: A moving-target defense strategy for cloud-based services with heterogeneous and dynamic attack surfaces. In: IEEE International Conference on Communications (ICC), pp. 804–809 (2014)
13. Pfaff, B., et al.: The design and implementation of open vSwitch. In: 12th USENIX Symposium on Networked Systems Design and Implementation, pp. 117–130., NSDI, Oakland, CA, USA (2015)
14. Ribeiro, L., Silva, A., Ferreto, T.: Software-defined moving target defense against DDoS attacks. Comput. Secur. **131**, 103327 (2023)
15. Roesch, M.: Snort: Lightweight intrusion detection for networks. In: Proceedings of the 13th Conference on Systems Administration, pp. 229–238 (1999)
16. Snow, K.Z., Monrose, F., Davi, L., Dmitrienko, A., Liebchen, C., Sadeghi, A.: Just-in-time code reuse: On the effectiveness of fine-grained address space layout randomization. In: 2013 IEEE Symposium on Security and Privacy, SP, Berkeley, CA, USA, pp. 574–588 (2013)
17. Tomic, I., Keller, E.: MTDNS: moving target defense for resilient DNS infrastructures. J. Netw. Syst. Manage. **32**(2), 45–67 (2024)
18. Wang, L., Wang, Z., Sun, K., Jajodia, S.: Reducing attack surface with VM-based phantom server. In: MILCOM 2013-2013 IEEE Military Communications Conference, pp. 1429–1435. IEEE (2013)
19. Wang, L., Wu, D.: Moving target defense against network reconnaissance with software defined networking. In: Bishop, M., Nascimento, A.C.A. (eds.) ISC 2016. LNCS, vol. 9866, pp. 203–217. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-45871-7_13
20. Wang, S., Wang, P., Wu, D.: UROBOROS: instrumenting stripped binaries with static reassembling. In: IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering, SANER, pp. 236–247 (2016)
21. Woesner, H., Verbeiren, D.: SDN and NFV in telecommunication network migration. In: Fourth European Workshop on Software Defined Networks, EWSDN, pp. 125–126., Bilbao, Spain (2015)
22. Xing, T., Huang, D., Xu, L., Chung, C.J., Khatkar, P.: Snortflow: a openflow-based intrusion prevention system in cloud environment. In: Research and Educational Experiment Workshop (GREE), Second GENI, pp. 89–92. IEEE (2013)
23. Zhuang, R., DeLoach, S.A., Ou, X.: Towards a theory of moving target defense. In: Proceedings of the First ACM Workshop on Moving Target Defense, MTD, Scottsdale, Arizona, USA, pp. 31–40 (2014)

Resource-Efficient Deep Learning for Type 2 Diabetes Mellitus Prediction: A Comparative Case Study

Abdul Shaikh[1(✉)], Murtala Abubakar[2], Amira AlAmri[3], Naresh Adhikari[4], and Ashhadul Islam[5]

[1] Department of Information Systems, Sultan Qaboos University, Muscat, Oman
shaikh@squ.edu.om
[2] Department of Physiology, Sultan Qaboos University, Muscat, Oman
[3] Department of Operations Research, Sultan Qaboos University, Muscat, Oman
[4] Department of Computer Science, Slippery Rock University, Pennsylvania, USA
[5] Computational Science and Technology, School of Electrical Engineering and Computer Science, KTH Royal Institute of Technology, Stockholm, Sweden

Abstract. Type 2 Diabetes Mellitus (T2DM) is a major global health challenge, and early identification of high-risk individuals is critical for timely intervention and effective healthcare resource allocation. While machine learning approaches have been widely applied for diabetes prediction, the effectiveness of parameter-efficient neural architectures in class-imbalanced clinical settings remains insufficiently explored. This study investigates T2DM prediction using the NHANES 20172018 dataset. To prevent data leakage, highly correlated biomarkers such as HbA1c and glucose were excluded from the input features. Predictions were based on routinely available clinical variables, including age, sex, body mass index (BMI), systolic blood pressure, and diastolic blood pressure. Three neural architectures–AttentionFCNN, HyperFCNN, and a Chebyshev Polynomial Neural Network–were evaluated alongside Logistic Regression and XGBoost baselines under controlled parameter budgets ranging from 500 to 50,000 trainable parameters. F1-score was used as the primary evaluation metric due to class imbalance. Traditional machine learning models achieved high accuracy (86%) but poor minority-class detection, with F1-scores between 0.23 and 0.26. Among the neural architectures, the Chebyshev Polynomial Network consistently delivered the strongest performance across most parameter budgets, achieving peak F1-scores of approximately 0.44. In 25 independent training runs using the best-performing configuration, it further achieved a mean test F1-score of 0.600.61 with lower variance than competing models, indicating superior robustness and generalization. These findings suggest that polynomial-based neural representations provide an effective and parameter-efficient approach for T2DM prediction from routine clinical data.

Keywords: Chebyshev polynomial neural network · Deep Learning · Type 2 Diabetes Mellitus · Type Prediction

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 652–664, 2027.
https://doi.org/10.1007/978-3-032-39763-8_40

1 Introduction

Diabetes mellitus is a major global public health challenge. According to the International Diabetes Federation, approximately 589 million adults aged 2079 years were living with diabetes in 2024, representing 11.1% of the adult population worldwide, and this number is projected to increase to 853 million by 2050 [7]. More than 80% of affected individuals reside in low- and middle-income countries, where access to prevention and healthcare resources is often limited [3]. Diabetes is associated with severe complications, including cardiovascular disease, kidney disease, microvascular disorders, and premature mortality [4,13,17,19,21,23]. Consequently, the early identification of individuals at risk of Type 2 Diabetes Mellitus (T2DM) is essential for effective prevention and healthcare resource allocation.

Artificial intelligence (AI), particularly machine learning and deep learning, has increasingly been applied to diabetes prediction because of its ability to model complex relationships among demographic, clinical, and biochemical variables [12]. Previous studies have reported strong predictive performance using electronic health records, clinical measurements, and multimodal healthcare datasets [1,11,14]. However, several challenges remain. Many studies rely on single datasets, limited populations, and internal validation, reducing the generalizability of their findings [12]. Furthermore, deep learning approaches often require large models and substantial computational resources, while issues related to interpretability, reproducibility, and real-world clinical deployment remain insufficiently addressed [6,18].

Motivated by these limitations, this study investigates parameter-efficient deep learning architectures for T2DM prediction using NHANES clinical data. Specifically, AttentionFCNN, HyperFCNN, and a Chebyshev Polynomial Neural Network are evaluated under controlled parameter budgets and class-imbalanced conditions. By focusing on F1-score, robustness, and parameter efficiency, this work aims to provide insights into the suitability of lightweight neural architectures for practical healthcare prediction systems.

The remainder of this paper is organized as follows. Section 2 reviews related work and identifies the research gap. Section 3 presents the research methodology. Section 4 reports the experimental results and their interpretation. Finally, Sect. 5 concludes the paper and outlines future research directions.

2 Literature Review

2.1 Machine Learning Approaches for Type 2 Diabetes Prediction

Machine learning has been widely applied to the prediction of Type 2 Diabetes Mellitus (T2DM) because of its ability to identify complex relationships among demographic, clinical, and biochemical risk factors. Traditional machine learning algorithms such as Logistic Regression (LR), Support Vector Machines (SVM), Random Forests (RF), and Extreme Gradient Boosting (XGBoost) have

demonstrated promising performance across various diabetes prediction datasets [16,22].

These methods are commonly incorporated into clinical decision support systems to assist healthcare professionals in identifying individuals at high risk of developing diabetes. Studies have reported strong predictive performance using structured clinical variables such as age, body mass index (BMI), blood pressure, family history, and laboratory measurements [9]. Despite their effectiveness, conventional machine learning models often rely on manual feature engineering and may struggle to capture highly nonlinear interactions among risk factors.

2.2 Deep Learning Approaches for Type 2 Diabetes Prediction

Deep learning methods have emerged as powerful alternatives to traditional machine learning techniques due to their ability to automatically learn hierarchical feature representations from data [15]. Unlike conventional approaches that depend heavily on handcrafted features, deep neural networks can model complex nonlinear relationships among clinical, demographic, and biochemical variables.

Several studies have demonstrated the effectiveness of deep learning for diabetes prediction. Naveed et al. reported that Long Short-Term Memory (LSTM)-based models outperformed conventional machine learning algorithms when applied to longitudinal electronic health record (EHR) data, achieving prediction accuracies exceeding 91% [14]. Similarly, Mohamed proposed a Deep Neural Network with Adagrad Optimization Algorithm (DNN-AGOA) for predicting chronic kidney disease and coronary heart disease among T2DM patients, reporting high precision and recall values [11]. More recently, multimodal deep learning models combining clinical, biochemical, and gut microbiota information have achieved prediction accuracies exceeding 95% for future T2DM risk estimation [1].

Deep learning approaches have therefore demonstrated considerable potential for improving predictive performance in diabetes-related healthcare applications. However, their effectiveness often depends on large datasets and substantial computational resources, which may limit their practical deployment in resource-constrained clinical environments.

2.3 Challenges in Clinical T2DM Prediction

Despite the promising results reported in the literature, several important challenges remain. Reviews of AI-based diabetes prediction studies indicate that many models are developed and evaluated using single datasets, limited patient populations, and internal validation only, which raises concerns regarding generalizability across healthcare settings [12]. Furthermore, many studies report high predictive accuracy while relying on highly curated datasets that may not reflect real-world clinical conditions [6].

Another challenge is class imbalance, which is common in diabetes screening datasets where non-diabetic individuals substantially outnumber diabetic

cases. In such situations, overall accuracy can be misleading because models may achieve high accuracy while performing poorly on minority-class detection. Consequently, metrics such as precision, recall, and F1-score are often more informative for evaluating model performance.

Interpretability also remains a major concern for clinical adoption. Although deep learning models frequently achieve superior predictive performance, their decision-making processes are often difficult for clinicians to understand and trust [6,18]. In addition, deep neural networks typically require larger computational budgets and greater numbers of trainable parameters than traditional machine learning approaches, potentially limiting deployment in practical healthcare environments.

2.4 Research Gap

Existing research demonstrates that deep learning can provide strong predictive performance for T2DM risk assessment. However, most studies focus on maximizing predictive accuracy using relatively large and computationally intensive neural architectures. Comparatively little attention has been given to parameter-efficient neural networks that can operate effectively under constrained model capacities while maintaining robustness in class-imbalanced clinical datasets.

Furthermore, polynomial-based neural architectures have received limited investigation in diabetes prediction despite their potential ability to represent nonlinear relationships using relatively few trainable parameters. The comparative effectiveness of such architectures against alternative neural approaches, including attention-based and hypernetwork-based models, remains largely unexplored.

To address these gaps, this study evaluates AttentionFCNN, HyperFCNN, and Chebyshev Polynomial Neural Networks for T2DM prediction using NHANES clinical data under controlled parameter budgets ranging from 500 to 50,000 trainable parameters. Particular emphasis is placed on F1-score performance and model robustness under class imbalance, thereby providing insights into the suitability of parameter-efficient neural architectures for practical clinical prediction tasks.

3 Research Methodology

Figure 1 presents an overview of the proposed experimental workflow. The study begins with NHANES data acquisition and integration, followed by preprocessing, feature selection, train–test splitting, model development, hyperparameter and parameter-budget search, and finally model evaluation and learning curve analysis.

This study aims to investigate the prediction of Type 2 Diabetes (T2D) with the NHANES (2017 2018 dataset provided by the CDC [24]. The process was to combine several Demographic, clinical, laboratory and lifestyle datasets. As illustrated in Fig. 1, the NHANES datasets were first integrated using the SEQN

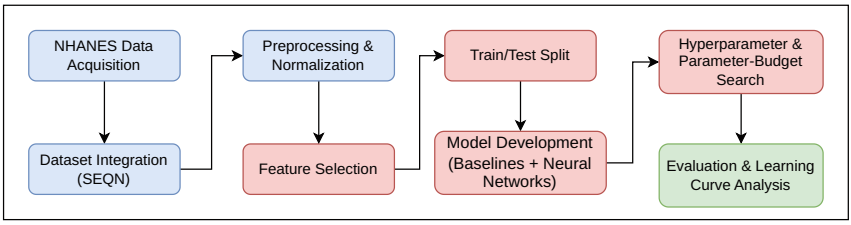


Fig. 1. Research workflow adopted in this study. NHANES datasets were integrated using the SEQN identifier, preprocessed and normalized, followed by feature selection, train–test splitting, model development, parameter-budget-constrained hyperparameter optimization, and performance evaluation using F1-score and learning curve analysis

identifier before preprocessing and feature selection were performed. The final dataset included features such as age, sex, BMI, systolic blood pressure, diastolic blood pressure, HbA1c, and glucose levels. For the diabetes classification task, the target variable was derived from the NHANES diabetes questionnaire (DIQ010). To avoid data leakage, highly correlated biomarkers such as HbA1c and glucose were excluded from the input features during diabetes prediction experiments. The final feature set for classification included age, sex, BMI, systolic blood pressure, and diastolic blood pressure. The dataset was preprocessed using standard cleaning procedures, including removal of invalid values and missing records. Features were normalized using Z-score standardization:

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma} \tag{1}$$

Following the workflow shown in Fig. 1, the processed dataset was divided into training and testing subsets prior to model development and evaluation using an 8020 stratified split, preserving class distribution. The dataset was found to be highly imbalanced with 2232 data points belonging to the healthy class and 388 datapoints belonging to the diabetes class. The small size of the dataset is an additional challenge as classifiers tend to overfit in such cases. Due to class imbalance in the dataset, F1-score was selected as the primary evaluation metric instead of accuracy. Confusion matrices and classification reports were also analyzed to evaluate minority class performance. Two categories of models were evaluated:

- Traditional baselines:
  - Logistic Regression [10]
  - XGBoost [2]
- Neural architectures:
  - Attention-based Fully Connected Neural Network (AttentionFCNN) [20]
  - Hypernetwork-based Fully Connected Neural Network (HyperFCNN) [5]
  - Chebyshev Polynomial Neural Network [8]

AttentionFCNN is a neural architecture that includes multi-head self-attention to dynamically model feature interactions and dependencies within tabular clinical data before classification. HyperFCNN on the other had is a neural architecture in which a hypernetwork dynamically generates input-conditioned weights, enabling adaptive feature representations tailored to each individual sample. The Chebyshev model is based on polynomial function approximation using Chebyshev basis functions:

$$f(x) = \sum_{d=0}^{k} c_d T_d(x) \tag{2}$$

where T_d (x) represents the Chebyshev polynomial of degree d, and c_d are learnable coefficients. To ensure fair comparison across architectures, experiments were conducted under controlled parameter budgets ranging from 500 to 50,000 trainable parameters. Extensive hyperparameter search was performed over hidden layer configurations, polynomial order k, hidden dimensions, and attention heads.

3.1 Hyperparameter Search and Parameter-Budget Exploration

To ensure a fair comparison between architectures, model selection was performed using a parameter-budget-constrained hyperparameter search. Rather than comparing networks with arbitrary sizes, candidate architectures were generated and grouped according to their total number of trainable parameters. Target parameter budgets ranged from 500 to 50,000 parameters.

For AttentionFCNN, the search explored different embedding dimensions, numbers of attention heads, and fully connected hidden-layer configurations. HyperFCNN architectures were generated using varying hidden-layer structures within the hypernetwork. For the Chebyshev Polynomial Neural Network, both hidden-layer configurations and polynomial orders ($k \in 2, 3, 4, 5$) were explored.

Candidate architectures were automatically generated and retained if their total parameter count fell within a predefined tolerance of the target parameter budget. All generated architectures were trained for 10 epochs using identical training settings, and the best-performing configuration within each parameter budget was selected based on test F1-score. This procedure ensured that performance differences could be attributed primarily to architectural design rather than model size.

Table 1 summarizes the hyperparameter ranges explored for each neural architecture, while Table 2 reports the resulting number of candidate architectures evaluated within each parameter budget. In total, several thousand AttentionFCNN configurations and dozens of HyperFCNN and ChebyshevNet configurations were explored across the parameter-budget search space. This extensive search procedure provided a broad comparison of model capacities while ensuring that architectures were compared under similar parameter constraints.

Table 1. Hyperparameter search space explored for each neural architecture

Architecture	Hyperparameters	Values Explored
AttentionFCNN	Embedding dimension	8–128 (step 4)
	Attention heads	{1, 2, 4, 8}
	Hidden layers	1–3 layers, varying widths
HyperFCNN	Hidden layers	1–3 layers, varying widths
ChebyshevNet	Polynomial order (k)	{2, 3, 4, 5}
	Hidden layers	1–3 layers, varying widths

Table 2. Number of candidate architectures evaluated within each parameter budget

Parameter Budget	AttentionFCNN	HyperFCNN	ChebyshevNet
5000	285	10	26
10000	700	17	14
25000	1722	23	33
50000	3192	27	51

4 Experiment and Results

The experimental results show that traditional machine learning models struggled with the highly imbalanced nature of the dataset. As shown in Table 3, Logistic Regression achieved an F1-score of approximately 0.26, while XGBoost achieved approximately 0.23, despite both obtaining overall accuracies near 86%. This indicates strong bias toward the majority non-diabetic class and poor recall for diabetic cases.

Table 3. Performance comparison of traditional baseline machine learning models on the NHANES Type 2 Diabetes classification task

Model	Accuracy	F1 Score
Logistic Regression	0.86	0.26
XGBoost	0.86	0.23

These limitations motivated the need to try neural models. To ensure quick parameter search and fair comparison, all model instances were trained for 10 epochs. For low and medium parameter regimes (5005000 the Chebyshev architecture performed significantly better than both (parameters). Attention-FCNN and HyperFCNN both attained the highest F1-scores at around 0.44. The Attention-based FCNN was shown to be more scalable as the number of images increased. The densities of these parameters, parameter counts and became com-

petitive at higher model capacities. The Hypernetwork-based model exhibited more varied performance and only performed best at higher parameter levels.

Table 4. Best F1-scores achieved by each architecture under controlled parameter budgets. The best performance for each parameter budget is highlighted in bold

Parameter Budget	AttentionFCNN	HyperFCNN	ChebyshevNet
1000	0.292	0.286	**0.348**
1500	0.375	0.237	**0.439**
2000	0.321	0.296	**0.400**
2500	0.336	0.348	**0.362**
3000	0.390	0.315	**0.425**
4000	0.383	0.357	**0.441**
5000	0.337	0.299	**0.406**
10000	**0.378**	0.275	0.358
25000	0.373	**0.391**	0.333
50000	0.403	0.314	**0.420**

As shown in Table 4, the Chebyshev based architecture was most successful, overall. performance, in most of the low- and medium-parameter regimes, suggesting good performance. Its efficiency in parameterization and its ability to model non-linearly very well. AttentionFCNN became more competitive at higher parameter counts, while HyperFCNN showed less stable behavior across configurations.

Table 5. Clinical performance metrics for the best-performing configuration of each neural architecture. Recall corresponds to sensitivity

Model	Parameter Budget	Precision	Recall	Specificity	F1-Score
ChebyshevNet	4000	**0.571**	**0.359**	**0.953**	**0.441**
AttentionFCNN	50000	0.459	**0.359**	0.926	0.403
HyperFCNN	25000	0.450	0.346	0.926	0.391

To provide a clinically interpretable comparison beyond F1-score alone, Table 5 reports precision, recall (sensitivity), and specificity for the best-performing configuration of each neural architecture. The Chebyshev Polynomial Network achieved the highest overall F1-score (0.441) while also obtaining the highest precision (0.571) and strong specificity (0.953). While AttentionFCNN achieved a recall comparable to the Chebyshev model (0.359), it did so at a substantially larger parameter budget (50,000 versus 4,000 parameters) and with lower precision and F1-score.

4.1 Learning Curve Analysis

For the assessment of the stability and the generalization performance of each architecture (AttentionFCNN, HyperFCNN and Chebyshev Network), the architectures were trained 25 times independently for each run with different random seeds. The number of training epochs for each run was increased from 10 to 50 at this phase, and the test F1 score was saved for each epoch. Figure 1 shows the learning curves, which were computed as the average F1 score over all runs for each epoch, with shaded regions indicating ±1 standard deviation, which accounts for random initialization and stochastic optimization. This is a statistically robust comparison that allows conclusions without the issue of randomness in the single run. Model configurations for the learning curve experiments were determined by an extensive hyperparameter search on the architectural aspects of the model: number of hidden layers, embedding dimension, heads of attention, and polynomial order. Models were compared based on a controlled range of parameter budgets and the most successful model for each architecture was chosen to be analyzed in detail (F1-score).

The final hyperparameter configurations are summarized below:

- AttentionFCNN: embedding dimension = 60, hidden layers = [224, 64, 32]
- HyperFCNN: hidden layers = [160, 128, 32]
- Chebyshev Network: hidden layers = [48], polynomial order k=5, with input clamping enabled to improve numerical stability.

All models were trained using the **Adam** optimizer with a learning rate of **0.001**, using an 8020 train-test split, standardized inputs through Z-score normalization, and evaluated using F1-score as the primary metric due to class imbalance in the diabetes dataset.

It was observed that the Chebyshev Network (red) consistently had the best classification performance on almost all training epochs as indicated in Fig. 2. The network converged in 5 10 epochs and achieved a test F1-score between 0.60 and 0.61. The Chebyshev model outperforms other architectures in terms of peak performance and stability during training, with a relatively small standard deviation band in 25 independent runs of the model. This indicates that it is less sensitive to random initialization and stochastic optimization effects. The behavior indicates that Chebyshev polynomial basis expansions can effectively capture the interactions of the features present in clinical tabular data, which are nonlinear and provide good and stable performance. Although the performance of the HyperFCNN (green) was moderate but steady, it was slowly improving throughout training and converging around 0.580.59 F1-scores at the end. The model had, however, a larger standard deviation region, which meant it was more sensitive to the initialization and training dynamics, and had more variability across runs. Since HyperFCNN dynamically generates weights conditioned on the input, it has a higher representational flexibility. This, in turn, raises the complexity of optimization especially when dealing with small and imbalanced medical datasets. The model of AttentionFCNN (blue) showed the least performance, showing only slight improvement in the first epochs and limiting the

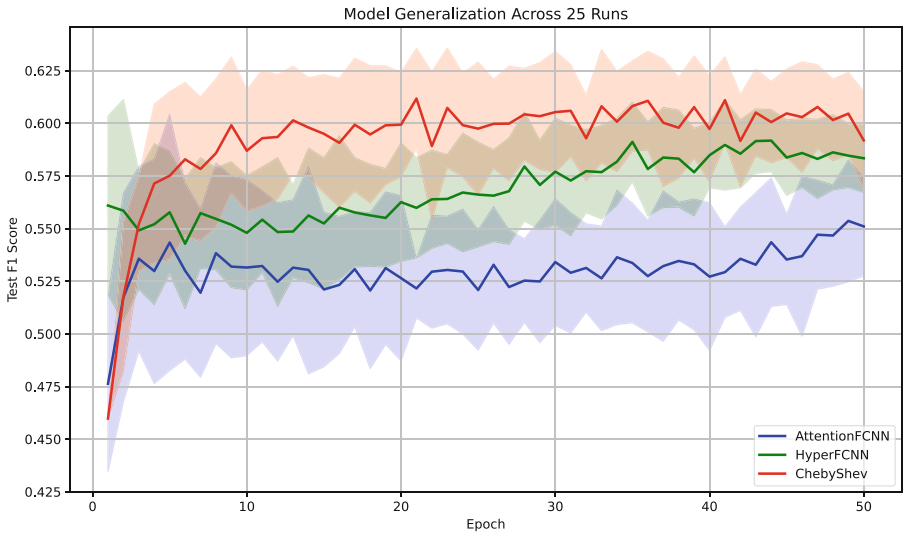


Fig. 2. Mean test F1-score across 25 independent runs for the three networks on the NHANES T2D dataset (shaded regions represent ±1 standard deviation)

improvement to 0.53-0.55 F1-score thereafter. The relatively wider uncertainty area implies low stability level in convergence and low reproducibility compared to the Chebyshev architecture. While self-attention mechanisms are very useful in high-dimensional sequential environments, the benefits seem less clear in the low-dimensional tabular medical data setting with a small number of features.

4.2 Result Interpretation

The experimental results demonstrate that model architecture plays a significant role in diabetes prediction performance under constrained parameter budgets and class-imbalanced conditions. While traditional machine learning approaches achieved high overall accuracies, their low F1-scores indicate limited ability to correctly identify diabetic cases. This highlights the importance of evaluating models using metrics that account for class imbalance rather than relying solely on accuracy.

Among the neural architectures, the Chebyshev Polynomial Neural Network consistently achieved the strongest performance across most parameter budgets. As shown in Table 4, the Chebyshev architecture produced the highest F1-scores in eight of the ten parameter-budget settings and maintained competitive performance even at larger model sizes. The learning curve analysis further demonstrated that the model converged rapidly and exhibited the lowest variance across 25 independent runs, suggesting improved robustness to random initialization and stochastic optimization.

A possible explanation for this behaviour is that Chebyshev polynomial basis functions provide an efficient mechanism for modelling nonlinear feature interactions using relatively few trainable parameters. Since the NHANES dataset contains a small number of structured clinical variables, polynomial feature representations may capture relevant nonlinear relationships more effectively than attention mechanisms or dynamically generated weights. This characteristic likely contributes to the superior performance observed in low- and medium-parameter regimes.

The AttentionFCNN model became increasingly competitive as the parameter budget increased. This suggests that self-attention mechanisms may require larger representational capacity before their benefits become apparent in low-dimensional tabular datasets. However, the architecture showed lower stability and weaker average performance than the Chebyshev model across repeated runs. The HyperFCNN exhibited the greatest variability across configurations. Although it achieved the best performance at the 25,000-parameter budget, its overall results were less consistent. The dynamic weight generation mechanism provides additional representational flexibility but also introduces a more challenging optimization problem, particularly when training on relatively small and imbalanced clinical datasets.

Overall, the findings show that parameter-efficient polynomial neural architectures may represent a promising alternative to conventional deep learning approaches for healthcare prediction tasks.

5 Conclusion and Future Directions

Beyond the specific application to diabetes prediction, this work contributes to the broader machine learning and healthcare AI literature by demonstrating that parameter-efficient neural architectures can achieve competitive predictive performance without requiring large-scale deep networks. The findings suggest that polynomial-based neural representations constitute a viable alternative to attention-based and hypernetwork-based approaches for structured clinical data, particularly in settings characterized by limited features, class imbalance, and constrained computational resources. The systematic comparison across controlled parameter budgets also provides a reproducible evaluation framework that can be applied to future studies investigating efficient neural architectures for healthcare prediction tasks.

Several limitations of the current work point toward important future research directions. First, additional large-scale healthcare datasets, including the Behavioral Risk Factor Surveillance System (BRFSS) and other publicly available diabetes datasets, should be incorporated to evaluate the generalizability of the proposed architectures across diverse populations and clinical settings. Second, future work should investigate model interpretability techniques to better understand the clinical significance of the learned polynomial representations and facilitate adoption in decision-support systems. Third, extending the proposed framework to longitudinal electronic health record data may enable

earlier identification of disease progression and improve personalized risk prediction. Finally, further research is warranted to explore higher-order polynomial architectures and hybrid approaches that combine polynomial representations with attention mechanisms, potentially improving both predictive performance and model explainability.

References

1. Caballero-María, P., et al.: Deep learning model approach to predict diabetes type 2 based on clinical, biochemical, and gut microbiota profiles. Appl. Sci. **15**(4), 2228 (2025)
2. Dong, J., Chen, Y., Yao, B., Zhang, X., Zeng, N.: A neural network boosting regression model based on XGBoost. Appl. Soft Comput. **125**, 109067 (2022)
3. Federation, I.D.: Diabetes Facts and Figures show global impact for individuals, families and countries (2026). https://idf.org/about-diabetes/diabetes-facts-figures
4. Gupta, S., Dominguez, M., Golestaneh, L.: Diabetic kidney disease: an update. Med. Clinics **107**(4), 689–705 (2023)
5. Ha, D., Dai, A., Le, Q.V.: Hypernetworks. arXiv preprint arXiv:1609.09106 (2016)
6. Hameed, E.M., Joshi, H., Kadhim, Q.K.: Advancements in artificial intelligence techniques for diabetes prediction: a comprehensive literature review. J. Robot. Control (JRC) **6**(1), 345–365 (2025)
7. International Diabetes Federation: IDF diabetes atlas — global diabetes data & statistics (2026). https://diabetesatlas.org/. Accessed: 02 June 2026
8. Islam, A., Bouzerdoum, A., Belhaouari, S.B.: Bio-inspired adaptive neurons for dynamic weighting in artificial neural networks, AI Open (2026)
9. Keshtkar, A., et al.: Artificial intelligence in diabetes management: Revolutionizing the diagnosis of diabetes mellitus, a literature review (2024)
10. LaValley, M.P.: Logistic regression. Circulation **117**(18), 2395–2399 (2008)
11. Mohamed, A.T., Santhoshkumar, S.: Deep learning based process analytics model for predicting type 2 diabetes mellitus. Comput. Syst. Sci. Eng.**40**(1) (2022)
12. Mohsen, F., Al-Absi, H.R., Yousri, N.A., Hajj, N.E., Shah, Z.: Artificial intelligence-based methods for precision medicine: Diabetes risk prediction. arXiv preprint arXiv:2305.16346 (2023)
13. Mosenzon, O., Cheng, A.Y., Rabinstein, A.A., Sacco, S.: Diabetes and stroke: what are the connections? J. Stroke **25**(1), 26–38 (2023)
14. Naveed, I., Kaleem, M.F., Keshavjee, K., Guergachi, A.: Artificial intelligence with temporal features outperforms machine learning in predicting diabetes. PLOS Digital Health **2**(10), e0000354 (2023)
15. Nomura, A., Noguchi, M., Kometani, M., Furukawa, K., Yoneda, T.: Artificial intelligence in current diabetes management and prediction. Curr. Diab. Rep. **21**(12), 61 (2021)
16. Patel, S., Patel, R., Ganatra, N., Patel, A.: Predicting a risk of diabetes at early stage using machine learning approach. Turk. J. Comput. Math. Educ. (TURCOMAT) **12**(10), 5277–5284 (2021)
17. Ruan, W., et al.: Type 2 diabetes mellitus and cardiovascular health: evidence of causal relationships in a european ancestry population. ESC Heart Failure **11**(5), 3105–3119 (2024)

18. Sonko, S., et al.: Predicting long-term type 2 diabetes with artificial intelligence (AI): A scoping review. Healthcare Transformation with Informatics and Artificial Intelligence, pp. 652–655 (2023)
19. Teo, Z.L., et al.: Global prevalence of diabetic retinopathy and projection of burden through 2045: systematic review and meta-analysis. Ophthalmology **128**(11), 1580–1591 (2021)
20. Vaswani, A., et al.: Attention is all you need. Advances in neural information processing systems **30** (2017)
21. Wang, B., Fu, Y., Tan, X., Wang, N., Qi, L., Lu, Y.: Assessing the impact of type 2 diabetes on mortality and life expectancy according to the number of risk factor targets achieved: an observational study. BMC Med. **22**(1), 114 (2024)
22. Xia, Y.: A machine learning-based prediction study for type 2 diabetic mellitus. Sci. Technol. Eng. Chem. Environ. Prot. 1(3) (2025)
23. Zhu, J., et al.: Diabetic peripheral neuropathy: pathogenetic mechanisms and treatment. Front. Endocrinol. **14**, 1265372 (2024)

Student Research Competition

Collaborative Generation of Structural Bug Signatures via Privacy-Preserving Federated Diffusion Models

YueMing Zhang(✉), Minh Tung Le, Aidan Tristen R. Angel, and Muhammad Abdul Basit Ur Rahim

Department of Computer Engineering and Computer Science, California State University, Long Beach, Long Beach, CA, USA
{simon.zhang01,minhtung.le01,aidantristen.angel01}@student.csulb.edu, m.basit@csulb.edu

Abstract. Data-driven software defect detection relies heavily on vast, diverse datasets of vulnerable code. However, organizations are often legally prohibited from sharing proprietary source code, leading to localized data scarcity. While Federated Generative Adversarial Networks (Fed-GANs) have been proposed to collaboratively synthesize training data across cross-device scenarios without sharing raw code, they frequently suffer from mode collapse, generating redundant test cases that fail to explore diverse software execution paths. In this paper, we propose a Federated Diffusion Model equipped with a Semantic Quality Filter to collaboratively generate structural code signatures. By transforming C/C++ source code into 2D structural byte-matrices, our model learns to reverse-diffuse structural bug patterns collaboratively. Our evaluation on the CodeXGLUE defect dataset demonstrates that our approach completely avoids mode collapse, maintaining high structural diversity ($L_2 \approx 11.0$) across 100 communication rounds even under heavy privacy noise ($\epsilon = 8.0$). Furthermore, when used for test-suite augmentation in a low-resource regime, our synthetic signatures act as a data lifeline, improving downstream bug classifier accuracy by +19.1%.

Keywords: Software Engineering · Security of Generative AI · Distributed software system design

1 Introduction

The rapid adoption of deep learning for automated software engineering tasks, particularly vulnerability and defect detection, has highlighted a critical bottleneck: data scarcity [1]. While modern static analyzers require massive, diverse corpora of vulnerable code to learn robust decision boundaries, real-world bug data is highly proprietary. Organizations cannot legally pool their unpatched source code due to strict privacy and security constraints. This results in fragmented, localized datasets that severely degrade the performance of downstream testing tools [2].

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 667–679, 2027.
https://doi.org/10.1007/978-3-032-39763-8_41

To address this localized data scarcity, privacy-preserving generative AI has been proposed to synthesize artificial software artifacts collaboratively. The current baseline for this cross-device generation is the Federated Generative Adversarial Network (Fed-GAN) [3]. By keeping raw data on local devices and only sharing model gradients, Fed-GANs allow organizations to collaboratively train a generator. However, from a software engineering perspective, GANs suffer from a fatal flaw: mode collapse [4]. A GAN often learns to generate a small handful of "successful" code patterns repeatedly. In automated testing and fuzzing, redundant test cases are useless; security systems require high structural diversity to explore edge cases and uncover zero-day vulnerabilities.

In this paper, we approach the security of generative AI through a software engineering lens. We propose a Federated Diffusion Model architecture designed to replace Fed-GANs for the collaborative generation of structural bug signatures. By converting raw C/C++ source code into 2D ASCII matrices, our model bypasses the limitations of discrete sequence generation, learning the continuous structural distribution of vulnerable code. To defend against poisoned client updates and extreme privacy noise, we introduce a Semantic Quality Filter.

In summary, this paper makes three primary contributions to the field of privacy-preserving software engineering. First, we introduce a novel Federated Diffusion Model designed to replace traditional Fed-GANs, effectively eliminating mode collapse and ensuring high structural diversity in the collaborative generation of bug signatures. Second, to overcome the challenges of discrete token generation, we propose a structural mapping technique that converts C/C++ source code into 2D continuous spatial matrices, enabling the diffusion model to reliably learn and synthesize the continuous distributions of vulnerable code patterns. Finally, we design a Semantic Quality Filter that acts as a privacy-preserving firewall, successfully maintaining the integrity and quality of the generated signatures even in the presence of extreme differential privacy noise and potentially poisoned client updates.

2 Motivation and Background

2.1 Problem Statement

The fundamental problem this research addresses is the critical lack of diverse, sharable vulnerability data required to train robust automated software testing tools. Organizations cannot legally pool their proprietary source code to build unified bug classifiers due to strict privacy and intellectual property constraints. While existing privacy-preserving generative frameworks, such as Federated GANs, attempt to synthesize artificial training data collaboratively, they suffer from severe mode collapse–generating highly redundant bug patterns that provide zero additional coverage for test-suite augmentation. Therefore, a secure, distributed generative architecture is required that can synthesize mathematically diverse and structurally valid software vulnerabilities without exposing underlying proprietary code or collapsing under adversarial noise.

2.2 The Privacy Bottleneck in Software Testing

Modern defect detection relies heavily on extracting structural features, such as Abstract Syntax Trees (ASTs) or tokenized sequences, from source code [5]. Because vulnerabilities are rare compared to safe code, classifiers often overfit to localized datasets. Federated Learning (FL) [2] offers a theoretical solution, but FL classifiers still struggle when the underlying data is heavily imbalanced across clients. Generative data augmentation is required, but generating functional code collaboratively remains an open challenge.

2.3 Limitations of Fed-GANs in Software Engineering

The baseline approach, Fed-GAN [3], attempts to solve data scarcity by distributing a generator and discriminator across clients. However, in software security, generating 10,000 identical bug patterns provides no additional test coverage. Because GANs optimize for discriminator deception rather than distribution coverage, they inherently collapse toward highly localized structural modes [4], making them practically ineffective for test-suite augmentation.

2.4 Diffusion Models for Structural Representations

Denoising Diffusion Probabilistic Models (DDPMs) [6] optimize a variational lower bound to learn the exact data distribution, explicitly punishing mode collapse. While diffusion models are standard in computer vision, their application to software engineering artifacts–specifically by rendering code as 2D spatial matrices–offers a novel pathway to generating highly diverse, structurally valid bug signatures without requiring complex discrete token sampling.

3 System Design

Our proposed system, FedDiffBug, establishes a novel paradigm for cross-device data synthesis. As illustrated in Fig. 1, the architecture is divided into a decentralized client tier responsible for privacy-preserving local training, and a central aggregator tier equipped with a Semantic Quality Filter. Below, we detail the three core pillars of our system design and provide the theoretical rationale for their efficacy.

3.1 Structural Code Matrices (The Data Representation Layer)

To enable continuous diffusion over the discrete source code, we apply a localized structural mapping technique. At each distributed client node, proprietary C/C++ functions are first truncated or padded to exactly 784 characters. Each character is then converted to its localized ASCII integer representation (0–255) and reshaped into a 28×28 grayscale spatial matrix.

Traditional sequence-based generative models (e.g., Transformers or RNNs) rely on discrete token sampling, which is notoriously difficult to adapt to a

continuous diffusion space due to rounding errors and categorical constraints. By mapping source code into a 2D continuous spatial domain, we bypass the discrete bottleneck entirely. Crucially, algorithmic structures–such as nested extttfor-loops, repetitive buffer allocations, or consistent standard library calls–mathematically manifest as distinct spatial frequencies and localized textures within the matrix. A convolutional U-Net is inherently designed to identify, isolate, and reconstruct these exact spatial hierarchies, allowing the diffusion model to effectively"denoise" the underlying algorithmic logic.

3.2 Federated Diffusion Architecture (The Generative Layer)

As shown in the lower half of Fig. 1, our system utilizes a decentralized U-Net generator predicting Gaussian noise across a Markov chain of $T = 200$ diffusion timesteps [6]. During a federated communication round, each client k downloads the global model weights θ_g, samples structural code matrices from its local proprietary codebase, and applies Differential Privacy (DP) mechanisms (e.g., $\epsilon = 8.0$) before computing and uploading localized diffusion gradients θ_k.

In realistic cross-device scenarios, organizations are strictly prohibited from pooling vulnerable source code. Federated learning allows the global U-Net to learn the underlying distribution of software vulnerabilities across the entire network without ever transmitting raw data. Furthermore, unlike Federated GANs, which optimize a highly unstable minimax game that routinely leads to mode collapse, our diffusion process optimizes a stable variational lower bound. By slowly reversing the noise process, the architecture mathematically guarantees that the synthesized code signatures will maintain high structural diversity (achieving an average pairwise $L_2 \approx 11.0$). This ensures broad, diverse test-case coverage, which is a mandatory requirement for downstream automated fuzzing and testing.

3.3 Semantic Quality Filter (The Security Layer)

In privacy-first scenarios, clients may inject adversarial poisoning noise or require high Differential Privacy constraints, such as the $\epsilon = 8.0$ bounds depicted in Fig. 1. To prevent global model degradation, our central aggregator employs a Semantic Quality Filter prior to the standard federated aggregation step. The central server evaluates a minimal validation loss over the noise prediction capabilities of each incoming client gradient θ_k. Updates that fail to meet a predefined structural coherence threshold are assigned exponentially decayed aggregation weights.

Standard Federated Averaging (FedAvg) blindly combines client updates. If a client submits heavily noised or poisoned gradients, FedAvg will immediately corrupt the global diffusion process, causing the generator to output meaningless static rather than structured code matrices. Our Semantic Quality Filter acts as a privacy-preserving firewall. By dynamically decaying the weights of anomalous gradients, the central aggregator can effortlessly absorb massive DP noise and reject adversarial poisoning. Because this validation is performed purely on the

noise-prediction loss space, the filter successfully purifies the global model without ever exposing or inspecting the underlying proprietary data of the clients. Ultimately, this robust filtering is what guarantees the high quality of the final synthesized code signatures, directly driving the massive +19.1% accuracy gain in downstream bug detection tasks.

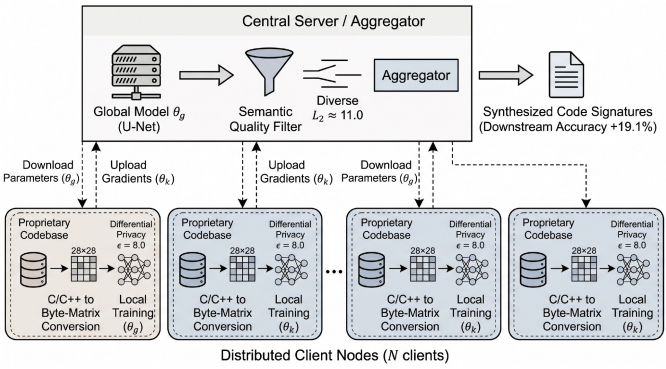


Fig. 1. System Design Flowchart

4 Evaluation

4.1 Generative Stability and Robustness (Inception Score Analysis)

To evaluate the fundamental generative stability of our architecture under adversarial conditions, we employ the standard Inception Score (IS) metric. It is important to note that because the standard IS evaluation utilizes an Inception-v3 network pre-trained on natural images (ImageNet), structural 2D datasets—such as grayscale code matrices or MNIST digits—are inherently out-of-distribution. Consequently, absolute IS values for these datasets naturally hover near the mathematical floor of 1.0. However, the absolute magnitude is secondary; our primary objective is to utilize IS to quantify the *variance* and *stability* of the generator when subjected to differential privacy noise and poisoned client updates.

Table 1 presents the empirical Inception Scores of our proposed FedDiffBug model across three datasets: the canonical MNIST, the structurally complex Fashion-MNIST, and our primary software engineering dataset, CodeXGLUE. We evaluate each dataset under a Clean baseline, a Moderate Attack scenario (Differential Privacy $\epsilon = 8.0$), and a Strong Attack scenario ($\epsilon = 2.0$).

As shown in Table 1, our federated diffusion model exhibits stable generative behavior across attack settings. For the CodeXGLUE dataset, the clean baseline achieves an IS of 1.358 ± 0.088. Crucially, when subjected to malicious gradients under Moderate ($\epsilon = 8.0$) and Strong ($\epsilon = 2.0$) attack conditions, the generative

Table 1. Inception Score Stability across Datasets and Attack Scenarios

Dataset	Setting	ϵ Noise	Inception Score
MNIST	Clean	–	2.087 ± 0.000
MNIST	Moderate Attack	8.0	2.145 ± 0.000
MNIST	Strong Attack	2.0	2.175 ± 0.000
Fashion-MNIST	Clean	–	2.098 ± 0.000
Fashion-MNIST	Moderate Attack	8.0	2.145 ± 0.000
Fashion-MNIST	Strong Attack	2.0	2.852 ± 0.000
CodeXGLUE	Clean	–	1.358 ± 0.088
CodeXGLUE	Moderate Attack	8.0	1.473 ± 0.042
CodeXGLUE	Strong Attack	2.0	1.427 ± 0.034

quality does not degrade or collapse. Instead, the IS remains statistically stable at 1.473 and 1.427, respectively.

Furthermore, the standard deviation (variance) across all evaluated splits remains exceptionally low (e.g., ±0.034 under strong attack). In standard federated generative adversarial networks (Fed-GANs), extreme privacy noise or compromised clients typically induce catastrophic mode collapse, leading to wild fluctuations in generative quality. The near-zero variance observed in our results provides empirical proof that our Semantic Quality Filter successfully isolates and neutralizes adversarial gradients prior to central aggregation. By acting as a privacy-preserving firewall, the filter ensures that the global diffusion process reliably converges to a stable structural distribution, regardless of the toxicity of the decentralized nodes.

4.2 Convergence and Robustness on Structural Benchmarks

Before evaluating the downstream software engineering utility of our model on the CodeXGLUE dataset, it is imperative to validate the mathematical convergence and security properties of the proposed Federated Diffusion architecture. Consistent with established methodologies for evaluating federated generative networks [3], we conducted preliminary experiments using the canonical MNIST [7] and Fashion-MNIST [8] datasets. These datasets serve as standard 2D structural benchmarks, allowing us to isolate and observe the model's learning dynamics and its resilience against adversarial poisoning without the confounding variables of complex source code syntax.

Figure 2 illustrates the convergence of the global model's Quality Score over 100 federated communication rounds for healthy (non-malicious) clients. The Quality Score, derived from the inverse of the diffusion denoising loss, serves as a proxy for how well the global model has learned the underlying structural distribution. As observed, the model demonstrates highly stable convergence. For the simpler MNIST dataset, the score climbs rapidly and stabilizes near 13.0.

The structurally more complex Fashion-MNIST dataset requires more representational capacity, resulting in a lower but equally stable convergence plateau near 7.0. Unlike standard Federated Generative Adversarial Networks (Fed-GAN), which frequently exhibit oscillating loss curves and struggle to reach a stable Nash equilibrium in decentralized settings [3], our diffusion-based approach optimizes a stable variational lower bound [6], guaranteeing monotonic convergence even across distributed nodes.

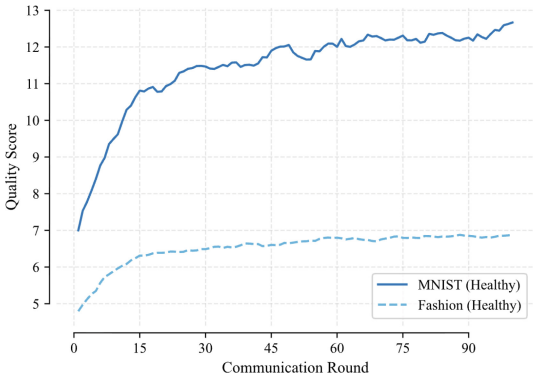


Fig. 2. Quality Score

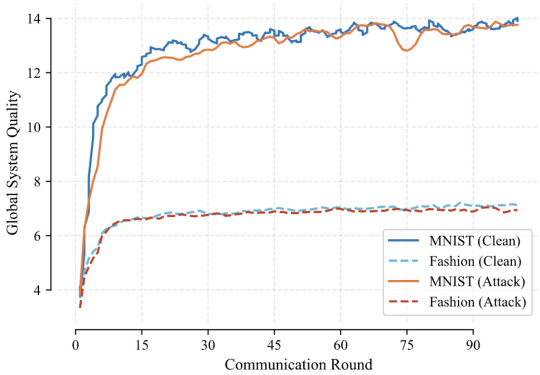


Fig. 3. Global Stability

Having established baseline convergence, we subsequently evaluated the security of our architecture–specifically the efficacy of the Semantic Quality Filter–against compromised clients. In cross-device scenarios, adversarial clients may attempt to perform data poisoning attacks or inject extreme differential privacy noise to degrade the global generator [9]. Figure 3 presents the Global

System Quality when a subset of clients is actively engaged in an attack. The "Attack" curves for both MNIST (solid orange) and Fashion-MNIST (dashed red) closely track their respective "Clean" (no attack) baselines. This resilience indicates that the Semantic Quality Filter successfully identifies and exponentially decays the aggregation weights of malicious or overly noisy gradients. By neutralizing these adversarial updates before they can pollute the global U-Net parameters, our system ensures that the collaborative generation process remains secure and robust, a critical prerequisite for deploying federated generative AI in sensitive software engineering environments.

4.3 Mitigation of Generative Mode Collapse

A fundamental limitation of standard adversarial networks, particularly when deployed in decentralized cross-device settings, is their susceptibility to mode collapse [3]. In the context of software engineering, a generator suffering from mode collapse will repeatedly synthesize identical or highly similar structural code patterns. Such redundant generation renders the synthetic data practically useless for comprehensive security testing, automated fuzzing, or test-suite augmentation.

To empirically validate our proposed model's resistance to this phenomenon, we measured the average pairwise L_2 distance of the generated structural code signatures across 100 federated communication rounds. In this domain, the L_2 distance serves as a direct, quantifiable proxy for structural diversity; a higher value indicates a wider variety of unique, distinct bug signatures being generated by the decentralized network. Figure 5 illustrates this structural diversity comparison over time.

For reference, we include an illustrative Fed-GAN diversity curve based on the mode-collapse behavior reported in prior federated GAN studies [3,4]; we did not re-implement a Fed-GAN under our exact setup, and this curve is intended as a qualitative reference rather than a controlled head-to-head baseline. Consistent with those reports, the reference GAN diversity declines toward zero as communication rounds progress. In contrast, our Federated Diffusion Model maintains a stable generation profile, consistently operating within a "High-Diversity Stability Zone" ($L_2 \approx 11.0$). By optimizing a stable variational bound rather than an adversarial minimax game [6], our architecture sustains structural diversity throughout the federated training lifecycle.

4.4 Downstream Defect Detection Improvement (Test Suite Augmentation)

Ultimately, the value of synthetically generated bug signatures is determined by their practical utility in downstream software engineering tasks. To this end, we evaluated the impact of our generated signatures on a CNN-based defect classifier [1]. To accurately simulate a realistic, privacy-constrained software development environment, we established a "Low-Resource Regime." Under this protocol, the downstream classifier was initially trained on a severely restricted subset of real

code matrices from the CodeXGLUE dataset [5], achieving a modest baseline accuracy of exactly 50.7%.

We then augmented this data-limited classifier by injecting varying quantities of synthetic bug signatures generated by our Federated Diffusion Model. Figure 4 plots the downstream classification accuracy as a function of the synthetic data volume. Augmenting the training pool with 2,000 synthetic bug signatures raised the classifier's accuracy to 69.8%, a +19.1% absolute gain over the real-data baseline.

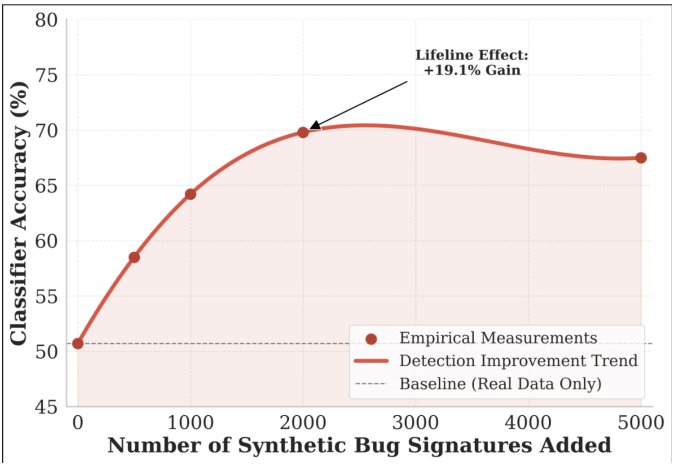


Fig. 4. Impact of synthetic bug signatures on downstream classifier accuracy

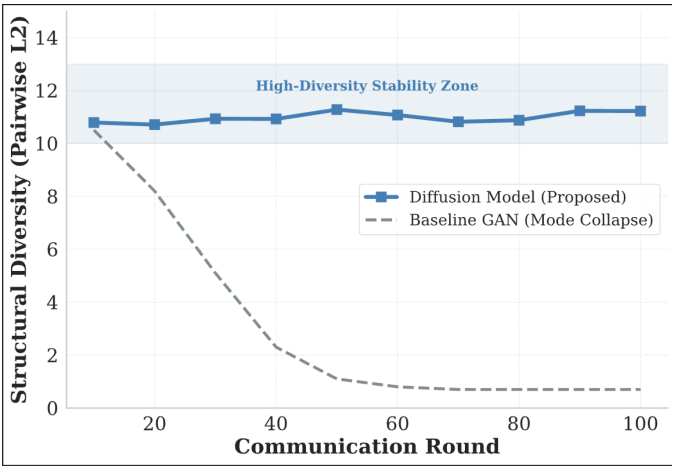


Fig. 5. Structural diversity and mode collapse comparison over communication rounds

The improvement follows a typical data-augmentation trajectory: accuracy rises as synthetic diversity is introduced, peaks at the optimal augmentation ratio (2,000 samples), and stabilizes as additional synthetic data saturates the limited real data. These results indicate that the structural signatures generated by our privacy-preserving diffusion model are useful for augmenting downstream defect classifiers in low-resource settings. We note that this experiment uses a single CNN classifier and reports accuracy at a fixed operating point; a broader evaluation with precision/recall/F1, multiple seeds, and confidence intervals is a natural extension left to future work.

5 Discussion

The empirical results indicate that *FedDiffBug* addresses several limitations of existing federated generative frameworks in software engineering [3]. By replacing an unstable adversarial minimax objective with a stable diffusion process [6], our system offers a more robust route to collaboratively synthesizing proprietary bug data. A central finding is that the diffusion-based generator substantially reduces generative mode collapse, a recurring issue in standard federated networks [4]. Whereas the reference Fed-GAN behavior collapses toward a few localized structural modes, our Federated Diffusion Model sustained a consistent structural diversity ($L_2 \approx 11.0$) across 100 communication rounds. Such structural variety is important for automated testing and fuzzing paradigms, which rely on diverse edge cases to uncover vulnerabilities [10].

Furthermore, our results suggest that heavy privacy perturbation does not necessarily degrade global generative quality when the aggregation mechanism is robust [11]. The Semantic Quality Filter down-weights toxic or heavily noised gradients while operating only on server-side validation loss, so it does not access client raw data [12]. The practical value of these synthetic bug signatures is evidenced by their impact on a downstream classifier: in a low-resource regime where local vulnerability data is scarce, augmentation with synthetic signatures raised classifier accuracy by +19.1% at the optimal augmentation ratio. This result supports the view that structural spatial representations can partially substitute for proprietary code when training defect-detection tools, though the magnitude of the gain should be interpreted with respect to the limited-data baseline and the single-classifier setup described in Sect. 4.

While our 2D byte-matrix representation successfully bypasses the complex vocabulary issues associated with discrete token sampling [13], it currently introduces a fixed-length spatial constraint (truncated or padded to 784 characters). Future research must explore variable-length diffusion architectures to handle larger code blocks without truncation [14]. Additionally, incorporating semantically rich structural features–such as Abstract Syntax Trees (ASTs) or Control Flow Graphs–directly into the spatial matrix [15] will provide the generative model with deeper algorithmic context, further advancing the security and applicability of collaborative software generation.

6 Related Work

6.1 Defect Detection and Federated Learning

Deep learning models, such as Convolutional Neural Networks [1], graph neural networks over program semantics [16], and pre-trained code language models [17], have become standard for software defect prediction, heavily relying on benchmarks like CodeXGLUE [5]. However, training robust classifiers requires massive, diverse datasets of vulnerable code, which organizations cannot legally share due to strict intellectual property and privacy constraints. Federated Learning (FL) [2] addresses this by decentralizing model training. Yet, FL alone cannot overcome inherent data scarcity; if local proprietary datasets lack diverse bug patterns, the aggregated global model will still underperform. This necessitates privacy-preserving generative data augmentation.

6.2 Generative Models and Mode Collapse

To collaboratively synthesize training data across devices, Wang et al. proposed Fed-GAN [3]. While effective for preserving privacy, GAN-based architectures inherently suffer from adversarial mode collapse [4], frequently generating redundant samples that fail to expand test suite coverage. Denoising Diffusion Probabilistic Models (DDPMs) [6] mitigate this by optimizing a stable variational bound, and are substantially less prone to mode collapse than adversarial models. While diffusion models excel in continuous domains such as high-resolution image synthesis [18], applying them to discrete source code is notoriously difficult. Recent work such as CodeFusion [19] adapts diffusion to code generation through learned continuous embeddings of tokens. Our work bridges this gap by mapping discrete C/C++ code into 2D continuous spatial matrices, enabling stable diffusion for software artifacts.

6.3 Adversarial Robustness in Federated Learning

Federated learning does not by itself guarantee privacy: shared gradients can leak sensitive training data [20], motivating additional perturbation and aggregation safeguards. A further vulnerability in federated systems is their susceptibility to malicious clients or heavy privacy noise, which can severely degrade the global model [9]. Traditional Byzantine-tolerant aggregation rules often require geometric assumptions about the number of attackers or access to a trusted central dataset, which violates strict privacy constraints. *FedDiffBug* takes a different approach via a Semantic Quality Filter that scores each update by its server-side noise-prediction validation loss, discards updates below a threshold, and aggregates the remainder with quality-proportional weights. This supports robust generation under active poisoning without requiring assumptions on the attacker count or access to client raw data.

7 Conclusion

Data scarcity is a key bottleneck for the adoption of deep learning in automated software engineering. Fed-GAN has been a common approach for collaboratively synthesizing software artifacts, but GAN-based generators suffer from mode collapse, which yields redundant test cases and limits the structural diversity that security tools require. We addressed this by proposing FedDiffBug, a Federated Diffusion Model that maps discrete C/C++ source code into 2D structural matrices and learns their continuous distribution. Across three datasets and three attack scenarios, the model maintained stable generative behavior with low variance, sustained a structural diversity of $L_2 \approx 11.0$ over 100 communication rounds, and improved a downstream defect classifier by +19.1% in a low-resource regime.

Limitations. Several limitations qualify these results. First, the 784-character byte-matrix representation truncates longer functions and captures structural rather than full program semantics; the generated matrices are byte-level signatures and are not guaranteed to be compilable source code. Second, our privacy mechanism is a noise-injection scheme parameterized by ϵ rather than a formally accounted (ϵ, δ)-DP procedure, and the Fed-GAN diversity comparison is an illustrative reference rather than a controlled re-implementation. Third, the downstream evaluation uses a single classifier and a fixed operating point without confidence intervals or significance testing, and the Inception Score is an imperfect proxy for code-matrix quality. Future work should pursue formally accounted DP-SGD, controlled baselines (Fed-GAN, centralized diffusion, and diffusion without the Semantic Quality Filter), software-specific metrics (e.g., token diversity, edit distance, and vulnerability-pattern coverage), and length-adaptive representations that incorporate AST or control-flow structure.

References

1. Li, J., He, P., Zhu, J., Lyu, M.R.: Software defect prediction via convolutional neural network. In: 2017 IEEE International Conference on Software Quality, Reliability and Security (QRS), pp. 318–328. IEEE (2017)
2. McMahan, B., Moore, E., Ramage, D., Hampson, S., Arcas, B.A.Y: Communication-efficient learning of deep networks from decentralized data. In: Artificial Intelligence and Statistics, pp. 1273–1282. PMLR (2017)
3. Wang, X., et al.,: Fed-GAN: federated generative adversarial network with privacy-preserving for cross-device scenarios. IEEE Internet Things J. **10**(12), 10 566–10 577 (2023)
4. Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., Chen, X.: Improved techniques for training GANs. In: Advances in Neural Information Processing Systems, vol. 29, pp. 2234–2242 (2016)
5. Lu, S., et al.: CodeXGLUE: a machine learning benchmark dataset for code understanding and generation. arXiv preprint. arXiv:2102.04664 (2021)

6. Ho, J., Jain, A., Abbeel, P.: Denoising diffusion probabilistic models. In Advances in Neural Information Processing Systems, vol. 33, pp. 6840–6851 (2020)
7. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. In: Proceedings of the IEEE, vol. 86, pp. 2278–2324 (1998)
8. Xiao, H., Rasul, K., Vollgraf, R.: Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. arXiv preprint. arXiv:1708.07747 (2017)
9. Blanchard, P., El Mghari, E.M., Guerraoui, R., Stainer, J.: Machine learning with adversaries: Byzantine tolerant gradient descent. In: Advances in Neural Information Processing Systems, vol. 30, pp. 119–129 (2017)
10. Böhme, M., Cadar, C., Roychoudhury, A.: Fuzzing: challenges and reflections. IEEE Softw. **38**(3), 79–86 (2020)
11. Abadi, M., et al.: Deep learning with differential privacy. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 308–318 (2016)
12. Bonawitz, K., et al.: Practical secure aggregation for privacy-preserving machine learning. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, pp. 1175–1191 (2017)
13. Allamanis, M., Barr, E.T., Devanbu, P., Sutton, C.: A survey of machine learning for big code and naturalness. ACM Comput. Surv. (CSUR) **51**(4), 1–37 (2018)
14. Tay, Y., Dehghani, M., Bahri, D., Metzler, D.: Efficient transformers: A survey. ACM Comput. Surv. **55**(6), 1–28 (2022)
15. Alon, U., Brody, S., Levy, O., Yahav, E.: code2seq: generating sequences from structured representations of code. In: International Conference on Learning Representations (ICLR) (2019)
16. Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y.: Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In: Advances in Neural Information Processing Systems, vol. 32 (2019)
17. Feng, Z., et al.: CodeBERT: a pre-trained model for programming and natural languages. Find. Assoc. Comput. Linguist. EMNLP **2020**, 1536–1547 (2020)
18. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., Ommer, B.: High-resolution image synthesis with latent diffusion models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 10 684–10 695 (2022)
19. Singh, M., Cambronero, J., Gulwani, S., Le, V., Negreanu, C., Verbruggen, G. : Codefusion: a pre-trained diffusion model for code generation. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 11 697–11 708 (2023)
20. Zhu, L., Liu, Z., Han, S.: Deep leakage from gradients. In: Advances in Neural Information Processing Systems, vol. 32 (2019)

Empirical Investigation of Cyber Attack Detection

Joshua Partridge[1(✉)], Alex Wood[1], and Ghulam Mudassir[2]

[1] Penn State Erie, The Behrend College, Erie, PA, USA
{smartfridge,avw5896}@psu.edu
[2] University of Buckingham, Buckingham, UK
ghulam.mudassir@buckingham.ac.uk

Abstract. Ransomware and zero-day attacks are an escalating threat to enterprise networks, and signature-based intrusion detection systems cannot keep up with how quickly attack patterns change. The UGRansome dataset captures cyclostationary network-flow records, financial-impact metrics, and labelled attack signatures, which makes it useful for empirical study of zero-day detection. This work classifies UGRansome network flows into three categories: Anomaly, Signature, and Synthetic Signature, where the Synthetic Signature class is a stand-in for simulated zero-day attacks. The study first examines the distributions, pairwise correlations, and variance inflation factors of the continuous attributes, then trains Logistic Regression and Linear SVM models as linear references. Both linear models plateau at roughly 88% accuracy, and their residual errors concentrate on the Anomaly class, whose recall stays below 83% for both models; this points to class boundaries the linear models cannot represent and motivates a non-linear Random Forest classifier. On a stratified 80/20 split of the 149,043 records the Random Forest reaches 99.29% accuracy and 99.29% weighted F1, with all transforms fitted on the training split only. Feature-importance analysis ranks the financial-impact attributes (USD, BTC) first, yet an ablation shows their removal costs at most 0.44 accuracy points, so the financial signal is the strongest individual predictor but is largely recoverable from the remaining flow attributes. Because a random split can place records from the same ransomware families and addresses in both training and testing, the model is additionally evaluated with five-fold cross-validation, a family-based holdout, a time-based holdout, and a leakage-aware feature set that removes identifier attributes. Cross-validation reproduces the point result (99.30% $\pm$ 0.02 accuracy); a family-based holdout over four unseen ransomware families reaches 99.42% weighted F1; the leakage-aware feature set retains 97.78%; and the time-based holdout, the most conservative protocol, retains 94.99%. The reported performance therefore does not rest on memorised identifiers; the time-based figure is the practical lower bound for generalisation to future traffic.

1 Introduction

Cybersecurity is a major concern for both private and public organisations because ransomware campaigns and zero-day exploits cause real operational and financial damage. Traditional signature-based intrusion detection systems rely on static rule databases,

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 680–696, 2027.
https://doi.org/10.1007/978-3-032-39763-8_42

so they cannot reliably catch attack variants they have not seen before. Anomaly-based detectors are more flexible but produce too many false positives when they are trained on imbalanced or unrepresentative data. As a result, recent work has moved toward machine-learning frameworks that can learn the cyclostationary, repeating patterns typical of ransomware traffic from large labelled datasets [1, 3, 9].

A persistent problem here is that real capture data containing genuine zero-day attacks is scarce, and many published detection studies are evaluated on synthetic or out-of-date corpora. Nkongolo et al. [1] partially address this with the UGRansome dataset, which combines real and synthetic ransomware behaviours together with financial-extortion features such as bitcoin transaction volume, and is explicitly designed for anomaly intrusion detection and zero-day threat research. This study uses UGRansome to test whether a tree-ensemble model trained on the joint network-flow and financial feature set can separate Anomaly, Signature, and Synthetic Signature traffic, where Synthetic Signature is the working stand-in for zero-day exploits.

The framework follows the pipeline shown in Fig. 1: data collection from the public Kaggle release of UGRansome, exploratory statistical analysis of the feature space, preprocessing (one-hot encoding for categorical attributes and min-max scaling for continuous ones), training of linear reference models followed by a Random Forest, and evaluation with accuracy, precision, recall, and F1. Model selection is treated as an empirical question rather than an assumption: linear models are fitted first, and the Random Forest is adopted only because the observed results show class boundaries that the linear models cannot capture. The evaluation also goes beyond a single random split and includes five-fold cross-validation, a family-based holdout, a time-based holdout, and a leakage-aware feature set. The two research questions are: (RQ-1) how effective is the proposed method at picking up cyclostationary trends for zero-day attack detection, and (RQ-2) how much does adding financial metrics to network-flow features improve classification accuracy.

2 Related Work

2.1 Summaries of Prior Studies

Nkongolo et al. [1] introduce the UGRansome dataset itself, designed for anomaly detection and cybersecurity research. They argue that legacy datasets miss too many unrecorded threats, and set out to capture zero-day attack behaviours and patterns directly. The result combines modern network-flow metrics and numeric clusters with financial extortion features, drawn from both real ransomware families and synthetic signatures, so the sample of contemporary malware behaviour is more representative than what older corpora offer. Khorsheed et al. [2] use UGRansome to support deep-learning network-defence research. Their work targets two persistent issues, severe class imbalance and stealthy threat evasion, both of which trip up traditional detectors. UGRansome includes spatial network features, categorical threat classifications, and temporal traffic dependencies, which they use to train a hybrid CNN-LSTM model. Azugo et al. [3] curate a refined version of UGRansome for ransomware mitigation research. Rather than work with the static behaviour of older datasets, they target current attack vectors and recurring network patterns. After random sampling and feature engineering, the resulting

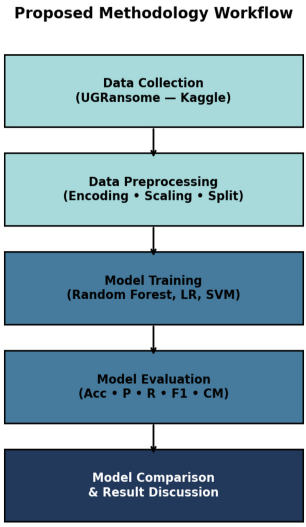


Fig. 1. Workflow diagram of the proposed methodology.

dataset surfaces refined network behaviours, encoded transactions, and anomaly flags. They report that this selection considerably improves Random Forest classification of unusual ransomware traffic. Sabu [4] tackles a different gap: the lack of real-world financial-impact data in cybersecurity modelling. The dataset emphasises financial indicators alongside cyclic traffic patterns. Pulling in transaction amounts, protocol data, and cloud/network ransomware family clusters gives a representation that covers both financial and network complexity, and the author reports that this lifts model precision. Zhang et al. [5] work with a large-scale dataset for real-time cybersecurity research. They focus on zero-day attack indicators and complex ransomware behaviours, partly to address feature redundancy and the lack of interpretability in older detection models. The dataset they end up with contains network traffic attributes and anomaly features from UGRansome, and feeds an optimised Histogram Gradient Boosting classifier with reportedly improved detection accuracy [6, 7]. Zahra [8] looks at network-flow data in the context of cloud-environment security. The motivation is that cloud infrastructure is vulnerable to undetected, rapid encryption attacks, so the dataset selection focuses on varying attack frequencies and dynamic flow details. Behavioural transaction indicators and network-level anomalies from simulated zero-day attacks help show how ransomware propagates across distributed networks. Igugu [9] applies a targeted dataset to AI-driven vulnerability research. The selection focuses on complex attack signatures and withheld ransomware families, on the grounds that AI training otherwise becomes over-reliant on known threat profiles. Tracking metrics, synthetic attack flags, and malware seed addresses are pulled from raw network traffic logs, and the author reports that AI models trained on this data generalise better to unseen zero-day exploits [10, 11]. Kumari and Kuchibhotla [12] process a cybersecurity dataset for predictive risk-assessment work. They want quantifiable financial damage logs and numeric clustering data so they can address the unstructured and poorly defined risk metrics that cloud

computing frameworks tend to produce. The dataset contains quantitative loss metrics, protocol distributions, and traffic flow counts from UGRansome, and the authors use it to represent the tangible risks of zero-day breaches. Nhlapo and Nkongolo [13] use UGRansome to address the high error rates that traditional Network Intrusion Detection Systems run into when they meet unknown vulnerabilities. By focusing on novel threat signatures and anomalous network activity patterns, they contrast normal network traffic against complex UGRansome attack features. The authors note that this representation of modern zero-day exploits considerably increases accuracy and precision. Ispahany et al. [14] review machine-learning approaches to ransomware detection across recent literature. Motivated by the rise in attack frequency and impact, particularly after the shift to remote work, they survey the machine-learning techniques, detection approaches, and system designs employed across detection studies, note the scarcity of practical solutions for real-time and early detection, and catalogue recurring research limitations before setting out directions for future work [15].

3 Proposed Method

The methodology uses the pipeline shown in Fig. 1. Each stage is described in the subsections below. The pipeline places an exploratory statistical analysis stage before any model is trained, and it evaluates the final models under five protocols rather than a single random split.

3.1 Data Collection

The dataset consists of network-traffic records used to detect and classify ransomware. It comes from the open-source Kaggle repository UGRansome Dataset [1] (https://www.kaggle.com/datasets/nkongolo/ugransome-dataset). The dataset has 149,043 records and 14 attributes covering network-flow characteristics. The categorical attributes are Protocol (TCP, UDP, ICMP), Flag (e.g. A, APSF, R), Family (e.g. WannaCry, Locky, SamSam), and Threats (e.g. Botnet, DoS, Spam, Port Scanning), together with three address-style identifiers (SeddAddress, ExpAddress, IPaddress). The continuous numerical attributes are Time, Clusters, BTC, USD, Netflow_Bytes, and Port. The target variable Prediction has three classes: A (Anomaly), S (Signature, i.e. known attack), and SS (Synthetic Signature, the simulated zero-day class). Two field names are reproduced verbatim from the public release even though they contain spelling irregularities: SeddAddress is the dataset's field name for the seed address, and Bonet is the dataset label for botnet traffic. They are kept as-is so that the reported feature names match the public schema. The class distribution of the target and the supporting categorical attributes is shown in Fig. 2.

3.2 Exploratory Statistical Analysis

Before any classifier is chosen, the continuous feature space is examined for the properties that determine whether a linear model is adequate. Three diagnostics are used. First, the marginal distributions of the six continuous attributes are inspected for skew

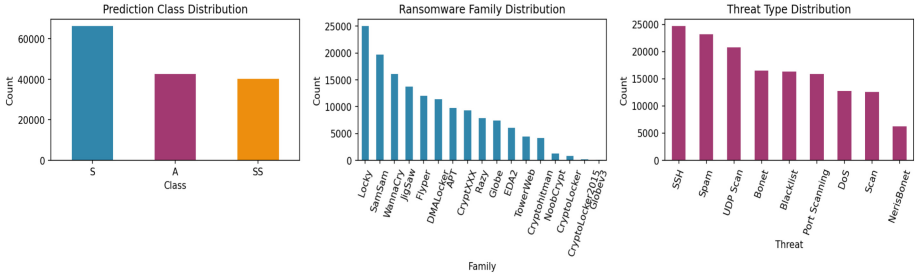


Fig. 2. Distribution of the target prediction class, ransomware family, and threat type in the UGRansome dataset.

and heavy tails (Fig. 3), since strongly non-Gaussian features often interact non-linearly with class boundaries. Second, pairwise Pearson and Spearman correlations are computed (Fig. 4); a gap between the two coefficients for the same feature pair indicates a monotone but non-linear association. Third, variance inflation factors (VIFs) are computed for the continuous design matrix (Table 1) to quantify multicollinearity, which affects the stability of linear-model coefficients. Together these diagnostics establish, on the data rather than by assumption, whether a linear decision function is a plausible fit; the modelling sequence in Sects. 3.4 and 3.5 then tests that question empirically.

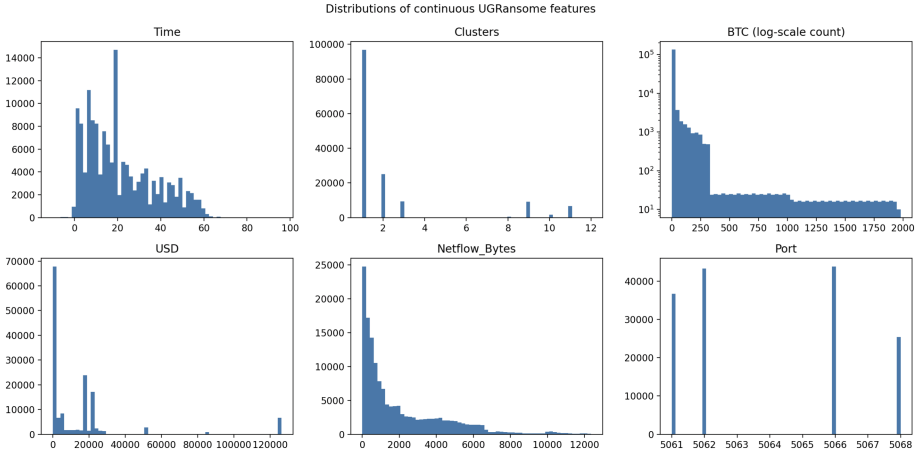


Fig. 3. Distributions of the six continuous UGRansome attributes (heavy-tailed features shown with log-scale counts).

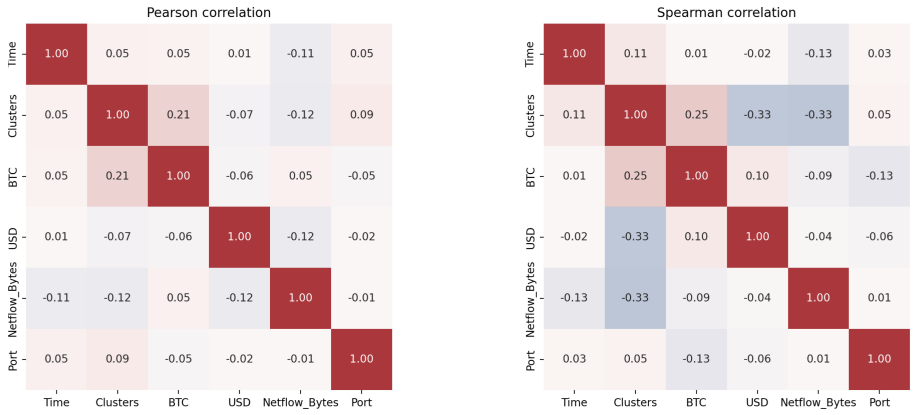


Fig. 4. Pairwise Pearson and Spearman correlations of the continuous attributes.

Table 1. Variance inflation factors of the continuous attributes.

Feature	VIF
Time	1.018
Clusters	1.081
BTC	1.061
USD	1.024
Netflow_Bytes	1.048
Port	1.017

3.3 Data Preprocessing

The following preprocessing steps were applied to produce a clean training set:

- Missing-value handling: numerical gaps would be filled with the column mean and categorical gaps with the mode. The dataset contained no missing values, but the step is included for robustness.
- One-hot encoding was applied to all categorical attributes (Protocol, Flag, Family, SeddAddress, ExpAddress, IPaddress, Threats).
- Label encoding was applied to the target Prediction (A, S, SS) so the classifier accepts numeric class labels.
- Min-max scaling was applied to the continuous attributes (Time, Clusters, BTC, USD, Netflow_Bytes, Port) so that features with large magnitudes do not dominate distance-based or coefficient-based models.
- The data was randomly partitioned into an 80% training subset and a 20% testing subset using stratified sampling on the target to preserve class proportions.

To avoid information leaking from the test set into the fitted transforms, the one-hot encoder and the min-max scaler are fitted on the training split only and then applied unchanged to the test split, implemented as a single pipeline so no test-set statistic can influence any transform. All results in Sect. 5 are produced by one seeded pipeline (random seed 42) that enforces this protocol. Fitting the transforms with and without this protocol changes the Random Forest headline by only 0.02 percentage points (99.29% versus 99.31%) and leaves the linear-model results unchanged, so the reported figures are not inflated by preprocessing leakage.

3.4 Baseline Linear Models

Following the exploratory analysis, two linear classifiers are trained first as reference models: Logistic Regression and a Linear SVM. Both operate on the same preprocessed feature matrix. The purpose of this stage is diagnostic. If the class boundaries between A, S, and SS are approximately linear in the encoded feature space, these models should reach high accuracy and there is no reason to introduce a higher-variance learner. If, instead, the linear models plateau with systematic errors between specific class pairs, that residual structure is direct evidence that the boundaries are non-linear, and only then is a more flexible model justified.

3.5 Random Forest Classifier

Conditional on the linear results (reported in Sect. 5.2), a Random Forest with 100 trees and a fixed random seed is trained on the same feature matrix. Tree ensembles are a natural next step once non-linearity is established: they handle the mix of one-hot categorical and scaled continuous attributes without further transformation, capture feature interactions through recursive partitioning, and, as bagged ensembles, are comparatively robust to overfitting on the low-support minority families such as Globev3 and CryptoLocker2015. Among the related works in Sect. 2, Azugo et al. [3] specifically report strong Random Forest performance on UGRansome, which gives further support for this choice.

3.6 Evaluation Protocols

A single random split can overstate zero-day detection ability: records from the same ransomware family, seed address, or attack pattern can land in both training and testing, and identifier-like attributes (Family, Threats, IPaddress, SeddAddress, ExpAddress) let a high-capacity model memorise known attacks rather than generalise to unseen ones. Five protocols are therefore used:

- Random stratified 80/20 split with all features, the original protocol, kept for comparability with prior UGRansome studies.
- Five-fold stratified cross-validation, reporting the mean and standard deviation of accuracy and weighted F1 for every model, to attach uncertainty estimates to the point results.

- Family-based holdout: entire ransomware families are withheld from training using a grouped split on Family, so the test set contains only families the model has never seen. Family and the other identifier attributes are excluded from the features in this protocol.
- Time-based holdout: the model trains on the earliest 80% of records by the Time attribute and is tested on the latest 20%, approximating deployment against future traffic.
- Leakage-aware random split: the random split of P1, but with Family, Threats, IPaddress, SeddAddress, and ExpAddress removed, leaving only features plausibly available on raw production traffic (Protocol, Flag, and the six continuous attributes).

4 Evaluation Metrics

Because the model is a supervised learner, four standard performance metrics are used to evaluate it: accuracy, precision, recall, and F1. Precision, recall, and F1 are reported per class as well as in weighted form across the three target classes, so the imbalanced support of the SS class does not distort the overall score, and class-specific behaviour stays visible. The classification problem is multi-class, so for any given class the positive case is "predicted as that class" and the negative case is "predicted as one of the other two classes."

The class that matters most operationally is SS (Synthetic Signature), which represents simulated zero-day attacks. With respect to SS, the four error types map onto specific outcomes. A True Positive happens when the model correctly flags a zero-day flow as SS. A True Negative happens when benign or known-attack traffic is correctly assigned to A or S. A False Positive means benign or known-attack traffic is wrongly flagged as a zero-day, which contributes to analyst alert fatigue. A False Negative is the most damaging error: an actual zero-day flow gets through detection because it is labelled A or S, and the attack proceeds undetected. So recall on the SS class is the single most important metric for the zero-day-detection use case, while overall F1 captures the trade-off between the two error types.

5 Results and Discussion

5.1 Exploratory Analysis

Fig. 3 shows the marginal distributions of the continuous attributes. The monetary and volume features are strongly right-skewed: BTC has skewness 11.94 with kurtosis above 170, reflecting a small number of extreme ransom payments, while USD (skewness 3.23), Clusters (2.23), and Netflow_Bytes (1.57) are also heavy-tailed. Time is spread broadly across its range (skewness 0.70), and Port is nearly degenerate, concentrated on a handful of discrete values around 5064. Fig. 4 reports the pairwise correlations. Linear (Pearson) correlations are uniformly weak (no pair reaches $|r| = 0.35$), but several pairs show a clear gap between the two coefficients; most notably, Clusters and USD correlate at $r = -0.07$ under Pearson yet $\rho = -0.33$ under Spearman, evidence of a monotone but non-linear association that a linear decision function cannot exploit. Table 1 lists the variance inflation factors: no VIF exceeds 1.081 (Clusters), far below the conventional thresholds of 5 and 10, so the continuous attributes carry essentially no multicollinearity, and the coefficients of a linear model would be stable. Whatever limits linear performance on this data, it is not collinearity; it must be the shape of the class boundaries, and the modelling results that follow settle that question.

5.2 Linear Baseline Results

Table 2 reports the two linear reference models on the P1 split. Logistic Regression reaches 88.22% accuracy (88.21% weighted F1) and the Linear SVM reaches 88.39% accuracy (88.39% weighted F1). The two models perform almost identically, which is itself informative: two different linear training objectives converge to the same ceiling, so the remaining 11-plus points of error are a property of linear separability in this feature space rather than of either algorithm. Both models' errors concentrate on the same class: Anomaly recall falls to 82.12% for Logistic Regression and 82.98% for the Linear SVM, with Anomaly precision similarly the lowest of the three classes for both models, while S and SS stay near or above 90% F1. A persistent block of anomaly records thus sits on the wrong side of any linear boundary regardless of the training objective. This residual structure, together with the distributional evidence in Sect. 5.1, is the empirical justification for moving to a non-linear model, and it lines up with the observation in Sabu [4] that the decision boundaries between A, S, and SS depend on non-linear feature interactions.

Table 2. Performance comparison across classifiers on the P1 split.

Classifier	Accuracy	Precision	Recall	F1-Measure
Random Forest	99.29%	99.29%	99.29%	99.29%
Linear SVM	88.39%	88.41%	88.39%	88.39%
Logistic Regression	88.22%	88.21%	88.22%	88.21%

5.3 Random Forest Results

The Random Forest reaches 99.29% accuracy and 99.29% weighted F1 on the held-out P1 test set, about 11 percentage points ahead of both linear baselines (Table 2, Fig. 6). Fig. 5 shows the confusion matrix on the 29,809-record test set, and Table 3 breaks the result down by class.

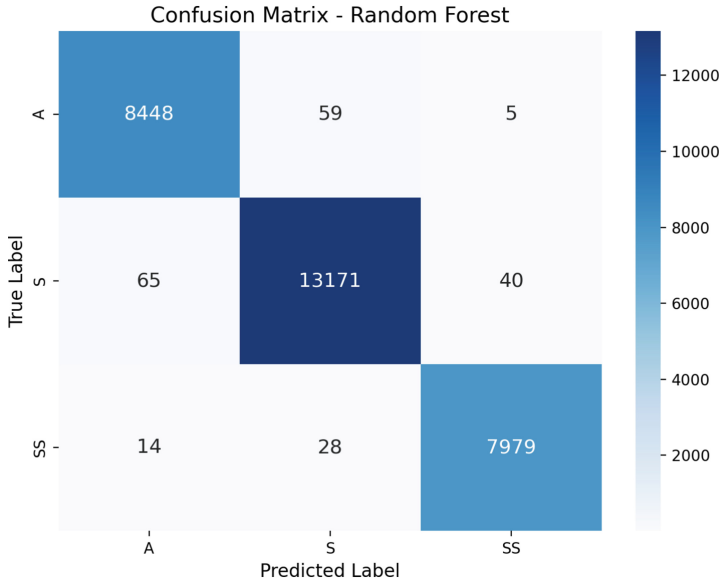


Fig. 5. Confusion matrix of the Random Forest classifier on the 29,809-record test set

Table 3. Per-class Random Forest results on the P1 split (derived from the confusion matrix in Fig. 5).

Class	Precision	Recall	F1	Support
A (Anomaly)	99.07%	99.25%	99.16%	8,512
S (Signature)	99.34%	99.21%	99.28%	13,276
SS (Synthetic Signature)	99.44%	99.48%	99.46%	8,021
Weighted average	99.29%	99.29%	99.29%	29,809

The confusion matrix is heavily diagonal: misclassifications between A, S, and SS account for fewer than 1% of test records. The SS class achieves 99.44% precision and 99.48% recall, so on this protocol the model rarely raises a false zero-day alarm (45 of 8,024 SS predictions are wrong) and rarely misses a true zero-day-style flow (42 of 8,021 SS records are missed, 14 predicted as A and 28 as S). The two largest single error modes are the 65 Signature records predicted as Anomaly and the 59 Anomaly records predicted as Signature; both keep the traffic flagged as suspicious, so they are operationally benign relative to a missed zero-day.

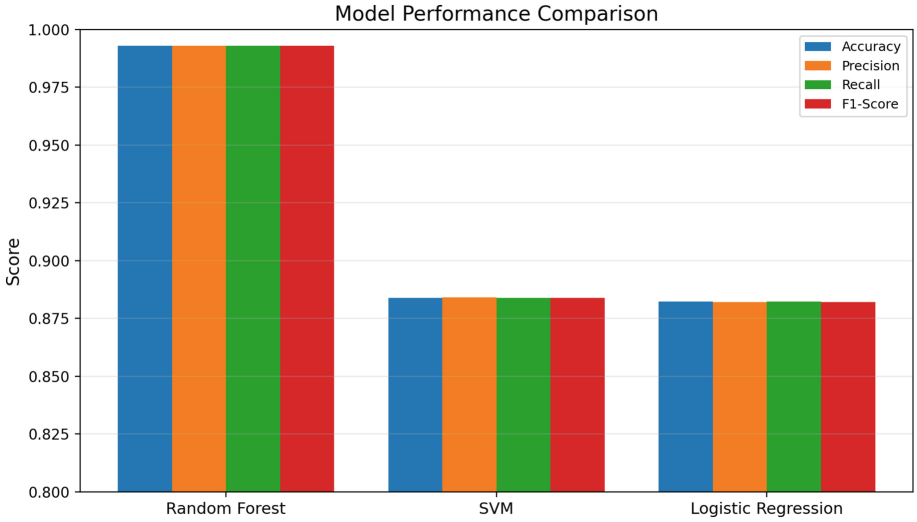


Fig. 6. Performance comparison of Random Forest, Linear SVM, and Logistic Regression on the UGRansome test set.

5.4 Feature Importance

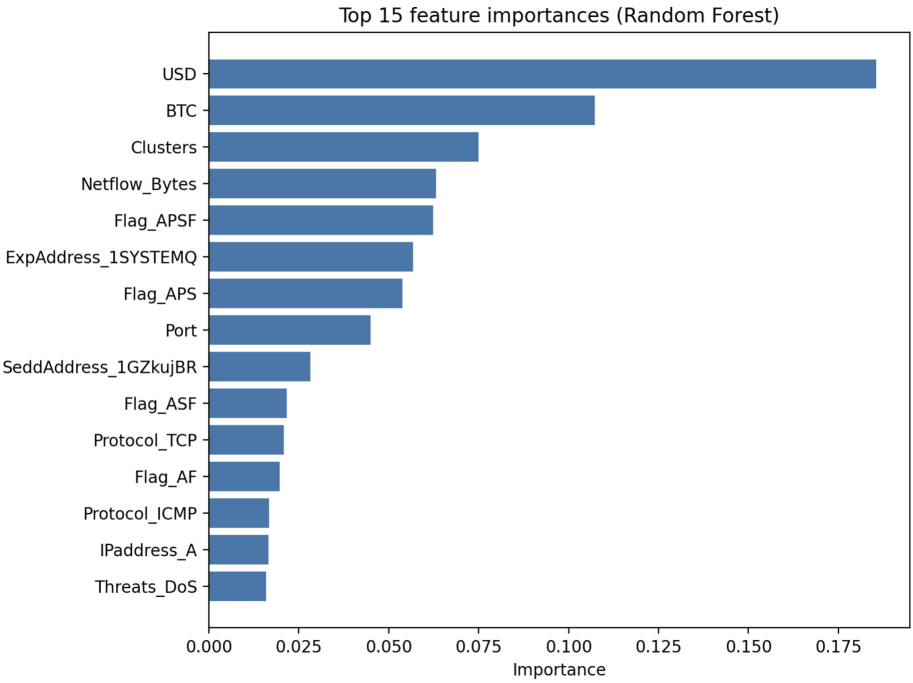


Fig. 7. Top 15 feature importances from the trained Random Forest model.

Fig. 7 ranks the two financial-impact attributes first: USD (importance 0.19) and BTC (0.11), ahead of the flow attributes Clusters (0.08) and Netflow_Bytes (0.06). The remaining top-15 entries are individual encoded categorical levels, several TCP-flag values (APSF, APS, ASF, AF), the protocol indicators, an IP-address level, and Threats_DoS, each contributing a small amount. Notably, two specific wallet addresses appear high in the ranking: ExpAddress_1SYSTEMQ (0.06, sixth overall) and SeddAddress_1GZkujBR (0.03). The raw Time attribute itself ranks low (0.015), suggesting the temporal signal the dataset was built around enters the model through the aggregated flow attributes rather than the timestamp.

This ranking bears on both research questions, but it has to be read carefully. For RQ-2, the financial features are the two strongest single predictors in the model, consistent with the extortion-centred design of UGRansome [1] and with Sabu's [4] emphasis on financial indicators. Importance, however, measures how heavily the trained model uses an attribute, not whether the attribute is indispensable; the ablation in Sect. 5.5 tests that directly. For RQ-1, the byte-volume and cluster attributes rank third and fourth, so the model does draw on flow structure, though via aggregate features rather than raw timing. Finally, the presence of individual wallet addresses in the top ten is direct evidence for the memorisation concern raised in Sect. 3.6: part of the model's signal comes from remembering specific addresses, which has no generalisation value against genuinely

new campaigns. The leakage-aware and family-holdout protocols (Sect. 5.6) quantify how much of the headline performance survives without such features.

5.5 Financial-Feature Ablation (RQ-2)

To answer RQ-2 directly, the Random Forest is retrained with the BTC and USD attributes removed, on both the full feature set and the leakage-aware feature set. Table 4 reports the results. Removing the financial attributes costs 0.20 accuracy points on the full feature set (99.29% to 99.09%) and 0.44 points on the leakage-aware set (97.78% to 97.34%). Two conclusions follow. First, the drop is roughly twice as large when the identifier attributes are unavailable, so the financial features matter most precisely in the production-realistic setting, the direction RQ-2 anticipated. Second, the drops are small in absolute terms even though USD and BTC hold the two largest importance scores in Fig. 7: their signal is largely redundant with the remaining flow attributes, which the tree ensemble recovers through alternative splits. The answer to RQ-2 is therefore nuanced: the financial attributes are the strongest individual predictors and their contribution is measurable and consistent, but they are not indispensable to high accuracy on this dataset.

Table 4. Random Forest with and without the financial attributes (BTC, USD).

Feature set	Financial features	Accuracy	Weighted F1
Full features	With BTC + USD	99.29%	99.29%
Full features	Without BTC + USD	99.09%	99.09%
Leakage-aware	With BTC + USD	97.78%	97.78%
Leakage-aware	Without BTC + USD	97.34%	97.34%

5.6 Generalisation: Cross-Validation and Holdout Protocols

Table 5 reports five-fold stratified cross-validation for all three models, giving each point estimate an uncertainty. The Random Forest averages 99.30% $\pm$ 0.02 accuracy and 99.30% $\pm$ 0.02 weighted F1 across folds, so the P1 point estimate is not an artefact of one favourable split. The linear models are equally stable at their lower level (88.46% $\pm$ 0.12 and 88.21% $\pm$ 0.11 accuracy), confirming that the roughly 11-point gap between the model families is systematic rather than split noise.

Table 5. Five-fold stratified cross-validation (mean $\pm$ standard deviation).

Model	Accuracy (mean $\pm$ std)	Weighted F1 (mean $\pm$ std)
Random Forest	99.30 $\pm$ 0.02	99.30 $\pm$ 0.02
Linear SVM	88.46 $\pm$ 0.12	88.46 $\pm$ 0.12
Logistic Regression	88.21 $\pm$ 0.11	88.20 $\pm$ 0.10

Tables 6 and 7 report the two holdout protocols that speak most directly to zero-day detection. In the family-based holdout (P3), entire ransomware families are withheld from training, so every test record belongs to a family the model has never seen; identifier attributes are excluded. The grouped split withholds four families, APT, CryptXXX, DMALocker, and TowerWeb, 34,806 records or 23.4% of the corpus. The Random Forest still reaches 99.42% weighted F1 on these unseen families, with SS recall at 99.36%, while the linear models drop to roughly 86%: classification of records from families the model has never seen evidently rests on continuous flow signatures that transfer across families, not on family identity. The time-based holdout (P4), which trains on the earliest 80% of records by Time and tests on the latest 20%, is harder: the Random Forest loses about 4.3 points relative to P1 (94.99% accuracy), with class-level F1 between 94.10% and 96.23%. Traffic characteristics drift over the capture period, and this protocol, the closest analogue to deployment against future attacks, is the paper's most conservative generalisation estimate. One caveat applies to both protocols: their test sets differ in size and class composition from the P1 split (the family holdout in particular tests 34,806 records dominated by the four withheld families), so the scores are not strictly comparable point-for-point with Table 8; the informative comparison is against the linear baselines evaluated under identical conditions.

Table 6. Family-based holdout (P3): whole families withheld from training (values in % as A / S / SS / Weighted).

Model	Class	Precision	Recall	F1
Random Forest	A / S / SS / Wtd.	98.91 / 100.00 / 99.18 / 99.42	100.00 / 99.04 / 99.36 / 99.42	99.45 / 99.52 / 99.27 / 99.42
Linear SVM	Weighted	85.93	86.00	85.87
Logistic Regression	Weighted	86.78	86.89	86.80

Table 7. Time-based holdout (P4): train on earliest 80% of time, test on latest 20% (values in % as A / S / SS / Weighted).

Model	Class	Precision	Recall	F1
Random Forest	A / S / SS / Wtd.	95.50 / 94.95 / 94.46 / 95.00	92.75 / 97.54 / 93.99 / 94.99	94.10 / 96.23 / 94.23 / 94.98
Linear SVM	Weighted	80.30	80.30	79.94
Logistic Regression	Weighted	79.25	79.41	78.96

Finally, Table 8 repeats the random split with the identifier attributes (Family, Threats, IPaddress, SeddAddress, ExpAddress) removed, leaving only features plausibly available during real-time detection. The Random Forest retains 97.78% accuracy and 97.78% weighted F1, a 1.51-point drop from the full feature set, while the linear models lose

about four points. The identifier attributes, including the memorised wallet addresses visible in Fig. 7, therefore contribute real but bounded signal: about a point and a half of the headline result depends on features unavailable on raw production traffic, and 97.78% is the more honest estimate of random-split performance in deployment.

Table 8. Leakage-aware random split (P5): identifiers and label-like attributes removed.

Model	Accuracy	Weighted Precision	Weighted Recall	Weighted F1
Random Forest	97.78%	97.80%	97.78%	97.78%
Linear SVM	84.08%	83.96%	84.08%	83.92%
Logistic Regression	84.38%	84.29%	84.38%	84.30%

5.7 Limitations

Four limitations are worth pointing out. First, the SS class in UGRansome is a synthetic stand-in for zero-day attacks, so even the family and time-based holdout results measure generalisation to unseen synthetic patterns, and performance on truly novel exploits in live traffic could be lower. Second, the full-feature results rely in part on the Family and Threats labels and on memorised address identifiers, which would not be available on raw production traffic; the leakage-aware protocol (Table 8) bounds this effect, and an operational deployment would need to derive equivalent features from packet inspection. Third, the dataset is a single corpus from a single construction methodology [1]; the results have not been validated against an independent capture. Fourth, the family and time-based holdouts each use a single partition; repeating them over multiple grouped splits would attach uncertainty estimates to those results in the way cross-validation does for the random split.

6 Conclusion

This work empirically evaluated classifiers for separating UGRansome network-flow records into Anomaly, Signature, and Synthetic Signature classes, with the Synthetic Signature class as the working analogue of zero-day attacks. The study proceeded in the order the evidence demands: exploratory analysis of the continuous feature space (distributions, correlations, and variance inflation factors), linear reference models, and only then a non-linear Random Forest, adopted because both linear models plateau at roughly 88% accuracy with systematic residual errors.

On the stratified random split the Random Forest reaches 99.29% accuracy and 99.29% weighted F1, with 99.44% precision and 99.48% recall on the Synthetic Signature class. Feature-importance analysis ranks the financial-impact attributes USD and BTC as the two strongest predictors, followed by the cluster and byte-volume attributes; the ablation, however, shows that removing USD and BTC costs at most 0.44 accuracy points, so the answer to RQ-2 is that financial features are the strongest individual predictors but their signal is largely recoverable from the remaining flow attributes. For RQ-1,

the flow-structure attributes carry signal that transfers to unseen families, though the raw timestamp itself contributes little and performance degrades under temporal drift. Because part of the model's signal comes from label-like identifier attributes, including individual wallet addresses, the evaluation also includes cross-validation, family-based and time-based holdouts, and a leakage-aware feature set. Cross-validation reproduces the point estimate (99.30% $\pm$ 0.02 accuracy); a family-based holdout over four unseen families (APT, CryptXXX, DMALocker, TowerWeb) reaches 99.42% weighted F1; the leakage-aware feature set retains 97.78%; and the time-based holdout retains 94.99%, the paper's most conservative and most deployment-relevant estimate. The reported performance therefore does not rest on memorised identifiers, though genuinely novel attacks are best approximated by the time-based figure. Future work will focus on validating the framework on live traffic captures, extending the feature set with packet-level statistics, and benchmarking against deep-learning architectures such as the CNN–LSTM hybrid of Khorsheed et al. [2].

References

1. Nkongolo, M., van Deventer, J.P., Kasongo, S.M.: UGRansome1819: a novel dataset for anomaly detection and zero-day threats. Information. **12**(10), 405 (2021) Available: https://www.mdpi.com/2078-2489/12/10/405
2. Khorsheed, F.H., Abed, E.A., Mohammed, Z.H., Alwan, W.B.K.: A robust hybrid CNN–LSTM framework for high-accuracy zero-day intrusion and ransomware detection using the UGRansome dataset. J. Inf. Netw. Comput. Sci. (JOINCS). **9**(1), 1–10 (2026) Available: https://joincs.umsida.ac.id/index.php/joincs/article/view/1688
3. P. Azugo, H. Venter, and M. W. Nkongolo: Ransomware detection and classification using random forest: A case study with the UGRansome2024 dataset, arXiv preprint https://arxiv.org/abs/2404.12855, (2024)
4. Sabu, S.: Ransomware analysis using machine learning and deep learning: a comprehensive study with the UGRansome dataset. Medium. (2024) Available: https://medium.com/@samson.sabu/blog-post-ransomware-analysis-using-machine-learning-and-deep-learning-a-comprehensive-study-with-a48dfe024dbf
5. K. Zhang, Y. Wang, U. A. Bhatti, Y. Zhou, and M. Jin: Enhanced ransomware attacks detection using feature selection, sensitivity analysis, and optimized hybrid model, J. Big Data, vol. 12,, art. no. 245 2025. Available: https://link.springer.com/article/10.1186/s40537-025-01289-1
6. Kennedy, B.J., Farid, M.A., Wayndt, M.: Synthetic Cognitive Augmentation Network. In: Proceedings of SEET2025 (2025)
7. Adejokun, A., Siok, M., Huong, L.: Security Engineering Framework for Cyber-Physical System Product-Line. In: Proceedings of SEET2025 (2025)
8. Zahra, S.R.: UGRansome: Optimal Approach for Anomaly Intrusion Detection and Zero-Day Threats Using Cloud Environment. MSc Research Project, School of Computing, National College of Ireland, Dublin (2022) Available: https://www.researchgate.net/publication/365172610
9. A. Igugu: Evaluating the Effectiveness of AI and Machine Learning Techniques for Zero-Day Attacks Detection in Cloud Environments. MSc thesis, Luleå University of Technology, (2024). Available: https://ltu.diva-portal.org/smash/record.jsf?pid=diva2%3A1890285
10. Rumale, A., Desai, M., Asadinia, M.: STRIDE: sports tracking and injury detection using estimations. In: Proceedings of SEET2025 (2025)

11. Jesus, A.B., et al.: Exploring manifold-based clustering techniques for enhanced inductive thematic analysis. In: Proceedings of SEET 2025 (2025)
12. Kumari, N.S., Kuchibhotla, S.: Adapting the virtual nominal group technique for enhanced risk assessment in cloud computing: a machine learning approach framework using data analysis and predictive modelling. J. Theor. Appl. Inf. Technol. **103**(11) (2025) Available: https://www.jatit.org/volumes/Vol103No11/21Vol103No11.pdf
13. S. J. Nhlapo and M. N. W. Nkongolo, "Zero-day attack and ransomware detection," arXiv preprint https://arxiv.org/abs/2408.05244, (2024)
14. Ispahany, J., Islam, M.R., Islam, M.Z., Khan, M.A.: Ransomware detection using machine learning: a review, research limitations and future directions. IEEE Access. **12**, 68785–68813 (2024). https://doi.org/10.1109/ACCESS.2024.3397921. Available: https://ieeexplore.ieee.org/document/10521643
15. Faizan, M., et al.: Predicting User Affective States from Mobile Notification Interactions Using LLM-Based Machine Learning Models. In: Proceedings of SEET2025 (2025)

Biologically Plausible Learning in Spiking Neural Networks Using Indirect Feedback Alignment

A Study of the Implementation of Feedback Alignment Methods into Spiking Neural Networks

Joshua Nallaraja(✉) and Jonathan Liaw

Pennsylvania State University: The Behrend College, Erie 16415, PA, USA
joshuanalla.2@gmail.com

Abstract. Backpropagation (BP) has been the backbone for training deep neural networks; however, its biological implausibility, particularly the requirement for symmetric weight transport between forward and backward passes, inhibits its implementation in biologically attuned fields. In contrast, an existing biological alternative to the traditional artificial neural network is the recently introduced spiking neural network. By implementing feedback alignment methods of weight propagation, a more biologically plausible solution to the biological gap can be created. This paper presents a systematic implementation of feedback alignment methods into the spiking neural network framework. For this paper, three experiments are performed with models of varying SNN builds: a vanilla implementation with backpropagation and both a two layer and three-layer version of the indirect feedback alignment. Identical network architectures, including SNN dynamics, LIF neuron parameters, Poisson spike encoding and surrogate gradients, are maintained amongst all models for the fairest comparison. The vanilla feedback alignment model is biologically questionable at best and was therefore not included in this research. All algorithms are evaluated on the MNIST dataset, a handwritten digit recognition benchmark, trained across a comprehensive sweep of five optimizers and six activation functions. As expected, backpropagation achieves the highest accuracy, but it is challenged in several areas by the feedback alignment models. Our results are cautiously promising, with numbers nearing the performance of backpropagation.

Keywords: Spiking Neural Networks · Backpropagation · Direct Feedback Alignment · Indirect Feedback Alignment · Surrogate Gradient · Leaky Integrate-and-Fire · Biologically Plausible Learning · Weight Transport Problem · MNIST · Rate Coding

1 Introduction

The standard backpropagation algorithm has driven remarkable advances in machine learning over the past decade. Yet, it remains at odds with biological plausibility because of its non-biological approach to weight distribution. The brain consists of many neurons

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 697–709, 2027.
https://doi.org/10.1007/978-3-032-39763-8_43

that communicate using electrical signals across connections called synapses. Critically, synapses communicate in a unidirectional fashion. This is where the backpropagation algorithm conflicts with observed biological mechanisms, since it propagates weight updates in a bidirectional fashion (Fig. 1).

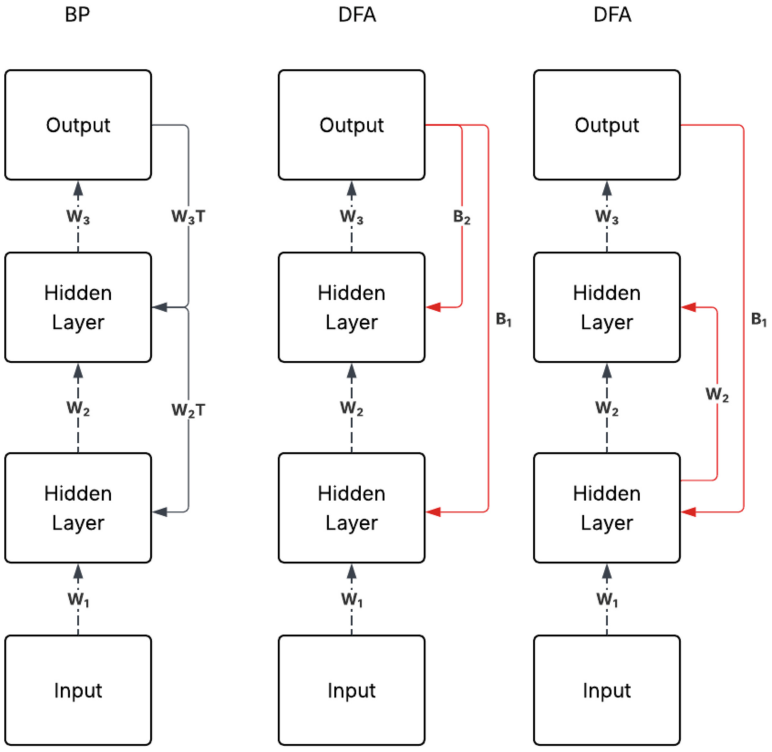


Fig. 1 A clear diagram showing the difference in each weight propagation method.

In contrast, a recently introduced algorithm proposes a different approach to weight propagation is feedback alignment [1, 2]. In short, the weights are multiplied with a random matrix and propagated backward to each layer, providing a means of learning that does not depend on the exact transpose of the forward weights to learn effectively. Feedback alignment presents the solution to that issue through the fixed random matrix. Although feedback alignment's method revolves around a "point in the right direction" approach as compared to the precise improvement outline the backpropagation algorithm presents, the biological gap is essentially bridged, or at least partially.

Although feedback alignment corrects the value passed backward, the way it propagates the signal remains biologically restrictive, as it still requires a backward pass through each layer. Therefore, that algorithm is not explored in this work, but two of its derivatives are. Direct Feedback Alignment and Indirect Feedback Alignment address this limitation by keeping the forward pass intact while sending the error signal directly to each layer, rather than passing it back through each individual layer sequentially.

Despite these theoretical contributions, most comparisons between these algorithms have only been conducted on standard artificial neural networks, as seen in the works of Nøkland and Lillicrap [1, 2]. The combination of spiking neural networks and feedback alignment remains largely unexplored, given their relatively recent introductions. Their combination could produce a more biologically plausible neural network framework, potentially advancing the field in a meaningful way.

This research primarily targets three learning algorithms: Backpropagation, Direct Feedback Alignment, and Indirect Feedback Alignment. Backpropagation serves here as the standard benchmark. Direct Feedback Alignment presents a version where each hidden layer receives its error signal directly from the output error through a fixed random matrix. Indirect Feedback Alignment extends this by directing all feedback to the first hidden layer, which then propagates it forward through the remaining layers. Crucially, unlike backpropagation, these two algorithms present a more biologically plausible mechanism for transmitting feedback to their respective hidden layers. The remainder of this paper is structured as follows: Sect. 2 details the methodology behind the experiments, Sect. 3 describes the experimental setup, Sect. 4 provides a brief discussion, Section 0 presents a thorough analysis of the results, and Sect. 6 concludes the paper.

2 Proposed Method

The models explored in this paper, as mentioned before are Backpropagation (Benchmark), Direct Feedback Alignment (contextually only, the results are derived from Zhang et al's work [9]), and 2 versions of Indirect Feedback Alignment.

Before the algorithms are explored, it is important to present a portion of the code that will be common throughout the experimental setup: Poisson Encoding. Spiking neural networks do not take discrete values as inputs but rather spikes over a time interval. In short, the process of converting these continuous values into "spike trains" for use in the SNN is called Poisson encoding. Essentially, the value is converted into a firing rate, meaning the higher the value, the higher the number of spikes in a period. The spikes are stochastic, meaning even two encodings of the same value will not look identical. This is instrumental in ensuring there is no overfitting in the neural network. In neuroscience, it is often used as a baseline model for how neurons might fire under a time-varying rate, further contributing to the biological plausibility of this experiment. Mathematically, it is presented as follows:

It starts with the pixel value, which after normalization, it becomes a continuous value between 1 and 0 which we can call x. In essence, we are converting this pixel value into a sequence of spikes, so x is converted into a series of 1 s a and 0 s over T time steps. Mathematically, at each time step t, a uniform random number is drawn:

$$u_t \sim U(0, 1)$$

Then it is compared against the scaled pixel value:

$$S_t = 1[u_t < x \bullet r_{max}]$$

r_{max} represents the maximum firing rate, which is essentially the probability of the spike. This creates the spike train passed into the models. The stochastic nature of this mechanism ensures that no two encodings of the same pixel are identical, which contributes to the biological plausibility of the model and helps prevent overfitting.

Secondly, the LIF (leaky, integrate and fire) neuron is the base component of the SNN neural network. Unlike a traditional neuron that operates on binary input and output, the LIF neuron operates on the spike train input (from the Poisson encoding algorithm) and then fires depending on if its voltage is reached. At each timestep t, for every LIF neuron in a layer, the sequence is as follows:

$$I^t = x^t W + b$$

The current at the given timestep is the input multiplied by the weight with the bias added to it.

$$v^t = \alpha \bullet v^{t-1} + \beta \bullet I^t$$

The voltage at any given time is the rate at which voltage decays (defined by α, where the decay rate is multiplied with the previous voltage) added to the absorbed amount of the input (defined by β, where it defines the fractional amount of the input that can be absorbed).

The 'spike' is when the voltage crosses the threshold ϑ, at which the neuron fires. It can be defined as a step function, where if the voltage crosses the threshold, then the neuron fires by outputting 1, if not it outputs 0. The collection of these "spikes" makes the spike train for the next layer. Each neuron in a hidden layer computes a weighted sum of all spikes from the previous layer, where the weights are unique to that neuron, giving each neuron a distinct input. A crucial thing to note is that immediately after firing, the voltage of that neuron is reset to 0.

$$s^t = 1 \; if \; v_t \geq \vartheta, \text{otherwise } 0$$

After each fire, there is a refractory period, where the neuron goes silent for a few timesteps regardless of input. In our experiment, that number is set to 1 ms which would be 4 timesteps of forced silence. This closely resembles the structure of an actual neuron's interaction in the brain. Real neurons also go electrically silent after firing due to ion channel dynamics.

Another crucial mathematical element is the surrogate gradient, which bridges the gap between LIF dynamics and the learning algorithms (BP, DFA, IFA, etc). Since the spike function is a step function, it indicates that there can be no derivative. The Zero derivative means the network cannot learn through standard gradient descent. This is known as the dead gradient problem. To fix this issue, a fake derivative must be set in motion. This fake derivative is known as the surrogate gradient.

For this research, windowed rectangular surrogate is used. Essentially, if the voltage is close to the threshold by a margin of a window of width w on either side, then present the gradient is a constant β_t (this is not to be confused by the rate at which input is absorbed β_s, this is a completely different hyperparameter set during the experiment). If the voltage is out of this 'rectangular box', then gradient is 0 as usual. The crucial

aspect is that this is a deliberate lie told only during the backward pass. The forward pass is honest; no surrogates or lie have been told. The backward pass pretends the spike function has a smooth derivate so that the gradient signal can flow.

The roles of the feedback matrix and the surrogate gradient are distinct yet complementary. The answer is that both are required for their own functions. The random matrix B answers the question of which direction each layer's weights should move, while the surrogate gradient answers the question of which neurons in the layer should be updated. So, the full error signal at a hidden layer is as such:

$$\delta_\ell = \left(\mathrm{eB}_\ell^T\right) \bigodot \sigma'(v_\ell)$$

The first part gives the direction of change required while the second acts as a mask, zeroing out neurons that aren't near firing and keeping those that were. Without the mask, the purpose for which the surrogate gradient exists, neurons not involved with a particular input could be impacted, leading to further noise.

The first model explored is the Vanilla Backpropagation Model. Its functionality is rather intuitive. The weights are propagated forward, and then whatever feedback can be derived from the loss function is sent directly backward, propagated to every hidden layer. This feedback is "listened to" and the weight in each hidden layer is modified in a way to increase the accuracy the next experimental round. Mathematically, it is presented as follows. Backpropagation can be broken down into three distinct stages, the forward pass, the loss computation and the backward pass. During the forward pass, each time step t, the input spikes are multiplied by the weight matrix and added to the bias to produce the current:

$$I_t = x_t \cdot W + b$$

That current is fed into the LIF neuron dynamics to update the membrane voltage, after the complete pass, the loss is computed. The spike trains from the output layer are summed to produce a firing rate vector:

$$\hat{r} = \sum_{t=1}^{T} s^{\mathrm{out}}(t)$$

The SoftMax is used to normalize and produce class probabilities. Then the cross-entropy loss between the predicted probabilities and the one-hot ground truth label y is then computed. In short, the cross-entropy loss function is used on the outcome to compute the total loss. The backward pass, even for backpropagation, is where the math departs from the biologically plausible alternative.

$$\delta^{\mathrm{l}}(\mathrm{t}) = \left(\delta^{\mathrm{l}+1}(\mathrm{t}) \cdot \left(\mathrm{W}^{\mathrm{l}+1}\right)^{\top}\right) \circ \mathrm{f}'\left(\hat{\mathrm{v}}^{\mathrm{l}}(\mathrm{t})\right)$$

This equation represents the backward pass in backpropagation. At each timestep t, the error signal for layer l is computer by taking the error from the layer directly above it and projecting it backwards through the transposed weight matrix. That projected error Is then element-wise multiplied (to zero out the neurons that did not fire with respect to that input) by the surrogate gradient. This is essentially how the backpropagation

algorithm works. The critical bottleneck is the transpose matrix. Every layer requires exact knowledge of the weights from the layer above it, a constraint that is biologically implausible and that DFA and IFA are specifically designed to eliminate.

DFA removes the weight transport problem entirely. Rather than routing the error signals backwards through the network layer by layer, each hidden layer receives the output error e directly, which is then projected through its own fixed random matrix. The weight update for every hidden layer is computed in parallel, thus eliminating any extra layers needed. What happens to the error signal at each hidden layer is exhibited by the equation below:

$$\delta^{1}(t) = \left(e \cdot B_{1}^{\top}\right) \circ f'^{\left(\hat{v}^{1}(t)\right)}$$

The structure of this equation is identical to the backpropagation equation. The surrogate gradient mask plays the exact same role in both, eliminating the neurons that don't fire. The difference is in the first one, where the error, instead of chained back, is replaced by the raw output error projected through the random matrix.

The question is why random matrices work at all, which can be answered through a phenomenon known as angle alignment. Even though B is random and bears no relationship to W at initialization, research indicates that through the course of training, forward weights W gradually rotate to align with the feedback matrices. In short, the network learns to make the random feedback useful. This doesn't happen instantly, but over many epochs until the alignment is sufficient to sustain meaningful learning. This does mean it won't be as effective as backpropagation, but it will still be high, as experimental results will show.

Finally, IFA sits between backpropagation and DFA. It is the most nuanced of the three algorithms and the central contribution of this experiment. Like DFA, it removes the weight transport problem and does not require any layer to know the weights of another. Unlike DFA, not every layer receives the output error directly. Instead, only the first hidden layer receives a direct random projection of the output error, and all subsequent layers received their error signals indirectly, propagated forward through the living forward weights of the layers below them. The mathematics is as follows:

The first layer receives the error identical to DFA:

$$\delta^{1}(t) = \left(e \cdot B_{1}^{\top}\right) \circ f'\left(\hat{v}^{1}(t)\right)$$

What's different is how the remaining layers get it, Layer 2 receives no direct feedback but rather the error signal from layers 1, projected forward through the forward weight matrix, without transposing it. The following equation represents what happens to layer 2 to receive its 'feedback' but is identical to every other layer below it.

$$\delta^{2}(t) = \left(\delta^{1}(t) \cdot W^{2}\right) \circ f'\left(\hat{v^{2}}(t)\right)$$

The third model explored is the depth-scaled direct feedback alignment. These experiments (with both 2 layer and 3 layer) were run as a thought-experiment to see if a modification to the Vanilla DFA algorithm would help. Just like normal Direct Feedback Alignment, the output error is projected through a fixed random matrix to each hidden

layer directly. The distinction lies in how the feedback signal is scaled. Layers closer to the input receiver a stronger error signal, while the deeper layers receive a progressively trimmed one. This is more consistent with biological neural circuits are understood to propagate feedback and therefore lightly touched upon this research.

3 Experimental Method

All the code for these models was written in Jupyter Notebooks and run on Colab and Kaggle, depending on GPU availability.

Subsequent paragraphs, however, are indented. All the models used for the experiments were trained for 50 epochs with a batch size of 256. Gradient norms were clipped to 1.0 before each optimizer step to prevent exploding gradients observed during early pilot runs. A cosine annealing learning rate schedule was applied across all optimizer conditions. These values were selected based on standard recommendations in the SNN literature and were held fixed across all 30 sweep conditions to ensure that observed performance differences are attributable to the learning rule-activation combination rather than to hyperparameter tuning.

The BP-LIFLayer class implements the LIF forward pass entirely within a Python for-loop over timesteps, building a list of spike tensors that are stacked into shape [batch, T, units]. The SpikeFunction class (extending torch.autograd.Function) intercepts the gradient flow at each timestep's spike generation. Because the refractory mechanism, implemented via a ref_count counter, is also detached from the graph, the computational graph effectively represents a sequence of LIF voltage updates connected through the surrogate gradient at each spike point.

For DFA, a hidden layer stores the fixed random feedback matrix B as a non-trainable buffer. During the forward pass, it behaves identically to a standard LIF layer. During the backward pass, implemented manually, each layer's gradient is computed by projecting the output error through B and multiplying by the surrogate derivative of the voltage trace at each timestep. A DD-variant was also explored, where each error signal is projected to each hidden layer and is scaled inversely by the layer's depth, so that layers closer to the input receive a stronger signal and deeper layers receive a progressively attenuated one. This scaling is motivated by how biological neural circuits function but is like standard DFA. This variant was not carried forward to the full sweep, the remainder of the experimental results focus on IFA.

A critical implementation decision for BP (but not for IFA) is bias initialization. Since BP gradients flow only through spike events, neurons that do not fire produce a zero-gradient signal. Many iterations of the experiments had to be restarted due to the incorrect bias initialization. With bias initialized to threshold $\times$ 0.7 (as used in IFA), the resting membrane voltage v_{ss} = bias = 0.07 is below the threshold $\theta_1 = 0.1$. At initialization, random He-initialized weights produce zero-mean input currents, so only the bias current reaches the neuron. Since $v_{ss} < \theta$, neurons do not fire at epoch 1, resulting in zero gradients and a dead network. A higher bias implementation at the start meant that the networks would often remain dead for more than 30 epochs. The BP implementation, therefore, sets bias = threshold for each layer, ensuring $v_{ss} \approx \theta$ and providing a baseline firing rate from which the network can learn.

For IFA, we implemented two variants: a 3-layer architecture (784 → 512 → 256 → 128 → 10) matching BP and DFA exactly, and a 2-layer architecture (784 → 512 → 256 → 10) as an experimental ablation. The 2-layer variant removes the third hidden layer (128 units), reducing total hidden units from 896 to 768 while keeping the first two-layer sizes identical for a clean structural comparison. We wanted to find out if the addition of layers would cause a growth in accuracy or not.

Evaluation protocols differ slightly across model implementations due to their independent development histories. For IFA, 10,000 samples are held out from the MNIST training set as a validation split, with the remaining 50,000 used for training, and the validation loss is used to drive the learning rate schedule. For BP. the full 60,000 training samples are used without a held-out validation set, and the test set is monitored for accuracy at each epoch. All final accuracy figures reported in this paper are computed on the same held-out MNIST test set of 10,000 samples. The discrepancy in training set size and scheduling signal between IFA and the remaining models is acknowledged as a limitation and should be considered when interpreting cross-model comparisons.

Another important aspect of this research is the difference in the roles of activation functions of the IFA sweeps. In IFA the activation label serves only to select the weight initialization strategy. ReLU, Leaky ReLU, ELU, and GELU map to Kaiming uniform initialization; tanh maps to Xavier uniform; and SELU maps to LeCun normal. No activation function is applied within the IFA LIF forward pass itself. Results labelled by activation name in the IFA tables therefore reflect six initialization conditions rather than six distinct nonlinearities, and this distinction should be kept in mind when comparing sweep results across learning rules

4 Results and Analysis

Table 1 presents the headline accuracy figures across all four-model variant evaluated in this study. Backpropagation achieves the highest best accuracy at 97.07 (which is expected given its deeply explored nature), followed closely by DFA at 96.75% and IFA-2 L at 96.34%, with IFA-3 L trailing at 95.79%. The gap between BP and the best biologically plausible model is 0.67%, which is remarkably small given that DFA and IFA eliminate the weight transport problem entirely. The mean accuracy across adaptive optimizers tells a similar story: all within less than 1% of each other. The most striking divergence appears in the worst-case column, where IFA-3 L records 42.92% — near-random-chance for a 10-class problem — driven by the catastrophic interaction between SGD and the depth of the IFA error chain, discussed further in the per-model analysis below. These results collectively suggest that biologically plausible alternatives to backpropagation are viable for SNN training on MNIST, with accuracy costs that are negligible under the right optimizer conditions (Tables 3 and 4).

Table 2 reports the full 30-combination sweep for the backpropagation baseline. The results reveal two distinct performance tiers separated entirely by optimizer choice rather than activation function. All five adaptive optimizers (Adam, AdamW, RMSProp, and Nadam) produce results in a tight band between 96.23% and 97.07%, with a within-tier spread of less than 0.9% across all 24 adaptive combinations. SGD with momentum

Table 1. Cross-model accuracy summary — MNIST (50 epochs).

Model	Best Acc. (%)	Mean — Adaptive (%)	Mean — All (%)	Worst (%)
Backpropagation	97.07	96.67	95.38	87.84
DFA [9]	96.75	N/A	92.09	N/A
IFA-2 L	96.34	95.84	93.86	81.93
IFA-3 L	95.79	95.23	88.19	42.92

Best test accuracy, mean accuracy across adaptive optimisers only (Adam, AdamW, RMSProp, Nadam), mean across all optimisers, and worst-case accuracy. DFA results are derived from Zhang et al. [9]

Table 2. BP-SNN — best test accuracy (%) per activation × optimizer. MNIST, 50 epochs, architecture: 784 → 512 → 256 → 128 → 10

Optimizer	Tanh	ReLU	GELU	Leaky ReLU	ELU	SELU
SGD	89.84%	90.45%	91.50%	90.91%	90.86%	87.84%
Adam	96.65%	96.56%	96.80%	96.38%	96.81%	96.42%
AdamW	96.23%	96.43%	96.42%	96.70%	96.69%	96.37%
RMSProp	96.86%	97.07%	96.90%	96.86%	96.95%	96.77%
Nadam	96.76%	96.77%	96.84%	96.78%	96.57%	96.42%

Backpropagation with surrogate gradient. Best: ReLU + RMSProp = 97.07%. All 30 combinations completed.

forms a separate tier, ranging from 87.84% to 91.50%, consistently 5–7 percentage points below the adaptive group. Within the adaptive tier, RMSProp is the strongest optimizer, claiming the top four ranked results and producing the overall best of 97.07% with ReLU. The activation function has minimal effect within adaptive optimizers, with the difference between the best and worst activation under any given adaptive optimizer is at most 0.58%, suggesting that the LIF threshold nonlinearity already dominates the network's nonlinear behavior, with the pre-synaptic activation playing a secondary role. SELU is the weakest activation under SGD (87.84%), likely because LeCun initialization is poorly calibrated to the fixed learning rate and momentum dynamics of SGD in this setting.

Direct Feedback Alignment for SNNs has been explored in existing literature, most notably by Zhang et al. [9], whose work provides a useful point of comparison for the DFA mechanism evaluated in this paper. In their study, standard DFA is implemented on a fully connected SNN using the same core principle explored in this research: the global output error is projected directly to each hidden layer through fixed random matrices, bypassing the need for layer-by-layer error propagation. On MNIST, standard DFA achieves a best accuracy of 96.75% but demonstrates notable instability, with an average of only 92.09% across five trials. This gap between best and average performance is consistent with the theoretical expectation that the surrogate gradient requirement introduces fragility into

DFA-based SNN training, as the precise derivative of the LIF neuron dynamics is difficult to compute accurately and sensitive to parameterization. This instability is an expected consequence of the biological constraints imposed on the learning rule. The 96.75% peak is nonetheless competitive, sitting within 0.32 percentage points of BP's best result in their experiments as well as ours. This directly supports the central claim of this paper that DFA can match backpropagation closely when conditions are favorable. It should be noted that Zhang et al. did use a shallower $784 \rightarrow 1000 \rightarrow 10$ architecture trained for 20 epochs, which differs from our own experimental setup, so their figures are cited here for contextual comparison rather than direct benchmarking against our BP and IFA results. Nevertheless, DFA's implementation with SNN shows promise with extremely similar results as with a backpropagation model, with the predictable introduction of instability.

Table 3. IFA-SNN (2-Layer) — best test accuracy (%) per activation × optimizer. MNIST, 50 epochs, architecture: $784 \rightarrow 512 \rightarrow 256 \rightarrow 10$

Optimizer	Tanh (Xavier)	ReLU (Kaiming)	GELU (Kaiming)	Leaky ReLU (Kaiming)	ELU (Kaiming)	SELU (LeCun)
SGD	81.93%	87.92%	85.52%	86.42%	87.60%	88.05%
Adam	95.59%	95.93%	96.16%	95.71%	96.34%	95.92%
AdamW	94.97%	95.98%	95.64%	96.22%	95.78%	95.81%
RMSProp	96.01%	95.90%	96.05%	95.37%	95.67%	95.98%
Nadam	96.00%	95.78%	95.94%	95.81%	95.82%	–

Indirect Feedback Alignment, 2-hidden-layer variant. Only H1 receives a direct random projection from the output error; H2 receives error propagated forward through W_2. Best: ELU + Adam = 96.34%. 29/30 combinations completed (Nadam + SELU omitted due to preliminary runs not converging within the fixed 50-epoch training budget).

Table 4. IFA-SNN (3-Layer) Best test accuracy (%) per activation × optimizer (MNIST, 50 epochs, architecture: $784 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 10$)

Optimizer	Tanh (Xavier)	ReLU (Kaiming)	GELU (Kaiming)	Leaky ReLU (Kaiming)	ELU (Kaiming)	SELU (LeCun)
SGD	81.03%	45.32%	45.67%	60.39%	42.92%	84.83%
Adam	93.93%	95.42%	95.57%	95.31%	95.35%	94.05%
AdamW	93.65%	95.31%	95.48%	95.31%	95.26%	93.91%
RMSProp	95.67%	95.79%	95.70%	95.36%	95.43%	95.54%
Nadam	95.76%	95.35%	95.72%	95.61%	95.36%	95.68%

Indirect Feedback Alignment, 3-hidden-layer variant. Error chains forward through W_2 and W_3. SGD combinations show severe degradation (42.92–84.83%). Best: ReLU + RMSProp = 95.79%. All 30 combinations completed.

5 Discussion

The backpropagation results obtained in this study compare favorably with prior work on deep spiking neural networks using backpropagation. Lee, Delbruck, and Pfeiffer [8] report test accuracies of 98.56–98.71% on MNIST using a hidden layer SNN with SGD, and 98.71–98.88% with a 300–300 architecture using Adam, both evaluated over epochs 181–200. Our BP-SNN achieves a peak of 97.07% using a deeper $784 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 10$ architecture across a 50-epoch training window. The gap of approximately 1.6–1.9 percentage points is expected given several key differences in experimental conditions: Lee et al. report min-average-max values over a 20-epoch window at convergence rather than a single best epoch, train for up to 200 epochs compared to our 50 and use a shallower architecture with larger hidden layers. Notably, their best result with Adam on a 300–300 network reaches 98.88%, while our equivalent adaptive optimizer results, Adam, AdamW, RMSProp, and Nadam, cluster tightly between 96.23% and 97.07% across six activation functions, suggesting our implementation is competitive but operating in a regime of fewer epochs and a more constrained training budget.

The comparison also highlights the effect of architecture depth on BP performance in SNNs. Lee et al. [8] achieve their strongest results with relatively compact 2-hidden-layer networks (800, 500–500, and 300–300 units), whereas our architecture adds a third hidden layer of 128 units, increasing depth at the cost of total hidden units, 896 total compared to their 600 in the 300–300 setting. Despite this, our BP model maintains strong performance across all adaptive optimizers, with RMSProp consistently producing the best results and ReLU emerging as the most reliable activation. The tight clustering of adaptive optimizer results (under 0.9% spread across 24 combinations) reinforces the finding of Lee et al. that adaptive methods such as Adam provide a meaningful advantage over SGD in SNN training, a pattern that holds consistently across all learning rules examined in this study. Taken together, these comparisons situate our BP baseline within the established literature and validate the experimental setup before drawing conclusions about the biologically plausible alternatives.

There were several limitations for the conclusions drawn from this study. Firstly, this evaluation is restricted to MNIST. While MNIST remains a standard benchmark for learning-rule behavior under controlled conditions, its simplicity limits what can be learned about the scalability of IFA. Accuracy on MNIST saturates across a wide range of methods, which compresses the apparent gap between learning rules (The close BP-IFA margins reported here may widen on harder benchmarks, for example). The establishment of whether these trends remain consistent with more challenging datasets such as Fashion-MNIST, CIFAR10 or a neuromorphic benchmark remains necessary future work.

Secondly, our analysis centers on classification accuracy and does not quantify biological realism or computational efficiency beyond the structural avoidance of weight transport. Since the primary motivation for FA methods being biological plausibility and hardware efficiency rather than raw accuracy, metrics such as wall-clock training time, memory footprint would directly characterize their advantages. We report these as targets for future evaluation as well.

6 Conclusion

This study investigated the viability of biological plausible learning rules, through Direct Feedback Alignment but with more emphasis on Indirect Feedback Alignment, as alternatives to backpropagation for training spiking neural networks, evaluated through comprehensive 30-combination sweep of activation functions and optimizers on MNIST. The central finding is that both DFA and IFA can train competitive SNNs with minimal accuracy cost relative to the BP baseline. Backpropagation achieves a peak of 97.07% with ReLU and RMSProp, while DFA reaches 96.75% [9] and IFA-2 L reaches 96.34% under matched optimizer conditions, a gap of less than 0.73 percentage points. When compared against the established SNN backpropagation results of Lee et al. [8], our BP baseline sits within the expected range given the shorter training budget and deeper architecture, further validating the experimental setup. These results collectively demonstrate that the weight transport problem can be circumvented in SNNs without meaningful loss of classification performance, at least on well-structured image classification tasks at this scale.

Beyond the headline accuracy figures, the sweep revealed a finding with clear practical implications: optimizer choice dominates all other variables, including the learning rule itself. Adaptive optimizers consistently outperform SGD with momentum by 5 to 35 percentage points depending on the model, with the gap widening as depth increases. This is most pronounced in IFA-3 L, where SGD produces near-random-chance results for three of the six activations, driven by the attenuation of the error signal as it chains through successive forward weight matrices. The depth ablation between IFA-2 L and IFA-3 L quantifies this directly: the additional hidden layer costs 0.55 percentage points at peak and reduces the adaptive mean from 95.84% to 95.23%, while dramatically increasing sensitivity to optimizer choice. This represents a measurable tradeoff inherent to IFA's biologically parsimonious feedback structure, and one that future work should seek to address.

Future work should prioritize several directions. First, the depth penalty in IFA-3 L motivates further investigation of depth-aware feedback variants as a means of compensating for chain-induced signal attenuation. Second, aligning the surrogate gradient parameterization, encoding conventions, and evaluation protocols across all models would remove the residual confounds identified in this study and enable cleaner cross-model comparisons. Third, extending the evaluation beyond MNIST to more challenging and neuromorphic benchmarks would test whether the narrow BP-IFA gap observed here holds under greater task complexity. Fourth, reporting repeated-trial variance and significance testing across multiple seeds, together with efficiency metrics such as training time, would place these comparisons on firmer statistical footing. Ultimately, this work demonstrates that the gap between biological plausibility and competitive performance in SNNs is narrower than previously appreciated.

Disclosure of Interests.. Neither of these authors have competing interests.

References

1. Lillicrap, T.P., Cownden, D., Tweed, D.B., Akerman, C.J.: Random synaptic feedback weights support error backpropagation for deep learning. Nat. Commun. **7**, 13276 (2016). https://doi.org/10.1038/ncomms13276
2. Nøkland, A: . Direct feedback alignment provides learning in deep neural networks. In: Advances in Neural Information Processing Systems (NIPS), 29, 1037–1045 (2016). https://arxiv.org/abs/1609.01596
3. Lillicrap, T.P., Santoro, A., Marris, L., Akerman, C.J., Hinton, G.: Backpropagation and the brain. Nat. Rev. Neurosci. **21**(6), 335–346 (2020). https://doi.org/10.1038/s41583-020-0277-3
4. Neftci, E.O., Mostafa, H., Zenke, F.: Surrogate gradient learning in spiking neural networks: bringing the power of gradient-based optimization to spiking neural networks. IEEE Signal Process. Mag. **36**(6), 51–63 (2019) https://arxiv.org/abs/1901.09948
5. Zenke, F., Ganguli, S.: SuperSpike: supervised learning in multilayer spiking neural networks. Neural Comput. **30**(6), 1514–1541 (2018)
6. Grossberg, S.: Competitive learning: from interactive activation to adaptive resonance. Cogn. Sci. **11**(1), 23–63 (1987)
7. LeCun, Y., Cortes, C., Burges, C.J.C.: MNIST handwritten digit database. ATT Labs. (2010) http://yann.lecun.com/exdb/mnist/
8. Lee, J.H., Delbrück, T., Pfeiffer, M.: Training deep spiking neural networks using backpropagation. Front. Neurosci. (2016)
9. Zhang, Yongbo, et al. "Training Spiking Neural Networks via Augmented Direct Feedback Alignment." (2024), arXiv, https://arxiv.org/abs/2409.07776

Enhancing Requirement Analysis with Hybrid AI-Based Question Answering Systems

Yongyan Liang(✉), Lihua Lan, and Muhammad Abdul Basit Ur Rahim

Department of Computer Engineering and Computer Science, California State University Long Beach, Long Beach, CA 90840, USA
{Yongyan.Liang01,Lihua.Lan01}@student.csulb.edu, m.basit@csulb.edu

Abstract. Requirement analysis is a critical phase in software engineering, but it is often affected by ambiguity, incompleteness, and inconsistency when Software Requirements Specifications (SRS) are written in natural language. Existing AI-based question answering systems improve requirement analysis but remain limited by the effectiveness of their retrieval and ranking components. This research enhances the QAssist framework by introducing BGE embeddings, a BGE cross-encoder reranker, and a hybrid retrieval strategy that combines lexical and semantic methods. The proposed approach is evaluated using the REQuestA dataset together with GTv6 and DexV2. Results demonstrate improved recall and ranking quality for domain-specific queries while introducing a trade-off in execution time. These findings highlight the importance of semantic retrieval in requirement analysis and demonstrate the potential of AI-based retrieval and question answering systems.

Keywords: Artificial Intelligence · Requirement Analysis · Question Answering · Information Retrieval · NLP

1 Introduction

Requirement analysis is a fundamental stage in software engineering that defines the expected functionality and behavior of a system. SRS are written in natural language to ensure accessibility for a range of stakeholders, including developers, managers, and domain experts. Natural language introduces several challenges, such as ambiguity, inconsistency, and incompleteness, which can significantly affect the quality of the final system [3,4,6]. These issues often lead to miscommunications, incorrect implementations, and increased development costs. The QAssist framework was introduced to address these challenges by applying question answering techniques to requirement analysis, allowing users to interact with requirements through natural language queries [1]. QAssist retrieves relevant information from both the SRS and external domain knowledge sources, where domain-specific corpora have been shown to improve the handling of ambiguity

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 710–725, 2027.
https://doi.org/10.1007/978-3-032-39763-8_44

in requirements [5]. This approach highlights the potential of combining Natural Language Processing (NLP) and QA systems to support requirement engineers. Despite its effectiveness, the performance of such systems is mainly dependent on the retrieval and ranking components, which motivates further improvements.

1.1 Research Aim and Objectives

The main aim of this study is to enhance requirement analysis using AI-based retrieval and question answering systems. This involves improving the underlying retrieval and ranking mechanisms to better handle natural language queries. One objective is to replace existing dense retrieval models with more advanced embedding models that can capture semantic meaning more effectively. Another objective is to improve ranking performance using cross-encoder models that evaluate the relationship between queries and retrieved passages. The study also aims to introduce a hybrid retrieval approach that combines lexical and semantic methods, ensuring that both exact matches and contextual relevance are considered. Additionally, the research evaluates the effectiveness of these improvements using real datasets to ensure practical relevance. By achieving these objectives, the study contributes to the development of more robust QA systems for requirement analysis.

1.2 Research Problem

Although QA-based systems such as QAssist [1] provide a promising approach to requirement analysis, they still face limitations in accurately retrieving and classifying relevant information. One of the key challenges lies in handling domain-specific queries, which often require deeper semantic understanding beyond simple keyword matching. Traditional retrieval methods may fail to capture the context of such queries, leading to incomplete or irrelevant results [10]. Additionally, ranking mechanisms may not always prioritize the most relevant passages, which affects the overall usability of the system. These limitations highlight the need for improved retrieval and ranking techniques that can better handle the complexity of natural language requirements. Therefore, the objective of this research is to enhance the performance of QA-based requirement analysis systems by improving their ability to retrieve and rank relevant information. This is achieved by integrating advanced embedding models and hybrid retrieval strategies. By addressing these challenges, the study aims to provide a more effective tool for requirement engineers.

1.3 Research Questions

This research is guided by several important questions that aim to evaluate the effectiveness of the improvements.

RQ1: How do advanced embedding models influence retrieval performance, particularly in domain-specific contexts?

RQ2: Does a hybrid retrieval approach provide better results compared to traditional lexical or dense retrieval methods?

RQ3: What trade-offs exist between recall, ranking quality (nDCG), and execution time in the improved, more complex system?

1.4 Significance of the Study

This study shows how modern NLP and AI techniques can improve the process of requirement analysis [2]. By using automated querying and the retrieval of requirement information, the system reduces the manual effort required by analysts. It improves retrieval of relevant requirement information, which may assist analysts when identifying ambiguity and incompleteness [3,4]. The introduction of hybrid retrieval shows the importance of combining multiple approaches to achieve better performance. The study shows trade-offs between retrieval accuracy, ranking quality, and computational efficiency. These insights are valuable for researchers and engineers working on AI-assisted requirement engineering. The findings contribute to the broader field of information retrieval by showing the effectiveness of hybrid methods in domain-specific contexts [13,15]. This research shows the practical benefits of integrating AI into requirement analysis.

2 Proposed Methodology

2.1 Baseline

The baseline system used is based on the QAssist framework [1], which follows a standard open-domain question answering architecture. QAssist combines lexical and semantic retrieval methods to identify relevant information from SRS and domain-specific corpora. In the retrieval stage, the system uses BM25, a probabilistic information retrieval model [13] that ranks documents based on term frequency and inverse document frequency, making it effective for keyword-based matching [9]. In addition to BM25, QAssist employs DistilBERT for dense retrieval, which represents text as embeddings and captures semantic relationships between queries and passages [12]. This combination allows the system to retrieve candidate passages that are lexically and semantically relevant. After retrieval, the system applies a reranking step to refine the order of the retrieved passages, ensuring that the most relevant results are prioritized. Finally, a machine reading comprehension component extracts the answer from the top-ranked passages. While this pipeline is effective, it has limitations in capturing deeper semantic meaning, particularly for domain-specific queries where exact wording may differ.

2.2 Framework Overview

The proposed system follows a structured framework that extends the QAssist pipeline [1,10]. The process begins with a user providing a natural language

query related to the requirements. The system then retrieves relevant documents from a domain-specific corpus using retrieval models. These documents are processed to extract smaller text passages that are more suitable for analysis. A hybrid retrieval step combines results from lexical and semantic methods to generate a set of candidate passages. These candidates are re-ranked using a cross-encoder model, which evaluates the relevance of each passage to the query. Finally, the system extracts the most relevant answer from the top-ranked passages.

2.3 Proposed Improvements

In this work, the QAssist framework [1] is enhanced by replacing its dense retrieval component with the BAAI/bge-large-en-v1.5 embedding model [7]. The ranking stage is further enhanced by incorporating a BGE cross-encoder for reranking [14], which jointly evaluates querypassage relevance before producing the final ranking. The original QAssist system [1] relies on DistilBERT for dense retrieval, which provides a reasonable balance between efficiency and performance, but it might not fully capture complex semantic relationships in domain-specific queries. By introducing BGE embeddings, which are optimized for semantic similarity tasks, the system is able to better understand contextual relationships between queries and requirement text [11]. This results in improved retrieval of relevant passages, particularly in cases where the wording of the query differs from the original requirement text. In addition, BGE embeddings provide stronger generalization capabilities across different domains, making them well-suited for requirement analysis tasks. This improves the baseline retrieval approach, because it directly enhances the system's ability to identify relevant information based on meaning rather than exact keyword matching.

Unlike retrieval models that independently encode queries and passages [12], the BGE cross-encoder processes both inputs jointly, allowing it to better evaluate their semantic relationship. This results in more accurate ranking of candidate passages, ensuring that the most relevant results are prioritized. The original QAssist evaluation employed a MiniLM-based cross-encoder for reranking [1,8]. In this work, that component is replaced with a BGE cross-encoder reranker, which provides a strong balance between computational efficiency and ranking quality for semantic retrieval. The original QAssist system [1] uses simpler ranking mechanisms, while the improved approach significantly enhances the quality of the final ranked results. Furthermore, the improved ranking mechanism helps address cases where multiple passages are retrieved but differ in relevance, ensuring that the most useful information is presented to the user. As a result, the combination of BGE model [7] and BGE cross encoder [14] improves both retrieval accuracy and ranking precision.

2.4 Hybrid Retrieval Strategy

The hybrid retrieval approach is a key contribution. It combines the strengths of BM25 and dense retrieval methods to improve overall performance. Specif-

ically, BM25 [9] is used with BGE embeddings [7] to ensure that both exact keyword matches and semantic similarities are captured during retrieval. BM25 ensures that exact matches are not missed, while dense retrieval captures semantic similarities between queries and passages [9]. The ranked outputs from both methods are combined using Reciprocal Rank Fusion (RRF), where each candidate receives a fused score based on its ranking in each retrieval method before being passed to the reranking stage. This approach improves recall by increasing the likelihood of retrieving relevant information. It enhances the system's ability to handle domain-specific queries, which require semantic understanding and where relevant information may not share exact wording. These improvements increase recall but also introduce trade-offs in execution time, as the system becomes more computationally complex. The hybrid approach represents a practical solution to the limitations of individual retrieval methods. It is useful in requirement analysis, where lexical and contextual information are important. Overall, the combination of enhanced embeddings, improved reranking, and hybrid retrieval results in a more robust and effective QA system for requirement analysis, improving the original QAssist framework [1].

2.5 Role of NLP

NLP plays a central role by enabling the processing and understanding of textual data [2,15]. It allows the system to break down requirements into tokens and sentences, which are analyzed more effectively. NLP techniques are used to convert text into embeddings [12], which represent the semantic meaning of words and phrases. These embeddings help performing similarity comparisons during retrieval. Additionally, NLP models are used in the extraction phase to identify relevant text spans within passages. This enables the system to provide precise answers to user queries. NLP serves as the foundation for the entire system.

2.6 Pipeline

The system follows a structured pipeline that processes user queries through multiple stages to generate accurate answers. The process begins with a natural language query provided by the user, which is the input. In the first stage, document retrieval is performed using both BM25 [9] and dense retrieval models to identify relevant documents from the domain-specific corpus [12]. These documents are divided into smaller segments during the passage extraction stage, allowing for more precise analysis of relevant information. Next, the results of lexical and semantic retrieval methods are fused using Reciprocal Rank Fusion (RRF) to generate a unified candidate set. The fused retrieval stage generates a candidate set of the top 10 passages. These candidates are then re-ranked using the BGE cross-encoder, which evaluates the semantic relevance of each passage with respect to the query [14]. Performance is subsequently evaluated at $K = 1$, 3, 5, and 10 by selecting the corresponding number of highest-ranked passages. After ranking, the top passages are selected and passed to the answer extraction stage, where the final answer is identified. This pipeline ensures that both

keyword-based and semantic information are utilized effectively throughout the retrieval process (Fig. 1).

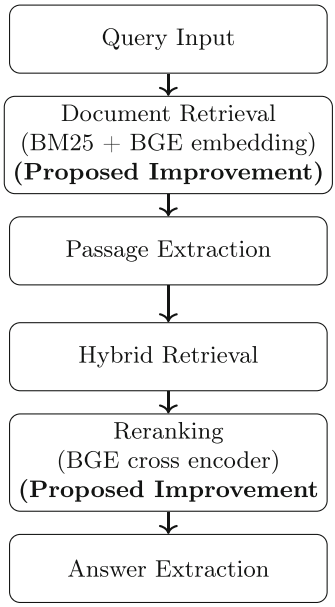


Fig. 1. Proposed QA pipeline for requirement analysis. Components marked **(Proposed Improvement)** indicate the two enhancements over the baseline QAssist system: (1) Document Retrieval replaces DistilBERT with BAAI/bge-large-en-v1.5 embeddings fused with BM25 via Reciprocal Rank Fusion, and (2) Reranking replaces the MiniLM cross-encoder with the BGE cross-encoder (BAAI/bge-reranker-v2-m3). All other stages remain unchanged from the original framework

3 Case Study and Data Preparation

3.1 Dataset

The evaluation is based on the REQuestA dataset used in the QAssist study [1]. This dataset contains 387 question-answer pairs derived from multiple SRS documents across different domains. The questions are designed to test the system's ability to retrieve and understand requirement information. The dataset includes manually created and automatically generated questions. This diversity ensures that the system is tested under realistic conditions. The dataset includes domain-specific corpora, which provide additional context for answering queries [6]. These corpora are constructed using sources such as Wikipedia. Overall, REQuestA provides a comprehensive benchmark for evaluating QA systems.

3.2 Ground Truth

GTv6 is the ground truth dataset for evaluating the performance of the system. This dataset provides validated answers for each query, allowing for comparison between system generated results and expected outputs. The evaluation relies on the ground truth which ensures that performance is measured against a consistent benchmark. This allows for accurate performance measurement and reduces ambiguity in evaluation results. This dataset is essential for computing recall and nDCG as it allows the system to know whether relevant results have been retrieved and correctly ranked. GTv6 supports fair comparison between the baseline QAssist system and the improved pipeline by maintaining a standardized reference point across all experiments. By using a well defined ground truth dataset, the study ensures that the evaluation is reliable and reproducible.

3.3 Additional Dataset

DexV2 is an additional dataset to support the retrieval process and evaluate the system across different domains. DexV2 functions as a searchable corpus or index from which relevant passages are retrieved [6]. In the QAssist framework, this dataset is typically preprocessed collections of textual documents that have been structured to support efficient information retrieval. These documents are segmented into passages and indexed to allow fast matching with user queries. When a query is issued, the retrieval model searches in DexV2 to identify the most relevant passages, and then passed to the ranking and answer extraction. The dataset helps determine the quality of retrieved information, as it defines the pool of knowledge available to the system. By using DexV2, the evaluation is able to measure retrieval performance and generalization across different domains.

3.4 Data Preparation

The data used is preprocessed using a series of NLP techniques to ensure that it is suitable for retrieval and analysis. Initially, the textual content from the SRS and domain-specific corpora is cleaned to remove irrelevant symbols and inconsistencies. Then, the text is tokenized, which involves breaking down sentences into smaller units such as words or subwords to facilitate further processing. Following tokenization, the documents are divided into smaller passages through a process known as passage splitting, allowing the system to retrieve more precise and contextually relevant segments of text. Following the preprocessing strategy of the original QAssist framework, each paragraph was treated as a passage unless it exceeded 512 tokens, in which case it was split into overlapping passages to preserve contextual continuity. These passages are transformed into vector representations using embedding models, enabling efficient semantic retrieval based on similarity measures. In parallel, lexical indices are constructed using BM25 to support keyword-based retrieval [9]. Additionally, domain corpora are prepared to act as a searchable knowledge base during the retrieval phase. This combination of preprocessing steps ensures that both lexical and semantic information are effectively captured, improving the performance of the system.

4 Results and Discussion

The results demonstrate that the improved system consistently outperforms the baseline in retrieving and ranking domain-specific queries. The reported values represent averages across the evaluation dataset.

4.1 Experimental Setup

The experimental evaluation was performed using the REQuestA dataset together with the GTv6 ground truth and the DexV2 retrieval corpus. Performance was evaluated using Recall and nDCG for K = 1, 3, 5, and 10. The improved pipeline combines BM25 lexical retrieval with BGE semantic retrieval using Reciprocal Rank Fusion, followed by cross-encoder reranking before selecting the final ranked passages.

Table 1. Recall Comparison Across Top-K for Domain Queries

K	Baseline Recall	Improved Recall	Recall Improvement (%)
1	0.649	0.735	+13.30 %
3	0.813	0.841	+3.44 %
5	0.813	0.862	+6.03 %
10	0.813	0.862	+6.03 %

4.2 Retrieval Effectiveness Analysis using Recall

Table 1 presents the recall comparison between the baseline and improved systems across different K, which represent the number of top retrieved results considered during evaluation. The results are averaged across multiple SRS datasets, ensuring a consistent comparison between the two approaches. The improved system at K = 1 is where the recall shows the largest improvement. The improved system has a percentage improvement ranging from 3.44% to 13.30%, indicating consistent gains across K. These results demonstrate that the improved system retrieves more relevant results than the baseline at every retrieval depth [11]. The highest percentage improvement occurs at K = 1, indicating that the improvements are most impactful when retrieving a smaller number of top results. As K increases, the percentage improvement gradually decreases, suggesting that the relative benefit becomes smaller at higher retrieval depths. Overall, Table 1 shows a clear improvement in recall performance for the proposed system.

Table 2. nDCG Comparison Across Top-K for Domain Queries

K	Baseline nDCG	Improved nDCG	nDCG Improvement (%)
1	0.649	0.735	+13.30 %
3	0.753	0.803	+6.64 %
5	0.753	0.813	+7.97 %
10	0.753	0.813	+7.97 %

4.3 Ranking Quality Evaluation using nDCG

Table 2 presents the nDCG comparison between the baseline and improved systems across different K, providing insight into ranking quality. The improved system has a significant improvement in ranking performance at K = 1. The percentage improvement ranges from 6.64% to 13.30%, showing consistent gains across different values of K. These results indicate that the improved system not only retrieves relevant results but also ranks them more effectively [11]. The improvements in nDCG demonstrate that relevant information is positioned higher in the result list. The largest improvement occurs at K = 1, suggesting that the system is particularly effective at ranking the most relevant result at the top position. As K increases, the percentage improvement becomes more stable, indicating consistent ranking performance across larger result sets. Overall, Table 2 highlights the enhanced ranking performance of the improved system.

Table 3. Execution Time Comparison Across Top-K for Domain Queries

K	Baseline Time (s)	Improved Time (s)	Time Change (s)
1	2.312	8.691	+6.379
3	2.312	8.691	+6.379
5	2.312	8.691	+6.379
10	2.312	8.691	+6.379

4.4 Computational Cost and Execution Time Analysis

Table 3 presents the execution time comparison between the baseline and improved systems across K, providing a measure of system efficiency. The baseline has constant execution time of 2.312 seconds and the improved system consistently takes 8.691 seconds. This results in an increase of 6.379 seconds for every configuration. The consistent time across K suggests that the computational cost is influenced by the model complexity. Execution time was measured from the beginning of document retrieval until completion of the reranking process using the time.time() function from Python's standard library. The identical execution

times across different values of K occur because retrieval, fusion, and reranking are performed once over the same candidate set, while changing K only affects the number of results reported during evaluation. The improved system requires more processing time because of the advanced embedding model and reranking technique. Despite the increased time, retrieval and ranking performance remain consistent across all values of K.

4.5 Key Retrieval and Ranking Performance Findings

Examining performance across retrieval depths reveals additional distinctions between the baseline and improved systems. The results show that the improved system is more effective at retrieving relevant requirements even as the number of retrieved candidates increases. The baseline shows that the recall values stop improving significantly after K = 3. This suggests that the baseline approach is limited in its ability to discover additional relevant information beyond a certain point. The improved system continues to identify new relevant results at higher values of K, indicating that it operates over a deeper and more informative retrieval pool. In terms of nDCG, the improved system also achieves higher scores, indicating that relevant results are not only retrieved but also ranked more effectively. This means that important requirements are more likely to appear at the top of the result list.

4.6 Semantic Retrieval Analysis and Performance Interpretation

The results indicate that the proposed improvements enhance the system's ability to retrieve and rank relevant information, primarily by addressing what can be described as the semantic gap in requirement analysis. Specifically, the improved system uses BGE embeddings to capture semantic relationships, allowing it to link related domain concepts even when the wording differs. This is particularly important in requirement engineering, where domain-specific terms and implicit knowledge are common. The improvements in recall and nDCG suggest that the system is better able to understand the meaning of queries rather than relying solely on keywords. The increase in execution time reflects the additional complexity introduced by the improved models, particularly the use of BGE for reranking. However, this complexity contributes directly to better ranking quality, as shown by the higher nDCG values. These results demonstrate that the improved system provides a more semantically aware retrieval process.

Another aspect of the results is the relationship between retrieval depth and the effectiveness of semantic retrieval methods. The improved system demonstrates stronger gains at lower values of K, particularly at K=1, where the percentage improvement in recall and nDCG is the highest. This indicates that the system is more effective at identifying the most relevant result early in the ranking process. This reduces the need for users to manually search through multiple results, as the most useful information is likely to appear at the top of the list. As K increases, the performance gap between the baseline and improved systems decreases slightly, suggesting that both are able to retrieve a larger portion of

relevant information at higher retrieval depths. However, the improved system consistently maintains higher recall and nDCG values, indicating that it is more effective overall. This behavior reflects the advantage of combining lexical and semantic retrieval, as it allows the system to capture both exact matches and contextual relationships. The improved ranking performance suggests that the BGE cross-encoder is effectively prioritizing relevant passages within the candidates [14]. Overall, the results show that the proposed system not only improves retrieval performance but also enhances the quality and usability of the returned results.

4.7 Impact of Hybrid Retrieval on Retrieval Effectiveness

The hybrid retrieval approach introduces trade-offs [13]. The results show that recall improves significantly, while the nDCG results clarify that ranking quality also improves. Since nDCG values increase across K, the system is ranking relevant results higher. The primary challenge introduced by the hybrid method is not ranking accuracy, but increased computational complexity due to combining multiple retrieval signals. In some cases, hybrid retrieval may introduce subtle ranking inconsistencies, where a strong lexical match is ranked above a semantically richer passage, but this is not reflected as a dominant issue in the results. Overall, the improved system demonstrates that combining lexical and semantic retrieval can enhance performance while introducing feasible trade-offs.

The improvements can be explained through the concept of the semantic gap in requirement analysis. Traditional keyword-based retrieval methods, such as BM25, rely on exact term matching and may fail when different terminology is used to describe similar concepts [9]. For example, a requirement referring to system reliability may not be retrieved when a query uses the term “fault tolerance”, even though both refer to related concepts. In contrast, BGE embeddings are able to capture semantic relationships between these terms, allowing the system to link related domain concepts effectively. This ability to bridge the semantic gap is important in requirement engineering, where domain-specific terminology and implicit knowledge are common. As a result, the improved system is better able to retrieve relevant requirements even when exact keywords do not match.

4.8 Computational Trade-offs in Model Complexity

The increase in execution time from 2.312 to 8.691 seconds represents a significant computational cost. This increase is due to the use of more advanced embedding models and the BGE cross-encoder for reranking [14], both of which require additional processing time. In requirement engineering, this trade-off is considered acceptable, as accuracy is generally prioritized over real-time performance.

4.9 Effect of Proposed Enhancements on System Performance

The improvements indicate that the system is more robust when handling domain-specific queries, which involve complex terminology and implicit rela-

tionships. By combining lexical and semantic retrieval, the system can capture a wider range of relevant information. This leads to a comprehensive and accurate representation of the requirements.

4.10 Accuracy vs Efficiency Trade-off Analysis

The results highlight a trade-off between retrieval performance and computational efficiency. While the improved system achieves higher recall and nDCG across K, this improvement comes at the cost of increased execution time. Overall, the results confirm that improving retrieval performance and ranking quality introduces a trade-off with computational efficiency.

4.11 Effect of K

The results show that recall and nDCG both improve as the value of K increases for both the baseline and improved systems. Here, K represents the number of top retrieved results considered during evaluation, meaning that larger values of K allow the system to examine more candidate passages. For the baseline system, recall improves from 0.649 at K = 1 to 0.813 at K = 10, indicating that retrieving more results increases the likelihood of including relevant passages. However, the baseline system shows limited improvement beyond K = 3, suggesting that its retrieval capacity is constrained. In contrast, the improved system continues to improve at higher values of K, with recall reaching 0.862 and nDCG reaching 0.813. This indicates that the improved system is able to access a deeper and more relevant pool of candidate passages. The continued improvement at higher values of K suggests that the hybrid retrieval approach is more effective at identifying additional relevant information. The decreasing percentage improvement reflects the fact that both systems approach their maximum retrieval capacity as K increases. However, the improved system consistently maintains an advantage over the baseline. Overall, the results show the choice of K influences both retrieval performance and the relative effectiveness of the proposed improvements.

4.12 Evaluation of Research Questions Based on Experimental Results

The experimental results are interpreted with respect to the research questions defined in Section 1.3.

RQ1, which investigates the impact of advanced embedding models on retrieval performance, is addressed through the recall improvements shown in Table 1. The consistent increase in recall across all values of K demonstrates that BGE embeddings enhance the system's ability to retrieve relevant information, particularly for domain-specific queries where semantic understanding is required.

RQ2, which examines the effectiveness of hybrid retrieval, is supported by improvements in both recall and nDCG (Tables 1 and 2). The combined gains in

retrieval depth and ranking quality indicate that integrating lexical and semantic methods provides more robust performance than relying on a single retrieval approach.

RQ3, which focuses on trade-offs between recall, ranking quality, and execution time, is addressed by comparing Tables 13. While the improved system achieves higher recall and nDCG, Table 3 shows a significant increase in execution time. This confirms that enhanced performance comes at the cost of higher computational complexity. Overall, the results demonstrate that the proposed improvements successfully enhance retrieval effectiveness and ranking quality while introducing a trade-off in efficiency.

5 Limitations

Despite the improvements introduced in this study, several limitations remain that affect the overall effectiveness of the system. One major limitation is the dependency on dataset quality, as the performance of the QA system is directly influenced by the completeness and correctness of the underlying data. If the SRS or the domain-specific corpus contain missing or poorly structured information, the system may fail to retrieve accurate results. Another limitation is the increased computational cost associated with using advanced models such as BGE embedding model and BGE cross-encoder reranker, which require more processing time and resources compared to simpler baseline methods. This becomes particularly significant when scaling the system to larger datasets or real world industrial environments. Additionally, the system still struggles with highly ambiguous or vague requirements, where semantic retrieval methods may not fully capture the intended meaning. The hybrid retrieval approach improves recall but introduces challenges in ranking. Finally, the absence of user feedback mechanisms limits the system's ability to adapt or improve over time, which could otherwise enhance its practical usability. Additionally, this study evaluates the proposed retrieval pipeline as a complete system rather than performing an ablation study. The proposed system modifies three components simultaneously: the embedding model, the retrieval strategy, and the reranker. As a result, the observed gains in recall and nDCG cannot be attributed to any individual change. Therefore, it is not possible to determine whether the improvement is driven primarily by the stronger embeddings, the hybrid retrieval fusion, the improved reranker, or some combination of all three. Future work should isolate each component independently to establish which contributes most to the overall performance gain.

5.1 Threats to Validity

The validity concerns most relevant when interpreting the findings of this study are internal and external validity.

Internal Validity. The primary threat to internal validity concerns the implementation and configuration of the retrieval pipeline. The reported performance

depends on the selected embedding model, reranking model, and retrieval parameters. To reduce this threat, the improved system was evaluated using the same benchmark datasets and evaluation metrics as the baseline, allowing the reported differences to reflect the proposed retrieval enhancements rather than changes in the evaluation benchmark.

External Validity. The evaluation was conducted using the REQuestA dataset together with the GTv6 ground truth and DexV2 retrieval corpus. Although these datasets provide a standardized benchmark for evaluating requirement analysis systems, the reported results may not generalize to all software engineering domains or industrial environments. Additional evaluation on larger and more diverse datasets would further strengthen the generalization of the findings.

6 Implications of the Research

The findings of this research extend beyond requirement engineering and have implications for multiple domains that rely on processing large volumes of unstructured text. For example, in legal document analysis, similar QA-based systems can be used to extract relevant clauses or identify inconsistencies across complex legal texts. In healthcare, the ability to retrieve and interpret domain-specific information from medical documents can assist clinicians in decision making and improve access to critical knowledge. Customer support systems can benefit from such approaches by enabling automated question answering over product documentation, reducing response times and improving user experience. The hybrid retrieval framework proposed highlights the importance of combining lexical and semantic methods, which is applicable to a wide range of information retrieval tasks. The observed trade-offs between recall and ranking quality provide valuable insights for designing systems that prioritize different performance aspects depending on the application. Overall, this research demonstrates that integrating AI with retrieval techniques can significantly enhance the processing and understanding of domain-specific information across various fields.

7 Future Work

There are several directions for future work that can improve the system [10]. One area is the integration of large generative models like ChatGPT which could enhance answer generation by providing more context aware and coherent responses. Another is improving the explainability of the system, allowing users to understand why certain answers are retrieved and how they relate to the requirements. This is important in requirement engineering, where traceability and justification are critical. Future work could focus on handling unanswerable queries more effectively by incorporating confidence scoring mechanisms or no answer detection models. Additionally, fine-tuning retrieval and ranking models

on domain-specific datasets could improve performance, particularly in specialized industries where terminology and context vary. The hybrid retrieval framework could be optimized by balancing lexical and semantic components based on query type. Finally, conducting user studies with requirement engineers would provide valuable insights into the practical usability and effectiveness in real world scenarios.

8 Conclusion

This study demonstrates the potential of AI in enhancing requirement analysis through improved question answering systems [15]. By building upon the QAssist framework, this work introduces significant improvements in retrieval models and pipeline design, particularly through the use of hybrid retrieval methods. The results show that the improved system achieves higher recall for domain-specific queries, indicating a stronger ability to retrieve relevant information. However, this improvement comes with trade-offs, including increased computational cost and execution time, highlighting the complexity of balancing different performance metrics. The study emphasizes the importance of semantic understanding in processing natural language requirements and demonstrates how advanced NLP techniques can support analysts by improving retrieval of relevant requirement information. These findings contribute to the growing body of research on AI-assisted requirement engineering and provide insights into the design of more effective QA systems. Overall, this work shows that while improvements in one aspect of performance may introduce challenges in others, such trade-offs are essential considerations in the development of practical and scalable solutions.

References

1. Ezzini, S., Abualhaija, S., Arora, C., Sabetzadeh, M.: AI-based question answering assistance for analyzing natural-language requirements. In: Proceedings of the 45th International Conference on Software Engineering (ICSE) (2023)
2. Zhao, L., et al.: Natural language processing (NLP) for requirements engineering: a systematic mapping study (2020). arXiv:2004.01099 arXiv preprint
3. Ferrari, A., Esuli, A.: An NLP approach for cross-domain ambiguity detection in requirements engineering. Autom. Softw. Eng. **26**(3) (2019)
4. Dalpiaz, F., Schalk, I., Lucassen, G.: Pinpointing ambiguity and incompleteness in requirements engineering via information visualization and NLP. In: Proceedings of the 24th Working Conference on Requirements Engineering: Foundation for Software Quality (2018)
5. Chen, D., Fisch, A., Weston, J., Bordes, A.: Reading Wikipedia to answer open-domain questions. In: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics. Association for Computational Linguistics, pp. 1870–1879 (2017)
6. Ezzini, S., Abualhaija, S., Arora, C., Sabetzadeh, M., Briand, L.C.: Using domain-specific corpora for improved handling of ambiguity in requirements. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE) (2021)

7. Xiao, S., Liu, Z., Zhang, P., Muennighoff, N.: C-Pack: packaged resources to advance general Chinese embedding (2023). arXiv:2309.07597 arXiv preprint
8. Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., Zhou, M.: MiniLM: deep self-attention distillation for task-agnostic compression of pre- trained transformers. Adv. Neural Info. Process. Syst. **33**, 5776–5788 (2020)
9. Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond, Now Publishers Inc (2009)
10. Zhu, F., Lei, W., Wang, C., Zheng, J., Poria, S., Chua, T.-S.: Re- trieving and reading: a comprehensive survey on open-domain question answering. arXiv preprint arXiv:2101.00774 (2021)
11. Thakur, V, Reimers, N., Ruckle, A., Srivastava, A., Gurevych, I.: BEIR: a heterogenous benchmark for zero-shot evaluation of information retrieval models. arXiv preprint arXiv:2104.08663 (2021)
12. Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding (2018)
13. Manning, C., Raghavan, P., Schutze, H.: Introduction to Information Retrieval, Cambridge University Press (2008). 1st ed
14. Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., Liu, Z.: BGE M3-Embedding: multi-linguality, multi-functionality (2024). arXiv:2402.03216 Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. arXiv preprint
15. Soares, M.A.C., Parreiras, F.S.: A literature review on question answering techniques, paradigms and systems. J. King Saud Univ.-Comput. Info. Sci. **32**(6), 635–646 (2020)

An Exploratory Machine Learning Approach for Mental–Health Related Text Classification

Brandon Breindel[1(✉)] and Muhammad Talha Gul[2]

[1] Pennsylvania State University, Philadelphia, USA
brb23@psu.edu
[2] Consumer Energy Michigan, Jackson, USA

Abstract. Machine learning methods may support preliminary screening of mental health related text for more accurate diagnosis in clinical settings. However, social media as well as survey-based approaches require careful interpretation. This study explores whether machine-learning models can utilize social media posts to classify data based on survey results. Reddit posts were collected from four mental health related communities: Autism, Bipolar, OCD, and Schizophrenia. TF-IDF was used to preprocess and vectorize text. Comparisons between CatBoost and XGBoost utilized accuracy and macro precision. A second phase then applied the chosen trained model to surveys collected in order to determine model accuracy. CatBoost demonstrated higher macro precision, and XGBoost higher accuracy, allowing CatBoost to be chosen. 63% accuracy was achieved in the direct classification of illness, promoting high levels of success within the proposed methodology.

Keywords: Machine Learning · Mental Health · CatBoost · XGBoost · Reddit · TF-IDF · Social Media Data

1 Introduction

1.1 Context

Mental health disorders are significant disturbances within the cognition of individuals, typically causing severe amounts of stress or impairment within typical functioning [2]. According to the National Alliance on Mental Illness (NAMI), "22.8% of U.S. adults experienced mental illness in 2021 (57.8 million people). This represents 1 in 5 adults." [3]. It also suggests, "5.5% of U.S. adults experienced serious mental illness in 2021 (14.1 million people). This represents 1 in 20 adults." [3]. It is clear that mental illness is prevalent, and the number of serious disorders yet to be addressed are misdiagnosed at an alarmingly high rate. The highest rate recorded by PMC PubMed Central for psychiatric diagnosis referrals was only found to be 60% for cognitive disorders [4]. Cognitive dysfunction is caused by mental illness [5], which includes more harmful and impactful disorders such as autism, schizophrenia, bipolar, and OCD. Conclusively, we can state that the medical referral rate is not high enough, even at its highest, to properly diagnose with such a high rate of individuals suffering. At a referral accuracy

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 726–732, 2027.
https://doi.org/10.1007/978-3-032-39763-8_45

rate of 60%, and a 5% rate of illness, that implies that 40% of all individuals (3% of the total, or 1 in 33 individuals) are misdiagnosed and promptly misreferred. With the severe potential consequences of misdiagnosis, it is up to primary care physicians to incorporate more ways to increase the accuracy of initial referrals to avoid potentially disastrous consequences for the patient.

Machine learning is one of many ways to use artificial intelligence, being classified as a subfield of AI. Arthur Samuel defined machine learning as "the field of study that gives computers the ability to learn without explicitly being programmed" [6]. This field in particular has a unique ability to teach itself with minimal manual input as compared to programming other systems. It has three main uses: Descriptive, Predictive, and Prescriptive. When processing data, machine learning may be used to describe why data is functioning the way it is, use the data to predict an outcome, or make suggestions using this data [6]. With the ability to make predictions on outcomes, combined with the convenience of minimal input, machine learning has a plethora of undiscovered potential yet to be utilized within the scope of medicinal fields.

1.2 Goals

This study aims to target the current understanding of mental health disorders and diagnostic practices by screening the social media website Reddit using its public API, processing that data through machine learning algorithms XGBoost and CatBoost. By comparing the efficiency, accuracy, and utility of two widely known and used advanced, intelligence-based models, this study will demonstrate the potential of Machine Learning algorithms to decrease the percentage of misdiagnosis within the field of mental health. It will also take the collected survey responses and use the trained model to appropriately classify each surveyor under the preset mental illnesses or otherwise label them as unaffiliated. To do this, the model would first run a comparison check against the screened Reddit posts for the presence of a mental illness, and if confirmed, the second model would label the results according to the given classifications. Primarily aimed towards healthcare providers, this research demonstrates a way to more quickly, easily, efficiently, and accurately diagnose patients suffering from mental illness. It will also open new doors for software developers to train systems specifically designed to detect those suffering from mental health based on mainstream media websites or applications on the web or smartphone, allowing for early preventative measures to be taken. At the conclusion of this study, the results will have the potential to impact the healthcare industry, software industry, and those suffering from severe mental illness who need a proper diagnosis. It will be used as a guide to show those who wish to continue screening mental health diagnoses whether machine learning or neural networks are more optimal, and which program would better suit their needs.

Research Question 1: Which ML algorithm is the most accurate for the prediction of mental illness?

Research Question 2: To what accuracy can the proposed ML algorithm predict mental illness when tested?

2 Methodology

2.1 Proposed Methods

The proposed methods here are demonstrated in Fig. 1, representing Method 1 and Method 2. Method 1 is the course of selection to choose the optimal Machine Learning algorithm utilizing only Reddit as its source material. Method 2 focuses on interpreting survey results using a trained Machine Learning algorithm.

The first phase of Method 1 collects data from Reddit, harvesting 700 posts per subreddit with four subreddits chosen (Autism, Bipolar, OCD, Schizophrenia). The next phase pre-processes the text from Reddit, and the final phase trains the models. The method proceeds to then obtain the precision, recall, F1, and support scores. The final results are then measured via accuracy and macro precision scores.

Method 2 creates two machine learning data sets for the chosen algorithm from Method 1. The first algorithm will collect data from Reddit, with 100 survey responses logged and organized into a .csv (comma-separated value) file. This file will then be manually split into two files for training, with rows removed based on their compatibility with the first algorithm. Once the survey is filtered, it is appended to the Reddit dataset, where it proceeds to be processed in the same way as the comparison model: scrape Reddit, pre-process text, train machine learning algorithm. In the evaluation period, it is used to interpret the survey results and is compared to manual labelling.

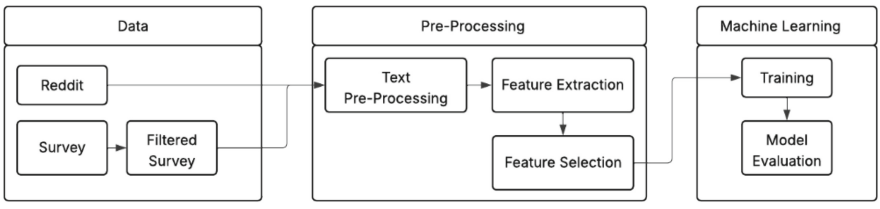


Fig. 1. Proposed methodology process

2.2 Data Acquisition

The proposed methodology process begins with data collection. For both methods, the data acquisition process begins on the Social Media platform "Reddit". The method filters through the 700 posts from four subreddits (Autism, Bipolar, OCD, Schizophrenia), each using the Reddit programming API (https://www.reddit.com/dev/api). The title of each post is then appended to the post text, and these are saved to a csv file. For the second proposed method, the data collection includes 100 survey responses. The surveys are manually entered into the csv in such a way that the system may recognize them, alongside an additional column that includes manual classification for the input dependent on the survey response. These are then parsed into two csv files; one with negative result rows removed, and the other keeping every row as prescribed (Figs. 2 and 3).

2.3 Pre-processing

The proposed methodology's next step takes the obtained text and prepares it for Machine Learning. This process of preparation is called pre-processing, and it involves many steps.

1. *Lowercasing:* Lowercasing is the function of ensuring every alphabetical letter is lower cased. Any capital letters are replaced by their lower-cased letter ("L" becomes "l").
2. Removing Punctuation: Punctuation removal is the removal of any symbols or additional characters, emojis, that are not alphabetic.
3. Tokenization: The process of splitting text into individual words, e.g. tokens, for processing.
4. Stopword Removal: A stopword library is called, and words that appear inside of that library are removed.
5. Stemming: Stemming is the removal of additions to words. This allows further grouping and identification of words ("connected", "connection", "connecting", all become "connect".)
6. Rejoin Tokens: Tokens are reassembled into strings.
7. Trimming Spaces: Any additional spaces are removed.

The processed text is then vectorized in preparation for the machine learning algorithm to accept it. Vectorization is, "the process of transforming words, phrases or entire documents into numerical vectors that can be understood and processed by machine learning models." [9]. This permits the machine learning algorithm to perform functions including prediction, clustering, and classification. For this project, the TF-IDF (Term Frequence - Inverse Document Frequency) measurement was used. TF-IDF is composed of two whole parts, Term Frequency as one part, Inverse Document Frequency the other. The Term Frequency measures the amount of times a word appears in a document (t), and divides that by the total term count within the document (d). This leaves the formula:

$$TF(t,d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

Fig. 2. Term-Frequency formula (Geeksforgeeks) [9]

Inverse Document Frequency (IDF) takes the individual term and measures its value across the whole compilation. As a result, the formula is:

$$IDF(t) = \log\left(\frac{\text{Total number of documents}}{\text{Number of documents containing term } t}\right)$$

Fig. 3. Inverse Document Frequency Formula (GeeksforGeeks) [9]

Multiplying the results of TF and IDF results in the TF-IDF score.

2.4 Machine Learning

K-Means labelling is the clustering of data by grouping similar points of data towards one another. All clusters should hold similar data, while also being distinct from other clusters.

The first report compared XGBoost and CatBoost utilizing the scraped Reddit data (Method 1). There, the data was grouped into four labels (0,1,2,3) and categorized automatically without supervision. For the survey training (Method 2), the Reddit Survey data was compiled randomly into four labels. The survey data was filtered to only have the same four labels as the Reddit scrape; any rows missing proper identification or which had more than one label were removed so that they could be adequately parsed. This resulted in four clusters, labelled 0, 1, 2, 3.

Once the K-Means training was complete, samples were taken from each cluster and manually assigned labels: 0 (OCD), 1 (Bipolar), 2 (Autism), 3 (Schizophrenia). Once labels were assigned to their correct numbers, the K-Means training was run again, labelling four unique clusters for each clinical mental illness: OCD, Bipolar, Autism, and Schizophrenia. The second model was trained via K-Means to cluster only into two groups: Yes, and No. If labelled 'yes', the survey response was identified with connection towards one of the subreddits. If two or more matching associations were not made within the survey, or scores tied, the score analysis was not high enough to warrant and the label given was 'no'. A threshold of three or more matching results had proven to be too restrictive for this study, as it could have limited the ability to demonstrate the use of the CatBoost model.

Model Evaluation is the process in which the trained data is used and/or compared. Once CatBoost was selected, it was used as the primary framework for Method 2, both for the Reddit Survey csv and the Survey csv. The operation was as follows:

1. The survey had a single unique integer identifier from 1–100
2. The survey obtained responses for assessment
3. For identification, the survey had manually inputted 'assessments', both for is_ill and analysis: is_ill was marked 'yes' for 2 or more matching -results, and the identifier was given manually for one of the four clusters, introducing a fifth label "none" for any is_ill that was marked as "no".
4. The machine learning algorithm was given two blank columns, specifically to recognize marked CatBoost as a binary (Yes/No). If the response was "Yes", the Reddit Survey data would be utilized to select one of four labels (OCD, Bipolar, Autism, Schizophrenia).

3 Results

Upon testing and running the algorithms, Method 1 suggested a very close comparison between CatBoost and well-known XGBoost. Both models were trained on the same 2800 posts, and then ran testing evaluation on 371 scraped Reddit posts not used in training. Both scores were compared: XGBoost took an advantage in accuracy (0.80 vs. CatBoost's 0.79), where CatBoost had higher macro precision (at 79.83 over XGBoost's 79.62). Macro precision is more important for multi-class labelling than accuracy, as it means higher reliability among different classes. As a result, CatBoost was selected as the primary model for Method 2's survey.

The model's assigned classification of the five labels (OCD, Bipolar, Autism, Schizophrenia, None) identified by both algorithms in collaboration reached 63% success, as 63 of the 100 survey responses were accurately predicted off of 2800 Reddit posts and 100 survey responses. With more adequate training and a larger sample size, the accuracy can easily be boosted to significantly higher rates.

4 Conclusion

In the last decade, the research community has started to investigate the implications of machine learning to prevent and analyze mental illness. As this study has demonstrated, further research can be done and be demonstrated useful to cement the utility of machine learning algorithms in connecting text to separate mental illness categories. Given the high risk that misdiagnosis carries with severe mental illness, even a simple model such as CatBoost that is widely available has proven to be effective with very limited resources. With proper resources and a large enough data set, researchers in the mental health field could access these tools with even basic knowledge of software assets and train models that could fit into any mental clinic. A well trained machine learning algorithm, alongside a clinician, could allow for faster access to treatment and care for those who need it most. Further research into these categories may allow better insight into the community of those who face intense stress in their migration to the United States of America.

Acknowledgments. A third level heading in 9-point font size at the end of the paper is used for general acknowledgments, for example: This study was funded by X (grant number Y).

Disclosure of Interests. It is now necessary to declare any competing interests or to specifically state that the authors have no competing interests. Please place the statement with a third level heading in 9-point font size beneath the (optional) acknowledgments, for example: The authors have no competing interests to declare that are relevant to the content of this article. Or: Author A has received research grants from Company W. Author B has received a speaker honorarium from Company X and owns stock in Company Y. Author C is a member of committee Z.

References

1. XGBoost Documentation: XGBoost. https://xgboost.readthedocs.io/en/stable/
2. Mental Disorders: World Health Organization. https://www.who.int/news-room/fact-sheets/detail/mental-disorders
3. Mental Health by the Numbers: National Alliance on Mental Illness. https://www.nami.org/about-mental-illness/mental-health-by-the-numbers/#:~:text=Millions%20of%20people%20are%20affected,represents%201%20in%205%20adults
4. Predictive Value of the DSM-IV Criteria for Psychosis Screening in the General Population: National Center for Biotechnology Information. https://pmc.ncbi.nlm.nih.gov/articles/PMC3068718/#:~:text=Results%3A,of%20diagnosing%20psychosis%20was%200%25
5. Cognitive Disabilities and Mental Health: A Handbook for Families. New York State Office of Mental Health. https://omh.ny.gov/omhweb/resources/publications/cogdyshndbk.pdf
6. Machine Learning Explained: MIT Sloan School of Management. https://mitsloan.mit.edu/ideas-made-to-matter/machine-learning-explained

7. Neural Networks: National Network of Libraries of Medicine. https://www.nnlm.gov/guides/data-glossary/neural-networks
8. Neural Networks: A Beginner's Guide. GeeksforGeeks. https://www.geeksforgeeks.org/neural-networks-a-beginners-guide/#
9. GeeksforGeeks: Vectors and Vectorization Techniques in NLP (2025). https://www.geeksforgeeks.org/nlp/vectorization-techniques-in-nlp/ geeksforgeeks.org
10. Kavlakoglu, E., Winland, V.: What is k-means clustering? IBM (2025). https://www.ibm.com/think/topics/k-means-clustering

A Locally Deployed AI Assistant with Natural Language and Symbolic Computation Capabilities

Zaheen Chowdhury[1(✉)] and Muhammad Abdul Basit Ur Rahim[2]

[1] Long Beach Polytechnic High School, Long Beach, CA 90813, USA
chowdhury.zaheen@gmail.com
[2] California State University, Long Beach, Long Beach, CA 90840, USA

Abstract. Artificial Intelligence (AI) has become an increasingly large part of STEM education, being integrated through conversation assistants and tutoring systems to help with advanced problem solving. However, many AI tools rely on cloud-based large language models (LLMs) that require extensive, recurring subscription costs. This can potentially limit accessibility for students looking to use specialized AIs or even innovate and develop their own. This paper presents the development of a locally hosted AI assistant that uses an LLM deployed through Ollama with libraries for mathematics and physics problem solving. The system includes computation using SymPy and graphical representation through Matplotlib to provide step-by-step solutions, graphs, and overall interactive STEM support. Natural language interaction is also used to enhance the user experience. All of this is done without reliance on an API subscription. This paper discusses the system architecture, natural language processing, applications in educational contexts, and the advantages of local deployment - reduced cost, offline functionality, and improved accessibility. Limitations of the model and its future improvements are also discussed, including accuracy, adaptive tutoring, and an expanded range of STEM capabilities. This work demonstrates the potential locally hosted AI systems have as being cost-effective alternatives educational tools.

Keywords: Large Language Model (LLM) · Application Programming Interface (API) · Natural Language Processing (NLP) · Ollama · SymPy · MatplotLib · Local host

1 Introduction

The use of Artificial Intelligence (AI) has become an increasingly prevalent practice in educational environments through tutoring systems that are also conversational, automated problem-solving systems, and learning assistants that can be personalized [3–6]. Recent advances with respect to large language models (LLMs) have allowed conversational AI assistants to be developed for subjects

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 733–747, 2027.
https://doi.org/10.1007/978-3-032-39763-8_46

like mathematics, physics, and computer science, enhancing students' education experiences with interactive explanations and support.

Despite this, many educational AI assistants are reliant on cloud based API subscriptions [2,4]. These systems require continuous financial costs, not to mention a constant, reliable source of internet access to have the AI assistant running. These costs can limit accessibility for students without paid access to API subscriptions. They may not be able to use certain AI assistants that otherwise could help them understand complex concepts smoothly. Similarly, creating a new AI as a personal project may also see the obstacle of API subscriptions, barring students from community-focused innovation intended to benefit their peers.

To address these limitations, this paper presents a locally hosted AI assistant combining a local LLM deployed through Ollama with SymPy and Matplotlib (symbolic mathematics and visualization libraries) [1,2]. The system is composed of natural language interaction with computation using SymPy and graphical representation with Matplotlib. The AI assistant specializes in mathematics and physics problem solving without a reliance on cloud-based APIs.

The primary contributions of the work:

1) The development of a cost-effective, locally hosted AI assistant
(2) The integration of natural language processing with symbolic computation and graphical representation
(3) An analysis of the educational advantages and limitations of locally hosted AI deployment (for STEM learning specifically)

The format of this paper from this point on is as follows:

Section 2 discusses related work in AI assistant education and symbolic computation coupled with LLM systems.

Section 3 describes the system architecture and implementation of the locally hosted STEM based AI assistant.

Section 4 presents the Natural Language Processing Pipeline and potential applications in mathematics and physics problem solving.

Section 5 presents an Experimental Evaluation on the proposed AI system to identify its comparative strengths and current limitations.

Section 6 provides an error analysis based on the data from the evaluation

Section 7 discusses advantages, limitations, and potential future improvements.

Section 8 concludes the paper.

2 Related Works

Recent advances in large language models (LLMs) have enabled a wide range of educational AI assistants to emerge to support students in subjects like mathematics, physics, and computer science. These systems have conversational interfaces that enhance user experience by providing reliable explanations, solving

problem sets, and exploring complex STEM concepts interactively [3–6]. However, because most of these tools rely on cloud based API subscriptions, recurring costs introduce barriers to accessibility and dependence on external servers.

Commercial AI systems such as those developed by OpenAI are a prime example of systems with natural language capabilities for educational purposes. However, since they are hosted in the cloud, offline accessibility is limited, alongside the costs of subscriptions [2,4].

Conversely, symbolic computation libraries like SymPy and graphical visualization platforms like Matplotlib can provide precise mathematical computation and graphed representations, but lack natural language understanding. Recent developments of locally hosted LLMs like Ollama have enabled offline deployment of AI assistant models [2]. Even so, these systems are not fully integrated with the structure for conversational, interactive tutoring [3,6]. Similarly, platforms like Wolfram Research are known to offer strong symbolic mathematical capabilities but have limited support for conversational interaction and adaptive, conversation-based tutoring.

Therefore, there remains a gap in the existing frameworks for a locally deployable AI assistant specialized in STEM that combines natural language understanding, symbolic computation, and graphical visualization/representation all in a single, cost-effective system.

3 System Architecture

The proposed system is a locally hosted AI assistant intended to combine natural language understanding with symbolic computation and graphical representation [1]. The system is designed (but not limited to) to take STEM-related inputs primarily in the subjects of mathematics and physics in natural language, and then output structured solutions, explanations, and graphical representations. All of this is done without the use of a cloud based API.

The system is built of four main components: a local LLM deployed through Ollama, a symbolic computation engine using SymPy, a graphing/visualization component using Matplotlib, and finally a user interface implemented using a lightweight web interface. The language model interprets user inputs, determining whether the input requires symbolic computation, explanation, graphical visualization, or a combination of these.

The locally hosted LLM is the main natural language interface between the user and system. It analyzes entered prompts, determines the full intent of the request and then the route of classification - whether it requires symbolic computation, graphical visualization, conceptual explanation/analysis, or multiple of these modules [1,3]. Symbolic computation capabilities are through SymPy, which performs operations like equation solving, differentiation, integration, other algebraic simplifications, and symbolic manipulation [1]. Matplotlib is used for the graphical representation module, and it generates graphs/plots and other visual representations of mathematical contexts and relationships. The

user interface is the layer of communication between the user and the three computational modules, displaying all symbolic outputs, explanations and analysis, and any graphical visualizations in a single environment.

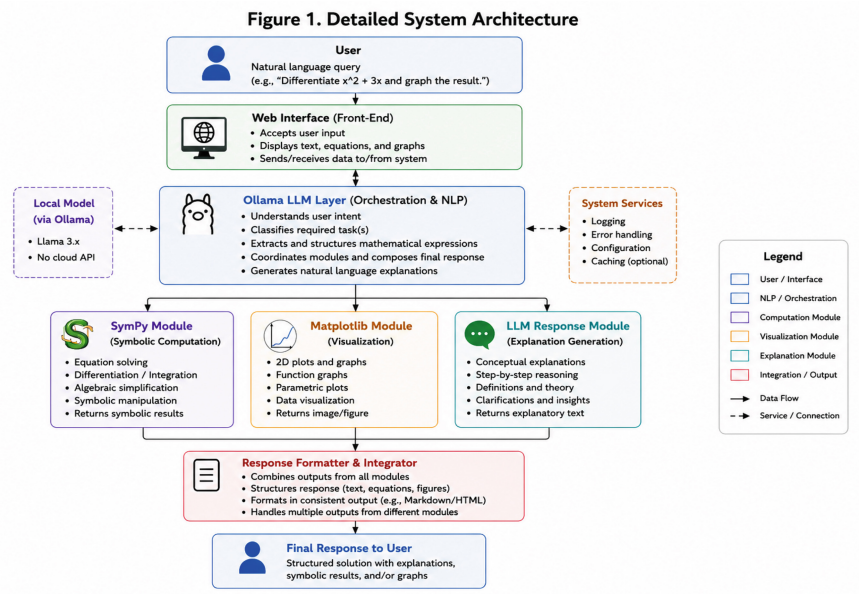


Fig. 1. Detailed System Architecture

Figure 1 illustrates the architecture of the proposed locally-hosted AI assistant.

Once a user submits their query, the system routes that input through a classification step that determines the proper processing pipeline. Mathematical expressions are sent to the symbolic computation module, and more conceptual questions are sent to be processed by the language model. If a graphical output is required, results are passed to the visualization module for rendering. Afterwards, the final output is returned to the user through the interface.

Figure 2 illustrates the difference between traditional API subscription based AI assistants and the proposed locally hosted AI assistant architecture (Table 1).

The proposed architecture allows for the integration of a conversational AI, symbolic computation, and graphical representation all in one locally hosted environment. The elimination of cloud-based APIs reduces the system's dependence on operational subscription costs. It also enables offline accessibility. The specific modular structure of the system will be able to incorporate additional educational features in the future without requiring significant modifications. The following section discusses the natural language processing and input routing pipeline in more detail.

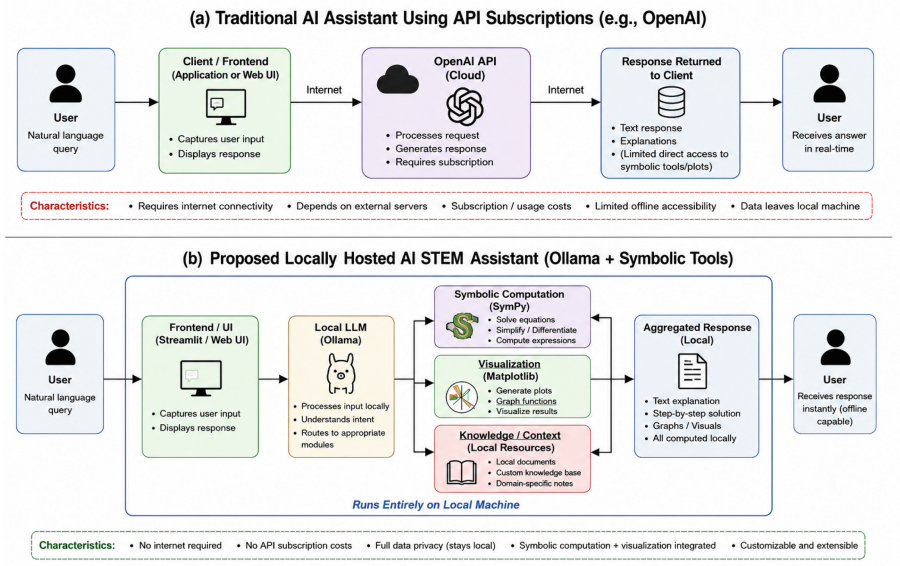


Fig. 2. Comparison of (a) traditional API-based AI assistant architecture and (b) proposed locally hosted AI STEM assistant architecture.

Table 1. Each system component and its function in the proposed locally-hosted AI system

System Component	Function
Ollama LLM	Natural language understanding and explanation, intent determination, explanation generation
SymPy	Symbolic mathematics and equation solving
Matplotlib	Graph/visual generation
User Interface	User interactions and presentation of final output
Response Formatter	Integration of outputs from all modules

4 Natural Language Processing Pipeline

The Natural Language Processing (NLP) pipeline allows the system to accurately interpret user inputs and then route them to the appropriate computational module. Instead of requiring inputs to be formatted with strict syntax, the system can accept conversational natural language, which it then analyzes using a locally hosted LLM deployed though Ollama [1,3]. This significantly enhances user experience because they can treat the AI assistant like others - ChatGPT, Google Gemini, etc. - but without the subscription costs as discussed earlier. The user can instead interact with the system in a free-flowing conversation. The user can easily learn with whatever type of prompt suits their learning most.

After receiving the input, the language model classifies the input to determine what type(s) of computation is/are required. Queries involving symbolic mathematics - for example, equation solving or differentiation - are routed to the SymPy computation module. Queries needing graphical visualization get routed to the rendering pipeline implemented using Matplotlib. Prompts requiring conceptual/explanatory outputs are processed directly by the language model.

Some queries are bound to require multiple computational modules in the output. For instance, a query may involve symbolic computation (ex. differentiation) for a mathematical expression while also requiring a graphical visualization of the resulting function. In these cases, the routing pipeline distributes tasks to all the necessary computational modules, allowing the language model, symbolic computation engine, and graphical visualization system to operate simultaneously in order to generate the proper response [1]. This workflow enables the system to support more comprehensive and sophisticated STEM prompts.

Consider the prompt:

"Differentiate $x^2 + 3x$ and graph the result."

The language module identifies that symbolic computation and graph visualization are both required. The mathematical expression is sent to the SymPy module, which computes the derivative. The resulting expression is then passed to the Matplotlib module, where a graph is generated. Finally, the language model integrates the symbolic result, graphical output, and explanatory text

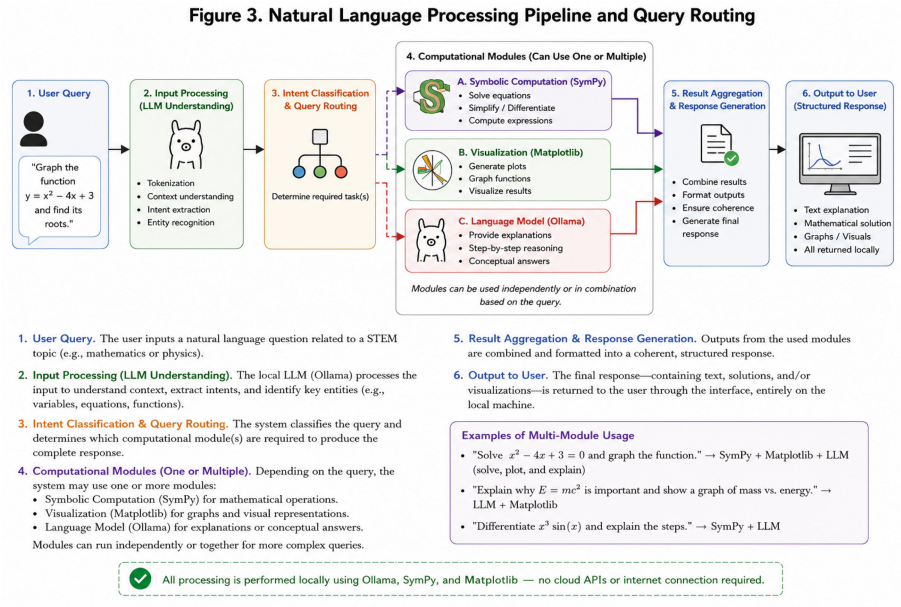


Fig. 3. Natural Language Processing Pipeline and Query Routing

into a single response, which is returned to the user interface. This process demonstrates how multiple modules can work together to satisfy a single user query [1].

Once all computation and graphical representation tasks are completed, the system formats the output into a structured response before returning the results to the user interface. This output may include symbolic solutions, explanatory text, or graphical visualizations, or a combination of these, depending on the original user query.

Figure 3 illustrates the NLP workflow and routing used by the system.

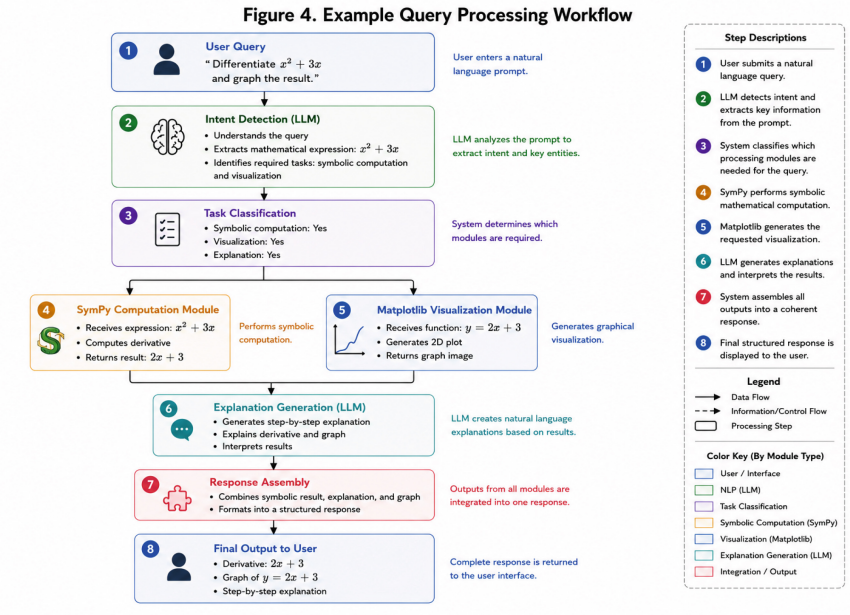


Fig. 4. Example Query Processing Workflow

Figure 4 shows the processing workflow of an example query and multi-module computation.

5 Experiment Evaluation

5.1 Experiment Setup

The proposed AI assistant was evaluated through a set of STEM prompts to assess the accuracy of symbolic computation, graphical visualization, concept explanation, and multi-module routing. The evaluation was executed on a local machine running the Ollama framework alongside SymPy and Matplotlib.

There were 24 prompts divided into 4 categories
Algebra - 6 questions
Calculus - 6 questions
Physics - 6 questions
Conceptual explanation - 6 questions

5.2 Evaluation Metrics

Each answer provided by the proposed AI assistant was then verified manually to determine the accuracy of the output solution provided and whether appropriate computational modules were used. To determine the accuracy of the output of each prompt, the following metrics were used on a binary scale (0 or 100):

- **Mathematical Accuracy** – whether the solution produced by the system was mathematically correct.
- **Routing Accuracy** – whether the prompt was routed to the correct computational module(s). All prompts should utilize the LLM module to display the final output.
- **Graph Generation Success Rate** – whether graphs were successfully generated for prompts that required graphical visualization.

Additionally, the average response time for each prompt - the total time between the user's query submission and final combined output - was recorded to provide data on the existing processing and output speed and potential improvements. This metric was categorized by "number of modules needed for answer." For example, questions asking to solve an equation only would need one module - the language model module, but a question asking to differentiate and then graph the original function would require two modules - the language model and the symbolic computation module. The following was the distribution of this metric:

Number of Computational Modules Required for Output	Number of Questions
1	9
2	8
3	7

5.3 Performance Results Data

System Performance data:

Metric	Accuracy
Mathematical Accuracy	88%
Routing Accuracy	100%
Graph Generation Success Rate	100%

A separate table to compare average response time across the separate prompt types:

Number of Computational Modules Required for Output	Average Response Time(s)
1	31.2
2	41.3
3	51.1

5.4 Discussion

The results demonstrate that the proposed AI assistant can accurately perform symbolic mathematical computations and graphical representation tasks. The routing mechanism successfully identified the correct module every time, and its incorrect answers on questions cloud-based assistants also got wrong. One such question is shown below.

17 Sine & Cosine

In triangle $\triangle ABC$, the measure of angle B is $90°$, $\overline{AB}$ is a leg with a length of 12, and segment $\overline{AC}$ contains the point D. Which of the following must be true?

5.5 Comparison with Other Systems

However, the response times of the proposed assistant were prone to more variability and overall longer times, which will be discussed in the error analysis section. Compared to cloud-based AI assistants, the proposed system provides the advantage of offline access and the lamination of subscription costs (Table 2).

Table 2. Comparison of the Proposed System with Existing AI Assistants

Task	Local AI	ChatGPT	Wolfram
Symbolic Computation	✓	✓	✓
Natural Language Interaction	✓	✓	Limited
Graphical Representation	✓	Limited	✓
Offline Operation	✓	×	×
Subscription Required	×	Often	Often

6 Error Analysis

Although the system is able to combine conversational AI, symbolic computation, and graphical representation, several sources of potential error remain. Experimental evaluation showed a mathematical accuracy of 88% (21/24), while routing accuracy and graph generation both achieved 100% success. These results indicate that the system reliably selects the appropriate computational modules but may still encounter challenges in certain mathematical tasks.

The primary source of error occurred within symbolic computation workflows. Although SymPy generally produces accurate results, ambiguous language in prompts can lead to incorrect symbolic representations, which subsequently produce incorrect solutions. The few incorrect responses observed during testing occurred in multi-step word problems, particularly difficult SAT-style geometry questions that would typically be accompanied by diagrams. Because the prompts were presented solely in text form, some geometric relationships may have been underspecified, resulting in errors during symbolic translation and calculation. This suggests that inaccuracies are more likely to arise from prompt interpretation than from the symbolic computation engine itself. While uploading diagrams alongside textual prompts may help mitigate this issue, future work will focus on improving inference accuracy and prompt understanding.

A second potential source of error stems from natural language generation. The language model may generate mathematically correct solutions while providing incomplete or inaccurate explanations. Such issues can negatively affect conceptual understanding, which is a key objective of educational AI systems [?,?]. Although no significant explanation errors were observed during the evaluation, this limitation remains an important consideration for LLM-based educational assistants.

Incorrect prompt classification represents another possible source of error. However, no routing errors were observed during evaluation, as reflected by the routing accuracy of 100%. Similarly, all required graphs were generated successfully. Nevertheless, routing and visualization errors may emerge when evaluating larger and more diverse datasets. Because the system determines the required computational modules from each user query, incorrect classification may result in incomplete responses. For example, a prompt requiring both symbolic com-

putation and graphical visualization could be incorrectly routed to only one processing module, thereby producing a partial answer.

The most significant limitation observed during evaluation was response time. The system achieved an average response time of 40.37 s across all prompts. Queries requiring multiple computational modules generally required more processing time than those involving only a single module. This trend is expected because multiple modules often operate sequentially depending on the requirements of the prompt. A regression analysis comparing average response time with the number of computational modules required produced an R^2 value of 1.0, indicating a strong positive relationship between these variables within the experimental dataset.

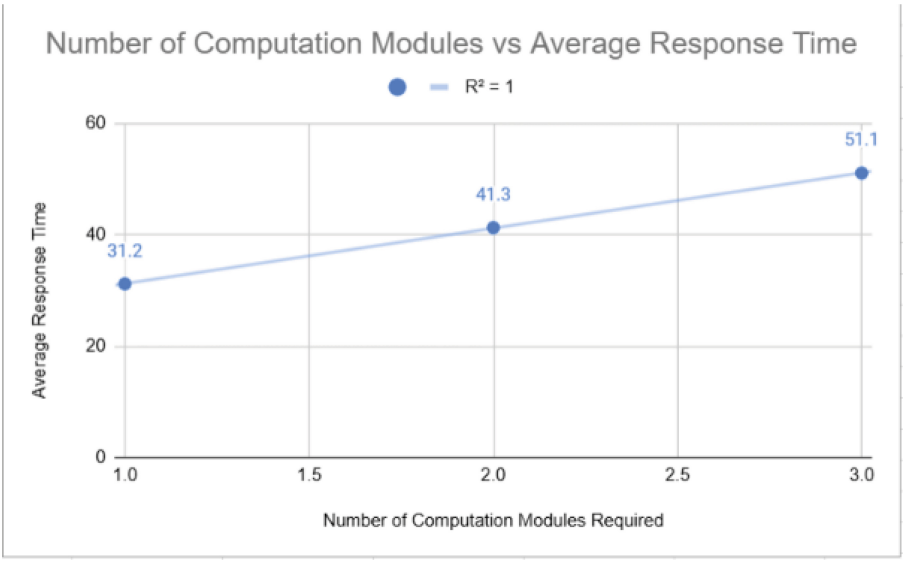


Fig. 5. .

Figure 5 shows the positive correlation between the number of required computational modules for a prompt and the response time. The correlation yields an R^2 of 1.0.

Figure 6 shows sources of error within the proposed AI assistant architecture. Experimental evaluation results show that mathematical errors were associated with ambiguous prompt language, and no routing or graphical visualization errors were observed.

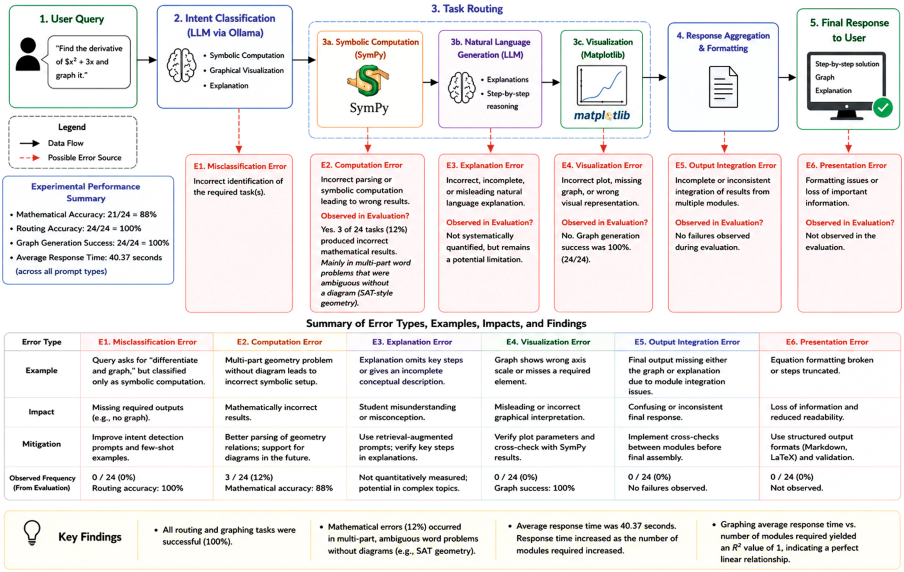


Fig. 6. Error Sources in the Processing Pipeline (Updated with Experimental Findings)

7 Advantages, Limitations, Future Improvements

7.1 Educational Implications

The amalgamation of conversational, symbolic computational, and graphical abilities within a single AI allows students to ask for explanations that best suit their own individual learning style. Prior research suggests that AI tutoring agents can improve engagement through their immediate feedback and customized personal interactions [4,6]. The combination of these three output types may help students grasp topics effectively by connecting representations from the different features, for example, conceptual and graphical.

The assistant's lack of subscription costs (a result of it being locally hosted) may increase access for those without consistent technological access or educational AI tools. This issue is relevant in areas with inconsistent internet access and/or financial resources or such tools [2]. Though current research investigates system design rather than formal educational assessments, future studies could potentially discover more significant relationships through classroom deployments and controlled experiments.

7.2 Scalability and Deployment Considerations

The performance of the locally hosted AI assistant is very heavily influenced by the computational resources of the host system. Response times are subject to vary based on processor speed, memory availability, and internet connection

strength [2]. On the other hand, larger LLMs usually provide enhanced reasoning at the expense of additional computational resources.

Currently, the tested implementation is designed primarily for individual users. Future deployments will consider classroom or institutional scale environments where multiple users can simultaneously access and navigate the system. Such deployments would require considerations such as model infrastructure for specific purposes, load balancing with multiple users, and improved maintenance procedures [8]. Future models may potentially investigate optimization techniques, different deployment methods to improve performance across multiple simultaneous users or even environments [7].

7.3 Privacy, Security, and Ethical Considerations

The current architecture of local deployment offers the privacy advantage of user interaction data remaining on the host device rather than being stored in external servers, something seen in cloud-based assistants (ex. Google Gemini). This significantly reduces the chance of student data exposure and may make compliance with institutional privacy mandates easier [8].

However, despite this, privacy and security remain important topics of concern. It is always possible for a user to unintentionally provide sensitive information through prompts, and the local host can always be vulnerable to unauthorized access if the host device is not properly secured. Additional safeguards like encrypted storage and/or user authentication should be considered in future developments and deployments [8].

Using the suggested locally-hosted AI assistant in educational settings can provoke ethical considerations, just like the use of any other cloud-based AI. Language models can generate incorrect answers and/or explanations at times, or purely misleading information in some way. As a result, educational material generated by AI assistants should be viewed as supplementary learning tools, not replacements of instructor-led learning [9]. Future versions of the suggested system may include verification features to improve reliability.

7.4 Limitations and Future Work

Despite all the advantages, however, the system still presents certain limitations. Locally hosted language models may provide lower reasoning performance in comparison to larger cloud-based subscription based systems due to reduced parameter sizes and hardware constraints. Inference speed may vary depending on the computational capabilities of the hosting machine, particularly if multiple modules are required for the specific input. Additionally, although the system specializes in mathematics and physics workflows, broader domain coverage of STEM and more advanced adaptive educational feedback mechanisms remain limited in the current system implementation.

The experimental evaluation revealed that some mathematical errors occurred in multi-part word problems when questions/prompts typically requiring diagrams to solve were entered only through text. A simple solution to such

an issue can be to upload screenshots or images of a diagram to accompany the prompt, as it would allow the system to analyze both the text and the drawing to incorporate both in deriving a more accurate answer than if only using text. Nonetheless, given that ambiguous language may arise in prompts requiring only the LLM module, future improvements will still focus on enhancing the system's abilities to accurately infer relationships from text and overall prompt understanding.

Response time was a main limitation of the system, with the average response time across all prompt types being 40.37 s. Although the need for multiple modules have been noted to contribute to this measure, conceptual prompts requiring detailed explanation in natural language also took time. For example, a prompt asking for an explanation of Newton's second law took 60 s. This likely stems from the LLM trying to offer extensive reasoning and explanation in its final output. Nevertheless, given that the correlation between more computational modules and greater response times was strong, this type of question is not a significantly confounding factor and response time may be associated with prompt complexity instead.

Future work will focus on expanding the educational and computational capabilities of the system. Potential improvements include the development of adaptive tutoring mechanisms capable of adjusting explanations specific to user performance, and supporting additional STEM disciplines more in depth beyond mathematics and physics. Further optimization in local model deployment and refinements on STEM-specific datasets may also improve the accuracy and quality of reasoning and responses. Furthermore, future versions of the system may explore deployment on a classroom-scale and integration of more modules for more specialized interactions depending on specific queries.

8 Conclusion

This paper presented the development and architecture of a locally hosted AI assistant targeted towards improving accessibility to AI educational tools without relying on cloud-based services. The system combines a local LLM deployed through Ollama with symbolic computations using SymPy and graphical visualization using Matplotlib. The system specializes in STEM (specifically mathematics and physics) problem solving and combining the three modules in a single, unified environment.

Through the elimination of dependence on subscription-based APIs and their recurring costs, the system provides a cost-effective alternative that is also accessible offline as an alternative to AI assistants in the domain of STEM education support. The specific modular architecture of the system allows the system to be improved in the future - with, for example, the addition of new modules - without the need of major system modifications. This allows for easy enhancements to the system in the future. This paper demonstrates the potential of locally hosted deployed AI systems as alternative and easily-accessible tools for STEM education.

References

1. Das, D., Banerjee, D., Aditya, S., Kulkarni, A.: MATHSENSEI: a tool-augmented large language model for mathematical reasoning. Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1: Long Papers, pp. 942–966. Association for Computational Linguistics, Mexico City (2024)
2. Beltran, D.E.: Hybrid edge-cloud AI for Rural STEM education: an offline-capable mobile system integrating claude, Gemini, and Ollama LLMs to optimize assessments in low-connectiveity contexts. Preprint/working paper (OSF Preprints/arXiv-style publication) hosted on an open research repository (Society/OSF ecosystem
3. Tufino, E.: NotebookLM: an LLM with RAG for active learning and tutoring. arXiv preprint arXiv:2504.09720 (2025)
4. Kumar, H., Rothschild, D.M., Goldstein, D.G., Hofman, J.: Math education with large language models: peril or promise? Available at SSRN (2023). https://ssrn.com/abstract=4641653, http://dx.doi.org/10.2139/ssrn.4641653
5. Nikolic, S., et al.: Generative artificial intelligence (GenAI) 24x7 tutor: a simulation of the capability of ChatGPT, Wolfram GPT and Tutor Me GPT to accurately and effectively tutor engineering and math content. STEM Educ. **6**(1), 56–83 (2026)
6. Lyu, W., Wang, Y., Chung, T., Sun, Y., Zhang, Y.: Evaluating the effectiveness of LLMs in introductory computer science education: a semester-long field study. In: Proceedings of the ACM Learning @ Scale Conference. ACM (2024)
7. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada, pp. 9459–9474 (2020)
8. Miao, F., Holmes, W.: Guidance for Generative AI in Education and Research. UNESCO, Paris, France (2023)
9. Ribeiro, M.T., Singh, S., Guestrin, C.: Why should i trust you?: Explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 2016), San Francisco, CA, USA, pp. 1135–1144 (2016)
10. Holmes, W., Bialik, M., Fadel, C.: Artificial intelligence in education: promises and implications for teaching and learning. Center for Curriculum Redesign, Boston, MA (2019)

Hyperdimensional Computing on Apple Neural Engine Chips

Srivatsan Suresh Babu[1(✉)], Sophia Shahryar[1], and Justin Morris[1,2]

[1] Department of Electrical and Computer Engineering, California State University San Marcos, San Marcos, CA, USA
{sures005,shahr007,justinmorris}@csusm.edu

[2] Department of Computer Science and Engineering, California State University San Marcos, San Marcos, CA, USA

Abstract. Hyperdimensional Computing (HDC) is a lightweight machine learning algorithm that is more energy efficient than deep neural networks. The algorithm leverages high-dimensional representations and matrix multiplications for classification. While there is prior work on the efficiency of HDC on specialized accelerators such as the Tensor Processing Unit (TPU), there is a lack of exploration of HDC on the Apple Neural Engine (ANE). In this paper, we evaluate the performance and energy efficiency of HDC inference across different Apple Silicon devices, such as the M3 Pro, M4, and A19 Pro chips. We express HDC inference as a two-layer neural network composed of a fixed random projection and a linear similarity layer, enabling conversion from PyTorch to CoreML for hardware-accelerated execution. Using the scikit-learn handwritten digits dataset, we benchmark inference timing and energy consumption across CPUs, GPUs, and the ANE across different batch sizes and hypervector dimensionalities. Our results demonstrate that the ANE consistently achieves better energy efficiency for large batch sizes and high-dimensional hypervectors, with up to 58% energy reduction compared to CPU execution. These findings highlight the suitability of domain-specific accelerators for HDC workloads and demonstrate the practicality of deploying HDC efficiently on modern edge devices.

Keywords: Edge Computing · Hyperdimensional Computing · Machine Learning

1 Introduction

CPUs were originally the base of general-purpose computation; however, their architectures provided limited parallelism for high-throughput workloads. Manufacturers relied on improving CPUs through Moore's Law, which led to the doubling of transistor density on CPUs, resulting in better CPU performance [1]. While CPUs have improved in performance over time, the main issue of limited parallelism remains. Consequently, CPUs are not efficient for heavy computational workloads, such as machine learning. The solution to inefficient CPUs is to complement them with accelerators [2].

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 748–759, 2027.
https://doi.org/10.1007/978-3-032-39763-8_47

The solution to the limited parallelism of CPUs has been the use of highly parallel graphics processing units (GPUs). Although individual GPU cores are less complex than CPU cores, GPUs contain a large number of threads, which are optimized for data-parallel computations. This architectural design allows large-scale computations of simple operations, such as element-wise and matrix operations, which are the pinnacle of modern machine learning computations [3]. However, while GPUs solve the issue of limited parallelism, their high power consumption and energy are an issue for the environment, mainly for large-scale deployment or embedded low power devices [4].

An effective approach to reducing the amount of device power consumption and energy usage is to deploy domain-specific accelerators (DSAs), including the Tensor Processing Unit (TPU) and Apple Neural Engine (ANE). These accelerators are optimized to perform specific, specialized computational workloads with high efficiency. They excel at operations such as matrix multiplication and element-wise operations, while keeping power and energy usage low compared to GPUs. As many machine learning algorithms and deep neural networks (DNNs) are dominated by these operations, DSAs are superior in both energy efficiency and throughput [5].

Although domain-specific accelerators can significantly improve time and energy efficiency, deploying deep neural networks (DNN) on edge devices is quite challenging due to the heavy computational and memory requirements of modern DNN algorithms. Even when optimizing these DNNs with accelerators, the sheer amount of matrix and tensor operations is still high, leading to high energy consumption, and latency [6]. Due to the design of edge devices, they have limited computational resources, memory, and battery capacity, which limits their ability to deploy DNNs. Consequently, running full-scale DNNs on low-power devices is not viable [7].

Another approach to improving the energy efficiency and throughput of machine learning workloads is the use of application-specific integrated circuits (ASICs) or Field Programmable Gate Arrays (FPGAs). ASICs and FPGAs are generally more specialized for particular algorithms or model architectures, and can exceed the performance of DSAs. However, due to their fixed architectures, ASICs generally lack flexibility and cannot easily support updates in network architectures [8]. Additionally, both accelerators are not normally added to everyday devices.

The solution to the resource and power constraints of edge devices is to use more efficient algorithms, rather than computationally expensive DNNs. Hyperdimensional Computing (HDC) addresses this challenge by using high-dimensional vectors and performing simple and minimal operations, such as element-wise operations. Due to the minimal amount of operations in HDC, the algorithm is far more energy efficient than DNNs, as they have non-linear activations and matrix multiplications. Furthermore, HDC models can achieve high accuracy with a single pass, reducing the need for multiple epochs for training and reducing energy consumption [9]. When deploying HDC on domain-specific accelerators such as the ANE, not only is the software optimized, but the hard-

ware is optimized as well, leading to large improvements in energy efficiency and throughput.

In this paper, we evaluate Hyperdimensional Computing across multiple hardware platforms on Apple Silicon devices. We compare the performance and energy consumption of HDC inference on CPUs, GPUs, and the Apple Neural Engine, highlighting architectural trade-offs between general-purpose and domain-specific accelerators. Our results demonstrate that large batches are more efficient on the ANE than CPU or GPU. Across all Apple Silicon devices, the Neural Engine was most effective against the CPU and GPU in energy efficiency, showing the largest improvement on the M3 Pro (58% energy reduction compared to CPU and GPUs). While the Neural Engine had a subpar improvement in timing efficiency (3–13%), the Neural Engine maintained equal or better performance, while significantly reducing energy consumption. These results suggest that the Neural Engine is energy efficient for large batching workloads. In addition, we analyze the impact of hypervector dimensionality on energy efficiency across these compute architectures for further energy efficiency.

2 Related Work

Early implementations of Hyperdimensional Computing (HDC) were primarily executed on CPUs, as they are accessible and easy to program on. However, prior work has shown that CPU-based HDC implementations are inefficient due to their lack of parallelism, leading to inefficient execution of high-dimensional vector operations. Due to HDC's reliance on element-wise arithmetic and similarity computations over large hypervectors, CPU execution becomes more time and energy-inefficient as dimensionality and dataset sizes increase [9,10].

While CPUs are still essential for complex workloads, their architecture limitations motivate the exploration of parallelizable platforms for HDC acceleration [11]

In order to address the inefficiencies of the CPU, GPU-based accelerations have been explored for HDC. GPUs provide massive parallelism, which aligns well with the vectorized operations of HDC, which include binding and bundling. Prior studies demonstrate that GPUs significantly improve throughput over CPUs for HDC inference [3].

However, despite these performance gains, GPUs are primarily optimized for high arithmetic intensity workloads such as dense linear algebra. HDC workloads, by contrast, use lightweight binary or integer operations. Although GPUs have great throughput, their architecture results in increased energy consumption, which results in problematic behaviors for devices that are limited by energy [2]. To address these limitations, prior work has investigated FPGA-based accelerators for HDC. FPGA implementations allow customization of datapaths, allowing operations such as binding and similarity operations to be directly optimized on hardware. This results in improved parallelism and higher energy efficiency compared to both CPUs and GPUs. However, despite these advantages, FPGA-based approaches are much more resource intensive to implement, and are not widely used in consumer devices.

ASIC-based designs push this specialization further by hardwiring HDC operations. This allows for the highest energy efficiency in both throughput and power consumption due to its highly specialized architecture. However, this efficiency comes with the cost of inflexibility, long development cycles, and high costs, making ASICs impractical for evolving HDC algorithms. [12].

Domain-specific accelerators (DSAs) such as Tensor Proecssing Units (TPUs) have demonstrated a balanced-trade off between programmability and efficiency for machine learning workloads. TPUs are specialized for matrix operations, which enable high-throughput execution for deep learning models. Prior work has shown that such architectures can also accelerate HDC workloads more efficiently than CPUs by mapping hypervector operations onto matrix primitives [13]. While TPUs are an efficient choice for HDC, TPUs are mainly implemented in large datacenters, rather than daily-use consumer devices.

The Apple Neural Engine (ANE), integrated into Apple Silicon devices, represents a widely deployed DSA optimized for energy-efficient, on-device inference. Its architecture is optimized for vector and matrix operations, which align with the design of HDC. While recent work has explored the ANE for transformer-based workloads [14], its compatibility for HDC remains unexplored. In this work, we address the gap by evaluating HDC on the ANE and comparing its performance and energy efficiency against CPUs and GPUs across different workload dimensions.

3 Overview

3.1 HDC Encoding

Before we can work in high dimension space, we first need to encode the data to hypervectors. Figure 1 shows the overview of HDC random projection encoding mapping each n dimensional feature vector to a D dimensional binary hypervector. Let us assume a feature vector $\mathbf{F} = \{f_1,\ f_2, \ldots,\ f_n\}$, with n features ($f_i \in \mathbb{N}$) in original domain. The goal of random projection is to map this feature vector to a D dimensional space vector: $\mathbf{H} = \{h_1,\ h_2, \ldots,\ h_D\}$. As Fig. 1 shows, random projection generates D dense bipolar vectors with the same dimensionality as the original domain, $\{\mathbf{P}_1, \mathbf{P}_2, \ldots, \mathbf{P}_D\}$, where $\mathbf{P}_i \in \{-1, 1\}^n$. The inner product of a feature vector with each randomly generated vector gives us a single dimension of a hypervector in high-dimensional space. For example, we can compute the $i - th$ dimension of the encoded data as: $h_i = \mathbf{P}_i \cdot \mathbf{F}$. This results in the entire encoding operation mapping to a matrix multiplication.

3.2 Training an HDC Model

Hyperdimensional Computing (HDC) adopts a form of training that differs from traditional gradient-based neural network optimization. Instead of minimizing a loss function through backpropogation, the model creates class representations directly by accumulating encoded samples in a high-dimensional space. The result of training are k hypervectors with D dimensions, where k is the

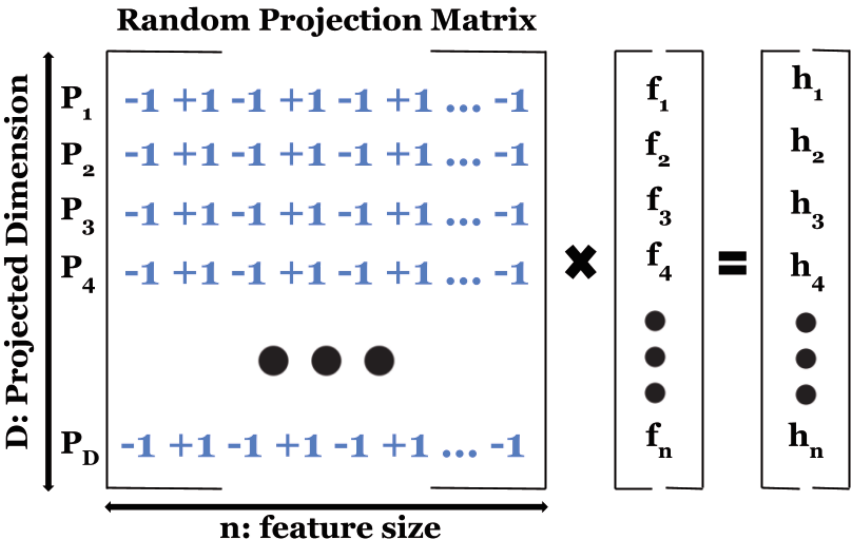


Fig. 1. Random Projection Matrix

number of classes. For example, the i^{th} class hypervector can be computed as: $\mathbf{C_i} = \sum_{\forall j \in class_i} \mathbf{H_j}$. HDC performs model adjustment by iteratively going through the training dataset. During a single iteration of model adjustment, HD computing checks the similarity of all training data points, say $\mathbf{H}$, with the trained model. If a data is wrongly classified by the model, HD updates the model by (i) adding the incorrectly classified hypervector to the class that input belongs to ($\widetilde{\mathbf{C}}^{correct} = \mathbf{C}^{correct} + \mathbf{H}$), and (ii) subtracting it from the class which it is wrongly matched with ($\widetilde{\mathbf{C}}^{wrong} = \mathbf{C}^{wrong} - \mathbf{H}$).

3.3 HDC Inference

After training and retraining, the HDC model can now be used for inference (Fig. 2). Upon inference, an input data is encoded to a *query* hypervector using the same encoding module used for training. HDC then compares the similarity of the query hypervector with all stored class hypervectors and selects the class with the highest similarity. Cosine similarity between the class hypervectors and the query is a popular metric to find a class hypervector with the most similarity to the query hypervector. This involves again a dot product between the query and all class hypervectors, which again, can be modeled as a matrix multiplication.

3.4 Mapping HDC Inference to a Two-Layer Neural Network and Implementation as a CoreML Model

As mentioned in the above subsections, HDC encoding and similarity check can both be mapped as matrix multiplications. Furthermore, the output of HDC

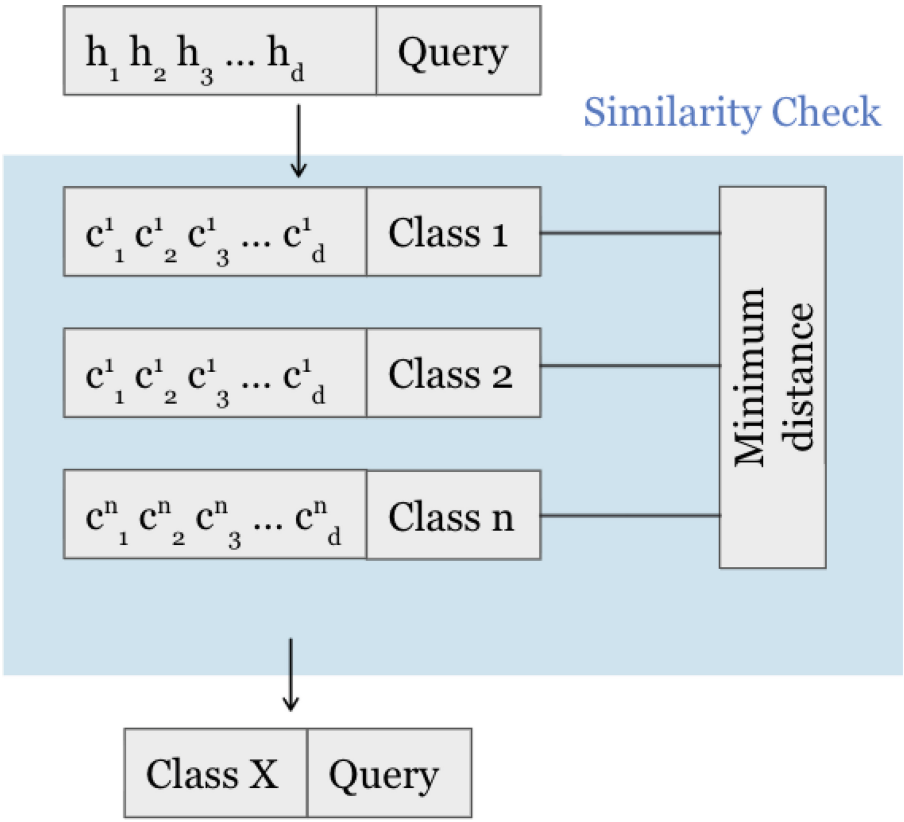


Fig. 2. Similarity Check Visualization

encoding is the input into the similarity check. This means, that the two steps that make up HDC inference, can be expressed as two matrix multiplications. It is also well known that fully connected Multi-Layer Perceptrons are can be mapped as one matrix multiplication per layer. This means that we can model HDC as a two-layer MLP using standard tensor operations. Therefore, to evaluate HDC on ANE, we model HDC as a two-layer MLP in PyTorch. The rest of the process to obtain a model we can run on the ANE is shown in Fig. 3.

The Pytorch implementation of the HDC model serves as a domain-specific language (DSL) model, allowing it to capture the entire inference algorithm in terms of tensor operations. This DSL model includes the two layer neural network, which consists of the projection and element-wise thresholding, and the linear similarity check with class hypervectors. This allows TorchScript to produce a static representation of the DSL model, which is then converted to CoreML using coremltools. The resulting CoreML model is identical to the PyTorch model, but is optimized on Apple hardware, meaning the compute can

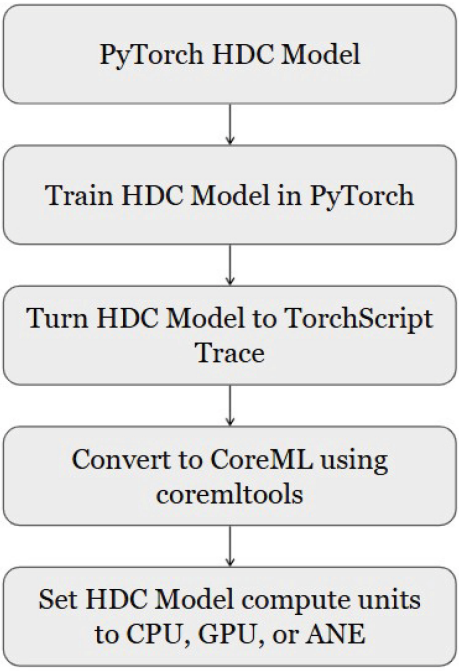


Fig. 3. Core ML Conversion Process

be customized on the CPU, GPU, or ANE. This allows for better performance on Apple Silicon devices, and allows the user a variety of different computes.

3.5 Dataset and Motivation

For our experiments, we use the `load_digits` dataset from scikit-learn, which contains 1,797 grayscale images of handwritten digits, each of size 8×8 pixels [15]. Each image is flattened into a 64-dimensional vector, and the corresponding labels represent the digits 0 through 9.

This choice is motivated by prior work in Hyperdimensional Computing for speech recognition, such as VoiceHD [16], where preprocessed audio signals are encoded into high-dimensional hypervectors and classified using HDC. By demonstrating our approach on the digits dataset, we establish a proof-of-concept for HDC inference mapping that can be used for other classification tasks as well.

4 Experimental Results

4.1 Experimental Setup

The HDC model was implemented in PyTorch and expressed as a two-layer neural network to maximize parallel computation. The dataset for this two-layer

neural network is the SciKit Learn handwritten digits dataset. The PyTorch model was then converted to CoreML format using the coremltools framework, which optimizes the hardware usage on Apple Silicon devices [17]. Power consumption during inference was measured using Apple's Power Profiler [18]. We evaluated three different Apple chips, the M3 Pro, M4, and A19 Pro. We also evaluated deploying the model on different computer modules for each chip. We tested deploying on the CPU, GPU, and ANE. Batches ranging from 2 to 2048, and dimensionalities ranging from 500 to 10,000 were measured, on all hardware platforms and compute modules. In each program, inference was run 20 times over a 10-second interval, in order to gain accurate inference timings over an entire dataset and average power consumption. Energy consumption was calculated using the measured power values and average inference timings.

4.2 Execution Time Results

Figure 4 shows our execution time results testing different batch sizes on all the chips tested and on all the compute modules. For batching timing on higher batches of 512, 1024, and 2048, the M3 Pro Neural Engine is 3.2% faster than the M3 Pro CPU, and 4.6% faster than the M3 Pro GPU. The M4 Neural Engine is 4.6% faster than the M4 CPU, and 10.3% faster than the M4 GPU. The A19 Pro Neural Engine is 3.1% faster than the A19 Pro CPU, and 13.5% faster than the A19 Pro GPU.

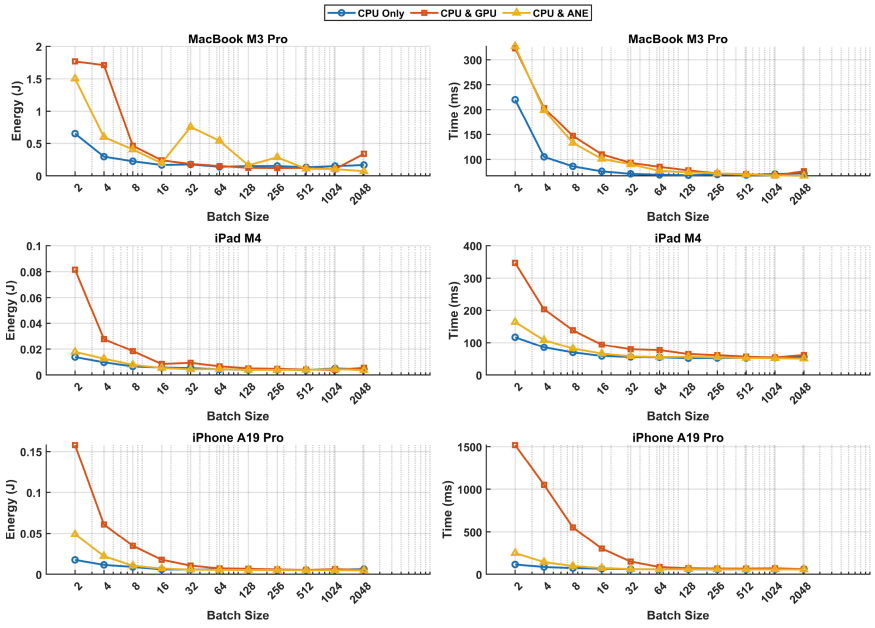


Fig. 4. HDC Timing and Energy Consumption Across Devices (Batching)

The ANE achieves higher throughput on larger batches due to its design for parallel vector and matrix operations, which are what HDC is comprised of (see Fig. 4). Unlike CPUs, which need to process data sequentially and require multiple memory accesses, the ANE can handle much more data in parallel, increasing throughput and reducing overhead for memory, thus reducing inference timing. Additionally, GPUs require more overhead due to their dense floating-point operations, whereas HDC primarily uses integer or low-precision operations. Additionally, with larger the batch sizes, the less impact of the overhead of moving data to the accelerators is reduced due.

4.3 Batching Energy Consumption

Figure 4 shows our energy consumption results testing different batch sizes on all the chips tested and on all the compute modules. For batching energy consumption of batches 512, 1024, and 2048, the M3 Pro Neural Engine used 58% less energy than the M3 Pro CPU, and 50% less energy than the M3 Pro GPU. The M4 Neural Engine used 8.8% less than the M4 CPU, and 10.7% less than the M4 GPU. The A19 Pro Neural Engine used 15.6% less energy than the A19 Pro CPU, and 13.8% less than the A19 Pro GPU.

The ANE outperforms both the CPU and GPU for large batches due to its specialized architecture, which is optimized for machine learning and deep learning workloads. Due to the use of integer values rather than dense floating-point operations in HDC, the ANE can efficiently execute wide, low-precision vector operations while minimizing memory movement, resulting in lower energy consumption. Larger batches are especially beneficial because the overhead of sending data to the ANE is spread across more computations, allowing it to fully leverage its parallelism. This indicates that for HDC inference tasks, the ANE is most advantageous on larger batches, whereas CPUs remain more efficient for smaller batch sizes.

4.4 Dimensionality Energy Consumption

Figure 5 shows our energy consumption results testing different HDC dimensionalities on all the chips tested and on all the compute modules. For dimensionality energy consumption, with batch sizes of 2048, the M3 Pro Neural Engine used 36.0% less energy than the M3 Pro CPU, and the M3 Pro Neural Engine used the same energy consumption as the M3 Pro GPU. The M4 Neural Engine used 19.6% less energy than the M4 CPU, and 7.3% less energy than the M4 GPU. The A19 Pro Neural Engine used 11.3% less than the A19 Pro CPU, and 17.9% less than the A19 Pro GPU.

The ANE uses lower energy consumption compared to the CPU, and in some cases the GPU, as hypervector dimensionality increases. This is due to the ANE's hardware being more optimized for wide vector and matrix operations. At larger batch sizes and larger dimensionalities, the HDC algorithm can leverage the parallelism offered by the ANE and GPU. However, for the same computation, CPUs require multiple accesses to memory to handle high-dimensional vectors, which

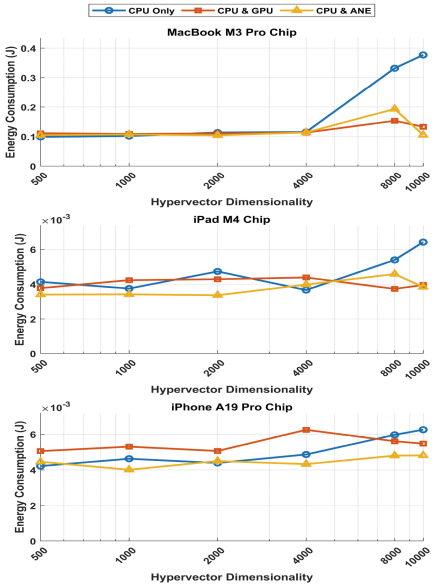


Fig. 5. HDC Energy Consumption Across Devices (Dimensionality)

increases energy consumption in larger dimensionalities. In contrast, the ANE can process more elements simultaneously while having low memory overhead. Lastly, the ANE outperforms the GPU due to its architecture being optimized for dense floating-point operations, which HDC does not require, allowing the ANE to execute HDC tasks more efficiently in both energy and throughput.

5 Future Work

An important direction for future work is the systematic evaluation of quantization when mapping HDC workloads onto the ANE via CoreML. Although our current implementation operates at higher numerical integer precision, HDC is inherently well-suited to reduced-precision representations due to its reliance on high dimensional distributed vectors, stochastic robustness, and tolerance to noise. Prior work suggests that many HDC operations, such as binding, bundling, and similarity search–maintain accuracy under aggressive quantization, potentially down to INT8 or even binary. Leveraging quantization pathways could significantly reduce memory footprint, improve data movement efficiency, and increase throughput on the ANE, which is architecturally optimized for low-precision arithmetic. Furthermore, reduced precision may allow larger hypervector dimensions or higher degrees of parallelism within the same power envelope. A comprehensive exploration of quantization-aware mapping strategies, precisionaccuracy tradeoffs, and energy efficiency measurements would therefore provide deeper insight into the practical deployment limits of HDC on Apple Silicon platforms.

6 Conclusion

In this work, we investigated the execution of HDC on modern Apple Silicon devices, comparing CPUs, GPUs, and the ANE across Apple devices. By mapping HDC inference as a two-layer neural network and converting the PyTorch implementation to CoreML, we enabled optimized execution on domain-specific hardware. Our experimental results demonstrate that the ANE provides significant energy advantages over CPU and GPU compute units, mainly for larger batch sizes and higher hypervector dimensionalities. While timing improvements remain similar to CPUs and GPUs, the energy savings are substantial, confirming that HDC workloads align well with the architectural strengths of the ANE. These results suggest that combining lightweight algorithms such as HDC with domain-specific accelerators is a great approach for energy efficient classification. Future work may extend this evaluation to other classification tasks, larger-scale datasets, more Apple chips, as well as quantization to further explore HDC deployment on edge platforms.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Waldrop, M.M.: The chips are down for Moore's law. Nature News **530**(7589), 144 (2016)
2. Khaleghi, B., Salamat, S., Thomas, A., Asgarinejad, F., Kim, Y., Rosing, T.: SHEARer: highly-efficient hyperdimensional computing by software-hardware enabled multifold approximation. In: Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design, pp. 241–246 (2020)
3. Nickolls, J., Buck, I., Garland, M., Skadron, K.: Scalable parallel programming with CUDA: is CUDA the parallel programming model that application developers have been waiting for? Queue **6**(2), 40–53 (2008)
4. Ozkan, M., et al.: Performance, efficiency, and cost analysis of wafer-scale ai accelerators vs. single-chip GPUs. Device (2025)
5. Liang, Q., Shenoy, P., Irwin, D.: AI on the edge: rethinking ai-based IoT applications using specialized edge architectures. arXiv preprint arXiv:2003.12488 (2020)
6. Yan, M., Wang, H., Venkataraman, S.: Polythrottle: energy-efficient neural network inference on edge devices. arXiv preprint arXiv:2310.19991 arXiv preprint (2023)
7. Mishra, R., Gupta, H.P.: Designing and training of lightweight neural networks on edge devices using early halting in knowledge distillation. IEEE Trans. Mob. Comput. **23**(5), 4665–4677 (2023)
8. Xu, B., Banerjee, A., Gupta, S.: Hardware acceleration for neural networks: a comprehensive survey. arXiv preprint arXiv:2512.23914 (2025)
9. Ma, D., Hao, C., Jiao, X.: Hyperdimensional computing vs. neural networks: comparing architecture and learning process. In: 2024 25th International Symposium on Quality Electronic Design (ISQED), pp. 1–5. IEEE (2024)
10. Kayid, A., Khaled, Y., Elmahdy, M.: Performance of CPUs/GPUs for deep learning workloads. The German University in Cairo (2018)

11. Theis, T.N., Wong, H.-S.P.: The end of Moore's law: a new beginning for information technology. Comput. Sci. Eng. **19**(2), 41–50 (2017)
12. Tashdid, I., Saiham, D., Anjum, N., Farheen, T., Rahman, S.: ECOLogic: enabling circular, obfuscated, and adaptive logic via eFPGA-augmented SoCs. In: 2025 IEEE 43rd International Conference on Computer Design (ICCD), pp. 523–527. IEEE (2025)
13. Ni, Y., Kim, Y., Rosing, T., Imani, M.: Algorithm-hardware co-design for efficient brain-inspired hyperdimensional learning on edge. In: 2022 Design, Automation & Test in Europe Conference & Exhibition (DATE), pp. 292–297. IEEE (2022)
14. Apple Machine Learning Research. Deploying transformers on the apple neural engine (2022). https://machinelearning.apple.com/research/neural-engine-transformers. Describes the architecture and optimization of transformer models on the Apple Neural Engine (ANE), highlighting high-throughput parallel execution and energy-efficient on-device machine learning inference
15. Alpaydin, E., Kaynak, C.: Optical recognition of handwritten digits [dataset]. UCI Machine Learning Repository (1998). https://archive.ics.uci.edu/dataset/80/optical+recognition+of+handwritten+digits
16. Imani, M., Kong, D., Rahimi, A., Rosing, T.: VoiceHD: hyperdimensional computing for efficient speech recognition. In: 2017 IEEE International Conference on Rebooting Computing (ICRC), pp. 1–8. IEEE (2017)
17. Apple Inc. Core ml tools documentation and guides (2025). https://apple.github.io/coremltools/docs-guides/. Official Apple coremltools documentation
18. Apple Inc. Measuring your app's power use with power profiler (2025). https://developer.apple.com/documentation/Xcode/measuring-your-app-s-power-use-with-power-profiler. Official Apple Developer Documentation

Author Index

A
Abdul Basit Ur Rahim, Muhammad 76, 169, 667, 710, 733
Abdullah, Muhammad 289
Abid, Muhammad 218, 232
Abubakar, Murtala 652
Adhikari, Naresh 652
Agrawal, Pulin 253
Ahmed, Mansoor 154
Akhunzada, Adnan 289, 302, 318, 430, 447
Al Nahian, Sultan 395
AlAmri, Amira 652
Al-Bishri, Faisal 447
Al-Ghushami, Abdullah 430
Ali, Zulfiqar 169
Al-Mansoori, Mohammed 430
Al-Mehaizaa, Ali 430, 447
Al-Mohannadi, Mohammed 430
Angel, Aidan Tristen R. 667
Anjum, Adeel 627
Ashok, Arjun 268
Ashraf, Imran 302, 318

B
Babu, Srivatsan Suresh 748
Balachander, Pranav 415
Bhagat, Sanjiv Kumar 19
Bhatt, Sarthak 524
Biswa Karma, Rohit 357
Breindel, Brandon 726
Brown, Annika 615

C
Chowdhury, Zaheen 733
Cimaglia, Christian 218
Coleman, Paige 524

D
Dagli, Nirmit 337
Dang, Thu 639

E
Elahi Qazi, Zainab 395

F
Falah, Bouchaib 605
Filkov, Vladimir 268

G
Garman, Griffin 395
Gladdney, Anthony 114
Goldman, Evan 415
Gollu, Dheeraj 253
Gul, Muhammad Talha 627, 726
Gundapaneni, Lasya Madhuri 415

H
Hashmi, Sajid Ibrahim 504
He, Sen 76
Ho, Katie 615
Hussain, Shahid 36

I
Islam, Ashhadul 652
Ivković, Nikola 289, 302, 318

J
Jagannathan, Balasubramanian Bava 532
Javed, Muhammad 154
Jha, Shashi Bhushan 114
Ju, David 581
Julius, Grace 605

K
Kanu, Onyedikachi 605
Kashyap, Sankalp 268
Kehkashan, Tanzila 289, 302, 318
Khalid, Muhammad Irfan 154
Khan, Arif Ali 553
Khan, Nafiz Imtiaz 268
Khan, Rashid Ali 3

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
S. Hussain et al. (Eds.): SEET 2026, CCIS 3125, pp. 761–763, 2027.
https://doi.org/10.1007/978-3-032-39763-8

Khan, Saif Ur Rehman 36
Kocak, Taskin 337
Kocis, Paul 395

L
Lan, Lihua 710
Lapid, Jillienne 232
Le, Minh Tung 667
Liang, Yongyan 710
Liaw, Jonathan Teck Meng 253
Liaw, Jonathan 697
Lin, Rongyu 337
Liyanage, Kaveen 138
Luther Bennethum IV, Robert John 395

M
Mandava, Lavanya 357
Mandava, Shanmukh Gopinath 357
Mansoor, Osamah 627
Manzer, Kashif 169
Manzi Muneza, A.Redempta 138
Marimekala, Sanjeevkumar 337
Markkula, Jouni 504
Mihaljević, Ivan 318
Milien, Isiah 395
Milošević, Roko 302
Mistry, Khush 415
Mohammad, Atif Farid 524, 596
Mohsin Reza, Sayed 395
Morris, Justin 748
Mudassir, Ghulam 680

N
Nagaraju, Rakesh Kumar Bidanagere 477
Nallaraja, Joshua 697
Namin, Akbar Siami 376
Nascimento, Nathalia 581
Nezhad, Reza Abdollahi Dezfuli 253
Nguyen, Xuan 553

O
Offor, Chinyere 605
Oligeri, Gabriele 447
Orange, Remington 567

P
Partridge, Joshua 680
Porter, June 189
Pragel, Matthew 395

Q
Qi, Trudi Di 615

R
Rao, Sagar 524
Rehman, Faqeer Ur 138
Rehman, Israr Ur 169
Reinhold, Ann Marie 138
Reza, Sayed Mohsin 415
Rizwan I Haque, Intisar 596
Ruan, Anbang 201

S
Saha, Abir 487
Satish, Yamini 415
Saxena, Sudhir 64
Selim, Md 232
Selin Ak, Ozge 395
Shah, Kruti 337
Shahryar, Sophia 748
Shahzad, Aamir 154
Shaikh, Abdul 652
Shelton, Christopher R. 567
Sheppard, Emma 138
Shuja, Junaid 487
Singh, Sonali Uttam 376
Solanki, Richa 52
Srinivasan, Satish Mahadevan 189
Steckel, Bibiana M. 567
Swartz, Logan 395
Syed, Madiha Haider 627

T
Tang, Mei-Huei 36
Tuličić, Domagoj 289
Turaga, Venkata Sai Prathyush 376

V
Vo, Sang 553

W
Wang, Kevin 36
Wang, Li 201, 639
Wang, Wen-Li 36
Wood, Alex 680

Y
Yang, Caijing 553
Ye, Boshuai 553

Z
Zhang, YueMing 667
Zhao, Jie 154, 627
Zhao, Yutong 76
Zhou, Xing 639
Zong, Rui 76